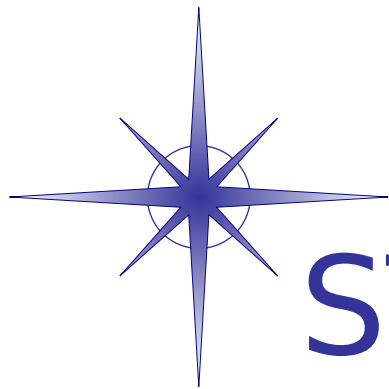




STAR-Dundee

STAR-System Documentation

Version 4.01

Generated by Doxygen 1.8.10

Fri Nov 29 2019 11:49:22

Contents

1	Introduction and Overview	1
1.1	Introduction	1
1.2	Supported Operating Systems	2
1.3	Installation Instructions	2
1.3.1	Installation Instructions for Windows	2
1.3.1.1	Windows XP USB Issue	2
1.3.1.2	Windows Vista USB Issue	3
1.3.1.3	Universal CRT	3
1.3.2	Installation Instructions for Linux	3
1.3.2.1	Command Line Installer	4
1.3.3	Installation Instructions for Linux with Secure Boot Enabled	4
1.3.4	Installation Instructions for QNX	5
1.3.5	Installation Instructions for VxWorks	5
1.3.6	Installation Instructions for RTEMS	5
1.4	Getting Started	5
1.4.1	Getting Started on Windows	6
1.4.1.1	Getting Started on Cygwin	6
1.4.2	Getting Started on Linux	6
1.4.3	Getting Started on QNX	6
1.4.4	Getting Started on VxWorks	6
1.4.5	Getting Started on RTEMS	6
1.5	Migrating from Previous STAR-Dundee APIs	7
1.6	Supported Devices	7
1.7	Device User Manuals	8
1.8	Graphical Applications	9
1.9	CUBA Software	9
1.10	STAR-System APIs	9
1.10.1	STAR-API	9
1.10.2	Generic Configuration API	10
1.10.3	RMAP Packet Library	10
1.10.4	RMAP Target API	10

1.10.5	Triggering API	11
1.10.6	PCI Mk2 Packet Subsystem API	11
1.11	Supported Languages	11
1.12	Example Programs	11
1.12.1	STAR-System Test	11
1.12.2	STAR Performance Tester	12
1.12.3	Device Configuration APIs Examples	12
1.12.4	RMAP Packet Library Example	13
1.13	Known Issues	13
1.14	Change Log	13
1.14.1	Changes between STAR-System v4.00 and v4.01 - 29th November 2019	13
1.14.2	Changes between STAR-System v3.10 and v4.00 - 19th November 2019	13
1.15	Support and Contact Information	16
2	STAR-System Applications	19
2.1	Graphical Applications	19
2.1.1	Device Configuration Application	19
2.1.2	Transmit Application	20
2.1.3	Receive Application	20
2.1.4	Source Application	20
2.1.5	Sink Application	21
2.1.6	Error Injection Application	21
2.1.7	Device Update Application	21
2.1.8	Time-code Application	21
2.1.9	Command-Line Arguments	21
2.1.10	Running the GUI Applications on QNX	22
2.1.11	Qt Open Source Components	22
2.2	CUBA Software	32
2.2.1	Data Format Conventions	32
2.2.2	SpaceWire Interactive Mode	33
2.2.3	RMAP Interactive Mode	33
2.2.4	Command File Directives	34
2.2.4.1	#INCLUDELIST Directive	34
2.2.4.2	#ALIAS Directive	35
2.2.5	SpaceWire Command Stream Mode	35
2.2.5.1	SpaceWire Send Command	35
2.2.5.2	SpaceWire Receive Command	36
2.2.5.3	Loop Command	36
2.2.5.4	Change Base Command	37
2.2.5.5	Example SpaceWire Command Stream Input File	37

2.2.5.6	SpaceWire Command Stream Output Files	37
2.2.5.7	Example SpaceWire Command Stream Output File	38
2.2.6	RMAP Command Stream Mode	38
2.2.6.1	RMAP Command Stream Input Files	38
2.2.6.2	Loop Command	39
2.2.6.3	Example RMAP Command Stream Input File	39
2.2.6.4	RMAP Command Stream Output Files	40
2.2.6.5	Example RMAP Command Stream Output File	40
2.2.7	CUBA Command Line Arguments	41
2.2.8	Backwards Compatibility Mode	42
3	STAR-System Devices	45
3.1	SpaceWire PCI Mk2 and cPCI Mk2	45
3.1.1	Overview	45
3.1.1.1	Summary	45
3.1.1.2	Hardware Description	45
3.1.2	Getting Started	47
3.1.3	Interfaces	47
3.1.3.1	SpaceWire Ports	47
3.1.3.2	Front Panel LEDs	47
3.1.4	Functions	48
3.1.4.1	SpaceWire Router	48
3.1.4.2	Interface Mode	48
3.1.4.3	Routing Mode	49
3.1.4.4	Link Operating Speed	50
3.1.4.5	Time-codes	51
3.1.5	Electrical Characteristics	51
3.1.6	Switching Characteristics	52
3.1.7	Grounding Information	53
3.2	SpaceWire PCIe	53
3.2.1	Overview	53
3.2.1.1	Summary	53
3.2.1.2	Hardware Description	53
3.2.2	Getting Started	54
3.2.3	Interfaces	55
3.2.3.1	SpaceWire Ports	55
3.2.3.2	Power LED	55
3.2.3.3	Front Panel LEDs	55
3.2.4	Functions	56
3.2.4.1	SpaceWire Router	56

3.2.4.2	Interface Mode	56
3.2.4.3	Routing Mode	57
3.2.4.4	Link Operating Speed	58
3.2.4.5	Time-codes	58
3.2.4.6	RMAP Target	59
3.2.5	Electrical Characteristics	60
3.2.6	Switching Characteristics	61
3.2.7	Grounding Information	62
3.3	SpaceWire Brick Mk2	63
3.3.1	Overview	63
3.3.1.1	Summary	63
3.3.1.2	Hardware Description	63
3.3.2	Getting Started	64
3.3.3	Interfaces	64
3.3.3.1	SpaceWire Ports and LEDs	64
3.3.4	Functions	65
3.3.4.1	SpaceWire Router	65
3.3.4.2	Interface Mode	65
3.3.4.3	Routing Mode	67
3.3.4.4	Link Operating Speed	67
3.3.4.5	Time-codes	68
3.3.5	Electrical Characteristics	69
3.3.6	Switching Characteristics	69
3.3.7	Grounding Information	70
3.3.8	EMC Conformity	71
3.4	SpaceWire Router Mk2S	71
3.4.1	Overview	71
3.4.1.1	Summary	71
3.4.1.2	Hardware Description	72
3.4.2	Getting Started	73
3.4.3	Interfaces	73
3.4.3.1	SpaceWire Ports and LEDs	73
3.4.3.2	External FIFO Ports	74
3.4.3.3	Trigger In and Out	79
3.4.4	Functions	79
3.4.4.1	SpaceWire Router	79
3.4.4.2	Interface Mode	79
3.4.4.3	Routing Mode	81
3.4.4.4	Link Operating Speed	82
3.4.4.5	Time-codes	83

3.4.5	Electrical Characteristics	83
3.4.6	Switching Characteristics	84
3.4.7	Grounding Information	85
3.4.8	EMC Conformity	85
3.5	SpaceWire Brick Mk3	87
3.5.1	Overview	87
3.5.1.1	Summary	87
3.5.1.2	Hardware Description	87
3.5.2	Getting Started	88
3.5.3	Rear Panel Interfaces	89
3.5.3.1	USB Status LED	89
3.5.3.2	Micro-B USB Connector	90
3.5.4	Front Panel Interfaces	90
3.5.5	Functions	91
3.5.5.1	SpaceWire Router	91
3.5.5.2	Interface Mode	92
3.5.5.3	Routing Mode	93
3.5.5.4	Link Operating Speed	93
3.5.5.5	Time-codes	94
3.5.5.6	Packet Timestamping	94
3.5.6	Electrical Characteristics	96
3.5.6.1	SpaceWire Connectors	96
3.5.6.2	External Trigger Connectors	97
3.5.6.3	FMECA Protection	98
3.5.7	Switching Characteristics	98
3.5.8	Grounding Information	98
3.5.9	EMC Conformity	99
3.6	SpaceWire PXI Interface and PXI Interface Mk2	99
3.6.1	Overview	99
3.6.2	Hardware Description	100
3.6.2.1	SpaceWire PXI Interface Card	101
3.6.2.2	SpaceWire PXI Interface with RMAP Target	102
3.6.3	Getting Started	103
3.6.3.1	Front Panel Interfaces	104
3.6.3.2	SpaceWire Interfaces	104
3.6.3.3	Trigger Interfaces	105
3.6.4	Supported Systems	106
3.6.4.1	cPCI Compatibility	107
3.6.4.2	PXI Compatibility	107
3.6.4.3	PXle Compatibility	108

3.6.5	Router and SpaceWire Interfaces	109
3.6.5.1	Interface Mode	109
3.6.5.2	Routing Mode	109
3.6.5.3	Link Operating Speed	110
3.6.5.4	Time-codes	110
3.6.5.5	Packet Timestamping	110
3.6.6	RMAP Target	112
3.6.6.1	Configuration and Access Modes	113
3.6.6.2	Memory Access	115
3.6.7	SpaceWire Interface Tristate Mode	115
3.6.8	Electrical Characteristics	119
3.6.8.1	Absolute Maximum Ratings	119
3.6.8.2	SpaceWire Functional Diagram	120
3.6.8.3	External Trigger Functional Diagram	121
3.6.9	Switching Characteristics	122
3.7	SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2	122
3.7.1	Overview	123
3.7.2	Hardware Description	123
3.7.2.1	SpaceWire SpaceWire PXI 12 Port Router Card	124
3.7.3	Getting Started	124
3.7.3.1	Front Panel Interfaces	125
3.7.3.2	SpaceWire Interfaces	125
3.7.4	Supported Systems	126
3.7.4.1	cPCI Compatibility	127
3.7.4.2	PXI Compatibility	128
3.7.4.3	PXle Compatibility	129
3.7.5	Router and SpaceWire Interfaces	130
3.7.5.1	Interface Mode	130
3.7.5.2	Routing Mode	131
3.7.5.3	Link Operating Speed	131
3.7.5.4	Time-codes	132
3.7.6	SpaceWire Interface Tristate Mode	132
3.7.7	Electrical Characteristics	136
3.7.7.1	Absolute Maximum Ratings	136
3.7.7.2	SpaceWire Functional Diagram	137
3.7.7.3	External Trigger Functional Diagram	138
3.7.8	Switching Characteristics	138
3.8	Other STAR-System Devices	139
3.8.1	Devices with Interface Capabilities	139
3.8.1.1	SpaceWire Physical Layer Tester	139

3.8.1.2	STAR Fire	140
3.8.1.3	STAR Fire Mk3	140
3.8.2	Devices without Interface Capabilities	140
3.8.2.1	SpaceWire Conformance Tester Mk2	140
3.8.2.2	SpaceWire EGSE and SpaceWire EGSE Mk2	140
3.8.2.3	SpaceWire Link Analyser Mk3	140
3.8.2.4	SpaceWire Recorder and SpaceWire Recorder Mk2	140
3.9	Device Update Application	141
3.9.1	Using the Device Update Application	141
3.9.1.1	Before Running the Device Update Application	141
3.9.1.2	Performing a Device Update	141
3.9.1.3	Cancelling a Device Update	142
3.9.1.4	Errors During a Device Update	143
3.9.1.5	Other Messages and Notifications	143
4	Other Useful Information	145
4.1	VxWorks Installation and Getting Started Guide	145
4.1.1	Introduction	145
4.1.2	Installation Contents	145
4.1.3	VxWorks Image Include Components	146
4.1.4	Initialising The Driver	146
4.1.5	Quick Start Guide	147
4.1.5.1	Running The Pre-compiled Test Programs	147
4.1.5.2	Building the example programs	147
4.2	RTEMS Installation and Getting Started Guide	148
4.2.1	Introduction	148
4.2.2	Supported Targets	148
4.2.3	Installation	148
4.2.4	Driver Module	148
4.2.5	Device Configuration Service	148
4.2.6	RTEMS Workspace Configuration Options	148
4.3	Multi-Threading and Multi-Process Support	149
4.3.1	Multi-Threading Support	149
4.3.2	Multi-Process Support	149
4.4	Device Configuration Service	149
4.5	Error Logs	150
4.5.1	Log File Name and Location	150
4.5.2	Log File Format	150
4.6	Configuration API Functions Supported by Device Types	151
4.6.1	Generic Configuration API	151

4.6.2	SpaceWire cPCI Mk2 and SpaceWire PCI Mk2	160
4.6.3	SpaceWire PCIe and SpaceWire Physical Layer Tester	162
4.6.4	SpaceWire Brick Mk2	164
4.6.5	SpaceWire Router Mk2S	166
4.6.6	SpaceWire Brick Mk3	168
4.6.7	SpaceWire PXI Devices	170
4.7	Static Analysis	171
4.8	Change Log	171
4.8.1	STAR-System v4.01 - 29th November 2019	172
4.8.2	STAR-System v4.00 - 19th November 2019	172
4.8.3	STAR-System v4.00 Beta 4 - 16th October 2019	173
4.8.4	STAR-System v4.00 Beta 3 - 24th October 2018	174
4.8.5	STAR-System v4.00 Beta 2 - 16th October 2018	174
4.8.6	STAR-System v4.00 Beta 1 - 25th September 2018	175
4.8.7	STAR-System v3.10 - 23rd February 2018	175
4.8.8	STAR-System v3.9 - 6th December 2017	176
4.8.9	STAR-System v3.8 - 9th November 2017	176
4.8.10	STAR-System v3.7 - 13th October 2017	176
4.8.11	STAR-System v3.6 - 3rd April 2017	176
4.8.12	STAR-System v3.5 - 17th March 2017	177
4.8.13	STAR-System v3.4 - 9th March 2017	177
4.8.14	STAR-System v3.3 - 20th December 2016	178
4.8.15	STAR-System v3.2 - 24th November 2016	178
4.8.16	STAR-System v3.1 - 21st October 2016	178
4.8.17	STAR-System v3.0 - 26th August 2016	179
4.8.18	STAR-System v3.0 Beta 9 - 26th July 2016	179
4.8.19	STAR-System v3.0 Beta 8 - 6th May 2016	180
4.8.20	STAR-System v3.0 Beta 7 - 8th March 2016	180
4.8.21	STAR-System v3.0 Beta 6 - 21st January 2016	180
4.8.22	STAR-System v3.0 Beta 5 - 25th November 2015	180
4.8.23	STAR-System v3.0 Beta 4 - 13th November 2015	180
4.8.24	STAR-System v3.0 Beta 3 - 4th September 2015	181
4.8.25	STAR-System v3.0 Beta 2 - 24th July 2015	181
4.8.26	STAR-System v3.0 Beta 1 - 31st March 2015	181
4.8.27	STAR-System v2.4.1 - 4th August 2014	183
4.8.28	STAR-System v2.4 - 2nd May 2014	183
4.8.29	STAR-System v2.3 - 31st January 2014	183
4.8.30	STAR-System v2.2 - 5th November 2013	183
4.8.31	STAR-System v2.1 - 1st August 2013	184
4.8.32	STAR-System v2.0 - 3rd July 2013	184

4.8.33	STAR-System v2.0 Beta 4 - 21st May 2013	185
4.8.34	STAR-System v2.0 Beta 3 - 12th April 2013	185
4.8.35	STAR-System v2.0 Beta 2 - 19th March 2013	186
4.8.36	STAR-System v2.0 Beta 1 - 8th February 2013	186
4.8.37	STAR-System v1.12 - 14th November 2012	187
4.8.38	STAR-System v1.11 - 31st October 2012	188
4.8.39	STAR-System v1.10 - 27th September 2012	188
4.8.40	STAR-System v1.9 - 7th September 2012	188
4.8.41	STAR-System v1.8 - 24th August 2012	188
4.8.42	STAR-System v1.7 - 20th July 2012	189
4.8.43	STAR-System v1.6 - 29th June 2012	189
4.8.44	STAR-System v1.5 - 23rd April 2012	190
4.8.45	STAR-System v1.4 - 16th March 2012	191
4.8.46	STAR-System v1.3 - 14th February 2012	191
4.8.47	STAR-System v1.2 - 13th February 2012	191
4.8.48	STAR-System v1.1 - 20th December 2011	192
4.8.49	STAR-System v1.0 - 17th October 2011	194
4.8.50	STAR-System v0.8 (Beta 1) - 22nd August 2011	194
5	Module Index	195
5.1	STAR-System C APIs	195
6	File Index	197
6.1	File List	197
7	Module Documentation	199
7.1	STAR-API	199
7.1.1	Detailed Description	200
7.1.2	Introduction	200
7.1.3	Structure	200
7.2	General API Functions	201
7.2.1	Detailed Description	201
7.2.2	Typedef Documentation	202
7.2.2.1	STAR_APP_ID	202
7.2.3	Function Documentation	202
7.2.3.1	STAR_destroyString(_Post_ptr_invalid_ In_z_ char *string)	202
7.2.3.2	STAR_destroyVersionInfo(_Post_ptr_invalid_ STAR_VERSION_INFO *p↔ VersionInfo)	202
7.2.3.3	STAR_destroyVersionInfoList(_Post_ptr_invalid_ STAR_VERSION_INFO *p↔ VersionInfoList)	202
7.2.3.4	STAR_getAllVersions(_Out_ U32 *count)	203
7.2.3.5	STAR_getApiVersion(void)	203

7.2.3.6	STAR_getApplicationAttachedToChannel(STAR_DEVICE_ID deviceId, unsigned int channelNumber)	203
7.2.3.7	STAR_getApplicationID(void)	204
7.2.3.8	STAR_getApplicationName(void)	204
7.2.3.9	STAR_getApplicationNameForID(STAR_APP_ID appId)	204
7.2.3.10	STAR_setApplicationName(_In_z_ char *name)	205
7.3	Device and Driver Management	206
7.3.1	Detailed Description	210
7.3.2	Macro Definition Documentation	210
7.3.2.1	STAR_CHANNEL_MASK	210
7.3.2.2	STAR_DEVICE_ALL	210
7.3.2.3	STAR_DEVICE_BRICK_MK2	211
7.3.2.4	STAR_DEVICE_BRICK_MK3	211
7.3.2.5	STAR_DEVICE_CONFIG_NOT_SUPPORTED	211
7.3.2.6	STAR_DEVICE_CONFIG_SUPPORTED	211
7.3.2.7	STAR_DEVICE_CONFORMANCE_TESTER	211
7.3.2.8	STAR_DEVICE_CONFORMANCE_TESTER_MK2	212
7.3.2.9	STAR_DEVICE_CPCI_MK2	212
7.3.2.10	STAR_DEVICE_EGSE	212
7.3.2.11	STAR_DEVICE_EGSE_MK2	212
7.3.2.12	STAR_DEVICE_IP_TUNNEL	212
7.3.2.13	STAR_DEVICE_LINK_ANALYSER	212
7.3.2.14	STAR_DEVICE_LINK_ANALYSER_MK2	213
7.3.2.15	STAR_DEVICE_LINK_ANALYSER_MK3	213
7.3.2.16	STAR_DEVICE_PCI	213
7.3.2.17	STAR_DEVICE_PCI_MK2	213
7.3.2.18	STAR_DEVICE_PCIE	213
7.3.2.19	STAR_DEVICE_PXI_INTERFACE	213
7.3.2.20	STAR_DEVICE_PXI_INTERFACE_MK2	214
7.3.2.21	STAR_DEVICE_PXI_RMAP	214
7.3.2.22	STAR_DEVICE_PXI_RMAP_MK2	214
7.3.2.23	STAR_DEVICE_PXI_ROUTER_12	214
7.3.2.24	STAR_DEVICE_PXI_ROUTER_8	214
7.3.2.25	STAR_DEVICE_PXI_ROUTER_MK2	215
7.3.2.26	STAR_DEVICE_RECORDER	215
7.3.2.27	STAR_DEVICE_ROUTER_MK2S	215
7.3.2.28	STAR_DEVICE_ROUTER_USB	215
7.3.2.29	STAR_DEVICE_RTC	215
7.3.2.30	STAR_DEVICE_SERIAL_SIZE	215
7.3.2.31	STAR_DEVICE_SPLT	216

7.3.2.32	STAR_DEVICE_STAR_FIRE	216
7.3.2.33	STAR_DEVICE_STAR_FIRE_MK3	216
7.3.2.34	STAR_DEVICE_STAR_ULTRA_PCIE	216
7.3.2.35	STAR_DEVICE_TRAFFIC_GENERATOR	216
7.3.2.36	STAR_DEVICE_TXRX_NOT_SUPPORTED	216
7.3.2.37	STAR_DEVICE_TXRX_SUPPORTED	217
7.3.2.38	STAR_DEVICE_TYPE	217
7.3.2.39	STAR_DEVICE_UNKNOWN	217
7.3.2.40	STAR_DEVICE_USB_BRICK	217
7.3.2.41	STAR_DEVICE_WBS_II	217
7.3.3	Typedef Documentation	218
7.3.3.1	STAR_DEVICE_ID	218
7.3.3.2	STAR_DEVICE_LISTENER_ID	218
7.3.3.3	STAR_DeviceEventFunc	218
7.3.3.4	STAR_DRIVER_ID	218
7.3.3.5	STAR_DRIVER_LISTENER_ID	218
7.3.3.6	STAR_DriverEventFunc	218
7.3.4	Enumeration Type Documentation	219
7.3.4.1	STAR_BUS_TYPE	219
7.3.4.2	STAR_DRIVER_TYPE	219
7.3.5	Function Documentation	220
7.3.5.1	STAR_destroyDeviceList(_Post_ptr_invalid_ STAR_DEVICE_ID *pDeviceList)	220
7.3.5.2	STAR_destroyDriverList(_Post_ptr_invalid_ STAR_DRIVER_ID *pDriverList)	221
7.3.5.3	STAR_getDeviceBusType(STAR_DEVICE_ID deviceId)	221
7.3.5.4	STAR_getDeviceBusTypeAsString(STAR_DEVICE_ID deviceId)	221
7.3.5.5	STAR_getDeviceChannels(STAR_DEVICE_ID deviceId)	222
7.3.5.6	STAR_getDeviceConfigCapabilities(STAR_DEVICE_ID deviceId)	222
7.3.5.7	STAR_getDeviceDriver(STAR_DEVICE_ID deviceId)	223
7.3.5.8	STAR_getDeviceFirmwareVersion(STAR_DEVICE_ID deviceId)	223
7.3.5.9	STAR_getDeviceIndex(STAR_DEVICE_ID deviceId)	223
7.3.5.10	STAR_getDeviceList(_Out_ U32 *count)	224
7.3.5.11	STAR_getDeviceListForDriver(STAR_DRIVER_ID driverId, _Out_ U32 *count)	225
7.3.5.12	STAR_getDeviceListForType(STAR_DEVICE_TYPE deviceType, _Out_ U32 *pCount)	225
7.3.5.13	STAR_getDeviceListForTypes(STAR_DEVICE_TYPE deviceTypes[], U32 numRequestedTypes, _Out_ U32 *pCount)	226
7.3.5.14	STAR_getDeviceName(STAR_DEVICE_ID deviceId)	227
7.3.5.15	STAR_getDeviceSerialNumber(STAR_DEVICE_ID deviceId)	227
7.3.5.16	STAR_getDeviceTxRxCapabilities(STAR_DEVICE_ID deviceId)	227
7.3.5.17	STAR_getDeviceType(STAR_DEVICE_ID deviceId)	228

7.3.5.18	STAR_getDeviceTypeAsString(STAR_DEVICE_ID deviceId)	228
7.3.5.19	STAR_getDriverList(int physicalDeviceDrivers, int virtualDeviceDrivers, _Out_ U32 *count)	229
7.3.5.20	STAR_getDriverType(STAR_DRIVER_ID driverId)	230
7.3.5.21	STAR_getDriverVersion(STAR_DRIVER_ID driverId)	230
7.3.5.22	STAR_getLocalDeviceAttachedToChannel(STAR_DEVICE_ID deviceId, unsigned int channelNumber)	231
7.3.5.23	STAR_isChannelOpen(STAR_DEVICE_ID deviceId, unsigned int channelNumber)	231
7.3.5.24	STAR_registerDeviceListener(_In_ STAR_DeviceEventFunc listenerFunc, _In_opt_ void *pContextInfo, int notifyForCurrent)	231
7.3.5.25	STAR_registerDeviceListenerForDevice(STAR_DEVICE_ID deviceId, _In_ STAR_DeviceEventFunc listenerFunc, _In_opt_ void *pContextInfo)	232
7.3.5.26	STAR_registerDeviceListenerForDriver(STAR_DRIVER_ID driverId, _In_ STAR_DeviceEventFunc listenerFunc, _In_opt_ void *pContextInfo, int notifyForCurrent)	232
7.3.5.27	STAR_registerDriverListener(STAR_DriverEventFunc listenerFunc, _In_opt_ void *pContextInfo, int notifyForCurrent)	232
7.3.5.28	STAR_resetDevice(STAR_DEVICE_ID deviceId)	233
7.3.5.29	STAR_resetDeviceName(STAR_DEVICE_ID deviceId)	233
7.3.5.30	STAR_setDeviceName(STAR_DEVICE_ID deviceId, _In_z_ char *newName)	233
7.3.5.31	STAR_unregisterDeviceListener(STAR_DEVICE_LISTENER_ID listenerId)	234
7.3.5.32	STAR_unregisterDriverListener(STAR_DRIVER_LISTENER_ID listenerId)	234
7.4	Channel Management	235
7.4.1	Detailed Description	236
7.4.2	Typedef Documentation	236
7.4.2.1	STAR_CHANNEL_ID	236
7.4.2.2	STAR_CHANNEL_LISTENER_ID	236
7.4.2.3	STAR_ChannelEventFunc	236
7.4.3	Enumeration Type Documentation	237
7.4.3.1	STAR_CHANNEL_DIRECTION	237
7.4.3.2	STAR_CHANNEL_TYPE	237
7.4.4	Function Documentation	237
7.4.4.1	STAR_closeChannel(STAR_CHANNEL_ID channelId)	237
7.4.4.2	STAR_getChannelDirection(STAR_CHANNEL_ID channelId)	238
7.4.4.3	STAR_openChannelBetweenLocalDevices(STAR_DEVICE_ID deviceA, unsigned char channelA, STAR_DEVICE_ID deviceB, unsigned char channelB)	238
7.4.4.4	STAR_openChannelToLocalDevice(STAR_DEVICE_ID device, STAR_CHANNEL_DIRECTION direction, unsigned char channelNumber, int isQueued)	239
7.4.4.5	STAR_openChannelToLocalDeviceEx(_In_ STAR_DEVICE_ID device, _In_ STAR_CHANNEL_DIRECTION direction, _In_ unsigned char channelNumber, _In_ int isQueued)	239
7.4.4.6	STAR_registerChannelListener(_In_ STAR_ChannelEventFunc listenerFunc, _In_opt_ void *pContextInfo, int notifyForCurrent)	240

7.4.4.7	STAR_registerChannelListenerForDevice(STAR_DEVICE_ID deviceId, _In_ STAR_ChannelEventFunc listenerFunc, _In_opt_ void *pContextInfo, int notifyForCurrent)	241
7.4.4.8	STAR_registerChannelListenerForDriver(STAR_DRIVER_ID driverId, _In_ STAR_ChannelEventFunc listenerFunc, _In_opt_ void *pContextInfo, int notifyForCurrent)	242
7.4.4.9	STAR_unregisterChannelListener(STAR_CHANNEL_LISTENER_ID listenerId)	242
7.5	Stream Items	243
7.5.1	Detailed Description	246
7.5.2	Data Structure Documentation	246
7.5.2.1	struct STAR_LINK_STATE_EVENT	246
7.5.2.2	struct STAR_SPACEWIRE_ADDRESS	247
7.5.2.3	struct STAR_DATA_CHUNK	248
7.5.2.4	struct STAR_SPACEWIRE_PACKET	249
7.5.2.5	struct STAR_TIMECODE	250
7.5.2.6	struct STAR_LINK_SPEED_EVENT	250
7.5.2.7	struct STAR_ERROR_IN_DATA_INJECT	251
7.5.2.8	struct STAR_TIMESTAMP_EVENT	252
7.5.2.9	struct STAR_BROADCAST_MESSAGE	253
7.5.2.10	struct STAR_STREAM_ITEM	254
7.5.3	Enumeration Type Documentation	255
7.5.3.1	STAR_BROADCAST_MESSAGE_STATUS_FLAG	255
7.5.3.2	STAR_EOP_TYPE	255
7.5.3.3	STAR_ERROR_IN_DATA_TYPE	256
7.5.3.4	STAR_STREAM_ITEM_TYPE	256
7.5.3.5	STAR_TIMESTAMP_DIRECTION	257
7.5.3.6	STAR_TIMESTAMP_TYPE	257
7.5.4	Function Documentation	257
7.5.4.1	STAR_createAddress(_In_count_(pathLen) unsigned char *path, U16 pathLen)	257
7.5.4.2	STAR_createBroadcastMessage(_In_ U8 channel, _In_ U8 type, _In_ U8 status, _In_ U32 dataWord1, _In_ U32 dataWord2)	258
7.5.4.3	STAR_createDataChunk(_In_opt_count_(dataLen) unsigned char *data, unsigned int dataLen, int isStart, STAR_EOP_TYPE eopType)	258
7.5.4.4	STAR_createErrorInData(STAR_ERROR_IN_DATA_TYPE errorType)	259
7.5.4.5	STAR_createPacket(_In_opt_ STAR_SPACEWIRE_ADDRESS *address, _In_opt_count_(dataLen) unsigned char *data, unsigned int dataLen, STAR_EOP_TYPE eopType)	259
7.5.4.6	STAR_createTimeCode(U8 value)	260
7.5.4.7	STAR_destroyAddress(_In_ _Post_ptr_invalid_ STAR_SPACEWIRE_ADDRESS *address)	260
7.5.4.8	STAR_destroyPacketData(_In_ _Post_ptr_invalid_ unsigned char *pData)	260
7.5.4.9	STAR_destroyStreamItem(_In_ _Post_ptr_invalid_ STAR_STREAM_ITEM *item)	261

7.5.4.10	STAR_getBroadcastMessageChannel(_In_ STAR_BROADCAST_MESSAGE *pBroadcastMessage)	261
7.5.4.11	STAR_getBroadcastMessageDataWord1(_In_ STAR_BROADCAST_MESSAGE *pBroadcastMessage)	261
7.5.4.12	STAR_getBroadcastMessageDataWord2(_In_ STAR_BROADCAST_MESSAGE *pBroadcastMessage)	262
7.5.4.13	STAR_getBroadcastMessageDelayedFlag(_In_ STAR_BROADCAST_MESSAGE *pBroadcastMessage)	262
7.5.4.14	STAR_getBroadcastMessageLateFlag(_In_ STAR_BROADCAST_MESSAGE *pBroadcastMessage)	262
7.5.4.15	STAR_getBroadcastMessageStatus(_In_ STAR_BROADCAST_MESSAGE *pBroadcastMessage)	263
7.5.4.16	STAR_getBroadcastMessageType(_In_ STAR_BROADCAST_MESSAGE *pBroadcastMessage)	263
7.5.4.17	STAR_getChunkData(_In_ STAR_DATA_CHUNK *chunk, _Out_ unsigned int *len)	263
7.5.4.18	STAR_getChunkDataLength(_In_ STAR_DATA_CHUNK *chunk)	264
7.5.4.19	STAR_getChunkEop(_In_ STAR_DATA_CHUNK *chunk)	264
7.5.4.20	STAR_getChunkSop(_In_ STAR_DATA_CHUNK *chunk)	264
7.5.4.21	STAR_getLinkSpeedEventPort(_In_ STAR_LINK_SPEED_EVENT *pLinkSpeedEvent)	265
7.5.4.22	STAR_getLinkSpeedEventSpeed(_In_ STAR_LINK_SPEED_EVENT *pLinkSpeedEvent)	265
7.5.4.23	STAR_getLinkStateEventCount(_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent)	265
7.5.4.24	STAR_getLinkStateEventPort(_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent)	266
7.5.4.25	STAR_getPacketAddress(_In_ STAR_SPACEWIRE_PACKET *packet)	266
7.5.4.26	STAR_getPacketData(_In_ STAR_SPACEWIRE_PACKET *packet, _Out_ unsigned int *dataLength)	266
7.5.4.27	STAR_getPacketEOP(_In_ STAR_SPACEWIRE_PACKET *packet)	267
7.5.4.28	STAR_getPacketLength(_In_ STAR_SPACEWIRE_PACKET *packet)	267
7.5.4.29	STAR_getTimeCodeValue(_In_ STAR_TIMECODE *pTimeCode)	268
7.5.4.30	STAR_getTimestampEventDirection(_In_ STAR_TIMESTAMP_EVENT *pTimestampEvent)	268
7.5.4.31	STAR_getTimestampEventEndValue(_In_ STAR_TIMESTAMP_EVENT *pTimestampEvent, U32 syncPulseFrequency, _Out_ U32 *pSeconds, _Out_ U32 *pRemainder)	268
7.5.4.32	STAR_getTimestampEventRawCounters(_In_ STAR_TIMESTAMP_EVENT *pTimestampEvent, _Out_opt_ U32 *pStartSyncPulseCount, _Out_opt_ U32 *pStartClockCycleCount, _Out_opt_ U32 *pStartTotalCycleCount, _Out_opt_ U32 *pEndSyncPulseCount, _Out_opt_ U32 *pEndClockCycleCount, _Out_opt_ U32 *pEndTotalCycleCount, _Out_opt_ U32 *pClockFrequency)	269
7.5.4.33	STAR_getTimestampEventStartValue(_In_ STAR_TIMESTAMP_EVENT *pTimestampEvent, U32 syncPulseFrequency, _Out_ U32 *pSeconds, _Out_ U32 *pRemainder)	270

7.5.4.34	STAR_getTimestampEventType(_In_ STAR_TIMESTAMP_EVENT *pTimestampEvent)	270
7.5.4.35	STAR_isLinkStateEventDisconnectError(_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent)	271
7.5.4.36	STAR_isLinkStateEventEscapeError(_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent)	271
7.5.4.37	STAR_isLinkStateEventLinkRunning(_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent)	271
7.5.4.38	STAR_isLinkStateEventParityError(_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent)	272
7.5.4.39	STAR_isLinkStateEventReceiveCreditError(_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent)	272
7.5.4.40	STAR_isLinkStateEventTransmitCreditError(_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent)	272
7.5.4.41	STAR_setPacketAddress(_Inout_ STAR_SPACEWIRE_PACKET *packet, _In_ STAR_SPACEWIRE_ADDRESS *address)	273
7.5.4.42	STAR_setPacketEOP(_In_ STAR_SPACEWIRE_PACKET *packet, STAR_EOP_TYPE eopType)	274
7.5.4.43	STAR_setTimeCodeValue(_In_ STAR_TIMECODE *pTimeCode, U8 value)	274
7.6	Transfer Operations	275
7.6.1	Detailed Description	277
7.6.2	Typedef Documentation	277
7.6.2.1	STAR_TRANSFER_OPERATION	277
7.6.2.2	STAR_TransferOperationListenerFunc	277
7.6.3	Enumeration Type Documentation	278
7.6.3.1	STAR_RECEIVE_MASK	278
7.6.3.2	STAR_TRANSFER_STATUS	278
7.6.4	Function Documentation	279
7.6.4.1	STAR_cancelTransferOperation(_In_ STAR_TRANSFER_OPERATION *pTransferOperation)	279
7.6.4.2	STAR_cancelTransferOperationWaits(_In_ STAR_TRANSFER_OPERATION *pTransferOperation)	279
7.6.4.3	STAR_createRxOperation(int itemCount, STAR_RECEIVE_MASK mask)	280
7.6.4.4	STAR_createRxOperationEx(_In_count_(numBuffers) U8 **ppBuffers, _In_ unsigned int numBuffers, _In_ unsigned int bufferSize, _In_ STAR_RECEIVE_MASK mask)	280
7.6.4.5	STAR_createTxOperation(_In_count_(streamItemCount) STAR_STREAM_ITEM **ppItems, unsigned int streamItemCount)	281
7.6.4.6	STAR_destroyTransferItemList(_In_ _Post_ptr_invalid_ STAR_STREAM_ITEM **transferItemList, unsigned int transferItemListCount, int destroyItems)	282
7.6.4.7	STAR_disposeTransferOperation(_In_ _Post_ptr_invalid_ STAR_TRANSFER_OPERATION *pTransferOperation)	282
7.6.4.8	STAR_getNextTransferItem(STAR_STREAM_ITEM *pStreamItem)	283
7.6.4.9	STAR_getSpaceWireAddressPath(STAR_SPACEWIRE_ADDRESS *pSpaceWireAddress)	284

7.6.4.10	STAR_getSpaceWireAddressPathLength(STAR_SPACEWIRE_ADDRESS *pSpaceWireAddress)	284
7.6.4.11	STAR_getTransferItem(_In_ STAR_TRANSFER_OPERATION *pReceiveOperation, unsigned int index)	284
7.6.4.12	STAR_getTransferItemCount(_In_ STAR_TRANSFER_OPERATION *pReceiveOperation)	285
7.6.4.13	STAR_getTransferItemList(_In_ STAR_TRANSFER_OPERATION *pReceiveOperation, _Out_ unsigned int *count)	285
7.6.4.14	STAR_getTransferOperationStatus(_In_ STAR_TRANSFER_OPERATION *pTransferOperation)	286
7.6.4.15	STAR_receivePacket(_In_ STAR_CHANNEL_ID channelId, _Out_ bytewidth (*pPacketLength) void *pPacketData, _Inout_ unsigned int *pPacketLength, _Out_ STAR_EOP_TYPE *pEopType, _In_ int timeout)	286
7.6.4.16	STAR_registerTransferCompletionListener(_In_ STAR_TRANSFER_OPERATION *pTransferOperation, STAR_TransferOperationListenerFunc listenerFunc, _In_opt_ void *pContextInfo)	287
7.6.4.17	STAR_rxOperationExGetBroadcastMessage(_In_ STAR_TRANSFER_OPERATION *pTransferOp, _In_ unsigned int item, _Out_ STAR_BROADCAST_MESSAGE *pBroadcastMessage)	287
7.6.4.18	STAR_rxOperationExGetDataChunk(_In_ STAR_TRANSFER_OPERATION *pTransferOp, _In_ unsigned int item, _Out_ STAR_DATA_CHUNK *pDataChunk)	288
7.6.4.19	STAR_rxOperationExGetItemType(_In_ STAR_TRANSFER_OPERATION *pTransferOp, _In_ unsigned int item)	288
7.6.4.20	STAR_rxOperationExGetNumItems(_In_ STAR_TRANSFER_OPERATION *pTransferOp)	288
7.6.4.21	STAR_rxOperationExGetStatus(_In_ STAR_TRANSFER_OPERATION *pTransferOp)	289
7.6.4.22	STAR_submitTransferOperation(_In_ STAR_CHANNEL_ID channelId, _In_ STAR_TRANSFER_OPERATION *transferOp)	289
7.6.4.23	STAR_submitTransferOperationList(STAR_CHANNEL_ID channelId, _In_ count(count) STAR_TRANSFER_OPERATION **transferOps, unsigned int count)	290
7.6.4.24	STAR_transmitPacket(_In_ STAR_CHANNEL_ID channelId, _In_opt_ bytewidth(packetLength) void *pPacketData, _In_ unsigned int packetLength, _In_ STAR_EOP_TYPE eopType, _In_ int timeout)	290
7.6.4.25	STAR_unregisterTransferCompletionListener(_In_ STAR_TRANSFER_OPERATION *pTransferOperation, STAR_TransferOperationListenerFunc listenerFunc)	291
7.6.4.26	STAR_waitOnTransferOperationCompletion(_In_ STAR_TRANSFER_OPERATION *pTransferOperation, int timeout)	291
7.7	Virtual Devices and Drivers	292
7.7.1	Detailed Description	292
7.7.2	Function Documentation	292
7.7.2.1	STAR_isDeviceVirtual(STAR_DEVICE_ID deviceId)	292
7.7.2.2	STAR_isDriverVirtual(STAR_DRIVER_ID driverId)	292
7.8	Link Speed	294
7.8.1	Detailed Description	294

7.8.2	Enumeration Type Documentation	294
7.8.2.1	STAR_CFG_PCIMK2_LINK_FREQ	294
7.8.3	Function Documentation	295
7.8.3.1	CFG_PCIMK2_getLinkClockFrequency(STAR_DEVICE_ID deviceID, U8 linkNum, STAR_CFG_PCIMK2_LINK_FREQ *pLinkFreq)	295
7.8.3.2	CFG_PCIMK2_setLinkClockFrequency(STAR_DEVICE_ID deviceID, U8 linkNum, STAR_CFG_PCIMK2_LINK_FREQ linkFreq)	295
7.9	Link Speed	297
7.9.1	Detailed Description	297
7.9.2	Enumeration Type Documentation	297
7.9.2.1	STAR_CFG_BRICK_MK2_LINK_FREQ	297
7.9.3	Function Documentation	298
7.9.3.1	CFG_BRICK_MK2_getLinkClockFrequency(STAR_DEVICE_ID deviceID, U8 linkNum, _Out_ STAR_CFG_BRICK_MK2_LINK_FREQ *pLinkFreq)	298
7.9.3.2	CFG_BRICK_MK2_setLinkClockFrequency(STAR_DEVICE_ID deviceID, U8 linkNum, STAR_CFG_BRICK_MK2_LINK_FREQ linkFreq)	298
7.10	Interface Mode	300
7.10.1	Detailed Description	300
7.10.2	Function Documentation	300
7.10.2.1	CFG_BRICK_MK2_disableIdentifySourceOnPort(STAR_DEVICE_ID deviceID, U8 portNum)	300
7.10.2.2	CFG_BRICK_MK2_disableInterfaceModeOnPort(STAR_DEVICE_ID deviceID, U8 portNum)	301
7.10.2.3	CFG_BRICK_MK2_enableIdentifySourceOnPort(STAR_DEVICE_ID deviceID, U8 portNum)	301
7.10.2.4	CFG_BRICK_MK2_enableInterfaceModeOnPort(STAR_DEVICE_ID deviceID, U8 portNum)	303
7.10.2.5	CFG_BRICK_MK2_getIdentifySourceOnPortEnabled(STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int *pEnabled)	303
7.10.2.6	CFG_BRICK_MK2_getInterfaceModeOnPortEnabled(STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int *pEnabled)	304
7.10.2.7	CFG_BRICK_MK2_getPortRoutingAddress(STAR_DEVICE_ID deviceID, U8 portNum, _Out_ U8 *pAddress)	304
7.10.2.8	CFG_BRICK_MK2_setPortRoutingAddress(STAR_DEVICE_ID deviceID, U8 portNum, U8 address)	305
7.11	Error Injection	306
7.11.1	Detailed Description	306
7.11.2	Function Documentation	306
7.11.2.1	CFG_BRICK_MK2_injectErrors(STAR_DEVICE_ID deviceID, U8 portNum, _In_ STAR_CFG_BRICK_MK2_ERRORS *pErrors)	306
7.12	Events	307
7.12.1	Detailed Description	307
7.12.2	Function Documentation	307

7.12.2.1	CFG_BRICK_MK2_disableSpeedChangeEventsOnPort(STAR_DEVICE_ID deviceID, U8 portNum)	307
7.12.2.2	CFG_BRICK_MK2_disableStateChangeEventsOnPort(STAR_DEVICE_ID deviceID, U8 portNum)	309
7.12.2.3	CFG_BRICK_MK2_enableSpeedChangeEventsOnPort(STAR_DEVICE_ID deviceID, U8 portNum)	309
7.12.2.4	CFG_BRICK_MK2_enableStateChangeEventsOnPort(STAR_DEVICE_ID deviceID, U8 portNum)	310
7.12.2.5	CFG_BRICK_MK2_getSpeedChangeEventsOnPortEnabled(STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int *pEnabled)	310
7.12.2.6	CFG_BRICK_MK2_getStateChangeEventsOnPortEnabled(STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int *pEnabled)	311
7.13	Measured Link Speed	312
7.13.1	Detailed Description	312
7.13.2	Macro Definition Documentation	312
7.13.2.1	STAR_CFG_BRICK_MK2_LINK_SPEED_UNITS_KBPS	312
7.14	Link Speed	313
7.14.1	Detailed Description	313
7.14.2	Function Documentation	313
7.14.2.1	CFG_BRICK_MK3_getBaseTransmitClock(STAR_DEVICE_ID deviceID, U8 linkNum, _Out_ STAR_CFG_MK2_BASE_TRANSMIT_CLOCK *pClockRateParams)	313
7.14.2.2	CFG_BRICK_MK3_getTransmitClock(STAR_DEVICE_ID deviceID, U8 linkNum, _Out_ STAR_CFG_MK2_BASE_TRANSMIT_CLOCK *pClockRateParams)	314
7.14.2.3	CFG_BRICK_MK3_setBaseTransmitClock(STAR_DEVICE_ID deviceID, U8 linkNum, STAR_CFG_MK2_BASE_TRANSMIT_CLOCK clockRateParams)	314
7.14.2.4	CFG_BRICK_MK3_setTransmitClock(STAR_DEVICE_ID deviceID, U8 linkNum, STAR_CFG_MK2_BASE_TRANSMIT_CLOCK clockRateParams)	315
7.15	Timestamping	317
7.15.1	Detailed Description	318
7.15.2	Enumeration Type Documentation	318
7.15.2.1	STAR_CFG_BRICK_MK3_PULSE_FREQ	318
7.15.2.2	STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD	318
7.15.3	Function Documentation	318
7.15.3.1	CFG_BRICK_MK3_disableRxTimestampEventsOnPort(STAR_DEVICE_ID deviceID, U8 portNum)	318
7.15.3.2	CFG_BRICK_MK3_enableRxTimestampEventsOnPort(STAR_DEVICE_ID deviceID, U8 portNum)	319
7.15.3.3	CFG_BRICK_MK3_getPulseGeneratorFrequency(STAR_DEVICE_ID deviceID, _Out_ STAR_CFG_BRICK_MK3_PULSE_FREQ *pFrequency)	319
7.15.3.4	CFG_BRICK_MK3_getRxTimestampEventsOnPortEnabled(STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int *pEnabled)	320
7.15.3.5	CFG_BRICK_MK3_getTimestampMethod(STAR_DEVICE_ID deviceID, _Out_ STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD *pTimestampMethod)	320

7.15.3.6	CFG_BRICK_MK3_getTimestampValue(STAR_DEVICE_ID deviceId, _Out_ U32 *pValue)	321
7.15.3.7	CFG_BRICK_MK3_setPulseGeneratorFrequency(STAR_DEVICE_ID deviceId, STAR_CFG_BRICK_MK3_PULSE_FREQ frequency)	321
7.15.3.8	CFG_BRICK_MK3_setTimestampMethod(STAR_DEVICE_ID deviceId, STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD timestampMethod)	322
7.15.3.9	CFG_BRICK_MK3_setTimestampValue(STAR_DEVICE_ID deviceId, U32 value)	322
7.16	Link Speed	324
7.16.1	Detailed Description	324
7.16.2	Function Documentation	324
7.16.2.1	CFG_PXI_getBaseTransmitClock(STAR_DEVICE_ID deviceId, U8 linkNum, _Out_ STAR_CFG_MK2_BASE_TRANSMIT_CLOCK *clockRateParams)	324
7.16.2.2	CFG_PXI_getLinkRateDivider(STAR_DEVICE_ID deviceId, _Out_ U8 *pDivider)	325
7.16.2.3	CFG_PXI_getTransmitClock(STAR_DEVICE_ID deviceId, U8 linkNum, _Out_ STAR_CFG_MK2_BASE_TRANSMIT_CLOCK *clockRateParams)	325
7.16.2.4	CFG_PXI_setBaseTransmitClock(STAR_DEVICE_ID deviceId, U8 linkNum, STAR_CFG_MK2_BASE_TRANSMIT_CLOCK clockRateParams)	326
7.16.2.5	CFG_PXI_setLinkRateDivider(STAR_DEVICE_ID deviceId, U8 divider)	327
7.16.2.6	CFG_PXI_setTransmitClock(STAR_DEVICE_ID deviceId, U8 linkNum, STAR_CFG_MK2_BASE_TRANSMIT_CLOCK clockRateParams)	327
7.17	Error Injection	329
7.17.1	Detailed Description	329
7.17.2	Function Documentation	329
7.17.2.1	CFG_PXI_injectError(STAR_DEVICE_ID deviceId, unsigned char port, SPW_ERROR error)	329
7.18	Interface Mode	330
7.18.1	Detailed Description	330
7.18.2	Function Documentation	330
7.18.2.1	CFG_PXI_disableIdentifySourceOnPort(STAR_DEVICE_ID deviceId, U8 portNum)	330
7.18.2.2	CFG_PXI_disableInterfaceModeOnPort(STAR_DEVICE_ID deviceId, U8 portNum)	331
7.18.2.3	CFG_PXI_enableIdentifySourceOnPort(STAR_DEVICE_ID deviceId, U8 portNum)	331
7.18.2.4	CFG_PXI_enableInterfaceModeOnPort(STAR_DEVICE_ID deviceId, U8 portNum)	332
7.18.2.5	CFG_PXI_getIdentifySourceOnPortEnabled(STAR_DEVICE_ID deviceId, U8 portNum, _Out_ int *pEnabled)	332
7.18.2.6	CFG_PXI_getInterfaceModeOnPortEnabled(STAR_DEVICE_ID deviceId, U8 portNum, _Out_ int *pEnabled)	332
7.18.2.7	CFG_PXI_getPortRoutingAddress(STAR_DEVICE_ID deviceId, U8 portNum, _Out_ U8 *pAddress)	333
7.18.2.8	CFG_PXI_setPortRoutingAddress(STAR_DEVICE_ID deviceId, U8 portNum, U8 address)	333
7.19	Events	335
7.19.1	Detailed Description	335
7.19.2	Function Documentation	335

7.19.2.1	CFG_PXI_disableSpeedChangeEventsOnPort(STAR_DEVICE_ID deviceId, U8 portNum)	335
7.19.2.2	CFG_PXI_disableStateChangeEventsOnPort(STAR_DEVICE_ID deviceId, U8 portNum)	336
7.19.2.3	CFG_PXI_disableTimeCodeEventsOnPort(STAR_DEVICE_ID deviceId, U8 portNum)	336
7.19.2.4	CFG_PXI_enableSpeedChangeEventsOnPort(STAR_DEVICE_ID deviceId, U8 portNum)	337
7.19.2.5	CFG_PXI_enableStateChangeEventsOnPort(STAR_DEVICE_ID deviceId, U8 portNum)	337
7.19.2.6	CFG_PXI_enableTimeCodeEventsOnPort(STAR_DEVICE_ID deviceId, U8 portNum)	338
7.19.2.7	CFG_PXI_getSpeedChangeEventsOnPortEnabled(STAR_DEVICE_ID deviceId, U8 portNum, _Out_ int *pEnabled)	338
7.19.2.8	CFG_PXI_getStateChangeEventsOnPortEnabled(STAR_DEVICE_ID deviceId, U8 portNum, _Out_ int *pEnabled)	338
7.19.2.9	CFG_PXI_getTimeCodeEventsOnPortEnabled(STAR_DEVICE_ID deviceId, U8 portNum, _Out_ int *pEnabled)	339
7.20	RMAP Target API	340
7.20.1	Detailed Description	340
7.21	Manual Authorisation	342
7.21.1	Detailed Description	342
7.21.2	Data Structure Documentation	342
7.21.2.1	struct RmapCommandParameters	342
7.21.3	Enumeration Type Documentation	343
7.21.3.1	REJECTION_REASON	343
7.21.4	Function Documentation	344
7.21.4.1	RMAP_TARGET_PXI_IF_authouriseWaitingCommand(STAR_DEVICE_ID deviceId, U32 target)	344
7.21.4.2	RMAP_TARGET_PXI_IF_getWaitingCommand(STAR_DEVICE_ID deviceId, U32 target, RmapCommandParameters *pCommand)	344
7.21.4.3	RMAP_TARGET_PXI_IF_isAuthRequired(STAR_DEVICE_ID deviceId, U32 target)	344
7.21.4.4	RMAP_TARGET_PXI_IF_rejectWaitingCommand(STAR_DEVICE_ID deviceId, U32 target, REJECTION_REASON reason)	345
7.22	Configuration	346
7.22.1	Detailed Description	348
7.22.2	Data Structure Documentation	348
7.22.2.1	struct TARGET_ENGINE	348
7.22.3	Typedef Documentation	349
7.22.3.1	TARGET_AUTH_CMD_MASK	349
7.22.4	Enumeration Type Documentation	349
7.22.4.1	IF_MODE	349
7.22.4.2	TARGET_AUTH_CMD	349
7.22.4.3	TARGET_AUTH_MODE	349

7.22.4.4	TARGET_STATUS	349
7.22.5	Function Documentation	350
7.22.5.1	RMAP_TARGET_PXI_IF_getAddressOffset(STAR_DEVICE_ID deviceId, U32 target, U32 *pOffset)	350
7.22.5.2	RMAP_TARGET_PXI_IF_getAuthCommands(STAR_DEVICE_ID deviceId, U32 target, TARGET_AUTH_CMD_MASK *pCommands)	350
7.22.5.3	RMAP_TARGET_PXI_IF_getAuthControlMode(STAR_DEVICE_ID deviceId, U32 target, TARGET_AUTH_MODE *pMode)	350
7.22.5.4	RMAP_TARGET_PXI_IF_getAuthKeyRange(STAR_DEVICE_ID deviceId, U32 target, U8 *pLowest, U8 *pHighest)	351
7.22.5.5	RMAP_TARGET_PXI_IF_getAuthLogicalAddressRange(STAR_DEVICE_ID deviceId, U32 target, U8 *pLowest, U8 *pHighest)	351
7.22.5.6	RMAP_TARGET_PXI_IF_getAuthMemoryAddressRange(STAR_DEVICE_ID deviceId, U32 target, U32 *pLowerBoundary, U32 *pUpperBoundary)	352
7.22.5.7	RMAP_TARGET_PXI_IF_getAuthProtocolId(STAR_DEVICE_ID deviceId, U32 target, U8 *pProtocolId)	352
7.22.5.8	RMAP_TARGET_PXI_IF_getInterfaceMode(STAR_DEVICE_ID deviceId, U32 port, IF_MODE *pMode)	353
7.22.5.9	RMAP_TARGET_PXI_IF_getStatus(STAR_DEVICE_ID deviceId, U32 target, TARGET_STATUS *pStatus)	354
7.22.5.10	RMAP_TARGET_PXI_IF_readMemory(STAR_DEVICE_ID deviceId, U32 target, U32 address, U32 length, unsigned char *pBuffer)	354
7.22.5.11	RMAP_TARGET_PXI_IF_setAddressOffset(STAR_DEVICE_ID deviceId, U32 target, U32 offset)	355
7.22.5.12	RMAP_TARGET_PXI_IF_setAuthCommands(STAR_DEVICE_ID deviceId, U32 target, TARGET_AUTH_CMD_MASK commands)	355
7.22.5.13	RMAP_TARGET_PXI_IF_setAuthControlMode(STAR_DEVICE_ID deviceId, U32 target, TARGET_AUTH_MODE mode)	356
7.22.5.14	RMAP_TARGET_PXI_IF_setAuthKeyRange(STAR_DEVICE_ID deviceId, U32 target, U8 lowest, U8 highest)	356
7.22.5.15	RMAP_TARGET_PXI_IF_setAuthLogicalAddressRange(STAR_DEVICE_ID deviceId, U32 target, U8 lowest, U8 highest)	357
7.22.5.16	RMAP_TARGET_PXI_IF_setAuthMemoryAddressRange(STAR_DEVICE_ID deviceId, U32 target, U32 lowerBoundary, U32 upperBoundary)	357
7.22.5.17	RMAP_TARGET_PXI_IF_setAuthProtocolId(STAR_DEVICE_ID deviceId, U32 target, U8 protocolId)	358
7.22.5.18	RMAP_TARGET_PXI_IF_setInterfaceMode(STAR_DEVICE_ID deviceId, U32 port, IF_MODE mode)	358
7.22.5.19	RMAP_TARGET_PXI_IF_writeMemory(STAR_DEVICE_ID deviceId, U32 target, U32 address, U32 length, const unsigned char *pBuffer)	358
7.23	Notifications	360
7.23.1	Detailed Description	361
7.23.2	Typedef Documentation	361
7.23.2.1	NOTIF_TYPE_MASK	361
7.23.2.2	NotifListenerFunc	361
7.23.2.3	NotifListenerWithContextFunc	362

7.23.3	Enumeration Type Documentation	362
7.23.3.1	NOTIF_TYPE	362
7.23.4	Function Documentation	362
7.23.4.1	RMAP_TARGET_PXI_IF_getEnabledNotifications(STAR_DEVICE_ID deviceld, U32 target, NOTIF_TYPE_MASK *pNotifications)	363
7.23.4.2	RMAP_TARGET_PXI_IF_registerNotificationListener(STAR_DEVICE_ID deviceld, U32 target, NOTIF_TYPE type, NotifListenerFunc listenerFunc)	364
7.23.4.3	RMAP_TARGET_PXI_IF_registerNotificationListenerWithContext(STAR_DEVICE_ID deviceld, U32 target, NOTIF_TYPE type, NotifListenerWithContextFunc listenerFunc, void *pContext)	364
7.23.4.4	RMAP_TARGET_PXI_IF_setEnabledNotifications(STAR_DEVICE_ID deviceld, U32 target, NOTIF_TYPE_MASK notifications)	365
7.23.4.5	RMAP_TARGET_PXI_IF_startReceivingNotifications(STAR_DEVICE_ID deviceld)	365
7.23.4.6	RMAP_TARGET_PXI_IF_stopReceivingNotifications(STAR_DEVICE_ID deviceld)	366
7.23.4.7	RMAP_TARGET_PXI_IF_unregisterNotificationListener(STAR_DEVICE_ID deviceld, U32 target, NOTIF_TYPE type)	366
7.23.4.8	RMAP_TARGET_PXI_IF_unregisterNotificationListenerWithContext(STAR_DEVICE_ID deviceld, U32 target, NOTIF_TYPE type)	366
7.24	Triggering API	368
7.24.1	Detailed Description	368
7.24.2	Input Events	368
7.24.3	Output Actions	371
7.24.4	Internal Triggers	373
7.24.5	Triggering Resources	375
7.24.6	PCIe Differences	377
7.24.7	Examples	378
7.24.7.1	Transmitting Time-Codes Synchronised by an External Trigger	378
7.24.7.2	Packet Transmission Synchronised by a Counter	378
7.24.7.3	Multiple Packet Transmission Synchronised by Time-Codes	379
7.25	Events	380
7.25.1	Detailed Description	383
7.25.2	Typedef Documentation	383
7.25.2.1	COUNTER_ACTION_MASK	383
7.25.2.2	COUNTER_EVENT_MASK	383
7.25.2.3	EXT_TRIGGER_ACTION_MASK	383
7.25.2.4	EXT_TRIGGER_EVENT_MASK	383
7.25.2.5	PORT_ACTION_MASK	384
7.25.2.6	PORT_EVENT_MASK	384
7.25.2.7	TIME_CODE_ACTION_MASK	384
7.25.2.8	TIME_CODE_EVENT_MASK	384
7.25.2.9	TRIGGER_EVENT_MASK	384

7.25.3	Enumeration Type Documentation	384
7.25.3.1	COUNTER_EVENT	384
7.25.3.2	EXT_TRIGGER_EVENT	385
7.25.3.3	PORT_EVENT	385
7.25.3.4	TIME_CODE_EVENT	385
7.25.3.5	TRIGGER_EVENT	386
7.25.4	Function Documentation	386
7.25.4.1	TRIGGER_BRICK_MK3_getCounterInputEvents(STAR_DEVICE_ID deviceld, U32 counter, U32 trigger, COUNTER_EVENT_MASK *pEvents)	386
7.25.4.2	TRIGGER_BRICK_MK3_getExtTriggerInputEvents(STAR_DEVICE_ID deviceld, U32 extTrigger, U32 trigger, EXT_TRIGGER_EVENT_MASK *pEvents)	386
7.25.4.3	TRIGGER_BRICK_MK3_getPortInputEvents(STAR_DEVICE_ID deviceld, U32 port, U32 trigger, PORT_EVENT_MASK *pEvents)	387
7.25.4.4	TRIGGER_BRICK_MK3_getTimeCodeInputEvents(STAR_DEVICE_ID deviceld, U32 timeCode, U32 trigger, TIME_CODE_EVENT_MASK *pEvents)	387
7.25.4.5	TRIGGER_BRICK_MK3_getTriggerInputEvents(STAR_DEVICE_ID deviceld, U32 causeTrigger, U32 trigger, TRIGGER_EVENT_MASK *pEvents)	387
7.25.4.6	TRIGGER_BRICK_MK3_setCounterInputEvents(STAR_DEVICE_ID deviceld, U32 counter, U32 trigger, COUNTER_EVENT_MASK events)	388
7.25.4.7	TRIGGER_BRICK_MK3_setExtTriggerInputEvents(STAR_DEVICE_ID deviceld, U32 extTrigger, U32 trigger, EXT_TRIGGER_EVENT_MASK events)	388
7.25.4.8	TRIGGER_BRICK_MK3_setPortInputEvents(STAR_DEVICE_ID deviceld, U32 port, U32 trigger, PORT_EVENT_MASK events)	389
7.25.4.9	TRIGGER_BRICK_MK3_setTimeCodeInputEvents(STAR_DEVICE_ID deviceld, U32 timeCode, U32 trigger, TIME_CODE_EVENT_MASK events)	389
7.25.4.10	TRIGGER_BRICK_MK3_setTriggerInputEvents(STAR_DEVICE_ID deviceld, U32 causeTrigger, U32 trigger, TRIGGER_EVENT_MASK events)	390
7.25.4.11	TRIGGER_PCIE_IF_getCounterInputEvents(STAR_DEVICE_ID deviceld, U32 counter, U32 trigger, COUNTER_EVENT_MASK *pEvents)	390
7.25.4.12	TRIGGER_PCIE_IF_getPortInputEvents(STAR_DEVICE_ID deviceld, U32 port, U32 trigger, PORT_EVENT_MASK *pEvents)	391
7.25.4.13	TRIGGER_PCIE_IF_getTriggerInputEvents(STAR_DEVICE_ID deviceld, U32 causeTrigger, U32 trigger, TRIGGER_EVENT_MASK *pEvents)	391
7.25.4.14	TRIGGER_PCIE_IF_setCounterInputEvents(STAR_DEVICE_ID deviceld, U32 counter, U32 trigger, COUNTER_EVENT_MASK events)	392
7.25.4.15	TRIGGER_PCIE_IF_setPortInputEvents(STAR_DEVICE_ID deviceld, U32 port, U32 trigger, PORT_EVENT_MASK events)	392
7.25.4.16	TRIGGER_PCIE_IF_setTriggerInputEvents(STAR_DEVICE_ID deviceld, U32 causeTrigger, U32 trigger, TRIGGER_EVENT_MASK events)	392
7.25.4.17	TRIGGER_PXI_IF_getCounterInputEvents(STAR_DEVICE_ID deviceld, U32 counter, U32 trigger, COUNTER_EVENT_MASK *pEvents)	393
7.25.4.18	TRIGGER_PXI_IF_getExtTriggerInputEvents(STAR_DEVICE_ID deviceld, U32 extTrigger, U32 trigger, EXT_TRIGGER_EVENT_MASK *pEvents)	393
7.25.4.19	TRIGGER_PXI_IF_getPortInputEvents(STAR_DEVICE_ID deviceld, U32 port, U32 trigger, PORT_EVENT_MASK *pEvents)	394

7.25.4.20	TRIGGER_PXI_IF_getTimeCodeInputEvents(STAR_DEVICE_ID deviceId, U32 timeCode, U32 trigger, TIME_CODE_EVENT_MASK *pEvents)	394
7.25.4.21	TRIGGER_PXI_IF_getTriggerInputEvents(STAR_DEVICE_ID deviceId, U32 causeTrigger, U32 trigger, TRIGGER_EVENT_MASK *pEvents)	395
7.25.4.22	TRIGGER_PXI_IF_setCounterInputEvents(STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_EVENT_MASK events)	396
7.25.4.23	TRIGGER_PXI_IF_setExtTriggerInputEvents(STAR_DEVICE_ID deviceId, U32 extTrigger, U32 trigger, EXT_TRIGGER_EVENT_MASK events)	396
7.25.4.24	TRIGGER_PXI_IF_setPortInputEvents(STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_EVENT_MASK events)	397
7.25.4.25	TRIGGER_PXI_IF_setTimeCodeInputEvents(STAR_DEVICE_ID deviceId, U32 timeCode, U32 trigger, TIME_CODE_EVENT_MASK events)	397
7.25.4.26	TRIGGER_PXI_IF_setTriggerInputEvents(STAR_DEVICE_ID deviceId, U32 causeTrigger, U32 trigger, TRIGGER_EVENT_MASK events)	398
7.25.4.27	TRIGGER_PXI_ROUTER_getCounterInputEvents(STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_EVENT_MASK *pEvents)	398
7.25.4.28	TRIGGER_PXI_ROUTER_getPortInputEvents(STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_EVENT_MASK *pEvents)	399
7.25.4.29	TRIGGER_PXI_ROUTER_getTimeCodeInputEvents(STAR_DEVICE_ID deviceId, U32 timeCode, U32 trigger, TIME_CODE_EVENT_MASK *pEvents)	399
7.25.4.30	TRIGGER_PXI_ROUTER_getTriggerInputEvents(STAR_DEVICE_ID deviceId, U32 causeTrigger, U32 trigger, TRIGGER_EVENT_MASK *pEvents)	400
7.25.4.31	TRIGGER_PXI_ROUTER_setCounterInputEvents(STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_EVENT_MASK events)	400
7.25.4.32	TRIGGER_PXI_ROUTER_setPortInputEvents(STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_EVENT_MASK events)	400
7.25.4.33	TRIGGER_PXI_ROUTER_setTimeCodeInputEvents(STAR_DEVICE_ID deviceId, U32 timeCode, U32 trigger, TIME_CODE_EVENT_MASK events)	401
7.25.4.34	TRIGGER_PXI_ROUTER_setTriggerInputEvents(STAR_DEVICE_ID deviceId, U32 causeTrigger, U32 trigger, TRIGGER_EVENT_MASK events)	401
7.26	Actions	403
7.26.1	Detailed Description	405
7.26.2	Enumeration Type Documentation	405
7.26.2.1	COUNTER_ACTION	405
7.26.2.2	EXT_TRIGGER_ACTION	405
7.26.2.3	PORT_ACTION	406
7.26.2.4	TIME_CODE_ACTION	406
7.26.3	Function Documentation	406
7.26.3.1	TRIGGER_BRICK_MK3_getCounterOutputActions(STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_ACTION_MASK *pActions)	406
7.26.3.2	TRIGGER_BRICK_MK3_getExtTriggerOutputActions(STAR_DEVICE_ID deviceId, U32 extTrigger, U32 trigger, EXT_TRIGGER_ACTION_MASK *pActions)	407
7.26.3.3	TRIGGER_BRICK_MK3_getPortOutputActions(STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_ACTION_MASK *pActions)	407

7.26.3.4	TRIGGER_BRICK_MK3_getTimeCodeOutputActions(STAR_DEVICE_ID deviceId, U32 timeCode, U32 trigger, TIME_CODE_ACTION_MASK *pActions)	408
7.26.3.5	TRIGGER_BRICK_MK3_setCounterOutputActions(STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_ACTION_MASK actions)	408
7.26.3.6	TRIGGER_BRICK_MK3_setExtTriggerOutputActions(STAR_DEVICE_ID deviceId, U32 extTrigger, U32 trigger, EXT_TRIGGER_ACTION_MASK actions)	409
7.26.3.7	TRIGGER_BRICK_MK3_setPortOutputActions(STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_ACTION_MASK actions)	409
7.26.3.8	TRIGGER_BRICK_MK3_setTimeCodeOutputActions(STAR_DEVICE_ID deviceId, U32 timeCode, U32 trigger, TIME_CODE_ACTION_MASK actions)	409
7.26.3.9	TRIGGER_PCIE_IF_getCounterOutputActions(STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_ACTION_MASK *pActions)	410
7.26.3.10	TRIGGER_PCIE_IF_getPortOutputActions(STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_ACTION_MASK *pActions)	410
7.26.3.11	TRIGGER_PCIE_IF_setCounterOutputActions(STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_ACTION_MASK actions)	411
7.26.3.12	TRIGGER_PCIE_IF_setPortOutputActions(STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_ACTION_MASK actions)	411
7.26.3.13	TRIGGER_PXI_IF_getCounterOutputActions(STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_ACTION_MASK *pActions)	412
7.26.3.14	TRIGGER_PXI_IF_getExtTriggerOutputActions(STAR_DEVICE_ID deviceId, U32 extTrigger, U32 trigger, EXT_TRIGGER_ACTION_MASK *pActions)	412
7.26.3.15	TRIGGER_PXI_IF_getPortOutputActions(STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_ACTION_MASK *pActions)	412
7.26.3.16	TRIGGER_PXI_IF_getTimeCodeOutputActions(STAR_DEVICE_ID deviceId, U32 timeCode, U32 trigger, TIME_CODE_ACTION_MASK *pActions)	413
7.26.3.17	TRIGGER_PXI_IF_setCounterOutputActions(STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_ACTION_MASK actions)	413
7.26.3.18	TRIGGER_PXI_IF_setExtTriggerOutputActions(STAR_DEVICE_ID deviceId, U32 extTrigger, U32 trigger, EXT_TRIGGER_ACTION_MASK actions)	414
7.26.3.19	TRIGGER_PXI_IF_setPortOutputActions(STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_ACTION_MASK actions)	414
7.26.3.20	TRIGGER_PXI_IF_setTimeCodeOutputActions(STAR_DEVICE_ID deviceId, U32 timeCode, U32 trigger, TIME_CODE_ACTION_MASK actions)	415
7.26.3.21	TRIGGER_PXI_ROUTER_getCounterOutputActions(STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_ACTION_MASK *pActions)	415
7.26.3.22	TRIGGER_PXI_ROUTER_getPortOutputActions(STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_ACTION_MASK *pActions)	416
7.26.3.23	TRIGGER_PXI_ROUTER_getTimeCodeOutputActions(STAR_DEVICE_ID deviceId, U32 timeCode, U32 trigger, TIME_CODE_ACTION_MASK *pActions)	417
7.26.3.24	TRIGGER_PXI_ROUTER_setCounterOutputActions(STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_ACTION_MASK actions)	417
7.26.3.25	TRIGGER_PXI_ROUTER_setPortOutputActions(STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_ACTION_MASK actions)	418
7.26.3.26	TRIGGER_PXI_ROUTER_setTimeCodeOutputActions(STAR_DEVICE_ID deviceId, U32 timeCode, U32 trigger, TIME_CODE_ACTION_MASK actions)	418
7.27	Configuration	420

7.27.1	Detailed Description	428
7.27.2	Enumeration Type Documentation	428
7.27.2.1	TRIGGER_INPUT_MODE	428
7.27.3	Function Documentation	428
7.27.3.1	TRIGGER_BRICK_MK3_disableCounterAutoReload(STAR_DEVICE_ID device↔ Id, U32 counter)	428
7.27.3.2	TRIGGER_BRICK_MK3_disableCounterLoadZero(STAR_DEVICE_ID deviceId, U32 counter)	429
7.27.3.3	TRIGGER_BRICK_MK3_disableCounterStartMode(STAR_DEVICE_ID device↔ Id, U32 counter)	429
7.27.3.4	TRIGGER_BRICK_MK3_disableCounterStartStopMode(STAR_DEVICE_ID↔ D deviceId, U32 counter)	430
7.27.3.5	TRIGGER_BRICK_MK3_disableCounterTriggerCount(STAR_DEVICE_ID↔ D deviceId, U32 counter)	430
7.27.3.6	TRIGGER_BRICK_MK3_disableExtTriggerEdgeDetectMode(STAR_DEVICE_ID↔ ID deviceId, U32 extTrigger)	430
7.27.3.7	TRIGGER_BRICK_MK3_disableExtTriggerInvert(STAR_DEVICE_ID deviceId, U32 extTrigger)	431
7.27.3.8	TRIGGER_BRICK_MK3_disableExtTriggerOutput(STAR_DEVICE_ID deviceId, U32 extTrigger)	431
7.27.3.9	TRIGGER_BRICK_MK3_disablePortTransmitPacketMode(STAR_DEVICE_ID deviceId, U32 port)	432
7.27.3.10	TRIGGER_BRICK_MK3_disableTrigger(STAR_DEVICE_ID deviceId, U32 trigger)	432
7.27.3.11	TRIGGER_BRICK_MK3_enableCounterAutoReload(STAR_DEVICE_ID device↔ Id, U32 counter)	433
7.27.3.12	TRIGGER_BRICK_MK3_enableCounterLoadZero(STAR_DEVICE_ID deviceId, U32 counter)	433
7.27.3.13	TRIGGER_BRICK_MK3_enableCounterStartMode(STAR_DEVICE_ID deviceId, U32 counter)	434
7.27.3.14	TRIGGER_BRICK_MK3_enableCounterStartStopMode(STAR_DEVICE_ID↔ D deviceId, U32 counter)	435
7.27.3.15	TRIGGER_BRICK_MK3_enableCounterTriggerCount(STAR_DEVICE_ID↔ D deviceId, U32 counter)	435
7.27.3.16	TRIGGER_BRICK_MK3_enableExtTriggerEdgeDetectMode(STAR_DEVICE_ID deviceId, U32 extTrigger)	436
7.27.3.17	TRIGGER_BRICK_MK3_enableExtTriggerInvert(STAR_DEVICE_ID deviceId, U32 extTrigger)	436
7.27.3.18	TRIGGER_BRICK_MK3_enableExtTriggerOutput(STAR_DEVICE_ID deviceId, U32 extTrigger)	437
7.27.3.19	TRIGGER_BRICK_MK3_enablePortTransmitPacketMode(STAR_DEVICE_ID deviceId, U32 port)	437
7.27.3.20	TRIGGER_BRICK_MK3_enableTrigger(STAR_DEVICE_ID deviceId, U32 trigger)	438
7.27.3.21	TRIGGER_BRICK_MK3_forceCounterReload(STAR_DEVICE_ID deviceId, U32 counter)	438
7.27.3.22	TRIGGER_BRICK_MK3_forceTrigger(STAR_DEVICE_ID deviceId, U32 trigger)	439

7.27.3.23 TRIGGER_BRICK_MK3_getCounterAutoReloadEnabled(STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)	439
7.27.3.24 TRIGGER_BRICK_MK3_getCounterLoadZeroEnabled(STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)	439
7.27.3.25 TRIGGER_BRICK_MK3_getCounterReloadValue(STAR_DEVICE_ID deviceId, U32 counter, U32 *pReloadValue)	440
7.27.3.26 TRIGGER_BRICK_MK3_getCounterStartModeEnabled(STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)	440
7.27.3.27 TRIGGER_BRICK_MK3_getCounterStartStopModeEnabled(STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)	441
7.27.3.28 TRIGGER_BRICK_MK3_getCounterTriggerCountEnabled(STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)	441
7.27.3.29 TRIGGER_BRICK_MK3_getCounterValue(STAR_DEVICE_ID deviceId, U32 counter, U32 *pValue)	441
7.27.3.30 TRIGGER_BRICK_MK3_getExtTriggerEdgeDetectModeEnabled(STAR_DEVICE_ID deviceId, U32 extTrigger, U32 *pEnabled)	442
7.27.3.31 TRIGGER_BRICK_MK3_getExtTriggerExtend(STAR_DEVICE_ID deviceId, U32 extTrigger, U32 *pExtend)	442
7.27.3.32 TRIGGER_BRICK_MK3_getExtTriggerInvertEnabled(STAR_DEVICE_ID deviceId, U32 extTrigger, U32 *pEnabled)	443
7.27.3.33 TRIGGER_BRICK_MK3_getExtTriggerOutputEnabled(STAR_DEVICE_ID deviceId, U32 extTrigger, U32 *pEnabled)	443
7.27.3.34 TRIGGER_BRICK_MK3_getPortTransmitPacketModeEnabled(STAR_DEVICE_ID deviceId, U32 port, U32 *pEnabled)	443
7.27.3.35 TRIGGER_BRICK_MK3_getTriggerAndMask(STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_TYPE type, U32 *pMask)	444
7.27.3.36 TRIGGER_BRICK_MK3_getTriggerEnabled(STAR_DEVICE_ID deviceId, U32 trigger, U32 *pEnabled)	444
7.27.3.37 TRIGGER_BRICK_MK3_getTriggerInputMode(STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_INPUT_MODE *pMode)	445
7.27.3.38 TRIGGER_BRICK_MK3_getTriggerInvertMask(STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_TYPE type, U32 *pMask)	445
7.27.3.39 TRIGGER_BRICK_MK3_setCounterReloadValue(STAR_DEVICE_ID deviceId, U32 counter, U32 reloadValue)	446
7.27.3.40 TRIGGER_BRICK_MK3_setExtTriggerExtend(STAR_DEVICE_ID deviceId, U32 extTrigger, U32 extend)	447
7.27.3.41 TRIGGER_BRICK_MK3_setTriggerAndMask(STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_TYPE type, U32 mask)	447
7.27.3.42 TRIGGER_BRICK_MK3_setTriggerInputMode(STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_INPUT_MODE mode)	448
7.27.3.43 TRIGGER_BRICK_MK3_setTriggerInvertMask(STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_TYPE type, U32 mask)	448
7.27.3.44 TRIGGER_PCIE_IF_disableCounterAutoReload(STAR_DEVICE_ID deviceId, U32 counter)	449
7.27.3.45 TRIGGER_PCIE_IF_disableCounterLoadZero(STAR_DEVICE_ID deviceId, U32 counter)	449

7.27.3.46 TRIGGER_PCIE_IF_disableCounterStartMode(STAR_DEVICE_ID deviceId, U32 counter)	450
7.27.3.47 TRIGGER_PCIE_IF_disableCounterStartStopMode(STAR_DEVICE_ID deviceId, U32 counter)	450
7.27.3.48 TRIGGER_PCIE_IF_disableCounterTriggerCount(STAR_DEVICE_ID deviceId, U32 counter)	451
7.27.3.49 TRIGGER_PCIE_IF_disableTrigger(STAR_DEVICE_ID deviceId, U32 trigger)	452
7.27.3.50 TRIGGER_PCIE_IF_enableCounterAutoReload(STAR_DEVICE_ID deviceId, U32 counter)	452
7.27.3.51 TRIGGER_PCIE_IF_enableCounterLoadZero(STAR_DEVICE_ID deviceId, U32 counter)	453
7.27.3.52 TRIGGER_PCIE_IF_enableCounterStartMode(STAR_DEVICE_ID deviceId, U32 counter)	453
7.27.3.53 TRIGGER_PCIE_IF_enableCounterStartStopMode(STAR_DEVICE_ID deviceId, U32 counter)	453
7.27.3.54 TRIGGER_PCIE_IF_enableCounterTriggerCount(STAR_DEVICE_ID deviceId, U32 counter)	454
7.27.3.55 TRIGGER_PCIE_IF_enableTrigger(STAR_DEVICE_ID deviceId, U32 trigger)	454
7.27.3.56 TRIGGER_PCIE_IF_forceCounterReload(STAR_DEVICE_ID deviceId, U32 counter)	455
7.27.3.57 TRIGGER_PCIE_IF_forceTrigger(STAR_DEVICE_ID deviceId, U32 trigger)	455
7.27.3.58 TRIGGER_PCIE_IF_getCounterAutoReloadEnabled(STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)	455
7.27.3.59 TRIGGER_PCIE_IF_getCounterLoadZeroEnabled(STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)	456
7.27.3.60 TRIGGER_PCIE_IF_getCounterReloadValue(STAR_DEVICE_ID deviceId, U32 counter, U32 *pReloadValue)	456
7.27.3.61 TRIGGER_PCIE_IF_getCounterStartModeEnabled(STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)	457
7.27.3.62 TRIGGER_PCIE_IF_getCounterStartStopModeEnabled(STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)	457
7.27.3.63 TRIGGER_PCIE_IF_getCounterTriggerCountEnabled(STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)	457
7.27.3.64 TRIGGER_PCIE_IF_getCounterValue(STAR_DEVICE_ID deviceId, U32 counter, U32 *pValue)	458
7.27.3.65 TRIGGER_PCIE_IF_getTriggerAndMask(STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_TYPE type, U32 *pMask)	458
7.27.3.66 TRIGGER_PCIE_IF_getTriggerEnabled(STAR_DEVICE_ID deviceId, U32 trigger, U32 *pEnabled)	459
7.27.3.67 TRIGGER_PCIE_IF_getTriggerInputMode(STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_INPUT_MODE *pMode)	459
7.27.3.68 TRIGGER_PCIE_IF_getTriggerInvertMask(STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_TYPE type, U32 *pMask)	459
7.27.3.69 TRIGGER_PCIE_IF_setCounterReloadValue(STAR_DEVICE_ID deviceId, U32 counter, U32 reloadValue)	460
7.27.3.70 TRIGGER_PCIE_IF_setTriggerAndMask(STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_TYPE type, U32 mask)	460

7.27.3.71 TRIGGER_PCIE_IF_setTriggerInputMode(STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_INPUT_MODE mode)	461
7.27.3.72 TRIGGER_PCIE_IF_setTriggerInvertMask(STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_TYPE type, U32 mask)	461
7.27.3.73 TRIGGER_PXI_IF_disableCounterAutoReload(STAR_DEVICE_ID deviceId, U32 counter)	462
7.27.3.74 TRIGGER_PXI_IF_disableCounterLoadZero(STAR_DEVICE_ID deviceId, U32 counter)	462
7.27.3.75 TRIGGER_PXI_IF_disableCounterStartMode(STAR_DEVICE_ID deviceId, U32 counter)	463
7.27.3.76 TRIGGER_PXI_IF_disableCounterStartStopMode(STAR_DEVICE_ID deviceId, U32 counter)	463
7.27.3.77 TRIGGER_PXI_IF_disableCounterTriggerCount(STAR_DEVICE_ID deviceId, U32 counter)	463
7.27.3.78 TRIGGER_PXI_IF_disableExtTriggerEdgeDetectMode(STAR_DEVICE_ID deviceId, U32 extTrigger)	464
7.27.3.79 TRIGGER_PXI_IF_disableExtTriggerInvert(STAR_DEVICE_ID deviceId, U32 extTrigger)	464
7.27.3.80 TRIGGER_PXI_IF_disableExtTriggerOutput(STAR_DEVICE_ID deviceId, U32 extTrigger)	465
7.27.3.81 TRIGGER_PXI_IF_disablePortTransmitPacketMode(STAR_DEVICE_ID deviceId, U32 port)	465
7.27.3.82 TRIGGER_PXI_IF_disableTrigger(STAR_DEVICE_ID deviceId, U32 trigger)	465
7.27.3.83 TRIGGER_PXI_IF_enableCounterAutoReload(STAR_DEVICE_ID deviceId, U32 counter)	466
7.27.3.84 TRIGGER_PXI_IF_enableCounterLoadZero(STAR_DEVICE_ID deviceId, U32 counter)	466
7.27.3.85 TRIGGER_PXI_IF_enableCounterStartMode(STAR_DEVICE_ID deviceId, U32 counter)	467
7.27.3.86 TRIGGER_PXI_IF_enableCounterStartStopMode(STAR_DEVICE_ID deviceId, U32 counter)	467
7.27.3.87 TRIGGER_PXI_IF_enableCounterTriggerCount(STAR_DEVICE_ID deviceId, U32 counter)	467
7.27.3.88 TRIGGER_PXI_IF_enableExtTriggerEdgeDetectMode(STAR_DEVICE_ID deviceId, U32 extTrigger)	468
7.27.3.89 TRIGGER_PXI_IF_enableExtTriggerInvert(STAR_DEVICE_ID deviceId, U32 extTrigger)	468
7.27.3.90 TRIGGER_PXI_IF_enableExtTriggerOutput(STAR_DEVICE_ID deviceId, U32 extTrigger)	469
7.27.3.91 TRIGGER_PXI_IF_enablePortTransmitPacketMode(STAR_DEVICE_ID deviceId, U32 port)	469
7.27.3.92 TRIGGER_PXI_IF_enableTrigger(STAR_DEVICE_ID deviceId, U32 trigger)	470
7.27.3.93 TRIGGER_PXI_IF_forceCounterReload(STAR_DEVICE_ID deviceId, U32 counter)	470
7.27.3.94 TRIGGER_PXI_IF_forceTrigger(STAR_DEVICE_ID deviceId, U32 trigger)	470
7.27.3.95 TRIGGER_PXI_IF_getCounterAutoReloadEnabled(STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)	471

7.27.3.96 TRIGGER_PXI_IF_getCounterLoadZeroEnabled(STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)	471
7.27.3.97 TRIGGER_PXI_IF_getCounterReloadValue(STAR_DEVICE_ID deviceId, U32 counter, U32 *pReloadValue)	472
7.27.3.98 TRIGGER_PXI_IF_getCounterStartModeEnabled(STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)	472
7.27.3.99 TRIGGER_PXI_IF_getCounterStartStopModeEnabled(STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)	472
7.27.3.100 TRIGGER_PXI_IF_getCounterTriggerCountEnabled(STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)	473
7.27.3.101 TRIGGER_PXI_IF_getCounterValue(STAR_DEVICE_ID deviceId, U32 counter, U32 *pValue)	473
7.27.3.102 TRIGGER_PXI_IF_getExtTriggerEdgeDetectModeEnabled(STAR_DEVICE_ID deviceId, U32 extTrigger, U32 *pEnabled)	474
7.27.3.103 TRIGGER_PXI_IF_getExtTriggerExtend(STAR_DEVICE_ID deviceId, U32 extTrigger, U32 *pExtend)	475
7.27.3.104 TRIGGER_PXI_IF_getExtTriggerInvertEnabled(STAR_DEVICE_ID deviceId, U32 extTrigger, U32 *pEnabled)	475
7.27.3.105 TRIGGER_PXI_IF_getExtTriggerOutputEnabled(STAR_DEVICE_ID deviceId, U32 extTrigger, U32 *pEnabled)	476
7.27.3.106 TRIGGER_PXI_IF_getPortTransmitPacketModeEnabled(STAR_DEVICE_ID deviceId, U32 port, U32 *pEnabled)	476
7.27.3.107 TRIGGER_PXI_IF_getTriggerAndMask(STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_TYPE type, U32 *pMask)	476
7.27.3.108 TRIGGER_PXI_IF_getTriggerEnabled(STAR_DEVICE_ID deviceId, U32 trigger, U32 *pEnabled)	477
7.27.3.109 TRIGGER_PXI_IF_getTriggerInputMode(STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_INPUT_MODE *pMode)	477
7.27.3.110 TRIGGER_PXI_IF_getTriggerInvertMask(STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_TYPE type, U32 *pMask)	478
7.27.3.111 TRIGGER_PXI_IF_setCounterReloadValue(STAR_DEVICE_ID deviceId, U32 counter, U32 reloadValue)	478
7.27.3.112 TRIGGER_PXI_IF_setExtTriggerExtend(STAR_DEVICE_ID deviceId, U32 extTrigger, U32 extend)	479
7.27.3.113 TRIGGER_PXI_IF_setTriggerAndMask(STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_TYPE type, U32 mask)	479
7.27.3.114 TRIGGER_PXI_IF_setTriggerInputMode(STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_INPUT_MODE mode)	480
7.27.3.115 TRIGGER_PXI_IF_setTriggerInvertMask(STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_TYPE type, U32 mask)	480
7.27.3.116 TRIGGER_PXI_ROUTER_disableCounterAutoReload(STAR_DEVICE_ID deviceId, U32 counter)	480
7.27.3.117 TRIGGER_PXI_ROUTER_disableCounterLoadZero(STAR_DEVICE_ID deviceId, U32 counter)	481
7.27.3.118 TRIGGER_PXI_ROUTER_disableCounterStartMode(STAR_DEVICE_ID deviceId, U32 counter)	481

7.27.3.119	TRIGGER_PXI_ROUTER_disableCounterStartStopMode(STAR_DEVICE_ID deviceld, U32 counter)	482
7.27.3.120	TRIGGER_PXI_ROUTER_disableCounterTriggerCount(STAR_DEVICE_ID D deviceld, U32 counter)	482
7.27.3.121	TRIGGER_PXI_ROUTER_disablePortTransmitPacketMode(STAR_DEVICE_ID deviceld, U32 port)	482
7.27.3.122	TRIGGER_PXI_ROUTER_disableTrigger(STAR_DEVICE_ID deviceld, U32 trig- ger)	483
7.27.3.123	TRIGGER_PXI_ROUTER_enableCounterAutoReload(STAR_DEVICE_ID D deviceld, U32 counter)	483
7.27.3.124	TRIGGER_PXI_ROUTER_enableCounterLoadZero(STAR_DEVICE_ID device Id, U32 counter)	484
7.27.3.125	TRIGGER_PXI_ROUTER_enableCounterStartMode(STAR_DEVICE_ID device Id, U32 counter)	484
7.27.3.126	TRIGGER_PXI_ROUTER_enableCounterStartStopMode(STAR_DEVICE_ID deviceld, U32 counter)	484
7.27.3.127	TRIGGER_PXI_ROUTER_enableCounterTriggerCount(STAR_DEVICE_ID D deviceld, U32 counter)	485
7.27.3.128	TRIGGER_PXI_ROUTER_enablePortTransmitPacketMode(STAR_DEVICE_ID deviceld, U32 port)	485
7.27.3.129	TRIGGER_PXI_ROUTER_enableTrigger(STAR_DEVICE_ID deviceld, U32 trigger)	486
7.27.3.130	TRIGGER_PXI_ROUTER_forceCounterReload(STAR_DEVICE_ID deviceld, U32 counter)	486
7.27.3.131	TRIGGER_PXI_ROUTER_forceTrigger(STAR_DEVICE_ID deviceld, U32 trigger)	487
7.27.3.132	TRIGGER_PXI_ROUTER_getCounterAutoReloadEnabled(STAR_DEVICE_ID deviceld, U32 counter, U32 *pEnabled)	487
7.27.3.133	TRIGGER_PXI_ROUTER_getCounterLoadZeroEnabled(STAR_DEVICE_ID D deviceld, U32 counter, U32 *pEnabled)	488
7.27.3.134	TRIGGER_PXI_ROUTER_getCounterReloadValue(STAR_DEVICE_ID deviceld, U32 counter, U32 *pReloadValue)	489
7.27.3.135	TRIGGER_PXI_ROUTER_getCounterStartModeEnabled(STAR_DEVICE_ID deviceld, U32 counter, U32 *pEnabled)	489
7.27.3.136	TRIGGER_PXI_ROUTER_getCounterStartStopModeEnabled(STAR_DEVICE_ID _ID deviceld, U32 counter, U32 *pEnabled)	490
7.27.3.137	TRIGGER_PXI_ROUTER_getCounterTriggerCountEnabled(STAR_DEVICE_ID deviceld, U32 counter, U32 *pEnabled)	490
7.27.3.138	TRIGGER_PXI_ROUTER_getCounterValue(STAR_DEVICE_ID deviceld, U32 counter, U32 *pValue)	490
7.27.3.139	TRIGGER_PXI_ROUTER_getPortTransmitPacketModeEnabled(STAR_DEVICE_ID CE_ID deviceld, U32 port, U32 *pEnabled)	491
7.27.3.140	TRIGGER_PXI_ROUTER_getTriggerAndMask(STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_TYPE type, U32 *pMask)	491
7.27.3.141	TRIGGER_PXI_ROUTER_getTriggerEnabled(STAR_DEVICE_ID deviceld, U32 trigger, U32 *pEnabled)	492
7.27.3.142	TRIGGER_PXI_ROUTER_getTriggerInputMode(STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_INPUT_MODE *pMode)	492

7.27.3.143	TRIGGER_PXI_ROUTER_getTriggerInvertMask(STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_TYPE type, U32 *pMask)	492
7.27.3.144	TRIGGER_PXI_ROUTER_setCounterReloadValue(STAR_DEVICE_ID deviceId, U32 counter, U32 reloadValue)	493
7.27.3.145	TRIGGER_PXI_ROUTER_setTriggerAndMask(STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_TYPE type, U32 mask)	493
7.27.3.146	TRIGGER_PXI_ROUTER_setTriggerInputMode(STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_INPUT_MODE mode)	494
7.27.3.147	TRIGGER_PXI_ROUTER_setTriggerInvertMask(STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_TYPE type, U32 mask)	494
7.28	Link Speed	496
7.28.1	Detailed Description	496
7.28.2	Function Documentation	496
7.28.2.1	CFG_ROUTER_MK2S_disablePrecisionTransmitRate(STAR_DEVICE_ID deviceId)	496
7.28.2.2	CFG_ROUTER_MK2S_enablePrecisionTransmitRate(STAR_DEVICE_ID deviceId)	497
7.28.2.3	CFG_ROUTER_MK2S_getPrecisionTransmitRate(STAR_DEVICE_ID deviceId, _Out_ double *pTransmitRate)	497
7.28.2.4	CFG_ROUTER_MK2S_getPrecisionTransmitRateEnabled(STAR_DEVICE_ID deviceId, _Out_ int *pEnabled)	498
7.28.2.5	CFG_ROUTER_MK2S_getPrecisionTransmitRateInUse(STAR_DEVICE_ID deviceId, _Out_ int *pInUse)	498
7.28.2.6	CFG_ROUTER_MK2S_setPrecisionTransmitRate(STAR_DEVICE_ID deviceId, double transmitRate)	499
7.29	Events	500
7.29.1	Detailed Description	500
7.29.2	Function Documentation	501
7.29.2.1	CFG_MK2_disableSpeedChangeEventsOnPort(STAR_DEVICE_ID deviceId, U8 portNum)	501
7.29.2.2	CFG_MK2_disableStateChangeEventsOnPort(STAR_DEVICE_ID deviceId, U8 portNum)	501
7.29.2.3	CFG_MK2_disableTimeCodeEventsOnPort(STAR_DEVICE_ID deviceId, U8 portNum)	502
7.29.2.4	CFG_MK2_enableSpeedChangeEventsOnPort(STAR_DEVICE_ID deviceId, U8 portNum)	502
7.29.2.5	CFG_MK2_enableStateChangeEventsOnPort(STAR_DEVICE_ID deviceId, U8 portNum)	503
7.29.2.6	CFG_MK2_enableTimeCodeEventsOnPort(STAR_DEVICE_ID deviceId, U8 portNum)	503
7.29.2.7	CFG_MK2_getSpeedChangeEventsOnPortEnabled(STAR_DEVICE_ID deviceId, U8 portNum, _Out_ int *pEnabled)	504
7.29.2.8	CFG_MK2_getStateChangeEventsOnPortEnabled(STAR_DEVICE_ID deviceId, U8 portNum, _Out_ int *pEnabled)	504
7.29.2.9	CFG_MK2_getTimeCodeEventsOnPortEnabled(STAR_DEVICE_ID deviceId, U8 portNum, _Out_ int *pEnabled)	505

7.30 Link Speed	506
7.30.1 Detailed Description	506
7.30.2 Data Structure Documentation	506
7.30.2.1 struct STAR_CFG_MK2_BASE_TRANSMIT_CLOCK	506
7.30.3 Function Documentation	507
7.30.3.1 CFG_MK2_getBaseTransmitClock(STAR_DEVICE_ID deviceID, U8 linkNum, ↔ _Out_ STAR_CFG_MK2_BASE_TRANSMIT_CLOCK *pClockRateParams)	507
7.30.3.2 CFG_MK2_getLinkRateDivider(STAR_DEVICE_ID deviceID, U8 linkNum, ↔ Out_ U8 *pDivider)	508
7.30.3.3 CFG_MK2_getTransmitClock(STAR_DEVICE_ID deviceID, U8 linkNum, _Out_↔ STAR_CFG_MK2_BASE_TRANSMIT_CLOCK *pClockRateParams)	508
7.30.3.4 CFG_MK2_setBaseTransmitClock(STAR_DEVICE_ID deviceID, U8 linkNum, STAR_CFG_MK2_BASE_TRANSMIT_CLOCK clockRateParams)	509
7.30.3.5 CFG_MK2_setLinkRateDivider(STAR_DEVICE_ID deviceID, U8 linkNum, U8 di- vider)	510
7.30.3.6 CFG_MK2_setTransmitClock(STAR_DEVICE_ID deviceID, U8 linkNum, STAR↔ _CFG_MK2_BASE_TRANSMIT_CLOCK clockRateParams)	511
7.31 Measured Link Speed	512
7.31.1 Detailed Description	512
7.31.2 Macro Definition Documentation	512
7.31.2.1 STAR_CFG_MK2_LINK_SPEED_UNITS_KBPS	512
7.31.3 Function Documentation	512
7.31.3.1 CFG_MK2_getMeasuredLinkSpeed(STAR_DEVICE_ID deviceID, U8 portNum, _Out_ U16 *pLinkSpeed)	512
7.32 Time-code Master	514
7.32.1 Detailed Description	514
7.32.2 Function Documentation	515
7.32.2.1 CFG_MK2_disableExternalTimeCodeSelection(STAR_DEVICE_ID deviceID)	515
7.32.2.2 CFG_MK2_disableTimeCodeCounterBypassMode(STAR_DEVICE_ID deviceID)	516
7.32.2.3 CFG_MK2_disableTimeCodeMaster(STAR_DEVICE_ID deviceID)	516
7.32.2.4 CFG_MK2_enableExternalTimeCodeSelection(STAR_DEVICE_ID deviceID)	517
7.32.2.5 CFG_MK2_enableTimeCodeCounterBypassMode(STAR_DEVICE_ID deviceID)	517
7.32.2.6 CFG_MK2_enableTimeCodeMaster(STAR_DEVICE_ID deviceID)	517
7.32.2.7 CFG_MK2_getExternalTimeCodeSelectionEnabled(STAR_DEVICE_ID device↔ ID, int *pEnabled)	518
7.32.2.8 CFG_MK2_getTimeCodeCounterBypassModeEnabled(STAR_DEVICE_ID↔ D deviceID, int *pEnabled)	518
7.32.2.9 CFG_MK2_getTimeCodeMasterEnabled(STAR_DEVICE_ID deviceID, int *p↔ Enabled)	519
7.32.2.10 CFG_MK2_getTimeCodePeriod(STAR_DEVICE_ID deviceID, _Out_ U32 *p↔ Period)	519
7.32.2.11 CFG_MK2_setTimeCodePeriod(STAR_DEVICE_ID deviceID, U32 period)	520
7.33 Device Information	521

7.33.1	Detailed Description	521
7.33.2	Data Structure Documentation	521
7.33.2.1	struct STAR_CFG_MK2_HARDWARE_INFO	521
7.33.3	Function Documentation	522
7.33.3.1	CFG_MK2_getHardwareInfo(STAR_DEVICE_ID deviceId, _Out_ STAR_CFG_MK2_HARDWARE_INFO *pInfo)	522
7.33.3.2	CFG_MK2_hardwareInfoToString(STAR_CFG_MK2_HARDWARE_INFO info, _Out_z_cap_c_(STAR_CFG_MK2_VERSION_STR_MAX_LEN) char *pVersion, _Out_z_cap_c_(STAR_CFG_MK2_BUILD_DATE_STR_MAX_LEN) char *pBuildDate)	523
7.33.3.3	CFG_MK2_identify(STAR_DEVICE_ID deviceId)	523
7.34	Interface Mode	524
7.34.1	Detailed Description	525
7.34.2	Function Documentation	525
7.34.2.1	CFG_MK2_disableIdentifySource(STAR_DEVICE_ID deviceId)	525
7.34.2.2	CFG_MK2_disableIdentifySourceOnPort(STAR_DEVICE_ID deviceId, U8 portNum)	525
7.34.2.3	CFG_MK2_disableInterfaceMode(STAR_DEVICE_ID deviceId)	525
7.34.2.4	CFG_MK2_disableInterfaceModeOnPort(STAR_DEVICE_ID deviceId, U8 portNum)	526
7.34.2.5	CFG_MK2_enableIdentifySource(STAR_DEVICE_ID deviceId)	526
7.34.2.6	CFG_MK2_enableIdentifySourceOnPort(STAR_DEVICE_ID deviceId, U8 portNum)	527
7.34.2.7	CFG_MK2_enableInterfaceMode(STAR_DEVICE_ID deviceId)	527
7.34.2.8	CFG_MK2_enableInterfaceModeOnPort(STAR_DEVICE_ID deviceId, U8 portNum)	527
7.34.2.9	CFG_MK2_getIdentifySourceEnabled(STAR_DEVICE_ID deviceId, int *pEnabled)	528
7.34.2.10	CFG_MK2_getIdentifySourceOnPortEnabled(STAR_DEVICE_ID deviceId, U8 portNum, int *pEnabled)	528
7.34.2.11	CFG_MK2_getInterfaceModeEnabled(STAR_DEVICE_ID deviceId, int *pEnabled)	529
7.34.2.12	CFG_MK2_getInterfaceModeOnPortEnabled(STAR_DEVICE_ID deviceId, U8 portNum, int *pEnabled)	529
7.34.2.13	CFG_MK2_getPortRoutingAddress(STAR_DEVICE_ID deviceId, U8 portNum, _Out_ U8 *pAddress)	530
7.34.2.14	CFG_MK2_setPortRoutingAddress(STAR_DEVICE_ID deviceId, U8 portNum, U8 address)	530
7.35	Error Injection	532
7.35.1	Detailed Description	532
7.35.2	Enumeration Type Documentation	532
7.35.2.1	SPW_ERROR	532
7.35.3	Function Documentation	533
7.35.3.1	CFG_MK2_injectError(STAR_DEVICE_ID deviceId, unsigned char port, SPW_ERROR error)	533
7.36	Periodic Actions	534

7.36.1	Detailed Description	534
7.36.2	Function Documentation	534
7.36.2.1	CFG_MK2_getDeviceClockRate(STAR_DEVICE_ID deviceId)	534
7.36.2.2	CFG_MK2_startPeriodicAction(STAR_DEVICE_ID deviceId, unsigned char port, U32 count, SPW_ACTION action)	535
7.36.2.3	CFG_MK2_stopPeriodicAction(STAR_DEVICE_ID deviceId, unsigned char port)	535
7.37	PCI Mk2 Packet Subsystem	536
7.37.1	Detailed Description	537
7.37.2	Data Structure Documentation	538
7.37.2.1	struct PKT_SUBSYS_STATISTICS	538
7.37.3	Typedef Documentation	539
7.37.3.1	PKT_SUBSYS_StatisticsFunc	539
7.37.3.2	STATISTICS_LOOP_ID	539
7.37.4	Enumeration Type Documentation	539
7.37.4.1	PKT_SUBSYS_PATTERN	539
7.37.5	Function Documentation	539
7.37.5.1	PKT_SUBSYS_cancelWaitsForPacketCheckingError(STAR_DEVICE_ID deviceId, unsigned int subsystem)	540
7.37.5.2	PKT_SUBSYS_fillBufferWithPattern(PKT_SUBSYS_PATTERN pattern, U16 bufferSize, _Out_bytecap_(bufferSize) void *pBuffer)	541
7.37.5.3	PKT_SUBSYS_getPacketCheckingMismatchCount(STAR_DEVICE_ID deviceId, unsigned int subsystem, U64 *errorCount)	541
7.37.5.4	PKT_SUBSYS_getSinkDataPointers(STAR_DEVICE_ID deviceId, unsigned int subsystem, U16 *startPointer, U16 *endPointer)	542
7.37.5.5	PKT_SUBSYS_getStatistics(STAR_DEVICE_ID deviceId, unsigned int subsystem, PKT_SUBSYS_STATISTICS *pStatistics)	542
7.37.5.6	PKT_SUBSYS_readMemory(STAR_DEVICE_ID deviceId, U16 address, U16 readSize, _Inout_bytecap_(readSize) void *pBuffer)	543
7.37.5.7	PKT_SUBSYS_setGeneratorBlockingParameters(STAR_DEVICE_ID deviceId, unsigned int subsystem, U16 blockFrequency, U16 blockDuration)	543
7.37.5.8	PKT_SUBSYS_setPacketCheckingFormat(STAR_DEVICE_ID deviceId, unsigned int subsystem, U16 headerPointer, U16 headerLength, U16 bodyPointer, U16 bodyLength, U32 packetLength, STAR_EOP_TYPE eop)	544
7.37.5.9	PKT_SUBSYS_setPacketGeneratorFormat(STAR_DEVICE_ID deviceId, unsigned int subsystem, U16 headerPointer, U16 headerLength, U16 bodyPointer, U16 bodyLength, U32 packetLength, STAR_EOP_TYPE eop, U16 gap)	545
7.37.5.10	PKT_SUBSYS_setPacketSinkParameters(STAR_DEVICE_ID deviceId, unsigned int subsystem, U16 memoryPointer, U16 memoryLength)	546
7.37.5.11	PKT_SUBSYS_setSinkBlockingParameters(STAR_DEVICE_ID deviceId, unsigned int subsystem, U16 blockFrequency, U16 blockDuration)	547
7.37.5.12	PKT_SUBSYS_startGetStatisticsLoop(STAR_DEVICE_ID deviceId, unsigned int subsystem, PKT_SUBSYS_StatisticsFunc statisticsHandler)	547
7.37.5.13	PKT_SUBSYS_startPacketChecking(STAR_DEVICE_ID deviceId, unsigned int subsystem, int disableSinkOnError, U16 disableSinkdelay)	548

7.37.5.14	PKT_SUBSYS_startPacketGeneration(STAR_DEVICE_ID deviceId, unsigned int subsystem, U16 sequenceLength)	548
7.37.5.15	PKT_SUBSYS_startSink(STAR_DEVICE_ID deviceId, unsigned int subsystem)	549
7.37.5.16	PKT_SUBSYS_stopGetStatisticsLoop(STATISTICS_LOOP_ID loopId)	549
7.37.5.17	PKT_SUBSYS_stopPacketChecking(STAR_DEVICE_ID deviceId, unsigned int subsystem)	550
7.37.5.18	PKT_SUBSYS_stopPacketGeneration(STAR_DEVICE_ID deviceId, unsigned int subsystem)	550
7.37.5.19	PKT_SUBSYS_stopSink(STAR_DEVICE_ID deviceId, unsigned int subsystem)	550
7.37.5.20	PKT_SUBSYS_waitForPacketCheckingError(STAR_DEVICE_ID deviceId, unsigned int subsystem)	551
7.37.5.21	PKT_SUBSYS_waitForPacketGenerationSequenceComplete(STAR_DEVICE_ID deviceId, unsigned int subsystem, unsigned int timeout)	551
7.37.5.22	PKT_SUBSYS_writeMemory(STAR_DEVICE_ID deviceId, U16 address, U16 writeSize, _In_bytecount_(writeSize) void *pBuffer)	552
7.38	Generic Configuration API	553
7.38.1	Detailed Description	554
7.39	Other Device Configuration APIs	555
7.39.1	Detailed Description	556
7.39.2	Registers	556
7.40	Remote Devices	557
7.40.1	Detailed Description	557
7.40.2	Function Documentation	558
7.40.2.1	STAR_CFG_createRemoteDeviceIdentifier(STAR_DEVICE_ID deviceId, unsigned char localChannel, char *name, STAR_SPACEWIRE_ADDRESS *pathTo, STAR_SPACEWIRE_ADDRESS *returnPath)	558
7.40.2.2	STAR_CFG_destroyRemoteDeviceDescriptionList(_Post_ptr_invalid_ STAR_DEVICE_ID *pRemoteDeviceDescriptionList)	558
7.40.2.3	STAR_CFG_destroyRemoteDeviceIdentifier(STAR_DEVICE_ID remoteDeviceId)	558
7.40.2.4	STAR_CFG_getRemoteDeviceDescriptionList(_Out_ U32 *count)	559
7.40.2.5	STAR_CFG_getRemoteDeviceDescriptionNameString(STAR_DEVICE_ID deviceId)	559
7.40.2.6	STAR_CFG_getRemoteDeviceLocalChannel(STAR_DEVICE_ID remoteDeviceId, unsigned char *localChannel)	560
7.40.2.7	STAR_CFG_getRemoteDeviceLocalDevice(STAR_DEVICE_ID remoteDeviceId, STAR_DEVICE_ID *localDeviceId)	561
7.40.2.8	STAR_CFG_getRemoteDevicePath(STAR_DEVICE_ID remoteDeviceId, _Out_opt_ STAR_SPACEWIRE_ADDRESS **pPath)	561
7.40.2.9	STAR_CFG_getRemoteDeviceRetPath(STAR_DEVICE_ID remoteDeviceId, _Out_opt_ STAR_SPACEWIRE_ADDRESS **pRetPath)	561
7.41	Router Configuration API	563
7.41.1	Detailed Description	563
7.42	Mk2 Compatible Interface and Router Device Configuration API	564
7.42.1	Detailed Description	564

7.43	PCI Mk2 Configuration API	565
7.43.1	Detailed Description	565
7.44	Brick Mk2 Configuration API	566
7.44.1	Detailed Description	566
7.45	Brick Mk3 Configuration API	567
7.45.1	Detailed Description	567
7.46	Router Mk2S Configuration API	568
7.46.1	Detailed Description	568
7.47	PXI Configuration API	569
7.47.1	Detailed Description	569
7.48	Device Information	570
7.48.1	Detailed Description	571
7.48.2	Data Structure Documentation	571
7.48.2.1	struct STAR_CFG_DEVICE_IDENTIFIER_INFO	571
7.48.2.2	struct STAR_CFG_NETWORK_DISCOVERY_INFO	571
7.48.3	Macro Definition Documentation	572
7.48.3.1	STAR_CFG_DEVICE_STR_MAX_LEN	572
7.48.3.2	STAR_CFG_MANUFACTURER_STR_MAX_LEN	572
7.48.4	Enumeration Type Documentation	573
7.48.4.1	STAR_CFG_DEVICE_TYPE	573
7.48.5	Function Documentation	573
7.48.5.1	CFG_ROUTER_getDeviceIdentificationInfo(STAR_DEVICE_ID deviceID, _Out_ STAR_CFG_DEVICE_IDENTIFIER_INFO *pInfo)	573
7.48.5.2	CFG_ROUTER_getDeviceManufacturerAsString(STAR_CFG_DEVICE_IDENTIFIER_INFO *pDeviceIdentifierInfo, _Out_z_cap_c_(STAR_CFG_MANUFACTURER_STR_MAX_LEN) char *manufacturerStr)	573
7.48.5.3	CFG_ROUTER_getDeviceTypeAsString(STAR_CFG_DEVICE_IDENTIFIER_INFO *pDeviceIdentifierInfo, _Out_z_cap_c_(STAR_CFG_DEVICE_STR_MAX_LEN) char *deviceTypeStr)	574
7.48.5.4	CFG_ROUTER_getNetworkDiscoveryInfo(STAR_DEVICE_ID deviceID, _Out_ STAR_CFG_NETWORK_DISCOVERY_INFO *pInfo)	574
7.49	Port Status and Control	576
7.49.1	Detailed Description	577
7.49.2	Data Structure Documentation	577
7.49.2.1	struct STAR_CFG_CONFIG_PORT_ERRORS	577
7.49.2.2	struct STAR_CFG_SPW_LINK_ERRORS	579
7.49.2.3	struct STAR_CFG_SPW_LINK_STATUS	580
7.49.2.4	struct STAR_CFG_EXTERNAL_PORT_ERRORS	581
7.49.2.5	struct STAR_CFG_EXTERNAL_PORT_STATUS	581
7.49.3	Typedef Documentation	582
7.49.3.1	PORT_STATUS_CONTROL	582
7.49.4	Enumeration Type Documentation	582

7.49.4.1	STAR_CFG_PORT_TYPE	582
7.49.4.2	STAR_CFG_SPW_LINK_STATE	583
7.49.5	Function Documentation	583
7.49.5.1	CFG_ROUTER_clearPortErrors(STAR_DEVICE_ID deviceID, U8 portNum) . . .	583
7.49.5.2	CFG_ROUTER_getConfigPortErrors(PORT_STATUS_CONTROL portStatus← Control, _Out_ STAR_CFG_CONFIG_PORT_ERRORS *errors)	584
7.49.5.3	CFG_ROUTER_getExternalPortErrors(PORT_STATUS_CONTROL port← StatusControl, _Out_ STAR_CFG_EXTERNAL_PORT_ERRORS *errors) . . .	584
7.49.5.4	CFG_ROUTER_getExternalPortStatus(PORT_STATUS_CONTROL port← StatusControl, _Out_ STAR_CFG_EXTERNAL_PORT_STATUS *status)	585
7.49.5.5	CFG_ROUTER_getPortConnection(PORT_STATUS_CONTROL portStatus← Control)	586
7.49.5.6	CFG_ROUTER_getPortStatusControl(STAR_DEVICE_ID deviceID, U8 portNum, _Out_ PORT_STATUS_CONTROL *portStatusControl)	586
7.49.5.7	CFG_ROUTER_getPortType(PORT_STATUS_CONTROL portStatusControl) . .	587
7.49.5.8	CFG_ROUTER_getSpaceWireLinkErrors(PORT_STATUS_CONTROL link← StatusControl, _Out_ STAR_CFG_SPW_LINK_ERRORS *errors)	587
7.49.5.9	CFG_ROUTER_getSpaceWireLinkStatus(PORT_STATUS_CONTROL link← StatusControl, _Out_ STAR_CFG_SPW_LINK_STATUS *status)	588
7.49.5.10	CFG_ROUTER_setSpaceWireLinkStatus(STAR_DEVICE_ID deviceID, U8 link← Num, STAR_CFG_SPW_LINK_STATUS linkStatus)	588
7.49.5.11	CFG_ROUTER_startLink(STAR_DEVICE_ID deviceID, U8 linkNum)	589
7.49.5.12	CFG_ROUTER_stopLink(STAR_DEVICE_ID deviceID, U8 linkNum)	589
7.50	Routing Table	590
7.50.1	Detailed Description	590
7.50.2	Data Structure Documentation	590
7.50.2.1	struct STAR_CFG_GAR_ENTRY	590
7.50.3	Function Documentation	591
7.50.3.1	CFG_ROUTER_getRoutingTableEntry(STAR_DEVICE_ID deviceID, U8 logical← Address, _Out_ STAR_CFG_GAR_ENTRY *entry)	591
7.50.3.2	CFG_ROUTER_setRoutingTableEntry(STAR_DEVICE_ID deviceID, U8 logical← Address, STAR_CFG_GAR_ENTRY entry)	592
7.51	User Configurable Registers	593
7.51.1	Detailed Description	593
7.51.2	Function Documentation	593
7.51.2.1	CFG_MK2_getTimeCodeDistributionPorts(STAR_DEVICE_ID deviceID, _Out_ ← U32 *pPortMask)	593
7.51.2.2	CFG_MK2_setTimeCodeDistributionPorts(STAR_DEVICE_ID deviceID, U32 portMask)	594
7.51.2.3	CFG_ROUTER_getGeneralPurpose(STAR_DEVICE_ID deviceID, _Out_ REG← ISTER *value)	594
7.51.2.4	CFG_ROUTER_getNetworkIdentity(STAR_DEVICE_ID deviceID, _Out_ REGI← STER *value)	595

7.51.2.5	CFG_ROUTER_getTimeCodeDistributionPorts(STAR_DEVICE_ID deviceId, _Out_ U32 *portMask)	595
7.51.2.6	CFG_ROUTER_setGeneralPurpose(STAR_DEVICE_ID deviceId, REGISTER value)	596
7.51.2.7	CFG_ROUTER_setNetworkIdentity(STAR_DEVICE_ID deviceId, REGISTER value)	596
7.51.2.8	CFG_ROUTER_setTimeCodeDistributionPorts(STAR_DEVICE_ID deviceId, U32 portMask)	597
7.52	Router Control and Status Configuration	599
7.52.1	Detailed Description	600
7.52.2	Data Structure Documentation	600
7.52.2.1	struct STAR_CFG_ROUTER_GLOBAL_STATE	600
7.52.3	Enumeration Type Documentation	601
7.52.3.1	STAR_CFG_PORT_TIMEOUT	601
7.52.3.2	STAR_CFG_TIMEOUT_MODE	601
7.52.4	Function Documentation	602
7.52.4.1	CFG_MK2_getTimeCodeFlagMode(STAR_DEVICE_ID deviceId, _Out_ U8 *pMode)	602
7.52.4.2	CFG_MK2_setTimeCodeFlagMode(STAR_DEVICE_ID deviceId, U8 mode)	602
7.52.4.3	CFG_ROUTER_getRouterGlobalSettings(STAR_DEVICE_ID deviceId, _Out_ STAR_CFG_ROUTER_GLOBAL_STATE *state)	603
7.52.4.4	CFG_ROUTER_getTimeCodeFlagMode(STAR_DEVICE_ID deviceId, _Out_ U8 *mode)	604
7.52.4.5	CFG_ROUTER_getTimeCodeValue(STAR_DEVICE_ID deviceId, _Out_ U8 *value)	604
7.52.4.6	CFG_ROUTER_setRouterGlobalSettings(STAR_DEVICE_ID deviceId, STAR_CFG_ROUTER_GLOBAL_STATE state)	605
7.52.4.7	CFG_ROUTER_setTimeCodeFlagMode(STAR_DEVICE_ID deviceId, U8 mode)	605
7.53	User	606
7.53.1	Detailed Description	606
7.53.2	Function Documentation	606
7.53.2.1	CFG_getGeneralPurpose(STAR_DEVICE_ID deviceId, _Out_ REGISTER *pValue)	606
7.53.2.2	CFG_getNetworkIdentity(STAR_DEVICE_ID deviceId, _Out_ REGISTER *pValue)	607
7.53.2.3	CFG_getTimeCodeDistributionPorts(STAR_DEVICE_ID deviceId, _Out_ U32 *pPortMask)	607
7.53.2.4	CFG_setGeneralPurpose(STAR_DEVICE_ID deviceId, REGISTER value)	608
7.53.2.5	CFG_setNetworkIdentity(STAR_DEVICE_ID deviceId, REGISTER value)	608
7.53.2.6	CFG_setTimeCodeDistributionPorts(STAR_DEVICE_ID deviceId, U32 portMask)	609
7.53.2.7	CFG_updateDeviceInfo(STAR_DEVICE_ID deviceId)	609
7.54	Hardware	610
7.54.1	Detailed Description	610
7.54.2	Data Structure Documentation	610

7.54.2.1	struct STAR_CFG_FPGA_INFO	610
7.54.3	Function Documentation	611
7.54.3.1	CFG_FPGAInfoToString(STAR_DEVICE_ID deviceId, _In_ STAR_CFG_FPGA_I↔ A_INFO *pFPGAInfo, _Out_z_cap_c_(STAR_CFG_MK2_VERSION_STR_M↔ AX_LEN) char *pVersion, _Out_z_cap_c_(STAR_CFG_MK2_BUILD_DATE_↔ STR_MAX_LEN) char *pBuildDate)	611
7.54.3.2	CFG_getFPGAInfo(STAR_DEVICE_ID deviceId, _Out_ STAR_CFG_FPGA_I↔ NFO *pFPGAInfo)	612
7.54.3.3	CFG_identify(STAR_DEVICE_ID deviceId)	612
7.55	Port Status and Control	614
7.55.1	Detailed Description	615
7.55.2	Function Documentation	615
7.55.2.1	CFG_clearPortErrors(STAR_DEVICE_ID deviceId, U8 portNum)	615
7.55.2.2	CFG_getConfigPortErrors(PORT_STATUS_CONTROL portStatusControl, _↔ Out_ STAR_CFG_CONFIG_PORT_ERRORS *pErrors)	615
7.55.2.3	CFG_getExternalPortErrors(PORT_STATUS_CONTROL portStatusControl, _↔ Out_ STAR_CFG_EXTERNAL_PORT_ERRORS *pErrors)	616
7.55.2.4	CFG_getExternalPortStatus(PORT_STATUS_CONTROL portStatusControl, _↔ Out_ STAR_CFG_EXTERNAL_PORT_STATUS *pStatus)	616
7.55.2.5	CFG_getPortConnection(PORT_STATUS_CONTROL portStatusControl)	617
7.55.2.6	CFG_getPortStatusControl(STAR_DEVICE_ID deviceId, U8 portNum, _Out_↔ PORT_STATUS_CONTROL *pPortStatusControl)	618
7.55.2.7	CFG_getPortType(PORT_STATUS_CONTROL portStatusControl)	618
7.55.2.8	CFG_getSpaceWireLinkErrors(PORT_STATUS_CONTROL portStatusControl, _Out_ STAR_CFG_SPW_LINK_ERRORS *pErrors)	619
7.55.2.9	CFG_getSpaceWireLinkStatus(PORT_STATUS_CONTROL portStatusControl, _Out_ STAR_CFG_SPW_LINK_STATUS *pStatus)	619
7.55.2.10	CFG_setSpaceWireLinkStatus(STAR_DEVICE_ID deviceId, U8 linkNum, _In_↔ STAR_CFG_SPW_LINK_STATUS *pLinkStatus)	620
7.55.2.11	CFG_startLink(STAR_DEVICE_ID deviceId, U8 linkNum)	620
7.55.2.12	CFG_stopLink(STAR_DEVICE_ID deviceId, U8 linkNum)	621
7.56	Device Identifier	622
7.56.1	Detailed Description	622
7.56.2	Function Documentation	622
7.56.2.1	CFG_getDeviceIdentificationInfo(STAR_DEVICE_ID deviceId, _Out_ STAR_C↔ FG_DEVICE_IDENTIFIER_INFO *pInfo)	622
7.56.2.2	CFG_getDeviceManufacturerAsString(_In_ STAR_CFG_DEVICE_IDENTIFIE↔ R_INFO *pDeviceIdentifierInfo, _Out_z_cap_c_(STAR_CFG_MANUFACTUR↔ ER_STR_MAX_LEN) char *pManufacturerStr)	623
7.56.2.3	CFG_getDeviceTypeAsString(_In_ STAR_CFG_DEVICE_IDENTIFIER_INFO *pDeviceIdentifierInfo, _Out_z_cap_c_(STAR_CFG_DEVICE_STR_MAX_LEN) char *pDeviceTypeStr)	623
7.56.2.4	CFG_getNetworkDiscoveryInfo(STAR_DEVICE_ID deviceId, _Out_ STAR_CF↔ G_NETWORK_DISCOVERY_INFO *pInfo)	624
7.57	Group Adaptive Routing	625

7.57.1	Detailed Description	625
7.57.2	Function Documentation	625
7.57.2.1	CFG_getRoutingTableEntry(STAR_DEVICE_ID deviceId, U8 logicalAddress, _↵ Out_ STAR_CFG_GAR_ENTRY *pEntry)	625
7.57.2.2	CFG_setRoutingTableEntry(STAR_DEVICE_ID deviceId, U8 logicalAddress, _↵ In_ STAR_CFG_GAR_ENTRY *pEntry)	626
7.58	Interface Mode	628
7.58.1	Detailed Description	629
7.58.2	Function Documentation	629
7.58.2.1	CFG_disableIdentifySource(STAR_DEVICE_ID deviceId)	629
7.58.2.2	CFG_disableIdentifySourceOnPort(STAR_DEVICE_ID deviceId, U8 portNum)	629
7.58.2.3	CFG_disableInterfaceMode(STAR_DEVICE_ID deviceId)	629
7.58.2.4	CFG_disableInterfaceModeOnPort(STAR_DEVICE_ID deviceId, U8 portNum)	630
7.58.2.5	CFG_enableIdentifySource(STAR_DEVICE_ID deviceId)	630
7.58.2.6	CFG_enableIdentifySourceOnPort(STAR_DEVICE_ID deviceId, U8 portNum)	631
7.58.2.7	CFG_enableInterfaceMode(STAR_DEVICE_ID deviceId)	631
7.58.2.8	CFG_enableInterfaceModeOnPort(STAR_DEVICE_ID deviceId, U8 portNum)	632
7.58.2.9	CFG_getIdentifySourceEnabled(STAR_DEVICE_ID deviceId, _Out_ int *p↵ Enabled)	632
7.58.2.10	CFG_getIdentifySourceOnPortEnabled(STAR_DEVICE_ID deviceId, U8 port↵ Num, _Out_ int *pEnabled)	632
7.58.2.11	CFG_getInterfaceModeEnabled(STAR_DEVICE_ID deviceId, _Out_ int *pEnabled)	633
7.58.2.12	CFG_getInterfaceModeOnPortEnabled(STAR_DEVICE_ID deviceId, U8 port↵ Num, _Out_ int *pEnabled)	633
7.58.2.13	CFG_getPortRoutingAddress(STAR_DEVICE_ID deviceId, U8 portNum, _Out↵ _ U8 *pAddress)	634
7.58.2.14	CFG_setPortRoutingAddress(STAR_DEVICE_ID deviceId, U8 portNum, U8 ad↵ dress)	634
7.59	Time-codes	636
7.59.1	Detailed Description	636
7.59.2	Function Documentation	637
7.59.2.1	CFG_disableExternalTimeCodeSelection(STAR_DEVICE_ID deviceId)	637
7.59.2.2	CFG_disableTimeCodeCounterBypassMode(STAR_DEVICE_ID deviceId)	638
7.59.2.3	CFG_disableTimeCodeMaster(STAR_DEVICE_ID deviceId)	638
7.59.2.4	CFG_enableExternalTimeCodeSelection(STAR_DEVICE_ID deviceId)	639
7.59.2.5	CFG_enableTimeCodeCounterBypassMode(STAR_DEVICE_ID deviceId)	639
7.59.2.6	CFG_enableTimeCodeMaster(STAR_DEVICE_ID deviceId)	639
7.59.2.7	CFG_getExternalTimeCodeSelectionEnabled(STAR_DEVICE_ID deviceId, _↵ Out_ int *pEnabled)	640
7.59.2.8	CFG_getTimeCodeCounterBypassModeEnabled(STAR_DEVICE_ID deviceId, ↵ _Out_ int *pEnabled)	640
7.59.2.9	CFG_getTimeCodeMasterEnabled(STAR_DEVICE_ID deviceId, _Out_ int *p↵ Enabled)	641

7.59.2.10	CFG_getTimeCodePeriod(STAR_DEVICE_ID deviceID, _Out_ U32 *pPeriod)	641
7.59.2.11	CFG_setTimeCodePeriod(STAR_DEVICE_ID deviceID, U32 period)	642
7.60	Events	643
7.60.1	Detailed Description	643
7.60.2	Function Documentation	643
7.60.2.1	CFG_disableSpeedChangeEventsOnPort(STAR_DEVICE_ID deviceID, U8 portNum)	643
7.60.2.2	CFG_disableStateChangeEventsOnPort(STAR_DEVICE_ID deviceID, U8 portNum)	644
7.60.2.3	CFG_disableTimeCodeEventsOnPort(STAR_DEVICE_ID deviceID, U8 portNum)	644
7.60.2.4	CFG_enableSpeedChangeEventsOnPort(STAR_DEVICE_ID deviceID, U8 portNum)	645
7.60.2.5	CFG_enableStateChangeEventsOnPort(STAR_DEVICE_ID deviceID, U8 portNum)	645
7.60.2.6	CFG_enableTimeCodeEventsOnPort(STAR_DEVICE_ID deviceID, U8 portNum)	646
7.60.2.7	CFG_getSpeedChangeEventsOnPortEnabled(STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int *pEnabled)	646
7.60.2.8	CFG_getStateChangeEventsOnPortEnabled(STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int *pEnabled)	647
7.60.2.9	CFG_getTimeCodeEventsOnPortEnabled(STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int *pEnabled)	647
7.61	Link Speed	649
7.61.1	Detailed Description	649
7.61.2	Function Documentation	649
7.61.2.1	CFG_getMeasuredLinkSpeed(STAR_DEVICE_ID deviceID, U8 portNum, _Out_ U16 *pLinkSpeed)	649
7.62	Links	650
7.62.1	Detailed Description	650
7.62.2	Function Documentation	650
7.62.2.1	CFG_getTransmitSignallingRate(STAR_DEVICE_ID deviceID, U8 linkNum, _Out_ U32 *pSignallingRate)	650
7.62.2.2	CFG_setTransmitSignallingRate(STAR_DEVICE_ID deviceID, U8 linkNum, U32 signallingRate)	651
7.63	Precision Links	652
7.63.1	Detailed Description	652
7.63.2	Function Documentation	652
7.63.2.1	CFG_disablePrecisionTransmitRate(STAR_DEVICE_ID deviceID)	652
7.63.2.2	CFG_enablePrecisionTransmitRate(STAR_DEVICE_ID deviceID)	653
7.63.2.3	CFG_getPrecisionTransmitRate(STAR_DEVICE_ID deviceID, _Out_ double *pTransmitRate)	653
7.63.2.4	CFG_getPrecisionTransmitRateEnabled(STAR_DEVICE_ID deviceID, _Out_ int *pEnabled)	654
7.63.2.5	CFG_getPrecisionTransmitRateInUse(STAR_DEVICE_ID deviceID, _Out_ int *pInUse)	654

7.63.2.6	CFG_setPrecisionTransmitRate(STAR_DEVICE_ID deviceID, double transmitRate)	655
7.64	Error Injection	656
7.64.1	Detailed Description	656
7.64.2	Function Documentation	656
7.64.2.1	CFG_injectError(STAR_DEVICE_ID deviceID, U8 portNum, SPW_ERROR error)	656
7.64.2.2	CFG_injectErrors(STAR_DEVICE_ID deviceID, U8 portNum, _In_ STAR_CFG↔ _BRICK_MK2_ERRORS *pErrors)	657
7.65	Timestamping	658
7.65.1	Detailed Description	658
7.65.2	Function Documentation	658
7.65.2.1	CFG_disableRxTimestampEventsOnPort(STAR_DEVICE_ID deviceID, U8 portNum)	658
7.65.2.2	CFG_enableRxTimestampEventsOnPort(STAR_DEVICE_ID deviceID, U8 port↔ Num)	659
7.65.2.3	CFG_getPulseGeneratorFrequency(STAR_DEVICE_ID deviceID, _Out_ STAR↔ _CFG_BRICK_MK3_PULSE_FREQ *pFrequency)	659
7.65.2.4	CFG_getRxTimestampEventsOnPortEnabled(STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int *pEnabled)	660
7.65.2.5	CFG_getTimestampMethod(STAR_DEVICE_ID deviceID, _Out_ STAR_CFG↔ BRICK_MK3_TIMESTAMP_METHOD *pMethod)	660
7.65.2.6	CFG_getTimestampValue(STAR_DEVICE_ID deviceID, _Out_ U32 *pValue)	661
7.65.2.7	CFG_setPulseGeneratorFrequency(STAR_DEVICE_ID deviceID, STAR_CFG↔ _BRICK_MK3_PULSE_FREQ frequency)	661
7.65.2.8	CFG_setTimestampMethod(STAR_DEVICE_ID deviceID, STAR_CFG_BRICK↔ _MK3_TIMESTAMP_METHOD method)	662
7.65.2.9	CFG_setTimestampValue(STAR_DEVICE_ID deviceID, U32 value)	662
7.66	Periodic	664
7.66.1	Detailed Description	664
7.66.2	Function Documentation	664
7.66.2.1	CFG_getDeviceClockRate(STAR_DEVICE_ID deviceID)	664
7.66.2.2	CFG_startPeriodicAction(STAR_DEVICE_ID deviceID, unsigned char port, U32 count, SPW_ACTION action)	665
7.66.2.3	CFG_stopPeriodicAction(STAR_DEVICE_ID deviceID, unsigned char port)	665
7.67	Defines	666
7.67.1	Detailed Description	668
7.67.2	Macro Definition Documentation	668
7.67.2.1	CFG_STATUS_NOT_COMPATIBLE_WITH_THIS_DEVICES_FPGA_VERSION ON	668
7.67.2.2	CFG_STATUS_SUCCESS	668
7.68	RMAP Packet Library	669
7.68.1	Detailed Description	671
7.68.2	Functionality Provided	671
7.68.3	Supported Platforms	671

7.68.4	Using The Library	671
7.68.5	Java Library	672
7.68.6	Byte Alignment	672
7.68.7	Contact Information	672
7.68.8	Data Structure Documentation	672
7.68.8.1	struct RMAP_PACKET	672
7.68.9	Macro Definition Documentation	674
7.68.9.1	RMAP_COMMAND_BIT	674
7.68.9.2	RMAP_GET_VERSION_EDIT	674
7.68.9.3	RMAP_GET_VERSION_MAJOR	675
7.68.9.4	RMAP_GET_VERSION_MINOR	675
7.68.9.5	RMAP_GET_VERSION_PATCH	675
7.68.9.6	RMAP_INCREMENT_ADDRESS_BIT	676
7.68.9.7	RMAP_PROTOCOL_IDENTIFIER	676
7.68.9.8	RMAP_REPLY_ADDRESS_LENGTH_BITS	676
7.68.9.9	RMAP_REPLY_BIT	676
7.68.9.10	RMAP_RESERVED_BIT	676
7.68.9.11	RMAP_VERIFY_BEFORE_WRITE_BIT	677
7.68.9.12	RMAP_WRITE_OPERATION_BIT	677
7.68.9.13	RMAPPACKETLIBRARY_CC	677
7.68.10	Enumeration Type Documentation	677
7.68.10.1	RMAP_PACKET_TYPE	677
7.68.10.2	RMAP_STATUS	677
7.68.11	Function Documentation	678
7.68.11.1	RMAP_CalculateCRC(void *pBuffer, unsigned long len)	678
7.68.11.2	RMAP_CalculateCRCWithSeed(void *pBuffer, unsigned long len, U8 crc)	678
7.68.11.3	RMAP_FreeBuffer(void *pBuffer)	679
7.68.11.4	RMAP_GetVersion(void)	679
7.68.11.5	RMAP_IsCRCValid(void *pBuffer, unsigned long len, U8 crc)	679
7.69	Functions for Building RMAP Packets	681
7.69.1	Detailed Description	683
7.69.2	Function Documentation	683
7.69.2.1	RMAP_BuildReadCommandPacket(U8 *pTargetAddress, unsigned long target↵ AddressLength, U8 *pReplyAddress, unsigned long replyAddressLength, char incrementAddress, U8 key, U16 transactionIdentifier, U32 readAddress, U8 extendedReadAddress, U32 dataLength, unsigned long *pRawPacketLength, RMAP_PACKET *pPacketStruct, char alignment)	683
7.69.2.2	RMAP_BuildReadModifyWriteCommandPacket(U8 *pTargetAddress, unsigned long targetAddressLength, U8 *pReplyAddress, unsigned long replyAddress↵ Length, U8 key, U16 transactionIdentifier, U32 readModifyWriteAddress, U8 extendedReadModifyWriteAddress, U8 dataAndMaskLength, U8 *pData, U8 *pMask, unsigned long *pRawPacketLength, RMAP_PACKET *pPacketStruct, char alignment)	684

7.69.2.3	RMAP_BuildReadModifyWriteRegisterPacket(U8 *pTargetAddress, unsigned long targetAddressLength, U8 *pReplyAddress, unsigned long replyAddressLength, U8 key, U16 transactionIdentifier, U32 readModifyWriteAddress, U8 extendedReadModifyWriteAddress, U32 registerValue, U32 mask, unsigned long *pRawPacketLength, RMAP_PACKET *pPacketStruct, char alignment)	685
7.69.2.4	RMAP_BuildReadModifyWriteReplyPacket(U8 *pInitiatorAddress, unsigned long initiatorAddressLength, U8 targetAddress, RMAP_STATUS status, U16 transactionIdentifier, unsigned long dataLength, U8 *pData, unsigned long *pRawPacketLength, RMAP_PACKET *pPacketStruct, char alignment)	686
7.69.2.5	RMAP_BuildReadRegisterPacket(U8 *pTargetAddress, unsigned long targetAddressLength, U8 *pReplyAddress, unsigned long replyAddressLength, char incrementAddress, U8 key, U16 transactionIdentifier, U32 readAddress, U8 extendedReadAddress, unsigned long *pRawPacketLength, RMAP_PACKET *pPacketStruct, char alignment)	687
7.69.2.6	RMAP_BuildReadReplyPacket(U8 *pInitiatorAddress, unsigned long initiatorAddressLength, U8 targetAddress, char incrementAddress, RMAP_STATUS status, U16 transactionIdentifier, U8 *pData, U32 dataLength, unsigned long *pRawPacketLength, RMAP_PACKET *pPacketStruct, char alignment)	688
7.69.2.7	RMAP_BuildWriteCommandPacket(U8 *pTargetAddress, unsigned long targetAddressLength, U8 *pReplyAddress, unsigned long replyAddressLength, char verifyBeforeWrite, char acknowledge, char incrementAddress, U8 key, U16 transactionIdentifier, U32 writeAddress, U8 extendedWriteAddress, U8 *pData, U32 dataLength, unsigned long *pRawPacketLength, RMAP_PACKET *pPacketStruct, char alignment)	689
7.69.2.8	RMAP_BuildWriteRegisterPacket(U8 *pTargetAddress, unsigned long targetAddressLength, U8 *pReplyAddress, unsigned long replyAddressLength, char verifyBeforeWrite, char acknowledge, char incrementAddress, U8 key, U16 transactionIdentifier, U32 writeAddress, U8 extendedWriteAddress, U32 registerValue, unsigned long *pRawPacketLength, RMAP_PACKET *pPacketStruct, char alignment)	690
7.69.2.9	RMAP_BuildWriteReplyPacket(U8 *pInitiatorAddress, unsigned long initiatorAddressLength, U8 targetAddress, char verifyBeforeWrite, char incrementAddress, RMAP_STATUS status, U16 transactionIdentifier, unsigned long *pRawPacketLength, RMAP_PACKET *pPacketStruct, char alignment)	691
7.69.2.10	RMAP_CalculateReadCommandPacketLength(unsigned long targetAddressLength, unsigned long replyAddressLength, char alignment)	691
7.69.2.11	RMAP_CalculateReadModifyWriteCommandPacketLength(unsigned long targetAddressLength, unsigned long replyAddressLength, U32 dataAndMaskLength, char alignment)	692
7.69.2.12	RMAP_CalculateReadModifyWriteReplyPacketLength(unsigned long initiatorAddressLength, U32 dataLength, char alignment)	692
7.69.2.13	RMAP_CalculateReadReplyPacketLength(unsigned long initiatorAddressLength, U32 dataLength, char alignment)	693
7.69.2.14	RMAP_CalculateWriteCommandPacketLength(unsigned long targetAddressLength, unsigned long replyAddressLength, U32 dataLength, char alignment)	693
7.69.2.15	RMAP_CalculateWriteReplyPacketLength(unsigned long initiatorAddressLength, char alignment)	694

7.69.2.16	RMAP_FillReadCommandPacket(U8 *pTargetAddress, unsigned long targetAddressLength, U8 *pReplyAddress, unsigned long replyAddressLength, char incrementAddress, U8 key, U16 transactionIdentifier, U32 readAddress, U8 extendedReadAddress, U32 dataLength, unsigned long *pRawPacketLength, RMAP_PACKET *pPacketStruct, char alignment, U8 *pRawPacket, unsigned long rawPacketLength)	694
7.69.2.17	RMAP_FillReadModifyWriteCommandPacket(U8 *pTargetAddress, unsigned long targetAddressLength, U8 *pReplyAddress, unsigned long replyAddressLength, U8 key, U16 transactionIdentifier, U32 readModifyWriteAddress, U8 extendedReadModifyWriteAddress, U8 dataAndMaskLength, U8 *pData, U8 *pMask, unsigned long *pRawPacketLength, RMAP_PACKET *pPacketStruct, char alignment, U8 *pRawPacket, unsigned long rawPacketLength)	695
7.69.2.18	RMAP_FillReadModifyWriteReplyPacket(U8 *pInitiatorAddress, unsigned long initiatorAddressLength, U8 targetAddress, RMAP_STATUS status, U16 transactionIdentifier, unsigned long dataLength, U8 *pData, unsigned long *pRawPacketLength, RMAP_PACKET *pPacketStruct, char alignment, U8 *pRawPacket, unsigned long rawPacketLength)	696
7.69.2.19	RMAP_FillReadReplyPacket(U8 *pInitiatorAddress, unsigned long initiatorAddressLength, U8 targetAddress, char incrementAddress, RMAP_STATUS status, U16 transactionIdentifier, U8 *pData, U32 dataLength, unsigned long *pRawPacketLength, RMAP_PACKET *pPacketStruct, char alignment, U8 *pRawPacket, unsigned long rawPacketLength)	697
7.69.2.20	RMAP_FillWriteCommandPacket(U8 *pTargetAddress, unsigned long targetAddressLength, U8 *pReplyAddress, unsigned long replyAddressLength, char verifyBeforeWrite, char acknowledge, char incrementAddress, U8 key, U16 transactionIdentifier, U32 writeAddress, U8 extendedWriteAddress, U8 *pData, U32 dataLength, unsigned long *pRawPacketLength, RMAP_PACKET *pPacketStruct, char alignment, U8 *pRawPacket, unsigned long rawPacketLength)	698
7.69.2.21	RMAP_FillWriteReplyPacket(U8 *pInitiatorAddress, unsigned long initiatorAddressLength, U8 targetAddress, char verifyBeforeWrite, char incrementAddress, RMAP_STATUS status, U16 transactionIdentifier, unsigned long *pRawPacketLength, RMAP_PACKET *pPacketStruct, char alignment, U8 *pRawPacket, unsigned long rawPacketLength)	699
7.69.2.22	RMAP_SetProtocolIdentifier(RMAP_PACKET *pPacketStruct, U8 protocolIdentifier)	700
7.70	Functions for Interpreting RMAP Packets	701
7.70.1	Detailed Description	702
7.70.2	Function Documentation	702
7.70.2.1	RMAP_CheckPacketValid(void *pRawPacket, unsigned long packetLength, RMAP_PACKET *pPacketStruct, char checkPacketTooLong)	702
7.70.2.2	RMAP_CheckPacketValidIgnoreProtocol(void *pRawPacket, unsigned long packetLength, RMAP_PACKET *pPacketStruct, char checkPacketTooLong)	703
7.70.2.3	RMAP_GetAddress(RMAP_PACKET *pPacketStruct, U8 *pExtendedAddress)	703
7.70.2.4	RMAP_GetData(RMAP_PACKET *pPacketStruct, U32 *pDataLength)	704
7.70.2.5	RMAP_GetDataCRC(RMAP_PACKET *pPacketStruct)	704
7.70.2.6	RMAP_GetHeaderCRC(RMAP_PACKET *pPacketStruct)	705
7.70.2.7	RMAP_GetIncrementAddress(RMAP_PACKET *pPacketStruct)	705
7.70.2.8	RMAP_GetKey(RMAP_PACKET *pPacketStruct)	705
7.70.2.9	RMAP_GetMask(RMAP_PACKET *pPacketStruct, U8 *pMaskLength)	706

7.70.2.10	RMAP_GetPacketType(RMAP_PACKET *pPacketStruct)	706
7.70.2.11	RMAP_GetPerformAcknowledgement(RMAP_PACKET *pPacketStruct)	707
7.70.2.12	RMAP_GetProtocolIdentifier(RMAP_PACKET *pPacketStruct)	707
7.70.2.13	RMAP_GetReadLength(RMAP_PACKET *pPacketStruct)	708
7.70.2.14	RMAP_GetReplyAddress(RMAP_PACKET *pPacketStruct, unsigned long *pReplyAddressLength)	708
7.70.2.15	RMAP_GetStatus(RMAP_PACKET *pPacketStruct)	708
7.70.2.16	RMAP_GetTargetAddress(RMAP_PACKET *pPacketStruct, unsigned long *pTargetAddressLength)	709
7.70.2.17	RMAP_GetTransactionID(RMAP_PACKET *pPacketStruct)	709
7.70.2.18	RMAP_GetVerifyBeforeWrite(RMAP_PACKET *pPacketStruct)	710
7.71	STAR-Dundee Types	711
7.71.1	Detailed Description	711
7.71.2	Macro Definition Documentation	711
7.71.2.1	FALSE	711
7.71.2.2	TRUE	711
7.71.2.3	U16_MAX	712
7.71.3	Typedef Documentation	712
7.71.3.1	REGISTER	712
7.71.3.2	U16	712
7.71.3.3	U32	712
7.71.3.4	U8	712
8	File Documentation	713
8.1	cfg_api_brick_mk2.h File Reference	713
8.1.1	Detailed Description	714
8.2	cfg_api_brick_mk2_types.h File Reference	714
8.2.1	Detailed Description	715
8.2.2	Data Structure Documentation	715
8.2.2.1	struct STAR_CFG_BRICK_MK2_ERRORS	715
8.3	cfg_api_brick_mk3.h File Reference	716
8.3.1	Detailed Description	717
8.4	cfg_api_brick_mk3_types.h File Reference	717
8.4.1	Detailed Description	718
8.5	cfg_api_generic.c File Reference	718
8.5.1	Detailed Description	723
8.5.2	Function Documentation	723
8.5.2.1	CFG_getRouterGlobalSettings(STAR_DEVICE_ID deviceID, _Out_ STAR_CFG_ROUTER_GLOBAL_STATE *pState)	723
8.5.2.2	CFG_getTimeCodeFlagMode(STAR_DEVICE_ID deviceID, _Out_ U8 *pMode)	724
8.5.2.3	CFG_getTimeCodeValue(STAR_DEVICE_ID deviceID, _Out_ U8 *pValue)	724

8.5.2.4	CFG_setRouterGlobalSettings(STAR_DEVICE_ID deviceId, _In_ STAR_CFG↔ _ROUTER_GLOBAL_STATE *pState)	725
8.5.2.5	CFG_setTimeCodeFlagMode(STAR_DEVICE_ID deviceId, U8 mode)	725
8.6	cfg_api_generic.h File Reference	726
8.6.1	Detailed Description	733
8.6.2	Function Documentation	733
8.6.2.1	CFG_getRouterGlobalSettings(STAR_DEVICE_ID deviceId, _Out_ STAR_CFG↔ G_ROUTER_GLOBAL_STATE *pState)	733
8.6.2.2	CFG_getTimeCodeFlagMode(STAR_DEVICE_ID deviceId, _Out_ U8 *pMode)	734
8.6.2.3	CFG_getTimeCodeValue(STAR_DEVICE_ID deviceId, _Out_ U8 *pValue)	734
8.6.2.4	CFG_setRouterGlobalSettings(STAR_DEVICE_ID deviceId, _In_ STAR_CFG↔ _ROUTER_GLOBAL_STATE *pState)	734
8.6.2.5	CFG_setTimeCodeFlagMode(STAR_DEVICE_ID deviceId, U8 mode)	735
8.7	cfg_api_generic_common.h File Reference	735
8.7.1	Detailed Description	736
8.8	cfg_api_mk2.h File Reference	736
8.8.1	Detailed Description	739
8.8.2	Macro Definition Documentation	739
8.8.2.1	CFG_API_MK2_MODULE_NAME	739
8.9	cfg_api_mk2_types.h File Reference	739
8.9.1	Detailed Description	740
8.9.2	Enumeration Type Documentation	740
8.9.2.1	SPW_ACTION	740
8.10	cfg_api_pci_mk2.h File Reference	740
8.10.1	Detailed Description	741
8.10.2	Macro Definition Documentation	741
8.10.2.1	CFG_API_PCI_MK2_MODULE_NAME	741
8.11	cfg_api_pci_mk2_types.h File Reference	741
8.11.1	Detailed Description	741
8.12	cfg_api_pxi.h File Reference	742
8.12.1	Detailed Description	743
8.13	cfg_api_remote.h File Reference	743
8.13.1	Detailed Description	744
8.14	cfg_api_router.h File Reference	744
8.14.1	Detailed Description	746
8.15	cfg_api_router_mk2s.h File Reference	746
8.15.1	Detailed Description	747
8.16	cfg_api_router_types.h File Reference	747
8.16.1	Detailed Description	748
8.17	common.h File Reference	749
8.17.1	Detailed Description	749

8.17.2	Macro Definition Documentation	749
8.17.2.1	STAR_API_CC	749
8.17.2.2	STAR_API_MODULE_NAME	750
8.18	general.h File Reference	750
8.18.1	Detailed Description	753
8.19	packet_subsystem.h File Reference	753
8.19.1	Detailed Description	755
8.19.2	Macro Definition Documentation	755
8.19.2.1	PKT_SUBSYS_MODULE_NAME	755
8.20	rmap_packet_library.h File Reference	755
8.20.1	Detailed Description	760
8.21	rmap_target_pxi_if.h File Reference	760
8.21.1	Detailed Description	762
8.22	rmap_target_types.h File Reference	762
8.22.1	Detailed Description	764
8.23	star-api.h File Reference	764
8.23.1	Detailed Description	764
8.24	star-dundee_types.h File Reference	764
8.24.1	Detailed Description	765
8.25	stream_item.h File Reference	765
8.25.1	Detailed Description	768
8.26	stream_item_types.h File Reference	768
8.26.1	Detailed Description	769
8.27	transfer.h File Reference	770
8.27.1	Detailed Description	771
8.28	transfer_types.h File Reference	772
8.28.1	Detailed Description	772
8.29	triggering_brick_mk3.h File Reference	772
8.29.1	Detailed Description	776
8.30	triggering_pcie.h File Reference	776
8.30.1	Detailed Description	779
8.31	triggering_pxi_if.h File Reference	779
8.31.1	Detailed Description	783
8.32	triggering_pxi_ro.h File Reference	783
8.32.1	Detailed Description	786
8.33	triggering_types.h File Reference	786
8.33.1	Detailed Description	788
8.33.2	Data Structure Documentation	788
8.33.2.1	struct TRIGGER_MATRIX	788
8.33.3	Enumeration Type Documentation	789

8.33.3.1	TRIGGER_DEVICE	789
8.33.3.2	TRIGGER_TYPE	789
8.34	types.h File Reference	789
8.34.1	Detailed Description	792
8.34.2	Macro Definition Documentation	792
8.34.2.1	CFG_INFO_ID_TYPE	792
8.35	version.h File Reference	792
8.35.1	Detailed Description	793
8.35.2	Data Structure Documentation	793
8.35.2.1	struct STAR_VERSION_INFO	793
8.35.3	Macro Definition Documentation	794
8.35.3.1	POPULATE_VERSION	794
8.35.3.2	PRINT_FULL_VERSION_INFO	795
8.35.3.3	PRINT_VERSION	795
8.35.3.4	PRINT_VERSION_EDIT	795
8.35.3.5	PRINT_VERSION_NUM	795
8.35.3.6	PRINT_VERSION_PATCH	796
8.35.3.7	VERSION_EDIT	796
8.35.3.8	VERSION_MAJOR	796
8.35.3.9	VERSION_MINOR	796
8.35.3.10	VERSION_PATCH	796
9	Example Documentation	797
9.1	advanced_address_example.c	797
9.2	advanced_continuous_receive_example.c	799
9.3	advanced_multiple_packet_example.c	800
9.4	advanced_receive_example.c	802
9.5	advanced_send_and_receive_example.c	803
9.6	advanced_send_example.c	805
9.7	advanced_two_way_example.c	806
9.8	all_version_example.c	809
9.9	api_test_app.c	809
9.10	api_version_example.c	810
9.11	application_name_example.c	810
9.12	brickMk2_configuration_example.c	811
9.13	channel_callback_example.c	812
9.14	check_packet_example.c	814
9.15	crc_example.c	817
9.16	device_identifier_example.c	817
9.17	driver_list_example.c	819

9.18	firmware_version_example.c	820
9.19	link_events.c	821
9.20	mk2_configuration_example.c	827
9.21	packet_subsystem_example.c	829
9.22	pciMk2_configuration_example.c	834
9.23	port_control_status_example.c	835
9.24	read_command_packet_example.c	837
9.25	read_reply_packet_example.c	838
9.26	rmap_target_example.c	839
9.27	rmw_command_packet_example.c	844
9.28	rmw_reply_packet_example.c	845
9.29	router_configuration_example.c	846
9.30	routerMk2S_configuration_example.c	849
9.31	routing_table_example.c	850
9.32	simple_receive_example.c	851
9.33	simple_send_and_receive_example.c	852
9.34	simple_send_example.c	853
9.35	simple_two_way_example.c	854
9.36	star-test.c	856
9.37	star_performance_tester.c	890
9.38	time-code_example.c	920
9.39	time-code_test.c	921
9.40	timestamp_test.c	927
9.41	transfer_callback_example.c	932
9.42	transfer_operation_list_send_example.c	934
9.43	transfer_operation_list_send_receive_example.c	935
9.44	transmit_errors_example.c	937
9.45	triggering_example.c	939
9.46	ui.c	954
9.47	user_registers_example.c	957
9.48	utilities.c	958
9.49	utility.c	963
9.50	version_example.c	966
9.51	write_command_packet_example.c	967
9.52	write_reply_packet_example.c	968
Index		971

Chapter 1

Introduction and Overview

Note that the HTML version of this documentation is available after installing STAR-System, and the latest version can be viewed online by registered users at <https://www.star-dundee.com/help/star-system/index.xhtml>. A PDF version of this documentation is also available for download from the STAR-Dundee website, again for registered users. See the [Support and Contact Information](#) section for more information on registering your product.

1.1 Introduction

STAR-System is the STAR-Dundee software suite provided with all new and future STAR-Dundee interface and router devices. It also provides the drivers used by other STAR-System devices. A full list of supported devices is provided in the [STAR-System Devices](#) section. If you are using a device which is not simply an interface or router, then it will include a separate user manual, focussing on its unique capabilities. Some of the features of STAR-System are still available for use with these devices and further information is included in the [Other STAR-System Devices](#) section.

The aims of STAR-System include:

- To provide high bandwidth and low latency packet transmission and reception.
- To provide additional test and debug capabilities than were available in previous STAR-Dundee software packages.
- To simplify migration between devices, operating systems and programming languages.

STAR-System was designed with the intention of not only allowing you to transmit and receive packets at high data rates, but also to provide you with the tools you're likely to need when testing or debugging a new device, for example. These include transmitting packets terminated with an EEP or with no end of packet marker, transmitting time-codes at a particular point in a packet, receiving packets and time-codes in the order in which they were carried across the SpaceWire link, etc.

STAR-System provides a consistent interface across multiple operating systems (see [Supported Operating Systems](#)). It provides a consistent interface for accessing numerous STAR-Dundee device types (see [Supported Devices](#)), with support for each new device added as that device is completed. It also supports multiple programming and scripting languages, with more to be added in the future, in a manner that is as consistent as is possible across these languages (see [Supported Languages](#)).

1.2 Supported Operating Systems

STAR-System is provided for Windows and Linux operating systems. The following versions of Windows are supported:

- Windows XP (32- and 64-bit) (to be removed in v4.02)
- Windows Server 2003 (32- and 64-bit) (to be removed in v4.02)
- Windows Vista (32- and 64-bit)
- Windows Server 2008 (32- and 64-bit)
- Windows 7 (32- and 64-bit)
- Windows 8 and 8.1 (32- and 64-bit)
- Windows Server 2012 (64-bit)
- Windows 10 (32- and 64-bit)

The Windows versions of STAR-System also support compiling and running applications under Cygwin.

Note that as Microsoft have dropped support for Windows XP and Windows Server 2003, STAR-System will drop support for Windows XP and Windows Server 2003 from version 4.02 onwards.

2.6, 3.x and 4.0 - 5.3.9 Linux kernels for i386 and x86-64 processors are supported. An ARM beta release has been developed for Raspberry Pi and BeagleBone devices and will be included in a future release. For further information, or support for other ARM devices, please contact us.

QNX, VxWorks and RTEMS versions of STAR-System are available separately.

If you require support for other operating systems or other Linux kernels or architectures, please contact us using the [contact information](#) below.

1.3 Installation Instructions

1.3.1 Installation Instructions for Windows

To install STAR-System on Windows, double-click the `.msi` file for the Windows architecture you are installing on. For example, to install on a 64-bit version of Windows, double-click `star-system_win_x86-64_v4.01.↵.msi`. To install on a 32-bit version of Windows, double-click `star-system_win_x86-32_v4.01.msi`.

After double-clicking the appropriate `.msi` file, the STAR-System installer should begin. Follow the instructions to install STAR-System, selecting the features that you wish to install.

If you encounter issues installing STAR-System on Windows, please make sure that you have the latest root certificates installed on your machine. These are available from Windows Update. It is recommended that you have the latest service pack and updates installed, particularly if using an older version of Windows (see below).

1.3.1.1 Windows XP USB Issue

There is a bug in Windows XP which can cause PCs to bug check (blue screen) when transmitting and receiving over USB at the high rates possible using the STAR-System USB Driver. This will cause your PC to crash and reboot. A fix is available from Microsoft and it is recommended that this fix is installed (along with all other important updates) when running on Windows XP.

The fix can be obtained from: <http://support.microsoft.com/kb/969238>.

1.3.1.2 Windows Vista USB Issue

There is a bug in Windows Vista prior to Service Pack 2 which can cause USB traffic to be transferred out of sequence when transmitting and receiving over USB at the high rates possible using the STAR-System USB Driver. This can cause SpaceWire packets to be corrupted or received out of order. A fix for this issue was included in Service Pack 2 and it is highly recommended that the latest service pack is installed (along with all other important updates) when running on Windows Vista.

1.3.1.3 Universal CRT

The GUI applications require components from the Universal CRT which is built into Windows 10 and older versions (Vista onwards) that have had updates applied. If you try to run the GUI applications on an older version of Windows that has not been updated then you may see an error stating that "api-ms-win-crt-runtime-l1-1-0.dll" is missing. To resolve this error you must either install the latest Windows updates or the [KB2999226](#) package.

1.3.2 Installation Instructions for Linux

Prior to installing STAR-System on Linux, you should ensure that the gcc compiler, make and the kernel headers are installed on the target machine. These are required to build the drivers for your system.

On some systems, e.g. RedHat, it may also be necessary to install the "linux-kernel-devel" and/or "build-essential" packages in order to build the driver modules. On some versions of CentOS it may be necessary to install "kernel-devel-\$(uname -r)" and/or "kernel-headers-\$(uname -r)".

To use the GUI applications you will also need to install Qt. Further information on installing the necessary Qt packages for Linux is available in the [Graphical Applications](#) section.

To install STAR-System on Linux, first extract the .tgz archive, then double-click the installation wizard for the Linux architecture you are installing on, or run the installation from the command line. For example, to install on a 64-bit x86 Linux kernel, run `star-system_linux_x86-64_v4.01`. To install on a 32-bit x86 kernel, run `star-system_linux_x86-32_v4.01`.

After running the appropriate install script, the STAR-System installer GUI should begin. Follow the instructions to install STAR-System, selecting the features that you wish to install.

STAR-System requires root access during the installation process to install the drivers and associated files. Although on some Linux distributions the installer will ask for the appropriate password, note that on some distributions the installer will instead fail if the Linux distribution does not ask for the appropriate permissions. On these distributions you will need to execute the script as root. This can be done at the command line using `su` or `sudo`, depending on your distribution. For example, on Fedora:

```
$ su -  
$ ./star-system_linux_x86-32_v4.01
```

On Ubuntu, the following command can be used:

```
$ sudo ./star-system_linux_x86-32_v4.01
```

The installer builds drivers specifically for the current kernel. If you upgrade your kernel, or wish to use STAR-System on another kernel, please run the following build script to rebuild the drivers:

```
$ ./build_star-system.sh
```

Note that as with the installer you will need to be root or use `sudo` to run this script.

1.3.2.1 Command Line Installer

We also provide a command line installation script, `star-system_install_v4.01.sh` that includes a variety of options to allow feature selection. All features can be installed with the following command:

```
$ ./star-system_install_v4.01.sh --all
```

The help argument can be used to display the available options:

```
$ ./star-system_install_v4.01.sh --help
```

Note again that as with the installer above, you will need to be root or use `sudo` to run this script.

1.3.3 Installation Instructions for Linux with Secure Boot Enabled

Some Linux kernels require that kernel modules, such as the STAR-System drivers, be cryptographically signed by a trusted key in order to be loaded. This can often be the case for kernels running on UEFI systems with Secure Boot enabled.

If an error message such as:

```
Modprobe: ERROR: could not insert '<driver name>': Required key not
available
```

is encountered during installation, this may indicate that Secure Boot is enabled.

If it is possible to disable Secure Boot via the UEFI or BIOS settings, this should allow the drivers to be loaded.

If this is not possible, another method is to use the `mokutil` tool to disable Secure Boot. You may have to install this tool using your Linux distribution's package manager. To disable Secure Boot, use the command:

```
$ sudo mokutil --disable-validation
```

You will be asked to provide a password, which will be required after rebooting.

On rebooting a prompt will be displayed allowing Secure Boot to be disabled. You will either be asked for the password you entered earlier, or for specific characters from the password.

If you cannot disable Secure Boot, or would prefer to retain it, the following steps describe the procedure required to sign and install the driver modules.

Firstly, create a directory for the signing keys:

```
$ mkdir ~/.ssl
```

```
$ cd ~/.ssl
```

Next, create the required keys:

```
$ openssl req -new -x509 -newkey rsa:2048 -keyout star.priv -outform DER
-out star.der -nodes -days 36500 -subj "/CN=STAR-Dundee/"
```

NOTE : it is important to keep your private key secure, as it could be used to compromise any system which has your public key enrolled.

The next step is to sign the STAR-Dundee USB and PCI drivers :

```
$ sudo /usr/src/kernels/$(uname -r)/scripts/sign-file sha256 star.priv  
star.der $(modinfo -n star_spw_usb)
```

```
$ sudo /usr/src/kernels/$(uname -r)/scripts/sign-file sha256 star.priv  
star.der $(modinfo -n star_spw_pci)
```

Finally, the key must be "enrolled" to allow it to be recognised by the Secure Boot system:

```
$ sudo mokutil --import star.der
```

You will be asked to provide a password, which will be required after rebooting.

Reboot your PC, and select "Enroll MOK" from the displayed menu. The new key should be shown as a numeric entry, and the details can be checked by selecting the "View" option. Once the identity of the key has been confirmed, select "Continue" from the menu, then "Yes" when asked to confirm enrolling the key.

You will then be asked for the password you entered when enrolling the key, either as the full password, or specific characters from it.

The system will reboot again, and the signed drivers should now be available.

Note that key creation and enrolling only needs to be done once, and the same keys can be used for future updates or installations of STAR-System.

1.3.4 Installation Instructions for QNX

To install STAR-System on QNX, first extract the .tgz to the directory you wish to install the software in, then double-click the install script, or run the install script from the command line. For example, run:

```
$ ./install_star-system.sh
```

Reboot the system and STAR-System should now be installed and ready to use.

1.3.5 Installation Instructions for VxWorks

Installation instructions for VxWorks are available in the [VxWorks Installation and Getting Started Guide](#).

1.3.6 Installation Instructions for RTEMS

Installation instructions for RTEMS are available in the [RTEMS Installation and Getting Started Guide](#).

1.4 Getting Started

There are a few things you should be aware of before you start using STAR-System and these are described below. After reading the section below for the operating system you're using, take a look at the [Graphical Applications](#) or some of the example programs such as [STAR-System Test](#) or [STAR Performance Tester](#) to start using the software, or read about the [STAR-System APIs](#) you can use to write your own applications.

1.4.1 Getting Started on Windows

After installation on Windows is complete, STAR-System should be ready to use. Note that the installer installs a process, `star_conf_service.exe` which is automatically executed when the installer completes, and will start when a user logs in to Windows. This process provides the Device Configuration Service for configuring devices, and should not consume any resources other than when STAR-System is in use. You will receive an error if you attempt to use some STAR-System features without this process running. See the [Device Configuration Service](#) section for further information.

1.4.1.1 Getting Started on Cygwin

The example STAR-System applications provided as source code include Makefiles which allow these applications to be built under Cygwin. The header files and libraries also allow you to build your own STAR-System applications under Cygwin. For 32-bit applications you can use the normal STAR-System Windows `.lib` files. For 64-bit applications, some additional Cygwin-specific `.a` files are provided. Instructing your linker to look in the correct directory for the libraries (e.g. `lib/x86-64`) will result in the correct libraries being picked up. No further action is required. The example Makefiles demonstrate this.

1.4.2 Getting Started on Linux

After installation on Linux is complete, STAR-System should be ready to use. Note that the installer installs a process, `star_conf_service` which is automatically executed when the installer completes, and will run whenever your PC is started. This process provides the Device Configuration Service for configuring devices, and should not consume any resources other than when STAR-System is in use. You will receive an error if you attempt to use STAR-System without this process running. The start-up script which runs the Device Configuration Service is named `star_device_config` and is installed in the `/etc/rc.d/init.d/` or `/etc/init.d/` directory. See the [Device Configuration Service](#) section for further information.

1.4.3 Getting Started on QNX

After installation on QNX is complete, and the system rebooted, STAR-System should be ready to use.

On start-up, if a STAR-Dundee device is detected, the driver and the Device Configuration Service are started automatically. The driver and the Device Configuration Service are installed to `/sbin`.

For information, to start the driver manually, run `./star_pci`. To stop the driver, press CTRL + C or kill the process using `/bin/kill` (see the instructions for the Device Configuration Service below).

The Device Configuration Service must be running in order to use the device configuration functions. To start the service manually, run `./star_conf_service`. To stop the service send it the sigint signal:

```
/bin/kill -SIGINT <process_number>
```

To obtain the process number, execute `ps` and look for the `star_conf_service` process. See the [Device Configuration Service](#) section for further information.

1.4.4 Getting Started on VxWorks

Getting started instructions for VxWorks are available in the [VxWorks Installation and Getting Started Guide](#).

1.4.5 Getting Started on RTEMS

Getting started instructions for RTEMS are available in the [RTEMS Installation and Getting Started Guide](#).

1.5 Migrating from Previous STAR-Dundee APIs

STAR-Dundee's older drivers and APIs for our PCI and USB devices were developed and refined over a number of years. As a result, these systems are very reliable and provide very high performance.

However, these systems also have their limitations. The old PCI Driver in particular has been in existence for quite some time, and as a result there are issues on newer operating system releases. For example, it does not support 64-bit versions of Windows and Linux. A further limitation of these older systems is that a different API is used to access the PCI devices than is used to access the USB devices. As a result of this, it's quite awkward to write software which will work with both types of devices. Writing an application using STAR-System's APIs means that not only will it work with the currently supported STAR-Dundee devices, it will also work with future STAR-Dundee devices.

It is possible to migrate code from these older APIs to STAR-System and STAR-Dundee can provide support on this. Note also that the original SpaceWire Brick can be upgraded to the SpaceWire Brick Mk2, the SpaceWire PCI-2 can be upgraded to the SpaceWire PCI Mk2, the SpaceWire cPCI can be upgraded to the SpaceWire cPCI Mk2, and the SpaceWire Router Mk2 can be upgraded to the SpaceWire Router Mk2S, giving STAR-System support. Please contact us using the [contact information](#) below if you would like to upgrade a device.

Below is a list of some of the advantages of STAR-System over previous STAR-Dundee systems:

- A consistent API for new and future devices.
- An API which is consistent across all supported operating systems.
- Support for additional operating systems such as QNX and RTEMS (available separately).
- Support for 64-bit versions of Windows and Linux.
- Support for running 32-bit applications on 64-bit operating systems.
- Support for newer Linux kernels.
- Support for multiple applications accessing the same device at one time.
- APIs developed specifically for test and development, not just transmitting and receiving packets.
- Improved API documentation and examples.
- Improved performance with smaller packets.
- Direct support for multiple programming languages, including C and C++, with Python support planned in the future.
- Separately available LabVIEW wrapper.

If there are any additional features you would like added, please contact us using the [contact information](#) below.

1.6 Supported Devices

The SpaceWire PCI Mk2 and SpaceWire cPCI Mk2 were the first of the new STAR-Dundee devices to be supported by STAR-System. STAR-System was then updated to support the STAR-Dundee SpaceWire PCIe device. Version 2.0 of STAR-System added support for USB devices, specifically the STAR-Dundee SpaceWire Brick Mk2 and the SpaceWire Router Mk2S. Version 3.0 added support for the SpaceWire Brick Mk3 and SpaceWire PXI devices.

Version 3.4 added support for the STAR Fire Mk3 device, while version 3.5 added support for the SpaceWire PXI 12 Port Router device. Version 4.00 added support for the STAR Fire Mk3 and the SpaceWire PXI Mk2 devices.

List of directly supported devices:

- SpaceWire Brick Mk2
- SpaceWire Brick Mk3
- SpaceWire cPCI Mk2
- SpaceWire PCI Mk2
- SpaceWire PCIe
- SpaceWire Physical Layer Tester
- SpaceWire PXI and PXI Mk2 Devices
- SpaceWire Router Mk2S
- STAR Fire and STAR Fire Mk3

STAR-System also provides the drivers for other devices which do not offer standard transmit and receive functions. These include:

- SpaceWire Conformance Tester Mk2
- SpaceWire EGSE and EGSE Mk2
- SpaceWire Link Analyser Mk3
- SpaceWire Recorder and Recorder Mk2

Although STAR-System cannot currently be used with any other devices directly, it is still possible to configure other devices over SpaceWire using STAR-System's Device Configuration APIs and the Device Configuration application. The configuration that is possible includes getting and setting routing tables, starting and stopping links, and getting error information. In addition to the devices supported directly by STAR-System, the Device Configuration APIs and Device Configuration application currently support configuring the following devices over a SpaceWire network:

- SpaceWire-USB Brick
- SpaceWire Router-USB
- SpaceWire Router Mk2
- SpaceWire SpW-10X Router (AT7910E)

Other devices developed using STAR-Dundee's Router IP core (see <https://www.star-dundee.com/products/spacewire-ip-cores>) can also be configured over a SpaceWire network.

If you require support for another device, please contact us using the [contact information](#) below.

1.7 Device User Manuals

The user manuals for the devices supported by STAR-System are available in the following sections:

- [SpaceWire PCI Mk2 and cPCI Mk2](#)

- [SpaceWire PCIe](#)
- [SpaceWire Brick Mk2](#)
- [SpaceWire Router Mk2S](#)
- [SpaceWire Brick Mk3](#)
- [SpaceWire PXI Interface and PXI Interface Mk2](#)
- [SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2](#)

Information about other devices supported by STAR-System is available in the [Other STAR-System Devices](#) section. Instructions to upgrade the firmware of a device are included in the [Device Update Application](#) section.

1.8 Graphical Applications

In addition to the command line example applications, a number of Graphical User Interface (GUI) applications are included, to provide simple, yet powerful, methods to interact with STAR-System and the supported devices. These applications are described in further detail in the [Graphical Applications](#) section.

1.9 CUBA Software

A further useful application provided with STAR-System is the CUBA software. This is an interactive tool designed to assist in the development and testing of SpaceWire systems.

The CUBA Software can be used to transmit and receive SpaceWire packets, either as raw packet data or using the RMAP protocol. Packet data can be entered at the command line, or read from files which allow simple scripting of the actions to be performed.

Received data can be displayed in the console or written to output files.

For more information on using the CUBA Software, refer to the [STAR-System CUBA Software](#) section.

1.10 STAR-System APIs

The STAR-System APIs described below are the C APIs which can be used to access supported STAR-System devices. The C++ API uses a similar structure.

1.10.1 STAR-API

Library:	star-api
Include:	star-api.h

The main Application Programming Interface (API) provided with STAR-System is STAR-API. This provides all the functions for transmitting and receiving traffic, getting the list of connected devices and drivers, registering for notification of events, etc.

All traffic transmitted and received using STAR-API is done so over channels to devices. A channel to a device must be opened before traffic can be transmitted or received. For most devices, when the device is in interface mode, a channel corresponds to a port on the device, e.g. all traffic transmitted over channel 1 will be sent over port 1 of the device. When a device is in router mode, a channel corresponds to an external port on the router. Channels can be opened to transmit traffic and/or to receive traffic. This allows one application to transmit traffic over a port, while another application receives from that port. Once an application has opened a channel to a device, no other application can open that same channel until it has been closed.

1.10.2 Generic Configuration API

Library:	<code>star_conf_api_generic</code>
Include:	<code>cfg_api_generic.h</code> , <code>cfg_api_generic_common.h</code>

The Generic Configuration API provides functions for configuring all STAR-System devices. These devices may be connected to the local PC or may be connected over a SpaceWire network. A number of functions common to all devices are provided.

For example, there are functions for getting and setting the routing tables of all routing devices and for starting and stopping each of the links on a device. Newer STAR-Dundee devices support a feature which causes the device to flash its LEDs when requested so the device can be visually identified. This can be useful when there are multiple devices in a PC, or multiple devices on a network. As this feature is not present in older devices, they will return an error code to indicate the function is not supported on the device.

1.10.3 RMAP Packet Library

Library:	<code>rmap_packet_library</code>
Include:	<code>rmap_packet_library.h</code>

The RMAP Packet Library was provided with STAR-Dundee's older USB and PCI APIs. The Remote Memory Access Protocol (RMAP) is a protocol designed specifically for reading and writing memory over SpaceWire. Its simplicity and flexibility means that it can also be used for many other purposes. For example, the Device Configuration API uses RMAP to read and write to registers of SpaceWire devices in order to configure them.

The RMAP Packet Library provides functions for constructing each type of RMAP packet, checking whether a packet is a valid RMAP packet, and accessing the fields in an RMAP packet. The STAR-System version of the library is named differently to previous versions to avoid conflicts with older versions, and to be consistent with the naming convention used throughout STAR-System.

1.10.4 RMAP Target API

Library:	<code>star_rmap_target</code>
Include:	<code>rmap_target_pxi_if.h</code> , <code>rmap_target_types.h</code>

The RMAP Target API provides functions for configuring and controlling the RMAP targets within PXI Interface with RMAP Target devices. The RMAP targets can operate in automatic or manual authorisation mode. The API provides functions to set the automatic authorisation parameters, allowing ranges of valid values for the RMAP header parameters. When using the manual authorisation mode, the API provides functions to retrieve any pending commands and manually authorise or reject them.

In addition, there are functions to enable notifications when commands require authorisation or when commands are completed. Callback functions can be registered with the API and there are functions to start and stop receiving notifications.

1.10.5 Triggering API

Library:	star_triggering
Include:	triggering_pxi_if.h , triggering_brick_mk3.h , triggering_pcie.h , triggering_types.h

The Triggering API provides functions for configuring and controlling the triggering capabilities of the Brick Mk3, PXI Interface, PXI 12 Port Router and PCIe devices. Triggering allows periodic or sporadic events to cause actions inside the device. There are resources that can generate events such as external triggers, SpaceWire ports, time-code interfaces and counters. These events can be configured to cause internal triggers to be set which can cause actions on the external triggers, SpaceWire ports, time-code interfaces and counters.

1.10.6 PCI Mk2 Packet Subsystem API

Library:	star_packet_subsystem
Include:	packet_subsystem.h

The PCI Mk2 device contains a single hardware packet generator and checker subsystem. This subsystem can be used to automatically generate and check SpaceWire packets at high speed without using the resources of the host PC.

1.11 Supported Languages

Both C and C++ versions of the APIs are provided. Documentation for the C++ API is available in a separate document, and this provides an object-oriented version of the C API. The C API can also be used in C++ code, while both versions of the APIs can be imported in to other programming languages such as Java and the .NET languages, scripting languages such as Python and Tcl, and in to LabVIEW.

A LabVIEW version of the APIs is available separately, while Python versions of the APIs are planned in the future. Support for other languages is also planned.

If there is a particular language or application you would like us to support, please contact us using the [contact information](#) below.

1.12 Example Programs

STAR-System comes with a number of example programs with source code, demonstrating the features of each API. Some of these applications are described below.

1.12.1 STAR-System Test

STAR-System Test provides a number of common operations that you might want to try out when first using your device. These include loopback tests (transmitting packets out of one link and receiving them on another), double

loopback tests (transmitting and receiving on two links at the same time), transmitting packets to another destination, receiving packets from another source, and transmitting and receiving files.

These features have been provided with STAR-Dundee's test programs for previous PCI and USB systems, although STAR-System Test is more powerful than previous incarnations. It allows you to run multiple instances of the software and, for example, transmit from one software instance and receive in another. It also allows you to perform loopback tests between two devices. For example, if you have a SpaceWire PXI Interface device and a SpaceWire Brick Mk3 device, you can connect a SpaceWire cable between them and transmit packets from one to the other.

The source code for STAR-System Test can be used as a basis for your own code, with numerous examples of transmitting and receiving packets provided. The software also provides a Get Version option, so you can find exactly which versions of the various modules of STAR-System you have installed, and which devices are connected to your PC.

1.12.2 STAR Performance Tester

STAR-Dundee strives to get the best performance from our APIs, drivers and devices. For this reason, we use a number of tools to determine the throughput, latency and other performance figures we can achieve. STAR Performance Tester is one of those tools. It provides options to transmit and/or receive packets at the fastest possible rate achievable, using various packet sizes or random packet sizes, and to test the latency when transmitting and receiving. While the tests are being executed, the CPU usage is monitored and recorded, including the total CPU usage and the application's CPU usage. The rates, average packet transmission time (latency) and CPU usage achieved can be written to file, and the results files can then be imported in to Excel to graph data rates, packet rates, etc.

If you're interested to see the throughput that can be achieved using STAR-System with a particular packet size, for example, then STAR Performance Tester can give you that information. The source code provided can be very useful in your own code, as it is designed to achieve the best possible performance from the API and drivers.

Note that you can use the Performance Tester to transmit packets, receive packets, or to do both. If you set the Performance Tester to only transmit, you will need another application to receive those packets, or to configure hardware to drop the packets. It is a similar situation when receiving. You can run two instances of the Performance Tester, either on the same PC or on different PCs, with one instance transmitting and a second instance receiving, for example.

Also note that the latency times displayed are the average of the total time that it takes to transmit and receive a packet. This includes the time that the packet is on the SpaceWire link, so is larger than the actual latency introduced by the software. The data and packets rates shown are the rates at which packets are being transmitted on the link.

The Performance Tester will accept the test parameters as command line arguments. Just pass all the arguments that the application will ask for in order. For string arguments, you can include these in quotes if they contain spaces, or specify two quotes if an empty string is to be provided (e.g. an address path with no bytes).

A `-usechannel0` argument can also be passed to the Performance Tester, to enable the use of channel 0 to transmit and/or receive packets. This argument must be provided prior to any test parameters.

An application note is available from the STAR-Dundee website, describing the performance that can be achieved with a PCI Mk2 using the Performance Tester, and explaining in further detail the meaning of the figures. See <https://www.star-dundee.com/knowledge-base/star-system-performance>.

1.12.3 Device Configuration APIs Examples

The Device Configuration APIs Examples consist of a number of different small applications which demonstrate the use of functions in the Device Configuration APIs. These examples include getting and setting routing table entries, getting and setting the properties of ports, starting and stopping links, and enabling and disabling interface mode of the PCI Mk2.

These examples are used in the documentation for the APIs, and are useful starting points when writing code which uses the Device Configuration APIs.

1.12.4 RMAP Packet Library Example

The RMAP Packet Library Example software doesn't perform a particularly useful task, but its code provides detailed examples of how the RMAP Packet Library can be used to build various types of RMAP packets, check the validity of received RMAP packets, and access the fields of an RMAP packet.

Code from the RMAP Packet Library Example software is included in the documentation for the RMAP Packet Library, to demonstrate the use of the library's functions.

1.13 Known Issues

The following list contains all known issues with this release of STAR-System:

- The `STAR_setPacketData()` function has been removed from this release of STAR-API as setting the data of a packet which had already been submitted to be transmitted over a channel would cause an error.
- As a result of the `STAR_setPacketData()` issue, STAR Performance Tester's random data test does not use a different length for each packet sent, but instead picks some random lengths which it uses repeatedly.
- Resetting the device, e.g. by calling `STAR_resetDevice()`, while a packet is being transmitted and/or received on the device, can cause the device to get in to a bad state.

If you encounter any problems not included in this list, please contact us using the [contact information](#) below.

1.14 Change Log

A full list of the changes introduced in each version is available in the [Change Log](#) section.

1.14.1 Changes between STAR-System v4.00 and v4.01 - 29th November 2019

- Fixed bug in STAR-System Test and Time-code example programs due to incorrect error checking.
- Added Makefile and Visual Studio projects for device configuration examples.
- Linux kernel compatibility tested up to v5.4.
- Added port information to link state and speed change event structs, and also API functions `STAR_getLink↔StateEventPort()` and `STAR_getLinkSpeedEventPort()` to access it.

1.14.2 Changes between STAR-System v3.10 and v4.00 - 19th November 2019

- **New features:**
 - Added [Generic Configuration API](#).
 - Added ability to transmit and receive webcam image data.
 - Added device serial number to default device name so that it is easier to differentiate between multiple devices of the same type.
 - Added an icon to the Error Injection application.

- Added support for Conformance Tester Mk2 and EGSE Mk2 devices.
- Added port action to disable LVDS ([PORT_ACTION_DISABLE_LVDS](#)) to Triggering API.
- Added support for PXI Mk2 cards.
- Added timestamp support for [SpaceWire PXI Interface](#) and [PXI Interface Mk2](#) cards.
- Added support for colour version of SpaceFibre Camera in Sink.
- Added time-code support for PXI cards.
- Custom device names are now stored in the device so that they persist when moved between computers.

• **New APIs and functions:**

- Added function [STAR_resetDeviceName\(\)](#) to allow a custom device name to be reset to a device's default name.
- Added [STAR_getSpaceWireAddressPath\(\)](#) and [STAR_getSpaceWireAddressPathLength\(\)](#) functions.
- Added functions [CFG_MK2_getTimeCodeDistributionPorts\(\)](#) and [CFG_MK2_setTimeCode↔DistributionPorts\(\)](#). They are used to obtain and specify which output ports time-codes are forwarded on. They are used with Brick Mk2, Brick Mk3, Router Mk2S and PXI devices. In future we recommend that the generic configuration functions [CFG_getTimeCodeDistributionPorts\(\)](#) and [CFG_setTime↔CodeDistributionPorts\(\)](#) are used instead.
- Added functions [CFG_MK2_getTimeCodeFlagMode\(\)](#) and [CFG_MK2_setTimeCodeFlagMode\(\)](#). They are used to get and set the time-code flag interpretation mode. They are used with Brick Mk2, Brick Mk3, Router Mk2S and PXI devices. In future we recommend that the generic configuration functions [CFG_getTimeCodeFlagMode\(\)](#) and [CFG_setTimeCodeFlagMode\(\)](#) are used instead.
- Added the following functions to support SpaceFibre broadcast message stream items:
 - * [STAR_getBroadcastMessageChannel\(\)](#)
 - * [STAR_getBroadcastMessageDataWord1\(\)](#)
 - * [STAR_getBroadcastMessageDataWord2\(\)](#)
 - * [STAR_getBroadcastMessageDelayedFlag\(\)](#)
 - * [STAR_getBroadcastMessageLateFlag\(\)](#)
 - * [STAR_getBroadcastMessageStatus\(\)](#)
 - * [STAR_getBroadcastMessageType\(\)](#)
- Added the following functions to support an alternative receive method using pre-allocated buffers:
 - * [STAR_openChannelToLocalDeviceEx\(\)](#)
 - * [STAR_createRxOperationEx\(\)](#)
 - * [STAR_rxOperationExGetNumItems\(\)](#)
 - * [STAR_rxOperationExGetItemType\(\)](#)
 - * [STAR_rxOperationExGetDataChunk\(\)](#)
 - * [STAR_rxOperationExGetBroadcastMessage\(\)](#)
 - * [STAR_rxOperationExGetStatus\(\)](#)

• **Bug fixes:**

- Fixed bug where Receive application could crash if an empty packet was received.
- Fixed bug that allowed time-code events to be enabled on incompatible versions of the [SpaceWire PCIe](#) card.
- Fixed bug where Sink showed packet too short errors when sequence error was encountered.
- Fixed possible crash when receiving link events on wrong channel.
- Fixed bug in Device Configuration Service that could result in the device not being recognised.
- Fixed bug that caused large packets to be split when received by the STAR Fire Mk3.
- Fixed bug that meant that time-codes could not be received in isolation on a channel.
- Fixed issue in [api_test_app](#) time-code example where the wrong channel was being used.
- Fixed bug in Device Configuration application where transmit frequency was not updated for [SpaceWire PXI 12 Port Router](#) and [PXI 12 Port Router Mk2](#).

- Fixed bug where channel remained open if Linux application was terminated abruptly using Ctrl+C.
- Fixed issue with 32-bit/64-bit compatibility on recent Linux kernels.
- Fixed an issue in the Windows PCI driver that caused devices to get into a bad state after being reset.
- Fixed an issue in the Linux PCI and USB drivers that allowed invalid channels to be opened.
- Fixed an issue in the Source application that allowed an empty schedule to be transmitted.
- Resolved issue where base selection dropdowns in the Device Configuration application would disappear on Linux.
- Fixed potential bug during initialisation of Link Analyser Mk3, STAR Fire Mk3, EGSE Mk2 and Conformance Tester Mk2.
- Fixed bug in "Select Packet Format" dialog in Sink application where leaving "Check the content of received packets for errors" unchecked would result in an error.
- Fixed bug in `CFG_BRICK_MK3_setTimestampValue()` where incorrect timestamp value was being set. In future we recommend that the generic configuration function `CFG_setTimestampValue()` is used instead.
- Fixed bug in Time-codes GUI application where incorrect time-code period was being reported.
- Fixed bug where Sink application could crash when recording to file.
- Fixed bug in Device Configuration application where multiplier and divisor "Set" button was not being enabled in all circumstances.
- Fixed bug that could cause the Device Configuration application to get into a bad state when refreshing a remote device.
- Fixed bug in Source and Sink applications where it was possible to create a packet format with same name.
- Fixed bug in time-code handling for [SpaceWire PXI Interface](#) and [PXI Interface Mk2](#) and [SpaceWire PXI 12 Port Router](#) and [PXI 12 Port Router Mk2](#) cards.
- Fixed problem in `STAR_openChannelBetweenLocalDevices()`.
- Fixed bug where Sink application didn't always terminate when closed.
- Fixed bug that prevented Transmit application from transmitting over channel 0.
- Fixed issue that could cause increased CPU usage in the Sink application.
- Fixed bug that caused an error in the Performance Tester when log-like pattern was specified.
- Fixed bug in Time-codes GUI application where master frequency was not loaded correctly for the Brick Mk3.

• **Other improvements:**

- Devices are now listed under "STAR-Dundee Devices" in Device Manager on Windows.
- Brand new Linux installer allowing feature selection including the ability to install Qt5 versions of the GUI applications, required for new webcam features.
- Changes to Device Update application to improve usability.
- Improved component scaling on high DPI and normal DPI displays.
- Improved [SpaceWire PCI Mk2](#) and [cPCI Mk2](#) device configuration example.
- Improvements to look and feel of GUI applications including maximise and minimise buttons.
- Removed EEP statistics from the Source application as this only applies to the Sink.
- Sink application statistics now only show errors that are being checked.
- Documented installation steps for Linux installations with secure boot enabled.
- Documented PXI timestamping features.
- Linux kernel compatibility tested up to v5.3.9.
- Various performance and documentation improvements.
- Removed several functions from header files that were previously deprecated:

- * CFG_BRICK_MK2_getMeasuredLinkSpeed()
replaced by [CFG_getMeasuredLinkSpeed\(\)](#)
- * CFG_PCIMK2_getHardwareInfo()
replaced by [CFG_getFPGAInfo\(\)](#)
- * CFG_PCIMK2_hardwareInfoToString()
replaced by [CFG_FPGAInfoToString\(\)](#)
- * CFG_PCIMK2_enableTimeCodeMaster()
replaced by [CFG_enableTimeCodeMaster\(\)](#)
- * CFG_PCIMK2_disableTimeCodeMaster()
replaced by [CFG_disableTimeCodeMaster\(\)](#)
- * CFG_PCIMK2_setTimeCodePeriod()
replaced by [CFG_setTimeCodePeriod\(\)](#)
- * CFG_PCIMK2_getTimeCodePeriod()
replaced by [CFG_getTimeCodePeriod\(\)](#)
- * CFG_PCIMK2_identify()
replaced by [CFG_identify\(\)](#)
- * CFG_PCIMK2_enableInterfaceMode()
replaced by [CFG_enableInterfaceMode\(\)](#)
- * CFG_PCIMK2_enableInterfaceModeOnPort()
replaced by [CFG_enableInterfaceModeOnPort\(\)](#)
- * CFG_PCIMK2_disableInterfaceMode()
replaced by [CFG_disableInterfaceMode\(\)](#)
- * CFG_PCIMK2_disableInterfaceModeOnPort()
replaced by [CFG_disableInterfaceModeOnPort\(\)](#)
- * CFG_PCIMK2_enableIdentifySource()
replaced by [CFG_enableIdentifySource\(\)](#)
- * CFG_PCIMK2_disableIdentifySource()
replaced by [CFG_disableIdentifySource\(\)](#)
- * CFG_PCIMK2_enableIdentifySourceOnPort()
replaced by [CFG_enableIdentifySourceOnPort\(\)](#)
- * CFG_PCIMK2_disableIdentifySourceOnPort()
replaced by [CFG_disableIdentifySourceOnPort\(\)](#)
- * CFG_PCIMK2_setPortRoutingAddress()
replaced by [CFG_setPortRoutingAddress\(\)](#)
- * CFG_PCIMK2_getPortRoutingAddress()
replaced by [CFG_getPortRoutingAddress\(\)](#)
- * CFG_PCIMK2_setLinkRateDivider()
replaced by [CFG_setTransmitSignallingRate\(\)](#)
- * CFG_PCIMK2_getLinkRateDivider()
replaced by [CFG_getTransmitSignallingRate\(\)](#)
- * STAR_CFG_getRemoteDeviceDescriptionName()
replaced by [STAR_CFG_getRemoteDeviceDescriptionNameString\(\)](#)

1.15 Support and Contact Information

For software updates and access to further technical information, please register your STAR-Dundee products at <http://www.star-dundee.com/product-registration> (or click the Product Registration link on our website: <http://www.star-dundee.com>). After registering your devices, you will be given an account with permissions to download updates for those devices. You can also register to receive e-mail notifications when new updates are available.

If you wish to contact STAR-Dundee with a support enquiry, please e-mail support@star-dundee.com. For general enquiries, please contact enquiries@star-dundee.com. Our full address is as follows:

STAR-Dundee Ltd.
STAR House
166 Nethergate
Dundee
DD1 4EE
Scotland, UK

Chapter 2

STAR-System Applications

STAR-System includes many applications which make it simple to perform common tasks.

These include graphical applications and command-line applications, some of which are provided as example code.

The [Graphical Applications](#) are described on a separate page.

The [CUBA Software](#), a command line application for performing and scripting transmit and receive operations or RMAP commands, is also documented on a separate page.

2.1 Graphical Applications

This section contains information on the Graphical User Interface (GUI) applications, provided with STAR-System.

Each GUI application contains context-sensitive help, explaining the purpose of each component of the application and how it can be used. The help can be accessed by pressing Shift + F1 when a component has focus, or by clicking the ? button in the application's title bar, then clicking on the component of interest. The help text will disappear when you click elsewhere in the application.

Note

These applications make use of Nokia's Qt libraries. The necessary libraries are installed by the STAR-↔ System installer for Windows and QNX.

On Linux Qt may be installed by default, or will be available to download using your Linux distribution's package manager. STAR-System provides both Qt4 and Qt5 versions of the GUI applications. Qt5 brings the ability to transmit and receive webcam data to the Source and Sink applications and improves support for high DPI displays. The packages required for Qt4 are libqtcore4 and libqtgui4. Qt5 may require qt5-default or qt5-qtbase and libqt5multimedia5-plugins or qt5-qtmultimedia depending on the distribution. The STAR-↔ System applications have been successfully tested on Qt versions 4.7, 4.8, and 5.5 and above.

Please see the [licence information](#) section at the end of this section for important information regarding the Qt Libraries.

Each of the GUI applications is described below. A description of the command-line arguments that can be passed to each of the applications is also provided in the [Command-Line Arguments](#) section.

2.1.1 Device Configuration Application

The Device Configuration application provides a window in which the properties of a device can be configured. From this application, links can be started or stopped, the device can be reset, the link speed can be changed, the routing

table can be modified, etc.

The tabbed interface provides a tab for viewing the properties, and configuring those properties, for each of the ports on the device, including the configuration port and each of the external ports. Tabs are also included to view and configure the properties of the device which are local to the PC, such as the name used to refer to the device, and also the general device properties. The properties which are configurable can be changed by altering the value and pressing the corresponding Set button, which will only become enabled when the value has been changed.

Using the Device Configuration application, it is also possible to configure other supported devices on a Space↔Wire network. To do this, check the Remote device check box, enter the path and return path to the device's configuration port (the path to the local device's configuration port are provided as an example), then click the Display Properties button. The properties of the remote device should be shown, although as this device is not local, the Local Properties tab will not be present. Note that to configure a remote device, the Device Configuration application needs to open a channel to transmit and receive the configuration packets. This means this channel cannot be used for data packets while the remote device's properties are being read or set.

Note that PCI/cPCI Mk2 and PCIe devices prior to version 1.07 have a bug which means that reading the value of the port routing registers always returns 0. This means that when viewing the properties of such devices, the port routing address will always be 0, and interface mode on port and identify source on port will always be disabled. The latest firmware version for these devices resolves this issue, and can be downloaded from the STAR-Dundee website. See the [Support and Contact Information](#) section for more information on registering your product.

2.1.2 Transmit Application

The Transmit application is a simple application for transmitting packets.

It allows the bytes of a packet to be entered in to a text box, then transmitted using the specified device and channel by pressing the Transmit Packet button. The base in which the packet is entered can be specified by changing the base in the drop down list. The end of packet marker that is used to terminate the packets transmitted can also be specified.

2.1.3 Receive Application

The Receive application is a simple application for receiving packets.

It allows packets to be received by pressing the Receive Packets button. This will cause all packets received on the specified device and channel to be displayed, along with their end of packet marker type. The base in which the packets are displayed can be altered by changing the base in the drop down list. Packets will continue to be received until the Stop Receiving button is pressed. The packets on display can be cleared by clicking the Clear Packets button.

2.1.4 Source Application

The Source application is a more powerful application for transmitting packets with complex formats at high speed.

It allows a packet format, and a schedule to which those packets should be transmitted, to be specified. Packets can be transmitted repeatedly or a specified number of times, and time delays can be included between packets. The packet format specified is saved for future use, and previously saved packet formats can be edited and reused. The same packet format can also be used in the [Sink Application](#) application to check the format of received packets.

Once the format and schedule have been defined, the packets can be transmitted. While the packets are being transmitted, the transmit statistics can be displayed. These statistics show the total number of characters transmitted and the characters transmitted in the last second. Packet transmission can be stopped at any time by clicking the appropriate button.

2.1.5 Sink Application

The Sink application is a more powerful application for receiving packets at high speed, and checking the contents of packets for errors.

The Sink application allows packets to be received and the content of those packets to be checked for any deviation from the expected format. Received packets, or fields within those packets, can also be written to file. While the packets are being received, the receive statistics can be displayed. These statistics show the total number of characters received and the characters received in the last second. Any errors detected in the received packets will also be displayed in the statistics. Packet reception can be stopped at any time by clicking the appropriate button.

2.1.6 Error Injection Application

The Error Injection application allows errors to be injected onto a SpaceWire link. The types of error that may be injected are described in the documentation for [SPW_ERROR](#).

2.1.7 Device Update Application

The Device Update application allows in-the-field updating of the FPGAs in STAR-Dundee devices. When an update is required or recommended for a device, the appropriate update files will be made available for download from the STAR-Dundee website. See the [Support and Contact Information](#) section for more information on registering your product. These update files will be processed by the Device Update application, and applied to the target devices.

Details on the use of the Device Update Application can be found in the [Device Update Application](#) page.

2.1.8 Time-code Application

The Time-code application allows time-codes to be transmitted and received on STAR-Dundee devices. Other functions such as enabling a device as a time-code master and enabling time-code distribution ports are also provided.

2.1.9 Command-Line Arguments

All STAR-System GUI Applications also support a number of command-line arguments to facilitate device set-up at program start. The following arguments are supported:

- **-0** or **--zero** Includes channel 0 as an option in the Device/Port Selector drop-down box options.
- **-s** [serial #] or **--serial** [serial #] Selects the device with serial number *serial #* as the default selected device from the Device Selector drop-down box.
*Note: If there is no device connected with serial number **serial #**, the argument is ignored.*
- **-c** [ch #] or **--channel** [ch #] Selects *ch #* as the default channel from the Device/Port Selector drop-down box.
*Note: If the selected device does not have channel **ch #**, 1 (or 0) is chosen as the default.*

Note: All the above arguments additionally support the Windows-based "/" delimiter for both long and short versions of the arguments.

2.1.10 Running the GUI Applications on QNX

The STAR-System GUI applications are implemented using Qt for QNX. This uses its own windowing system that is incompatible with QNX's Photon environment.

To use the GUI applications on QNX, QNX must be running in text mode and **io-display** must be running. To enter text mode from Photon, either **kill** the **Photon** process, or click Launch->Logout(End Photon session). In the login dialog, click Shutdown. The shutdown dialog appears. Enable Exit to text mode and click Ok.

Set up the following environment variables:

```
export QWS_KEYBOARD=qnx
export QWS_MOUSE_PROTO=qnx
export QWS_DISPLAY=qnx
```

To enable the mouse and keyboard under Qt, **devi-hid** must be run as follows: `/usr/photon/bin/devi-hid -Pr kbd mouse` This will prevent your current shell from accepting keyboard and mouse input, so be sure to either run this from a script that launches the GUI applications afterwards, or have a remote terminal available to launch applications.

When launching the GUI applications to run under QNX, the first application must be run with the option **-qws** in order to initialise the Qt windowing system and be designated as the GUI server. Only one Qt application should have this option specified.

2.1.11 Qt Open Source Components

The STAR-System GUI applications make use of the Qt4 and Qt5 libraries. These components are licensed under the terms of the GNU Lesser General Public License version 2.1 and version 3 respectively.

The source code for these libraries may be downloaded from our website:

- [Qt 4.8.0](#)
- [Qt 5.9.0](#)

GNU LESSER GENERAL PUBLIC LICENSE
Version 2.1, February 1999

Copyright (C) 1991, 1999 Free Software Foundation, Inc.
51 Franklin Street, Fifth Floor, Boston, MA 02110-1301 USA
Everyone is permitted to copy and distribute verbatim copies

of this license document, but changing it is not allowed.

[This is the first released version of the Lesser GPL. It also counts as the successor of the GNU Library Public License, version 2, hence the version number 2.1.]

Preamble

The licenses for most software are designed to take away your freedom to share and change it. By contrast, the GNU General Public Licenses are intended to guarantee your freedom to share and change free software--to make sure the software is free for all its users.

This license, the Lesser General Public License, applies to some specially designated software packages--typically libraries--of the Free Software Foundation and other authors who decide to use it. You can use it too, but we suggest you first think carefully about whether this license or the ordinary General Public License is the better strategy to use in any particular case, based on the explanations below.

When we speak of free software, we are referring to freedom of use, not price. Our General Public Licenses are designed to make sure that you have the freedom to distribute copies of free software (and charge for this service if you wish); that you receive source code or can get it if you want it; that you can change the software and use pieces of it in new free programs; and that you are informed that you can do these things.

To protect your rights, we need to make restrictions that forbid distributors to deny you these rights or to ask you to surrender these rights. These restrictions translate to certain responsibilities for you if you distribute copies of the library or if you modify it.

For example, if you distribute copies of the library, whether gratis or for a fee, you must give the recipients all the rights that we gave you. You must make sure that they, too, receive or can get the source code. If you link other code with the library, you must provide complete object files to the recipients, so that they can relink them with the library after making changes to the library and recompiling it. And you must show them these terms so they know their rights.

We protect your rights with a two-step method: (1) we copyright the library, and (2) we offer you this license, which gives you legal permission to copy, distribute and/or modify the library.

To protect each distributor, we want to make it very clear that there is no warranty for the free library. Also, if the library is modified by someone else and passed on, the recipients should know that what they have is not the original version, so that the original author's reputation will not be affected by problems that might be introduced by others.

Finally, software patents pose a constant threat to the existence of any free program. We wish to make sure that a company cannot effectively restrict the users of a free program by obtaining a restrictive license from a patent holder. Therefore, we insist that any patent license obtained for a version of the library must be consistent with the full freedom of use specified in this license.

Most GNU software, including some libraries, is covered by the ordinary GNU General Public License. This license, the GNU Lesser General Public License, applies to certain designated libraries, and is quite different from the ordinary General Public License. We use this license for certain libraries in order to permit linking those libraries into non-free programs.

When a program is linked with a library, whether statically or using a shared library, the combination of the two is legally speaking a combined work, a derivative of the original library. The ordinary General Public License therefore permits such linking only if the entire combination fits its criteria of freedom. The Lesser General Public License permits more lax criteria for linking other code with the library.

We call this license the "Lesser" General Public License because it does Less to protect the user's freedom than the ordinary General Public License. It also provides other free software developers Less of an advantage over competing non-free programs. These disadvantages are the reason we use the ordinary General Public License for many libraries. However, the Lesser license provides advantages in certain special circumstances.

For example, on rare occasions, there may be a special need to encourage the widest possible use of a certain library, so that it becomes a de-facto standard. To achieve this, non-free programs must be allowed to use the library. A more frequent case is that a free library does the same job as widely used non-free libraries. In this case, there is little to gain by limiting the free library to free software only, so we use the Lesser General Public License.

In other cases, permission to use a particular library in non-free programs enables a greater number of people to use a large body of free software. For example, permission to use the GNU C Library in non-free programs enables many more people to use the whole GNU operating system, as well as its variant, the GNU/Linux operating system.

Although the Lesser General Public License is Less protective of the users' freedom, it does ensure that the user of a program that is linked with the Library has the freedom and the wherewithal to run that program using a modified version of the Library.

The precise terms and conditions for copying, distribution and modification follow. Pay close attention to the difference between a "work based on the library" and a "work that uses the library". The former contains code derived from the library, whereas the latter must be combined with the library in order to run.

GNU LESSER GENERAL PUBLIC LICENSE TERMS AND CONDITIONS FOR COPYING, DISTRIBUTION AND MODIFICATION

0. This License Agreement applies to any software library or other program which contains a notice placed by the copyright holder or other authorized party saying it may be distributed under the terms of this Lesser General Public License (also called "this License"). Each licensee is addressed as "you".

A "library" means a collection of software functions and/or data prepared so as to be conveniently linked with application programs (which use some of those functions and data) to form executables.

The "Library", below, refers to any such software library or work which has been distributed under these terms. A "work based on the Library" means either the Library or any derivative work under copyright law: that is to say, a work containing the Library or a portion of it, either verbatim or with modifications and/or translated straightforwardly into another language. (Hereinafter, translation is included without limitation in the term "modification".)

"Source code" for a work means the preferred form of the work for making modifications to it. For a library, complete source code means all the source code for all modules it contains, plus any associated interface definition files, plus the scripts used to control compilation and installation of the library.

Activities other than copying, distribution and modification are not covered by this License; they are outside its scope. The act of running a program using the Library is not restricted, and output from such a program is covered only if its contents constitute a work based on the Library (independent of the use of the Library in a tool for writing it). Whether that is true depends on what the Library does and what the program that uses the Library does.

1. You may copy and distribute verbatim copies of the Library's complete source code as you receive it, in any medium, provided that you conspicuously and appropriately publish on each copy an

appropriate copyright notice and disclaimer of warranty; keep intact all the notices that refer to this License and to the absence of any warranty; and distribute a copy of this License along with the Library.

You may charge a fee for the physical act of transferring a copy, and you may at your option offer warranty protection in exchange for a fee.

2. You may modify your copy or copies of the Library or any portion of it, thus forming a work based on the Library, and copy and distribute such modifications or work under the terms of Section 1 above, provided that you also meet all of these conditions:

- a) The modified work must itself be a software library.
- b) You must cause the files modified to carry prominent notices stating that you changed the files and the date of any change.
- c) You must cause the whole of the work to be licensed at no charge to all third parties under the terms of this License.
- d) If a facility in the modified Library refers to a function or a table of data to be supplied by an application program that uses the facility, other than as an argument passed when the facility is invoked, then you must make a good faith effort to ensure that, in the event an application does not supply such function or table, the facility still operates, and performs whatever part of its purpose remains meaningful.

(For example, a function in a library to compute square roots has a purpose that is entirely well-defined independent of the application. Therefore, Subsection 2d requires that any application-supplied function or table used by this function must be optional: if the application does not supply it, the square root function must still compute square roots.)

These requirements apply to the modified work as a whole. If identifiable sections of that work are not derived from the Library, and can be reasonably considered independent and separate works in themselves, then this License, and its terms, do not apply to those sections when you distribute them as separate works. But when you distribute the same sections as part of a whole which is a work based on the Library, the distribution of the whole must be on the terms of this License, whose permissions for other licensees extend to the entire whole, and thus to each and every part regardless of who wrote it.

Thus, it is not the intent of this section to claim rights or contest your rights to work written entirely by you; rather, the intent is to exercise the right to control the distribution of derivative or collective works based on the Library.

In addition, mere aggregation of another work not based on the Library with the Library (or with a work based on the Library) on a volume of a storage or distribution medium does not bring the other work under the scope of this License.

3. You may opt to apply the terms of the ordinary GNU General Public License instead of this License to a given copy of the Library. To do this, you must alter all the notices that refer to this License, so that they refer to the ordinary GNU General Public License, version 2, instead of to this License. (If a newer version than version 2 of the ordinary GNU General Public License has appeared, then you can specify that version instead if you wish.) Do not make any other change in these notices.

Once this change is made in a given copy, it is irreversible for that copy, so the ordinary GNU General Public License applies to all subsequent copies and derivative works made from that copy.

This option is useful when you wish to copy part of the code of the Library into a program that is not a library.

4. You may copy and distribute the Library (or a portion or derivative of it, under Section 2) in object code or executable form under the terms of Sections 1 and 2 above provided that you accompany it with the complete corresponding machine-readable source code, which must be distributed under the terms of Sections 1 and 2 above on a medium customarily used for software interchange.

If distribution of object code is made by offering access to copy from a designated place, then offering equivalent access to copy the source code from the same place satisfies the requirement to distribute the source code, even though third parties are not compelled to copy the source along with the object code.

5. A program that contains no derivative of any portion of the Library, but is designed to work with the Library by being compiled or linked with it, is called a "work that uses the Library". Such a work, in isolation, is not a derivative work of the Library, and therefore falls outside the scope of this License.

However, linking a "work that uses the Library" with the Library creates an executable that is a derivative of the Library (because it contains portions of the Library), rather than a "work that uses the library". The executable is therefore covered by this License. Section 6 states terms for distribution of such executables.

When a "work that uses the Library" uses material from a header file that is part of the Library, the object code for the work may be a derivative work of the Library even though the source code is not. Whether this is true is especially significant if the work can be linked without the Library, or if the work is itself a library. The threshold for this to be true is not precisely defined by law.

If such an object file uses only numerical parameters, data structure layouts and accessors, and small macros and small inline functions (ten lines or less in length), then the use of the object file is unrestricted, regardless of whether it is legally a derivative work. (Executables containing this object code plus portions of the Library will still fall under Section 6.)

Otherwise, if the work is a derivative of the Library, you may distribute the object code for the work under the terms of Section 6. Any executables containing that work also fall under Section 6, whether or not they are linked directly with the Library itself.

6. As an exception to the Sections above, you may also combine or link a "work that uses the Library" with the Library to produce a work containing portions of the Library, and distribute that work under terms of your choice, provided that the terms permit modification of the work for the customer's own use and reverse engineering for debugging such modifications.

You must give prominent notice with each copy of the work that the Library is used in it and that the Library and its use are covered by this License. You must supply a copy of this License. If the work during execution displays copyright notices, you must include the copyright notice for the Library among them, as well as a reference directing the user to the copy of this License. Also, you must do one of these things:

a) Accompany the work with the complete corresponding machine-readable source code for the Library including whatever changes were used in the work (which must be distributed under Sections 1 and 2 above); and, if the work is an executable linked with the Library, with the complete machine-readable "work that uses the Library", as object code and/or source code, so that the user can modify the Library and then relink to produce a modified executable containing the modified Library. (It is understood that the user who changes the contents of definitions files in the Library will not necessarily be able to recompile the application to use the modified definitions.)

b) Use a suitable shared library mechanism for linking with the

Library. A suitable mechanism is one that (1) uses at run time a copy of the library already present on the user's computer system, rather than copying library functions into the executable, and (2) will operate properly with a modified version of the library, if the user installs one, as long as the modified version is interface-compatible with the version that the work was made with.

c) Accompany the work with a written offer, valid for at least three years, to give the same user the materials specified in Subsection 6a, above, for a charge no more than the cost of performing this distribution.

d) If distribution of the work is made by offering access to copy from a designated place, offer equivalent access to copy the above specified materials from the same place.

e) Verify that the user has already received a copy of these materials or that you have already sent this user a copy.

For an executable, the required form of the "work that uses the Library" must include any data and utility programs needed for reproducing the executable from it. However, as a special exception, the materials to be distributed need not include anything that is normally distributed (in either source or binary form) with the major components (compiler, kernel, and so on) of the operating system on which the executable runs, unless that component itself accompanies the executable.

It may happen that this requirement contradicts the license restrictions of other proprietary libraries that do not normally accompany the operating system. Such a contradiction means you cannot use both them and the Library together in an executable that you distribute.

7. You may place library facilities that are a work based on the Library side-by-side in a single library together with other library facilities not covered by this License, and distribute such a combined library, provided that the separate distribution of the work based on the Library and of the other library facilities is otherwise permitted, and provided that you do these two things:

a) Accompany the combined library with a copy of the same work based on the Library, uncombined with any other library facilities. This must be distributed under the terms of the Sections above.

b) Give prominent notice with the combined library of the fact that part of it is a work based on the Library, and explaining where to find the accompanying uncombined form of the same work.

8. You may not copy, modify, sublicense, link with, or distribute the Library except as expressly provided under this License. Any attempt otherwise to copy, modify, sublicense, link with, or distribute the Library is void, and will automatically terminate your rights under this License. However, parties who have received copies, or rights, from you under this License will not have their licenses terminated so long as such parties remain in full compliance.

9. You are not required to accept this License, since you have not signed it. However, nothing else grants you permission to modify or distribute the Library or its derivative works. These actions are prohibited by law if you do not accept this License. Therefore, by modifying or distributing the Library (or any work based on the Library), you indicate your acceptance of this License to do so, and all its terms and conditions for copying, distributing or modifying the Library or works based on it.

10. Each time you redistribute the Library (or any work based on the Library), the recipient automatically receives a license from the original licensor to copy, distribute, link with or modify the Library subject to these terms and conditions. You may not impose any further restrictions on the recipients' exercise of the rights granted herein. You are not responsible for enforcing compliance by third parties with

this License.

11. If, as a consequence of a court judgment or allegation of patent infringement or for any other reason (not limited to patent issues), conditions are imposed on you (whether by court order, agreement or otherwise) that contradict the conditions of this License, they do not excuse you from the conditions of this License. If you cannot distribute so as to satisfy simultaneously your obligations under this License and any other pertinent obligations, then as a consequence you may not distribute the Library at all. For example, if a patent license would not permit royalty-free redistribution of the Library by all those who receive copies directly or indirectly through you, then the only way you could satisfy both it and this License would be to refrain entirely from distribution of the Library.

If any portion of this section is held invalid or unenforceable under any particular circumstance, the balance of the section is intended to apply, and the section as a whole is intended to apply in other circumstances.

It is not the purpose of this section to induce you to infringe any patents or other property right claims or to contest validity of any such claims; this section has the sole purpose of protecting the integrity of the free software distribution system which is implemented by public license practices. Many people have made generous contributions to the wide range of software distributed through that system in reliance on consistent application of that system; it is up to the author/donor to decide if he or she is willing to distribute software through any other system and a licensee cannot impose that choice.

This section is intended to make thoroughly clear what is believed to be a consequence of the rest of this License.

12. If the distribution and/or use of the Library is restricted in certain countries either by patents or by copyrighted interfaces, the original copyright holder who places the Library under this License may add an explicit geographical distribution limitation excluding those countries, so that distribution is permitted only in or among countries not thus excluded. In such case, this License incorporates the limitation as if written in the body of this License.

13. The Free Software Foundation may publish revised and/or new versions of the Lesser General Public License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns.

Each version is given a distinguishing version number. If the Library specifies a version number of this License which applies to it and "any later version", you have the option of following the terms and conditions either of that version or of any later version published by the Free Software Foundation. If the Library does not specify a license version number, you may choose any version ever published by the Free Software Foundation.

14. If you wish to incorporate parts of the Library into other free programs whose distribution conditions are incompatible with these, write to the author to ask for permission. For software which is copyrighted by the Free Software Foundation, write to the Free Software Foundation; we sometimes make exceptions for this. Our decision will be guided by the two goals of preserving the free status of all derivatives of our free software and of promoting the sharing and reuse of software generally.

NO WARRANTY

15. BECAUSE THE LIBRARY IS LICENSED FREE OF CHARGE, THERE IS NO WARRANTY FOR THE LIBRARY, TO THE EXTENT PERMITTED BY APPLICABLE LAW. EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES PROVIDE THE LIBRARY "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE LIBRARY IS WITH YOU. SHOULD THE LIBRARY PROVE DEFECTIVE, YOU ASSUME

THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION.

16. IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN WRITING WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MAY MODIFY AND/OR REDISTRIBUTE THE LIBRARY AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OR INABILITY TO USE THE LIBRARY (INCLUDING BUT NOT LIMITED TO LOSS OF DATA OR DATA BEING RENDERED INACCURATE OR LOSSES SUSTAINED BY YOU OR THIRD PARTIES OR A FAILURE OF THE LIBRARY TO OPERATE WITH ANY OTHER SOFTWARE), EVEN IF SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

END OF TERMS AND CONDITIONS

How to Apply These Terms to Your New Libraries

If you develop a new library, and you want it to be of the greatest possible use to the public, we recommend making it free software that everyone can redistribute and change. You can do so by permitting redistribution under these terms (or, alternatively, under the terms of the ordinary General Public License).

To apply these terms, attach the following notices to the library. It is safest to attach them to the start of each source file to most effectively convey the exclusion of warranty; and each file should have at least the "copyright" line and a pointer to where the full notice is found.

```
\<one line to give the library's name and a brief idea of what it does.\>
Copyright (C) \<year\> \<name of author\>
```

```
This library is free software; you can redistribute it and/or
modify it under the terms of the GNU Lesser General Public
License as published by the Free Software Foundation; either
version 2.1 of the License, or (at your option) any later version.
```

```
This library is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
Lesser General Public License for more details.
```

```
You should have received a copy of the GNU Lesser General Public
License along with this library; if not, write to the Free Software
Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301 USA
```

Also add information on how to contact you by electronic and paper mail.

You should also get your employer (if you work as a programmer) or your school, if any, to sign a "copyright disclaimer" for the library, if necessary. Here is a sample; alter the names:

```
Yoyodyne, Inc., hereby disclaims all copyright interest in the
library 'Frob' (a library for tweaking knobs) written by James Random Hacker.
```

```
\<signature of Ty Coon\>, 1 April 1990
Ty Coon, President of Vice
```

That's all there is to it!

GNU LESSER GENERAL PUBLIC LICENSE Version 3, 29 June 2007

Copyright (C) 2007 Free Software Foundation, Inc. <<http://fsf.org/>>
Everyone is permitted to copy and distribute verbatim copies
of this license document, but changing it is not allowed.

This version of the GNU Lesser General Public License incorporates the terms and conditions of version 3 of the GNU General Public License, supplemented by the additional permissions listed below.

0. Additional Definitions.

As used herein, "this License" refers to version 3 of the GNU Lesser General Public License, and the "GNU GPL" refers to version 3 of the GNU General Public License.

"The Library" refers to a covered work governed by this License, other than an Application or a Combined Work as defined below.

An "Application" is any work that makes use of an interface provided by the Library, but which is not otherwise based on the Library. Defining a subclass of a class defined by the Library is deemed a mode of using an interface provided by the Library.

A "Combined Work" is a work produced by combining or linking an Application with the Library. The particular version of the Library with which the Combined Work was made is also called the "Linked Version".

The "Minimal Corresponding Source" for a Combined Work means the Corresponding Source for the Combined Work, excluding any source code for portions of the Combined Work that, considered in isolation, are based on the Application, and not on the Linked Version.

The "Corresponding Application Code" for a Combined Work means the object code and/or source code for the Application, including any data and utility programs needed for reproducing the Combined Work from the Application, but excluding the System Libraries of the Combined Work.

1. Exception to Section 3 of the GNU GPL.

You may convey a covered work under sections 3 and 4 of this License without being bound by section 3 of the GNU GPL.

2. Conveying Modified Versions.

If you modify a copy of the Library, and, in your modifications, a facility refers to a function or data to be supplied by an Application that uses the facility (other than as an argument passed when the facility is invoked), then you may convey a copy of the modified version:

- a) under this License, provided that you make a good faith effort to ensure that, in the event an Application does not supply the function or data, the facility still operates, and performs whatever part of its purpose remains meaningful, or
- b) under the GNU GPL, with none of the additional permissions of this License applicable to that copy.

3. Object Code Incorporating Material from Library Header Files.

The object code form of an Application may incorporate material from a header file that is part of the Library. You may convey such object code under terms of your choice, provided that, if the incorporated material is not limited to numerical parameters, data structure layouts and accessors, or small macros, inline functions and templates (ten or fewer lines in length), you do both of the following:

- a) Give prominent notice with each copy of the object code that the Library is used in it and that the Library and its use are covered by this License.
- b) Accompany the object code with a copy of the GNU GPL and this license document.

4. Combined Works.

You may convey a Combined Work under terms of your choice that, taken together, effectively do not restrict modification of the portions of the Library contained in the Combined Work and reverse engineering for debugging such modifications, if you also do each of the following:

- a) Give prominent notice with each copy of the Combined Work that the Library is used in it and that the Library and its use are covered by this License.
- b) Accompany the Combined Work with a copy of the GNU GPL and this license document.
- c) For a Combined Work that displays copyright notices during execution, include the copyright notice for the Library among these notices, as well as a reference directing the user to the copies of the GNU GPL and this license document.
- d) Do one of the following:
 - 0) Convey the Minimal Corresponding Source under the terms of this License, and the Corresponding Application Code in a form suitable for, and under terms that permit, the user to recombine or relink the Application with a modified version of the Linked Version to produce a modified Combined Work, in the manner specified by section 6 of the GNU GPL for conveying Corresponding Source.
 - 1) Use a suitable shared library mechanism for linking with the Library. A suitable mechanism is one that (a) uses at run time a copy of the Library already present on the user's computer system, and (b) will operate properly with a modified version of the Library that is interface-compatible with the Linked Version.
- e) Provide Installation Information, but only if you would otherwise be required to provide such information under section 6 of the GNU GPL, and only to the extent that such information is necessary to install and execute a modified version of the Combined Work produced by recombining or relinking the Application with a modified version of the Linked Version. (If you use option 4d0, the Installation Information must accompany the Minimal Corresponding Source and Corresponding Application Code. If you use option 4d1, you must provide the Installation Information in the manner specified by section 6 of the GNU GPL for conveying Corresponding Source.)

5. Combined Libraries.

You may place library facilities that are a work based on the Library side by side in a single library together with other library facilities that are not Applications and are not covered by this License, and convey such a combined library under terms of your choice, if you do both of the following:

- a) Accompany the combined library with a copy of the same work based on the Library, uncombined with any other library facilities, conveyed under the terms of this License.
- b) Give prominent notice with the combined library that part of it is a work based on the Library, and explaining where to find the accompanying uncombined form of the same work.

6. Revised Versions of the GNU Lesser General Public License.

The Free Software Foundation may publish revised and/or new versions of the GNU Lesser General Public License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns.

Each version is given a distinguishing version number. If the Library as you received it specifies that a certain numbered version of the GNU Lesser General Public License "or any later version" applies to it, you have the option of following the terms and conditions either of that published version or of any later version published by the Free Software Foundation. If the Library as you received it does not specify a version number of the GNU Lesser General Public License, you may choose any version of the GNU Lesser

General Public License ever published by the Free Software Foundation.

If the Library as you received it specifies that a proxy can decide whether future versions of the GNU Lesser General Public License shall apply, that proxy's public statement of acceptance of any version is permanent authorization for you to choose that version for the Library.

2.2 CUBA Software

This section contains information on the STAR-System CUBA Software application.

The STAR-System CUBA Software is designed to support the development and testing of SpaceWire systems, allowing interactive or scripted sending and receiving of SpaceWire packets, including use of the RMAP protocol.

The STAR-System CUBA Software can be used to send and receive raw SpaceWire packets, either entered and displayed at the command line, or reading from command files and outputting received packets to an output file. It can also perform similar tasks using the SpaceWire Remote Memory Access Protocol (RMAP). RMAP commands to be performed may be entered at the command line, or supplied as a list of commands in a command file. The results of these operations can then be displayed on the command line or output to a file.

The CUBA Software can be run from the start menu, in which case it will start in its fully interactive mode. Alternatively it may be run from the command line with a number of optional arguments which can be used to modify its behaviour. These arguments are detailed in the [CUBA Command Line Arguments](#) section.

If there are no compatible SpaceWire devices connected when the software is run, it will display a message and exit. If there is only one device connected, this device will be used. If there is more than one device connected, and a valid device number has not been specified at the command line, the software will display a list of available devices and ask for one to be selected.

If the selected device has more than one channel available for use, and a valid channel number has not been specified at the command line, the software will ask for a channel number to be selected.

The command line options allow the application to be run in one of the command stream modes without requiring any input from the user. This can be useful for executing test scripts, for example, and the software can also be called from a batch file or shell script.

The following sections describe use of the CUBA software to send and receive raw SpaceWire packets in interactive and command stream modes, and similarly its use for RMAP packets in interactive and command stream modes.

2.2.1 Data Format Conventions

When entering data values for SpaceWire and RMAP packets, both in interactive mode and in command files, there are a number of conventions which may be used for numeric and other data.

The initial default base is hexadecimal, although this can be changed by commands in both interactive and command stream modes. Space-separated data values can be entered in the default base, and the base can be changed per data value if required by preceding it by a base change indicator. These are:

- 'b or 'B for binary - e.g. 'b10110100
- 'o or 'O for octal - e.g. 'O264
- 'd or 'D for decimal - e.g. 'd180
- 'x or 'X for hex - e.g. 'Xb4

ASCII character strings may be included in data fields by enclosing the required characters in double quotes - e.g. "Test string". The ASCII values of the characters will be used, and sent as individual bytes.

The contents of a data file may be used by entering the file name preceded by a hyphen - e.g. -testfile.dat. If the file name contains spaces it can be enclosed in double quotes - e.g. -"another file.txt". The contents of the file will be sent as raw data bytes.

All of the above data input modes can be combined as needed to define the data to be sent. Assuming the default base to be hex, the following is a valid data definition:

```
12 34 "characters" 'd234 'b10101100 -datafile.txt fe ff
```

Note that when the CUBA Software is operating in backwards compatibility mode, slightly different conventions apply. See the [Backwards Compatibility Mode](#) section for more details.

2.2.2 SpaceWire Interactive Mode

The SpaceWire Interactive Command/Reply Mode allows the bytes of a packet to be sent to be entered at the command line. After the bytes have been entered, pressing Enter will cause the bytes to be sent in a packet terminated by an EOP.

Once the packet has been sent, the software will wait for packets to be received. Each packet received will be shown at the command line, including the end of packet marker type. To stop the software waiting for packets, press any key. To receive packets without first sending one, just press Enter (without specifying any bytes to be sent) when the software asks for a packet to be sent.

The base used when entering packets to be sent and displaying any responses may be altered by entering base=<base>, where <base> is 2, 0b or i for binary; 8, 0o or o for octal; 10, 0d or n for decimal; or 16, 0x or h for hexadecimal. Note that when entering the base, any numbers entered are assumed to be decimal, regardless of the current base.

To exit the program, x should be entered.

2.2.3 RMAP Interactive Mode

The RMAP Interactive Command/Reply Mode allows RMAP commands to be specified at the command line. The software prompts for each of the fields in the command to be entered.

When this mode is entered a list of possible commands to be performed is provided:

- Read command
- Write command
- Read-modify-write command
- Change the default base
- Delete all fixed fields
- Exit

Selecting to perform a read, write or read-modify-write command will result in the software asking for values for each of the fields for those commands. If a value for a field has already been provided when performing a previous command, that value will be shown as the default value. Pressing Enter without entering a value will cause the software to use the default, otherwise a value can be entered. This will then become the new default.

If a value is entered followed by an x, then that value will become fixed, and all future commands will not ask for this field to be specified, the fixed value will be used instead. A default value can also be set to fixed by simply entering x. Fixed fields may be deleted by selecting the appropriate option from the main menu.

When a boolean value is expected, t, true, y or yes may be entered to indicate true, while f, false, n or no may be entered to indicate false. If a series of bytes is expected, each byte should be entered separated by a space, while strings can also be used, as described in the [Data Format Conventions](#) section.

Once the fields of a command have been specified, the command will be sent. If a response is expected by the command, the software will wait for the response. If a response is received it will be displayed. To stop waiting for a response, a key can be pressed. The main menu will be displayed once the command has completed or has been stopped.

The table below shows the valid fields for RMAP Interactive Command/Reply mode.

Fields for RMAP Interactive Command/Reply Mode

Field Name	Type	Command Write	Read	R-M-W
Target Address	Series of bytes	Yes	Yes	Yes
Initiator or Reply Address	Series of bytes	Yes	Yes	Yes
Verify Before Write	Boolean	Yes	—	—
Acknowledge	Boolean	Yes	—	—
Increment Address	Boolean	Yes	Yes	—
Key	1 byte number	Yes	Yes	Yes
Transaction Identifier	2 byte number	Yes	Yes	Yes
Read Write Address	4 byte number	Yes	Yes	Yes
Extended Read Write Address	1 byte number	Yes	Yes	Yes
Data Length	3 byte number	—	Yes	—
Header CRC	1 byte number	Yes	Yes	Yes
Data	Series of bytes	Yes	—	Yes
Mask	4 byte number	—	—	Yes
Data CRC	1 byte number	Yes	—	Yes

The base used when entering field values and displaying the field values in responses may be altered by selecting the appropriate menu option. The base should then be specified, with 2, 0b or i for binary; 8, 0o, or o for octal; 10, 0d or n for decimal; or 16, 0x or h for hexadecimal. Note that when entering the base, any numbers entered are assumed to be decimal, regardless of the current base. Once the base has been specified, the software will return to the menu.

To exit the program, x should be entered.

2.2.4 Command File Directives

Both the SpaceWire and RMAP command stream modes support common directives which are processed prior to processing and executing the commands in the input file provided. These are similar to pre-processor directives used in C and C++ programming, for example. All directives begin with a # character. The supported directives are described below.

2.2.4.1 #INCLUDELIST Directive

The #INCLUDELIST directive allows other command files to be included in a command file. This is similar to a #include in C and C++. The name of the file to be included should follow the #INCLUDELIST directive, and the

command should be terminated with a semi-colon. The filename may contain spaces and the #INCLUDELIST may be specified in any mixture of upper and lowercase.

Where an #INCLUDELIST directive is encountered, the CUBA Software will replace the directive with the command file specified in the directive before processing and executing the commands. The included file could in turn include further files.

The example below includes the file `include file.cuba`. Note that quotes are not required around the filename, even though it contains spaces

```
#IncludeList include file.cuba;
```

2.2.4.2 #ALIAS Directive

The #ALIAS directive allows a string in the file and any included files to be replaced with a value specified in the #ALIAS directive. This is similar to a #define macro in C and C++. The #ALIAS directive should be followed by the alias string, an equals sign (=) and then the value to replace the alias string with. The directive should be terminated with a semi-colon. As with the #INCLUDELIST directive, the case of the #ALIAS text is not important. However the case of the alias string is important. Only text that is exactly the same as the string specified will be replaced.

Where an #ALIAS directive is encountered, the CUBA software will remove the alias directive and replace all occurrences of the alias string following the directive, including those included using the #INCLUDELIST directive, with the alias value after the equals sign. This value can include spaces and can be useful for specifying commonly used values, particularly those that might need to be changed in future.

For example, the path address to a device can be represented using an #ALIAS directive, as shown below:

```
#ALIAS ADDRESS=1 3;  
  
S D:ADDRESS aa bb cc dd ee ff;  
S D:ADDRESS 01 23 45 67 89;
```

2.2.5 SpaceWire Command Stream Mode

In the SpaceWire Command Stream Mode a file containing a list of SpaceWire commands to be performed must be specified. A file to output any packets received can also be specified, or left as blank if the received packets are of no interest.

A file to dump the pre-processed input file can also be specified. This will produce a file with all comments removed, and all directives (such as #ALIAS and #INCLUDELIST directives) processed. This can be useful to debug any problems with a command file.

There are four types of commands which can be included in a SpaceWire Command Stream: send packets, receive packets, loop commands or change the base in use. These are described below. Each command must be terminated with a semi-colon (;) and comments may be included by placing them between /* and */. The information in a comment is ignored.

2.2.5.1 SpaceWire Send Command

Send commands begin with S. An optional N: parameter can be used to indicate the number of packets to be sent. Each packet will be identical. The N: should be followed by the number of packets. If this parameter is not specified only one copy of the packet is sent.

By default, all packets are terminated with a normal end of packet marker. An optional E: parameter allows the end of packet marker type to be included at the end of the packet to be specified. The E: should be followed by O, N or F to indicate an EOP should be used (eOp, No or False), or E, Y or T to indicate an EEP should be used (eEp, Yes or True). The value provided is case-insensitive.

The data to be sent should be included following a D: parameter, each byte separated by a space. The following is an example send command (it is assumed the base is currently hexadecimal):

```
S D:11 22 33 44 55 66 77 88 99 aa bb cc dd ee ff;
```

To send this packet ten times, terminated by an EEP, the following command could be used:

```
S N:a E:t D:11 22 33 44 55 66 77 88 99 aa bb cc dd ee ff;
```

2.2.5.2 SpaceWire Receive Command

Receive commands begin with R. An optional N: parameter can be used to indicate the number of packets to be received. The N: should be followed by the number of packets. The following is an example receive command to receive a single packet:

```
R;
```

To receive 10 packets, the following command could be used:

```
R N:0d10;
```

Received packets can also be written to a file using the optional F: parameter. The file to write to should be included after the F: parameter, and should be placed in quotes if it contains spaces. The file specified will be appended to (not overwritten), so that the same file can be used for multiple commands. The data will be written to the file as raw bytes. The following example will write the received data to the file rmap_output:

```
R L:0d10 F:rmap_output;
```

Note that there are no markers inserted in the file to indicate the start or end of packets.

2.2.5.3 Loop Command

Loop commands begin with L and allow subsequent commands to be repeated. An N: parameter should be used to indicate the number of commands to be repeated. The N: should be followed by the number of commands to repeat. A T: parameter should be used to indicate the number of times to repeat the commands. The T: should be followed by the number of times to repeat the commands.

The following is an example loop command to send 5 packets and receive 5 packets:

```
L N:2 T:5;
  S D:00 11 22 33 44 55 66 77 88 99 aa bb cc dd ee ff;
  R L:10;
```

In the above example, the two commands following the loop command are repeated five times. Indentation has been used to highlight the commands being repeated.

Loop commands can be nested within one another. When this is done, the loop command and all the commands below it that are to be repeated are treated as one command. The following commands send 20 packets and receive twenty responses:

```
L N:2 T:5;
  S N:4 D:00 11 22 33 44 55 66 77 88 99 aa bb cc dd ee ff;
  L N:2 T:2;
    R L:10;
    R L:10;
```

The first loop command will repeat the send command and the second loop command five times. The second loop command will repeat the two receive commands twice. As this loop command is being repeated 5 times, the two receive commands are each repeated a total of ten times.

2.2.5.4 Change Base Command

To change the base of values in the file, the B command can be used. Following the B and a space, the base to use should be entered, with 2, 0b or i indicating binary; 8, 0o or o indicating octal; 10, 0d or n indicating decimal; and 16, 0x or h indicating hexadecimal. Note that the values for the base are always entered in decimal. The following is an example command to change the base to decimal:

```
B 10;
```

The base used in output files can be changed in a similar way, using the O command. The following is an example command to change the output file's base to octal:

```
O 8;
```

Only the last command to change the output file's base will be used.

Note that the base commands are not included in loop commands, so cannot be repeated. The input base affects all text below the command, while the output base affects the entire output file.

2.2.5.5 Example SpaceWire Command Stream Input File

The following is an example command stream input file, containing comments explaining what the file does:

```
/* Send 10 packets */
S N:a D:1 3 11 22 33 44 55 66 77 88 99 aa bb cc dd ee ff;

/* Change the base to decimal */
B 10;

/* Receive 10 packets, 16 bytes in length */
R N:10 L:16;
```

2.2.5.6 SpaceWire Command Stream Output Files

The output files produced when in SpaceWire Command Stream Mode contain the results of each receive operation. Each line in the output file corresponds to a receive operation in the command file (a receive operation may have multiple lines in the output file, e.g. if it is meant to receive multiple packets). The first character on the line indicates the success or failure of the operation, with the following values possible:

- F - Full packet received
- P - Partial packet received
- T - Timeout occurred (Partial packet might have been received)
- E - Error occurred

Following this character and a space is the contents of any packets read. The bytes of the packet will be shown in hexadecimal, unless an alternative base has been specified in the command file. If an end of packet marker has been received for a packet, this shall be included at the end of the line, with EOP indicating a normal end of packet marker and EEP indicating an error end of packet marker.

A comment line is included at the top of every output file, indicating the base in use.

The following is an example of a successfully read packet terminated with an EOP:

```
/* Base = Hexadecimal */

F 11 22 33 44 55 66 77 88 99 EOP
```

2.2.5.7 Example SpaceWire Command Stream Output File

The following is an example command stream output file, in which 8 packets have been successfully received terminated by an EOP, one has been received terminated by an EEP, and the final one timed out:

```
/* Base = Hexadecimal */
F 11 22 33 44 55 66 77 88 99 AA BB CC DD EE FF EOP
F 11 22 33 44 55 66 77 88 99 AA BB CC DD EE FF EOP
F 11 22 33 44 55 66 77 88 99 AA BB CC DD EE FF EOP
F 11 22 33 44 55 66 77 88 99 AA BB CC DD EE FF EOP
F 11 22 33 44 55 66 77 88 99 AA BB CC DD EE FF EOP
F 11 22 33 44 55 66 77 88 99 AA BB CC DD EE FF EOP
F 11 22 33 44 55 66 77 88 99 AA BB CC DD EE FF EOP
F 11 22 33 44 55 66 77 88 99 AA BB CC DD EE FF EOP
F 11 22 33 44 55 EEP
T
```

2.2.6 RMAP Command Stream Mode

In the RMAP Command Stream Mode a file containing a list of RMAP commands to be performed must be specified. A file to output any responses to these commands can also be specified, or left as blank if the responses are of no interest.

2.2.6.1 RMAP Command Stream Input Files

RMAP Command Stream input files take the form of a number of fields, each followed by a value. The end of a command is indicated by a blank line. The first command must have all fields required for that command specified, other than the CRC field, which is automatically calculated. Subsequent commands can omit a field if it has been specified in a previous command.

The table below shows the fields which should be provided for each type of command. Note that R-M-W refers to the read-modify-write command. The first field specified should be the type of the command, which can be a write, read or read-modify-write. The field name does not need to be specified when specifying the command type. Subsequent fields can be in any order. To specify a field, the field's full name should be supplied, or 2 or more characters of its short name. This should be followed by a colon, then the value of the field, and then a semi-colon.

For the command type, w, r or m should be specified. If a series of bytes is expected, each byte should be a value between 0 and 0xFF, and should be separated from the next byte by a space. Additionally, strings and files can also be used. If a boolean value is expected, then either t, true, y or yes can be used to indicate true, while f, false, n or no can be used to indicate false.

If a 1 byte number is expected, a value between 0 and 0xFF should be entered.

If a 2 byte number is expected, a value between 0 and 0xFFFF should be entered.

If a 3 byte number is expected, a value between 0 and 0xFFFFFF should be entered.

If a 4 byte number is expected, a value between 0 and 0xFFFFFFFF should be entered.

The table below shows the valid fields for RMAP Command Stream mode.

Fields for RMAP Command Stream Input Files

Field Name	Short Name	Type	Command Write	Read	R-M-W
------------	------------	------	---------------	------	-------

CommandType	Command	W, R or M	W	R	M
TargetAddress	Target	Series of bytes	Yes	Yes	Yes
ReplyAddress	Reply	Series of bytes	Yes	Yes	Yes
VerifyBefore↔ Write	Verify	Boolean	Yes	—	—
Acknowledge	Acknowledge	Boolean	Yes	—	—
Increment↔ Address	Increment	Boolean	Yes	Yes	—
Key	Key	1 byte number	Yes	Yes	Yes
Transaction↔ Identifier	Transaction	2 byte number	Yes	Yes	Yes
ReadWrite↔ Address	Address	4 byte number	Yes	Yes	Yes
Extended↔ ReadWrite↔ Address	Extended	1 byte number	Yes	Yes	Yes
DataLength	Length	3 byte number	—	Yes	—
HeaderCRC	HeaderCRC	1 byte number	Yes	Yes	Yes
Data	Data	Series of bytes	Yes	—	Yes
Mask	Mask	4 byte number	—	—	Yes
DataCRC	CRC	1 byte number	Yes	—	Yes

Refer to the [Data Format Conventions](#) section for information on data formats for the contents of the fields

For command types that expect responses containing data (read and read-modify-write commands), an optional Filename field can be provided to specify a file to write the data to. The file to write to should be included after the Filename field, and should be placed in quotes if it contains spaces. The file specified will be appended to (not overwritten), so that the same file can be used for multiple commands. The data will be written to the file as raw bytes.

2.2.6.2 Loop Command

The RMAP Command Stream Mode includes a loop command similar to the one provided for the SpaceWire Command Stream Mode. Loop commands have a command type of l and allow subsequent commands to be repeated. A NumberToRepeat field should be used to indicate the number of commands to be repeated, while a TimesTo↔Repeat parameter should be used to indicate the number of times to repeat the commands. Both fields accept numerical values, and only the first 2 characters of the field names are required.

Loop commands can be nested within one another. When this is done, the loop command and all the commands below it that are to be repeated are treated as one command.

Note that the base commands are not included in loop commands, so cannot be repeated.

2.2.6.3 Example RMAP Command Stream Input File

The following is an example RMAP command stream input file, containing comments explaining what the file does. Indentation has been used to highlight the commands repeated in loop commands.

```
/* Send a write command to address 0x106, using full field names */
CommandType: w;
DestinationAddress: 1 0 FE;
VerifyBeforeWrite: T;
Acknowledge: T;
IncrementTarget: F;
DestinationKey: 20;
SourceAddress: 3 FE;
TransactionIdentifier: 0;
ExtendedReadWriteAddress: 0;
ReadWriteAddress: 106;
Data: 01 23 45 67;
```

```

/* Change the output base to decimal */
B;
Output: 10;

/* Loop over the next 3 commands (which includes a loop) twice */
l;
TimesToRepeat: 2;
numbertorepeat: 3;

    /* Send a write command to address 0x107, using short field names and using defaults */
    w;
    Address: 107;
    Data: 89 ab cd ef;

    /* Send a write command to address 0x108, using decimal and octal values, */
    /* (very) short field names and using defaults */
    w;
    Ad: 108;
    Da: 0o21 34 0o63 68;

    /* Loop over the next command twice */
    l;
    ti: 2;
    nu: 1;

    /* Read address 108, and write the data to rmap_out */
    r;
    file: rmap_out;

```

2.2.6.4 RMAP Command Stream Output Files

RMAP Command Stream output files contain the fields and their values of any received packets. These can be acknowledges to write commands, or responses to read and read-modify-write commands containing the data read.

If an expected response was not received, due to a timeout, then an error will be included in the file. This will show the type of the expected command and the expected transaction identifier.

The base in use to show the field values is shown in a comment at the top of the file.

2.2.6.5 Example RMAP Command Stream Output File

The following is an example command stream output file, in which the responses to two RMAP write commands have been successfully received, while a third has timed out:

```

/* Base = Hexadecimal */

Write Response
-----
Reply Address: FE
Data verification is enabled
Incrementing addresses is not enabled.
Status = success
Target Address: FE
TransactionIdentifier: 0
Header CRC: bd

Write Response
-----

```

```

Reply Address: FE
Data verification is enabled
Incrementing addresses is not enabled.
Status = success
Target Address: FE
TransactionIdentifier: 0
Header CRC: ba

```

Error receiving acknowledgement for command 3

2.2.7 CUBA Command Line Arguments

The arguments that can be passed to the CUBA Software are described below:

-help	Display information about the application and the arguments that can be passed to it, then exit.								
-device=<device_num>	If more than one SpaceWire device is connected to the PC in use, the number of the device to be used by the software to send and receive packets can be specified by providing its number. The number of the first device connected is 0.								
-channel=<channel_num>	If more than one SpaceWire channel is present on the selected device, the number of the channel to be used by the software to transmit and receive packets can be specified by providing its number.								
-all_devices=<enable>	By default the CUBA Software only accesses devices which can transmit and receive packets. Setting this parameter non-zero allows the device to be selected from all available SpaceWire devices								
-mode=<mode>	Sets the mode the software operates in, where <mode> is one of the following values: <table> <tr> <td>interactive_↔ spacewire or iscr</td><td>Interactive SpaceWire Command/Reply Mode</td></tr> <tr> <td>interactive_↔ rmap or ircr</td><td>Interactive RMAP Command/Reply Mode</td></tr> <tr> <td>spacewire_↔ command or scs</td><td>SpaceWire Command Stream Mode</td></tr> <tr> <td>rmap_command or rcs</td><td>RMAP Command Stream Mode</td></tr> </table>	interactive_↔ spacewire or iscr	Interactive SpaceWire Command/Reply Mode	interactive_↔ rmap or ircr	Interactive RMAP Command/Reply Mode	spacewire_↔ command or scs	SpaceWire Command Stream Mode	rmap_command or rcs	RMAP Command Stream Mode
interactive_↔ spacewire or iscr	Interactive SpaceWire Command/Reply Mode								
interactive_↔ rmap or ircr	Interactive RMAP Command/Reply Mode								
spacewire_↔ command or scs	SpaceWire Command Stream Mode								
rmap_command or rcs	RMAP Command Stream Mode								

<code>-command_list=<file></code>	Specify the path to a file containing a command list. This argument is only valid in the command stream modes, and is ignored in the interactive modes.
<code>-output=<file></code>	Specify the path to a file where any output will be recorded. This argument is only valid in the command stream modes, and is ignored in the interactive modes. The <code><file></code> may be left blank if no output file is required.
<code>-timeout=<timeout></code>	Sets the timeout to be applied for any read operations, specified in seconds. Values with a decimal point are permitted. This argument is only valid in the command stream modes, and is ignored in the interactive modes. If this option is not specified, the default value is 10 seconds.
<code>-dump_cmdlist=<file></code>	Specify the path to a file which will be updated to contain the command file to be used after all pre-processing has been performed. This argument is only valid in the command stream modes, and is ignored in the interactive modes.
<code>-backwards_compatibility=<1 or 0></code>	Enables or disables the backwards compatibility mode. If backwards compatibility is enabled, all scripts written for the original CUBA Software for the old SpaceWire USB API will work in the command stream modes. Note that there are some minor differences in data representation as noted below in the Backwards Compatibility Mode section.

2.2.8 Backwards Compatibility Mode

Backwards Compatibility Mode is provided to allow the use of command stream files which were created for use with the previous version of the CUBA Software supplied with the Brick (Mk1), Router-USB and Router Mk2. There are a number of differences, particularly in the format of numeric data, which should be noted when using this mode.

To change the base used for a data item, the base indicator is preceded by a zero, i.e. '0b' for binary, '0o' for octal, '0d' for decimal and '0x' for hexadecimal (the base is not case-sensitive). This can lead to errors when using hexadecimal as the default base and the values 0b or 0d are used. These will be interpreted as base change requests with no following value, and will result in those values being omitted from the data.

In order to enter the values 0b and 0d they must either be input as simply 'b' and 'd', or explicitly in hex as '0x0b' and '0x0d'.

A further potential source of errors in command file data blocks is that data entered over multiple lines must have a trailing space at the end of each line. This separator must be present, otherwise the value at the end of the line will be concatenated with the first value on the next line. E.g. data entered as

```
Data: 12 34 ... .. 98 76 (no trailing space)
ab cd ... ..
```

would be interpreted as 12 34 98 76ab cd Since data is sent as bytes, the value '76ab' would be transmitted as only 'ab', omitting the '76' value.

Note that these are problems present in the original CUBA Software.

In backwards compatibility mode character strings may be entered using the single quote character. However, in the STAR-System version this character has been redefined to signal base change (see [Data Format Conventions](#)) and cannot be used for strings.

Command stream files which have been used successfully with the older version of the CUBA Software will continue to work correctly with the STAR-System CUBA Software when backwards compatibility is enabled. When writing

new command files we recommend using the new conventions for formatting data.

Chapter 3

STAR-System Devices

The following sections describe the hardware devices supported by STAR-System, and how they can be updated:

- [SpaceWire PCI Mk2 and cPCI Mk2](#)
- [SpaceWire PCIe](#)
- [SpaceWire Brick Mk2](#)
- [SpaceWire Router Mk2S](#)
- [SpaceWire Brick Mk3](#)
- [SpaceWire PXI Interface and PXI Interface Mk2](#)
- [SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2](#)
- [Other STAR-System Devices](#)
- [Device Update Application](#)

3.1 SpaceWire PCI Mk2 and cPCI Mk2

Documentation for the STAR-Dundee SpaceWire PCI Mk2 and cPCI Mk2 devices is provided in this section.

3.1.1 Overview

3.1.1.1 Summary

This user manual provides an overview of the interfaces and features of the SpaceWire PCI Mk2 hardware.

For detailed information on using the SpaceWire PCI Mk2 card please consult the API reference documentation.

3.1.1.2 Hardware Description

The SpaceWire PCI Mk2 provides a PCI to SpaceWire interface device with three SpaceWire interfaces. Efficient host software support is provided to support rapid sending and receiving of SpaceWire packets into host PC memory.

A block diagram of the SpaceWire PCI Mk2 board is shown below.

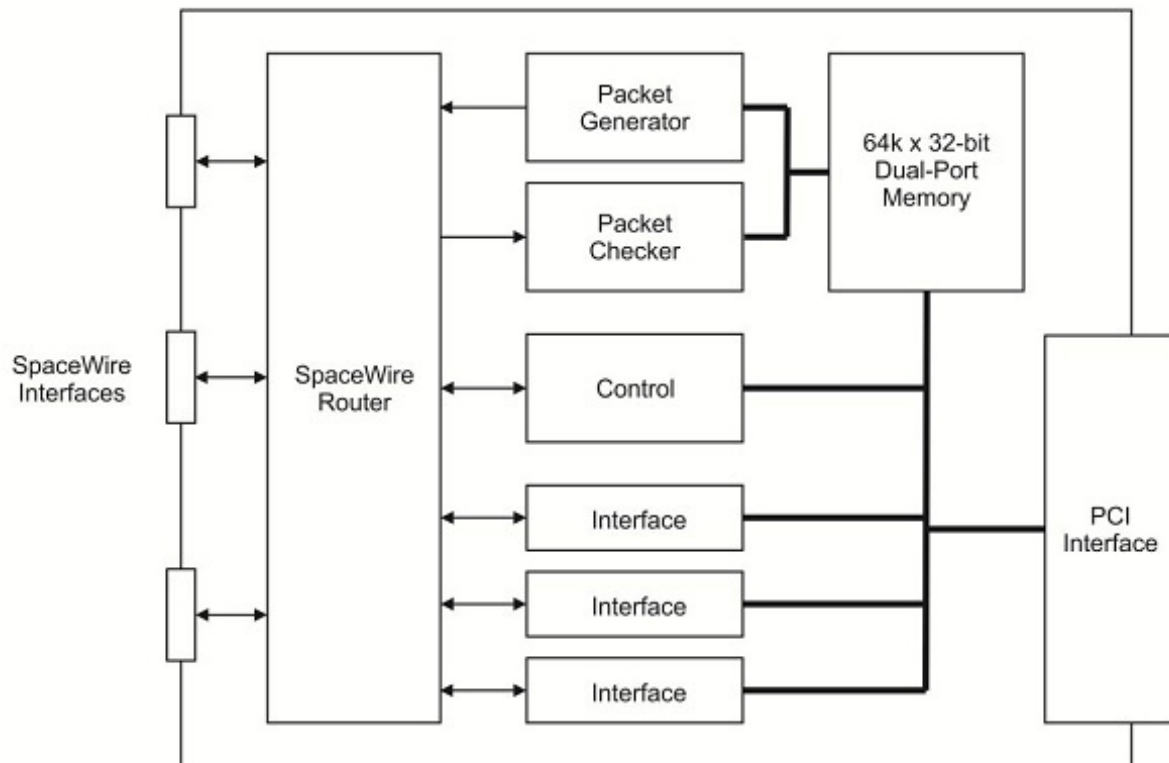


Figure 3.1: SpaceWire PCI Mk2 hardware overview

The three SpaceWire interfaces of the SpaceWire PCI Mk2 are fully compliant with the SpaceWire standard and operate at up to 200 Mbit/s. They are connected to a SpaceWire router which supports two modes of operation, interface mode and router mode. In router mode, packets from any SpaceWire port can be routed to any another SpaceWire port, or the PCI interface. In interface mode, the SpaceWire router is passive and packets which are received on a SpaceWire link are passed directly to the PCI channel for that link. There are three independent channels from the SpaceWire router to the PCI interface, so traffic flowing over one port cannot block traffic for another port. In addition, there is a separate control channel, so that the host PC is always able to access the control, configuration and status space of the SpaceWire PCI Mk2 regardless of the data flow.

The PCI interface is 32 bits wide and can operate at 33 or 66 MHz. It contains a DMA controller for rapid transfer of data to and from the SpaceWire PCI Mk2 board.

A hardware packet generator and checker are included on the SpaceWire PCI Mk2 to automatically generate and check SpaceWire packets at high speed without using host computer resources. The packets to be generated are stored in a dual-port memory on the PCI Mk2 board. Packets of any length can be generated with a separate header and cargo as required. When a packet is larger than the amount of RAM available on the on board dual port RAM the cargo part of the packet will be resent multiple times to satisfy the packet length required. The speed of packet data generation and the gap between packets can be controlled.

The packet checker receives an incoming packet and checks it against a template held in the dual-port memory, while the incoming packet can be stored in the dual-port memory. Any mismatches can be flagged to the host computer. The packet checker is very useful for the automatic testing of packets from high data-rate instruments. When interface mode is used the SpaceWire router can be configured to automatically route packets from any SpaceWire port to the packet checker. Alternatively in router mode the packets must have a header byte which routes packets to the packet checker address.

3.1.2 Getting Started

Note: Before inserting the SpaceWire PCI Mk2 card, the software and driver programs should first be installed on the host computer operating system. This ensures that the SpaceWire PCI Mk2 card is recognised by the host when it boots up.

The SpaceWire PCI Mk2 card fits into any standard PCI slot on a typical desktop PC. The card can be inserted into a standard 3.3 or 5 Volt PCI system.

3.1.3 Interfaces

3.1.3.1 SpaceWire Ports

The SpaceWire port at the bottom of the board (i.e. closest to the PCI bus, or motherboard), is SpaceWire port 3 and the furthest away is port 1. The port numbering is shown below.

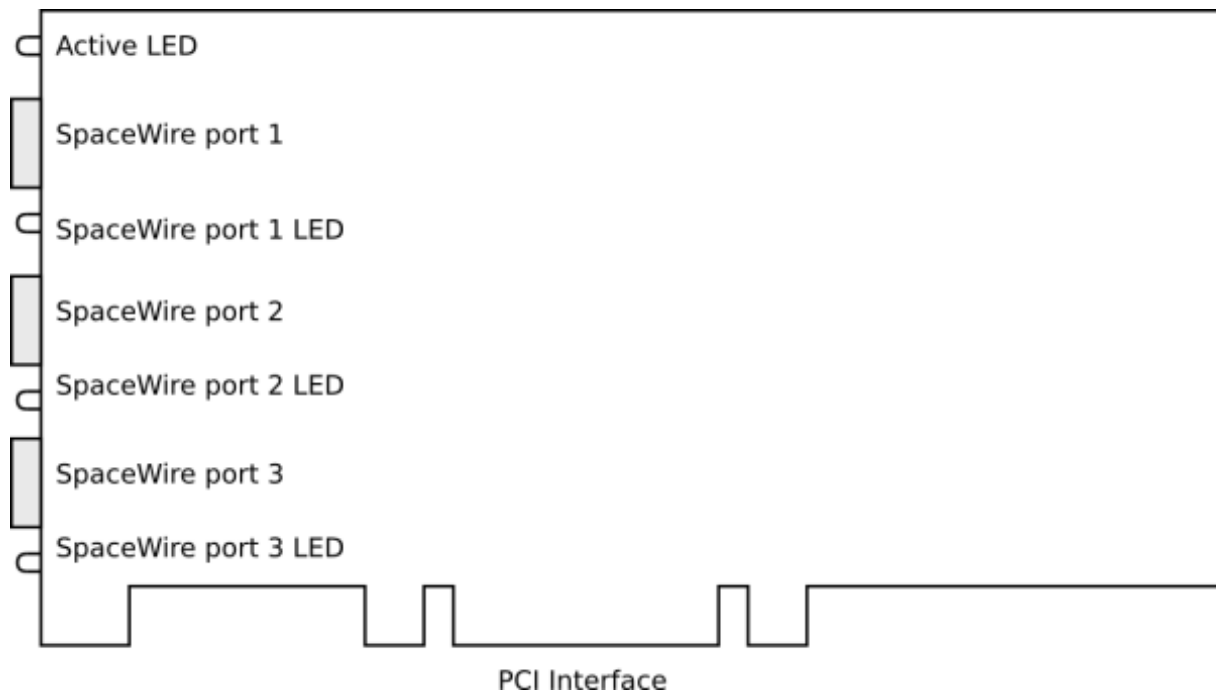


Figure 3.2: SpaceWire ports and LED positions

3.1.3.2 Front Panel LEDs

The PCI card has four front panel LEDs as shown in the image above. The LEDs are single colour red LEDs which are either on or off.

The functions of the SpaceWire port LEDs are defined in the table below.

Function	LED Mode	Description
----------	----------	-------------

No event	OFF	The LED is turned OFF when the link is not running and no errors are indicated.
Link running	ON	The LED is turned ON when the link is running. The link running LED is held for 200 ms after the link has stopped running and no error occurs.
Error	Fast flashing	The LED is flashed quickly at intervals of 100 ms.
Identify	Slow flashing	Identify mode is a convenience function which allows the user to visually determine which SpaceWire PCI board the software is talking to. The LED is flashed for a few seconds at intervals of 250 ms.

Please note that there is an issue with all SpaceWire interfaces where a small amount of noise on the inputs can be translated by the LVDS receivers into a digital pattern and then interpreted by the receiver as a bit sequence. Since this bit sequence is invalid it causes the SpaceWire CODEC to start up and then disconnect. The SpaceWire interface LEDs flash red to indicate an error due to this noise. So, if you get a burst of noise on the input then you will see red flashes for around half a second on the LEDs. To avoid this happening all the time we have a bias resistor network on the input of the LVDS receivers that provides a small bias current. This filters out most of the noise, but occasionally depending on the environment there may still be enough noise to trigger the reset to ready cycle and you see this as a red flash on the LEDs. This is the expected/normal operation. If you have any concerns, please contact support@star-dundee.com.

3.1.4 Functions

3.1.4.1 SpaceWire Router

The SpaceWire router inside this unit is compatible with the AT7910E radiation tolerant router (also known as the SpW-10X) manufactured by Atmel. The SpaceWire interfaces operate at a maximum rate of 200 Mbit/s.

The SpaceWire router supports two modes of operation, interface mode and routing mode.

The default mode is interface mode. The router will revert to interface mode after a device reset is performed.

The SpaceWire router has 9 ports in total as listed below:

- 1 internal router configuration port
- 3 external SpaceWire ports
- 4 PCI interface external ports which are connected to the three SpaceWire ports and the configuration port
- 1 Packet generator/checker port

3.1.4.2 Interface Mode

In interface mode, a packet which is received on an external port, from a SpaceWire link or from the configuration port, will always be routed to the port specified by the Port Routing Register of the port. The SpaceWire PCI Mk2 card supports interface mode by utilising the port routing registers of the SpaceWire router. The default values are listed in the image and table below.

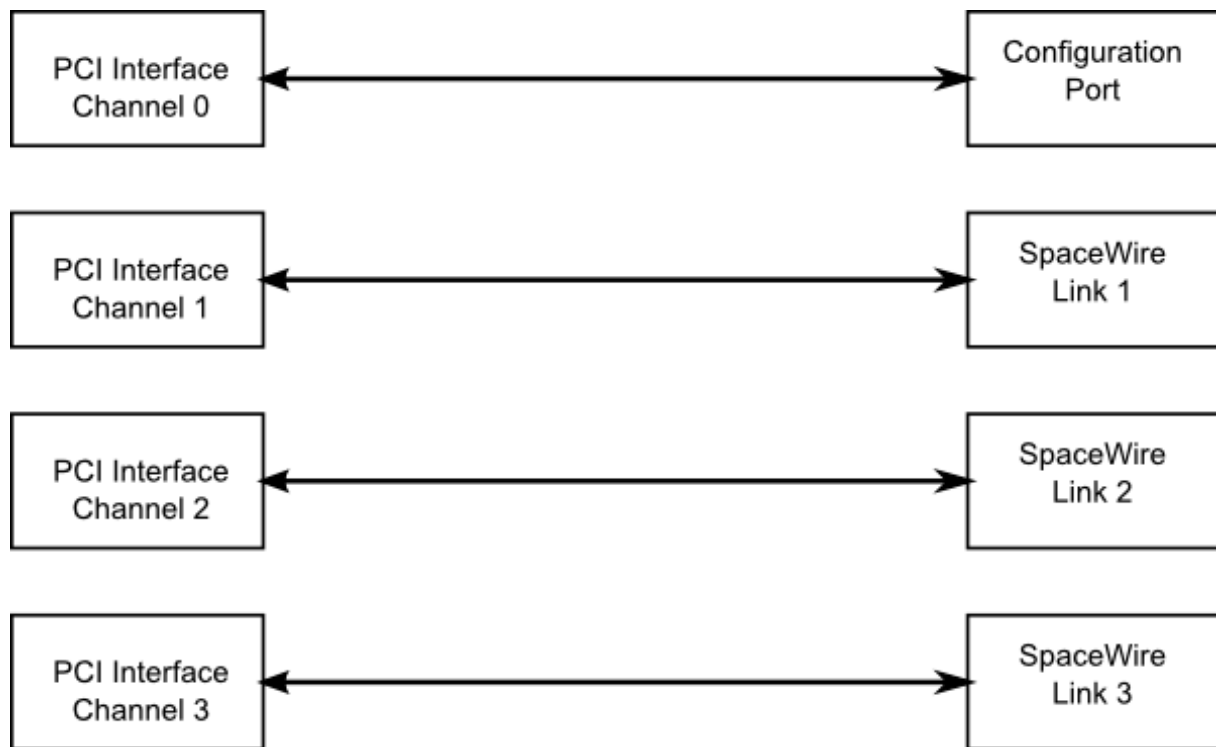


Figure 3.3: Interface mode routing connections

Source port	Destination port	Description
0	4	Configuration to external port 1 (host channel 0)
1	5	SpaceWire link 1 to external port 2 (host channel 1)
2	6	SpaceWire link 2 to external port 3 (host channel 2)
3	7	SpaceWire link 3 to external port 4 (host channel 3)
4	0	PCI interface channel 0 to configuration port
5	1	PCI interface channel 1 to SpaceWire link 1
6	2	PCI interface channel 2 to SpaceWire link 2
7	3	PCI interface channel 3 to SpaceWire link 3
8	1	Packet generator SpaceWire link 1

3.1.4.3 Routing Mode

In routing mode packets are routed dependent on the first byte of each SpaceWire packet. The routing algorithm is described below.

- Packets with known physical addresses are routed to the port indicated by the physical address at the head of the packet, i.e. a packet with address 0 is routed to the configuration port.
- Packets with unknown physical addresses, 9 to 31, are discarded and an address error is set in the source

port register.

- Packets with logical address headers which are valid are routed to one of the set ports in the logical address lookup table (accessed via the configuration port).
- Packets with logical address headers which are marked as invalid are discarded and an address error is set in the source port register.

3.1.4.4 Link Operating Speed

The SpaceWire link operating speed for each SpaceWire link is controlled by register settings. The clock selection logic for each link is described in the diagram below.

The default link operating speed is 200 Mbit/s after link start-up and 10 Mbit/s during link initialisation.

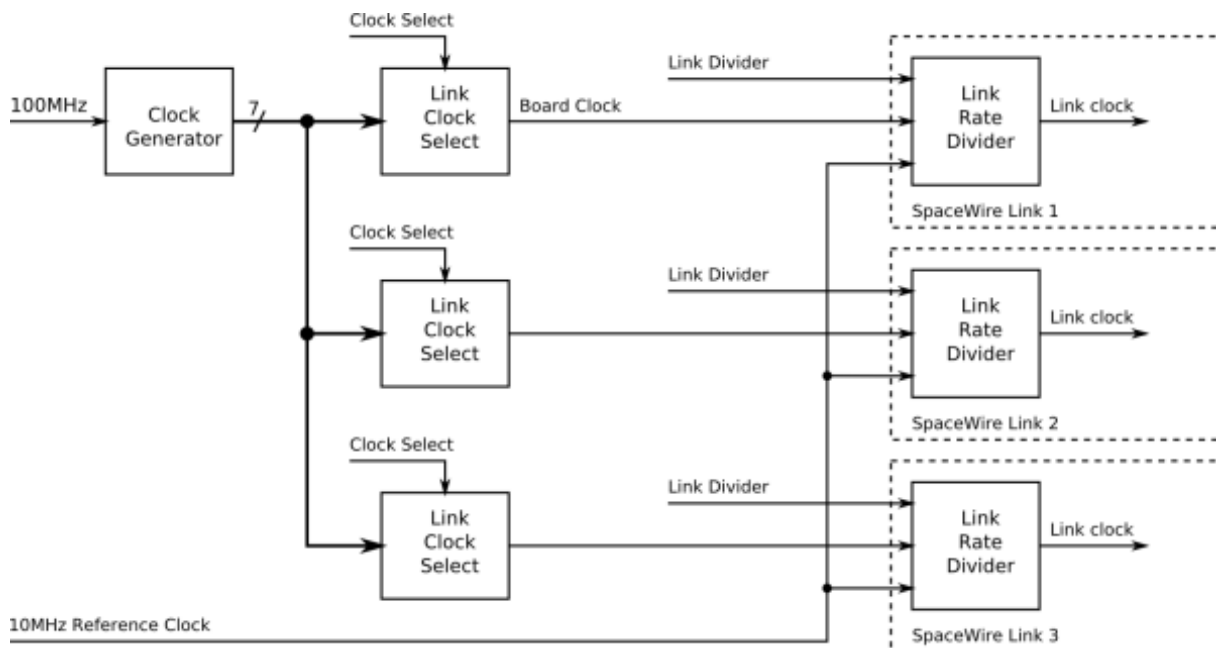


Figure 3.4: Link operating speed circuit diagram

The link operating speed is set using the following variables:

- Clock Generator:
 - Generates a set of clock frequencies for each link.
- Link Clock Select (one for each SpaceWire link):
 - 120, 128, 140, 150, 160, 180, 200 MHz
- Link Rate Divider (one for each SpaceWire link):
 - Divide by an even integer value.

The clock generator generates seven clock frequencies which can be individually selected by the link clock select modules.

Each link clock select module chooses between the set of clock generator frequencies using a link clock select register which is accessible through the SpaceWire router configuration port.

The link rate can be programmed using the functions provided by the PCI Mk2 Configuration API and the Mk2 Device and Interface Configuration API. Please refer to the API documentation for further details on how to set this clock speed.

3.1.4.5 Time-codes

A time-code can be received from the PCI interface or from a SpaceWire link. When a time-code is received, its value is checked against the value of the internal time-code register in the SpaceWire router. If the received time-code is one more than the value of the time-code register then the time-code will be written to the time-code register and propagated to other ports, except the port that was the source of the time-code.

Time-codes are only propagated on ports which are enabled to send time-codes as defined by the time-code control register. Note that of the external ports, only port 4, corresponding to channel 0, can currently be used to send or receive time-codes.

The router has an internal time-code generation master, which can be controlled by the SpaceWire router configuration port to generate time-codes at a user defined rate.

An overview of the SpaceWire router time-code interface is shown below.

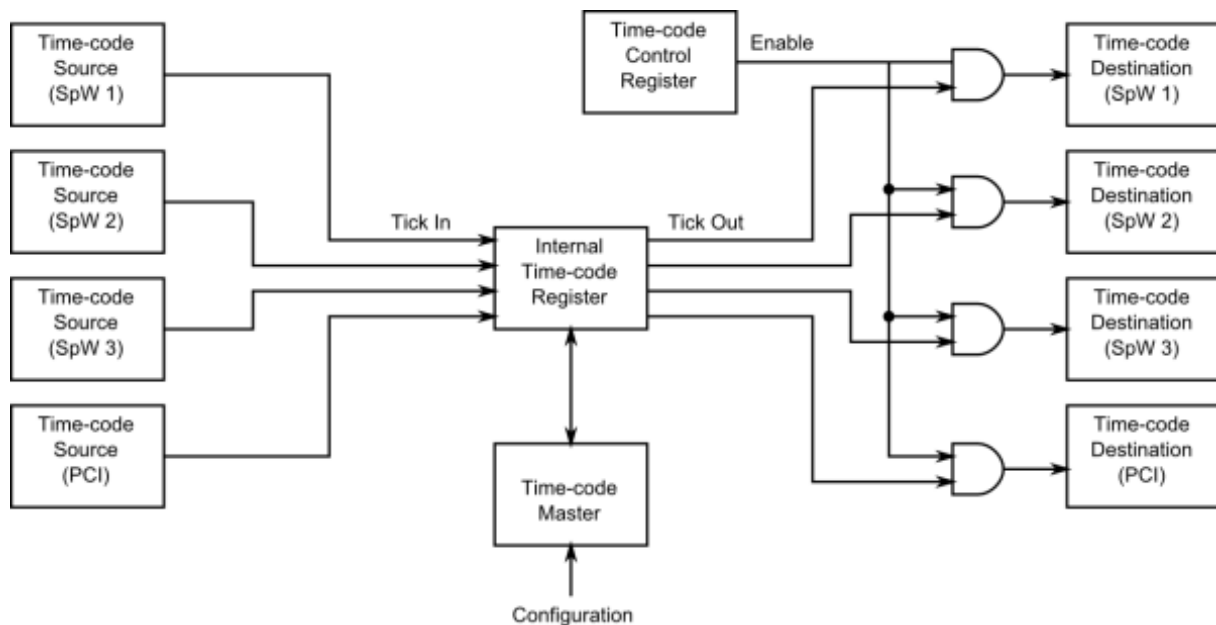


Figure 3.5: Time-code processing architecture

3.1.5 Electrical Characteristics

The SpaceWire interface absolute maximum input ratings are defined in the table below.

	Description	V _{MIN}	V _{MAX}
V _{IN}	Input voltage relative to GND	-0.5	3.8

The SpaceWire connector LVDS DC characteristics are listed in the table below.

	V _{ID} mV, Min	mV, Max	V _{ICM} V, Min	V, Max	V _{OD} mV, Min	mV, Max	V _{OCM} V, Min	V, Max
Din, Sin	100	-	0.2	2.8				

Dout, Sout					250	400	1.125	1.375
---------------	--	--	--	--	-----	-----	-------	-------

V_{ID}	Input differential voltage
V_{ICM}	Input common mode voltage
V_{OD}	Output differential voltage
V_{OCM}	Output common mode voltage

The common mode and differential identifiers are described in the diagram below.

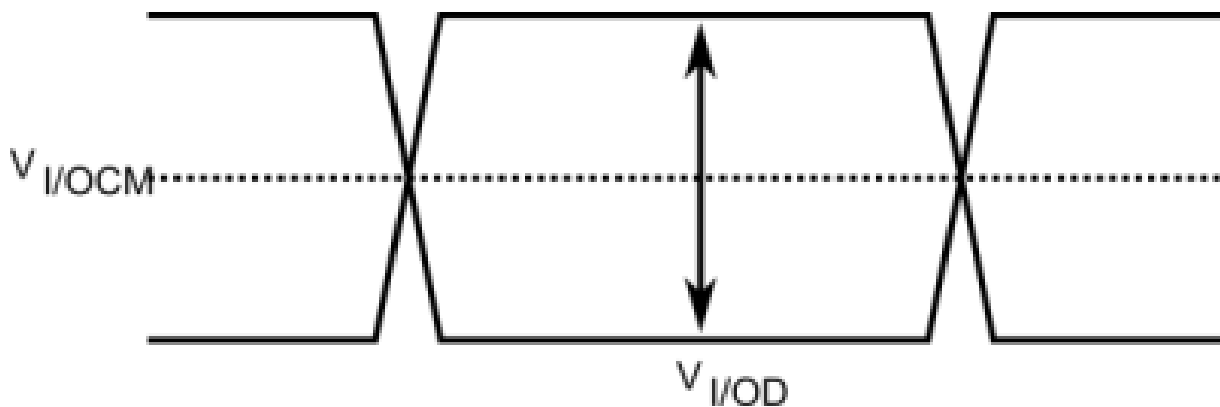


Figure 3.6: SpaceWire LVDS electrical specifications

3.1.6 Switching Characteristics

The SpaceWire interface data/strobe input switching characteristics are defined in the figure and table below.

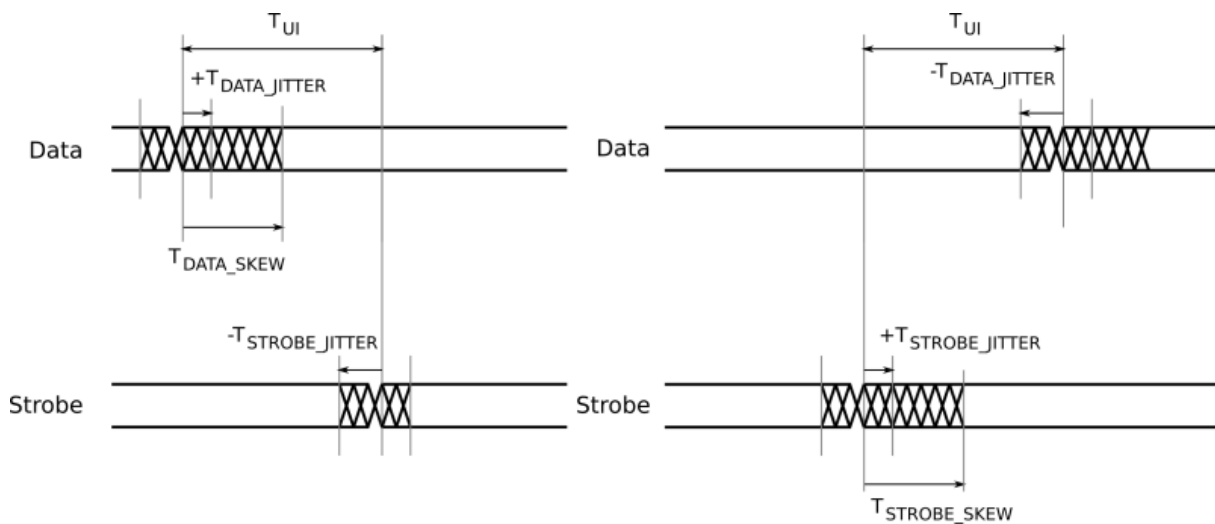


Figure 3.7: SpaceWire input switching characteristics

	Description	ns (Max)
T_{UI}	Transmitter bit rate	5
$T_{DATA_SKEW_IN} (MAX)$	Maximum data skew which can be tolerated by the receiver.	$3 - T_{DATA_JITTER_IN} - T_{STROBE_JITTER_IN}$
$T_{STROBE_SKEW_IN} (MAX)$	Maximum strobe skew which can be tolerated by the receiver.	$3 - T_{DATA_JITTER_IN} - T_{STROBE_JITTER_IN}$

The data/strobe output switching characteristics are defined in the table below.

	Description	ns (Max)
T_{SKEW_OUT}	Maximum data or strobe skew	0
T_{JITTER_OUT}	Maximum data/strobe jitter	0.7 (Peak to Peak)

3.1.7 Grounding Information

When using LVDS it is very important that the grounds of the equipment at the two ends of each SpaceWire link are connected. Otherwise a significant common mode voltage can exist between the two units resulting in the maximum input voltage at one end or the other being exceeded. This could then result in damage to either unit.

The SpaceWire outer shield is connected to the backshell of the connectors at either end of a SpaceWire cable and when screwed into the mating SpaceWire connector on a unit ought to make a connection to the chassis and ground plane of that unit. Properly connecting a SpaceWire cable would then suffice to make the required ground connection between the two units being connected by SpaceWire.

STAR-Dundee equipment always has a low impedance connection between the backshell of the connector, the chassis and ground plane of the electronics in the unit. STAR-Dundee cables have the outer shield connected to the backshell of the connectors at either end of the cable.

When testing a new SpaceWire unit, it is recommended that you check that there is a low impedance connection between the backshell of the connector on your unit under test (UUT) and the chassis and ground plane in the UUT. If this is not low impedance then that problem should be resolved before connecting any SpaceWire equipment to the UUT. If there is a low impedance connection then please check the cable you are using and that there is a low impedance connection between the backshells of the connector at either end of the cable.

3.2 SpaceWire PCIe

Documentation for the STAR-Dundee SpaceWire PCIe device is provided in this section.

3.2.1 Overview

3.2.1.1 Summary

This document provides an overview of the interfaces and features of the SpaceWire PCIe hardware.

For detailed information on using the SpaceWire PCIe card please consult the API reference documentation.

3.2.1.2 Hardware Description

The SpaceWire PCIe provides a PCIe to SpaceWire interface device with three SpaceWire interfaces. Efficient host software support is provided to support rapid sending and receiving of SpaceWire packets into host PC memory.

A block diagram of the SpaceWire PCIe board is shown below.

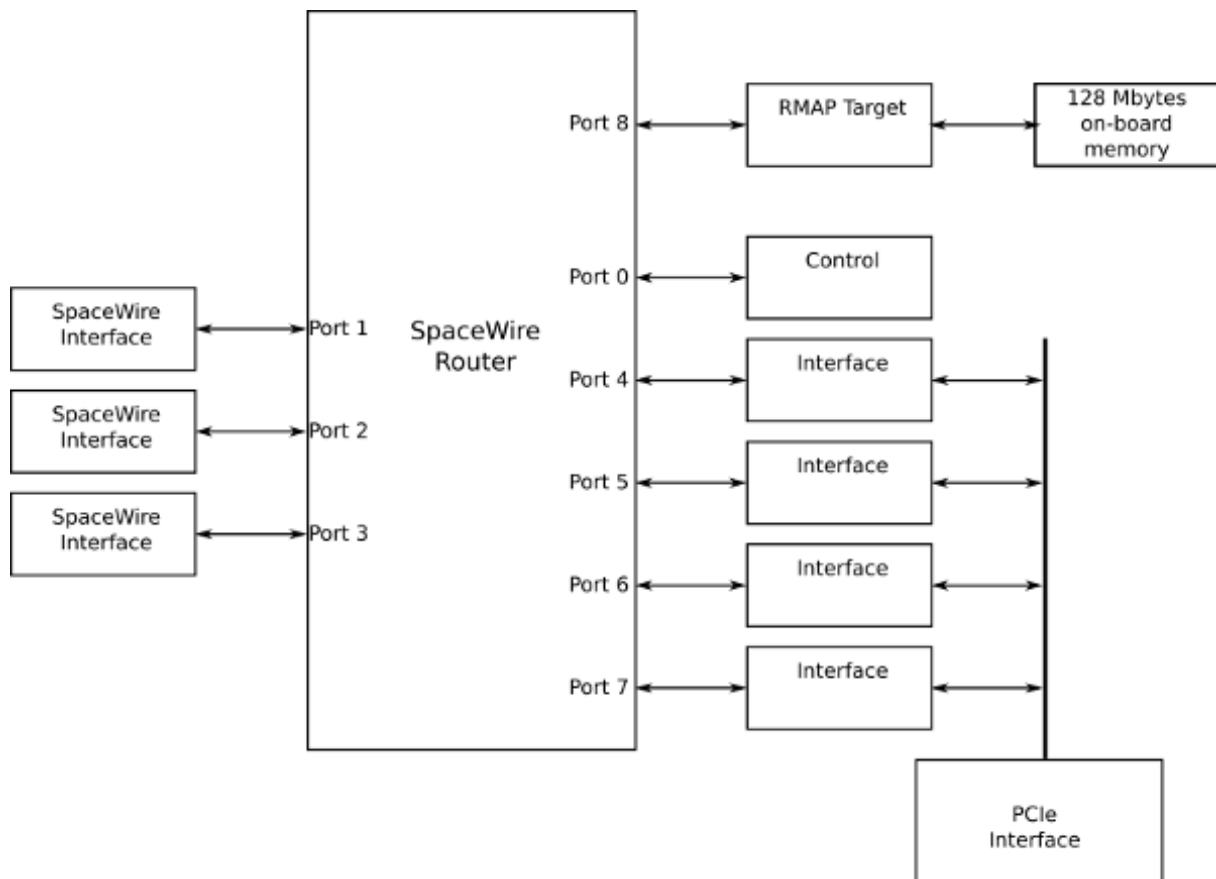


Figure 3.8: SpaceWire PCIe hardware overview

The three SpaceWire interfaces of the SpaceWire PCIe are fully compliant with the SpaceWire standard and operate at up to 300 Mbit/s. They are connected to a SpaceWire router which supports two modes of operation, interface mode and router mode. In router mode, packets from any SpaceWire port can be routed to any another SpaceWire port, or the PCIe interface. In interface mode, the SpaceWire router is passive and packets which are received on a SpaceWire link are passed directly to the PCIe channel for that link. There are three independent channels from the SpaceWire router to the PCIe interface, so traffic flowing over one port cannot block traffic for another port. In addition, there is a separate control channel, so that the host PC is always able to access the control, configuration and status space of the SpaceWire PCIe regardless of the data flow.

The single lane PCIe interface is revision 1.1 compliant. It contains a DMA controller for rapid transfer of data to and from the SpaceWire PCIe board.

The SpaceWire PCIe includes support for fault injection including support for parity errors, credit errors, escape errors, data corruption and EEP termination of packets.

3.2.2 Getting Started

Note: Before inserting the SpaceWire PCIe card, the STAR-System software should first be installed on the host computer operating system. This ensures that the SpaceWire PCIe card is recognised by the host when it boots up.

The SpaceWire PCIe card fits into any standard PCIe slot on a typical desktop PC. It can fit into x1, x4 and x16 slots which are compliant to PCIe revision 1.1, 2 or 3. The card is powered entirely through the PCIe bus; no extra power supplies from the PC's ATX power supply are required.

3.2.3 Interfaces

3.2.3.1 SpaceWire Ports

The SpaceWire port at the bottom of the board (i.e. closest to the PCIe bus, or motherboard), is SpaceWire port 3 and the furthest away is port 1. The port numbering is shown below.

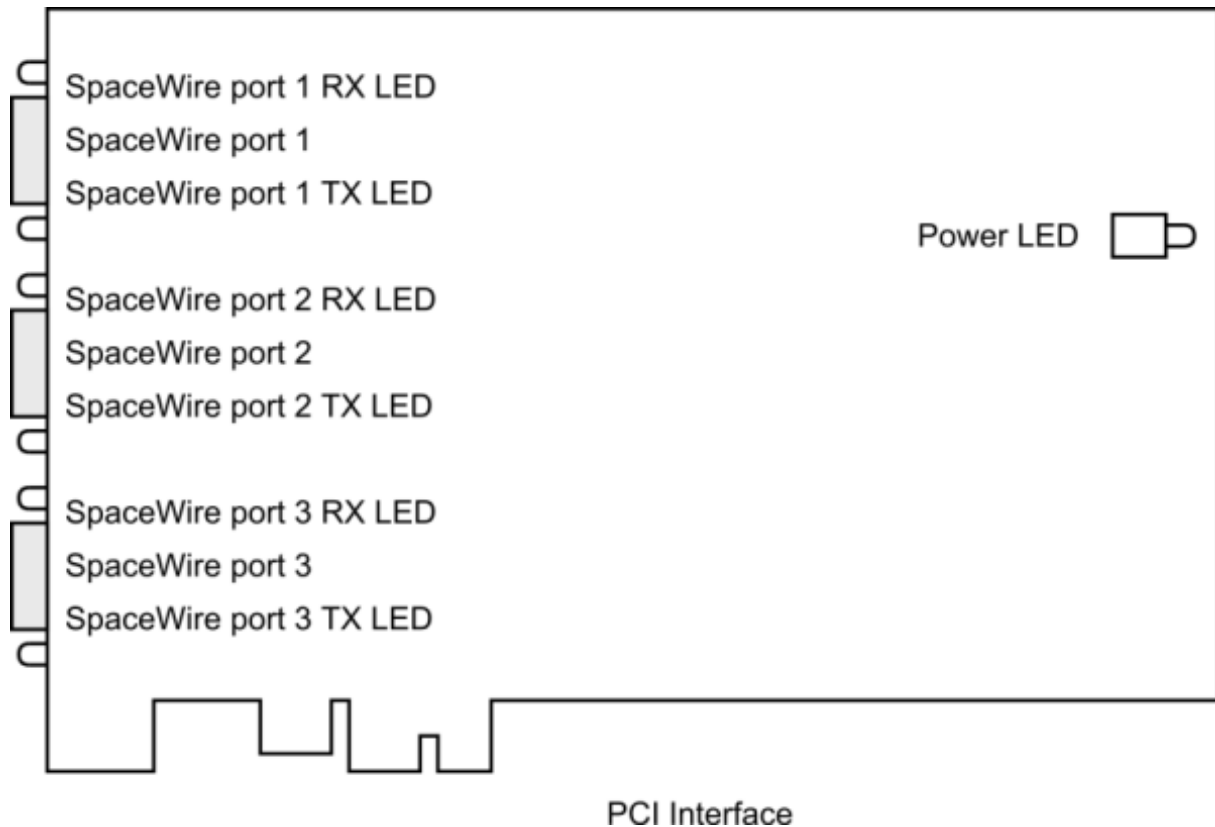


Figure 3.9: SpaceWire ports and LED positions

3.2.3.2 Power LED

When the PCIe card is powered up, the Power LED should glow a steady white. Should this LED glow, or flash any other colour, or fail to illuminate when power is applied to the board, then a hardware problem has occurred. Under these circumstances, the board must immediately be taken out of use and STAR-Dundee should be contacted for further instruction.

3.2.3.3 Front Panel LEDs

The PCIe card has six front panel tri-colour LEDs, two for each SpaceWire port. When viewed from the rear of the PC, the LED to the left of the link shows Received data, the LED to the right of the link shows Transmitted data.

The functions of the SpaceWire port LEDs are defined in the table below.

Function	LED Colour	Description
No event	Amber	LED is steady amber.
Link running	Green	The LED is turned green when the link is running. The link running LED is held for approximately half a second after the link has stopped running and no error occurs.
Error	Red	The LED is turned red.
Data Transmitted/Received	Blue	When data is received on the link, the left hand LED turns blue. When data is transmitted on the link, the right hand LED turns blue.
Identify	Flashing purple	The SpaceWire PCIe card can be forced to flash its front panel LEDs to allow the device to be easily identified among a group of other devices. When this function is used the LEDs will flash purple for a short period of time.

Please note that there is an issue with all SpaceWire interfaces where a small amount of noise on the inputs can be translated by the LVDS receivers into a digital pattern and then interpreted by the receiver as a bit sequence. Since this bit sequence is invalid it causes the SpaceWire CODEC to start up and then disconnect. The SpaceWire interface LEDs go red on disconnect (ErrorReset state) and amber when ready (Ready state). So, if you get a burst of noise on the input then you will see a red flash of around half a second on the LEDs. To avoid this happening all the time we have a bias resistor network on the input of the LVDS receivers that provides a small bias current. This filters out most of the noise, but occasionally depending on the environment there may still be enough noise to trigger the reset to ready cycle and you see this as a red flash on the LEDs. This is the expected/normal operation. If you have any concerns, please contact support@star-dundee.com.

3.2.4 Functions

3.2.4.1 SpaceWire Router

The SpaceWire router inside this unit is compatible with the AT7910E radiation tolerant router (also known as the SpW-10X) manufactured by Atmel. The SpaceWire interfaces operate at a maximum rate of 300 Mbit/s.

The SpaceWire router supports two modes of operation, interface mode and routing mode.

The default mode is interface mode. The router will revert to interface mode after a device reset is performed.

The SpaceWire router has 9 ports in total as listed below:

- 1 internal router configuration port
- 3 external SpaceWire ports
- 4 PCIe interface external ports which are connected to the three SpaceWire ports and the configuration port
- RMAP Target port

3.2.4.2 Interface Mode

In interface mode, a packet which is received on an external port, from a SpaceWire link or from the configuration port, will always be routed to the port specified by the Port Routing Register of the port. The SpaceWire PCIe card supports interface mode by utilising the port routing registers of the SpaceWire router. The default values are listed in the image and table below.

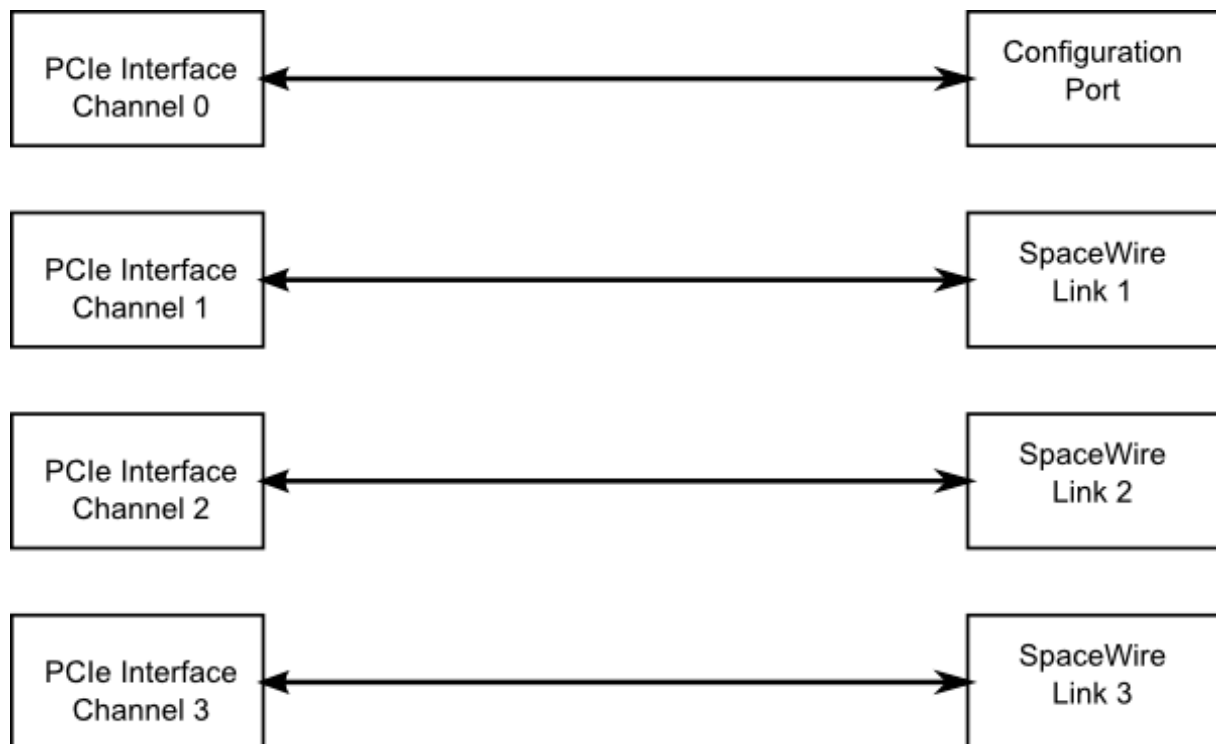


Figure 3.10: Interface mode routing connections

Source port	Destination port	Description
0	4	Configuration to external port 1 (host channel 0)
1	5	SpaceWire link 1 to external port 2 (host channel 1)
2	6	SpaceWire link 2 to external port 3 (host channel 2)
3	7	SpaceWire link 3 to external port 4 (host channel 3)
4	0	PCIe interface channel 0 to configuration port
5	1	PCIe interface channel 1 to SpaceWire link 1
6	2	PCIe interface channel 2 to SpaceWire link 2
7	3	PCIe interface channel 3 to SpaceWire link 3

3.2.4.3 Routing Mode

In routing mode packets are routed dependent on the first byte of each SpaceWire packet. The routing algorithm is described below.

- Packets with known physical addresses are routed to the port indicated by the physical address at the head of the packet, i.e. a packet with address 0 is routed to the configuration port.
- Packets with unknown physical addresses, 9 to 31, are discarded and an address error is set in the source port register.

- Packets with logical address headers which are valid are routed to one of the set ports in the logical address lookup table (accessed via the configuration port).
- Packets with logical address headers which are marked as invalid are discarded and an address error is set in the source port register.

3.2.4.4 Link Operating Speed

The link operating speed for each SpaceWire port is controlled by configuration parameters which can be set by the API or GUI application. The link clock rate selection logic for each port is shown in the figure below.

The default link operating speed is 200 Mbit/s after link start-up and 10 Mbit/s during link initialisation.

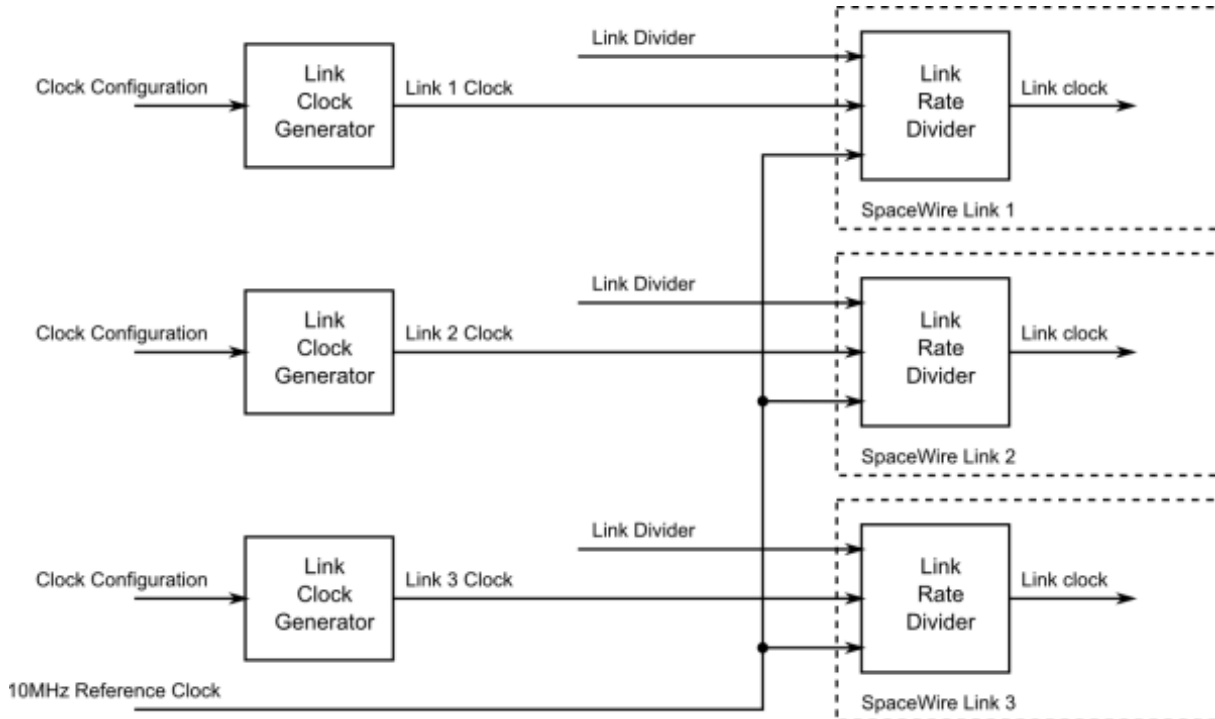


Figure 3.11: Link operating speed circuit diagram

The link operating speed for each SpaceWire link is set using a programmable clock generator frequency. This is set by configuring a multiply and divide value to the input 100 MHz clock.

The link rate can be programmed using the functions provided by the Mk2 Device and Interface Configuration API. Please refer to the API documentation for further details on how to set this clock speed.

3.2.4.5 Time-codes

A time-code can be received from the PCI interface or from a SpaceWire link. When a time-code is received, its value is checked against the value of the internal time-code register in the SpaceWire router. If the received time-code is one more than the value of the time-code register then the time-code will be written to the time-code register and propagated to other ports, except the port that was the source of the time-code.

Time-codes are only propagated on ports which are enabled to send time-codes as defined by the time-code control register. Note that of the external ports, only port 4, corresponding to channel 0, can currently be used to send or receive time-codes.

The router has an internal time-code generation master, which can be controlled by the SpaceWire router configuration port to generate time-codes at a user defined rate.

An overview of the SpaceWire router time-code interface is shown below.

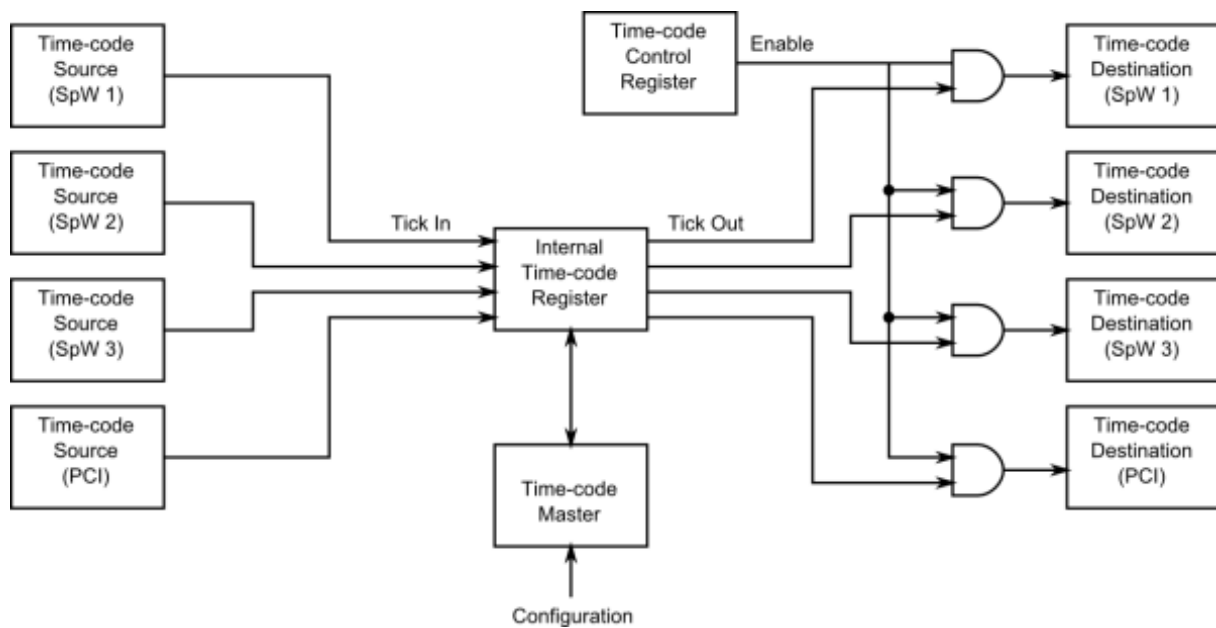


Figure 3.12: Time-code processing architecture

3.2.4.6 RMAP Target

The SpaceWire PCIe card includes an RMAP target port for rapid prototyping of a remote RMAP memory space. The port has access to 128 Mbytes of DDR3 memory. The port is accessible through the SpaceWire router allowing access from the PCIe interface or from the SpaceWire interfaces.

Supported authorisation parameters:

Target logical address: 0x00 - 0xFF	0xFE is the default target logical address
Protocol ID: 0x01	0x01 is the protocol ID for RMAP
Instruction:	The RMAP command type
Incrementing Read (command code 0b0011)	
Incrementing Write (command code 0b1001)	
Incrementing Write with reply (command code 0b1011)	
Read Modify Write (command code 0b0111)	
Key: 0x00 - 0xFF	The key allows the target application to filter commands
Initiator logical address: 0x00 - 0xFF	The logical address a reply should be sent to
Extended address: 0x02	To access the appropriate target address space
Data length: 0x0000_0000 - 0x00FF_FFFC	
Addressable range: 0x0000_0000 - 0xFFFF_FFFF	
Usable address range: 0x0000_0000 - 0x01FF_FFFF (addressed as 32 bit values)	

The bottom two bits of the data length field must be zero. Otherwise, the command will not be authorised.

All accesses are 32 bit aligned and the address in the RMAP command is interpreted as a word aligned address, i.e. address 0 and address 1 are separated by 4 bytes.

The external DDR3 memory range is 0x0000_0000 - 0x01FF_FFFF (128 Mbytes / 4)

The target does not support address range checking and will authorise access to any 32 bit address.

The extended address should be set to 0x02 for access to the 128 Mbyte external memory block.

3.2.5 Electrical Characteristics

The SpaceWire interface absolute maximum input ratings are defined in the table below.

	Description	V _{MIN}	V _{MAX}
V _{IN}	Input voltage relative to GND	-0.5	3.8

The SpaceWire connector LVDS DC characteristics are listed in the table below.

	V _{ID} mV, Min	mV, Max	V _{ICM} V, Min	V, Max	V _{OD} mV, Min	mV, Max	V _{OCM} V, Min	V, Max
Din, Sin	100	-	0.2	2.8				
Dout, Sout					250	400	1.125	1.375

V _{ID}	Input differential voltage
V _{ICM}	Input common mode voltage
V _{OD}	Output differential voltage
V _{OCM}	Output common mode voltage

The common mode and differential identifiers are described in the diagram below.

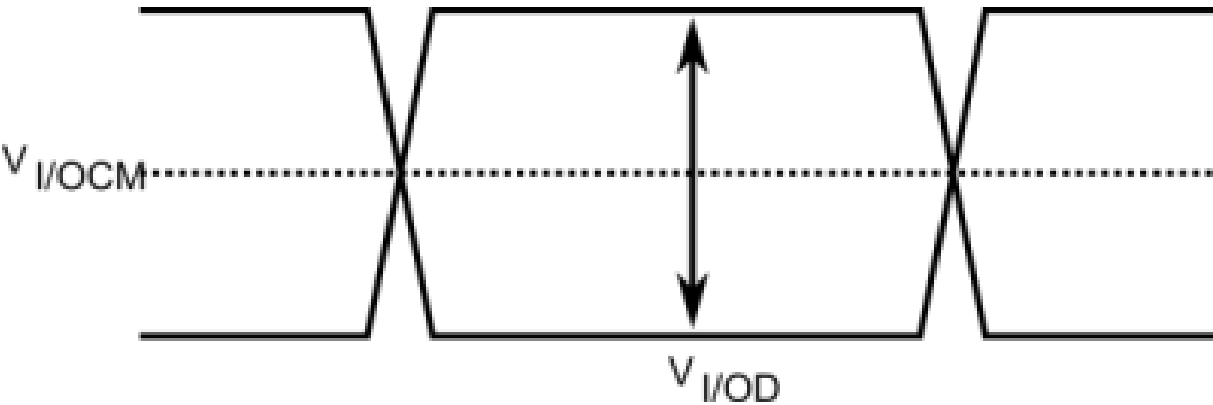


Figure 3.13: SpaceWire LVDS electrical specifications

3.2.6 Switching Characteristics

The SpaceWire interface data/strobe input switching characteristics are defined in the figure and table below.

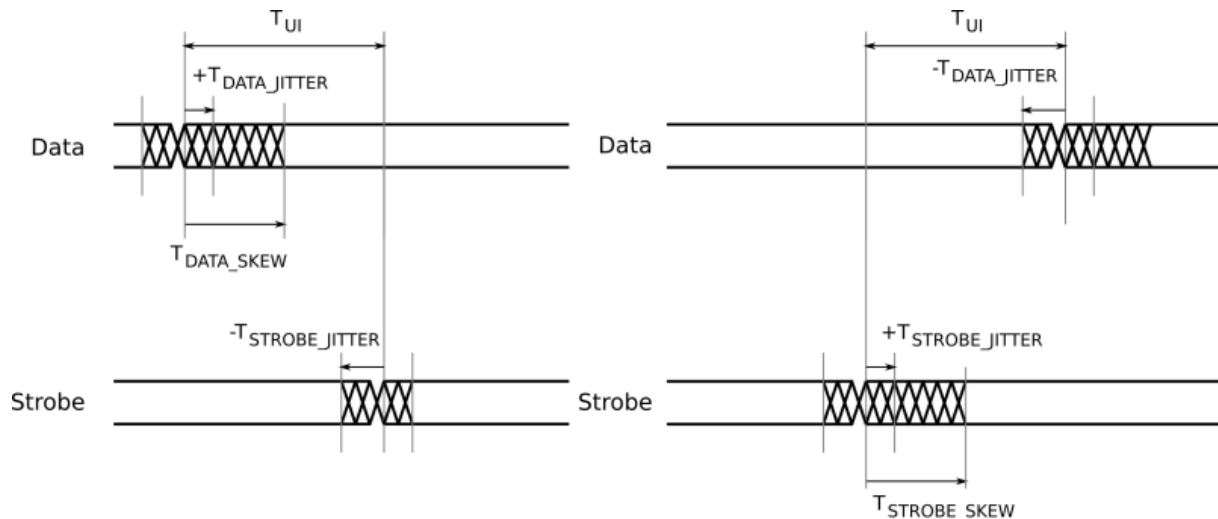


Figure 3.14: SpaceWire input switching characteristics

	Description	ns (Max)
T_{UI}	Transmitter bit rate	5
$T_{DATA_SKEW_IN} (MAX)$	Maximum data skew which can be tolerated by the receiver.	$3 - T_{DATA_JITTER_IN} - T_{STROBE_JITTER_IN}$
$T_{STROBE_SKEW_IN} (MAX)$	Maximum strobe skew which can be tolerated by the receiver.	$3 - T_{DATA_JITTER_IN} - T_{STROBE_JITTER_IN}$

The data/strobe output switching characteristics are defined in the table below.

	Description	ns (Max)
T_{SKEW_OUT}	Maximum data or strobe skew	0
T_{JITTER_OUT}	Maximum data/strobe jitter	0.7 (Peak to Peak)

3.2.7 Grounding Information

When using LVDS it is very important that the grounds of the equipment at the two ends of each SpaceWire link are connected. Otherwise a significant common mode voltage can exist between the two units resulting in the maximum input voltage at one end or the other being exceeded. This could then result in damage to either unit.

The SpaceWire outer shield is connected to the backshell of the connectors at either end of a SpaceWire cable and when screwed into the mating SpaceWire connector on a unit ought to make a connection to the chassis and ground plane of that unit. Properly connecting a SpaceWire cable would then suffice to make the required ground connection between the two units being connected by SpaceWire.

STAR-Dundee equipment always has a low impedance connection between the backshell of the connector, the chassis and ground plane of the electronics in the unit. STAR-Dundee cables have the outer shield connected to the backshell of the connectors at either end of the cable.

When testing a new SpaceWire unit, it is recommended that you check that there is a low impedance connection between the backshell of the connector on your unit under test (UUT) and the chassis and ground plane in the UUT. If this is not low impedance then that problem should be resolved before connecting any SpaceWire equipment to the UUT. If there is a low impedance connection then please check the cable you are using and that there is a low impedance connection between the backshells of the connector at either end of the cable.

3.3 SpaceWire Brick Mk2

Documentation for the STAR-Dundee SpaceWire Brick Mk2 device is provided in this section.

3.3.1 Overview

3.3.1.1 Summary

This document provides an overview of the interfaces and features of the SpaceWire Brick Mk2 hardware.

For detailed information on using the SpaceWire Brick Mk2 device please consult the API reference documentation.

3.3.1.2 Hardware Description

The SpaceWire Brick Mk2 provides a USB to SpaceWire interface device with two SpaceWire interfaces. Efficient host software is provided to support rapid sending and receiving of SpaceWire packets into host PC memory.

A block diagram of the SpaceWire Brick Mk2 is shown below.

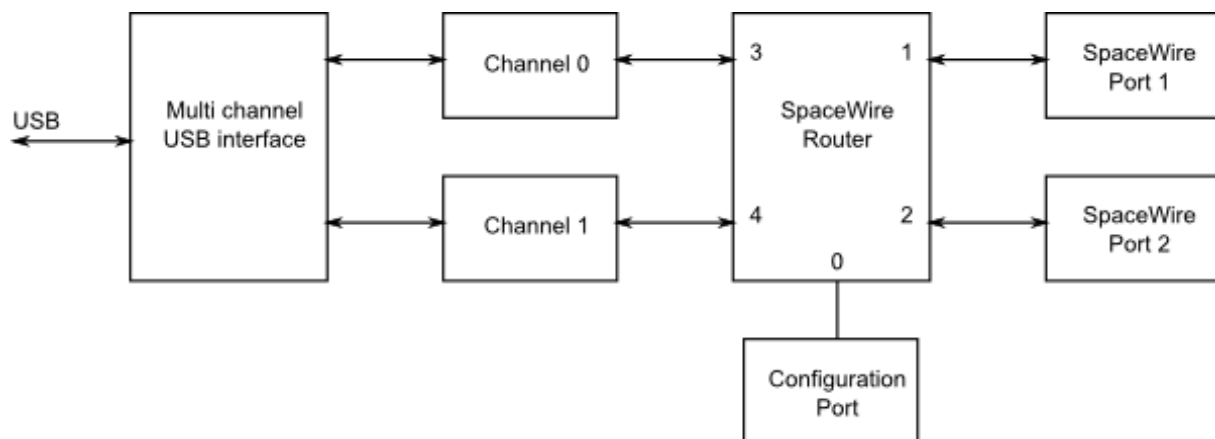


Figure 3.15: SpaceWire Brick Mk2 hardware overview

The two SpaceWire interfaces of the SpaceWire Brick Mk2 are fully compliant with the SpaceWire standard and operate at up to 200 Mbit/s. They are connected to a SpaceWire router which supports two modes of operation, interface mode and router mode. In router mode, packets from any SpaceWire port can be routed to any other SpaceWire port, or the USB interface. In interface mode, the SpaceWire router is passive and packets which are received on a SpaceWire link are passed directly to the USB data channel. The behaviour when transmitting is described in the [Interface Mode](#) section. There is only one data channel which is shared between the two ports, so traffic flowing over one port may block traffic on the other port. There is also a separate control channel used to ensure that the host PC is always able to access the control, configuration and status space of the Brick Mk2 regardless of the data flow.

The USB interface is compliant to USB version 2.0, and can also be used in USB 1.1 and USB 3.0 ports.

The SpaceWire Brick Mk2 includes support for fault injection including support for parity errors, credit errors, escape errors, data corruption and EEP termination of packets.

3.3.2 Getting Started

The SpaceWire Brick Mk2 is powered from the USB cable; therefore no external power supply is required. The USB cable is inserted in the USB socket located at the rear of the Brick Mk2.

When plugged into a host PC or laptop the Brick Mk2 performs a power negotiation stage with the host PC before powering up the router. The LEDs at the front of the Brick Mk2 indicate whether the host has accepted the request from the SpaceWire Brick Mk2 to draw power from the USB bus. When first plugged in, the LEDs remain off until power has been granted to the device. Once the host has accepted the power request from the brick then the LEDs will turn to red for approximately 5 seconds while the brick disconnects and reconnects from the USB bus as a fully powered device.

After this phase the LEDs will default to their normal operating state e.g. with no cables connected all four LEDs are amber.

The SpaceWire Brick Mk2 software and drivers, provided by STAR-System, should be installed on the host computer before connecting the Brick Mk2. This ensures that the Brick Mk2 is recognised by the host when it is connected.

3.3.3 Interfaces

3.3.3.1 SpaceWire Ports and LEDs

The SpaceWire port lowest down is SpaceWire router link 1 and the highest is router link 2. The USB port is router links 3 and 4. The port numbering is shown below.

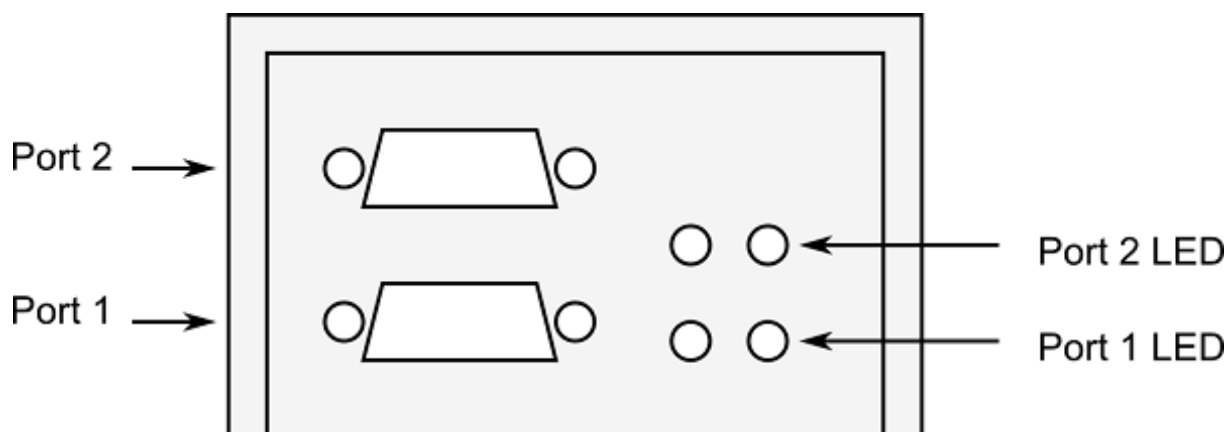


Figure 3.16: SpaceWire ports and LED positions

The Brick Mk2 has four tri-colour LEDs, two for each SpW port. The LED on the left next to each link shows received data for that link, the LED on the right next to each link shows transmitted data for that link. The functions of the SpaceWire port LEDs are defined in the table below.

Function	LED Colour	Description
No event	Amber	The left and right hand LEDs are steady amber.
Link running	Green	The left and right hand LEDs are turned green when the link is running. The link running LEDs are held for approximately half a second after the link has stopped running and no error occurs.
Error	Red	The left and right hand LEDs are turned red.
Data Transmitted/Received	Blue	When data is received on the link, the left hand LED turns blue. When data is transmitted on the link, the right hand LED turns blue.
Identify	Flashing purple	The SpaceWire Brick Mk2 can be forced to flash its front panel LEDs to allow the device to be easily identified among a group of other devices. When this function is used the LEDs will flash purple for a short period of time.

Please note that there is an issue with all SpaceWire interfaces where a small amount of noise on the inputs can be translated by the LVDS receivers into a digital pattern and then interpreted by the receiver as a bit sequence. Since this bit sequence is invalid it causes the SpaceWire CODEC to start up and then disconnect. The SpaceWire interface LEDs go red on disconnect (ErrorReset state) and amber when ready (Ready state). So, if you get a burst of noise on the input then you will see a red flash of around half a second on the LEDs. To avoid this happening all the time we have a bias resistor network on the input of the LVDS receivers that provides a small bias current. This filters out most of the noise, but occasionally depending on the environment there may still be enough noise to trigger the reset to ready cycle and you see this as a red flash on the LEDs. This is the expected/normal operation. If you have any concerns, please contact support@star-dundee.com.

3.3.4 Functions

3.3.4.1 SpaceWire Router

The SpaceWire router inside this unit is compatible with the AT7910E radiation tolerant router (also known as the SpW-10X) manufactured by Atmel.

The SpaceWire router supports two modes of operation, interface mode and routing mode.

The default mode is interface mode. The router will revert to interface mode after a device reset is performed.

The SpaceWire router has 5 ports in total as listed below:

- 1 internal router configuration port
- 2 external SpaceWire ports
- 2 USB interface external ports, one of which may be used by the two SpaceWire ports, while the other may be used by the configuration port

3.3.4.2 Interface Mode

In interface mode, a packet which is received on a SpaceWire link will always be routed to the port specified by the port routing register of the port. Packets received from the USB external ports or the configuration port are routed

based on the first byte, as for routing mode. The SpaceWire Brick Mk2 supports interface mode by utilising the port routing registers of the SpaceWire router. The default values are listed in the image and table below.

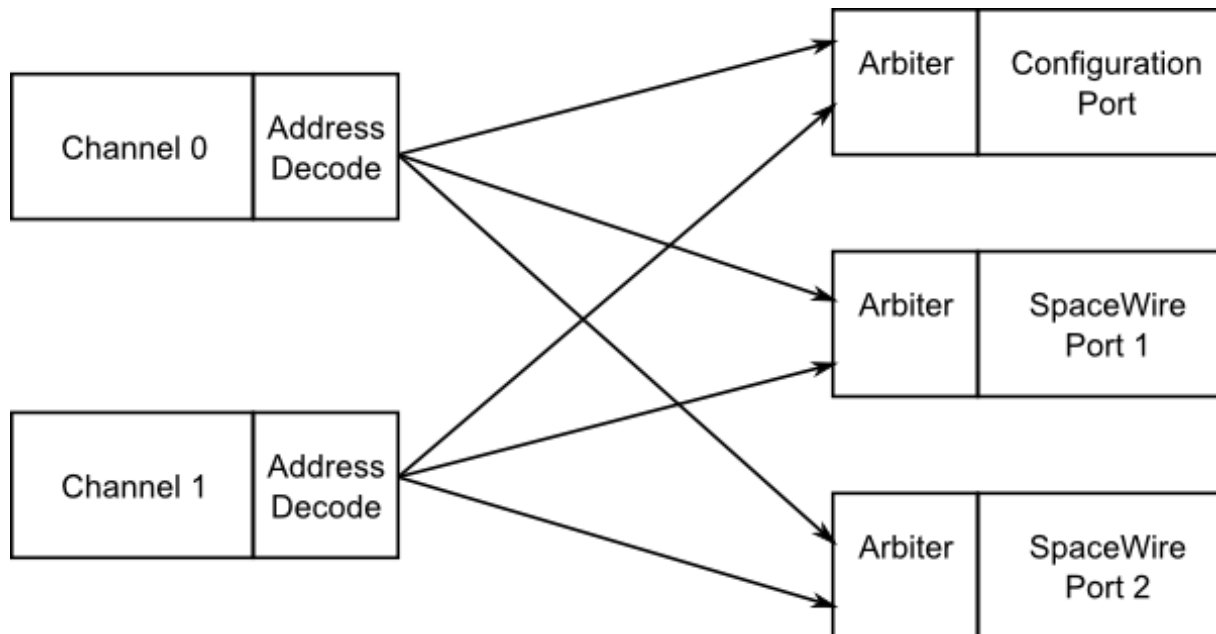


Figure 3.17: Interface mode routing connections, USB to SpaceWire

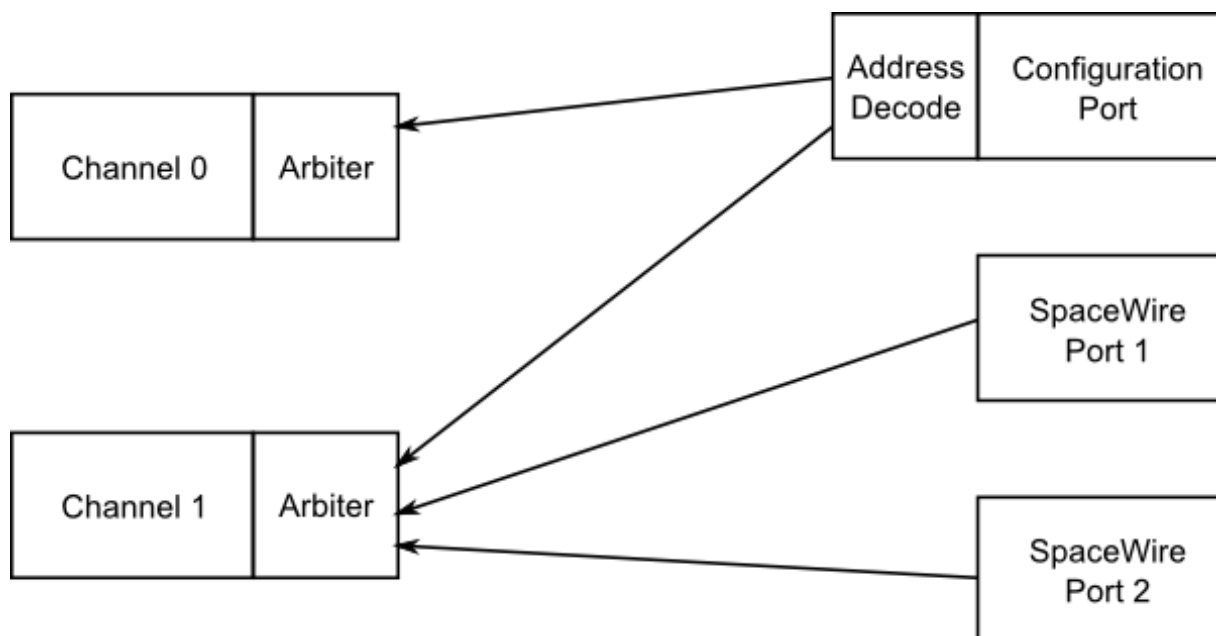


Figure 3.18: Interface mode routing connections, SpaceWire to USB

Source port	Destination port	Description
0	3 or 4	Configuration to USB interface channel (Reply packets routed based on command source)
1	4	SpaceWire link 1 to USB interface channel 1
2	4	SpaceWire link 2 to USB interface channel 1
3	0, 1 or 2	USB interface channel 0 to configuration port, SpaceWire links 1 and 2 (Packets routed based on first byte)
4	0, 1 or 2	USB interface channel 1 to configuration port, SpaceWire links 1 and 2 (Packets routed based on first byte)

3.3.4.3 Routing Mode

In routing mode packets are routed dependent on the first byte of each SpaceWire packet. The routing algorithm is described below.

- Packets with known physical addresses are routed to the port indicated by the physical address at the head of the packet, i.e. a packet with address 0 is routed to the configuration port.
- Packets with unknown physical addresses, 9 to 31, are discarded and an address error is set in the source port register.
- Packets with logical address headers which are valid are routed to one of the set ports in the logical address lookup table (accessed via the configuration port).
- Packets with logical address headers which are marked as invalid are discarded and an address error is set in the source port register.

3.3.4.4 Link Operating Speed

The SpaceWire link operating speed for each SpaceWire link is controlled by register settings. The clock selection logic for each link is described in the diagram below.

The default link operating speed is 200 Mbit/s after link start-up and 10 Mbit/s during link initialisation.

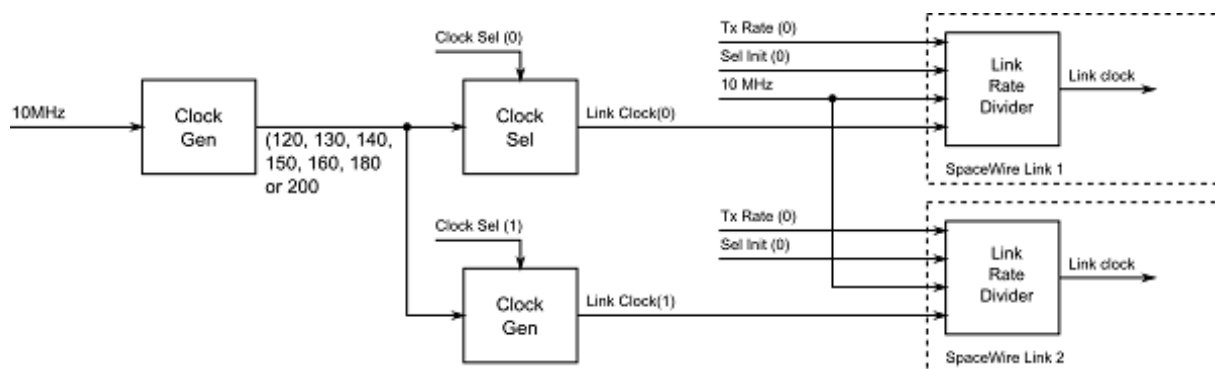


Figure 3.19: Link operating speed circuit diagram

The link operating speed for each SpaceWire link is set using the following variables:

- Clock Generator:
 - Generates a set of clock frequencies for each link.
- Link clock select (one for each SpaceWire link):
 - 120, 130, 140, 150, 160, 180, 200 MHz
- Link rate divider (one for each SpaceWire link):
 - Divide by an even integer value.

The clock generator generates seven clock frequencies which can be individually selected by the link clock select modules.

Each link clock select module chooses between the set of clock generator frequencies using a link clock select register which is accessible through the SpaceWire router configuration port.

The link rate can be programmed using the functions provided by the Mk2 Device and Interface Configuration API and the Brick Mk2 Configuration API. Please refer to the API documentation for further details on how to set this clock speed.

3.3.4.5 Time-codes

A time-code can be received from the USB interface or from a SpaceWire link. When a time-code is received, its value is checked against the value of the internal time-code register in the SpaceWire router. If the received time-code is one more than the value of the time-code register then the time-code will be written to the time-code register and propagated to other ports, except the port that was the source of the Time-code.

Time-codes are only propagated on ports which are enabled to send time-codes as defined by the time-code control register. Note that of the external USB ports, only port 3, corresponding to channel 0, can currently be used to send or receive time-codes.

The router has an internal time-code generation master, which can be controlled by the SpaceWire router configuration port to generate time-codes at a user defined rate.

An overview of the SpaceWire router time-code interface is shown below.

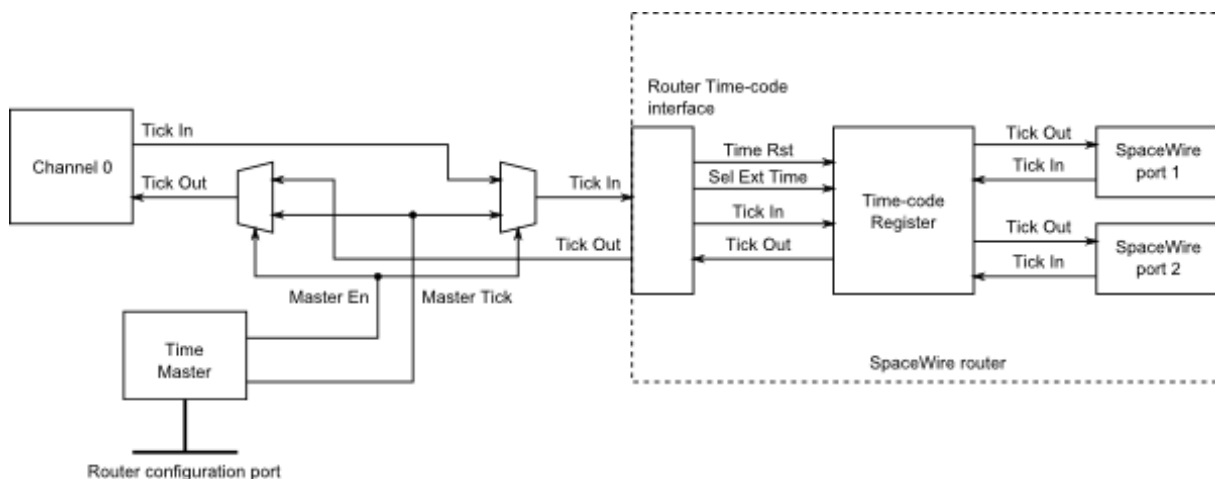


Figure 3.20: Time-code processing architecture

3.3.5 Electrical Characteristics

The SpaceWire interface absolute maximum input ratings are defined in the table below.

	Description	V_{MIN}	V_{MAX}
V_{IN}	Input voltage relative to GND	-0.5	3

The SpaceWire connector LVDS DC characteristics are listed in the table below.

	V_{ID} mV, Min	mV, Max	V_{ICM} V, Min	V, Max	V_{OD} mV, Min	mV, Max	V_{OCM} V, Min	V, Max
Din, Sin	100	-	0.2	2.1				
Dout, Sout					250	400	1.125	1.375

V_{ID}	Input differential voltage
V_{ICM}	Input common mode voltage
V_{OD}	Output differential voltage
V_{OCM}	Output common mode voltage

The common mode and differential identifiers are described in the diagram below.

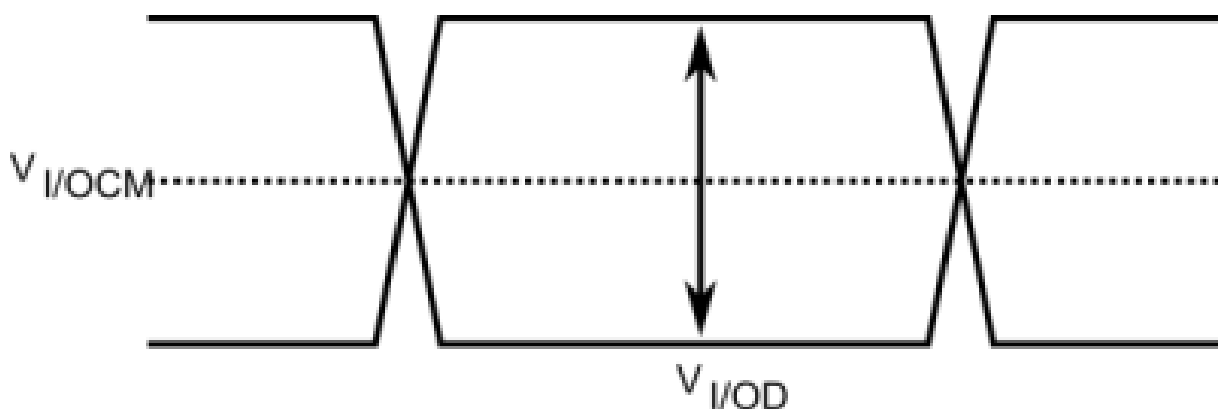


Figure 3.21: SpaceWire LVDS electrical specifications

3.3.6 Switching Characteristics

The SpaceWire interface data/strobe input switching characteristics are defined in the figure and table below.

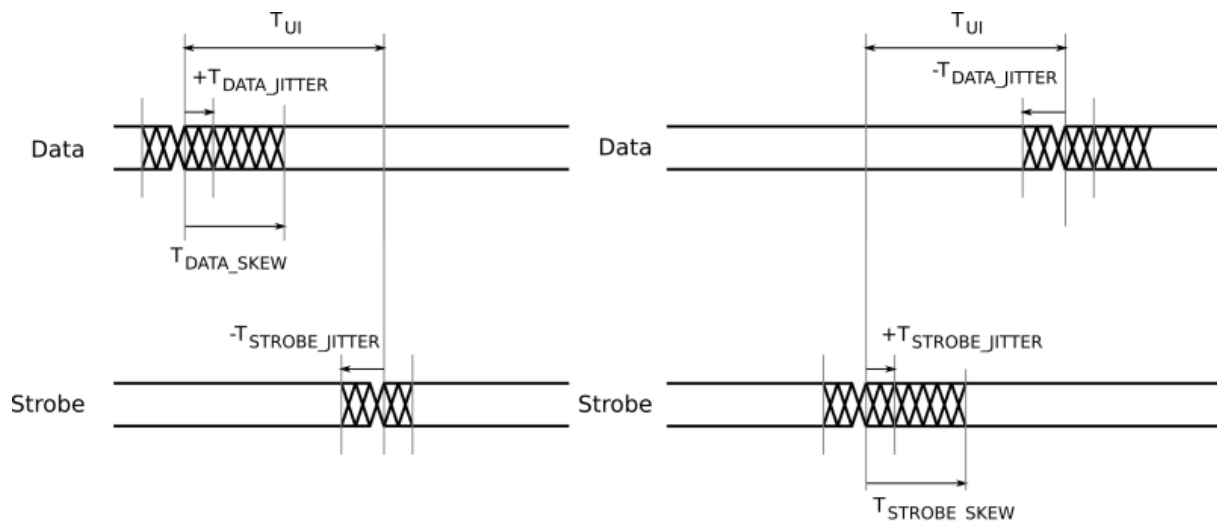


Figure 3.22: SpaceWire input switching characteristics

	Description	ns (Max)
T_{UI}	Transmitter bit rate	5
$T_{DATA_SKEW_IN} (MAX)$	Maximum data skew which can be tolerated by the receiver.	$2 - T_{DATA_JITTER_IN} - T_{STROBE_JITTER_IN}$
$T_{STROBE_SKEW_IN} (MAX)$	Maximum strobe skew which can be tolerated by the receiver.	$2 - T_{DATA_JITTER_IN} - T_{STROBE_JITTER_IN}$

The data/strobe output switching characteristics are defined in the table below.

	Description	ns (Max)
T_{SKEW_OUT}	Maximum data or strobe skew	0
T_{JITTER_OUT}	Maximum data/strobe jitter	0.7 (Peak to Peak)

3.3.7 Grounding Information

When using LVDS it is very important that the grounds of the equipment at the two ends of each SpaceWire link are connected. Otherwise a significant common mode voltage can exist between the two units resulting in the maximum input voltage at one end or the other being exceeded. This could then result in damage to either unit.

The SpaceWire outer shield is connected to the backshell of the connectors at either end of a SpaceWire cable and when screwed into the mating SpaceWire connector on a unit ought to make a connection to the chassis and ground plane of that unit. Properly connecting a SpaceWire cable would then suffice to make the required ground connection between the two units being connected by SpaceWire.

STAR-Dundee equipment always has a low impedance connection between the backshell of the connector, the chassis and ground plane of the electronics in the unit. STAR-Dundee cables have the outer shield connected to the backshell of the connectors at either end of the cable.

When testing a new SpaceWire unit, it is recommended that you check that there is a low impedance connection between the backshell of the connector on your unit under test (UUT) and the chassis and ground plane in the UUT. If this is not low impedance then that problem should be resolved before connecting any SpaceWire equipment to the UUT. If there is a low impedance connection then please check the cable you are using and that there is a low impedance connection between the backshells of the connector at either end of the cable.

3.3.8 EMC Conformity

Declaration of Conformity

We

STAR-Dundee Ltd
STAR House, 166 Nethergate, Dundee, DD1 4EE, Scotland, UK

declare under our sole responsibility that the product

SpaceWire Brick Mk2

to which this declaration relates is in conformity with the following standard(s) or other normative documents

EN 61326-1:1997 +A1: 1998 +A2:2001
47CFR:2002
ANSI C63-4:1992
CFR47 (2002) Part 15, Sub part B

following the provisions of the EMC Directive.

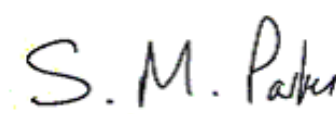
Dundee, 21st December 2012	
(place and date of issue)	(name and signature of authorised person)

Figure 3.23: S.M. Parkes

WARNING This is a class A product. In a domestic environment this product may cause radio interference in which case the user may be required to take adequate measures

3.4 SpaceWire Router Mk2S

Documentation for the STAR-Dundee SpaceWire Router Mk2S device is provided in this section.

3.4.1 Overview

3.4.1.1 Summary

This document provides an overview of the interfaces and features of the SpaceWire Router Mk2S hardware.

For detailed information on using the SpaceWire Router Mk2S device please consult the API reference documentation.

3.4.1.2 Hardware Description

The SpaceWire Router Mk2S provides a USB to SpaceWire interface device with eight SpaceWire interfaces. It also has two external FIFO ports and an external time-code port to permit interfacing to development boards and equipment. Efficient host software is provided to support rapid sending and receiving of SpaceWire packets into host PC memory.

A block diagram of the SpaceWire Router Mk2S is shown below.

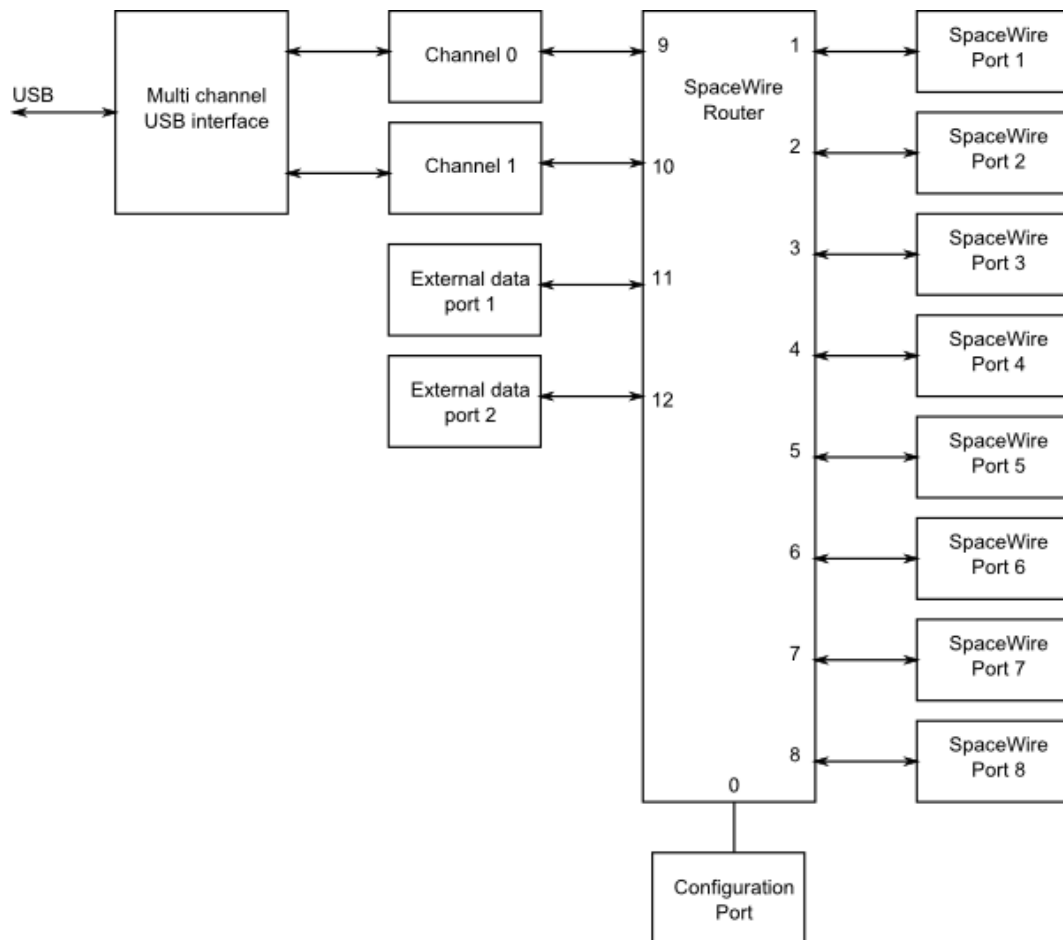


Figure 3.24: SpaceWire Router Mk2S hardware overview

The eight SpaceWire interfaces of the SpaceWire Router Mk2S are fully compliant with the SpaceWire standard and operate at up to 200 Mbit/s. They are connected to a SpaceWire router which supports two modes of operation, interface mode and router mode. In router mode, packets from any SpaceWire port can be routed to any other SpaceWire port, or the USB interface. In interface mode, the SpaceWire router is passive and packets which are received on a SpaceWire link are passed directly to the USB data channel. There is only one data channel which is shared between the eight ports, so traffic flowing over one port may block traffic on another port. There is also a separate control channel used to ensure that the host PC is always able to access the control, configuration and status space of the Router Mk2S regardless of the data flow.

The USB interface is compliant to USB version 2.0, and can also be used in USB 1.1 and USB 3.0 ports.

The SpaceWire Router Mk2S includes support for fault injection including support for parity errors, credit errors, escape errors, data corruption and EEP termination of packets.

3.4.2 Getting Started

The SpaceWire Router Mk2S is supplied with a power supply and USB cable. The power supply jack should be plugged into the power socket at the rear of the SpaceWire Router Mk2S unit. When inserted, the SpaceWire transmit and receive LEDs should illuminate. The USB cable is inserted in the USB socket located at the rear of the Router Mk2S.

The SpaceWire Router Mk2S software and drivers, provided by STAR-System, should be installed on the host computer before connecting the Router Mk2S. This ensures that the Router Mk2S is recognised by the host when it is connected.

Note: The USB cable need not be inserted if the Router Mk2S is part of a network and is only used to route data in the network. The USB cable is only needed if the Router Mk2S unit is to provide an interface from a SpaceWire network to a PC.

3.4.3 Interfaces

3.4.3.1 SpaceWire Ports and LEDs

The Router Mk2S has eight SpaceWire ports numbered 1 to 8. The USB port is router links 9 and 10. The Router Mk2S has two tri-colour LEDs for each SpaceWire port. The LED on the left next to each link shows received data for that link, the LED on the right next to each link shows transmitted data for that link. The functions of the SpaceWire port LEDs are defined in the table below.

Function	LED Colour	Description
No event	Amber	The left and right hand LEDs are steady amber.
Link running	Green	The left and right hand LEDs are turned green when the link is running. The link running LEDs are held for approximately half a second after the link has stopped running and no error occurs.
Error	Red	The left and right hand LEDs are turned red.
Data Transmitted/Received	Blue	When data is received on the link, the left hand LED turns blue. When data is transmitted on the link, the right hand LED turns blue.
Identify	Flashing purple	The SpaceWire Router Mk2S can be forced to flash its front panel LEDs to allow the device to be easily identified among a group of other devices. When this function is used the LEDs will flash purple for a short period of time.

Please note that there is an issue with all SpaceWire interfaces where a small amount of noise on the inputs can be translated by the LVDS receivers into a digital pattern and then interpreted by the receiver as a bit sequence. Since this bit sequence is invalid it causes the SpaceWire CODEC to start up and then disconnect. The SpaceWire interface LEDs go red on disconnect (ErrorReset state) and amber when ready (Ready state). So, if you get a burst of noise on the input then you will see a red flash of around half a second on the LEDs. To avoid this happening all the time we have a bias resistor network on the input of the LVDS receivers that provides a small bias current. This filters out most of the noise, but occasionally depending on the environment there may still be enough noise to trigger the reset to ready cycle and you see this as a red flash on the LEDs. This is the expected/normal operation. If you have any concerns, please contact support@star-dundee.com.

3.4.3.2 External FIFO Ports

The Router Mk2S has two external FIFO ports which may be used to interface development equipment. Each FIFO port is synchronous and runs at 30MHz. All signals are LVCMOS logic at 3.3V. The connector is a standard fine pitch SCSI-2 D-type, e.g. Molex 1587-6040. It is recommended that an insulation displacement (IDC) mating part is used. A typical mating part would be a Multicom 8E050P-32000AF.

The pinout for both external FIFO ports is as follows:

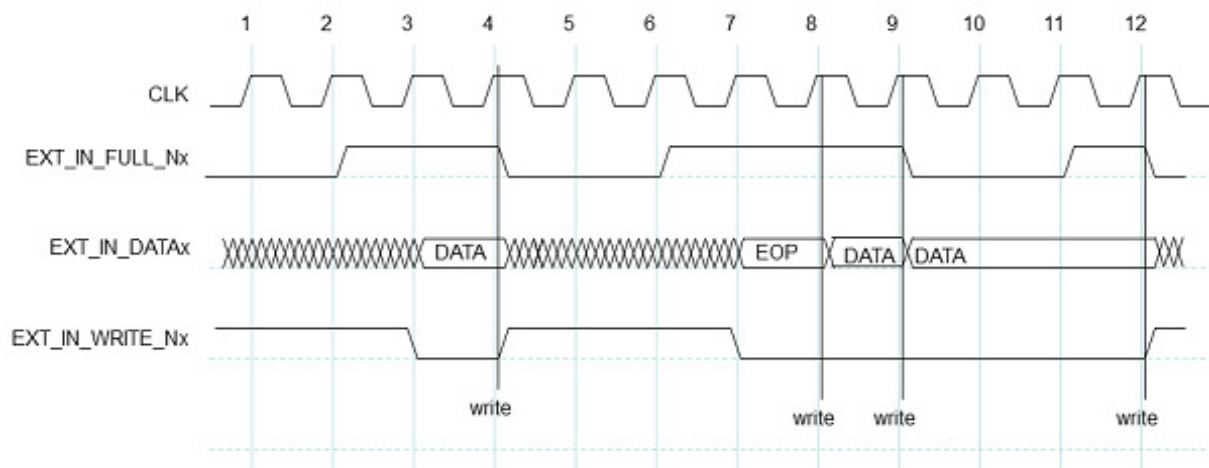
Pin	Signal	Symbol
1	Data Out (8)	EXT_OUT_DATA(0)
2	Data Out (7)	EXT_OUT_DATA(1)
3	Data Out (6)	EXT_OUT_DATA(2)
4	Data Out (5)	EXT_OUT_DATA(3)
5	Data Out (4)	EXT_OUT_DATA(4)
6	Data Out (3)	EXT_OUT_DATA(5)
7	Data Out (2)	EXT_OUT_DATA(6)
8	Data Out (1)	EXT_OUT_DATA(7)
9	Data Out (0)	EXT_OUT_DATA(8)
10	Output FIFO Empty (Active Low)	EXT_OUT_EMPTY_N
11	Input FIFO Full (Active Low)	EXT_IN_FULL_N
12	<i>Do Not Connect</i>	NC
13	System Clock (30MHz)	SYSCLK
14	Data In (8)	EXT_IN_DATA(0)
15	Data In (7)	EXT_IN_DATA(1)
16	Data In (6)	EXT_IN_DATA(2)
17	Data In (5)	EXT_IN_DATA(3)
18	Data In (4)	EXT_IN_DATA(4)
19	Data In (3)	EXT_IN_DATA(5)
20	Data In (2)	EXT_IN_DATA(6)
21	Data In (1)	EXT_IN_DATA(7)
22	Data In (0)	EXT_IN_DATA(8)
23	Input FIFO Write (Active Low)	EXT_IN_WRITE_N
24	Output FIFO Read (Active Low)	EXT_OUT_READ_N
25	<i>Do Not Connect</i>	NC
26	Ground	
27	Ground	
28	Ground	
29	Ground	
30	Ground	
31	Ground	
32	Ground	
33	Ground	
34	Ground	
35	Ground	
36	Ground	
37	Ground	

38	Ground	
39	Ground	
40	Ground	
41	Ground	
42	Ground	
43	Ground	
44	Ground	
45	Ground	
46	Ground	
47	Ground	
48	Ground	
49	Ground	
50	Ground	

Data into and out of the FIFO ports is 9 bits wide. The lowest 8 bits (0-7) are data, where bit 0 is the least significant; the highest bit (8) selects whether this word is data (set to zero) or an end of packet marker (set to one). If an end of packet marker is selected (bit 8 = '1') the data bits (0-7) are used to select an EOP (0x00) or an EEP (0x01). All other end of packet codes are reserved and should not be used.

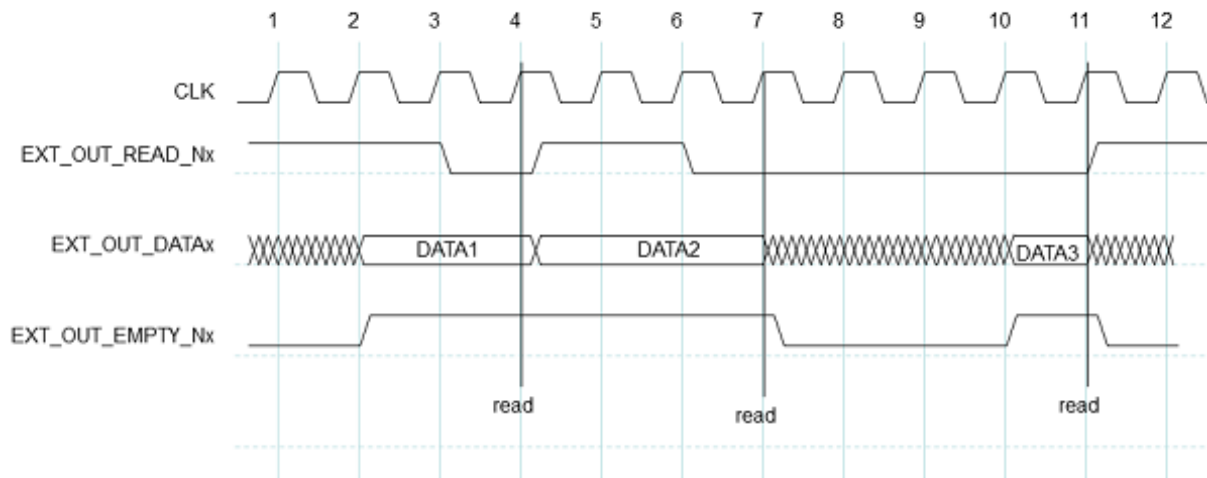
All data and control inputs (EXT_IN_DATA signals, EXT_IN_WRITE_N and EXT_OUT_READ_N) have internal pull-ups. Clock signals are source terminated for 50Ω.

3.4.3.2.1 External Port Write Cycle



The operation of the external port during write operations starts with the EXT_IN_FULL_N signal being de-asserted by the router (at clock cycle 2 above) to indicate to the external system that the router has room for more data and is ready to receive it through the external port. The external system then puts data onto the EXT_IN_DATA data lines and asserts EXT_IN_WRITE_N to transfer data into the external port on the next rising edge of SYSClk. As long as there is room for new data (EXT_IN_FULL_N is inactive) the write access is performed as long as EXT_IN_WRITE_N is active. If no room is available the write access is ignored (cycles 10 and 11 in the diagram above) and will be performed when room has become available if EXT_IN_WRITE_N is still active. Therefore the data (EXT_IN_DATA) must be valid at that time.

3.4.3.2.2 External Port Read Cycle



Reading of the external port is illustrated in the diagram above. When data is available in the external port FIFO then it is placed on the EXT_OUT_DATA bus and the EXT_OUT_EMPTY_N signal is asserted to signal to the external system that data is available. This is done synchronously to the SYSCLK signal (e.g. clock cycle 2 in the diagram). When it is ready, the external system asserts the EXT_OUT_READ_N signal synchronously with the SYSCLK signal (e.g. clock cycle 3) and the data is then read out of the external port on the next rising edge of the SYSCLK (e.g. start of clock cycle 4). If there is no more data available in the FIFO then the EXT_OUT_EMPTY_N is de-asserted once the data has been read. If the FIFO contains more data to transfer then the EXT_OUT_EMPTY_N remains asserted, the new data is placed on the EXT_OUT_DATA bus and the external system can read it as soon as it is ready. The read access is ignored if there is no data available (EXT_OUT_EMPTY_N is active).

3.4.3.2.3 External Time-code Port

The Router Mk2S has a single time-code port which may be used to interface development equipment. All signals are LVCMOS logic at 3.3V. The connector is identical to that used for the external FIFO ports (SCSI-2 D-type, e.g. Molex 1587-6040). Again, it is recommended that an insulation displacement (IDC) mating part is used. A typical mating part would be a Multicomp 8E050P-32000AF.

The pinout for the external time-code port is as follows:

Pin	Signal	Symbol
1	Time Out (7)	EXT_TIME_OUT(7)
2	Time Out (6)	EXT_TIME_OUT(6)
3	Time Out (5)	EXT_TIME_OUT(5)
4	Time Out (4)	EXT_TIME_OUT(4)
5	Time Out (3)	EXT_TIME_OUT(3)
6	Time Out (2)	EXT_TIME_OUT(2)
7	Time Out (1)	EXT_TIME_OUT(1)
8	Time Out (0)	EXT_TIME_OUT(0)
9	Tick Out	EXT_TICK_OUT
10	Do Not Connect	NC
11	Do Not Connect	NC
12	Do Not Connect	NC
13	Do Not Connect	NC

14	Time In (7)	EXT_TIME_IN(7)
15	Time In (6)	EXT_TIME_IN(6)
16	Time In (5)	EXT_TIME_IN(5)
17	Time In (4)	EXT_TIME_IN(4)
18	Time In (3)	EXT_TIME_IN(3)
19	Time In (2)	EXT_TIME_IN(2)
20	Time In (1)	EXT_TIME_IN(1)
21	Time In (0)	EXT_TIME_IN(0)
22	Select External Time Data	SEL_EXT_TIME
23	Tick In	EXT_TICK_IN
24	Reset Internal Time Counter	TIME_CTR_RST
25	Router Reset (Active Low)	SYSRST_N
26	Ground	
27	Ground	
28	Ground	
29	Ground	
30	Ground	
31	Ground	
32	Ground	
33	Ground	
34	Ground	
35	Ground	
36	Ground	
37	Ground	
38	Ground	
39	Ground	
40	Ground	
41	Ground	
42	Ground	
43	Ground	
44	Ground	
45	Ground	
46	Ground	
47	Ground	
48	Ground	
49	Ground	
50	Ground	

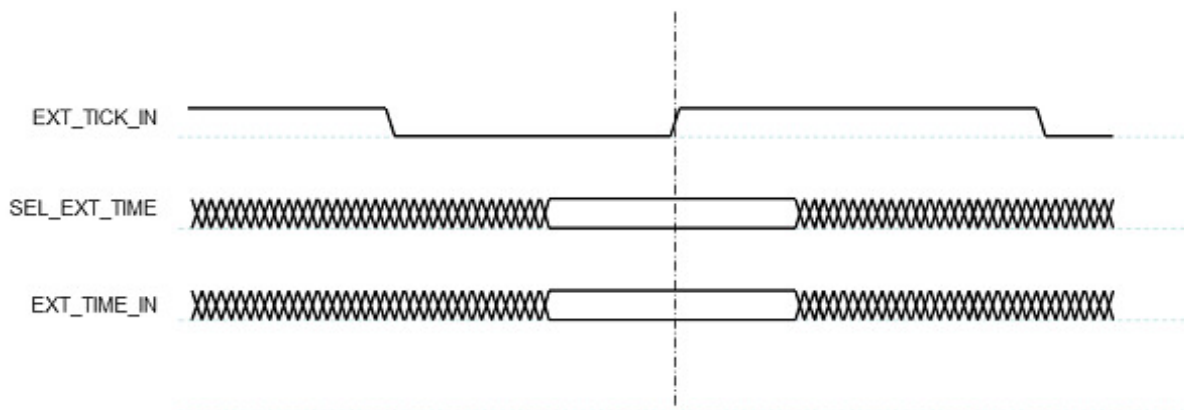
The time-code interface consists of a time-code source for generating time-codes to be transmitted over the SpaceWire link (the EXT_TIME_IN signals, SEL_EXT_TIME and EXT_TICK_IN) and a time-code sink for receiving time-codes from the SpaceWire links (the EXT_TIME_OUT signals and EXT_TICK_OUT). The tick signals indicate that a time-code should be transmitted (EXT_TICK_IN) or that one has been received (EXT_TICK_OUT) the time signals indicate the value of the time-code. Bit 0 is the least significant bit. The operation of the SEL_EXT_TIME and TIME_CTR_RST signals are described below.

All time inputs (EXT_TIME_IN signals, SEL_EXT_TIME, EXT_TICK_IN and TIME_CTR_RST) have internal pull-downs.

The SYSRST_N signal is bidirectional and can be used to reset the entire router. SYSRST_N is both logically and electrically equivalent to pressing the reset button. This signal is active low and has an internal pull-up. It must be driven by an open-drain driver. The router may also be reset via USB (software) and by the reset button. In both of these cases this signal will go low. If reset functionality is not required, it is safest to leave this signal unconnected.

3.4.3.2.4 Generating Time-codes

Time-codes can be generated by the router on request of the external system to which it is attached. A time-code is generated whenever the router detects a rising edge on the EXT_TICK_IN signal. The value of the time-code to be transmitted is either taken from the EXT_TIME_IN inputs or from the time-code counter inside the router. The time-code source used depends on the value of the SEL_EXT_TIME signal when EXT_TICK_IN signal has a rising edge. If SEL_EXT_TIME is 1 then the EXT_TIME_IN inputs are used to provide the contents of the time-code.

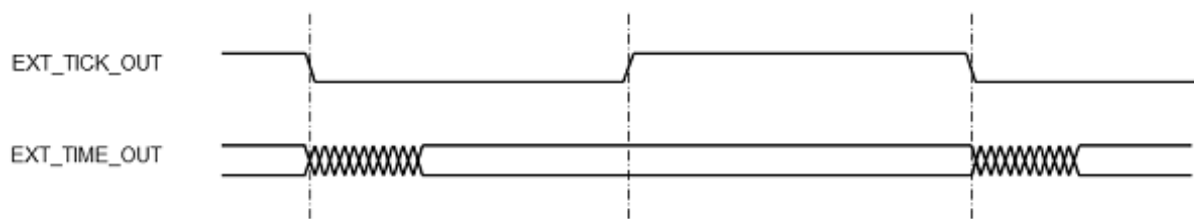


If SEL_EXT_TIME is 0 then the internal time-code counter provides the least-significant 6-bits of the time-code and the top two bits of EXT_TIME_IN (bits 7 and 6) provide the most-significant 2-bits. When using the EXT_TIME_IN inputs to provide the complete time-code, the time-code is only broadcast if it is a valid time-code i.e. is one more than the internal time register of the router (for more information refer to the SpaceWire standard).

Note: Only one router or node in a SpaceWire network should operate as a time master generating time-codes (again, see the SpaceWire standard for more information).

3.4.3.2.5 Receiving Time-codes

When a valid time-code is received by the router the value of this time-code (flags plus time value) will be placed on the EXT_TIME_OUT outputs and the EXT_TICK_OUT signal will be set to zero. The EXT_TICK_OUT signal is set to one a short time later, once the EXT_TIME_OUT outputs have stabilised, to indicate that these outputs are valid.



They then remain valid until the next time-code is received and the EXT_TICK_OUT signal will be set to zero.

3.4.3.2.6 Time Counter Reset

When a rising edge is detected on TIME_CTR_RST then the internal time-code register is reset to zero.

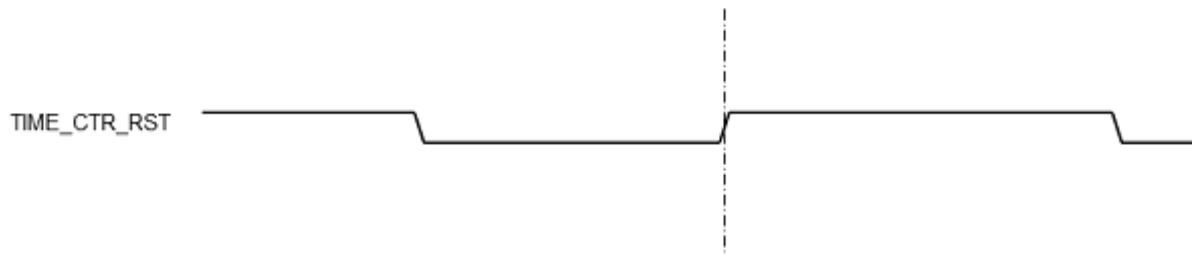


Figure 3.25: Time-code reset interface

3.4.3.3 Trigger In and Out

Trigger in and out connectors provide a mechanism by which SpaceWire transmission may be synchronised to an external source (trigger in) and external equipment (e.g. an oscilloscope) may be synchronised to SpaceWire events. Trigger connectors are standard SMB-style and operate at 3.3V LVCMOS levels.

3.4.4 Functions

3.4.4.1 SpaceWire Router

The SpaceWire router inside this unit is compatible with the AT7910E radiation tolerant router (also known as the SpW-10X) manufactured by Atmel.

The SpaceWire router supports two modes of operation, interface mode and routing mode.

The default mode is router mode. The Router Mk2S will revert to router mode after a device reset is performed.

The SpaceWire router has 13 ports in total as listed below:

- 1 internal router configuration port
- 8 external SpaceWire ports
- 2 USB interface external ports, one of which may be used by the SpaceWire ports, while the other may be used by the configuration port
- 8 external FIFO ports

3.4.4.2 Interface Mode

In interface mode, a packet which is received on a SpaceWire link will always be routed to the port specified by the port routing register of the port. Packets received from the USB external ports or the configuration port are routed based on the first byte, as for routing mode. The SpaceWire Router Mk2S supports interface mode by utilising the port routing registers of the SpaceWire router. The default values are listed in the image and table below.

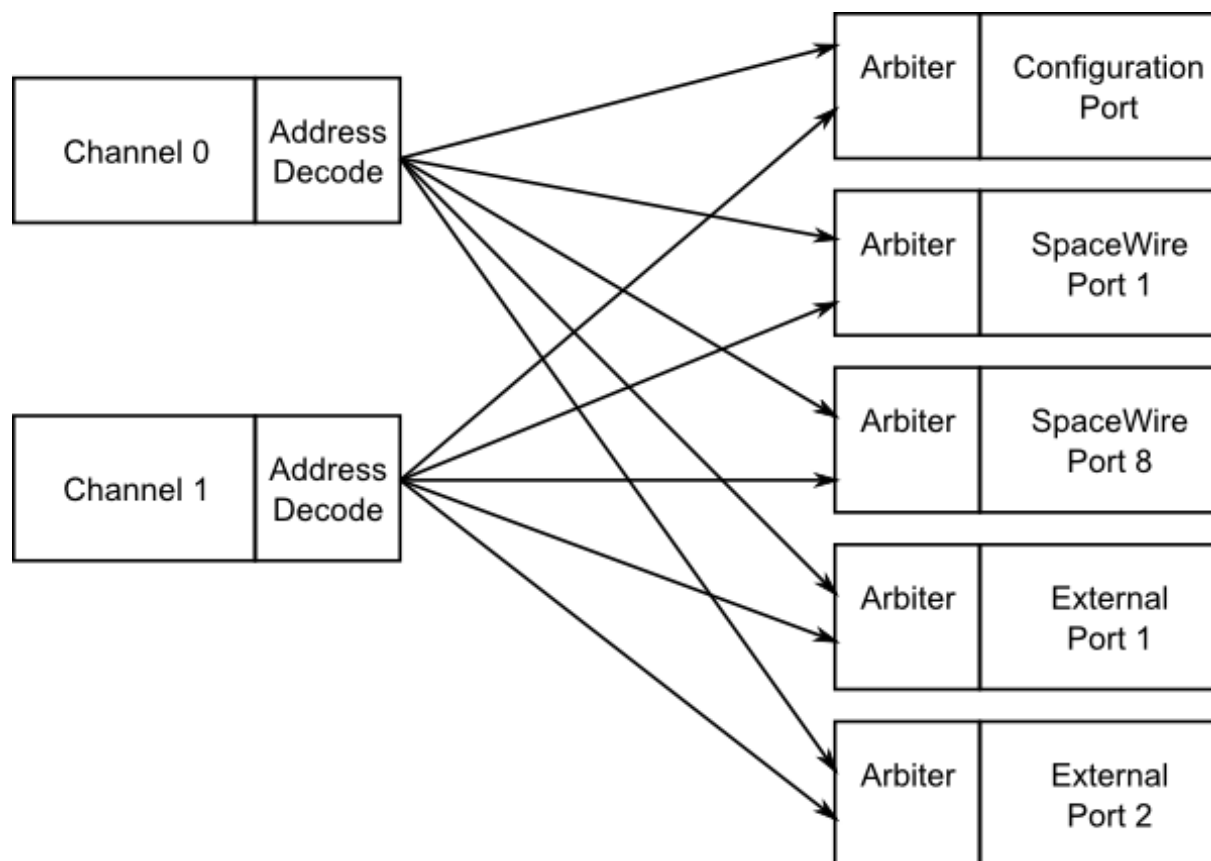


Figure 3.26: Interface mode routing connections, USB to SpaceWire

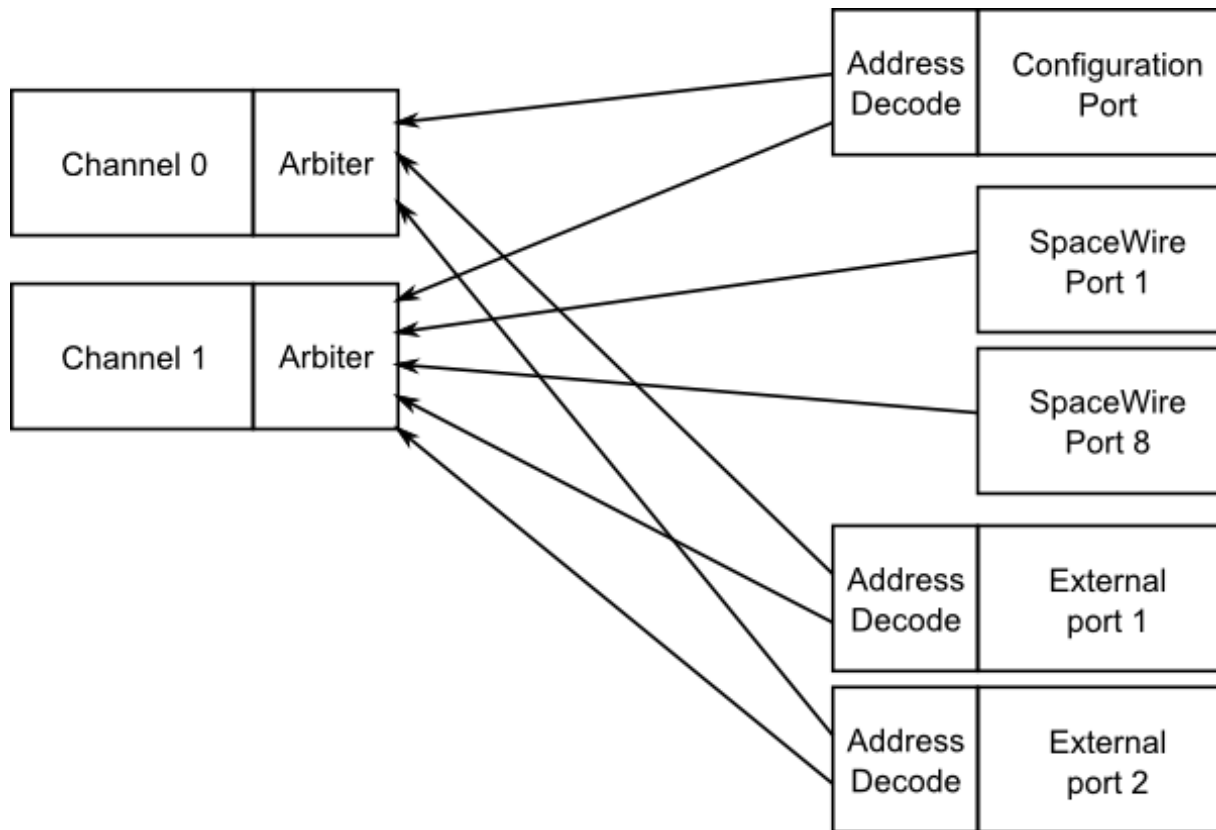


Figure 3.27: Interface mode routing connections, SpaceWire to USB

Source port	Destination port	Description
0	9 or 10	Configuration to USB interface channel (Reply packets routed based on command source)
1 to 8	10	SpaceWire links to USB interface channel 1
11, 12	10	External FIFO ports to USB interface channel 1
9	0 to 8, 11 or 12	USB interface channel 0 to configuration port, SpaceWire links, external FIFO ports (Packets routed based on first byte)
10	0 to 8, 11 or 12	USB interface channel 1 to configuration port, SpaceWire links, external FIFO ports (Packets routed based on first byte)

3.4.4.3 Routing Mode

In routing mode packets are routed dependent on the first byte of each SpaceWire packet. The routing algorithm is described below.

- Packets with known physical addresses are routed to the port indicated by the physical address at the head of the packet, i.e. a packet with address 0 is routed to the configuration port.
- Packets with unknown physical addresses, 9 to 31, are discarded and an address error is set in the source port register.
- Packets with logical address headers which are valid are routed to one of the set ports in the logical address lookup table (accessed via the configuration port).
- Packets with logical address headers which are marked as invalid are discarded and an address error is set in the source port register.

3.4.4.4 Link Operating Speed

The SpaceWire link operating speed for each SpaceWire link is controlled by register settings. The clock selection logic for each link is described in the diagram below.

The default link operating speed is 200 Mbit/s after link start-up and 10 Mbit/s during link initialisation.

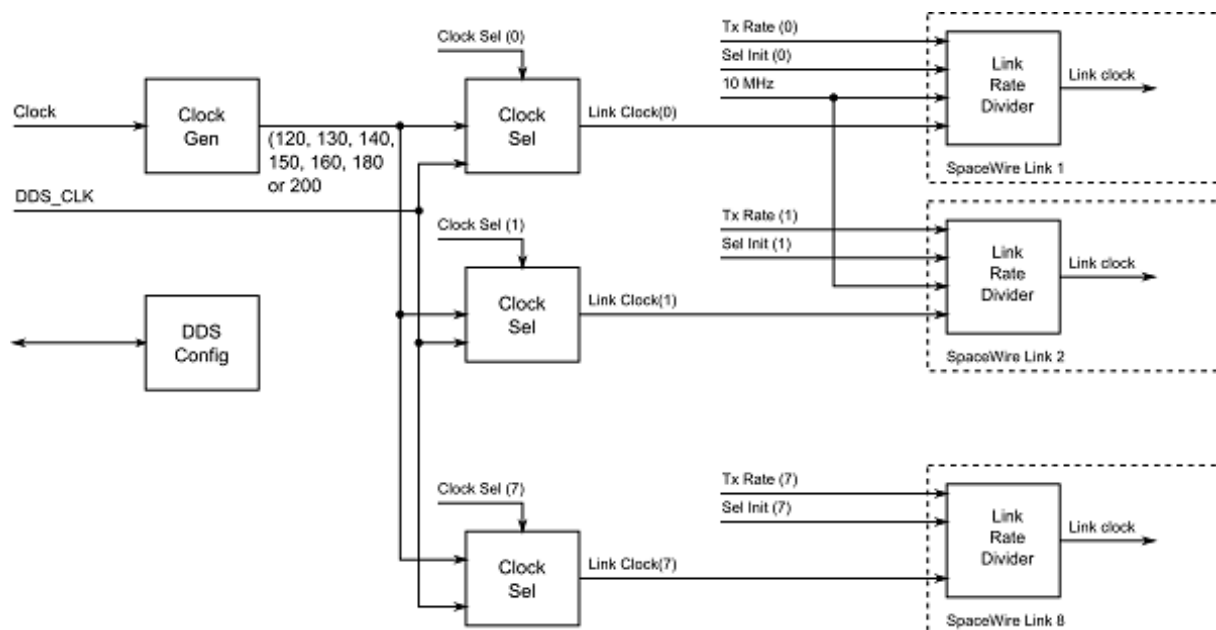


Figure 3.28: Link operating speed circuit diagram

The link operating speed for each SpaceWire link is set using the following variables:

- Clock Generator:
 - Generates a set of clock frequencies for each link.
- Link clock select (one for each SpaceWire link):
 - 120, 130, 140, 150, 160, 180, 200 MHz
- Link rate divider (one for each SpaceWire link):
 - Divide by an even integer value.

The clock generator generates seven clock frequencies which can be individually selected by the link clock select modules.

Each link clock select module chooses between the set of clock generator frequencies using a link clock select register which is accessible through the SpaceWire router configuration port.

In addition, a precision transmit rate mode can be enabled, which generates any frequency the user requires between 2 and 200 Mbit/s and is accurate to more than 0.1 Mbit/s.

The link rate can be programmed using the functions provided by the Mk2 Device and Interface Configuration API and the Brick Mk2 Configuration API. The precision transmit rate can be configured using the Router Mk2S Configuration API. Please refer to the API documentation for further details on how to set these values.

3.4.4.5 Time-codes

A time-code can be received from the USB interface or from a SpaceWire link. When a time-code is received, its value is checked against the value of the internal time-code register in the SpaceWire router. If the received time-code is one more than the value of the time-code register then the time-code will be written to the time-code register and propagated to other ports, except the port that was the source of the time-code.

Time-codes are only propagated on ports which are enabled to send time-codes as defined by the time-code control register. Note that of the external USB ports, only port 9, corresponding to channel 0, can currently be used to send or receive time-codes.

The router has an internal time-code generation master, which can be controlled by the SpaceWire router configuration port to generate time-codes at a user defined rate.

An overview of the SpaceWire router time-code interface is shown below.

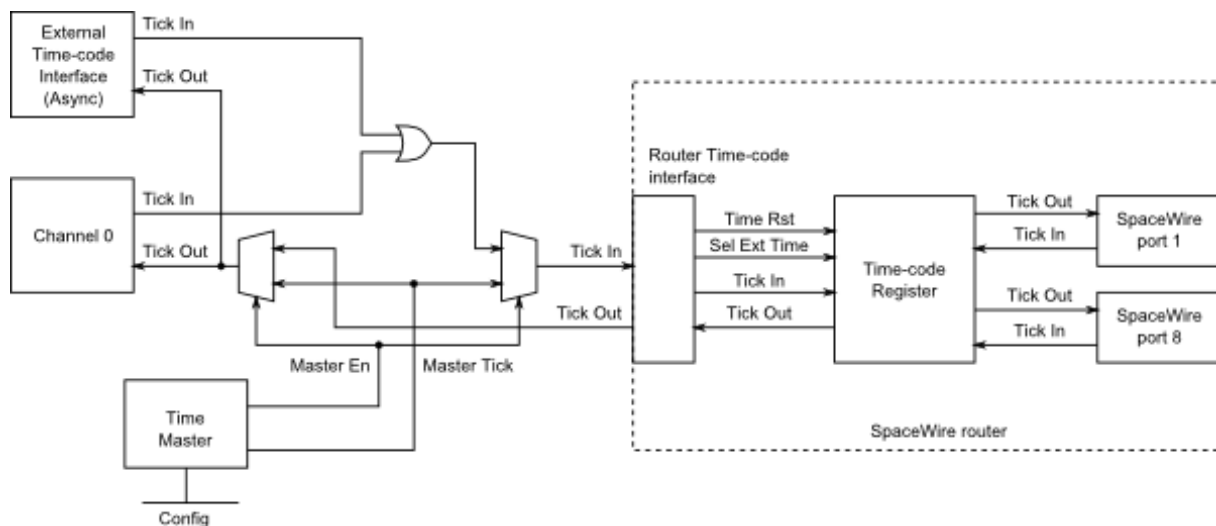


Figure 3.29: Time-code processing architecture

3.4.5 Electrical Characteristics

The SpaceWire interface absolute maximum input ratings are defined in the table below.

	Description	V _{MIN}	V _{MAX}
V _{IN}	Input voltage relative to GND	-0.5	3

The SpaceWire connector LVDS DC characteristics are listed in the table below.

	V_{ID} mV, Min	mV, Max	V_{ICM} V, Min	V, Max	V_{OD} mV, Min	mV, Max	V_{OCM} V, Min	V, Max
Din, Sin	100	-	0.2	2.1				
Dout, Sout					250	400	1.125	1.375

V_{ID}	Input differential voltage
V_{ICM}	Input common mode voltage
V_{OD}	Output differential voltage
V_{OCM}	Output common mode voltage

The common mode and differential identifiers are described in the diagram below.

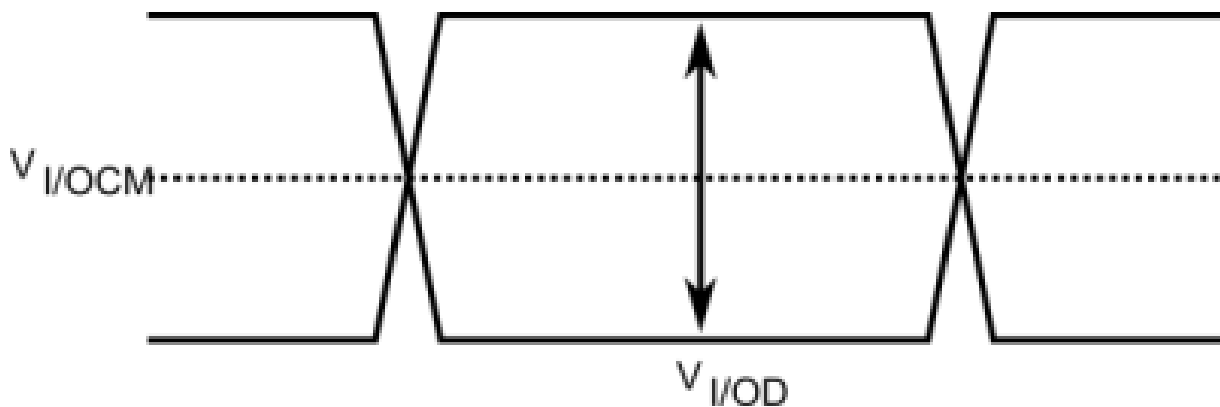


Figure 3.30: SpaceWire LVDS electrical specifications

3.4.6 Switching Characteristics

The SpaceWire interface data/strobe input switching characteristics are defined in the figure and table below.

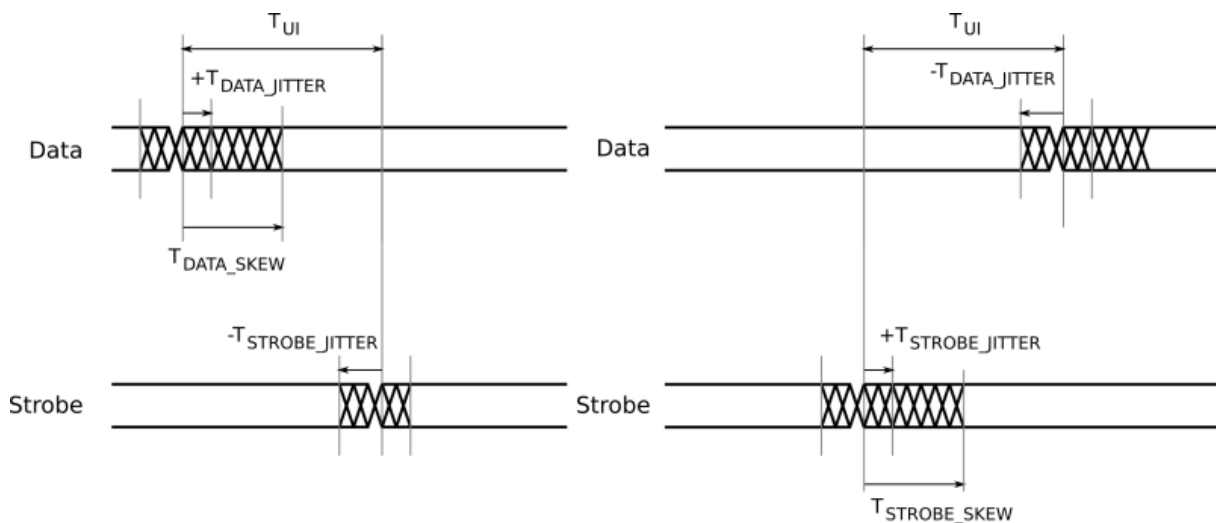


Figure 3.31: SpaceWire input switching characteristics

	Description	ns (Max)
T_{UI}	Transmitter bit rate	5
$T_{DATA_SKEW_IN} (MAX)$	Maximum data skew which can be tolerated by the receiver.	$2 - T_{DATA_JITTER_IN} - T_{STROBE_JITTER_IN}$
$T_{STROBE_SKEW_IN} (MAX)$	Maximum strobe skew which can be tolerated by the receiver.	$2 - T_{DATA_JITTER_IN} - T_{STROBE_JITTER_IN}$

The data/strobe output switching characteristics are defined in the table below.

	Description	ns (Max)
T_{SKEW_OUT}	Maximum data or strobe skew	0
T_{JITTER_OUT}	Maximum data/strobe jitter	0.7 (Peak to Peak)

3.4.7 Grounding Information

When using LVDS it is very important that the grounds of the equipment at the two ends of each SpaceWire link are connected. Otherwise a significant common mode voltage can exist between the two units resulting in the maximum input voltage at one end or the other being exceeded. This could then result in damage to either unit.

The SpaceWire outer shield is connected to the backshell of the connectors at either end of a SpaceWire cable and when screwed into the mating SpaceWire connector on a unit ought to make a connection to the chassis and ground plane of that unit. Properly connecting a SpaceWire cable would then suffice to make the required ground connection between the two units being connected by SpaceWire.

STAR-Dundee equipment always has a low impedance connection between the backshell of the connector, the chassis and ground plane of the electronics in the unit. STAR-Dundee cables have the outer shield connected to the backshell of the connectors at either end of the cable.

When testing a new SpaceWire unit, it is recommended that you check that there is a low impedance connection between the backshell of the connector on your unit under test (UUT) and the chassis and ground plane in the UUT. If this is not low impedance then that problem should be resolved before connecting any SpaceWire equipment to the UUT. If there is a low impedance connection then please check the cable you are using and that there is a low impedance connection between the backshells of the connector at either end of the cable.

3.4.8 EMC Conformity

Declaration of Conformity

We

STAR-Dundee Ltd
STAR House, 166 Nethergate, Dundee, DD1 4EE, Scotland, UK

declare under our sole responsibility that the product

SpaceWire Router Mk2S

to which this declaration relates is in conformity with the following standard(s) or other normative documents

EN 61326-1:1997 +A1: 1998 +A2:2001
47CFR:2002
ANSI C63-4:1992
CFR47 (2002) Part 15, Sub part B

following the provisions of the EMC Directive.

Dundee, 21st December 2012	
(place and date of issue)	(name and signature of authorised person)

Figure 3.32: S.M. Parkes

WARNING This is a class A product. In a domestic environment this product may cause radio interference in which case the user may be required to take adequate measures

3.5 SpaceWire Brick Mk3

Documentation for the STAR-Dundee SpaceWire Brick Mk3 device is provided in this section.

3.5.1 Overview

3.5.1.1 Summary

This user manual provides an overview of the interfaces and features of the SpaceWire Brick Mk3 hardware.

For detailed information on using the SpaceWire Brick Mk3 device please consult the STAR-System API reference documentation.

3.5.1.2 Hardware Description

The SpaceWire Brick Mk3 provides a USB to SpaceWire interface device with two SpaceWire interfaces. Efficient host software is provided to support rapid sending and receiving of SpaceWire packets into host PC memory.

A block diagram of the SpaceWire Brick Mk3 is shown below.

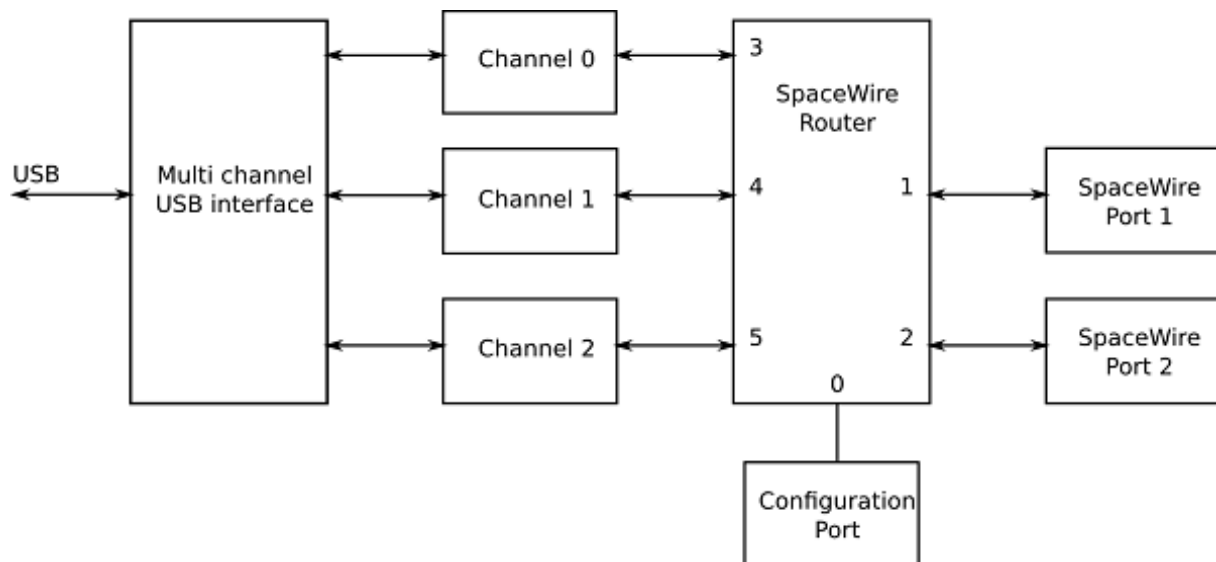


Figure 3.33: SpaceWire Brick Mk3 hardware overview

The two SpaceWire interfaces of the SpaceWire Brick Mk3 are fully compliant with the SpaceWire standard and operate at up to 300 Mbit/s. They are connected to a SpaceWire router which supports two modes of operation, interface mode and router mode. In router mode, packets from any SpaceWire port can be routed to any other SpaceWire port, or the USB interface. In interface mode, the SpaceWire router is passive and packets which are received on a SpaceWire link are passed directly to the USB data channel for that link. There are two independent channels from the SpaceWire router to the USB interface, so traffic flowing over one port cannot block traffic for another port. In addition, there is a separate control channel, so that the host PC is always able to access the control, configuration and status space of the Brick Mk3 regardless of the data flow.

The USB interface is compliant to USB version 3.0, as well as USB versions 1.1 and 2.0. This means it can be used in older PCs, albeit with lower throughput and higher latency achievable on the USB bus.

The SpaceWire Brick Mk3 includes support for fault injection including support for parity errors, credit errors, escape errors, data corruption and EEP termination of packets.

3.5.2 Getting Started

The SpaceWire Brick Mk3 is powered from the USB cable; therefore no external power supply is required. The USB cable is inserted in the USB socket located at the rear of the Brick Mk3.

When plugged into a host PC or laptop the Brick Mk3 performs a power negotiation stage with the host PC before powering up the router. The LEDs at the front of the Brick Mk3 indicate whether the host has accepted the request from the SpaceWire Brick Mk3 to draw power from the USB bus. When first plugged in, the LEDs remain off until power has been granted to the device.

After this phase the LEDs will default to their normal operating state e.g. with no cables connected all four LEDs are amber.

The SpaceWire Brick Mk3 software and drivers, provided by STAR-System, should be installed on the host computer before connecting the Brick Mk3. This ensures that the Brick Mk3 is recognised by the host when it is connected.

3.5.3 Rear Panel Interfaces

The Rear panel features the Micro-B USB 3.0 connector with an LED to indicate USB activity and "power good". An image of this is shown below.



Figure 3.34: Micro USB 3.0 connector and LED

3.5.3.1 USB Status LED

The USB status LED provides three functions:

1. Low level power good indicator
2. Active USB interface version
3. USB data transaction in progress

3.5.3.1.1 Low level power good indicator

If the USB Status LED fails to light up (along with the LEDs on the front panel) after a few seconds when the Brick Mk3 is connected to a USB port, then it is possible that the Host PC has not granted sufficient power on the USB bus to start up. In this event, disconnect the Brick Mk3, and try plugging it into another USB port on the same computer. If it fails a second time, try using a USB hub with an external power supply.

If the USB Status LED fails to light up when the device is plugged into a few different computers, or if it turns off in the middle of a test, then this is an indication that one of the power supplies has failed. The Brick Mk3 should be powered down and removed from the test system. Contact STAR-Dundee support at support@star-dundee.com for further advice in this situation.

3.5.3.1.2 Active USB interface version

The colour of the USB status LED is used to indicate the version of USB that the device is operating under. This is useful as the Brick MK3 is capable of operation under USB 1.1, 2.0 and 3.0. The colours are summarised in the table below.

USB Status LED Colour	Active USB Interface Version
Blue	USB 3.0
Green	USB 2.0
Red	USB 1.1

Note that it is sometimes possible to plug the Brick Mk3 into a USB 3.0 socket in such a way that it starts up in USB 2.0 mode. This is usually because the Type-A plug is inserted very slowly or, at a slight angle to the connector. If this happens, try unplugging and plugging the device.

3.5.3.1.3 USB data transaction in progress

The USB status LED can indicate when traffic is passing over the USB interface by blinking on and off. This behaviour is summarised in the table below.

USB Status LED	Status
Steady	No traffic is passing over the USB link
Blinking on and off rapidly	Traffic is passing over the USB link

Note that the LED will blink when data passes in either direction over any of the three USB endpoints.

3.5.3.2 Micro-B USB Connector

A suitable Type-A to Micro-B cable is supplied with the Brick Mk3 along with a ferrite attached for EMC purposes. This cable can be plugged into any Type-A USB 1.1, 2.0 or 3.0 port on a host PC. The Brick will have higher throughput and lower latency on a USB 3.0 port compared to a USB 2.0 port, so it is recommended to use USB 3.0 when possible. Computers with a mixture of USB 2.0 and USB 3.0 ports usually colour the plastic portion of a USB 3.0 port blue to distinguish them. You can also look at the USB Status LED to see in which USB mode the Brick Mk3 is operating.

Although it is possible to plug in other Type-A to Micro-B USB 2.0 or 3.0 cables, STAR-Dundee strongly recommends the use of the supplied cable as poor quality "off-the-shelf" cables can cause signal integrity problems in USB 3.0. EMC compliance may also be affected by using any other third party cables.

3.5.4 Front Panel Interfaces

The SpaceWire port on the left of the front panel is SpaceWire router link 1 and the port on the right is router link 2. The USB port is router links 3, 4 and 5. The front panel is shown below.

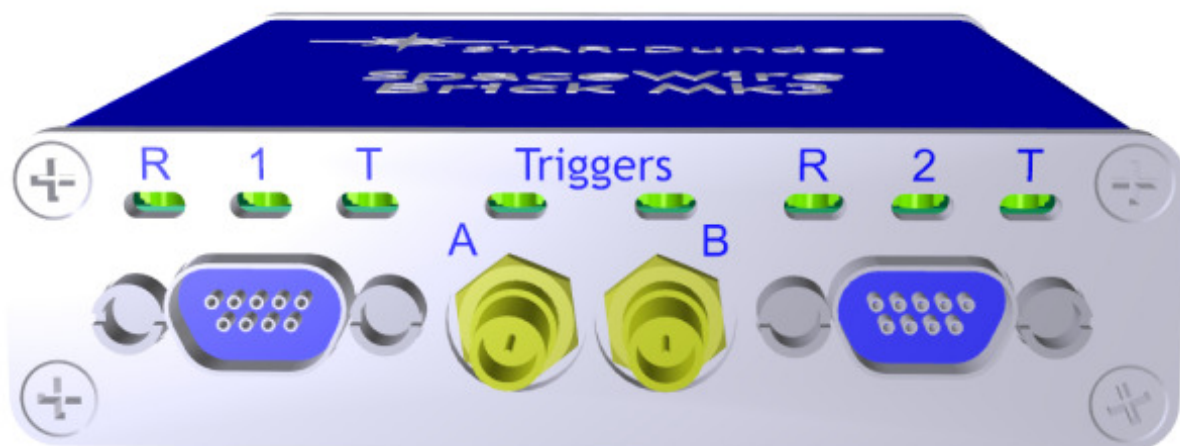


Figure 3.35: SpaceWire ports and LED positions

The Brick Mk3 has six multicolour LEDs, three for each SpW port. The LED on the left next to each link shows received data for that link, the LED on the right next to each link shows transmitted data for that link. The middle LED is currently unused. The functions of the SpaceWire port LEDs are defined in the table below.

Function	LED Colour	Description
No event	Amber	The left and right hand LEDs are steady amber.
Link running	Green	The left and right hand LEDs are turned green when the link is running. The link running LEDs are held for approximately half a second after the link has stopped running and no error occurs.
Error	Red	The left and right hand LEDs are turned red.
Data Transmitted/Received	Blue	When data is received on the link, the left hand LED turns blue. When data is transmitted on the link, the right hand LED turns blue.
No Credit	White	When no credit is available on the link for the device to transmit for a period of time, the right hand LED turns white. When the Brick Mk3 has provided no credit on the link for the device at the other end of the link to transmit for a period of time, the left hand LED turns white.
Identify	Flashing purple	The SpaceWire Brick Mk3 can be forced to flash its front panel LEDs to allow the device to be easily identified among a group of other devices. When this function is used the LEDs will flash purple for a short period of time.

Please note that there is an issue with all SpaceWire interfaces where a small amount of noise on the inputs can be translated by the LVDS receivers into a digital pattern and then interpreted by the receiver as a bit sequence. Since this bit sequence is invalid it causes the SpaceWire CODEC to start up and then disconnect. The SpaceWire interface LEDs go red on disconnect (ErrorReset state) and amber when ready (Ready state). So, if you get a burst of noise on the input then you will see a red flash of around half a second on the LEDs. To avoid this happening all the time we have a bias resistor network on the input of the LVDS receivers that provides a small bias current. This filters out most of the noise, but occasionally depending on the environment there may still be enough noise to trigger the reset to ready cycle and you see this as a red flash on the LEDs. This is the expected/normal operation. If you have any concerns, please contact support@star-undee.com.

3.5.5 Functions

3.5.5.1 SpaceWire Router

The SpaceWire router inside this unit is compatible with the AT7910E radiation tolerant router (also known as the SpW-10X) manufactured by Atmel.

The SpaceWire router supports two modes of operation, interface mode and routing mode.

The default mode is interface mode. The router will revert to interface mode after power cycling or a device reset is performed.

The SpaceWire router has 6 ports in total as listed below:

- 1 internal router configuration port
- 2 external SpaceWire ports
- 3 USB interface external ports. By default two of these are used by the two SpaceWire ports, while the other is used by the configuration port

3.5.5.2 Interface Mode

In interface mode, a packet which is received on a SpaceWire link will always be routed to the port specified by the port routing register of the port. The SpaceWire Brick Mk3 supports interface mode by utilising the port routing registers of the SpaceWire router. The default values are listed in the image and table below.

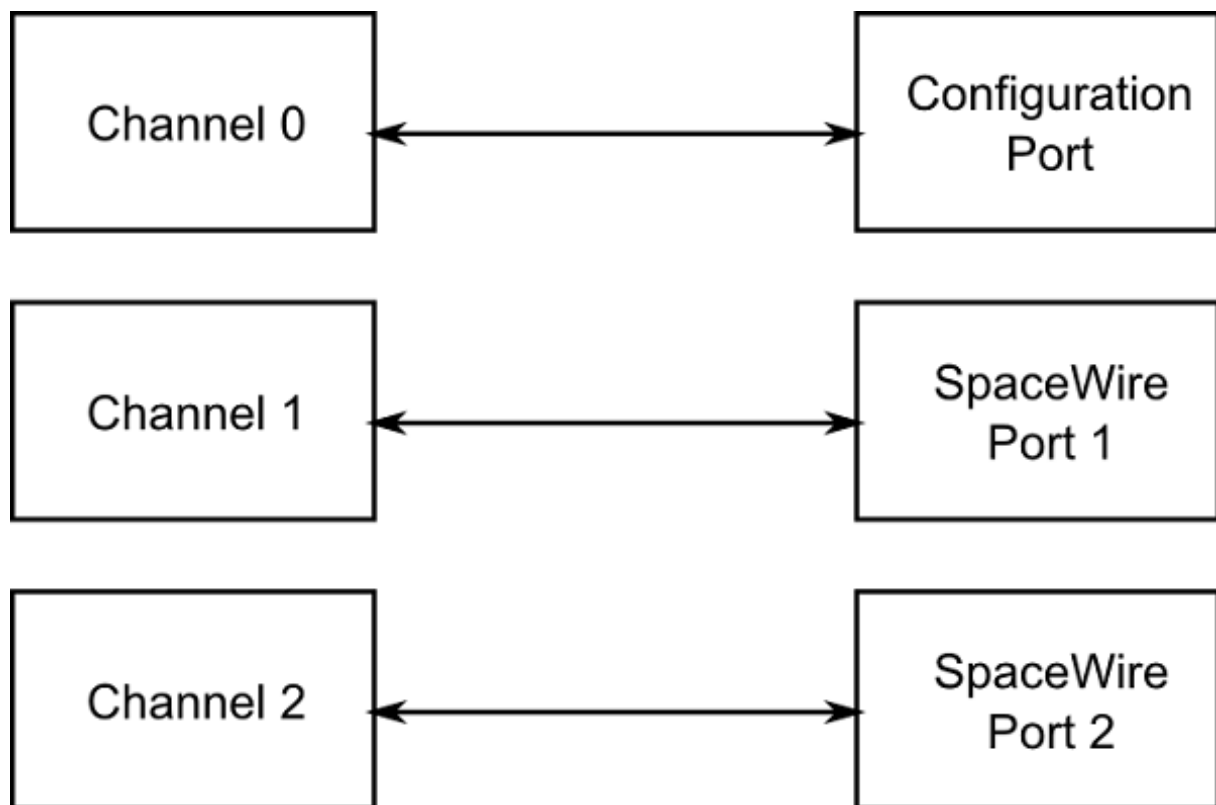


Figure 3.36: Interface mode routing connections

Source port	Destination port	Description
0	3	Configuration to external port 1 (host channel 0)
1	4	SpaceWire link 1 to external port 2 (host channel 1)
2	5	SpaceWire link 2 to external port 3 (host channel 2)

3	0	USB interface channel 0 to configuration port
4	1	USB interface channel 1 to SpaceWire link 1
5	2	USB interface channel 2 to SpaceWire link 2

3.5.5.3 Routing Mode

In routing mode packets are routed dependent on the first byte of each SpaceWire packet. The routing algorithm is described below.

- Packets with known physical addresses are routed to the port indicated by the physical address at the head of the packet, i.e. a packet with address 0 is routed to the configuration port.
- Packets with unknown physical addresses, 6 to 31, are discarded and an address error is set in the source port register.
- Packets with logical address headers which are valid are routed to one of the set ports in the logical address lookup table (accessed via the configuration port).
- Packets with logical address headers which are marked as invalid are discarded and an address error is set in the source port register.

3.5.5.4 Link Operating Speed

The link operating speed for each SpaceWire port is controlled by configuration parameters which can be set by the API or GUI application. The link clock rate selection logic for each port is shown in the figure below.

The default link operating speed is 200 Mbit/s after link start-up and 10 Mbit/s during link initialisation.

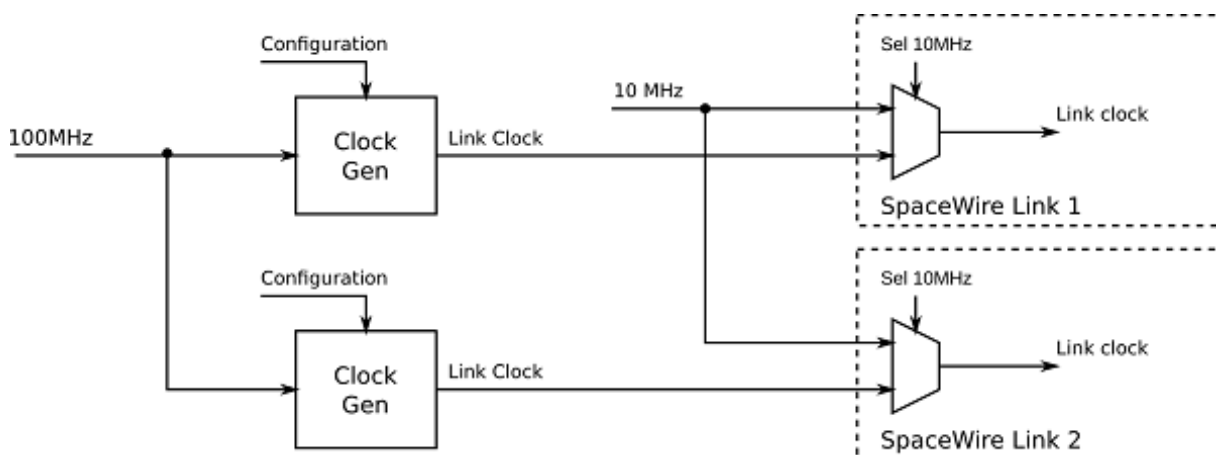


Figure 3.37: Link operating speed circuit diagram

The link operating speed for each SpaceWire link is set using a programmable clock generator frequency. This is set by configuring a multiply and divide value to the input 100 MHz clock.

The link rate can be programmed using the functions provided by the Mk2 Device and Interface Configuration API. Please refer to the API documentation for further details on how to set this clock speed.

3.5.5.5 Time-codes

A time-code can be received from the USB interface or from a SpaceWire link. When a time-code is received, its value is checked against the value of the internal time-code register in the SpaceWire router. If the received time-code is one more than the value of the time-code register then the time-code will be written to the time-code register and propagated to other ports, except the port that was the source of the time-code.

Time-codes are only propagated on ports which are enabled to send time-codes as defined by the time-code control register. Note that of the external USB ports, only port 3, corresponding to channel 0, can currently be used to send or receive time-codes.

The router has an internal time-code generation master, which can be controlled by the SpaceWire router configuration port to generate time-codes at a user defined rate.

An overview of the SpaceWire router time-code interface is shown below.

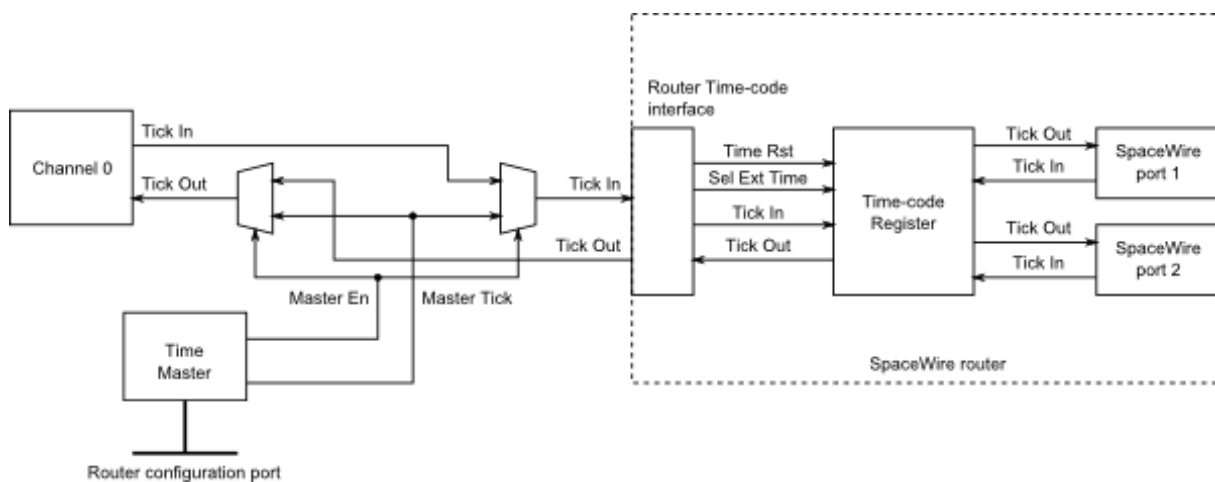


Figure 3.38: Time-code processing architecture

3.5.5.6 Packet Timestamping

The SpaceWire Brick Mk3 provides an optional timestamp feature for each received packet. A timestamp will be recorded when the first byte of a packet is received and when the end of packet marker is received. The timestamp is synchronised with an external pulse input on SMB trigger A.

Multiple SpaceWire Brick Mk3 devices can be synchronised using the external pulse external time source to synchronise time across the devices.

3.5.5.6.1 Detailed Description

An overview of the timestamp functions provided in by the SpaceWire Brick Mk3 are shown in the figure below. Configuration settings are shown in *italics*.

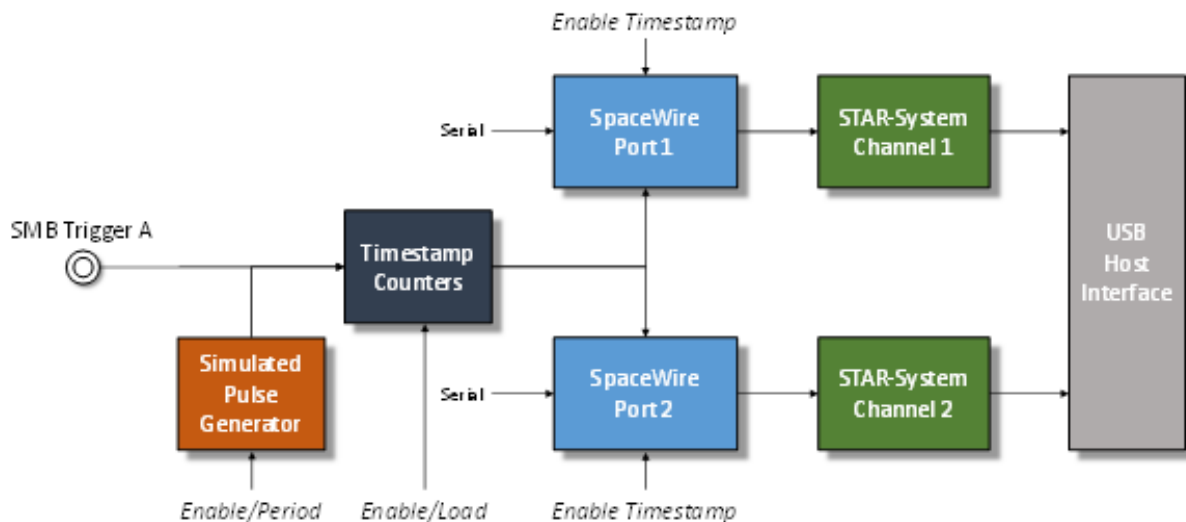


Figure 3.39: Timestamp system overview

Each SpaceWire port has a SpaceWire receiver which decodes serial data into link characters and data characters. Each data character can be an 8-bit packet data byte or an end of packet marker. The timestamping mechanism will record the timestamp for each start of packet byte and for each end of packet marker, effectively the start time and end time of each packet.

The timestamp counter is implemented as two counters, one which increments on the input SMB trigger or the pulse generator and one which increments on the system clock. The counters are identified as "SyncPulse" and "ClockCycles" respectively.

The packet start and end times are recorded using a combination of the counters described above, plus a capture register which captures the total number of clock cycles which elapsed in the previous period. An overview of the counter and register operation is listed below.

- The "SyncPulse" counter can be loaded by software to provide a starting value for the timestamp. The counter will increment on each trigger pulse.
- The "ClockCycles" counter is incremented on each system clock cycle and is reset on each pulse of the trigger. It provides a count of the system clock cycles which have elapsed since the last synchronisation pulse.
- The "TotalCycles" register is captured into a storage register on each synchronisation pulse to provide a mechanism to convert the "ClockCycles" elapsed time into fractions of the synchronisation period, i.e. $\frac{\text{ClockCycles}}{\text{TotalCycles}}$ is the fraction between 0 and 1 of the synchronisation period at which the event occurred.

The timestamp is synchronised using an external trigger pulse which supports frequencies equal to or greater than 1 Hz. The mechanism provides additional meta-data for each received SpaceWire packet which can be received by applications to compute the time at which the packet was decoded by the SpaceWire receiver.

A simulated pulse generator module can be used to simulate the external SMB trigger for testing and development purposes. The generator is programmed with the period between pulses in system clock cycles, and can be enabled or disabled.

3.5.5.6.2 Computing the "User" time from the recorded Timestamp

A method to convert the timestamp fields into "User" time is listed below. The raw timestamp is returned by the API therefore user applications can implement a different algorithm if required.

The synchronisation pulse frequency is set by the user application. In the equations below the frequency is referred to as F_{sync} .

The computed time is defined as $x.y$ where x is the number of seconds and y indicates the fraction of seconds between 0 and 1.

The example steps to compute the "User" time are listed below.

1. The first step is to convert the hardware format into seconds and sub seconds as shown below.

$$Seconds = SyncPulse / F_{sync}$$

$$SubSeconds = ClockCycles / (TotalCycles * F_{sync})$$

2. The resultant values are then added together.

$$x.y = Seconds + SubSeconds$$

An example case is listed below.

1. The application setup is listed below:

- (a) F_{sync} is 10 Hz
- (b) "SyncPulse" counter is 1234
- (c) "ClockCycles" counter is 234567
- (d) "TotalCycles" counter is 301210

2. The resultant time in seconds and fractions of seconds is computed as:

$$\begin{aligned} Seconds &= 1234/10 \\ &= 123.4 \end{aligned}$$

$$\begin{aligned} SubSeconds &= \frac{234567}{301210 * F_{sync}} \\ &= 77.8749045516E - 3 \end{aligned}$$

$$x.y = 123.477874905seconds$$

3.5.6 Electrical Characteristics

3.5.6.1 SpaceWire Connectors

The SpaceWire interface **absolute maximum** input ratings are defined in the table below. The device can withstand these maximum values but use at or near these values is not recommended.

		Description	V _{MIN}	V _{MAX}
V _{IN}	Data/Strobe Out	Input voltage relative to GND	-0.5	3.8
	Data/Strobe In	Input voltage relative to GND	-5	6
	Triggers	Voltage applied to triggers	-0.5	6

Additional clamping diodes on all SpaceWire signals will limit the voltage applied to these signals between -0.5V and 3V.

Additional clamping diodes on all trigger signals will limit the voltage applied to these signals between -0.5V and 5V.

The SpaceWire connector LVDS DC characteristics are listed in the table below.

	V_{ID} mV, Min	mV, Max	V_{ICM} V, Min	V, Max	V_{OD} mV, Min	mV, Typ	mV, Max	V_{OCM} V, Min	V, Typ	V, Max
Din, Sin	100	300	-4	5						
Dout, Sout					250	310	450	1.125	1.17	1.375

V_{ID}	Input differential voltage
V_{ICM}	Input common mode voltage
V_{OD}	Output differential voltage
V_{OCM}	Output common mode voltage

The common mode and differential identifiers are described in the diagram below.

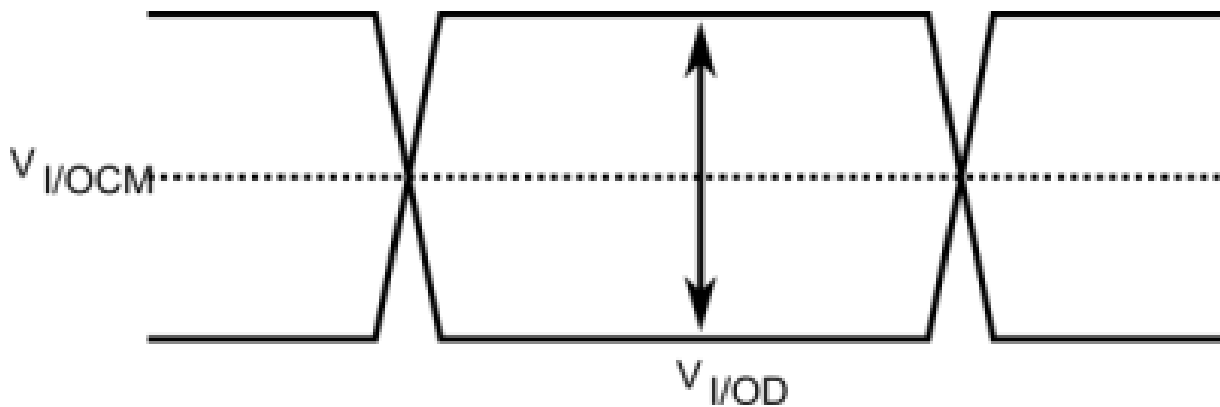


Figure 3.40: SpaceWire LVDS electrical specifications

3.5.6.2 External Trigger Connectors

The DC characteristics of the external trigger are listed in the table below. The output is driven by 3.3 Volt LVCMOS logic.

	V_{IL} V, Min	V, Max	V_{IH} V, Min	V, Max	V_{OL} V, Max	V_{OH} V, Min
Trigger IN	0	1.1	1.9	5.5		
Trigger OUT					0.55	2.3

V_{IL}	Input low voltage
V_{IH}	Input high voltage
V_{OL}	Output low voltage
V_{OH}	Output high voltage

3.5.6.3 FMECA Protection

A FMECA analysis report for the Brick Mk3 is available on request.

3.5.7 Switching Characteristics

The SpaceWire interface data/strobe input switching characteristics are defined in the figure and table below.

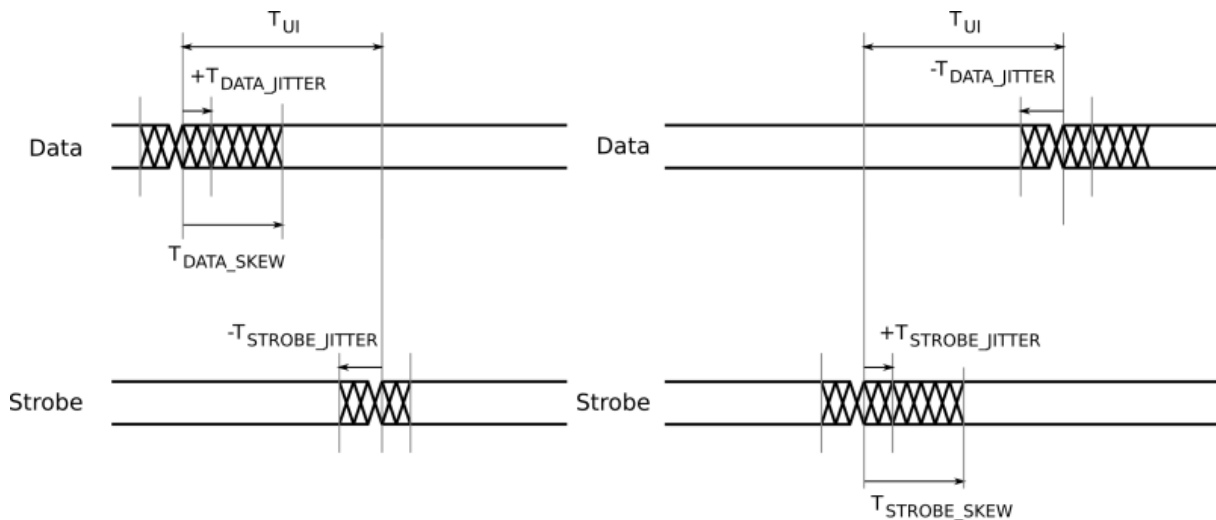


Figure 3.41: SpaceWire input switching characteristics

	Description	ns (Max)
T_{UI}	Transmitter bit rate.	3.333
$T_{DATA_SKEW_IN} (MAX)$	Maximum data skew which can be tolerated by the receiver.	$2 \cdot T_{DATA_JITTER_IN} - T_{STROBE_JITTER_IN}$
$T_{STROBE_SKEW_IN} (MAX)$	Maximum strobe skew which can be tolerated by the receiver.	$2 \cdot T_{DATA_JITTER_IN} - T_{STROBE_JITTER_IN}$

The data/strobe output switching characteristics are defined in the table below.

	Description	ns (Max)
T_{SKEW_OUT}	Maximum data or strobe skew.	0
T_{JITTER_OUT}	Maximum data/strobe jitter.	0.7 (Peak to Peak)

3.5.8 Grounding Information

When using LVDS it is very important that the grounds of the equipment at the two ends of each SpaceWire link are connected. Otherwise a significant common mode voltage can exist between the two units resulting in the maximum input voltage at one end or the other being exceeded. This could then result in damage to either unit.

The SpaceWire outer shield is connected to the backshell of the connectors at either end of a SpaceWire cable and when screwed into the mating SpaceWire connector on a unit ought to make a connection to the chassis and ground plane of that unit. Properly connecting a SpaceWire cable would then suffice to make the required ground connection between the two units being connected by SpaceWire.

STAR-Dundee equipment always has a low impedance connection between the backshell of the connector, the chassis and ground plane of the electronics in the unit. STAR-Dundee cables have the outer shield connected to the backshell of the connectors at either end of the cable.

When testing a new SpaceWire unit, it is recommended that you check that there is a low impedance connection between the backshell of the connector on your unit under test (UUT) and the chassis and ground plane in the UUT. If this is not low impedance then that problem should be resolved before connecting any SpaceWire equipment to the UUT. If there is a low impedance connection then please check the cable you are using and that there is a low impedance connection between the backshells of the connector at either end of the cable.

3.5.9 EMC Conformity

Declaration of Conformity

We

STAR-Dundee Ltd
STAR House, 166 Nethergate, Dundee, DD1 4EE, Scotland, UK

declare under our sole responsibility that the product

SpaceWire Brick Mk3

to which this declaration relates is in conformity with the following standard(s) or other normative documents

EN55022:2010
EN55024:2010

following the provisions of the EMC Directive.

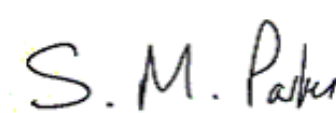
Dundee, 19th February 2015	
(place and date of issue)	(name and signature of authorised person)

Figure 3.42: S.M. Parkes

This is a class B product, and may therefore be used in a domestic environment.

3.6 SpaceWire PXI Interface and PXI Interface Mk2

Documentation for the STAR-Dundee SpaceWire PXI and PXI Mk2, Interface and Interface with RMAP Target devices is provided in this section.

3.6.1 Overview

This document provides an overview of the interfaces and features of the SpaceWire PXI and PXI Mk2, Interface card and Interface card with RMAP Target card.

The SpaceWire PXI card is a versatile SpaceWire Interface, Router or optional RMAP target device. The card is designed to be an efficient host interface to a SpaceWire system or network, by utilising a cPCI or PXI backplane.

The features of the SpaceWire PXI Interface card types are listed below.

- **SpaceWire PXI Interface:** 4 port SpaceWire Interface device with 4 data channels and 1 control channel for direct access to the SpaceWire ports through the PXI interface.
- **SpaceWire PXI Interface with RMAP Target:** 4 port SpaceWire Interface device with 4 embedded configurable Remote Memory Access Protocol (RMAP) targets and 1 GByte of DDR3 memory.

The Interface and RMAP Target cards can take advantage of the trigger connections on the PXI card front panel to sequence events in the card such as the transmission of time-codes or data packets. The interface card has four general purpose SMB trigger connectors which can be used to connect to the PXI backplane, sequence data transfers and events, and indicate events to an external device such as an oscilloscope.

The RMAP Target card has an option to directly connect up to four configurable RMAP targets to the SpaceWire interfaces to allow the card to act as a test card for remote memory transfers. Each RMAP target can be controlled and configured independently, and has access of up to 1 GByte of on board DDR3 memory.

Note that an upgrade is available for the Interface to include the RMAP Target functionality. Please contact STAR-Dundee for more information.

3.6.2 Hardware Description

The SpaceWire PXI card provides a cPCI or PXI system with four SpaceWire interfaces. Efficient host software is provided to support rapid sending and receiving of SpaceWire packets into host PC memory.

The SpaceWire PXI card is shown in the figure below.

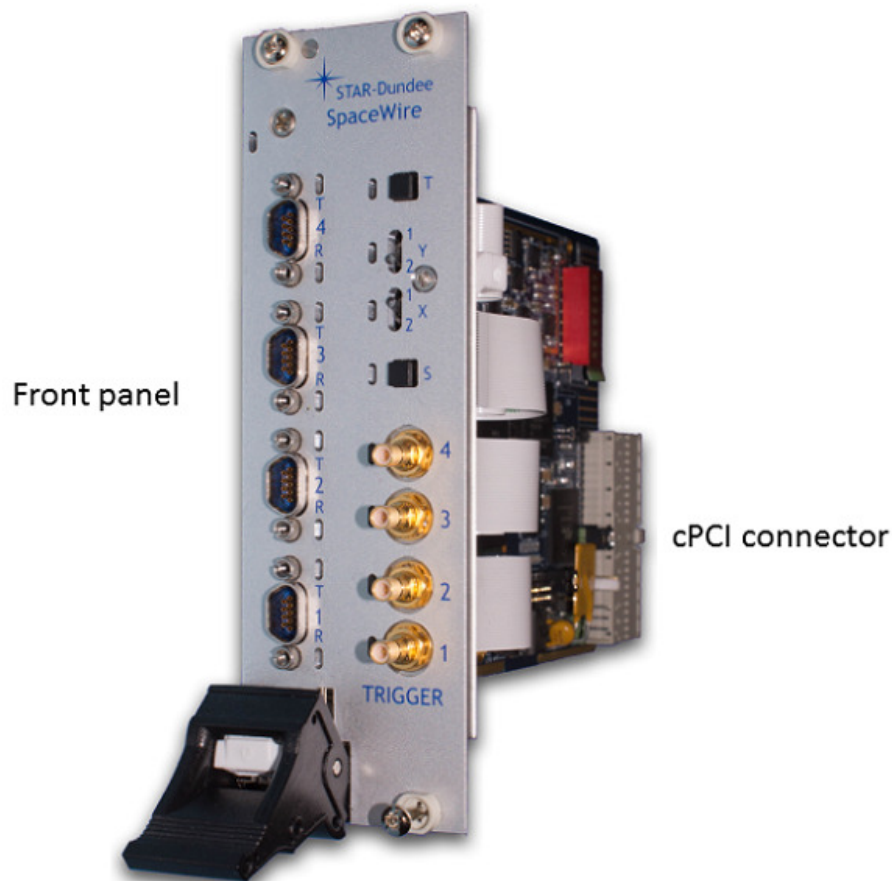


Figure 3.43: SpaceWire PXI Interface card

The card is comprised of a front panel where the SpaceWire and trigger interfaces are located, and a rear cPCI connector for insertion into a cPCI, PXI or PXIe rack - see the [Supported Systems](#) section.

The following sections describe the features of both cards.

3.6.2.1 SpaceWire PXI Interface Card

An overview of the SpaceWire PXI Interface card is shown in the figure below.

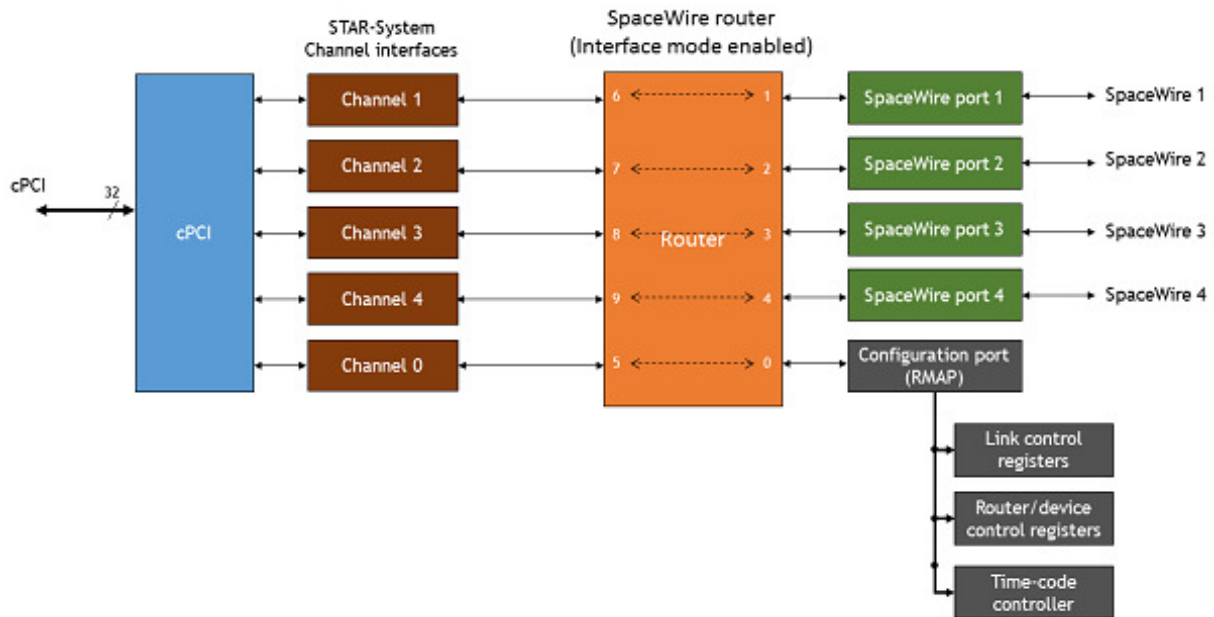


Figure 3.44: SpaceWire PXI Interface hardware overview

The four SpaceWire interfaces of the SpaceWire PXI Interface card are fully compliant with the SpaceWire standard and operate at up to 200 Mbit/s. They are connected to a SpaceWire router which supports two modes of operation, interface mode and router mode, where interface mode is the default mode. In Router mode, packets from any SpaceWire port can be routed to any other SpaceWire port, or the cPCI interface. In interface mode, the SpaceWire router is passive and packets which are received on a SpaceWire port are passed directly to the USB data channel for that port.

There are four independent channels from the SpaceWire router to the cPCI interface, so traffic flowing over one port cannot block traffic for another port. In addition, there is a separate control channel, so that the host PC is always able to access the control, configuration and status space of the PXI card regardless of the data flow.

The SpaceWire PXI card includes support for fault injection including support for parity errors, credit errors, escape errors, data corruption and EEP termination of packets.

3.6.2.2 SpaceWire PXI Interface with RMAP Target

An overview of the functional blocks in the SpaceWire interface/RMAP card are shown in the figure below.

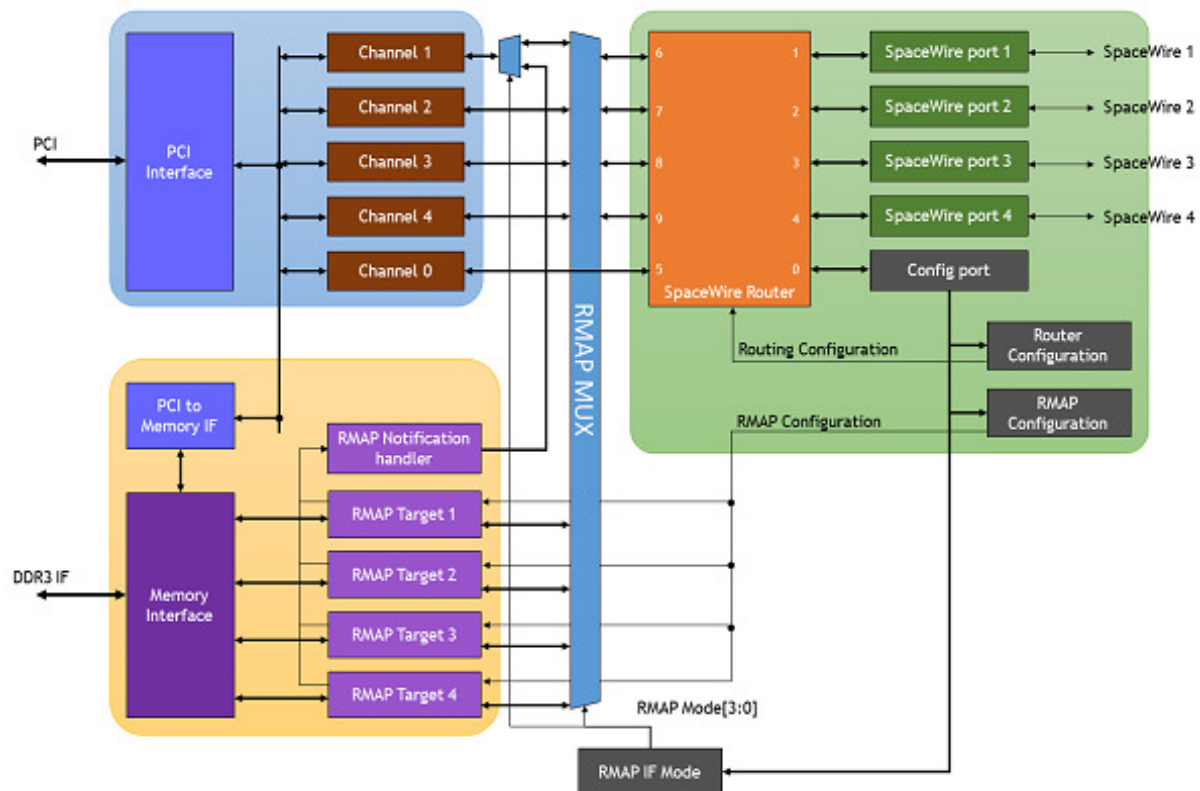


Figure 3.45: SpaceWire PXI Interface with RMAP Target hardware overview

The cPCI interface supports five channel interfaces, one of which is used by the STAR-System API for configuration purposes. The other channels are available for data transfer between the SpaceWire packets and application software dependent on the RMAP Mode setting. The SpaceWire ports and cPCI channel interfaces operate in a similar way to the PXI Interface card, listed in the section above.

The RMAP system comprises four RMAP targets, a notification handler and an access port to 1 GByte of on-board memory.

The SpaceWire interfaces and Router are configured using the STAR-System configuration API over the configuration port of the Router. The RMAP targets are also configured in the same way and are supported by the STAR-System **RMAP Target API**.

The RMAP IF Mode configuration bits control the RMAP MUX multiplexer and the RMAP Notification multiplexer. The configuration is identified as "RMAP Mode" in each of the following sections.

The SpaceWire PXI card includes support for fault injection including support for parity errors, credit errors, escape errors, data corruption and EEP termination of packets.

3.6.3 Getting Started

The SpaceWire PXI card is powered over a cPCI or PXI backplane; therefore no external power supply is required. Correct anti-static discharge procedures should be observed when inserting the PXI card into the rack to ensure that no damage occurs to the components on the printed circuit board. Once the board is fully seated in the rack and the ejector handle is closed, it should be screwed in place using the four securing screws before use.

The SpaceWire PXI software and drivers, provided by STAR-System, should be installed on the host computer before connecting the SpaceWire PXI card. This ensures that the device is recognised by the host when it is

connected.

This document describes the host interface connections which are supported by the SpaceWire PXI card.

3.6.3.1 Front Panel Interfaces

The front panel interfaces of the SpaceWire PXI card are shown in the figure below.



Figure 3.46: SpaceWire PXI card front panel interfaces and LED positions

3.6.3.2 SpaceWire Interfaces

The SpaceWire PXI card has two multicolour LEDs for each SpW port. The lower LED on each port shows received data for that port, and the higher LED on each port shows transmitted data for that port. The functions of the SpaceWire port LEDs are defined in the table below.

Function	LED Colour	Description
No event	Amber	The left and right hand LEDs are steady amber. Default state of the port, no activity and no NULLs transferred.

Link running	Green	The left and right hand LEDs are turned green when the link is running. The link running LEDs are held for approximately half a second after the link has stopped running and no error occurs.
Error	Red	The left and right hand LEDs are turned red if an error is detected when the port is in the Run state.
Data Transmitted/Received	Blue	When data is received on the link, the left hand LED turns blue. When data is transmitted on the link, the right hand LED turns blue.
No Credit	White	When no credit is available on the link for the device to transmit for a period of time, the right hand LED turns white. When the device has provided no credit on the link for the device at the other end of the link to transmit for a period of time, the left hand LED turns white.
Identify	Flashing purple	The SpaceWire PXI devices can be forced to flash their front panel LEDs to allow the device to be easily identified among a group of other devices. When this function is used the LEDs will flash purple for a short period of time.

Please note that there is an issue with all SpaceWire interfaces where a small amount of noise on the inputs can be translated by the LVDS receivers into a digital pattern and then interpreted by the receiver as a bit sequence. Since this bit sequence is invalid it causes the SpaceWire CODEC to start up and then disconnect. The SpaceWire interface LEDs go red on disconnect (ErrorReset state) and amber when ready (Ready state). So, if you get a burst of noise on the input then you will see a red flash of around half a second on the LEDs. To avoid this happening all the time we have a bias resistor network on the input of the LVDS receivers that provides a small bias current. This filters out most of the noise, but occasionally depending on the environment there may still be enough noise to trigger the reset to ready cycle and you see this as a red flash on the LEDs. This is the expected/normal operation. If you have any concerns, please contact support@star-dundee.com.

3.6.3.3 Trigger Interfaces

The SpaceWire PXI card has four bi-directional trigger SMB trigger interfaces which can be used to trigger actions such as sending a SpaceWire packet or indicate activity by providing a trigger output pulse. The triggers are 5V tolerant and the output trigger pulse can be configured using the STAR-System [Triggering API](#).

The trigger LEDs are set dependent on the trigger direction and the state of the trigger. When the trigger is an input the LED indicates if the trigger is enabled and when an external trigger occurs. When the trigger is an output the LED indicates if the trigger is armed and when the trigger is set. The LED colour map for the trigger LEDs is listed in the table below.

Function	LED Colour	Description
Not active	Amber	Default state of the LED. The trigger is not enabled or armed.
Armed/Enabled	Green	Set to green when the trigger is an input and is enabled or when the trigger is an output and the trigger is armed.

Trigger	Blue	Set to blue when an input or output trigger occurs and the trigger is armed or enabled.
---------	------	---

3.6.4 Supported Systems

The cPCI and PXI specifications which are referenced in this section are listed in the table below.

Reference	Document	Description
[1]	PICMG 2.0 D3.0 CompactPCI Specification - 1999-10-01	cPCI Specification
[2]	PXI Hardware Specification - Revision 2.1 February 4, 2003	PXI Specification
[3]	PXI Express Hardware Specification - Revision 1.0 August 22, 2005	PXIe Specification

The SpaceWire PXI card is a standard 3U cPCI/PXI card which uses the 32-bit form factor defined in those standards. The card dimensions are listed below.

- Height: 100mm
- Length: 160 mm

An overview of the 3U 32-bit format PXI card is shown below and defined in Figure 2.1 of the PXI specification [2].

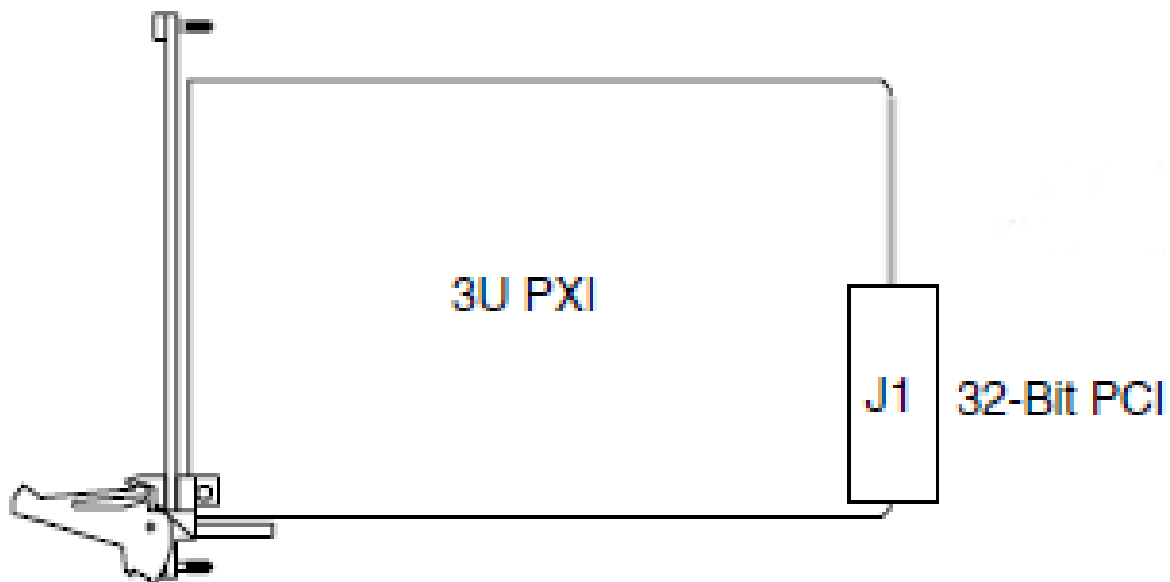


Figure 3.47: 3U PXI card form factor

3.6.4.1 cPCI Compatibility

The SpaceWire PXI card is compatible with all cPCI Peripheral slots defined in section 2.1 of the cPCI specification [1].

Peripheral slots are identified using a circle as shown in the reproduced figure below.

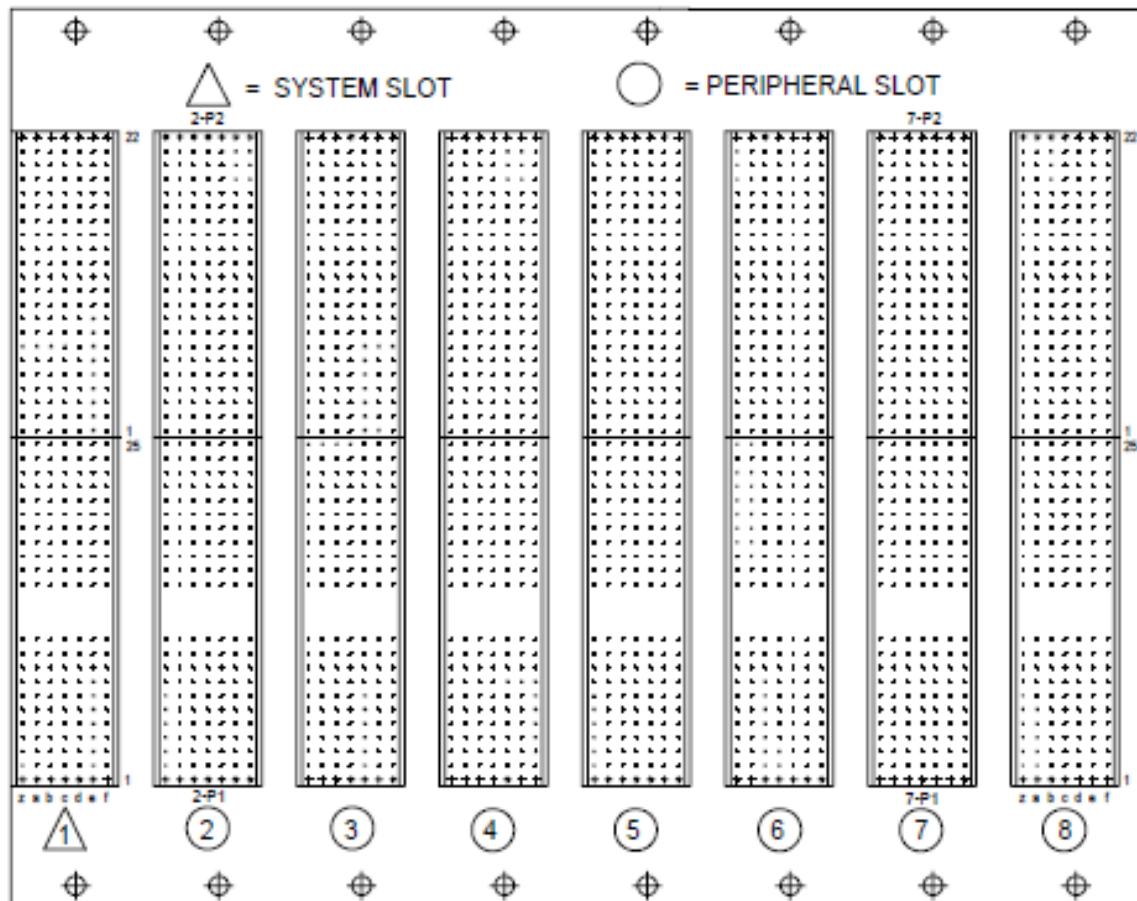


Figure 3.48: cPCI peripheral slots

3.6.4.2 PXI Compatibility

The SpaceWire PXI card is compatible with all PXI Peripheral slots defined in section 2.1 of the PXI specification [2].

Peripheral slots are identified using a circle as shown in the reproduced figure below.

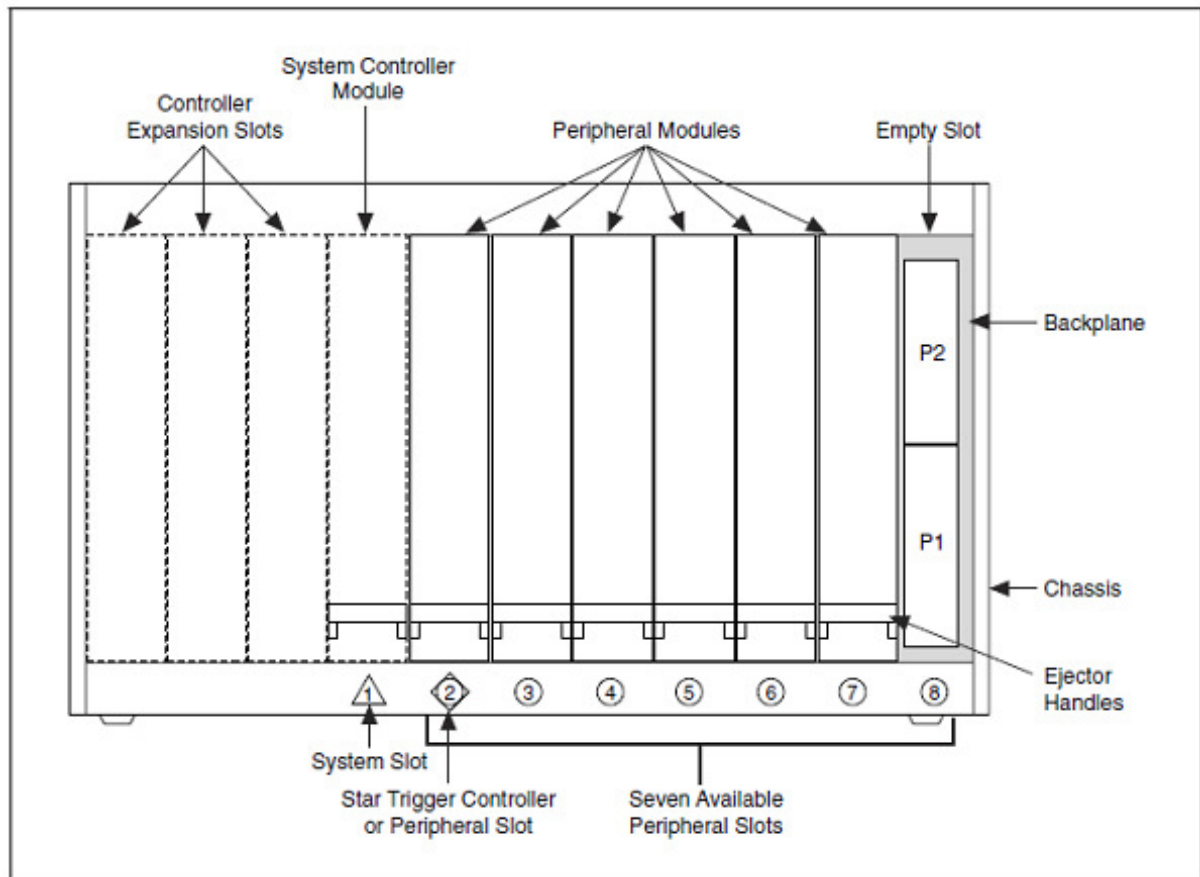


Figure 3.49: PXI peripheral slots

3.6.4.3 PXIe Compatibility

The SpaceWire PXI card is compatible with all PXIe PXI-1 and PXI Hybrid slots defined in section 2.1.1.5 and 2.1.1.6 of the PXIe specification [3].

PXI-1 slots are identified using a circle as shown in the reproduced figure below.

PXI Express Hybrid slots are identified using a filled circle with a capital H outside the circle, as shown in the reproduced figure below.

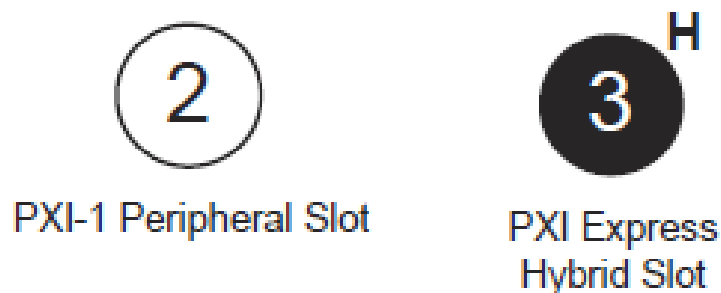


Figure 3.50: Compatible PXIe peripheral slots

3.6.5 Router and SpaceWire Interfaces

The SpaceWire router inside this unit is compatible with the AT7910E radiation tolerant router (also known as the SpW-10X) manufactured by Atmel. The SpaceWire router can be configured using the STAR-System device configuration API functions.

The SpaceWire router has 10 ports in total as listed below:

- 1 internal router configuration port
- 4 external SpaceWire ports
- 5 cPCI channel interface external ports. By default four of these are used by the SpaceWire ports, while the other is used by the configuration port

The SpaceWire router supports two modes of operation, interface mode and routing mode.

The default mode is interface mode. The router will revert to interface mode after power cycling or a device reset is performed.

3.6.5.1 Interface Mode

In interface mode, a packet which is received on a SpaceWire port will always be routed by the port routing configuration. The SpaceWire PXI card supports interface mode by utilising the port routing settings of the SpaceWire router. The default values are listed in the table below.

Source port	Destination port	Description
0	5	Configuration to external port 1 (host channel 0)
1	6	SpaceWire port 1 to external port 2 (host channel 1)
2	7	SpaceWire port 2 to external port 3 (host channel 2)
3	8	SpaceWire port 3 to external port 4 (host channel 3)
4	9	SpaceWire port 4 to external port 5 (host channel 4)
5	0	cPCI channel 0 to configuration port
6	1	cPCI channel 1 to SpaceWire port 1
7	2	cPCI channel 2 to SpaceWire port 2
8	3	cPCI channel 3 to SpaceWire port 3
9	4	cPCI channel 4 to SpaceWire port 4

3.6.5.2 Routing Mode

In routing mode packets are routed dependent on the first byte of each SpaceWire packet. The routing algorithm is described below.

- Packets with known physical addresses are routed to the port indicated by the physical address at the head of the packet, i.e. a packet with address 0 is routed to the configuration port.

- Packets with unknown physical addresses, 10 to 31, are discarded and an address error is set in the source port register.
- Packets with logical address headers which are valid are routed to one of the set ports in the logical address lookup table (accessed via the configuration port).
- Packets with logical address headers which are marked as invalid are discarded and an address error is set in the source port register.

3.6.5.3 Link Operating Speed

The SpaceWire link operating speed for each SpaceWire port is controlled by the STAR-System device configuration APIs. The link clock rate selection logic for each port is shown in the figure below.



Figure 3.51: SpaceWire link operating speed for each SpaceWire port

- Each SpaceWire link can be programmed to a link rate between 2 and 200 Mbit/s using the transmit clock configuration functions of the STAR-System device configuration APIs.
- The link operating speed after power on is 200 Mbit/s.
- Each SpaceWire link will operate at 10 Mbit/s at start-up.
- When the reconfigurable clock generator is reprogrammed the link rate will switch to 10 Mbit/s until the programming has completed, when the link is running.

3.6.5.4 Time-codes

A time-code can be received from the cPCI interface or from a SpaceWire link. When a time-code is received, its value is checked against the value of the internal time-code register in the SpaceWire router. If the received time-code is one more than the value of the time-code register then the time-code will be written to the time-code register and propagated to other ports, except the port that was the source of the time-code.

Time-codes are only propagated on ports which are enabled to send time-codes as defined by the time-code control register. Note that of the external cPCI ports, only port 5, corresponding to channel 0, can be used to send or receive time-codes.

The router has an internal time-code generation master, which can be controlled by the SpaceWire router configuration port to generate time-codes at a user defined rate.

3.6.5.5 Packet Timestamping

The SpaceWire PXI Interface provides an optional timestamp feature for each received packet. A timestamp will be recorded when the first byte of a packet is received and when the end of packet marker is received. The timestamp is synchronised with an external pulse input on SMB trigger A.

Multiple SpaceWire PXI Interface devices can be synchronised using the external pulse external time source to synchronise time across the devices.

3.6.5.5.1 Detailed Description

An overview of the timestamp functions provided by the SpaceWire PXI Interface are shown in the figure below. Configuration settings are shown in *italics*.

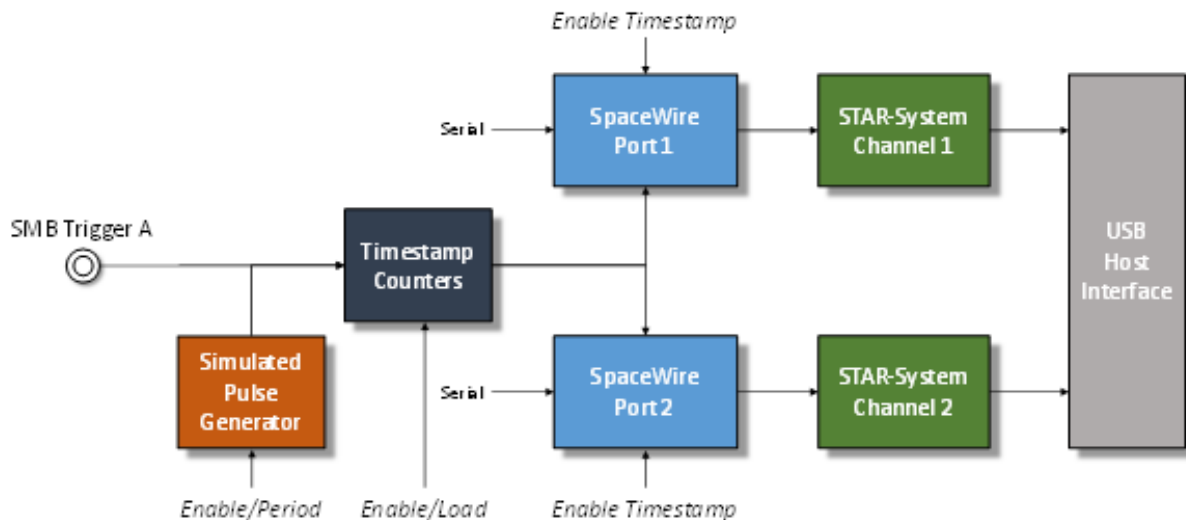


Figure 3.52: Timestamp system overview

Each SpaceWire port has a SpaceWire receiver which decodes serial data into link characters and data characters. Each data character can be an 8-bit packet data byte or an end of packet marker. The timestamping mechanism will record the timestamp for each start of packet byte and for each end of packet marker, effectively the start time and end time of each packet.

The timestamp counter is implemented as two counters, one which increments on the input SMB trigger or the pulse generator and one which increments on the system clock. The counters are identified as "SyncPulse" and "ClockCycles" respectively.

The packet start and end times are recorded using a combination of the counters described above, plus a capture register which captures the total number of clock cycles which elapsed in the previous period. An overview of the counter and register operation is listed below.

- The "SyncPulse" counter can be loaded by software to provide a starting value for the timestamp. The counter will increment on each trigger pulse.
- The "ClockCycles" counter is incremented on each system clock cycle and is reset on each pulse of the trigger. It provides a count of the system clock cycles which have elapsed since the last synchronisation pulse.
- The "TotalCycles" register is captured into a storage register on each synchronisation pulse to provide a mechanism to convert the "ClockCycles" elapsed time into fractions of the synchronisation period, i.e. $\frac{\text{ClockCycles}}{\text{TotalCycles}}$ is the fraction between 0 and 1 of the synchronisation period at which the event occurred.

The timestamp is synchronised using an external trigger pulse which supports frequencies equal to or greater than 1 Hz. The mechanism provides additional meta-data for each received SpaceWire packet which can be received by applications to compute the time at which the packet was decoded by the SpaceWire receiver.

A simulated pulse generator module can be used to simulate the external SMB trigger for testing and development purposes. The generator is programmed with the period between pulses in system clock cycles, and can be enabled

or disabled.

3.6.5.5.2 Computing the "User" time from the recorded Timestamp

A method to convert the timestamp fields into "User" time is listed below. The raw timestamp is returned by the API therefore user applications can implement a different algorithm if required.

The synchronisation pulse frequency is set by the user application. In the equations below the frequency is referred to as F_{sync} .

The computed time is defined as x.y where x is the number of seconds and y indicates the fraction of seconds between 0 and 1.

The example steps to compute the "User" time are listed below.

1. The first step is to convert the hardware format into seconds and sub seconds as shown below.

$$\begin{aligned} Seconds &= SyncPulse / F_{sync} \\ SubSeconds &= ClockCycles / (TotalCycles * F_{sync}) \end{aligned}$$

2. The resultant values are then added together.

$$x.y = Seconds + SubSeconds$$

An example case is listed below.

1. The application setup is listed below:

- (a) F_{sync} is 10 Hz
- (b) "SyncPulse" counter is 1234
- (c) "ClockCycles" counter is 234567
- (d) "TotalCycles" counter is 301210

2. The resultant time in seconds and fractions of seconds is computed as:

$$\begin{aligned} Seconds &= 1234 / 10 \\ &= 123.4 \\ SubSeconds &= \frac{234567}{301210 * F_{sync}} \\ &= 77.8749045516E - 3 \\ x.y &= 123.477874905seconds \end{aligned}$$

3.6.6 RMAP Target

The SpaceWire PXI Interface card with RMAP target has four embedded RMAP target ports which are compliant with the RMAP protocol standard document, ECSS-E-ST-50-52C.

Each RMAP target can be accessed through the SpaceWire ports or cPCI channel interfaces (with some restrictions) dependent on the configuration which is set through the configuration API. Each target can directly access up to 1 Gbyte of on-board DDR3 memory.

An overview of the RMAP functionality available for each of the RMAP targets is listed below and shown in the following figure.

- Decode RMAP command packets.
- Encode RMAP reply packets.
- Authorise commands using two modes of operation:
 - Notify the host when an RMAP command packet arrives. Wait for the host to authorise the command.
 - Authorise a command using a set of authorisation register which are configured by the host.
- Access memory directly or through a logical to physical address offset memory mapping.
 - Host notification can be enabled when the memory access has completed.

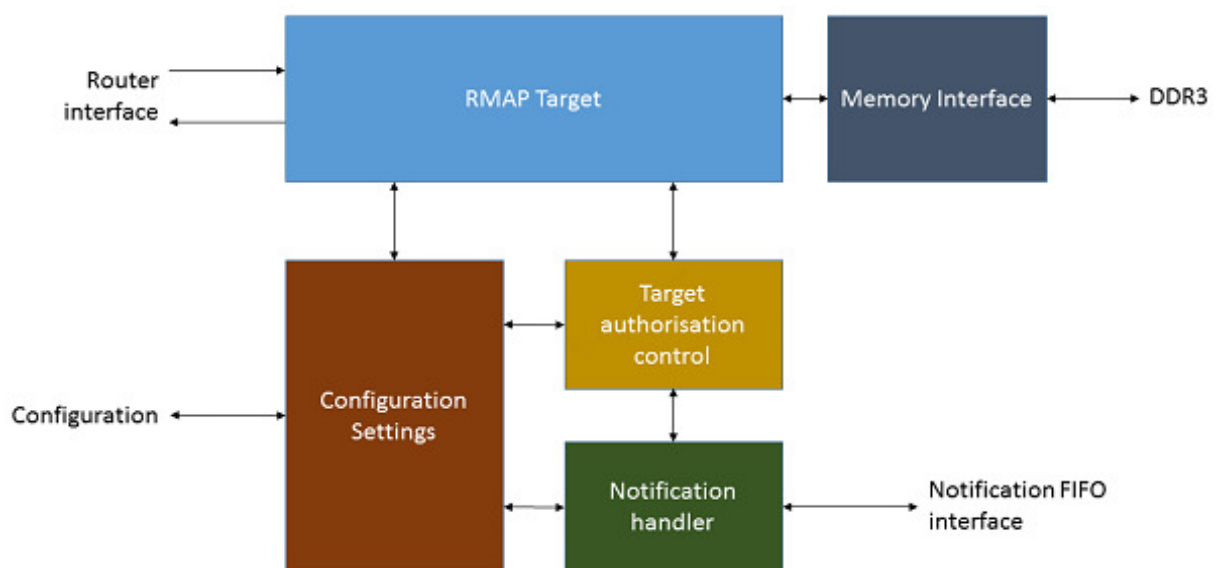


Figure 3.53: RMAP target architecture overview

3.6.6.1 Configuration and Access Modes

The configuration of the RMAP Target card is set using the STAR-System [RMAP Target API](#). The configuration items which determine the set-up of the RMAP card flow set-up is listed below.

Configuration	Default	Description
RMAP Mode[0]	Disabled	The RMAP mode flags change the settings of the RMAP MUX which determine if the RMAP target ports or the cPCI channel interfaces are connected to the SpaceWire Router ports. RMAP Mode[0] also controls the RMAP Notification multiplexer. When RMAP Mode[0] is enabled, any RMAP notifications will be sent to the STAR-System API using cPCI channel 1.
RMAP Mode[3:1]	Disabled	The RMAP mode flags change the settings of the RMAP MUX which determine if the RMAP target ports or the cPCI channel interfaces are connected to the SpaceWire Router ports.
Interface Mode	Enabled	The routing configuration of the SpaceWire router can be changed between Router Mode and Interface Mode. The default is Interface Mode. In Interface Mode, packets are routed dependent on a configurable port number and the leading address of the packet is not used. The default set-up connects SpaceWire port 1 to Router port 6, SpaceWire port 2 to Router port 7, and so on. The configuration port is connected to Router port 5. In Router Mode, packets are routed dependent on the leading packet address. Advanced routing and interface mode configuration set-ups are supported where some ports are configured in interface mode and have a fixed routing set-up, and some ports are configured to check the leading packet address to determine the packet destination.

RMAP Target Configuration	The RMAP Target configuration is set by the STAR-System RMAP Target API. Each target can be configured independently to access a different region of the 1 GByte memory space and respond to different logical addresses, commands, etc. The target configuration also sets the notification.
----------------------------------	---

3.6.6.2 Memory Access

The on board memory interface is accessible from the RMAP targets and cPCI interface. The functionality of the memory interface is listed below:

- Four RMAP target interfaces for remote memory access and simulation.
- cPCI interface which is shared with RMAP target 1.
- Each RMAP target can be assigned a region of memory using a physical memory OFFSET and a lower and upper limit to memory access. Wrapping memory accesses are not supported.

The physical memory location which is accessed by an RMAP target is defined by the target OFFSET register and the address field in the RMAP packet.

The *RMAPOFFSET* address is defined in 1Mbyte regions as shown below.

$$PHYSICALMEMORYADDRESS = (2^{20} \times RMAPOFFSET) + RMAPADDRESS$$

The physical memory location which is accessed by an RMAP target is defined by the local bus memory OFFSET register (*LOCALOFFSET*) and the PLX local bus address pins.

The *LOCALOFFSET* address is defined in 16Mbyte chunks.

$$PHYSICALMEMORYADDRESS = (2^{24} \times LOCALOFFSET) + LA$$

3.6.7 SpaceWire Interface Tristate Mode

The SpaceWire ports on the PXI card are comprised of four LVDS signal pairs, Data/Strobe in +/- and Data/Strobe out +/- . The Data/Strobe output pairs are connected to LVDS drivers which default to a high impedance Tristate mode after power on and after a device reset. The signal pairs are effectively disconnected when the tristate mode is enabled, providing protection for connected devices.

Each port includes a link monitoring component which can enable or disable the LVDS driver of a link dependent on a combination of the SpaceWire link settings, Start, Disabled or Tristate, and on the input bit pattern. The input bit pattern is monitored for a sequence of NULL characters without error to detect if a link should move out of Tristate mode and actively drive the Data/Strobe output signals.

The Tristate mode of each port is acted on by the following inputs.

- The default operating mode of all STAR-Dundee SpaceWire links is AutoStart. In the AutoStart mode the SpaceWire links remain in the Ready state and will wait for NULL characters before starting up.
If a link on the PXI card is in Tristate mode (LVDS drivers disabled) then the LinkEnabled check in the Ready state, **LinkEnabled=(Link Disabled=0 AND (LinkStart=1 OR (AutoStart=1 AND gotNULL=1))**, remains

LOW until 8 valid NULL characters have been successfully received without error, i.e. the state machine cannot move to Started until the LinkDisabled condition in the function is cleared.

Further information on the Ready state transition is shown in the figure below.

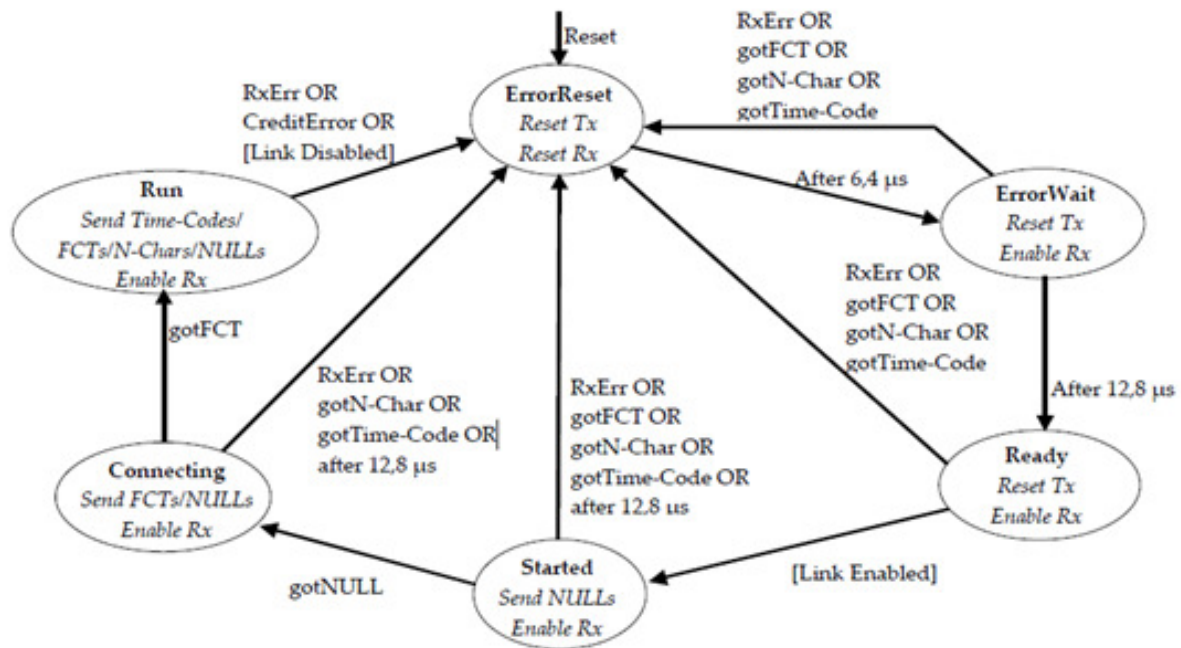


Figure 3.54: SpaceWire Interface state machine

- If the host software or network manager sets a link into LinkStart mode to force the link state machine to move from Ready to Started, or the Tristate mode enable bit is cleared, then the link will automatically move out of its Tristate mode and the SpaceWire port LVDS drivers will be enabled.
- If the host software or network manager sets the LinkDisabled mode to force the link into a disabled state, and the Tristate control bit is set, then the LVDS drivers of the corresponding SpaceWire port are disabled and are set to a high impedance state.

An overview of the Tristate mode settings and their effect on the link are shown in the state diagram below.

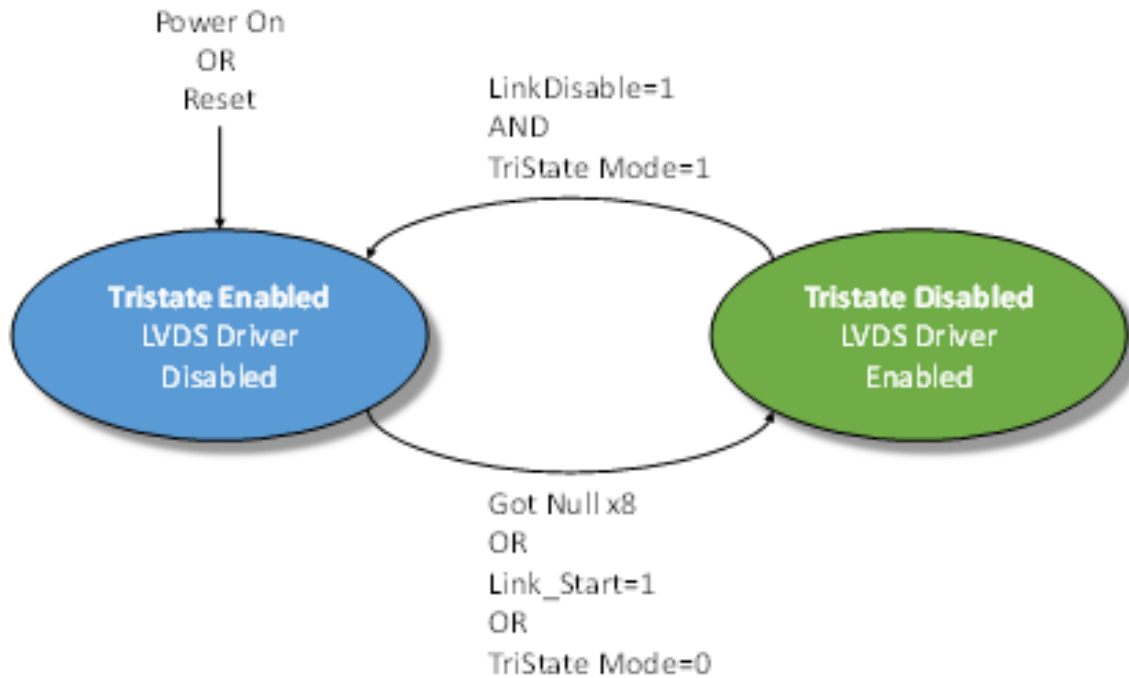


Figure 3.55: Tristate controller state machine

An approximate 60 microsecond delay occurs between the state machine moving from the Enabled to Disabled state, i.e. the LVDS drivers are not enabled until approximately 60 microseconds after the condition to enable the drivers has been detected.

A device which connects to the SpaceWire PXI card and sends NULLs can move through the SpaceWire link state machine a few times until the LVDS drivers are enabled. The exchange of silence period is 19.2 microseconds and a link which is enabled will send NULLs in the Started state for a period of 12.8 microseconds. The link will start normally after the 60 microsecond period – the user does not need to take any action to operate correctly with the PXI card in this mode.

An example start-up sequence when the PXI link is in AutoStart mode is shown in the figure below.

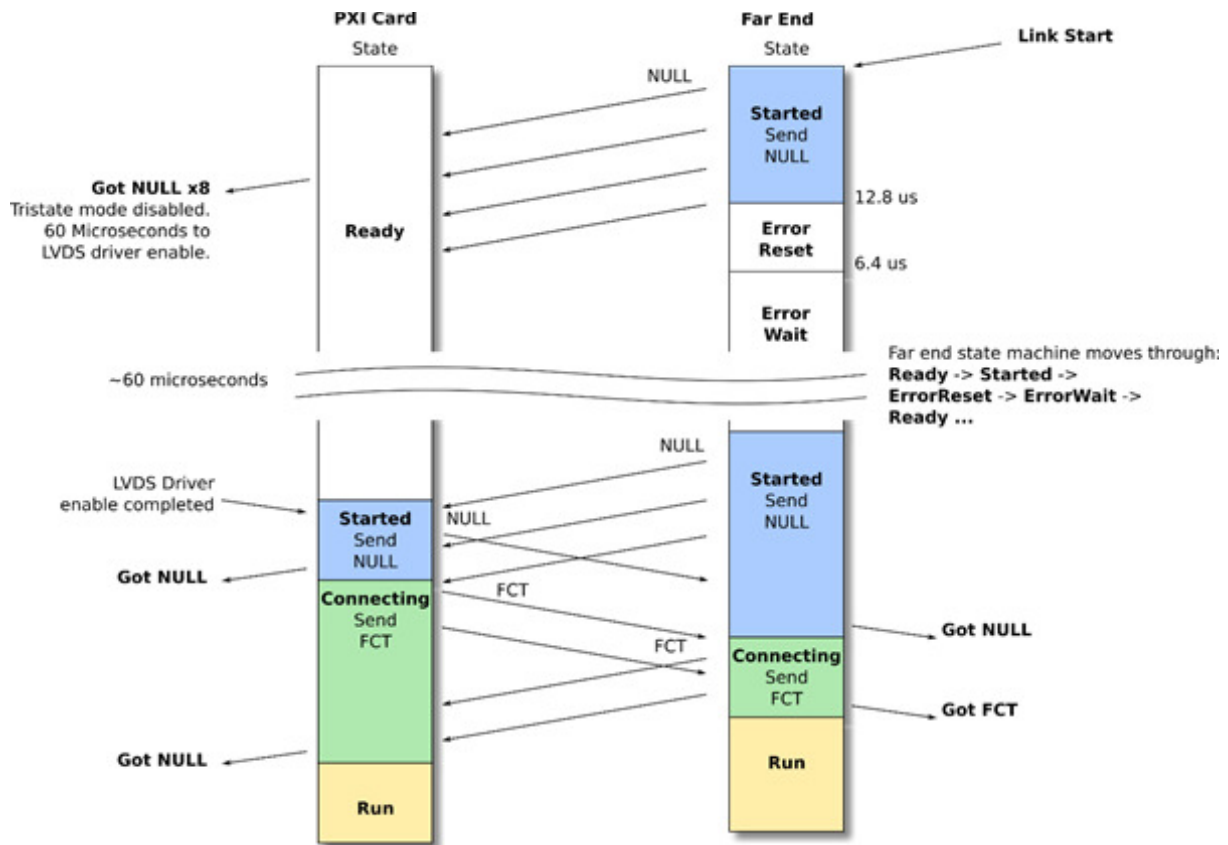


Figure 3.56: AutoStart link operation

In the figure above the Far End link state machine sends NULL characters after its Link Start control bit is set. The PXI card detects the NULL sequence and indicates to the Tristate controller state machine when 8 consecutive NULLs are received without error. The PXI card then enables its LVDS drivers after 60 microseconds and will remain in the Ready state until it detects a start-up NULL from the far end. Both ends then exchange NULL and FCT characters and start-up normally.

An example start-up sequence when the PXI link is set to start is shown below.

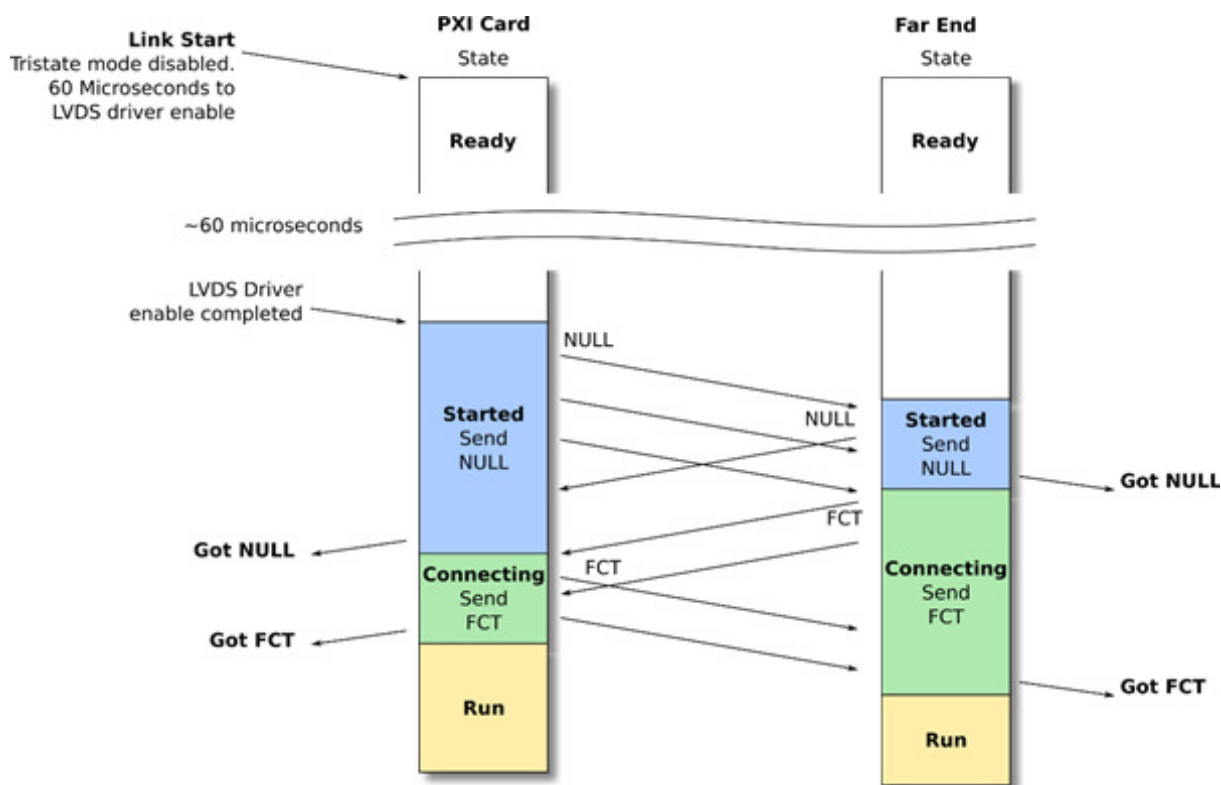


Figure 3.57: Link Start link operation

In the figure above the PXI Card Link Start bit is set and the LVDS driver enable sequence is started. After 60 microseconds the PXI card link will move to the Started state and will send NULL characters. The Far End will detect NULL characters, both ends will exchange NULLs and FCTs and will start-up normally.

Please note, if a SpaceWire Link should start-up on the first exchange of NULL characters then the Tristate setting for that link should be disabled.

3.6.8 Electrical Characteristics

3.6.8.1 Absolute Maximum Ratings

Parameter	Min	Max	Units
SpaceWire input voltage on Vin+, Vin- (see the figure in the following section)	-0.7	4.3	V
Differential input voltage on Vin+, - Vin- absolute value (see the figure in the following section)		1	V

External Trigger input voltage	-0.5	6	V
--------------------------------	------	---	---

3.6.8.2 SpaceWire Functional Diagram

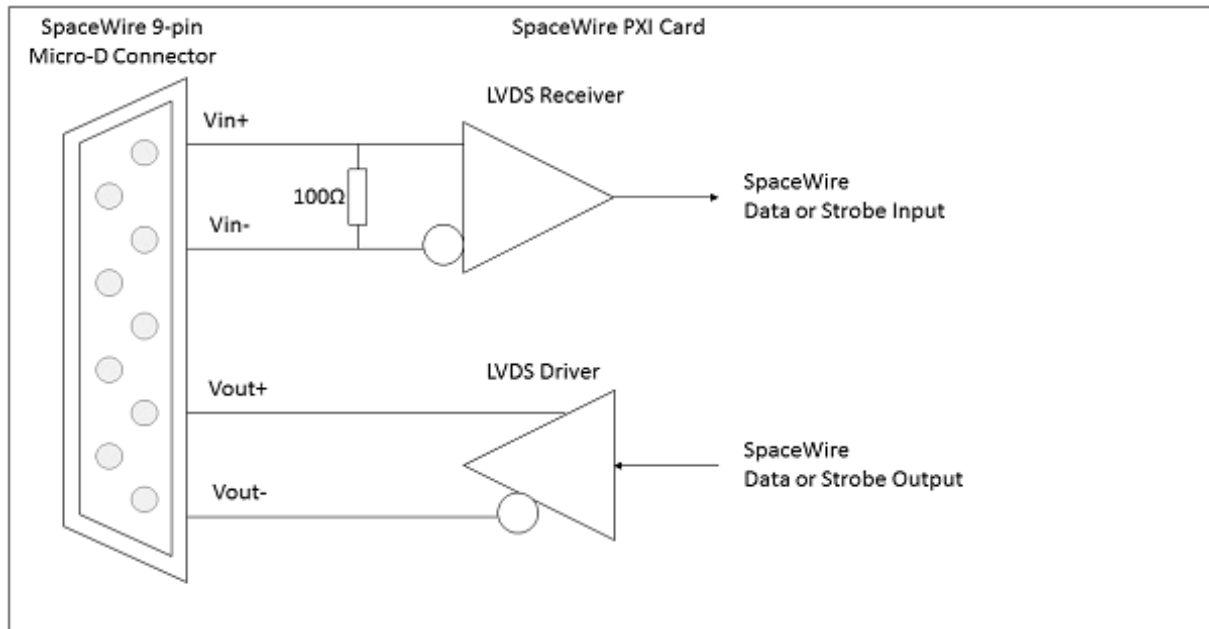


Figure 3.58: SpaceWire functional diagram (one LVDS driver/receiver shown)

Each SpaceWire port is comprised of a pair of LVDS drivers and LVDS receivers, one pair for data and strobe in, and one pair for data and strobe out.

3.6.8.2.1 SpaceWire Input Electrical Characteristics

The SpaceWire recommended input electrical characteristics are shown in the table below.

Parameter	Min	Max	Units
Differential Voltage range (Vin+ - Vin-, absolute value)	0.1	1	V
Common Mode +/- Differential voltage range (any combination of common-mode or input signals)	0	4	V
SpaceWire input Differential resistance	98	102	Ω

3.6.8.2.2 SpaceWire Output Electrical Characteristics

The SpaceWire output electrical characteristics are shown in the table below.

Parameter	Min	Typ (1)	Max	Units
SpaceWire output differential magnitude - 100 Ω load	247	310	454	mV

SpaceWire output common mode voltage - 100 Ω load		1.125		1.375	V
Output voltage to an unpowered unit	Vin+/Vin-	1.12	1.16	1.2	V
	Vout+/Vout- (Tri-state - Default at power on)	High Z	High Z	High Z	V
	Vout+/Vout- (Enabled - programmed by software)	0.898		1.602	V

(1) Typical conditions at 25 °C

Additional information

The STAR-Dundee PXI card can programmatically tristate its LVDS drivers.

3.6.8.3 External Trigger Functional Diagram

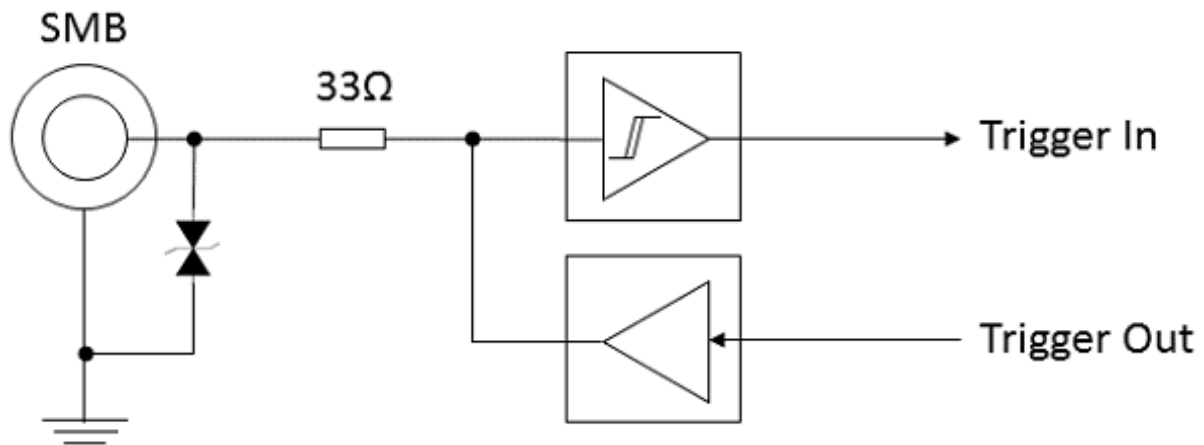


Figure 3.59: External trigger functional diagram

3.6.8.3.1 External Trigger Input Electrical Characteristics

The external trigger recommended input electrical characteristics are shown in the table below.

Parameter	Min	Max	Units
Logic 0 (low)	0	0.8	V
Logic 1 (high)	1.9	5.5	V

3.6.8.3.2 External Trigger Output Electrical Characteristics

The external trigger output electrical characteristics are shown in the table below. The output is driven by 3.3 Volt LVCMOS logic.

Parameter	Min	Max	Units
Logic 0 (low)		0.55	V

Logic 1 (high)	2.3		V
----------------	-----	--	---

3.6.9 Switching Characteristics

The SpaceWire interface data/strobe input switching characteristics are defined in the figure and table below.

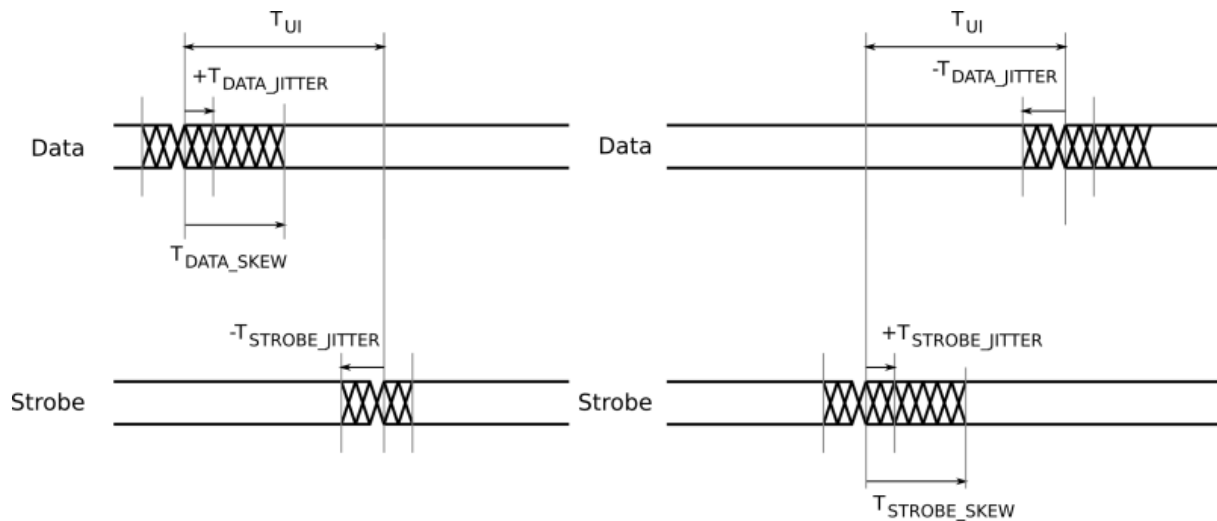


Figure 3.60: SpaceWire input switching characteristics

Parameter	Description	ns (Max)
T_{UI}	Transmitter bit rate.	5
$T_{DATA_SKEW_IN} (MAX)$	Maximum data skew which can be tolerated by the receiver.	$2 - T_{DATA_JITTER_IN} - T_{STROBE_JITTER_IN}$
$T_{STROBE_SKEW_IN} (MAX)$	Maximum strobe skew which can be tolerated by the receiver.	$2 - T_{DATA_JITTER_IN} - T_{STROBE_JITTER_IN}$

The data/strobe output switching characteristics are defined in the table below.

Parameter	Description	ns (Max)
T_{SKEW_OUT}	Maximum data or strobe skew.	0.1
T_{JITTER_OUT}	Maximum data/strobe jitter.	0.7 (Peak to Peak)

3.7 SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

Documentation for the STAR-Dundee SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2 card is provided in this section.

3.7.1 Overview

This document provides an overview of the interfaces and features of the SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2 card.

The SpaceWire PXI 12 Port Router card is a versatile SpaceWire Interface or Router device. The card is designed to be an efficient host interface to a SpaceWire system or network, by utilising a cPCI or PXI backplane.

3.7.2 Hardware Description

The SpaceWire PXI 12 Port Router card provides a cPCI or PXI system with 12 SpaceWire interfaces. Efficient host software is provided to support rapid sending and receiving of SpaceWire packets into host PC memory.

The SpaceWire PXI 12 Port Router card is shown in the figure below.

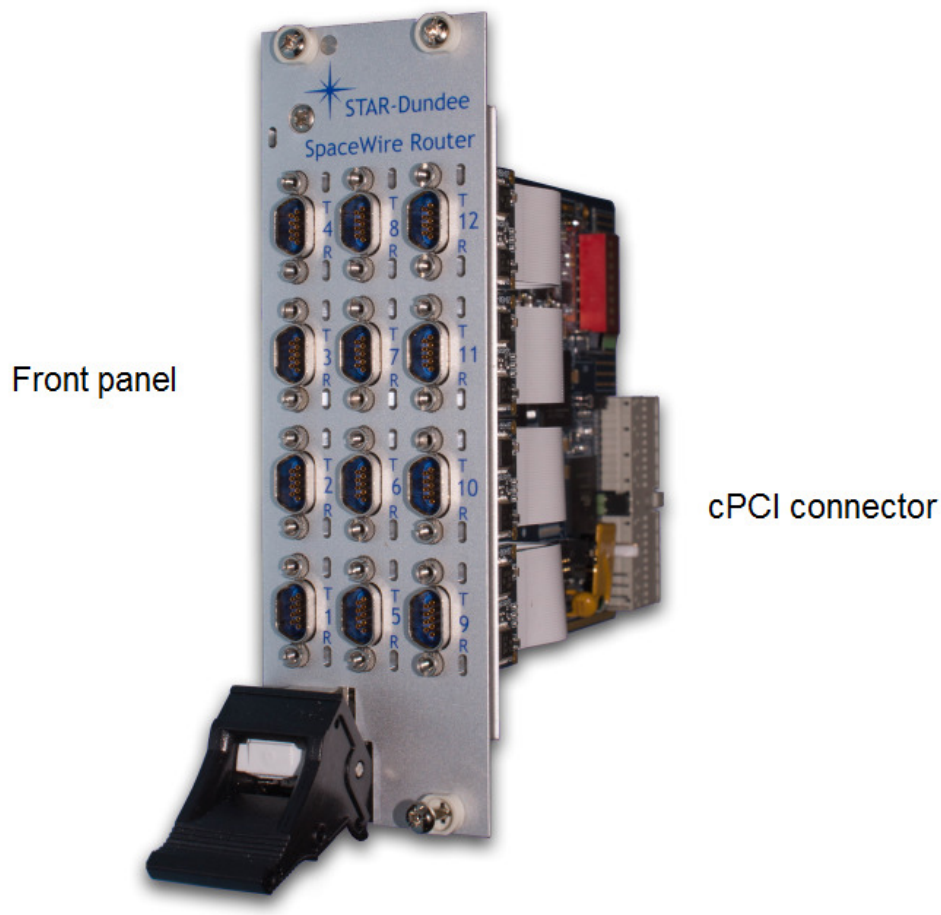


Figure 3.61: SpaceWire PXI 12 Port Router card

The card is comprised of a front panel where the SpaceWire interfaces are located, and a rear cPCI connector for insertion into a cPCI, PXI or PXIe rack - see the [Supported Systems](#) section.

The following section describes the features of the card.

3.7.2.1 SpaceWire SpaceWire PXI 12 Port Router Card

An overview of the SpaceWire PXI 12 Port Router card is shown in the figure below.

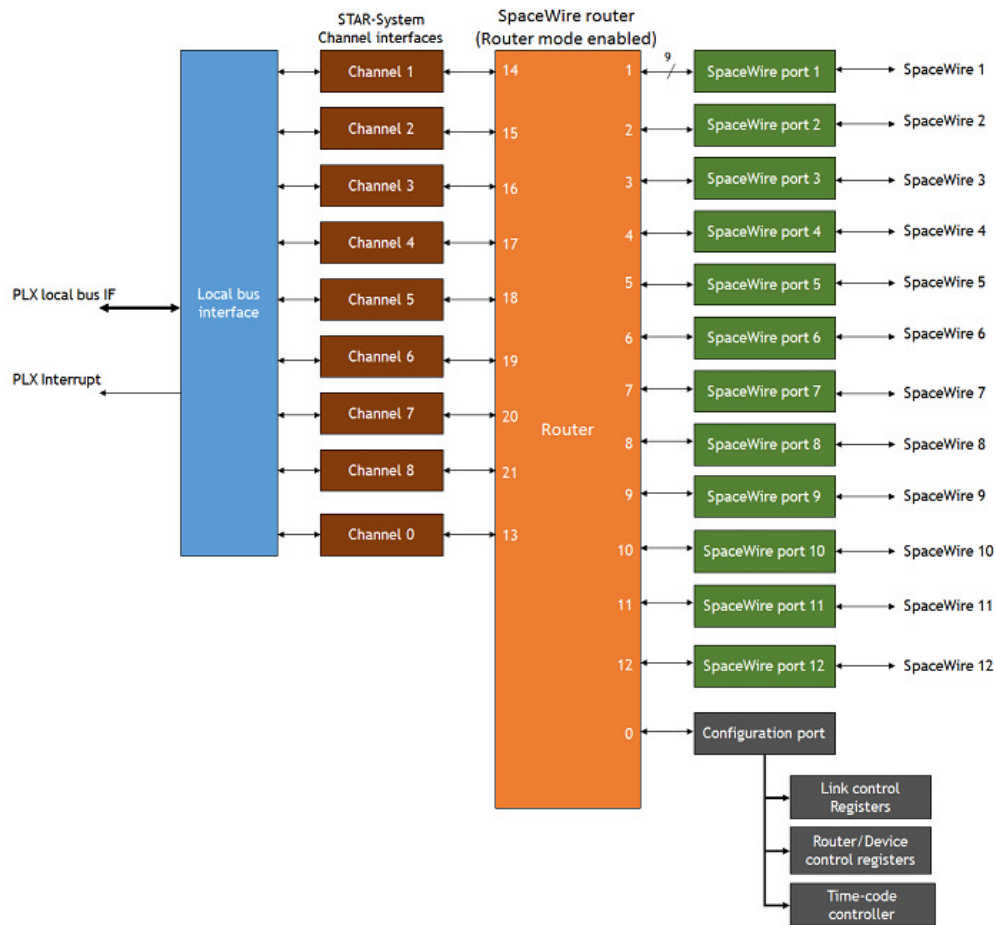


Figure 3.62: SpaceWire PXI 12 Port Router hardware overview

The 12 SpaceWire interfaces of the SpaceWire PXI Interface card are fully compliant with the SpaceWire standard and operate at up to 200 Mbit/s. They are connected to a SpaceWire router which supports two modes of operation, interface mode and router mode, where interface mode is the default mode. In Router mode, packets from any SpaceWire port can be routed to any other SpaceWire port, or the cPCI interface. In interface mode, the SpaceWire router is passive and packets which are received on a SpaceWire port are passed directly to the USB data channel for that port.

There are eight independent channels from the SpaceWire router to the cPCI interface, so traffic flowing over one port cannot block traffic for another port. In addition, there is a separate control channel, so that the host PC is always able to access the control, configuration and status space of the PXI card regardless of the data flow.

The SpaceWire PXI 12 Port Router card includes support for fault injection including support for parity errors, credit errors, escape errors, data corruption and EEP termination of packets.

3.7.3 Getting Started

The SpaceWire PXI 12 Port Router card is powered over a cPCI or PXI backplane; therefore no external power supply is required. Correct anti-static discharge procedures should be observed when inserting the PXI card into

the rack to ensure that no damage occurs to the components on the printed circuit board. Once the board is fully seated in the rack and the ejector handle is closed, it should be screwed in place using the four securing screws before use.

The SpaceWire PXI software and drivers, provided by STAR-System, should be installed on the host computer before connecting the SpaceWire PXI 12 Port Router card. This ensures that the device is recognised by the host when it is connected.

This document describes the host interface connections which are supported by the SpaceWire PXI 12 Port Router card.

3.7.3.1 Front Panel Interfaces

The front panel interfaces of the SpaceWire PXI 12 Port Router card are shown in the figure below.

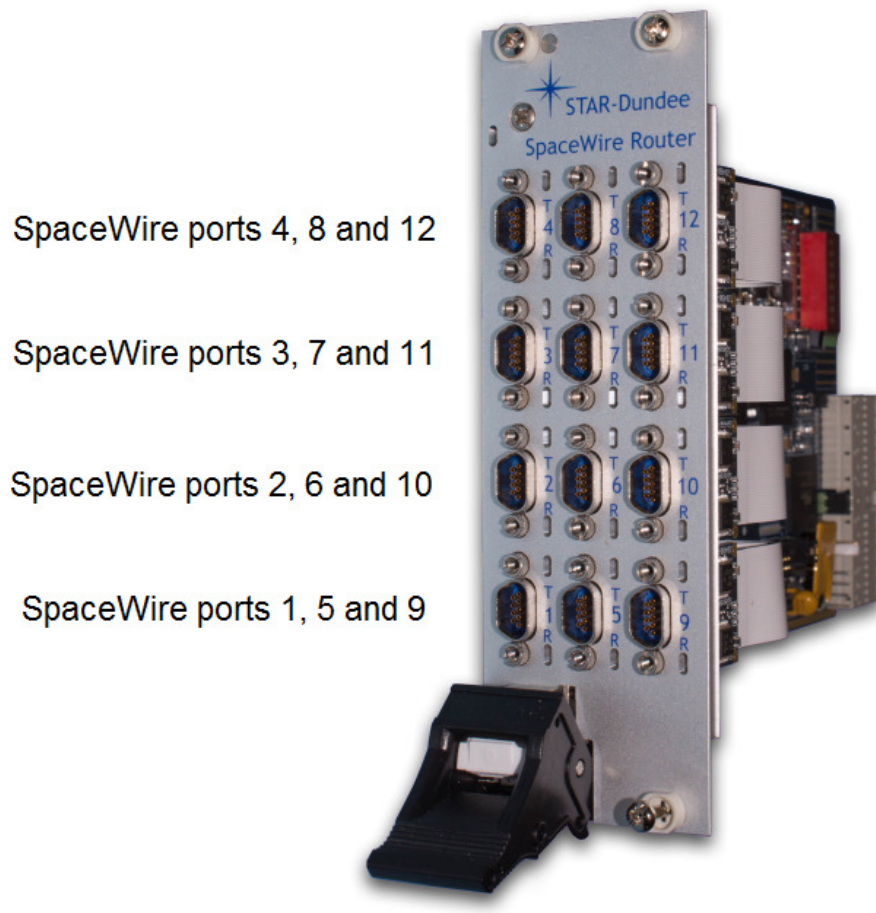


Figure 3.63: SpaceWire PXI 12 Port Router card front panel interfaces and LED positions

3.7.3.2 SpaceWire Interfaces

The SpaceWire PXI 12 Port Router card has two multicolour LEDs for each SpW port. The lower LED on each port shows received data for that port, and the higher LED on each port shows transmitted data for that port. The functions of the SpaceWire port LEDs are defined in the table below.

Function	LED Colour	Description
No event	Amber	The left and right hand LEDs are steady amber. Default state of the port, no activity and no NULLs transferred.
Link running	Green	The left and right hand LEDs are turned green when the link is running. The link running LEDs are held for approximately half a second after the link has stopped running and no error occurs.
Error	Red	The left and right hand LEDs are turned red if an error is detected when the port is in the Run state.
Data Transmitted/Received	Blue	When data is received on the link, the left hand LED turns blue. When data is transmitted on the link, the right hand LED turns blue.
No Credit	White	When no credit is available on the link for the device to transmit for a period of time, the right hand LED turns white. When the device has provided no credit on the link for the device at the other end of the link to transmit for a period of time, the left hand LED turns white.
Identify	Flashing purple	The SpaceWire PXI devices can be forced to flash their front panel LEDs to allow the device to be easily identified among a group of other devices. When this function is used the LEDs will flash purple for a short period of time.

Please note that there is an issue with all SpaceWire interfaces where a small amount of noise on the inputs can be translated by the LVDS receivers into a digital pattern and then interpreted by the receiver as a bit sequence. Since this bit sequence is invalid it causes the SpaceWire CODEC to start up and then disconnect. The SpaceWire interface LEDs go red on disconnect (ErrorReset state) and amber when ready (Ready state). So, if you get a burst of noise on the input then you will see a red flash of around half a second on the LEDs. To avoid this happening all the time we have a bias resistor network on the input of the LVDS receivers that provides a small bias current. This filters out most of the noise, but occasionally depending on the environment there may still be enough noise to trigger the reset to ready cycle and you see this as a red flash on the LEDs. This is the expected/normal operation. If you have any concerns, please contact support@star-dundee.com.

3.7.4 Supported Systems

The cPCI and PXI specifications which are referenced in this section are listed in the table below.

Reference	Document	Description
[1]	PICMG 2.0 D3.0 CompactPCI Specification - 1999-10-01	cPCI Specification
[2]	PXI Hardware Specification - Revision 2.1 February 4, 2003	PXI Specification
[3]	PXI Express Hardware Specification - Revision 1.0 August 22, 2005	PXIe Specification

The SpaceWire PXI 12 Port Router card is a standard 3U cPCI/PXI card which uses the 32-bit form factor defined in those standards. The card dimensions are listed below.

- Height: 100mm
- Length: 160 mm

An overview of the 3U 32-bit format PXI card is shown below and defined in Figure 2.1 of the PXI specification [2].

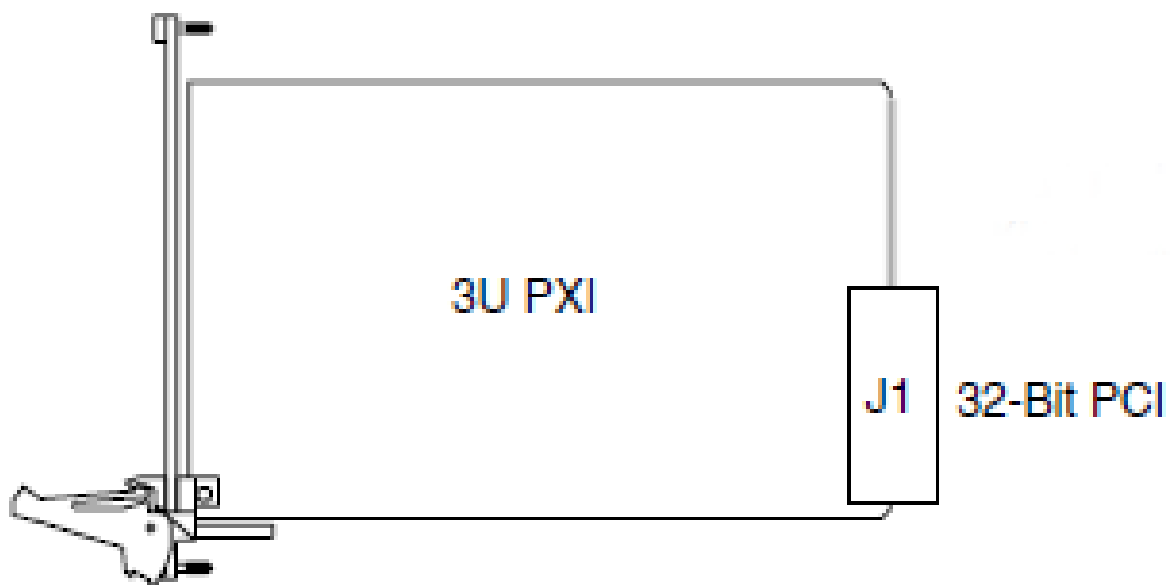


Figure 3.64: 3U PXI card form factor

3.7.4.1 cPCI Compatibility

The SpaceWire PXI 12 Port Router card is compatible with all cPCI Peripheral slots defined in section 2.1 of the cPCI specification [1].

Peripheral slots are identified using a circle as shown in the reproduced figure below.

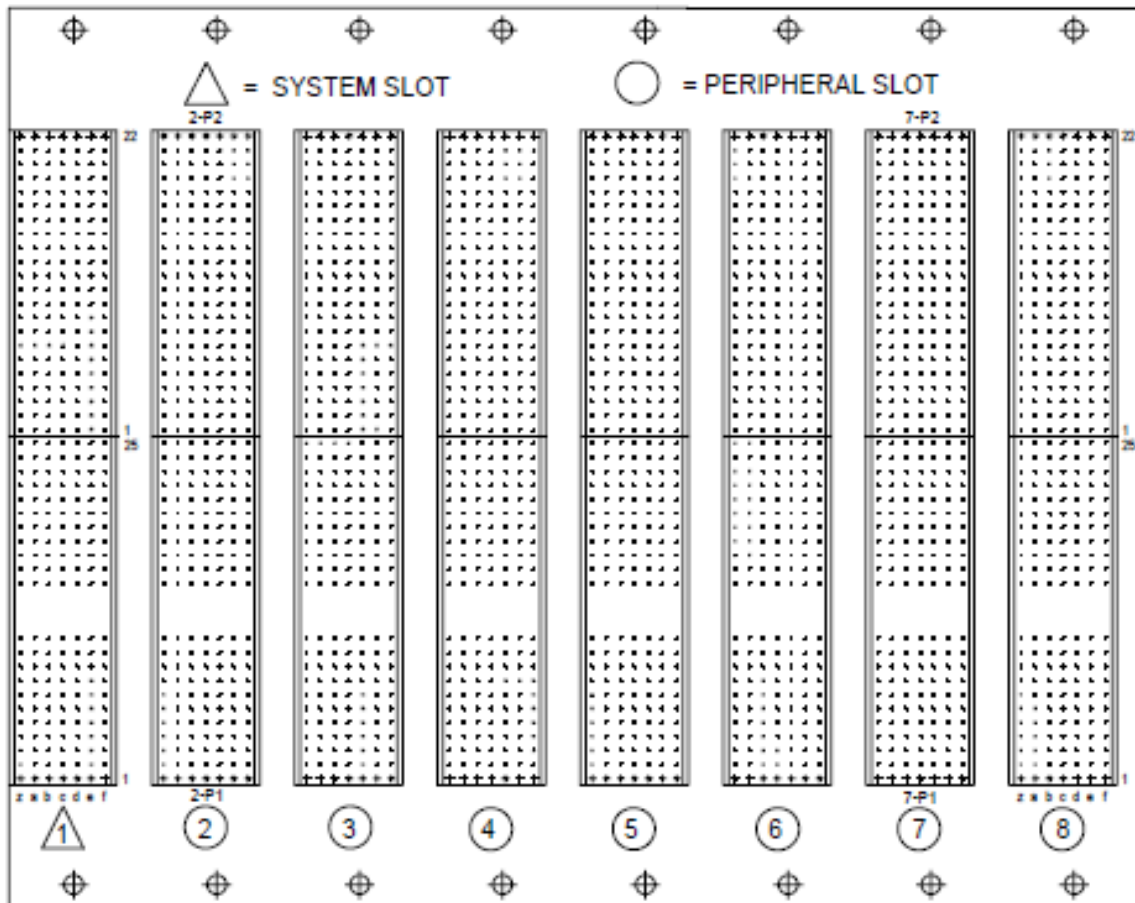


Figure 3.65: cPCI peripheral slots

3.7.4.2 PXI Compatibility

The SpaceWire PXI 12 Port Router card is compatible with all PXI Peripheral slots defined in section 2.1 of the PXI specification [2].

Peripheral slots are identified using a circle as shown in the reproduced figure below.

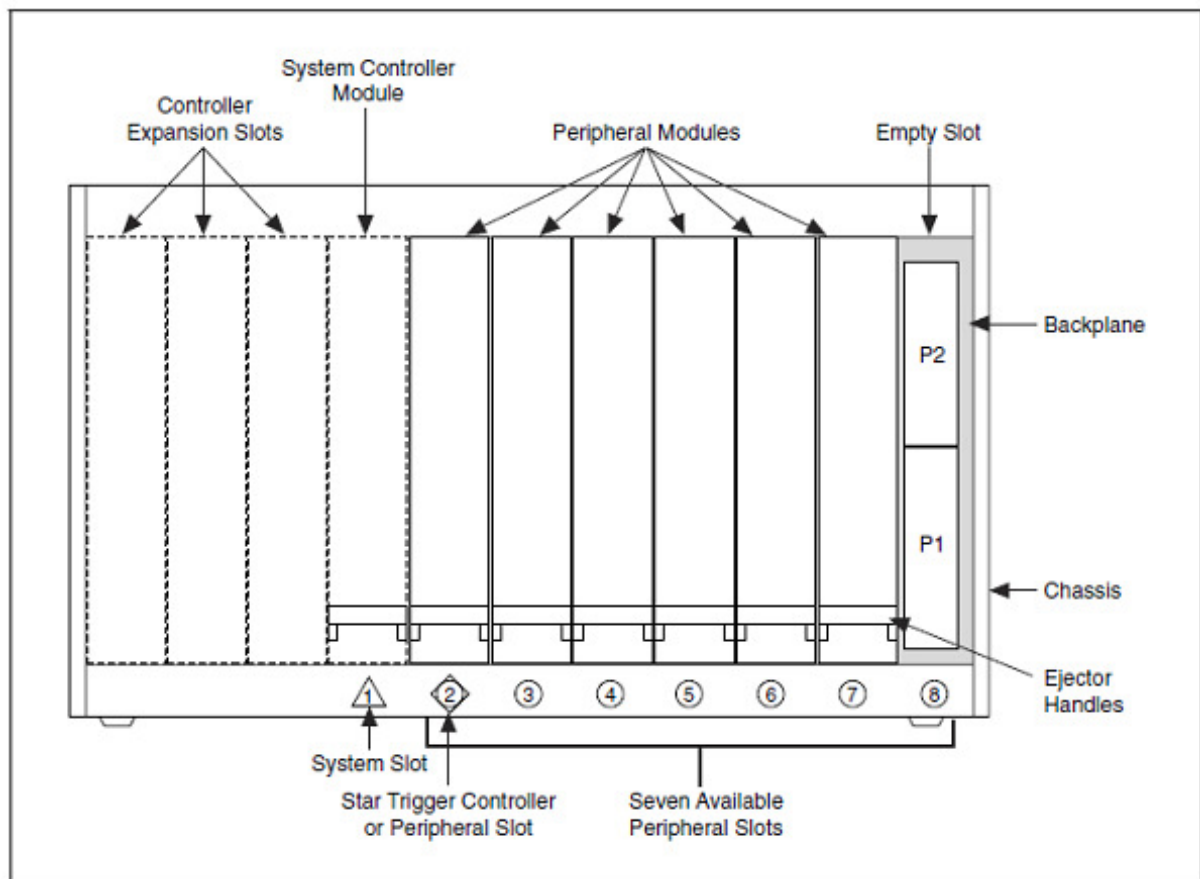


Figure 3.66: PXI peripheral slots

3.7.4.3 PXIe Compatibility

The SpaceWire PXI 12 Port Router card is compatible with all PXIe PXI-1 and PXI Hybrid slots defined in section 2.1.1.5 and 2.1.1.6 of the PXIe specification [3].

PXI-1 slots are identified using a circle as shown in the reproduced figure below.

PXI Express Hybrid slots are identified using a filled circle with a capital H outside the circle, as shown in the reproduced figure below.

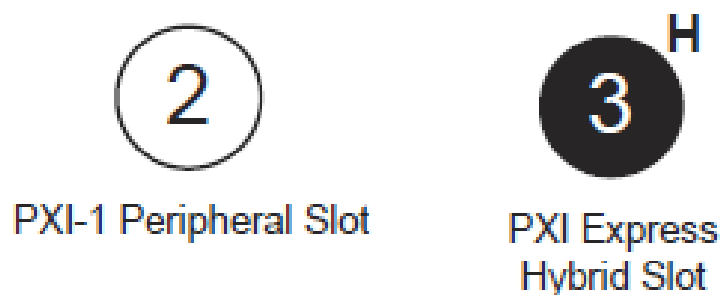


Figure 3.67: Compatible PXIe peripheral slots

3.7.5 Router and SpaceWire Interfaces

The SpaceWire router inside this unit is compatible with the AT7910E radiation tolerant router (also known as the SpW-10X) manufactured by Atmel. The SpaceWire router can be configured using the STAR-System device configuration API functions.

The SpaceWire router has 22 ports in total as listed below:

- 1 internal router configuration port
- 12 external SpaceWire ports
- 9 cPCI channel interface external ports. By default eight of these are used by the SpaceWire ports, while the other is used by the configuration port

The SpaceWire router supports two modes of operation, interface mode and routing mode.

The default mode is routing mode. The router will revert to routing mode after power cycling or a device reset is performed.

3.7.5.1 Interface Mode

In interface mode, a packet which is received on a SpaceWire port will always be routed by the port routing configuration. The SpaceWire PXI 12 Port Router card supports interface mode by utilising the port routing settings of the SpaceWire router. The default values are listed in the table below.

Source port	Destination port	Description
0	13	Configuration to external port 1 (host channel 0)
1	14	SpaceWire port 1 to external port 2 (host channel 1)
2	15	SpaceWire port 2 to external port 3 (host channel 2)
3	16	SpaceWire port 3 to external port 4 (host channel 3)
4	17	SpaceWire port 4 to external port 5 (host channel 4)
5	18	SpaceWire port 5 to external port 6 (host channel 5)
6	19	SpaceWire port 6 to external port 7 (host channel 6)
7	20	SpaceWire port 7 to external port 8 (host channel 7)
8	21	SpaceWire port 8 to external port 9 (host channel 8)
9	N/A	Not connected
10	N/A	Not connected
11	N/A	Not connected
12	N/A	Not connected
13	0	cPCI channel 0 to configuration port

14	1	cPCI channel 1 to SpaceWire port 1
15	2	cPCI channel 2 to SpaceWire port 2
16	3	cPCI channel 3 to SpaceWire port 3
17	4	cPCI channel 4 to SpaceWire port 4
18	5	cPCI channel 5 to SpaceWire port 5
19	6	cPCI channel 6 to SpaceWire port 6
20	7	cPCI channel 7 to SpaceWire port 7
21	8	cPCI channel 8 to SpaceWire port 8

3.7.5.2 Routing Mode

In routing mode packets are routed dependent on the first byte of each SpaceWire packet. The routing algorithm is described below.

- Packets with known physical addresses are routed to the port indicated by the physical address at the head of the packet, i.e. a packet with address 0 is routed to the configuration port.
- Packets with unknown physical addresses, 22 to 31, are discarded and an address error is set in the source port register.
- Packets with logical address headers which are valid are routed to one of the set ports in the logical address lookup table (accessed via the configuration port).
- Packets with logical address headers which are marked as invalid are discarded and an address error is set in the source port register.

3.7.5.3 Link Operating Speed

The SpaceWire link operating speed for each pair of SpaceWire ports is controlled by the STAR-System device configuration APIs. The link clock rate selection logic for each pair of ports is shown in the figure below.

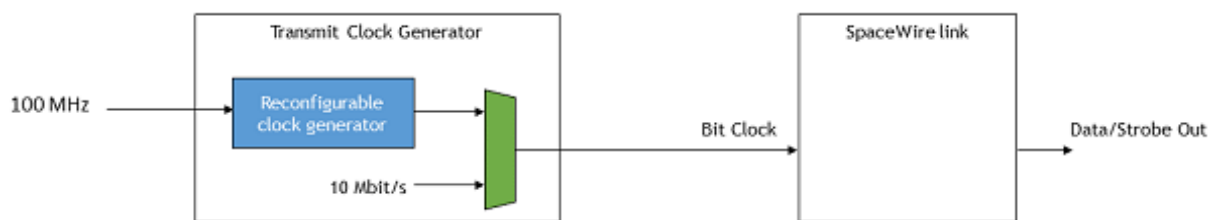


Figure 3.68: SpaceWire link operating speed for each pair of SpaceWire ports

- Each pair of SpaceWire links can be programmed to a link rate between 2 and 200 Mbit/s using the transmit clock configuration functions of the STAR-System device configuration APIs.
- The link operating speed after power on is 200 Mbit/s.

- Each SpaceWire link will operate at 10 Mbit/s at start-up.
- When the reconfigurable clock generator is reprogrammed the link rate will switch to 10 Mbit/s until the programming has completed, when the link is running.

3.7.5.4 Time-codes

A time-code can be received from the cPCI interface or from a SpaceWire link. When a time-code is received, its value is checked against the value of the internal time-code register in the SpaceWire router. If the received time-code is one more than the value of the time-code register then the time-code will be written to the time-code register and propagated to other ports, except the port that was the source of the time-code.

Time-codes are only propagated on ports which are enabled to send time-codes as defined by the time-code control register. Note that of the external cPCI ports, only port 5, corresponding to channel 0, can be used to send or receive time-codes.

The router has an internal time-code generation master, which can be controlled by the SpaceWire router configuration port to generate time-codes at a user defined rate.

3.7.6 SpaceWire Interface Tristate Mode

The SpaceWire ports on the PXI card are comprised of four LVDS signal pairs, Data/Strobe in +/- and Data/Strobe out +/- . The Data/Strobe output pairs are connected to LVDS drivers which default to a high impedance Tristate mode after power on and after a device reset. The signal pairs are effectively disconnected when the tristate mode is enabled, providing protection for connected devices.

Each port includes a link monitoring component which can enable or disable the LVDS driver of a link dependent on a combination of the SpaceWire link settings, Start, Disabled or Tristate, and on the input bit pattern. The input bit pattern is monitored for a sequence of NULL characters without error to detect if a link should move out of Tristate mode and actively drive the Data/Strobe output signals.

The Tristate mode of each port is acted on by the following inputs.

- The default operating mode of all STAR-Dundee SpaceWire links is AutoStart. In the AutoStart mode the SpaceWire links remain in the Ready state and will wait for NULL characters before starting up.

If a link on the PXI card is in Tristate mode (LVDS drivers disabled) then the LinkEnabled check in the Ready state, **LinkEnabled=(Link Disabled=0 AND (LinkStart=1 OR (AutoStart=1 AND gotNULL=1))**, remains LOW until 8 valid NULL characters have been successfully received without error, i.e. the state machine cannot move to Started until the LinkDisabled condition in the function is cleared.

Further information on the Ready state transition is shown in the figure below.

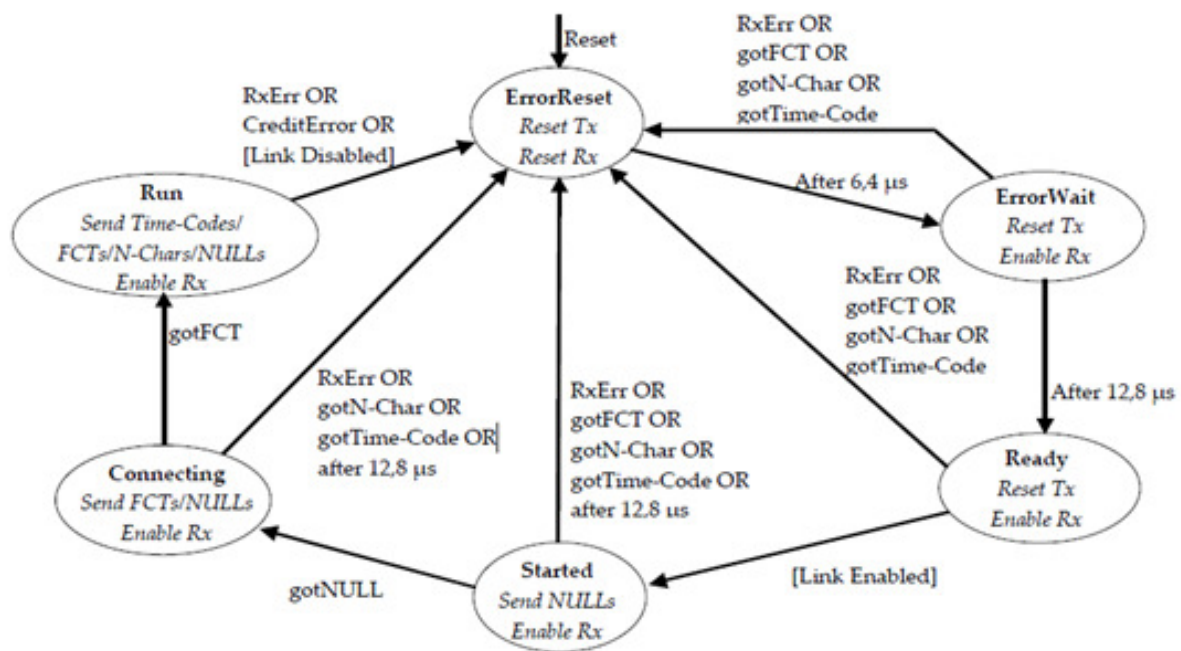


Figure 3.69: SpaceWire Interface state machine

- If the host software or network manager sets a link into LinkStart mode to force the link state machine to move from Ready to Started, or the Tristate mode enable bit is cleared, then the link will automatically move out of its Tristate mode and the SpaceWire port LVDS drivers will be enabled.
- If the host software or network manager sets the LinkDisabled mode to force the link into a disabled state, and the Tristate control bit is set, then the LVDS drivers of the corresponding SpaceWire port are disabled and are set to a high impedance state.

An overview of the Tristate mode settings and their effect on the link are shown in the state diagram below.

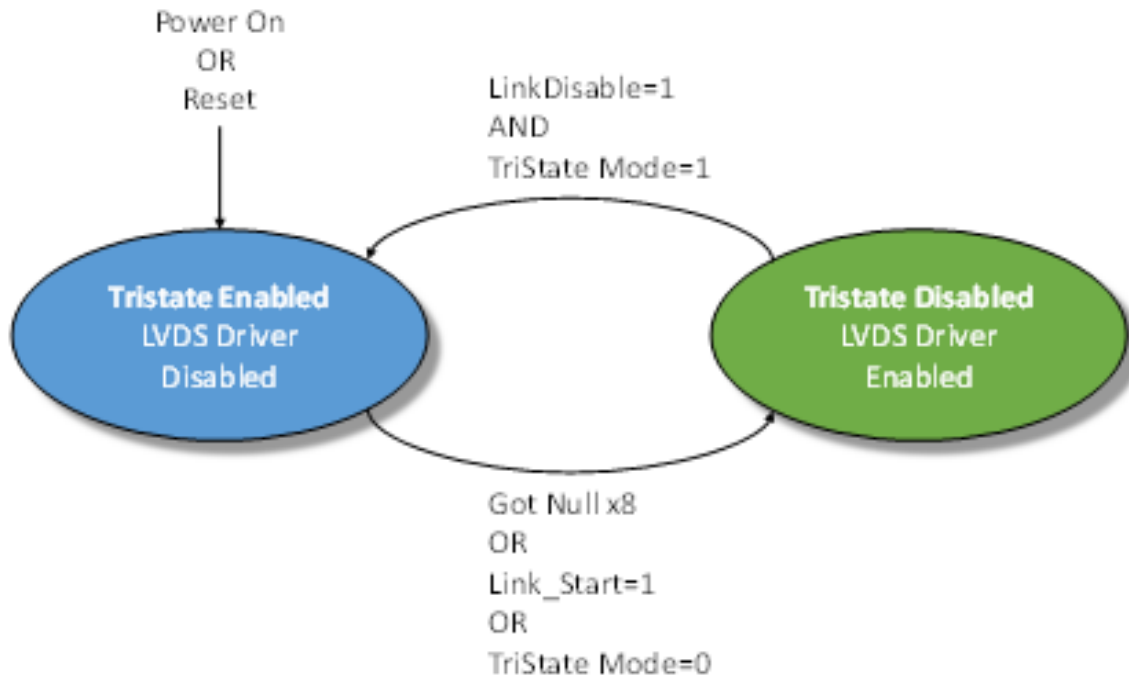


Figure 3.70: Tristate controller state machine

An approximate 60 microsecond delay occurs between the state machine moving from the Enabled to Disabled state, i.e. the LVDS drivers are not enabled until approximately 60 microseconds after the condition to enable the drivers has been detected.

A device which connects to the SpaceWire PXI card and sends NULLs can move through the SpaceWire link state machine a few times until the LVDS drivers are enabled. The exchange of silence period is 19.2 microseconds and a link which is enabled will send NULLs in the Started state for a period of 12.8 microseconds. The link will start normally after the 60 microsecond period – the user does not need to take any action to operate correctly with the PXI card in this mode.

An example start-up sequence when the PXI link is in AutoStart mode is shown in the figure below.

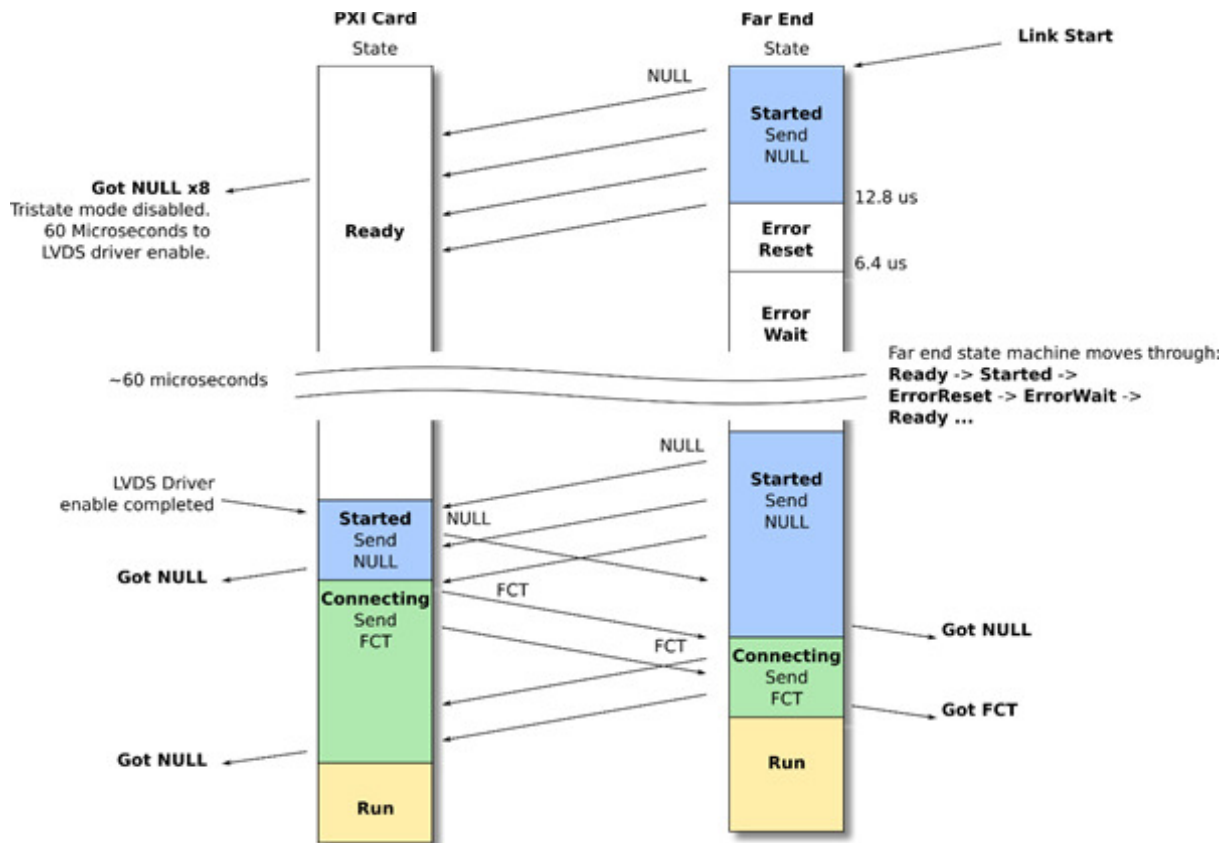


Figure 3.71: AutoStart link operation

In the figure above the Far End link state machine sends NULL characters after its Link Start control bit is set. The PXI card detects the NULL sequence and indicates to the Tristate controller state machine when 8 consecutive NULLs are received without error. The PXI card then enables its LVDS drivers after 60 microseconds and will remain in the Ready state until it detects a start-up NULL from the far end. Both ends then exchange NULL and FCT characters and start-up normally.

An example start-up sequence when the PXI link is set to start is shown below.

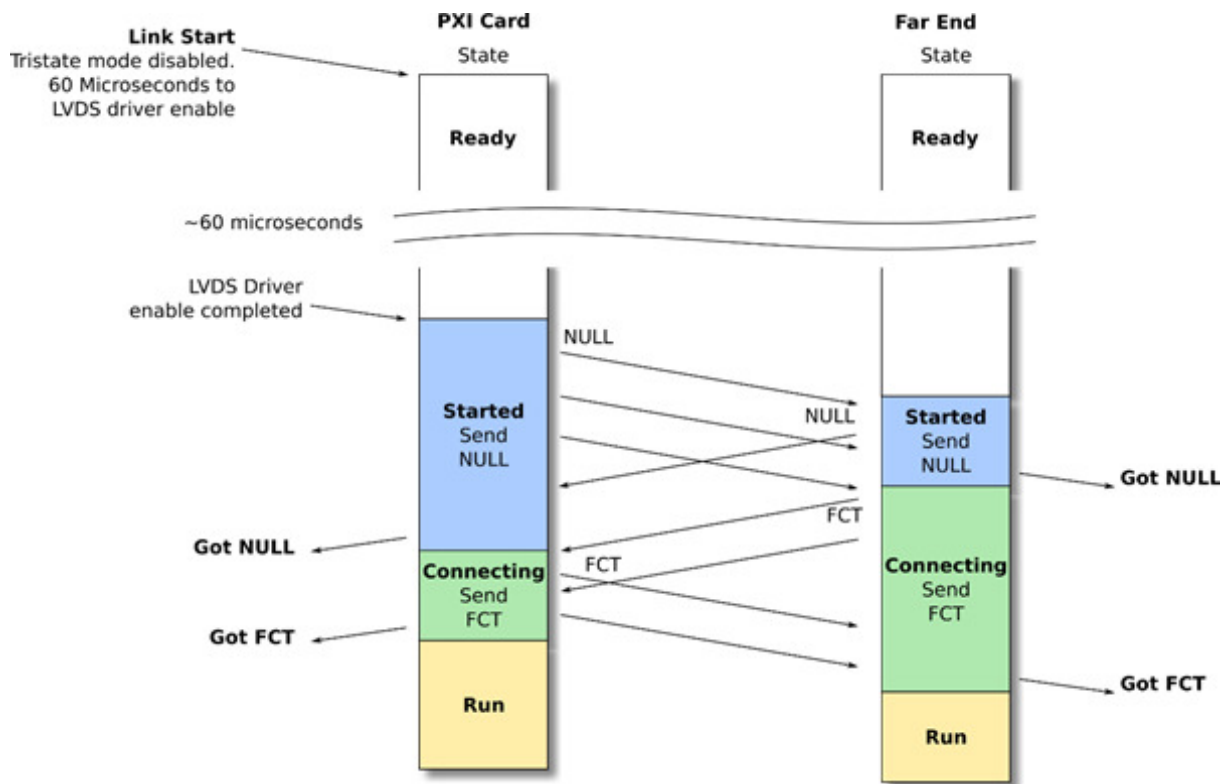


Figure 3.72: Link Start link operation

In the figure above the PXI Card Link Start bit is set and the LVDS driver enable sequence is started. After 60 microseconds the PXI card link will move to the Started state and will send NULL characters. The Far End will detect NULL characters, both ends will exchange NULLs and FCTs and will start-up normally.

Please note, if a SpaceWire Link should start-up on the first exchange of NULL characters then the Tristate setting for that link should be disabled.

3.7.7 Electrical Characteristics

3.7.7.1 Absolute Maximum Ratings

Parameter	Min	Max	Units
SpaceWire input voltage on Vin+, Vin- (see the figure in the following section)	-0.7	4.3	V
Differential input voltage on Vin+, - Vin- absolute value (see the figure in the following section)		1	V

External Trigger input voltage	-0.5	6	V
--------------------------------	------	---	---

3.7.7.2 SpaceWire Functional Diagram

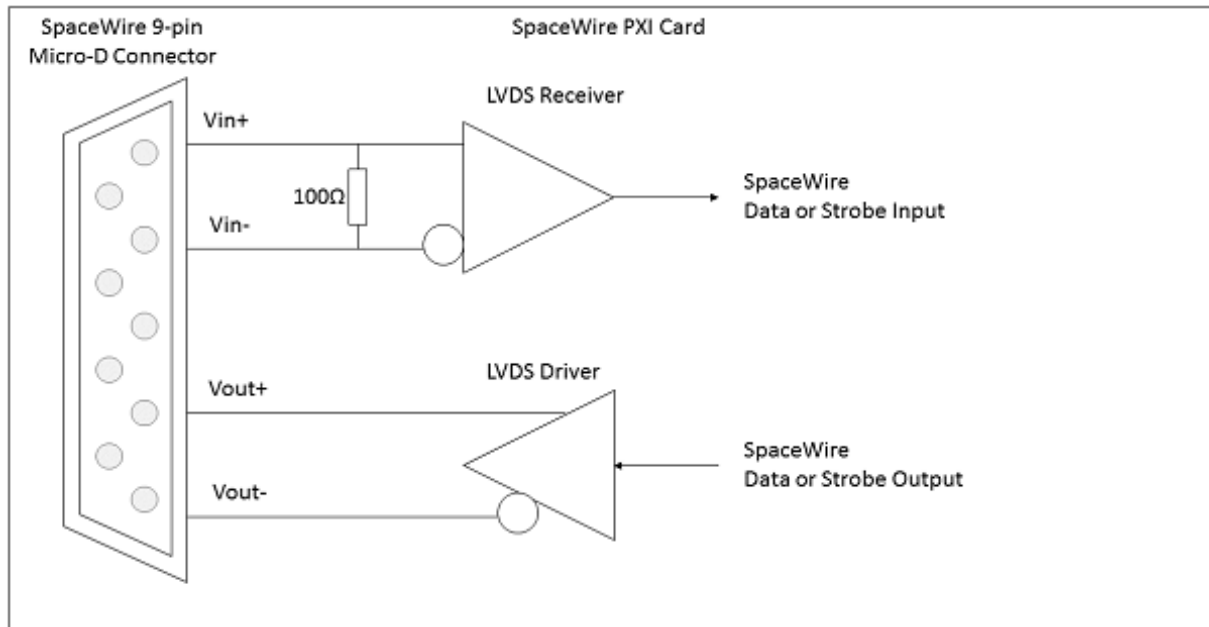


Figure 3.73: SpaceWire functional diagram (one LVDS driver/receiver shown)

Each SpaceWire port is comprised of a pair of LVDS drivers and LVDS receivers, one pair for data and strobe in, and one pair for data and strobe out.

3.7.7.2.1 SpaceWire Input Electrical Characteristics

The SpaceWire recommended input electrical characteristics are shown in the table below.

Parameter	Min	Max	Units
Differential Voltage range (Vin+ - Vin-, absolute value)	0.1	1	V
Common Mode +/- Differential voltage range (any combination of common-mode or input signals)	0	4	V
SpaceWire input Differential resistance	98	102	Ω

3.7.7.2.2 SpaceWire Output Electrical Characteristics

The SpaceWire output electrical characteristics are shown in the table below.

Parameter	Min	Typ (1)	Max	Units
SpaceWire output differential magnitude - 100 Ω load	247	310	454	mV

SpaceWire output common mode voltage - 100 Ω load		1.125		1.375	V
Output voltage to an unpowered unit	Vin+/Vin-	1.12	1.16	1.2	V
	Vout+/Vout- (Tri-state - Default at power on)	High Z	High Z	High Z	V
	Vout+/Vout- (Enabled - programmed by software)	0.898		1.602	V

(1) Typical conditions at 25 °C

Additional information

The STAR-Dundee PXI card can programmatically tristate its LVDS drivers.

3.7.7.3 External Trigger Functional Diagram

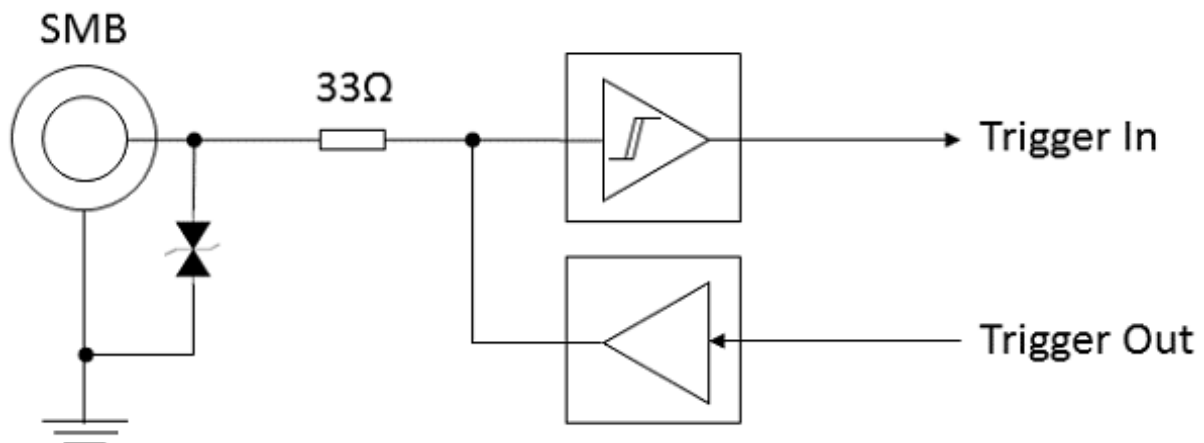


Figure 3.74: External trigger functional diagram

3.7.8 Switching Characteristics

The SpaceWire interface data/strobe input switching characteristics are defined in the figure and table below.

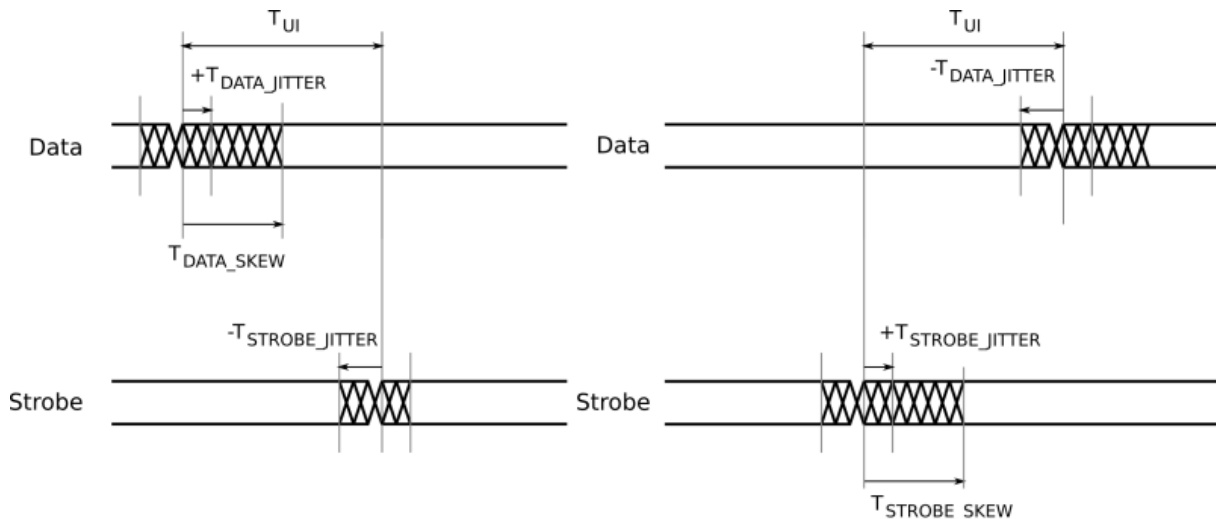


Figure 3.75: SpaceWire input switching characteristics

Parameter	Description	ns (Max)
T_{UI}	Transmitter bit rate.	5
$T_{DATA_SKEW_IN} (MAX)$	Maximum data skew which can be tolerated by the receiver.	$2 - T_{DATA_JITTER_IN} - T_{STROBE_JITTER_IN}$
$T_{STROBE_SKEW_IN} (MAX)$	Maximum strobe skew which can be tolerated by the receiver.	$2 - T_{DATA_JITTER_IN} - T_{STROBE_JITTER_IN}$

The data/strobe output switching characteristics are defined in the table below.

Parameter	Description	ns (Max)
T_{SKEW_OUT}	Maximum data or strobe skew.	0.1
T_{JITTER_OUT}	Maximum data/strobe jitter.	0.7 (Peak to Peak)

3.8 Other STAR-System Devices

In addition to the STAR-System devices with their own individual user manuals, there are several other STAR-System devices which rely on STAR-System. These are described below.

3.8.1 Devices with Interface Capabilities

The following STAR-System devices offer similar functionality to the interface and router devices which have individual user manuals included in this documentation. They can therefore be used with the STAR-System applications and APIs. However, they also offer lots of other functionality so have separate user manuals which also describe their individual capabilities.

3.8.1.1 SpaceWire Physical Layer Tester

The **SpaceWire Physical Layer Tester**, or SPLT, provides the ability to manipulate SpaceWire signals to test a connected unit. While those signals are being manipulated, STAR-System can be used to transmit and receive packets on the SPLT's four ports. The SPLT's user manual explains how each of these ports can be accessed.

3.8.1.2 STAR Fire

The STAR Fire can act as a SpaceWire to SpaceFibre bridge, a SpaceFibre link analyser, or as an interface for transmitting and receiving on the SpaceWire and SpaceFibre ports. The initial version of the STAR Fire did not support STAR-System and relied on STAR-Dundee's older USB Driver. The device and software were soon updated, though, to support STAR-System for transmitting and receiving packets. Internally this updated version of the product is known as the STAR Fire Mk2. Please see the user manual for the STAR Fire to learn how packets can be routed to/from the SpaceWire and SpaceFibre ports.

3.8.1.3 STAR Fire Mk3

Like the original **STAR Fire**, the **STAR Fire Mk3** can act as a SpaceWire to SpaceFibre bridge, a SpaceFibre link analyser, or as an interface for transmitting and receiving on the SpaceWire and SpaceFibre links. The **STAR Fire Mk3** provides a higher speed USB 3.0 interface and much improved software. The STAR Fire Mk3 user manual provides information on routing packets to the SpaceWire and SpaceFibre ports, including using the USB 3.0 interface to transmit and receive packets at very high speed over the SpaceFibre ports.

3.8.2 Devices without Interface Capabilities

The following STAR-System devices use the STAR-System drivers, but do not offer the ability to transmit and receive packets using the STAR-System APIs. Basic properties such as their name and serial number can be obtained, however, and device updates can be performed using the [Device Update Application](#), for example. These devices instead offer lots of other capabilities which are documented in their individual user manuals.

3.8.2.1 SpaceWire Conformance Tester Mk2

The **SpaceWire Conformance Tester Mk2** allows a device's conformance to the SpaceWire standard to be tested. It does this by transmitting specific patterns of bits on the link and checking the bits that are received. Due to this unique nature, it doesn't offer an interface for transmitting and receiving packets through STAR-System.

3.8.2.2 SpaceWire EGSE and SpaceWire EGSE Mk2

The SpaceWire EGSE and Device Simulator and its successor the **SpaceWire EGSE Mk2**, can be used to perform real-time simulations of an instrument. This is achieved by writing scripts describing what the instrument should do, which are then downloaded to the device and executed on that device in real-time. Although the EGSE products do not offer standard interface capabilities, they do include an API which is built on top of STAR-System. Further information is available in the EGSE and EGSE Mk2 user manuals.

3.8.2.3 SpaceWire Link Analyser Mk3

The **SpaceWire Link Analyser Mk3** can be used to monitor and record the information crossing a SpaceWire link. The recorded data can be viewed at various levels, from the individual bits through to a higher layer protocol level. Although the Link Analyser Mk3 doesn't include the ability to transmit and receive packets, it does include an API for configuring the capture of traffic, etc. This is built on top of STAR-System and further information is available in the Link Analyser Mk3's user manual.

3.8.2.4 SpaceWire Recorder and SpaceWire Recorder Mk2

The SpaceWire Recorder and its successor the **SpaceWire Recorder Mk2**, can be used to monitor and record the traffic crossing up to four SpaceWire links. The Recorders include a large SSD for storing this data, meaning that the traffic can be recorded for much longer than is possible using the Link Analyser Mk3, although the Recorders are unable to record at the bit level.

3.9 Device Update Application

Documentation for the STAR-System Device Update Application is provided in this section.

3.9.1 Using the Device Update Application

3.9.1.1 Before Running the Device Update Application

Before running the Device Update Application, make sure that you have the correct firmware update file for your device.

Note also that updating devices can take a considerable length of time, sometimes several hours, and it is important that the computer used for updating, and the device being updated, are not powered down or disconnected during the update process. Doing so may leave your device in an inoperative state.

3.9.1.2 Performing a Device Update

The image below shows the Device Update Application in use.

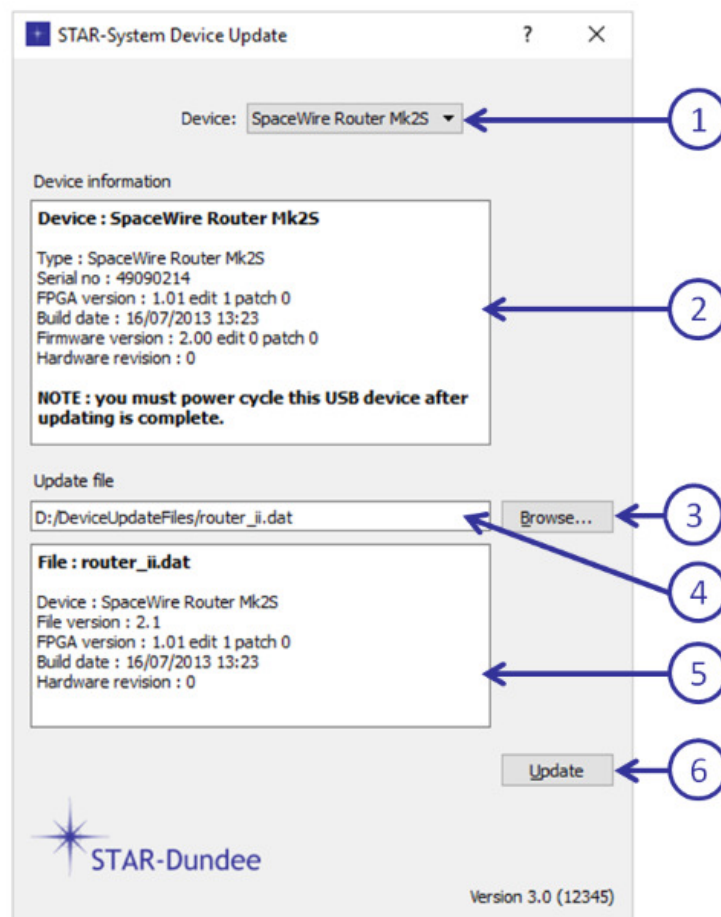


Figure 3.76: Device Update Application Main Window

The controls for the application are as follows:

1. Device selector dropdown - used to select the device to be updated

2. Device information panel - provides information about the selected device
3. Update file browse button - displays a file selection dialog for the update file
4. Update file path and name - displays the update file name, and can be used to enter the file name
5. Update file information - displays information about the update file
6. Update button - starts the update process

Note that the device selected in this example is a USB device, and the device information panel includes the information that it should be power cycled on completion of the update. For a PCI, PCIe, cPCI or PXI device, this will indicate that the computer should be power cycled to complete the update.

Once a device and an appropriate update file have been selected, the Update button (6) can be used to start the update process. A progress dialog will be displayed showing the status of the update:

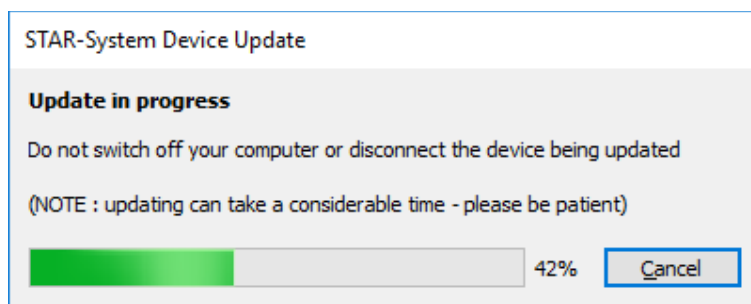


Figure 3.77: Device Update Progress Dialog

Again, please note that the update can take a considerable time.

Once the update is complete, a message box will be displayed:

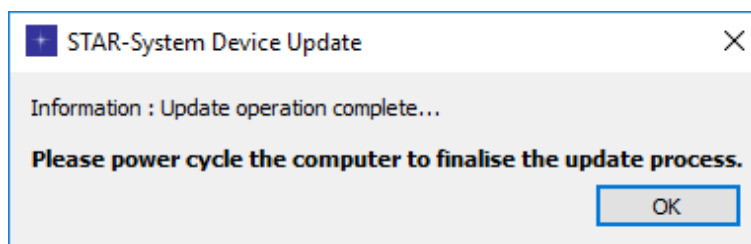


Figure 3.78: Device Update Complete

In this example the update was performed on a PCI, PCIe, cPCI or PXI device, and the computer must be power cycled to complete the update. For a USB device, the message will indicate that the device should be power cycled.

3.9.1.3 Cancelling a Device Update

The update can be cancelled at any time by pressing the Cancel button in the progress dialog, but please note that this may leave your device in an inoperative state. An additional dialog will be displayed to protect against accidental cancellation:

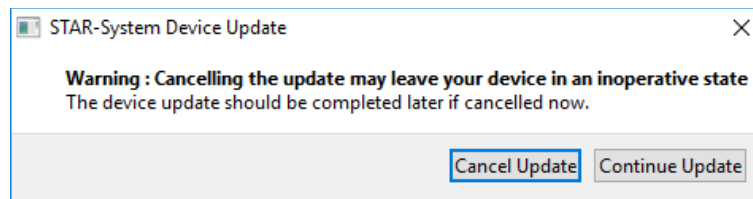


Figure 3.79: Device Update Cancel Dialog

If the update is cancelled, an additional notification will be displayed:

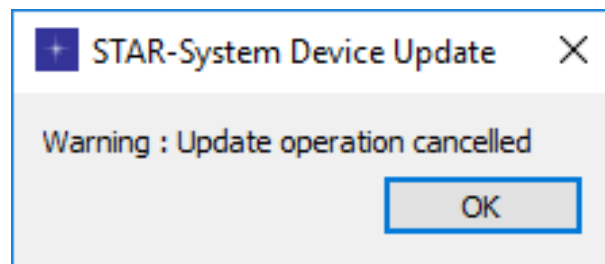


Figure 3.80: Device Update Cancelled Message

3.9.1.4 Errors During a Device Update

If an error occurs during the update process, a message box will be displayed giving details of the error, such as:

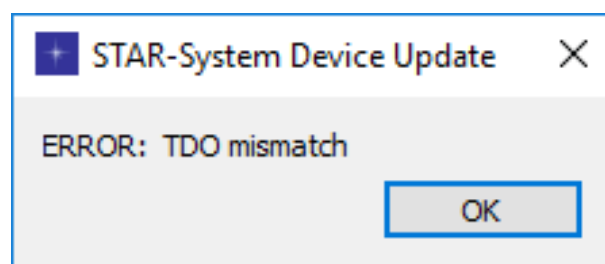


Figure 3.81: Device Update Error Message

If this occurs, please contact support@star-dundee.com with details of the error message, the device you were updating and the update file being used.

3.9.1.5 Other Messages and Notifications

There are a number of other messages and notifications which may appear when starting a device update operation, and each is described below.

If the update file is not for the type of device selected, an error message will be displayed. Either the correct device type or correct update file must be selected to allow the update to proceed.

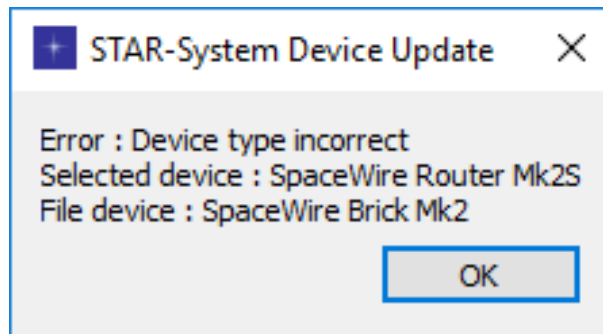


Figure 3.82: Device Type Incorrect Message

If the version of the update file is earlier than the version currently programmed into the device, a warning will be displayed. The update may be performed, but this should only be done if you are sure you wish to revert your device to an earlier version.

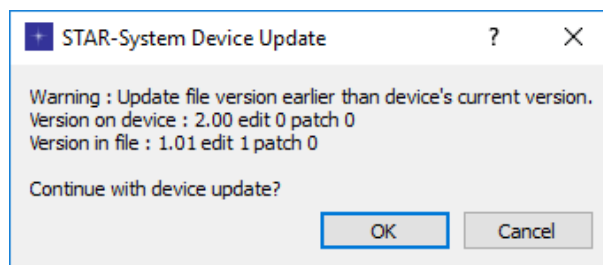


Figure 3.83: File Version Earlier Dialog

If the version of the update file is the same as the version currently programmed into the device, a warning will be displayed. The update may be performed, but this should only be done if you are sure this is what is required.

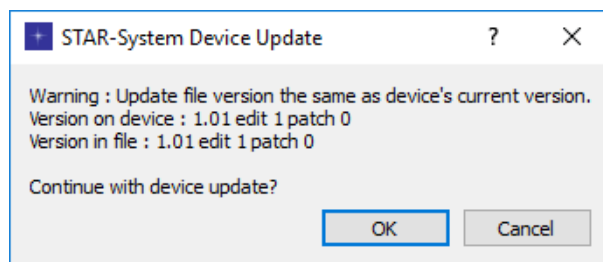


Figure 3.84: File Version The Same Dialog

Chapter 4

Other Useful Information

The following sections describe other information which may be useful when installing and using STAR-System:

- Installation instructions for specific operating systems:
 - [VxWorks Installation and Getting Started Guide](#)
 - [RTEMS Installation and Getting Started Guide](#)
- [Multi-Threading and Multi-Process Support](#)
- [Device Configuration Service](#)
- [Error Logs](#)
- Information on the C API:
 - [Configuration API Functions Supported by Device Types](#)
 - [Static Analysis](#)
- [Change Log](#)

4.1 VxWorks Installation and Getting Started Guide

4.1.1 Introduction

The VxWorks version of STAR-System provides a custom driver in the form of compiled libraries for the appropriate toolchain, low level device initialisation code provided as source, and example application code.

4.1.2 Installation Contents

This section contains a description of files provided.

`/bin`

- `device-configuration-service.out`: contains the `STAR_runConfigService()` function which runs the Device Configuration Service. This provides the facility to read or modify the properties of devices.
- `Star-SystemPerformanceTester.out`: Star-System Performance Tester functions can be run interactively from the `main` method or by providing command line arguments. Note that the file-handling functions have not been enabled for VxWorks.

- `Star-SystemTest.out`: A suite of general purpose STAR-System test functions. This can be run interactively by invoking `runInteractive()` or by providing command line arguments. Note that the file-handling functions have not been enabled for VxWorks.

/docs

- STAR-System documentation in HTML form.

/examples

- A suite of test/example code is provided including the source code to `Star-SystemPerformanceTester` and `Star-SystemTest.out`. Generic Makefiles are included but these may need to be adapted for use with your VxWorks setup.

/inc/star/

- Header files required to build STAR-System applications.

/lib

- STAR-System libraries (`.a`) compiled for specific VxWorks versions and toolchains.
- The following libraries should be included for all STAR-System application builds: `star-api.a`, `star-port.a`, `star-driver-interface.a`, and `star-VxWorks-driver.a`.
- The following libraries should also be included for all STAR-System applications using the configuration service: `star_conf_mssgr.a`, `star_conf_api_router.a`, `star_conf_api_mk2.a`, `star_conf_api_pci_mk2.a`, `star_rmap-initiator.a` and `rmap_packet_library.a`.

/sys

- This directory contains the driver board initialisation code provided as source; `sysSpaceWirePci.c` and `SpaceWirePci.h`. The function `sysSpaceWirePciInit()` must be called before running any STAR-System API calls.

4.1.3 VxWorks Image Include Components

In your VxWorks image build please make sure that the following components are included.

In operating system components (default), POSIX components: POSIX clocks (default), POSIX directory utilities (default), POSIX mman, POSIX process scheduling, POSIX semaphores (default), POSIX threads (default), POSIX timers (default) and sigevent notification library.

Depending on the BSP used, you may also need to include PCI and PCI autoconfiguration.

4.1.4 Initialising The Driver

Function `sysSpaceWirePciInit()` in file `sysSpaceWirePci.c`, adds an entry to the SpaceWire device resource table (globally defined) for each device found with the correct device and vendor ID. Function `sysSpaceWirePciInit()` must be called before running any STAR-System API calls and could be called from either `sysHwInit2()` or `userAppInit()` in the VxWorks image build. Before building, please define one of the targets at line 66 in `sysSpaceWirePci.c` or adapt the code for a new target. The default target is the Marvell CPC1750 PowerPC. Attempts to call `sysSpaceWirePciInit()` in `userAppInit()` may fail (depending on the BSP) if MMU memory mapping is required, in that instance please call it from either `sysHwInit()` or `sysHwInit2()`.

Function `sysSpaceWirePciInit()` may need to be modified for a specific target if there are any non-standard BSP issues. The following target specific implementations have been developed and tested by STAR-Dundee.

1. Marvell CPC1750 PowerPC, VxWorks 6.7
2. XCalibur 1002, VxWorks 6.7, 6.8

For any other target the following is suggested:

1. Create a new `#define` in `sysSpaceWirePci.c` for your new target.
2. Map BAR registers by reading the base address and storing in the resource table entry. If function `sysMmuMapAdd()` is required by your BSP then add an additional call `sysMmuMapAdd()` for BAR regions 0, 2 and 3.
3. Enable interrupts by obtaining the device IRQ and IRQ vector.
4. Specify whether the device is little or big endian.
5. Define any memory offset address for DMA.
6. Enable the PLX9056 command register.
7. Define read and write register functions.
8. Define interrupt connect and disconnect functions.
9. Define any other BSP cPCI specific tasks, e.g. the correct bus if there are multiple cPCI buses.

Please contact STAR-Dundee should any assistance be required for this procedure.

4.1.5 Quick Start Guide

4.1.5.1 Running The Pre-compiled Test Programs

Follow the instructions to initialise the driver in the above section. After booting VxWorks, download `Star-SystemTest.out` (for the correct toolchain). Connect links 1 and 3 with a SpaceWire cable then run test 2 to transmit and receive data by calling the `runInteractive()` function from a shell.

To test the router configuration service, download `device-configuration-service.out` and `Star-SystemTest.out`, call `STAR_runConfigServe()` which starts the configuration service, then call `runInteractive` and run the `IdentifyDevice` test which will flash the LEDs.

4.1.5.2 Building the example programs

The example programs are supplied with Makefiles which have not been configured for VxWorks. So to build the examples, either modify these Makefiles for your VxWorks set-up or use Workbench to create projects to build the test code. For example, to build STAR-System Test in Workbench please do the following:

1. Create a new downloadable kernel module project,
2. Add the STAR-System Test source code files to your project (or link to external content),
3. In the build properties, add the path to `/inc/star` to your include paths,
4. In the build properties include the 10 library files from the `/lib` directory, and
5. Build and download the test program.

4.2 RTEMS Installation and Getting Started Guide

4.2.1 Introduction

The RTEMS version of STAR-System includes all the APIs, examples and command-line test applications available on other STAR-System supported platforms. At the driver level, a PCI Driver is provided, supporting PCI Mk2, cPCI Mk2, PCIe and PXI devices.

4.2.2 Supported Targets

Currently only LEON targets are supported, with the code compiled for the LEON3. The RTEMS build of STAR-System makes use of the RTEMS Driver Manager, and so requires RTEMS version 4.10.

If you require support for another target, RTEMS version, or for USB devices, please contact STAR-Dundee using the information in the [Support and Contact Information](#) section.

4.2.3 Installation

The STAR-System RTEMS release is provided as a .tgz file which should be extracted to a location that can be accessed from your RTEMS build environment. The example and test applications can then be built using the following command in the directory of the program to be built:

```
make OPERATING_SYSTEM=RTEMS
```

4.2.4 Driver Module

The PCI Driver for STAR-System is provided as a library on RTEMS, named `libstar_pci_driver.a`. Any application which will make use of the STAR-System APIs to access a device must therefore link against this library, in addition to the other STAR-System libraries. The Makefiles for the STAR-System example applications demonstrate how to do this using `-lstar_pci_driver`.

4.2.5 Device Configuration Service

If you wish to read or modify the properties of a device by calling any of the Device Configuration API functions (those with names beginning `CFG_`) from your application, you must first call the `STAR_runConfigService()` function to start the Device Configuration Service. This function starts the Configuration Service, so should only be called once by an application. Further information on the functionality provided by the Device Configuration Service is available in the [Device Configuration Service](#) section.

4.2.6 RTEMS Workspace Configuration Options

There are no guarantees provided as to the resources used by the current version of STAR-System for RTEMS. The resources used by STAR-System include threads, mutexes, semaphores and condition variables. We therefore recommend setting these resources to be unlimited.

An example source file containing the definitions required to use the STAR-System PCI Driver is provided in the examples directory. This file, `star_rtems_init.c`, is used by all the example and test programs, and can also be used in your own applications. To start the Configuration Service, `STAR_CONFIG_API_REQUIRED` must be defined. This is performed in the example Makefiles by adding `-DSTAR_CONFIG_API_REQUIRED` during compilation.

4.3 Multi-Threading and Multi-Process Support

STAR-System and the various APIs provided, are designed to support multi-threaded and multi-process development.

4.3.1 Multi-Threading Support

All functions in the STAR-System APIs have been designed to be thread safe, and any function can be safely called in multiple threads at the same time.

Despite this thread-safety, there are some situations in which thread synchronisation may be required when calling STAR-System functions. If you are transmitting or receiving on a channel, in most situations you will only transmit from one thread at a time, or receive from one thread at a time. If, however, you wish to transmit from two different threads on the same channel, there is no guarantee of the order in which the packets will be sent, unless synchronisation is used between the threads. If you don't care about the order in which the packets are transmitted, one other thing to consider is that you should always transmit a full packet or make use of synchronisation between the threads. If you transmit the start of a packet, then transmit the end of the packet in a separate function call, the other thread may have transmitted a packet (or chunk of a packet) in the middle of this packet. Similar considerations should be made when receiving on one channel in multiple threads.

Note that the above doesn't apply to transmitting in one thread while receiving in another thread. These are completely independent operations, and it's unlikely synchronisation will be required, unless buffers or other resources are being shared between the two threads and may be modified or destroyed.

The documentation for individual functions highlight any potential issues when using these functions with multiple threads.

4.3.2 Multi-Process Support

STAR-System has also been designed to allow multiple applications to access a device at the same time. Any number of applications can call functions in the STAR-System APIs, and information such as device names can be set in one application and read in another.

Multiple applications can transmit and/or receive on a device at the same time. This is achieved through the use of channels, which must be opened by an application to a device before packets can be transmitted to that device or received from the device. Each application can open a different channel to a device, allowing one application to transmit to a device while another receives from the device, or for one application to transmit on link 1 while another transmits on link 2. See the [Channel Management](#) section of the STAR-API documentation for further information.

The Device Configuration APIs have been designed so that any number of applications (or threads) can query and/or configure a device simultaneously. The Device Configuration Service of STAR-System handles the synchronisation between each of the applications. See the [Device Configuration Service](#) for further information.

4.4 Device Configuration Service

The Device Configuration Service is a small application, named `star_conf_service.exe` on Windows and `star_conf_service` on other platforms, which runs in the background at all times.

The primary purpose of the Device Configuration Service is to allow multiple applications and/or threads to perform Device Configuration functions, either locally or over a SpaceWire network, all at the same time. It is responsible for ensuring each application/thread receives a response to the commands it initiated, and is effectively providing multiplexing/demultiplexing to allow multiple applications and threads to access a single resource. In addition, the Device Configuration Service also maintains state information, such as the channels which are open, the names assigned to devices, etc.

The intention of the Device Configuration Service is that it is always running. This reduces the time required for the first API function call to complete (as it doesn't need to wait on the process starting), and also allows the service to maintain state information after an application closes. By default, the Configuration Service will run at start-up on

Windows, Linux and QNX. When no STAR-System application is running, the process will not be doing anything, so it should be using very few resources. On VxWorks and RTEMS the service runs in a separate thread of the application, and must be started manually using a function call. See the [VxWorks Installation and Getting Started Guide](#) and [RTEMS Installation and Getting Started Guide](#) for further information.

4.5 Error Logs

STAR-System uses an error log to indicate any serious errors encountered during its operation.

If you encounter any unexpected behaviour, or have trouble debugging an issue, check the error log to see if it can provide you with useful information.

4.5.1 Log File Name and Location

The error log will be located in your user directory, e.g. on Windows Vista onwards:

```
C:\Users\Stuart\
```

on Windows XP:

```
C:\Documents and Settings\Stuart\
```

on Linux and QNX:

```
/home/stuart/
```

or if you're logged in as root on Linux and QNX:

```
/root/
```

The log file is named `star-system.log`, and will never grow above a fixed size (which is currently 1 MByte). When the maximum size is reached, the file is renamed (to `star-system.0.log`) and a new log file is started. If log files have already been archived in this manner, they will be renamed to e.g. `star-system.1.log` before `star-system.0.log` is created. The maximum number of archived logs that will be maintained is currently four. The oldest archived log will be deleted when a fifth is created.

4.5.2 Log File Format

The log file format is quite simple:

```
<Date> <Time> <Message Type> [<Module Name>] <Message>
```

A typical error message looks as follows:

```
24-08-2011 15:39:53 ERROR [STAR-API] Could not communicate with Device Configuration Service
```

The `<Date>` and `<Time>` are the date and time provided by the PC's clock at which the issue occurred. The `<Message Type>` contains one of the following possible values `ERROR`, `WARNING`, `DEBUG` or `TRACE`, indicating the severity of the issue. Debug and trace messages should only be displayed if you have been provided with a special build of a module in order to debug a problem. The `<Module Name>` indicates the module which generated the log message. Possible values for the module name include:

- STAR-API
- Driver Interface

- STAR-Port
- STAR Device Configuration Messenger
- STAR Device Configuration Service
- STAR RMAP Initiator

The Device Configuration APIs may also output messages to the log file.

If you encounter problems with any of these modules, please contact STAR-Dundee's support team for further assistance using the e-mail address support@star-dundee.com.

4.6 Configuration API Functions Supported by Device Types

The following tables show the features provided by each device and the Configuration API functions which can be used to access them.

The Generic Configuration API provides functions for configuring all features on all devices. If a feature is not supported by a device, the API function calls will return a value to indicate this. Please refer to the [Generic Configuration API](#) documentation for further information and [cfg_api_generic.h](#) for the list of possible return values.

The other Configuration APIs were used to configure devices prior to the introduction of the Generic Configuration API in v4.00 of STAR-System. These APIs provide functions which are often limited to a range or single device. It is recommended that the Generic Configuration API is used for all new developments.

4.6.1 Generic Configuration API

Your device may require a firmware update to be compatible with some Generic Configuration API function calls. For any such calls, the required firmware version is documented below.

Feature	Device	Function
Get device identification information	Router Mk2S	Yes CFG_getDeviceIdentificationInfo()
	Brick Mk2	Yes CFG_getDeviceManufacturerAs↵String()
	Brick Mk3	Yes CFG_getDeviceTypeAsString()
	PCI Mk2 / cPCI	Yes CFG_getNetworkDiscoveryInfo()
	PCIe	Yes
	Physical Layer Tester	Yes
	PXI Interface	Yes
	PXI Router	Yes
	PXI Interface Mk2	Yes
	PXI Router Mk2	Yes
Port status and control	Router Mk2S	Yes CFG_getPortStatusControl()
	Brick Mk2	Yes CFG_getPortType()
	Brick Mk3	Yes CFG_getPortConnection()
	PCI Mk2 / cPCI	Yes CFG_getConfigPortErrors()
	PCIe	Yes CFG_getSpaceWireLinkErrors()
	Physical Layer Tester	Yes CFG_getExternalPortErrors()
	PXI Interface	Yes CFG_getExternalPortStatus()
	PXI Router	Yes
	PXI Interface Mk2	Yes
	PXI Router Mk2	Yes

Clear port errors	Router Mk2S Brick Mk2 Brick Mk3 PCI Mk2 / cPCI PCle Physical Layer Tester PXI Interface PXI Router PXI Interface Mk2 PXI Router Mk2	Yes Yes Yes Yes Yes Yes Yes Yes Yes Yes Yes	CFG_clearPortErrors()
Get and set link status	Router Mk2S Brick Mk2 Brick Mk3 PCI Mk2 / cPCI PCle Physical Layer Tester PXI Interface PXI Router PXI Interface Mk2 PXI Router Mk2	Yes Yes Yes Yes Yes Yes Yes Yes Yes Yes Yes	CFG_getSpaceWireLinkStatus() CFG_setSpaceWireLinkStatus()
Start and stop links	Router Mk2S Brick Mk2 Brick Mk3 PCI Mk2 / cPCI PCle Physical Layer Tester PXI Interface PXI Router PXI Interface Mk2 PXI Router Mk2	Yes Yes Yes Yes Yes Yes Yes Yes Yes Yes Yes	CFG_startLink() CFG_stopLink()
Get and set routing table entries	Router Mk2S Brick Mk2 Brick Mk3 PCI Mk2 / cPCI PCle Physical Layer Tester PXI Interface PXI Router PXI Interface Mk2 PXI Router Mk2	Yes Yes Yes Yes Yes Yes Yes Yes Yes Yes Yes	CFG_getRoutingTableEntry() CFG_setRoutingTableEntry()

Get and set network identity	Router Mk2S Brick Mk2 Brick Mk3 PCI Mk2 / cPCI PCle Physical Layer Tester PXI Interface PXI Router PXI Interface Mk2 PXI Router Mk2	Yes Yes Yes Yes Yes Yes Yes Yes Yes Yes Yes	CFG_getNetworkIdentity() CFG_setNetworkIdentity()
Get and set general purpose register	Router Mk2S Brick Mk2 Brick Mk3 PCI Mk2 / cPCI PCle Physical Layer Tester PXI Interface PXI Router PXI Interface Mk2 PXI Router Mk2	Yes Yes Yes Yes Yes Yes Yes Yes Yes Yes Yes	CFG_getGeneralPurpose() CFG_setGeneralPurpose()
Get and set time-code distribution ports	Router Mk2S Brick Mk2 Brick Mk3 PCI Mk2 / cPCI PCle Physical Layer Tester PXI Interface PXI Router PXI Interface Mk2 PXI Router Mk2	Yes Yes Yes Yes Yes Yes Yes Yes Yes Yes Yes	CFG_getTimeCodeDistribution↔ Ports() CFG_setTimeCodeDistribution↔ Ports()
Get and set router global settings	Router Mk2S Brick Mk2 Brick Mk3 PCI Mk2 / cPCI PCle Physical Layer Tester PXI Interface PXI Router PXI Interface Mk2 PXI Router Mk2	Yes Yes Yes Yes Yes Yes Yes Yes Yes Yes Yes	CFG_getRouterGlobalSettings() CFG_setRouterGlobalSettings()

Get current time-code value	Router Mk2S Brick Mk2 Brick Mk3 PCI Mk2 / cPCI PCle Physical Layer Tester PXI Interface PXI Router PXI Interface Mk2 PXI Router Mk2	Yes Yes Yes Yes Yes Yes Yes Yes Yes Yes Yes	CFG_getTimeCodeValue()
Get and set time-code flag mode	Router Mk2S Brick Mk2 Brick Mk3 PCI Mk2 / cPCI PCle Physical Layer Tester PXI Interface PXI Router PXI Interface Mk2 PXI Router Mk2	Yes Yes Yes Yes Yes Yes Yes Yes Yes Yes Yes	CFG_getTimeCodeFlagMode() CFG_setTimeCodeFlagMode()
Enable and disable time-code master	Router Mk2S Brick Mk2 Brick Mk3 PCI Mk2 / cPCI PCle Physical Layer Tester PXI Interface PXI Router PXI Interface Mk2 PXI Router Mk2	Yes Yes Yes Yes Yes Yes Yes Yes Yes Yes Yes	CFG_enableTimeCodeMaster() CFG_disableTimeCodeMaster() CFG_getTimeCodeMaster↔ Enabled()
Get and set time-code period	Router Mk2S Brick Mk2 Brick Mk3 PCI Mk2 / cPCI PCle Physical Layer Tester PXI Interface PXI Router PXI Interface Mk2 PXI Router Mk2	Yes Yes Yes Yes Yes Yes Yes Yes Yes Yes Yes	CFG_getTimeCodePeriod() CFG_setTimeCodePeriod()

Enable and disable external time-code selection	Router Mk2S Brick Mk2 Brick Mk3 PCI Mk2 / cPCI PCle Physical Layer Tester PXI Interface PXI Router PXI Interface Mk2 PXI Router Mk2	Yes Yes Yes Yes Yes Yes Yes Yes Yes Yes Yes	CFG_enableExternalTimeCode↔ Selection() CFG_disableExternalTimeCode↔ Selection() CFG_getExternalTimeCode↔ SelectionEnabled()
Get FPGA information	Router Mk2S Brick Mk2 Brick Mk3 PCI Mk2 / cPCI PCle Physical Layer Tester PXI Interface PXI Router PXI Interface Mk2 PXI Router Mk2	Yes Yes Yes Yes Yes Yes Yes Yes Yes Yes Yes	CFG_getFPGAInfo() CFG_FPGAInfoToString()
Identify device	Router Mk2S Brick Mk2 Brick Mk3 PCI Mk2 / cPCI PCle Physical Layer Tester PXI Interface PXI Router PXI Interface Mk2 PXI Router Mk2	Yes Yes Yes Yes Yes Yes Yes Yes Yes Yes Yes	CFG_identify()
Enable and disable interface mode	Router Mk2S Brick Mk2 Brick Mk3 PCI Mk2 / cPCI PCle Physical Layer Tester PXI Interface PXI Router PXI Interface Mk2 PXI Router Mk2	Yes Yes Yes Yes Yes Yes Yes Yes Yes Yes Yes	CFG_enableInterfaceMode() CFG_disableInterfaceMode() CFG_getInterfaceModeEnabled()

Enable and disable identify source	Router Mk2S Brick Mk2 Brick Mk3 PCI Mk2 / cPCI PCle Physical Layer Tester PXI Interface PXI Router PXI Interface Mk2 PXI Router Mk2	Yes Yes Yes Yes Yes Yes Yes Yes Yes Yes Yes	CFG_enableIdentifySource() CFG_disableIdentifySource() CFG_getIdentifySourceEnabled()
Enable and disable interface mode on port	Router Mk2S Brick Mk2 Brick Mk3 PCI Mk2 / cPCI PCle Physical Layer Tester PXI Interface PXI Router PXI Interface Mk2 PXI Router Mk2	Yes Yes Yes Yes Yes Yes Yes Yes Yes Yes Yes	CFG_enableInterfaceModeOn↔ Port() CFG_disableInterfaceModeOn↔ Port() CFG_getInterfaceModeOnPort↔ Enabled()
Enable and disable identify source on port	Router Mk2S Brick Mk2 Brick Mk3 PCI Mk2 / cPCI PCle Physical Layer Tester PXI Interface PXI Router PXI Interface Mk2 PXI Router Mk2	Yes Yes Yes Yes Yes Yes Yes Yes Yes Yes Yes	CFG_enableIdentifySourceOn↔ Port() CFG_disableIdentifySourceOn↔ Port() CFG_getIdentifySourceOnPort↔ Enabled()
Get and set port routing address	Router Mk2S Brick Mk2 Brick Mk3 PCI Mk2 / cPCI PCle Physical Layer Tester PXI Interface PXI Router PXI Interface Mk2 PXI Router Mk2	Yes Yes Yes Yes Yes Yes Yes Yes Yes Yes Yes	CFG_getPortRoutingAddress() CFG_setPortRoutingAddress()
Inject error	Router Mk2S Brick Mk2 Brick Mk3 PCI Mk2 / cPCI PCle Physical Layer Tester PXI Interface PXI Router PXI Interface Mk2 PXI Router Mk2	Yes Yes Yes Yes Yes Yes Yes Yes Yes Yes Yes	CFG_injectError()

Inject errors	Router Mk2S Brick Mk2 Brick Mk3 PCI Mk2 / cPCI PCle Physical Layer Tester PXI Interface PXI Router PXI Interface Mk2 PXI Router Mk2	Yes Yes Yes No No No No No No No No	CFG_injectErrors()
Get measured link speed	Router Mk2S Brick Mk2 Brick Mk3 PCI Mk2 / cPCI PCle Physical Layer Tester PXI Interface PXI Router PXI Interface Mk2 PXI Router Mk2	Yes Yes Yes No Yes Yes Yes Yes Yes Yes Yes	CFG_getMeasuredLinkSpeed()
Enable and disable link state change events	Router Mk2S Brick Mk2 Brick Mk3 PCI Mk2 / cPCI PCle Physical Layer Tester PXI Interface PXI Router PXI Interface Mk2 PXI Router Mk2	Yes Yes Yes No Yes Yes Yes Yes Yes Yes Yes	CFG_enableStateChange↔ EventsOnPort() CFG_disableStateChange↔ EventsOnPort() CFG_getStateChangeEventsOn↔ PortEnabled()
Enable and disable link speed change events	Router Mk2S Brick Mk2 Brick Mk3 PCI Mk2 / cPCI PCle Physical Layer Tester PXI Interface PXI Router PXI Interface Mk2 PXI Router Mk2	Yes Yes Yes No Yes Yes Yes Yes Yes Yes Yes	CFG_enableSpeedChange↔ EventsOnPort() CFG_disableSpeedChange↔ EventsOnPort() CFG_getSpeedChangeEvents↔ OnPortEnabled()

Get and set transmit signalling rate	Router Mk2S Brick Mk2 Brick Mk3 (from v1.1) PCI Mk2 / cPCI PCIe (from v1.11) P. L. Tester (from v2.0) PXI Interface (from v1.2.3) PXI Router (from v1.2.4) PXI Interface Mk2 PXI Router Mk2	Yes Yes Yes Yes Yes Yes Yes Yes Yes Yes	CFG_getTransmitSignallingRate() CFG_setTransmitSignallingRate()
Precision transmit rate	Router Mk2S Brick Mk2 Brick Mk3 PCI Mk2 / cPCI PCIe Physical Layer Tester PXI Interface PXI Router PXI Interface Mk2 PXI Router Mk2	Yes No No No No No No No No No	CFG_enablePrecisionTransmitRate() CFG_disablePrecisionTransmitRate() CFG_getPrecisionTransmitRateEnabled() CFG_getPrecisionTransmitRateInUse() CFG_getPrecisionTransmitRate() CFG_setPrecisionTransmitRate()
Timestamping	Router Mk2S Brick Mk2 Brick Mk3 (from v1.2) PCI Mk2 / cPCI PCIe Physical Layer Tester PXI Interface PXI Router PXI Interface Mk2 PXI Router Mk2	No No Yes No No No Yes No Yes No	CFG_setTimestampMethod() CFG_getTimestampMethod() CFG_enableRxTimestampEventsOnPort() CFG_getRxTimestampEventsOnPortEnabled() CFG_disableRxTimestampEventsOnPort() CFG_setTimestampValue() CFG_getTimestampValue() CFG_setPulseGeneratorFrequency() CFG_getPulseGeneratorFrequency()
Periodic actions	Router Mk2S Brick Mk2 Brick Mk3 PCI Mk2 / cPCI PCIe Physical Layer Tester PXI Interface PXI Router PXI Interface Mk2 PXI Router Mk2	No No No Yes No No No No No No	CFG_getDeviceClockRate() CFG_startPeriodicAction() CFG_stopPeriodicAction()

Enable and disable time-code counter bypass mode	Router Mk2S	No	CFG_enableTimeCodeCounter↔
	Brick Mk2	No	BypassMode()
	Brick Mk3	No	CFG_disableTimeCodeCounter↔
	PCI Mk2 / cPCI	No	BypassMode()
	PCIe	Yes	CFG_getTimeCodeCounter↔
	Physical Layer Tester	No	BypassModeEnabled()
	PXI Interface	No	
	PXI Router	No	
	PXI Interface Mk2	No	
	PXI Router Mk2	No	
Enable and disable time-code events	Router Mk2S	No	CFG_enableTimeCodeEvents↔
	Brick Mk2	No	OnPort()
	Brick Mk3	No	CFG_disableTimeCodeEvents↔
	PCI Mk2 / cPCI	No	OnPort()
	PCIe	Yes	CFG_getTimeCodeEventsOn↔
	Physical Layer Tester	No	PortEnabled()
	PXI Interface (from v1.2.3)	Yes	
	PXI Router (from v1.2.4)	Yes	
	PXI Interface Mk2	Yes	
	PXI Router Mk2	Yes	

4.6.2 SpaceWire cPCI Mk2 and SpaceWire PCI Mk2

Further information on the features present in the SpaceWire cPCI Mk2 and PCI Mk2 can be found in the [SpaceWire PCI Mk2 and cPCI Mk2 user manual](#).

Feature	Function
Get device identification information	CFG_ROUTER_getDeviceIdentificationInfo() CFG_ROUTER_getDeviceManufacturerAsString() CFG_ROUTER_getDeviceTypeAsString() CFG_ROUTER_getNetworkDiscoveryInfo()
Port status and control	CFG_ROUTER_getPortStatusControl() CFG_ROUTER_getPortType() CFG_ROUTER_getPortConnection() CFG_ROUTER_getConfigPortErrors() CFG_ROUTER_getSpaceWireLinkErrors() CFG_ROUTER_getExternalPortErrors() CFG_ROUTER_getExternalPortStatus()
Clear port errors	CFG_ROUTER_clearPortErrors()
Get and set link status	CFG_ROUTER_getSpaceWireLinkStatus() CFG_ROUTER_setSpaceWireLinkStatus()
Start and stop links	CFG_ROUTER_startLink() CFG_ROUTER_stopLink()
Get and set routing table entries	CFG_ROUTER_getRoutingTableEntry() CFG_ROUTER_setRoutingTableEntry()
Get and set network identity	CFG_ROUTER_getNetworkIdentity() CFG_ROUTER_setNetworkIdentity()
Get and set general purpose register	CFG_ROUTER_getGeneralPurpose() CFG_ROUTER_setGeneralPurpose()
Get and set time-code distribution ports	CFG_ROUTER_getTimeCodeDistributionPorts() CFG_ROUTER_setTimeCodeDistributionPorts()
Get and set router global settings	CFG_ROUTER_getRouterGlobalSettings() CFG_ROUTER_setRouterGlobalSettings()
Get current time-code value	CFG_ROUTER_getTimeCodeValue()
Get and set time-code flag mode	CFG_ROUTER_getTimeCodeFlagMode() CFG_ROUTER_setTimeCodeFlagMode()
Get and set base transmit clock	CFG_PCIMK2_getLinkClockFrequency() CFG_PCIMK2_setLinkClockFrequency()
Get and set link rate divider	CFG_MK2_getLinkRateDivider() CFG_MK2_setLinkRateDivider()
Enable and disable time-code master	CFG_MK2_enableTimeCodeMaster() CFG_MK2_disableTimeCodeMaster() CFG_MK2_getTimeCodeMasterEnabled()
Get and set time-code period	CFG_MK2_getTimeCodePeriod() CFG_MK2_setTimeCodePeriod()
Enable and disable external time-code selection	CFG_MK2_enableExternalTimeCodeSelection() CFG_MK2_disableExternalTimeCodeSelection() CFG_MK2_getExternalTimeCodeSelectionEnabled()
Enable and disable time-code counter bypass mode	<i>Not supported</i>
Get hardware information	CFG_MK2_getHardwareInfo() CFG_MK2_hardwareInfoToString()
Identify device	CFG_MK2_identify()
Enable and disable interface mode	CFG_MK2_enableInterfaceMode() CFG_MK2_disableInterfaceMode() CFG_MK2_getInterfaceModeEnabled()

Enable and disable identify source	CFG_MK2_enableIdentifySource() CFG_MK2_disableIdentifySource() CFG_MK2_getIdentifySourceEnabled()
Enable and disable interface mode on port	CFG_MK2_enableInterfaceModeOnPort() CFG_MK2_disableInterfaceModeOnPort() CFG_MK2_getInterfaceModeOnPortEnabled()
Enable and disable identify source on port	CFG_MK2_enableIdentifySourceOnPort() CFG_MK2_disableIdentifySourceOnPort() CFG_MK2_getIdentifySourceOnPortEnabled()
Get and set port routing address	CFG_MK2_getPortRoutingAddress() CFG_MK2_setPortRoutingAddress()
Inject errors	CFG_MK2_injectError()
Periodic actions	CFG_MK2_getDeviceClockRate() CFG_MK2_startPeriodicAction() CFG_MK2_stopPeriodicAction()
Enable and disable link state change events	<i>Not supported</i>
Enable and disable link speed change events	<i>Not supported</i>
Enable and disable time-code events	<i>Not supported</i>
Get measured link speed	<i>Not supported</i>
Enable and disable precision transmit rate	<i>Not supported</i>
Get and set precision transmit rate	<i>Not supported</i>
Timestamping	<i>Not supported</i>

4.6.3 SpaceWire PCIe and SpaceWire Physical Layer Tester

Further information on the features present in the SpaceWire PCIe can be found in the [SpaceWire PCIe](#) user manual. A brief overview of the [SpaceWire Physical Layer Tester](#) is provided in the [Other STAR-System Devices](#) section.

Feature	Function
Get device identification information	CFG_ROUTER_getDeviceIdentificationInfo() CFG_ROUTER_getDeviceManufacturerAsString() CFG_ROUTER_getDeviceTypeAsString() CFG_ROUTER_getNetworkDiscoveryInfo()
Port status and control	CFG_ROUTER_getPortStatusControl() CFG_ROUTER_getPortType() CFG_ROUTER_getPortConnection() CFG_ROUTER_getConfigPortErrors() CFG_ROUTER_getSpaceWireLinkErrors() CFG_ROUTER_getExternalPortErrors() CFG_ROUTER_getExternalPortStatus()
Clear port errors	CFG_ROUTER_clearPortErrors()
Get and set link status	CFG_ROUTER_getSpaceWireLinkStatus() CFG_ROUTER_setSpaceWireLinkStatus()
Start and stop links	CFG_ROUTER_startLink() CFG_ROUTER_stopLink()
Get and set routing table entries	CFG_ROUTER_getRoutingTableEntry() CFG_ROUTER_setRoutingTableEntry()
Get and set network identity	CFG_ROUTER_getNetworkIdentity() CFG_ROUTER_setNetworkIdentity()
Get and set general purpose register	CFG_ROUTER_getGeneralPurpose() CFG_ROUTER_setGeneralPurpose()
Get and set time-code distribution ports	CFG_ROUTER_getTimeCodeDistributionPorts() CFG_ROUTER_setTimeCodeDistributionPorts()
Get and set router global settings	CFG_ROUTER_getRouterGlobalSettings() CFG_ROUTER_setRouterGlobalSettings()
Get current time-code value	CFG_ROUTER_getTimeCodeValue()
Get and set time-code flag mode	CFG_ROUTER_getTimeCodeFlagMode() CFG_ROUTER_setTimeCodeFlagMode()
Get and set base transmit clock	CFG_MK2_getBaseTransmitClock() CFG_MK2_setBaseTransmitClock() CFG_MK2_getTransmitClock() CFG_MK2_setTransmitClock()
Get and set link rate divider	CFG_MK2_getLinkRateDivider() CFG_MK2_setLinkRateDivider()
Enable and disable time-code master	CFG_MK2_enableTimeCodeMaster() CFG_MK2_disableTimeCodeMaster() CFG_MK2_getTimeCodeMasterEnabled()
Get and set time-code period	CFG_MK2_getTimeCodePeriod() CFG_MK2_setTimeCodePeriod()
Enable and disable external time-code selection	CFG_MK2_enableExternalTimeCodeSelection() CFG_MK2_disableExternalTimeCodeSelection() CFG_MK2_getExternalTimeCodeSelectionEnabled()

Enable and disable time-code counter bypass mode (PCIe only)	CFG_MK2_enableTimeCodeCounterBypassMode() CFG_MK2_disableTimeCodeCounterBypassMode() CFG_MK2_getTimeCodeCounterBypassModeEnabled()
Get hardware information	CFG_MK2_getHardwareInfo() CFG_MK2_hardwareInfoToString()
Identify device	CFG_MK2_identify()
Enable and disable interface mode	CFG_MK2_enableInterfaceMode() CFG_MK2_disableInterfaceMode() CFG_MK2_getInterfaceModeEnabled()
Enable and disable identify source	CFG_MK2_enableIdentifySource() CFG_MK2_disableIdentifySource() CFG_MK2_getIdentifySourceEnabled()
Enable and disable interface mode on port	CFG_MK2_enableInterfaceModeOnPort() CFG_MK2_disableInterfaceModeOnPort() CFG_MK2_getInterfaceModeOnPortEnabled()
Enable and disable identify source on port	CFG_MK2_enableIdentifySourceOnPort() CFG_MK2_disableIdentifySourceOnPort() CFG_MK2_getIdentifySourceOnPortEnabled()
Get and set port routing address	CFG_MK2_getPortRoutingAddress() CFG_MK2_setPortRoutingAddress()
Inject errors	CFG_MK2_injectError()
Periodic actions	<i>Not supported</i>
Enable and disable link state change events	CFG_MK2_enableStateChangeEventsOnPort() CFG_MK2_disableStateChangeEventsOnPort() CFG_MK2_getStateChangeEventsOnPortEnabled()
Enable and disable link speed change events	CFG_MK2_enableSpeedChangeEventsOnPort() CFG_MK2_disableSpeedChangeEventsOnPort() CFG_MK2_getSpeedChangeEventsOnPortEnabled()
Enable and disable time-code events (PCIe only)	CFG_MK2_enableTimeCodeEventsOnPort() CFG_MK2_disableTimeCodeEventsOnPort() CFG_MK2_getTimeCodeEventsOnPortEnabled()
Get measured link speed	CFG_MK2_getMeasuredLinkSpeed()
Enable and disable precision transmit rate	<i>Not supported</i>
Get and set precision transmit rate	<i>Not supported</i>
Timestamping	<i>Not supported</i>

4.6.4 SpaceWire Brick Mk2

Further information on the features present in the SpaceWire Brick Mk2 can be found in the [SpaceWire Brick Mk2 user manual](#).

Feature	Function
Get device identification information	CFG_ROUTER_getDeviceIdentificationInfo() CFG_ROUTER_getDeviceManufacturerAsString() CFG_ROUTER_getDeviceTypeAsString() CFG_ROUTER_getNetworkDiscoveryInfo()
Port status and control	CFG_ROUTER_getPortStatusControl() CFG_ROUTER_getPortType() CFG_ROUTER_getPortConnection() CFG_ROUTER_getConfigPortErrors() CFG_ROUTER_getSpaceWireLinkErrors() CFG_ROUTER_getExternalPortErrors() CFG_ROUTER_getExternalPortStatus()
Clear port errors	CFG_ROUTER_clearPortErrors()
Get and set link status	CFG_ROUTER_getSpaceWireLinkStatus() CFG_ROUTER_setSpaceWireLinkStatus()
Start and stop links	CFG_ROUTER_startLink() CFG_ROUTER_stopLink()
Get and set routing table entries	CFG_ROUTER_getRoutingTableEntry() CFG_ROUTER_setRoutingTableEntry()
Get and set network identity	CFG_ROUTER_getNetworkIdentity() CFG_ROUTER_setNetworkIdentity()
Get and set general purpose register	CFG_ROUTER_getGeneralPurpose() CFG_ROUTER_setGeneralPurpose()
Get and set time-code distribution ports	CFG_MK2_getTimeCodeDistributionPorts() CFG_MK2_setTimeCodeDistributionPorts()
Get and set router global settings	CFG_ROUTER_getRouterGlobalSettings() CFG_ROUTER_setRouterGlobalSettings()
Get current time-code value	CFG_ROUTER_getTimeCodeValue()
Get and set time-code flag mode	CFG_MK2_getTimeCodeFlagMode() CFG_MK2_setTimeCodeFlagMode()
Get and set base transmit clock	CFG_BRICK_MK2_getLinkClockFrequency() CFG_BRICK_MK2_setLinkClockFrequency()
Get and set link rate divider	CFG_MK2_getLinkRateDivider() CFG_MK2_setLinkRateDivider()
Enable and disable time-code master	CFG_MK2_enableTimeCodeMaster() CFG_MK2_disableTimeCodeMaster() CFG_MK2_getTimeCodeMasterEnabled()
Get and set time-code period	CFG_MK2_getTimeCodePeriod() CFG_MK2_setTimeCodePeriod()
Enable and disable external time-code selection	CFG_MK2_enableExternalTimeCodeSelection() CFG_MK2_disableExternalTimeCodeSelection() CFG_MK2_getExternalTimeCodeSelectionEnabled()
Enable and disable time-code counter bypass mode	<i>Not supported</i>
Get hardware information	CFG_MK2_getHardwareInfo() CFG_MK2_hardwareInfoToString()
Identify device	CFG_MK2_identify()
Enable and disable interface mode	CFG_MK2_enableInterfaceMode() CFG_MK2_disableInterfaceMode() CFG_MK2_getInterfaceModeEnabled()

Enable and disable identify source	CFG_MK2_enableIdentifySource() CFG_MK2_disableIdentifySource() CFG_MK2_getIdentifySourceEnabled()
Enable and disable interface mode on port	CFG_BRICK_MK2_enableInterfaceModeOnPort() CFG_BRICK_MK2_disableInterfaceModeOnPort() CFG_BRICK_MK2_getInterfaceModeOnPort↔Enabled()
Enable and disable identify source on port	CFG_BRICK_MK2_enableIdentifySourceOnPort() CFG_BRICK_MK2_disableIdentifySourceOnPort() CFG_BRICK_MK2_getIdentifySourceOnPort↔Enabled()
Get and set port routing address	CFG_BRICK_MK2_getPortRoutingAddress() CFG_BRICK_MK2_setPortRoutingAddress()
Inject errors	CFG_BRICK_MK2_injectErrors()
Periodic actions	<i>Not supported</i>
Enable and disable link state change events	CFG_BRICK_MK2_enableStateChangeEventsOn↔Port() CFG_BRICK_MK2_disableStateChangeEventsOn↔Port() CFG_BRICK_MK2_getStateChangeEventsOnPort↔Enabled()
Enable and disable link speed change events	CFG_BRICK_MK2_enableSpeedChangeEventsOn↔Port() CFG_BRICK_MK2_disableSpeedChangeEventsOn↔Port() CFG_BRICK_MK2_getSpeedChangeEventsOnPort↔Enabled()
Enable and disable time-code events	<i>Not supported</i>
Get measured link speed	CFG_MK2_getMeasuredLinkSpeed()
Enable and disable precision transmit rate	<i>Not supported</i>
Get and set precision transmit rate	<i>Not supported</i>
Timestamping	<i>Not supported</i>

4.6.5 SpaceWire Router Mk2S

Further information on the features present in the SpaceWire Router Mk2S can be found in the [SpaceWire Router Mk2S user manual](#).

Feature	Function
Get device identification information	CFG_ROUTER_getDeviceIdentificationInfo() CFG_ROUTER_getDeviceManufacturerAsString() CFG_ROUTER_getDeviceTypeAsString() CFG_ROUTER_getNetworkDiscoveryInfo()
Port status and control	CFG_ROUTER_getPortStatusControl() CFG_ROUTER_getPortType() CFG_ROUTER_getPortConnection() CFG_ROUTER_getConfigPortErrors() CFG_ROUTER_getSpaceWireLinkErrors() CFG_ROUTER_getExternalPortErrors() CFG_ROUTER_getExternalPortStatus()
Clear port errors	CFG_ROUTER_clearPortErrors()
Get and set link status	CFG_ROUTER_getSpaceWireLinkStatus() CFG_ROUTER_setSpaceWireLinkStatus()
Start and stop links	CFG_ROUTER_startLink() CFG_ROUTER_stopLink()
Get and set routing table entries	CFG_ROUTER_getRoutingTableEntry() CFG_ROUTER_setRoutingTableEntry()
Get and set network identity	CFG_ROUTER_getNetworkIdentity() CFG_ROUTER_setNetworkIdentity()
Get and set general purpose register	CFG_ROUTER_getGeneralPurpose() CFG_ROUTER_setGeneralPurpose()
Get and set time-code distribution ports	CFG_MK2_getTimeCodeDistributionPorts() CFG_MK2_setTimeCodeDistributionPorts()
Get and set router global settings	CFG_ROUTER_getRouterGlobalSettings() CFG_ROUTER_setRouterGlobalSettings()
Get current time-code value	CFG_ROUTER_getTimeCodeValue()
Get and set time-code flag mode	CFG_MK2_getTimeCodeFlagMode() CFG_MK2_setTimeCodeFlagMode()
Get and set base transmit clock	CFG_BRICK_MK2_getLinkClockFrequency() CFG_BRICK_MK2_setLinkClockFrequency()
Get and set link rate divider	CFG_MK2_getLinkRateDivider() CFG_MK2_setLinkRateDivider()
Enable and disable time-code master	CFG_MK2_enableTimeCodeMaster() CFG_MK2_disableTimeCodeMaster() CFG_MK2_getTimeCodeMasterEnabled()
Get and set time-code period	CFG_MK2_getTimeCodePeriod() CFG_MK2_setTimeCodePeriod()
Enable and disable external time-code selection	CFG_MK2_enableExternalTimeCodeSelection() CFG_MK2_disableExternalTimeCodeSelection() CFG_MK2_getExternalTimeCodeSelectionEnabled()
Enable and disable time-code counter bypass mode	<i>Not supported</i>
Get hardware information	CFG_MK2_getHardwareInfo() CFG_MK2_hardwareInfoToString()
Identify device	CFG_MK2_identify()
Enable and disable interface mode	CFG_MK2_enableInterfaceMode() CFG_MK2_disableInterfaceMode() CFG_MK2_getInterfaceModeEnabled()

Enable and disable identify source	CFG_MK2_enableIdentifySource() CFG_MK2_disableIdentifySource() CFG_MK2_getIdentifySourceEnabled()
Enable and disable interface mode on port	CFG_BRICK_MK2_enableInterfaceModeOnPort() CFG_BRICK_MK2_disableInterfaceModeOnPort() CFG_BRICK_MK2_getInterfaceModeOnPort↔Enabled()
Enable and disable identify source on port	CFG_BRICK_MK2_enableIdentifySourceOnPort() CFG_BRICK_MK2_disableIdentifySourceOnPort() CFG_BRICK_MK2_getIdentifySourceOnPort↔Enabled()
Get and set port routing address	CFG_BRICK_MK2_getPortRoutingAddress() CFG_BRICK_MK2_setPortRoutingAddress()
Inject errors	CFG_BRICK_MK2_injectErrors()
Periodic actions	<i>Not supported</i>
Enable and disable link state change events	CFG_BRICK_MK2_enableStateChangeEventsOn↔Port() CFG_BRICK_MK2_disableStateChangeEventsOn↔Port() CFG_BRICK_MK2_getStateChangeEventsOnPort↔Enabled()
Enable and disable link speed change events	CFG_BRICK_MK2_enableSpeedChangeEventsOn↔Port() CFG_BRICK_MK2_disableSpeedChangeEventsOn↔Port() CFG_BRICK_MK2_getSpeedChangeEventsOnPort↔Enabled()
Enable and disable time-code events	<i>Not supported</i>
Get measured link speed	CFG_MK2_getMeasuredLinkSpeed()
Enable and disable precision transmit rate	CFG_ROUTER_MK2S_enablePrecisionTransmit↔Rate() CFG_ROUTER_MK2S_disablePrecisionTransmit↔Rate() CFG_ROUTER_MK2S_getPrecisionTransmitRate↔Enabled() CFG_ROUTER_MK2S_getPrecisionTransmitRate↔InUse()
Get and set precision transmit rate	CFG_ROUTER_MK2S_getPrecisionTransmitRate() CFG_ROUTER_MK2S_setPrecisionTransmitRate()
Timestamping	<i>Not supported</i>

4.6.6 SpaceWire Brick Mk3

Further information on the features present in the SpaceWire Brick Mk3 can be found in the [SpaceWire Brick Mk3 user manual](#).

Feature	Function
Get device identification information	CFG_ROUTER_getDeviceIdentificationInfo() CFG_ROUTER_getDeviceManufacturerAsString() CFG_ROUTER_getDeviceTypeAsString() CFG_ROUTER_getNetworkDiscoveryInfo()
Port status and control	CFG_ROUTER_getPortStatusControl() CFG_ROUTER_getPortType() CFG_ROUTER_getPortConnection() CFG_ROUTER_getConfigPortErrors() CFG_ROUTER_getSpaceWireLinkErrors() CFG_ROUTER_getExternalPortErrors() CFG_ROUTER_getExternalPortStatus()
Clear port errors	CFG_ROUTER_clearPortErrors()
Get and set link status	CFG_ROUTER_getSpaceWireLinkStatus() CFG_ROUTER_setSpaceWireLinkStatus()
Start and stop links	CFG_ROUTER_startLink() CFG_ROUTER_stopLink()
Get and set routing table entries	CFG_ROUTER_getRoutingTableEntry() CFG_ROUTER_setRoutingTableEntry()
Get and set network identity	CFG_ROUTER_getNetworkIdentity() CFG_ROUTER_setNetworkIdentity()
Get and set general purpose register	CFG_ROUTER_getGeneralPurpose() CFG_ROUTER_setGeneralPurpose()
Get and set time-code distribution ports	CFG_MK2_getTimeCodeDistributionPorts() CFG_MK2_setTimeCodeDistributionPorts()
Get and set router global settings	CFG_ROUTER_getRouterGlobalSettings() CFG_ROUTER_setRouterGlobalSettings()
Get current time-code value	CFG_ROUTER_getTimeCodeValue()
Get and set time-code flag mode	CFG_MK2_getTimeCodeFlagMode() CFG_MK2_setTimeCodeFlagMode()
Get and set base transmit clock	CFG_BRICK_MK3_getBaseTransmitClock() CFG_BRICK_MK3_setBaseTransmitClock() CFG_BRICK_MK3_getTransmitClock() CFG_BRICK_MK3_setTransmitClock()
Get and set link rate divider	CFG_MK2_getLinkRateDivider() CFG_MK2_setLinkRateDivider()
Enable and disable time-code master	CFG_MK2_enableTimeCodeMaster() CFG_MK2_disableTimeCodeMaster() CFG_MK2_getTimeCodeMasterEnabled()
Get and set time-code period	CFG_MK2_getTimeCodePeriod() CFG_MK2_setTimeCodePeriod()
Enable and disable external time-code selection	CFG_MK2_enableExternalTimeCodeSelection() CFG_MK2_disableExternalTimeCodeSelection() CFG_MK2_getExternalTimeCodeSelectionEnabled()
Enable and disable time-code counter bypass mode	<i>Not supported</i>
Get hardware information	CFG_MK2_getHardwareInfo() CFG_MK2_hardwareInfoToString()

Identify device	CFG_MK2_identify()
-----------------	--------------------

Enable and disable interface mode	CFG_MK2_enableInterfaceMode() CFG_MK2_disableInterfaceMode() CFG_MK2_getInterfaceModeEnabled()
Enable and disable identify source	CFG_MK2_enableIdentifySource() CFG_MK2_disableIdentifySource() CFG_MK2_getIdentifySourceEnabled()
Enable and disable interface mode on port	CFG_BRICK_MK2_enableInterfaceModeOnPort() CFG_BRICK_MK2_disableInterfaceModeOnPort() CFG_BRICK_MK2_getInterfaceModeOnPort↔Enabled()
Enable and disable identify source on port	CFG_BRICK_MK2_enableIdentifySourceOnPort() CFG_BRICK_MK2_disableIdentifySourceOnPort() CFG_BRICK_MK2_getIdentifySourceOnPort↔Enabled()
Get and set port routing address	CFG_BRICK_MK2_getPortRoutingAddress() CFG_BRICK_MK2_setPortRoutingAddress()
Inject errors	CFG_BRICK_MK2_injectErrors()
Periodic actions	<i>Not supported</i>
Enable and disable link state change events	CFG_BRICK_MK2_enableStateChangeEventsOn↔Port() CFG_BRICK_MK2_disableStateChangeEventsOn↔Port() CFG_BRICK_MK2_getStateChangeEventsOnPort↔Enabled()
Enable and disable link speed change events	CFG_BRICK_MK2_enableSpeedChangeEventsOn↔Port() CFG_BRICK_MK2_disableSpeedChangeEventsOn↔Port() CFG_BRICK_MK2_getSpeedChangeEventsOnPort↔Enabled()
Enable and disable time-code events	<i>Not supported</i>
Get measured link speed	CFG_MK2_getMeasuredLinkSpeed()
Enable and disable precision transmit rate	<i>Not supported</i>
Get and set precision transmit rate	<i>Not supported</i>
Timestamping	CFG_BRICK_MK3_setTimestampMethod() CFG_BRICK_MK3_getTimestampMethod() CFG_BRICK_MK3_enableRxTimestampEventsOn↔Port() CFG_BRICK_MK3_getRxTimestampEventsOnPort↔Enabled() CFG_BRICK_MK3_disableRxTimestampEventsOn↔Port() CFG_BRICK_MK3_setTimestampValue() CFG_BRICK_MK3_getTimestampValue() CFG_BRICK_MK3_setPulseGeneratorFrequency() CFG_BRICK_MK3_getPulseGeneratorFrequency()

4.6.7 SpaceWire PXI Devices

Further information on the features present in the SpaceWire PXI devices can be found in the [SpaceWire PXI Interface and PXI Interface Mk2](#) or [SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2](#) user manuals.

Additionally, for PXI devices, at power-up their SpaceWire ports default to Tri-State. This means the LVDS drivers are disabled and hence in a high impedance state.

A port's LVDS drivers can be enabled either by starting the link, by receiving data on the link, or by explicitly disabling tri-state with [CFG_ROUTER_setSpaceWireLinkStatus\(\)](#)

Feature	Function
Get device identification information	CFG_ROUTER_getDeviceIdentificationInfo() CFG_ROUTER_getDeviceManufacturerAsString() CFG_ROUTER_getDeviceTypeAsString() CFG_ROUTER_getNetworkDiscoveryInfo()
Port status and control	CFG_ROUTER_getPortStatusControl() CFG_ROUTER_getPortType() CFG_ROUTER_getPortConnection() CFG_ROUTER_getConfigPortErrors() CFG_ROUTER_getSpaceWireLinkErrors() CFG_ROUTER_getExternalPortErrors() CFG_ROUTER_getExternalPortStatus()
Clear port errors	CFG_ROUTER_clearPortErrors()
Get and set link status	CFG_ROUTER_getSpaceWireLinkStatus() CFG_ROUTER_setSpaceWireLinkStatus()
Start and stop links	CFG_ROUTER_startLink() CFG_ROUTER_stopLink()
Get and set routing table entries	CFG_ROUTER_getRoutingTableEntry() CFG_ROUTER_setRoutingTableEntry()
Get and set network identity	CFG_ROUTER_getNetworkIdentity() CFG_ROUTER_setNetworkIdentity()
Get and set general purpose register	CFG_ROUTER_getGeneralPurpose() CFG_ROUTER_setGeneralPurpose()
Get and set time-code distribution ports	CFG_MK2_getTimeCodeDistributionPorts() CFG_MK2_setTimeCodeDistributionPorts()
Get and set router global settings	CFG_ROUTER_getRouterGlobalSettings() CFG_ROUTER_setRouterGlobalSettings()
Get current time-code value	CFG_ROUTER_getTimeCodeValue()
Get and set time-code flag mode	CFG_MK2_getTimeCodeFlagMode() CFG_MK2_setTimeCodeFlagMode()
Get and set base transmit clock	CFG_PXI_getBaseTransmitClock() CFG_PXI_setBaseTransmitClock() CFG_PXI_getTransmitClock() CFG_PXI_setTransmitClock()
Get and set link rate divider	CFG_PXI_getLinkRateDivider() CFG_PXI_setLinkRateDivider()
Enable and disable time-code master	CFG_MK2_enableTimeCodeMaster() CFG_MK2_disableTimeCodeMaster() CFG_MK2_getTimeCodeMasterEnabled()
Get and set time-code period	CFG_MK2_getTimeCodePeriod() CFG_MK2_setTimeCodePeriod()
Enable and disable external time-code selection	CFG_MK2_enableExternalTimeCodeSelection() CFG_MK2_disableExternalTimeCodeSelection() CFG_MK2_getExternalTimeCodeSelectionEnabled()

Enable and disable time-code counter bypass mode	<i>Not supported</i>
Get hardware information	CFG_MK2_getHardwareInfo() CFG_MK2_hardwareInfoToString()
Identify device	CFG_MK2_identify()
Enable and disable interface mode	CFG_MK2_enableInterfaceMode() CFG_MK2_disableInterfaceMode() CFG_MK2_getInterfaceModeEnabled()
Enable and disable identify source	CFG_MK2_enableIdentifySource() CFG_MK2_disableIdentifySource() CFG_MK2_getIdentifySourceEnabled()
Enable and disable interface mode on port	CFG_PXI_enableInterfaceModeOnPort() CFG_PXI_disableInterfaceModeOnPort() CFG_PXI_getInterfaceModeOnPortEnabled()
Enable and disable identify source on port	CFG_PXI_enableIdentifySourceOnPort() CFG_PXI_disableIdentifySourceOnPort() CFG_PXI_getIdentifySourceOnPortEnabled()
Get and set port routing address	CFG_PXI_getPortRoutingAddress() CFG_PXI_setPortRoutingAddress()
Inject errors	CFG_PXI_injectError()
Periodic actions	<i>Not supported</i>
Enable and disable link state change events	CFG_PXI_enableStateChangeEventsOnPort() CFG_PXI_disableStateChangeEventsOnPort() CFG_PXI_getStateChangeEventsOnPortEnabled()
Enable and disable link speed change events	CFG_PXI_enableSpeedChangeEventsOnPort() CFG_PXI_disableSpeedChangeEventsOnPort() CFG_PXI_getSpeedChangeEventsOnPortEnabled()
Enable and disable time-code events	CFG_PXI_enableTimeCodeEventsOnPort() CFG_PXI_disableTimeCodeEventsOnPort() CFG_PXI_getTimeCodeEventsOnPortEnabled()
Get measured link speed	CFG_MK2_getMeasuredLinkSpeed()
Enable and disable precision transmit rate	<i>Not supported</i>
Get and set precision transmit rate	<i>Not supported</i>
Timestamping	<i>Not supported</i>

4.7 Static Analysis

A number of functions in the STAR-System APIs make use of SAL Annotations.

SAL is the Microsoft source-code annotation language (SAL), which provides a set of annotations to describe how a function uses its parameters, for example, the assumptions it makes about them and the guarantees it makes on finishing. The header file `<sal.h>` defines the annotations.

When using the Microsoft compiler with the `\analyze` flag, these annotations are used to provide information to the developer about possible defects, such as buffer overruns and `NULL` pointer dereferences. To disable the use of annotations on Windows, define the pre-processor variable:

```
NO_STAR_ANNOTATIONS
```

Note that these annotations are defined as nothing for non-Windows platforms.

For more information, please see the documentation available on the MSDN Library:

- [Code Analysis for C/C++ Overview](#)
- [SAL Annotations](#)

4.8 Change Log

4.8.1 STAR-System v4.01 - 29th November 2019

- Fixed bug in STAR-System Test and Time-code example programs due to incorrect error checking.
- Added Makefile and Visual Studio projects for device configuration examples.
- Linux kernel compatibility tested up to v5.4.
- Added port information to link state and speed change event structs, and also API functions [STAR_getLink↔StateEventPort\(\)](#) and [STAR_getLinkSpeedEventPort\(\)](#) to access it.

4.8.2 STAR-System v4.00 - 19th November 2019

- Added [Generic Configuration API](#).
- Added the following functions to support SpaceFibre broadcast message stream items:
 - [STAR_getBroadcastMessageChannel\(\)](#)
 - [STAR_getBroadcastMessageDataWord1\(\)](#)
 - [STAR_getBroadcastMessageDataWord2\(\)](#)
 - [STAR_getBroadcastMessageDelayedFlag\(\)](#)
 - [STAR_getBroadcastMessageLateFlag\(\)](#)
 - [STAR_getBroadcastMessageStatus\(\)](#)
 - [STAR_getBroadcastMessageType\(\)](#)
- Added time-code support for PXI cards.
- Added the following functions to support an alternative receive method using pre-allocated buffers:
 - [STAR_openChannelToLocalDeviceEx\(\)](#)
 - [STAR_createRxOperationEx\(\)](#)
 - [STAR_rxOperationExGetNumItems\(\)](#)
 - [STAR_rxOperationExGetItemType\(\)](#)
 - [STAR_rxOperationExGetDataChunk\(\)](#)
 - [STAR_rxOperationExGetBroadcastMessage\(\)](#)
 - [STAR_rxOperationExGetStatus\(\)](#)
- Improvements to Linux console installation script and GUI installer.
- Improved component scaling on high DPI and normal DPI displays.
- Fixed problem in [STAR_openChannelBetweenLocalDevices\(\)](#).
- Fixed bug where Sink application didn't always terminate when closed.
- Fixed bug that prevented Transmit application from transmitting over channel 0.
- Fixed issue that could cause increased CPU usage in the Sink application.
- Fixed bug that caused an error in the Performance Tester when log-like pattern was specified.
- Fixed bug in Time-codes GUI application where master frequency was not loaded correctly for the Brick Mk3.
- Documented installation steps for Linux installations with secure boot enabled.
- Documented PXI timestamping features.
- Linux kernel compatibility tested up to v5.3.9.
- Various performance and documentation improvements.

- Removed several functions from header files that were previously deprecated:

- `CFG_BRICK_MK2_getMeasuredLinkSpeed()`
replaced by `CFG_getMeasuredLinkSpeed()`
- `CFG_PCIMK2_getHardwareInfo()`
replaced by `CFG_getFPGAInfo()`
- `CFG_PCIMK2_hardwareInfoToString()`
replaced by `CFG_FPGAInfoToString()`
- `CFG_PCIMK2_enableTimeCodeMaster()`
replaced by `CFG_enableTimeCodeMaster()`
- `CFG_PCIMK2_disableTimeCodeMaster()`
replaced by `CFG_disableTimeCodeMaster()`
- `CFG_PCIMK2_setTimeCodePeriod()`
replaced by `CFG_setTimeCodePeriod()`
- `CFG_PCIMK2_getTimeCodePeriod()`
replaced by `CFG_getTimeCodePeriod()`
- `CFG_PCIMK2_identify()`
replaced by `CFG_identify()`
- `CFG_PCIMK2_enableInterfaceMode()`
replaced by `CFG_enableInterfaceMode()`
- `CFG_PCIMK2_enableInterfaceModeOnPort()`
replaced by `CFG_enableInterfaceModeOnPort()`
- `CFG_PCIMK2_disableInterfaceMode()`
replaced by `CFG_disableInterfaceMode()`
- `CFG_PCIMK2_disableInterfaceModeOnPort()`
replaced by `CFG_disableInterfaceModeOnPort()`
- `CFG_PCIMK2_enableIdentifySource()`
replaced by `CFG_enableIdentifySource()`
- `CFG_PCIMK2_disableIdentifySource()`
replaced by `CFG_disableIdentifySource()`
- `CFG_PCIMK2_enableIdentifySourceOnPort()`
replaced by `CFG_enableIdentifySourceOnPort()`
- `CFG_PCIMK2_disableIdentifySourceOnPort()`
replaced by `CFG_disableIdentifySourceOnPort()`
- `CFG_PCIMK2_setPortRoutingAddress()`
replaced by `CFG_setPortRoutingAddress()`
- `CFG_PCIMK2_getPortRoutingAddress()`
replaced by `CFG_getPortRoutingAddress()`
- `CFG_PCIMK2_setLinkRateDivider()`
replaced by `CFG_setTransmitSignallingRate()`
- `CFG_PCIMK2_getLinkRateDivider()`
replaced by `CFG_getTransmitSignallingRate()`
- `STAR_CFG_getRemoteDeviceDescriptionName()`
replaced by `STAR_CFG_getRemoteDeviceDescriptionNameString()`

4.8.3 STAR-System v4.00 Beta 4 - 16th October 2019

- Brand new Linux installer allowing feature selection including the ability to install Qt5 versions of the GUI applications, required for new webcam features.
- Changes to Device Update application to improve usability.

- Added support for colour version of SpaceFibre Camera in Sink.
- Added port action to disable LVDS ([PORT_ACTION_DISABLE_LVDS](#)) to Triggering API.
- Added support for PXI Mk2 cards.
- Added timestamp support for [SpaceWire PXI Interface](#) and [PXI Interface Mk2](#) cards.
- Fixed potential bug during initialisation of Link Analyser Mk3, STAR Fire Mk3, EGSE Mk2 and Conformance Tester Mk2.
- Fixed possible hang when transmitting high resolution webcam data in Source application.
- Fixed bug in "Select Packet Format" dialog in Sink application where leaving "Check the content of received packets for errors" unchecked would result in an error.
- Fixed bug in [CFG_BRICK_MK3_setTimestampValue\(\)](#) where incorrect timestamp value was being set.
- Fixed bug in Time-codes GUI application where incorrect time-code period was being reported.
- Fixed bug where Sink application could crash when recording to file.
- Fixed bug in Device Configuration application where multiplier and divisor "Set" button was not being enabled in all circumstances.
- Fixed bug where Sink application would crash when selecting webcam field from "Frame Settings" dialog for second time.
- Fixed bug in Source application where webcam fields could not be deleted if the webcam is no longer attached.
- Fixed bug that could cause the Device Configuration application to get into a bad state when refreshing a remote device.
- Fixed bug in Source and Sink applications where it was possible to create a packet format with same name.
- Fixed bug in time-code handling for [SpaceWire PXI Interface](#) and [PXI Interface Mk2](#) and [SpaceWire PXI 12 Port Router](#) and [PXI 12 Port Router Mk2](#) cards.
- Various improvements to the documentation and internals.
- Linux kernel compatibility tested up to v5.0.0.

4.8.4 STAR-System v4.00 Beta 3 - 24th October 2018

- Added support for Conformance Tester Mk2 and EGSE Mk2 devices.
- Resolved issue where base selection dropdowns in the Device Configuration application would disappear on Linux.
- Devices are now listed under "STAR-Dundee Devices" in Device Manager on Windows.
- Linux kernel compatibility tested up to v4.19.

4.8.5 STAR-System v4.00 Beta 2 - 16th October 2018

- Fixed an issue in the Windows PCI driver that caused devices to get into a bad state after being reset.
- Fixed an issue in the Linux PCI and USB drivers that allowed invalid channels to be opened.
- Fixed an issue in the Source application that allowed an empty schedule to be transmitted.
- Added an icon to the Error Injection application.
- Various improvements to the documentation.

4.8.6 STAR-System v4.00 Beta 1 - 25th September 2018

- Added ability to transmit and receive webcam image data.
- Added device serial number to default device name so that it is easier to differentiate between multiple devices of the same type.
- Improved [SpaceWire PCI Mk2](#) and [cPCI Mk2](#) device configuration example.
- Improvements to look and feel of GUI applications including maximise and minimise buttons.
- Custom device names are now stored in the device so that they persist when moved between computers.
- Added function [STAR_resetDeviceName\(\)](#) to allow a custom device name to be reset to a device's default name.
- Removed EEP statistics from the Source application as this only applies to the Sink.
- Sink application statistics now only show errors that are being checked.
- Added [STAR_getSpaceWireAddressPath\(\)](#) and [STAR_getSpaceWireAddressPathLength\(\)](#) functions.
- Fixed bug where Receive application could crash if an empty packet was received.
- Fixed bug that allowed time-code events to be enabled on incompatible versions of the [SpaceWire PCIe](#) card.
- Fixed bug where Sink showed packet too short errors when sequence error was encountered.
- Fixed possible crash when receiving link events on wrong channel.
- Fixed bug in Device Configuration Service that could result in the device not being recognised.
- Fixed bug that caused large packets to be split when received by the STAR Fire Mk3.
- Fixed bug that meant that time-codes could not be received in isolation on a channel.
- Fixed issue in `api_test_app` time-code example where the wrong channel was being used.
- Fixed bug in Device Configuration application where transmit frequency was not updated for [SpaceWire PXI 12 Port Router](#) and [PXI 12 Port Router Mk2](#).
- Fixed bug where channel remained open if Linux application was terminated abruptly using Ctrl+C.
- Fixed issue with 32-bit/64-bit compatibility on recent Linux kernels.
- Various internal performance improvements.
- Various improvements to the documentation.
- Added functions [CFG_MK2_getTimeCodeDistributionPorts\(\)](#) and [CFG_MK2_setTimeCodeDistributionPorts\(\)](#). They are used to obtain and specify which output ports time-codes are forwarded on. They are used with Brick Mk2, Brick Mk3, Router Mk2S and PXI devices.
- Added functions [CFG_MK2_getTimeCodeFlagMode\(\)](#) and [CFG_MK2_setTimeCodeFlagMode\(\)](#). They are used to get and set the time-code flag interpretation mode. They are used with Brick Mk2, Brick Mk3, Router Mk2S and PXI devices.

4.8.7 STAR-System v3.10 - 23rd February 2018

- Fixed memory leak in timestamp feature.
- Added batch file `star_performance_tester.bat` to call `star_performance_tester.exe` so test results can be viewed on screen.
- Added function [STAR_getChannelDirection\(\)](#) to get the transfer direction in which a channel has been opened.
- Added function [STAR_getNextTransferItem\(\)](#) to get the next item in a list of received stream items.

4.8.8 STAR-System v3.9 - 6th December 2017

- Linux kernel compatibility tested up to V4.14.3.
- Fixed issue that meant that Windows drivers were not recognised in a fresh install of Windows 7 or earlier.
- Fixed bug that prevented hexadecimal entry in some fields of the Device Configuration application.

4.8.9 STAR-System v3.8 - 9th November 2017

- Fixed issue that meant that Windows drivers were not recognised in some new installs of Windows 10.

4.8.10 STAR-System v3.7 - 13th October 2017

- Added ability to save and load routing table data in Device Configuration application.
- Added example programs for [Triggering API](#) and [RMAP Target API](#).
- Added [PORT_ACTION_STOP_RECEPTION](#) to [Triggering API](#) (supported by [SpaceWire Brick Mk3](#) hardware version 1.03) which causes a port to stop receiving whilst it is set.
- Linux kernel compatibility tested up to V4.13.4.
- Added new features to the C++ API including Triggering and RMAP Target support. See the [C++ API documentation](#) for more information.
- Fixed bug in Linux PCI driver where data was occasionally lost for packets less than 20 bytes.
- Fixed bug that could cause application to crash during infinite receive operations.
- Fixed hang in Performance Tester when very small packets were defined.
- Fixed crash in Device Configuration application when accessing pre-STAR-System devices as remote devices.
- Fixed bug where it was possible for time-code flags to be incorrectly set.
- Fixed bug in Windows PCI driver which occasionally caused duplicate data to be transmitted.
- Fixed bug that could allow the same channel to be opened more than once on Linux.
- Random data fields in packet formats are no longer checked by the Sink application.
- Added [Triggering API](#) documentation for [SpaceWire PXI 12 Port Router](#) and [PXI 12 Port Router Mk2](#).
- Improved error handling for various API functions.
- Improvements to example project files and makefiles.
- Resolved various warnings in example code.

4.8.11 STAR-System v3.6 - 3rd April 2017

- Added support for timestamping received packets for the [SpaceWire Brick Mk3](#) device.
- Added support for injecting disconnect errors for the [SpaceWire Brick Mk2](#), [SpaceWire Brick Mk3](#), [SpaceWire PXI Interface](#) and [PXI Interface Mk2](#) and [SpaceWire PXI 12 Port Router](#) and [PXI 12 Port Router Mk2](#) devices.
- Added the [CFG_BRICK_MK3_setTimestampMethod\(\)](#), [CFG_BRICK_MK3_getTimestampMethod\(\)](#), [CFG_BRICK_MK3_enableRxTimestampEventsOnPort\(\)](#), [CFG_BRICK_MK3_getRxTimestampEventsOnPortEnabled\(\)](#), [CFG_BRICK_MK3_disableRxTimestampEventsOnPort\(\)](#), [CFG_BRICK_MK3_setTimestampValue\(\)](#), [CFG_BRICK_MK3_getTimestampValue\(\)](#), [CFG_BRICK_MK3_setPulseGeneratorFrequency\(\)](#) and [CFG_BRICK_MK3_getPulseGeneratorFrequency\(\)](#) functions to the [SpaceWire Brick Mk3](#) configuration API to support timestamping of received packets.

- Added the `STAR_getTimestampEventStartValue()`, `STAR_getTimestampEventEndValue()`, `STAR_getTimestampEventDirection()`, `STAR_getTimestampEventType()` and `STAR_getTimestampEventRawCounters()` functions to support timestamping of received packets.
- Added a description of the SpaceWire interface tristate behaviour for the ports on the [SpaceWire PXI Interface and PXI Interface Mk2](#) and [SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2](#) devices to their hardware manuals.
- Added example program to demonstrate the use of the timestamping feature for the [SpaceWire Brick Mk3](#).

4.8.12 STAR-System v3.5 - 17th March 2017

- Added support for the [SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2](#) device.
- Added support for the [SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2](#) device to the Triggering API.
- Fixed missing definitions in the Triggering API for Linux.
- Fixed bug that caused infinite receive operations to crash.
- Added additional documentation for the Triggering API.
- Added example program to demonstrate the use of the Triggering API.
- Added example program to demonstrate the use of the RMAP Target API.
- Fixed compiler warnings in the C++ API for old versions of GCC.
- Fixed compiler warnings in the C and C++ examples.
- Linux kernel compatibility tested up to V4.10.3.

4.8.13 STAR-System v3.4 - 9th March 2017

- Added millisecond resolution to timestamps in the STAR-System logs for Windows and Linux.
- Added support to the Device Configuration GUI application for the [STAR Fire Mk3](#) device.
- Added the `RMAP_TARGET_PXI_IF_registerNotificationListenerWithContext()` and `RMAP_TARGET_PXI_IF_unregisterNotificationListenerWithContext()` functions to the RMAP Target API to support user-supplied context variables when receiving RMAP target notification callbacks.
- Updated C++ API with new functionality. Information on the changes can be found in the change log section of the C++ API documentation.
- Fixed bug in the STAR-System Test application where the `STAR_INFINITE` value was incorrectly defined.
- Fixed bug in the Linux USB driver where if the size of a packet being transmitted was a multiple of the USB frame size, the transmission would hang.
- Fixed memory leak in the Link Events example program.
- Fixed memory leaks related to STAR-System callbacks.
- Improved resource clean-up during the shutdown procedure of STAR-System.
- Added external trigger voltage output information to the [SpaceWire Brick Mk3](#) and [SpaceWire PXI Interface and PXI Interface Mk2](#) user manuals in the documentation.
- Various improvements to the documentation.

4.8.14 STAR-System v3.3 - 20th December 2016

- Added support for injecting disconnect errors when using [SpaceWire PXI Interface](#) and [PXI Interface Mk2](#) devices in the Error Injection GUI application.
- Added an API Test App example project which demonstrates how to implement many common applications of STAR-System.
- Added [SpaceWire PXI Interface](#) and [PXI Interface Mk2](#) configuration functions to replace the [SpaceWire Brick Mk2](#) functions that are currently used to control interface mode on port, identify source, port routing addresses, state events and speed events.
- Fixed issue in the drivers to support Linux kernel 4.8's hardened user copy function.
- Fixed HTML bug when viewing the [SpaceWire Brick Mk2](#) and [SpaceWire Router Mk2S](#) hardware manuals.
- Fixed bug in the GUI applications when distinguishing between different device hardware versions.
- Various improvements to the documentation.

4.8.15 STAR-System v3.2 - 24th November 2016

- Added an example application to demonstrate the use of the link events API.
- Added support for STAR Fire Mk3 devices.
- Added support for link events to the [SpaceWire PXI Interface](#) and [PXI Interface Mk2](#) devices.
- Fixed bug in Sink GUI application which caused some recorded packet files to be lost.
- Fixed bug in Source and Sink GUI application where the statistics windows had incorrect font sizes on high DPI monitors.
- Fixed bug in the calculation of [SpaceWire PCI Mk2](#) and [cPCI Mk2](#) register addresses which caused some functions to stop working.
- Improved readability of the device and file information fields in the Device Update GUI application.

4.8.16 STAR-System v3.1 - 21st October 2016

- Added new RMAP Target GUI to configure the RMAP targets within the [SpaceWire PXI Interface with RMAP Target](#) devices.
- Resizing the statistics views in the Source and Sink GUI applications now scales the font size accordingly.
- Added support in the [Triggering API](#) for controlling the triggering capabilities of [SpaceWire PCIe](#) devices. The `CFG_MK2_startPeriodicAction()` and `CFG_MK2_stopPeriodicAction()` functions have been deprecated for the [SpaceWire PCIe](#) devices because their functionality has been replaced by the [Triggering API](#).
- Added the `CFG_MK2_enableTimeCodeCounterBypassMode()`, `CFG_MK2_getTimeCodeCounterBypassModeEnabled()` and `CFG_MK2_disableTimeCodeCounterBypassMode()` functions to configure the time-code counter bypass mode within supported devices. When enabled, this mode allows the device to forward any received time-codes, including their flag values, without checking them for validity. When disabled, only valid time-codes are forwarded. A checkbox for enabling/disabling this feature will now appear in the Device Configuration GUI if supported by the device. Note that this feature is currently only supported by [SpaceWire PCIe](#) devices with hardware version 1.12 or above.
- Fixed bug in Source GUI application where the application would freeze on Linux after clicking the stop transmit button.
- Fixed bug when getting incorrect stream items after receiving multiple packets in a transfer operation.
- Linux kernel compatibility tested up to V4.8.2.
- Application notes are now installed in the `application_notes` directory.

4.8.17 STAR-System v3.0 - 26th August 2016

- A PDF version of this documentation is now available for download from the STAR-Dundee website in the support area for registered users, while the latest version of the HTML documentation is also available at <https://www.star-dundee.com/system/files/help/star-system/index.xhtml>.
- Added `CFG_MK2_enableTimeCodeEventsOnPort()`, `CFG_MK2_getTimeCodeEventsOnPortEnabled()` and `CFG_MK2_disableTimeCodeEventsOnPort()` to configure time-code events on individual ports of a PCIe device.
- Fixed bug in Sink application when attempting to record traffic to a directory which is not writeable.
- Added `CFG_MK2_enableStateChangeEventsOnPort()`, `CFG_MK2_getStateChangeEventsOnPortEnabled()` and `CFG_MK2_disableStateChangeEventsOnPort()` to configure state change events on individual ports of PCIe and SPLT devices.
- Added `CFG_MK2_enableSpeedChangeEventsOnPort()`, `CFG_MK2_getSpeedChangeEventsOnPortEnabled()` and `CFG_MK2_disableSpeedChangeEventsOnPort()` to configure speed change events on individual ports of PCIe and SPLT devices.
- Updated `CFG_MK2_getMeasuredLinkSpeed()` to work with all devices which support this feature, and removed `CFG_PXI_getMeasuredLinkSpeed()`.
- Added `STAR_destroyTransferItemList()` to destroy stream items returned by a call to `STAR_getTransferItemList()`. If this function is not called for the returned array of stream items, the memory will be leaked. This is a change of functionality from previous releases, and all code using `STAR_getTransferItemList()` must be updated. This change was necessary to deal with a bug in `STAR_getTransferItemList()` in previous releases which meant that the stream items pointed to by the returned array could disappear as more packets were received.
- Stream items created by receive operations can now be destroyed by calling `STAR_destroyStreamItem()` or `STAR_destroyTransferItemList()`. This change was necessary to ensure infinite (or very large) receive operations did not consume all the memory on the PC.
- Added the RMAP Target API for controlling the RMAP target in the SpaceWire PXI Interface with RMAP Target.
- Added the Triggering API for controlling the triggering capabilities of SpaceWire PXI Interface and Brick Mk3 devices.

4.8.18 STAR-System v3.0 Beta 9 - 26th July 2016

- Fixed bug in Device Configuration application which meant that when the Reset button was pressed, the window was not refreshed to show the new values. This bug was introduced in STAR-System v3.0 Beta 5.
- Added `STAR_getDeviceConfigCapabilities()` function and associated definitions of `STAR_DEVICE_CONFIG_SUPPORTED` and `STAR_DEVICE_CONFIG_NOT_SUPPORTED`. These values are used by `STAR_getDeviceConfigCapabilities()` to indicate if a device can be configured and can also be passed to `STAR_getDeviceListForType()` to get all devices which can or cannot be configured.
- Fixed bug in Device Configuration application which meant that the link speed divider and multiplier / divisor input boxes incorrectly allowed spaces.
- Fixed bug in Sink application where application could crash if starting recording to a file that had already reached the size limit. Now the file is correctly rotated.
- Fixed issue in Windows and Linux USB Driver that could cause a USB device to not transmit a packet until a subsequent packet was queued up to be transmitted.
- Added fix to potential permissions issue when running multiple applications under Linux with different user accounts.
- Improvements to the documentation.
- Linux kernel compatibility tested up to V4.7.

4.8.19 STAR-System v3.0 Beta 8 - 6th May 2016

- Fixed bug which caused `STAR_closeChannel()` to complete before the channel was closed. This could cause subsequent calls to open a channel to fail as the channel was still open.
- Fixed bug in Sink application where some of the statistics values were misaligned.
- Fixed bug in Windows USB Driver which could cause a blue screen if a channel was closed while a transmit or receive operation was in progress.
- Resolved code signing issue that could present SmartScreen warning during installation in newer versions of Windows.
- Improved documentation for Brick Mk3.
- Improved error reporting in Device Update Software.
- Added `CFG_MK2_getMeasuredLinkSpeed()` and `CFG_PXI_getMeasuredLinkSpeed()` functions to retrieve current link speed on a port.
- Linux kernel compatibility tested up to V4.5.4.

4.8.20 STAR-System v3.0 Beta 7 - 8th March 2016

- Added support for devices with improved SpaceWire clock duty cycle.
- Improvements to C++ API examples.
- Fixed bug to ensure 32- and 64-bit libraries are in the correct locations in Linux systems.
- Added composite multiplier and divider for clock frequency control in Device Configuration application.

4.8.21 STAR-System v3.0 Beta 6 - 21st January 2016

- Fixed an issue with IOCTL handling when running 32-bit apps on a 64-bit Linux kernel.
- Linux kernel compatibility tested up to V4.3.
- Added support for SpaceFibre Router PXI devices.

4.8.22 STAR-System v3.0 Beta 5 - 25th November 2015

- Fixed an issue with the Device Configuration API function `CFG_ROUTER_getConfigPortErrors()` where a late EEP was flagged when an early EEP was detected.
- Fixed an issue with the PCIe card on Linux.

4.8.23 STAR-System v3.0 Beta 4 - 13th November 2015

- Added support for SpaceWire PXI cards, with associated Configuration API.
- Tested on Windows 10, 32- and 64-bit.
- Tested on Linux 4.0.0 kernel, 32- and 64-bit.
- Reverted Glibc to version 1.12 for compatibility with older Linux systems.
- Reverted Qt to version 4.6.2 for compatibility with older Linux systems.

4.8.24 STAR-System v3.0 Beta 3 - 4th September 2015

- Fixed LabVIEW bug where an application can hang on restart when running in a loop.
- Added new Periodic Action functions to allow repetitive actions to be scheduled on the PCI-Mk2, cPCI-Mk2 and PCIe devices.
- Added function `STAR_getDeviceListForTypes()` to allow passing in an array of device types and to return an array of device IDs for all connected devices of the requested types.

4.8.25 STAR-System v3.0 Beta 2 - 24th July 2015

- Added new base box feature to GUI applications, which means the numerical base that values are entered in can be specified.
- Updated the Statistics dialog in the Sink application to have green error boxes when there are no errors, and to change red when there are errors. This makes errors much easier to spot.
- Completed support for SpaceWire Brick Mk3 devices in the Device Update software.
- Added new device type constant `STAR_DEVICE_ALL`, which can be passed to `STAR_getDeviceListForType()` to get all devices.
Note that as a result of this addition, the value of `STAR_DEVICE_TXRX_SUPPORTED` has changed from the value used in version 3.0 Beta 1.
- The CUBA Software, Source, Sink, Transmit, Receive, Error Injection and Time-codes applications have all been updated to only use devices which can transmit and receive packets.
- The Performance Tester and STAR-System Test have been updated to demonstrate the use of `STAR_getDeviceListForType()` and to improve the code.
- Renamed `STAR_canDeviceTransmitAndReceive()` function to `STAR_getDeviceTxRxCapabilities()`.
- Fixed bug in STAR-System core which could theoretically cause a problem during initialisation of the API.
- Completed CUBA Software documentation.
- Improved instructions on Linux installation and required modules.
- Fixed bug in STAR-System core which meant that a handle to a device was always kept open even after that device was removed. This could cause an issue on Linux when adding and removing devices.
- Fixed bug in STAR-System core which meant an application may hang when being closed on Linux if the Configuration Service was not running.

4.8.26 STAR-System v3.0 Beta 1 - 31st March 2015

- Added support for SpaceWire Brick Mk3 devices.
- Added support for STAR Fire devices with STAR-System support.
- Added support for Conformance Tester devices with STAR-System support.
- Added new Time-code GUI application to transmit and receive time-codes, and to configure the properties of devices related to time-codes.
- Added help to all GUI applications.
- Updated GUI applications to support command-line options:
 - `-0` to enable the use of channel 0 in the application.
 - `-s` to specify the serial number of the device to be used by the application.

- `-c` to specify the channel on the device to be used by the application.

For more information, please see the [Graphical Applications](#) page.

- Added Error Injection GUI application to QNX installer.
- Minor improvements to the Device Configuration application when entering new values and when unable to read router properties.
- Added support for the HPPDSP, WBS and RTC devices to the Device Configuration GUI application.
- Updated Device Configuration GUI application to refresh all values on display following a reset.
- Added support for the STAR Fire, Link Analyser Mk2 and EGSE to the Device Update software.
- Updated Device Update application to support programming the EEPROM of Brick Mk3 devices, and a fast update mode for programming the FPGA of a Brick Mk3 (incomplete).
- Numerous minor improvements to the Source and Sink GUI applications when specifying the packet format.
- Replaced Copy and Paste buttons in the Source and Sink GUI application's Packet Format dialog with a Duplicate button.
- Fixed bug in Source and Sink GUI applications which meant a duplicate packet format was created when a packet format was renamed.
- Fixed bug in Source GUI application, which meant it would stop transmitting after it had transmitted 0xffffffff packets.
- Added data rate entry in the Statistics dialogs of the Source and Sink GUI applications.
- Added method to save packet formats to specific directories in Source and Sink GUI applications.
- Updated the Sink GUI application to allow unknown initial values for the sequence number field.
- Added the STAR-System version of the [CUBA Software](#).
- Improved Windows USB Driver, to provide additional buffer space and reduce the effect on the SpaceWire link of applications not receiving packets quickly enough. Unfortunately there is currently a bug in this code, see [Known Issues](#).
- Added `STAR_getDeviceListForType()` function.
- Added `STAR_canDeviceTransmitAndReceive()` function.
- Added definitions for all currently supported device types in `types.h`, along with definitions of `STAR_DEV↔ICE_TXRX_SUPPORTED` and `STAR_DEVICE_TXRX_NOT_SUPPORTED`. These values are used by the above two functions.
- Added `STAR_createErrorInData()` function and added support for transmitting errors in sequence with data characters, when using devices which support this feature.
- Added support for Linux kernels up to 3.19.
- Fixed bug in Linux PCI Driver, which meant that some PCs were unable to map memory required to DMA to the device.
- Fixed bug when setting the name of a remote device using `STAR_CFG_createRemoteDeviceIdentifier()`.
- Fixed bug when receiving time-codes which meant non-zero flag values were incorrectly reported.
- Fixed bug in `CFG_BRICK_MK2_enableIdentifySourceOnPort()`, writing a value to an incorrect register.
- Fixed bug in reset feature of PCI drivers, which meant access to the device immediately after reset may put the device in a bad state.
- Fixed bug when resetting the API, which resulted in issues with some LabVIEW applications using the ST↔AR-System LabVIEW Wrapper.
- Fixed bug in `CFG_MK2_setBaseTransmitClock()`, which could complete before the clock was successfully set, returning an error.
- Updated hardware manuals to address issues with channel numbering.

4.8.27 STAR-System v2.4.1 - 4th August 2014

- Added support for Linux kernel version 3.16.

4.8.28 STAR-System v2.4 - 2nd May 2014

- Release of STAR-System for RTEMS.

4.8.29 STAR-System v2.3 - 31st January 2014

- Fix to issue present in v2.2 release of Device Update software, which meant that device updates may fail with an error.
- Added support for 24-bit sequence numbers to the Source and Sink applications.
- Added support for SpaceWire Recorder device type.
- Support for CASTOR router configuration in Device Configuration application.
- Beta release of STAR-System for RTEMS.

4.8.30 STAR-System v2.2 - 5th November 2013

- Fixed problem with shortcuts to documentation on Linux. Wrong file extension was being used (.html rather than .xhtml).
- Fixed issues displaying some documentation files, including the hardware manuals and the GUI applications page, and resolved issues with tree view on left of documentation.
- Fixed memory leak in Device Configuration Service.
- Fixed bug in Device Configuration Service which could cause the service to get in to a bad state when a device was removed.
- Fixed bug in STAR-System core which could cause a crash if unexpected data was received from a device during device removal or addition.
- Fixed bug in STAR-System core which could cause a crash when submitting a receive operation which had previously been cancelled.
- Fix to compilation error in Linux USB Driver causing installation error when installed on kernels prior to 2.6.23.
- Fixed bug in STAR-System core which could cause a crash when creating a number of remote device identifiers.
- Fixed bug in Linux install script which may result in 64-bit libraries not being installed on 64-bit systems.
- Improvements to the Device Configuration application:
 - Errors on ports are now highlighted by changing the colour of the tab's text to red.
 - Clicking the Refresh button does not change the tab on display. Changing the device being displayed also does not change the tab.
- Improvements to the Performance Tester software:
 - Added support for `-usechannel0` argument, to allow channel 0 to be used to transmit and receive packets.
 - Fixed issue when performing rate and random tests, which could result in a receive test expecting to receive more packets than a transmit test would transmit.

- Fixed bug in Router Configuration Library: Invalid Destination Logical Address error was not being correctly reported.
- Added Cygwin support to Windows release.
- Fixed Time-flags mode was incorrectly reported for Brick Mk2, Router Mk2S and RTC devices.
- Fixed memory corruption leading to application crash when receiving link events.
- Added support for Linux kernels up to 3.11.
- Fix to issue in Windows PCI Driver, which could affect device update operations.

4.8.31 STAR-System v2.1 - 1st August 2013

- Fixed issues in Windows and Linux USB and PCI Drivers when hibernating a PC with a device connected which has a channel open.
- Fixed issues in documentation when using the document tree.
- Removed unnecessary dependency on libQtNetwork in GUI applications on Linux.
- On Windows, removed libraries for the current target (e.g. 32-bit libraries on 32-bit versions of Windows) from the lib install directory. Instead, only the libraries for each specific target are included in the appropriate sub-directory, e.g. lib/x86-32, lib/x86-64. Please use the libraries in these sub-directories when linking your code.
- Updated documentation for [STAR_TIMECODE](#) to indicate that time-codes can be transmitted on channel 0 only.
- Fixed bug in Linux USB Driver which could result in a kernel error when a USB device was removed on Linux.
- Removed ui.c and ui.h example files from API documentation, and moved them to their correct location in the examples.
- Added time-code test application to examples.
- Added FPGA version number information to STAR-System Test version information.
- Reduced size of documentation, and hence the installers. This was achieved by changing the images to be svg files. If you have problems viewing the images on your web browser, a version which uses gif files, and which should work on older browsers, is available for download from our website.

4.8.32 STAR-System v2.0 - 3rd July 2013

- Added C++ API.
- Fixed bug in Device Configuration application, which meant that invalid registers would be read for some devices, causing an error of 10 to be shown in the STAR-System log.
- Fixed bug in [STAR_CFG_destroyRemoteDeviceIdentifier\(\)](#) when passing in an invalid identifier, which would cause an exception.
- Added Error Injection GUI tool.
- Added function to support error injection for Mk2 compatible devices: [CFG_MK2_injectError\(\)](#).
- Fixed incorrect enum values for [STAR_CFG_PCIMK2_LINK_FREQ](#). Link Frequency values for 128MHz and 150MHz were swapped.
- Fixed issue with [CFG_ROUTER_stopLink\(\)](#), which did not reset the disable bit.
- Fixed bug in the Linux USB driver which caused a kernel panic if a device was unplugged while an operation was still in progress.

- Fixed bug that would cause `STAR_openChannelToLocalDevice()` to fail.
- Fixed bug that would cause configuration operations to time out.
- Fixed bug that could crash the device configuration service.
- Fixed memory leak in Device Configuration application, which could cause it to consume memory and eventually crash.

4.8.33 STAR-System v2.0 Beta 4 - 21st May 2013

- Fixed bug in STAR-API when calling `STAR_createRxOperation()` with a stream item count of -1 to receive an infinite number of stream items. This call would always return an error.
- Reduced the time that the Device Configuration Service will keep a channel open following an operation to be a minimum of 1 second and a maximum of 2 seconds. In the previous release it was 10 seconds, which meant the channel could not be used to transmit or receive packets for quite some time.
- Fixed `STAR_CFG_getRemoteDeviceDescriptionNameString()` not being exported correctly.
- Renamed GUI applications and their executable names as follows:
 - Simple Receive (`star_simple_rx`) => Receive (`star_receive`)
 - Simple Transmit (`star_simple_tx`) => Transmit (`star_transmit`)
 - Advanced Receive (`star_advanced_rx`) => Sink (`star_sink`)
 - Advanced Transmit (`star_advanced_tx`) => Source (`star_source`)

The Device Configuration application's filename has also been renamed from `star_config_gui` to `star_device_config`.

- Fixed bug in Sink application, when checking the packet format of a packet which contained a checksum/CRC and an address which was not present at the destination.
- Improved Source application to ensure random fields change between packets.
- Fixed bug in Linux PCI and USB drivers, which could cause a kernel oops when a device was first added (either by being plugged in, or when the driver was first loaded).
- Fixed bug in Linux USB driver, which meant a device reset would not work.
- Fixed bug in Windows USB Driver when transmitting, which could result in some packets not being transmitted.
- Fixed bug in `STAR_getApplicationID()` which meant it would fail if it was the first STAR-API function called in an application.
- Added noise warnings to Brick Mk2, PCIe and Router Mk2S hardware manuals.

4.8.34 STAR-System v2.0 Beta 3 - 12th April 2013

- Fixed bug in Device Configuration Service start-up script on Linux, which meant the service may not start on some Linux distributions. Also updated install and uninstall process to address related issues.
- Improved the end of line format used in header files and example code to use the operating system's native format.
- Addition of VxWorks documentation.
- Fixed synchronisation issue in STAR-System core, which could appear when registering a listener.
- Fixed bug in STAR-System core when receiving time-codes.
- Added the following functions to the Mk2 Configuration API to access external time-code selection settings:

- `CFG_MK2_enableExternalTimeCodeSelection()`
- `CFG_MK2_disableExternalTimeCodeSelection()`
- `CFG_MK2_getExternalTimeCodeSelectionEnabled()`
- Added support for enabling and disabling external time-code selection to the Device Configuration GUI.
- Corrected type of parameter expected by `STAR_CFG_destroyRemoteDeviceDescriptionList()`.
- Updated the following listener functions to return an error if a device or driver is not specified:
 - `STAR_registerDeviceListenerForDriver()`
 - `STAR_registerDeviceListenerForDevice()`
 - `STAR_registerChannelListenerForDriver()`
 - `STAR_registerChannelListenerForDevice()`
- Fixed issues in Device Configuration Service, which could cause it to crash in some circumstances.
- Updated example Visual Studio project files to link to the appropriate libraries for 32-bit and 64-bit builds.
- Improved detection of addition and removal of devices.

4.8.35 STAR-System v2.0 Beta 2 - 19th March 2013

- Improved performance of Windows USB Driver.
- Improved memory usage of Windows USB Driver, to address occasional problem when adding and removing a USB device repeatedly.
- Fixed bug in Linux USB Driver which could potentially result in packets being corrupted or received out of order.
- Added option to configure remote devices in the Device Configuration GUI.
- Updated `STAR_CFG_createRemoteDeviceIdentifier()` to allow NULL addresses for the path and/or return path arguments.
- Improved `cfg_api_remote.h` to remove errors when including from C++ code.
- Fixed bug in Linux USB and PCI drivers when accessing invalid channel numbers.
- Disabled Set buttons in Device Configuration GUI until the corresponding value has been changed.
- Updated the Linux installer to include the option to perform a command-line install.

4.8.36 STAR-System v2.0 Beta 1 - 8th February 2013

- First VxWorks release.
- Added support for USB devices, including the Brick Mk2, the Router Mk2S and the SpaceWire Physical Layer Tester.
- Added support for new link state event (`STAR_LINK_STATE_EVENT`) and link speed event (`STAR_LINK↔_SPEED_EVENT`) stream types, which can be received from USB devices (but not transmitted).
- Added functions to access link state and link speed events:
 - `STAR_getLinkStateEventCount()`
 - `STAR_isLinkStateEventDisconnectError()`
 - `STAR_isLinkStateEventEscapeError()`
 - `STAR_isLinkStateEventLinkRunning()`

- [STAR_isLinkStateEventParityError\(\)](#)
 - [STAR_isLinkStateEventReceiveCreditError\(\)](#)
 - [STAR_isLinkStateEventTransmitCreditError\(\)](#)
 - [STAR_getLinkSpeedEventSpeed\(\)](#)
- Fixed minor bugs in random and latency tests of STAR-System Performance Tester.
- Merged the STAR-Core and STAR-API modules in to a single module.
- Improved the documentation for remote devices.
- Fixed [STAR_CFG_getRemoteDeviceDescriptionList\(\)](#) returning incorrect type.
- Added [STAR_CFG_destroyRemoteDeviceDescriptionList\(\)](#) function.
- Fixed bug in Advanced Rx and Tx GUI applications, which resulted in packets containing incorrect bytes when specifying an address field containing more than one byte.
- Added Brick Mk2 and Router Mk2S Configuration APIs.
- Fixed bug in Windows Installer which meant Mk2 Configuration API was not included in the lib directory.
- Fixed access violation error when [STAR_registerTransferCompletionListener\(\)](#) was called before any other STAR-API function.
- Fixed bug in [STAR_unregisterTransferCompletionListener\(\)](#) which unregistered all transfer completion listeners, instead of just the one specified.
- Fixed Windows installer not including file [cfg_api_mk2_types.h](#).
- Fixed bug in [CFG_ROUTER_getNetworkDiscoveryInfo\(\)](#) which resulted in the number of ports being returned in portCount being one less than the actual number of ports.
- Updated Simple Tx GUI application to allow the end of packet marker type of packets sent to be specified.
- Fixed bug in [CFG_ROUTER_clearPortErrors\(\)](#) which meant that errors were not being correctly cleared.
- Fixed bug in Device Configuration GUI which meant that the error status was not updated when the error was cleared.
- Added support for the notify for current option in STAR-API's register listener functions.
- Updated STAR-System Test to accept a path address when transmitting packets. This is required when using USB devices, or any devices in routing mode.
- Added [Configuration API Functions Supported by Device Types](#) page, listing the features and functions supported by each device type.
- Added Device Update GUI application.
- Fixed bug in Linux PCI Driver when receiving packets, which could result in a packet received by the device not being passed up to the API.

4.8.37 STAR-System v1.12 - 14th November 2012

- Fixed synchronisation bug in STAR-System initialisation, which occurred when the first function call to the API in an application, was made in multiple threads at the same time or in a very short space of time.
- Added [STAR_getPacketAddress\(\)](#) function.
- Fixed [STAR_getPacketData\(\)](#) reporting incorrect buffer length when address present.
- Fixed bug in Windows PCI Driver which could cause the PC to freeze when a device was reset. Also added note to [STAR_resetDevice\(\)](#) explaining the limitations of this function, and added a known issue for this.

- Updated documentation to describe the Device Configuration Service and the multi-threading and multi-processor support in STAR-System.
- Fixed bug which caused issues with functions which made use of the driver type, including `STAR_getDriverType()` and `STAR_registerChannelListenerForDriver()`.
- Added `STAR_getDeviceDriver()` function to get the driver of a device.
- Added `STAR_isDeviceVirtual()` function to determine if a device is a virtual device.
- Added STAR-System Test and Performance Tester code to documentation as examples.
- Fixed bug in Windows PCI Driver when closing an application before closing channels which were open to transmit. Such channels would not be correctly cleaned up.

4.8.38 STAR-System v1.11 - 31st October 2012

- Update to Windows PCI driver to support firmware upgrades.

4.8.39 STAR-System v1.10 - 27th September 2012

- Fixed bug: `STAR_getDriverList()` now works correctly.
- Fixed bug: `CFG_MK2_setBaseTransmitClock()` now works correctly.
- Fixed bug: Callback functions registered with `STAR_registerTransferCompletionListener()` are now called correctly on completion of transmit operations.

4.8.40 STAR-System v1.9 - 7th September 2012

- Added: Advanced Rx GUI application can now record received packets and the fields of those packets to file.
- Fixed bug: Advanced Tx and Rx GUI applications did not check that a packet format had been selected when clicking OK in the Select Packet Format dialog.
- Fixed bug: STAR-System Test double loopback tests may encounter timeout errors when large packets or numbers of packets are used.
- Fixed bug: Advanced Tx application may transmit more packets than were requested when a number is specified.

4.8.41 STAR-System v1.8 - 24th August 2012

- Fixed bug: Linux PCI Driver may hang the system when transmitting and/or receiving on multiple links.
- Fixed bug: Attempts to create and transmit large (>65KB) packets would fail.
- Added: Support for multiple tests in the Performance Tester, so a double or triple loopback test can be performed.
- Added: Support for command line arguments in the Performance Tester.
- Change: The maximum permitted size for a single data chunk is now 0xffff. Attempts to create a data chunk larger than this using `STAR_createDataChunk()` will fail. `STAR_createPacket()` automatically breaks large input data buffers into data chunks of maximum size 0xffff.
- Fixed bug: Using Ctrl+C to quit an application on Windows or Linux which was transmitting and/or receiving could cause the application to hang.
- Improvement: Performance improvements to Windows PCI driver.

- Fixed bug: Copying a packet format in the Advanced Tx GUI application would result in an invalid packet format being created.
- Added: Checking of received packets added to the Advanced Rx GUI Application.
- Fixed bug: The Advanced Tx GUI application contained a bug building packets when a checksum covered a field prior to it in the packet, or covered a field which changed between packets, i.e. a sequence number.
- Fixed bug: The Advanced Tx GUI application contained a bug when adding a new field to a packet format with a checksum field selected. This could cause the checksum to cover the wrong fields.

4.8.42 STAR-System v1.7 - 20th July 2012

- Fixed bug: killing an application (e.g. using Ctrl+C) while it is transmitting or receiving on Windows could cause the application to get in to a bad state. Note that there is still another related bug related to this which can cause the application to hang.
- Fixed bug: restarting the Device Configuration Service after it had been stopped could fail. This can result in the Windows uninstall hanging if the Device Configuration Service had been stopped.
- Fixed bug: stopping the Device Configuration Service on Linux may not complete cleanly. This can result in the Linux install not starting the Device Configuration Service if an uninstall had previously been performed.
- Added: PCI Mk2 Packet Subsystem API.
- Change to End-User Licence Agreement to use new address for STAR-Dundee and to explicitly state the agreement covers "example software".
- Added byte ordering to sequence number and data pattern fields of Advanced Tx GUI.
- Added fixed value data pattern field type to Advanced Tx GUI.
- Added checksum/CRC field type to Advanced Tx GUI.
- Added length field type to Advanced Tx GUI.

4.8.43 STAR-System v1.6 - 29th June 2012

- Fixed bug: detaching from the STAR-System DLLs under Windows by calling FreeLibrary() resulted in deadlock.
- Fixed bug: attempting to receive on an already open channel in the Simple Rx GUI application would result in the software getting in to a bad state.
- Added ability to specify schedule and format of packets to be transmitted in Advanced Tx GUI.
- Updated Device Configuration GUI to only show time-code distribution checkbox for supported ports.
- Fixed bugs in the following functions introduced in v1.5, which resulted in the wrong port being accessed (specified port - 1):
 - CFG_MK2_enableInterfaceModeOnPort()
 - CFG_MK2_getInterfaceModeOnPortEnabled()
 - CFG_MK2_disableInterfaceModeOnPort()
 - CFG_MK2_enableIdentifySourceOnPort()
 - CFG_MK2_getIdentifySourceOnPortEnabled()
 - CFG_MK2_disableIdentifySourceOnPort()
 - CFG_MK2_setPortRoutingAddress()
 - CFG_MK2_getPortRoutingAddress()
- Added support for cPCI Mk2 device type and cPCI bus type to STAR-API.
- Improvements to GUI applications to remove warnings when running from console.

4.8.44 STAR-System v1.5 - 23rd April 2012

- Improved performance of Device Configuration APIs.
- Fixed bug in STAR-System test when receiving to file under Windows which caused a spurious carriage return character to be added after writing a line feed character.
- Fixed synchronisation bug when using Device Configuration APIs which could cause operations to fail, and improved error reporting.
- Fixed bug in Router Configuration API when getting a routing table entry using `CFG_ROUTER_getRoutingTableEntry()`. The port mask was not being set correctly.
- Fixed bug in PCI Mk2 Configuration API when setting the link clock frequency using `CFG_PCIMK2_getLinkClockFrequency()`.
- Fixed bug in Linux PCI Driver, which resulted in the wrong device type being returned by calls to `STAR_getDeviceType()` on Linux.
- Fixed bug which could cause a shared memory area to become inaccessible when an application was closed. This could result in various issues, e.g. attempts to get the name of a device might fail.
- Fixed bug in Mk2 Configuration API when enabling time-code master using `CFG_MK2_enableTimeCodeMaster()`.
- Fixed bug in Mk2 Configuration API which resulted in the wrong port being accessed when getting and setting interface mode properties for a port using the following functions:
 - `CFG_MK2_enableInterfaceModeOnPort()`
 - `CFG_MK2_disableInterfaceModeOnPort()`
 - `CFG_MK2_enableIdentifySourceOnPort()`
 - `CFG_MK2_disableIdentifySourceOnPort()`
 - `CFG_MK2_setPortRoutingAddress()`
 - `CFG_MK2_getPortRoutingAddress()`
- Added new functions to Mk2 Configuration API:
 - `CFG_MK2_getTimeCodeMasterEnabled()`
 - `CFG_MK2_getInterfaceModeEnabled()`
 - `CFG_MK2_getIdentifySourceEnabled()`
 - `CFG_MK2_getInterfaceModeOnPortEnabled()`
 - `CFG_MK2_getIdentifySourceOnPortEnabled()`
- GUI Prototype improvements:
 - Device Configuration GUI:
 - * Added routing table tab to configure the routing table of a device.
 - * Added missing features to existing tabs.
 - Simple Tx and Rx GUIs:
 - * Added option to display/enter packets in ASCII.
 - Added Advanced Tx and Rx GUIs.

4.8.45 STAR-System v1.4 - 16th March 2012

- First full QNX release.
- Updated Router Configuration API to fix bug when getting error status using `CFG_ROUTER_getConfigPortErrors()`, `CFG_ROUTER_getSpaceWireLinkErrors()` and `CFG_ROUTER_getExternalPortErrors()`. The error structure provided was not being cleared.
- Fixed bug in Linux version of `STAR_resetDevice()`. The return value was incorrect. STAR-System Test now displays an error if the device cannot be reset.
- Updated Linux install and uninstall scripts to improve error checking and clean-up.
- Fixed issue when closing applications which have used a Device Configuration API. There was a bug when detaching from the Device Configuration Messenger.
- Fixed issue when running two applications at the same time, which could cause one of the applications not to terminate correctly. This was caused by a bug in the clean-up code.
- Improved output of STAR-System Test option to display device and version information.
- Added prototype GUI applications to installers.
- Added 32-bit DLLs to 64-bit Windows installer, to allow 32-bit applications to run on 64-bit Windows. Note that 32-bit Linux applications already run on 64-bit Linux.
- Fixed bug in RMAP Packet Library's `RMAP_CheckPacketValid()` function when checking a write command packet or read reply packet has a data length of 0.

4.8.46 STAR-System v1.3 - 14th February 2012

- Added Mk2 Interface and Router Device Configuration API files missing from v1.2 install.
- Added identify device option to STAR-System Test.
- Improvements to PCI Mk2 Device Configuration API example.
- Minor improvements to documentation.

4.8.47 STAR-System v1.2 - 13th February 2012

- First full Linux release.
- First release with PCIe support.
- Added Mk2 Interface and Router Device Configuration API.
- Deprecated functions in PCI Mk2 Device Configuration API which are present in Mk2 Interface and Router Device Configuration API.
- Fixed bug which could cause Device Configuration API set functions to timeout and return an error, despite the operation actually being performed successfully.
- Fixed bug in `STAR_closeChannel()` which could result in resources being accessed after they'd been freed, causing a segmentation fault on Linux.
- Fixed issues in Device Configuration Service, which could cause the service to exit unexpectedly, or to hang.
- Fixed occasional problems when starting a new application on Linux after previously starting and stopping multiple applications.
- Fixed memory leaks in system core.
- Updated API header files to address namespace issues.
- Fixed problem with Linux installer which could result in a logout at the end of installation.

4.8.48 STAR-System v1.1 - 20th December 2011

- First Linux (beta) release.
- First stable API release.
- Added remote device features to documentation and added further detail to documentation.
- Corrected issues with remote device function prototypes.
- `len` parameter of `STAR_getChunkData()` changed from `unsigned long *` to `unsigned int *`, to be consistent with other parameters and return types.
- Renamed `STAR_waitOnTransferCompletion()` to `STAR_waitOnTransferOperationCompletion()` to be consistent with other function names.
- Renamed `STAR_getTransferStatus()` to `STAR_getTransferOperationStatus()` to be consistent with other function names.
- Renamed `STAR_cancelTransferWaits()` to `STAR_cancelTransferOperationWaits()` to be consistent with other function names.
- Renamed `STAR_cancelReceiveOp()` to `STAR_cancelTransferOperation()` to be consistent with other function names.
- Renamed `STAR_disposeTransferOp()` to `STAR_disposeTransferOperation()` to be consistent with other function names.
- Renamed `STAR_CONNECTION_ID` to `STAR_CHANNEL_ID` to improve and simplify channel-related names and functions.
- Renamed `STAR_CONNECTION_LISTENER_ID` to `STAR_CHANNEL_LISTENER_ID` to improve and simplify channel-related names and functions.
- Removed `STAR_CONNECTION_ENDPOINT_ID` to improve and simplify channel-related names and functions.
- Renamed `STAR_ConnectionEventFunc` to `STAR_ChannelEventFunc` and updated parameters to improve and simplify channel-related names and functions.
- Renamed `STAR_ENDPOINT_TYPE` to `STAR_CHANNEL_TYPE` and updated values to improve and simplify channel-related names and functions.
- Renamed `STAR_CONNECTION_DIRECTION` to `STAR_CHANNEL_DIRECTION` and updated values to improve and simplify channel-related names and functions.
- Renamed `STAR_registerConnectionListener()` to `STAR_registerChannelListener()` and updated parameters to improve and simplify channel-related names and functions.
- Renamed `STAR_registerConnectionListenerForDriver()` to `STAR_registerChannelListenerForDriver()` and updated parameters to improve and simplify channel-related names and functions.
- Renamed `STAR_registerConnectionListenerForDevice()` to `STAR_registerChannelListenerForDevice()` and updated parameters (including adding `notifyForCurrent`) to improve and simplify channel-related names and functions.
- Renamed `STAR_unregisterConnectionListener()` to `STAR_unregisterChannelListener()` and updated parameter type to improve and simplify channel-related names and functions.
- Renamed `STAR_createConnectionBetween()` to `STAR_openChannelBetweenLocalDevices()` and updated return type to improve and simplify channel-related names and functions.
- Renamed `STAR_createConnectionTo()` to `STAR_openChannelToLocalDevice()` and updated parameters and return type to improve and simplify channel-related names and functions.

- Renamed `STAR_destroyConnection()` to `STAR_closeChannel()` and updated parameter to improve and simplify channel-related names and functions.
- Renamed `STAR_isChannelConnected()` to `STAR_isChannelOpen()` and updated return type to improve and simplify channel-related names and functions.
- Renamed `STAR_getDeviceConnectedToChannel()` to `STAR_getLocalDeviceAttachedToChannel()` to improve and simplify channel-related names and functions.
- Renamed `STAR_getApplicationConnectedToChannel()` to `STAR_getApplicationAttachedToChannel()` to improve and simplify channel-related names and functions.
- Updated `STAR_submitTransferOperation()` to use updated parameter type to improve and simplify channel-related names and functions.
- Updated `STAR_submitTransferOperationList()` to use updated parameter type to improve and simplify channel-related names and functions.
- Renamed `STAR_getPacketEop()` to `STAR_getPacketEOP()` to be consistent with other function names.
- Renamed `STAR_setPacketEop()` to `STAR_setPacketEOP()` to be consistent with other function names.
- Removed the `isUnlimited` parameter from `STAR_createRxOperation()`, with an item count of `-1` now resulting in an unlimited receive.
- Updated test programs and example code to use new function names, parameters and return types.
- Fixed issue with RMAP Packet Library file name on Windows.
- Fixed issue starting and stopping the Device Configuration Service on Windows.
- Added `STAR_transmitPacket()` and `STAR_receivePacket()` to provide simple methods for transmitting and receiving a single packet.
- Fixed bug when attempting to create a transfer operation prior to opening a channel.
- Removed deprecated functions, included prior to STAR-System, from the RMAP Packet Library.
- Replaced `STAR_freeBuffer()` with `STAR_destroyVersionInfo()`, `STAR_destroyVersionInfoList()`, `STAR_destroyDeviceList()`, `STAR_destroyDriverList()` and `STAR_destroyPacketData()`.
- Renamed the following header files so they are lower case and with words separated by underscores:
 - `streamItem.h` -> `stream_item.h`
 - `streamItemTypes.h` -> `stream_item_types.h`
 - `transferTypes.h` -> `transfer_types.h`
 - `cfg-api-remote.h` -> `cfg_api_remote.h`
 - `cfg-api-pci-mk2.h` -> `cfg_api_pci_mk2.h`
 - `cfg-api-pci-mk2_types.h` -> `cfg_api_pci_mk2_types.h`
 - `cfg-api-router.h` -> `cfg_api_router.h`
 - `cfg-api-router-types.h` -> `cfg_api_router_types.h`
- Renamed the following library files so they are lower case and with words separated by underscores:
 - `star-conf_pcimk2` -> `star_conf_api_pci_mk2`
 - `star-conf_router` -> `star_conf_api_router`
- Added `STAR_destroyString()` and updated the parameters and return types of the following functions to return a string:
 - `STAR_getApplicationName()`
 - `STAR_getApplicationNameForID()`
 - `STAR_getDeviceBusTypeAsString()`
 - `STAR_getDeviceTypeAsString()`
 - `STAR_getDeviceName()`
 - `STAR_getDeviceSerialNumber()`

4.8.49 STAR-System v1.0 - 17th October 2011

- First Windows release.
- Improved performance.
- Documentation improvements.
- Support for configuring remote devices using the Device Configuration API added.
- Issue when accessing the Device Configuration Service when it is not running addressed.
- `STAR_getApplicationConnectedToChannel()` function updated to return an application identifier.
- `STAR_getApplicationNameForID()` function added to get an application's name.
- `STAR_getApplicationID()` function added to get the current application's identifier.
- `STAR_getDeviceBusTypeAsString()` function added to get a string representation of a device's bus type.
- Added support for error logging to QNX.
- STAR-System Test Improvements:
 - Displays each device's firmware version.
 - Provides an option to reset a device.
 - Command line arguments accepted to display the properties of the modules and devices.
 - Added additional tests to transmit and receive files.
- Performance Tester updated to record the CPU usage during a test.

4.8.50 STAR-System v0.8 (Beta 1) - 22nd August 2011

First QNX (beta) release.

Chapter 5

Module Index

5.1 STAR-System C APIs

Here is a list of all modules:

STAR-API	199
General API Functions	201
Device and Driver Management	206
Virtual Devices and Drivers	292
Channel Management	235
Stream Items	243
Transfer Operations	275
RMAP Target API	340
Manual Authorisation	342
Configuration	346
Notifications	360
Triggering API	368
Events	380
Actions	403
Configuration	420
PCI Mk2 Packet Subsystem	536
Generic Configuration API	553
User	606
Hardware	610
Port Status and Control	614
Device Identifier	622
Group Adaptive Routing	625
Interface Mode	628
Time-codes	636
Events	643
Link Speed	649
Links	650
Precision Links	652
Error Injection	656
Timestamping	658
Periodic	664
Defines	666
Other Device Configuration APIs	555
Remote Devices	557
Router Configuration API	563
Device Information	570

Port Status and Control	576
Routing Table	590
User Configurable Registers	593
Router Control and Status Configuration	599
Mk2 Compatible Interface and Router Device Configuration API	564
Events	500
Link Speed	506
Measured Link Speed	512
Time-code Master	514
Device Information	521
Interface Mode	524
Error Injection	532
Periodic Actions	534
PCI Mk2 Configuration API	565
Link Speed	294
Brick Mk2 Configuration API	566
Link Speed	297
Interface Mode	300
Error Injection	306
Events	307
Measured Link Speed	312
Brick Mk3 Configuration API	567
Link Speed	313
Timestamping	317
Router Mk2S Configuration API	568
Link Speed	496
PXI Configuration API	569
Link Speed	324
Error Injection	329
Interface Mode	330
Events	335
RMAP Packet Library	669
Functions for Building RMAP Packets	681
Functions for Interpreting RMAP Packets	701
STAR-Dundee Types	711

Chapter 6

File Index

6.1 File List

Here is a list of all documented files with brief descriptions:

cfg_api_brick_mk2.h	Functions used to configure STAR-Dundee Brick Mk2 and compatible devices	713
cfg_api_brick_mk2_types.h	Types used with the STAR-Dundee Brick Mk2 Configuration API	714
cfg_api_brick_mk3.h	Functions used to configure STAR-Dundee Brick Mk3 and compatible devices	716
cfg_api_brick_mk3_types.h	Types used with the STAR-Dundee Brick Mk3 Configuration API	717
cfg_api_generic.c	Generic Configuration API used to configure all STAR-Dundee devices	718
cfg_api_generic.h	Types used with the STAR-Dundee Generic Configuration API	726
cfg_api_generic_common.h	Additional types used with the STAR-Dundee Generic Configuration API	735
cfg_api_mk2.h	Functions used to configure STAR-Dundee Mk2 compatible devices	736
cfg_api_mk2_types.h	Types used with the STAR-Dundee Mk2 compatible device configuration API	739
cfg_api_pci_mk2.h	Functions used to configure STAR-Dundee PCI Mk2 devices	740
cfg_api_pci_mk2_types.h	Types used with the STAR-Dundee PCI Mk2 Configuration API	741
cfg_api_pxi.h	Functions used to configure STAR-Dundee PXI devices	742
cfg_api_remote.h	Functions used to create, configure and destroy paths to remote devices	743
cfg_api_router.h	Functions used to configure STAR-Dundee routing devices	744
cfg_api_router_mk2s.h	Functions used to configure STAR-Dundee Router Mk2S devices	746
cfg_api_router_types.h	Types used with the STAR-Dundee Router API	747
common.h	STAR-Dundee API macros and types required by STAR API modules	749
general.h	General functions provided by STAR-API	750
packet_subsystem.h	Definitions of the functions provided by the PCI Mk2 packet subsystem API	753

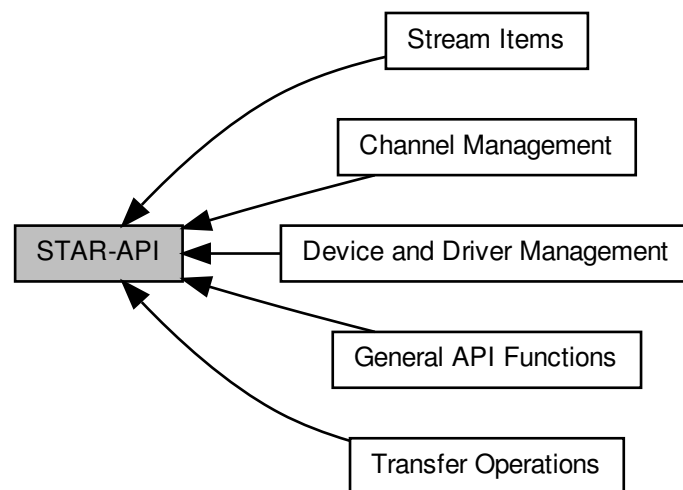
rmap_packet_library.h	Declarations of the functions provided by the STAR-Dundee RMAP Packet Library	755
rmap_target_pxi_if.h	Functions used to configure and control the RMAP target capabilities of the STAR-Dundee PXI Interface devices	760
rmap_target_types.h	Types used with the STAR-Dundee RMAP Target API	762
star-api.h	Declarations of all functions provided by STAR-API	764
star-dundee_types.h	Definitions of STAR-Dundee commonly used types	764
stream_item.h	Functions create and modify items that can be sent over a SpaceWire network	765
stream_item_types.h	Structures used to represent stream items such as packets, time-codes and link control to- kens	768
transfer.h	Declarations of functions provided by STAR-API relating to transfer operations	770
transfer_types.h	Types used to describe transmit and receive operations	772
triggering_brick_mk3.h	Functions used to configure and control the triggering capabilities of the STAR-Dundee Brick Mk3 devices	772
triggering_pcie.h	Functions used to configure and control the triggering capabilities of the STAR-Dundee PCIe Interface devices	776
triggering_pxi_if.h	Functions used to configure and control the triggering capabilities of the STAR-Dundee PXI Interface devices	779
triggering_pxi_ro.h	Functions used to configure and control the triggering capabilities of the STAR-Dundee PXI Router devices	783
triggering_types.h	Types used with the STAR-Dundee Triggering API	786
types.h	Types, structures and function types used by STAR-API	789
version.h	Version information for a module	792

Chapter 7

Module Documentation

7.1 STAR-API

Collaboration diagram for STAR-API:



Modules

- [General API Functions](#)

These are functions that provide information about the API and the STAR-System stack itself.

- [Device and Driver Management](#)

Functions providing information on devices and drivers present.

- [Channel Management](#)

Used to open and close channels, between applications and devices.

- [Stream Items](#)

Stream items are abstract representations of packets, time-codes and link control tokens that can be transmitted and received over a SpaceWire network.

- [Transfer Operations](#)

Transfer operations are used to transmit and receive packets and time-codes.

7.1.1 Detailed Description

7.1.2 Introduction

This is the documentation for STAR-API, the C API of STAR-System, the STAR-Dundee software stack. This API is used to manage STAR-Dundee SpaceWire devices and the channels between them, and to transmit and receive data across those channels.

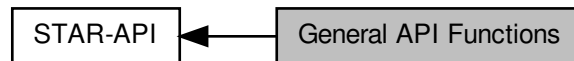
7.1.3 Structure

The API is broken down into a number of logically grouped sections. There are the General functions, which relate to the API itself, such as creating and disposing of devices and channels, and getting device and channel information.

7.2 General API Functions

These are functions that provide information about the API and the STAR-System stack itself.

Collaboration diagram for General API Functions:



Typedefs

- typedef CFG_INFO_ID_TYPE STAR_APP_ID
Unique value used to identify an application using STAR-System.

Functions

- STAR_VERSION_INFO *STAR_API_CC STAR_getApiVersion (void)
Gets the Version of STAR-API.
- void STAR_API_CC STAR_destroyVersionInfo (_Post_ptr_invalid_ STAR_VERSION_INFO *pVersionInfo)
Frees a version information structure previously created by a call to STAR_getApiVersion(), STAR_getDriverVersion() or STAR_getDeviceFirmwareVersion().
- _Ret_opt_cap_ count STAR_VERSION_INFO *STAR_API_CC STAR_getAllVersions (_Out_ U32 *count)
Gets an array of all the version info structures provided by all modules of STAR-System.
- void STAR_API_CC STAR_destroyVersionInfoList (_Post_ptr_invalid_ STAR_VERSION_INFO *pVersionInfoList)
Frees a version information list previously created by a call to STAR_getAllVersions().
- int STAR_API_CC STAR_setApplicationName (_In_z_ char *name)
Sets the name of the application using STAR-API.
- _Check_return_ _Ret_opt_z_ char *STAR_API_CC STAR_getApplicationName (void)
Gets the name of the calling process (this process), set by STAR_setApplicationName().
- STAR_APP_ID STAR_API_CC STAR_getApplicationID (void)
Gets the application ID of the calling process.
- _Check_return_ _Ret_opt_z_ char *STAR_API_CC STAR_getApplicationNameForID (STAR_APP_ID appld)
Gets the name of the application with the given ID.
- void STAR_API_CC STAR_destroyString (_Post_ptr_invalid_ _In_z_ char *string)
Destroy a string returned by STAR-API, such as from STAR_getApplicationName() or STAR_getDeviceTypeAsString().
- STAR_APP_ID STAR_API_CC STAR_getApplicationAttachedToChannel (STAR_DEVICE_ID deviceId, unsigned int channelNumber)
Returns the ID of an application attached to a given channel on a given device.

7.2.1 Detailed Description

These are functions that provide information about the API and the STAR-System stack itself.

7.2.2 Typedef Documentation

7.2.2.1 typedef CFG_INFO_ID_TYPE STAR_APP_ID

Unique value used to identify an application using STAR-System.

Added in version:

STAR-System v1.0 - 17th October 2011

7.2.3 Function Documentation

7.2.3.1 void STAR_API_CC STAR_destroyString (_Post_ptr_invalid_ _In_z_ char * *string*)

Destroy a string returned by STAR-API, such as from [STAR_getApplicationName\(\)](#) or [STAR_getDeviceTypeAsString\(\)](#).

Parameters

<i>in</i>	<i>string</i>	the string to be destroyed
-----------	---------------	----------------------------

Added in version:

STAR-System v1.1 - 20th December 2011

Examples:

[application_name_example.c](#), [device_identifier_example.c](#), [link_events.c](#), [mk2_configuration_example.c](#), [port_control_status_example.c](#), [router_configuration_example.c](#), [routing_table_example.c](#), [star-test.c](#), [star_performance_tester.c](#), [time-code_test.c](#), [triggering_example.c](#), [ui.c](#), [user_registers_example.c](#), and [utilities.c](#).

7.2.3.2 void STAR_API_CC STAR_destroyVersionInfo (_Post_ptr_invalid_ STAR_VERSION_INFO * *pVersionInfo*)

Frees a version information structure previously created by a call to [STAR_getApiVersion\(\)](#), [STAR_getDriverVersion\(\)](#) or [STAR_getDeviceFirmwareVersion\(\)](#).

Parameters

<i>pVersionInfo</i>	the version information structure to be freed
---------------------	---

Added in version:

STAR-System v1.1 - 20th December 2011

Examples:

[api_version_example.c](#), and [star-test.c](#).

7.2.3.3 void STAR_API_CC STAR_destroyVersionInfoList (_Post_ptr_invalid_ STAR_VERSION_INFO * *pVersionInfoList*)

Frees a version information list previously created by a call to [STAR_getAllVersions\(\)](#).

Parameters

<i>pVersionInfoList</i>	the version information list to be freed
-------------------------	--

Added in version:

STAR-System v1.1 - 20th December 2011

Examples:

`all_version_example.c`, and `star-test.c`.

7.2.3.4 `_Ret_opt_cap_count STAR_VERSION_INFO* STAR_API_CC STAR_getAllVersions ( _Out_ U32 * count )`

Gets an array of all the version info structures provided by all modules of STAR-System.

Parameters

<i>out</i>	<i>count</i>	Count of version information structures held in array.
------------	--------------	--

Returns

Array of version info structures containing STAR-System version info structures.

Note

This function returns a snapshot of the current state of the system and is not automatically updated. This array must be freed using `STAR_destroyVersionInfoList()` when no longer required.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

`all_version_example.c`, and `star-test.c`.

7.2.3.5 `STAR_VERSION_INFO* STAR_API_CC STAR_getApiVersion ( void )`

Gets the Version of STAR-API.

Returns

Pointer to a version structure that will contain STAR-API's version info, or NULL if a memory allocation error occurred.

Note

This structure must be freed using `STAR_destroyVersionInfo()` when no longer required.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

`api_version_example.c`.

7.2.3.6 `STAR_APP_ID STAR_API_CC STAR_getApplicationAttachedToChannel ( STAR_DEVICE_ID deviceId, unsigned int channelNumber )`

Returns the ID of an application attached to a given channel on a given device.

Parameters

<i>deviceId</i>	The device to check
<i>channelNumber</i>	The channel number to check

Returns

Requested application ID, or 0 if no application is attached

Added in version:

STAR-System v1.1 - 20th December 2011

7.2.3.7 STAR_APP_ID STAR_API_CC STAR_getApplicationID (void)

Gets the application ID of the calling process.

Returns

The application ID for the calling process

Added in version:

STAR-System v1.0 - 17th October 2011

7.2.3.8 _Check_return_ _Ret_opt_z_ char* STAR_API_CC STAR_getApplicationName (void)

Gets the name of the calling process (this process), set by [STAR_setApplicationName\(\)](#).

The string returned should be freed by calling [STAR_destroyString\(\)](#).

Returns

the application name as a null terminated string

Added in version:

STAR-System v1.0 - 17th October 2011

Declaration changed in version:

STAR-System v1.1 - 20th December 2011

Examples:

[application_name_example.c](#).

7.2.3.9 _Check_return_ _Ret_opt_z_ char* STAR_API_CC STAR_getApplicationNameForID (STAR_APP_ID appld)

Gets the name of the application with the given ID.

The string returned should be freed by calling [STAR_destroyString\(\)](#).

Parameters

<i>appld</i>	the application ID to get a corresponding name for
--------------	--

Returns

the application name as a null terminated string

Added in version:

STAR-System v1.0 - 17th October 2011

Declaration changed in version:

STAR-System v1.1 - 20th December 2011

7.2.3.10 int STAR_API_CC STAR_setApplicationName (_In_z_ char * *name*)

Sets the name of the application using STAR-API.

This is the name provided to other processes calling `STAR_getApplicationNameForID()`.

Parameters

<i>in</i>	<i>name</i>	Null terminated string containing the application name
-----------	-------------	--

Returns

1 if app name was successfully set, else 0

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

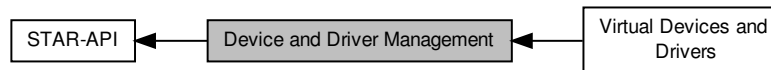
Examples:

`api_test_app.c`, `application_name_example.c`, `link_events.c`, `star-test.c`, `star_performance_tester.c`, `time-code_test.c`, and `timestamp_test.c`.

7.3 Device and Driver Management

Functions providing information on devices and drivers present.

Collaboration diagram for Device and Driver Management:



Modules

- Virtual Devices and Drivers

These are functions that are used to manage the properties of virtual drivers and devices.

Macros

- `#define STAR_DEVICE_TYPE U32`
Type of device that a device is.
- `#define STAR_DEVICE_SERIAL_SIZE 16`
Size in bytes (including null terminator) of a device serial number.
- `#define STAR_CHANNEL_MASK U32`
Type used to represent channels that exist on a device.
- `#define STAR_DEVICE_UNKNOWN 0U`
The "unknown" device type.
- `#define STAR_DEVICE_PCI 1U`
The STAR-Dundee SpaceWire PCI-2 and cPCI device type.
- `#define STAR_DEVICE_ROUTER_USB 2U`
The STAR-Dundee SpaceWire Router-USB device type.
- `#define STAR_DEVICE_USB_BRICK 3U`
The STAR-Dundee SpaceWire-USB Brick device type.
- `#define STAR_DEVICE_LINK_ANALYSER 4U`
The STAR-Dundee SpaceWire Link Analyser device type.
- `#define STAR_DEVICE_CONFORMANCE_TESTER 5U`
The STAR-Dundee SpaceWire Conformance Tester device type.
- `#define STAR_DEVICE_IP_TUNNEL 6U`
The STAR-Dundee SpaceWire IP-Tunnel device type.
- `#define STAR_DEVICE_ROUTER_MK2S 7U`
The STAR-Dundee SpaceWire Router Mk2S device type.
- `#define STAR_DEVICE_PCI_MK2 8U`
The STAR-Dundee SpaceWire PCI Mk2 device type.
- `#define STAR_DEVICE_BRICK_MK2 9U`
The STAR-Dundee SpaceWire Brick Mk2 device type.
- `#define STAR_DEVICE_LINK_ANALYSER_MK2 10U`
The STAR-Dundee SpaceWire Link Analyser Mk2 device type.
- `#define STAR_DEVICE_PCIE 11U`
The STAR-Dundee SpaceWire PCI Express device type.

- `#define STAR_DEVICE_RTC 12U`
The STAR-Dundee SpaceWire RTC device type.
- `#define STAR_DEVICE_EGSE 13U`
The STAR-Dundee SpaceWire EGSE device type.
- `#define STAR_DEVICE_CPCI_MK2 14U`
The STAR-Dundee SpaceWire cPCI Mk2 device type.
- `#define STAR_DEVICE_SPLT 15U`
The STAR-Dundee SpaceWire Physical Layer Tester device type.
- `#define STAR_DEVICE_STAR_FIRE 16U`
The STAR-Dundee STAR Fire device type.
- `#define STAR_DEVICE_WBS_II 17U`
The STAR-Dundee Wide Band Spectrometer II device type.
- `#define STAR_DEVICE_RECORDER 18U`
The STAR-Dundee SpaceWire Recorder device type.
- `#define STAR_DEVICE_BRICK_MK3 19U`
The STAR-Dundee SpaceWire Brick Mk3 device type.
- `#define STAR_DEVICE_TRAFFIC_GENERATOR 20U`
The Shimafuji Traffic Generator device type.
- `#define STAR_DEVICE_PXI_INTERFACE 21U`
The STAR-Dundee SpaceWire PXI Interface device type.
- `#define STAR_DEVICE_PXI_RMAP 22U`
The STAR-Dundee SpaceWire PXI RMAP device type.
- `#define STAR_DEVICE_PXI_ROUTER_8 23U`
The STAR-Dundee SpaceWire PXI 8 port Router device type.
- `#define STAR_DEVICE_PXI_ROUTER_12 24U`
The STAR-Dundee SpaceWire PXI 12 port Router device type.
- `#define STAR_DEVICE_SPFIPCI 25U`
The STAR-Dundee SpaceFibre PCI Express device type.
- `#define STAR_DEVICE_STAR_FIRE_MK3 26U`
The STAR-Dundee STAR Fire Mk3 device type.
- `#define STAR_DEVICE_PCI_MK3 27U`
The STAR-Dundee SpaceWire PCI Mk3 device type.
- `#define STAR_DEVICE_LINK_ANALYSER_MK3 28U`
The STAR-Dundee SpaceWire Link Analyser Mk3 device type.
- `#define STAR_DEVICE_CONFORMANCE_TESTER_MK2 29U`
The STAR-Dundee SpaceWire Conformance Tester Mk2 device type.
- `#define STAR_DEVICE_EGSE_MK2 30U`
The STAR-Dundee SpaceWire EGSE Mk2 device type.
- `#define STAR_DEVICE_GBE_BRICK 31U`
The STAR-Dundee SpaceWire GbE Brick device type.
- `#define STAR_DEVICE_STAR_ULTRA_PCIE 32U`
The STAR-Dundee STAR-Ultra-PCIe device type.
- `#define STAR_DEVICE_PXI_INTERFACE_MK2 33U`
The STAR-Dundee SpaceWire PXI Interface Mk2 device type.
- `#define STAR_DEVICE_PXI_RMAP_MK2 34U`
The STAR-Dundee SpaceWire PXI RMAP Mk2 device type.
- `#define STAR_DEVICE_PXI_ROUTER_MK2 35U`
The STAR-Dundee SpaceWire PXI Router Mk2 device type.
- `#define STAR_DEVICE_CONFIG_SUPPORTED 0x00ffffbU`
A device which is capable of being configured.
- `#define STAR_DEVICE_CONFIG_NOT_SUPPORTED 0x00ffffcU`

- *A device which is not capable of being configured.*
- `#define STAR_DEVICE_TXRX_SUPPORTED 0x00ffffdU`
A device which is capable of transmitting and receiving packets.
- `#define STAR_DEVICE_TXRX_NOT_SUPPORTED 0x00ffffeU`
A device which is not capable of transmitting and receiving packets.
- `#define STAR_DEVICE_ALL 0x00fffffU`
All devices.

Typedefs

- `typedef CFG_INFO_ID_TYPE STAR_DRIVER_ID`
Unique value used to identify an individual driver.
- `typedef CFG_INFO_ID_TYPE STAR_DEVICE_ID`
Unique value used to identify an individual device.
- `typedef CFG_INFO_ID_TYPE STAR_DEVICE_LISTENER_ID`
Unique value used to identify an individual device listener.
- `typedef CFG_INFO_ID_TYPE STAR_DRIVER_LISTENER_ID`
Unique value used to identify an individual driver listener.
- `typedef void(STAR_API_CC * STAR_DriverEventFunc) (STAR_DRIVER_LISTENER_ID driverListener↵ Identifier, STAR_DRIVER_ID driverIdentifier, int driverAdded, void *pContextInfo)`
The function type for driver listener functions which are called when a new driver is added, or an existing one removed.
- `typedef void(STAR_API_CC * STAR_DeviceEventFunc) (STAR_DEVICE_LISTENER_ID deviceListener↵ Identifier, STAR_DRIVER_ID driverIdentifier, STAR_DEVICE_ID deviceIdIdentifier, int deviceAdded, void *p↵ ContextInfo)`
The function type for device listener functions which are called when a device is added or removed.

Enumerations

- `enum STAR_BUS_TYPE {`
 `STAR_BUS_UNKNOWN = 0, STAR_BUS_PCI = 1, STAR_BUS_USB = 2, STAR_BUS_PCIE = 3,`
 `STAR_BUS_TCP = 4, STAR_BUS_CPCI = 5, STAR_BUS_VIRTUAL = 255 }`
The different types of bus that can be used to connect a SpaceWire device to a PC.
- `enum STAR_DRIVER_TYPE {`
 `STAR_DRIVER_INVALID = 0, STAR_DRIVER_TYPE_USB = 1, STAR_DRIVER_TYPE_PCI = 2, STAR_↵`
 `DRIVER_TYPE_TCPIP = 4,`
 `STAR_DRIVER_TYPE_VIRTUAL = 5 }`
Available driver types.

Functions

- `_Ret_opt_cap_ count STAR_DRIVER_ID *STAR_API_CC STAR_getDriverList (int physicalDeviceDrivers,`
 `int virtualDeviceDrivers, _Out_ U32 *count)`
Gets an array of driver identifiers for all drivers of the requested type(s).
- `void STAR_API_CC STAR_destroyDriverList (_Post_ptr_invalid_ STAR_DRIVER_ID *pDriverList)`
Frees a driver list previously created by a call to `STAR_getDriverList()`.
- `_Check_return_ STAR_DRIVER_LISTENER_ID STAR_API_CC STAR_registerDriverListener (STAR_↵`
 `DriverEventFunc listenerFunc, _In_opt_ void *pContextInfo, int notifyForCurrent)`
Registers a Call-back function that will be called whenever a Driver is added or removed.
- `int STAR_API_CC STAR_unregisterDriverListener (STAR_DRIVER_LISTENER_ID listenerId)`
Unregister a Call-back function that was previously registered to be called whenever a driver is added or removed.
- `STAR_VERSION_INFO *STAR_API_CC STAR_getDriverVersion (STAR_DRIVER_ID driverId)`

- Gets the Version of a given driver.*

 - `STAR_DRIVER_TYPE STAR_API_CC STAR_getDriverType (STAR_DRIVER_ID driverId)`

Gets the type (USB/PCI/specific virtual type identifier) of a given driver.
- `_Ret_opt_cap_ count STAR_DEVICE_ID *STAR_API_CC STAR_getDeviceList (_Out_ U32 *count)`

Gets an array of identifiers for all devices present for all drivers.
- `_Ret_opt_cap_ pCount STAR_DEVICE_ID *STAR_API_CC STAR_getDeviceListForType (STAR_DEVICE_ID deviceType, _Out_ U32 *pCount)`

Gets an array of identifiers for all devices present for the given device type.
- `_Ret_opt_cap_ pCount STAR_DEVICE_ID *STAR_API_CC STAR_getDeviceListForTypes (STAR_DEVICE_ID deviceTypes[], U32 numRequestedTypes, _Out_ U32 *pCount)`

Gets an array of identifiers for all devices present for the given device types in an array.
- `_Ret_opt_cap_ count STAR_DEVICE_ID *STAR_API_CC STAR_getDeviceListForDriver (STAR_DRIVER_ID driverId, _Out_ U32 *count)`

Gets an array of device identifiers for all devices present for a specific driver.
- `void STAR_API_CC STAR_destroyDeviceList (_Post_ptr_invalid_ STAR_DEVICE_ID *pDeviceList)`

Frees a device list previously created by a call to `STAR_getDeviceList()` or `STAR_getDeviceListForDriver()`.
- `_Check_return_ STAR_DEVICE_LISTENER_ID STAR_API_CC STAR_registerDeviceListener (_In_ STAR_DEVICE_ID deviceId, STAR_DeviceEventFunc listenerFunc, _In_opt_ void *pContextInfo, int notifyForCurrent)`

Registers a Call-back function that will be called whenever a device is added or removed.
- `_Check_return_ STAR_DEVICE_LISTENER_ID STAR_API_CC STAR_registerDeviceListenerForDriver (STAR_DRIVER_ID driverId, _In_ STAR_DeviceEventFunc listenerFunc, _In_opt_ void *pContextInfo, int notifyForCurrent)`

Registers a Call-back function that will be called whenever a device for a specific driver is added or removed.
- `_Check_return_ STAR_DEVICE_LISTENER_ID STAR_API_CC STAR_registerDeviceListenerForDevice (STAR_DEVICE_ID deviceId, _In_ STAR_DeviceEventFunc listenerFunc, _In_opt_ void *pContextInfo)`

Registers a Call-back function that will be called whenever a specific device is removed.
- `int STAR_API_CC STAR_unregisterDeviceListener (STAR_DEVICE_LISTENER_ID listenerId)`

Unregister a Call-back function that was previously registered to be called whenever a device is added or removed.
- `int STAR_API_CC STAR_resetDevice (STAR_DEVICE_ID deviceId)`

Resets a device.
- `STAR_VERSION_INFO *STAR_API_CC STAR_getDeviceFirmwareVersion (STAR_DEVICE_ID deviceId)`

Gets the version of a device's firmware.
- `STAR_BUS_TYPE STAR_API_CC STAR_getDeviceBusType (STAR_DEVICE_ID deviceId)`

Gets the bus type (PCI,USB, Virtual, etc) for the device.
- `_Check_return_ _Ret_opt_z_ char *STAR_API_CC STAR_getDeviceBusTypeAsString (STAR_DEVICE_ID deviceId)`

Gets a string representation of a device's bus type.
- `int STAR_API_CC STAR_getDeviceIndex (STAR_DEVICE_ID deviceId)`

Gets the device index by Driver.
- `STAR_DEVICE_TYPE STAR_API_CC STAR_getDeviceType (STAR_DEVICE_ID deviceId)`

Gets the device's type identifier.
- `_Check_return_ _Ret_opt_z_ char *STAR_API_CC STAR_getDeviceTypeAsString (STAR_DEVICE_ID deviceId)`

Gets a string representation of a device's type.
- `STAR_DRIVER_ID STAR_API_CC STAR_getDeviceDriver (STAR_DEVICE_ID deviceId)`

Gets the identifier of the device's driver.
- `_Check_return_ _Ret_opt_z_ char *STAR_API_CC STAR_getDeviceName (STAR_DEVICE_ID deviceId)`

Gets the name of the device identified by a given identifier.
- `int STAR_API_CC STAR_setDeviceName (STAR_DEVICE_ID deviceId, _In_z_ char *newName)`

Sets the name of the device (friendly name).
- `int STAR_API_CC STAR_resetDeviceName (STAR_DEVICE_ID deviceId)`

Resets a device's name to its default value.

- `_Check_return_ _Ret_opt_z_ char *STAR_API_CC STAR_getDeviceSerialNumber (STAR_DEVICE_ID deviceId)`
Gets the serial number (unique alphanumeric identifier) of the device identified by a given identifier.
- `STAR_DEVICE_TYPE STAR_API_CC STAR_getDeviceTxRxCapabilities (STAR_DEVICE_ID deviceId)`
Determine whether the device is capable of transmitting and receiving packets on channels.
- `STAR_DEVICE_TYPE STAR_API_CC STAR_getDeviceConfigCapabilities (STAR_DEVICE_ID deviceId)`
Determine whether the device is capable of being configured.
- `STAR_CHANNEL_MASK STAR_API_CC STAR_getDeviceChannels (STAR_DEVICE_ID deviceId)`
Gets the channels on a device.
- `STAR_CHANNEL_TYPE STAR_API_CC STAR_isChannelOpen (STAR_DEVICE_ID deviceId, unsigned int channelNumber)`
Returns whether a given channel on a device is open and, if it is open, whether it is connected to a device or an application.
- `STAR_DEVICE_ID STAR_API_CC STAR_getLocalDeviceAttachedToChannel (STAR_DEVICE_ID deviceId, unsigned int channelNumber)`
Returns the id of a device attached to a given channel on a given device.

7.3.1 Detailed Description

Functions providing information on devices and drivers present.

7.3.2 Macro Definition Documentation

7.3.2.1 `#define STAR_CHANNEL_MASK U32`

Type used to represent channels that exist on a device.

(bit 0 set = channel 0 exists, bit 2 set = channel 2 exists etc...).

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

`link_events.c`, `star-test.c`, `star_performance_tester.c`, and `ui.c`.

7.3.2.2 `#define STAR_DEVICE_ALL 0x00ffffU`

All devices.

Added in version:

STAR-System v3.0 Beta 2 - 24th July 2015

Examples:

`star-test.c`.

7.3.2.3 #define STAR_DEVICE_BRICK_MK2 9U

The STAR-Dundee SpaceWire Brick Mk2 device type.

Added in version:

STAR-System v3.0 Beta 1 - 31st March 2015

Examples:

brickMk2_configuration_example.c, and link_events.c.

7.3.2.4 #define STAR_DEVICE_BRICK_MK3 19U

The STAR-Dundee SpaceWire Brick Mk3 device type.

Added in version:

STAR-System v3.0 Beta 1 - 31st March 2015

Examples:

link_events.c, timestamp_test.c, and triggering_example.c.

7.3.2.5 #define STAR_DEVICE_CONFIG_NOT_SUPPORTED 0x00ffffcU

A device which is not capable of being configured.

Added in version:

STAR-System v3.0 Beta 9 - 26th July 2016

7.3.2.6 #define STAR_DEVICE_CONFIG_SUPPORTED 0x00ffffbU

A device which is capable of being configured.

Added in version:

STAR-System v3.0 Beta 9 - 26th July 2016

Examples:

device_identifier_example.c, mk2_configuration_example.c, port_control_status_example.c, router_↵
configuration_example.c, routing_table_example.c, star-test.c, time-code_test.c, and user_registers_↵
example.c.

7.3.2.7 #define STAR_DEVICE_CONFORMANCE_TESTER 5U

The STAR-Dundee SpaceWire Conformance Tester device type.

Added in version:

STAR-System v3.0 Beta 1 - 31st March 2015

7.3.2.8 `#define STAR_DEVICE_CONFORMANCE_TESTER_MK2 29U`

The STAR-Dundee SpaceWire Conformance Tester Mk2 device type.

Added in version:

[STAR-System v4.00 Beta 3 - 24th October 2018](#)

7.3.2.9 `#define STAR_DEVICE_CPCI_MK2 14U`

The STAR-Dundee SpaceWire cPCI Mk2 device type.

Added in version:

[STAR-System v3.0 Beta 1 - 31st March 2015](#)

Examples:

[link_events.c](#).

7.3.2.10 `#define STAR_DEVICE_EGSE 13U`

The STAR-Dundee SpaceWire EGSE device type.

Added in version:

[STAR-System v3.0 Beta 1 - 31st March 2015](#)

7.3.2.11 `#define STAR_DEVICE_EGSE_MK2 30U`

The STAR-Dundee SpaceWire EGSE Mk2 device type.

Added in version:

[STAR-System v4.00 Beta 3 - 24th October 2018](#)

7.3.2.12 `#define STAR_DEVICE_IP_TUNNEL 6U`

The STAR-Dundee SpaceWire IP-Tunnel device type.

Added in version:

[STAR-System v3.0 Beta 1 - 31st March 2015](#)

7.3.2.13 `#define STAR_DEVICE_LINK_ANALYSER 4U`

The STAR-Dundee SpaceWire Link Analyser device type.

Added in version:

[STAR-System v3.0 Beta 1 - 31st March 2015](#)

7.3.2.14 #define STAR_DEVICE_LINK_ANALYSER_MK2 10U

The STAR-Dundee SpaceWire Link Analyser Mk2 device type.

Added in version:

[STAR-System v3.0 Beta 1 - 31st March 2015](#)

7.3.2.15 #define STAR_DEVICE_LINK_ANALYSER_MK3 28U

The STAR-Dundee SpaceWire Link Analyser Mk3 device type.

Added in version:

[STAR-System v4.00 Beta 3 - 24th October 2018](#)

7.3.2.16 #define STAR_DEVICE_PCI 1U

The STAR-Dundee SpaceWire PCI-2 and cPCI device type.

Added in version:

[STAR-System v3.0 Beta 1 - 31st March 2015](#)

7.3.2.17 #define STAR_DEVICE_PCI_MK2 8U

The STAR-Dundee SpaceWire PCI Mk2 device type.

Added in version:

[STAR-System v3.0 Beta 1 - 31st March 2015](#)

Examples:

[link_events.c](#), [packet_subsystem_example.c](#), and [pciMk2_configuration_example.c](#).

7.3.2.18 #define STAR_DEVICE_PCIE 11U

The STAR-Dundee SpaceWire PCI Express device type.

Added in version:

[STAR-System v3.0 Beta 1 - 31st March 2015](#)

Examples:

[link_events.c](#), and [triggering_example.c](#).

7.3.2.19 #define STAR_DEVICE_PXI_INTERFACE 21U

The STAR-Dundee SpaceWire PXI Interface device type.

Added in version:

[STAR-System v3.0 Beta 4 - 13th November 2015](#)

Examples:

[link_events.c](#), and [triggering_example.c](#).

7.3.2.20 `#define STAR_DEVICE_PXI_INTERFACE_MK2 33U`

The STAR-Dundee SpaceWire PXI Interface Mk2 device type.

Added in version:

STAR-System v4.00 Beta 4 - 16th October 2019

Examples:

[link_events.c](#).

7.3.2.21 `#define STAR_DEVICE_PXI_RMAP 22U`

The STAR-Dundee SpaceWire PXI RMAP device type.

Added in version:

STAR-System v3.0 Beta 4 - 13th November 2015

Examples:

[link_events.c](#), [rmap_target_example.c](#), and [triggering_example.c](#).

7.3.2.22 `#define STAR_DEVICE_PXI_RMAP_MK2 34U`

The STAR-Dundee SpaceWire PXI RMAP Mk2 device type.

Added in version:

STAR-System v4.00 Beta 4 - 16th October 2019

Examples:

[link_events.c](#).

7.3.2.23 `#define STAR_DEVICE_PXI_ROUTER_12 24U`

The STAR-Dundee SpaceWire PXI 12 port Router device type.

Added in version:

STAR-System v3.5 - 17th March 2017

Examples:

[link_events.c](#), and [triggering_example.c](#).

7.3.2.24 `#define STAR_DEVICE_PXI_ROUTER_8 23U`

The STAR-Dundee SpaceWire PXI 8 port Router device type.

Added in version:

STAR-System v3.0 Beta 4 - 13th November 2015

7.3.2.25 #define STAR_DEVICE_PXI_ROUTER_MK2 35U

The STAR-Dundee SpaceWire PXI Router Mk2 device type.

Added in version:

[STAR-System v4.00 Beta 4 - 16th October 2019](#)

Examples:

[link_events.c](#).

7.3.2.26 #define STAR_DEVICE_RECORDER 18U

The STAR-Dundee SpaceWire Recorder device type.

Added in version:

[STAR-System v3.0 Beta 1 - 31st March 2015](#)

7.3.2.27 #define STAR_DEVICE_ROUTER_MK2S 7U

The STAR-Dundee SpaceWire Router Mk2S device type.

Added in version:

[STAR-System v3.0 Beta 1 - 31st March 2015](#)

Examples:

[brickMk2_configuration_example.c](#), [link_events.c](#), and [routerMk2S_configuration_example.c](#).

7.3.2.28 #define STAR_DEVICE_ROUTER_USB 2U

The STAR-Dundee SpaceWire Router-USB device type.

Added in version:

[STAR-System v3.0 Beta 1 - 31st March 2015](#)

7.3.2.29 #define STAR_DEVICE_RTC 12U

The STAR-Dundee SpaceWire RTC device type.

Added in version:

[STAR-System v3.0 Beta 1 - 31st March 2015](#)

7.3.2.30 #define STAR_DEVICE_SERIAL_SIZE 16

Size in bytes (including null terminator) of a device serial number.

Added in version:

[STAR-System v1.0 - 17th October 2011](#)

7.3.2.31 `#define STAR_DEVICE_SPLT 15U`

The STAR-Dundee SpaceWire Physical Layer Tester device type.

Added in version:

[STAR-System v3.0 Beta 1 - 31st March 2015](#)

Examples:

[link_events.c](#).

7.3.2.32 `#define STAR_DEVICE_STAR_FIRE 16U`

The STAR-Dundee STAR Fire device type.

Added in version:

[STAR-System v3.0 Beta 1 - 31st March 2015](#)

7.3.2.33 `#define STAR_DEVICE_STAR_FIRE_MK3 26U`

The STAR-Dundee STAR Fire Mk3 device type.

Added in version:

[STAR-System v3.2 - 24th November 2016](#)

Examples:

[link_events.c](#).

7.3.2.34 `#define STAR_DEVICE_STAR_ULTRA_PCIE 32U`

The STAR-Dundee STAR-Ultra-PCle device type.

Added in version:

[STAR-System v4.00 - 19th November 2019](#)

7.3.2.35 `#define STAR_DEVICE_TRAFFIC_GENERATOR 20U`

The Shimafuji Traffic Generator device type.

Added in version:

[STAR-System v3.0 Beta 1 - 31st March 2015](#)

7.3.2.36 `#define STAR_DEVICE_TXRX_NOT_SUPPORTED 0x00fffffeU`

A device which is not capable of transmitting and receiving packets.

Added in version:

[STAR-System v3.0 Beta 1 - 31st March 2015](#)

7.3.2.37 #define STAR_DEVICE_TXRX_SUPPORTED 0x00ffffdU

A device which is capable of transmitting and receiving packets.

Added in version:

STAR-System v3.0 Beta 1 - 31st March 2015

Declaration changed in version:

STAR-System v3.0 Beta 2 - 24th July 2015

Examples:

star-test.c, and star_performance_tester.c.

7.3.2.38 #define STAR_DEVICE_TYPE U32

Type of device that a device is.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

brickMk2_configuration_example.c, link_events.c, packet_subsystem_example.c, pciMk2_configuration_↔
example.c, routerMk2S_configuration_example.c, star-test.c, and ui.c.

7.3.2.39 #define STAR_DEVICE_UNKNOWN 0U

The "unknown" device type.

Added in version:

STAR-System v3.0 Beta 1 - 31st March 2015

Examples:

star-test.c, and triggering_example.c.

7.3.2.40 #define STAR_DEVICE_USB_BRICK 3U

The STAR-Dundee SpaceWire-USB Brick device type.

Added in version:

STAR-System v3.0 Beta 1 - 31st March 2015

7.3.2.41 #define STAR_DEVICE_WBS_II 17U

The STAR-Dundee Wide Band Spectrometer II device type.

Added in version:

STAR-System v3.0 Beta 1 - 31st March 2015

7.3.3 Typedef Documentation

7.3.3.1 typedef CFG_INFO_ID_TYPE STAR_DEVICE_ID

Unique value used to identify an individual device.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

7.3.3.2 typedef CFG_INFO_ID_TYPE STAR_DEVICE_LISTENER_ID

Unique value used to identify an individual device listener.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

7.3.3.3 typedef void(STAR_API_CC * STAR_DeviceEventFunc) (STAR_DEVICE_LISTENER_ID deviceListenerIdentifier, STAR_DRIVER_ID driverIdentifier, STAR_DEVICE_ID deviceId, int deviceAdded, void *pContextInfo)

The function type for device listener functions which are called when a device is added or removed.

Parameters

	<i>deviceListenerIdentifier</i>	the device listener that this event corresponds to
	<i>driverIdentifier</i>	the driver on which the device was added or removed
	<i>deviceId</i>	the device which was added or removed
	<i>deviceAdded</i>	whether the device has been added (1) or removed (0)
in	<i>pContextInfo</i>	a pointer to context information that can be passed to the function

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

7.3.3.4 typedef CFG_INFO_ID_TYPE STAR_DRIVER_ID

Unique value used to identify an individual driver.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

7.3.3.5 typedef CFG_INFO_ID_TYPE STAR_DRIVER_LISTENER_ID

Unique value used to identify an individual driver listener.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

7.3.3.6 typedef void(STAR_API_CC * STAR_DriverEventFunc) (STAR_DRIVER_LISTENER_ID driverListenerIdentifier, STAR_DRIVER_ID driverIdentifier, int driverAdded, void *pContextInfo)

The function type for driver listener functions which are called when a new driver is added, or an existing one removed.

Parameters

	<i>driverListenerIdentifier</i>	the driver listener that this event corresponds to
	<i>driverIdentifier</i>	The driver which has been added or removed
	<i>driverAdded</i>	Whether the driver was added (1) or removed (0)
in	<i>pContextInfo</i>	A pointer to optional context information that can be passed to the function

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

7.3.4 Enumeration Type Documentation

7.3.4.1 enum STAR_BUS_TYPE

The different types of bus that can be used to connect a SpaceWire device to a PC.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Enumerator

STAR_BUS_UNKNOWN The bus type of a device is unknown.

STAR_BUS_PCI The PCI bus type.

STAR_BUS_USB The USB bus type.

STAR_BUS_PCIE The PCI Express (PCIe) bus type.

Added in version:

STAR-System v1.2 - 13th February 2012

STAR_BUS_TCP The TCP/IP bus type, used by Ethernet devices.

STAR_BUS_CPCI The cPCI bus type.

Added in version:

STAR-System v1.6 - 29th June 2012

STAR_BUS_VIRTUAL The bus type for virtual devices.

7.3.4.2 enum STAR_DRIVER_TYPE

Available driver types.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Enumerator

STAR_DRIVER_INVALID Invalid.

STAR_DRIVER_TYPE_USB USB Driver.

STAR_DRIVER_TYPE_PCI PCI Driver.

STAR_DRIVER_TYPE_TCPIP TCP/IP Driver.

STAR_DRIVER_TYPE_VIRTUAL Virtual Driver.

7.3.5 Function Documentation

7.3.5.1 void **STAR_API_CC** STAR_destroyDeviceList (_Post_ptr_invalid_ STAR_DEVICE_ID * *pDeviceList*)

Frees a device list previously created by a call to [STAR_getDeviceList\(\)](#) or [STAR_getDeviceListForDriver\(\)](#).

Parameters

pDeviceList the device list to be freed

Added in version:

STAR-System v1.1 - 20th December 2011

Examples:

advanced_send_example.c, device_identifier_example.c, driver_list_example.c, firmware_version_example.c, mk2_configuration_example.c, port_control_status_example.c, rmap_target_example.c, router_configuration_example.c, routing_table_example.c, simple_send_example.c, star-test.c, star_performance_tester.c, time-code_test.c, timestamp_test.c, transmit_errors_example.c, triggering_example.c, ui.c, user_registers_example.c, and utilities.c.

7.3.5.2 void STAR_API_CC STAR_destroyDriverList (_Post_ptr_invalid_ STAR_DRIVER_ID * pDriverList)

Frees a driver list previously created by a call to STAR_getDriverList().

Parameters

pDriverList the driver list to be freed

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

utilities.c.

7.3.5.3 STAR_BUS_TYPE STAR_API_CC STAR_getDeviceBusType (STAR_DEVICE_ID deviceld)

Gets the bus type (PCI,USB, Virtual, etc) for the device.

Parameters

deviceld the identifier of the device to get the bus type of

Returns

1 if the driver is virtual, otherwise 0

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

7.3.5.4 _Check_return_ _Ret_opt_z_ char* STAR_API_CC STAR_getDeviceBusTypeAsString (STAR_DEVICE_ID deviceld)

Gets a string representation of a device's bus type.

The string returned should be freed by calling STAR_destroyString().

Parameters

<i>deviceld</i>	the identifier of the device to get the bus type of
-----------------	---

Returns

the device's bus type as a null terminated string

Added in version:

STAR-System v1.0 - 17th October 2011

Declaration changed in version:

STAR-System v1.1 - 20th December 2011

Examples:

star-test.c.

7.3.5.5 **STAR_CHANNEL_MASK STAR_API_CC STAR_getDeviceChannels (STAR_DEVICE_ID *deviceld*)**

Gets the channels on a device.

Parameters

<i>deviceld</i>	The device to check
-----------------	---------------------

Returns

A bit mask representing the channels on the device

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

link_events.c, star-test.c, star_performance_tester.c, ui.c, and utilities.c.

7.3.5.6 **STAR_DEVICE_TYPE STAR_API_CC STAR_getDeviceConfigCapabilities (STAR_DEVICE_ID *deviceld*)**

Determine whether the device is capable of being configured.

This function can be used to determine whether a device can be configured using the Configuration APIs ([STAR_DEVICE_CONFIG_SUPPORTED](#)), or whether it is a special device such as a Link Analyser or Conformance Tester ([STAR_DEVICE_CONFIG_NOT_SUPPORTED](#)) which does not have a router with a configuration port.

Parameters

<i>deviceld</i>	the identifier of the device to get whether the device can be configured
-----------------	--

Returns

a value indicating whether the device is capable of being configured ([STAR_DEVICE_CONFIG_SUPPORTED](#)), not capable ([STAR_DEVICE_CONFIG_NOT_SUPPORTED](#)), or 0 if an error occurred

Added in version:

STAR-System v3.0 Beta 9 - 26th July 2016

Examples:

star-test.c.

7.3.5.7 STAR_DRIVER_ID STAR_API_CC STAR_getDeviceDriver (STAR_DEVICE_ID *deviceld*)

Gets the identifier of the device's driver.

Parameters

<i>deviceld</i>	the identifier of the device to get the driver of
-----------------	---

Returns

the driver for the device

Added in version:

STAR-System v1.12 - 14th November 2012

7.3.5.8 STAR_VERSION_INFO* STAR_API_CC STAR_getDeviceFirmwareVersion (STAR_DEVICE_ID *deviceld*)

Gets the version of a device's firmware.

Parameters

<i>deviceld</i>	Identifier for device to be checked
-----------------	-------------------------------------

Returns

Pointer to a version structure that will contain the device's version info, or NULL if a memory allocation error occurred.

Note

This structure must be freed using [STAR_destroyVersionInfo\(\)](#) when no longer required.

Added in version:

STAR-System v1.0 - 17th October 2011

Examples:

[firmware_version_example.c](#), and [star-test.c](#).

7.3.5.9 int STAR_API_CC STAR_getDeviceIndex (STAR_DEVICE_ID *deviceld*)

Gets the device index by Driver.

Provided for backwards compatibility reasons, not for typical usage.

Parameters

<i>deviceld</i>	The device to check
-----------------	---------------------

Returns

The index number of the device, or -1 if the function call was unsuccessful

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

[star-test.c](#).

7.3.5.10 `_Ret_opt_cap_count STAR_DEVICE_ID* STAR_API_CC STAR_getDeviceList ( _Out_ U32 * count )`

Gets an array of identifiers for all devices present for all drivers.

Parameters

<i>out</i>	<i>count</i>	Count of device identifiers held in the array.
------------	--------------	--

Returns

Array of identifiers for all devices present.

Note

This function returns a snapshot of the current state of the system and is not automatically updated. This array must be freed using [STAR_destroyDeviceList\(\)](#) when no longer required.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

[firmware_version_example.c](#), [star-test.c](#), and [utilities.c](#).

7.3.5.11 `_Ret_opt_cap_count STAR_DEVICE_ID* STAR_API_CC STAR_getDeviceListForDriver ( STAR_DRIVER_ID driverId, _Out_ U32 * count )`

Gets an array of device identifiers for all devices present for a specific driver.

Parameters

	<i>driverId</i>	ID of driver to get devices for, obtained from a call to STAR_getDriverList()
<i>out</i>	<i>count</i>	Count of device identifiers held in the array.

Returns

Array of identifiers for devices for the given driver.

Note

This function returns a snapshot of the current state of the system and is not automatically updated. This array must be freed using [STAR_destroyDeviceList\(\)](#) when no longer required.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

7.3.5.12 `_Ret_opt_cap_pCount STAR_DEVICE_ID* STAR_API_CC STAR_getDeviceListForType ( STAR_DEVICE_TYPE deviceType, _Out_ U32 * pCount )`

Gets an array of identifiers for all devices present for the given device type.

The device type may be the type of a specific device, e.g. [STAR_DEVICE_BRICK_MK3](#), or it may be one of the special device types, [STAR_DEVICE_TXRX_SUPPORTED](#), [STAR_DEVICE_TXRX_NOT_SUPPORTED](#), [STAR_DEVICE_CONFIG_SUPPORTED](#) or [STAR_DEVICE_CONFIG_NOT_SUPPORTED](#) to get the list of all devices which can or cannot transmit and receive packets or be configured, or [STAR_DEVICE_ALL](#) to get all devices.

Parameters

	<i>deviceType</i>	The type of device to get the list for
out	<i>pCount</i>	A pointer to a variable which will be updated to include the count of device identifiers held in the returned array.

Returns

Array of identifiers for all devices of the given type present.

Note

This function returns a snapshot of the current state of the system and is not automatically updated. This array must be freed using [STAR_destroyDeviceList\(\)](#) when no longer required.

Added in version:

STAR-System v3.0 Beta 1 - 31st March 2015

Examples:

[device_identifier_example.c](#), [mk2_configuration_example.c](#), [port_control_status_example.c](#), [rmap_target_example.c](#), [router_configuration_example.c](#), [routing_table_example.c](#), [star-test.c](#), [star_performance_tester.c](#), [time-code_test.c](#), [timestamp_test.c](#), and [user_registers_example.c](#).

7.3.5.13 `_Ret_opt_cap_ pCount STAR_DEVICE_ID* STAR_API_CC STAR_getDeviceListForTypes ( STAR_DEVICE_TYPE deviceTypes[], U32 numRequestedTypes, _Out_ U32 * pCount )`

Gets an array of identifiers for all devices present for the given device types in an array.

Each device type may be the type of a specific device, e.g. [STAR_DEVICE_BRICK_MK3](#), or it may be one of the special device types, [STAR_DEVICE_TXRX_SUPPORTED](#), [STAR_DEVICE_TXRX_NOT_SUPPORTED](#), [STAR_DEVICE_CONFIG_SUPPORTED](#) or [STAR_DEVICE_CONFIG_NOT_SUPPORTED](#), to get the list of all devices which can or cannot transmit and receive packets or be configured, or [STAR_DEVICE_ALL](#) to get all devices.

Parameters

	<i>deviceTypes</i>	An array of device types to get the list for
	<i>numRequestedTypes</i>	The number of entries in the deviceTypes array
out	<i>pCount</i>	A pointer to a variable which will be updated to include the count of device identifiers held in the returned array.

Returns

Array of identifiers for all devices of the given type present.

Note

This function returns a snapshot of the current state of the system and is not automatically updated. This array must be freed using [STAR_destroyDeviceList\(\)](#) when no longer required.

Added in version:

STAR-System v3.0 Beta 3 - 4th September 2015

Examples:

[triggering_example.c](#), and [ui.c](#).

7.3.5.14 `_Check_return_ _Ret_opt_z_ char* STAR_API_CC STAR_getDeviceName ( STAR_DEVICE_ID deviceld )`

Gets the name of the device identified by a given identifier.

The string returned should be freed by calling [STAR_destroyString\(\)](#).

Parameters

<i>deviceld</i>	the identifier of the device to get the type of
-----------------	---

Returns

the device's name as a null terminated string

Added in version:

[STAR-System v0.8 \(Beta 1\) - 22nd August 2011](#)

Declaration changed in version:

[STAR-System v1.1 - 20th December 2011](#)

Examples:

[device_identifier_example.c](#), [mk2_configuration_example.c](#), [port_control_status_example.c](#), [router_configuration_example.c](#), [routing_table_example.c](#), [star-test.c](#), [star_performance_tester.c](#), [time-code_test.c](#), [timestamp_test.c](#), [triggering_example.c](#), [ui.c](#), [user_registers_example.c](#), and [utilities.c](#).

7.3.5.15 `_Check_return_ _Ret_opt_z_ char* STAR_API_CC STAR_getDeviceSerialNumber ( STAR_DEVICE_ID deviceld )`

Gets the serial number (unique alphanumeric identifier) of the device identified by a given identifier.

The string returned should be freed by calling [STAR_destroyString\(\)](#).

Parameters

<i>deviceld</i>	the identifier of the device to get the serial number of
-----------------	--

Returns

the device's serial number as a null terminated string

Added in version:

[STAR-System v1.0 - 17th October 2011](#)

Declaration changed in version:

[STAR-System v1.1 - 20th December 2011](#)

Examples:

[star-test.c](#), [timestamp_test.c](#), [triggering_example.c](#), and [utilities.c](#).

7.3.5.16 `STAR_DEVICE_TYPE STAR_API_CC STAR_getDeviceTxRxCapabilities ( STAR_DEVICE_ID deviceld )`

Determine whether the device is capable of transmitting and receiving packets on channels.

This function can be used to determine whether a device can be used to route packets over channels and in and out of the device's SpaceWire links ([STAR_DEVICE_TXRX_SUPPORTED](#)), or whether it is a special device such as a Link Analyser or Conformance Tester ([STAR_DEVICE_TXRX_NOT_SUPPORTED](#)) which cannot route packets. Note that some devices, such as the EGSE may be capable of transmitting and receiving packets, but a value of [STAR_DEVICE_TXRX_NOT_SUPPORTED](#) will be returned, as these devices do not simply route any packets transmitted or received on channels, and must be specially configured to transmit or receive packets.

Parameters

<i>deviceld</i>	the identifier of the device to get whether the device can transmit and receive
-----------------	---

Returns

a value indicating whether the device is capable of transmitting and receiving packets ([STAR_DEVICE_TX↔RX_SUPPORTED](#)), not capable ([STAR_DEVICE_TXRX_NOT_SUPPORTED](#)), or 0 if an error occurred

Added in version:

STAR-System v3.0 Beta 2 - 24th July 2015

7.3.5.17 STAR_DEVICE_TYPE STAR_API_CC STAR_getDeviceType (STAR_DEVICE_ID *deviceld*)

Gets the device's type identifier.

Parameters

<i>deviceld</i>	the identifier of the device to get the type of
-----------------	---

Returns

the type of the device

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

[brickMk2_configuration_example.c](#), [link_events.c](#), [pciMk2_configuration_example.c](#), [routerMk2S_configuration↔_example.c](#), and [triggering_example.c](#).

7.3.5.18 _Check_return_ _Ret_opt_z_char* STAR_API_CC STAR_getDeviceTypeAsString (STAR_DEVICE_ID *deviceld*)

Gets a string representation of a device's type.

The string returned should be freed by calling [STAR_destroyString\(\)](#).

Parameters

<i>deviceld</i>	the identifier of the device to get the type of
-----------------	---

Returns

the device's type as a null terminated string

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Declaration changed in version:

STAR-System v1.1 - 20th December 2011

Examples:

[link_events.c](#), and [star-test.c](#).

7.3.5.19 `_Ret_opt_cap_count STAR_DRIVER_ID* STAR_API_CC STAR_getDriverList ( int physicalDeviceDrivers, int virtualDeviceDrivers, _Out_ U32 * count )`

Gets an array of driver identifiers for all drivers of the requested type(s).

Parameters

<i>physicalDeviceDrivers</i>	Whether to include physical device drivers in the list
<i>virtualDeviceDrivers</i>	Whether to include virtual device drivers in the list
<i>out</i> <i>count</i>	Count of driver identifiers held in the array.

Returns

Array of identifiers for drivers.

Note

This function returns a snapshot of the current state of the system and is not automatically updated. This array must be freed using [STAR_destroyDriverList\(\)](#) when no longer required.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

[driver_list_example.c](#).

7.3.5.20 STAR_DRIVER_TYPE STAR_API_CC STAR_getDriverType (STAR_DRIVER_ID *driverId*)

Gets the type (USB/PCI/specific virtual type identifier) of a given driver.

Parameters

<i>driverId</i>	Identifier for driver to be checked
-----------------	-------------------------------------

Returns

Type of the driver

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

[driver_list_example.c](#).

7.3.5.21 STAR_VERSION_INFO* STAR_API_CC STAR_getDriverVersion (STAR_DRIVER_ID *driverId*)

Gets the Version of a given driver.

Parameters

<i>driverId</i>	ID of driver to get version for, obtained from a call to STAR_getDriverList()
-----------------	---

Returns

Pointer to a version structure that will contain specific drivers version info, or NULL if there was an error.

Note

This structure must be freed using `STAR_destroyVersionInfo()` when no longer required.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

7.3.5.22 **STAR_DEVICE_ID STAR_API_CC STAR_getLocalDeviceAttachedToChannel (STAR_DEVICE_ID *deviceld*, unsigned int *channelNumber*)**

Returns the id of a device attached to a given channel on a given device.

Parameters

<i>deviceld</i>	The device to check
<i>channelNumber</i>	The channel number to check

Returns

The attached device, or 0 if no device is attached

Added in version:

STAR-System v1.1 - 20th December 2011

7.3.5.23 **STAR_CHANNEL_TYPE STAR_API_CC STAR_isChannelOpen (STAR_DEVICE_ID *deviceld*, unsigned int *channelNumber*)**

Returns whether a given channel on a device is open and, if it is open, whether it is connected to a device or an application.

Parameters

<i>deviceld</i>	The device to check
<i>channelNumber</i>	The channel number to check

Returns

Whether the channel is connected a device (`STAR_CHANNEL_TYPE_DEVICE`) or application (`STAR_CHANNEL_TYPE_APPLICATION`) if the channel is open, or not open (`STAR_CHANNEL_TYPE_NOT_OPEN`) if the channel is not open

Added in version:

STAR-System v1.1 - 20th December 2011

7.3.5.24 **_Check_return_ STAR_DEVICE_LISTENER_ID STAR_API_CC STAR_registerDeviceListener (_In_ STAR_DeviceEventFunc *listenerFunc*, _In_opt_ void * *pContextInfo*, int *notifyForCurrent*)**

Registers a Call-back function that will be called whenever a device is added or removed.

Parameters

<i>in</i>	<i>listenerFunc</i>	Call-back Function
<i>in</i>	<i>pContextInfo</i>	Optional pointer to some context information that will be passed to the listener function when it is called
	<i>notifyForCurrent</i>	Whether the function should be called for all existing devices to indicate that they've been added

Returns

Non-Zero if call-back was registered successfully

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

7.3.5.25 `_Check_return_ STAR_DEVICE_LISTENER_ID STAR_API_CC STAR_registerDeviceListenerForDevice ( STAR_DEVICE_ID deviceld, _In_ STAR_DeviceEventFunc listenerFunc, _In_opt_ void * pContextInfo )`

Registers a Call-back function that will be called whenever a specific device is removed.

Parameters

	<i>deviceld</i>	Device to register device listener for
	<i>listenerFunc</i>	Call-back Function
<i>in</i>	<i>pContextInfo</i>	Optional A pointer to some context information that will be passed to the listener function when it is called

Returns

Non-Zero if call-back was registered successfully

7.3.5.26 `_Check_return_ STAR_DEVICE_LISTENER_ID STAR_API_CC STAR_registerDeviceListenerForDriver ( STAR_DRIVER_ID driverId, _In_ STAR_DeviceEventFunc listenerFunc, _In_opt_ void * pContextInfo, int notifyForCurrent )`

Registers a Call-back function that will be called whenever a device for a specific driver is added or removed.

Parameters

	<i>driverId</i>	Driver to register device listener for
	<i>listenerFunc</i>	Call-back Function
<i>in</i>	<i>pContextInfo</i>	Optional pointer to some context information that will be passed to the listener function when it is called
	<i>notifyForCurrent</i>	Whether the function should be called for all existing devices to indicate that they've been added

Returns

Non-Zero if call-back was registered successfully

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

7.3.5.27 `_Check_return_ STAR_DRIVER_LISTENER_ID STAR_API_CC STAR_registerDriverListener ( STAR_DriverEventFunc listenerFunc, _In_opt_ void * pContextInfo, int notifyForCurrent )`

Registers a Call-back function that will be called whenever a Driver is added or removed.

Parameters

	<i>listenerFunc</i>	Call-back Function
in	<i>pContextInfo</i>	Optional pointer to some context information that will be passed to the listener function when it is called
	<i>notifyForCurrent</i>	Whether the function should be called for all existing drivers to indicate that they've been added

Returns

1 if call-back was registered successfully, else 0

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

7.3.5.28 int STAR_API_CC STAR_resetDevice (STAR_DEVICE_ID *deviceld*)

Resets a device.

This function will return an error if called with a Virtual Device's identifier.

Note

Calling this function while packets are being transmitted and/or received can cause the device to get in to a bad state.

Parameters

	<i>deviceld</i>	Device to be reset
--	-----------------	--------------------

Returns

1 if the device was successfully reset, else 0

Added in version:

STAR-System v1.0 - 17th October 2011

Examples:

star-test.c.

7.3.5.29 int STAR_API_CC STAR_resetDeviceName (STAR_DEVICE_ID *deviceld*)

Resets a device's name to its default value.

Parameters

	<i>deviceld</i>	Device to have it's name reset
--	-----------------	--------------------------------

Returns

1 if the device's name was successfully reset, else 0

Added in version:

STAR-System v4.00 Beta 1 - 25th September 2018

7.3.5.30 int STAR_API_CC STAR_setDeviceName (STAR_DEVICE_ID *deviceld*, _In_z_ char * *newName*)

Sets the name of the device (friendly name).

Parameters

	<i>deviceId</i>	The device to set a name for
<i>in</i>	<i>newName</i>	The new name for the device (newName must be null terminated and shorter than 256 characters). If newName is NULL then the device name is reset to the default.

Returns

1 if the name could be set, otherwise 0

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

7.3.5.31 int STAR_API_CC STAR_unregisterDeviceListener (STAR_DEVICE_LISTENER_ID listenerId)

Unregister a Call-back function that was previously registered to be called whenever a device is added or removed.

Parameters

<i>listenerId</i>	Device Listener ID returned from a previous call to STAR_registerDeviceListener
-------------------	---

Returns

1 if call-back was unregistered successfully, else 0

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

7.3.5.32 int STAR_API_CC STAR_unregisterDriverListener (STAR_DRIVER_LISTENER_ID listenerId)

Unregister a Call-back function that was previously registered to be called whenever a driver is added or removed.

Parameters

<i>listenerId</i>	Driver Listener ID returned from a previous call to STAR_registerDriverListener
-------------------	---

Returns

1 if call-back was unregistered successfully

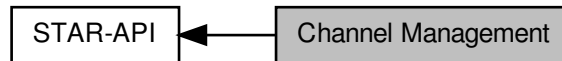
Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

7.4 Channel Management

Used to open and close channels, between applications and devices.

Collaboration diagram for Channel Management:



Typedefs

- typedef `CFG_INFO_ID_TYPE STAR_CHANNEL_ID`
Unique value used to identify an open channel.
- typedef `CFG_INFO_ID_TYPE STAR_CHANNEL_LISTENER_ID`
Unique value used to identify an individual channel listener.
- typedef `void(STAR_API_CC * STAR_ChannelEventFunc) (STAR_CHANNEL_LISTENER_ID channelIdentifier, STAR_DRIVER_ID driverIdentifier, STAR_DEVICE_ID deviceIdentifier, STAR_CHANNEL_ID channelIdentifier, int channelOpened, U8 channelNumber, void *pContextInfo)`
The function type for channel listener functions which are called when a device channel is opened or closed.

Enumerations

- enum `STAR_CHANNEL_TYPE { STAR_CHANNEL_TYPE_NOT_OPEN, STAR_CHANNEL_TYPE_DEVICE, STAR_CHANNEL_TYPE_APPLICATION }`
Channel types.
- enum `STAR_CHANNEL_DIRECTION { STAR_CHANNEL_DIRECTION_IN = 1, STAR_CHANNEL_DIRECTION_OUT = 2, STAR_CHANNEL_DIRECTION_INOUT }`
Channel directions.

Functions

- `_Check_return_ STAR_CHANNEL_ID STAR_API_CC STAR_openChannelBetweenLocalDevices (STAR_DEVICE_ID deviceA, unsigned char channelA, STAR_DEVICE_ID deviceB, unsigned char channelB)`
Open a channel between two devices on the specified channel numbers.
- `_Check_return_ STAR_CHANNEL_ID STAR_API_CC STAR_openChannelToLocalDevice (STAR_DEVICE_ID device, STAR_CHANNEL_DIRECTION direction, unsigned char channelNumber, int isQueued)`
Open a channel to a device on the specified channel number.
- `_Check_return_ STAR_CHANNEL_ID STAR_API_CC STAR_openChannelToLocalDeviceEx (_In_ STAR_DEVICE_ID device, _In_ STAR_CHANNEL_DIRECTION direction, _In_ unsigned char channelNumber, _In_ int isQueued)`
This function is an alternative to STAR_openChannelToLocalDevice that uses an optimised receive strategy.
- `int STAR_API_CC STAR_closeChannel (STAR_CHANNEL_ID channelId)`
Close a previously opened channel.
- `_Check_return_ STAR_CHANNEL_DIRECTION STAR_API_CC STAR_getChannelDirection (STAR_CHANNEL_ID channelId)`
Get the direction information of an open channel.

- `_Check_return_ STAR_CHANNEL_LISTENER_ID STAR_API_CC STAR_registerChannelListener (_In_↔ STAR_ChannelEventFunc listenerFunc, _In_opt_ void *pContextInfo, int notifyForCurrent)`
Registers a Call-back function that will be called whenever any channel is opened or closed.
- `_Check_return_ STAR_CHANNEL_LISTENER_ID STAR_API_CC STAR_registerChannelListenerForDriver (STAR_DRIVER_ID driverId, _In_ STAR_ChannelEventFunc listenerFunc, _In_opt_ void *pContextInfo, int notifyForCurrent)`
Registers a Call-back function that will be called whenever a channel is opened or closed on a device which has the specific driver.
- `_Check_return_ STAR_CHANNEL_LISTENER_ID STAR_API_CC STAR_registerChannelListenerFor↔ Device (STAR_DEVICE_ID deviceId, _In_ STAR_ChannelEventFunc listenerFunc, _In_opt_ void *p↔ ContextInfo, int notifyForCurrent)`
Registers a Call-back function that will be called whenever a channel on the specific device is opened or closed.
- `int STAR_API_CC STAR_unregisterChannelListener (STAR_CHANNEL_LISTENER_ID listenerId)`
Unregister a Call-back function that was previously registered to be called whenever a channel is opened or closed.

7.4.1 Detailed Description

Used to open and close channels, between applications and devices.

7.4.2 Typedef Documentation

7.4.2.1 `typedef CFG_INFO_ID_TYPE STAR_CHANNEL_ID`

Unique value used to identify an open channel.

Added in version:

STAR-System v1.1 - 20th December 2011

7.4.2.2 `typedef CFG_INFO_ID_TYPE STAR_CHANNEL_LISTENER_ID`

Unique value used to identify an individual channel listener.

Added in version:

STAR-System v1.1 - 20th December 2011

7.4.2.3 `typedef void(STAR_API_CC * STAR_ChannelEventFunc) (STAR_CHANNEL_LISTENER_ID channelListenerIdentifier, STAR_DRIVER_ID driverIdentifier, STAR_DEVICE_ID deviceIdIdentifier, STAR_CHANNEL_ID channelIdIdentifier, int channelOpened, U8 channelNumber, void *pContextInfo)`

The function type for channel listener functions which are called when a device channel is opened or closed.

Parameters

<i>channel↔ ListenerIdentifier</i>	the channel listener that this event corresponds to
<i>driverIdentifier</i>	the driver of the device on which the channel was opened or closed
<i>deviceIdIdentifier</i>	the device on which the channel was opened or closed

	<i>channelIdentifier</i>	the identifier of the channel which has been opened or closed
	<i>channelOpened</i>	whether the channel was opened (1) or closed (0)
	<i>channelNumber</i>	the channel number on the device of the channel which was opened or closed
in	<i>pContextInfo</i>	a pointer to context information that can be passed to the function

Note

The calling convention for this function type is [STAR_API_CC](#)

Added in version:

[STAR-System v1.1 - 20th December 2011](#)

7.4.3 Enumeration Type Documentation**7.4.3.1 enum STAR_CHANNEL_DIRECTION**

Channel directions.

Added in version:

[STAR-System v1.1 - 20th December 2011](#)

Enumerator

STAR_CHANNEL_DIRECTION_IN Channel may be used to receive traffic.

STAR_CHANNEL_DIRECTION_OUT Channel may be used to transmit traffic.

STAR_CHANNEL_DIRECTION_INOUT Channel may be used to both receive and transmit traffic.

7.4.3.2 enum STAR_CHANNEL_TYPE

Channel types.

Added in version:

[STAR-System v1.1 - 20th December 2011](#)

Enumerator

STAR_CHANNEL_TYPE_NOT_OPEN Not open.

STAR_CHANNEL_TYPE_DEVICE Attached to a device.

STAR_CHANNEL_TYPE_APPLICATION Attached to an application.

7.4.4 Function Documentation**7.4.4.1 int STAR_API_CC STAR_closeChannel (STAR_CHANNEL_ID channelId)**

Close a previously opened channel.

Parameters

	<i>channelId</i>	Channel to close
--	------------------	------------------

Returns

1 if the channel was successfully closed, else 0. If a channel could not be closed, it was not open or has already been closed

Added in version:

STAR-System v1.1 - 20th December 2011

Examples:

advanced_address_example.c, advanced_continuous_receive_example.c, advanced_multiple_packet_example.c, advanced_receive_example.c, advanced_send_and_receive_example.c, advanced_send_example.c, advanced_two_way_example.c, brickMk2_configuration_example.c, channel_callback_example.c, link_events.c, rmap_target_example.c, simple_receive_example.c, simple_send_and_receive_example.c, simple_send_example.c, simple_two_way_example.c, star-test.c, star_performance_tester.c, time-code_example.c, time-code_test.c, timestamp_test.c, transfer_callback_example.c, transfer_operation_list_send_example.c, transfer_operation_list_send_receive_example.c, and transmit_errors_example.c.

7.4.4.2 `_Check_return_STAR_CHANNEL_DIRECTION STAR_API_CC STAR_getChannelDirection ( STAR_CHANNEL_ID channelID )`

Get the direction information of an open channel.

Parameters

<i>channelID</i>	Channel to get the direction information for
------------------	--

Returns

Direction information of the channel, or 0 if an error occurred

Added in version:

STAR-System v3.10 - 23rd February 2018

7.4.4.3 `_Check_return_STAR_CHANNEL_ID STAR_API_CC STAR_openChannelBetweenLocalDevices ( STAR_DEVICE_ID deviceA, unsigned char channelA, STAR_DEVICE_ID deviceB, unsigned char channelB )`

Open a channel between two devices on the specified channel numbers.

Parameters

<i>deviceA</i>	First device to attach to
<i>channelA</i>	Channel number on first device to attach to
<i>deviceB</i>	Second device to attach to
<i>channelB</i>	Channel number on second device to attach to

Returns

Channel identifier of new channel or 0 if an error occurred

Added in version:

STAR-System v1.1 - 20th December 2011

7.4.4.4 `_Check_return_STAR_CHANNEL_ID STAR_API_CC STAR_openChannelToLocalDevice ( STAR_DEVICE_ID device, STAR_CHANNEL_DIRECTION direction, unsigned char channelNumber, int isQueued )`

Open a channel to a device on the specified channel number.

When the channel is no longer required it should be closed by calling `STAR_closeChannel()`. Note that a channel can only be opened once in a particular direction, but it is possible, for example, to open a channel to receive, then open it again to transmit.

The `isQueued` parameter determines whether received traffic is buffered when it is received. As well as buffering in the device, there is also buffering in the device driver, and if an application is open with a channel open to receive data, there is also buffering in STAR-API for this channel. As these buffers become full, the device will stop issuing credit, so the transmitting device can no longer transmit.

The `isQueued` parameter can be thought of as a way to override this behaviour. When it is set to `FALSE`, packets will be received as normal. However, if the STAR-API buffer becomes full, the API will keep reading data from the driver. If another packet is received from the driver before the application reads a packet from the channel, the API will drop a packet from its buffer and read in the new packet.

The benefit of passing `FALSE` for the `isQueued` parameter is that the link shouldn't become blocked, as the device should always be issuing credit. The obvious disadvantage is that packets can be dropped, so this isn't a mode that is often used. In most circumstances it is recommended that `TRUE` is provided for this argument, although note that the argument will be ignored for transmit-only channels.

Parameters

<i>device</i>	the device to attach to
<i>direction</i>	whether this channel is for transmitting data from this application or receiving data into it, or both. Note that there is no performance penalty in opening a channel in both directions when only receiving, for example. It simply stops the channel from being opened for transmitting in another process or thread
<i>channelNumber</i>	the channel number on the device to attach to
<i>isQueued</i>	whether traffic received on this channel should be buffered if there is no receive op waiting to receive it

Returns

Channel identifier of new channel or 0 if an error occurred

Added in version:

STAR-System v1.1 - 20th December 2011

Examples:

`advanced_address_example.c`, `advanced_continuous_receive_example.c`, `advanced_multiple_packet_example.c`, `advanced_send_and_receive_example.c`, `advanced_two_way_example.c`, `brickMk2_configuration_example.c`, `channel_callback_example.c`, `link_events.c`, `rmap_target_example.c`, `simple_send_and_receive_example.c`, `simple_two_way_example.c`, `star-test.c`, `star_performance_tester.c`, `time-code_example.c`, `time-code_test.c`, `timestamp_test.c`, `transfer_callback_example.c`, `transfer_operation_list_send_example.c`, `transfer_operation_list_send_receive_example.c`, `ui.c`, and `utilities.c`.

7.4.4.5 `_Check_return_STAR_CHANNEL_ID STAR_API_CC STAR_openChannelToLocalDeviceEx ( _In_ STAR_DEVICE_ID device, _In_ STAR_CHANNEL_DIRECTION direction, _In_ unsigned char channelNumber, _In_ int isQueued )`

This function is an alternative to `STAR_openChannelToLocalDevice` that uses an optimised receive strategy.

Instead of creating dynamically allocated packets, data is copied directly into pre-allocated buffers.

To utilise the optimised receive strategy, the channel should be used with receive transfer operations created with the `STAR_createRxOperationEx` function.

When the channel is no longer required it should be closed by calling `STAR_closeChannel()`. Note that a channel can only be opened once in a particular direction, but it is possible, for example, to open a channel to receive, then open it again to transmit.

The `isQueued` parameter determines whether received traffic is buffered when it is received. As well as buffering in the device, there is also buffering in the device driver, and if an application is open with a channel open to receive data, there is also buffering in STAR-API for this channel. As these buffers become full, the device will stop issuing credit, so the transmitting device can no longer transmit.

The `isQueued` parameter can be thought of as a way to override this behaviour. When it is set to `FALSE`, data will be received as normal. However, if the STAR-API buffer becomes full, the API will keep reading data from the driver. If more data is received from the driver before the application reads it from the channel, the API will drop the data from its buffer and read in the new data.

The benefit of passing `FALSE` for the `isQueued` parameter is that the link shouldn't become blocked, as the device should always be issuing credit. The obvious disadvantage is that data can be dropped, so this isn't a mode that is often used. In most circumstances it is recommended that `TRUE` is provided for this argument, although note that the argument will be ignored for transmit-only channels.

Parameters

<i>device</i>	the device to attach to
<i>direction</i>	whether this channel is for transmitting data from this application or receiving data into it, or both. Note that there is no performance penalty in opening a channel in both directions when only receiving, for example. It simply stops the channel from being opened for transmitting in another process or thread
<i>channelNumber</i>	the channel number on the device to attach to
<i>isQueued</i>	whether traffic received on this channel should be buffered if there is no receive op waiting to receive it

Returns

Channel identifier of new channel or 0 if an error occurred

Added in version:

STAR-System v4.00 - 19th November 2019

7.4.4.6 `_Check_return_STAR_CHANNEL_LISTENER_ID STAR_API_CC STAR_registerChannelListener ( _In_ STAR_ChannelEventFunc listenerFunc, _In_opt_ void * pContextInfo, int notifyForCurrent )`

Registers a Call-back function that will be called whenever any channel is opened or closed.

Parameters

	<i>listenerFunc</i>	Call-back Function
<i>in</i>	<i>pContextInfo</i>	A pointer to some context information that will be passed to the listener function when it is called
	<i>notifyForCurrent</i>	Whether the function should be called for all existing channels to indicate that they've been opened

Returns

Non-Zero if call-back was registered successfully

Added in version:

STAR-System v1.1 - 20th December 2011

Examples:

`channel_callback_example.c`.

7.4.4.7 `_Check_return_ STAR_CHANNEL_LISTENER_ID STAR_API_CC STAR_registerChannelListenerForDevice ( STAR_DEVICE_ID deviceId, _In_ STAR_ChannelEventFunc listenerFunc, _In_opt_ void * pContextInfo, int notifyForCurrent )`

Registers a Call-back function that will be called whenever a channel on the specific device is opened or closed.

Parameters

	<i>deviceld</i>	Device to register channel listener for
	<i>listenerFunc</i>	Call-back Function
in	<i>pContextInfo</i>	A pointer to some context information that will be passed to the listener function when it is called
	<i>notifyForCurrent</i>	Whether the function should be called for all existing channels to indicate that they've been opened

Returns

Non-Zero if call-back was registered successfully

Added in version:

STAR-System v1.1 - 20th December 2011

7.4.4.8 `_Check_return_ STAR_CHANNEL_LISTENER_ID STAR_API_CC STAR_registerChannelListenerForDriver ( STAR_DRIVER_ID driverId, _In_ STAR_ChannelEventFunc listenerFunc, _In_opt_ void * pContextInfo, int notifyForCurrent )`

Registers a Call-back function that will be called whenever a channel is opened or closed on a device which has the specific driver.

Parameters

	<i>driverId</i>	Driver to register channel listener for
	<i>listenerFunc</i>	Call-back Function
in	<i>pContextInfo</i>	A pointer to some context information that will be passed to the listener function when it is called
	<i>notifyForCurrent</i>	Whether the function should be called for all existing channels to indicate that they've been opened

Returns

Non-Zero if call-back was registered successfully

Added in version:

STAR-System v1.1 - 20th December 2011

7.4.4.9 `int STAR_API_CC STAR_unregisterChannelListener ( STAR_CHANNEL_LISTENER_ID listenerId )`

Unregister a Call-back function that was previously registered to be called whenever a channel is opened or closed.

Parameters

<i>listenerId</i>	Channel Listener ID returned from a previous call to a registerChannelListener function
-------------------	---

Returns

1 if call-back was unregistered successfully, else 0

Added in version:

STAR-System v1.1 - 20th December 2011

Examples:

`channel_callback_example.c.`

7.5 Stream Items

Stream items are abstract representations of packets, time-codes and link control tokens that can be transmitted and received over a SpaceWire network.

Collaboration diagram for Stream Items:



Data Structures

- struct `STAR_LINK_STATE_EVENT`
Events that can occur which affect the state of the SpaceWire link. [More...](#)
- struct `STAR_SPACEWIRE_ADDRESS`
A structure used to describe a SpaceWire address. [More...](#)
- struct `STAR_DATA_CHUNK`
A structure used to describe a chunk of contiguous SpaceWire data, within a single packet. [More...](#)
- struct `STAR_SPACEWIRE_PACKET`
A structure used to describe a SpaceWire packet. [More...](#)
- struct `STAR_TIMECODE`
A structure used to describe a SpaceWire time-code. [More...](#)
- struct `STAR_LINK_SPEED_EVENT`
A structure used to describe a link speed change event. [More...](#)
- struct `STAR_ERROR_IN_DATA_INJECT`
A structure used to describe an error in data event. [More...](#)
- struct `STAR_TIMESTAMP_EVENT`
A structure used to describe a timestamp on receipt of data on the link. [More...](#)
- struct `STAR_BROADCAST_MESSAGE`
A structure used to describe a SpaceFibre broadcast message. [More...](#)
- struct `STAR_STREAM_ITEM`
A wrapper for Stream Item objects. [More...](#)

Enumerations

- enum `STAR_STREAM_ITEM_TYPE` {
`STAR_STREAM_ITEM_TYPE_SPACEWIRE_PACKET`, `STAR_STREAM_ITEM_TYPE_TIMECODE`, `STAR_STREAM_ITEM_TYPE_LINK_STATE_EVENT`, `STAR_STREAM_ITEM_TYPE_DATA_CHUNK`,
`STAR_STREAM_ITEM_TYPE_LINK_SPEED_EVENT`, `STAR_STREAM_ITEM_TYPE_ERROR_INJECT`, `STAR_STREAM_ITEM_TYPE_TIMESTAMP_EVENT`, `STAR_STREAM_ITEM_TYPE_BROADCAST_MESSAGE`
}

The kinds of object that are represented as stream items.
- enum `STAR_EOP_TYPE` { `STAR_EOP_TYPE_INVALID`, `STAR_EOP_TYPE_EOP`, `STAR_EOP_TYPE_EEP`, `STAR_EOP_TYPE_NONE` }
The kinds of End of packet marker that may be present.

- enum `STAR_ERROR_IN_DATA_TYPE` { `STAR_ERROR_IN_DATA_PARITY`, `STAR_ERROR_IN_DATA_DISCONNECT` }

The types of error that may be injected in to data in a packet.

- enum `STAR_TIMESTAMP_TYPE` { `STAR_TIMESTAMP_TYPE_DATA`, `STAR_TIMESTAMP_TYPE_LINK_STATE`, `STAR_TIMESTAMP_TYPE_LINK_SPEED`, `STAR_TIMESTAMP_TYPE_TIMECODE`, `STAR_TIMESTAMP_TYPE_ERROR` }

An enum used to define different types of timestamps that can be received.

- enum `STAR_TIMESTAMP_DIRECTION` { `STAR_TIMESTAMP_DIRECTION_TRANSMIT`, `STAR_TIMESTAMP_DIRECTION_RECEIVE` }

An enum used to define the direction of a timestamp.

- enum `STAR_BROADCAST_MESSAGE_STATUS_FLAG` { `STAR_BROADCAST_MESSAGE_STATUS_FLAG_LATE` = 0x1, `STAR_BROADCAST_MESSAGE_STATUS_FLAG_DELAYED` = 0x2 }

An enum used to define the status flags of a broadcast message.

Functions

- void `STAR_API_CC STAR_destroyStreamItem` (`_In_ _Post_ptr_invalid_ STAR_STREAM_ITEM *item`)
Disposes of a given stream item.
- `_Check_return_ STAR_STREAM_ITEM *STAR_API_CC STAR_createPacket` (`_In_opt_ STAR_SPACEWIRE_ADDRESS *address`, `_In_opt_count_ (dataLen) unsigned char *data`, `unsigned int dataLen`, `STAR_EOP_TYPE eopType`)
Creates a new SpaceWire packet stream item from user provided data.
- `_Check_return_ STAR_SPACEWIRE_ADDRESS *STAR_API_CC STAR_createAddress` (`_In_count_ (pathLen) unsigned char *path`, `U16 pathLen`)
Creates a SpaceWire address.
- void `STAR_API_CC STAR_destroyAddress` (`_In_ _Post_ptr_invalid_ STAR_SPACEWIRE_ADDRESS *address`)
Disposes of a SpaceWire address created with `STAR_createAddress()`.
- int `STAR_API_CC STAR_setPacketAddress` (`_Inout_ STAR_SPACEWIRE_PACKET *packet`, `_In_ STAR_SPACEWIRE_ADDRESS *address`)
Updates the address of a given packet.
- `_Ret_opt_bytecap_x_ dataLength unsigned char *STAR_API_CC STAR_getPacketData` (`_In_ STAR_SPACEWIRE_PACKET *packet`, `_Out_ unsigned int *dataLength`)
Gets the packet's data as an array of bytes.
- void `STAR_API_CC STAR_destroyPacketData` (`_In_ _Post_ptr_invalid_ unsigned char *pData`)
Frees packet data previously created by a call to `STAR_getPacketData()`.
- `STAR_SPACEWIRE_ADDRESS *STAR_API_CC STAR_getPacketAddress` (`_In_ STAR_SPACEWIRE_PACKET *packet`)
Gets a copy of a packet's address structure.
- unsigned int `STAR_API_CC STAR_getPacketLength` (`_In_ STAR_SPACEWIRE_PACKET *packet`)
Gets the length of a given packet.
- `STAR_EOP_TYPE STAR_API_CC STAR_getPacketEOP` (`_In_ STAR_SPACEWIRE_PACKET *packet`)
Gets the end of packet marker type of a packet.
- int `STAR_API_CC STAR_setPacketEOP` (`_In_ STAR_SPACEWIRE_PACKET *packet`, `STAR_EOP_TYPE eopType`)
Sets the end of packet marker type of a packet.
- `_Check_return_ STAR_STREAM_ITEM *STAR_API_CC STAR_createDataChunk` (`_In_opt_count_ (dataLen) unsigned char *data`, `unsigned int dataLen`, `int isStart`, `STAR_EOP_TYPE eopType`)
Creates a data chunk stream item, used to refer to part of a packet.
- unsigned char `*STAR_API_CC STAR_getChunkData` (`_In_ STAR_DATA_CHUNK *chunk`, `_Out_ unsigned int *len`)

- Gets a pointer to a data chunk stream item's data.*

 - unsigned int `STAR_API_CC STAR_getChunkDataLength` (`_In_ STAR_DATA_CHUNK *chunk`)

Gets the length of a data chunk stream item's data.
- `STAR_EOP_TYPE STAR_API_CC STAR_getChunkEop` (`_In_ STAR_DATA_CHUNK *chunk`)

Gets the End Of Packet marker type of a data chunk stream item.
- int `STAR_API_CC STAR_getChunkSop` (`_In_ STAR_DATA_CHUNK *chunk`)

Gets whether a data chunk stream item is at the start of a packet.
- `STAR_STREAM_ITEM *STAR_API_CC STAR_createTimeCode` (U8 value)

Creates a Time-code stream item.
- U8 `STAR_API_CC STAR_getTimeCodeValue` (`_In_ STAR_TIMECODE *pTimeCode`)

Gets the value of a given time-code stream item.
- void `STAR_API_CC STAR_setTimeCodeValue` (`_In_ STAR_TIMECODE *pTimeCode`, U8 value)

Sets the value of a given time-code stream item.
- `STAR_STREAM_ITEM *STAR_API_CC STAR_createErrorInData` (`STAR_ERROR_IN_DATA_TYPE errorType`)

Creates an error in data stream item.
- U8 `STAR_API_CC STAR_getLinkStateEventCount` (`_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent`)

Gets the count of a given link state event stream item, indicating the number of times the specified event has occurred.
- U8 `STAR_API_CC STAR_getLinkStateEventPort` (`_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent`)

Gets the port related to a given link state change event stream item.
- int `STAR_API_CC STAR_isLinkStateEventDisconnectError` (`_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent`)

Gets whether a link state event stream item includes a disconnect error.
- int `STAR_API_CC STAR_isLinkStateEventEscapeError` (`_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent`)

Gets whether a link state event stream item includes an escape error.
- int `STAR_API_CC STAR_isLinkStateEventLinkRunning` (`_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent`)

Gets whether a link state event stream item includes a link running state change.
- int `STAR_API_CC STAR_isLinkStateEventParityError` (`_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent`)

Gets whether a link state event stream item includes a parity error.
- int `STAR_API_CC STAR_isLinkStateEventReceiveCreditError` (`_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent`)

Gets whether a link state event stream item includes a receive credit error.
- int `STAR_API_CC STAR_isLinkStateEventTransmitCreditError` (`_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent`)

Gets whether a link state event stream item includes a transmit credit error.
- U32 `STAR_API_CC STAR_getLinkSpeedEventSpeed` (`_In_ STAR_LINK_SPEED_EVENT *pLinkSpeedEvent`)

Gets the link speed of a given link speed change event stream item.
- U8 `STAR_API_CC STAR_getLinkSpeedEventPort` (`_In_ STAR_LINK_SPEED_EVENT *pLinkSpeedEvent`)

Gets the port related to a given link speed change event stream item.
- void `STAR_API_CC STAR_getTimestampEventStartValue` (`_In_ STAR_TIMESTAMP_EVENT *pTimestampEvent`, U32 syncPulseFrequency, `_Out_ U32 *pSeconds`, `_Out_ U32 *pRemainder`)

Gets the start time value from a timestamp event stream item.
- void `STAR_API_CC STAR_getTimestampEventEndValue` (`_In_ STAR_TIMESTAMP_EVENT *pTimestampEvent`, U32 syncPulseFrequency, `_Out_ U32 *pSeconds`, `_Out_ U32 *pRemainder`)

Gets the end time value from a timestamp event stream item.
- `STAR_TIMESTAMP_TYPE STAR_API_CC STAR_getTimestampEventType` (`_In_ STAR_TIMESTAMP_EVENT *pTimestampEvent`)

Gets the timestamp type information from a timestamp event stream item.

- `STAR_TIMESTAMP_DIRECTION STAR_API_CC STAR_getTimestampEventDirection (_In_ STAR_TIMESTAMP_EVENT *pTimestampEvent)`
Gets the timestamp direction information from a timestamp event stream item.
- `void STAR_API_CC STAR_getTimestampEventRawCounters (_In_ STAR_TIMESTAMP_EVENT *pTimestampEvent, _Out_opt_ U32 *pStartSyncPulseCount, _Out_opt_ U32 *pStartClockCycleCount, _Out_opt_ U32 *pStartTotalCycleCount, _Out_opt_ U32 *pEndSyncPulseCount, _Out_opt_ U32 *pEndClockCycleCount, _Out_opt_ U32 *pEndTotalCycleCount, _Out_opt_ U32 *pClockFrequency)`
Gets the timestamp counter values as returned by the hardware, from a timestamp event stream item.
- `STAR_STREAM_ITEM *STAR_API_CC STAR_createBroadcastMessage (_In_ U8 channel, _In_ U8 type, _In_ U8 status, _In_ U32 dataWord1, _In_ U32 dataWord2)`
Creates a broadcast message stream item.
- `U8 STAR_API_CC STAR_getBroadcastMessageChannel (_In_ STAR_BROADCAST_MESSAGE *pBroadcastMessage)`
Gets the broadcast message channel from a broadcast message stream item.
- `U8 STAR_API_CC STAR_getBroadcastMessageType (_In_ STAR_BROADCAST_MESSAGE *pBroadcastMessage)`
Gets the broadcast message type from a broadcast message stream item.
- `U8 STAR_API_CC STAR_getBroadcastMessageStatus (_In_ STAR_BROADCAST_MESSAGE *pBroadcastMessage)`
Gets the broadcast message status from a broadcast message stream item.
- `U8 STAR_API_CC STAR_getBroadcastMessageLateFlag (_In_ STAR_BROADCAST_MESSAGE *pBroadcastMessage)`
Gets the broadcast message late flag from a broadcast message stream item.
- `U8 STAR_API_CC STAR_getBroadcastMessageDelayedFlag (_In_ STAR_BROADCAST_MESSAGE *pBroadcastMessage)`
Gets the broadcast message delayed flag from a broadcast message stream item.
- `U32 STAR_API_CC STAR_getBroadcastMessageDataWord1 (_In_ STAR_BROADCAST_MESSAGE *pBroadcastMessage)`
Gets the broadcast message first data word from a broadcast message stream item.
- `U32 STAR_API_CC STAR_getBroadcastMessageDataWord2 (_In_ STAR_BROADCAST_MESSAGE *pBroadcastMessage)`
Gets the broadcast message second data word from a broadcast message stream item.

7.5.1 Detailed Description

Stream items are abstract representations of packets, time-codes and link control tokens that can be transmitted and received over a SpaceWire network.

The API has been designed such that users only interact with pointers to item types, so they need not have any knowledge of what is actually contained within the various item structure types. Therefore, any objects required must be created and destroyed using the API, and all uses of the object are through the provided item accessor and mutator functions. Data structure members may change in future versions of this API.

7.5.2 Data Structure Documentation

7.5.2.1 struct STAR_LINK_STATE_EVENT

Events that can occur which affect the state of the SpaceWire link.

Note

Link state events are not supported by the PCI Mk2 card. The appropriate link state change event enable function must be called to allow these events to be generated.

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

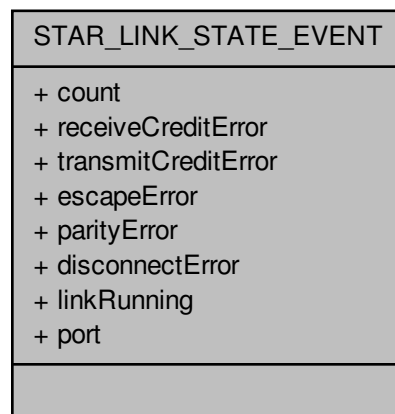
Declaration changed in version:

STAR-System v4.00 - 19th November 2019

Examples:

[link_events.c](#).

Collaboration diagram for STAR_LINK_STATE_EVENT:

**Data Fields**

U8	count	The number of times that the event(s) has occurred.
unsigned int	disconnect↔ Error: 1	Disconnect error.
unsigned int	escapeError: 1	Escape error.
unsigned int	linkRunning: 1	Link running.
unsigned int	parityError: 1	Parity error.
U8	port	The port on which the events occurred.
unsigned int	receiveCredit↔ Error: 1	Receive credit error.
unsigned int	transmitCredit↔ Error: 1	Transmit credit error.

7.5.2.2 struct STAR_SPACEWIRE_ADDRESS

A structure used to describe a SpaceWire address.

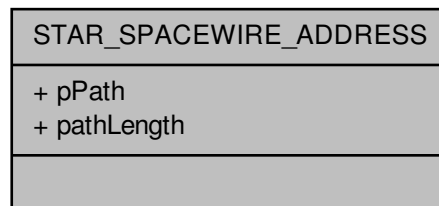
Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

[advanced_address_example.c](#), [link_events.c](#), [star-test.c](#), [star_performance_tester.c](#), [transfer_operation_list_send_receive_example.c](#), and [ui.c](#).

Collaboration diagram for STAR_SPACEWIRE_ADDRESS:



Data Fields

U16	pathLength	The length of the address.
U8 *	pPath	Array of SpaceWire path address elements.

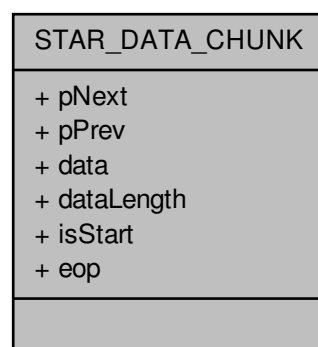
7.5.2.3 struct STAR_DATA_CHUNK

A structure used to describe a chunk of contiguous SpaceWire data, within a single packet.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Collaboration diagram for STAR_DATA_CHUNK:



Data Fields

unsigned char *	data	Pointer to the data buffer itself.
U32	dataLength	Length of data in the buffer.
STAR_EOP_TYPE	eop	The type of end of packet marker this data chunk has, if any.
int	isStart	Whether the chunk of data represents the start of a new packet.
struct star_data_chunk *	pNext	
struct star_data_chunk *	pPrev	

7.5.2.4 struct STAR_SPACEWIRE_PACKET

A structure used to describe a SpaceWire packet.

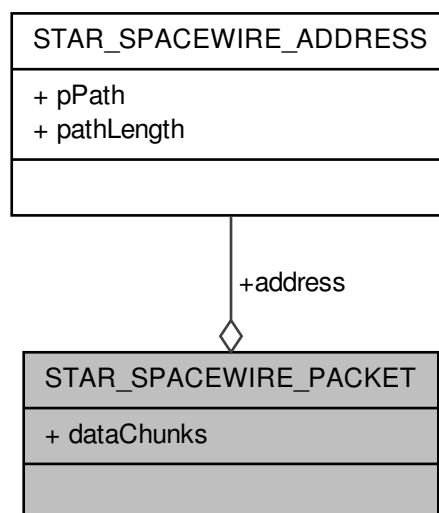
Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

advanced_address_example.c, advanced_continuous_receive_example.c, advanced_multiple_packet_example.c, advanced_receive_example.c, advanced_send_and_receive_example.c, advanced_two_way_example.c, star-test.c, star_performance_tester.c, timestamp_test.c, transfer_callback_example.c, and transfer_operation_list_send_receive_example.c.

Collaboration diagram for STAR_SPACEWIRE_PACKET:



Data Fields

STAR_SPAC↔ EWIRE_ADD↔ RESS *	address	The packets address (optional) Note This member is provided for convenience only. There is no difference between specifying a packet with an address of [0xFE, 0xAB] and data of [0x01, 0x02, 0x03, 0x04], and specifying a packet with no explicit address, and data of [0xFE, 0xAB, 0x01, 0x02, 0x03, 0x04]
void *	dataChunks	The data that makes up the packet.

7.5.2.5 struct STAR_TIMECODE

A structure used to describe a SpaceWire time-code.

Note

Currently some STAR-System compatible devices only support transmitting time-codes using channel 0.

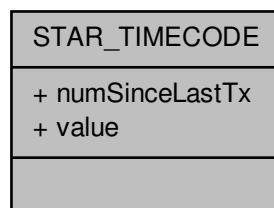
Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

[time-code_test.c](#).

Collaboration diagram for STAR_TIMECODE:



Data Fields

U16	numSinceLastTx	Count of the number of time-codes since the last time-code was transmitted.
U8	value	Value of the time-code.

7.5.2.6 struct STAR_LINK_SPEED_EVENT

A structure used to describe a link speed change event.

Note

Link speed events are not supported by the PCI Mk2 card. The appropriate link speed change event enable function must be called to allow these events to be generated.

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

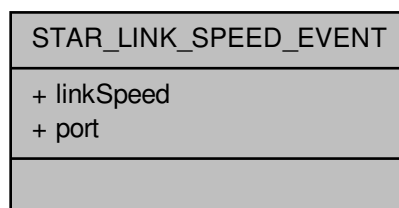
Declaration changed in version:

STAR-System v4.00 - 19th November 2019

Examples:

brickMk2_configuration_example.c, and link_events.c.

Collaboration diagram for STAR_LINK_SPEED_EVENT:

**Data Fields**

U32	linkSpeed	The new link speed which has been adopted, represented in bit/s. Note that this value is approximate.
U8	port	The port on which the event occurred.

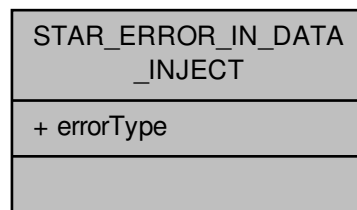
7.5.2.7 struct STAR_ERROR_IN_DATA_INJECT

A structure used to describe an error in data event.

Note

Not all devices support this item type, and that receiving these types is not possible.

Collaboration diagram for STAR_ERROR_IN_DATA_INJECT:

**Data Fields**

STAR_ERROR_IN_DATA_INJECT	errorType	Type of error to inject on a following data character.
---------------------------	-----------	--

7.5.2.8 struct STAR_TIMESTAMP_EVENT

A structure used to describe a timestamp on receipt of data on the link.

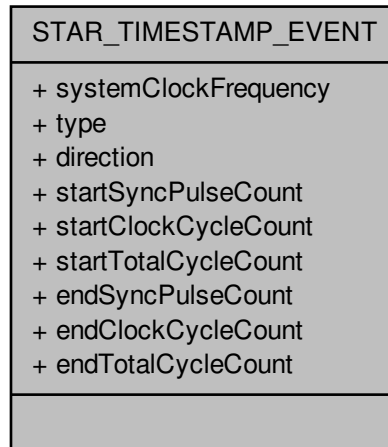
Added in version:

STAR-System v3.6 - 3rd April 2017

Examples:

timestamp_test.c.

Collaboration diagram for STAR_TIMESTAMP_EVENT:



Data Fields

STAR_TIMESTAMP_DIRECTION	direction	Whether timestamp relates to transmitted or received data.
U32	endClockCycleCount	The number of clock cycles counted when the end of the packet was transmitted or received.
U32	endSyncPulseCount	The number of synchronisation pulses when the end of the packet was transmitted or received.
U32	endTotalCycleCount	The number of clock cycles counted when the last synchronisation pulse before the end of the packet was received.
U32	startClockCycleCount	The number of clock cycles counted when the start of the packet was transmitted or received.
U32	startSyncPulseCount	The number of synchronisation pulses when the start of the packet was transmitted or received.
U32	startTotalCycleCount	The number of clock cycles counted when the last synchronisation pulse before the start of the packet was received.
U32	systemClockFrequency	The system clock frequency of the device.
STAR_TIMESTAMP_TYPE	type	The type of data that the timestamp represents.

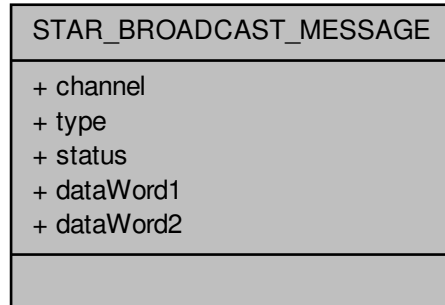
7.5.2.9 struct STAR_BROADCAST_MESSAGE

A structure used to describe a SpaceFibre broadcast message.

Added in version:

STAR-System v4.00 - 19th November 2019

Collaboration diagram for STAR_BROADCAST_MESSAGE:



Data Fields

U8	channel	The broadcast message's channel (BC field)
U32	dataWord1	The broadcast message's first data word.
U32	dataWord2	The broadcast message's second data word.
U8	status	The broadcast message's status (STATUS field).
U8	type	The broadcast message's type (B_TYPE field).

7.5.2.10 struct STAR_STREAM_ITEM

A wrapper for Stream Item objects.

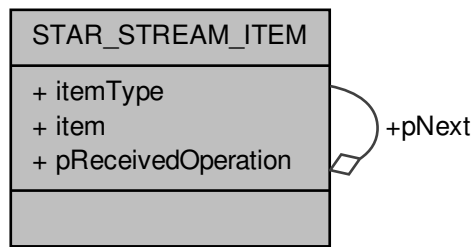
Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

advanced_address_example.c, advanced_continuous_receive_example.c, advanced_multiple_packet_example.c, advanced_receive_example.c, advanced_send_and_receive_example.c, advanced_send_example.c, advanced_two_way_example.c, brickMk2_configuration_example.c, link_events.c, rmap_target_example.c, star-test.c, star_performance_tester.c, time-code_example.c, time-code_test.c, timestamp_test.c, transfer_callback_example.c, transfer_operation_list_send_example.c, transfer_operation_list_send_receive_example.c, and transmit_errors_example.c.

Collaboration diagram for STAR_STREAM_ITEM:



Data Fields

void *	item	A stream item object.
STAR_STREAM_ITEM_TYPE	itemType	The type of stream item pointed to by item.
struct STAR_STREAM_ITEM *	pNext	
void *	pReceivedOperation	

7.5.3 Enumeration Type Documentation

7.5.3.1 enum STAR_BROADCAST_MESSAGE_STATUS_FLAG

An enum used to define the status flags of a broadcast message.

Added in version:

STAR-System v4.00 - 19th November 2019

Enumerator

STAR_BROADCAST_MESSAGE_STATUS_FLAG_LATE The LATE flag of a broadcast message status field.

STAR_BROADCAST_MESSAGE_STATUS_FLAG_DELAYED The DELAYED flag of a broadcast message status field.

7.5.3.2 enum STAR_EOP_TYPE

The kinds of End of packet marker that may be present.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Enumerator

STAR_EOP_TYPE_INVALID Error occurred determining EOP type.

STAR_EOP_TYPE_EOP End of Packet marker.
STAR_EOP_TYPE_EEP Error End of Packet marker.
STAR_EOP_TYPE_NONE No End of Packet marker present.

7.5.3.3 enum STAR_ERROR_IN_DATA_TYPE

The types of error that may be injected in to data in a packet.

Added in version:

[STAR-System v3.0 Beta 1 - 31st March 2015](#)

Enumerator

STAR_ERROR_IN_DATA_PARITY Parity error.
STAR_ERROR_IN_DATA_DISCONNECT Disconnect error.

7.5.3.4 enum STAR_STREAM_ITEM_TYPE

The kinds of object that are represented as stream items.

Used by [STAR_STREAM_ITEM](#) to show what kind of item it is wrapping.

Added in version:

[STAR-System v0.8 \(Beta 1\) - 22nd August 2011](#)

Enumerator

STAR_STREAM_ITEM_TYPE_SPACEWIRE_PACKET Packet, see [STAR_SPACEWIRE_PACKET](#).
STAR_STREAM_ITEM_TYPE_TIMECODE Time-code, see: [STAR_TIMECODE](#).
STAR_STREAM_ITEM_TYPE_LINK_STATE_EVENT A change in the state of a link, see: [STAR_LINK_STATE_EVENT](#).

Added in version:

[STAR-System v2.0 Beta 1 - 8th February 2013](#)

STAR_STREAM_ITEM_TYPE_DATA_CHUNK Contiguous chunk of data within a single packet, see: [STAR_DATA_CHUNK](#).

STAR_STREAM_ITEM_TYPE_LINK_SPEED_EVENT A change in the speed of a link, see: [STAR_LINK_SPEED_EVENT](#).

Added in version:

[STAR-System v2.0 Beta 1 - 8th February 2013](#)

STAR_STREAM_ITEM_TYPE_ERROR_INJECT An error injection control word, see: [STAR_ERROR_IN_DATA_INJECT](#).

STAR_STREAM_ITEM_TYPE_TIMESTAMP_EVENT Timestamp of the last data received on the link, see: [STAR_TIMESTAMP_EVENT](#).

Added in version:

[STAR-System v3.6 - 3rd April 2017](#)

STAR_STREAM_ITEM_TYPE_BROADCAST_MESSAGE SpaceFibre broadcast message, see: [STAR_BROADCAST_MESSAGE](#).

Added in version:

[STAR-System v4.00 - 19th November 2019](#)

Examples:

[link_events.c](#).

7.5.3.5 enum STAR_TIMESTAMP_DIRECTION

An enum used to define the direction of a timestamp.

Added in version:

STAR-System v3.6 - 3rd April 2017

Enumerator

STAR_TIMESTAMP_DIRECTION_TRANSMIT Timestamp is applied to transmitted data.

STAR_TIMESTAMP_DIRECTION_RECEIVE Timestamp is applied to received data.

7.5.3.6 enum STAR_TIMESTAMP_TYPE

An enum used to define different types of timestamps that can be received.

Added in version:

STAR-System v3.6 - 3rd April 2017

Enumerator

STAR_TIMESTAMP_TYPE_DATA Timestamp for data.

STAR_TIMESTAMP_TYPE_LINK_STATE Timestamp for link state event.

STAR_TIMESTAMP_TYPE_LINK_SPEED Timestamp for link speed event.

STAR_TIMESTAMP_TYPE_TIMECODE Timestamp for time-code.

STAR_TIMESTAMP_TYPE_ERROR Timestamp for error in data.

7.5.4 Function Documentation

7.5.4.1 _Check_return_ STAR_SPACEWIRE_ADDRESS* STAR_API_CC STAR_createAddress (_In_count_(pathLen) unsigned char * path, U16 pathLen)

Creates a SpaceWire address.

The address should be destroyed when it is no longer required using [STAR_destroyAddress\(\)](#).

Parameters

<i>in</i>	<i>path</i>	Address path
-----------	-------------	--------------

Note

The contents of this buffer are copied into data structures managed by the API. It is safe to dispose of this buffer after this function completes.

Parameters

<i>pathLen</i>	Length of the path buffer
----------------	---------------------------

Returns

Pointer to newly created address

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

[advanced_address_example.c](#), [link_events.c](#), [star-test.c](#), [star_performance_tester.c](#), [transfer_operation_list_send_receive_example.c](#), and [ui.c](#).

7.5.4.2 **STAR_STREAM_ITEM*** **STAR_API_CC** **STAR_createBroadcastMessage** (*_In_* U8 *channel*, *_In_* U8 *type*, *_In_* U8 *status*, *_In_* U32 *dataWord1*, *_In_* U32 *dataWord2*)

Creates a broadcast message stream item.

This stream item should be destroyed when it is no longer required by calling [STAR_destroyStreamItem\(\)](#).

Parameters

<i>channel</i>	The broadcast message channel
<i>type</i>	The broadcast message type
<i>status</i>	The broadcast message status
<i>dataWord1</i>	The first data word of the broadcast message
<i>dataWord2</i>	The second data word of the broadcast message

Returns

Pointer to newly created broadcast message, or NULL if an error occurred.

Added in version:

STAR-System v4.00 - 19th November 2019

7.5.4.3 **_Check_return_** **STAR_STREAM_ITEM*** **STAR_API_CC** **STAR_createDataChunk** (*_In_opt_* count (*dataLen*) *unsigned char ** *data*, *unsigned int* *dataLen*, *int* *isStart*, **STAR_EOP_TYPE** *eopType*)

Creates a data chunk stream item, used to refer to part of a packet.

This stream item should be destroyed when it is no longer required by calling [STAR_destroyStreamItem\(\)](#).

Parameters

<i>in</i>	<i>data</i>	Pointer to start of chunk data
-----------	-------------	--------------------------------

Note

The contents of this buffer are copied into data structures managed by the API. It is safe to dispose of this buffer after this function completes.

Parameters

<i>dataLen</i>	The length of the chunk. The maximum size permitted for a data chunk is 0xffff. Attempts to create a chunk larger than this will fail.
<i>isStart</i>	Whether the chunk is the start of a packet (1) or not (0)
<i>eopType</i>	The end of packet marker type for the chunk (can be No EOP)

Returns

Pointer to newly created data chunk, or NULL if an error occurred.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

[transmit_errors_example.c](#).

7.5.4.4 **STAR_STREAM_ITEM*** STAR_API_CC STAR_createErrorInData (**STAR_ERROR_IN_DATA_TYPE** *errorType*)

Creates an error in data stream item.

This stream item can be transmitted to cause an error to occur on the next data character to be transmitted. The stream item should be destroyed when it is no longer required by calling [STAR_destroyStreamItem\(\)](#).

Parameters

<i>errorType</i>	the type of error to inject.
------------------	------------------------------

Returns

Error in data stream item.

Added in version:

STAR-System v3.0 Beta 1 - 31st March 2015

Examples:

[transmit_errors_example.c](#).

7.5.4.5 **_Check_return_ STAR_STREAM_ITEM*** STAR_API_CC STAR_createPacket (**_In_opt_ STAR_SPACEWIRE_ADDRESS** * *address*, **_In_opt_count_**(*dataLen*) unsigned char * *data*, unsigned int *dataLen*, **STAR_EOP_TYPE** *eopType*)

Creates a new SpaceWire packet stream item from user provided data.

This stream item should be destroyed when it is no longer required by calling [STAR_destroyStreamItem\(\)](#).

Parameters

<i>in</i>	<i>address</i>	Optional pointer to address created with STAR_createAddress
-----------	----------------	---

Note

It is safe to dispose of this buffer after this function completes.

Parameters

<i>in</i>	<i>data</i>	Optional pointer to data buffer
-----------	-------------	---------------------------------

Note

The contents of this buffer are copied into data structures managed by the API. It is safe to dispose of this buffer after this function completes.

Parameters

<i>dataLen</i>	Length of the data buffer
<i>eopType</i>	End of packet marker type for the packet (may be none)

Returns

SpaceWire Packet item

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

[advanced_address_example.c](#), [advanced_multiple_packet_example.c](#), [advanced_send_and_receive_example.c](#), [advanced_send_example.c](#), [advanced_two_way_example.c](#), [rmap_target_example.c](#), [star-test.c](#), [star_performance_tester.c](#), [timestamp_test.c](#), [transfer_operation_list_send_example.c](#), and [transfer_operation_list_send_receive_example.c](#).

7.5.4.6 STAR_STREAM_ITEM* STAR_API_CC STAR_createTimeCode (U8 value)

Creates a Time-code stream item.

This stream item should be destroyed when it is no longer required by calling [STAR_destroyStreamItem\(\)](#).

Parameters

<i>value</i>	Time-code value

Returns

Time-code stream item

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

[time-code_example.c](#), and [time-code_test.c](#).

7.5.4.7 void STAR_API_CC STAR_destroyAddress (_In_ _Post_ptr_invalid_ STAR_SPACEWIRE_ADDRESS * address)

Disposes of a SpaceWire address created with [STAR_createAddress\(\)](#).

Parameters

<i>in</i>	<i>address</i>	SpaceWire address to destroy

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

[advanced_address_example.c](#), [link_events.c](#), [star-test.c](#), and [star_performance_tester.c](#).

7.5.4.8 void STAR_API_CC STAR_destroyPacketData (_In_ _Post_ptr_invalid_ unsigned char * pData)

Frees packet data previously created by a call to [STAR_getPacketData\(\)](#).

Parameters

<i>pData</i>	the data to be freed

Added in version:

STAR-System v1.1 - 20th December 2011

Examples:

advanced_address_example.c, advanced_continuous_receive_example.c, advanced_multiple_packet_example.c, advanced_receive_example.c, advanced_send_and_receive_example.c, advanced_two_way_example.c, star-test.c, timestamp_test.c, transfer_callback_example.c, and transfer_operation_list_send_receive_example.c.

7.5.4.9 void STAR_API_CC STAR_destroyStreamItem (_In_ Post_ptr_invalid_ STAR_STREAM_ITEM * item)

Disposes of a given stream item.

All stream items which have been explicitly created, e.g. by calls to STAR_createDataChunk(), STAR_createPacket(), etc. should be destroyed using this function.

Stream items returned by STAR_getTransferItem() can optionally be disposed using this function. If these stream items are not disposed, they will continue to exist until the receive operation is disposed of by calling STAR_disposeTransferOperation(). For infinite receive operations (or receives for large numbers of packets) it is necessary to destroy the received stream items to ensure the PC's memory is not filled.

Parameters

in	item	Pointer to item to destroy
----	------	----------------------------

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

advanced_address_example.c, advanced_send_and_receive_example.c, advanced_send_example.c, advanced_two_way_example.c, rmap_target_example.c, star-test.c, star_performance_tester.c, time-code_example.c, time-code_test.c, timestamp_test.c, transfer_operation_list_send_example.c, transfer_operation_list_send_receive_example.c, and transmit_errors_example.c.

7.5.4.10 U8 STAR_API_CC STAR_getBroadcastMessageChannel (_In_ STAR_BROADCAST_MESSAGE * pBroadcastMessage)

Gets the broadcast message channel from a broadcast message stream item.

Parameters

pBroadcastMessage	the broadcast message
-------------------	-----------------------

Returns

the broadcast message's channel

Added in version:

STAR-System v4.00 - 19th November 2019

7.5.4.11 U32 STAR_API_CC STAR_getBroadcastMessageDataWord1 (_In_ STAR_BROADCAST_MESSAGE * pBroadcastMessage)

Gets the broadcast message first data word from a broadcast message stream item.

Parameters

<i>pBroadcastMessage</i>	the broadcast message
--------------------------	-----------------------

Returns

the broadcast message's first data word

Added in version:

STAR-System v4.00 - 19th November 2019

7.5.4.12 U32 STAR_API_CC STAR_getBroadcastMessageDataWord2 (_In_ STAR_BROADCAST_MESSAGE * pBroadcastMessage)

Gets the broadcast message second data word from a broadcast message stream item.

Parameters

<i>pBroadcastMessage</i>	the broadcast message
--------------------------	-----------------------

Returns

the broadcast message's second data word

Added in version:

STAR-System v4.00 - 19th November 2019

7.5.4.13 U8 STAR_API_CC STAR_getBroadcastMessageDelayedFlag (_In_ STAR_BROADCAST_MESSAGE * pBroadcastMessage)

Gets the broadcast message delayed flag from a broadcast message stream item.

Parameters

<i>pBroadcastMessage</i>	the broadcast message
--------------------------	-----------------------

Returns

the broadcast message's delayed flag

Added in version:

STAR-System v4.00 - 19th November 2019

7.5.4.14 U8 STAR_API_CC STAR_getBroadcastMessageLateFlag (_In_ STAR_BROADCAST_MESSAGE * pBroadcastMessage)

Gets the broadcast message late flag from a broadcast message stream item.

Parameters

<i>pBroadcastMessage</i>	the broadcast message
--------------------------	-----------------------

Returns

the broadcast message's late flag

Added in version:

STAR-System v4.00 - 19th November 2019

7.5.4.15 U8 STAR_API_CC STAR_getBroadcastMessageStatus (_In_ STAR_BROADCAST_MESSAGE * *pBroadcastMessage*)

Gets the broadcast message status from a broadcast message stream item.

Parameters

<i>pBroadcastMessage</i>	the broadcast message
--------------------------	-----------------------

Returns

the broadcast message's status

Added in version:

STAR-System v4.00 - 19th November 2019

7.5.4.16 U8 STAR_API_CC STAR_getBroadcastMessageType (_In_ STAR_BROADCAST_MESSAGE * *pBroadcastMessage*)

Gets the broadcast message type from a broadcast message stream item.

Parameters

<i>pBroadcastMessage</i>	the broadcast message
--------------------------	-----------------------

Returns

the broadcast message's type

Added in version:

STAR-System v4.00 - 19th November 2019

7.5.4.17 unsigned char* STAR_API_CC STAR_getChunkData (_In_ STAR_DATA_CHUNK * *chunk*, _Out_ unsigned int * *len*)

Gets a pointer to a data chunk stream item's data.

Parameters

<i>chunk</i>	Data chunk
<i>len</i>	Pointer to user provided value that will be updated to the length of the returned data

Returns

Pointer to data, or NULL if no data is present. Note that this is a pointer to the chunk's data, not a copy of the data, so the data should not be freed, and will cease to exist if the data chunk is destroyed.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Declaration changed in version:

STAR-System v1.1 - 20th December 2011

7.5.4.18 unsigned int **STAR_API_CC** STAR_getChunkDataLength (_In_ STAR_DATA_CHUNK * *chunk*)

Gets the length of a data chunk stream item's data.

Parameters

<i>chunk</i>	Data chunk
--------------	------------

Returns

Length of data member

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

7.5.4.19 STAR_EOP_TYPE **STAR_API_CC** STAR_getChunkEop (_In_ STAR_DATA_CHUNK * *chunk*)

Gets the End Of Packet marker type of a data chunk stream item.

Parameters

<i>chunk</i>	Data chunk
--------------	------------

Returns

The EOP type of the chunk or STAR_EOP_TYPE_INVALID if an if an invalid chunk was passed in

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

7.5.4.20 int **STAR_API_CC** STAR_getChunkSop (_In_ STAR_DATA_CHUNK * *chunk*)

Gets whether a data chunk stream item is at the start of a packet.

Parameters

<i>chunk</i>	Data chunk
--------------	------------

Returns

Whether the chunk is at the start of a packet (1) or not (0), or -1 if an invalid chunk was passed in

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

7.5.4.21 U8 STAR_API_CC STAR_getLinkSpeedEventPort (_In_ STAR_LINK_SPEED_EVENT * *pLinkSpeedEvent*)

Gets the port related to a given link speed change event stream item.

Parameters

<i>pLinkSpeedEvent</i>	the link speed event to get the port for
------------------------	--

Returns

the port related to the link speed event

Added in version:

STAR-System v4.01 - 29th November 2019

Examples:

link_events.c.

7.5.4.22 U32 STAR_API_CC STAR_getLinkSpeedEventSpeed (_In_ STAR_LINK_SPEED_EVENT * *pLinkSpeedEvent*)

Gets the link speed of a given link speed change event stream item.

Parameters

<i>pLinkSpeedEvent</i>	the link speed event to get the link speed from
------------------------	---

Returns

the link speed from the link speed event

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

Examples:

brickMk2_configuration_example.c, and link_events.c.

7.5.4.23 U8 STAR_API_CC STAR_getLinkStateEventCount (_In_ STAR_LINK_STATE_EVENT * *pLinkStateEvent*)

Gets the count of a given link state event stream item, indicating the number of times the specified event has occurred.

Parameters

<i>pLinkStateEvent</i>	the link state event to get the count for
------------------------	---

Returns

the link state event's count

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

7.5.4.24 U8 STAR_API_CC STAR_getLinkStateEventPort (_In_ STAR_LINK_STATE_EVENT * *pLinkStateEvent*)

Gets the port related to a given link state change event stream item.

Parameters

<i>pLinkStateEvent</i>	the link state event to get the port for
------------------------	--

Returns

the port related to the link state event

Added in version:

STAR-System v4.01 - 29th November 2019

Examples:

[link_events.c](#).

7.5.4.25 STAR_SPACEWIRE_ADDRESS* STAR_API_CC STAR_getPacketAddress (_In_ STAR_SPACEWIRE_PACKET * *packet*)

Gets a copy of a packet's address structure.

Note that received packets will not have an address structure set. This function is only useful for accessing the address member of a packet already created by the user.

Note

As this function creates a copy of the address, this must be freed with a call to [STAR_destroyAddress\(\)](#) when it is no longer in use.

Parameters

<i>in</i>	<i>packet</i>	Packet to get address member from.
-----------	---------------	------------------------------------

Returns

Pointer to copy of packet's address member.

Added in version:

STAR-System v1.12 - 14th November 2012

7.5.4.26 _Ret_opt_bytecap_x_ dataLength unsigned char* STAR_API_CC STAR_getPacketData (_In_ STAR_SPACEWIRE_PACKET * *packet*, _Out_ unsigned int * *dataLength*)

Gets the packet's data as an array of bytes.

This function copies the data from the various chunks that make up a packet into a new array.

Parameters

in	<i>packet</i>	SpaceWire packet
out	<i>dataLength</i>	Pointer to value to be updated to be size of array

Returns

Pointer to data, or NULL if no data is present

Note

As this function creates a new buffer to store the data, this buffer must be freed with a call to [STAR_destroyPacketData\(\)](#) when it is no longer in use.

Added in version:

[STAR-System v0.8 \(Beta 1\) - 22nd August 2011](#)

Examples:

[advanced_address_example.c](#), [advanced_continuous_receive_example.c](#), [advanced_multiple_packet_example.c](#), [advanced_receive_example.c](#), [advanced_send_and_receive_example.c](#), [advanced_two_way_example.c](#), [star-test.c](#), [timestamp_test.c](#), [transfer_callback_example.c](#), and [transfer_operation_list_send_receive_example.c](#).

7.5.4.27 STAR_EOP_TYPE STAR_API_CC STAR_getPacketEOP (_In_ STAR_SPACEWIRE_PACKET * *packet*)

Gets the end of packet marker type of a packet.

Parameters

<i>packet</i>	SpaceWire packet
---------------	------------------

Returns

The EOP type of the packet

Added in version:

[STAR-System v1.1 - 20th December 2011](#)

Examples:

[star_performance_tester.c](#).

7.5.4.28 unsigned int STAR_API_CC STAR_getPacketLength (_In_ STAR_SPACEWIRE_PACKET * *packet*)

Gets the length of a given packet.

Parameters

in	<i>packet</i>	SpaceWire packet
----	---------------	------------------

Returns

The length in bytes of the packet (data + address)

Added in version:

[STAR-System v0.8 \(Beta 1\) - 22nd August 2011](#)

Examples:

[star-test.c](#), and [star_performance_tester.c](#).

7.5.4.29 U8 STAR_API_CC STAR_getTimeCodeValue (_In_ STAR_TIMECODE * pTimeCode)

Gets the value of a given time-code stream item.

Parameters

<i>pTimeCode</i>	Time-code to get value of
------------------	---------------------------

Returns

Time-code value

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

time-code_test.c.

7.5.4.30 STAR_TIMESTAMP_DIRECTION STAR_API_CC STAR_getTimestampEventDirection (_In_ STAR_TIMESTAMP_EVENT * pTimestampEvent)

Gets the timestamp direction information from a timestamp event stream item.

Parameters

<i>pTimestampEvent</i>	the timestamp event to get the direction of
------------------------	---

Returns

the timestamp direction, or -1 if an invalid timestamp was passed in

Added in version:

STAR-System v3.6 - 3rd April 2017

7.5.4.31 void STAR_API_CC STAR_getTimestampEventEndValue (_In_ STAR_TIMESTAMP_EVENT * pTimestampEvent, U32 syncPulseFrequency, _Out_ U32 * pSeconds, _Out_ U32 * pRemainder)

Gets the end time value from a timestamp event stream item.

This value relates to the end of the data being received in whole seconds and the remainder in nanoseconds.

Parameters

	<i>pTimestampEvent</i>	the timestamp event to get timestamp from
	<i>syncPulseFrequency</i>	the frequency of synchronisation pulses in hertz
out	<i>pSeconds</i>	outputs the number of whole seconds of the timestamp
out	<i>pRemainder</i>	outputs the remainder in nanoseconds

Added in version:

STAR-System v3.6 - 3rd April 2017

Examples:

timestamp_test.c.

7.5.4.32 void STAR_API_CC STAR_getTimestampEventRawCounters (_In_ STAR_TIMESTAMP_EVENT * pTimestampEvent, _Out_opt_ U32 * pStartSyncPulseCount, _Out_opt_ U32 * pStartClockCycleCount, _Out_opt_ U32 * pStartTotalCycleCount, _Out_opt_ U32 * pEndSyncPulseCount, _Out_opt_ U32 * pEndClockCycleCount, _Out_opt_ U32 * pEndTotalCycleCount, _Out_opt_ U32 * pClockFrequency)

Gets the timestamp counter values as returned by the hardware, from a timestamp event stream item.

Parameters

	<i>pTimestampEvent</i>	the timestamp event to get timestamp from
out	<i>pStartSyncPulseCount</i>	outputs the start sync pulse counter
out	<i>pStartClockCycleCount</i>	outputs the start clock cycle counter
out	<i>pStartTotalCycleCount</i>	outputs the start clock cycle counter value recorded at the previous sync pulse
out	<i>pEndSyncPulseCount</i>	outputs the end sync pulse counter
out	<i>pEndClockCycleCount</i>	outputs the end sync pulse counter
out	<i>pEndTotalCycleCount</i>	outputs the end clock cycle counter value recorded at the previous sync pulse
out	<i>pClockFrequency</i>	outputs the clock frequency

Added in version:

STAR-System v3.6 - 3rd April 2017

7.5.4.33 void **STAR_API_CC** STAR_getTimestampEventStartValue (_In_ **STAR_TIMESTAMP_EVENT** * *pTimestampEvent*, **U32** *syncPulseFrequency*, _Out_ **U32** * *pSeconds*, _Out_ **U32** * *pRemainder*)

Gets the start time value from a timestamp event stream item.

This value relates to the start of the data being received in whole seconds and the remainder in nanoseconds.

Parameters

	<i>pTimestampEvent</i>	the timestamp event to get the timestamp from
	<i>syncPulseFrequency</i>	the frequency of synchronisation pulses in hertz
out	<i>pSeconds</i>	outputs the number of whole seconds of the timestamp
out	<i>pRemainder</i>	outputs the remainder in nanoseconds

Added in version:

STAR-System v3.6 - 3rd April 2017

Examples:

timestamp_test.c.

7.5.4.34 **STAR_TIMESTAMP_TYPE** **STAR_API_CC** STAR_getTimestampEventType (_In_ **STAR_TIMESTAMP_EVENT** * *pTimestampEvent*)

Gets the timestamp type information from a timestamp event stream item.

Parameters

<i>pTimestamp</i> ←→	the timestamp event to get the type of
<i>Event</i>	

Returns

the timestamp type, or -1 if an invalid timestamp was passed in

Added in version:

STAR-System v3.6 - 3rd April 2017

7.5.4.35 `int STAR_API_CC STAR_isLinkStateEventDisconnectError ( _In_ STAR_LINK_STATE_EVENT * pLinkStateEvent )`

Gets whether a link state event stream item includes a disconnect error.

Parameters

<i>pLinkStateEvent</i>	the link state event to check for a disconnect error
------------------------	--

Returns

whether the link state event includes a disconnect error

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

7.5.4.36 `int STAR_API_CC STAR_isLinkStateEventEscapeError ( _In_ STAR_LINK_STATE_EVENT * pLinkStateEvent )`

Gets whether a link state event stream item includes an escape error.

Parameters

<i>pLinkStateEvent</i>	the link state event to check for an escape error
------------------------	---

Returns

whether the link state event includes an escape error

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

7.5.4.37 `int STAR_API_CC STAR_isLinkStateEventLinkRunning ( _In_ STAR_LINK_STATE_EVENT * pLinkStateEvent )`

Gets whether a link state event stream item includes a link running state change.

Parameters

<i>pLinkStateEvent</i>	the link state event to check for a link running state change
------------------------	---

Returns

whether the link state event includes a link running state change

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

Examples:

`link_events.c`.

7.5.4.38 `int STAR_API_CC STAR_isLinkStateEventParityError ( _In_ STAR_LINK_STATE_EVENT * pLinkStateEvent )`

Gets whether a link state event stream item includes a parity error.

Parameters

<i>pLinkStateEvent</i>	the link state event to check for a parity error
------------------------	--

Returns

whether the link state event includes a parity error

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

7.5.4.39 `int STAR_API_CC STAR_isLinkStateEventReceiveCreditError ( _In_ STAR_LINK_STATE_EVENT * pLinkStateEvent )`

Gets whether a link state event stream item includes a receive credit error.

Parameters

<i>pLinkStateEvent</i>	the link state event to check for a receive credit error
------------------------	--

Returns

whether the link state event includes a receive credit error

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

7.5.4.40 `int STAR_API_CC STAR_isLinkStateEventTransmitCreditError ( _In_ STAR_LINK_STATE_EVENT * pLinkStateEvent )`

Gets whether a link state event stream item includes a transmit credit error.

Parameters

<i>pLinkStateEvent</i>	the link state event to check for a transmit credit error
------------------------	---

Returns

whether the link state event includes a transmit credit error

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

7.5.4.41 `int STAR_API_CC STAR_setPacketAddress ( _Inout_ STAR_SPACEWIRE_PACKET * packet, _In_ STAR_SPACEWIRE_ADDRESS * address )`

Updates the address of a given packet.

Parameters

<i>in</i>	<i>packet</i>	SpaceWire packet
<i>in</i>	<i>address</i>	SpaceWire packet

Returns

1 if the update was successful, else 0

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

7.5.4.42 `int STAR_API_CC STAR_setPacketEOP ( _In_ STAR_SPACEWIRE_PACKET * packet, STAR_EOP_TYPE eopType )`

Sets the end of packet marker type of a packet.

Parameters

<i>in</i>	<i>packet</i>	SpaceWire packet
	<i>eopType</i>	End of packet marker type

Returns

1 if EOP was successfully set, else 0

Added in version:

STAR-System v1.1 - 20th December 2011

7.5.4.43 `void STAR_API_CC STAR_setTimeCodeValue ( _In_ STAR_TIMECODE * pTimeCode, U8 value )`

Sets the value of a given time-code stream item.

Parameters

<i>pTimeCode</i>	time-code to set value of
<i>value</i>	new value

Returns

none

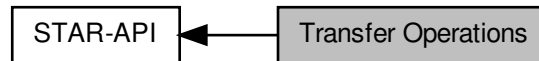
Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

7.6 Transfer Operations

Transfer operations are used to transmit and receive packets and time-codes.

Collaboration diagram for Transfer Operations:



Typedefs

- typedef void **STAR_TRANSFER_OPERATION**
A structure used to identify a transfer operation.
- typedef void(**STAR_API_CC** * **STAR_TransferOperationListenerFunc**) (**STAR_TRANSFER_OPERATION** *pOperation, **STAR_TRANSFER_STATUS** status, void *pContextInfo)
The function type for transfer operation listener functions which are called when a transfer operation completes, is cancelled, or encounters an error.

Enumerations

- enum **STAR_TRANSFER_STATUS** {
STAR_TRANSFER_STATUS_NOT_STARTED, **STAR_TRANSFER_STATUS_STARTED**, **STAR_TRANSFER_STATUS_COMPLETE**, **STAR_TRANSFER_STATUS_CANCELLED**,
STAR_TRANSFER_STATUS_ERROR }
Possible states a transfer operation can be in.
- enum **STAR_RECEIVE_MASK** {
STAR_RECEIVE_PACKETS = 1U << 0, **STAR_RECEIVE_CHUNKS** = 1U << 1, **STAR_RECEIVE_TIMECODES** = 1U << 2, **STAR_RECEIVE_FCTS** = 1U << 3,
STAR_RECEIVE_NULL = 1U << 4, **STAR_RECEIVE_LINK_STATE_EVENTS** = 1U << 5, **STAR_RECEIVE_LINK_SPEED_EVENTS** = 1U << 6, **STAR_RECEIVE_TIMESTAMP_EVENTS** = 1U << 7,
STAR_RECEIVE_BROADCAST_MESSAGES = 1U << 8 }
Flags used to determine what kind of stream items to receive on a receive operation.

Functions

- **STAR_TRANSFER_STATUS** **STAR_API_CC** **STAR_receivePacket** (_In_ **STAR_CHANNEL_ID** channelId, _Out_bytecap_(*pPacketLength) void *pPacketData, _Inout_ unsigned int *pPacketLength, _Out_ **STAR_EOP_TYPE** *pEopType, _In_ int timeout)
Receive a single packet on a previously opened channel in to the buffer specified.
- **STAR_TRANSFER_STATUS** **STAR_API_CC** **STAR_transmitPacket** (_In_ **STAR_CHANNEL_ID** channelId, _In_opt_bytecount_ (packetLength) void *pPacketData, _In_ unsigned int packetLength, _In_ **STAR_EOP_TYPE** eopType, _In_ int timeout)
Transmit a single packet on a previously opened channel from the buffer specified.
- _Check_return_ **STAR_TRANSFER_OPERATION** ***STAR_API_CC** **STAR_createTxOperation** (_In_count_ (streamItemCount) **STAR_STREAM_ITEM** **pplItems, unsigned int streamItemCount)
Creates a transmit operation that can be submitted later.

- `_Check_return_ STAR_TRANSFER_OPERATION *STAR_API_CC STAR_createRxOperation (int item↵
Count, STAR_RECEIVE_MASK mask)`
Creates a receive operation that can then be submitted.
- `_Check_return_ STAR_TRANSFER_OPERATION *STAR_API_CC STAR_createRxOperationEx (_In_↵
count_(numBuffers) U8 **ppBuffers, _In_ unsigned int numBuffers, _In_ unsigned int bufferSize, _In_ ST↵
AR_RECEIVE_MASK mask)`
*This function is an alternative to STAR_createRxOperation that creates receive operations to be submitted to a
channel opened with the STAR_openChannelToLocalDeviceEx function.*
- `int STAR_API_CC STAR_submitTransferOperation (_In_ STAR_CHANNEL_ID channelId, _In_ STAR_TR↵
ANSFER_OPERATION *transferOp)`
Submits a single transfer operation.
- `int STAR_API_CC STAR_submitTransferOperationList (STAR_CHANNEL_ID channelId, _In_count_(count)
STAR_TRANSFER_OPERATION **transferOps, unsigned int count)`
Submits a list of transfer operations.
- `STAR_TRANSFER_STATUS STAR_API_CC STAR_waitOnTransferOperationCompletion (_In_ STAR_T↵
RANSFER_OPERATION *pTransferOperation, int timeout)`
Blocks until a given transfer operation has completed, or the timeout period expires.
- `STAR_TRANSFER_STATUS STAR_API_CC STAR_getTransferOperationStatus (_In_ STAR_TRANSFE↵
R_OPERATION *pTransferOperation)`
Gets the status of a transfer operation.
- `void STAR_API_CC STAR_cancelTransferOperationWaits (_In_ STAR_TRANSFER_OPERATION *p↵
TransferOperation)`
Cancels any waits in progress on a transfer operation.
- `unsigned int STAR_API_CC STAR_getTransferItemCount (_In_ STAR_TRANSFER_OPERATION *p↵
ReceiveOperation)`
*Gets the current count of stream items received by the operation and available to be obtained using STAR_get↵
TransferItem().*
- `STAR_STREAM_ITEM *STAR_API_CC STAR_getTransferItem (_In_ STAR_TRANSFER_OPERATIO↵
N *pReceiveOperation, unsigned int index)`
Gets the stream item at a given index of a receive operation.
- `STAR_STREAM_ITEM *STAR_API_CC STAR_getNextTransferItem (STAR_STREAM_ITEM *pStream↵
Item)`
Gets the next stream item in the list of received stream items.
- `_Ret_opt_cap_count STAR_STREAM_ITEM **STAR_API_CC STAR_getTransferItemList (_In_ STAR_T↵
RANSFER_OPERATION *pReceiveOperation, _Out_ unsigned int *count)`
Gets the entire list of transfer items associated with a receive.
- `void STAR_API_CC STAR_destroyTransferItemList (_In_ _Post_ptr_invalid_ STAR_STREAM_ITE↵
M **transferItemList, unsigned int transferItemListCount, int destroyItems)`
*Destroys a given stream item array, returned by a call to STAR_getTransferItemList(), optionally destroying each of
the elements in the array.*
- `int STAR_API_CC STAR_cancelTransferOperation (_In_ STAR_TRANSFER_OPERATION *pTransfer↵
Operation)`
Cancels a given transfer operation.
- `int STAR_API_CC STAR_disposeTransferOperation (_In_ _Post_ptr_invalid_ STAR_TRANSFER_OPERA↵
TION *pTransferOperation)`
*Cancels a transfer operation (if it is not already complete) and indicates that the memory associated with the operation
should be freed when the operation is no longer in use.*
- `int STAR_API_CC STAR_registerTransferCompletionListener (_In_ STAR_TRANSFER_OPERATION *p↵
TransferOperation, STAR_TransferOperationListenerFunc listenerFunc, _In_opt_ void *pContextInfo)`
Registers a call-back function that will be called when an operation completes.
- `int STAR_API_CC STAR_unregisterTransferCompletionListener (_In_ STAR_TRANSFER_OPERATION
*pTransferOperation, STAR_TransferOperationListenerFunc listenerFunc)`
Unregisters a call-back function that will be called when an operation completes.

- `U8 *STAR_API_CC STAR_getSpaceWireAddressPath (STAR_SPACEWIRE_ADDRESS *pSpaceWireAddress)`
Access function to get a SpaceWire address path.
- `U16 STAR_API_CC STAR_getSpaceWireAddressPathLength (STAR_SPACEWIRE_ADDRESS *pSpaceWireAddress)`
Access function to get a SpaceWire address path length.
- `int STAR_API_CC STAR_rxOperationExGetNumItems (_In_ STAR_TRANSFER_OPERATION *pTransferOp)`
Gets the number of items received by an ex receive operation.
- `STAR_STREAM_ITEM_TYPE STAR_API_CC STAR_rxOperationExGetItemType (_In_ STAR_TRANSFER_OPERATION *pTransferOp, _In_ unsigned int item)`
Gets the stream item type for an item received by an ex receive operation.
- `int STAR_API_CC STAR_rxOperationExGetDataChunk (_In_ STAR_TRANSFER_OPERATION *pTransferOp, _In_ unsigned int item, _Out_ STAR_DATA_CHUNK *pDataChunk)`
Gets a data chunk from an item received by an ex receive operation.
- `int STAR_API_CC STAR_rxOperationExGetBroadcastMessage (_In_ STAR_TRANSFER_OPERATION *pTransferOp, _In_ unsigned int item, _Out_ STAR_BROADCAST_MESSAGE *pBroadcastMessage)`
Gets a broadcast message from an item received by an ex receive operation.
- `STAR_TRANSFER_STATUS STAR_API_CC STAR_rxOperationExGetStatus (_In_ STAR_TRANSFER_OPERATION *pTransferOp)`
Gets the status of an ex receive operation.

7.6.1 Detailed Description

Transfer operations are used to transmit and receive packets and time-codes.

This group contains functions and data structures used to create, submit, and destroy these operations.

7.6.2 Typedef Documentation

7.6.2.1 typedef void STAR_TRANSFER_OPERATION

A structure used to identify a transfer operation.

Note

It is not intended for users of this library to access the members of this structure directly. Instead, use the provided accessor functions.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

7.6.2.2 typedef void(STAR_API_CC * STAR_TransferOperationListenerFunc) (STAR_TRANSFER_OPERATION *pOperation, STAR_TRANSFER_STATUS status, void *pContextInfo)

The function type for transfer operation listener functions which are called when a transfer operation completes, is cancelled, or encounters an error.

Parameters

<i>pOperation</i>	the operation which has completed
<i>status</i>	the status of the operation
<i>pContextInfo</i>	a pointer to user-specific data that is provided when the completion listener is registered

Note

The calling convention for this function type is [STAR_API_CC](#)

Added in version:

[STAR-System v0.8 \(Beta 1\) - 22nd August 2011](#)

7.6.3 Enumeration Type Documentation**7.6.3.1 enum STAR_RECEIVE_MASK**

Flags used to determine what kind of stream items to receive on a receive operation.

Added in version:

[STAR-System v0.8 \(Beta 1\) - 22nd August 2011](#)

Enumerator

[STAR_RECEIVE_PACKETS](#) [STAR_SPACEWIRE_PACKET](#)

[STAR_RECEIVE_CHUNKS](#) [STAR_DATA_CHUNK](#)

[STAR_RECEIVE_TIMECODES](#) [STAR_TIMECODE](#)

[STAR_RECEIVE_FCTS](#) Flow control characters. Note that transmitting and receiving FCTs is not currently supported.

[STAR_RECEIVE_NULL](#) Null characters. Note that transmitting and receiving Nulls is not currently supported.

[STAR_RECEIVE_LINK_STATE_EVENTS](#) [STAR_LINK_STATE_EVENT](#)

Added in version:

[STAR-System v2.0 Beta 1 - 8th February 2013](#)

[STAR_RECEIVE_LINK_SPEED_EVENTS](#) [STAR_LINK_SPEED_EVENT](#)

Added in version:

[STAR-System v2.0 Beta 1 - 8th February 2013](#)

[STAR_RECEIVE_TIMESTAMP_EVENTS](#) [STAR_TIMESTAMP_EVENT](#)

Added in version:

[STAR-System v3.6 - 3rd April 2017](#)

[STAR_RECEIVE_BROADCAST_MESSAGES](#) [STAR_BROADCAST_MESSAGE](#)

Added in version:

[STAR-System v4.00 - 19th November 2019](#)

7.6.3.2 enum STAR_TRANSFER_STATUS

Possible states a transfer operation can be in.

Added in version:

[STAR-System v0.8 \(Beta 1\) - 22nd August 2011](#)

Enumerator

STAR_TRANSFER_STATUS_NOT_STARTED Not yet started. When a transfer operation is created, this will be its status.

STAR_TRANSFER_STATUS_STARTED Transfer has begun. When a transfer operation is submitted, this will be its status, until it has completed.

STAR_TRANSFER_STATUS_COMPLETE Transfer has completed. Once a transmit operation has successfully transmitted all its traffic, or a receive operation has received all its requested traffic, this will be its status.

STAR_TRANSFER_STATUS_CANCELLED Transfer was cancelled. When a transfer is cancelled by calling [STAR_cancelTransferOperation\(\)](#), this will be its status.

STAR_TRANSFER_STATUS_ERROR An error occurred while processing the transfer. This will be the status of a transfer operation if there was an error creating it, submitting it, or there was an error while transmitting or receiving.

7.6.4 Function Documentation

7.6.4.1 `int STAR_API_CC STAR_cancelTransferOperation ( _In_ STAR_TRANSFER_OPERATION * pTransferOperation )`

Cancels a given transfer operation.

Note

It is not currently possible to cancel a transmit operation.

Parameters

<i>pTransferOperation</i>	Operation to cancel
---------------------------	---------------------

Returns

1 if a receive was cancelled, else 0

Added in version:

STAR-System v1.1 - 20th December 2011

Examples:

[brickMk2_configuration_example.c](#), and [link_events.c](#).

7.6.4.2 `void STAR_API_CC STAR_cancelTransferOperationWaits ( _In_ STAR_TRANSFER_OPERATION * pTransferOperation )`

Cancels any waits in progress on a transfer operation.

Note that this does not cancel the transfer operation it simply cancels any calls to [STAR_waitOnTransferOperationCompletion\(\)](#).

Parameters

<i>pTransferOperation</i>	Operation to cancel waits on
---------------------------	------------------------------

Added in version:

STAR-System v1.1 - 20th December 2011

7.6.4.3 `_Check_return_ STAR_TRANSFER_OPERATION* STAR_API_CC STAR_createRxOperation ( int itemCount, STAR_RECEIVE_MASK mask )`

Creates a receive operation that can then be submitted.

The receive operation returned should be disposed of by calling `STAR_disposeTransferOperation()` when it is no longer required.

Parameters

<i>itemCount</i>	The maximum number of stream items to receive (the size of an individual stream item is not limited). This can be -1 to receive an unlimited number of items
------------------	--

Note

When a receive operation receiving an unlimited (or very large) number of items, the received stream items must be destroyed to ensure the received stream items do not consume all the PC's memory.

Parameters

<i>mask</i>	Bitmask with flags set for the type of traffic one wishes to receive
-------------	--

Note

It is not possible to receive whole packets at the same time as link control tokens/data chunks

Returns

a pointer to a `STAR_TRANSFER_OPERATION` object

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Declaration changed in version:

STAR-System v1.1 - 20th December 2011

Examples:

`advanced_address_example.c`, `advanced_continuous_receive_example.c`, `advanced_multiple_packet_example.c`, `advanced_receive_example.c`, `advanced_send_and_receive_example.c`, `advanced_two_way_example.c`, `brickMk2_configuration_example.c`, `link_events.c`, `star-test.c`, `star_performance_tester.c`, `time-code_test.c`, `timestamp_test.c`, `transfer_callback_example.c`, and `transfer_operation_list_send_receive_example.c`.

7.6.4.4 `_Check_return_ STAR_TRANSFER_OPERATION* STAR_API_CC STAR_createRxOperationEx ( _In_count (numBuffers) U8 ** ppBuffers, _In_ unsigned int numBuffers, _In_ unsigned int bufferSize, _In_ STAR_RECEIVE_MASK mask )`

This function is an alternative to `STAR_createRxOperation` that creates receive operations to be submitted to a channel opened with the `STAR_openChannelToLocalDeviceEx` function.

Using the alternative receive operations, data is copied directly into pre-allocate buffers instead of dynamically allocated packets.

Note that receive operations created with `STAR_createRxOperation` should not be submitted to a channel opened using `STAR_openChannelToLocalDeviceEx`. Similarly, receive operations created with `STAR_createRxOperationEx` should not be submitted to a channel opened using `STAR_openChannelToLocalDevice`.

The receive operation returned should be disposed of by calling `STAR_disposeTransferOperation()` when it is no longer required.

Parameters

<i>ppBuffers</i>	An array of pointers to pre-allocated buffers that received stream items will be written to.
<i>numBuffers</i>	The number of buffers in the <i>ppBuffers</i> array.
<i>bufferSize</i>	The size of the buffers in the <i>ppBuffers</i> array.

Note

the size of the buffers should be large enough to receive the largest chunk generated by the hardware, which is generally 4096 or 8192 bytes.

Parameters

<i>mask</i>	Bitmask with flags set for the type of traffic one wishes to receive.
-------------	---

Note

Due to the use of fixed-size pre-allocated buffers, `STAR_RECEIVE_CHUNK` should be used to receive data instead of `STAR_RECEIVE_PACKET`.

Returns

a pointer to a `STAR_TRANSFER_OPERATION`

Added in version:

STAR-System v4.00 - 19th November 2019

7.6.4.5 `_Check_return_ STAR_TRANSFER_OPERATION* STAR_API_CC STAR_createTxOperation (` `_In_count_ (streamItemCount) STAR_STREAM_ITEM ** ppltems, unsigned int streamItemCount )`

Creates a transmit operation that can be submitted later.

The transmit operation returned should be disposed of by calling `STAR_disposeTransferOperation()` when it is no longer required.

Parameters

<i>ppltems</i>	Array of pointers to Stream Items to be sent
----------------	--

Note

It is safe to free this array after the function has completed.

Parameters

<i>streamItem</i> ↔ <i>Count</i>	Count of the items in the <i>ppltems</i> array
-------------------------------------	--

Returns

Pointer to a `STAR_TRANSFER_OPERATION`

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

`advanced_address_example.c`, `advanced_multiple_packet_example.c`, `advanced_send_and_receive_↔
example.c`, `advanced_send_example.c`, `advanced_two_way_example.c`, `rmap_target_example.c`, `star-test.c`,
`star_performance_tester.c`, `time-code_example.c`, `time-code_test.c`, `timestamp_test.c`, `transfer_operation_↔
list_send_example.c`, `transfer_operation_list_send_receive_example.c`, and `transmit_errors_example.c`.

7.6.4.6 void **STAR_API_CC** STAR_destroyTransferItemList (_In_ _Post_ptr_invalid_ STAR_STREAM_ITEM **
transferItemList, unsigned int transferItemListCount, int destroyItems)

Destroys a given stream item array, returned by a call to [STAR_getTransferItemList\(\)](#), optionally destroying each of the elements in the array.

If the stream items are not destroyed they will continue to exist until destroyed by calling [STAR_destroyStreamItem\(\)](#) or by disposing of the receive operation by calling [STAR_disposeTransferOperation\(\)](#). For infinite receive operations (or receives for large numbers of packets) it is necessary to destroy the received stream items to ensure the PC's memory is not filled.

Parameters

in	transferItemList	The array of stream items to destroy
	transferItemListCount	The number of elements in the array of stream items
	destroyItems	If non-zero, each stream item will also be destroyed

Added in version:

STAR-System v3.0 - 26th August 2016

Examples:

star_performance_tester.c.

7.6.4.7 int **STAR_API_CC** STAR_disposeTransferOperation (_In_ _Post_ptr_invalid_ STAR_TRANSFER_OPERATION
* pTransferOperation)

Cancels a transfer operation (if it is not already complete) and indicates that the memory associated with the operation should be freed when the operation is no longer in use.

Note that this is a dispose function rather than a destroy function, as it only indicates to the API that the transfer operation should be destroyed when it is possible to do so. If the API is in the process of transmitting or receiving traffic on the operation it may not be possible to destroy the operation immediately.

Parameters

	pTransferOperation	a pointer to the operation to be disposed of
--	--------------------	--

Returns

1 if operation was disposed of successfully, otherwise 0

Added in version:

STAR-System v1.1 - 20th December 2011

Examples:

advanced_address_example.c, advanced_continuous_receive_example.c, advanced_multiple_packet_example.c, advanced_receive_example.c, advanced_send_and_receive_example.c, advanced_send_example.c, advanced_two_way_example.c, brickMk2_configuration_example.c, link_events.c, star-test.c, star_performance_tester.c, time-code_example.c, time-code_test.c, timestamp_test.c, transfer_callback_example.c, transfer_operation_list_send_example.c, transfer_operation_list_send_receive_example.c, and transmit_errors_example.c.

7.6.4.8 `STAR_STREAM_ITEM* STAR_API_CC STAR_getNextTransferItem ( STAR_STREAM_ITEM * pStreamItem )`

Gets the next stream item in the list of received stream items.

This is quicker than using `STAR_getTransferItem()`.

Parameters

<i>pStreamItem</i>	Current stream item
--------------------	---------------------

Returns

Next stream item

Added in version:

STAR-System v3.10 - 23rd February 2018

Examples:

star-test.c.

7.6.4.9 U8* STAR_API_CC STAR_getSpaceWireAddressPath (STAR_SPACEWIRE_ADDRESS * pSpaceWireAddress)

Access function to get a SpaceWire address path.

Parameters

<i>pSpaceWire↔ Address</i>	A pointer to a STAR_SPACEWIRE_ADDRESS struct.
--------------------------------	--

Returns

A pointer to a SpaceWire address path.

Added in version:

STAR-System v4.00 Beta 1 - 25th September 2018

7.6.4.10 U16 STAR_API_CC STAR_getSpaceWireAddressPathLength (STAR_SPACEWIRE_ADDRESS * pSpaceWireAddress)

Access function to get a SpaceWire address path length.

Parameters

<i>pSpaceWire↔ Address</i>	A pointer to a STAR_SPACEWIRE_ADDRESS struct.
--------------------------------	--

Returns

A SpaceWire address path length.

Added in version:

STAR-System v4.00 Beta 1 - 25th September 2018

Examples:

star_performance_tester.c.

7.6.4.11 STAR_STREAM_ITEM* STAR_API_CC STAR_getTransferItem (_In_ STAR_TRANSFER_OPERATION * pReceiveOperation, unsigned int index)

Gets the stream item at a given index of a receive operation.

Parameters

<i>pReceive↔ Operation</i>	Operation to get item from
<i>index</i>	Item within operation to access

Returns

Item requested

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

[advanced_address_example.c](#), [advanced_continuous_receive_example.c](#), [advanced_multiple_packet_example.c](#), [advanced_receive_example.c](#), [advanced_send_and_receive_example.c](#), [advanced_two_way_example.c](#), [brickMk2_configuration_example.c](#), [link_events.c](#), [star-test.c](#), [time-code_test.c](#), [timestamp_test.c](#), [transfer_callback_example.c](#), and [transfer_operation_list_send_receive_example.c](#).

7.6.4.12 unsigned int **STAR_API_CC** STAR_getTransferItemCount (_In_ STAR_TRANSFER_OPERATION * *pReceiveOperation*)

Gets the current count of stream items received by the operation and available to be obtained using [STAR_getTransferItem\(\)](#).

Note that this may be less than the number of stream items received if some of those stream items have been destroyed by calling [STAR_destroyStreamItem\(\)](#) or [STAR_destroyTransferItemList\(\)](#) with the destroyItems parameter set to a non-zero value.

Parameters

<i>pReceive↔ Operation</i>	Operation to get received item count for
--------------------------------	--

Returns

Count of received stream items

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

[advanced_multiple_packet_example.c](#), [link_events.c](#), [star-test.c](#), and [timestamp_test.c](#).

7.6.4.13 _Ret_opt_cap_count STAR_STREAM_ITEM** **STAR_API_CC** STAR_getTransferItemList (_In_ STAR_TRANSFER_OPERATION * *pReceiveOperation*, _Out_ unsigned int * *count*)

Gets the entire list of transfer items associated with a receive.

The array returned must be destroyed by calling [STAR_destroyTransferItemList\(\)](#).

Parameters

in	<i>pReceiveOperation</i>	Operation to get items from
out	<i>count</i>	Count of items in returned array

Returns

Array of pointers stream item associated with this receive

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

star_performance_tester.c.

7.6.4.14 **STAR_TRANSFER_STATUS STAR_API_CC STAR_getTransferOperationStatus (_In_ STAR_TRANSFER_OPERATION * *pTransferOperation*)**

Gets the status of a transfer operation.

Parameters

<i>pTransferOperation</i>	Operation to get status of
---------------------------	----------------------------

Returns

result status of the operation

Added in version:

STAR-System v1.1 - 20th December 2011

Examples:

brickMk2_configuration_example.c, and star_performance_tester.c.

7.6.4.15 **STAR_TRANSFER_STATUS STAR_API_CC STAR_receivePacket (_In_ STAR_CHANNEL_ID channelId, _Out_bytecap_ *pPacketLength void * *pPacketData*, _Inout_ unsigned int * *pPacketLength*, _Out_ STAR_EOP_TYPE * *pEopType*, _In_ int *timeout*)**

Receive a single packet on a previously opened channel in to the buffer specified.

This function is provided to simplify the process of receiving packets, but is not as efficient or as flexible as using [STAR_createRxOperation\(\)](#), [STAR_submitTransferOperation\(\)](#), etc. This function should not be used if high performance or low CPU utilisation is required. Note that if the received packet is longer than the buffer provided, the end of the packet will be dropped and the end of packet marker will indicate there was no end of packet marker. Note also that a limitation of this function is that the operation may timeout, but a packet is consumed and will not be received in a subsequent call to the function. To work around these issues use the alternative functions mentioned above.

Parameters

<i>channelId</i>	The identifier of a previously opened channel. The channel must have been opened as an in or in/out channel
<i>pPacketData</i>	Pointer to a previously allocated buffer which will be updated to contain the received packet
<i>pPacketLength</i>	Pointer to a variable which should contain the length of the buffer, and which will be updated to contain the length of the packet
<i>pEopType</i>	Pointer to a variable which will be updated to contain the end of packet marker type of the packet
<i>timeout</i>	The maximum time in milliseconds to wait for the packet to be received, or -1 to wait indefinitely

Returns

the status of the operation

Added in version:

STAR-System v1.1 - 20th December 2011

Examples:

[simple_receive_example.c](#), [simple_send_and_receive_example.c](#), and [simple_two_way_example.c](#).

7.6.4.16 `int STAR_API_CC STAR_registerTransferCompletionListener ( _In_ STAR_TRANSFER_OPERATION * pTransferOperation, STAR_TransferOperationListenerFunc listenerFunc, _In_opt_ void * pContextInfo )`

Registers a call-back function that will be called when an operation completes.

Note

Each callback is made using a separate thread, to ensure that STAR-System internal processing is not blocked by long-running callbacks. Due to operating system scheduling it is not possible to guarantee that callbacks will execute in the same order as they are triggered.

Parameters

<i>pTransfer↔ Operation</i>	Operation to add listener for
<i>listenerFunc</i>	Call-back function
<i>pContextInfo</i>	A pointer to memory which will be provided when the call-back function is called

Returns

Non-Zero if call-back was registered successfully

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

[transfer_callback_example.c](#).

7.6.4.17 `int STAR_API_CC STAR_rxOperationExGetBroadcastMessage ( _In_ STAR_TRANSFER_OPERATION * pTransferOp, _In_ unsigned int item, _Out_ STAR_BROADCAST_MESSAGE * pBroadcastMessage )`

Gets a broadcast message from an item received by an ex receive operation.

Parameters

<i>pTransferOp</i>	A pointer to a STAR_TRANSFER_OPERATION.
<i>item</i>	The index of the item to get the broadcast message from.
<i>pBroadcastMessage</i>	A pointer to a broadcast message that will be populated with data from the item.

Returns

1 if a broadcast message was successfully populated from the item, or -1 if an error occurred.

Added in version:

STAR-System v4.00 - 19th November 2019

7.6.4.18 `int STAR_API_CC STAR_rxOperationExGetDataChunk ( _In_ STAR_TRANSFER_OPERATION * pTransferOp, _In_ unsigned int item, _Out_ STAR_DATA_CHUNK * pDataChunk )`

Gets a data chunk from an item received by an ex receive operation.

Parameters

<i>pTransferOp</i>	A pointer to a STAR_TRANSFER_OPERATION.
<i>item</i>	The index of the item to get the data chunk from.
<i>pDataChunk</i>	A pointer to a data chunk that will be populated with data from the item.

Returns

1 if a data chunk was successfully populated from the item, or -1 if an error occurred.

Added in version:

STAR-System v4.00 - 19th November 2019

7.6.4.19 `STAR_STREAM_ITEM_TYPE STAR_API_CC STAR_rxOperationExGetItemType ( _In_ STAR_TRANSFER_OPERATION * pTransferOp, _In_ unsigned int item )`

Gets the stream item type for an item received by an ex receive operation.

Parameters

<i>pTransferOp</i>	A pointer to a STAR_TRANSFER_OPERATION.
<i>item</i>	The index of the item to get the stream item type for.

Returns

The stream item type for an item received by an ex receive operation.

Added in version:

STAR-System v4.00 - 19th November 2019

7.6.4.20 `int STAR_API_CC STAR_rxOperationExGetNumItems ( _In_ STAR_TRANSFER_OPERATION * pTransferOp )`

Gets the number of items received by an ex receive operation.

Parameters

<i>pTransferOp</i>	A pointer to a STAR_TRANSFER_OPERATION.
--------------------	---

Returns

The number of items received by an ex receive operation, or -1 if an error occurred.

Added in version:

STAR-System v4.00 - 19th November 2019

7.6.4.21 **STAR_TRANSFER_STATUS STAR_API_CC STAR_rxOperationExGetStatus (_In_ STAR_TRANSFER_OPERATION * pTransferOp)**

Gets the status of an ex receive operation.

Parameters

<i>pTransferOp</i>	A pointer to a STAR_TRANSFER_OPERATION.
--------------------	---

Returns

The status of the ex receive operation.

Added in version:

STAR-System v4.00 - 19th November 2019

7.6.4.22 **int STAR_API_CC STAR_submitTransferOperation (_In_ STAR_CHANNEL_ID channelId, _In_ STAR_TRANSFER_OPERATION * transferOp)**

Submits a single transfer operation.

Once a transfer operation has completed, it can be resubmitted. This can be useful for receiving a number of packets in a loop, or transmitting the same packet repeatedly. Note that if an operation is resubmitted before it has completed, the original operation will be cancelled and then the new one submitted.

Parameters

	<i>channelId</i>	Channel to submit operation on
<i>in</i>	<i>transferOp</i>	STAR_TRANSFER_OPERATION to submit

Returns

1 if operation was successfully submitted, else 0

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Declaration changed in version:

STAR-System v1.1 - 20th December 2011

Examples:

advanced_address_example.c, advanced_continuous_receive_example.c, advanced_multiple_packet_example.c, advanced_receive_example.c, advanced_send_and_receive_example.c, advanced_send_example.c, advanced_two_way_example.c, brickMk2_configuration_example.c, link_events.c, rmap_target_example.c, star-test.c, star_performance_tester.c, time-code_example.c, time-code_test.c, timestamp_test.c, transfer_callback_example.c, and transmit_errors_example.c.

7.6.4.23 `int STAR_API_CC STAR_submitTransferOperationList ( STAR_CHANNEL_ID channelId, _In_count_(count) STAR_TRANSFER_OPERATION ** transferOps, unsigned int count )`

Submits a list of transfer operations.

Once a transfer operation has completed, it can be resubmitted. This can be useful for receiving a number of packets in a loop, or transmitting the same packet repeatedly. Note that if an operation is resubmitted before it has completed, the original operation will be cancelled and then the new one submitted. This means it is not useful to submit a list containing multiple instances of the same operation.

Parameters

	<i>channelId</i>	Channel to submit operations on
<i>in</i>	<i>transferOps</i>	Array of pointers to STAR_TRANSFER_OPERATION to be submitted
	<i>count</i>	Count of operations in array

Returns

1 if all operations were successfully submitted, else 0

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Declaration changed in version:

STAR-System v1.1 - 20th December 2011

Examples:

[transfer_operation_list_send_example.c](#), and [transfer_operation_list_send_receive_example.c](#).

7.6.4.24 `STAR_TRANSFER_STATUS STAR_API_CC STAR_transmitPacket ( _In_ STAR_CHANNEL_ID channelId, _In_opt_bytecount_(packetLength) void * pPacketData, _In_ unsigned int packetLength, _In_ STAR_EOP_TYPE eopType, _In_ int timeout )`

Transmit a single packet on a previously opened channel from the buffer specified.

This function is provided to simplify the process of transmitting packets, but is not as efficient or as flexible as using [STAR_createTxOperation\(\)](#), [STAR_submitTransferOperation\(\)](#), etc. This function should not be used if high performance or low CPU utilisation is required.

Parameters

<i>channelId</i>	The identifier of a previously opened channel. The channel must have been opened as an out or in/out channel
<i>pPacketData</i>	A pointer to a previously allocated buffer which should contain the packet to be transmitted
<i>packetLength</i>	The length of the buffer, and therefore the length of the packet
<i>eopType</i>	The end of packet marker type to be added to the end of the packet
<i>timeout</i>	The maximum time in milliseconds to wait for the packet to be transmitted, or -1 to wait indefinitely

Returns

the status of the operation

Added in version:

STAR-System v1.1 - 20th December 2011

Examples:

[simple_send_and_receive_example.c](#), [simple_send_example.c](#), and [simple_two_way_example.c](#).

7.6.4.25 `int STAR_API_CC STAR_unregisterTransferCompletionListener ( _In_ STAR_TRANSFER_OPERATION * pTransferOperation, STAR_TransferOperationListenerFunc listenerFunc )`

Unregisters a call-back function that will be called when an operation completes.

Parameters

<i>pTransfer↔ Operation</i>	Operation to remove listener for
<i>listenerFunc</i>	Call-back function

Returns

Non-Zero if call-back was unregistered successfully

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

[transfer_callback_example.c](#).

7.6.4.26 `STAR_TRANSFER_STATUS STAR_API_CC STAR_waitOnTransferOperationCompletion ( _In_ STAR_TRANSFER_OPERATION * pTransferOperation, int timeout )`

Blocks until a given transfer operation has completed, or the timeout period expires.

Note that the operation will continue to transmit or receive once this function returns if it has not yet completed. The operation will only be stopped if it completes, there is an error, or it is cancelled by calling [STAR_cancelTransfer↔Operation\(\)](#). If the operation passed to this function has been submitted but has not yet completed when the function returns, the return value will be [STAR_TRANSFER_STATUS_STARTED](#) as the operation has started but not yet completed. This function can be called multiple times for the same operation to wait until that operation completes.

Parameters

<i>in</i>	<i>pTransfer↔ Operation</i>	Operation to wait on
	<i>timeout</i>	Max time in milliseconds to wait, or -1 to wait indefinitely

Returns

result status of the operation

Added in version:

STAR-System v1.1 - 20th December 2011

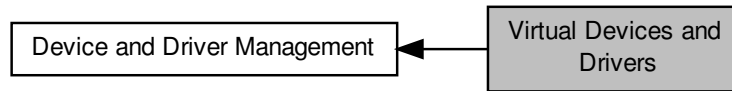
Examples:

[advanced_address_example.c](#), [advanced_continuous_receive_example.c](#), [advanced_multiple_packet↔example.c](#), [advanced_receive_example.c](#), [advanced_send_and_receive_example.c](#), [advanced_send↔example.c](#), [advanced_two_way_example.c](#), [brickMk2_configuration_example.c](#), [link_events.c](#), [rmap_target↔example.c](#), [star-test.c](#), [star_performance_tester.c](#), [time-code_example.c](#), [time-code_test.c](#), [timestamp_test.c](#), [transfer_operation_list_send_example.c](#), [transfer_operation_list_send_receive_example.c](#), and [transmit↔errors_example.c](#).

7.7 Virtual Devices and Drivers

These are functions that are used to manage the properties of virtual drivers and devices.

Collaboration diagram for Virtual Devices and Drivers:



Functions

- `int STAR_API_CC STAR_isDriverVirtual (STAR_DRIVER_ID driverId)`
Gets whether or not a given driver is virtual.
- `int STAR_API_CC STAR_isDeviceVirtual (STAR_DEVICE_ID deviceId)`
Gets whether or not a given device is virtual.

7.7.1 Detailed Description

These are functions that are used to manage the properties of virtual drivers and devices.

7.7.2 Function Documentation

7.7.2.1 `int STAR_API_CC STAR_isDeviceVirtual ( STAR_DEVICE_ID deviceId )`

Gets whether or not a given device is virtual.

Parameters

<i>deviceId</i>	Identifier for device to be checked
-----------------	-------------------------------------

Returns

1 if the device is virtual, otherwise 0

Added in version:

STAR-System v1.12 - 14th November 2012

7.7.2.2 `int STAR_API_CC STAR_isDriverVirtual ( STAR_DRIVER_ID driverId )`

Gets whether or not a given driver is virtual.

Parameters

<i>driverId</i>	Identifier for driver to be checked
-----------------	-------------------------------------

Returns

1 if the driver is virtual, otherwise 0

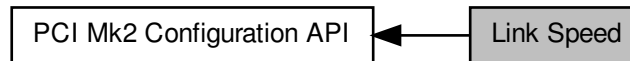
Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

7.8 Link Speed

Functions in this group deal with setting the link speeds on the device.

Collaboration diagram for Link Speed:



Enumerations

- enum `STAR_CFG_PCIMK2_LINK_FREQ` {
`STAR_CFG_PCIMK2_LINK_FREQ_120` = 0, `STAR_CFG_PCIMK2_LINK_FREQ_128` = 5, `STAR_CFG_PCIMK2_LINK_FREQ_140` = 1, `STAR_CFG_PCIMK2_LINK_FREQ_150` = 6,
`STAR_CFG_PCIMK2_LINK_FREQ_160` = 2, `STAR_CFG_PCIMK2_LINK_FREQ_180` = 3, `STAR_CFG_PCIMK2_LINK_FREQ_200` = 4 }

Frequencies a link may run at.

Functions

- int `STAR_API_CC_CFG_PCIMK2_setLinkClockFrequency` (`STAR_DEVICE_ID` deviceId, U8 linkNum, `STAR_CFG_PCIMK2_LINK_FREQ` linkFreq)
Sets the Link Clock Frequency on a given link.
- int `STAR_API_CC_CFG_PCIMK2_getLinkClockFrequency` (`STAR_DEVICE_ID` deviceId, U8 linkNum, `STAR_CFG_PCIMK2_LINK_FREQ` *pLinkFreq)
Gets the Link Clock Frequency for a given link.

7.8.1 Detailed Description

Functions in this group deal with setting the link speeds on the device.

7.8.2 Enumeration Type Documentation

7.8.2.1 enum `STAR_CFG_PCIMK2_LINK_FREQ`

Frequencies a link may run at.

These are divided by a divider set by `CFG_PCIMK2_setLinkRateDivider()` to generate the desired link speed.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Enumerator

`STAR_CFG_PCIMK2_LINK_FREQ_120` 120MHz

`STAR_CFG_PCIMK2_LINK_FREQ_128` 128MHz

Declaration changed in version:

STAR-System v2.0 - 3rd July 2013

STAR_CFG_PCIMK2_LINK_FREQ_140 140MHz

STAR_CFG_PCIMK2_LINK_FREQ_150 150MHz

Declaration changed in version:

STAR-System v2.0 - 3rd July 2013

STAR_CFG_PCIMK2_LINK_FREQ_160 160MHz

STAR_CFG_PCIMK2_LINK_FREQ_180 180MHz

STAR_CFG_PCIMK2_LINK_FREQ_200 200MHz

7.8.3 Function Documentation

7.8.3.1 `int STAR_API_CC_CFG_PCIMK2_getLinkClockFrequency ( STAR_DEVICE_ID deviceID, U8 linkNum, STAR_CFG_PCIMK2_LINK_FREQ * pLinkFreq )`

Gets the Link Clock Frequency for a given link.

Note

This function is for use with SpaceWire PCI Mk2 and cPCI Mk2 devices only. For SpaceWire PCIe and SPLT devices please use `CFG_MK2_getBaseTransmitClock()` instead. For SpaceWire Brick Mk2 and SpaceWire Router Mk2S devices please use `CFG_BRICK_MK2_getLinkClockFrequency()`. For SpaceWire Brick Mk3 devices, please use `CFG_BRICK_MK3_getBaseTransmitClock()`.

Parameters

	<i>deviceID</i>	Device to get the links frequency from.
	<i>linkNum</i>	Link to get frequency for.
out	<i>pLinkFreq</i>	Pointer to value that will be updated with the given links frequency.

Returns

1 if source frequency was successfully obtained, else 0.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Supported devices:

SpaceWire PCI Mk2 and cPCI Mk2

7.8.3.2 `int STAR_API_CC_CFG_PCIMK2_setLinkClockFrequency ( STAR_DEVICE_ID deviceID, U8 linkNum, STAR_CFG_PCIMK2_LINK_FREQ linkFreq )`

Sets the Link Clock Frequency on a given link.

The transmit rate of a link is given by:

$$LinkTransmitRate = \frac{LinkClockRate}{LinkClockRateDivider}$$

Where *LinkClockRate* is linkFreq and *LinkClockRateDivider* is set by `CFG_MK2_setLinkRateDivider()`.

Note

This function is for use with [SpaceWire PCI Mk2](#) and [cPCI Mk2](#) devices only. For [SpaceWire PCIe](#) and [SPLT](#) devices please use [CFG_MK2_setBaseTransmitClock\(\)](#) instead. For [SpaceWire Brick Mk2](#) and [SpaceWire Router Mk2S](#) devices please use [CFG_BRICK_MK2_setLinkClockFrequency\(\)](#). For [SpaceWire Brick Mk3](#) devices, please use [CFG_BRICK_MK3_setBaseTransmitClock\(\)](#).

Parameters

<i>deviceID</i>	Device to modify a links frequency on.
<i>linkNum</i>	Link to have its frequency modified.
<i>linkFreq</i>	Link clock frequency to be set.

Returns

1 if source frequency was successfully set, else 0.

Added in version:

[STAR-System v0.8 \(Beta 1\)](#) - 22nd August 2011

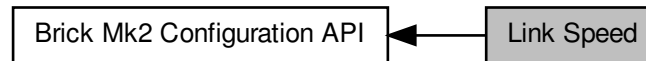
Supported devices:

[SpaceWire PCI Mk2](#) and [cPCI Mk2](#)

7.9 Link Speed

Functions in this group deal with setting the link speeds on the device.

Collaboration diagram for Link Speed:



Enumerations

- enum `STAR_CFG_BRICK_MK2_LINK_FREQ` {
`STAR_CFG_BRICK_MK2_LINK_FREQ_120` = 0, `STAR_CFG_BRICK_MK2_LINK_FREQ_130` = 1, `STAR_CFG_BRICK_MK2_LINK_FREQ_140` = 2, `STAR_CFG_BRICK_MK2_LINK_FREQ_150` = 3,
`STAR_CFG_BRICK_MK2_LINK_FREQ_160` = 4, `STAR_CFG_BRICK_MK2_LINK_FREQ_180` = 5, `STAR_CFG_BRICK_MK2_LINK_FREQ_200` = 6 }

Frequencies a link on a Brick Mk2 compatible device may run at.

Functions

- int `STAR_API_CC_CFG_BRICK_MK2_setLinkClockFrequency` (`STAR_DEVICE_ID` deviceID, U8 linkNum, `STAR_CFG_BRICK_MK2_LINK_FREQ` linkFreq)
Sets the Link Clock Frequency on a given link.
- int `STAR_API_CC_CFG_BRICK_MK2_getLinkClockFrequency` (`STAR_DEVICE_ID` deviceID, U8 linkNum, `_Out_ STAR_CFG_BRICK_MK2_LINK_FREQ` *pLinkFreq)
Gets the Link Clock Frequency for a given link.

7.9.1 Detailed Description

Functions in this group deal with setting the link speeds on the device.

7.9.2 Enumeration Type Documentation

7.9.2.1 enum `STAR_CFG_BRICK_MK2_LINK_FREQ`

Frequencies a link on a Brick Mk2 compatible device may run at.

These are divided by a divider set by `CFG_MK2_setLinkRateDivider()` to generate the desired link speed.

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

Enumerator

`STAR_CFG_BRICK_MK2_LINK_FREQ_120` 120MHz

`STAR_CFG_BRICK_MK2_LINK_FREQ_130` 130MHz

STAR_CFG_BRICK_MK2_LINK_FREQ_140 140MHz
STAR_CFG_BRICK_MK2_LINK_FREQ_150 150MHz
STAR_CFG_BRICK_MK2_LINK_FREQ_160 160MHz
STAR_CFG_BRICK_MK2_LINK_FREQ_180 180MHz
STAR_CFG_BRICK_MK2_LINK_FREQ_200 200MHz

7.9.3 Function Documentation

7.9.3.1 **int STAR_API_CC_CFG_BRICK_MK2_getLinkClockFrequency (STAR_DEVICE_ID deviceID, U8 linkNum, _Out_ STAR_CFG_BRICK_MK2_LINK_FREQ * pLinkFreq)**

Gets the Link Clock Frequency for a given link.

Note

This function is for use with SpaceWire Brick Mk2 and SpaceWire Router Mk2S devices only. For SpaceWire PCIe and SpaceWire Physical Layer Tester devices please use [CFG_MK2_getBaseTransmitClock\(\)](#). For SpaceWire PCI Mk2 and cPCI Mk2 devices please use [CFG_PCIMK2_setLinkClockFrequency\(\)](#). For SpaceWire Brick Mk3 devices, please use [CFG_BRICK_MK3_getBaseTransmitClock\(\)](#).

Parameters

	<i>deviceID</i>	Device to get the link's frequency from.
	<i>linkNum</i>	Link to get frequency for.
out	<i>pLinkFreq</i>	Pointer to value that will be updated with the given links frequency.

Returns

1 if source frequency was successfully obtained, else 0.

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

Supported devices:

SpaceWire Brick Mk2, SpaceWire Router Mk2S

7.9.3.2 **int STAR_API_CC_CFG_BRICK_MK2_setLinkClockFrequency (STAR_DEVICE_ID deviceID, U8 linkNum, STAR_CFG_BRICK_MK2_LINK_FREQ linkFreq)**

Sets the Link Clock Frequency on a given link.

The transmit rate of a link is given by:

$$LinkTransmitRate = \frac{LinkClockRate}{LinkClockRateDivider}$$

Where *LinkClockRate* is linkFreq and *LinkClockRateDivider* is set by [CFG_MK2_setLinkRateDivider\(\)](#).

Note

This function is for use with SpaceWire Brick Mk2 and SpaceWire Router Mk2S devices only. For SpaceWire PCIe and SpaceWire Physical Layer Tester devices please use [CFG_MK2_setBaseTransmitClock\(\)](#). For SpaceWire PCI Mk2 and cPCI Mk2 devices please use [CFG_PCIMK2_setLinkClockFrequency\(\)](#). For SpaceWire Brick Mk3 devices, please use [CFG_BRICK_MK3_setBaseTransmitClock\(\)](#).

Parameters

<i>deviceID</i>	Device to modify a link's frequency on.
<i>linkNum</i>	Link to have its frequency modified.
<i>linkFreq</i>	Link clock frequency to be set.

Returns

1 if source frequency was successfully set, else 0.

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

Supported devices:

SpaceWire Brick Mk2, SpaceWire Router Mk2S

7.10 Interface Mode

Functions in this group deal with managing the device in Interface mode.

Collaboration diagram for Interface Mode:



Functions

- `int STAR_API_CC_CFG_BRICK_MK2_enableInterfaceModeOnPort (STAR_DEVICE_ID deviceId, U8 portNum)`
Selectively enables interface mode on a given port of a supported USB device.
- `int STAR_API_CC_CFG_BRICK_MK2_getInterfaceModeOnPortEnabled (STAR_DEVICE_ID deviceId, U8 portNum, _Out_ int *pEnabled)`
Gets whether interface mode is enabled on a specific port of a supported USB device.
- `int STAR_API_CC_CFG_BRICK_MK2_disableInterfaceModeOnPort (STAR_DEVICE_ID deviceId, U8 portNum)`
Selectively disables interface mode on a given port of a supported USB device.
- `int STAR_API_CC_CFG_BRICK_MK2_enableIdentifySourceOnPort (STAR_DEVICE_ID deviceId, U8 portNum)`
Enables identification of the source port of a received packet on a given port, when in interface mode.
- `int STAR_API_CC_CFG_BRICK_MK2_getIdentifySourceOnPortEnabled (STAR_DEVICE_ID deviceId, U8 portNum, _Out_ int *pEnabled)`
Gets whether identify source is enabled for a specific port.
- `int STAR_API_CC_CFG_BRICK_MK2_disableIdentifySourceOnPort (STAR_DEVICE_ID deviceId, U8 portNum)`
Disables source port identification for a given port.
- `int STAR_API_CC_CFG_BRICK_MK2_setPortRoutingAddress (STAR_DEVICE_ID deviceId, U8 portNum, U8 address)`
Sets the address a packet received on a given port should be routed to when interface mode is enabled.
- `int STAR_API_CC_CFG_BRICK_MK2_getPortRoutingAddress (STAR_DEVICE_ID deviceId, U8 portNum, _Out_ U8 *pAddress)`
Gets the address a packet received on a given port should be routed to when interface mode is enabled.

7.10.1 Detailed Description

Functions in this group deal with managing the device in Interface mode.

7.10.2 Function Documentation

7.10.2.1 `int STAR_API_CC_CFG_BRICK_MK2_disableIdentifySourceOnPort ( STAR_DEVICE_ID deviceId, U8 portNum )`

Disables source port identification for a given port.

Note

This function is used by [SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#) and [SpaceWire Brick Mk3](#) devices. Please refer to the [Configuration API Functions Supported by Device Types](#) page for details of which functions are supported by each device.

Parameters

<i>deviceID</i>	Device to disable source port identification for a given port on.
<i>portNum</i>	Port to disable source identification on.

Returns

1 if source identification was successfully disabled, else 0.

Added in version:

[STAR-System v2.0 Beta 1 - 8th February 2013](#)

Supported devices:

[SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#), [SpaceWire Brick Mk3](#)

7.10.2.2 int **STAR_API_CC_CFG_BRICK_MK2_disableInterfaceModeOnPort** (**STAR_DEVICE_ID** *deviceID*, **U8** *portNum*)

Selectively disables interface mode on a given port of a supported USB device.

When interface mode is disabled, the device operates in routing mode.

Note

This function is used by [SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#) and [SpaceWire Brick Mk3](#) devices. Please refer to the [Configuration API Functions Supported by Device Types](#) page for details of which functions are supported by each device.

Parameters

<i>deviceID</i>	Device to disable interface mode for a port on.
<i>portNum</i>	Port to disable interface mode on.

Returns

1 if interface mode was successfully disabled, else 0.

Added in version:

[STAR-System v2.0 Beta 1 - 8th February 2013](#)

Supported devices:

[SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#), [SpaceWire Brick Mk3](#)

7.10.2.3 int **STAR_API_CC_CFG_BRICK_MK2_enableIdentifySourceOnPort** (**STAR_DEVICE_ID** *deviceID*, **U8** *portNum*)

Enables identification of the source port of a received packet on a given port, when in interface mode.

Interface mode ([CFG_MK2_enableInterfaceMode\(\)](#)) and Identify Source ([CFG_MK2_enableIdentifySource\(\)](#)) must be enabled globally for this to have an effect.

Note

This function is used by [SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#) and [SpaceWire Brick Mk3](#) devices. Please refer to the [Configuration API Functions Supported by Device Types](#) page for details of which functions are supported by each device.

Parameters

<i>deviceID</i>	Device to enable source port identification for a given port on.
<i>portNum</i>	Port to enable source identification on.

Returns

1 if source identification was successfully enabled, else 0.

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

Supported devices:

SpaceWire Brick Mk2, SpaceWire Router Mk2S, SpaceWire Brick Mk3

7.10.2.4 `int STAR_API_CC CFG_BRICK_MK2_enableInterfaceModeOnPort ( STAR_DEVICE_ID deviceID, U8 portNum )`

Selectively enables interface mode on a given port of a supported USB device.

Interface mode must be enabled globally (`CFG_MK2_enableInterfaceMode()`) for interface mode on a port to be enabled.

Note

This function is used by [SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#) and [SpaceWire Brick Mk3](#) devices. Please refer to the [Configuration API Functions Supported by Device Types](#) page for details of which functions are supported by each device.

Parameters

<i>deviceID</i>	Device to enable interface mode for a port on.
<i>portNum</i>	Port to enable interface mode on.

Returns

1 if interface mode was successfully enabled, else 0.

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

Supported devices:

SpaceWire Brick Mk2, SpaceWire Router Mk2S, SpaceWire Brick Mk3

7.10.2.5 `int STAR_API_CC CFG_BRICK_MK2_getIdentifySourceOnPortEnabled ( STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int * pEnabled )`

Gets whether identify source is enabled for a specific port.

Note

This function is used by [SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#) and [SpaceWire Brick Mk3](#) devices. Please refer to the [Configuration API Functions Supported by Device Types](#) page for details of which functions are supported by each device.

Parameters

<i>deviceID</i>	Device to check.
<i>portNum</i>	Port to check.
<i>pEnabled</i>	User supplied value that will be updated to 1 if identify source is enabled, else 0.

Returns

1 if the information could be obtained from the device, else 0.

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

Supported devices:

SpaceWire Brick Mk2, SpaceWire Router Mk2S, SpaceWire Brick Mk3

7.10.2.6 `int STAR_API_CC_CFG_BRICK_MK2_getInterfaceModeOnPortEnabled ( STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int * pEnabled )`

Gets whether interface mode is enabled on a specific port of a supported USB device.

Note

This function is used by [SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#) and [SpaceWire Brick Mk3](#) devices. Please refer to the [Configuration API Functions Supported by Device Types](#) page for details of which functions are supported by each device.

Parameters

<i>deviceID</i>	Device to get interface mode enabled for a port on.
<i>portNum</i>	Port to get information for.
<i>pEnabled</i>	User supplied value that will be updated to 1 if interface mode is enabled, else 0.

Returns

1 if the information could be obtained from the device, else 0.

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

Supported devices:

SpaceWire Brick Mk2, SpaceWire Router Mk2S, SpaceWire Brick Mk3

7.10.2.7 `int STAR_API_CC_CFG_BRICK_MK2_getPortRoutingAddress ( STAR_DEVICE_ID deviceID, U8 portNum, _Out_ U8 * pAddress )`

Gets the address a packet received on a given port should be routed to when interface mode is enabled.

Note

This is an advanced feature and not necessary for normal usage. This function is used by [SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#) and [SpaceWire Brick Mk3](#) devices. Please refer to the [Configuration API Functions Supported by Device Types](#) page for details of which functions are supported by each device.

Parameters

	<i>deviceID</i>	Device to get port routing address from.
	<i>portNum</i>	Port to get port routing address from.
out	<i>pAddress</i>	Pointer to value that will be updated to contain the address.

Returns

1 if the address could be obtained, else 0.

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

Supported devices:

SpaceWire Brick Mk2, SpaceWire Router Mk2S, SpaceWire Brick Mk3

7.10.2.8 `int STAR_API_CC_CFG_BRICK_MK2_setPortRoutingAddress ( STAR_DEVICE_ID deviceID, U8 portNum, U8 address )`

Sets the address a packet received on a given port should be routed to when interface mode is enabled.

Note

This is an advanced feature and not necessary for normal usage.

This function is used by [SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#) and [SpaceWire Brick Mk3](#) devices. Please refer to the [Configuration API Functions Supported by Device Types](#) page for details of which functions are supported by each device.

Parameters

	<i>deviceID</i>	Device to have one of its port's port routing address changed.
	<i>portNum</i>	Port to have its port routing address modified.
	<i>address</i>	New port routing address.

Returns

1 if the address could be set, else 0.

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

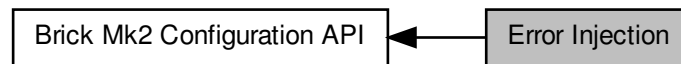
Supported devices:

SpaceWire Brick Mk2, SpaceWire Router Mk2S, SpaceWire Brick Mk3

7.11 Error Injection

Functions in this group deal with error injection.

Collaboration diagram for Error Injection:



Functions

- `int STAR_API_CC_CFG_BRICK_MK2_injectErrors (STAR_DEVICE_ID deviceID, U8 portNum, _In_ STAR_CFG_BRICK_MK2_ERRORS *pErrors)`
Inject the specified errors on a given port.

7.11.1 Detailed Description

Functions in this group deal with error injection.

7.11.2 Function Documentation

7.11.2.1 `int STAR_API_CC_CFG_BRICK_MK2_injectErrors ( STAR_DEVICE_ID deviceID, U8 portNum, _In_ STAR_CFG_BRICK_MK2_ERRORS * pErrors )`

Inject the specified errors on a given port.

Note

This function is used by [SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#) and [SpaceWire Brick Mk3](#) devices. Please refer to the [Configuration API Functions Supported by Device Types](#) page for details of which functions are supported by each device.

Parameters

<i>deviceID</i>	Device to inject the errors on.
<i>portNum</i>	Port to inject the errors on.
<i>pErrors</i>	Pointer to structure specifying which errors are to be injected.

Returns

1 if the errors were successfully injected, else 0.

Added in version:

[STAR-System v2.0 Beta 1](#) - 8th February 2013

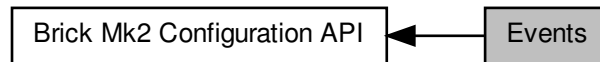
Supported devices:

[SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#), [SpaceWire Brick Mk3](#)

7.12 Events

Functions in this group deal with enabling and disabling link events.

Collaboration diagram for Events:



Functions

- `int STAR_API_CC_CFG_BRICK_MK2_enableStateChangeEventsOnPort (STAR_DEVICE_ID deviceId, U8 portNum)`
Selectively enables state change events on a given port.
- `int STAR_API_CC_CFG_BRICK_MK2_getStateChangeEventsOnPortEnabled (STAR_DEVICE_ID deviceId, U8 portNum, _Out_ int *pEnabled)`
Gets whether state change events are enabled on a specific port.
- `int STAR_API_CC_CFG_BRICK_MK2_disableStateChangeEventsOnPort (STAR_DEVICE_ID deviceId, U8 portNum)`
Selectively disables state change events on a given port.
- `int STAR_API_CC_CFG_BRICK_MK2_enableSpeedChangeEventsOnPort (STAR_DEVICE_ID deviceId, U8 portNum)`
Selectively enables speed change events on a given port.
- `int STAR_API_CC_CFG_BRICK_MK2_getSpeedChangeEventsOnPortEnabled (STAR_DEVICE_ID deviceId, U8 portNum, _Out_ int *pEnabled)`
Gets whether speed change events are enabled on a specific port.
- `int STAR_API_CC_CFG_BRICK_MK2_disableSpeedChangeEventsOnPort (STAR_DEVICE_ID deviceId, U8 portNum)`
Selectively disables speed change events on a given port.

7.12.1 Detailed Description

Functions in this group deal with enabling and disabling link events.

7.12.2 Function Documentation

7.12.2.1 `int STAR_API_CC_CFG_BRICK_MK2_disableSpeedChangeEventsOnPort ( STAR_DEVICE_ID deviceId, U8 portNum )`

Selectively disables speed change events on a given port.

Disabling speed change events will result in speed change event traffic no longer being received on channel 0 when the link speed changes.

Note

This function is used by [SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#) and [SpaceWire Brick Mk3](#) devices. Please refer to the [Configuration API Functions Supported by Device Types](#) page for details of which functions are supported by each device.

Parameters

<i>deviceId</i>	Device to disable speed change events for a port on.
<i>portNum</i>	Port to disable speed change events on.

Returns

1 if speed change events were successfully disabled, else 0.

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

Supported devices:

SpaceWire Brick Mk2, SpaceWire Router Mk2S, SpaceWire Brick Mk3

7.12.2.2 int STAR_API_CC_CFG_BRICK_MK2_disableStateChangeEventsOnPort (STAR_DEVICE_ID deviceId, U8 portNum)

Selectively disables state change events on a given port.

Disabling state change events will result in state change event traffic no longer being received on channel 0 when the link changes to running or disconnected.

Note

This function is used by [SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#) and [SpaceWire Brick Mk3](#) devices. Please refer to the [Configuration API Functions Supported by Device Types](#) page for details of which functions are supported by each device.

Parameters

<i>deviceId</i>	Device to disable state change events for a port on.
<i>portNum</i>	Port to disable state change events on.

Returns

1 if state change events were successfully disabled, else 0.

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

Supported devices:

SpaceWire Brick Mk2, SpaceWire Router Mk2S, SpaceWire Brick Mk3

7.12.2.3 int STAR_API_CC_CFG_BRICK_MK2_enableSpeedChangeEventsOnPort (STAR_DEVICE_ID deviceId, U8 portNum)

Selectively enables speed change events on a given port.

Enabling speed change events will result in speed change event traffic being received on channel 0 when the link speed changes.

Note

This function is used by [SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#) and [SpaceWire Brick Mk3](#) devices. Please refer to the [Configuration API Functions Supported by Device Types](#) page for details of which functions are supported by each device.

Parameters

<i>deviceID</i>	Device to enable speed change events for a port on.
<i>portNum</i>	Port to enable speed change events on.

Returns

1 if speed change events were successfully enabled, else 0.

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

Supported devices:

SpaceWire Brick Mk2, SpaceWire Router Mk2S, SpaceWire Brick Mk3

7.12.2.4 `int STAR_API_CC_CFG_BRICK_MK2_enableStateChangeEventsOnPort ( STAR_DEVICE_ID deviceID, U8 portNum )`

Selectively enables state change events on a given port.

Enabling state change events will result in state change event traffic being received on channel 0 when the link changes to running or disconnected.

Note

This function is used by [SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#) and [SpaceWire Brick Mk3](#) devices. Please refer to the [Configuration API Functions Supported by Device Types](#) page for details of which functions are supported by each device.

Parameters

<i>deviceID</i>	Device to enable state change events for a port on.
<i>portNum</i>	Port to enable state change events on.

Returns

1 if state change events were successfully enabled, else 0.

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

Supported devices:

SpaceWire Brick Mk2, SpaceWire Router Mk2S, SpaceWire Brick Mk3

7.12.2.5 `int STAR_API_CC_CFG_BRICK_MK2_getSpeedChangeEventsOnPortEnabled ( STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int * pEnabled )`

Gets whether speed change events are enabled on a specific port.

Note

This function is used by [SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#) and [SpaceWire Brick Mk3](#) devices. Please refer to the [Configuration API Functions Supported by Device Types](#) page for details of which functions are supported by each device.

Parameters

<i>deviceID</i>	Device to get whether speed change events are enabled for a port.
<i>portNum</i>	Port to get information for.
<i>pEnabled</i>	User supplied value that will be updated to 1 if speed change events are enabled, else 0.

Returns

1 if the information could be obtained from the device, else 0.

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

Supported devices:

SpaceWire Brick Mk2, SpaceWire Router Mk2S, SpaceWire Brick Mk3

7.12.2.6 `int STAR_API_CC_CFG_BRICK_MK2_getStateChangeEventsOnPortEnabled ( STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int * pEnabled )`

Gets whether state change events are enabled on a specific port.

Note

This function is used by [SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#) and [SpaceWire Brick Mk3](#) devices. Please refer to the [Configuration API Functions Supported by Device Types](#) page for details of which functions are supported by each device.

Parameters

<i>deviceID</i>	Device to get whether state change events are enabled for a port.
<i>portNum</i>	Port to get information for.
<i>pEnabled</i>	User supplied value that will be updated to 1 if state change events are enabled, else 0.

Returns

1 if the information could be obtained from the device, else 0.

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

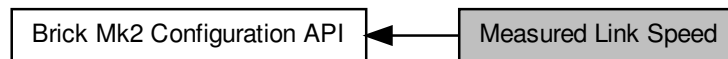
Supported devices:

SpaceWire Brick Mk2, SpaceWire Router Mk2S, SpaceWire Brick Mk3

7.13 Measured Link Speed

Functions in this group deal with reading the measured link speed.

Collaboration diagram for Measured Link Speed:



Macros

- `#define STAR_CFG_BRICK_MK2_LINK_SPEED_UNITS_KBPS 200`
The units in Kbit/s used to represent the link speed returned when calling `CFG_BRICK_MK2_getMeasuredLinkSpeed()`.

7.13.1 Detailed Description

Functions in this group deal with reading the measured link speed.

7.13.2 Macro Definition Documentation

7.13.2.1 `#define STAR_CFG_BRICK_MK2_LINK_SPEED_UNITS_KBPS 200`

The units in Kbit/s used to represent the link speed returned when calling `CFG_BRICK_MK2_getMeasuredLinkSpeed()`.

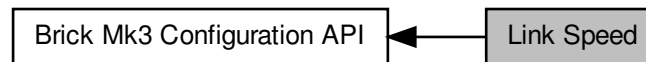
Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

7.14 Link Speed

Functions in this group deal with setting the link speeds of Brick Mk3 devices.

Collaboration diagram for Link Speed:



Functions

- `int STAR_API_CC_CFG_BRICK_MK3_setBaseTransmitClock (STAR_DEVICE_ID deviceID, U8 linkNum, STAR_CFG_MK2_BASE_TRANSMIT_CLOCK clockRateParams)`
Sets the base transmit clock rate on a given link of a Brick Mk3 device:
- `int STAR_API_CC_CFG_BRICK_MK3_getBaseTransmitClock (STAR_DEVICE_ID deviceID, U8 linkNum, ↔ _Out_ STAR_CFG_MK2_BASE_TRANSMIT_CLOCK *pClockRateParams)`
Gets the base transmit clock rate parameters for a given link of a Brick Mk3 device.
- `int STAR_API_CC_CFG_BRICK_MK3_setTransmitClock (STAR_DEVICE_ID deviceID, U8 linkNum, STAR_CFG_MK2_BASE_TRANSMIT_CLOCK clockRateParams)`
Sets the transmit clock rate on a given link.
- `int STAR_API_CC_CFG_BRICK_MK3_getTransmitClock (STAR_DEVICE_ID deviceID, U8 linkNum, _Out_ STAR_CFG_MK2_BASE_TRANSMIT_CLOCK *pClockRateParams)`
Gets the transmit clock rate parameters for a given link.

7.14.1 Detailed Description

Functions in this group deal with setting the link speeds of Brick Mk3 devices.

7.14.2 Function Documentation

7.14.2.1 `int STAR_API_CC_CFG_BRICK_MK3_getBaseTransmitClock ( STAR_DEVICE_ID deviceID, U8 linkNum, _Out_ STAR_CFG_MK2_BASE_TRANSMIT_CLOCK * pClockRateParams )`

Gets the base transmit clock rate parameters for a given link of a Brick Mk3 device.

Note

This function is currently for use with [SpaceWire Brick Mk3](#) devices before hardware version v1.01. For later versions use `CFG_BRICK_MK3_getTransmitClock()`.

The [SpaceWire PCI Mk2](#) and [cPCI Mk2](#), [SpaceWire PCIe](#), [SpaceWire Physical Layer Tester](#), [SpaceWire Brick Mk2](#) and [SpaceWire Router Mk2S](#) do not support this function. For [SpaceWire PCI Mk2](#) and [cPCI Mk2](#) devices, please use `CFG_PCIMK2_getLinkClockFrequency()` instead. For [SpaceWire PCIe](#) and [SpaceWire Physical Layer Tester](#) devices, please use either `CFG_MK2_getBaseTransmitClock()` or `CFG_MK2_get↔TransmitClock()` depending on hardware version. For [SpaceWire Brick Mk2](#) and [SpaceWire Router Mk2S](#) devices, please use `CFG_BRICK_MK2_getLinkClockFrequency()`.

No check is made by this function to ensure it is being used with a compatible device.

Parameters

	<i>deviceID</i>	Device to get the base transmit clock rate from.
	<i>linkNum</i>	Link to get base transmit clock rate for.
out	<i>pClockRateParams</i>	A pointer to an existing structure that will be updated to contain the given link's base transmit clock rate parameters.

Returns

1 if the base transmit clock frequency parameters were successfully obtained, else 0.

Added in version:

STAR-System v3.0 Beta 1 - 31st March 2015

Supported devices:

SpaceWire Brick Mk3 (earlier than version 1.01)

7.14.2.2 `int STAR_API_CC_CFG_BRICK_MK3_getTransmitClock ( STAR_DEVICE_ID deviceID, U8 linkNum, _Out_ STAR_CFG_MK2_BASE_TRANSMIT_CLOCK * pClockRateParams )`

Gets the transmit clock rate parameters for a given link.

Note

This function is currently for use with [SpaceWire Brick Mk3](#) devices from hardware version v1.01. For earlier versions use [CFG_BRICK_MK3_getBaseTransmitClock\(\)](#).

No check is made by this function to ensure it is being used with a compatible device.

Parameters

	<i>deviceID</i>	Device to get the transmit clock rate from.
	<i>linkNum</i>	Link to get the transmit clock rate for.
out	<i>pClockRateParams</i>	Pointer to value that will be updated with the given link's transmit clock rate parameters.

Returns

1 if transmit clock frequency parameters were successfully obtained, else 0.

Added in version:

STAR-System v3.0 Beta 7 - 8th March 2016

Supported devices:

SpaceWire Brick Mk3 (version 1.01 or greater)

7.14.2.3 `int STAR_API_CC_CFG_BRICK_MK3_setBaseTransmitClock ( STAR_DEVICE_ID deviceID, U8 linkNum, STAR_CFG_MK2_BASE_TRANSMIT_CLOCK clockRateParams )`

Sets the base transmit clock rate on a given link of a Brick Mk3 device:

$$BaseClockRate = \frac{100\text{MHz} \times MULTIPLIER}{DIVISOR}$$

The *MULTIPLIER* and *DIVISOR* are properties of the `STAR_CFG_MK2_BASE_TRANSMIT_CLOCK` structure, passed as a parameter. The output clock rate must be between 5 MHz and 200 MHz.

The transmit rate of the link is given by:

$$LinkTransmitRate = \frac{BaseClockRate}{LinkClockRateDivider} \times 2$$

The *LinkClockRateDivider* can be set using the `CFG_MK2_setLinkRateDivider()` function.

Note

This function is currently for use with [SpaceWire Brick Mk3](#) devices before hardware version v1.01. For later versions use `CFG_BRICK_MK3_setTransmitClock()`.

The [SpaceWire PCI Mk2](#) and [cPCI Mk2](#), [SpaceWire PCIe](#), [SpaceWire Physical Layer Tester](#), [SpaceWire Brick Mk2](#) and [SpaceWire Router Mk2S](#) do not support this function. For [SpaceWire PCI Mk2](#) and [cPCI Mk2](#) devices, please use `CFG_PCIMK2_setLinkClockFrequency()` instead. For [SpaceWire PCIe](#) and [SpaceWire Physical Layer Tester](#) devices, please use either `CFG_MK2_setBaseTransmitClock()` or `CFG_MK2_setTransmitClock()` depending on hardware version. For [SpaceWire Brick Mk2](#) and [SpaceWire Router Mk2S](#) devices, please use `CFG_BRICK_MK2_setLinkClockFrequency()`.

No check is made by this function to ensure it is being used with a compatible device.

Parameters

<i>deviceID</i>	Device to modify a link's base transmit clock rate on.
<i>linkNum</i>	Link to have its base transmit clock rate modified.
<i>clockRateParams</i>	Structure containing the base transmit clock rate parameters to be set.

Returns

1 if link's Base transmit clock frequency was successfully set, else 0.

Added in version:

[STAR-System v3.0 Beta 1](#) - 31st March 2015

Supported devices:

[SpaceWire Brick Mk3](#) (earlier than version 1.01)

7.14.2.4 `int STAR_API_CC CFG_BRICK_MK3_setTransmitClock ( STAR_DEVICE_ID deviceID, U8 linkNum, STAR_CFG_MK2_BASE_TRANSMIT_CLOCK clockRateParams )`

Sets the transmit clock rate on a given link.

The transmit rate of a link is given by:

$$LinkTransmitRate = \frac{100\text{MHz} \times MULTIPLIER}{DIVISOR} \times 2$$

The *MULTIPLIER* and *DIVISOR* are properties of the `STAR_CFG_MK2_BASE_TRANSMIT_CLOCK` structure, passed as a parameter. The output clock rate will be between 1 MHz and 200 MHz giving data rates between 2 Mbit/s and 400 Mbit/s.

Note that for data rates below 10 Mbit/s, an exact rate may not be achievable and it may be rounded up or down slightly.

Note

This function is currently for use with [SpaceWire Brick Mk3](#) devices from hardware version v1.01. For earlier versions use [CFG_BRICK_MK3_setBaseTransmitClock\(\)](#).

No check is made by this function to ensure it is being used with a compatible device.

Parameters

<i>deviceID</i>	Device to modify a link's transmit clock rate on.
<i>linkNum</i>	Link to have its transmit clock rate modified.
<i>clockRate</i> ↔	Transmit clock rate parameters to be set.
<i>Params</i>	

Returns

1 if link's transmit clock frequency was successfully set, else 0.

Added in version:

[STAR-System v3.0 Beta 7 - 8th March 2016](#)

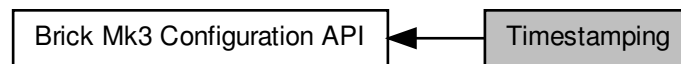
Supported devices:

[SpaceWire Brick Mk3](#) (version 1.01 or greater)

7.15 Timestamping

Functions in this group deal with timestamping related functions for the Brick Mk3.

Collaboration diagram for Timestamping:



Enumerations

- enum STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD { STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD_NONE = 0, STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD_EXTERNAL_TRIGGER = 1, STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD_PULSE_GENERATOR = 2 }
Timestamps can be generated by either an external trigger signal or by an internal pulse generator.
- enum STAR_CFG_BRICK_MK3_PULSE_FREQ { STAR_CFG_BRICK_MK3_PULSE_FREQ_1 = 0, STAR_CFG_BRICK_MK3_PULSE_FREQ_10 = 1, STAR_CFG_BRICK_MK3_PULSE_FREQ_100 = 2, STAR_CFG_BRICK_MK3_PULSE_FREQ_1000 = 3 }
Defines frequencies that can be used by the global pulse generator.

Functions

- int STAR_API_CC_CFG_BRICK_MK3_setTimestampMethod (STAR_DEVICE_ID deviceId, STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD timestampMethod)
Sets the timestamping method to use for the given device.
- int STAR_API_CC_CFG_BRICK_MK3_getTimestampMethod (STAR_DEVICE_ID deviceId, _Out_ STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD *pTimestampMethod)
Gets the timestamping method that is currently in use for the given device.
- int STAR_API_CC_CFG_BRICK_MK3_enableRxTimestampEventsOnPort (STAR_DEVICE_ID deviceId, U8 portNum)
Selectively enables receive timestamp events on a given port.
- int STAR_API_CC_CFG_BRICK_MK3_getRxTimestampEventsOnPortEnabled (STAR_DEVICE_ID deviceId, U8 portNum, _Out_ int *pEnabled)
Gets whether receive timestamp events are enabled on a specific port.
- int STAR_API_CC_CFG_BRICK_MK3_disableRxTimestampEventsOnPort (STAR_DEVICE_ID deviceId, U8 portNum)
Selectively disables receive timestamp events on a given port.
- int STAR_API_CC_CFG_BRICK_MK3_setTimestampValue (STAR_DEVICE_ID deviceId, U32 value)
Sets the current timestamp value for the given device which will be incremented on the next synchronisation pulse.
- int STAR_API_CC_CFG_BRICK_MK3_getTimestampValue (STAR_DEVICE_ID deviceId, _Out_ U32 *pValue)
Gets the current timestamp value.
- int STAR_API_CC_CFG_BRICK_MK3_setPulseGeneratorFrequency (STAR_DEVICE_ID deviceId, STAR_CFG_BRICK_MK3_PULSE_FREQ frequency)
Sets the frequency of each generated pulse.
- int STAR_API_CC_CFG_BRICK_MK3_getPulseGeneratorFrequency (STAR_DEVICE_ID deviceId, _Out_ STAR_CFG_BRICK_MK3_PULSE_FREQ *pFrequency)
Gets the frequency of each generated pulse.

7.15.1 Detailed Description

Functions in this group deal with timestamping related functions for the Brick Mk3.

The functions in this group can be used to receive start of packet and end of packet timestamps for data that is received on either SpaceWire link.

7.15.2 Enumeration Type Documentation

7.15.2.1 enum `STAR_CFG_BRICK_MK3_PULSE_FREQ`

Defines frequencies that can be used by the global pulse generator.

Added in version:

STAR-System v3.6 - 3rd April 2017

Enumerator

`STAR_CFG_BRICK_MK3_PULSE_FREQ_1` 1Hz
`STAR_CFG_BRICK_MK3_PULSE_FREQ_10` 10Hz
`STAR_CFG_BRICK_MK3_PULSE_FREQ_100` 100Hz
`STAR_CFG_BRICK_MK3_PULSE_FREQ_1000` 1KHz

7.15.2.2 enum `STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD`

Timestamps can be generated by either an external trigger signal or by an internal pulse generator.

This enum defines different methods of timestamping that are available.

Added in version:

STAR-System v3.6 - 3rd April 2017

Enumerator

`STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD_NONE` No timestamp method.
`STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD_EXTERNAL_TRIGGER` External trigger used for timestamp sync.
`STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD_PULSE_GENERATOR` Internal pulse generator used for timestamp sync.

7.15.3 Function Documentation

7.15.3.1 int `STAR_API_CC CFG_BRICK_MK3_disableRxTimestampEventsOnPort ( STAR_DEVICE_ID deviceID, U8 portNum )`

Selectively disables receive timestamp events on a given port.

Disabling receive timestamp events will result in no timestamp information being provided after the packet.

Note

This function is currently for use with [SpaceWire Brick Mk3](#) devices from hardware version v1.02 and [SpaceWire PXI Interface and PXI Interface Mk2](#) devices.

No check is made by this function to ensure it is being used with a compatible device.

Parameters

<i>deviceID</i>	Device to disable receive timestamp events for a port on.
<i>portNum</i>	Port to disable receive timestamp events on.

Returns

1 if receive timestamp events were successfully disabled, else 0.

Added in version:

STAR-System v3.6 - 3rd April 2017

Supported devices:

SpaceWire Brick Mk3 (version 1.02 or greater), SpaceWire PXI Interface and PXI Interface Mk2

7.15.3.2 int STAR_API_CC_CFG_BRICK_MK3_enableRxTimestampEventsOnPort (STAR_DEVICE_ID *deviceID*, U8 *portNum*)

Selectively enables receive timestamp events on a given port.

Enabling receive timestamp events will result in the timestamp that the last packet was received at to be provided as a STAR_STREAM_ITEM_TYPE_RX_TIMESTAMP.

Note

This function is currently for use with SpaceWire Brick Mk3 devices from hardware version v1.02 and SpaceWire PXI Interface and PXI Interface Mk2 devices.

No check is made by this function to ensure it is being used with a compatible device.

Parameters

<i>deviceID</i>	Device to enable receive timestamp events for a port on.
<i>portNum</i>	Port to enable receive timestamp events on.

Returns

1 if receive timestamp packets were successfully enabled, else 0.

Added in version:

STAR-System v3.6 - 3rd April 2017

Supported devices:

SpaceWire Brick Mk3 (version 1.02 or greater), SpaceWire PXI Interface and PXI Interface Mk2

7.15.3.3 int STAR_API_CC_CFG_BRICK_MK3_getPulseGeneratorFrequency (STAR_DEVICE_ID *deviceID*, _Out_ STAR_CFG_BRICK_MK3_PULSE_FREQ * *pFrequency*)

Gets the frequency of each generated pulse.

Note

This function is currently for use with SpaceWire Brick Mk3 devices from hardware version v1.02 and SpaceWire PXI Interface and PXI Interface Mk2 devices.

No check is made by this function to ensure it is being used with a compatible device.

Parameters

	<i>deviceId</i>	Device to get the pulse generator frequency value for.
out	<i>pFrequency</i>	Pointer to value that will be updated with the frequency between each pulse cycle.

Returns

1 if frequency value was successfully obtained, else 0.

Added in version:

STAR-System v3.6 - 3rd April 2017

Supported devices:

SpaceWire Brick Mk3 (version 1.02 or greater), SpaceWire PXI Interface and PXI Interface Mk2

7.15.3.4 `int STAR_API_CC_CFG_BRICK_MK3_getRxTimestampEventsOnPortEnabled ( STAR_DEVICE_ID deviceId, U8 portNum, _Out_ int * pEnabled )`

Gets whether receive timestamp events are enabled on a specific port.

Note

This function is currently for use with SpaceWire Brick Mk3 devices from hardware version v1.02 and SpaceWire PXI Interface and PXI Interface Mk2 devices.

No check is made by this function to ensure it is being used with a compatible device.

Parameters

	<i>deviceId</i>	Device to get whether receive timestamp events are enabled for a port.
	<i>portNum</i>	Port to get receive timestamp events for.
out	<i>pEnabled</i>	User supplied value that will be updated to 1 if receive timestamp events are enabled, else 0.

Returns

1 if the information could be obtained from the device, else 0.

Added in version:

STAR-System v3.6 - 3rd April 2017

Supported devices:

SpaceWire Brick Mk3 (version 1.02 or greater), SpaceWire PXI Interface and PXI Interface Mk2

7.15.3.5 `int STAR_API_CC_CFG_BRICK_MK3_getTimestampMethod ( STAR_DEVICE_ID deviceId, _Out_ STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD * pTimestampMethod )`

Gets the timestamping method that is currently in use for the given device.

Note

This function is currently for use with SpaceWire Brick Mk3 devices from hardware version v1.02 and SpaceWire PXI Interface and PXI Interface Mk2 devices.

No check is made by this function to ensure it is being used with a compatible device.

Parameters

	<i>deviceID</i>	Device to get the timestamping method for.
out	<i>pTimestamp↔ Method</i>	Pointer to value that will be updated with the given link's timestamp method.

Returns

1 if timestamp method was successfully obtained, else 0.

Added in version:

STAR-System v3.6 - 3rd April 2017

Supported devices:

SpaceWire Brick Mk3 (version 1.02 or greater), SpaceWire PXI Interface and PXI Interface Mk2

7.15.3.6 int STAR_API_CC_CFG_BRICK_MK3_getTimestampValue (STAR_DEVICE_ID *deviceID*, _Out_ U32 * *pValue*)

Gets the current timestamp value.

Note

This function is currently for use with SpaceWire Brick Mk3 devices from hardware version v1.02 and Space↔Wire PXI Interface and PXI Interface Mk2 devices.

No check is made by this function to ensure it is being used with a compatible device.

Parameters

	<i>deviceID</i>	Device to get the timestamp value for.
out	<i>pValue</i>	Pointer to value that will be updated with the given link's timestamp value.

Returns

1 if timestamp value was successfully obtained, else 0.

Added in version:

STAR-System v3.6 - 3rd April 2017

Supported devices:

SpaceWire Brick Mk3 (version 1.02 or greater), SpaceWire PXI Interface and PXI Interface Mk2

7.15.3.7 int STAR_API_CC_CFG_BRICK_MK3_setPulseGeneratorFrequency (STAR_DEVICE_ID *deviceID*, STAR_CFG_BRICK_MK3_PULSE_FREQ *frequency*)

Sets the frequency of each generated pulse.

Note

This function is currently for use with SpaceWire Brick Mk3 devices from hardware version v1.02 and Space↔Wire PXI Interface and PXI Interface Mk2 devices.

No check is made by this function to ensure it is being used with a compatible device.

Parameters

<i>deviceID</i>	Device to set the pulse generator frequency for.
<i>frequency</i>	The frequency value to set.

Returns

1 if frequency was successfully set, else 0.

Added in version:

STAR-System v3.6 - 3rd April 2017

Supported devices:

SpaceWire Brick Mk3 (version 1.02 or greater), SpaceWire PXI Interface and PXI Interface Mk2

7.15.3.8 `int STAR_API_CC_CFG_BRICK_MK3_setTimestampMethod ( STAR_DEVICE_ID deviceID, STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD timestampMethod )`

Sets the timestamping method to use for the given device.

Note

This function is currently for use with SpaceWire Brick Mk3 devices from hardware version v1.02 and SpaceWire PXI Interface and PXI Interface Mk2 devices.

No check is made by this function to ensure it is being used with a compatible device.

The device will trigger on the rising edge of the external pulse but the Triggering API can be used to change it to trigger on the falling edge using e.g. `TRIGGER_BRICK_MK3_enableExtTriggerEdgeDetectMode()` and `TRIGGER_BRICK_MK3_enableExtTriggerInvert()`.

Parameters

<i>deviceID</i>	Device to set the timestamping method for.
<i>timestampMethod</i>	Timestamp method to enable for device.

Returns

1 if timestamp method was successfully set, else 0.

Added in version:

STAR-System v3.6 - 3rd April 2017

Supported devices:

SpaceWire Brick Mk3 (version 1.02 or greater), SpaceWire PXI Interface and PXI Interface Mk2

7.15.3.9 `int STAR_API_CC_CFG_BRICK_MK3_setTimestampValue ( STAR_DEVICE_ID deviceID, U32 value )`

Sets the current timestamp value for the given device which will be incremented on the next synchronisation pulse.

Note

This function is currently for use with SpaceWire Brick Mk3 devices from hardware version v1.02 and SpaceWire PXI Interface and PXI Interface Mk2 devices.

No check is made by this function to ensure it is being used with a compatible device.

Parameters

<i>deviceId</i>	Device to set the timestamp value for.
<i>value</i>	Timestamp value to set for device.

Returns

1 if timestamp value was successfully set, else 0.

Added in version:

STAR-System v3.6 - 3rd April 2017

Supported devices:

SpaceWire Brick Mk3 (version 1.02 or greater), SpaceWire PXI Interface and PXI Interface Mk2

7.16 Link Speed

Functions in this group deal with setting the link speeds of PXI devices.

Collaboration diagram for Link Speed:



Functions

- `int STAR_API_CC_CFG_PXI_setLinkRateDivider (STAR_DEVICE_ID deviceId, U8 divider)`
Sets the Link rate divider for a PXI device.
- `int STAR_API_CC_CFG_PXI_getLinkRateDivider (STAR_DEVICE_ID deviceId, _Out_ U8 *pDivider)`
Gets the Link rate divider for a PXI device.
- `int STAR_API_CC_CFG_PXI_setBaseTransmitClock (STAR_DEVICE_ID deviceId, U8 linkNum, STAR_CFG_MK2_BASE_TRANSMIT_CLOCK clockRateParams)`
Sets the base transmit clock rate on a given link:
- `int STAR_API_CC_CFG_PXI_getBaseTransmitClock (STAR_DEVICE_ID deviceId, U8 linkNum, _Out_ STAR_CFG_MK2_BASE_TRANSMIT_CLOCK *clockRateParams)`
Gets the base transmit clock rate parameters for a given link.
- `int STAR_API_CC_CFG_PXI_setTransmitClock (STAR_DEVICE_ID deviceId, U8 linkNum, STAR_CFG_MK2_BASE_TRANSMIT_CLOCK clockRateParams)`
Sets the transmit clock rate on a given link.
- `int STAR_API_CC_CFG_PXI_getTransmitClock (STAR_DEVICE_ID deviceId, U8 linkNum, _Out_ STAR_CFG_MK2_BASE_TRANSMIT_CLOCK *clockRateParams)`
Gets the transmit clock rate parameters for a given link.

7.16.1 Detailed Description

Functions in this group deal with setting the link speeds of PXI devices.

7.16.2 Function Documentation

7.16.2.1 `int STAR_API_CC_CFG_PXI_getBaseTransmitClock ( STAR_DEVICE_ID deviceId, U8 linkNum, _Out_ STAR_CFG_MK2_BASE_TRANSMIT_CLOCK * clockRateParams )`

Gets the base transmit clock rate parameters for a given link.

Note

This function is currently for use with PXI devices before hardware version V1.1. For later versions use `CFG_PXI_getTransmitClock()`.

No check is made by this function to ensure it is being used with a compatible device.

Parameters

	<i>deviceID</i>	Device to get the base transmit clock rate from.
	<i>linkNum</i>	Link to get base transmit clock rate for.
out	<i>clockRateParams</i>	Pointer to value that will be updated with the given link's Base transmit clock rate parameters.

Returns

1 if Base transmit clock frequency parameters were successfully obtained, else 0.

Added in version:

STAR-System v3.0 Beta 4 - 13th November 2015

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (prior to version 1.1), SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2 (prior to version 1.1)

7.16.2.2 int STAR_API_CC_CFG_PXI_getLinkRateDivider (STAR_DEVICE_ID *deviceID*, _Out_ U8 * *pDivider*)

Gets the Link rate divider for a PXI device.

Note

This function is currently for use with PXI devices before hardware version V1.1. No check is made by this function to ensure it is being used with a compatible device.

Parameters

	<i>deviceID</i>	Device to get a links divider from..
out	<i>pDivider</i>	Value of the divider.

Returns

1 if was successfully obtained, else 0.

Added in version:

STAR-System v3.0 Beta 4 - 13th November 2015

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

7.16.2.3 int STAR_API_CC_CFG_PXI_getTransmitClock (STAR_DEVICE_ID *deviceID*, U8 *linkNum*, _Out_ STAR_CFG_MK2_BASE_TRANSMIT_CLOCK * *clockRateParams*)

Gets the transmit clock rate parameters for a given link.

Note

This function is currently for use with PXI devices from hardware version V1.1. For earlier versions use `CFG_PXI_getBaseTransmitClock()`.

No check is made by this function to ensure it is being used with a compatible device.

For the SpaceWire PXI 12 Port Router device, clocks are shared by pairs of links. For example, getting the transmit clock frequency for link 1 will return the transmit clock frequency also shared by link 2. Similarly, links 3 and 4, 5 and 6, 7 and 8, 9 and 10, and 11 and 12 share clocks. Note that the odd-numbered link must be used to get the transmit clock frequency for each pair of links.

Parameters

	<i>deviceID</i>	Device to get the transmit clock rate from.
	<i>linkNum</i>	Link to get the transmit clock rate for.
out	<i>clockRate↔ Params</i>	Pointer to value that will be updated with the given link's transmit clock rate parameters.

Returns

1 if transmit clock frequency parameters were successfully obtained, else 0.

Added in version:

STAR-System v3.0 Beta 7 - 8th March 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (version 1.1 or greater), SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2 (version 1.1 or greater)

7.16.2.4 `int STAR_API_CC_CFG_PXI_setBaseTransmitClock ( STAR_DEVICE_ID deviceID, U8 linkNum, STAR_CFG_MK2_BASE_TRANSMIT_CLOCK clockRateParams )`

Sets the base transmit clock rate on a given link:

$$BaseClockRate = \frac{100\text{MHz} \times MULTIPLIER}{DIVISOR}$$

The *MULTIPLIER* and *DIVISOR* are properties of the `STAR_CFG_MK2_BASE_TRANSMIT_CLOCK` structure, passed as a parameter. The output clock rate must be between 5 MHz and 200 MHz.

The transmit rate of the link is given by:

$$LinkTransmitRate = \frac{BaseClockRate}{LinkClockRateDivider} \times 2$$

The *LinkClockRateDivider* can be set using the `CFG_PXI_setLinkRateDivider()` function.

Note

This function is currently for use with PXI devices before hardware version V1.1. For later versions use `CFG_PXI_setTransmitClock()`.

No check is made by this function to ensure it is being used with a compatible device.

Parameters

	<i>deviceID</i>	Device to modify a link's base transmit clock rate on.
	<i>linkNum</i>	Link to have its base transmit clock rate modified.
	<i>clockRate↔ Params</i>	Base transmit clock rate parameters to be set.

Returns

1 if link's Base transmit clock frequency was successfully set, else 0.

Added in version:

STAR-System v3.0 Beta 4 - 13th November 2015

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (prior to version 1.1), SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2 (prior to version 1.1)

7.16.2.5 `int STAR_API_CC_CFG_PXI_setLinkRateDivider ( STAR_DEVICE_ID deviceID, U8 divider )`

Sets the Link rate divider for a PXI device.

The transmit rate of a link is given by:

$$LinkTransmitRate = \frac{BaseClockRate}{LinkClockRateDivider} \times 2$$

Where *BaseClockRate* is set by the appropriate function (i.e. `CFG_PXI_setBaseTransmitClock()`) and *LinkClockRateDivider* = divider.

Note

It is never necessary to set a link rate divider higher than 100, as the minimum transmit rate permitted by SpaceWire is 2 Mbit/s (200 Mbit/s Max / 100).

If an odd number (other than 1) is specified as the divider parameter, the previous even integer will be used instead. For example: if a divider value of 3 is specified, the actual divider set shall be 2.

This function is currently for use with PXI devices before hardware version V1.1. No check is made by this function to ensure it is being used with a compatible device.

Parameters

<i>deviceID</i>	Device to modify a links divider on.
<i>divider</i>	Value of the new divider. Valid input: 1, Even numbers 2 through 126.

Returns

1 if divider was successfully set, else 0.

Added in version:

STAR-System v3.0 Beta 4 - 13th November 2015

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

7.16.2.6 `int STAR_API_CC_CFG_PXI_setTransmitClock ( STAR_DEVICE_ID deviceID, U8 linkNum, STAR_CFG_MK2_BASE_TRANSMIT_CLOCK clockRateParams )`

Sets the transmit clock rate on a given link.

The transmit rate of a link is given by:

$$LinkTransmitRate = \frac{100\text{MHz} \times MULTIPLIER}{DIVISOR} \times 2$$

The *MULTIPLIER* and *DIVISOR* are properties of the `STAR_CFG_MK2_BASE_TRANSMIT_CLOCK` structure, passed as a parameter. The output clock rate will be between 1 MHz and 200 MHz giving data rates between 2 Mbit/s and 400 Mbit/s

Note that for data rates below 10 Mbit/s, an exact rate may not be achievable and it may be rounded up or down slightly.

Note

This function is currently for use with PXI devices from hardware version V1.1. For earlier versions use [CFG_PXI_setBaseTransmitClock\(\)](#).

No check is made by this function to ensure it is being used with a compatible device.

For the SpaceWire PXI 12 Port Router device, clocks are shared by pairs of links. For example, setting the transmit clock frequency for link 1 will also affect link 2. Similarly, links 3 and 4, 5 and 6, 7 and 8, 9 and 10, and 11 and 12 share clocks. Note that the odd-numbered link must be used to set the transmit clock frequency for each pair of links.

Parameters

<i>deviceID</i>	Device to modify a link's transmit clock rate on.
<i>linkNum</i>	Link to have its transmit clock rate modified.
<i>clockRate</i> ↔ <i>Params</i>	Transmit clock rate parameters to be set.

Returns

1 if link's transmit clock frequency was successfully set, else 0.

Added in version:

[STAR-System v3.0 Beta 7 - 8th March 2016](#)

Supported devices:

[SpaceWire PXI Interface and PXI Interface Mk2](#) (version 1.1 or greater), [SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2](#) (version 1.1 or greater)

7.17 Error Injection

Functions in this group deal with injecting errors onto SpaceWire links.

Collaboration diagram for Error Injection:



Functions

- `int STAR_API_CC_CFG_PXI_injectError (STAR_DEVICE_ID deviceId, unsigned char port, SPW_ERROR error)`

Immediately injects the specified error on a given device's port.

7.17.1 Detailed Description

Functions in this group deal with injecting errors onto SpaceWire links.

7.17.2 Function Documentation

7.17.2.1 `int STAR_API_CC_CFG_PXI_injectError ( STAR_DEVICE_ID deviceId, unsigned char port, SPW_ERROR error )`

Immediately injects the specified error on a given device's port.

Note

This function is currently only compatible with [SpaceWire PXI Interface and PXI Interface Mk2](#) devices. No check is made by this function to ensure it is being used with a compatible device.

Parameters

<i>deviceId</i>	Identifier of the device to have the error injected on.
<i>port</i>	Port the error is to be injected on.
<i>error</i>	SpaceWire error to be injected.

Returns

1 if the error information was successfully sent to the device, else 0.

Added in version:

[STAR-System v3.0 Beta 4 - 13th November 2015](#)

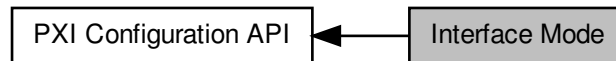
Supported devices:

[SpaceWire PXI Interface and PXI Interface Mk2](#), [SpaceWire PXI 12 Port Router](#) and [PXI 12 Port Router Mk2](#)

7.18 Interface Mode

Functions in this group deal with managing the device in Interface mode.

Collaboration diagram for Interface Mode:



Functions

- `int STAR_API_CC_CFG_PXI_enableInterfaceModeOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`
Selectively enables interface mode on a given port of a supported PXI device.
- `int STAR_API_CC_CFG_PXI_getInterfaceModeOnPortEnabled (STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int *pEnabled)`
Gets whether interface mode is enabled on a specific port of a supported PXI device.
- `int STAR_API_CC_CFG_PXI_disableInterfaceModeOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`
Selectively disables interface mode on a given port of a supported PXI device.
- `int STAR_API_CC_CFG_PXI_enableIdentifySourceOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`
Enables identification of the source port of a received packet on a given port, when in interface mode.
- `int STAR_API_CC_CFG_PXI_getIdentifySourceOnPortEnabled (STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int *pEnabled)`
Gets whether identify source is enabled for a specific port.
- `int STAR_API_CC_CFG_PXI_disableIdentifySourceOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`
Disables source port identification for a given port.
- `int STAR_API_CC_CFG_PXI_setPortRoutingAddress (STAR_DEVICE_ID deviceID, U8 portNum, U8 address)`
Sets the address a packet received on a given port should be routed to when interface mode is enabled.
- `int STAR_API_CC_CFG_PXI_getPortRoutingAddress (STAR_DEVICE_ID deviceID, U8 portNum, _Out_ U8 *pAddress)`
Gets the address a packet received on a given port should be routed to when interface mode is enabled.

7.18.1 Detailed Description

Functions in this group deal with managing the device in Interface mode.

7.18.2 Function Documentation

7.18.2.1 `int STAR_API_CC_CFG_PXI_disableIdentifySourceOnPort ( STAR_DEVICE_ID deviceID, U8 portNum )`

Disables source port identification for a given port.

Parameters

<i>deviceID</i>	Device to disable source port identification for a given port on.
<i>portNum</i>	Port to disable source identification on.

Returns

1 if source identification was successfully disabled, else 0.

Added in version:

STAR-System v3.3 - 20th December 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

7.18.2.2 int **STAR_API_CC_CFG_PXI_disableInterfaceModeOnPort** (**STAR_DEVICE_ID** *deviceID*, **U8** *portNum*)

Selectively disables interface mode on a given port of a supported PXI device.

When interface mode is disabled, the device operates in routing mode.

Parameters

<i>deviceID</i>	Device to disable interface mode for a port on.
<i>portNum</i>	Port to disable interface mode on.

Returns

1 if interface mode was successfully disabled, else 0.

Added in version:

STAR-System v3.3 - 20th December 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

7.18.2.3 int **STAR_API_CC_CFG_PXI_enableIdentifySourceOnPort** (**STAR_DEVICE_ID** *deviceID*, **U8** *portNum*)

Enables identification of the source port of a received packet on a given port, when in interface mode.

Interface mode (**CFG_MK2_enableInterfaceMode()**) and Identify Source (**CFG_MK2_enableIdentifySource()**) must be enabled globally for this to have an effect.

Parameters

<i>deviceID</i>	Device to enable source port identification for a given port on.
<i>portNum</i>	Port to enable source identification on.

Returns

1 if source identification was successfully enabled, else 0.

Added in version:

STAR-System v3.3 - 20th December 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

7.18.2.4 `int STAR_API_CC_CFG_PXI_enableInterfaceModeOnPort ( STAR_DEVICE_ID deviceID, U8 portNum )`

Selectively enables interface mode on a given port of a supported PXI device.

Interface mode must be enabled globally (`CFG_MK2_enableInterfaceMode()`) for interface mode on a port to be enabled.

Parameters

<i>deviceID</i>	Device to enable interface mode for a port on.
<i>portNum</i>	Port to enable interface mode on.

Returns

1 if interface mode was successfully enabled, else 0.

Added in version:

STAR-System v3.3 - 20th December 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

7.18.2.5 `int STAR_API_CC_CFG_PXI_getIdentifySourceOnPortEnabled ( STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int * pEnabled )`

Gets whether identify source is enabled for a specific port.

Parameters

<i>deviceID</i>	Device to check.
<i>portNum</i>	Port to check.
<i>pEnabled</i>	User supplied value that will be updated to 1 if identify source is enabled, else 0.

Returns

1 if the information could be obtained from the device, else 0.

Added in version:

STAR-System v3.3 - 20th December 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

7.18.2.6 `int STAR_API_CC_CFG_PXI_getInterfaceModeOnPortEnabled ( STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int * pEnabled )`

Gets whether interface mode is enabled on a specific port of a supported PXI device.

Parameters

<i>deviceID</i>	Device to get interface mode enabled for a port on.
-----------------	---

<i>portNum</i>	Port to get information for.
<i>pEnabled</i>	User supplied value that will be updated to 1 if interface mode is enabled, else 0.

Returns

1 if the information could be obtained from the device, else 0.

Added in version:

STAR-System v3.3 - 20th December 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

7.18.2.7 `int STAR_API_CC_CFG_PXI_getPortRoutingAddress ( STAR_DEVICE_ID deviceID, U8 portNum, _Out_ U8 * pAddress )`

Gets the address a packet received on a given port should be routed to when interface mode is enabled.

Note

This is an advanced feature and not necessary for normal usage.

Parameters

	<i>deviceID</i>	Device to get port routing address from.
	<i>portNum</i>	Port to get port routing address from.
<i>out</i>	<i>pAddress</i>	Pointer to value that will be updated to contain the address.

Returns

1 if the address could be obtained, else 0.

Added in version:

STAR-System v3.3 - 20th December 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

7.18.2.8 `int STAR_API_CC_CFG_PXI_setPortRoutingAddress ( STAR_DEVICE_ID deviceID, U8 portNum, U8 address )`

Sets the address a packet received on a given port should be routed to when interface mode is enabled.

Note

This is an advanced feature and not necessary for normal usage.

Parameters

<i>deviceID</i>	Device to have one of its port's port routing address changed.
-----------------	--

<i>portNum</i>	Port to have its port routing address modified.
<i>address</i>	New port routing address.

Returns

1 if the address could be set, else 0.

Added in version:

STAR-System v3.3 - 20th December 2016

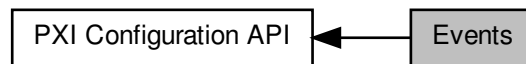
Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

7.19 Events

Functions in this group deal with enabling and disabling link events.

Collaboration diagram for Events:



Functions

- `int STAR_API_CC CFG_PXI_enableStateChangeEventsOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`
Selectively enables state change events on a given port.
- `int STAR_API_CC CFG_PXI_getStateChangeEventsOnPortEnabled (STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int *pEnabled)`
Gets whether state change events are enabled on a specific port.
- `int STAR_API_CC CFG_PXI_disableStateChangeEventsOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`
Selectively disables state change events on a given port.
- `int STAR_API_CC CFG_PXI_enableSpeedChangeEventsOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`
Selectively enables speed change events on a given port.
- `int STAR_API_CC CFG_PXI_getSpeedChangeEventsOnPortEnabled (STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int *pEnabled)`
Gets whether speed change events are enabled on a specific port.
- `int STAR_API_CC CFG_PXI_disableSpeedChangeEventsOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`
Selectively disables speed change events on a given port.
- `int STAR_API_CC CFG_PXI_enableTimeCodeEventsOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`
Selectively enables time-code notifications on a channel attached to a given port.
- `int STAR_API_CC CFG_PXI_getTimeCodeEventsOnPortEnabled (STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int *pEnabled)`
Gets whether time-code notifications are enabled on a specific port.
- `int STAR_API_CC CFG_PXI_disableTimeCodeEventsOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`
Selectively disables time-code notifications on a channel attached to a given port.

7.19.1 Detailed Description

Functions in this group deal with enabling and disabling link events.

7.19.2 Function Documentation

7.19.2.1 `int STAR_API_CC CFG_PXI_disableSpeedChangeEventsOnPort ( STAR_DEVICE_ID deviceID, U8 portNum )`

Selectively disables speed change events on a given port.

Disabling speed change events will result in speed change event traffic no longer being received on channel 0 when the link speed changes.

Parameters

<i>deviceID</i>	Device to disable speed change events for a port on.
<i>portNum</i>	Port to disable speed change events on.

Returns

1 if speed change events were successfully disabled, else 0.

Added in version:

STAR-System v3.3 - 20th December 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

7.19.2.2 int STAR_API_CC_CFG_PXI_disableStateChangeEventsOnPort (STAR_DEVICE_ID *deviceID*, U8 *portNum*)

Selectively disables state change events on a given port.

Disabling state change events will result in state change event traffic no longer being received on channel 0 when the link changes to running or disconnected.

Parameters

<i>deviceID</i>	Device to disable state change events for a port on.
<i>portNum</i>	Port to disable state change events on.

Returns

1 if state change events were successfully disabled, else 0.

Added in version:

STAR-System v3.3 - 20th December 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

7.19.2.3 int STAR_API_CC_CFG_PXI_disableTimeCodeEventsOnPort (STAR_DEVICE_ID *deviceID*, U8 *portNum*)

Selectively disables time-code notifications on a channel attached to a given port.

Note

This function is currently only compatible with [SpaceWire PXI Interface and PXI Interface Mk2](#) Interface devices from hardware version v1.2 edit 3 and with [SpaceWire PXI Interface and PXI Interface Mk2](#) Router devices from hardware version v1.2 edit 4. No check is made by this function to ensure it is being used with a compatible device.

Parameters

<i>deviceID</i>	Device to disable time-code notifications for a port on.
<i>portNum</i>	Port to disable time-code notifications on.

Returns

1 if time-code notifications were successfully disabled, else 0.

Added in version:

STAR-System v4.00 - 19th November 2019

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (all Mk2 versions and Mk1 version 1.2 edit 3 or greater),
SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2 (all Mk2 versions and Mk1 version 1.2 edit 4 or greater)

7.19.2.4 `int STAR_API_CC_CFG_PXI_enableSpeedChangeEventsOnPort ( STAR_DEVICE_ID deviceID, U8 portNum )`

Selectively enables speed change events on a given port.

Enabling speed change events will result in speed change event traffic being received on channel 0 when the link speed changes.

Parameters

<i>deviceID</i>	Device to enable speed change events for a port on.
<i>portNum</i>	Port to enable speed change events on.

Returns

1 if speed change events were successfully enabled, else 0.

Added in version:

STAR-System v3.3 - 20th December 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

7.19.2.5 `int STAR_API_CC_CFG_PXI_enableStateChangeEventsOnPort ( STAR_DEVICE_ID deviceID, U8 portNum )`

Selectively enables state change events on a given port.

Enabling state change events will result in state change event traffic being received on channel 0 when the link changes to running or disconnected.

Parameters

<i>deviceID</i>	Device to enable state change events for a port on.
<i>portNum</i>	Port to enable state change events on.

Returns

1 if state change events were successfully enabled, else 0.

Added in version:

STAR-System v3.3 - 20th December 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

7.19.2.6 int STAR_API_CC_CFG_PXI_enableTimeCodeEventsOnPort (STAR_DEVICE_ID *deviceId*, U8 *portNum*)

Selectively enables time-code notifications on a channel attached to a given port.

Note

This function is currently only compatible with SpaceWire PXI Interface and PXI Interface Mk2 Interface devices from hardware version v1.2 edit 3 and with SpaceWire PXI Interface and PXI Interface Mk2 Router devices from hardware version v1.2 edit 4. No check is made by this function to ensure it is being used with a compatible device.

Parameters

<i>deviceId</i>	Device to enable time-code notifications for a port on.
<i>portNum</i>	Port to enable time-code notifications on.

Returns

1 if time-codes were successfully enabled, else 0.

Added in version:

STAR-System v4.00 - 19th November 2019

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (all Mk2 versions and Mk1 version 1.2 edit 3 or greater), SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2 (all Mk2 versions and Mk1 version 1.2 edit 4 or greater)

7.19.2.7 int STAR_API_CC_CFG_PXI_getSpeedChangeEventsOnPortEnabled (STAR_DEVICE_ID *deviceId*, U8 *portNum*, _Out_ int * *pEnabled*)

Gets whether speed change events are enabled on a specific port.

Parameters

<i>deviceId</i>	Device to get whether speed change events are enabled for a port.
<i>portNum</i>	Port to get information for.
<i>pEnabled</i>	User supplied value that will be updated to 1 if speed change events are enabled, else 0.

Returns

1 if the information could be obtained from the device, else 0.

Added in version:

STAR-System v3.3 - 20th December 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

7.19.2.8 int STAR_API_CC_CFG_PXI_getStateChangeEventsOnPortEnabled (STAR_DEVICE_ID *deviceId*, U8 *portNum*, _Out_ int * *pEnabled*)

Gets whether state change events are enabled on a specific port.

Parameters

<i>deviceID</i>	Device to get whether state change events are enabled for a port.
<i>portNum</i>	Port to get information for.
<i>pEnabled</i>	User supplied value that will be updated to 1 if state change events are enabled, else 0.

Returns

1 if the information could be obtained from the device, else 0.

Added in version:

STAR-System v3.3 - 20th December 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

7.19.2.9 `int STAR_API_CC_CFG_PXI_getTimeCodeEventsOnPortEnabled ( STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int * pEnabled )`

Gets whether time-code notifications are enabled on a specific port.

Note

This function is currently only compatible with [SpaceWire PXI Interface and PXI Interface Mk2](#) Interface devices from hardware version v1.2 edit 3 and with [SpaceWire PXI Interface and PXI Interface Mk2 Router](#) devices from hardware version v1.2 edit 4. No check is made by this function to ensure it is being used with a compatible device.

Parameters

<i>deviceID</i>	Device to get whether time-code notifications are enabled for a port.
<i>portNum</i>	Port to get information for.
<i>pEnabled</i>	User supplied value that will be updated to 1 if time-code notifications on port are enabled, else 0.

Returns

1 if the information could be obtained from the device, else 0.

Added in version:

STAR-System v4.00 - 19th November 2019

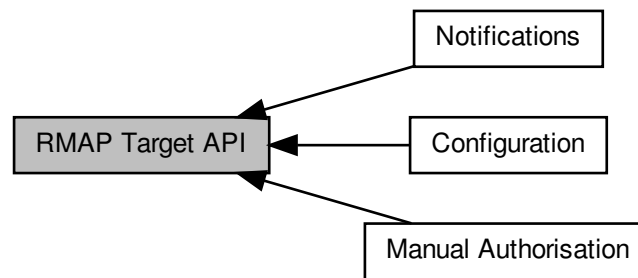
Supported devices:

[SpaceWire PXI Interface and PXI Interface Mk2](#) (all Mk2 versions and Mk1 version 1.2 edit 3 or greater), [SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2](#) (all Mk2 versions and Mk1 version 1.2 edit 4 or greater)

7.20 RMAP Target API

The RMAP Target API provides functions for configuring and controlling the RMAP targets within PXI Interface with RMAP Target devices.

Collaboration diagram for RMAP Target API:



Modules

- **Manual Authorisation**

Functions in this group deal with the manual RMAP target authorisation.

- **Configuration**

Functions in this group deal with setting and getting RMAP target configuration parameters.

- **Notifications**

Functions in this group deal with the RMAP target notification handling.

7.20.1 Detailed Description

The RMAP Target API provides functions for configuring and controlling the RMAP targets within PXI Interface with RMAP Target devices.

The RMAP targets can operate in automatic or manual authorisation mode. The API provides functions to set the automatic authorisation parameters, allowing ranges of valid values for the RMAP header parameters. When using the manual authorisation mode, the API provides functions to retrieve any pending commands and manually authorise or reject them.

In addition, there are functions to enable notifications when commands require authorisation or when commands are completed. Callback functions can be registered with the API and there are functions to start and stop receiving notifications.

More information about the RMAP target hardware can be found in the [SpaceWire PXI Interface and PXI Interface Mk2 Device User Manual](#).

Several examples of how to use the RMAP Target API are included in an application note entitled "Using the STAR-System RMAP Target API for the PXI Interface Board". The application note can be found in the following directory on Windows, assuming the default installation directory:

```
C:\Program Files\STAR-Dundee\STAR-System\application_notes
```

Or the following directory on Linux:

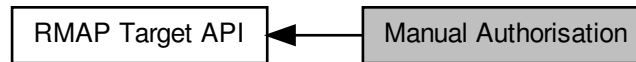
```
/usr/local/STAR-Dundee/STAR-System/application_notes
```

Additionally, there is a full code example include in the list of examples (see `rmap_target_example.c`).

7.21 Manual Authorisation

Functions in this group deal with the manual RMAP target authorisation.

Collaboration diagram for Manual Authorisation:



Data Structures

- struct [RmapCommandParameters](#)
Parameters of an RMAP command. [More...](#)

Enumerations

- enum [REJECTION_REASON](#) { [REJECTION_REASON_LOGICAL_ADDRESS](#), [REJECTION_REASON_KEY](#), [REJECTION_REASON_PROTOCOL_ID](#), [REJECTION_REASON_OTHER](#) }
Reason for rejecting a waiting RMAP command.

Functions

- int [STAR_API_CC RMAP_TARGET_PXI_IF_isAuthRequired](#) (STAR_DEVICE_ID deviceId, U32 target)
Gets whether or not there is an RMAP command waiting to be authorised.
- int [STAR_API_CC RMAP_TARGET_PXI_IF_getWaitingCommand](#) (STAR_DEVICE_ID deviceId, U32 target, [RmapCommandParameters](#) *pCommand)
Gets the parameters of the RMAP command waiting to be authorised.
- int [STAR_API_CC RMAP_TARGET_PXI_IF_authoriseWaitingCommand](#) (STAR_DEVICE_ID deviceId, U32 target)
Authorises a waiting RMAP command.
- int [STAR_API_CC RMAP_TARGET_PXI_IF_rejectWaitingCommand](#) (STAR_DEVICE_ID deviceId, U32 target, [REJECTION_REASON](#) reason)
Rejects a waiting RMAP command.

7.21.1 Detailed Description

Functions in this group deal with the manual RMAP target authorisation.

7.21.2 Data Structure Documentation

7.21.2.1 struct [RmapCommandParameters](#)

Parameters of an RMAP command.

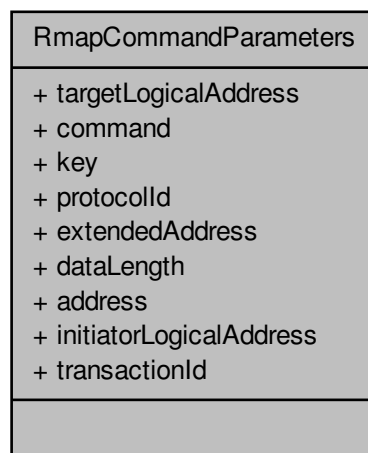
Added in version:

STAR-System v3.0 - 26th August 2016

Examples:

rmap_target_example.c.

Collaboration diagram for RmapCommandParameters:



Data Fields

U32	address	
U8	command	
U32	dataLength	
U8	extended↔ Address	
U8	initiatorLogical↔ Address	
U8	key	
U8	protocolId	
U8	targetLogical↔ Address	
U16	transactionId	

7.21.3 Enumeration Type Documentation

7.21.3.1 enum REJECTION_REASON

Reason for rejecting a waiting RMAP command.

Added in version:

STAR-System v3.0 - 26th August 2016

7.21.4 Function Documentation

7.21.4.1 `int STAR_API_CC RMAP_TARGET_PXI_IF_authoriseWaitingCommand ( STAR_DEVICE_ID deviceld, U32 target )`

Authorises a waiting RMAP command.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to authorise a waiting RMAP command for.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

Examples:

`rmap_target_example.c`.

7.21.4.2 `int STAR_API_CC RMAP_TARGET_PXI_IF_getWaitingCommand ( STAR_DEVICE_ID deviceld, U32 target, RmapCommandParameters * pCommand )`

Gets the parameters of the RMAP command waiting to be authorised.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to get the waiting RMAP command parameters from.
<i>pCommand</i>	Pointer to a value to be updated with the RMAP command parameters.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

Examples:

`rmap_target_example.c`.

7.21.4.3 `int STAR_API_CC RMAP_TARGET_PXI_IF_isAuthRequired ( STAR_DEVICE_ID deviceld, U32 target )`

Gets whether or not there is an RMAP command waiting to be authorised.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to check whether authorisation is required.

Returns

1 if the target has a command waiting to be authorised, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

7.21.4.4 `int STAR_API_CC RMAP_TARGET_PXI_IF_rejectWaitingCommand ( STAR_DEVICE_ID deviceld, U32 target, REJECTION_REASON reason )`

Rejects a waiting RMAP command.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to reject a waiting RMAP command for.
<i>reason</i>	Reason for rejecting the waiting RMAP command.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

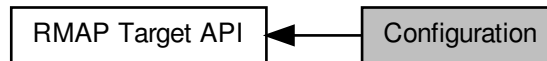
Examples:

`rmap_target_example.c`.

7.22 Configuration

Functions in this group deal with setting and getting RMAP target configuration parameters.

Collaboration diagram for Configuration:



Data Structures

- struct `TARGET_ENGINE`
Hardware capabilities. [More...](#)

Typedefs

- typedef U32 `TARGET_AUTH_CMD_MASK`
Mask describing which commands are authorised.

Enumerations

- enum `TARGET_AUTH_MODE` { `TARGET_AUTH_MODE_MANUAL` = 0, `TARGET_AUTH_MODE_AUTO←`
`MATIC` = 1 }
Method of authorising incoming RMAP commands.
- enum `TARGET_STATUS` {
 `TARGET_STATUS_SUCCESS` = 0, `TARGET_STATUS_GENERAL_ERROR` = 1, `TARGET_STATUS_H←`
 `EADER_EOP_ERROR` = 2, `TARGET_STATUS_HEADER_EEP_ERROR` = 3,
 `TARGET_STATUS_PID_ERROR` = 4, `TARGET_STATUS_REPLY_ERROR` = 5, `TARGET_STATUS_HE←`
 `ADER_CRC_ERROR` = 6, `TARGET_STATUS_HEADER_EEP_AFTER_CRC` = 7,
 `TARGET_STATUS_PACKET_TYPE_ERROR` = 8, `TARGET_STATUS_COMMAND_TYPE_ERROR` = 9,
 `TARGET_STATUS_RMW_DATALEN_ERROR` = 10, `TARGET_STATUS_CARGO_TOO_LARGE` = 11,
 `TARGET_STATUS_KEY_ERROR` = 12, `TARGET_STATUS_LOGICAL_ADDR_ERROR` = 13, `TARGET←`
 `_STATUS_AUTHORISED_ERROR` = 14, `TARGET_STATUS_VERIFY_BUFFER_OVERRUN` = 15,
 `TARGET_STATUS_DATA_CRC_ERROR` = 16, `TARGET_STATUS_BUS_ERROR` = 17, `TARGET_STA←`
 `TUS_DATA_EOP_ERROR` = 18, `TARGET_STATUS_DATA_EEP_ERROR` = 19,
 `TARGET_STATUS_DATA_EEP_AFTER_CRC` = 20 }
Status of the RMAP target.
- enum `TARGET_AUTH_CMD` {
 `TARGET_AUTH_CMD_READ` = (1 << 0), `TARGET_AUTH_CMD_READ_INCR` = (1 << 1), `TARGET←`
 `_AUTH_CMD_READ_MODIFY_WRITE` = (1 << 2), `TARGET_AUTH_CMD_WRITE` = (1 << 3),
 `TARGET_AUTH_CMD_WRITE_INCR` = (1 << 4), `TARGET_AUTH_CMD_WRITE_REPLY` = (1 << 5),
 `TARGET_AUTH_CMD_WRITE_INCR_REPLY` = (1 << 6), `TARGET_AUTH_CMD_WRITE_VERIFY` = (1
 << 7),
 `TARGET_AUTH_CMD_WRITE_VERIFY_INCR` = (1 << 8), `TARGET_AUTH_CMD_WRITE_VERIFY_R←`
 `EPLY` = (1 << 9), `TARGET_AUTH_CMD_WRITE_VERIFY_INCR_REPLY` = (1 << 10), `TARGET_AUT←`
 `H_CMD_ALL` = (0x7FF),
 `TARGET_AUTH_CMD_NONE` = (0) }

Command types which are authorised by the target.

- enum IF_MODE { IF_MODE_RMAP_DISABLED = 0, IF_MODE_RMAP_ENABLED = 1 }

Control whether or not the RMAP target is enabled for a port.

Functions

- int STAR_API_CC RMAP_TARGET_PXI_IF_getStatus (STAR_DEVICE_ID deviceId, U32 target, TARGET_STATUS *pStatus)
Gets the RMAP target status for a specific target.
- int STAR_API_CC RMAP_TARGET_PXI_IF_setAddressOffset (STAR_DEVICE_ID deviceId, U32 target, U32 offset)
Sets the address offset for a specific target.
- int STAR_API_CC RMAP_TARGET_PXI_IF_getAddressOffset (STAR_DEVICE_ID deviceId, U32 target, U32 *pOffset)
Gets the address offset from a specific target.
- int STAR_API_CC RMAP_TARGET_PXI_IF_setAuthControlMode (STAR_DEVICE_ID deviceId, U32 target, TARGET_AUTH_MODE mode)
Sets the authorisation control mode for a specific target.
- int STAR_API_CC RMAP_TARGET_PXI_IF_getAuthControlMode (STAR_DEVICE_ID deviceId, U32 target, TARGET_AUTH_MODE *pMode)
Gets the authorisation control mode from a specific target.
- int STAR_API_CC RMAP_TARGET_PXI_IF_setAuthLogicalAddressRange (STAR_DEVICE_ID deviceId, U32 target, U8 lowest, U8 highest)
Sets the authorised target logical address range for a specific target.
- int STAR_API_CC RMAP_TARGET_PXI_IF_getAuthLogicalAddressRange (STAR_DEVICE_ID deviceId, U32 target, U8 *pLowest, U8 *pHighest)
Gets the authorised target logical address range from a specific target.
- int STAR_API_CC RMAP_TARGET_PXI_IF_setAuthProtocolId (STAR_DEVICE_ID deviceId, U32 target, U8 protocolId)
Sets the authorised protocol ID for a specific target.
- int STAR_API_CC RMAP_TARGET_PXI_IF_getAuthProtocolId (STAR_DEVICE_ID deviceId, U32 target, U8 *pProtocolId)
Gets the authorised protocol ID from a specific target.
- int STAR_API_CC RMAP_TARGET_PXI_IF_setAuthCommands (STAR_DEVICE_ID deviceId, U32 target, TARGET_AUTH_CMD_MASK commands)
Sets the authorised RMAP commands for a specific target.
- int STAR_API_CC RMAP_TARGET_PXI_IF_getAuthCommands (STAR_DEVICE_ID deviceId, U32 target, TARGET_AUTH_CMD_MASK *pCommands)
Gets the authorised RMAP commands from a specific target.
- int STAR_API_CC RMAP_TARGET_PXI_IF_setAuthKeyRange (STAR_DEVICE_ID deviceId, U32 target, U8 lowest, U8 highest)
Sets the authorised key range for a specific target.
- int STAR_API_CC RMAP_TARGET_PXI_IF_getAuthKeyRange (STAR_DEVICE_ID deviceId, U32 target, U8 *pLowest, U8 *pHighest)
Gets the authorised key range from a specific target.
- int STAR_API_CC RMAP_TARGET_PXI_IF_setAuthMemoryAddressRange (STAR_DEVICE_ID deviceId, U32 target, U32 lowerBoundary, U32 upperBoundary)
Sets the authorised memory address range for a specific target.
- int STAR_API_CC RMAP_TARGET_PXI_IF_getAuthMemoryAddressRange (STAR_DEVICE_ID deviceId, U32 target, U32 *pLowerBoundary, U32 *pUpperBoundary)
Gets the authorised memory address range from a specific target.
- int STAR_API_CC RMAP_TARGET_PXI_IF_setInterfaceMode (STAR_DEVICE_ID deviceId, U32 port, IF_MODE mode)

Sets the RMAP interface mode for a specific port.

- int `STAR_API_CC RMAP_TARGET_PXI_IF_getInterfaceMode` (STAR_DEVICE_ID deviceId, U32 port, IF↔_MODE *pMode)

Gets the RMAP interface mode for a specific port.

- int `STAR_API_CC RMAP_TARGET_PXI_IF_readMemory` (STAR_DEVICE_ID deviceId, U32 target, U32 address, U32 length, unsigned char *pBuffer)

Reads an area of RMAP target memory into a user-supplied buffer.

- int `STAR_API_CC RMAP_TARGET_PXI_IF_writeMemory` (STAR_DEVICE_ID deviceId, U32 target, U32 address, U32 length, const unsigned char *pBuffer)

Writes an area of RMAP target memory from a user-supplied buffer.

7.22.1 Detailed Description

Functions in this group deal with setting and getting RMAP target configuration parameters.

7.22.2 Data Structure Documentation

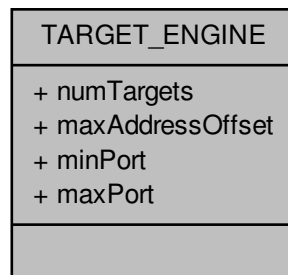
7.22.2.1 struct TARGET_ENGINE

Hardware capabilities.

Added in version:

STAR-System v3.0 - 26th August 2016

Collaboration diagram for TARGET_ENGINE:



Data Fields

U32	maxAddress↔Offset	
U32	maxPort	
U32	minPort	

U32	numTargets	
-----	------------	--

7.22.3 Typedef Documentation

7.22.3.1 typedef U32 TARGET_AUTH_CMD_MASK

Mask describing which commands are authorised.

Added in version:

STAR-System v3.0 - 26th August 2016

7.22.4 Enumeration Type Documentation

7.22.4.1 enum IF_MODE

Control whether or not the RMAP target is enabled for a port.

Added in version:

STAR-System v3.0 - 26th August 2016

Enumerator

IF_MODE_RMAP_DISABLED When RMAP is disabled, the port acts as a normal SpaceWire interface.

IF_MODE_RMAP_ENABLED When RMAP is enabled, the port acts as an RMAP target.

7.22.4.2 enum TARGET_AUTH_CMD

Command types which are authorised by the target.

Added in version:

STAR-System v3.0 - 26th August 2016

7.22.4.3 enum TARGET_AUTH_MODE

Method of authorising incoming RMAP commands.

Added in version:

STAR-System v3.0 - 26th August 2016

Enumerator

TARGET_AUTH_MODE_MANUAL When set to manual, each incoming command must be authorised or rejected by software.

TARGET_AUTH_MODE_AUTOMATIC When set to automatic, each incoming command is automatically authorised or rejected based on the authorised parameter values.

7.22.4.4 enum TARGET_STATUS

Status of the RMAP target.

Added in version:

STAR-System v3.0 - 26th August 2016

7.22.5 Function Documentation

7.22.5.1 `int STAR_API_CC RMAP_TARGET_PXI_IF_getAddressOffset ( STAR_DEVICE_ID deviceld, U32 target, U32 * pOffset )`

Gets the address offset from a specific target.

The address offset determines where the target's memory region begins.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to get address offset from.
<i>pOffset</i>	Pointer to a value to be updated with the target's address offset.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

7.22.5.2 `int STAR_API_CC RMAP_TARGET_PXI_IF_getAuthCommands ( STAR_DEVICE_ID deviceld, U32 target, TARGET_AUTH_CMD_MASK * pCommands )`

Gets the authorised RMAP commands from a specific target.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to get authorised RMAP commands from.
<i>pCommands</i>	Pointer to a value to be updated with the target's authorised commands.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

7.22.5.3 `int STAR_API_CC RMAP_TARGET_PXI_IF_getAuthControlMode ( STAR_DEVICE_ID deviceld, U32 target, TARGET_AUTH_MODE * pMode )`

Gets the authorisation control mode from a specific target.

Authorisation can be set to either TARGET_AUTH_MODE_MANUAL or TARGET_AUTH_MODE_AUTOMATIC. When set to TARGET_AUTH_MODE_MANUAL, the authorisation of RMAP commands is done manually by the user application. When set to TARGET_AUTH_MODE_AUTOMATIC, the authorisation of RMAP commands is done automatically using the expected parameter fields.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to get authorisation mode from.
<i>pMode</i>	Pointer to a value to be updated with the target's authorisation mode.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

7.22.5.4 `int STAR_API_CC RMAP_TARGET_PXI_IF_getAuthKeyRange ( STAR_DEVICE_ID deviceld, U32 target, U8 * pLowest, U8 * pHighest )`

Gets the authorised key range from a specific target.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to get authorised key range from.
<i>pLowest</i>	Pointer to a value to be updated with the lowest key authorised.
<i>pHighest</i>	Pointer to a value to be updated with the highest key authorised.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

7.22.5.5 `int STAR_API_CC RMAP_TARGET_PXI_IF_getAuthLogicalAddressRange ( STAR_DEVICE_ID deviceld, U32 target, U8 * pLowest, U8 * pHighest )`

Gets the authorised target logical address range from a specific target.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to get authorised logical address range from.
<i>pLowest</i>	Pointer to a value to be updated with the lowest target logical address authorised.
<i>pHighest</i>	Pointer to a value to be updated with the highest target logical address authorised.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

7.22.5.6 `int STAR_API_CC RMAP_TARGET_PXI_IF_getAuthMemoryAddressRange ( STAR_DEVICE_ID deviceld, U32 target, U32 * pLowerBoundary, U32 * pUpperBoundary )`

Gets the authorised memory address range from a specific target.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to get authorised memory address range from.
<i>pLowerBoundary</i>	Pointer to a value to be updated with the lowest memory address authorised.
<i>pUpperBoundary</i>	Pointer to a value to be updated with the highest memory address authorised.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

7.22.5.7 `int STAR_API_CC RMAP_TARGET_PXI_IF_getAuthProtocolId ( STAR_DEVICE_ID deviceld, U32 target, U8 * pProtocolId )`

Gets the authorised protocol ID from a specific target.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to get authorised protocol ID from.
<i>pProtocolId</i>	Pointer to a value to be updated with the target's authorised protocol ID.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

7.22.5.8 `int STAR_API_CC RMAP_TARGET_PXI_IF_getInterfaceMode ( STAR_DEVICE_ID deviceld, U32 port, IF_MODE * pMode )`

Gets the RMAP interface mode for a specific port.

Parameters

<i>deviceld</i>	Device containing the port.
<i>port</i>	Number of the port (6 - 9).
<i>pMode</i>	Pointer to a value to be updated with the port's interface mode.

Returns

1 if the port's RMAP interface mode was successfully modified, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

7.22.5.9 `int STAR_API_CC RMAP_TARGET_PXI_IF_getStatus ( STAR_DEVICE_ID deviceld, U32 target, TARGET_STATUS * pStatus )`

Gets the RMAP target status for a specific target.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to get status from.
<i>pStatus</i>	Pointer to a value to be updated with the target's status.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

7.22.5.10 `int STAR_API_CC RMAP_TARGET_PXI_IF_readMemory ( STAR_DEVICE_ID deviceld, U32 target, U32 address, U32 length, unsigned char * pBuffer )`

Reads an area of RMAP target memory into a user-supplied buffer.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to read memory from.
<i>address</i>	The address in the target to read from.
<i>length</i>	The length to read.
<i>pBuffer</i>	Pointer to a buffer to read memory into.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

7.22.5.11 int STAR_API_CC RMAP_TARGET_PXI_IF_setAddressOffset (STAR_DEVICE_ID *deviceld*, U32 *target*, U32 *offset*)

Sets the address offset for a specific target.

The address offset determines where the target's memory region begins.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to set address offset for.
<i>offset</i>	Address offset in MBytes.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

Examples:

`rmap_target_example.c`.

7.22.5.12 int STAR_API_CC RMAP_TARGET_PXI_IF_setAuthCommands (STAR_DEVICE_ID *deviceld*, U32 *target*, TARGET_AUTH_CMD_MASK *commands*)

Sets the authorised RMAP commands for a specific target.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to set authorised RMAP commands for.
<i>commands</i>	Authorised commands mask.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

Examples:

rmap_target_example.c.

7.22.5.13 int **STAR_API_CC** RMAP_TARGET_PXI_IF_setAuthControlMode (STAR_DEVICE_ID *deviceld*, U32 *target*, TARGET_AUTH_MODE *mode*)

Sets the authorisation control mode for a specific target.

Authorisation can be set to either TARGET_AUTH_MODE_MANUAL or TARGET_AUTH_MODE_AUTOMATIC. When set to TARGET_AUTH_MODE_MANUAL, the authorisation of RMAP commands is done manually by the user application. When set to TARGET_AUTH_MODE_AUTOMATIC, the authorisation of RMAP commands is done automatically using the expected parameter fields.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to set authorisation mode for.
<i>mode</i>	Authorisation mode.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

Examples:

rmap_target_example.c.

7.22.5.14 int **STAR_API_CC** RMAP_TARGET_PXI_IF_setAuthKeyRange (STAR_DEVICE_ID *deviceld*, U32 *target*, U8 *lowest*, U8 *highest*)

Sets the authorised key range for a specific target.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to set authorised key range for.
<i>lowest</i>	Lowest key authorised.
<i>highest</i>	Highest key authorised.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

Examples:

[rmap_target_example.c](#).

7.22.5.15 `int STAR_API_CC RMAP_TARGET_PXI_IF_setAuthLogicalAddressRange ( STAR_DEVICE_ID deviceld, U32 target, U8 lowest, U8 highest )`

Sets the authorised target logical address range for a specific target.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to set authorised logical address range for.
<i>lowest</i>	Lowest target logical address authorised.
<i>highest</i>	Highest target logical address authorised.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

Examples:

[rmap_target_example.c](#).

7.22.5.16 `int STAR_API_CC RMAP_TARGET_PXI_IF_setAuthMemoryAddressRange ( STAR_DEVICE_ID deviceld, U32 target, U32 lowerBoundary, U32 upperBoundary )`

Sets the authorised memory address range for a specific target.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to set authorised memory address range for.
<i>lowerBoundary</i>	Lowest memory address authorised.
<i>upperBoundary</i>	Highest memory address authorised.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

Examples:

[rmap_target_example.c](#).

7.22.5.17 `int STAR_API_CC RMAP_TARGET_PXI_IF_setAuthProtocolId ( STAR_DEVICE_ID deviceld, U32 target, U8 protocolld )`

Sets the authorised protocol ID for a specific target.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to set authorised protocol ID for.
<i>protocolld</i>	Authorised protocol ID.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

Examples:

`rmap_target_example.c`.

7.22.5.18 `int STAR_API_CC RMAP_TARGET_PXI_IF_setInterfaceMode ( STAR_DEVICE_ID deviceld, U32 port, IF_MODE mode )`

Sets the RMAP interface mode for a specific port.

Parameters

<i>deviceld</i>	Device containing the port.
<i>port</i>	Number of the port (6 - 9).
<i>mode</i>	Interface mode of the port.

Returns

1 if the port's RMAP interface mode was successfully modified, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

Examples:

`rmap_target_example.c`.

7.22.5.19 `int STAR_API_CC RMAP_TARGET_PXI_IF_writeMemory ( STAR_DEVICE_ID deviceld, U32 target, U32 address, U32 length, const unsigned char * pBuffer )`

Writes an area of RMAP target memory from a user-supplied buffer.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to write memory to.
<i>address</i>	The address in the target to write to.
<i>length</i>	The length to write.
<i>pBuffer</i>	Pointer to a buffer to write memory from.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

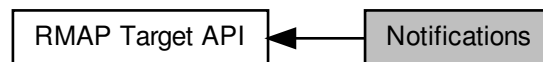
Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

7.23 Notifications

Functions in this group deal with the RMAP target notification handling.

Collaboration diagram for Notifications:



Typedefs

- typedef U32 NOTIF_TYPE_MASK
Mask describing which notifications are enabled.
- typedef void(STAR_API_CC * NotifListenerFunc) (U32 target, void *pNotif)
Notification listener function pointer: target is the index of the target which generated the notification.
- typedef void(STAR_API_CC * NotifListenerWithContextFunc) (STAR_DEVICE_ID deviceId, U32 target, void *pNotif, void *pContext)
Notification listener with context function pointer: deviceId is the ID of the device that issued the notification target is the index of the target which generated the notification.

Enumerations

- enum NOTIF_TYPE { NOTIF_TYPE_AUTH_REQUEST = (1 << 0), NOTIF_TYPE_CMD_COMPLETE = (1 << 2), NOTIF_TYPE_ALL = (0x5), NOTIF_TYPE_NONE = (0) }
Types of notifications.

Functions

- int STAR_API_CC RMAP_TARGET_PXI_IF_setEnabledNotifications (STAR_DEVICE_ID deviceId, U32 target, NOTIF_TYPE_MASK notifications)
Set the notifications that are enabled for a target.
- int STAR_API_CC RMAP_TARGET_PXI_IF_getEnabledNotifications (STAR_DEVICE_ID deviceId, U32 target, NOTIF_TYPE_MASK *pNotifications)
Get the notifications that are enabled for a target.
- int STAR_API_CC RMAP_TARGET_PXI_IF_registerNotificationListener (STAR_DEVICE_ID deviceId, U32 target, NOTIF_TYPE type, NotifListenerFunc listenerFunc)
Register a notification listener function.
- int STAR_API_CC RMAP_TARGET_PXI_IF_registerNotificationListenerWithContext (STAR_DEVICE_ID deviceId, U32 target, NOTIF_TYPE type, NotifListenerWithContextFunc listenerFunc, void *pContext)
Register a notification listener with context function.
- int STAR_API_CC RMAP_TARGET_PXI_IF_unregisterNotificationListener (STAR_DEVICE_ID deviceId, U32 target, NOTIF_TYPE type)
Unregister a notification listener function.
- int STAR_API_CC RMAP_TARGET_PXI_IF_unregisterNotificationListenerWithContext (STAR_DEVICE_ID deviceId, U32 target, NOTIF_TYPE type)
Unregister a notification listener with context function.

- int `STAR_API_CC RMAP_TARGET_PXI_IF_startReceivingNotifications (STAR_DEVICE_ID deviceId)`
Start receiving notifications for a device.
- int `STAR_API_CC RMAP_TARGET_PXI_IF_stopReceivingNotifications (STAR_DEVICE_ID deviceId)`
Stop receiving notifications for a device.

7.23.1 Detailed Description

Functions in this group deal with the RMAP target notification handling.

7.23.2 Typedef Documentation

7.23.2.1 typedef U32 NOTIF_TYPE_MASK

Mask describing which notifications are enabled.

Added in version:

STAR-System v3.0 - 26th August 2016

7.23.2.2 typedef void(STAR_API_CC * NotifListenerFunc) (U32 target, void *pNotif)

Notification listener function pointer: target is the index of the target which generated the notification.

pNotif is a pointer to the notification packet which has one of two formats depending on the notification type.

Authorisation request notification:

Byte [0] : 0xFE
 Byte [1] : 0xF0
 Byte [2] : 0x10 (Notification type)
 Byte [3] : Notification packet cargo length
 Byte [4] : Target index
 Byte [5] : Current time-code at time of event

Command complete notification:

Byte [0] : 0xFE
 Byte [1] : 0xF0
 Byte [2] : 0x12 (Notification type)
 Byte [3] : Notification packet cargo length
 Byte [4] : Target index
 Byte [5] : Current time-code at time of event
 Byte [6] : Target logical address
 Byte [7] : Protocol ID
 Byte [8] : Command
 Byte [9] : Key
 Byte [10] : Initiator logical address
 Byte [11 - 12] : Transaction ID (MSB first)
 Byte [13] : Extended address
 Byte [14 - 17] : Address (MSB first)
 Byte [18 - 20] : Data length (MSB first)
 Byte [21] : Status

Added in version:

STAR-System v3.0 - 26th August 2016

7.23.2.3 `typedef void(STAR_API_CC * NotifListenerWithContextFunc) (STAR_DEVICE_ID deviceld, U32 target, void *pNotif, void *pContext)`

Notification listener with context function pointer: deviceld is the ID of the device that issued the notification target is the index of the target which generated the notification.

pNotif is a pointer to the notification packet which has one of two formats depending on the notification type. pContext is a pointer to a user supplied context variable

Authorisation request notification:

Byte [0] : 0xFE
 Byte [1] : 0xF0
 Byte [2] : 0x10 (Notification type)
 Byte [3] : Notification packet cargo length
 Byte [4] : Target index
 Byte [5] : Current time-code at time of event

Command complete notification:

Byte [0] : 0xFE
 Byte [1] : 0xF0
 Byte [2] : 0x12 (Notification type)
 Byte [3] : Notification packet cargo length
 Byte [4] : Target index
 Byte [5] : Current time-code at time of event
 Byte [6] : Target logical address
 Byte [7] : Protocol ID
 Byte [8] : Command
 Byte [9] : Key
 Byte [10] : Initiator logical address
 Byte [11 - 12] : Transaction ID (MSB first)
 Byte [13] : Extended address
 Byte [14 - 17] : Address (MSB first)
 Byte [18 - 20] : Data length (MSB first)
 Byte [21] : Status

Added in version:

STAR-System v3.0 - 26th August 2016

7.23.3 Enumeration Type Documentation

7.23.3.1 enum NOTIF_TYPE

Types of notifications.

Added in version:

STAR-System v3.0 - 26th August 2016

Enumerator

NOTIF_TYPE_AUTH_REQUEST Authorisation request notification.

NOTIF_TYPE_CMD_COMPLETE Command complete notification.

NOTIF_TYPE_ALL All notifications.

NOTIF_TYPE_NONE No notifications.

7.23.4 Function Documentation

7.23.4.1 `int STAR_API_CC RMAP_TARGET_PXI_IF_getEnabledNotifications ( STAR_DEVICE_ID deviceld, U32 target, NOTIF_TYPE_MASK * pNotifications )`

Get the notifications that are enabled for a target.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to get enabled notifications from.
<i>pNotifications</i>	Pointer to a value to be updated with the enabled notifications.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

7.23.4.2 **int STAR_API_CC RMAP_TARGET_PXI_IF_registerNotificationListener (STAR_DEVICE_ID *deviceld*, U32 *target*, NOTIF_TYPE *type*, NotifListenerFunc *listenerFunc*)**

Register a notification listener function.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to register a notification listener function for.
<i>type</i>	Type of notification to register the listener function for.
<i>listenerFunc</i>	Pointer to a notification listener function.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

7.23.4.3 **int STAR_API_CC RMAP_TARGET_PXI_IF_registerNotificationListenerWithContext (STAR_DEVICE_ID *deviceld*, U32 *target*, NOTIF_TYPE *type*, NotifListenerWithContextFunc *listenerFunc*, void * *pContext*)**

Register a notification listener with context function.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to register a notification listener function for.
<i>type</i>	Type of notification to register the listener function for.
<i>listenerFunc</i>	Pointer to a notification listener with context function.
<i>pContext</i>	Pointer to a user-supplied context.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.4 - 9th March 2017

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

Examples:

rmap_target_example.c.

7.23.4.4 `int STAR_API_CC RMAP_TARGET_PXI_IF_setEnabledNotifications ( STAR_DEVICE_ID deviceld, U32 target, NOTIF_TYPE_MASK notifications )`

Set the notifications that are enabled for a target.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to set enabled notifications for.
<i>notifications</i>	Enabled notifications mask

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

Examples:

rmap_target_example.c.

7.23.4.5 `int STAR_API_CC RMAP_TARGET_PXI_IF_startReceivingNotifications ( STAR_DEVICE_ID deviceld )`

Start receiving notifications for a device.

Parameters

<i>deviceld</i>	Device to start receiving notifications for.
-----------------	--

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

Examples:

[rmap_target_example.c](#).

7.23.4.6 `int STAR_API_CC RMAP_TARGET_PXI_IF_stopReceivingNotifications ( STAR_DEVICE_ID deviceld )`

Stop receiving notifications for a device.

Parameters

<i>deviceld</i>	Device to stop receiving notifications for.
-----------------	---

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

Examples:

[rmap_target_example.c](#).

7.23.4.7 `int STAR_API_CC RMAP_TARGET_PXI_IF_unregisterNotificationListener ( STAR_DEVICE_ID deviceld, U32 target, NOTIF_TYPE type )`

Unregister a notification listener function.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to unregister a notification listener with context function for.
<i>type</i>	Type of notification to unregister the listener with context function for.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

Examples:

[rmap_target_example.c](#).

7.23.4.8 `int STAR_API_CC RMAP_TARGET_PXI_IF_unregisterNotificationListenerWithContext ( STAR_DEVICE_ID deviceld, U32 target, NOTIF_TYPE type )`

Unregister a notification listener with context function.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to unregister a notification listener with context function for.
<i>type</i>	Type of notification to unregister the listener with context function for.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.4 - 9th March 2017

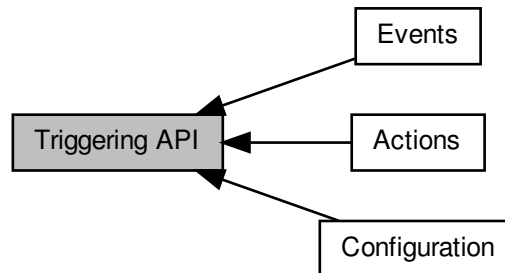
Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

7.24 Triggering API

The Triggering API provides functions for configuring and controlling the triggering capabilities of the Brick Mk3, PXI Interface, PXI Interface Mk2, PXI Router, PXI Router Mk2 and PCIe devices.

Collaboration diagram for Triggering API:



Modules

- **Events**

Functions in this group deal with setting and getting trigger input events.

- **Actions**

Functions in this group deal with setting and getting trigger output actions.

- **Configuration**

Functions in this group deal with configuring the different triggering subsystems.

7.24.1 Detailed Description

The Triggering API provides functions for configuring and controlling the triggering capabilities of the Brick Mk3, PXI Interface, PXI Interface Mk2, PXI Router, PXI Router Mk2 and PCIe devices.

The Triggering API allows events that occur on triggering resources such as external trigger ports, counters, time-code interfaces or SpaceWire ports to cause actions. For example, the triggers could be configured to transmit a packet on a port whenever an external trigger detects the rising edge of an input signal.

The three main concepts of the Triggering API are:

- **Input Events:** these are events that occur on triggering resources.
- **Output Actions:** these are actions that are performed by triggering resources.
- **Internal Triggers:** these are used to control the input events that cause output actions.

These concepts are described in the following sections.

7.24.2 Input Events

An input event is an event that occurs on one of the triggering resources. These events may be configured to set one or more internal triggers. Each triggering resource has one or more events that can occur. For example, SpaceWire ports have events for when they detect certain characters, errors or receive/transmit time-codes.

The following tables provide a description of the input events for each triggering resource.

External Trigger Input Events

Input Event	Description	Supported Devices			
		Brick Mk3	PXI Interface	PXI Router	PCle Interface
EXT_TRIGGER_EVENT_IN	This event occurs when the external trigger receives a signal.	Yes	Yes	No	No

Counter Input Events

Input Event	Description	Supported Devices			
		Brick Mk3	PXI Interface	PXI Router	PCle Interface
COUNTER_EVENT_COUNT	This event occurs when the counter decrements.	Yes	Yes	Yes	Yes
COUNTER_EVENT_ZERO_SINGLE	This event occurs when the counter reaches zero.	Yes	Yes	Yes	Yes
COUNTER_EVENT_ZERO	This event is active while the counter is equal to zero.	Yes	Yes	Yes	Yes
COUNTER_EVENT_RELOAD	This event occurs when the counter reloads.	Yes	Yes	Yes	Yes

SpaceWire Port Input Events

Input Event	Description	Supported Devices			
		Brick Mk3	PXI Interface	PXI Router	PCle Interface
PORT_EVENT_RX_SOP	This event occurs when the port receives a Start of Packet (SOP).	Yes	Yes	Yes	Yes
PORT_EVENT_RX_EOP	This event occurs when the port receives an End of Packet (EOP).	Yes	Yes	Yes	Yes

PORT_EVE↔ NT_RX_EEP	This event occurs when the port receives an Error End of Packet (EEP).	Yes	Yes	Yes	Yes
PORT_EVE↔ NT_TX_SOP	This event occurs when the port transmits an SOP.	Yes	Yes	Yes	Yes
PORT_EVE↔ NT_TX_EOP	This event occurs when the port transmits an EOP.	Yes	Yes	Yes	Yes
PORT_EVE↔ NT_TX_PKT↔ _PENDING	This event is active when a port has a packet queued for transmission.	Yes	Yes	Yes	Yes
PORT_EVE↔ NT_RUNNING	This event is active when the link attached to a port is running.	Yes	Yes	Yes	Yes
PORT_EVE↔ NT_PARITY↔ _ERROR	This event occurs when a port detects a parity error.	Yes	Yes	Yes	Yes
PORT_EVE↔ NT_ESCAPE↔ _ERROR	This event occurs when a port detects an escape error.	Yes	Yes	Yes	Yes
PORT_EVE↔ NT_CREDIT↔ _ERROR	This event occurs when a port detects a credit error.	Yes	Yes	Yes	Yes
PORT_EVE↔ NT_DISCON↔ NECT	This event occurs when the link attached to a port disconnects.	Yes	Yes	Yes	Yes
PORT_EVE↔ NT_RX_TIM↔ ECODE	This event occurs when a port receives a time-code.	Yes	Yes	Yes	Yes

PORT_EVENT_TX_TIMECODE	This event occurs when a port transmits a time-code.	Yes	Yes	Yes	No
------------------------	--	-----	-----	-----	----

Time-Code Interface Input Events

Input Event	Description	Supported Devices			
		Brick Mk3	PXI Interface	PXI Router	PCIe Interface
TIME_CODE_EVENT_TRIGGER	This event occurs when a time-code interface receives its next valid time-code.	Yes	Yes	Yes	No

7.24.3 Output Actions

An output action is an action that is performed by a triggering resource. The actions are caused by the setting of an internal trigger, if configured to do so. Each triggering resource has one or more actions that can be performed. For example, a SpaceWire port can inject errors or transmit packets, if configured in packet transmit mode.

The following tables provide a description of the output actions for each triggering resource.

External Trigger Output Actions

Output Action	Description	Supported Devices			
		Brick Mk3	PXI Interface	PXI Router	PCIe Interface
EXT_TRIGGER_ACTION_OUT	This action causes an external trigger to output a signal.	Yes	Yes	No	No

Counter Output Actions

Output Action	Description	Supported Devices			
		Brick Mk3	PXI Interface	PXI Router	PCIe Interface
COUNTER_ACTION_COUNT	This action causes a counter to decrement by one.	Yes	Yes	Yes	Yes
COUNTER_ACTION_RELOAD	This action causes a counter to be set to its configured reload value.	Yes	Yes	Yes	Yes

COUNTER_ ACTION_START	This action causes a counter to be start, if start mode is enabled.	Yes	Yes	Yes	Yes
COUNTER_ ACTION_STOP	This action causes a counter to be stopped, if start-stop mode is enabled.	Yes	Yes	Yes	Yes

SpaceWire Port Output Actions

Output Action	Description	Supported Devices			
		Brick Mk3	PXI Interface	PXI Router	PCIe Interface
PORT_ACTION_TRANSMIT_PKT	This action causes a port to transmit a queued packet, if transmit packet mode is enabled for the port.	Yes	Yes	Yes	Yes
PORT_ACTION_DISCONNECT	This action causes the link attached to a port to disconnect.	Yes	Yes	Yes	Yes
PORT_ACTION_PARITY_ERROR	This action causes a port to inject a parity error.	Yes	Yes	Yes	Yes
PORT_ACTION_ESCAPE_ERROR	This action causes a port to inject an escape error.	Yes	Yes	Yes	Yes
PORT_ACTION_INSERT_FCT	This action causes a port to insert a Flow Control Token (FCT).	Yes	Yes	Yes	Yes
PORT_ACTION_SUPPRESS_FCT	This action causes a port to suppress an FCT.	Yes	Yes	Yes	Yes
PORT_ACTION_INCREMENT_CREDIT	This action causes a port to increment its credit.	Yes	Yes	Yes	Yes

PORT_ACTION_DECREMENT_CREDIT	This action causes a port to decrement its credit.	Yes	Yes	Yes	Yes
PORT_ACTION_STOP_RECEPTION	This action causes a port to stop receiving whilst it is set.	Yes	No	No	No
PORT_ACTION_DISABLE_LVDS	This action causes a port to disable its LVDS drivers whilst it is set.	Yes	No	No	No

Time-Code Interface Output Actions

Output Action	Description	Supported Devices			
		Brick Mk3	PXI Interface	PXI Router	PCIe Interface
TIME_CODE_ACTION_TX	This action causes a time-code interface to transmit its next valid time-code.	Yes	Yes	Yes	No

7.24.4 Internal Triggers

Internal triggers are used to control which input events cause output actions. An internal trigger can receive one or more input events which cause it to be set. Once set, the internal trigger causes one or more output actions.

An internal trigger is configured by taking a mask, for each triggering resource, that describes which events will cause it to be set. The internal trigger is then configured to operate in AND or OR mode. If operating in AND mode, the internal trigger will be set when all of its input events occur. If operating in OR mode, the internal trigger will be set when any of its input events occur. To control which output actions that the internal trigger will cause, it takes a mask, for each triggering resource, that describes which output actions will be performed when it is set.

The following diagram illustrates the input events, internal triggers, and output actions architecture.

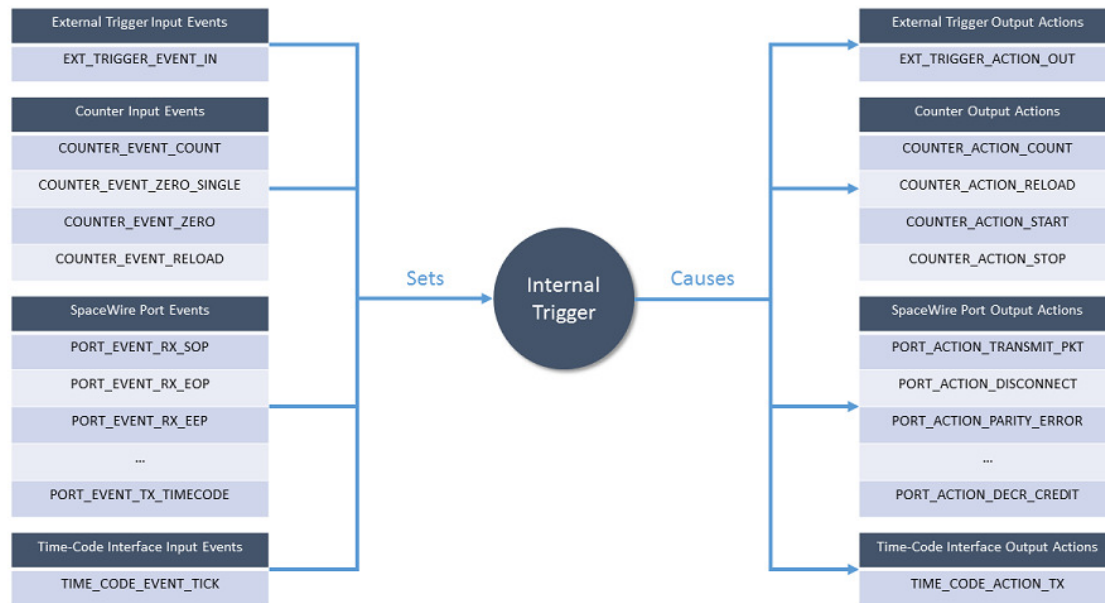


Figure 7.1: Triggering API Architecture

The diagram above shows that an internal trigger is set by one or more input events. Once set, the internal trigger causes one or more output actions.

The following diagram shows an example of a triggering configuration.



Figure 7.2: Triggering API Example

In this example, a time-code received event on the time-code interface is configured to set internal trigger 0. Once set, internal trigger 0 causes a packet to be transmitted on SpaceWire port 1. The code for this example, using a Brick Mk3, is shown below.

- ~~~{.c} /* Enable port transmit packet mode on port 1 */ TRIGGER_BRICK_MK3_enablePortTransmit(
PacketMode(deviceId, 1);
/* TIME_CODE_EVENT_TICK event on the time-code interface causes internal trigger 0 to be set */ TRIG←
GER_BRICK_MK3_setTimeCodeInputEvents(deviceId, 0, 0, TIME_CODE_EVENT_TICK);
/* Internal trigger 0 causes PORT_ACTION_TRANSMIT_PKT action on port 1 */ TRIGGER_BRICK_MK3←
_setPortOutputActions(deviceId, 1, 0, PORT_ACTION_TRANSMIT_PKT);
/* Enable internal trigger 0 */ TRIGGER_BRICK_MK3_enableTrigger(deviceId, 0);
- ~~~

In the code listed above, SpaceWire port 1 is first configured to have "Port Transmit Mode" enabled. This means that any packets submitted for transmission on the port by STAR-System will be held in a buffer until explicitly instructed to transmit by an action. In the second line, internal trigger 0 is configured to be set by

the TIME_CODE_EVENT_TICK event from the time-code interface. Next, internal trigger 0 is configured to cause the PORT_ACTION_TRANSMIT_PKT action on SpaceWire port 1 when set. Finally, internal trigger 0 is enabled to allow it to receive events and cause actions.

7.24.5 Triggering Resources

There are five types of triggering resource: external triggers, counters, SpaceWire ports, time-code interfaces and internal triggers. Each device that is supported by the Triggering API has zero or more of each of these resources. The following table lists the number of triggering resources in each supported device.

Triggering Resources

Type	Description	Number of Resources			
		Brick Mk3	PXI Interface	PXI Router	PCle Interface (see note below)
External Trigger	An external trigger can receive signals causing events and transmit signals as actions.	2	4	0	0
Counter	A counter can operate in free-running mode as a timer or in triggered counting mode as a counter.	4	4	4	6
SpaceWire Port	A SpaceWire port can cause events when it receives characters or errors and perform actions such as transmitting packets or injecting errors.	2	4	12	3

Time-Code Interface	A time-code interface can cause events by receiving valid time-codes and transmit valid time-codes as actions.	1	1	1	0
Internal Trigger	An internal trigger is set by receiving one or more input events. When set, it causes actions to be performed on other triggering resources.	8	8	4	6

Note that the number of resources supported by PCIe devices was increased in version 1.11e06. Previous versions support only 3 counters and internal triggers.

The external trigger, counter, SpaceWire port and internal trigger triggering resources have various configuration settings. The following tables describe the configuration options for each of these triggering resources.

External Trigger Configuration

Setting	Description
Edge Detect Mode	When enabled, this setting causes the input event to occur on the rising edge of the input signal.
Invert	When enabled, this setting inverts the input signal before it reaches the detection mechanism.
Output	When enabled, the output action causes the external trigger to output a signal for the duration specified by the Extend value.
Extend Value	The extend value is the duration, in clock cycles, of the external trigger's output signal caused by an output action.

Counter Configuration

Setting	Description
Auto Reload	When enabled, this setting causes the counter to be automatically reloaded with its configured reload value when the counter reaches zero.
Trigger Count	When enabled, the counter must explicitly receive a count action before it is decremented. This allows the counter to operate as a counter rather than a free-running timer.
Start Mode	When enabled, the counter will wait for a start action before any counting or reloading can take place. When the counter reaches zero, it may be reloaded but no further counting or reloading will take place until the next start action occurs.

Start Stop Mode	When enabled, the counter will wait for a start action before any counting or reloading can take place. The counter will continue to count and reload until a stop action occurs.
Load Zero	When enabled, reload triggered events will only occur when the counter value is zero.
Reload Value	The reload value is the value that the counter will be set to when it is reloaded.
Force Reload	The force reload setting causes the counter to be reloaded.

SpaceWire Port Configuration

Setting	Description
Transmit Packet Mode	When enabled, packets are queued for transmission on the SpaceWire port. If a packet transmit action occurs, a single packet will be transmitted from the queue.

Internal Trigger Configuration

Setting	Description
Input Mode	An internal trigger can be configured to operate in AND or OR mode. When operating in AND mode, the internal trigger is set when all of its input events occur. When operating in OR mode, the internal trigger is set when any of its input events occur.
Enabled	When enabled, the internal trigger can be set by its input events and cause its output actions to occur.
Force Trigger	The force trigger setting causes the internal trigger to be set.
Invert Mask	The invert mask determines, for each triggering resource, if the input events should be inverted before reaching the internal trigger.
AND Mask	The AND Mask determines, for each triggering resource, if the input events should be checked when the internal trigger is operating in AND mode.

7.24.6 PCIe Differences

The PCIe device's triggering support has several differences compared to the Brick Mk3 and PXI devices. These differences are listed below.

- There is no time-code interface so the functions to trigger on valid time-code events and perform time-code transmit actions are not supported. However, the SpaceWire port time-code receive event is still supported so it can be used to trigger on time-codes.
- The SpaceWire port time-code transmit event is not supported by the PCIe device.
- There are no explicit packet transmit mode enable/disable functions. Rather, if any packet transmit action is enabled on a port, packet transmit mode is enabled implicitly.

7.24.7 Examples

The following sections describe several examples showing how to use the Triggering API.

7.24.7.1 Transmitting Time-Codes Synchronised by an External Trigger

In this example, the triggers of a Brick Mk3 will be configured so that time-codes are transmitted by the Brick Mk3 every time the rising edge of an external trigger input signal is detected. The code for this example is listed below.

- ```

~~~~{.c} /* Enable external trigger 0 edge detect mode */ TRIGGER_BRICK_MK3_enableExtTriggerEdge(
DetectMode(deviceId, 0);

/* EXT_TRIGGER_EVENT_IN event on external trigger 0 causes internal trigger 0 to be set */ TRIGGER_
_BRICK_MK3_setExtTriggerInputEvents(deviceId, 0, 0, EXT_TRIGGER_EVENT_IN);

/* Internal trigger 0 causes TIME_CODE_ACTION_TX action on the time-code interface */ TRIGGER_BR
ICK_MK3_setTimeCodeOutputActions(deviceId, 0, 0, TIME_CODE_ACTION_TX);

/* Enable internal trigger 0 */ TRIGGER_BRICK_MK3_enableTrigger(deviceId, 0);

```

- ~~~~

In the code listed above, external trigger 0 is first configured to enable edge detect mode. This means that the EXT\_TRIGGER\_EVENT\_IN event will occur once, on the rising edge of the input signal. Otherwise, the event would be active for as long as the input signal is high (unless the invert setting is enabled, in which case the event would be active for as long as the input signal is low).

Next, internal trigger 0 is configured so that the EXT\_TRIGGER\_EVENT\_IN event from external trigger 0 will cause it to be set. Internal trigger 0 is also configured so that, when set, it causes the TIME\_CODE\_ACTION\_TX action on the time-code interface. Finally, internal trigger 0 is enabled to allow the event to cause the action.

### 7.24.7.2 Packet Transmission Synchronised by a Counter

In this example, the triggers of a Brick Mk3 will be configured so that packets submitted for transmission by STAR-System to port 1 will be queued and transmitted every time a free-running counter reloads. The code for this example is listed below.

- ```

~~~~{.c} /* Set counter 0 reload value to 60 million clock cycles */ TRIGGER_BRICK_MK3_setCounter
ReloadValue(deviceId, 0, 60000000);

/* Enable counter auto reload */ TRIGGER_BRICK_MK3_enableCounterAutoReload(deviceId, 0);

/* Enable port transmit packet mode on port 1 */ TRIGGER_BRICK_MK3_enablePortTransmitPacket
Mode(deviceId, 1);

/* COUNTER_EVENT_RELOAD event on counter 0 causes internal trigger 0 to be set */ TRIGGER_BRI
CK_MK3_setCounterInputEvents(deviceId, 0, 0, COUNTER_EVENT_RELOAD);

/* Internal trigger 0 causes PORT_ACTION_TRANSMIT_PKT action on port 1 */ TRIGGER_BRICK_MK3_
_setPortOutputActions(deviceId, 1, 0, PORT_ACTION_TRANSMIT_PKT);

/* Enable internal trigger 0 */ TRIGGER_BRICK_MK3_enableTrigger(deviceId, 0);

```

- ~~~~

In the code listed above, counter 0 is first configured to set its reload value to 60 million clock cycles and enable its auto-reload setting. SpaceWire port 1 is then configured to enable its transmit packet mode.

Next, internal trigger 0 is configured so that the COUNTER\_EVENT\_RELOAD event from counter 0 will cause it to be set. Internal trigger 0 is also configured so that, when set, it causes the PORT\_ACTION\_TRANSMIT\_PKT action on SpaceWire port 1. Finally, internal trigger 0 is enabled to allow the event to cause the action.



### 7.24.7.3 Multiple Packet Transmission Synchronised by Time-Codes

In this example, the triggers of a Brick Mk3 will be configured so that packets submitted for transmission by STAR-System to port 1 will be queued and transmitted in groups of 4 every time the Brick Mk3 receives a valid time-code. The code for this example is listed below.

- ~~~{.c} /\* Enable counter trigger count on counter 0 so the counter only counts on triggers \*/ TRIGGER\_↵  
\_BRICK\_MK3\_enableCounterTriggerCount(deviceId, 0);  
/\* Set counter 0 reload value to 3 \*/ TRIGGER\_BRICK\_MK3\_setCounterReloadValue(deviceId, 0, 3);  
/\* Enable port transmit packet mode on port 1 \*/ TRIGGER\_BRICK\_MK3\_enablePortTransmitPacket\_↵  
Mode(deviceId, 1);  
/\* TIME\_CODE\_EVENT\_TICK event on the time-code interface causes internal trigger 0 to be set \*/ TRIG\_↵  
GER\_BRICK\_MK3\_setTimeCodeInputEvents(deviceId, 0, 0, TIME\_CODE\_EVENT\_TICK);  
/\* Internal trigger 0 causes COUNTER\_ACTION\_RELOAD action on counter 0 \*/ TRIGGER\_BRICK\_MK3\_↵  
\_setCounterOutputActions(deviceId, 0, 0, COUNTER\_ACTION\_RELOAD);  
/\* Internal trigger 0 causes PORT\_ACTION\_TRANSMIT\_PKT action on port 1 \*/ TRIGGER\_BRICK\_MK3\_↵  
\_setPortOutputActions(deviceId, 1, 0, PORT\_ACTION\_TRANSMIT\_PKT);  
/\* PORT\_EVENT\_TX\_EOP event on port 1 causes internal trigger 1 to be set \*/ TRIGGER\_BRICK\_MK3\_↵  
\_setPortInputEvents(deviceId, 1, 1, PORT\_EVENT\_TX\_EOP);  
/\* Internal trigger 1 causes COUNTER\_ACTION\_COUNT action on counter 0 \*/ TRIGGER\_BRICK\_MK3\_↵  
\_setCounterOutputActions(deviceId, 0, 1, COUNTER\_ACTION\_COUNT);  
/\* COUNTER\_EVENT\_COUNT event on counter 0 causes internal trigger 2 to be set \*/ TRIGGER\_BRIC\_↵  
K\_MK3\_setCounterInputEvents(deviceId, 0, 2, COUNTER\_EVENT\_COUNT);  
/\* Internal trigger 2 causes PORT\_ACTION\_TRANSMIT\_PKT action on port 1 \*/ TRIGGER\_BRICK\_MK3\_↵  
\_setPortOutputActions(deviceId, 1, 2, PORT\_ACTION\_TRANSMIT\_PKT);  
/\* Enable internal triggers 0, 1 and 2 \*/ TRIGGER\_BRICK\_MK3\_enableTrigger(deviceId, 0); TRIGGER\_B\_↵  
RICK\_MK3\_enableTrigger(deviceId, 1); TRIGGER\_BRICK\_MK3\_enableTrigger(deviceId, 2);
- ~~~

In the code listed above, counter 0 is configured to enable its trigger count mode. This means that the counter will only decrement when it receives a COUNTER\_ACTION\_COUNT action. Additionally, counter 0 has its reload value set to 3. Auto-reload is not enabled in this case because reloading will be controlled by the triggers. SpaceWire Port 1 is then configured to have its transmit packet mode enabled as described in the previous example.

Next, internal trigger 0 is configured so that the TIME\_CODE\_EVENT\_TICK event from the time-code interface will cause it to be set. Internal trigger 0 is also configured so that, when set, it causes two actions to be performed: firstly, counter 0 is reloaded using the COUNTER\_ACTION\_RELOAD action, and secondly, the first packet in the group is transmitted on port 1 using the PORT\_ACTION\_TRANSMIT\_PKT action. Internal trigger 1 is configured so that the PORT\_EVENT\_TX\_EOP event on port 1 will cause it to be set. Internal trigger 1 is also configured so that, when set, it causes the COUNTER\_ACTION\_COUNT action on counter 0. Internal trigger 2 is configured so that the COUNTER\_EVENT\_COUNT event from counter 0 causes it to be set. Internal trigger 2 is also configured so that, when set, it causes the PORT\_ACTION\_TRANSMIT\_PKT action on port 1. Finally, internal triggers 0, 1 and 2 are enabled to allow the events to cause the actions.

To summarise this example, when a time-code is received by the Brick Mk3, it reloads the packet counter and transmits the first packet in the group of 4. When the EOP from the packet is transmitted, it decrements the packet counter. When the packet counter is decremented it transmits another packet. The transmitting EOPs continue causing the packet counter to decrement and more packets to be transmitted until the counter reaches 0, at which point the transmission stops. When the next time-code is received, the packet counter is reloaded and the process begins again, transmitting the next group of 4 packets.

## 7.25 Events

Functions in this group deal with setting and getting trigger input events.

Collaboration diagram for Events:



### Typedefs

- typedef U32 EXT\_TRIGGER\_EVENT\_MASK  
*External trigger event mask.*
- typedef U32 EXT\_TRIGGER\_ACTION\_MASK  
*External trigger action mask.*
- typedef U32 COUNTER\_EVENT\_MASK  
*Counter event mask.*
- typedef U32 COUNTER\_ACTION\_MASK  
*Counter action mask.*
- typedef U32 PORT\_EVENT\_MASK  
*Port event mask.*
- typedef U32 PORT\_ACTION\_MASK  
*Port action mask.*
- typedef U32 TIME\_CODE\_EVENT\_MASK  
*Time-code event mask.*
- typedef U32 TIME\_CODE\_ACTION\_MASK  
*Time-code action mask.*
- typedef U32 TRIGGER\_EVENT\_MASK  
*Trigger event mask.*

### Enumerations

- enum EXT\_TRIGGER\_EVENT { EXT\_TRIGGER\_EVENT\_IN = 1 << 0, EXT\_TRIGGER\_EVENT\_NONE = 0, EXT\_TRIGGER\_EVENT\_ALL = 0xf }
  - enum COUNTER\_EVENT { COUNTER\_EVENT\_COUNT = 1 << 0, COUNTER\_EVENT\_ZERO\_SINGLE = 1 << 1, COUNTER\_EVENT\_ZERO = 1 << 2, COUNTER\_EVENT\_RELOAD = 1 << 3, COUNTER\_EVENT\_NONE = 0, COUNTER\_EVENT\_ALL = 0xf }
- Counter events.*

- enum PORT\_EVENT {  
PORT\_EVENT\_RX\_SOP = 1 << 0, PORT\_EVENT\_RX\_EOP = 1 << 1, PORT\_EVENT\_RX\_EEP = 1 << 2, PORT\_EVENT\_TX\_SOP = 1 << 3,  
PORT\_EVENT\_TX\_EOP = 1 << 4, PORT\_EVENT\_TX\_PKT\_PENDING = 1 << 5, PORT\_EVENT\_RUNNING = 1 << 6, PORT\_EVENT\_PARITY\_ERROR = 1 << 7,  
PORT\_EVENT\_ESCAPE\_ERROR = 1 << 8, PORT\_EVENT\_CREDIT\_ERROR = 1 << 9, PORT\_EVENT\_T\_DISCONNECT = 1 << 10, PORT\_EVENT\_RX\_TIME\_CODE = 1 << 11,  
PORT\_EVENT\_TX\_TIME\_CODE = 1 << 12, PORT\_EVENT\_NONE = 0, PORT\_EVENT\_ALL = 0x1fff, PORT\_EVENT\_ALL\_PCIE = 0xffff }

*Port events.*

- enum TIME\_CODE\_EVENT { TIME\_CODE\_EVENT\_TICK = 1 << 0, TIME\_CODE\_EVENT\_NONE = 0, TIME\_CODE\_EVENT\_ALL = 0x1 }

*Time-code events.*

- enum TRIGGER\_EVENT { TRIGGER\_EVENT\_IN = 1 << 0, TRIGGER\_EVENT\_NONE = 0, TRIGGER\_EVENT\_ALL = 0x1 }

*Internal trigger events.*

## Functions

- int STAR\_API\_CC TRIGGER\_BRICK\_MK3\_setExtTriggerInputEvents (STAR\_DEVICE\_ID deviceId, U32 extTrigger, U32 trigger, EXT\_TRIGGER\_EVENT\_MASK events)  
*Sets the input events for an external trigger which will cause an internal trigger to be set.*
- int STAR\_API\_CC TRIGGER\_BRICK\_MK3\_setCounterInputEvents (STAR\_DEVICE\_ID deviceId, U32 counter, U32 trigger, COUNTER\_EVENT\_MASK events)  
*Sets the input events for a counter which will cause an internal trigger to be set.*
- int STAR\_API\_CC TRIGGER\_BRICK\_MK3\_setPortInputEvents (STAR\_DEVICE\_ID deviceId, U32 port, U32 trigger, PORT\_EVENT\_MASK events)  
*Sets the input events for a port which will cause an internal trigger to be set.*
- int STAR\_API\_CC TRIGGER\_BRICK\_MK3\_setTimeCodeInputEvents (STAR\_DEVICE\_ID deviceId, U32 timeCode, U32 trigger, TIME\_CODE\_EVENT\_MASK events)  
*Sets the input events for a time-code engine which will cause an internal trigger to be set.*
- int STAR\_API\_CC TRIGGER\_BRICK\_MK3\_setTriggerInputEvents (STAR\_DEVICE\_ID deviceId, U32 causeTrigger, U32 trigger, TRIGGER\_EVENT\_MASK events)  
*Sets the input events for an internal trigger which will cause another internal trigger to be set.*
- int STAR\_API\_CC TRIGGER\_BRICK\_MK3\_getExtTriggerInputEvents (STAR\_DEVICE\_ID deviceId, U32 extTrigger, U32 trigger, EXT\_TRIGGER\_EVENT\_MASK \*pEvents)  
*Gets the input events from an external trigger which will cause an internal trigger to be set.*
- int STAR\_API\_CC TRIGGER\_BRICK\_MK3\_getCounterInputEvents (STAR\_DEVICE\_ID deviceId, U32 counter, U32 trigger, COUNTER\_EVENT\_MASK \*pEvents)  
*Gets the input events from a counter which will cause an internal trigger to be set.*
- int STAR\_API\_CC TRIGGER\_BRICK\_MK3\_getPortInputEvents (STAR\_DEVICE\_ID deviceId, U32 port, U32 trigger, PORT\_EVENT\_MASK \*pEvents)  
*Gets the input events from a port which will cause an internal trigger to be set.*
- int STAR\_API\_CC TRIGGER\_BRICK\_MK3\_getTimeCodeInputEvents (STAR\_DEVICE\_ID deviceId, U32 timeCode, U32 trigger, TIME\_CODE\_EVENT\_MASK \*pEvents)  
*Gets the input events from a time-code engine which will cause an internal trigger to be set.*
- int STAR\_API\_CC TRIGGER\_BRICK\_MK3\_getTriggerInputEvents (STAR\_DEVICE\_ID deviceId, U32 causeTrigger, U32 trigger, TRIGGER\_EVENT\_MASK \*pEvents)  
*Gets the input events for an internal trigger which will cause another internal trigger to be set.*
- int STAR\_API\_CC TRIGGER\_PCIE\_IF\_setCounterInputEvents (STAR\_DEVICE\_ID deviceId, U32 counter, U32 trigger, COUNTER\_EVENT\_MASK events)  
*Sets the input events for a counter which will cause an internal trigger to be set.*
- int STAR\_API\_CC TRIGGER\_PCIE\_IF\_setPortInputEvents (STAR\_DEVICE\_ID deviceId, U32 port, U32 trigger, PORT\_EVENT\_MASK events)

*Sets the input events for a port which will cause an internal trigger to be set.*

- int `STAR_API_CC TRIGGER_PCIE_IF_setTriggerInputEvents` (STAR\_DEVICE\_ID deviceId, U32 cause↵ Trigger, U32 trigger, TRIGGER\_EVENT\_MASK events)

*Sets the input events for an internal trigger which will cause another internal trigger to be set.*

- int `STAR_API_CC TRIGGER_PCIE_IF_getCounterInputEvents` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 trigger, COUNTER\_EVENT\_MASK \*pEvents)

*Gets the input events from a counter which will cause an internal trigger to be set.*

- int `STAR_API_CC TRIGGER_PCIE_IF_getPortInputEvents` (STAR\_DEVICE\_ID deviceId, U32 port, U32 trigger, PORT\_EVENT\_MASK \*pEvents)

*Gets the input events from a port which will cause an internal trigger to be set.*

- int `STAR_API_CC TRIGGER_PCIE_IF_getTriggerInputEvents` (STAR\_DEVICE\_ID deviceId, U32 cause↵ Trigger, U32 trigger, TRIGGER\_EVENT\_MASK \*pEvents)

*Gets the input events for an internal trigger which will cause another internal trigger to be set.*

- int `STAR_API_CC TRIGGER_PXI_IF_setExtTriggerInputEvents` (STAR\_DEVICE\_ID deviceId, U32 ext↵ Trigger, U32 trigger, EXT\_TRIGGER\_EVENT\_MASK events)

*Sets the input events for an external trigger which will cause an internal trigger to be set.*

- int `STAR_API_CC TRIGGER_PXI_IF_setCounterInputEvents` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 trigger, COUNTER\_EVENT\_MASK events)

*Sets the input events for a counter which will cause an internal trigger to be set.*

- int `STAR_API_CC TRIGGER_PXI_IF_setPortInputEvents` (STAR\_DEVICE\_ID deviceId, U32 port, U32 trigger, PORT\_EVENT\_MASK events)

*Sets the input events for a port which will cause an internal trigger to be set.*

- int `STAR_API_CC TRIGGER_PXI_IF_setTimeCodeInputEvents` (STAR\_DEVICE\_ID deviceId, U32 time↵ Code, U32 trigger, TIME\_CODE\_EVENT\_MASK events)

*Sets the input events for a time-code engine which will cause an internal trigger to be set.*

- int `STAR_API_CC TRIGGER_PXI_IF_setTriggerInputEvents` (STAR\_DEVICE\_ID deviceId, U32 cause↵ Trigger, U32 trigger, TRIGGER\_EVENT\_MASK events)

*Sets the input events for an internal trigger which will cause another internal trigger to be set.*

- int `STAR_API_CC TRIGGER_PXI_IF_getExtTriggerInputEvents` (STAR\_DEVICE\_ID deviceId, U32 ext↵ Trigger, U32 trigger, EXT\_TRIGGER\_EVENT\_MASK \*pEvents)

*Gets the input events from an external trigger which will cause an internal trigger to be set.*

- int `STAR_API_CC TRIGGER_PXI_IF_getCounterInputEvents` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 trigger, COUNTER\_EVENT\_MASK \*pEvents)

*Gets the input events from a counter which will cause an internal trigger to be set.*

- int `STAR_API_CC TRIGGER_PXI_IF_getPortInputEvents` (STAR\_DEVICE\_ID deviceId, U32 port, U32 trigger, PORT\_EVENT\_MASK \*pEvents)

*Gets the input events from a port which will cause an internal trigger to be set.*

- int `STAR_API_CC TRIGGER_PXI_IF_getTimeCodeInputEvents` (STAR\_DEVICE\_ID deviceId, U32 time↵ Code, U32 trigger, TIME\_CODE\_EVENT\_MASK \*pEvents)

*Gets the input events from a time-code engine which will cause an internal trigger to be set.*

- int `STAR_API_CC TRIGGER_PXI_IF_getTriggerInputEvents` (STAR\_DEVICE\_ID deviceId, U32 cause↵ Trigger, U32 trigger, TRIGGER\_EVENT\_MASK \*pEvents)

*Gets the input events for an internal trigger which will cause another internal trigger to be set.*

- int `STAR_API_CC TRIGGER_PXI_ROUTER_setCounterInputEvents` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 trigger, COUNTER\_EVENT\_MASK events)

*Sets the input events for a counter which will cause an internal trigger to be set.*

- int `STAR_API_CC TRIGGER_PXI_ROUTER_setPortInputEvents` (STAR\_DEVICE\_ID deviceId, U32 port, U32 trigger, PORT\_EVENT\_MASK events)

*Sets the input events for a port which will cause an internal trigger to be set.*

- int `STAR_API_CC TRIGGER_PXI_ROUTER_setTimeCodeInputEvents` (STAR\_DEVICE\_ID deviceId, U32 timeCode, U32 trigger, TIME\_CODE\_EVENT\_MASK events)

*Sets the input events for a time-code engine which will cause an internal trigger to be set.*

- `int STAR_API_CC TRIGGER_PXI_ROUTER_setTriggerInputEvents (STAR_DEVICE_ID deviceId, U32 causeTrigger, U32 trigger, TRIGGER_EVENT_MASK events)`  
*Sets the input events for an internal trigger which will cause another internal trigger to be set.*
- `int STAR_API_CC TRIGGER_PXI_ROUTER_getCounterInputEvents (STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_EVENT_MASK *pEvents)`  
*Gets the input events from a counter which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_PXI_ROUTER_getPortInputEvents (STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_EVENT_MASK *pEvents)`  
*Gets the input events from a port which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_PXI_ROUTER_getTimeCodeInputEvents (STAR_DEVICE_ID deviceId, U32 timeCode, U32 trigger, TIME_CODE_EVENT_MASK *pEvents)`  
*Gets the input events from a time-code engine which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_PXI_ROUTER_getTriggerInputEvents (STAR_DEVICE_ID deviceId, U32 causeTrigger, U32 trigger, TRIGGER_EVENT_MASK *pEvents)`  
*Gets the input events for an internal trigger which will cause another internal trigger to be set.*

### 7.25.1 Detailed Description

Functions in this group deal with setting and getting trigger input events.

### 7.25.2 Typedef Documentation

#### 7.25.2.1 typedef U32 COUNTER\_ACTION\_MASK

Counter action mask.

Added in version:

STAR-System v3.0 - 26th August 2016

#### 7.25.2.2 typedef U32 COUNTER\_EVENT\_MASK

Counter event mask.

Added in version:

STAR-System v3.0 - 26th August 2016

#### 7.25.2.3 typedef U32 EXT\_TRIGGER\_ACTION\_MASK

External trigger action mask.

Added in version:

STAR-System v3.0 - 26th August 2016

#### 7.25.2.4 typedef U32 EXT\_TRIGGER\_EVENT\_MASK

External trigger event mask.

Added in version:

STAR-System v3.0 - 26th August 2016

#### 7.25.2.5 `typedef U32 PORT_ACTION_MASK`

Port action mask.

Added in version:

STAR-System v3.0 - 26th August 2016

#### 7.25.2.6 `typedef U32 PORT_EVENT_MASK`

Port event mask.

Added in version:

STAR-System v3.0 - 26th August 2016

#### 7.25.2.7 `typedef U32 TIME_CODE_ACTION_MASK`

Time-code action mask.

Added in version:

STAR-System v3.0 - 26th August 2016

#### 7.25.2.8 `typedef U32 TIME_CODE_EVENT_MASK`

Time-code event mask.

Added in version:

STAR-System v3.0 - 26th August 2016

#### 7.25.2.9 `typedef U32 TRIGGER_EVENT_MASK`

Trigger event mask.

Added in version:

STAR-System v3.0 - 26th August 2016

### 7.25.3 Enumeration Type Documentation

#### 7.25.3.1 `enum COUNTER_EVENT`

Counter events.

Added in version:

STAR-System v3.0 - 26th August 2016

Enumerator

**`COUNTER_EVENT_COUNT`** Event occurs when the counter decrements.

**`COUNTER_EVENT_ZERO_SINGLE`** Event occurs when the counter reaches zero (once)

**`COUNTER_EVENT_ZERO`** Event is active while the counter is equal to zero.

**`COUNTER_EVENT_RELOAD`** Event occurs when the counter reloads.

**`COUNTER_EVENT_NONE`** No events.

**`COUNTER_EVENT_ALL`** All events.

### 7.25.3.2 enum EXT\_TRIGGER\_EVENT

External trigger events.

Added in version:

STAR-System v3.0 - 26th August 2016

Enumerator

**EXT\_TRIGGER\_EVENT\_IN** Event occurs when the external trigger has received input.

**EXT\_TRIGGER\_EVENT\_NONE** No events.

**EXT\_TRIGGER\_EVENT\_ALL** All events.

### 7.25.3.3 enum PORT\_EVENT

Port events.

Added in version:

STAR-System v3.0 - 26th August 2016

Enumerator

**PORT\_EVENT\_RX\_SOP** Event occurs when the port receives an SOP.

**PORT\_EVENT\_RX\_EOP** Event occurs when the port receives an EOP.

**PORT\_EVENT\_RX\_EEP** Event occurs when the port receives an EEP.

**PORT\_EVENT\_TX\_SOP** Event occurs when the port transmits an SOP.

**PORT\_EVENT\_TX\_EOP** Event occurs when the port transmits an EOP.

**PORT\_EVENT\_TX\_PKT\_PENDING** Event is active when the port has a packet queued for transmission.

**PORT\_EVENT\_RUNNING** Event is active when the link attached to the port is running.

**PORT\_EVENT\_PARITY\_ERROR** Event occurs when the port detects a parity error.

**PORT\_EVENT\_ESCAPE\_ERROR** Event occurs when the port detects an escape error.

**PORT\_EVENT\_CREDIT\_ERROR** Event occurs when the port detects a credit error.

**PORT\_EVENT\_DISCONNECT** Event occurs when the port disconnects.

**PORT\_EVENT\_RX\_TIME\_CODE** Event occurs when the port receives a time-code.

**PORT\_EVENT\_TX\_TIME\_CODE** Event occurs when the port transmits a time-code.

**PORT\_EVENT\_NONE** No events.

**PORT\_EVENT\_ALL** All events.

### 7.25.3.4 enum TIME\_CODE\_EVENT

Time-code events.

Added in version:

STAR-System v3.0 - 26th August 2016

Enumerator

**TIME\_CODE\_EVENT\_TICK** Event occurs when the next valid time-code is received.

**TIME\_CODE\_EVENT\_NONE** No events.

**TIME\_CODE\_EVENT\_ALL** All events.

### 7.25.3.5 enum TRIGGER\_EVENT

Internal trigger events.

Added in version:

STAR-System v3.0 - 26th August 2016

Enumerator

**TRIGGER\_EVENT\_IN** Event occurs when the internal trigger has received input.

**TRIGGER\_EVENT\_NONE** No events.

**TRIGGER\_EVENT\_ALL** All events.

## 7.25.4 Function Documentation

**7.25.4.1** `int STAR_API_CC TRIGGER_BRICK_MK3_getCounterInputEvents ( STAR_DEVICE_ID deviceld, U32 counter, U32 trigger, COUNTER_EVENT_MASK * pEvents )`

Gets the input events from a counter which will cause an internal trigger to be set.

Parameters

|                 |                                                                |
|-----------------|----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                 |
| <i>counter</i>  | Counter to get input events from.                              |
| <i>trigger</i>  | Internal trigger which is affected by the input events.        |
| <i>pEvents</i>  | Pointer to a value which will be updated with the events mask. |

Returns

1 if the operation was successful, else 0

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire Brick Mk3

**7.25.4.2** `int STAR_API_CC TRIGGER_BRICK_MK3_getExtTriggerInputEvents ( STAR_DEVICE_ID deviceld, U32 extTrigger, U32 trigger, EXT_TRIGGER_EVENT_MASK * pEvents )`

Gets the input events from an external trigger which will cause an internal trigger to be set.

Parameters

|                   |                                                                |
|-------------------|----------------------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.                        |
| <i>extTrigger</i> | External trigger to get input events from.                     |
| <i>trigger</i>    | Internal trigger which is affected by the input events.        |
| <i>pEvents</i>    | Pointer to a value which will be updated with the events mask. |

Returns

1 if the operation was successful, else 0

Added in version:

STAR-System v3.0 - 26th August 2016



**Supported devices:**

SpaceWire Brick Mk3

**7.25.4.3** `int STAR_API_CC TRIGGER_BRICK_MK3_getPortInputEvents ( STAR_DEVICE_ID deviceld, U32 port, U32 trigger, PORT_EVENT_MASK * pEvents )`

Gets the input events from a port which will cause an internal trigger to be set.

**Parameters**

|                 |                                                                |
|-----------------|----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                                    |
| <i>port</i>     | Port to get input events from.                                 |
| <i>trigger</i>  | Internal trigger which is affected by the input events.        |
| <i>pEvents</i>  | Pointer to a value which will be updated with the events mask. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

**7.25.4.4** `int STAR_API_CC TRIGGER_BRICK_MK3_getTimeCodeInputEvents ( STAR_DEVICE_ID deviceld, U32 timeCode, U32 trigger, TIME_CODE_EVENT_MASK * pEvents )`

Gets the input events from a time-code engine which will cause an internal trigger to be set.

**Parameters**

|                 |                                                                |
|-----------------|----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the time-code engine.                        |
| <i>timeCode</i> | Time-code engine to get input events from.                     |
| <i>trigger</i>  | Internal trigger which is affected by the input events.        |
| <i>pEvents</i>  | Pointer to a value which will be updated with the events mask. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

**7.25.4.5** `int STAR_API_CC TRIGGER_BRICK_MK3_getTriggerInputEvents ( STAR_DEVICE_ID deviceld, U32 causeTrigger, U32 trigger, TRIGGER_EVENT_MASK * pEvents )`

Gets the input events for an internal trigger which will cause another internal trigger to be set.

**Parameters**

|                     |                                                                |
|---------------------|----------------------------------------------------------------|
| <i>deviceld</i>     | Device containing the internal trigger.                        |
| <i>causeTrigger</i> | Internal trigger to get input events from.                     |
| <i>trigger</i>      | Internal trigger which is affected by the input events.        |
| <i>pEvents</i>      | Pointer to a value which will be updated with the events mask. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

7.25.4.6 `int STAR_API_CC TRIGGER_BRICK_MK3_setCounterInputEvents ( STAR_DEVICE_ID deviceld, U32 counter, U32 trigger, COUNTER_EVENT_MASK events )`

Sets the input events for a counter which will cause an internal trigger to be set.

**Parameters**

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                          |
| <i>counter</i>  | Counter to set input events for.                        |
| <i>trigger</i>  | Internal trigger which is affected by the input events. |
| <i>events</i>   | Mask describing the input events.                       |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

**Examples:**

timestamp\_test.c, and triggering\_example.c.

7.25.4.7 `int STAR_API_CC TRIGGER_BRICK_MK3_setExtTriggerInputEvents ( STAR_DEVICE_ID deviceld, U32 extTrigger, U32 trigger, EXT_TRIGGER_EVENT_MASK events )`

Sets the input events for an external trigger which will cause an internal trigger to be set.

**Parameters**

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceld</i> | Device containing the external trigger. |
|-----------------|-----------------------------------------|

|                   |                                                         |
|-------------------|---------------------------------------------------------|
| <i>extTrigger</i> | External trigger to set input events for.               |
| <i>trigger</i>    | Internal trigger which is affected by the input events. |
| <i>events</i>     | Mask describing the input events.                       |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

**Examples:**

triggering\_example.c.

**7.25.4.8** `int STAR_API_CC TRIGGER_BRICK_MK3_setPortInputEvents ( STAR_DEVICE_ID deviceld, U32 port, U32 trigger, PORT_EVENT_MASK events )`

Sets the input events for a port which will cause an internal trigger to be set.

**Parameters**

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                             |
| <i>port</i>     | Port to set input events for.                           |
| <i>trigger</i>  | Internal trigger which is affected by the input events. |
| <i>events</i>   | Mask describing the input events.                       |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

**Examples:**

triggering\_example.c.

**7.25.4.9** `int STAR_API_CC TRIGGER_BRICK_MK3_setTimeCodeInputEvents ( STAR_DEVICE_ID deviceld, U32 timeCode, U32 trigger, TIME_CODE_EVENT_MASK events )`

Sets the input events for a time-code engine which will cause an internal trigger to be set.

**Parameters**

---

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <i>deviceld</i> | Device containing the time-code engine.                 |
| <i>timeCode</i> | Time-code engine to set input events for.               |
| <i>trigger</i>  | Internal trigger which is affected by the input events. |
| <i>events</i>   | Mask describing the input events.                       |

---

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

**Examples:**

triggering\_example.c.

7.25.4.10 `int STAR_API_CC TRIGGER_BRICK_MK3_setTriggerInputEvents ( STAR_DEVICE_ID deviceld, U32 causeTrigger, U32 trigger, TRIGGER_EVENT_MASK events )`

Sets the input events for an internal trigger which will cause another internal trigger to be set.

**Parameters**


---

|                     |                                                         |
|---------------------|---------------------------------------------------------|
| <i>deviceld</i>     | Device containing the internal trigger.                 |
| <i>causeTrigger</i> | Internal trigger to set input events for.               |
| <i>trigger</i>      | Internal trigger which is affected by the input events. |
| <i>events</i>       | Mask describing the input events.                       |

---

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

7.25.4.11 `int STAR_API_CC TRIGGER_PCIE_IF_getCounterInputEvents ( STAR_DEVICE_ID deviceld, U32 counter, U32 trigger, COUNTER_EVENT_MASK * pEvents )`

Gets the input events from a counter which will cause an internal trigger to be set.

**Parameters**


---

|                 |                                   |
|-----------------|-----------------------------------|
| <i>deviceld</i> | Device containing the counter.    |
| <i>counter</i>  | Counter to get input events from. |

---

|                |                                                                |
|----------------|----------------------------------------------------------------|
| <i>trigger</i> | Internal trigger which is affected by the input events.        |
| <i>pEvents</i> | Pointer to a value which will be updated with the events mask. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

**7.25.4.12** `int STAR_API_CC_TRIGGER_PCIE_IF_getPortInputEvents ( STAR_DEVICE_ID deviceld, U32 port, U32 trigger, PORT_EVENT_MASK * pEvents )`

Gets the input events from a port which will cause an internal trigger to be set.

**Parameters**

|                 |                                                                |
|-----------------|----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                                    |
| <i>port</i>     | Port to get input events from.                                 |
| <i>trigger</i>  | Internal trigger which is affected by the input events.        |
| <i>pEvents</i>  | Pointer to a value which will be updated with the events mask. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

**7.25.4.13** `int STAR_API_CC_TRIGGER_PCIE_IF_getTriggerInputEvents ( STAR_DEVICE_ID deviceld, U32 causeTrigger, U32 trigger, TRIGGER_EVENT_MASK * pEvents )`

Gets the input events for an internal trigger which will cause another internal trigger to be set.

**Parameters**

|                     |                                                                |
|---------------------|----------------------------------------------------------------|
| <i>deviceld</i>     | Device containing the internal trigger.                        |
| <i>causeTrigger</i> | Internal trigger to get input events from.                     |
| <i>trigger</i>      | Internal trigger which is affected by the input events.        |
| <i>pEvents</i>      | Pointer to a value which will be updated with the events mask. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

7.25.4.14 `int STAR_API_CC TRIGGER_PCIE_IF_setCounterInputEvents ( STAR_DEVICE_ID deviceld, U32 counter, U32 trigger, COUNTER_EVENT_MASK events )`

Sets the input events for a counter which will cause an internal trigger to be set.

#### Parameters

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                          |
| <i>counter</i>  | Counter to set input events for.                        |
| <i>trigger</i>  | Internal trigger which is affected by the input events. |
| <i>events</i>   | Mask describing the input events.                       |

#### Returns

1 if the operation was successful, else 0

#### Added in version:

STAR-System v3.1 - 21st October 2016

#### Supported devices:

SpaceWire PCIe

#### Examples:

triggering\_example.c.

7.25.4.15 `int STAR_API_CC TRIGGER_PCIE_IF_setPortInputEvents ( STAR_DEVICE_ID deviceld, U32 port, U32 trigger, PORT_EVENT_MASK events )`

Sets the input events for a port which will cause an internal trigger to be set.

#### Parameters

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                             |
| <i>port</i>     | Port to set input events for.                           |
| <i>trigger</i>  | Internal trigger which is affected by the input events. |
| <i>events</i>   | Mask describing the input events.                       |

#### Returns

1 if the operation was successful, else 0

#### Added in version:

STAR-System v3.1 - 21st October 2016

#### Supported devices:

SpaceWire PCIe

7.25.4.16 `int STAR_API_CC TRIGGER_PCIE_IF_setTriggerInputEvents ( STAR_DEVICE_ID deviceld, U32 causeTrigger, U32 trigger, TRIGGER_EVENT_MASK events )`

Sets the input events for an internal trigger which will cause another internal trigger to be set.

**Parameters**

|                     |                                                         |
|---------------------|---------------------------------------------------------|
| <i>deviceld</i>     | Device containing the internal trigger.                 |
| <i>causeTrigger</i> | Internal trigger to set input events for.               |
| <i>trigger</i>      | Internal trigger which is affected by the input events. |
| <i>events</i>       | Mask describing the input events.                       |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

7.25.4.17 **int STAR\_API\_CC TRIGGER\_PXI\_IF\_getCounterInputEvents ( STAR\_DEVICE\_ID *deviceld*, U32 *counter*, U32 *trigger*, COUNTER\_EVENT\_MASK \* *pEvents* )**

Gets the input events from a counter which will cause an internal trigger to be set.

**Parameters**

|                 |                                                                |
|-----------------|----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                 |
| <i>counter</i>  | Counter to get input events from.                              |
| <i>trigger</i>  | Internal trigger which is affected by the input events.        |
| <i>pEvents</i>  | Pointer to a value which will be updated with the events mask. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

7.25.4.18 **int STAR\_API\_CC TRIGGER\_PXI\_IF\_getExtTriggerInputEvents ( STAR\_DEVICE\_ID *deviceld*, U32 *extTrigger*, U32 *trigger*, EXT\_TRIGGER\_EVENT\_MASK \* *pEvents* )**

Gets the input events from an external trigger which will cause an internal trigger to be set.

**Parameters**

|                   |                                                                |
|-------------------|----------------------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.                        |
| <i>extTrigger</i> | External trigger to get input events from.                     |
| <i>trigger</i>    | Internal trigger which is affected by the input events.        |
| <i>pEvents</i>    | Pointer to a value which will be updated with the events mask. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**7.25.4.19** `int STAR_API_CC TRIGGER_PXI_IF_getPortInputEvents ( STAR_DEVICE_ID deviceld, U32 port, U32 trigger, PORT_EVENT_MASK * pEvents )`

Gets the input events from a port which will cause an internal trigger to be set.

**Parameters**

|                 |                                                                |
|-----------------|----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                                    |
| <i>port</i>     | Port to get input events from.                                 |
| <i>trigger</i>  | Internal trigger which is affected by the input events.        |
| <i>pEvents</i>  | Pointer to a value which will be updated with the events mask. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**7.25.4.20** `int STAR_API_CC TRIGGER_PXI_IF_getTimeCodeInputEvents ( STAR_DEVICE_ID deviceld, U32 timeCode, U32 trigger, TIME_CODE_EVENT_MASK * pEvents )`

Gets the input events from a time-code engine which will cause an internal trigger to be set.

**Parameters**

|                 |                                                                |
|-----------------|----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the time-code engine.                        |
| <i>timeCode</i> | Time-code engine to get input events from.                     |
| <i>trigger</i>  | Internal trigger which is affected by the input events.        |
| <i>pEvents</i>  | Pointer to a value which will be updated with the events mask. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2



7.25.4.21 `int STAR_API_CC TRIGGER_PXI_IF_getTriggerInputEvents ( STAR_DEVICE_ID deviceld, U32 causeTrigger, U32 trigger, TRIGGER_EVENT_MASK * pEvents )`

Gets the input events for an internal trigger which will cause another internal trigger to be set.

**Parameters**

|                     |                                                                |
|---------------------|----------------------------------------------------------------|
| <i>deviceld</i>     | Device containing the internal trigger.                        |
| <i>causeTrigger</i> | Internal trigger to get input events from.                     |
| <i>trigger</i>      | Internal trigger which is affected by the input events.        |
| <i>pEvents</i>      | Pointer to a value which will be updated with the events mask. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**7.25.4.22** `int STAR_API_CC TRIGGER_PXI_IF_setCounterInputEvents ( STAR_DEVICE_ID deviceld, U32 counter, U32 trigger, COUNTER_EVENT_MASK events )`

Sets the input events for a counter which will cause an internal trigger to be set.

**Parameters**

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                          |
| <i>counter</i>  | Counter to set input events for.                        |
| <i>trigger</i>  | Internal trigger which is affected by the input events. |
| <i>events</i>   | Mask describing the input events.                       |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**Examples:**

triggering\_example.c.

**7.25.4.23** `int STAR_API_CC TRIGGER_PXI_IF_setExtTriggerInputEvents ( STAR_DEVICE_ID deviceld, U32 extTrigger, U32 trigger, EXT_TRIGGER_EVENT_MASK events )`

Sets the input events for an external trigger which will cause an internal trigger to be set.

**Parameters**

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceld</i> | Device containing the external trigger. |
|-----------------|-----------------------------------------|

|                   |                                                         |
|-------------------|---------------------------------------------------------|
| <i>extTrigger</i> | External trigger to set input events for.               |
| <i>trigger</i>    | Internal trigger which is affected by the input events. |
| <i>events</i>     | Mask describing the input events.                       |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**Examples:**

triggering\_example.c.

**7.25.4.24** `int STAR_API_CC TRIGGER_PXI_IF_setPortInputEvents ( STAR_DEVICE_ID deviceld, U32 port, U32 trigger, PORT_EVENT_MASK events )`

Sets the input events for a port which will cause an internal trigger to be set.

**Parameters**

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                             |
| <i>port</i>     | Port to set input events for.                           |
| <i>trigger</i>  | Internal trigger which is affected by the input events. |
| <i>events</i>   | Mask describing the input events.                       |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**Examples:**

triggering\_example.c.

**7.25.4.25** `int STAR_API_CC TRIGGER_PXI_IF_setTimeCodeInputEvents ( STAR_DEVICE_ID deviceld, U32 timeCode, U32 trigger, TIME_CODE_EVENT_MASK events )`

Sets the input events for a time-code engine which will cause an internal trigger to be set.

**Parameters**

---

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <i>deviceld</i> | Device containing the time-code engine.                 |
| <i>timeCode</i> | Time-code engine to set input events for.               |
| <i>trigger</i>  | Internal trigger which is affected by the input events. |
| <i>events</i>   | Mask describing the input events.                       |

---

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**Examples:**

triggering\_example.c.

```
7.25.4.26 int STAR_API_CC TRIGGER_PXI_IF_setTriggerInputEvents (STAR_DEVICE_ID deviceld, U32 causeTrigger,
U32 trigger, TRIGGER_EVENT_MASK events)
```

Sets the input events for an internal trigger which will cause another internal trigger to be set.

**Parameters**


---

|                     |                                                         |
|---------------------|---------------------------------------------------------|
| <i>deviceld</i>     | Device containing the internal trigger.                 |
| <i>causeTrigger</i> | Internal trigger to set input events for.               |
| <i>trigger</i>      | Internal trigger which is affected by the input events. |
| <i>events</i>       | Mask describing the input events.                       |

---

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

```
7.25.4.27 int STAR_API_CC TRIGGER_PXI_ROUTER_getCounterInputEvents (STAR_DEVICE_ID deviceld, U32
counter, U32 trigger, COUNTER_EVENT_MASK * pEvents)
```

Gets the input events from a counter which will cause an internal trigger to be set.

**Parameters**


---

|                 |                                   |
|-----------------|-----------------------------------|
| <i>deviceld</i> | Device containing the counter.    |
| <i>counter</i>  | Counter to get input events from. |

---

|                |                                                                |
|----------------|----------------------------------------------------------------|
| <i>trigger</i> | Internal trigger which is affected by the input events.        |
| <i>pEvents</i> | Pointer to a value which will be updated with the events mask. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**7.25.4.28** int **STAR\_API\_CC\_TRIGGER\_PXI\_ROUTER\_getPortInputEvents** ( **STAR\_DEVICE\_ID** *deviceld*, **U32** *port*, **U32** *trigger*, **PORT\_EVENT\_MASK** \* *pEvents* )

Gets the input events from a port which will cause an internal trigger to be set.

**Parameters**

|                 |                                                                |
|-----------------|----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                                    |
| <i>port</i>     | Port to get input events from.                                 |
| <i>trigger</i>  | Internal trigger which is affected by the input events.        |
| <i>pEvents</i>  | Pointer to a value which will be updated with the events mask. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**7.25.4.29** int **STAR\_API\_CC\_TRIGGER\_PXI\_ROUTER\_getTimeCodeInputEvents** ( **STAR\_DEVICE\_ID** *deviceld*, **U32** *timeCode*, **U32** *trigger*, **TIME\_CODE\_EVENT\_MASK** \* *pEvents* )

Gets the input events from a time-code engine which will cause an internal trigger to be set.

**Parameters**

|                 |                                                                |
|-----------------|----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the time-code engine.                        |
| <i>timeCode</i> | Time-code engine to get input events from.                     |
| <i>trigger</i>  | Internal trigger which is affected by the input events.        |
| <i>pEvents</i>  | Pointer to a value which will be updated with the events mask. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

7.25.4.30 **int STAR\_API\_CC TRIGGER\_PXI\_ROUTER\_getTriggerInputEvents ( STAR\_DEVICE\_ID *deviceld*, U32 *causeTrigger*, U32 *trigger*, TRIGGER\_EVENT\_MASK \* *pEvents* )**

Gets the input events for an internal trigger which will cause another internal trigger to be set.

#### Parameters

|                     |                                                                |
|---------------------|----------------------------------------------------------------|
| <i>deviceld</i>     | Device containing the internal trigger.                        |
| <i>causeTrigger</i> | Internal trigger to get input events from.                     |
| <i>trigger</i>      | Internal trigger which is affected by the input events.        |
| <i>pEvents</i>      | Pointer to a value which will be updated with the events mask. |

#### Returns

1 if the operation was successful, else 0

#### Added in version:

STAR-System v3.5 - 17th March 2017

#### Supported devices:

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

7.25.4.31 **int STAR\_API\_CC TRIGGER\_PXI\_ROUTER\_setCounterInputEvents ( STAR\_DEVICE\_ID *deviceld*, U32 *counter*, U32 *trigger*, COUNTER\_EVENT\_MASK *events* )**

Sets the input events for a counter which will cause an internal trigger to be set.

#### Parameters

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                          |
| <i>counter</i>  | Counter to set input events for.                        |
| <i>trigger</i>  | Internal trigger which is affected by the input events. |
| <i>events</i>   | Mask describing the input events.                       |

#### Returns

1 if the operation was successful, else 0

#### Added in version:

STAR-System v3.5 - 17th March 2017

#### Supported devices:

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

#### Examples:

triggering\_example.c.

7.25.4.32 **int STAR\_API\_CC TRIGGER\_PXI\_ROUTER\_setPortInputEvents ( STAR\_DEVICE\_ID *deviceld*, U32 *port*, U32 *trigger*, PORT\_EVENT\_MASK *events* )**

Sets the input events for a port which will cause an internal trigger to be set.

**Parameters**

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                             |
| <i>port</i>     | Port to set input events for.                           |
| <i>trigger</i>  | Internal trigger which is affected by the input events. |
| <i>events</i>   | Mask describing the input events.                       |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**Examples:**

[triggering\\_example.c](#).

**7.25.4.33** `int STAR_API_CC TRIGGER_PXI_ROUTER_setTimeCodeInputEvents ( STAR_DEVICE_ID deviceld, U32 timeCode, U32 trigger, TIME_CODE_EVENT_MASK events )`

Sets the input events for a time-code engine which will cause an internal trigger to be set.

**Parameters**

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <i>deviceld</i> | Device containing the time-code engine.                 |
| <i>timeCode</i> | Time-code engine to set input events for.               |
| <i>trigger</i>  | Internal trigger which is affected by the input events. |
| <i>events</i>   | Mask describing the input events.                       |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**Examples:**

[triggering\\_example.c](#).

**7.25.4.34** `int STAR_API_CC TRIGGER_PXI_ROUTER_setTriggerInputEvents ( STAR_DEVICE_ID deviceld, U32 causeTrigger, U32 trigger, TRIGGER_EVENT_MASK events )`

Sets the input events for an internal trigger which will cause another internal trigger to be set.

**Parameters**

---

|                     |                                                         |
|---------------------|---------------------------------------------------------|
| <i>deviceId</i>     | Device containing the internal trigger.                 |
| <i>causeTrigger</i> | Internal trigger to set input events for.               |
| <i>trigger</i>      | Internal trigger which is affected by the input events. |
| <i>events</i>       | Mask describing the input events.                       |

---

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

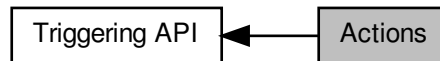
SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2



## 7.26 Actions

Functions in this group deal with setting and getting trigger output actions.

Collaboration diagram for Actions:



### Enumerations

- enum `EXT_TRIGGER_ACTION` { `EXT_TRIGGER_ACTION_OUT` = 1 << 0, `EXT_TRIGGER_ACTION_NONE` = 0, `EXT_TRIGGER_ACTION_ALL` = 0x1 }
- *External trigger actions.*
- enum `COUNTER_ACTION` { `COUNTER_ACTION_COUNT` = 1 << 0, `COUNTER_ACTION_RELOAD` = 1 << 1, `COUNTER_ACTION_START` = 1 << 2, `COUNTER_ACTION_STOP` = 1 << 3, `COUNTER_ACTION_NONE` = 0, `COUNTER_ACTION_ALL` = 0xf }
- *Counter actions.*
- enum `PORT_ACTION` { `PORT_ACTION_TRANSMIT_PKT` = 1 << 0, `PORT_ACTION_DISCONNECT` = 1 << 1, `PORT_ACTION_PARITY_ERROR` = 1 << 2, `PORT_ACTION_ESCAPE_ERROR` = 1 << 3, `PORT_ACTION_INSERT_FCT` = 1 << 4, `PORT_ACTION_SUPPRESS_FCT` = 1 << 5, `PORT_ACTION_INCR_CREDIT` = 1 << 6, `PORT_ACTION_DECR_CREDIT` = 1 << 7, `PORT_ACTION_STOP_RECEPTION` = 1 << 8, `PORT_ACTION_DISABLE_LVDS` = 1 << 9, `PORT_ACTION_NONE` = 0, `PORT_ACTION_ALL` = 0x3ff, `PORT_ACTION_ALL_PCIE` = 0x1ff }
- *Port actions.*
- enum `TIME_CODE_ACTION` { `TIME_CODE_ACTION_TX` = 1 << 0, `TIME_CODE_ACTION_NONE` = 0, `TIME_CODE_ACTION_ALL` = 0x1 }
- *Time-code actions.*

### Functions

- int `STAR_API_CC TRIGGER_BRICK_MK3_setExtTriggerOutputActions` (`STAR_DEVICE_ID` deviceId, U32 extTrigger, U32 trigger, `EXT_TRIGGER_ACTION_MASK` actions)  
*Sets the output actions for an external trigger that are caused by an internal trigger being set.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_setCounterOutputActions` (`STAR_DEVICE_ID` deviceId, U32 counter, U32 trigger, `COUNTER_ACTION_MASK` actions)  
*Sets the output actions for a counter that are caused by an internal trigger being set.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_setPortOutputActions` (`STAR_DEVICE_ID` deviceId, U32 port, U32 trigger, `PORT_ACTION_MASK` actions)  
*Sets the output actions for a port that are caused by an internal trigger being set.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_setTimeCodeOutputActions` (`STAR_DEVICE_ID` deviceId, U32 timeCode, U32 trigger, `TIME_CODE_ACTION_MASK` actions)  
*Sets the output actions for a time-code engine that are caused by an internal trigger being set.*

- `int STAR_API_CC TRIGGER_BRICK_MK3_getExtTriggerOutputActions (STAR_DEVICE_ID deviceId, U32 extTrigger, U32 trigger, EXT_TRIGGER_ACTION_MASK *pActions)`  
*Gets the output actions from an external trigger that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_getCounterOutputActions (STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_ACTION_MASK *pActions)`  
*Gets the output actions from a counter that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_getPortOutputActions (STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_ACTION_MASK *pActions)`  
*Gets the output actions from a port that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_getTimeCodeOutputActions (STAR_DEVICE_ID deviceId, U32 timeCode, U32 trigger, TIME_CODE_ACTION_MASK *pActions)`  
*Gets the output actions from a time-code engine that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_PCIE_IF_setCounterOutputActions (STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_ACTION_MASK actions)`  
*Sets the output actions for a counter that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_PCIE_IF_setPortOutputActions (STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_ACTION_MASK actions)`  
*Sets the output actions for a port that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_PCIE_IF_getCounterOutputActions (STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_ACTION_MASK *pActions)`  
*Gets the output actions from a counter that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_PCIE_IF_getPortOutputActions (STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_ACTION_MASK *pActions)`  
*Gets the output actions from a port that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_PXI_IF_setExtTriggerOutputActions (STAR_DEVICE_ID deviceId, U32 extTrigger, U32 trigger, EXT_TRIGGER_ACTION_MASK actions)`  
*Sets the output actions for an external trigger that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_PXI_IF_setCounterOutputActions (STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_ACTION_MASK actions)`  
*Sets the output actions for a counter that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_PXI_IF_setPortOutputActions (STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_ACTION_MASK actions)`  
*Sets the output actions for a port that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_PXI_IF_setTimeCodeOutputActions (STAR_DEVICE_ID deviceId, U32 timeCode, U32 trigger, TIME_CODE_ACTION_MASK actions)`  
*Sets the output actions for a time-code engine that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_PXI_IF_getExtTriggerOutputActions (STAR_DEVICE_ID deviceId, U32 extTrigger, U32 trigger, EXT_TRIGGER_ACTION_MASK *pActions)`  
*Gets the output actions from an external trigger that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_PXI_IF_getCounterOutputActions (STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_ACTION_MASK *pActions)`  
*Gets the output actions from a counter that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_PXI_IF_getPortOutputActions (STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_ACTION_MASK *pActions)`  
*Gets the output actions from a port that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_PXI_IF_getTimeCodeOutputActions (STAR_DEVICE_ID deviceId, U32 timeCode, U32 trigger, TIME_CODE_ACTION_MASK *pActions)`  
*Gets the output actions from a time-code engine that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_PXI_ROUTER_setCounterOutputActions (STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_ACTION_MASK actions)`  
*Sets the output actions for a counter that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_PXI_ROUTER_setPortOutputActions (STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_ACTION_MASK actions)`

*Sets the output actions for a port that are caused by an internal trigger being set.*

- int `STAR_API_CC_TRIGGER_PXI_ROUTER_setTimeCodeOutputActions` (STAR\_DEVICE\_ID deviceId, U32 timeCode, U32 trigger, TIME\_CODE\_ACTION\_MASK actions)

*Sets the output actions for a time-code engine that are caused by an internal trigger being set.*

- int `STAR_API_CC_TRIGGER_PXI_ROUTER_getCounterOutputActions` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 trigger, COUNTER\_ACTION\_MASK \*pActions)

*Gets the output actions from a counter that are caused by an internal trigger being set.*

- int `STAR_API_CC_TRIGGER_PXI_ROUTER_getPortOutputActions` (STAR\_DEVICE\_ID deviceId, U32 port, U32 trigger, PORT\_ACTION\_MASK \*pActions)

*Gets the output actions from a port that are caused by an internal trigger being set.*

- int `STAR_API_CC_TRIGGER_PXI_ROUTER_getTimeCodeOutputActions` (STAR\_DEVICE\_ID deviceId, U32 timeCode, U32 trigger, TIME\_CODE\_ACTION\_MASK \*pActions)

*Gets the output actions from a time-code engine that are caused by an internal trigger being set.*

## 7.26.1 Detailed Description

Functions in this group deal with setting and getting trigger output actions.

## 7.26.2 Enumeration Type Documentation

### 7.26.2.1 enum COUNTER\_ACTION

Counter actions.

Added in version:

STAR-System v3.0 - 26th August 2016

Enumerator

**COUNTER\_ACTION\_COUNT** Decrement the counter.

**COUNTER\_ACTION\_RELOAD** Reload the counter.

**COUNTER\_ACTION\_START** Start the counter.

**COUNTER\_ACTION\_STOP** Stop the counter.

**COUNTER\_ACTION\_NONE** No actions.

**COUNTER\_ACTION\_ALL** All actions.

### 7.26.2.2 enum EXT\_TRIGGER\_ACTION

External trigger actions.

Added in version:

STAR-System v3.0 - 26th August 2016

Enumerator

**EXT\_TRIGGER\_ACTION\_OUT** Output the external trigger.

**EXT\_TRIGGER\_ACTION\_NONE** No actions.

**EXT\_TRIGGER\_ACTION\_ALL** All actions.

### 7.26.2.3 enum PORT\_ACTION

Port actions.

Added in version:

STAR-System v3.0 - 26th August 2016

Declaration changed in version:

STAR-System v4.00 Beta 4 - 16th October 2019

Enumerator

**PORT\_ACTION\_TRANSMIT\_PKT** Transmit a pending packet.

**PORT\_ACTION\_DISCONNECT** Disconnect the port.

**PORT\_ACTION\_PARITY\_ERROR** Inject a parity error.

**PORT\_ACTION\_ESCAPE\_ERROR** Inject an escape error.

**PORT\_ACTION\_INSERT\_FCT** Insert an FCT.

**PORT\_ACTION\_SUPPRESS\_FCT** Suppress an FCT.

**PORT\_ACTION\_INCR\_CREDIT** Increment credit.

**PORT\_ACTION\_DECR\_CREDIT** Decrement credit.

**PORT\_ACTION\_STOP\_RECEPTION** Stop reception.

Added in version:

STAR-System v3.7 - 13th October 2017

Supported devices:

SpaceWire Brick Mk3 (v1.03 or later).

**PORT\_ACTION\_DISABLE\_LVDS** Disable LVDS drivers.

Added in version:

STAR-System v4.00 Beta 4 - 16th October 2019

Supported devices:

SpaceWire Brick Mk3 (v1.05 or later), SpaceWire PXI Interface and PXI Interface Mk2 (v1.05 or later),  
SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2 (v1.05 or later).

**PORT\_ACTION\_NONE** No actions.

**PORT\_ACTION\_ALL** All actions.

### 7.26.2.4 enum TIME\_CODE\_ACTION

Time-code actions.

Added in version:

STAR-System v3.0 - 26th August 2016

Enumerator

**TIME\_CODE\_ACTION\_TX** Transmit a time-code.

**TIME\_CODE\_ACTION\_NONE** No actions.

**TIME\_CODE\_ACTION\_ALL** All actions.

## 7.26.3 Function Documentation

7.26.3.1 **int STAR\_API\_CC TRIGGER\_BRICK\_MK3\_getCounterOutputActions ( STAR\_DEVICE\_ID *deviceld*, U32 *counter*, U32 *trigger*, COUNTER\_ACTION\_MASK \* *pActions* )**

Gets the output actions from a counter that are caused by an internal trigger being set.

**Parameters**

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>deviceId</i> | Device containing the counter.                                  |
| <i>counter</i>  | Counter to get output actions from.                             |
| <i>trigger</i>  | Internal trigger which causes the output actions.               |
| <i>pActions</i> | Pointer to a value which will be updated with the actions mask. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

**7.26.3.2** `int STAR_API_CC TRIGGER_BRICK_MK3_getExtTriggerOutputActions ( STAR_DEVICE_ID deviceId, U32 extTrigger, U32 trigger, EXT_TRIGGER_ACTION_MASK * pActions )`

Gets the output actions from an external trigger that are caused by an internal trigger being set.

**Parameters**

|                   |                                                                 |
|-------------------|-----------------------------------------------------------------|
| <i>deviceId</i>   | Device containing the external trigger.                         |
| <i>extTrigger</i> | External trigger to get output actions from.                    |
| <i>trigger</i>    | Internal trigger which causes the output actions.               |
| <i>pActions</i>   | Pointer to a value which will be updated with the actions mask. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

**7.26.3.3** `int STAR_API_CC TRIGGER_BRICK_MK3_getPortOutputActions ( STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_ACTION_MASK * pActions )`

Gets the output actions from a port that are caused by an internal trigger being set.

**Parameters**

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>deviceId</i> | Device containing the port.                                     |
| <i>port</i>     | Port to get output actions from.                                |
| <i>trigger</i>  | Internal trigger which causes the output actions.               |
| <i>pActions</i> | Pointer to a value which will be updated with the actions mask. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

**7.26.3.4** `int STAR_API_CC TRIGGER_BRICK_MK3_getTimeCodeOutputActions ( STAR_DEVICE_ID deviceld, U32 timeCode, U32 trigger, TIME_CODE_ACTION_MASK * pActions )`

Gets the output actions from a time-code engine that are caused by an internal trigger being set.

**Parameters**

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the time-code engine.                         |
| <i>timeCode</i> | Time-code engine to get output actions from.                    |
| <i>trigger</i>  | Internal trigger which causes the output actions.               |
| <i>pActions</i> | Pointer to a value which will be updated with the actions mask. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

**7.26.3.5** `int STAR_API_CC TRIGGER_BRICK_MK3_setCounterOutputActions ( STAR_DEVICE_ID deviceld, U32 counter, U32 trigger, COUNTER_ACTION_MASK actions )`

Sets the output actions for a counter that are caused by an internal trigger being set.

**Parameters**

|                 |                                                   |
|-----------------|---------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                    |
| <i>counter</i>  | Counter to set output actions for.                |
| <i>trigger</i>  | Internal trigger which causes the output actions. |
| <i>actions</i>  | Mask describing the output actions                |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

**Examples:**

triggering\_example.c.

**7.26.3.6** `int STAR_API_CC TRIGGER_BRICK_MK3_setExtTriggerOutputActions ( STAR_DEVICE_ID deviceld, U32 extTrigger, U32 trigger, EXT_TRIGGER_ACTION_MASK actions )`

Sets the output actions for an external trigger that are caused by an internal trigger being set.

#### Parameters

|                   |                                                   |
|-------------------|---------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.           |
| <i>extTrigger</i> | External trigger to set output actions for.       |
| <i>trigger</i>    | Internal trigger which causes the output actions. |
| <i>actions</i>    | Mask describing the output actions                |

#### Returns

1 if the operation was successful, else 0

#### Added in version:

STAR-System v3.0 - 26th August 2016

#### Supported devices:

SpaceWire Brick Mk3

#### Examples:

timestamp\_test.c.

**7.26.3.7** `int STAR_API_CC TRIGGER_BRICK_MK3_setPortOutputActions ( STAR_DEVICE_ID deviceld, U32 port, U32 trigger, PORT_ACTION_MASK actions )`

Sets the output actions for a port that are caused by an internal trigger being set.

#### Parameters

|                 |                                                   |
|-----------------|---------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                       |
| <i>port</i>     | Port to set output actions for.                   |
| <i>trigger</i>  | Internal trigger which causes the output actions. |
| <i>actions</i>  | Mask describing the output actions                |

#### Returns

1 if the operation was successful, else 0

#### Added in version:

STAR-System v3.0 - 26th August 2016

#### Supported devices:

SpaceWire Brick Mk3

#### Examples:

triggering\_example.c.

**7.26.3.8** `int STAR_API_CC TRIGGER_BRICK_MK3_setTimeCodeOutputActions ( STAR_DEVICE_ID deviceld, U32 timeCode, U32 trigger, TIME_CODE_ACTION_MASK actions )`

Sets the output actions for a time-code engine that are caused by an internal trigger being set.

**Parameters**

|                 |                                                   |
|-----------------|---------------------------------------------------|
| <i>deviceld</i> | Device containing the time-code engine.           |
| <i>timeCode</i> | Time-code engine to set output actions for.       |
| <i>trigger</i>  | Internal trigger which causes the output actions. |
| <i>actions</i>  | Mask describing the output actions                |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

**Examples:**

triggering\_example.c.

**7.26.3.9** `int STAR_API_CC_TRIGGER_PCIE_IF_getCounterOutputActions ( STAR_DEVICE_ID deviceld, U32 counter, U32 trigger, COUNTER_ACTION_MASK * pActions )`

Gets the output actions from a counter that are caused by an internal trigger being set.

**Parameters**

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                  |
| <i>counter</i>  | Counter to get output actions from.                             |
| <i>trigger</i>  | Internal trigger which causes the output actions.               |
| <i>pActions</i> | Pointer to a value which will be updated with the actions mask. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

**7.26.3.10** `int STAR_API_CC_TRIGGER_PCIE_IF_getPortOutputActions ( STAR_DEVICE_ID deviceld, U32 port, U32 trigger, PORT_ACTION_MASK * pActions )`

Gets the output actions from a port that are caused by an internal trigger being set.

**Parameters**

|                 |                             |
|-----------------|-----------------------------|
| <i>deviceld</i> | Device containing the port. |
|-----------------|-----------------------------|



|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>port</i>     | Port to get output actions from.                                |
| <i>trigger</i>  | Internal trigger which causes the output actions.               |
| <i>pActions</i> | Pointer to a value which will be updated with the actions mask. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

**7.26.3.11** `int STAR_API_CC TRIGGER_PCIE_IF_setCounterOutputActions ( STAR_DEVICE_ID deviceld, U32 counter, U32 trigger, COUNTER_ACTION_MASK actions )`

Sets the output actions for a counter that are caused by an internal trigger being set.

**Parameters**

|                 |                                                   |
|-----------------|---------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                    |
| <i>counter</i>  | Counter to set output actions for.                |
| <i>trigger</i>  | Internal trigger which causes the output actions. |
| <i>actions</i>  | Mask describing the output actions                |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

**7.26.3.12** `int STAR_API_CC TRIGGER_PCIE_IF_setPortOutputActions ( STAR_DEVICE_ID deviceld, U32 port, U32 trigger, PORT_ACTION_MASK actions )`

Sets the output actions for a port that are caused by an internal trigger being set.

**Parameters**

|                 |                                                   |
|-----------------|---------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                       |
| <i>port</i>     | Port to set output actions for.                   |
| <i>trigger</i>  | Internal trigger which causes the output actions. |
| <i>actions</i>  | Mask describing the output actions                |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.1 - 21st October 2016

Supported devices:

SpaceWire PCIe

Examples:

triggering\_example.c.

**7.26.3.13** `int STAR_API_CC TRIGGER_PXI_IF_getCounterOutputActions ( STAR_DEVICE_ID deviceld, U32 counter, U32 trigger, COUNTER_ACTION_MASK * pActions )`

Gets the output actions from a counter that are caused by an internal trigger being set.

**Parameters**

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                  |
| <i>counter</i>  | Counter to get output actions from.                             |
| <i>trigger</i>  | Internal trigger which causes the output actions.               |
| <i>pActions</i> | Pointer to a value which will be updated with the actions mask. |

**Returns**

1 if the operation was successful, else 0

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2

**7.26.3.14** `int STAR_API_CC TRIGGER_PXI_IF_getExtTriggerOutputActions ( STAR_DEVICE_ID deviceld, U32 extTrigger, U32 trigger, EXT_TRIGGER_ACTION_MASK * pActions )`

Gets the output actions from an external trigger that are caused by an internal trigger being set.

**Parameters**

|                   |                                                                 |
|-------------------|-----------------------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.                         |
| <i>extTrigger</i> | External trigger to get output actions from.                    |
| <i>trigger</i>    | Internal trigger which causes the output actions.               |
| <i>pActions</i>   | Pointer to a value which will be updated with the actions mask. |

**Returns**

1 if the operation was successful, else 0

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2

**7.26.3.15** `int STAR_API_CC TRIGGER_PXI_IF_getPortOutputActions ( STAR_DEVICE_ID deviceld, U32 port, U32 trigger, PORT_ACTION_MASK * pActions )`

Gets the output actions from a port that are caused by an internal trigger being set.

**Parameters**

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                                     |
| <i>port</i>     | Port to get output actions from.                                |
| <i>trigger</i>  | Internal trigger which causes the output actions.               |
| <i>pActions</i> | Pointer to a value which will be updated with the actions mask. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

7.26.3.16 **int STAR\_API\_CC TRIGGER\_PXI\_IF\_getTimeCodeOutputActions ( STAR\_DEVICE\_ID *deviceld*, U32 *timeCode*, U32 *trigger*, TIME\_CODE\_ACTION\_MASK \* *pActions* )**

Gets the output actions from a time-code engine that are caused by an internal trigger being set.

**Parameters**

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the time-code engine.                         |
| <i>timeCode</i> | Time-code engine to get output actions from.                    |
| <i>trigger</i>  | Internal trigger which causes the output actions.               |
| <i>pActions</i> | Pointer to a value which will be updated with the actions mask. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

7.26.3.17 **int STAR\_API\_CC TRIGGER\_PXI\_IF\_setCounterOutputActions ( STAR\_DEVICE\_ID *deviceld*, U32 *counter*, U32 *trigger*, COUNTER\_ACTION\_MASK *actions* )**

Sets the output actions for a counter that are caused by an internal trigger being set.

**Parameters**

|                 |                                                   |
|-----------------|---------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                    |
| <i>counter</i>  | Counter to set output actions for.                |
| <i>trigger</i>  | Internal trigger which causes the output actions. |
| <i>actions</i>  | Mask describing the output actions                |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**Examples:**

triggering\_example.c.

**7.26.3.18** `int STAR_API_CC TRIGGER_PXI_IF_setExtTriggerOutputActions ( STAR_DEVICE_ID deviceld, U32 extTrigger, U32 trigger, EXT_TRIGGER_ACTION_MASK actions )`

Sets the output actions for an external trigger that are caused by an internal trigger being set.

**Parameters**

|                   |                                                   |
|-------------------|---------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.           |
| <i>extTrigger</i> | External trigger to set output actions for.       |
| <i>trigger</i>    | Internal trigger which causes the output actions. |
| <i>actions</i>    | Mask describing the output actions                |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**7.26.3.19** `int STAR_API_CC TRIGGER_PXI_IF_setPortOutputActions ( STAR_DEVICE_ID deviceld, U32 port, U32 trigger, PORT_ACTION_MASK actions )`

Sets the output actions for a port that are caused by an internal trigger being set.

**Parameters**

|                 |                                                   |
|-----------------|---------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                       |
| <i>port</i>     | Port to set output actions for.                   |
| <i>trigger</i>  | Internal trigger which causes the output actions. |
| <i>actions</i>  | Mask describing the output actions                |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**Examples:**

triggering\_example.c.

**7.26.3.20** `int STAR_API_CC TRIGGER_PXI_IF_setTimeCodeOutputActions ( STAR_DEVICE_ID deviceld, U32 timeCode, U32 trigger, TIME_CODE_ACTION_MASK actions )`

Sets the output actions for a time-code engine that are caused by an internal trigger being set.

**Parameters**

|                 |                                                   |
|-----------------|---------------------------------------------------|
| <i>deviceld</i> | Device containing the time-code engine.           |
| <i>timeCode</i> | Time-code engine to set output actions for.       |
| <i>trigger</i>  | Internal trigger which causes the output actions. |
| <i>actions</i>  | Mask describing the output actions                |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**Examples:**

triggering\_example.c.

**7.26.3.21** `int STAR_API_CC TRIGGER_PXI_ROUTER_getCounterOutputActions ( STAR_DEVICE_ID deviceld, U32 counter, U32 trigger, COUNTER_ACTION_MASK * pActions )`

Gets the output actions from a counter that are caused by an internal trigger being set.

**Parameters**

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                  |
| <i>counter</i>  | Counter to get output actions from.                             |
| <i>trigger</i>  | Internal trigger which causes the output actions.               |
| <i>pActions</i> | Pointer to a value which will be updated with the actions mask. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

7.26.3.22 `int STAR_API_CC TRIGGER_PXI_ROUTER_getPortOutputActions ( STAR_DEVICE_ID deviceld, U32 port, U32 trigger, PORT_ACTION_MASK * pActions )`

Gets the output actions from a port that are caused by an internal trigger being set.

**Parameters**

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                                     |
| <i>port</i>     | Port to get output actions from.                                |
| <i>trigger</i>  | Internal trigger which causes the output actions.               |
| <i>pActions</i> | Pointer to a value which will be updated with the actions mask. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**7.26.3.23** int **STAR\_API\_CC\_TRIGGER\_PXI\_ROUTER\_getTimeCodeOutputActions** ( **STAR\_DEVICE\_ID** *deviceld*, **U32** *timeCode*, **U32** *trigger*, **TIME\_CODE\_ACTION\_MASK** \* *pActions* )

Gets the output actions from a time-code engine that are caused by an internal trigger being set.

**Parameters**

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the time-code engine.                         |
| <i>timeCode</i> | Time-code engine to get output actions from.                    |
| <i>trigger</i>  | Internal trigger which causes the output actions.               |
| <i>pActions</i> | Pointer to a value which will be updated with the actions mask. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**7.26.3.24** int **STAR\_API\_CC\_TRIGGER\_PXI\_ROUTER\_setCounterOutputActions** ( **STAR\_DEVICE\_ID** *deviceld*, **U32** *counter*, **U32** *trigger*, **COUNTER\_ACTION\_MASK** *actions* )

Sets the output actions for a counter that are caused by an internal trigger being set.

**Parameters**

|                 |                                                   |
|-----------------|---------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                    |
| <i>counter</i>  | Counter to set output actions for.                |
| <i>trigger</i>  | Internal trigger which causes the output actions. |
| <i>actions</i>  | Mask describing the output actions                |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**Examples:**

triggering\_example.c.

**7.26.3.25** `int STAR_API_CC TRIGGER_PXI_ROUTER_setPortOutputActions ( STAR_DEVICE_ID deviceld, U32 port, U32 trigger, PORT_ACTION_MASK actions )`

Sets the output actions for a port that are caused by an internal trigger being set.

**Parameters**

|                 |                                                   |
|-----------------|---------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                       |
| <i>port</i>     | Port to set output actions for.                   |
| <i>trigger</i>  | Internal trigger which causes the output actions. |
| <i>actions</i>  | Mask describing the output actions                |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**Examples:**

triggering\_example.c.

**7.26.3.26** `int STAR_API_CC TRIGGER_PXI_ROUTER_setTimeCodeOutputActions ( STAR_DEVICE_ID deviceld, U32 timeCode, U32 trigger, TIME_CODE_ACTION_MASK actions )`

Sets the output actions for a time-code engine that are caused by an internal trigger being set.

**Parameters**

|                 |                                                   |
|-----------------|---------------------------------------------------|
| <i>deviceld</i> | Device containing the time-code engine.           |
| <i>timeCode</i> | Time-code engine to set output actions for.       |
| <i>trigger</i>  | Internal trigger which causes the output actions. |
| <i>actions</i>  | Mask describing the output actions                |



**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

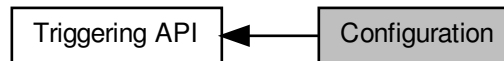
**Examples:**

triggering\_example.c.

## 7.27 Configuration

Functions in this group deal with configuring the different triggering subsystems.

Collaboration diagram for Configuration:



### Enumerations

- enum `TRIGGER_INPUT_MODE` { `TRIGGER_INPUT_MODE_OR` = 0, `TRIGGER_INPUT_MODE_AND` = 1 }  
*Internal trigger input modes.*

### Functions

- int `STAR_API_CC TRIGGER_BRICK_MK3_enableExtTriggerEdgeDetectMode` (`STAR_DEVICE_ID` deviceId, U32 extTrigger)  
*Enables edge detect mode for a specific external trigger.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_disableExtTriggerEdgeDetectMode` (`STAR_DEVICE_ID` deviceId, U32 extTrigger)  
*Disables edge detect mode for a specific external trigger.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_getExtTriggerEdgeDetectModeEnabled` (`STAR_DEVICE_ID` deviceId, U32 extTrigger, U32 \*pEnabled)  
*Gets whether edge detect mode is enabled for a specific external trigger.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_enableExtTriggerInvert` (`STAR_DEVICE_ID` deviceId, U32 extTrigger)  
*Enables invert for a specific external trigger.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_disableExtTriggerInvert` (`STAR_DEVICE_ID` deviceId, U32 extTrigger)  
*Disables invert for a specific external trigger.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_getExtTriggerInvertEnabled` (`STAR_DEVICE_ID` deviceId, U32 extTrigger, U32 \*pEnabled)  
*Gets whether invert is enabled for a specific external trigger.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_enableExtTriggerOutput` (`STAR_DEVICE_ID` deviceId, U32 extTrigger)  
*Enables output for a specific external trigger.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_disableExtTriggerOutput` (`STAR_DEVICE_ID` deviceId, U32 extTrigger)  
*Disables output for a specific external trigger.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_getExtTriggerOutputEnabled` (`STAR_DEVICE_ID` deviceId, U32 extTrigger, U32 \*pEnabled)  
*Gets whether output is enabled for a specific external trigger.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_setExtTriggerExtend` (`STAR_DEVICE_ID` deviceId, U32 extTrigger, U32 extend)  
*Sets the extend duration for a specific external trigger.*

- `int STAR_API_CC TRIGGER_BRICK_MK3_getExtTriggerExtend (STAR_DEVICE_ID deviceId, U32 extTrigger, U32 *pExtend)`  
*Gets the extend duration for a specific external trigger.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_enableCounterAutoReload (STAR_DEVICE_ID deviceId, U32 counter)`  
*Enables auto reload for a specific counter.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_disableCounterAutoReload (STAR_DEVICE_ID deviceId, U32 counter)`  
*Disables auto reload for a specific counter.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_getCounterAutoReloadEnabled (STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)`  
*Gets whether auto reload is enabled for a specific counter.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_enableCounterTriggerCount (STAR_DEVICE_ID deviceId, U32 counter)`  
*Enables trigger count for a specific counter.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_disableCounterTriggerCount (STAR_DEVICE_ID deviceId, U32 counter)`  
*Disables trigger count for a specific counter.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_getCounterTriggerCountEnabled (STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)`  
*Gets whether trigger count is enabled for a specific counter.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_enableCounterStartMode (STAR_DEVICE_ID deviceId, U32 counter)`  
*Enables start mode for a specific counter.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_disableCounterStartMode (STAR_DEVICE_ID deviceId, U32 counter)`  
*Disables start mode for a specific counter.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_getCounterStartModeEnabled (STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)`  
*Gets whether start mode is enabled for a specific counter.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_enableCounterStartStopMode (STAR_DEVICE_ID deviceId, U32 counter)`  
*Enables start stop mode for a specific counter.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_disableCounterStartStopMode (STAR_DEVICE_ID deviceId, U32 counter)`  
*Disables start stop mode for a specific counter.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_getCounterStartStopModeEnabled (STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)`  
*Gets whether start stop mode is enabled for a specific counter.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_enableCounterLoadZero (STAR_DEVICE_ID deviceId, U32 counter)`  
*Enables load zero for a specific counter.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_disableCounterLoadZero (STAR_DEVICE_ID deviceId, U32 counter)`  
*Disables load zero for a specific counter.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_getCounterLoadZeroEnabled (STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)`  
*Gets whether load zero is enabled for a specific counter.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_setCounterReloadValue (STAR_DEVICE_ID deviceId, U32 counter, U32 reloadValue)`  
*Sets the reload value for a specific counter.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_getCounterReloadValue (STAR_DEVICE_ID deviceId, U32 counter, U32 *pReloadValue)`

- Gets the reload value for a specific counter.*

  - int `STAR_API_CC_TRIGGER_BRICK_MK3_forceCounterReload` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Force the reload of a specific counter.*
- int `STAR_API_CC_TRIGGER_BRICK_MK3_getCounterValue` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pValue)

*Gets the counter value for a specific counter.*
- int `STAR_API_CC_TRIGGER_BRICK_MK3_enablePortTransmitPacketMode` (STAR\_DEVICE\_ID deviceId, U32 port)

*Enables packet transmit mode for a specific port.*
- int `STAR_API_CC_TRIGGER_BRICK_MK3_disablePortTransmitPacketMode` (STAR\_DEVICE\_ID deviceId, U32 port)

*Disables packet transmit mode for a specific port.*
- int `STAR_API_CC_TRIGGER_BRICK_MK3_getPortTransmitPacketModeEnabled` (STAR\_DEVICE\_ID deviceId, U32 port, U32 \*pEnabled)

*Gets whether packet transmit mode is enabled for a specific port.*
- int `STAR_API_CC_TRIGGER_BRICK_MK3_setTriggerInputMode` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_INPUT\_MODE mode)

*Sets the input mode for a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_BRICK_MK3_getTriggerInputMode` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_INPUT\_MODE \*pMode)

*Gets the input mode for a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_BRICK_MK3_enableTrigger` (STAR\_DEVICE\_ID deviceId, U32 trigger)

*Enables a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_BRICK_MK3_disableTrigger` (STAR\_DEVICE\_ID deviceId, U32 trigger)

*Disables a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_BRICK_MK3_getTriggerEnabled` (STAR\_DEVICE\_ID deviceId, U32 trigger, U32 \*pEnabled)

*Gets whether a trigger is enabled.*
- int `STAR_API_CC_TRIGGER_BRICK_MK3_forceTrigger` (STAR\_DEVICE\_ID deviceId, U32 trigger)

*Force the triggering of a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_BRICK_MK3_setTriggerInvertMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 mask)

*Sets the invert mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_BRICK_MK3_getTriggerInvertMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 \*pMask)

*Gets the invert mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_BRICK_MK3_setTriggerAndMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 mask)

*Sets the AND/OR mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_BRICK_MK3_getTriggerAndMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 \*pMask)

*Gets the AND/OR mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_PCIE_IF_enableCounterAutoReload` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Enables auto reload for a specific counter.*
- int `STAR_API_CC_TRIGGER_PCIE_IF_disableCounterAutoReload` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Disables auto reload for a specific counter.*
- int `STAR_API_CC_TRIGGER_PCIE_IF_getCounterAutoReloadEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)

*Gets whether auto reload is enabled for a specific counter.*
- int `STAR_API_CC_TRIGGER_PCIE_IF_enableCounterTriggerCount` (STAR\_DEVICE\_ID deviceId, U32 counter)

- Enables trigger count for a specific counter.*

  - int `STAR_API_CC_TRIGGER_PCIE_IF_disableCounterTriggerCount` (STAR\_DEVICE\_ID deviceId, U32 counter)
- Disables trigger count for a specific counter.*

  - int `STAR_API_CC_TRIGGER_PCIE_IF_getCounterTriggerCountEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)
- Gets whether trigger count is enabled for a specific counter.*

  - int `STAR_API_CC_TRIGGER_PCIE_IF_enableCounterStartMode` (STAR\_DEVICE\_ID deviceId, U32 counter)
- Enables start mode for a specific counter.*

  - int `STAR_API_CC_TRIGGER_PCIE_IF_disableCounterStartMode` (STAR\_DEVICE\_ID deviceId, U32 counter)
- Disables start mode for a specific counter.*

  - int `STAR_API_CC_TRIGGER_PCIE_IF_getCounterStartModeEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)
- Gets whether start mode is enabled for a specific counter.*

  - int `STAR_API_CC_TRIGGER_PCIE_IF_enableCounterStartStopMode` (STAR\_DEVICE\_ID deviceId, U32 counter)
- Enables start stop mode for a specific counter.*

  - int `STAR_API_CC_TRIGGER_PCIE_IF_disableCounterStartStopMode` (STAR\_DEVICE\_ID deviceId, U32 counter)
- Disables start stop mode for a specific counter.*

  - int `STAR_API_CC_TRIGGER_PCIE_IF_getCounterStartStopModeEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)
- Gets whether start stop mode is enabled for a specific counter.*

  - int `STAR_API_CC_TRIGGER_PCIE_IF_enableCounterLoadZero` (STAR\_DEVICE\_ID deviceId, U32 counter)
- Enables load zero for a specific counter.*

  - int `STAR_API_CC_TRIGGER_PCIE_IF_disableCounterLoadZero` (STAR\_DEVICE\_ID deviceId, U32 counter)
- Disables load zero for a specific counter.*

  - int `STAR_API_CC_TRIGGER_PCIE_IF_getCounterLoadZeroEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)
- Gets whether load zero is enabled for a specific counter.*

  - int `STAR_API_CC_TRIGGER_PCIE_IF_setCounterReloadValue` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 reloadValue)
- Sets the reload value for a specific counter.*

  - int `STAR_API_CC_TRIGGER_PCIE_IF_getCounterReloadValue` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pReloadValue)
- Gets the reload value for a specific counter.*

  - int `STAR_API_CC_TRIGGER_PCIE_IF_forceCounterReload` (STAR\_DEVICE\_ID deviceId, U32 counter)
- Force the reload of a specific counter.*

  - int `STAR_API_CC_TRIGGER_PCIE_IF_getCounterValue` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pValue)
- Gets the counter value for a specific counter.*

  - int `STAR_API_CC_TRIGGER_PCIE_IF_setTriggerInputMode` (STAR\_DEVICE\_ID deviceId, U32 trigger, T↔RIGGER\_INPUT\_MODE mode)
- Sets the input mode for a specific internal trigger.*

  - int `STAR_API_CC_TRIGGER_PCIE_IF_getTriggerInputMode` (STAR\_DEVICE\_ID deviceId, U32 trigger, T↔RIGGER\_INPUT\_MODE \*pMode)
- Gets the input mode for a specific internal trigger.*

  - int `STAR_API_CC_TRIGGER_PCIE_IF_enableTrigger` (STAR\_DEVICE\_ID deviceId, U32 trigger)
- Enables a specific internal trigger.*

  - int `STAR_API_CC_TRIGGER_PCIE_IF_disableTrigger` (STAR\_DEVICE\_ID deviceId, U32 trigger)

- Disables a specific internal trigger.*

  - int `STAR_API_CC_TRIGGER_PCIE_IF_getTriggerEnabled` (STAR\_DEVICE\_ID deviceId, U32 trigger, U32 \*pEnabled)

*Gets whether a trigger is enabled.*
- int `STAR_API_CC_TRIGGER_PCIE_IF_forceTrigger` (STAR\_DEVICE\_ID deviceId, U32 trigger)

*Force the triggering of a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_PCIE_IF_setTriggerInvertMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 mask)

*Sets the invert mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_PCIE_IF_getTriggerInvertMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 \*pMask)

*Gets the invert mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_PCIE_IF_setTriggerAndMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 mask)

*Sets the AND/OR mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_PCIE_IF_getTriggerAndMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 \*pMask)

*Gets the AND/OR mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_PXI_IF_enableExtTriggerEdgeDetectMode` (STAR\_DEVICE\_ID deviceId, U32 extTrigger)

*Enables edge detect mode for a specific external trigger.*
- int `STAR_API_CC_TRIGGER_PXI_IF_disableExtTriggerEdgeDetectMode` (STAR\_DEVICE\_ID deviceId, U32 extTrigger)

*Disables edge detect mode for a specific external trigger.*
- int `STAR_API_CC_TRIGGER_PXI_IF_getExtTriggerEdgeDetectModeEnabled` (STAR\_DEVICE\_ID deviceId, U32 extTrigger, U32 \*pEnabled)

*Gets whether edge detect mode is enabled for a specific external trigger.*
- int `STAR_API_CC_TRIGGER_PXI_IF_enableExtTriggerInvert` (STAR\_DEVICE\_ID deviceId, U32 extTrigger)

*Enables invert for a specific external trigger.*
- int `STAR_API_CC_TRIGGER_PXI_IF_disableExtTriggerInvert` (STAR\_DEVICE\_ID deviceId, U32 extTrigger)

*Disables invert for a specific external trigger.*
- int `STAR_API_CC_TRIGGER_PXI_IF_getExtTriggerInvertEnabled` (STAR\_DEVICE\_ID deviceId, U32 extTrigger, U32 \*pEnabled)

*Gets whether invert is enabled for a specific external trigger.*
- int `STAR_API_CC_TRIGGER_PXI_IF_enableExtTriggerOutput` (STAR\_DEVICE\_ID deviceId, U32 extTrigger)

*Enables output for a specific external trigger.*
- int `STAR_API_CC_TRIGGER_PXI_IF_disableExtTriggerOutput` (STAR\_DEVICE\_ID deviceId, U32 extTrigger)

*Disables output for a specific external trigger.*
- int `STAR_API_CC_TRIGGER_PXI_IF_getExtTriggerOutputEnabled` (STAR\_DEVICE\_ID deviceId, U32 extTrigger, U32 \*pEnabled)

*Gets whether output is enabled for a specific external trigger.*
- int `STAR_API_CC_TRIGGER_PXI_IF_setExtTriggerExtend` (STAR\_DEVICE\_ID deviceId, U32 extTrigger, U32 extend)

*Sets the extend duration for a specific external trigger.*
- int `STAR_API_CC_TRIGGER_PXI_IF_getExtTriggerExtend` (STAR\_DEVICE\_ID deviceId, U32 extTrigger, U32 \*pExtend)

*Gets the extend duration for a specific external trigger.*
- int `STAR_API_CC_TRIGGER_PXI_IF_enableCounterAutoReload` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Enables auto reload for a specific counter.*
- int `STAR_API_CC_TRIGGER_PXI_IF_disableCounterAutoReload` (STAR\_DEVICE\_ID deviceId, U32 counter)

- Disables auto reload for a specific counter.*

  - int `STAR_API_CC_TRIGGER_PXI_IF_getCounterAutoReloadEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)

*Gets whether auto reload is enabled for a specific counter.*
- int `STAR_API_CC_TRIGGER_PXI_IF_enableCounterTriggerCount` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Enables trigger count for a specific counter.*
- int `STAR_API_CC_TRIGGER_PXI_IF_disableCounterTriggerCount` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Disables trigger count for a specific counter.*
- int `STAR_API_CC_TRIGGER_PXI_IF_getCounterTriggerCountEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)

*Gets whether trigger count is enabled for a specific counter.*
- int `STAR_API_CC_TRIGGER_PXI_IF_enableCounterStartMode` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Enables start mode for a specific counter.*
- int `STAR_API_CC_TRIGGER_PXI_IF_disableCounterStartMode` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Disables start mode for a specific counter.*
- int `STAR_API_CC_TRIGGER_PXI_IF_getCounterStartModeEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)

*Gets whether start mode is enabled for a specific counter.*
- int `STAR_API_CC_TRIGGER_PXI_IF_enableCounterStartStopMode` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Enables start stop mode for a specific counter.*
- int `STAR_API_CC_TRIGGER_PXI_IF_disableCounterStartStopMode` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Disables start stop mode for a specific counter.*
- int `STAR_API_CC_TRIGGER_PXI_IF_getCounterStartStopModeEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)

*Gets whether start stop mode is enabled for a specific counter.*
- int `STAR_API_CC_TRIGGER_PXI_IF_enableCounterLoadZero` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Enables load zero for a specific counter.*
- int `STAR_API_CC_TRIGGER_PXI_IF_disableCounterLoadZero` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Disables load zero for a specific counter.*
- int `STAR_API_CC_TRIGGER_PXI_IF_getCounterLoadZeroEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)

*Gets whether load zero is enabled for a specific counter.*
- int `STAR_API_CC_TRIGGER_PXI_IF_setCounterReloadValue` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 reloadValue)

*Sets the reload value for a specific counter.*
- int `STAR_API_CC_TRIGGER_PXI_IF_getCounterReloadValue` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pReloadValue)

*Gets the reload value for a specific counter.*
- int `STAR_API_CC_TRIGGER_PXI_IF_forceCounterReload` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Force the reload of a specific counter.*
- int `STAR_API_CC_TRIGGER_PXI_IF_getCounterValue` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pValue)

*Gets the counter value for a specific counter.*
- int `STAR_API_CC_TRIGGER_PXI_IF_enablePortTransmitPacketMode` (STAR\_DEVICE\_ID deviceId, U32 port)

*Enables packet transmit mode for a specific port.*
- int `STAR_API_CC_TRIGGER_PXI_IF_disablePortTransmitPacketMode` (STAR\_DEVICE\_ID deviceId, U32 port)



- Disables packet transmit mode for a specific port.*
- int `STAR_API_CC_TRIGGER_PXI_IF_getPortTransmitPacketModeEnabled` (STAR\_DEVICE\_ID deviceId, U32 port, U32 \*pEnabled)
- Gets whether packet transmit mode is enabled for a specific port.*
- int `STAR_API_CC_TRIGGER_PXI_IF_setTriggerInputMode` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_INPUT\_MODE mode)
- Sets the input mode for a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_PXI_IF_getTriggerInputMode` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_INPUT\_MODE \*pMode)
- Gets the input mode for a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_PXI_IF_enableTrigger` (STAR\_DEVICE\_ID deviceId, U32 trigger)
- Enables a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_PXI_IF_disableTrigger` (STAR\_DEVICE\_ID deviceId, U32 trigger)
- Disables a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_PXI_IF_getTriggerEnabled` (STAR\_DEVICE\_ID deviceId, U32 trigger, U32 \*pEnabled)
- Gets whether a trigger is enabled.*
- int `STAR_API_CC_TRIGGER_PXI_IF_forceTrigger` (STAR\_DEVICE\_ID deviceId, U32 trigger)
- Force the triggering of a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_PXI_IF_setTriggerInvertMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 mask)
- Sets the invert mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_PXI_IF_getTriggerInvertMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 \*pMask)
- Gets the invert mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_PXI_IF_setTriggerAndMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 mask)
- Sets the AND/OR mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_PXI_IF_getTriggerAndMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 \*pMask)
- Gets the AND/OR mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_PXI_ROUTER_enableCounterAutoReload` (STAR\_DEVICE\_ID deviceId, U32 counter)
- Enables auto reload for a specific counter.*
- int `STAR_API_CC_TRIGGER_PXI_ROUTER_disableCounterAutoReload` (STAR\_DEVICE\_ID deviceId, U32 counter)
- Disables auto reload for a specific counter.*
- int `STAR_API_CC_TRIGGER_PXI_ROUTER_getCounterAutoReloadEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)
- Gets whether auto reload is enabled for a specific counter.*
- int `STAR_API_CC_TRIGGER_PXI_ROUTER_enableCounterTriggerCount` (STAR\_DEVICE\_ID deviceId, U32 counter)
- Enables trigger count for a specific counter.*
- int `STAR_API_CC_TRIGGER_PXI_ROUTER_disableCounterTriggerCount` (STAR\_DEVICE\_ID deviceId, U32 counter)
- Disables trigger count for a specific counter.*
- int `STAR_API_CC_TRIGGER_PXI_ROUTER_getCounterTriggerCountEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)
- Gets whether trigger count is enabled for a specific counter.*
- int `STAR_API_CC_TRIGGER_PXI_ROUTER_enableCounterStartMode` (STAR\_DEVICE\_ID deviceId, U32 counter)
- Enables start mode for a specific counter.*



- int `STAR_API_CC_TRIGGER_PXI_ROUTER_disableCounterStartMode` (STAR\_DEVICE\_ID deviceId, U32 counter)  
*Disables start mode for a specific counter.*
- int `STAR_API_CC_TRIGGER_PXI_ROUTER_getCounterStartModeEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)  
*Gets whether start mode is enabled for a specific counter.*
- int `STAR_API_CC_TRIGGER_PXI_ROUTER_enableCounterStartStopMode` (STAR\_DEVICE\_ID deviceId, U32 counter)  
*Enables start stop mode for a specific counter.*
- int `STAR_API_CC_TRIGGER_PXI_ROUTER_disableCounterStartStopMode` (STAR\_DEVICE\_ID deviceId, U32 counter)  
*Disables start stop mode for a specific counter.*
- int `STAR_API_CC_TRIGGER_PXI_ROUTER_getCounterStartStopModeEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)  
*Gets whether start stop mode is enabled for a specific counter.*
- int `STAR_API_CC_TRIGGER_PXI_ROUTER_enableCounterLoadZero` (STAR\_DEVICE\_ID deviceId, U32 counter)  
*Enables load zero for a specific counter.*
- int `STAR_API_CC_TRIGGER_PXI_ROUTER_disableCounterLoadZero` (STAR\_DEVICE\_ID deviceId, U32 counter)  
*Disables load zero for a specific counter.*
- int `STAR_API_CC_TRIGGER_PXI_ROUTER_getCounterLoadZeroEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)  
*Gets whether load zero is enabled for a specific counter.*
- int `STAR_API_CC_TRIGGER_PXI_ROUTER_setCounterReloadValue` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 reloadValue)  
*Sets the reload value for a specific counter.*
- int `STAR_API_CC_TRIGGER_PXI_ROUTER_getCounterReloadValue` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pReloadValue)  
*Gets the reload value for a specific counter.*
- int `STAR_API_CC_TRIGGER_PXI_ROUTER_forceCounterReload` (STAR\_DEVICE\_ID deviceId, U32 counter)  
*Force the reload of a specific counter.*
- int `STAR_API_CC_TRIGGER_PXI_ROUTER_getCounterValue` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pValue)  
*Gets the counter value for a specific counter.*
- int `STAR_API_CC_TRIGGER_PXI_ROUTER_enablePortTransmitPacketMode` (STAR\_DEVICE\_ID deviceId, U32 port)  
*Enables packet transmit mode for a specific port.*
- int `STAR_API_CC_TRIGGER_PXI_ROUTER_disablePortTransmitPacketMode` (STAR\_DEVICE\_ID deviceId, U32 port)  
*Disables packet transmit mode for a specific port.*
- int `STAR_API_CC_TRIGGER_PXI_ROUTER_getPortTransmitPacketModeEnabled` (STAR\_DEVICE\_ID deviceId, U32 port, U32 \*pEnabled)  
*Gets whether packet transmit mode is enabled for a specific port.*
- int `STAR_API_CC_TRIGGER_PXI_ROUTER_setTriggerInputMode` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_INPUT\_MODE mode)  
*Sets the input mode for a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_PXI_ROUTER_getTriggerInputMode` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_INPUT\_MODE \*pMode)  
*Gets the input mode for a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_PXI_ROUTER_enableTrigger` (STAR\_DEVICE\_ID deviceId, U32 trigger)  
*Enables a specific internal trigger.*

- int `STAR_API_CC TRIGGER_PXI_ROUTER_disableTrigger` (STAR\_DEVICE\_ID deviceId, U32 trigger)  
*Disables a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PXI_ROUTER_getTriggerEnabled` (STAR\_DEVICE\_ID deviceId, U32 trigger, U32 \*pEnabled)  
*Gets whether a trigger is enabled.*
- int `STAR_API_CC TRIGGER_PXI_ROUTER_forceTrigger` (STAR\_DEVICE\_ID deviceId, U32 trigger)  
*Force the triggering of a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PXI_ROUTER_setTriggerInvertMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 mask)  
*Sets the invert mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PXI_ROUTER_getTriggerInvertMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 \*pMask)  
*Gets the invert mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PXI_ROUTER_setTriggerAndMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 mask)  
*Sets the AND/OR mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PXI_ROUTER_getTriggerAndMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 \*pMask)  
*Gets the AND/OR mask for a trigger type for a specific internal trigger.*

### 7.27.1 Detailed Description

Functions in this group deal with configuring the different triggering subsystems.

### 7.27.2 Enumeration Type Documentation

#### 7.27.2.1 enum TRIGGER\_INPUT\_MODE

Internal trigger input modes.

Added in version:

STAR-System v3.0 - 26th August 2016

Enumerator

**TRIGGER\_INPUT\_MODE\_OR** Trigger is a sum of its inputs.

**TRIGGER\_INPUT\_MODE\_AND** Trigger is a product of its inputs.

### 7.27.3 Function Documentation

#### 7.27.3.1 int `STAR_API_CC TRIGGER_BRICK_MK3_disableCounterAutoReload` ( STAR\_DEVICE\_ID deviceId, U32 counter )

Disables auto reload for a specific counter.

When disabled, the counter will not automatically reload with its reload value when the counter reaches zero.

Parameters

---

|                 |                                |
|-----------------|--------------------------------|
| <i>deviceId</i> | Device containing the counter. |
|-----------------|--------------------------------|

---

---

|                |                                    |
|----------------|------------------------------------|
| <i>counter</i> | Counter to disable auto reload on. |
|----------------|------------------------------------|

---

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

**Examples:**

triggering\_example.c.

```
7.27.3.2 int STAR_API_CC TRIGGER_BRICK_MK3_disableCounterLoadZero (STAR_DEVICE_ID deviceld, U32 counter)
```

Disables load zero for a specific counter.

When disabled, reload triggered events occur even when the counter is not zero.

**Parameters**


---

|                 |                                  |
|-----------------|----------------------------------|
| <i>deviceld</i> | Device containing the counter.   |
| <i>counter</i>  | Counter to disable load zero on. |

---

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

```
7.27.3.3 int STAR_API_CC TRIGGER_BRICK_MK3_disableCounterStartMode (STAR_DEVICE_ID deviceld, U32 counter)
```

Disables start mode for a specific counter.

When disabled, the counter will not wait for a start action to occur before any counting or reloading can take place.

**Parameters**


---

|                 |                                   |
|-----------------|-----------------------------------|
| <i>deviceld</i> | Device containing the counter.    |
| <i>counter</i>  | Counter to disable start mode on. |

---

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

---

**Supported devices:**

SpaceWire Brick Mk3

**7.27.3.4 int STAR\_API\_CC TRIGGER\_BRICK\_MK3\_disableCounterStartStopMode ( STAR\_DEVICE\_ID *deviceld*, U32 *counter* )**

Disables start stop mode for a specific counter.

When disabled, the counter will not wait for a start action to occur before any counting or reloading can take place.

**Parameters**

|                 |                                        |
|-----------------|----------------------------------------|
| <i>deviceld</i> | Device containing the counter.         |
| <i>counter</i>  | Counter to disable start stop mode on. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

**7.27.3.5 int STAR\_API\_CC TRIGGER\_BRICK\_MK3\_disableCounterTriggerCount ( STAR\_DEVICE\_ID *deviceld*, U32 *counter* )**

Disables trigger count for a specific counter.

When disabled, the counter will be decremented every clock cycle.

**Parameters**

|                 |                                      |
|-----------------|--------------------------------------|
| <i>deviceld</i> | Device containing the counter.       |
| <i>counter</i>  | Counter to disable trigger count on. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

**7.27.3.6 int STAR\_API\_CC TRIGGER\_BRICK\_MK3\_disableExtTriggerEdgeDetectMode ( STAR\_DEVICE\_ID *deviceld*, U32 *extTrigger* )**

Disables edge detect mode for a specific external trigger.

When disabled, the trigger will always be set when the input trigger is high.

**Parameters**

---

|                   |                                                  |
|-------------------|--------------------------------------------------|
| <i>deviceId</i>   | Device containing the external trigger.          |
| <i>extTrigger</i> | External trigger to disable edge detect mode on. |

---

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

7.27.3.7 `int STAR_API_CC TRIGGER_BRICK_MK3_disableExtTriggerInvert ( STAR_DEVICE_ID deviceId, U32 extTrigger )`

Disables invert for a specific external trigger.

When disabled, the input trigger signal is not modified before the detection mechanism.

**Parameters**

---

|                   |                                         |
|-------------------|-----------------------------------------|
| <i>deviceId</i>   | Device containing the external trigger. |
| <i>extTrigger</i> | External trigger to disable invert on.  |

---

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

**Examples:**

timestamp\_test.c.

7.27.3.8 `int STAR_API_CC TRIGGER_BRICK_MK3_disableExtTriggerOutput ( STAR_DEVICE_ID deviceId, U32 extTrigger )`

Disables output for a specific external trigger.

When disabled, the external trigger's output action does not cause the trigger output signal to be pulsed.

**Parameters**

---

|                   |                                         |
|-------------------|-----------------------------------------|
| <i>deviceId</i>   | Device containing the external trigger. |
| <i>extTrigger</i> | External trigger to disable output on.  |

---

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

**Examples:**

timestamp\_test.c.

**7.27.3.9** `int STAR_API_CC TRIGGER_BRICK_MK3_disablePortTransmitPacketMode ( STAR_DEVICE_ID deviceld, U32 port )`

Disables packet transmit mode for a specific port.

When disabled, packets do not wait for a transmit packet action to occur before transmission.

**Parameters**

|                 |                                          |
|-----------------|------------------------------------------|
| <i>deviceld</i> | Device containing the port.              |
| <i>port</i>     | Port to disable packet transmit mode on. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

**7.27.3.10** `int STAR_API_CC TRIGGER_BRICK_MK3_disableTrigger ( STAR_DEVICE_ID deviceld, U32 trigger )`

Disables a specific internal trigger.

When disabled, the trigger is not set by its input events and does not cause output actions to occur.

**Parameters**

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger. |
| <i>trigger</i>  | Internal trigger to disable.            |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

**Examples:**

[timestamp\\_test.c](#).

**7.27.3.11 int STAR\_API\_CC TRIGGER\_BRICK\_MK3\_enableCounterAutoReload ( STAR\_DEVICE\_ID *deviceld*, U32 *counter* )**

Enables auto reload for a specific counter.

When enabled, the counter will automatically reload with its reload value when the counter reaches zero.

**Parameters**

|                 |                                   |
|-----------------|-----------------------------------|
| <i>deviceld</i> | Device containing the counter.    |
| <i>counter</i>  | Counter to enable auto reload on. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

**Examples:**

[timestamp\\_test.c](#), and [triggering\\_example.c](#).

**7.27.3.12 int STAR\_API\_CC TRIGGER\_BRICK\_MK3\_enableCounterLoadZero ( STAR\_DEVICE\_ID *deviceld*, U32 *counter* )**

Enables load zero for a specific counter.

When enabled, reload triggered events only occur when the counter is zero.

**Parameters**

|                 |                                 |
|-----------------|---------------------------------|
| <i>deviceld</i> | Device containing the counter.  |
| <i>counter</i>  | Counter to enable load zero on. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

7.27.3.13 `int STAR_API_CC TRIGGER_BRICK_MK3_enableCounterStartMode ( STAR_DEVICE_ID deviceld, U32 counter )`

Enables start mode for a specific counter.

When enabled, the counter will wait for a start action to occur before any counting or reloading can take place. When the counter reaches zero, the counter may be reloaded but no counting or reloading can take place until the next start action occurs.



**Parameters**

|                 |                                  |
|-----------------|----------------------------------|
| <i>deviceld</i> | Device containing the counter.   |
| <i>counter</i>  | Counter to enable start mode on. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

**7.27.3.14** `int STAR_API_CC TRIGGER_BRICK_MK3_enableCounterStartStopMode ( STAR_DEVICE_ID deviceld, U32 counter )`

Enables start stop mode for a specific counter.

When enabled, the counter will wait for a start action to occur before any counting or reloading can take place. The counter will continue to count and reload until a stop action occurs.

**Parameters**

|                 |                                       |
|-----------------|---------------------------------------|
| <i>deviceld</i> | Device containing the counter.        |
| <i>counter</i>  | Counter to enable start stop mode on. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

**Examples:**

`triggering_example.c`.

**7.27.3.15** `int STAR_API_CC TRIGGER_BRICK_MK3_enableCounterTriggerCount ( STAR_DEVICE_ID deviceld, U32 counter )`

Enables trigger count for a specific counter.

When enabled, the counter will only be decremented when a count action occurs.

**Parameters**

|                 |                                |
|-----------------|--------------------------------|
| <i>deviceld</i> | Device containing the counter. |
|-----------------|--------------------------------|

---

|                |                                     |
|----------------|-------------------------------------|
| <i>counter</i> | Counter to enable trigger count on. |
|----------------|-------------------------------------|

---

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

**Examples:**

triggering\_example.c.

**7.27.3.16** `int STAR_API_CC TRIGGER_BRICK_MK3_enableExtTriggerEdgeDetectMode ( STAR_DEVICE_ID deviceld, U32 extTrigger )`

Enables edge detect mode for a specific external trigger.

When enabled, the trigger will be set on the rising edge of the input trigger signal.

**Parameters**


---

|                   |                                                 |
|-------------------|-------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.         |
| <i>extTrigger</i> | External trigger to enable edge detect mode on. |

---

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

**Examples:**

timestamp\_test.c, and triggering\_example.c.

**7.27.3.17** `int STAR_API_CC TRIGGER_BRICK_MK3_enableExtTriggerInvert ( STAR_DEVICE_ID deviceld, U32 extTrigger )`

Enables invert for a specific external trigger.

When enabled, the input trigger signal is inverted before the detection mechanism.

**Parameters**


---

|                   |                                         |
|-------------------|-----------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger. |
| <i>extTrigger</i> | External trigger to enable invert on.   |

---

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

**Examples:**

timestamp\_test.c.

**7.27.3.18** `int STAR_API_CC TRIGGER_BRICK_MK3_enableExtTriggerOutput ( STAR_DEVICE_ID deviceld, U32 extTrigger )`

Enables output for a specific external trigger.

When enabled, the external trigger's output action causes the trigger output signal to be pulsed for a duration specified by the extend duration value.

**Parameters**

|                   |                                         |
|-------------------|-----------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger. |
| <i>extTrigger</i> | External trigger to enable output on.   |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

**Examples:**

timestamp\_test.c.

**7.27.3.19** `int STAR_API_CC TRIGGER_BRICK_MK3_enablePortTransmitPacketMode ( STAR_DEVICE_ID deviceld, U32 port )`

Enables packet transmit mode for a specific port.

When enabled, packets are only transmitted when a transmit packet action occurs.

**Parameters**

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceld</i> | Device containing the port.             |
| <i>port</i>     | Port to enable packet transmit mode on. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

**Examples:**

triggering\_example.c.

**7.27.3.20 int STAR\_API\_CC TRIGGER\_BRICK\_MK3\_enableTrigger ( STAR\_DEVICE\_ID *deviceld*, U32 *trigger* )**

Enables a specific internal trigger.

When enabled, the trigger can be set by its input events and cause output actions to occur.

**Parameters**

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger. |
| <i>trigger</i>  | Internal trigger to enable.             |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

**Examples:**

timestamp\_test.c, and triggering\_example.c.

**7.27.3.21 int STAR\_API\_CC TRIGGER\_BRICK\_MK3\_forceCounterReload ( STAR\_DEVICE\_ID *deviceld*, U32 *counter* )**

Force the reload of a specific counter.

**Parameters**

|                 |                                |
|-----------------|--------------------------------|
| <i>deviceld</i> | Device containing the counter. |
| <i>counter</i>  | Counter to reload.             |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

**Examples:**

[triggering\\_example.c](#).

### 7.27.3.22 `int STAR_API_CC TRIGGER_BRICK_MK3_forceTrigger ( STAR_DEVICE_ID deviceld, U32 trigger )`

Force the triggering of a specific internal trigger.

**Parameters**

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger. |
| <i>trigger</i>  | Internal trigger to trigger.            |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

### 7.27.3.23 `int STAR_API_CC TRIGGER_BRICK_MK3_getCounterAutoReloadEnabled ( STAR_DEVICE_ID deviceld, U32 counter, U32 * pEnabled )`

Gets whether auto reload is enabled for a specific counter.

**Parameters**

|                 |                                                                                  |
|-----------------|----------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                   |
| <i>counter</i>  | Counter to check.                                                                |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if auto reload is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

### 7.27.3.24 `int STAR_API_CC TRIGGER_BRICK_MK3_getCounterLoadZeroEnabled ( STAR_DEVICE_ID deviceld, U32 counter, U32 * pEnabled )`

Gets whether load zero is enabled for a specific counter.

**Parameters**

|                 |                                                                                |
|-----------------|--------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                 |
| <i>counter</i>  | Counter to check.                                                              |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if load zero is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

**7.27.3.25** `int STAR_API_CC_TRIGGER_BRICK_MK3_getCounterReloadValue ( STAR_DEVICE_ID deviceld, U32 counter, U32 * pReloadValue )`

Gets the reload value for a specific counter.

The reload value is the value that will be loaded into the counter when a reload occurs.

**Parameters**

|                     |                                                       |
|---------------------|-------------------------------------------------------|
| <i>deviceld</i>     | Device containing the counter.                        |
| <i>counter</i>      | Counter to get the reload value for.                  |
| <i>pReloadValue</i> | Pointer to value to be updated with the reload value. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

**7.27.3.26** `int STAR_API_CC_TRIGGER_BRICK_MK3_getCounterStartModeEnabled ( STAR_DEVICE_ID deviceld, U32 counter, U32 * pEnabled )`

Gets whether start mode is enabled for a specific counter.

**Parameters**

|                 |                                                                                 |
|-----------------|---------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                  |
| <i>counter</i>  | Counter to check.                                                               |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if start mode is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

## Supported devices:

SpaceWire Brick Mk3

**7.27.3.27** `int STAR_API_CC TRIGGER_BRICK_MK3_getCounterStartStopModeEnabled ( STAR_DEVICE_ID deviceld, U32 counter, U32 * pEnabled )`

Gets whether start stop mode is enabled for a specific counter.

## Parameters

|                 |                                                                                      |
|-----------------|--------------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                       |
| <i>counter</i>  | Counter to check.                                                                    |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if start stop mode is enabled, else 0. |

## Returns

1 if the operation was successful, else 0

## Added in version:

STAR-System v3.0 - 26th August 2016

## Supported devices:

SpaceWire Brick Mk3

**7.27.3.28** `int STAR_API_CC TRIGGER_BRICK_MK3_getCounterTriggerCountEnabled ( STAR_DEVICE_ID deviceld, U32 counter, U32 * pEnabled )`

Gets whether trigger count is enabled for a specific counter.

## Parameters

|                 |                                                                                    |
|-----------------|------------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                     |
| <i>counter</i>  | Counter to check.                                                                  |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if trigger count is enabled, else 0. |

## Returns

1 if the operation was successful, else 0

## Added in version:

STAR-System v3.0 - 26th August 2016

## Supported devices:

SpaceWire Brick Mk3

**7.27.3.29** `int STAR_API_CC TRIGGER_BRICK_MK3_getCounterValue ( STAR_DEVICE_ID deviceld, U32 counter, U32 * pValue )`

Gets the counter value for a specific counter.

**Parameters**

|                 |                                                        |
|-----------------|--------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                         |
| <i>counter</i>  | Counter to get the value for.                          |
| <i>pValue</i>   | Pointer to value to be updated with the counter value. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

**7.27.3.30** `int STAR_API_CC_TRIGGER_BRICK_MK3_getExtTriggerEdgeDetectModeEnabled ( STAR_DEVICE_ID deviceld, U32 extTrigger, U32 * pEnabled )`

Gets whether edge detect mode is enabled for a specific external trigger.

**Parameters**

|                   |                                                                                       |
|-------------------|---------------------------------------------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.                                               |
| <i>extTrigger</i> | External trigger to check.                                                            |
| <i>pEnabled</i>   | User supplied value that will be updated to 1 if edge detect mode is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

**7.27.3.31** `int STAR_API_CC_TRIGGER_BRICK_MK3_getExtTriggerExtend ( STAR_DEVICE_ID deviceld, U32 extTrigger, U32 * pExtend )`

Gets the extend duration for a specific external trigger.

The extend duration is the duration, in cycles, of the trigger output signal pulse when a trigger output action occurs.

**Parameters**

|                   |                                                                  |
|-------------------|------------------------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.                          |
| <i>extTrigger</i> | External trigger to get the extend duration for.                 |
| <i>pExtend</i>    | Point to value to be updated with the extend duration in cycles. |



**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

**7.27.3.32** `int STAR_API_CC TRIGGER_BRICK_MK3_getExtTriggerInvertEnabled ( STAR_DEVICE_ID deviceld, U32 extTrigger, U32 * pEnabled )`

Gets whether invert is enabled for a specific external trigger.

**Parameters**

|                   |                                                                             |
|-------------------|-----------------------------------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.                                     |
| <i>extTrigger</i> | External trigger to check.                                                  |
| <i>pEnabled</i>   | User supplied value that will be updated to 1 if invert is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

**7.27.3.33** `int STAR_API_CC TRIGGER_BRICK_MK3_getExtTriggerOutputEnabled ( STAR_DEVICE_ID deviceld, U32 extTrigger, U32 * pEnabled )`

Gets whether output is enabled for a specific external trigger.

**Parameters**

|                   |                                                                             |
|-------------------|-----------------------------------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.                                     |
| <i>extTrigger</i> | External trigger to check.                                                  |
| <i>pEnabled</i>   | User supplied value that will be updated to 1 if output is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

**7.27.3.34** `int STAR_API_CC TRIGGER_BRICK_MK3_getPortTransmitPacketModeEnabled ( STAR_DEVICE_ID deviceld, U32 port, U32 * pEnabled )`

Gets whether packet transmit mode is enabled for a specific port.

**Parameters**

|                 |                                                                                           |
|-----------------|-------------------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                                                               |
| <i>port</i>     | Port to check.                                                                            |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if packet transmit mode is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

**7.27.3.35** `int STAR_API_CC TRIGGER_BRICK_MK3_getTriggerAndMask ( STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_TYPE type, U32 * pMask )`

Gets the AND/OR mask for a trigger type for a specific internal trigger.

**Parameters**

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                         |
| <i>trigger</i>  | Internal trigger to get the AND/OR mask for a trigger type for. |
| <i>type</i>     | Trigger type.                                                   |
| <i>pMask</i>    | Pointer to value to be updated with the AND/OR mask.            |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

**7.27.3.36** `int STAR_API_CC TRIGGER_BRICK_MK3_getTriggerEnabled ( STAR_DEVICE_ID deviceld, U32 trigger, U32 * pEnabled )`

Gets whether a trigger is enabled.

**Parameters**

|                 |                                                                                  |
|-----------------|----------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                                          |
| <i>trigger</i>  | Internal trigger to check.                                                       |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if the trigger is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

**7.27.3.37** `int STAR_API_CC TRIGGER_BRICK_MK3_getTriggerInputMode ( STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_INPUT_MODE * pMode )`

Gets the input mode for a specific internal trigger.

The trigger input mode can be either TRIGGER\_INPUT\_MODE\_AND or TRIGGER\_INPUT\_MODE\_OR. If set to TRIGGER\_INPUT\_MODE\_AND, the trigger occurs when all of its input events occur. If set to TRIGGER\_INPUT\_MODE\_OR, the trigger occurs when any of its input events occur.

**Parameters**

|                 |                                                     |
|-----------------|-----------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.             |
| <i>trigger</i>  | Internal trigger to get trigger input mode for.     |
| <i>pMode</i>    | Pointer to value to be updated with the input mode. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

**7.27.3.38** `int STAR_API_CC TRIGGER_BRICK_MK3_getTriggerInvertMask ( STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_TYPE type, U32 * pMask )`

Gets the invert mask for a trigger type for a specific internal trigger.

The invert mask for a trigger type determines if a source of input events should have its signal inverted before reaching the internal trigger.

**Parameters**

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                         |
| <i>trigger</i>  | Internal trigger to get the invert mask for a trigger type for. |
| <i>type</i>     | Trigger type.                                                   |
| <i>pMask</i>    | Pointer to value to be updated with the invert mask.            |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

7.27.3.39 `int STAR_API_CC TRIGGER_BRICK_MK3_setCounterReloadValue ( STAR_DEVICE_ID deviceId, U32 counter, U32 reloadValue )`

Sets the reload value for a specific counter.

The reload value is the value that will be loaded into the counter when a reload occurs.

**Parameters**

|                    |                                      |
|--------------------|--------------------------------------|
| <i>deviceId</i>    | Device containing the counter.       |
| <i>counter</i>     | Counter to set the reload value for. |
| <i>reloadValue</i> | Reload value.                        |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

**Examples:**

timestamp\_test.c, and triggering\_example.c.

**7.27.3.40** `int STAR_API_CC TRIGGER_BRICK_MK3_setExtTriggerExtend ( STAR_DEVICE_ID deviceId, U32 extTrigger, U32 extend )`

Sets the extend duration for a specific external trigger.

The extend duration is the duration, in cycles, of the trigger output signal pulse when a trigger output action occurs.

**Parameters**

|                   |                                             |
|-------------------|---------------------------------------------|
| <i>deviceId</i>   | Device containing the external trigger.     |
| <i>extTrigger</i> | External trigger to set extend duration on. |
| <i>extend</i>     | Extend duration in cycles.                  |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

**Examples:**

timestamp\_test.c.

**7.27.3.41** `int STAR_API_CC TRIGGER_BRICK_MK3_setTriggerAndMask ( STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_TYPE type, U32 mask )`

Sets the AND/OR mask for a trigger type for a specific internal trigger.

**Parameters**

|                 |                                                            |
|-----------------|------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                    |
| <i>trigger</i>  | Internal trigger to set AND/OR mask for a trigger type on. |
| <i>type</i>     | Trigger type.                                              |
| <i>mask</i>     | AND/OR mask.                                               |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

**7.27.3.42** `int STAR_API_CC TRIGGER_BRICK_MK3_setTriggerInputMode ( STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_INPUT_MODE mode )`

Sets the input mode for a specific internal trigger.

The trigger input mode can be either TRIGGER\_INPUT\_MODE\_AND or TRIGGER\_INPUT\_MODE\_OR. If set to TRIGGER\_INPUT\_MODE\_AND, the trigger occurs when all source input events occur. If set to TRIGGER\_INPUT\_MODE\_OR, the trigger occurs when any source input events occur.

**Parameters**

|                 |                                             |
|-----------------|---------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.     |
| <i>trigger</i>  | Internal trigger to set the input mode for. |
| <i>mode</i>     | The input mode.                             |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

**7.27.3.43** `int STAR_API_CC TRIGGER_BRICK_MK3_setTriggerInvertMask ( STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_TYPE type, U32 mask )`

Sets the invert mask for a trigger type for a specific internal trigger.

The invert mask for a trigger type determines if a source of input events should have its signal inverted before reaching the internal trigger.

**Parameters**

|                 |                                                            |
|-----------------|------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                    |
| <i>trigger</i>  | Internal trigger to set invert mask for a trigger type on. |
| <i>type</i>     | Trigger type.                                              |
| <i>mask</i>     | Invert mask.                                               |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3

**Examples:**

triggering\_example.c.

#### 7.27.3.44 int STAR\_API\_CC TRIGGER\_PCIE\_IF\_disableCounterAutoReload ( STAR\_DEVICE\_ID *deviceld*, U32 *counter* )

Disables auto reload for a specific counter.

When disabled, the counter will not automatically reload with its reload value when the counter reaches zero.

**Parameters**

|                 |                                    |
|-----------------|------------------------------------|
| <i>deviceld</i> | Device containing the counter.     |
| <i>counter</i>  | Counter to disable auto reload on. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

#### 7.27.3.45 int STAR\_API\_CC TRIGGER\_PCIE\_IF\_disableCounterLoadZero ( STAR\_DEVICE\_ID *deviceld*, U32 *counter* )

Disables load zero for a specific counter.

When disabled, reload triggered events occur even when the counter is not zero.

**Parameters**

|                 |                                  |
|-----------------|----------------------------------|
| <i>deviceld</i> | Device containing the counter.   |
| <i>counter</i>  | Counter to disable load zero on. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

**7.27.3.46 int STAR\_API\_CC\_TRIGGER\_PCIE\_IF\_disableCounterStartMode ( STAR\_DEVICE\_ID *deviceld*, U32 *counter* )**

Disables start mode for a specific counter.

When disabled, the counter will not wait for a start action to occur before any counting or reloading can take place.

**Parameters**

|                 |                                   |
|-----------------|-----------------------------------|
| <i>deviceld</i> | Device containing the counter.    |
| <i>counter</i>  | Counter to disable start mode on. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

**7.27.3.47 int STAR\_API\_CC\_TRIGGER\_PCIE\_IF\_disableCounterStartStopMode ( STAR\_DEVICE\_ID *deviceld*, U32 *counter* )**

Disables start stop mode for a specific counter.

When disabled, the counter will not wait for a start action to occur before any counting or reloading can take place.

**Parameters**

|                 |                                        |
|-----------------|----------------------------------------|
| <i>deviceld</i> | Device containing the counter.         |
| <i>counter</i>  | Counter to disable start stop mode on. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe



7.27.3.48 `int STAR_API_CC TRIGGER_PCIE_IF_disableCounterTriggerCount ( STAR_DEVICE_ID deviceId, U32 counter )`

Disables trigger count for a specific counter.

When disabled, the counter will be decremented every clock cycle.

**Parameters**

---

|                 |                                      |
|-----------------|--------------------------------------|
| <i>deviceld</i> | Device containing the counter.       |
| <i>counter</i>  | Counter to disable trigger count on. |

---

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

**7.27.3.49 int STAR\_API\_CC TRIGGER\_PCIE\_IF\_disableTrigger ( STAR\_DEVICE\_ID *deviceld*, U32 *trigger* )**

Disables a specific internal trigger.

When disabled, the trigger is not set by its input events and does not cause output actions to occur.

**Parameters**

---

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger. |
| <i>trigger</i>  | Internal trigger to disable.            |

---

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

**7.27.3.50 int STAR\_API\_CC TRIGGER\_PCIE\_IF\_enableCounterAutoReload ( STAR\_DEVICE\_ID *deviceld*, U32 *counter* )**

Enables auto reload for a specific counter.

When enabled, the counter will automatically reload with its reload value when the counter reaches zero.

**Parameters**

---

|                 |                                   |
|-----------------|-----------------------------------|
| <i>deviceld</i> | Device containing the counter.    |
| <i>counter</i>  | Counter to enable auto reload on. |

---

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

**Examples:**

[triggering\\_example.c](#).

**7.27.3.51 int STAR\_API\_CC TRIGGER\_PCIE\_IF\_enableCounterLoadZero ( STAR\_DEVICE\_ID *deviceld*, U32 *counter* )**

Enables load zero for a specific counter.

When enabled, reload triggered events only occur when the counter is zero.

**Parameters**

|                 |                                 |
|-----------------|---------------------------------|
| <i>deviceld</i> | Device containing the counter.  |
| <i>counter</i>  | Counter to enable load zero on. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

[STAR-System v3.1 - 21st October 2016](#)

**Supported devices:**

[SpaceWire PCIe](#)

**7.27.3.52 int STAR\_API\_CC TRIGGER\_PCIE\_IF\_enableCounterStartMode ( STAR\_DEVICE\_ID *deviceld*, U32 *counter* )**

Enables start mode for a specific counter.

When enabled, the counter will wait for a counter start action to occur before any counting or reloading can take place. When the counter reaches zero, the counter may be reloaded but no counting or reloading can take place until the next start action occurs.

**Parameters**

|                 |                                  |
|-----------------|----------------------------------|
| <i>deviceld</i> | Device containing the counter.   |
| <i>counter</i>  | Counter to enable start mode on. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

[STAR-System v3.1 - 21st October 2016](#)

**Supported devices:**

[SpaceWire PCIe](#)

**7.27.3.53 int STAR\_API\_CC TRIGGER\_PCIE\_IF\_enableCounterStartStopMode ( STAR\_DEVICE\_ID *deviceld*, U32 *counter* )**

Enables start stop mode for a specific counter.

When enabled, the counter will wait for a start action to occur before any counting or reloading can take place. The counter will continue to count and reload until a stop action occurs.

**Parameters**

|                 |                                       |
|-----------------|---------------------------------------|
| <i>deviceld</i> | Device containing the counter.        |
| <i>counter</i>  | Counter to enable start stop mode on. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

7.27.3.54 `int STAR_API_CC TRIGGER_PCIE_IF_enableCounterTriggerCount ( STAR_DEVICE_ID deviceld, U32 counter )`

Enables trigger count for a specific counter.

When enabled, the counter will only be decremented when a count action occurs.

**Parameters**

|                 |                                     |
|-----------------|-------------------------------------|
| <i>deviceld</i> | Device containing the counter.      |
| <i>counter</i>  | Counter to enable trigger count on. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

7.27.3.55 `int STAR_API_CC TRIGGER_PCIE_IF_enableTrigger ( STAR_DEVICE_ID deviceld, U32 trigger )`

Enables a specific internal trigger.

When enabled, the trigger can be set by its input events and cause output actions to occur.

**Parameters**

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger. |
| <i>trigger</i>  | Internal trigger to enable.             |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

**Examples:**

triggering\_example.c.

**7.27.3.56** `int STAR_API_CC TRIGGER_PCIE_IF_forceCounterReload ( STAR_DEVICE_ID deviceld, U32 counter )`

Force the reload of a specific counter.

**Parameters**

|                 |                                |
|-----------------|--------------------------------|
| <i>deviceld</i> | Device containing the counter. |
| <i>counter</i>  | Counter to reload.             |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

**7.27.3.57** `int STAR_API_CC TRIGGER_PCIE_IF_forceTrigger ( STAR_DEVICE_ID deviceld, U32 trigger )`

Force the triggering of a specific internal trigger.

**Parameters**

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger. |
| <i>trigger</i>  | Internal trigger to trigger.            |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

**7.27.3.58** `int STAR_API_CC TRIGGER_PCIE_IF_getCounterAutoReloadEnabled ( STAR_DEVICE_ID deviceld, U32 counter, U32 * pEnabled )`

Gets whether auto reload is enabled for a specific counter.

**Parameters**

|                 |                                                                                  |
|-----------------|----------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                   |
| <i>counter</i>  | Counter to check.                                                                |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if auto reload is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

**7.27.3.59** int **STAR\_API\_CC\_TRIGGER\_PCIE\_IF\_getCounterLoadZeroEnabled** ( **STAR\_DEVICE\_ID** *deviceld*, **U32** *counter*, **U32** \* *pEnabled* )

Gets whether load zero is enabled for a specific counter.

**Parameters**

|                 |                                                                                |
|-----------------|--------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                 |
| <i>counter</i>  | Counter to check.                                                              |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if load zero is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

**7.27.3.60** int **STAR\_API\_CC\_TRIGGER\_PCIE\_IF\_getCounterReloadValue** ( **STAR\_DEVICE\_ID** *deviceld*, **U32** *counter*, **U32** \* *pReloadValue* )

Gets the reload value for a specific counter.

The reload value is the value that will be loaded into the counter when a reload occurs.

**Parameters**

|                     |                                                       |
|---------------------|-------------------------------------------------------|
| <i>deviceld</i>     | Device containing the counter.                        |
| <i>counter</i>      | Counter to get the reload value for.                  |
| <i>pReloadValue</i> | Pointer to value to be updated with the reload value. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.1 - 21st October 2016

Supported devices:

SpaceWire PCIe

**7.27.3.61** `int STAR_API_CC_TRIGGER_PCIE_IF_getCounterStartModeEnabled ( STAR_DEVICE_ID deviceld, U32 counter, U32 * pEnabled )`

Gets whether start mode is enabled for a specific counter.

**Parameters**

|                 |                                                                                 |
|-----------------|---------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                  |
| <i>counter</i>  | Counter to check.                                                               |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if start mode is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0

Added in version:

STAR-System v3.1 - 21st October 2016

Supported devices:

SpaceWire PCIe

**7.27.3.62** `int STAR_API_CC_TRIGGER_PCIE_IF_getCounterStartStopModeEnabled ( STAR_DEVICE_ID deviceld, U32 counter, U32 * pEnabled )`

Gets whether start stop mode is enabled for a specific counter.

**Parameters**

|                 |                                                                                      |
|-----------------|--------------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                       |
| <i>counter</i>  | Counter to check.                                                                    |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if start stop mode is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0

Added in version:

STAR-System v3.1 - 21st October 2016

Supported devices:

SpaceWire PCIe

**7.27.3.63** `int STAR_API_CC_TRIGGER_PCIE_IF_getCounterTriggerCountEnabled ( STAR_DEVICE_ID deviceld, U32 counter, U32 * pEnabled )`

Gets whether trigger count is enabled for a specific counter.

**Parameters**

|                 |                                                                                    |
|-----------------|------------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                     |
| <i>counter</i>  | Counter to check.                                                                  |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if trigger count is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

7.27.3.64 **int STAR\_API\_CC TRIGGER\_PCIE\_IF\_getCounterValue ( STAR\_DEVICE\_ID *deviceld*, U32 *counter*, U32 \* *pValue* )**

Gets the counter value for a specific counter.

**Parameters**

|                 |                                                        |
|-----------------|--------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                         |
| <i>counter</i>  | Counter to get the value for.                          |
| <i>pValue</i>   | Pointer to value to be updated with the counter value. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

7.27.3.65 **int STAR\_API\_CC TRIGGER\_PCIE\_IF\_getTriggerAndMask ( STAR\_DEVICE\_ID *deviceld*, U32 *trigger*, TRIGGER\_TYPE *type*, U32 \* *pMask* )**

Gets the AND/OR mask for a trigger type for a specific internal trigger.

**Parameters**

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                         |
| <i>trigger</i>  | Internal trigger to get the AND/OR mask for a trigger type for. |
| <i>type</i>     | Trigger type.                                                   |
| <i>pMask</i>    | Pointer to value to be updated with the AND/OR mask.            |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.1 - 21st October 2016



## Supported devices:

SpaceWire PCIe

7.27.3.66 `int STAR_API_CC TRIGGER_PCIE_IF_getTriggerEnabled ( STAR_DEVICE_ID deviceld, U32 trigger, U32 * pEnabled )`

Gets whether a trigger is enabled.

## Parameters

|                 |                                                                                  |
|-----------------|----------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                                          |
| <i>trigger</i>  | Internal trigger to check.                                                       |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if the trigger is enabled, else 0. |

## Returns

1 if the operation was successful, else 0

## Added in version:

STAR-System v3.1 - 21st October 2016

## Supported devices:

SpaceWire PCIe

7.27.3.67 `int STAR_API_CC TRIGGER_PCIE_IF_getTriggerInputMode ( STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_INPUT_MODE * pMode )`

Gets the input mode for a specific internal trigger.

The trigger input mode can be either TRIGGER\_INPUT\_MODE\_AND or TRIGGER\_INPUT\_MODE\_OR. If set to TRIGGER\_INPUT\_MODE\_AND, the trigger occurs when all of its input events occur. If set to TRIGGER\_INPUT\_MODE\_OR, the trigger occurs when any of its input events occur.

## Parameters

|                 |                                                     |
|-----------------|-----------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.             |
| <i>trigger</i>  | Internal trigger to get trigger input mode for.     |
| <i>pMode</i>    | Pointer to value to be updated with the input mode. |

## Returns

1 if the operation was successful, else 0

## Added in version:

STAR-System v3.1 - 21st October 2016

## Supported devices:

SpaceWire PCIe

7.27.3.68 `int STAR_API_CC TRIGGER_PCIE_IF_getTriggerInvertMask ( STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_TYPE type, U32 * pMask )`

Gets the invert mask for a trigger type for a specific internal trigger.

The invert mask for a trigger type determines if a source of input events should have its signal inverted before reaching the internal trigger.

**Parameters**

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                         |
| <i>trigger</i>  | Internal trigger to get the invert mask for a trigger type for. |
| <i>type</i>     | Trigger type.                                                   |
| <i>pMask</i>    | Pointer to value to be updated with the invert mask.            |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

7.27.3.69 **int STAR\_API\_CC TRIGGER\_PCIE\_IF\_setCounterReloadValue ( STAR\_DEVICE\_ID *deviceld*, U32 *counter*, U32 *reloadValue* )**

Sets the reload value for a specific counter.

The reload value is the value that will be loaded into the counter when a reload occurs.

**Parameters**

|                    |                                      |
|--------------------|--------------------------------------|
| <i>deviceld</i>    | Device containing the counter.       |
| <i>counter</i>     | Counter to set the reload value for. |
| <i>reloadValue</i> | Reload value.                        |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

**Examples:**

triggering\_example.c.

7.27.3.70 **int STAR\_API\_CC TRIGGER\_PCIE\_IF\_setTriggerAndMask ( STAR\_DEVICE\_ID *deviceld*, U32 *trigger*, TRIGGER\_TYPE *type*, U32 *mask* )**

Sets the AND/OR mask for a trigger type for a specific internal trigger.

**Parameters**

|                 |                                                            |
|-----------------|------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                    |
| <i>trigger</i>  | Internal trigger to set AND/OR mask for a trigger type on. |
| <i>type</i>     | Trigger type.                                              |
| <i>mask</i>     | AND/OR mask.                                               |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

**7.27.3.71** `int STAR_API_CC TRIGGER_PCIE_IF_setTriggerInputMode ( STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_INPUT_MODE mode )`

Sets the input mode for a specific internal trigger.

The trigger input mode can be either TRIGGER\_INPUT\_MODE\_AND or TRIGGER\_INPUT\_MODE\_OR. If set to TRIGGER\_INPUT\_MODE\_AND, the trigger occurs when all source input events occur. If set to TRIGGER\_INPUT\_MODE\_OR, the trigger occurs when any source input events occur.

**Parameters**

|                 |                                             |
|-----------------|---------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.     |
| <i>trigger</i>  | Internal trigger to set the input mode for. |
| <i>mode</i>     | The input mode.                             |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

**7.27.3.72** `int STAR_API_CC TRIGGER_PCIE_IF_setTriggerInvertMask ( STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_TYPE type, U32 mask )`

Sets the invert mask for a trigger type for a specific internal trigger.

The invert mask for a trigger type determines if a source of input events should have its signal inverted before reaching the internal trigger.

**Parameters**

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger. |
|-----------------|-----------------------------------------|

|                |                                                            |
|----------------|------------------------------------------------------------|
| <i>trigger</i> | Internal trigger to set invert mask for a trigger type on. |
| <i>type</i>    | Trigger type.                                              |
| <i>mask</i>    | Invert mask.                                               |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

**7.27.3.73** `int STAR_API_CC TRIGGER_PXI_IF_disableCounterAutoReload ( STAR_DEVICE_ID deviceld, U32 counter )`

Disables auto reload for a specific counter.

When disabled, the counter will not automatically reload with its reload value when the counter reaches zero.

**Parameters**

|                 |                                    |
|-----------------|------------------------------------|
| <i>deviceld</i> | Device containing the counter.     |
| <i>counter</i>  | Counter to disable auto reload on. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**7.27.3.74** `int STAR_API_CC TRIGGER_PXI_IF_disableCounterLoadZero ( STAR_DEVICE_ID deviceld, U32 counter )`

Disables load zero for a specific counter.

When disabled, reload triggered events occur even when the counter is not zero.

**Parameters**

|                 |                                  |
|-----------------|----------------------------------|
| <i>deviceld</i> | Device containing the counter.   |
| <i>counter</i>  | Counter to disable load zero on. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

#### 7.27.3.75 int STAR\_API\_CC TRIGGER\_PXI\_IF\_disableCounterStartMode ( STAR\_DEVICE\_ID *deviceld*, U32 *counter* )

Disables start mode for a specific counter.

When disabled, the counter will not wait for a start action to occur before any counting or reloading can take place.

##### Parameters

|                 |                                   |
|-----------------|-----------------------------------|
| <i>deviceld</i> | Device containing the counter.    |
| <i>counter</i>  | Counter to disable start mode on. |

##### Returns

1 if the operation was successful, else 0

##### Added in version:

STAR-System v3.0 - 26th August 2016

##### Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2

#### 7.27.3.76 int STAR\_API\_CC TRIGGER\_PXI\_IF\_disableCounterStartStopMode ( STAR\_DEVICE\_ID *deviceld*, U32 *counter* )

Disables start stop mode for a specific counter.

When disabled, the counter will not wait for a start action to occur before any counting or reloading can take place.

##### Parameters

|                 |                                        |
|-----------------|----------------------------------------|
| <i>deviceld</i> | Device containing the counter.         |
| <i>counter</i>  | Counter to disable start stop mode on. |

##### Returns

1 if the operation was successful, else 0

##### Added in version:

STAR-System v3.0 - 26th August 2016

##### Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2

#### 7.27.3.77 int STAR\_API\_CC TRIGGER\_PXI\_IF\_disableCounterTriggerCount ( STAR\_DEVICE\_ID *deviceld*, U32 *counter* )

Disables trigger count for a specific counter.

When disabled, the counter will be decremented every clock cycle.

##### Parameters

|                 |                                |
|-----------------|--------------------------------|
| <i>deviceld</i> | Device containing the counter. |
|-----------------|--------------------------------|

---

|                |                                      |
|----------------|--------------------------------------|
| <i>counter</i> | Counter to disable trigger count on. |
|----------------|--------------------------------------|

---

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**7.27.3.78** `int STAR_API_CC TRIGGER_PXI_IF_disableExtTriggerEdgeDetectMode ( STAR_DEVICE_ID deviceld, U32 extTrigger )`

Disables edge detect mode for a specific external trigger.

When disabled, the trigger will always be set when the input trigger is high.

**Parameters**

|                   |                                                  |
|-------------------|--------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.          |
| <i>extTrigger</i> | External trigger to disable edge detect mode on. |

---

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**7.27.3.79** `int STAR_API_CC TRIGGER_PXI_IF_disableExtTriggerInvert ( STAR_DEVICE_ID deviceld, U32 extTrigger )`

Disables invert for a specific external trigger.

When disabled, the input trigger signal is not modified before the detection mechanism.

**Parameters**

|                   |                                         |
|-------------------|-----------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger. |
| <i>extTrigger</i> | External trigger to disable invert on.  |

---

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**7.27.3.80 int STAR\_API\_CC TRIGGER\_PXI\_IF\_disableExtTriggerOutput ( STAR\_DEVICE\_ID *deviceld*, U32 *extTrigger* )**

Disables output for a specific external trigger.

When disabled, the external trigger's output action does not cause the trigger output signal to be pulsed.

**Parameters**

|                   |                                         |
|-------------------|-----------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger. |
| <i>extTrigger</i> | External trigger to disable output on.  |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**7.27.3.81 int STAR\_API\_CC TRIGGER\_PXI\_IF\_disablePortTransmitPacketMode ( STAR\_DEVICE\_ID *deviceld*, U32 *port* )**

Disables packet transmit mode for a specific port.

When disabled, packets do not wait for a transmit packet action to occur before transmission.

**Parameters**

|                 |                                          |
|-----------------|------------------------------------------|
| <i>deviceld</i> | Device containing the port.              |
| <i>port</i>     | Port to disable packet transmit mode on. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**7.27.3.82 int STAR\_API\_CC TRIGGER\_PXI\_IF\_disableTrigger ( STAR\_DEVICE\_ID *deviceld*, U32 *trigger* )**

Disables a specific internal trigger.

When disabled, the trigger is not set by its input events and does not cause output actions to occur.

**Parameters**

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger. |
| <i>trigger</i>  | Internal trigger to disable.            |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**7.27.3.83 int STAR\_API\_CC TRIGGER\_PXI\_IF\_enableCounterAutoReload ( STAR\_DEVICE\_ID *deviceld*, U32 *counter* )**

Enables auto reload for a specific counter.

When enabled, the counter will automatically reload with its reload value when the counter reaches zero.

**Parameters**

|                 |                                   |
|-----------------|-----------------------------------|
| <i>deviceld</i> | Device containing the counter.    |
| <i>counter</i>  | Counter to enable auto reload on. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**Examples:**

triggering\_example.c.

**7.27.3.84 int STAR\_API\_CC TRIGGER\_PXI\_IF\_enableCounterLoadZero ( STAR\_DEVICE\_ID *deviceld*, U32 *counter* )**

Enables load zero for a specific counter.

When enabled, reload triggered events only occur when the counter is zero.

**Parameters**

|                 |                                 |
|-----------------|---------------------------------|
| <i>deviceld</i> | Device containing the counter.  |
| <i>counter</i>  | Counter to enable load zero on. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2



#### 7.27.3.85 int STAR\_API\_CC TRIGGER\_PXI\_IF\_enableCounterStartMode ( STAR\_DEVICE\_ID *deviceld*, U32 *counter* )

Enables start mode for a specific counter.

When enabled, the counter will wait for a counter start action to occur before any counting or reloading can take place. When the counter reaches zero, the counter may be reloaded but no counting or reloading can take place until the next start action occurs.

##### Parameters

|                 |                                  |
|-----------------|----------------------------------|
| <i>deviceld</i> | Device containing the counter.   |
| <i>counter</i>  | Counter to enable start mode on. |

##### Returns

1 if the operation was successful, else 0

##### Added in version:

STAR-System v3.0 - 26th August 2016

##### Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2

#### 7.27.3.86 int STAR\_API\_CC TRIGGER\_PXI\_IF\_enableCounterStartStopMode ( STAR\_DEVICE\_ID *deviceld*, U32 *counter* )

Enables start stop mode for a specific counter.

When enabled, the counter will wait for a start action to occur before any counting or reloading can take place. The counter will continue to count and reload until a stop action occurs.

##### Parameters

|                 |                                       |
|-----------------|---------------------------------------|
| <i>deviceld</i> | Device containing the counter.        |
| <i>counter</i>  | Counter to enable start stop mode on. |

##### Returns

1 if the operation was successful, else 0

##### Added in version:

STAR-System v3.0 - 26th August 2016

##### Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2

##### Examples:

triggering\_example.c.

#### 7.27.3.87 int STAR\_API\_CC TRIGGER\_PXI\_IF\_enableCounterTriggerCount ( STAR\_DEVICE\_ID *deviceld*, U32 *counter* )

Enables trigger count for a specific counter.

When enabled, the counter will only be decremented when a count action occurs.

**Parameters**

---

|                 |                                     |
|-----------------|-------------------------------------|
| <i>deviceld</i> | Device containing the counter.      |
| <i>counter</i>  | Counter to enable trigger count on. |

---

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**Examples:**

[triggering\\_example.c](#).

**7.27.3.88** `int STAR_API_CC TRIGGER_PXI_IF_enableExtTriggerEdgeDetectMode ( STAR_DEVICE_ID deviceld, U32 extTrigger )`

Enables edge detect mode for a specific external trigger.

When enabled, the trigger will be set on the rising edge of the input trigger signal.

**Parameters**

---

|                   |                                                 |
|-------------------|-------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.         |
| <i>extTrigger</i> | External trigger to enable edge detect mode on. |

---

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**Examples:**

[triggering\\_example.c](#).

**7.27.3.89** `int STAR_API_CC TRIGGER_PXI_IF_enableExtTriggerInvert ( STAR_DEVICE_ID deviceld, U32 extTrigger )`

Enables invert for a specific external trigger.

When enabled, the input trigger signal is inverted before the detection mechanism.

**Parameters**

|                   |                                         |
|-------------------|-----------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger. |
| <i>extTrigger</i> | External trigger to enable invert on.   |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**7.27.3.90 int STAR\_API\_CC TRIGGER\_PXI\_IF\_enableExtTriggerOutput ( STAR\_DEVICE\_ID *deviceld*, U32 *extTrigger* )**

Enables output for a specific external trigger.

When enabled, the external trigger's output action causes the trigger output signal to be pulsed for a duration specified by the extend duration value.

**Parameters**

|                   |                                         |
|-------------------|-----------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger. |
| <i>extTrigger</i> | External trigger to enable output on.   |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**7.27.3.91 int STAR\_API\_CC TRIGGER\_PXI\_IF\_enablePortTransmitPacketMode ( STAR\_DEVICE\_ID *deviceld*, U32 *port* )**

Enables packet transmit mode for a specific port.

When enabled, packets are only transmitted when a transmit packet action occurs.

**Parameters**

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceld</i> | Device containing the port.             |
| <i>port</i>     | Port to enable packet transmit mode on. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**Examples:**

triggering\_example.c.

**7.27.3.92** int **STAR\_API\_CC** TRIGGER\_PXI\_IF\_enableTrigger ( STAR\_DEVICE\_ID *deviceld*, U32 *trigger* )

Enables a specific internal trigger.

When enabled, the trigger can be set by its input events and cause output actions to occur.

**Parameters**

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger. |
| <i>trigger</i>  | Internal trigger to enable.             |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**Examples:**

triggering\_example.c.

**7.27.3.93** int **STAR\_API\_CC** TRIGGER\_PXI\_IF\_forceCounterReload ( STAR\_DEVICE\_ID *deviceld*, U32 *counter* )

Force the reload of a specific counter.

**Parameters**

|                 |                                |
|-----------------|--------------------------------|
| <i>deviceld</i> | Device containing the counter. |
| <i>counter</i>  | Counter to reload.             |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**Examples:**

triggering\_example.c.

**7.27.3.94** int **STAR\_API\_CC** TRIGGER\_PXI\_IF\_forceTrigger ( STAR\_DEVICE\_ID *deviceld*, U32 *trigger* )

Force the triggering of a specific internal trigger.

**Parameters**

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger. |
| <i>trigger</i>  | Internal trigger to trigger.            |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**7.27.3.95** int **STAR\_API\_CC TRIGGER\_PXI\_IF\_getCounterAutoReloadEnabled** ( **STAR\_DEVICE\_ID** *deviceld*, **U32** *counter*, **U32** \* *pEnabled* )

Gets whether auto reload is enabled for a specific counter.

**Parameters**

|                 |                                                                                  |
|-----------------|----------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                   |
| <i>counter</i>  | Counter to check.                                                                |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if auto reload is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**7.27.3.96** int **STAR\_API\_CC TRIGGER\_PXI\_IF\_getCounterLoadZeroEnabled** ( **STAR\_DEVICE\_ID** *deviceld*, **U32** *counter*, **U32** \* *pEnabled* )

Gets whether load zero is enabled for a specific counter.

**Parameters**

|                 |                                                                                |
|-----------------|--------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                 |
| <i>counter</i>  | Counter to check.                                                              |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if load zero is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

7.27.3.97 `int STAR_API_CC TRIGGER_PXI_IF_getCounterReloadValue ( STAR_DEVICE_ID deviceld, U32 counter, U32 * pReloadValue )`

Gets the reload value for a specific counter.

The reload value is the value that will be loaded into the counter when a reload occurs.

#### Parameters

|                     |                                                       |
|---------------------|-------------------------------------------------------|
| <i>deviceld</i>     | Device containing the counter.                        |
| <i>counter</i>      | Counter to get the reload value for.                  |
| <i>pReloadValue</i> | Pointer to value to be updated with the reload value. |

#### Returns

1 if the operation was successful, else 0

#### Added in version:

STAR-System v3.0 - 26th August 2016

#### Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2

7.27.3.98 `int STAR_API_CC TRIGGER_PXI_IF_getCounterStartModeEnabled ( STAR_DEVICE_ID deviceld, U32 counter, U32 * pEnabled )`

Gets whether start mode is enabled for a specific counter.

#### Parameters

|                 |                                                                                 |
|-----------------|---------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                  |
| <i>counter</i>  | Counter to check.                                                               |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if start mode is enabled, else 0. |

#### Returns

1 if the operation was successful, else 0

#### Added in version:

STAR-System v3.0 - 26th August 2016

#### Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2

7.27.3.99 `int STAR_API_CC TRIGGER_PXI_IF_getCounterStartStopModeEnabled ( STAR_DEVICE_ID deviceld, U32 counter, U32 * pEnabled )`

Gets whether start stop mode is enabled for a specific counter.

#### Parameters

---

|                 |                                                                                      |
|-----------------|--------------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                       |
| <i>counter</i>  | Counter to check.                                                                    |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if start stop mode is enabled, else 0. |

---

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**7.27.3.100** `int STAR_API_CC TRIGGER_PXI_IF_getCounterTriggerCountEnabled ( STAR_DEVICE_ID deviceld, U32 counter, U32 * pEnabled )`

Gets whether trigger count is enabled for a specific counter.

**Parameters**

---

|                 |                                                                                    |
|-----------------|------------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                     |
| <i>counter</i>  | Counter to check.                                                                  |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if trigger count is enabled, else 0. |

---

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**7.27.3.101** `int STAR_API_CC TRIGGER_PXI_IF_getCounterValue ( STAR_DEVICE_ID deviceld, U32 counter, U32 * pValue )`

Gets the counter value for a specific counter.

**Parameters**

---

|                 |                                                        |
|-----------------|--------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                         |
| <i>counter</i>  | Counter to get the value for.                          |
| <i>pValue</i>   | Pointer to value to be updated with the counter value. |

---

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

7.27.3.102 `int STAR_API_CC TRIGGER_PXI_IF_getExtTriggerEdgeDetectModeEnabled ( STAR_DEVICE_ID deviceId,  
U32 extTrigger, U32 * pEnabled )`

Gets whether edge detect mode is enabled for a specific external trigger.



**Parameters**

|                   |                                                                                       |
|-------------------|---------------------------------------------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.                                               |
| <i>extTrigger</i> | External trigger to check.                                                            |
| <i>pEnabled</i>   | User supplied value that will be updated to 1 if edge detect mode is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**7.27.3.103** `int STAR_API_CC TRIGGER_PXI_IF_getExtTriggerExtend ( STAR_DEVICE_ID deviceld, U32 extTrigger, U32 * pExtend )`

Gets the extend duration for a specific external trigger.

The extend duration is the duration, in cycles, of the trigger output signal pulse when a trigger output action occurs.

**Parameters**

|                   |                                                                  |
|-------------------|------------------------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.                          |
| <i>extTrigger</i> | External trigger to get the extend duration for.                 |
| <i>pExtend</i>    | Point to value to be updated with the extend duration in cycles. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**7.27.3.104** `int STAR_API_CC TRIGGER_PXI_IF_getExtTriggerInvertEnabled ( STAR_DEVICE_ID deviceld, U32 extTrigger, U32 * pEnabled )`

Gets whether invert is enabled for a specific external trigger.

**Parameters**

|                   |                                                                             |
|-------------------|-----------------------------------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.                                     |
| <i>extTrigger</i> | External trigger to check.                                                  |
| <i>pEnabled</i>   | User supplied value that will be updated to 1 if invert is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**7.27.3.105** `int STAR_API_CC TRIGGER_PXI_IF_getExtTriggerOutputEnabled ( STAR_DEVICE_ID deviceld, U32 extTrigger, U32 * pEnabled )`

Gets whether output is enabled for a specific external trigger.

**Parameters**

|                   |                                                                             |
|-------------------|-----------------------------------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.                                     |
| <i>extTrigger</i> | External trigger to check.                                                  |
| <i>pEnabled</i>   | User supplied value that will be updated to 1 if output is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**7.27.3.106** `int STAR_API_CC TRIGGER_PXI_IF_getPortTransmitPacketModeEnabled ( STAR_DEVICE_ID deviceld, U32 port, U32 * pEnabled )`

Gets whether packet transmit mode is enabled for a specific port.

**Parameters**

|                 |                                                                                           |
|-----------------|-------------------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                                                               |
| <i>port</i>     | Port to check.                                                                            |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if packet transmit mode is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**7.27.3.107** `int STAR_API_CC TRIGGER_PXI_IF_getTriggerAndMask ( STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_TYPE type, U32 * pMask )`

Gets the AND/OR mask for a trigger type for a specific internal trigger.

**Parameters**

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                         |
| <i>trigger</i>  | Internal trigger to get the AND/OR mask for a trigger type for. |
| <i>type</i>     | Trigger type.                                                   |
| <i>pMask</i>    | Pointer to value to be updated with the AND/OR mask.            |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**7.27.3.108** `int STAR_API_CC TRIGGER_PXI_IF_getTriggerEnabled ( STAR_DEVICE_ID deviceld, U32 trigger, U32 * pEnabled )`

Gets whether a trigger is enabled.

**Parameters**

|                 |                                                                                  |
|-----------------|----------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                                          |
| <i>trigger</i>  | Internal trigger to check.                                                       |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if the trigger is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**7.27.3.109** `int STAR_API_CC TRIGGER_PXI_IF_getTriggerInputMode ( STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_INPUT_MODE * pMode )`

Gets the input mode for a specific internal trigger.

The trigger input mode can be either TRIGGER\_INPUT\_MODE\_AND or TRIGGER\_INPUT\_MODE\_OR. If set to TRIGGER\_INPUT\_MODE\_AND, the trigger occurs when all of its input events occur. If set to TRIGGER\_INPUT\_MODE\_OR, the trigger occurs when any of its input events occur.

**Parameters**

|                 |                                                 |
|-----------------|-------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.         |
| <i>trigger</i>  | Internal trigger to get trigger input mode for. |

---

|              |                                                     |
|--------------|-----------------------------------------------------|
| <i>pMode</i> | Pointer to value to be updated with the input mode. |
|--------------|-----------------------------------------------------|

---

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**7.27.3.110** `int STAR_API_CC TRIGGER_PXI_IF_getTriggerInvertMask ( STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_TYPE type, U32 * pMask )`

Gets the invert mask for a trigger type for a specific internal trigger.

The invert mask for a trigger type determines if a source of input events should have its signal inverted before reaching the internal trigger.

**Parameters**

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                         |
| <i>trigger</i>  | Internal trigger to get the invert mask for a trigger type for. |
| <i>type</i>     | Trigger type.                                                   |
| <i>pMask</i>    | Pointer to value to be updated with the invert mask.            |

---

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**7.27.3.111** `int STAR_API_CC TRIGGER_PXI_IF_setCounterReloadValue ( STAR_DEVICE_ID deviceld, U32 counter, U32 reloadValue )`

Sets the reload value for a specific counter.

The reload value is the value that will be loaded into the counter when a reload occurs.

**Parameters**

|                    |                                      |
|--------------------|--------------------------------------|
| <i>deviceld</i>    | Device containing the counter.       |
| <i>counter</i>     | Counter to set the reload value for. |
| <i>reloadValue</i> | Reload value.                        |

---

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**Examples:**

triggering\_example.c.

**7.27.3.112** `int STAR_API_CC TRIGGER_PXI_IF_setExtTriggerExtend ( STAR_DEVICE_ID deviceld, U32 extTrigger, U32 extend )`

Sets the extend duration for a specific external trigger.

The extend duration is the duration, in cycles, of the trigger output signal pulse when a trigger output action occurs.

**Parameters**

|                   |                                             |
|-------------------|---------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.     |
| <i>extTrigger</i> | External trigger to set extend duration on. |
| <i>extend</i>     | Extend duration in cycles.                  |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**7.27.3.113** `int STAR_API_CC TRIGGER_PXI_IF_setTriggerAndMask ( STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_TYPE type, U32 mask )`

Sets the AND/OR mask for a trigger type for a specific internal trigger.

**Parameters**

|                 |                                                            |
|-----------------|------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                    |
| <i>trigger</i>  | Internal trigger to set AND/OR mask for a trigger type on. |
| <i>type</i>     | Trigger type.                                              |
| <i>mask</i>     | AND/OR mask.                                               |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**7.27.3.114** `int STAR_API_CC TRIGGER_PXI_IF_setTriggerInputMode ( STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_INPUT_MODE mode )`

Sets the input mode for a specific internal trigger.

The trigger input mode can be either TRIGGER\_INPUT\_MODE\_AND or TRIGGER\_INPUT\_MODE\_OR. If set to TRIGGER\_INPUT\_MODE\_AND, the trigger occurs when all source input events occur. If set to TRIGGER\_INPUT\_MODE\_OR, the trigger occurs when any source input events occur.

#### Parameters

|                 |                                             |
|-----------------|---------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.     |
| <i>trigger</i>  | Internal trigger to set the input mode for. |
| <i>mode</i>     | The input mode.                             |

#### Returns

1 if the operation was successful, else 0

#### Added in version:

STAR-System v3.0 - 26th August 2016

#### Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2

**7.27.3.115** `int STAR_API_CC TRIGGER_PXI_IF_setTriggerInvertMask ( STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_TYPE type, U32 mask )`

Sets the invert mask for a trigger type for a specific internal trigger.

The invert mask for a trigger type determines if a source of input events should have its signal inverted before reaching the internal trigger.

#### Parameters

|                 |                                                            |
|-----------------|------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                    |
| <i>trigger</i>  | Internal trigger to set invert mask for a trigger type on. |
| <i>type</i>     | Trigger type.                                              |
| <i>mask</i>     | Invert mask.                                               |

#### Returns

1 if the operation was successful, else 0

#### Added in version:

STAR-System v3.0 - 26th August 2016

#### Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2

**7.27.3.116** `int STAR_API_CC TRIGGER_PXI_ROUTER_disableCounterAutoReload ( STAR_DEVICE_ID deviceld, U32 counter )`

Disables auto reload for a specific counter.

When disabled, the counter will not automatically reload with its reload value when the counter reaches zero.

**Parameters**

|                 |                                    |
|-----------------|------------------------------------|
| <i>deviceld</i> | Device containing the counter.     |
| <i>counter</i>  | Counter to disable auto reload on. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**7.27.3.117** `int STAR_API_CC TRIGGER_PXI_ROUTER_disableCounterLoadZero ( STAR_DEVICE_ID deviceld, U32 counter )`

Disables load zero for a specific counter.

When disabled, reload triggered events occur even when the counter is not zero.

**Parameters**

|                 |                                  |
|-----------------|----------------------------------|
| <i>deviceld</i> | Device containing the counter.   |
| <i>counter</i>  | Counter to disable load zero on. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**7.27.3.118** `int STAR_API_CC TRIGGER_PXI_ROUTER_disableCounterStartMode ( STAR_DEVICE_ID deviceld, U32 counter )`

Disables start mode for a specific counter.

When disabled, the counter will not wait for a start action to occur before any counting or reloading can take place.

**Parameters**

|                 |                                   |
|-----------------|-----------------------------------|
| <i>deviceld</i> | Device containing the counter.    |
| <i>counter</i>  | Counter to disable start mode on. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

7.27.3.119 `int STAR_API_CC TRIGGER_PXI_ROUTER_disableCounterStartStopMode ( STAR_DEVICE_ID deviceld, U32 counter )`

Disables start stop mode for a specific counter.

When disabled, the counter will not wait for a start action to occur before any counting or reloading can take place.

#### Parameters

|                 |                                        |
|-----------------|----------------------------------------|
| <i>deviceld</i> | Device containing the counter.         |
| <i>counter</i>  | Counter to disable start stop mode on. |

#### Returns

1 if the operation was successful, else 0

#### Added in version:

STAR-System v3.5 - 17th March 2017

#### Supported devices:

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

7.27.3.120 `int STAR_API_CC TRIGGER_PXI_ROUTER_disableCounterTriggerCount ( STAR_DEVICE_ID deviceld, U32 counter )`

Disables trigger count for a specific counter.

When disabled, the counter will be decremented every clock cycle.

#### Parameters

|                 |                                      |
|-----------------|--------------------------------------|
| <i>deviceld</i> | Device containing the counter.       |
| <i>counter</i>  | Counter to disable trigger count on. |

#### Returns

1 if the operation was successful, else 0

#### Added in version:

STAR-System v3.5 - 17th March 2017

#### Supported devices:

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

7.27.3.121 `int STAR_API_CC TRIGGER_PXI_ROUTER_disablePortTransmitPacketMode ( STAR_DEVICE_ID deviceld, U32 port )`

Disables packet transmit mode for a specific port.

When disabled, packets do not wait for a transmit packet action to occur before transmission.

#### Parameters

---



---

|                 |                                          |
|-----------------|------------------------------------------|
| <i>deviceld</i> | Device containing the port.              |
| <i>port</i>     | Port to disable packet transmit mode on. |

---

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**7.27.3.122 int STAR\_API\_CC TRIGGER\_PXI\_ROUTER\_disableTrigger ( STAR\_DEVICE\_ID *deviceld*, U32 *trigger* )**

Disables a specific internal trigger.

When disabled, the trigger is not set by its input events and does not cause output actions to occur.

**Parameters**

---

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger. |
| <i>trigger</i>  | Internal trigger to disable.            |

---

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**7.27.3.123 int STAR\_API\_CC TRIGGER\_PXI\_ROUTER\_enableCounterAutoReload ( STAR\_DEVICE\_ID *deviceld*, U32 *counter* )**

Enables auto reload for a specific counter.

When enabled, the counter will automatically reload with its reload value when the counter reaches zero.

**Parameters**

---

|                 |                                   |
|-----------------|-----------------------------------|
| <i>deviceld</i> | Device containing the counter.    |
| <i>counter</i>  | Counter to enable auto reload on. |

---

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**Examples:**

[triggering\\_example.c](#).

**7.27.3.124** `int STAR_API_CC TRIGGER_PXI_ROUTER_enableCounterLoadZero ( STAR_DEVICE_ID deviceld, U32 counter )`

Enables load zero for a specific counter.

When enabled, reload triggered events only occur when the counter is zero.

**Parameters**

|                 |                                 |
|-----------------|---------------------------------|
| <i>deviceld</i> | Device containing the counter.  |
| <i>counter</i>  | Counter to enable load zero on. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

[STAR-System v3.5 - 17th March 2017](#)

**Supported devices:**

[SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2](#)

**7.27.3.125** `int STAR_API_CC TRIGGER_PXI_ROUTER_enableCounterStartMode ( STAR_DEVICE_ID deviceld, U32 counter )`

Enables start mode for a specific counter.

When enabled, the counter will wait for a counter start action to occur before any counting or reloading can take place. When the counter reaches zero, the counter may be reloaded but no counting or reloading can take place until the next start action occurs.

**Parameters**

|                 |                                  |
|-----------------|----------------------------------|
| <i>deviceld</i> | Device containing the counter.   |
| <i>counter</i>  | Counter to enable start mode on. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

[STAR-System v3.5 - 17th March 2017](#)

**Supported devices:**

[SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2](#)

**7.27.3.126** `int STAR_API_CC TRIGGER_PXI_ROUTER_enableCounterStartStopMode ( STAR_DEVICE_ID deviceld, U32 counter )`

Enables start stop mode for a specific counter.

When enabled, the counter will wait for a start action to occur before any counting or reloading can take place. The counter will continue to count and reload until a stop action occurs.

**Parameters**

---

|                 |                                       |
|-----------------|---------------------------------------|
| <i>deviceld</i> | Device containing the counter.        |
| <i>counter</i>  | Counter to enable start stop mode on. |

---

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**Examples:**

triggering\_example.c.

**7.27.3.127** `int STAR_API_CC TRIGGER_PXI_ROUTER_enableCounterTriggerCount ( STAR_DEVICE_ID deviceld, U32 counter )`

Enables trigger count for a specific counter.

When enabled, the counter will only be decremented when a count action occurs.

**Parameters**

---

|                 |                                     |
|-----------------|-------------------------------------|
| <i>deviceld</i> | Device containing the counter.      |
| <i>counter</i>  | Counter to enable trigger count on. |

---

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**Examples:**

triggering\_example.c.

**7.27.3.128** `int STAR_API_CC TRIGGER_PXI_ROUTER_enablePortTransmitPacketMode ( STAR_DEVICE_ID deviceld, U32 port )`

Enables packet transmit mode for a specific port.

When enabled, packets are only transmitted when a transmit packet action occurs.

**Parameters**

---

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceld</i> | Device containing the port.             |
| <i>port</i>     | Port to enable packet transmit mode on. |

---

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**Examples:**

[triggering\\_example.c](#).

**7.27.3.129** `int STAR_API_CC TRIGGER_PXI_ROUTER_enableTrigger ( STAR_DEVICE_ID deviceld, U32 trigger )`

Enables a specific internal trigger.

When enabled, the trigger can be set by its input events and cause output actions to occur.

**Parameters**

---

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger. |
| <i>trigger</i>  | Internal trigger to enable.             |

---

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**Examples:**

[triggering\\_example.c](#).

**7.27.3.130** `int STAR_API_CC TRIGGER_PXI_ROUTER_forceCounterReload ( STAR_DEVICE_ID deviceld, U32 counter )`

Force the reload of a specific counter.

**Parameters**

---

|                 |                                |
|-----------------|--------------------------------|
| <i>deviceld</i> | Device containing the counter. |
|-----------------|--------------------------------|

---

---

|                |                    |
|----------------|--------------------|
| <i>counter</i> | Counter to reload. |
|----------------|--------------------|

---

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**Examples:**

triggering\_example.c.

**7.27.3.131** `int STAR_API_CC TRIGGER_PXI_ROUTER_forceTrigger ( STAR_DEVICE_ID deviceld, U32 trigger )`

Force the triggering of a specific internal trigger.

**Parameters**


---

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger. |
| <i>trigger</i>  | Internal trigger to trigger.            |

---

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**7.27.3.132** `int STAR_API_CC TRIGGER_PXI_ROUTER_getCounterAutoReloadEnabled ( STAR_DEVICE_ID deviceld, U32 counter, U32 * pEnabled )`

Gets whether auto reload is enabled for a specific counter.

**Parameters**


---

|                 |                                                                                  |
|-----------------|----------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                   |
| <i>counter</i>  | Counter to check.                                                                |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if auto reload is enabled, else 0. |

---

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

7.27.3.133 `int STAR_API_CC TRIGGER_PXI_ROUTER_getCounterLoadZeroEnabled ( STAR_DEVICE_ID deviceId, U32 counter, U32 * pEnabled )`

Gets whether load zero is enabled for a specific counter.

**Parameters**

|                 |                                                                                |
|-----------------|--------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                 |
| <i>counter</i>  | Counter to check.                                                              |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if load zero is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**7.27.3.134** `int STAR_API_CC TRIGGER_PXI_ROUTER_getCounterReloadValue ( STAR_DEVICE_ID deviceld, U32 counter, U32 * pReloadValue )`

Gets the reload value for a specific counter.

The reload value is the value that will be loaded into the counter when a reload occurs.

**Parameters**

|                     |                                                       |
|---------------------|-------------------------------------------------------|
| <i>deviceld</i>     | Device containing the counter.                        |
| <i>counter</i>      | Counter to get the reload value for.                  |
| <i>pReloadValue</i> | Pointer to value to be updated with the reload value. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**7.27.3.135** `int STAR_API_CC TRIGGER_PXI_ROUTER_getCounterStartModeEnabled ( STAR_DEVICE_ID deviceld, U32 counter, U32 * pEnabled )`

Gets whether start mode is enabled for a specific counter.

**Parameters**

|                 |                                                                                 |
|-----------------|---------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                  |
| <i>counter</i>  | Counter to check.                                                               |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if start mode is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

7.27.3.136 `int STAR_API_CC TRIGGER_PXI_ROUTER_getCounterStartStopModeEnabled ( STAR_DEVICE_ID deviceld, U32 counter, U32 * pEnabled )`

Gets whether start stop mode is enabled for a specific counter.

**Parameters**

|                 |                                                                                      |
|-----------------|--------------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                       |
| <i>counter</i>  | Counter to check.                                                                    |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if start stop mode is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

7.27.3.137 `int STAR_API_CC TRIGGER_PXI_ROUTER_getCounterTriggerCountEnabled ( STAR_DEVICE_ID deviceld, U32 counter, U32 * pEnabled )`

Gets whether trigger count is enabled for a specific counter.

**Parameters**

|                 |                                                                                    |
|-----------------|------------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                     |
| <i>counter</i>  | Counter to check.                                                                  |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if trigger count is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

7.27.3.138 `int STAR_API_CC TRIGGER_PXI_ROUTER_getCounterValue ( STAR_DEVICE_ID deviceld, U32 counter, U32 * pValue )`

Gets the counter value for a specific counter.



**Parameters**

|                 |                                                        |
|-----------------|--------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                         |
| <i>counter</i>  | Counter to get the value for.                          |
| <i>pValue</i>   | Pointer to value to be updated with the counter value. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**7.27.3.139** `int STAR_API_CC_TRIGGER_PXI_ROUTER_getPortTransmitPacketModeEnabled ( STAR_DEVICE_ID deviceld, U32 port, U32 * pEnabled )`

Gets whether packet transmit mode is enabled for a specific port.

**Parameters**

|                 |                                                                                           |
|-----------------|-------------------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                                                               |
| <i>port</i>     | Port to check.                                                                            |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if packet transmit mode is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**7.27.3.140** `int STAR_API_CC_TRIGGER_PXI_ROUTER_getTriggerAndMask ( STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_TYPE type, U32 * pMask )`

Gets the AND/OR mask for a trigger type for a specific internal trigger.

**Parameters**

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                         |
| <i>trigger</i>  | Internal trigger to get the AND/OR mask for a trigger type for. |
| <i>type</i>     | Trigger type.                                                   |
| <i>pMask</i>    | Pointer to value to be updated with the AND/OR mask.            |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**7.27.3.141** `int STAR_API_CC TRIGGER_PXI_ROUTER_getTriggerEnabled ( STAR_DEVICE_ID deviceld, U32 trigger, U32 * pEnabled )`

Gets whether a trigger is enabled.

**Parameters**

|                 |                                                                                  |
|-----------------|----------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                                          |
| <i>trigger</i>  | Internal trigger to check.                                                       |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if the trigger is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**7.27.3.142** `int STAR_API_CC TRIGGER_PXI_ROUTER_getTriggerInputMode ( STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_INPUT_MODE * pMode )`

Gets the input mode for a specific internal trigger.

The trigger input mode can be either TRIGGER\_INPUT\_MODE\_AND or TRIGGER\_INPUT\_MODE\_OR. If set to TRIGGER\_INPUT\_MODE\_AND, the trigger occurs when all of its input events occur. If set to TRIGGER\_INPUT\_MODE\_OR, the trigger occurs when any of its input events occur.

**Parameters**

|                 |                                                     |
|-----------------|-----------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.             |
| <i>trigger</i>  | Internal trigger to get trigger input mode for.     |
| <i>pMode</i>    | Pointer to value to be updated with the input mode. |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**7.27.3.143** `int STAR_API_CC TRIGGER_PXI_ROUTER_getTriggerInvertMask ( STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_TYPE type, U32 * pMask )`

Gets the invert mask for a trigger type for a specific internal trigger.

The invert mask for a trigger type determines if a source of input events should have its signal inverted before reaching the internal trigger.

**Parameters**

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                         |
| <i>trigger</i>  | Internal trigger to get the invert mask for a trigger type for. |
| <i>type</i>     | Trigger type.                                                   |
| <i>pMask</i>    | Pointer to value to be updated with the invert mask.            |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**7.27.3.144** `int STAR_API_CC TRIGGER_PXI_ROUTER_setCounterReloadValue ( STAR_DEVICE_ID deviceld, U32 counter, U32 reloadValue )`

Sets the reload value for a specific counter.

The reload value is the value that will be loaded into the counter when a reload occurs.

**Parameters**

|                    |                                      |
|--------------------|--------------------------------------|
| <i>deviceld</i>    | Device containing the counter.       |
| <i>counter</i>     | Counter to set the reload value for. |
| <i>reloadValue</i> | Reload value.                        |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**Examples:**

triggering\_example.c.

**7.27.3.145** `int STAR_API_CC TRIGGER_PXI_ROUTER_setTriggerAndMask ( STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_TYPE type, U32 mask )`

Sets the AND/OR mask for a trigger type for a specific internal trigger.

**Parameters**

|                 |                                                            |
|-----------------|------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                    |
| <i>trigger</i>  | Internal trigger to set AND/OR mask for a trigger type on. |
| <i>type</i>     | Trigger type.                                              |
| <i>mask</i>     | AND/OR mask.                                               |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**7.27.3.146** `int STAR_API_CC TRIGGER_PXI_ROUTER_setTriggerInputMode ( STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_INPUT_MODE mode )`

Sets the input mode for a specific internal trigger.

The trigger input mode can be either TRIGGER\_INPUT\_MODE\_AND or TRIGGER\_INPUT\_MODE\_OR. If set to TRIGGER\_INPUT\_MODE\_AND, the trigger occurs when all source input events occur. If set to TRIGGER\_INPU←T\_MODE\_OR, the trigger occurs when any source input events occur.

**Parameters**

|                 |                                             |
|-----------------|---------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.     |
| <i>trigger</i>  | Internal trigger to set the input mode for. |
| <i>mode</i>     | The input mode.                             |

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**7.27.3.147** `int STAR_API_CC TRIGGER_PXI_ROUTER_setTriggerInvertMask ( STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_TYPE type, U32 mask )`

Sets the invert mask for a trigger type for a specific internal trigger.

The invert mask for a trigger type determines if a source of input events should have its signal inverted before reaching the internal trigger.

**Parameters**

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger. |
|-----------------|-----------------------------------------|

---

|                |                                                            |
|----------------|------------------------------------------------------------|
| <i>trigger</i> | Internal trigger to set invert mask for a trigger type on. |
| <i>type</i>    | Trigger type.                                              |
| <i>mask</i>    | Invert mask.                                               |

---

**Returns**

1 if the operation was successful, else 0

**Added in version:**

STAR-System v3.5 - 17th March 2017

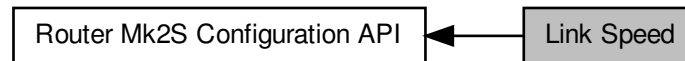
**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

## 7.28 Link Speed

Functions in this group deal with setting the link speeds on the device.

Collaboration diagram for Link Speed:



### Functions

- `int STAR_API_CC_CFG_ROUTER_MK2S_enablePrecisionTransmitRate (STAR_DEVICE_ID deviceID)`  
*Enables precision transmit rate on a SpaceWire Router Mk2S device.*
- `int STAR_API_CC_CFG_ROUTER_MK2S_getPrecisionTransmitRateEnabled (STAR_DEVICE_ID deviceID, _Out_ int *pEnabled)`  
*Determine if precision transmit rate is enabled on a Router Mk2S device.*
- `int STAR_API_CC_CFG_ROUTER_MK2S_disablePrecisionTransmitRate (STAR_DEVICE_ID deviceID)`  
*Disables precision transmit rate on a SpaceWire Router Mk2S device.*
- `int STAR_API_CC_CFG_ROUTER_MK2S_getPrecisionTransmitRateInUse (STAR_DEVICE_ID deviceID, _Out_ int *pInUse)`  
*Determine whether precision transmit rate is in use on a SpaceWire Router Mk2S device.*
- `int STAR_API_CC_CFG_ROUTER_MK2S_getPrecisionTransmitRate (STAR_DEVICE_ID deviceID, _Out_ double *pTransmitRate)`  
*Get the current precision transmit rate on a SpaceWire Router Mk2S device in Mbit/s.*
- `int STAR_API_CC_CFG_ROUTER_MK2S_setPrecisionTransmitRate (STAR_DEVICE_ID deviceID, double transmitRate)`  
*Set the current precision transmit rate on a SpaceWire Router Mk2S device in Mbit/s.*

### 7.28.1 Detailed Description

Functions in this group deal with setting the link speeds on the device.

### 7.28.2 Function Documentation

#### 7.28.2.1 `int STAR_API_CC_CFG_ROUTER_MK2S_disablePrecisionTransmitRate ( STAR_DEVICE_ID deviceID )`

Disables precision transmit rate on a SpaceWire Router Mk2S device.

Precision transmit rate allows the transmit rate of the device to be expressed accurately in Mbit/s as a floating point number using the function `CFG_ROUTER_MK2S_setPrecisionTransmitRate()`. Note that after being disabled there may be a short delay (less than 250 microseconds) before precision transmit rate is actually disabled. Call `CFG_ROUTER_MK2S_getPrecisionTransmitRateInUse` to determine if precision transmit rate is no longer in use.

**Parameters**

|                 |                                               |
|-----------------|-----------------------------------------------|
| <i>deviceID</i> | Device to disable precision transmit rate on. |
|-----------------|-----------------------------------------------|

**Returns**

1 if precision transmit rate was successfully disabled, else 0.

**Added in version:**

STAR-System v2.0 Beta 1 - 8th February 2013

**Supported devices:**

SpaceWire Router Mk2S

### 7.28.2.2 int STAR\_API\_CC CFG\_ROUTER\_MK2S\_enablePrecisionTransmitRate ( STAR\_DEVICE\_ID *deviceID* )

Enables precision transmit rate on a [SpaceWire Router Mk2S](#) device.

Precision transmit rate allows the transmit rate of the device to be expressed accurately in Mbit/s as a floating point number using the function [CFG\\_ROUTER\\_MK2S\\_setPrecisionTransmitRate\(\)](#). Note that after being enabled it can take up to 250 microseconds before precision transmit rate is actually used. Call [CFG\\_ROUTER\\_MK2S\\_getPrecisionTransmitRateInUse](#) to determine if precision transmit rate is actually in use.

**Parameters**

|                 |                                              |
|-----------------|----------------------------------------------|
| <i>deviceID</i> | Device to enable precision transmit rate on. |
|-----------------|----------------------------------------------|

**Returns**

1 if precision transmit rate was successfully enabled, else 0.

**Added in version:**

STAR-System v2.0 Beta 1 - 8th February 2013

**Supported devices:**

SpaceWire Router Mk2S

### 7.28.2.3 int STAR\_API\_CC CFG\_ROUTER\_MK2S\_getPrecisionTransmitRate ( STAR\_DEVICE\_ID *deviceID*, \_Out\_ double \* *pTransmitRate* )

Get the current precision transmit rate on a [SpaceWire Router Mk2S](#) device in Mbit/s.

Precision transmit rate allows the transmit rate of the device to be expressed accurately in Mbit/s as a floating point number using the function [CFG\\_ROUTER\\_MK2S\\_setPrecisionTransmitRate\(\)](#).

**Parameters**

|                      |                                                                                            |
|----------------------|--------------------------------------------------------------------------------------------|
| <i>deviceID</i>      | Device to get the precision transmit rate.                                                 |
| <i>pTransmitRate</i> | User supplied value that will be updated to contain the precision transmit rate in Mbit/s. |

**Returns**

1 if the information could be obtained from the device, else 0.

**Added in version:**

STAR-System v2.0 Beta 1 - 8th February 2013

**Supported devices:**

SpaceWire Router Mk2S

**7.28.2.4** `int STAR_API_CC CFG_ROUTER_MK2S_getPrecisionTransmitRateEnabled ( STAR_DEVICE_ID deviceId,  
_Out_ int * pEnabled )`

Determine if precision transmit rate is enabled on a Router Mk2S device.

Precision transmit rate allows the transmit rate of the device to be expressed accurately in Mbit/s as a floating point number using the function `CFG_ROUTER_MK2S_setPrecisionTransmitRate()`. Note that although precision transmit rate may be enabled, it may not yet be in use. It can take up to 250 microseconds before precision transmit rate is actually used. Call `CFG_ROUTER_MK2S_getPrecisionTransmitRateInUse` to determine if precision transmit rate is actually in use.

**Parameters**

|                 |                                                                                              |
|-----------------|----------------------------------------------------------------------------------------------|
| <i>deviceId</i> | Device to get whether precision transmit rate is enabled.                                    |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if precision transmit rate is enabled, else 0. |

**Returns**

1 if the information could be obtained from the device, else 0.

**Added in version:**

STAR-System v2.0 Beta 1 - 8th February 2013

**Supported devices:**

SpaceWire Router Mk2S

**7.28.2.5** `int STAR_API_CC CFG_ROUTER_MK2S_getPrecisionTransmitRateInUse ( STAR_DEVICE_ID deviceId, _Out_  
int * pInUse )`

Determine whether precision transmit rate is in use on a [SpaceWire Router Mk2S](#) device.

After being enabled or disabled, it can take up to 250 microseconds before precision transmit is actually used or not used. This function determines whether precision transmit rate is currently in use and can be used to determine if an enable or disable operation has completed.

**Parameters**

|                 |                                                                                             |
|-----------------|---------------------------------------------------------------------------------------------|
| <i>deviceId</i> | Device to get whether precision transmit rate is in use.                                    |
| <i>pInUse</i>   | User supplied value that will be updated to 1 if precision transmit rate is in use, else 0. |

**Returns**

1 if the information could be obtained from the device, else 0.

**Added in version:**

STAR-System v2.0 Beta 1 - 8th February 2013

**Supported devices:**

SpaceWire Router Mk2S



7.28.2.6 `int STAR_API_CC_CFG_ROUTER_MK2S_setPrecisionTransmitRate ( STAR_DEVICE_ID deviceId, double transmitRate )`

Set the current precision transmit rate on a [SpaceWire Router Mk2S](#) device in Mbit/s.

Precision transmit rate allows the transmit rate of the device to be expressed accurately in Mbit/s as a floating point number.

#### Parameters

|                     |                                                |
|---------------------|------------------------------------------------|
| <i>deviceId</i>     | Device to set the precision transmit rate for. |
| <i>transmitRate</i> | the precision transmit rate in Mbit/s to set.  |

#### Returns

1 if the rate for the device was successfully set, else 0.

#### Added in version:

[STAR-System v2.0 Beta 1](#) - 8th February 2013

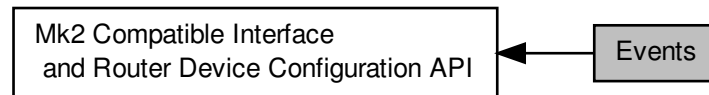
#### Supported devices:

[SpaceWire Router Mk2S](#)

## 7.29 Events

Functions in this group deal with enabling and disabling link events.

Collaboration diagram for Events:



### Functions

- `int STAR_API_CC CFG_MK2_enableStateChangeEventsOnPort (STAR_DEVICE_ID deviceID, U8 port↔Num)`  
*Selectively enables state change events on a given port.*
- `int STAR_API_CC CFG_MK2_getStateChangeEventsOnPortEnabled (STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int *pEnabled)`  
*Gets whether state change events are enabled on a specific port.*
- `int STAR_API_CC CFG_MK2_disableStateChangeEventsOnPort (STAR_DEVICE_ID deviceID, U8 port↔Num)`  
*Selectively disables state change events on a given port.*
- `int STAR_API_CC CFG_MK2_enableSpeedChangeEventsOnPort (STAR_DEVICE_ID deviceID, U8 port↔Num)`  
*Selectively enables speed change events on a given port.*
- `int STAR_API_CC CFG_MK2_getSpeedChangeEventsOnPortEnabled (STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int *pEnabled)`  
*Gets whether speed change events are enabled on a specific port.*
- `int STAR_API_CC CFG_MK2_disableSpeedChangeEventsOnPort (STAR_DEVICE_ID deviceID, U8 port↔Num)`  
*Selectively disables speed change events on a given port.*
- `int STAR_API_CC CFG_MK2_enableTimeCodeEventsOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`  
*Selectively enables time-code notifications on a the channel attached to a given port.*
- `int STAR_API_CC CFG_MK2_getTimeCodeEventsOnPortEnabled (STAR_DEVICE_ID deviceID, U8 port↔Num, _Out_ int *pEnabled)`  
*Gets whether time-code notifications are enabled on a specific port.*
- `int STAR_API_CC CFG_MK2_disableTimeCodeEventsOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`  
*Selectively disables time-code notifications on a the channel attached to a given port.*

### 7.29.1 Detailed Description

Functions in this group deal with enabling and disabling link events.

## 7.29.2 Function Documentation

### 7.29.2.1 `int STAR_API_CC_CFG_MK2_disableSpeedChangeEventsOnPort ( STAR_DEVICE_ID deviceID, U8 portNum )`

Selectively disables speed change events on a given port.

Disabling speed change events will result in speed change event traffic no longer being received on channel 0 when the link speed changes.

#### Note

This function is currently only compatible with [SpaceWire PCIe](#) devices from hardware version v1.10 edit 4, and [SpaceWire Physical Layer Tester](#) devices from hardware version v2.0. No check is made by this function to ensure it is being used with a compatible device.

On the [SpaceWire PCIe](#) link speed events are received on the channel associated with the port rather than channel 0.

#### Parameters

|                 |                                                      |
|-----------------|------------------------------------------------------|
| <i>deviceID</i> | Device to disable speed change events for a port on. |
| <i>portNum</i>  | Port to disable speed change events on.              |

#### Returns

1 if speed change events were successfully disabled, else 0.

#### Added in version:

[STAR-System v3.0 - 26th August 2016](#)

#### Supported devices:

[SpaceWire PCIe](#) (version 1.10e4 and greater), [SpaceWire Physical Layer Tester](#) (version 2.0 and greater)

### 7.29.2.2 `int STAR_API_CC_CFG_MK2_disableStateChangeEventsOnPort ( STAR_DEVICE_ID deviceID, U8 portNum )`

Selectively disables state change events on a given port.

Disabling state change events will result in state change event traffic no longer being received on channel 0 when the link changes to running or disconnected.

#### Note

This function is currently only compatible with [SpaceWire PCIe](#) devices from hardware version v1.10 edit 4, and [SpaceWire Physical Layer Tester](#) devices from hardware version v2.0. No check is made by this function to ensure it is being used with a compatible device.

On the [SpaceWire PCIe](#) state events are received on the channel associated with the port rather than channel 0.

#### Parameters

|                 |                                                      |
|-----------------|------------------------------------------------------|
| <i>deviceID</i> | Device to disable state change events for a port on. |
| <i>portNum</i>  | Port to disable state change events on.              |

#### Returns

1 if state change events were successfully disabled, else 0.

#### Added in version:

[STAR-System v3.0 - 26th August 2016](#)

**Supported devices:**

[SpaceWire PCIe](#) (version 1.10e4 and greater), [SpaceWire Physical Layer Tester](#) (version 2.0 and greater)

### 7.29.2.3 `int STAR_API_CC_CFG_MK2_disableTimeCodeEventsOnPort ( STAR_DEVICE_ID deviceID, U8 portNum )`

Selectively disables time-code notifications on a the channel attached to a given port.

**Note**

This function is currently only compatible with [SpaceWire PCIe](#) devices from hardware version v1.10 edit 4, with No check is made by this function to ensure it is being used with a compatible device.

**Parameters**

|                 |                                                          |
|-----------------|----------------------------------------------------------|
| <i>deviceID</i> | Device to disable time-code notifications for a port on. |
| <i>portNum</i>  | Port to disable time-code notifications on.              |

**Returns**

1 if time-code notifications were successfully disabled, else 0.

**Added in version:**

[STAR-System v3.0](#) - 26th August 2016

**Supported devices:**

[SpaceWire PCIe](#) (version 1.10e4 and greater)

### 7.29.2.4 `int STAR_API_CC_CFG_MK2_enableSpeedChangeEventsOnPort ( STAR_DEVICE_ID deviceID, U8 portNum )`

Selectively enables speed change events on a given port.

Enabling speed change events will result in speed change event traffic being received on channel 0 when the link speed changes.

**Note**

This function is currently only compatible with [SpaceWire PCIe](#) devices from hardware version v1.10 edit 4, and [SpaceWire Physical Layer Tester](#) devices from hardware version v2.0. No check is made by this function to ensure it is being used with a compatible device.

On the [SpaceWire PCIe](#) link speed events are received on the channel associated with the port rather than channel 0.

**Parameters**

|                 |                                                     |
|-----------------|-----------------------------------------------------|
| <i>deviceID</i> | Device to enable speed change events for a port on. |
| <i>portNum</i>  | Port to enable speed change events on.              |

**Returns**

1 if speed change events were successfully enabled, else 0.

**Added in version:**

[STAR-System v3.0](#) - 26th August 2016

**Supported devices:**

[SpaceWire PCIe](#) (version 1.10e4 and greater), [SpaceWire Physical Layer Tester](#) (version 2.0 and greater)

#### 7.29.2.5 `int STAR_API_CC_CFG_MK2_enableStateChangeEventsOnPort ( STAR_DEVICE_ID deviceID, U8 portNum )`

Selectively enables state change events on a given port.

Enabling state change events will result in state change event traffic being received on channel 0 when the link changes to running or disconnected.

##### Note

This function is currently only compatible with [SpaceWire PCIe](#) devices from hardware version v1.10 edit 4, and [SpaceWire Physical Layer Tester](#) devices from hardware version v2.0. No check is made by this function to ensure it is being used with a compatible device.

On the [SpaceWire PCIe](#) state events are received on the channel associated with the port rather than channel 0.

##### Parameters

|                 |                                                     |
|-----------------|-----------------------------------------------------|
| <i>deviceID</i> | Device to enable state change events for a port on. |
| <i>portNum</i>  | Port to enable state change events on.              |

##### Returns

1 if state change events were successfully enabled, else 0.

##### Added in version:

[STAR-System v3.0](#) - 26th August 2016

##### Supported devices:

[SpaceWire PCIe](#) (version 1.10e4 and greater), [SpaceWire Physical Layer Tester](#) (version 2.0 and greater)

#### 7.29.2.6 `int STAR_API_CC_CFG_MK2_enableTimeCodeEventsOnPort ( STAR_DEVICE_ID deviceID, U8 portNum )`

Selectively enables time-code notifications on a the channel attached to a given port.

##### Note

This function is currently only compatible with [SpaceWire PCIe](#) devices from hardware version v1.10 edit 4. No check is made by this function to ensure it is being used with a compatible device.

##### Parameters

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <i>deviceID</i> | Device to enable time-code notifications for a port on. |
| <i>portNum</i>  | Port to enable time-code notifications on.              |

##### Returns

1 if time-codes were successfully enabled, else 0.

##### Added in version:

[STAR-System v3.0](#) - 26th August 2016

##### Supported devices:

[SpaceWire PCIe](#) (version 1.10e4 and greater)

7.29.2.7 `int STAR_API_CC_CFG_MK2_getSpeedChangeEventsOnPortEnabled ( STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int * pEnabled )`

Gets whether speed change events are enabled on a specific port.

#### Note

This function is currently only compatible with [SpaceWire PCIe](#) devices from hardware version v1.10 edit 4, and [SpaceWire Physical Layer Tester](#) devices from hardware version v2.0. No check is made by this function to ensure it is being used with a compatible device.

#### Parameters

|                 |                                                                                           |
|-----------------|-------------------------------------------------------------------------------------------|
| <i>deviceID</i> | Device to get whether speed change events are enabled for a port.                         |
| <i>portNum</i>  | Port to get information for.                                                              |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if speed change events are enabled, else 0. |

#### Returns

1 if the information could be obtained from the device, else 0.

#### Added in version:

[STAR-System v3.0 - 26th August 2016](#)

#### Supported devices:

[SpaceWire PCIe](#) (version 1.10e4 and greater), [SpaceWire Physical Layer Tester](#) (version 2.0 and greater)

7.29.2.8 `int STAR_API_CC_CFG_MK2_getStateChangeEventsOnPortEnabled ( STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int * pEnabled )`

Gets whether state change events are enabled on a specific port.

#### Note

This function is currently only compatible with [SpaceWire PCIe](#) devices from hardware version v1.10 edit 4, and [SpaceWire Physical Layer Tester](#) devices from hardware version v2.0. No check is made by this function to ensure it is being used with a compatible device.

#### Parameters

|                 |                                                                                           |
|-----------------|-------------------------------------------------------------------------------------------|
| <i>deviceID</i> | Device to get whether state change events are enabled for a port.                         |
| <i>portNum</i>  | Port to get information for.                                                              |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if state change events are enabled, else 0. |

#### Returns

1 if the information could be obtained from the device, else 0.

#### Added in version:

[STAR-System v3.0 - 26th August 2016](#)

#### Supported devices:

[SpaceWire PCIe](#) (version 1.10e4 and greater), [SpaceWire Physical Layer Tester](#) (version 2.0 and greater)

7.29.2.9 `int STAR_API_CC_CFG_MK2_getTimeCodeEventsOnPortEnabled ( STAR_DEVICE_ID deviceId, U8 portNum,  
_Out_ int * pEnabled )`

Gets whether time-code notifications are enabled on a specific port.

#### Note

This function is currently only compatible with [SpaceWire PCIe](#) devices from hardware version v1.10 edit 4. No check is made by this function to ensure it is being used with a compatible device.

#### Parameters

|                 |                                                                                                       |
|-----------------|-------------------------------------------------------------------------------------------------------|
| <i>deviceId</i> | Device to get whether time-code notifications are enabled for a port.                                 |
| <i>portNum</i>  | Port to get information for.                                                                          |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if time-code notifications on port are enabled, else 0. |

#### Returns

1 if the information could be obtained from the device, else 0.

#### Added in version:

[STAR-System v3.0 - 26th August 2016](#)

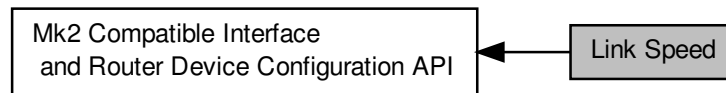
#### Supported devices:

[SpaceWire PCIe](#) (version 1.10e4 and greater)

## 7.30 Link Speed

Functions in this group deal with setting the link speeds on the device.

Collaboration diagram for Link Speed:



### Data Structures

- struct [STAR\\_CFG\\_MK2\\_BASE\\_TRANSMIT\\_CLOCK](#)

Base transmit clock rate control properties. [More...](#)

### Functions

- int [STAR\\_API\\_CC\\_CFG\\_MK2\\_setLinkRateDivider](#) (STAR\_DEVICE\_ID deviceId, U8 linkNum, U8 divider)  
Sets the Link rate divider for a given link.
- int [STAR\\_API\\_CC\\_CFG\\_MK2\\_getLinkRateDivider](#) (STAR\_DEVICE\_ID deviceId, U8 linkNum, \_Out\_ U8 \*pDivider)  
Gets the Link rate divider for a given link.
- int [STAR\\_API\\_CC\\_CFG\\_MK2\\_setBaseTransmitClock](#) (STAR\_DEVICE\_ID deviceId, U8 linkNum, STAR\_CFG\_MK2\_BASE\_TRANSMIT\_CLOCK clockRateParams)  
Sets the base transmit clock rate on a given link:
- int [STAR\\_API\\_CC\\_CFG\\_MK2\\_getBaseTransmitClock](#) (STAR\_DEVICE\_ID deviceId, U8 linkNum, \_Out\_ STAR\_CFG\_MK2\_BASE\_TRANSMIT\_CLOCK \*pClockRateParams)  
Gets the base transmit clock rate parameters for a given link.
- int [STAR\\_API\\_CC\\_CFG\\_MK2\\_setTransmitClock](#) (STAR\_DEVICE\_ID deviceId, U8 linkNum, STAR\_CFG\_MK2\_BASE\_TRANSMIT\_CLOCK clockRateParams)  
Sets the transmit clock rate on a given link.
- int [STAR\\_API\\_CC\\_CFG\\_MK2\\_getTransmitClock](#) (STAR\_DEVICE\_ID deviceId, U8 linkNum, \_Out\_ STAR\_CFG\_MK2\_BASE\_TRANSMIT\_CLOCK \*pClockRateParams)  
Gets the transmit clock rate parameters for a given link.

#### 7.30.1 Detailed Description

Functions in this group deal with setting the link speeds on the device.

#### 7.30.2 Data Structure Documentation

##### 7.30.2.1 struct STAR\_CFG\_MK2\_BASE\_TRANSMIT\_CLOCK

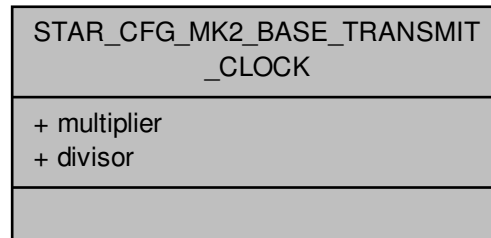
Base transmit clock rate control properties.



Added in version:

STAR-System v1.2 - 13th February 2012

Collaboration diagram for STAR\_CFG\_MK2\_BASE\_TRANSMIT\_CLOCK:



#### Data Fields

|     |            |                                                |
|-----|------------|------------------------------------------------|
| U16 | divisor    | Divisor value. Valid range 1:256 inclusive.    |
| U16 | multiplier | Multiplier value. Valid range 2:256 inclusive. |

### 7.30.3 Function Documentation

7.30.3.1 `int STAR_API_CC CFG_MK2_getBaseTransmitClock ( STAR_DEVICE_ID deviceId, U8 linkNum, _Out_ STAR_CFG_MK2_BASE_TRANSMIT_CLOCK * pClockRateParams )`

Gets the base transmit clock rate parameters for a given link.

#### Note

This function is currently for use with [SpaceWire PCIe](#) devices before hardwareversion v1.11 and [SpaceWire Physical Layer Tester](#) devices before hardware version v2.0.  
For later versions use [CFG\\_MK2\\_getTransmitClock\(\)](#).

The [SpaceWire PCI Mk2](#) and [cPCI Mk2](#), [SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#) and [SpaceWire Brick Mk3](#) do not support this function. For [SpaceWire PCI Mk2](#) and [cPCI Mk2](#) devices, please use [CFG\\_PCIMK2\\_getLinkClockFrequency\(\)](#) instead. For [SpaceWire Brick Mk2](#) and [SpaceWire Router Mk2S](#) devices, please use [CFG\\_BRICK\\_MK2\\_getLinkClockFrequency\(\)](#). For [SpaceWire Brick Mk3](#) devices, please use [CFG\\_BRICK\\_MK3\\_getBaseTransmitClock\(\)](#) or [CFG\\_BRICK\\_MK3\\_getTransmitClock\(\)](#) depending on hardware version.

No check is made by this function to ensure it is being used with a compatible device.

#### Parameters

|                 |                                                  |
|-----------------|--------------------------------------------------|
| <i>deviceId</i> | Device to get the base transmit clock rate from. |
| <i>linkNum</i>  | Link to get base transmit clock rate for.        |

|     |                               |                                                                                                  |
|-----|-------------------------------|--------------------------------------------------------------------------------------------------|
| out | <i>pClockRate↔<br/>Params</i> | Pointer to value that will be updated with the given link's Base transmit clock rate parameters. |
|-----|-------------------------------|--------------------------------------------------------------------------------------------------|

**Returns**

1 if Base transmit clock frequency parameters were successfully obtained, else 0.

**Added in version:**

STAR-System v1.2 - 13th February 2012

**Supported devices:**

SpaceWire PCIe (versions prior to 1.11), SpaceWire Physical Layer Tester (versions prior to 2.0)

**7.30.3.2** int STAR\_API\_CC\_CFG\_MK2\_getLinkRateDivider ( STAR\_DEVICE\_ID *deviceId*, U8 *linkNum*, \_Out\_ U8 \* *pDivider* )

Gets the Link rate divider for a given link.

**Parameters**

|     |                 |                                      |
|-----|-----------------|--------------------------------------|
|     | <i>deviceId</i> | Device to get a links divider from.. |
|     | <i>linkNum</i>  | Link to get divider for.             |
| out | <i>pDivider</i> | Value of the divider.                |

**Returns**

1 if was successfully obtained, else 0.

**Added in version:**

STAR-System v1.2 - 13th February 2012

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Router Mk2S, SpaceWire Physical Layer Tester

**7.30.3.3** int STAR\_API\_CC\_CFG\_MK2\_getTransmitClock ( STAR\_DEVICE\_ID *deviceId*, U8 *linkNum*, \_Out\_ STAR\_CFG\_MK2\_BASE\_TRANSMIT\_CLOCK \* *pClockRateParams* )

Gets the transmit clock rate parameters for a given link.

**Note**

This function is currently for use with SpaceWire PCIe devices from hardware version v1.11 and SpaceWire Physical Layer Tester devices from hardware version v2.0. For earlier versions use CFG\_MK2\_getBase↔TransmitClock().

No check is made by this function to ensure it is being used with a compatible device.

**Parameters**

|     |                                     |                                                                                             |
|-----|-------------------------------------|---------------------------------------------------------------------------------------------|
|     | <i>deviceID</i>                     | Device to get the transmit clock rate from.                                                 |
|     | <i>linkNum</i>                      | Link to get the transmit clock rate for.                                                    |
| out | <i>pClockRate↔</i><br><i>Params</i> | Pointer to value that will be updated with the given link's transmit clock rate parameters. |

**Returns**

1 if transmit clock frequency parameters were successfully obtained, else 0.

**Added in version:**

STAR-System v3.0 Beta 7 - 8th March 2016

**Supported devices:**

SpaceWire PCIe (versions 1.11 and greater), SpaceWire Physical Layer Tester (versions 2.0 and greater)

**7.30.3.4** `int STAR_API_CC_CFG_MK2_setBaseTransmitClock ( STAR_DEVICE_ID deviceID, U8 linkNum, STAR_CFG_MK2_BASE_TRANSMIT_CLOCK clockRateParams )`

Sets the base transmit clock rate on a given link:

$$BaseClockRate = \frac{100\text{MHz} \times MULTIPLIER}{DIVISOR}$$

The *MULTIPLIER* and *DIVISOR* are properties of the `STAR_CFG_MK2_BASE_TRANSMIT_CLOCK` structure, passed as a parameter. The output clock rate must be between 5 MHz and 200 MHz.

The transmit rate of the link is given by:

$$LinkTransmitRate = \frac{BaseClockRate}{LinkClockRateDivider} \times 2$$

The *LinkClockRateDivider* can be set using the `CFG_MK2_setLinkRateDivider()` function.

**Note**

This function is currently for use with SpaceWire PCIe devices before hardware version v1.11 and SpaceWire Physical Layer Tester devices before hardware version v2.0. For later versions use `CFG_MK2_setTransmitClock()`.

The SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire Brick Mk2, SpaceWire Router Mk2S and SpaceWire Brick Mk3 do not support this function. For SpaceWire PCI Mk2 and cPCI Mk2 devices, please use `CFG_PCIMK2_setLinkClockFrequency()` instead. For SpaceWire Brick Mk2 and SpaceWire Router Mk2S devices, please use `CFG_BRICK_MK2_setLinkClockFrequency()`. For SpaceWire Brick Mk3 devices, please use `CFG_BRICK_MK3_setBaseTransmitClock()` or `CFG_BRICK_MK3_setTransmitClock()` depending on hardware version.

No check is made by this function to ensure it is being used with a compatible device.

**Parameters**

|  |                 |                                                        |
|--|-----------------|--------------------------------------------------------|
|  | <i>deviceID</i> | Device to modify a link's base transmit clock rate on. |
|--|-----------------|--------------------------------------------------------|

|                   |                                                     |
|-------------------|-----------------------------------------------------|
| <i>linkNum</i>    | Link to have its base transmit clock rate modified. |
| <i>clockRate↔</i> | Base transmit clock rate parameters to be set.      |
| <i>Params</i>     |                                                     |

#### Returns

1 if link's Base transmit clock frequency was successfully set, else 0.

#### Added in version:

STAR-System v1.2 - 13th February 2012

#### Supported devices:

SpaceWire PCIe (versions prior to 1.11), SpaceWire Physical Layer Tester (versions prior to 2.0)

#### 7.30.3.5 int STAR\_API\_CC\_CFG\_MK2\_setLinkRateDivider ( STAR\_DEVICE\_ID *deviceId*, U8 *linkNum*, U8 *divider* )

Sets the Link rate divider for a given link.

The transmit rate of a link is given by:

$$LinkTransmitRate = \frac{BaseClockRate}{LinkClockRateDivider} \times 2$$

Where *BaseClockRate* is set by the appropriate function (e.g. `CFG_MK2_setBaseTransmitClock()` for a Space↔Wire PCIe device) and *LinkClockRateDivider* = divider.

#### Note

It is never necessary to set a link rate divider higher than 100, as the minimum transmit rate permitted by SpaceWire is 2 Mbit/s (i.e. 200 Mbit/s / 100).

If an odd number (other than 1) is specified as the divider parameter, the previous even integer will be used instead. For example: if a divider value of 3 is specified, the actual divider set shall be 2.

Link rate divider is limited to 64 on PCIe hardware versions prior to v1.11. We recommend upgrading to the latest firmware. If using the latest firmware then it is necessary to use `CFG_MK2_setTransmitClock()` rather than `CFG_MK2_setBaseTransmitClock()`. The `CFG_MK2_setLinkRateDivider()` function does not apply to PCIe hardware v1.11 or later and SPLT hardware v2.0 or later.

#### Parameters

|                 |                                                                       |
|-----------------|-----------------------------------------------------------------------|
| <i>deviceId</i> | Device to modify a links divider on.                                  |
| <i>linkNum</i>  | Link to have its divider modified.                                    |
| <i>divider</i>  | Value of the new divider. Valid input: 1, Even numbers 2 through 126. |

#### Returns

1 if divider was successfully set, else 0.

#### Added in version:

STAR-System v1.2 - 13th February 2012

#### Supported devices:

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Router Mk2S, SpaceWire Physical Layer Tester

7.30.3.6 `int STAR_API_CC_CFG_MK2_setTransmitClock ( STAR_DEVICE_ID deviceID, U8 linkNum, STAR_CFG_MK2_BASE_TRANSMIT_CLOCK clockRateParams )`

Sets the transmit clock rate on a given link.

The transmit rate of a link is given by:

$$LinkTransmitRate = \frac{100\text{MHz} \times MULTIPLIER}{DIVISOR} \times 2$$

The *MULTIPLIER* and *DIVISOR* are properties of the `STAR_CFG_MK2_BASE_TRANSMIT_CLOCK` structure, passed as a parameter. The output clock rate will be between 1 MHz and 200 MHz giving data rates between 2 Mbit/s and 400 Mbit/s.

Note that for data rates below 10 Mbit/s, an exact rate may not be achievable and it may be rounded up or down slightly.

#### Note

This function is currently for use with [SpaceWire PCIe](#) devices from hardware version v1.11 and [SpaceWire Physical Layer Tester](#) devices from hardware version v2.0. For earlier versions use `CFG_MK2_setBase↔TransmitClock()`.

No check is made by this function to ensure it is being used with a compatible device.

#### Parameters

|                         |                                                   |
|-------------------------|---------------------------------------------------|
| <i>deviceID</i>         | Device to modify a link's transmit clock rate on. |
| <i>linkNum</i>          | Link to have its transmit clock rate modified.    |
| <i>clockRate↔Params</i> | Transmit clock rate parameters to be set.         |

#### Returns

1 if link's transmit clock frequency was successfully set, else 0.

#### Added in version:

STAR-System v3.0 Beta 7 - 8th March 2016

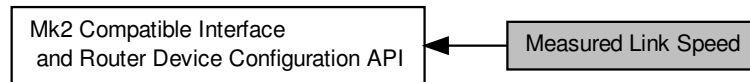
#### Supported devices:

[SpaceWire PCIe](#) (versions 1.11 and greater), [SpaceWire Physical Layer Tester](#) (versions 2.0 and greater)

## 7.31 Measured Link Speed

Functions in this group deal with reading the measured link speed.

Collaboration diagram for Measured Link Speed:



### Macros

- `#define STAR_CFG_MK2_LINK_SPEED_UNITS_KBPS 100`  
*The units in Kbit/s used to represent the link speed returned when calling `CFG_MK2_getMeasuredLinkSpeed()`.*

### Functions

- `int STAR_API_CC CFG_MK2_getMeasuredLinkSpeed (STAR_DEVICE_ID deviceId, U8 portNum, _Out_ U16 *pLinkSpeed)`  
*Gets the current link speed measured on a specific port.*

#### 7.31.1 Detailed Description

Functions in this group deal with reading the measured link speed.

#### 7.31.2 Macro Definition Documentation

##### 7.31.2.1 `#define STAR_CFG_MK2_LINK_SPEED_UNITS_KBPS 100`

The units in Kbit/s used to represent the link speed returned when calling `CFG_MK2_getMeasuredLinkSpeed()`.

Added in version:

STAR-System v3.0 Beta 8 - 6th May 2016

Examples:

`brickMk2_configuration_example.c`, and `link_events.c`.

#### 7.31.3 Function Documentation

##### 7.31.3.1 `int STAR_API_CC CFG_MK2_getMeasuredLinkSpeed ( STAR_DEVICE_ID deviceId, U8 portNum, _Out_ U16 *pLinkSpeed )`

Gets the current link speed measured on a specific port.

**Parameters**

|                   |                                                                                                                                                           |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>deviceID</i>   | Device to get the measured link speed for a port.                                                                                                         |
| <i>portNum</i>    | Port to get information for.                                                                                                                              |
| <i>pLinkSpeed</i> | User supplied value that will be updated to contain the measured link speed in 100 Kbit/s units, see <a href="#">STAR_CFG_MK2_LINK_SPEED_UNITS_KBPS</a> . |

**Returns**

1 if the information could be obtained from the device, else 0.

**Added in version:**

STAR-System v3.0 Beta 8 - 6th May 2016

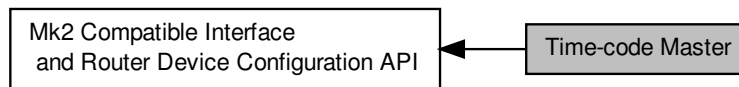
**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S, SpaceWire Physical Layer Tester

## 7.32 Time-code Master

Functions in this group deal with the management of the device as a time-code master.

Collaboration diagram for Time-code Master:



### Functions

- `int STAR_API_CC_CFG_MK2_enableTimeCodeMaster (STAR_DEVICE_ID deviceId)`  
*Enables the device as a time-code master.*
- `int STAR_API_CC_CFG_MK2_getTimeCodeMasterEnabled (STAR_DEVICE_ID deviceId, int *pEnabled)`  
*Gets whether the device has been enabled as a time-code master.*
- `int STAR_API_CC_CFG_MK2_disableTimeCodeMaster (STAR_DEVICE_ID deviceId)`  
*Disables the device as a time-code master.*
- `int STAR_API_CC_CFG_MK2_enableExternalTimeCodeSelection (STAR_DEVICE_ID deviceId)`  
*Enables external time-code selection for the device.*
- `int STAR_API_CC_CFG_MK2_getExternalTimeCodeSelectionEnabled (STAR_DEVICE_ID deviceId, int *pEnabled)`  
*Gets whether external time-code selection has been enabled for the device.*
- `int STAR_API_CC_CFG_MK2_disableExternalTimeCodeSelection (STAR_DEVICE_ID deviceId)`  
*Disables external time-code selection for the device.*
- `int STAR_API_CC_CFG_MK2_setTimeCodePeriod (STAR_DEVICE_ID deviceId, U32 period)`  
*Sets the period between time-code master ticks.*
- `int STAR_API_CC_CFG_MK2_getTimeCodePeriod (STAR_DEVICE_ID deviceId, _Out_ U32 *pPeriod)`  
*Gets the period between time-code master ticks.*
- `int STAR_API_CC_CFG_MK2_enableTimeCodeCounterBypassMode (STAR_DEVICE_ID deviceId)`  
*Enables time-code counter bypass mode for the device.*
- `int STAR_API_CC_CFG_MK2_getTimeCodeCounterBypassModeEnabled (STAR_DEVICE_ID deviceId, int *pEnabled)`  
*Gets whether time-code counter bypass mode has been enabled for the device.*
- `int STAR_API_CC_CFG_MK2_disableTimeCodeCounterBypassMode (STAR_DEVICE_ID deviceId)`  
*Disables time-code counter bypass mode for the device.*

### 7.32.1 Detailed Description

Functions in this group deal with the management of the device as a time-code master.



### 7.32.2 Function Documentation

#### 7.32.2.1 `int STAR_API_CC_CFG_MK2_disableExternalTimeCodeSelection ( STAR_DEVICE_ID deviceID )`

Disables external time-code selection for the device.

When external time-code selection is disabled and a time-code is transmitted from an application, the value specified for the time-code is ignored, and the next valid time-code is transmitted by the device.

**Parameters**


---

|                 |                                                                        |
|-----------------|------------------------------------------------------------------------|
| <i>deviceID</i> | The Mk2 compatible device to disable external time-code selection for. |
|-----------------|------------------------------------------------------------------------|

---

**Returns**

1 if external time-code selection was successfully disabled for the device, else 0.

**Added in version:**

STAR-System v2.0 Beta 3 - 12th April 2013

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S, SpaceWire Physical Layer Tester

### 7.32.2.2 int STAR\_API\_CC\_CFG\_MK2\_disableTimeCodeCounterBypassMode ( STAR\_DEVICE\_ID *deviceID* )

Disables time-code counter bypass mode for the device.

When time-code counter bypass mode is disabled, any time-code which is received will be checked against the current value of the counter for validity before being forwarded.

**Note**

This function is currently only supported by [SpaceWire PCIe](#) devices with at least hardware version v1.12.

**Parameters**


---

|                 |                                                                         |
|-----------------|-------------------------------------------------------------------------|
| <i>deviceID</i> | The Mk2 compatible device to disable time-code counter bypass mode for. |
|-----------------|-------------------------------------------------------------------------|

---

**Returns**

1 if time-code counter bypass mode was successfully disabled for the device, else 0.

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

[SpaceWire PCIe](#) (version 1.12 and greater)

### 7.32.2.3 int STAR\_API\_CC\_CFG\_MK2\_disableTimeCodeMaster ( STAR\_DEVICE\_ID *deviceID* )

Disables the device as a time-code master.

**Parameters**


---

|                 |                                                            |
|-----------------|------------------------------------------------------------|
| <i>deviceID</i> | The Mk2 compatible device to disable as a time-code master |
|-----------------|------------------------------------------------------------|

---

**Returns**

1 if the device was successfully unset as a time code master, else 0.

**Added in version:**

STAR-System v1.2 - 13th February 2012

---

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S, SpaceWire Physical Layer Tester

#### 7.32.2.4 `int STAR_API_CC_CFG_MK2_enableExternalTimeCodeSelection ( STAR_DEVICE_ID deviceId )`

Enables external time-code selection for the device.

When external time-code selection is enabled and a time-code is transmitted from an application, the value specified for the time-code is used. Note that the time-code will only be transmitted by the device if it is the next valid time-code.

**Parameters**

|                 |                                                                       |
|-----------------|-----------------------------------------------------------------------|
| <i>deviceId</i> | The Mk2 compatible device to enable external time-code selection for. |
|-----------------|-----------------------------------------------------------------------|

**Returns**

1 if external time-code selection was successfully enabled for the device, else 0.

**Added in version:**

STAR-System v2.0 Beta 3 - 12th April 2013

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S, SpaceWire Physical Layer Tester

#### 7.32.2.5 `int STAR_API_CC_CFG_MK2_enableTimeCodeCounterBypassMode ( STAR_DEVICE_ID deviceId )`

Enables time-code counter bypass mode for the device.

When time-code counter bypass mode is enabled, any time-code which is received will be forwarded out of the other ports on the router, without checking against the current value of the counter. The time-code register will be updated on each received time-code.

**Note**

This function is currently only supported by SpaceWire PCIe devices with at least hardware version v1.12.

**Parameters**

|                 |                                                                        |
|-----------------|------------------------------------------------------------------------|
| <i>deviceId</i> | The Mk2 compatible device to enable time-code counter bypass mode for. |
|-----------------|------------------------------------------------------------------------|

**Returns**

1 if time-code counter bypass mode was successfully enabled for the device, else 0.

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe (version 1.12 and greater)

#### 7.32.2.6 `int STAR_API_CC_CFG_MK2_enableTimeCodeMaster ( STAR_DEVICE_ID deviceId )`

Enables the device as a time-code master.

**Parameters**


---

|                 |                                                            |
|-----------------|------------------------------------------------------------|
| <i>deviceID</i> | The Mk2 compatible device to enable as a time-code master. |
|-----------------|------------------------------------------------------------|

---

**Returns**

1 if the device was successfully set as a time code master, else 0.

**Added in version:**

STAR-System v1.2 - 13th February 2012

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S, SpaceWire Physical Layer Tester

**7.32.2.7** `int STAR_API_CC_CFG_MK2_getExternalTimeCodeSelectionEnabled ( STAR_DEVICE_ID deviceID, int * pEnabled )`

Gets whether external time-code selection has been enabled for the device.

When external time-code selection is disabled and a time-code is transmitted from an application, the value specified for the time-code is ignored, and the next valid time-code is transmitted by the device. When external time-code selection is enabled and a time-code is transmitted from an application, the value specified for the time-code is used. Note that the time-code will only be transmitted by the device if it is the next valid time-code.

**Parameters**


---

|                 |                                                                                                                  |
|-----------------|------------------------------------------------------------------------------------------------------------------|
| <i>deviceID</i> | The Mk2 compatible device to check.                                                                              |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if external time-code selection is enabled for the device, else 0. |

---

**Returns**

1 if the information could be obtained from the device, else 0.

**Added in version:**

STAR-System v2.0 Beta 3 - 12th April 2013

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S, SpaceWire Physical Layer Tester

**7.32.2.8** `int STAR_API_CC_CFG_MK2_getTimeCodeCounterBypassModeEnabled ( STAR_DEVICE_ID deviceID, int * pEnabled )`

Gets whether time-code counter bypass mode has been enabled for the device.

When time-code counter bypass mode is disabled, any time-code which is received will be checked against against the current value of the counter for validity before being forwarded. When time-code counter bypass mode is enabled, any time-code which is received will be forwarded out of the other ports on the router, without checking against the current value of the counter. The time-code register will be updated on each received time-code.

**Note**

This function is currently only supported by SpaceWire PCIe devices with at least hardware version v1.12.

**Parameters**

|                 |                                                                                                                   |
|-----------------|-------------------------------------------------------------------------------------------------------------------|
| <i>deviceId</i> | The Mk2 compatible device to check.                                                                               |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if time-code counter bypass mode is enabled for the device, else 0. |

**Returns**

1 if the information could be obtained from the device, else 0.

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe (version 1.12 and greater)

### 7.32.2.9 int STAR\_API\_CC\_CFG\_MK2\_getTimeCodeMasterEnabled ( STAR\_DEVICE\_ID deviceId, int \* pEnabled )

Gets whether the device has been enabled as a time-code master.

**Parameters**

|                 |                                                                                                       |
|-----------------|-------------------------------------------------------------------------------------------------------|
| <i>deviceId</i> | The Mk2 compatible device to check.                                                                   |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if the device is enabled as a time-code master, else 0. |

**Returns**

1 if the information could be obtained from the device, else 0.

**Added in version:**

STAR-System v1.5 - 23rd April 2012

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S, SpaceWire Physical Layer Tester

### 7.32.2.10 int STAR\_API\_CC\_CFG\_MK2\_getTimeCodePeriod ( STAR\_DEVICE\_ID deviceId, \_Out\_ U32 \* pPeriod )

Gets the period between time-code master ticks.

This is the number of microseconds between time-codes.

**Parameters**

|            |                 |                                                                                    |
|------------|-----------------|------------------------------------------------------------------------------------|
|            | <i>deviceId</i> | Device to get time-code period from                                                |
| <i>out</i> | <i>pPeriod</i>  | A pointer to a variable that will be updated to contain the period in microseconds |

**Returns**

1 if the time-code period was successfully obtained, else 0.

**Added in version:**

STAR-System v1.2 - 13th February 2012

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S, SpaceWire Physical Layer Tester

**7.32.2.11 int STAR\_API\_CC\_CFG\_MK2\_setTimeCodePeriod ( STAR\_DEVICE\_ID *deviceId*, U32 *period* )**

Sets the period between time-code master ticks.

This is the number of microseconds between time-codes.

**Parameters**

|                 |                                      |
|-----------------|--------------------------------------|
| <i>deviceId</i> | Device to set time-code period for   |
| <i>period</i>   | The period to be set in microseconds |

**Returns**

1 if the time-code period was successfully set, else 0.

**Added in version:**

STAR-System v1.2 - 13th February 2012

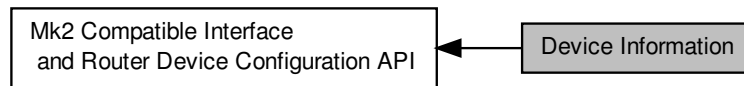
**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S, SpaceWire Physical Layer Tester

## 7.33 Device Information

Functions in this group deal with hardware information, such as obtaining version information, or flashing the LEDs.

Collaboration diagram for Device Information:



### Data Structures

- struct `STAR_CFG_MK2_HARDWARE_INFO`

Hardware version, and synthesis date of the FPGA code. [More...](#)

### Functions

- int `STAR_API_CC_CFG_MK2_getHardwareInfo` (STAR\_DEVICE\_ID deviceId, \_Out\_ STAR\_CFG\_MK2\_HARDWARE\_INFO \*pInfo)

Reads information about the hardware version of a device and updates the `STAR_CFG_MK2_HARDWARE_INFO` structure pointed to by the passed in pointer to contain the received version information.

- void `STAR_API_CC_CFG_MK2_hardwareInfoToString` (STAR\_CFG\_MK2\_HARDWARE\_INFO info, \_Out\_ z\_cap\_c\_(STAR\_CFG\_MK2\_VERSION\_STR\_MAX\_LEN) char \*pVersion, \_Out\_ z\_cap\_c\_(STAR\_CFG\_MK2\_BUILD\_DATE\_STR\_MAX\_LEN) char \*pBuildDate)

Creates a string representation of a `STAR_CFG_MK2_HARDWARE_INFO` structure.

- int `STAR_API_CC_CFG_MK2_identify` (STAR\_DEVICE\_ID deviceId)

Flashes the front panel LEDs of a device.

#### 7.33.1 Detailed Description

Functions in this group deal with hardware information, such as obtaining version information, or flashing the LEDs.

#### 7.33.2 Data Structure Documentation

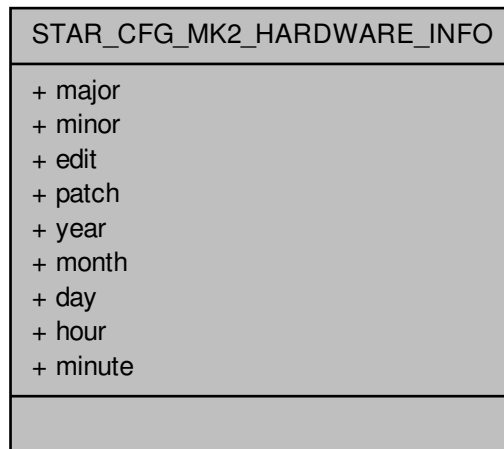
##### 7.33.2.1 struct STAR\_CFG\_MK2\_HARDWARE\_INFO

Hardware version, and synthesis date of the FPGA code.

Added in version:

STAR-System v1.2 - 13th February 2012

Collaboration diagram for STAR\_CFG\_MK2\_HARDWARE\_INFO:



#### Data Fields

|     |        |                                |
|-----|--------|--------------------------------|
| U8  | day    | Day value of time and date.    |
| U8  | edit   | Version number edit.           |
| U8  | hour   | Hour value of time and date.   |
| U8  | major  | Major version number.          |
| U8  | minor  | Minor version number.          |
| U8  | minute | Minute value of time and date. |
| U8  | month  | Month value of time and date.  |
| U8  | patch  | Version number patch.          |
| U16 | year   | Year value of time and date.   |

### 7.33.3 Function Documentation

7.33.3.1 `int STAR_API_CC CFG_MK2_getHardwareInfo ( STAR_DEVICE_ID deviceId, _Out_ STAR_CFG_MK2_HARDWARE_INFO * pInfo )`

Reads information about the hardware version of a device and updates the `STAR_CFG_MK2_HARDWARE_INFO` structure pointed to by the passed in pointer to contain the received version information.

#### Parameters

|     |                 |                                                                                                                                     |
|-----|-----------------|-------------------------------------------------------------------------------------------------------------------------------------|
|     | <i>deviceId</i> | Identifier for the Mk2 compatible device to get hardware info from.                                                                 |
| out | <i>pInfo</i>    | Pointer to a <code>STAR_CFG_MK2_HARDWARE_INFO</code> structure that will be updated to contain hardware information for the device. |

#### Returns

1 if the hardware version was successfully read.

Added in version:

STAR-System v1.2 - 13th February 2012



**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S, SpaceWire Physical Layer Tester

**7.33.3.2** void **STAR\_API\_CC** CFG\_MK2\_hardwareInfoToString ( **STAR\_CFG\_MK2\_HARDWARE\_INFO** info, \_Out\_z\_cap\_c\_(**STAR\_CFG\_MK2\_VERSION\_STR\_MAX\_LEN**) char \* *pVersion*, \_Out\_z\_cap\_c\_(**STAR\_CFG\_MK2\_BUILD\_DATE\_STR\_MAX\_LEN**) char \* *pBuildDate* )

Creates a string representation of a **STAR\_CFG\_MK2\_HARDWARE\_INFO** structure.

**Parameters**

|     |                   |                                                                                                                                                              |
|-----|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
|     | <i>info</i>       | Hardware information obtained from a call to <b>CFG_MK2_getHardwareInfo()</b> .                                                                              |
| out | <i>pVersion</i>   | String of length <b>STAR_CFG_MK2_VERSION_STR_MAX_LEN</b> that will be updated to contain a null-terminated string representation of the hardware version.    |
| out | <i>pBuildDate</i> | String of length <b>STAR_CFG_MK2_VERSION_STR_MAX_LEN</b> that will be updated to contain a null-terminated string representation of the hardware build date. |

**Added in version:**

STAR-System v1.2 - 13th February 2012

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S, SpaceWire Physical Layer Tester

**7.33.3.3** int **STAR\_API\_CC** CFG\_MK2\_identify ( **STAR\_DEVICE\_ID** deviceID )

Flashes the front panel LEDs of a device.

This can be used to identify the physical device to which a device identifier refers to.

**Parameters**

|                 |                                  |
|-----------------|----------------------------------|
| <i>deviceID</i> | Device to have its LEDs flashed. |
|-----------------|----------------------------------|

**Returns**

1 if the device successfully identified itself, else 0.

**Added in version:**

STAR-System v1.2 - 13th February 2012

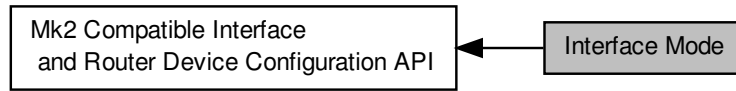
**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S, SpaceWire Physical Layer Tester

## 7.34 Interface Mode

Functions in this group deal managing the device in Interface mode.

Collaboration diagram for Interface Mode:



### Functions

- `int STAR_API_CC CFG_MK2_enableInterfaceMode (STAR_DEVICE_ID deviceID)`  
*In interface mode a packet which is received on an external port, from a SpaceWire link or from the configuration port will be routed to the port specified by the port routing register of the port (set by `CFG_MK2_setPortRoutingAddress()`).*
- `int STAR_API_CC CFG_MK2_getInterfaceModeEnabled (STAR_DEVICE_ID deviceID, int *pEnabled)`  
*Gets whether interface mode has been enabled on a device.*
- `int STAR_API_CC CFG_MK2_enableInterfaceModeOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`  
*Selectively enables interface mode on a given port.*
- `int STAR_API_CC CFG_MK2_getInterfaceModeOnPortEnabled (STAR_DEVICE_ID deviceID, U8 portNum, int *pEnabled)`  
*Gets whether interface mode is enabled on a specific port.*
- `int STAR_API_CC CFG_MK2_disableInterfaceMode (STAR_DEVICE_ID deviceID)`  
*Disables interface mode on a device.*
- `int STAR_API_CC CFG_MK2_disableInterfaceModeOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`  
*Selectively disables interface mode on a given port.*
- `int STAR_API_CC CFG_MK2_enableIdentifySource (STAR_DEVICE_ID deviceID)`  
*When interface mode is enabled on a port, this function globally enables the addition of a leading byte to each received packet indicating which port the packet was received on.*
- `int STAR_API_CC CFG_MK2_getIdentifySourceEnabled (STAR_DEVICE_ID deviceID, int *pEnabled)`  
*Gets whether identify source is enabled.*
- `int STAR_API_CC CFG_MK2_disableIdentifySource (STAR_DEVICE_ID deviceID)`  
*Disables identification of source ports for interface mode globally.*
- `int STAR_API_CC CFG_MK2_enableIdentifySourceOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`  
*Enables identification of source port during interface mode for a given port.*
- `int STAR_API_CC CFG_MK2_getIdentifySourceOnPortEnabled (STAR_DEVICE_ID deviceID, U8 portNum, int *pEnabled)`  
*Gets whether identify source is enabled for a specific port.*
- `int STAR_API_CC CFG_MK2_disableIdentifySourceOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`  
*Disables source port identification for a given port.*
- `int STAR_API_CC CFG_MK2_setPortRoutingAddress (STAR_DEVICE_ID deviceID, U8 portNum, U8 address)`  
*Sets the address a packet received on a given port should be routed to when interface mode is enabled.*
- `int STAR_API_CC CFG_MK2_getPortRoutingAddress (STAR_DEVICE_ID deviceID, U8 portNum, _Out_ U8 *pAddress)`  
*Gets the address a packet received on a given port should be routed to when interface mode is enabled.*

### 7.34.1 Detailed Description

Functions in this group deal managing the device in Interface mode.

### 7.34.2 Function Documentation

#### 7.34.2.1 `int STAR_API_CC_CFG_MK2_disableIdentifySource ( STAR_DEVICE_ID deviceID )`

Disables identification of source ports for interface mode globally.

##### Parameters

|                 |                                                  |
|-----------------|--------------------------------------------------|
| <i>deviceID</i> | Device to disable source port identification on. |
|-----------------|--------------------------------------------------|

##### Returns

1 if source identification was successfully disabled globally, else 0.

##### Added in version:

STAR-System v1.2 - 13th February 2012

##### Supported devices:

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S, SpaceWire Physical Layer Tester

#### 7.34.2.2 `int STAR_API_CC_CFG_MK2_disableIdentifySourceOnPort ( STAR_DEVICE_ID deviceID, U8 portNum )`

Disables source port identification for a given port.

##### Parameters

|                 |                                                                   |
|-----------------|-------------------------------------------------------------------|
| <i>deviceID</i> | Device to disable source port identification for a given port on. |
| <i>portNum</i>  | Port to disable source identification on.                         |

##### Returns

1 if source identification was successfully disabled, else 0.

##### Added in version:

STAR-System v1.2 - 13th February 2012

##### Supported devices:

SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester

#### 7.34.2.3 `int STAR_API_CC_CFG_MK2_disableInterfaceMode ( STAR_DEVICE_ID deviceID )`

Disables interface mode on a device.

When interface mode is disabled, the device operates in routing mode.

**Parameters**


---

|                 |                                      |
|-----------------|--------------------------------------|
| <i>deviceID</i> | Device to disable interface mode on. |
|-----------------|--------------------------------------|

---

**Returns**

1 if interface mode was successfully disabled, else 0.

**Added in version:**

STAR-System v1.2 - 13th February 2012

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S, SpaceWire Physical Layer Tester

#### 7.34.2.4 int **STAR\_API\_CC\_CFG\_MK2\_disableInterfaceModeOnPort** ( **STAR\_DEVICE\_ID** *deviceID*, **U8** *portNum* )

Selectively disables interface mode on a given port.

When interface mode is disabled, the device operates in routing mode.

**Parameters**


---

|                 |                                                 |
|-----------------|-------------------------------------------------|
| <i>deviceID</i> | Device to disable interface mode for a port on. |
| <i>portNum</i>  | Port to disable interface mode on.              |

---

**Returns**

1 if interface mode was successfully disabled, else 0.

**Added in version:**

STAR-System v1.2 - 13th February 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester

#### 7.34.2.5 int **STAR\_API\_CC\_CFG\_MK2\_enableIdentifySource** ( **STAR\_DEVICE\_ID** *deviceID* )

When interface mode is enabled on a port, this function globally enables the addition of a leading byte to each received packet indicating which port the packet was received on.

For each source port on which this behaviour is desired, a call to **CFG\_MK2\_enableIdentifySourceOnPort()** must be made.

**Parameters**


---

|                 |                                                 |
|-----------------|-------------------------------------------------|
| <i>deviceID</i> | Device to enable source port identification on. |
|-----------------|-------------------------------------------------|

---

**Returns**

1 if source identification was successfully enabled globally, else 0.

**Added in version:**

STAR-System v1.2 - 13th February 2012

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S, SpaceWire Physical Layer Tester

#### 7.34.2.6 `int STAR_API_CC_CFG_MK2_enableIdentifySourceOnPort ( STAR_DEVICE_ID deviceID, U8 portNum )`

Enables identification of source port during interface mode for a given port.

Interface mode (`CFG_MK2_enableInterfaceMode()`) and Identify Source (`CFG_MK2_enableIdentifySource()`) must be enabled globally for this to have an effect.

**Parameters**

|                 |                                                                  |
|-----------------|------------------------------------------------------------------|
| <i>deviceID</i> | Device to enable source port identification for a given port on. |
| <i>portNum</i>  | Port to enable source identification on.                         |

**Returns**

1 if source identification was successfully enabled, else 0.

**Added in version:**

STAR-System v1.2 - 13th February 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester

#### 7.34.2.7 `int STAR_API_CC_CFG_MK2_enableInterfaceMode ( STAR_DEVICE_ID deviceID )`

In interface mode a packet which is received on an external port, from a SpaceWire link or from the configuration port will be routed to the port specified by the port routing register of the port (set by `CFG_MK2_setPortRoutingAddress()`).

**Parameters**

|                 |                                     |
|-----------------|-------------------------------------|
| <i>deviceID</i> | Device to enable interface mode on. |
|-----------------|-------------------------------------|

**Returns**

1 if interface mode was successfully enabled, else 0.

**Added in version:**

STAR-System v1.2 - 13th February 2012

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S, SpaceWire Physical Layer Tester

#### 7.34.2.8 `int STAR_API_CC_CFG_MK2_enableInterfaceModeOnPort ( STAR_DEVICE_ID deviceID, U8 portNum )`

Selectively enables interface mode on a given port.

Interface mode must be enabled globally (`CFG_MK2_enableInterfaceMode()`) for interface mode on a port to be enabled.

**Parameters**

|                 |                                                |
|-----------------|------------------------------------------------|
| <i>deviceID</i> | Device to enable interface mode for a port on. |
| <i>portNum</i>  | Port to enable interface mode on.              |

**Returns**

1 if interface mode was successfully enabled, else 0.

**Added in version:**

STAR-System v1.2 - 13th February 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester

### 7.34.2.9 `int STAR_API_CC_CFG_MK2_getIdentifySourceEnabled ( STAR_DEVICE_ID deviceID, int * pEnabled )`

Gets whether identify source is enabled.

**Parameters**

|                 |                                                                                      |
|-----------------|--------------------------------------------------------------------------------------|
| <i>deviceID</i> | Device to check.                                                                     |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if identify source is enabled, else 0. |

**Returns**

1 if the information could be obtained from the device, else 0.

**Added in version:**

STAR-System v1.5 - 23rd April 2012

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S, SpaceWire Physical Layer Tester

### 7.34.2.10 `int STAR_API_CC_CFG_MK2_getIdentifySourceOnPortEnabled ( STAR_DEVICE_ID deviceID, U8 portNum, int * pEnabled )`

Gets whether identify source is enabled for a specific port.

**Note**

SpaceWire PCI Mk2 and cPCI Mk2 and SpaceWire PCIe devices prior to version 1.07 have a bug which means that the value returned in pEnabled is always 0.

**Parameters**

|                 |                  |
|-----------------|------------------|
| <i>deviceID</i> | Device to check. |
| <i>portNum</i>  | Port to check.   |

---

|                 |                                                                                      |
|-----------------|--------------------------------------------------------------------------------------|
| <i>pEnabled</i> | User supplied value that will be updated to 1 if identify source is enabled, else 0. |
|-----------------|--------------------------------------------------------------------------------------|

---

**Returns**

1 if the information could be obtained from the device, else 0.

**Added in version:**

STAR-System v1.5 - 23rd April 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester

#### 7.34.2.11 int STAR\_API\_CC\_CFG\_MK2\_getInterfaceModeEnabled ( STAR\_DEVICE\_ID *deviceId*, int \* *pEnabled* )

Gets whether interface mode has been enabled on a device.

**Parameters**


---

|                 |                                                                                     |
|-----------------|-------------------------------------------------------------------------------------|
| <i>deviceId</i> | The Mk2 compatible device to check.                                                 |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if interface mode is enabled, else 0. |

---

**Returns**

1 if the information could be obtained from the device, else 0

**Added in version:**

STAR-System v1.5 - 23rd April 2012

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S, SpaceWire Physical Layer Tester

#### 7.34.2.12 int STAR\_API\_CC\_CFG\_MK2\_getInterfaceModeOnPortEnabled ( STAR\_DEVICE\_ID *deviceId*, U8 *portNum*, int \* *pEnabled* )

Gets whether interface mode is enabled on a specific port.

**Note**

SpaceWire PCI Mk2 and cPCI Mk2 and SpaceWire PCIe devices prior to version 1.07 have a bug which means that the value returned in *pEnabled* is always 0.

**Parameters**


---

|                 |                                                                                     |
|-----------------|-------------------------------------------------------------------------------------|
| <i>deviceId</i> | Device to get interface mode enabled for a port on.                                 |
| <i>portNum</i>  | Port to get information for.                                                        |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if interface mode is enabled, else 0. |

---

**Returns**

1 if the information could be obtained from the device, else 0.

**Added in version:**

STAR-System v1.5 - 23rd April 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester

7.34.2.13 `int STAR_API_CC_CFG_MK2_getPortRoutingAddress ( STAR_DEVICE_ID deviceID, U8 portNum, _Out_ U8 * pAddress )`

Gets the address a packet received on a given port should be routed to when interface mode is enabled.

**Note**

This is an advanced feature and not necessary for normal usage.

SpaceWire PCI Mk2 and cPCI Mk2 and SpaceWire PCIe devices prior to version 1.07 have a bug which means that the value returned in pAddress is always 0.

**Parameters**

|     |                 |                                                               |
|-----|-----------------|---------------------------------------------------------------|
|     | <i>deviceID</i> | Device to get port routing address from.                      |
|     | <i>portNum</i>  | Port to get port routing address from.                        |
| out | <i>pAddress</i> | Pointer to value that will be updated to contain the address. |

**Returns**

1 if the address could be obtained, else 0.

**Added in version:**

STAR-System v1.2 - 13th February 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester

7.34.2.14 `int STAR_API_CC_CFG_MK2_setPortRoutingAddress ( STAR_DEVICE_ID deviceID, U8 portNum, U8 address )`

Sets the address a packet received on a given port should be routed to when interface mode is enabled.

**Note**

This is an advanced feature and not necessary for normal usage.

**Parameters**

|  |                 |                                                               |
|--|-----------------|---------------------------------------------------------------|
|  | <i>deviceID</i> | Device to have one of its ports port routing address changed. |
|  | <i>portNum</i>  | Port to have its port routing address modified.               |



---

|                |                           |
|----------------|---------------------------|
| <i>address</i> | New port routing address. |
|----------------|---------------------------|

---

**Returns**

1 if the address could be set, else 0.

**Added in version:**

STAR-System v1.2 - 13th February 2012

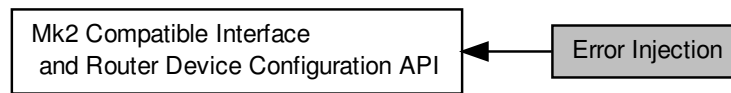
**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester

## 7.35 Error Injection

Functions in this group deal with injecting errors onto SpaceWire links.

Collaboration diagram for Error Injection:



### Enumerations

- enum `SPW_ERROR` {  
`SPW_ERROR_DISCONNECT` = 1, `SPW_ERROR_PARITY` = 2, `SPW_ERROR_ESCAPE` = 3, `SPW_ERROR_INSERT_FCT` = 4,  
`SPW_ERROR_SUPPRESS_FCT` = 5, `SPW_ERROR_INCREMENT_CREDIT` = 6, `SPW_ERROR_DECREMENT_CREDIT` = 7 }

*Errors that can be injected onto the SpaceWire link.*

### Functions

- int `STAR_API_CC_CFG_MK2_injectError` (`STAR_DEVICE_ID` deviceId, unsigned char port, `SPW_ERROR` error)

*Immediately injects the specified error on a given device's port.*

#### 7.35.1 Detailed Description

Functions in this group deal with injecting errors onto SpaceWire links.

#### 7.35.2 Enumeration Type Documentation

##### 7.35.2.1 enum `SPW_ERROR`

Errors that can be injected onto the SpaceWire link.

Added in version:

STAR-System v2.0 - 3rd July 2013

Enumerator

**`SPW_ERROR_DISCONNECT`** Causes the SpaceWire link to disconnect.

**`SPW_ERROR_PARITY`** Inserts a parity error on the next SpaceWire character to be transmitted by flipping the parity bit.

**`SPW_ERROR_ESCAPE`** Transmits an `ESCAPE_ESCAPE` code.

**`SPW_ERROR_INSERT_FCT`** Inserts an FCT character on the next character to be sent. This can be used to force FCT errors on the SpaceWire link.

***SPW\_ERROR\_SUPPRESS\_FCT*** Ignore the next FCT character received on the link. This can be used to simulate the loss of an FCT character.

***SPW\_ERROR\_INCREMENT\_CREDIT*** Increments the transmit credit available to send data by 8. This can be used to force a data overflow in the opposite end of the link, as too much data will be transmitted.

***SPW\_ERROR\_DECREMENT\_CREDIT*** Decrements the transmit credit count by 1.

### 7.35.3 Function Documentation

7.35.3.1 `int STAR_API_CC_CFG_MK2_injectError ( STAR_DEVICE_ID deviceId, unsigned char port, SPW_ERROR error )`

Immediately injects the specified error on a given device's port.

#### Note

This function is currently only compatible with [SpaceWire PCI Mk2 and cPCI Mk2](#), [SpaceWire Physical Layer Tester](#) and [SpaceWire PCIe](#) devices. No check is made by this function to ensure it is being used with a compatible device. For [SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#) and [SpaceWire Brick Mk3](#) devices, please use [CFG\\_BRICK\\_MK2\\_injectErrors\(\)](#).

Error injection makes use of the same on-board registers as periodic actions on devices which support it, therefore error injection cannot be used on those devices when periodic actions are in progress.

#### Parameters

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <i>deviceId</i> | Identifier of the device to have the error injected on. |
| <i>port</i>     | Port the error is to be injected on.                    |
| <i>error</i>    | SpaceWire error to be injected.                         |

#### Returns

1 if the error information was successfully sent to the device, else 0.

#### Added in version:

STAR-System v2.0 - 3rd July 2013

#### Supported devices:

[SpaceWire PCI Mk2 and cPCI Mk2](#), [SpaceWire PCIe](#), [SpaceWire Physical Layer Tester](#)

## 7.36 Periodic Actions

Functions in this group deal with controlling periodic actions on SpaceWire links.

Collaboration diagram for Periodic Actions:



### Functions

- `int STAR_API_CC_CFG_MK2_startPeriodicAction (STAR_DEVICE_ID deviceId, unsigned char port, U32 count, SPW_ACTION action)`  
*Starts a periodic action on a given device's port.*
- `int STAR_API_CC_CFG_MK2_stopPeriodicAction (STAR_DEVICE_ID deviceId, unsigned char port)`  
*Stops a periodic action on a given device's port.*
- `U32 STAR_API_CC_CFG_MK2_getDeviceClockRate (STAR_DEVICE_ID deviceId)`  
*Gets a device's internal clock rate.*

#### 7.36.1 Detailed Description

Functions in this group deal with controlling periodic actions on SpaceWire links.

#### 7.36.2 Function Documentation

##### 7.36.2.1 U32 STAR\_API\_CC\_CFG\_MK2\_getDeviceClockRate ( STAR\_DEVICE\_ID deviceId )

Gets a device's internal clock rate.

##### Parameters

|                 |                                    |
|-----------------|------------------------------------|
| <i>deviceId</i> | Identifier of the device to query. |
|-----------------|------------------------------------|

##### Returns

The device's clock rate in Hz, or zero if the device does not support periodic actions.

##### Added in version:

STAR-System v3.0 Beta 3 - 4th September 2015

##### Supported devices:

SpaceWire PCI Mk2 and cPCI Mk2

### 7.36.2.2 `int STAR_API_CC_CFG_MK2_startPeriodicAction ( STAR_DEVICE_ID deviceID, unsigned char port, U32 count, SPW_ACTION action )`

Starts a periodic action on a given device's port.

To stop a periodic action use `CFG_MK2_stopPeriodicAction()`.

#### Note

This function is currently only compatible with [SpaceWire PCI Mk2](#) and [cPCI Mk2](#) devices. No check is made by this function to ensure it is being used with a compatible device.

Periodic actions make use of the same on-board registers as error injection, therefore error injection cannot be used when periodic actions are in progress.

#### Parameters

|                 |                                                                                                                                                   |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>deviceID</i> | Identifier of the device to have periodic action created for.                                                                                     |
| <i>port</i>     | Port the periodic action is to be performed on.                                                                                                   |
| <i>count</i>    | Number of device clock cycles between action repetitions. See <a href="#">CFG_MK2_getDeviceClockRate()</a> for obtaining the device's clock rate. |
| <i>action</i>   | Action to be performed. This can be <a href="#">SPW_TRANSMIT_PACKET</a> or any of the <a href="#">SPW_ERROR</a> values.                           |

#### Returns

1 if the periodic action was successfully started, else 0.

#### Added in version:

[STAR-System v3.0 Beta 3 - 4th September 2015](#)

#### Supported devices:

[SpaceWire PCI Mk2](#) and [cPCI Mk2](#)

### 7.36.2.3 `int STAR_API_CC_CFG_MK2_stopPeriodicAction ( STAR_DEVICE_ID deviceID, unsigned char port )`

Stops a periodic action on a given device's port.

Periodic actions are started by calling `CFG_MK2_startPeriodicAction()`.

#### Note

This function is currently only compatible with [SpaceWire PCI Mk2](#) and [cPCI Mk2](#) devices. No check is made by this function to ensure it is being used with a compatible device.

#### Parameters

|                 |                                                          |
|-----------------|----------------------------------------------------------|
| <i>deviceID</i> | Identifier of the device performing the periodic action. |
| <i>port</i>     | Port the periodic action is being performed on.          |

#### Returns

1 if the periodic action was successfully stopped, else 0.

#### Added in version:

[STAR-System v3.0 Beta 3 - 4th September 2015](#)

#### Supported devices:

[SpaceWire PCI Mk2](#) and [cPCI Mk2](#)

## 7.37 PCI Mk2 Packet Subsystem

Functions in this group deal with setting the packet subsystem parameters for PCI Mk2 Devices.

### Data Structures

- struct `PKT_SUBSYS_STATISTICS`

*Statistics that can be obtained from the PCI Mk2 device packet sink subsystem. [More...](#)*

### Typedefs

- typedef void(`STAR_API_CC` \* `PKT_SUBSYS_StatisticsFunc`) (`STAR_DEVICE_ID` deviceId, unsigned int subsystem, `PKT_SUBSYS_STATISTICS` statistics)

*The function type used by the statistics loop function, which is called when a monitored subsystem's statistics are updated.*

- typedef U32 `STATISTICS_LOOP_ID`

*Type of identifier used to identify a statistics polling thread.*

### Enumerations

- enum `PKT_SUBSYS_PATTERN` {  
`PKT_SUBSYS_PATTERN_ALL_ZEROES`, `PKT_SUBSYS_PATTERN_ALL_ONES`, `PKT_SUBSYS_PATTERN_ALTERNATING_BITS`, `PKT_SUBSYS_PATTERN_INCREMENTING_BYTES`,  
`PKT_SUBSYS_PATTERN_DECREMENTING_BYTES`, `PKT_SUBSYS_PATTERN_WALKING_ONES`,  
`PKT_SUBSYS_PATTERN_WALKING_ZEROES` }

*Available patterns to use with `PKT_SUBSYS_fillBufferWithPattern()`*

### Functions

- void `STAR_API_CC` `PKT_SUBSYS_fillBufferWithPattern` (`PKT_SUBSYS_PATTERN` pattern, U16 bufferSize, U16 \*pBuffer)

*Convenience function that fills a user provided buffer with a known pattern.*

- int `STAR_API_CC` `PKT_SUBSYS_writeMemory` (`STAR_DEVICE_ID` deviceId, U16 address, U16 writeSize, U16 \*pBuffer)

*Writes a buffer to the PCI Mk2's dual port memory.*

- int `STAR_API_CC` `PKT_SUBSYS_readMemory` (`STAR_DEVICE_ID` deviceId, U16 address, U16 readSize, U16 \*pBuffer)

*Reads from the PCI Mk2's dual port memory to a user allocated buffer.*

- int `STAR_API_CC` `PKT_SUBSYS_setPacketGeneratorFormat` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem, U16 headerPointer, U16 headerLength, U16 bodyPointer, U16 bodyLength, U32 packetLength, `STAR_EOP_TYPE` eop, U16 gap)

*Sets up the format of the packet to be inserted by the packet generator into the router.*

- int `STAR_API_CC` `PKT_SUBSYS_setGeneratorBlockingParameters` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem, U16 blockFrequency, U16 blockDuration)

*Sets up the packet generation subsystem blocking parameters.*

- int `STAR_API_CC` `PKT_SUBSYS_startPacketGeneration` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem, U16 sequenceLength)

*Starts the packet generator inserting packets into the router.*

- int `STAR_API_CC` `PKT_SUBSYS_waitForPacketGenerationSequenceComplete` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem, unsigned int timeout)

*Blocks the current thread until packet generation has stopped.*

- int `STAR_API_CC_PKT_SUBSYS_stopPacketGeneration` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem)  
*Stops the packet generator inserting packets into the router.*
- int `STAR_API_CC_PKT_SUBSYS_setPacketSinkParameters` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem, U16 memoryPointer, U16 memoryLength)  
*Sets up the circular buffer for the packet sink subsystem.*
- int `STAR_API_CC_PKT_SUBSYS_setSinkBlockingParameters` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem, U16 blockFrequency, U16 blockDuration)  
*Sets up the packet sink subsystem blocking parameters.*
- int `STAR_API_CC_PKT_SUBSYS_startSink` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem)  
*Starts the packet sink reading packets into memory.*
- int `STAR_API_CC_PKT_SUBSYS_stopSink` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem)  
*Stops the packet sink reading packets into memory.*
- int `STAR_API_CC_PKT_SUBSYS_getSinkDataPointers` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem, U16 \*startPointer, U16 \*endPointer)  
*Gets the address for the start and end of valid data in the circular buffer in device memory.*
- int `STAR_API_CC_PKT_SUBSYS_startPacketChecking` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem, int disableSinkOnError, U16 disableSinkdelay)  
*Starts a packet checker running on a packet sink.*
- int `STAR_API_CC_PKT_SUBSYS_stopPacketChecking` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem)  
*Stops the packet checker.*
- int `STAR_API_CC_PKT_SUBSYS_waitForPacketCheckingError` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem)  
*Blocks the current thread until either a character mismatch is detected and the packet checker is automatically stopped, or the user manually stops the packet checker.*
- int `STAR_API_CC_PKT_SUBSYS_cancelWaitsForPacketCheckingError` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem)  
*Cancels all waits started by `PKT_SUBSYS_waitForPacketCheckingError()` for packet checker errors on a given device and subsystem.*
- int `STAR_API_CC_PKT_SUBSYS_setPacketCheckingFormat` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem, U16 headerPointer, U16 headerLength, U16 bodyPointer, U16 bodyLength, U32 packetLength, `STAR_EOP_TYPE` eop)  
*Sets up the packet checker to check packets match the format of packets provided by user entered parameters.*
- int `STAR_API_CC_PKT_SUBSYS_getPacketCheckingMismatchCount` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem, U64 \*errorCount)  
*Gets the number of mismatched data characters observed on the SpaceWire interface.*
- int `STAR_API_CC_PKT_SUBSYS_getStatistics` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem, `PKT_SUBSYS_STATISTICS` \*pStatistics)  
*Gets statistics for a given subsystem.*
- `STATISTICS_LOOP_ID` `STAR_API_CC_PKT_SUBSYS_startGetStatisticsLoop` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem, `PKT_SUBSYS_StatisticsFunc` statisticsHandler)  
*Starts a thread that polls statistics on a given subsystem, and calls a given function with the updated statistics when they are retrieved.*
- void `STAR_API_CC_PKT_SUBSYS_stopGetStatisticsLoop` (`STATISTICS_LOOP_ID` loopId)  
*Stops a statistics gathering loop set up by `PKT_SUBSYS_startGetStatisticsLoop()`.*

### 7.37.1 Detailed Description

Functions in this group deal with setting the packet subsystem parameters for PCI Mk2 Devices.

The PCI Mk2 device contains a single hardware packet generator and checker subsystem. This subsystem can be used to automatically generate and check SpaceWire packets at high speed without using the resources of the host PC.

On the device there is a dual-port memory that is used to store the format for packets to be generated and checked, and also used for the circular buffer used by the packet sink. Packets of any length can be generated with a separate header and cargo, which is specified by pointers to blocks of this shared memory. Additionally, it is possible to control the rate at which packets are sent. The user can set the number of clock cycles when no data is sent, (blocking) and also the number of clock cycles between packets (gap).

The packet sink can be used to read incoming packets directly to a user defined circular buffer in the dual-port memory. A packet checker can be run on each packets sink. The packet sink must be running on a subsystem in order for statistics to be gathered or packet checking to occur. Statistics are updated once per second.

The packet checker receives incoming packets and checks them against a template held in the dual-port memory. If a mismatched character is detected, then the packet checker stops the packet sink it is running on after a specified number of data characters are received. This allows the user to set up a circular buffer (packet sink) that can be read back to show what data preceded and followed the mismatched character, aiding debugging.

#### Attention

Currently the PCI Mk2 hardware supports only a single packet subsystem. Attempts to use subsystems 2 or 3 will result functions returning error. Note that subsystem 1 is connected to port 8 of the PCI Mk2 device's internal router.

## 7.37.2 Data Structure Documentation

### 7.37.2.1 struct PKT\_SUBSYS\_STATISTICS

Statistics that can be obtained from the PCI Mk2 device packet sink subsystem.

Statistics are updated once per second.

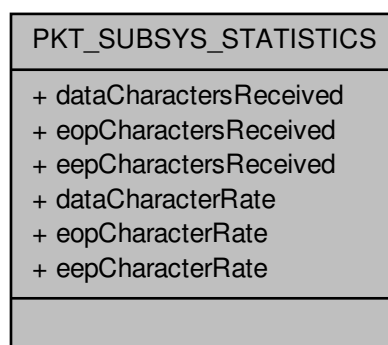
#### Added in version:

STAR-System v1.7 - 20th July 2012

#### Examples:

`packet_subsystem_example.c`.

Collaboration diagram for PKT\_SUBSYS\_STATISTICS:





## Data Fields

|     |                                  |                                                                                                                |
|-----|----------------------------------|----------------------------------------------------------------------------------------------------------------|
| U32 | dataCharacter↔<br>Rate           | Number of data characters received in the last sampling period. The sampling period is one second in duration. |
| U64 | data↔<br>Characters↔<br>Received | Number of data characters received since the PCI Mk2 was last reset.                                           |
| U32 | eepCharacter↔<br>Rate            | Number of EEP characters received in the last sampling period. The sampling period is one second in duration.  |
| U64 | eepCharacters↔<br>Received       | Number of EEP characters received since the PCI Mk2 was last reset.                                            |
| U32 | eopCharacter↔<br>Rate            | Number of EOP characters received in the last sampling period. The sampling period is one second in duration.  |
| U64 | eopCharacters↔<br>Received       | Number of EOP characters received since the PCI Mk2 was last reset.                                            |

## 7.37.3 Typedef Documentation

**7.37.3.1** `typedef void(STAR_API_CC * PKT_SUBSYS_StatisticsFunc) (STAR_DEVICE_ID deviceId, unsigned int subsystem, PKT_SUBSYS_STATISTICS statistics)`

The function type used by the statistics loop function, which is called when a monitored subsystem's statistics are updated.

## Parameters

|                   |                                                                       |
|-------------------|-----------------------------------------------------------------------|
| <i>deviceId</i>   | Identifier for the device statistics were obtained from.              |
| <i>subsystem</i>  | Subsystem number (indexed from 1) that statistics were obtained from. |
| <i>statistics</i> | Newly gathered statistics                                             |

Added in version:

STAR-System v1.7 - 20th July 2012

7.37.3.2 `typedef U32 STATISTICS_LOOP_ID`

Type of identifier used to identify a statistics polling thread.

Used when manually stopping a statistics polling thread.

Added in version:

STAR-System v1.7 - 20th July 2012

## 7.37.4 Enumeration Type Documentation

7.37.4.1 `enum PKT_SUBSYS_PATTERN`

Available patterns to use with `PKT_SUBSYS_fillBufferWithPattern()`

Added in version:

STAR-System v1.7 - 20th July 2012

## 7.37.5 Function Documentation

7.37.5.1 `int STAR_API_CC PKT_SUBSYS_cancelWaitsForPacketCheckingError ( STAR_DEVICE_ID deviceld, unsigned int subsystem )`

Cancels all waits started by `PKT_SUBSYS_waitForPacketCheckingError()` for packet checker errors on a given device and subsystem.

**Parameters**

|                  |                                                          |
|------------------|----------------------------------------------------------|
| <i>deviceld</i>  | Identifier for the device packet checking is started on. |
| <i>subsystem</i> | Subsystem number to configure (indexed from 1).          |

**Returns**

1 if packet sink was stopped, or 0 if an error occurred.

**Added in version:**

STAR-System v1.7 - 20th July 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2

**7.37.5.2** void **STAR\_API\_CC** PKT\_SUBSYS\_fillBufferWithPattern ( PKT\_SUBSYS\_PATTERN *pattern*, U16 *bufferSize*, \_Out\_bytecap\_(bufferSize) void \* *pBuffer* )

Convenience function that fills a user provided buffer with a known pattern.

**Parameters**

|                   |                                        |
|-------------------|----------------------------------------|
| <i>pattern</i>    | Pattern to generate.                   |
| <i>bufferSize</i> | Size in bytes of user provided buffer. |
| <i>pBuffer</i>    | Pointer to user allocated buffer.      |

**Added in version:**

STAR-System v1.7 - 20th July 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2

**Examples:**

packet\_subsystem\_example.c.

**7.37.5.3** int **STAR\_API\_CC** PKT\_SUBSYS\_getPacketCheckingMismatchCount ( STAR\_DEVICE\_ID *deviceld*, unsigned int *subsystem*, U64 \* *errorCount* )

Gets the number of mismatched data characters observed on the SpaceWire interface.

**Parameters**

|                              |                                                            |
|------------------------------|------------------------------------------------------------|
| <i>deviceld</i>              | Identifier for the device to start the packet checking on. |
| <i>subsystem</i>             | Subsystem number to configure (indexed from 1)             |
| <i>out</i> <i>errorCount</i> | Count of mismatched characters.                            |

**Returns**

1 if packet sink was started.

**Added in version:**

STAR-System v1.7 - 20th July 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2

**Examples:**

packet\_subsystem\_example.c.

**7.37.5.4** int **STAR\_API\_CC** PKT\_SUBSYS\_getSinkDataPointers ( STAR\_DEVICE\_ID *deviceld*, unsigned int *subsystem*, U16 \* *startPointer*, U16 \* *endPointer* )

Gets the address for the start and end of valid data in the circular buffer in device memory.

Note that if the endPointer is less than the startPointer then the end pointer has wrapped round since the sink was last started.

**Parameters**

|     |                     |                                                                       |
|-----|---------------------|-----------------------------------------------------------------------|
|     | <i>deviceld</i>     | Identifier for the device to get the memory location for.             |
|     | <i>subsystem</i>    | Subsystem number to get information from (indexed from 1).            |
| out | <i>startPointer</i> | 32-bit word address of start of acquired data in the circular buffer. |
| out | <i>endPointer</i>   | 32-bit word address of end of acquired data in the circular buffer.   |

**Returns**

1 if packet sink was stopped.

**Added in version:**

STAR-System v1.7 - 20th July 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2

**Examples:**

packet\_subsystem\_example.c.

**7.37.5.5** int **STAR\_API\_CC** PKT\_SUBSYS\_getStatistics ( STAR\_DEVICE\_ID *deviceld*, unsigned int *subsystem*, PKT\_SUBSYS\_STATISTICS \* *pStatistics* )

Gets statistics for a given subsystem.

Note that a sink must be started for statistics to be updated.

**Parameters**

|     |                    |                                                                                                                                        |
|-----|--------------------|----------------------------------------------------------------------------------------------------------------------------------------|
|     | <i>deviceld</i>    | Identifier for the device to stop the packet sink on.                                                                                  |
|     | <i>subsystem</i>   | Subsystem number to configure (indexed from 1).                                                                                        |
| out | <i>pStatistics</i> | Pointer to a user allocated PKT_SUBSYS_STATISTICS structure. that will be updated with the current values of the statistics registers. |

**Returns**

1 if packet sink was stopped.

**Added in version:**

STAR-System v1.7 - 20th July 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2

**7.37.5.6** `int STAR_API_CC PKT_SUBSYS_readMemory ( STAR_DEVICE_ID deviceld, U16 address, U16 readSize,  
_Inout_bytecap_(readSize) void * pBuffer )`

Reads from the PCI Mk2's dual port memory to a user allocated buffer.

#### Note

The dual port memory is made up of 64K 32 bit words. Therefore the valid address range is 0 through 65535. Attempts to read from addresses beyond this range will fail.

#### Parameters

|                |                 |                                                                                                                                      |
|----------------|-----------------|--------------------------------------------------------------------------------------------------------------------------------------|
|                | <i>deviceld</i> | Identifier for the device to read from.                                                                                              |
|                | <i>address</i>  | Word address to read from in the device's dual port memory. Address values 0 through 65536 are valid.                                |
|                | <i>readSize</i> | Size in bytes to read from the device memory into user provided buffer. This must be a multiple of 4 (only entire words may be read) |
| <i>in, out</i> | <i>pBuffer</i>  | Pointer to user allocated buffer.                                                                                                    |

#### Returns

0 if the memory was successfully read from to, else 1.

#### Added in version:

STAR-System v1.7 - 20th July 2012

#### Supported devices:

SpaceWire PCI Mk2 and cPCI Mk2

#### Examples:

`packet_subsystem_example.c`.

**7.37.5.7** `int STAR_API_CC PKT_SUBSYS_setGeneratorBlockingParameters ( STAR_DEVICE_ID deviceld, unsigned int  
subsystem, U16 blockFrequency, U16 blockDuration )`

Sets up the packet generation subsystem blocking parameters.

When blocked, no packet data will be inserted into the router by the packet generator.

#### Parameters

|  |                       |                                                                                           |
|--|-----------------------|-------------------------------------------------------------------------------------------|
|  | <i>deviceld</i>       | Identifier for the device to set up the packet generator subsystem for.                   |
|  | <i>subsystem</i>      | Subsystem number to configure (indexed from 1)                                            |
|  | <i>blockFrequency</i> | Number of clock cycles between the end of a blocking operation and the start of the next. |
|  | <i>blockDuration</i>  | Number of clock cycles which blocking will occur over.                                    |

#### Returns

1 if packet generation was stopped.

#### Added in version:

STAR-System v1.7 - 20th July 2012

#### Supported devices:

SpaceWire PCI Mk2 and cPCI Mk2

7.37.5.8 `int STAR_API_CC PKT_SUBSYS_setPacketCheckingFormat ( STAR_DEVICE_ID deviceld, unsigned int subsystem, U16 headerPointer, U16 headerLength, U16 bodyPointer, U16 bodyLength, U32 packetLength, STAR_EOP_TYPE eop )`

Sets up the packet checker to check packets match the format of packets provided by user entered parameters.

**Parameters**

|                      |                                                                                               |
|----------------------|-----------------------------------------------------------------------------------------------|
| <i>deviceId</i>      | Identifier for the device to set up the packet checking subsystem for.                        |
| <i>subsystem</i>     | Subsystem number to configure (indexed from 1)                                                |
| <i>headerPointer</i> | Address of the expected packet header in the device's memory.                                 |
| <i>headerLength</i>  | Length of the expected packet header in bytes.                                                |
| <i>bodyPointer</i>   | Starting address of the area in device memory to be used for the expected packet body values. |
| <i>bodyLength</i>    | Expected Length of the packet body in bytes.                                                  |
| <i>packetLength</i>  | Expected length of entire packet.                                                             |
| <i>eop</i>           | Type of end of packet identifier to be used. Valid values are EOP or EEP.                     |

**Returns**

1 if the packet format was successfully set.

**Added in version:**

STAR-System v1.7 - 20th July 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2

**Examples:**

`packet_subsystem_example.c`.

**7.37.5.9** `int STAR_API_CC PKT_SUBSYS_setPacketGeneratorFormat ( STAR_DEVICE_ID deviceId, unsigned int subsystem, U16 headerPointer, U16 headerLength, U16 bodyPointer, U16 bodyLength, U32 packetLength, STAR_EOP_TYPE eop, U16 gap )`

Sets up the format of the packet to be inserted by the packet generator into the router.

**Parameters**

|                      |                                                                                           |
|----------------------|-------------------------------------------------------------------------------------------|
| <i>deviceId</i>      | Identifier for the device to set up the packet generator subsystem for.                   |
| <i>subsystem</i>     | Subsystem number to configure (indexed from 1)                                            |
| <i>headerPointer</i> | Starting word address of the packet header in the device's memory.                        |
| <i>headerLength</i>  | Length of the packet header in bytes.                                                     |
| <i>bodyPointer</i>   | Starting word address of the area in device memory to be used for the packet body values. |
| <i>bodyLength</i>    | Number of consecutive words in memory the packet generator will use to build packets.     |
| <i>packetLength</i>  | Desired length of entire packet.                                                          |
| <i>eop</i>           | Type of end of packet identifier to be used. Valid values are EOP or EEP.                 |
| <i>gap</i>           | Number of clock cycles between the end of one packet and the start of the next.           |

**Returns**

1 if the packet format was successfully set.

**Added in version:**

STAR-System v1.7 - 20th July 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2

**Examples:**

`packet_subsystem_example.c`.

7.37.5.10 `int STAR_API_CC PKT_SUBSYS_setPacketSinkParameters ( STAR_DEVICE_ID deviceId, unsigned int subsystem, U16 memoryPointer, U16 memoryLength )`

Sets up the circular buffer for the packet sink subsystem.



**Parameters**

|                      |                                                                                            |
|----------------------|--------------------------------------------------------------------------------------------|
| <i>deviceld</i>      | Identifier for the device to set up the packet sink subsystem for.                         |
| <i>subsystem</i>     | Subsystem number to configure (indexed from 1)                                             |
| <i>memoryPointer</i> | Starting word address for the circular buffer used to record packets received by the sink. |
| <i>memoryLength</i>  | Length of the circular buffer in words.                                                    |

**Returns**

1 if the packet sink parameters were set.

**Added in version:**

STAR-System v1.7 - 20th July 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2

**Examples:**

packet\_subsystem\_example.c.

#### 7.37.5.11 int **STAR\_API\_CC\_PKT\_SUBSYS\_setSinkBlockingParameters** ( **STAR\_DEVICE\_ID** *deviceld*, unsigned int *subsystem*, U16 *blockFrequency*, U16 *blockDuration* )

Sets up the packet sink subsystem blocking parameters.

When blocked, no packet data will be read from router into packet sink memory.

**Parameters**

|                       |                                                                                           |
|-----------------------|-------------------------------------------------------------------------------------------|
| <i>deviceld</i>       | Identifier for the device to set up the packet generator subsystem for.                   |
| <i>subsystem</i>      | Subsystem number to configure (indexed from 1)                                            |
| <i>blockFrequency</i> | Number of clock cycles between the end of a blocking operation and the start of the next. |
| <i>blockDuration</i>  | Number of clock cycles which blocking will occur over.                                    |

**Returns**

1 if packet sink blocking parameters were set.

**Added in version:**

STAR-System v1.7 - 20th July 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2

#### 7.37.5.12 **STATISTICS\_LOOP\_ID STAR\_API\_CC\_PKT\_SUBSYS\_startGetStatisticsLoop** ( **STAR\_DEVICE\_ID** *deviceld*, unsigned int *subsystem*, **PKT\_SUBSYS\_StatisticsFunc** *statisticsHandler* )

Starts a thread that polls statistics on a given subsystem, and calls a given function with the updated statistics when they are retrieved.

If an error occurs obtaining the statistics, the thread terminates. The thread will keep running until stopped with **PKT\_SUBSYS\_stopGetStatisticsLoop()** even if the packet sink is disabled (though no statistics are updated when the packet sink is disabled).

**Note**

If packet checking is enabled and a character mismatch occurs, the packet sink is disabled. This causes statistics to stop being gathered until the sink is restarted.

**Parameters**

|                          |                                                                               |
|--------------------------|-------------------------------------------------------------------------------|
| <i>deviceld</i>          | Identifier for the device to stop the packet sink on.                         |
| <i>subsystem</i>         | Subsystem number to configure (indexed from 1)                                |
| <i>statisticsHandler</i> | Pointer to function that will be called when updated statistics are obtained. |

**Returns**

Identifier for the loop that can be used to stop it.

**Added in version:**

STAR-System v1.7 - 20th July 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2

**Examples:**

packet\_subsystem\_example.c.

**7.37.5.13** int **STAR\_API\_CC\_PKT\_SUBSYS\_startPacketChecking** ( **STAR\_DEVICE\_ID** *deviceld*, unsigned int *subsystem*, int *disableSinkOnError*, **U16** *disableSinkdelay* )

Starts a packet checker running on a packet sink.

If a character mismatch is detected the packet sink is disabled after a specified number of data characters are received. Note that a packet sink must be running to enable checking of incoming packets.

**Parameters**

|                           |                                                                                                                   |
|---------------------------|-------------------------------------------------------------------------------------------------------------------|
| <i>deviceld</i>           | Identifier for the device to start the packet checking on.                                                        |
| <i>subsystem</i>          | Subsystem number to configure (indexed from 1).                                                                   |
| <i>disableSinkOnError</i> | Whether to disable the packet sink if a mismatched character is received (1) or not (0).                          |
| <i>disableSinkdelay</i>   | The number of bytes to be received after a character mismatch is detected and before the packet sink is disabled. |

**Returns**

1 if packet checking was started.

**Added in version:**

STAR-System v1.7 - 20th July 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2

**Examples:**

packet\_subsystem\_example.c.

**7.37.5.14** int **STAR\_API\_CC\_PKT\_SUBSYS\_startPacketGeneration** ( **STAR\_DEVICE\_ID** *deviceld*, unsigned int *subsystem*, **U16** *sequenceLength* )

Starts the packet generator inserting packets into the router.

**Parameters**

|                       |                                                                                                                                       |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <i>deviceId</i>       | Identifier for the device to set up the packet generator subsystem for.                                                               |
| <i>subsystem</i>      | Subsystem number to configure (indexed from 1)                                                                                        |
| <i>sequenceLength</i> | Number of packets that will be inserted before generation stops. Specify a sequence length of zero to enable continuous transmission. |

**Returns**

1 if packet generation was started.

**Added in version:**

STAR-System v1.7 - 20th July 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2

**Examples:**

`packet_subsystem_example.c`.

#### 7.37.5.15 `int STAR_API_CC PKT_SUBSYS_startSink ( STAR_DEVICE_ID deviceId, unsigned int subsystem )`

Starts the packet sink reading packets into memory.

**Parameters**

|                  |                                                        |
|------------------|--------------------------------------------------------|
| <i>deviceId</i>  | Identifier for the device to start the packet sink on. |
| <i>subsystem</i> | Subsystem number to configure (indexed from 1)         |

**Returns**

1 if packet sink was started.

**Added in version:**

STAR-System v1.7 - 20th July 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2

**Examples:**

`packet_subsystem_example.c`.

#### 7.37.5.16 `void STAR_API_CC PKT_SUBSYS_stopGetStatisticsLoop ( STATISTICS_LOOP_ID loopId )`

Stops a statistics gathering loop set up by `PKT_SUBSYS_startGetStatisticsLoop()`.

**Parameters**

|               |                                  |
|---------------|----------------------------------|
| <i>loopId</i> | Identifier for the loop to stop. |
|---------------|----------------------------------|

**Added in version:**

STAR-System v1.7 - 20th July 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2

**Examples:**

packet\_subsystem\_example.c.

**7.37.5.17** int STAR\_API\_CC\_PKT\_SUBSYS\_stopPacketChecking ( STAR\_DEVICE\_ID *deviceld*, unsigned int *subsystem* )

Stops the packet checker.

**Parameters**

|                  |                                                          |
|------------------|----------------------------------------------------------|
| <i>deviceld</i>  | Identifier for the device packet checking is started on. |
| <i>subsystem</i> | Subsystem number to configure (indexed from 1).          |

**Returns**

1 if packet checking was stopped

**Added in version:**

STAR-System v1.7 - 20th July 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2

**Examples:**

packet\_subsystem\_example.c.

**7.37.5.18** int STAR\_API\_CC\_PKT\_SUBSYS\_stopPacketGeneration ( STAR\_DEVICE\_ID *deviceld*, unsigned int *subsystem* )

Stops the packet generator inserting packets into the router.

**Parameters**

|                  |                                                                         |
|------------------|-------------------------------------------------------------------------|
| <i>deviceld</i>  | Identifier for the device to set up the packet generator subsystem for. |
| <i>subsystem</i> | Subsystem number to configure (indexed from 1)                          |

**Returns**

1 if packet generation was stopped.

**Added in version:**

STAR-System v1.7 - 20th July 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2

**7.37.5.19** int STAR\_API\_CC\_PKT\_SUBSYS\_stopSink ( STAR\_DEVICE\_ID *deviceld*, unsigned int *subsystem* )

Stops the packet sink reading packets into memory.

**Parameters**

|                  |                                                       |
|------------------|-------------------------------------------------------|
| <i>deviceld</i>  | Identifier for the device to stop the packet sink on. |
| <i>subsystem</i> | Subsystem number to configure (indexed from 1)        |

**Returns**

1 if packet sink was stopped.

**Added in version:**

STAR-System v1.7 - 20th July 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2

**Examples:**

`packet_subsystem_example.c`.

**7.37.5.20** `int STAR_API_CC_PKT_SUBSYS_waitForPacketCheckingError ( STAR_DEVICE_ID deviceld, unsigned int subsystem )`

Blocks the current thread until either a character mismatch is detected and the packet checker is automatically stopped, or the user manually stops the packet checker.

**Parameters**

|                  |                                                          |
|------------------|----------------------------------------------------------|
| <i>deviceld</i>  | Identifier for the device packet checking is started on. |
| <i>subsystem</i> | Subsystem number to configure (indexed from 1).          |

**Returns**

1 if packet sink was stopped, or 0 if an error occurred.

**Added in version:**

STAR-System v1.7 - 20th July 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2

**7.37.5.21** `int STAR_API_CC_PKT_SUBSYS_waitForPacketGenerationSequenceComplete ( STAR_DEVICE_ID deviceld, unsigned int subsystem, unsigned int timeout )`

Blocks the current thread until packet generation has stopped.

**Parameters**

|                  |                                                                            |
|------------------|----------------------------------------------------------------------------|
| <i>deviceld</i>  | Identifier for the device with the packet generator subsystem to wait for. |
| <i>subsystem</i> | Subsystem number to wait on (indexed from 1)                               |
| <i>timeout</i>   | Timeout period in milliseconds. Provide a value of 0 to wait indefinitely. |

**Returns**

0 if error occurred communicating with device, 1 if packet generation completed or -1 if timeout occurred.

**Added in version:**

STAR-System v1.7 - 20th July 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2

**Examples:**

packet\_subsystem\_example.c.

**7.37.5.22** `int STAR_API_CC PKT_SUBSYS_writeMemory ( STAR_DEVICE_ID deviceId, U16 address, U16 writeSize,  
_In_bytecount_(writeSize) void * pBuffer )`

Writes a buffer to the PCI Mk2's dual port memory.

**Note**

The dual port memory is made up of 64K 32 bit words. Therefore the valid address range is 0 through 65535. Attempts to write to addresses beyond this range will fail.

**Parameters**

|           |                  |                                                                                                                                    |
|-----------|------------------|------------------------------------------------------------------------------------------------------------------------------------|
|           | <i>deviceId</i>  | Identifier for the device to write to.                                                                                             |
|           | <i>address</i>   | Word address to write to in the device's dual port memory. Address values 0 through 65536 are valid.                               |
|           | <i>writeSize</i> | Length in bytes of user provided buffer to write to device memory. This must be a multiple of 4 (only entire words may be written) |
| <i>in</i> | <i>pBuffer</i>   | Pointer to user allocated buffer.                                                                                                  |

**Returns**

0 if the memory was successfully written to, else 1.

**Added in version:**

STAR-System v1.7 - 20th July 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2

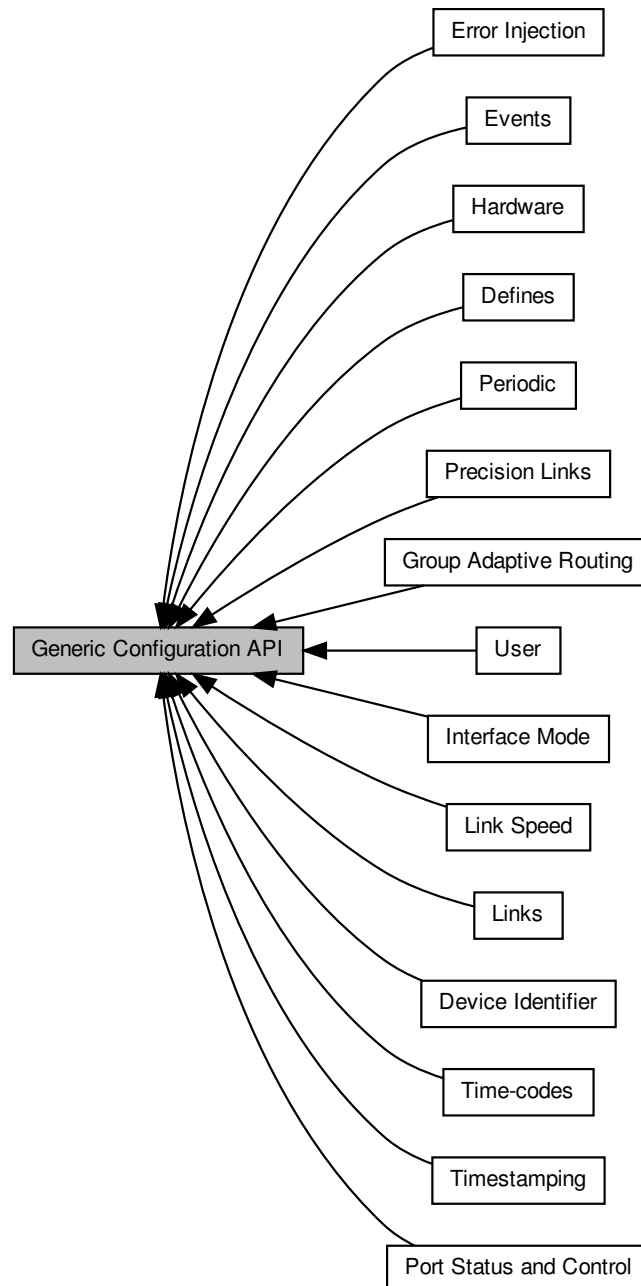
**Examples:**

packet\_subsystem\_example.c.

## 7.38 Generic Configuration API

The Generic Configuration API provides functions for configuring all features on all devices.

Collaboration diagram for Generic Configuration API:



### Modules

- User

*Functions in this group are user functions.*

- **Hardware**

*Functions in this group are hardware related functions.*

- **Port Status and Control**

*Functions in this group deal with port status and control.*

- **Device Identifier**

*Functions in this group deal with device identification.*

- **Group Adaptive Routing**

*Functions in this group deal with group adaptive routing.*

- **Interface Mode**

*Functions in this group deal with interface mode.*

- **Time-codes**

*Functions in this group deal with time-codes.*

- **Events**

*Functions in this group deal with events.*

- **Link Speed**

*Functions in this group deal with measuring link speed.*

- **Links**

*Functions in this group deal with controlling link speed.*

- **Precision Links**

*Functions in this group deal with controlling link speed with greater precision.*

- **Error Injection**

*Functions in this group deal with error injection.*

- **Timestamping**

*Functions in this group deal with timestampings.*

- **Periodic**

*Functions in this group deal with periodic functions.*

- **Defines**

*Function return error values are defined here.*

### 7.38.1 Detailed Description

The Generic Configuration API provides functions for configuring all features on all devices.

If a feature is not supported by a device, the API function calls will return a value to indicate this. Please refer to file [cfg\\_api\\_generic.h](#) for the possible return values.

As well as configuring devices connected directly to the PC, the Generic Device Configuration API can also be used to configure devices over a SpaceWire network. This is achieved by creating a remote device identifier (see [Remote Devices](#)) and passing this remote device identifier as the identifier of the device to be configured.

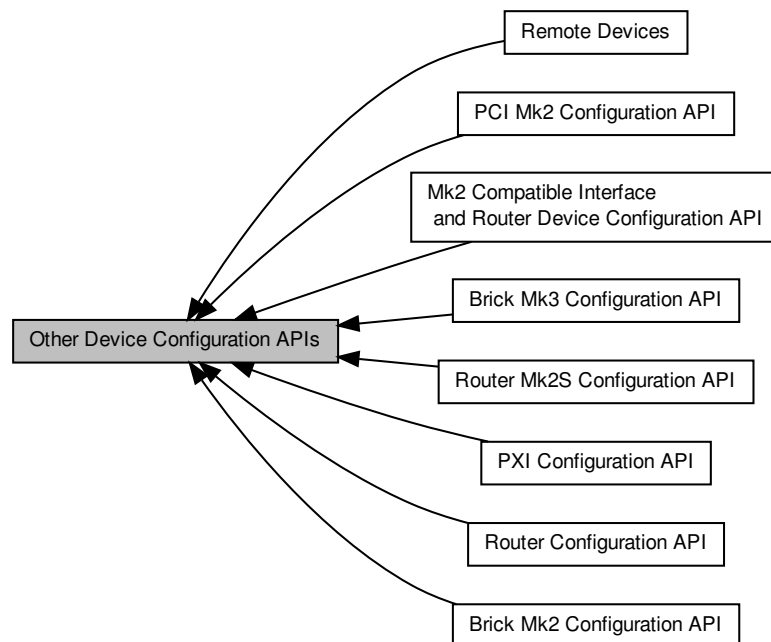
The [Configuration API Functions Supported by Device Types](#) lists the functions supported by each device.



## 7.39 Other Device Configuration APIs

The Device Configuration APIs provide functions for configuring devices.

Collaboration diagram for Other Device Configuration APIs:



### Modules

- **Remote Devices**

*Functions in this group deal with creating and destroying remote device identifiers, and obtaining the properties associated with a remote device identifier.*

- **Router Configuration API**

*The Router Configuration API provides functions for configuring features common to a number of SpaceWire routing devices.*

- **Mk2 Compatible Interface and Router Device Configuration API**

*The Mk2 compatible device API provides functions for configuring features common across the Mk2 range of devices, including the [SpaceWire Brick Mk3](#).*

- **PCI Mk2 Configuration API**

*The PCI Mk2 Configuration API provides functions for configuring features of the [SpaceWire PCI Mk2](#) and [cPCI Mk2](#) devices specific to those devices, and not covered in the [Router Configuration API](#) or [Mk2 Compatible Interface and Router Device Configuration API](#).*

- **Brick Mk2 Configuration API**

*The Brick Mk2 Configuration API provides functions for configuring features of [SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#) and [SpaceWire Brick Mk3](#) devices specific to those devices, and not covered in the [Router Configuration API](#) or [Mk2 Compatible Interface and Router Device Configuration API](#).*

- **Brick Mk3 Configuration API**

*The Brick Mk3 Configuration API provides functions for configuring features of [SpaceWire Brick Mk3](#) devices specific to those devices, and not covered in the [Router Configuration API](#), [Mk2 Compatible Interface and Router Device Configuration API](#) or [Brick Mk2 Configuration API](#).*

- Router Mk2S Configuration API

*The Router Mk2S Configuration API provides functions for configuring features specific to the SpaceWire Router Mk2S, and not covered in the Router Configuration API, Mk2 Compatible Interface and Router Device Configuration API or Brick Mk2 Configuration API.*

- PXI Configuration API

*The PXI Configuration API provides functions for configuring features of SpaceWire PXI Interface and PXI Interface Mk2 and SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2 devices specific to those devices, and not covered in the Router Configuration API, Mk2 Compatible Interface and Router Device Configuration API or Brick Mk2 Configuration API.*

### 7.39.1 Detailed Description

The Device Configuration APIs provide functions for configuring devices.

As well as configuring devices connected directly to the PC, the Device Configuration API can also be used to configure devices over a SpaceWire network. This is achieved by creating a remote device identifier (see [Remote Devices](#)) and passing this remote device identifier as the identifier of the device to be configured.

Note that the [Generic Configuration API](#) should be used for all new developments rather than the original Device Configuration APIs. The Generic Configuration API works with all device types, so different functions do not need called based on the device type, as can be the case with the other Configuration APIs.

The Device Configuration APIs consist of a number of sub-APIs which provide functions for configuring different devices. The [Router Configuration API](#) provides general functions for configuring router devices. Supported devices include the STAR-Dundee [SpaceWire PCI Mk2 and cPCI Mk2](#), [SpaceWire PCIe](#), [SPLT](#), [SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#), [SpaceWire Brick Mk3](#), the SpaceWire Router-USB, the SpaceWire Router Mk2, the SpaceWire-USB Brick and the ESA SpW-10X. Additional APIs then provide functions specific to individual devices. Included is the [Mk2 Compatible Interface and Router Device Configuration API](#) that supports all Mk2 compatible interface and router devices. This includes the [SpaceWire PCIe](#), [SpaceWire PCI Mk2 and cPCI Mk2](#), [SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#), [SpaceWire Brick Mk2](#) and [SpaceWire Brick Mk3](#) devices. Also, the [PCI Mk2 Configuration API](#) is included, which provides additional functions specifically for configuring the link speed of the [SpaceWire PCI Mk2 and cPCI Mk2](#). The [Brick Mk2 Configuration API](#) includes functions for configuring features specific to the [SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#) and [SpaceWire Brick Mk3](#), although note that the [SpaceWire Brick Mk3](#) uses separate base transmit clock functions provided by the [Brick Mk3 Configuration API](#). The [Router Mk2S Configuration API](#) includes functions for configuring the precision transmit rate features of the [SpaceWire Router Mk2S](#). The [PXI Configuration API](#) provides separate functions for configuring base transmit clock and link rate divider for the [SpaceWire PXI Interface and PXI Interface Mk2](#) devices.

The [Configuration API Functions Supported by Device Types](#) lists the functions supported by each device.

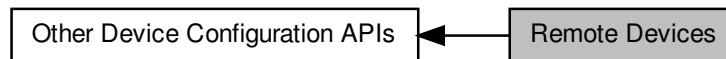
### 7.39.2 Registers

A number of the configuration functions in this API perform an operation on single registers. It is more efficient to read the value of this register once, and then keep this value in memory to get the required information/set required parameters, than to read the register for every operation. Functions that operate on a single register are grouped together in this API.

## 7.40 Remote Devices

Functions in this group deal with creating and destroying remote device identifiers, and obtaining the properties associated with a remote device identifier.

Collaboration diagram for Remote Devices:



### Functions

- `STAR_DEVICE_ID STAR_API_CC STAR_CFG_createRemoteDeviceIdentifier (STAR_DEVICE_ID deviceId, unsigned char localChannel, char *name, STAR_SPACEWIRE_ADDRESS *pathTo, STAR_SPACEWIRE_ADDRESS *returnPath)`  
*Creates a Remote Device Identifier.*
- `int STAR_API_CC STAR_CFG_destroyRemoteDeviceIdentifier (STAR_DEVICE_ID remoteDeviceId)`  
*Destroys a Remote Device Identifier.*
- `_Ret_opt_cap_ count STAR_DEVICE_ID *STAR_API_CC STAR_CFG_getRemoteDeviceDescriptionList (_Out_ U32 *count)`  
*Gets an array of all remote devices that have been created by all processes.*
- `void STAR_API_CC STAR_CFG_destroyRemoteDeviceDescriptionList (_Post_ptr_invalid_ STAR_DEVICE_ID *pRemoteDeviceDescriptionList)`  
*Frees a remote device description list previously created by a call to `STAR_CFG_getRemoteDeviceDescriptionList()`.*
- `int STAR_API_CC STAR_CFG_getRemoteDeviceLocalChannel (STAR_DEVICE_ID remoteDeviceId, unsigned char *localChannel)`  
*Gets the number of the local device channel used to communicate with the remote device on.*
- `int STAR_API_CC STAR_CFG_getRemoteDeviceLocalDevice (STAR_DEVICE_ID remoteDeviceId, STAR_DEVICE_ID *localDeviceId)`  
*Gets the local device used to communicate with the remote device on.*
- `int STAR_API_CC STAR_CFG_getRemoteDevicePath (STAR_DEVICE_ID remoteDeviceId, _Out_opt_ STAR_SPACEWIRE_ADDRESS **pPath)`  
*Gets the path to the specified remote device.*
- `int STAR_API_CC STAR_CFG_getRemoteDeviceRetPath (STAR_DEVICE_ID remoteDeviceId, _Out_opt_ STAR_SPACEWIRE_ADDRESS **pRetPath)`  
*Gets the return path from to the specified remote device.*
- `_Check_return_ _Ret_opt_z_ char *STAR_API_CC STAR_CFG_getRemoteDeviceDescriptionNameString (STAR_DEVICE_ID deviceId)`  
*Gets the name of the remote device description identified by a given identifier.*

### 7.40.1 Detailed Description

Functions in this group deal with creating and destroying remote device identifiers, and obtaining the properties associated with a remote device identifier.

A remote device identifier can only be used to configure a device using the Configuration APIs.

## 7.40.2 Function Documentation

### 7.40.2.1 **STAR\_DEVICE\_ID STAR\_API\_CC STAR\_CFG\_createRemoteDeviceIdentifier ( STAR\_DEVICE\_ID *deviceld*, unsigned char *localChannel*, char \* *name*, STAR\_SPACEWIRE\_ADDRESS \* *pathTo*, STAR\_SPACEWIRE\_ADDRESS \* *returnPath* )**

Creates a Remote Device Identifier.

Remote device Identifiers are used to describe to the Configuration API how to reach a remote device.

#### Parameters

|                     |                                                                                                                                                                                                                                                                                                                                                                                      |
|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>deviceld</i>     | Local Device communication with the remote device is initiated on.                                                                                                                                                                                                                                                                                                                   |
| <i>localChannel</i> | The number of the channel on the local device on which communication with the remote device is made. This channel cannot be used for any other purpose when a Device Configuration operation is being performed.                                                                                                                                                                     |
| <i>name</i>         | The name to be used to identify the remote device                                                                                                                                                                                                                                                                                                                                    |
| <i>pathTo</i>       | Path to the remote device. This should be the path to the remote device's configuration port (port 0), including a default logical address. This means that the path will normally end with "0 254".                                                                                                                                                                                 |
| <i>returnPath</i>   | Return path from the remote device to reach the localChannel. Configuration response packets are automatically sent out of the port on which the command is received, so this port number is not required. The path should also be terminated with a default logical address. This means that <i>returnPath</i> is normally one byte shorter than <i>pathTo</i> and ends with "254". |

#### Returns

Device ID for the remote device.

#### Added in version:

STAR-System v1.0 - 17th October 2011

#### Examples:

`link_events.c`.

### 7.40.2.2 **void STAR\_API\_CC STAR\_CFG\_destroyRemoteDeviceDescriptionList ( \_Post\_ptr\_invalid\_ STAR\_DEVICE\_ID \* *pRemoteDeviceDescriptionList* )**

Frees a remote device description list previously created by a call to `STAR_CFG_getRemoteDeviceDescriptionList()`.

#### Parameters

|                                                 |                       |
|-------------------------------------------------|-----------------------|
| <i>pRemote↔<br/>Device↔<br/>DescriptionList</i> | The list to be freed. |
|-------------------------------------------------|-----------------------|

#### Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

#### Declaration changed in version:

STAR-System v2.0 Beta 3 - 12th April 2013

### 7.40.2.3 **int STAR\_API\_CC STAR\_CFG\_destroyRemoteDeviceIdentifier ( STAR\_DEVICE\_ID *remoteDeviceld* )**

Destroys a Remote Device Identifier.

**Parameters**


---

|                       |                                                             |
|-----------------------|-------------------------------------------------------------|
| <i>remoteDeviceId</i> | The device identifier of the remote device to be destroyed. |
|-----------------------|-------------------------------------------------------------|

---

**Returns**

Whether the remote device identifier was successfully destroyed.

**Added in version:**

STAR-System v1.0 - 17th October 2011

**Examples:**

`link_events.c.`

#### 7.40.2.4 `_Ret_opt_cap_count STAR_DEVICE_ID* STAR_API_CC STAR_CFG_getRemoteDeviceDescriptionList ( _Out_ U32 * count )`

Gets an array of all remote devices that have been created by all processes.

**Parameters**


---

|            |              |                                                                                                              |
|------------|--------------|--------------------------------------------------------------------------------------------------------------|
| <i>out</i> | <i>count</i> | Pointer to user allocated variable that will be updated to contain the count of items in the returned array. |
|------------|--------------|--------------------------------------------------------------------------------------------------------------|

---

**Returns**

Array of remote device description identifiers.

**Note**

This function returns a snapshot of the current state of the system and is not automatically updated. This array must be freed using `STAR_CFG_destroyRemoteDeviceDescriptionList()` when no longer required.

**Added in version:**

STAR-System v1.0 - 17th October 2011

**Declaration changed in version:**

STAR-System v2.0 Beta 1 - 8th February 2013

#### 7.40.2.5 `_Check_return_ _Ret_opt_z_char* STAR_API_CC STAR_CFG_getRemoteDeviceDescriptionNameString ( STAR_DEVICE_ID deviceId )`

Gets the name of the remote device description identified by a given identifier.

The string returned should be freed by calling `STAR_destroyString()`.

**Parameters**


---

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceId</i> | The remote device description to check. |
|-----------------|-----------------------------------------|

---

**Returns**

Device name as a null terminated string.

**Added in version:**

STAR-System v2.0 Beta 1 - 8th February 2013

7.40.2.6 `int STAR_API_CC STAR_CFG_getRemoteDeviceLocalChannel ( STAR_DEVICE_ID remoteDeviceId, unsigned char * localChannel )`

Gets the number of the local device channel used to communicate with the remote device on.

**Parameters**

|     |                       |                                                    |
|-----|-----------------------|----------------------------------------------------|
|     | <i>remoteDeviceId</i> | Remote device to get information for.              |
| out | <i>localChannel</i>   | Value that will be updated with the local channel. |

**Returns**

1 on success, else 0.

**Added in version:**

STAR-System v1.0 - 17th October 2011

**7.40.2.7** `int STAR_API_CC STAR_CFG_getRemoteDeviceLocalDevice ( STAR_DEVICE_ID remoteDeviceId, STAR_DEVICE_ID * localDeviceID )`

Gets the local device used to communicate with the remote device on.

**Parameters**

|     |                       |                                                      |
|-----|-----------------------|------------------------------------------------------|
|     | <i>remoteDeviceId</i> | Remote device to get information for.                |
| out | <i>localDeviceID</i>  | Value that will be updated with the local device ID. |

**Returns**

1 on success, else 0.

**Added in version:**

STAR-System v1.0 - 17th October 2011

**7.40.2.8** `int STAR_API_CC STAR_CFG_getRemoteDevicePath ( STAR_DEVICE_ID remoteDeviceId, _Out_opt_ STAR_SPACEWIRE_ADDRESS ** pPath )`

Gets the path to the specified remote device.

The address should be destroyed when it is no longer required using `STAR_destroyAddress()`.

**Parameters**

|     |                       |                                                         |
|-----|-----------------------|---------------------------------------------------------|
|     | <i>remoteDeviceId</i> | Remote device to get information for.                   |
| out | <i>pPath</i>          | Value that will be updated with pointer to the address. |

**Returns**

1 on success, else 0.

**Added in version:**

STAR-System v1.0 - 17th October 2011

**7.40.2.9** `int STAR_API_CC STAR_CFG_getRemoteDeviceRetPath ( STAR_DEVICE_ID remoteDeviceId, _Out_opt_ STAR_SPACEWIRE_ADDRESS ** pRetPath )`

Gets the return path from to the specified remote device.

The address should be destroyed when it is no longer required using `STAR_destroyAddress()`.

**Parameters**

---

|     |                       |                                                         |
|-----|-----------------------|---------------------------------------------------------|
|     | <i>remoteDeviceId</i> | Remote device to get information for.                   |
| out | <i>pRetPath</i>       | Value that will be updated with pointer to the address. |

---

**Returns**

1 on success, else 0

**Added in version:**

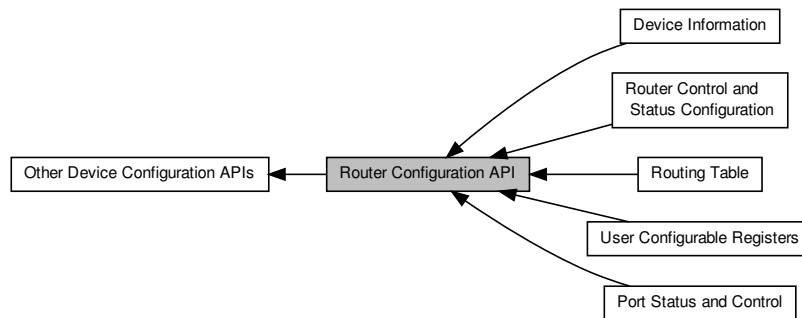
STAR-System v1.0 - 17th October 2011



## 7.41 Router Configuration API

The Router Configuration API provides functions for configuring features common to a number of SpaceWire routing devices.

Collaboration diagram for Router Configuration API:



### Modules

- **Device Information**

*Functions in this group deal with discovery of device information, such as device type, number of ports/links, and manufacturer information.*

- **Port Status and Control**

*Functions in this group deal with the management of the configuration port, link ports, and external ports.*

- **Routing Table**

*Functions in this group deal with the management of a device's routing table.*

- **User Configurable Registers**

*Functions in this group deal with reading and writing to the user configurable registers.*

- **Router Control and Status Configuration**

*Functions in this group deal with configuring the router's internal state, such as timeout periods self addressing, time code forwarding etc.*

### 7.41.1 Detailed Description

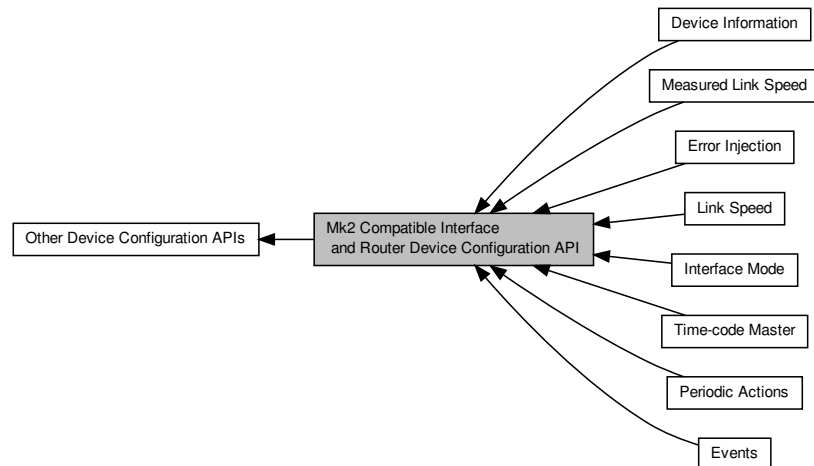
The Router Configuration API provides functions for configuring features common to a number of SpaceWire routing devices.

The devices directly supported by the Router Configuration API include the [SpaceWire PCI Mk2](#) and [cPCI Mk2](#), the [SpaceWire PCle](#), the [SpaceWire Brick Mk2](#), the [SpaceWire Router Mk2S](#), the [SpaceWire Brick Mk3](#), the [SpaceWire Router-USB](#), the [SpaceWire Router Mk2](#), the [SpaceWire-USB Brick](#) and the [ESA SpW-10X](#).

## 7.42 Mk2 Compatible Interface and Router Device Configuration API

The Mk2 compatible device API provides functions for configuring features common across the Mk2 range of devices, including the [SpaceWire Brick Mk3](#).

Collaboration diagram for Mk2 Compatible Interface and Router Device Configuration API:



### Modules

- [Events](#)

*Functions in this group deal with enabling and disabling link events.*

- [Link Speed](#)

*Functions in this group deal with setting the link speeds on the device.*

- [Measured Link Speed](#)

*Functions in this group deal with reading the measured link speed.*

- [Time-code Master](#)

*Functions in this group deal with the management of the device as a time-code master.*

- [Device Information](#)

*Functions in this group deal with hardware information, such as obtaining version information, or flashing the LEDs.*

- [Interface Mode](#)

*Functions in this group deal managing the device in Interface mode.*

- [Error Injection](#)

*Functions in this group deal with injecting errors onto SpaceWire links.*

- [Periodic Actions](#)

*Functions in this group deal with controlling periodic actions on SpaceWire links.*

### 7.42.1 Detailed Description

The Mk2 compatible device API provides functions for configuring features common across the Mk2 range of devices, including the [SpaceWire Brick Mk3](#).

## 7.43 PCI Mk2 Configuration API

The PCI Mk2 Configuration API provides functions for configuring features of the [SpaceWire PCI Mk2](#) and [cPCI Mk2](#) devices specific to those devices, and not covered in the [Router Configuration API](#) or [Mk2 Compatible Interface and Router Device Configuration API](#).

Collaboration diagram for PCI Mk2 Configuration API:



### Modules

- [Link Speed](#)

*Functions in this group deal with setting the link speeds on the device.*

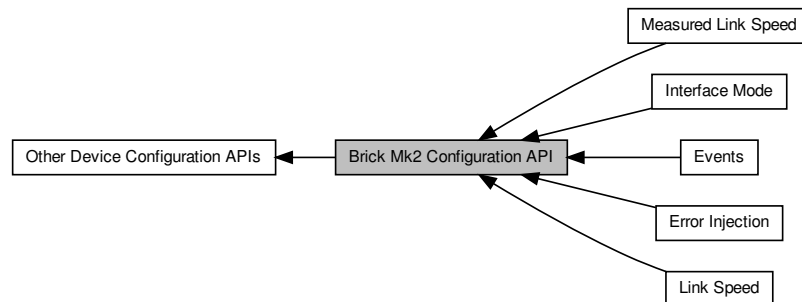
#### 7.43.1 Detailed Description

The PCI Mk2 Configuration API provides functions for configuring features of the [SpaceWire PCI Mk2](#) and [cPCI Mk2](#) devices specific to those devices, and not covered in the [Router Configuration API](#) or [Mk2 Compatible Interface and Router Device Configuration API](#).

## 7.44 Brick Mk2 Configuration API

The Brick Mk2 Configuration API provides functions for configuring features of [SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#) and [SpaceWire Brick Mk3](#) devices specific to those devices, and not covered in the [Router Configuration API](#) or [Mk2 Compatible Interface and Router Device Configuration API](#).

Collaboration diagram for Brick Mk2 Configuration API:



### Modules

- [Link Speed](#)

*Functions in this group deal with setting the link speeds on the device.*

- [Interface Mode](#)

*Functions in this group deal with managing the device in Interface mode.*

- [Error Injection](#)

*Functions in this group deal with error injection.*

- [Events](#)

*Functions in this group deal with enabling and disabling link events.*

- [Measured Link Speed](#)

*Functions in this group deal with reading the measured link speed.*

### 7.44.1 Detailed Description

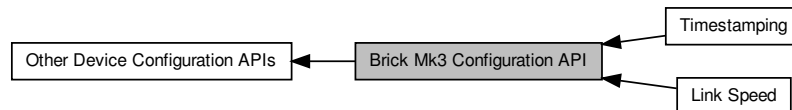
The Brick Mk2 Configuration API provides functions for configuring features of [SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#) and [SpaceWire Brick Mk3](#) devices specific to those devices, and not covered in the [Router Configuration API](#) or [Mk2 Compatible Interface and Router Device Configuration API](#).

Note that the [SpaceWire Brick Mk3](#) uses the base transmit clock functions provided by the [Mk2 Compatible Interface and Router Device Configuration API](#).

## 7.45 Brick Mk3 Configuration API

The Brick Mk3 Configuration API provides functions for configuring features of [SpaceWire Brick Mk3](#) devices specific to those devices, and not covered in the [Router Configuration API](#), [Mk2 Compatible Interface and Router Device Configuration API](#) or [Brick Mk2 Configuration API](#).

Collaboration diagram for Brick Mk3 Configuration API:



### Modules

- [Link Speed](#)

*Functions in this group deal with setting the link speeds of Brick Mk3 devices.*

- [Timestamping](#)

*Functions in this group deal with timestamping related functions for the Brick Mk3.*

### 7.45.1 Detailed Description

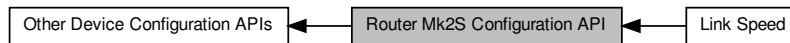
The Brick Mk3 Configuration API provides functions for configuring features of [SpaceWire Brick Mk3](#) devices specific to those devices, and not covered in the [Router Configuration API](#), [Mk2 Compatible Interface and Router Device Configuration API](#) or [Brick Mk2 Configuration API](#).

These functions are for configuring the base transmit clock of Brick Mk3 devices.

## 7.46 Router Mk2S Configuration API

The Router Mk2S Configuration API provides functions for configuring features specific to the [SpaceWire Router Mk2S](#), and not covered in the [Router Configuration API](#), [Mk2 Compatible Interface and Router Device Configuration API](#) or [Brick Mk2 Configuration API](#).

Collaboration diagram for Router Mk2S Configuration API:



### Modules

- [Link Speed](#)

*Functions in this group deal with setting the link speeds on the device.*

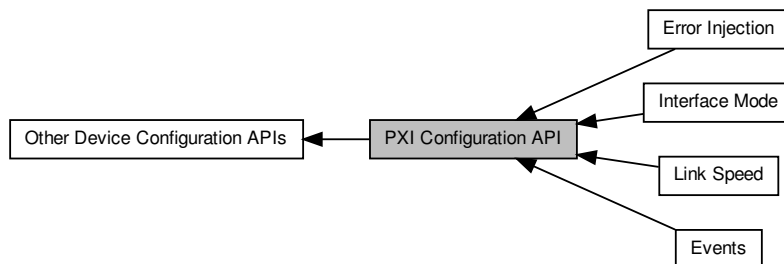
### 7.46.1 Detailed Description

The Router Mk2S Configuration API provides functions for configuring features specific to the [SpaceWire Router Mk2S](#), and not covered in the [Router Configuration API](#), [Mk2 Compatible Interface and Router Device Configuration API](#) or [Brick Mk2 Configuration API](#).

## 7.47 PXI Configuration API

The PXI Configuration API provides functions for configuring features of [SpaceWire PXI Interface](#) and [PXI Interface Mk2](#) and [SpaceWire PXI 12 Port Router](#) and [PXI 12 Port Router Mk2](#) devices specific to those devices, and not covered in the [Router Configuration API](#), [Mk2 Compatible Interface](#) and [Router Device Configuration API](#) or [Brick Mk2 Configuration API](#).

Collaboration diagram for PXI Configuration API:



### Modules

- [Link Speed](#)

*Functions in this group deal with setting the link speeds of PXI devices devices.*

- [Error Injection](#)

*Functions in this group deal with injecting errors onto SpaceWire links.*

- [Interface Mode](#)

*Functions in this group deal with managing the device in Interface mode.*

- [Events](#)

*Functions in this group deal with enabling and disabling link events.*

### 7.47.1 Detailed Description

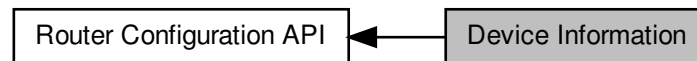
The PXI Configuration API provides functions for configuring features of [SpaceWire PXI Interface](#) and [PXI Interface Mk2](#) and [SpaceWire PXI 12 Port Router](#) and [PXI 12 Port Router Mk2](#) devices specific to those devices, and not covered in the [Router Configuration API](#), [Mk2 Compatible Interface](#) and [Router Device Configuration API](#) or [Brick Mk2 Configuration API](#).

These functions are for configuring the base transmit clock and link rate dividers of PXI devices.

## 7.48 Device Information

Functions in this group deal with discovery of device information, such as device type, number of ports/links, and manufacturer information.

Collaboration diagram for Device Information:



### Data Structures

- struct `STAR_CFG_DEVICE_IDENTIFIER_INFO`  
Information obtained from a Device's Identifier Register, identifying the router's version and type. [More...](#)
- struct `STAR_CFG_NETWORK_DISCOVERY_INFO`  
Information obtained from a device's network discovery register. [More...](#)

### Macros

- `#define STAR_CFG_MANUFACTURER_STR_MAX_LEN 256`  
Maximum length for the string representation of a manufacturer name.
- `#define STAR_CFG_DEVICE_STR_MAX_LEN 256`  
Maximum length for the string representation of a device type.

### Enumerations

- enum `STAR_CFG_DEVICE_TYPE` { `STAR_CFG_DEVICE_TYPE_ROUTER`, `STAR_CFG_DEVICE_TYPE_UNKNOWN`, `STAR_CFG_DEVICE_TYPE_INVALID` }  
Device types permitted within the network discovery register.

### Functions

- `int STAR_API_CC CFG_ROUTER_getDeviceIdentificationInfo (STAR_DEVICE_ID deviceId, _Out_ STAR_CFG_DEVICE_IDENTIFIER_INFO *pInfo)`  
Reads the device identifier information of a device, i.e.
- `void STAR_API_CC CFG_ROUTER_getDeviceManufacturerAsString (STAR_CFG_DEVICE_IDENTIFIER_INFO *pDeviceIdentifierInfo, _Out_z_cap_c_(STAR_CFG_MANUFACTURER_STR_MAX_LEN) char *manufacturerStr)`  
If the manufacturer is known, this function obtains a string representation of the manufacturers name.
- `void STAR_API_CC CFG_ROUTER_getDeviceTypeAsString (STAR_CFG_DEVICE_IDENTIFIER_INFO *pDeviceIdentifierInfo, _Out_z_cap_c_(STAR_CFG_DEVICE_STR_MAX_LEN) char *deviceTypeStr)`  
If the manufacturer and device type is known, this function obtains a string representation of the device's type.
- `int STAR_API_CC CFG_ROUTER_getNetworkDiscoveryInfo (STAR_DEVICE_ID deviceId, _Out_ STAR_CFG_NETWORK_DISCOVERY_INFO *pInfo)`  
Reads the network discovery information of a device.



### 7.48.1 Detailed Description

Functions in this group deal with discovery of device information, such as device type, number of ports/links, and manufacturer information.

### 7.48.2 Data Structure Documentation

#### 7.48.2.1 struct STAR\_CFG\_DEVICE\_IDENTIFIER\_INFO

Information obtained from a Device's Identifier Register, identifying the router's version and type.

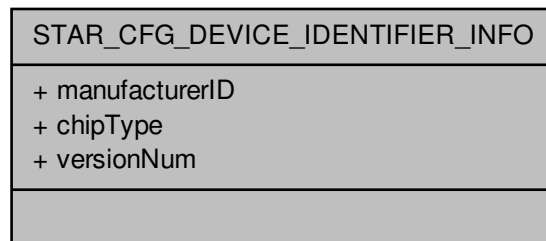
Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

`device_identifier_example.c`, and `star-test.c`.

Collaboration diagram for STAR\_CFG\_DEVICE\_IDENTIFIER\_INFO:



#### Data Fields

|     |                |                                                                        |
|-----|----------------|------------------------------------------------------------------------|
| U8  | chipType       | Identity code for the SpaceWire chip from the particular manufacturer. |
| U16 | manufacturerID | Manufacturer ID number (STAR-Dundee: 1)                                |
| U8  | versionNum     | Device version number.                                                 |

#### 7.48.2.2 struct STAR\_CFG\_NETWORK\_DISCOVERY\_INFO

Information obtained from a device's network discovery register.

Information that can be used to determine the network layout.

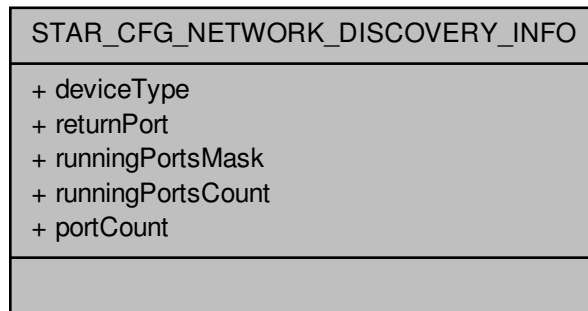
Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

`device_identifier_example.c`.

Collaboration diagram for STAR\_CFG\_NETWORK\_DISCOVERY\_INFO:



#### Data Fields

|                       |                        |                                                                                                 |
|-----------------------|------------------------|-------------------------------------------------------------------------------------------------|
| STAR_CFG_DEVICE_TYPE↔ | deviceType             | Type of device.                                                                                 |
| U8                    | portCount              | Count of the number of ports the device has.                                                    |
| U8                    | returnPort             | Indicates the input port number which was used to access the network discovery register.        |
| U8                    | runningPorts↔<br>Count | Count of running ports.                                                                         |
| U32                   | runningPorts↔<br>Mask  | Bitmask of ports which are in the run state.<br><br><b>Note</b><br>Bit 1 corresponds to port 1. |

### 7.48.3 Macro Definition Documentation

#### 7.48.3.1 #define STAR\_CFG\_DEVICE\_STR\_MAX\_LEN 256

Maximum length for the string representation of a device type.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

device\_identifier\_example.c, and star-test.c.

#### 7.48.3.2 #define STAR\_CFG\_MANUFACTURER\_STR\_MAX\_LEN 256

Maximum length for the string representation of a manufacturer name.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

`device_identifier_example.c`, and `star-test.c`.

**7.48.4 Enumeration Type Documentation****7.48.4.1 enum STAR\_CFG\_DEVICE\_TYPE**

Device types permitted within the network discovery register.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Enumerator**

**STAR\_CFG\_DEVICE\_TYPE\_ROUTER** Router device.

**STAR\_CFG\_DEVICE\_TYPE\_UNKNOWN** Unknown device type.

**STAR\_CFG\_DEVICE\_TYPE\_INVALID** Invalid device type.

**7.48.5 Function Documentation****7.48.5.1 int STAR\_API\_CC\_CFG\_ROUTER\_getDeviceIdentificationInfo ( STAR\_DEVICE\_ID *deviceId*, \_Out\_ STAR\_CFG\_DEVICE\_IDENTIFIER\_INFO \* *pInfo* )**

Reads the device identifier information of a device, i.e.  
the Manufacturer ID, Chip type and version.

**Parameters**

|     | <i>deviceId</i> | Device to obtain info from.                                                              |
|-----|-----------------|------------------------------------------------------------------------------------------|
| out | <i>pInfo</i>    | Structure that will be updated to contain the Identification information for the device. |

**Returns**

1 if device identifier info was successfully read, else 0.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S, SpaceWire Physical Layer Tester

**7.48.5.2 void STAR\_API\_CC\_CFG\_ROUTER\_getDeviceManufacturerAsString ( STAR\_CFG\_DEVICE\_IDENTIFIER\_INFO \* *pDeviceIdentifierInfo*, \_Out\_z\_cap\_c\_ (STAR\_CFG\_MANUFACTURER\_STR\_MAX\_LEN) char \* *manufacturerStr* )**

If the manufacturer is known, this function obtains a string representation of the manufacturers name.

**Parameters**

|     |                              |                                                                                                                                                                |
|-----|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| in  | <i>pDeviceIdentifierInfo</i> | Pointer to device identification information obtained from a call to <code>CFG_ROUTER_getDeviceIdentificationInfo()</code> .                                   |
| out | <i>manufacturerStr</i>       | User allocated zero terminated string of max length <code>STAR_CFG_MANUFACTURER_STR_MAX_LEN</code> that will be updated with the manufacturers name, if known. |

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S, SpaceWire Physical Layer Tester

**7.48.5.3** `void STAR_API_CC CFG_ROUTER_getDeviceTypeAsString ( STAR_CFG_DEVICE_IDENTIFIER_INFO * pDeviceIdentifierInfo, _Out_z_cap_c_(STAR_CFG_DEVICE_STR_MAX_LEN) char * deviceTypeStr )`

If the manufacturer and device type is known, this function obtains a string representation of the device's type.

**Parameters**

|     |                              |                                                                                                                                                   |
|-----|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| in  | <i>pDeviceIdentifierInfo</i> | Pointer to device identification information obtained from a call to <code>CFG_ROUTER_getDeviceIdentificationInfo()</code> .                      |
| out | <i>deviceTypeStr</i>         | User allocated zero terminated string of max length <code>STAR_CFG_DEVICE_STR_MAX_LEN</code> that will be updated with the device name, if known. |

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S, SpaceWire Physical Layer Tester

**7.48.5.4** `int STAR_API_CC CFG_ROUTER_getNetworkDiscoveryInfo ( STAR_DEVICE_ID deviceId, _Out_ STAR_CFG_NETWORK_DISCOVERY_INFO * pInfo )`

Reads the network discovery information of a device.

This information can be used by a network manager to determine the layout of the network.

**Parameters**

|     |                 |                                                                                             |
|-----|-----------------|---------------------------------------------------------------------------------------------|
|     | <i>deviceId</i> | Device to obtain info from.                                                                 |
| out | <i>pInfo</i>    | Structure that will be updated to contain the network discovery information for the device. |

**Returns**

1 if device identifier info was successfully read, else 0.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

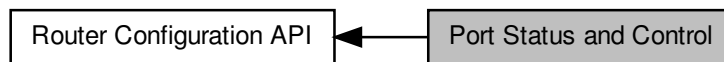
**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S, SpaceWire Physical Layer Tester

## 7.49 Port Status and Control

Functions in this group deal with the management of the configuration port, link ports, and external ports.

Collaboration diagram for Port Status and Control:



### Data Structures

- struct [STAR\\_CFG\\_CONFIG\\_PORT\\_ERRORS](#)  
*Errors that may be present on a device's configuration port. [More...](#)*
- struct [STAR\\_CFG\\_SPW\\_LINK\\_ERRORS](#)  
*Errors that may be present on a SpaceWire Link. [More...](#)*
- struct [STAR\\_CFG\\_SPW\\_LINK\\_STATUS](#)  
*Status of a SpaceWire link. [More...](#)*
- struct [STAR\\_CFG\\_EXTERNAL\\_PORT\\_ERRORS](#)  
*Errors that may be present on an external port. [More...](#)*
- struct [STAR\\_CFG\\_EXTERNAL\\_PORT\\_STATUS](#)  
*Status of an external port. [More...](#)*

### Typedefs

- typedef [REGISTER\\_PORT\\_STATUS\\_CONTROL](#)  
*The type used to represent a port status and control register value.*

### Enumerations

- enum [STAR\\_CFG\\_PORT\\_TYPE](#) { [STAR\\_CFG\\_PORT\\_TYPE\\_CONFIGURATION](#), [STAR\\_CFG\\_PORT\\_TYPE\\_LINK](#), [STAR\\_CFG\\_PORT\\_TYPE\\_EXTERNAL](#), [STAR\\_CFG\\_PORT\\_TYPE\\_INVALID](#) }  
*Port types for Routing devices.*
- enum [STAR\\_CFG\\_SPW\\_LINK\\_STATE](#) { [STAR\\_CFG\\_SPW\\_LINK\\_STATE\\_ERROR\\_RESET](#), [STAR\\_CFG\\_SPW\\_LINK\\_STATE\\_ERROR\\_WAIT](#), [STAR\\_CFG\\_SPW\\_LINK\\_STATE\\_READY](#), [STAR\\_CFG\\_SPW\\_LINK\\_STATE\\_STARTED](#), [STAR\\_CFG\\_SPW\\_LINK\\_STATE\\_CONNECTING](#), [STAR\\_CFG\\_SPW\\_LINK\\_STATE\\_RUN](#), [STAR\\_CFG\\_SPW\\_LINK\\_STATE\\_INVALID](#) }  
*The state of the interface state machine in the SpaceWire link.*

### Functions

- int [STAR\\_API\\_CFG\\_ROUTER\\_getPortStatusControl](#) ([STAR\\_DEVICE\\_ID](#) deviceID, U8 portNum, [\\_OUT\\_](#) [PORT\\_STATUS\\_CONTROL](#) \*portStatusControl)  
*Gets the value of a port or link's status/control register, which can then be used by the various functions within the Port Status and Control group.*

- `STAR_CFG_PORT_TYPE STAR_API_CC CFG_ROUTER_getPortType (PORT_STATUS_CONTROL portStatusControl)`  
*Identifies the type of port attributed to a given port number.*
- `U8 STAR_API_CC CFG_ROUTER_getPortConnection (PORT_STATUS_CONTROL portStatusControl)`  
*Identifies the output port to which the source port is currently connected whilst routing is in operation.*
- `int STAR_API_CC CFG_ROUTER_clearPortErrors (STAR_DEVICE_ID deviceId, U8 portNum)`  
*Clears all errors on a given port.*
- `int STAR_API_CC CFG_ROUTER_getConfigPortErrors (PORT_STATUS_CONTROL portStatusControl, _Out_ STAR_CFG_CONFIG_PORT_ERRORS *errors)`  
*Gets any errors present on the config port.*
- `int STAR_API_CC CFG_ROUTER_getSpaceWireLinkErrors (PORT_STATUS_CONTROL linkStatusControl, _Out_ STAR_CFG_SPW_LINK_ERRORS *errors)`  
*Gets any errors present on a SpaceWire link.*
- `int STAR_API_CC CFG_ROUTER_getSpaceWireLinkStatus (PORT_STATUS_CONTROL linkStatusControl, _Out_ STAR_CFG_SPW_LINK_STATUS *status)`  
*Gets the status of a SpaceWire Link.*
- `int STAR_API_CC CFG_ROUTER_setSpaceWireLinkStatus (STAR_DEVICE_ID deviceId, U8 linkNum, STAR_CFG_SPW_LINK_STATUS linkStatus)`  
*Sets the state of a SpaceWire Link.*
- `int STAR_API_CC CFG_ROUTER_startLink (STAR_DEVICE_ID deviceId, U8 linkNum)`  
*Starts a SpaceWire link by setting the start bit, and clearing the disable bit.*
- `int STAR_API_CC CFG_ROUTER_stopLink (STAR_DEVICE_ID deviceId, U8 linkNum)`  
*Stops a SpaceWire link by clearing the start bit, and setting the disable bit.*
- `int STAR_API_CC CFG_ROUTER_getExternalPortErrors (PORT_STATUS_CONTROL portStatusControl, _Out_ STAR_CFG_EXTERNAL_PORT_ERRORS *errors)`  
*Gets any errors present on an external port.*
- `int STAR_API_CC CFG_ROUTER_getExternalPortStatus (PORT_STATUS_CONTROL portStatusControl, _Out_ STAR_CFG_EXTERNAL_PORT_STATUS *status)`  
*Gets the status of an external port.*

### 7.49.1 Detailed Description

Functions in this group deal with the management of the configuration port, link ports, and external ports.

### 7.49.2 Data Structure Documentation

#### 7.49.2.1 struct STAR\_CFG\_CONFIG\_PORT\_ERRORS

Errors that may be present on a device's configuration port.

If a member of this structure is set, then the error it represents is present.

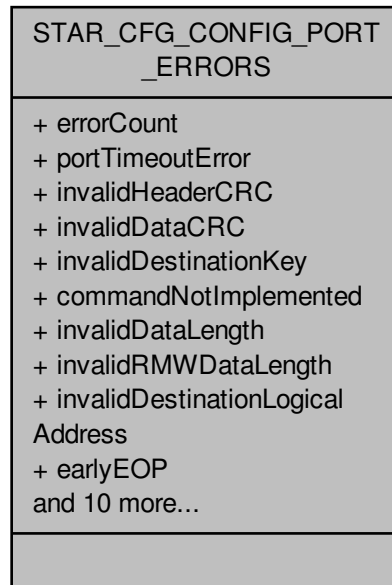
Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

`port_control_status_example.c`.

Collaboration diagram for STAR\_CFG\_CONFIG\_PORT\_ERRORS:



#### Data Fields

|      |                                  |                                                                                                                                                          |
|------|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| char | cargoTooLarge                    | The RMAP command packet is too large.                                                                                                                    |
| char | commandNotImplemented            | The command not implemented bit is set when the command code is a valid RMAP code but the command is not supported by the SpaceWire Router.              |
| char | earlyEEP                         | The early EEP bit is set when the command packet is terminated before the end of packet with an EEP.                                                     |
| char | earlyEOP                         | The early EOP bit is set when the command packet is terminated before the end of packet with an EOP.                                                     |
| char | errorCount                       | Count of errors present on the configuration port.                                                                                                       |
| char | invalidDataCRC                   | The invalid data CRC is set when the data field of the packet is corrupted and the CRC does not match the internally generated CRC.                      |
| char | invalidDataLength                | The invalid data length bit is set when a data length error is detected.                                                                                 |
| char | invalidDestinationKey            | The invalid destination key bit is set when the destination key in the command packet is invalid.                                                        |
| char | invalidDestinationLogicalAddress | The invalid destination logical address bit is set when the destination logical address in the command packet is not the default value of 254.           |
| char | invalidHeaderCRC                 | The Invalid header CRC bit is set when the header CRC is invalid.                                                                                        |
| char | invalidRegisterAddress           | The invalid register address bit is set when an unknown register address is given in the command packet or a write is attempted to a read only register. |



|      |                                         |                                                                                                                                                          |
|------|-----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| char | invalidRMW↔<br>DataLength               | The read modify write command data length is invalid. The expected length is 8.                                                                          |
| char | lateEEP                                 | The late EEP bit is set when the command packet is not terminated correctly and trailing bytes are detected before the end of packet.                    |
| char | lateEOP                                 | The late EOP bit is set when the command packet is not terminated correctly and trailing bytes are detected before the end of packet.                    |
| char | portTimeout↔<br>Error                   | The port timeout error bit is set when a timeout event is detected by the configuration port routing logic.                                              |
| char | sourceLogical↔<br>AddressError          | The source logical address error bit is set when an invalid source logical address is received.                                                          |
| char | sourcePath↔<br>AddressError             | This bit is set when an invalid source address path is received.<br><br><b>Note</b><br><br>This error is not used for the 10X.                           |
| char | unsupported↔<br>Protocol                | The unsupported protocol error bit is set when a command packet is received with a protocol identifier which is not the RMAP protocol identifier (0x01). |
| char | unusedRMAP↔<br>CommandOr↔<br>PacketType | The command code is an unused command code or the packet type is invalid.                                                                                |
| char | verifyBuffer↔<br>Overrun                | The verify buffer overrun error bit is set when a verified write command is performed and the data length is not 4.                                      |

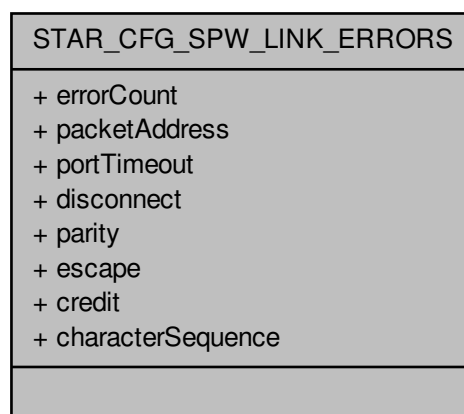
#### 7.49.2.2 struct STAR\_CFG\_SPW\_LINK\_ERRORS

Errors that may be present on a SpaceWire Link.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Collaboration diagram for STAR\_CFG\_SPW\_LINK\_ERRORS:



**Data Fields**

|      |                        |                                                                                                                                        |
|------|------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| char | character↔<br>Sequence | A character sequence error occurred on the link. A time-code or data character was received before the first FCT.                      |
| char | credit                 | A credit error occurred on the link.                                                                                                   |
| char | disconnect             | A disconnect error occurred on the link. No activity occurred on the link for the disconnect timeout period.                           |
| char | errorCount             | Count of errors present on the link port.                                                                                              |
| char | escape                 | An invalid escape code was received on the link. (ESC-ESC, ESC-EOP, or ESC-EEP).                                                       |
| char | packetAddress          | Packet received with an invalid address, either due to an unknown port, or an invalid logical address.                                 |
| char | parity                 | A parity was detected on a character parity bit.                                                                                       |
| char | portTimeout            | The port has become blocked for a period of time. A packet could not be routed to a destination port before the port timeout occurred. |

**7.49.2.3 struct STAR\_CFG\_SPW\_LINK\_STATUS**

Status of a SpaceWire link.

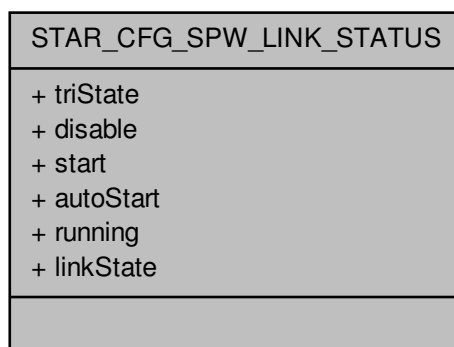
Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

[link\\_events.c](#), [port\\_control\\_status\\_example.c](#), and [timestamp\\_test.c](#).

Collaboration diagram for STAR\_CFG\_SPW\_LINK\_STATUS:

**Data Fields**

|      |           |                                                                                                                                                                                                                              |
|------|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| char | autoStart | When set the SpaceWire link will auto-start as defined in the SpaceWire standard. The SpaceWire port will wait until the other end of the link tries to make a connection (sending NULLs) and will then automatically start. |
|------|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|                                   |           |                                                                                                                                                                                                                  |
|-----------------------------------|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| char                              | disable   | When set then the SpaceWire link will be disabled as defined in the SpaceWire standard. The SpaceWire port will not start and will not respond to any attempt to make a connection by the other end of the link. |
| STAR_CFG_S↔<br>PW_LINK_ST↔<br>ATE | linkState | State of the interface state machine.                                                                                                                                                                            |
| char                              | running   | Set when the SpaceWire interface state machine is in the Run state.                                                                                                                                              |
| char                              | start     | When set then the SpaceWire link will initiate start-up as defined in the SpaceWire standard. The SpaceWire port will try to make a connection with the other end of the link.                                   |
| char                              | triState  | When set, the SpaceWire link LVDS drivers are in tri-state mode.                                                                                                                                                 |

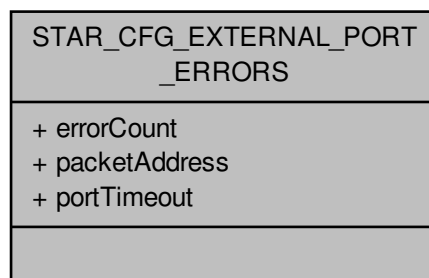
#### 7.49.2.4 struct STAR\_CFG\_EXTERNAL\_PORT\_ERRORS

Errors that may be present on an external port.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Collaboration diagram for STAR\_CFG\_EXTERNAL\_PORT\_ERRORS:



#### Data Fields

|      |               |                                                                                                                                                                                    |
|------|---------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| char | errorCount    | Count of errors present on the external port.                                                                                                                                      |
| char | packetAddress | Packet received with an invalid address, either due to an unknown port, or an invalid logical address. This is also generated when an empty packet is passed to the external port. |
| char | portTimeout   | The port has become blocked for a period of time. A packet could not be routed to a destination port before the port timeout occurred.                                             |

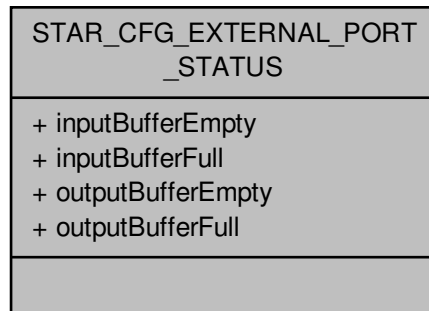
#### 7.49.2.5 struct STAR\_CFG\_EXTERNAL\_PORT\_STATUS

Status of an external port.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Collaboration diagram for STAR\_CFG\_EXTERNAL\_PORT\_STATUS:



#### Data Fields

|      |                        |                                                                                                                                                      |
|------|------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| char | inputBuffer↔<br>Empty  | The external port input buffer is empty.<br><br><b>Note</b><br>The input buffer writes data to the SpaceWire router.                                 |
| char | inputBufferFull        | The external port input buffer is full.                                                                                                              |
| char | outputBuffer↔<br>Empty | The external output port buffer is empty.<br><br><b>Note</b><br>The output buffer writes data to the external device connected to the external port. |
| char | outputBufferFull       | The external output port buffer is full.                                                                                                             |

### 7.49.3 Typedef Documentation

#### 7.49.3.1 typedef REGISTER\_PORT\_STATUS\_CONTROL

The type used to represent a port status and control register value.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

### 7.49.4 Enumeration Type Documentation

#### 7.49.4.1 enum STAR\_CFG\_PORT\_TYPE

Port types for Routing devices.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

## Enumerator

**STAR\_CFG\_PORT\_TYPE\_CONFIGURATION** Configuration port.  
**STAR\_CFG\_PORT\_TYPE\_LINK** SpaceWire Link port.  
**STAR\_CFG\_PORT\_TYPE\_EXTERNAL** External port.  
**STAR\_CFG\_PORT\_TYPE\_INVALID** Invalid value. This should never occur

## 7.49.4.2 enum STAR\_CFG\_SPW\_LINK\_STATE

The state of the interface state machine in the SpaceWire link.

## Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

## Enumerator

**STAR\_CFG\_SPW\_LINK\_STATE\_ERROR\_RESET** Error Reset.  
**STAR\_CFG\_SPW\_LINK\_STATE\_ERROR\_WAIT** Error Wait.  
**STAR\_CFG\_SPW\_LINK\_STATE\_READY** Ready.  
**STAR\_CFG\_SPW\_LINK\_STATE\_STARTED** Started.  
**STAR\_CFG\_SPW\_LINK\_STATE\_CONNECTING** Connecting.  
**STAR\_CFG\_SPW\_LINK\_STATE\_RUN** Run.  
**STAR\_CFG\_SPW\_LINK\_STATE\_INVALID** Invalid value.

## 7.49.5 Function Documentation

## 7.49.5.1 int STAR\_API\_CC\_CFG\_ROUTER\_clearPortErrors ( STAR\_DEVICE\_ID deviceID, U8 portNum )

Clears all errors on a given port.

## Note

Errors on a port are latched until cleared with this function.

## Parameters

|                 |                                      |
|-----------------|--------------------------------------|
| <i>deviceID</i> | Device to clear port errors on.      |
| <i>portNum</i>  | The port to have its errors cleared. |

## Returns

1 if the port port errors were successfully cleared, else 0.

## Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

## Supported devices:

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S, SpaceWire Physical Layer Tester

**7.49.5.2** `int STAR_API_CC CFG_ROUTER_getConfigPortErrors ( PORT_STATUS_CONTROL portStatusControl, Out_STAR_CFG_CONFIG_PORT_ERRORS * errors )`

Gets any errors present on the config port.

These are errors that arise when malformed or invalid configuration commands are sent to the configuration port.

#### Note

Errors on the configuration port are latched until cleared with `CFG_ROUTER_clearPortErrors()`.

#### Parameters

|     |                          |                                                                                                          |
|-----|--------------------------|----------------------------------------------------------------------------------------------------------|
|     | <i>portStatusControl</i> | Port Status/Control value obtained from a call to <code>CFG_ROUTER_getPortStatusControl()</code> .       |
| out | <i>errors</i>            | User provided error structure that will be updated to show any errors present on the configuration port. |

#### Returns

0 if the portStatusControl value passed in is not for a configuration port, otherwise 1.

#### Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

#### Supported devices:

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S, SpaceWire Physical Layer Tester

**7.49.5.3** `int STAR_API_CC CFG_ROUTER_getExternalPortErrors ( PORT_STATUS_CONTROL portStatusControl, Out_STAR_CFG_EXTERNAL_PORT_ERRORS * errors )`

Gets any errors present on an external port.

#### Note

Errors on a port are latched until cleared with `CFG_ROUTER_clearPortErrors()`.

#### Parameters

|     |                          |                                                                                                     |
|-----|--------------------------|-----------------------------------------------------------------------------------------------------|
|     | <i>portStatusControl</i> | Port Status/Control value obtained from a call to <code>CFG_ROUTER_getPortStatusControl()</code> .  |
| out | <i>errors</i>            | User provided error structure that will be updated to show any errors present on the external port. |

#### Returns

0 if the portStatusControl value passed in is not for an external port, otherwise 1.

#### Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

#### Supported devices:

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S, SpaceWire Physical Layer Tester

7.49.5.4 `int STAR_API_CC_CFG_ROUTER_getExternalPortStatus ( PORT_STATUS_CONTROL portStatusControl,  
_Out_ STAR_CFG_EXTERNAL_PORT_STATUS * status )`

Gets the status of an external port.

**Parameters**

|            |                                |                                                                                                          |
|------------|--------------------------------|----------------------------------------------------------------------------------------------------------|
|            | <i>portStatus↔<br/>Control</i> | Port Status/Control value obtained from a call to <code>CFG_ROUTER_getPort↔<br/>StatusControl()</code> . |
| <i>out</i> | <i>status</i>                  | User provided error structure that will be updated to show the status of the external port.              |

**Returns**

0 if the portStatusControl value passed in is not for an external port, otherwise 1.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S, SpaceWire Physical Layer Tester

#### 7.49.5.5 U8 STAR\_API\_CC CFG\_ROUTER\_getPortConnection ( PORT\_STATUS\_CONTROL portStatusControl )

Identifies the output port to which the source port is currently connected whilst routing is in operation.

**Parameters**

|                                |                                                                                                    |
|--------------------------------|----------------------------------------------------------------------------------------------------|
| <i>portStatus↔<br/>Control</i> | Port Status/Control value obtained from a call to <code>CFG_ROUTER_getPortStatusControl()</code> . |
|--------------------------------|----------------------------------------------------------------------------------------------------|

**Returns**

Output port connected. This will have a value of 31 if there is no current port connected.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S, SpaceWire Physical Layer Tester

#### 7.49.5.6 int STAR\_API\_CC CFG\_ROUTER\_getPortStatusControl ( STAR\_DEVICE\_ID deviceID, U8 portNum, \_Out\_ PORT\_STATUS\_CONTROL \* portStatusControl )

Gets the value of a port or link's status/control register, which can then be used by the various functions within the Port Status and Control group.

**Parameters**

|                 |                                           |
|-----------------|-------------------------------------------|
| <i>deviceID</i> | Device to obtain info from.               |
| <i>portNum</i>  | The port number to determine the type of. |



|                  |                                      |                                                                |
|------------------|--------------------------------------|----------------------------------------------------------------|
| <code>out</code> | <code>portStatus↔<br/>Control</code> | Value that will be updated with the ports status/control value |
|------------------|--------------------------------------|----------------------------------------------------------------|

**Returns**

1 if device port type info was successfully read, else 0.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S, SpaceWire Physical Layer Tester

#### 7.49.5.7 **STAR\_CFG\_PORT\_TYPE** **STAR\_API\_CC** **CFG\_ROUTER\_getPortType** ( **PORT\_STATUS\_CONTROL** *portStatusControl* )

Identifies the type of port attributed to a given port number.

**Parameters**

|                                      |                                                                                                    |
|--------------------------------------|----------------------------------------------------------------------------------------------------|
| <code>portStatus↔<br/>Control</code> | Port Status/Control value obtained from a call to <code>CFG_ROUTER_getPortStatusControl()</code> . |
|--------------------------------------|----------------------------------------------------------------------------------------------------|

**Returns**

Type of the port

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S, SpaceWire Physical Layer Tester

#### 7.49.5.8 **int** **STAR\_API\_CC** **CFG\_ROUTER\_getSpaceWireLinkErrors** ( **PORT\_STATUS\_CONTROL** *linkStatusControl*, **\_Out\_STAR\_CFG\_SPW\_LINK\_ERRORS** \* *errors* )

Gets any errors present on a SpaceWire link.

**Note**

Errors on a port are latched until cleared with `CFG_ROUTER_clearPortErrors()`.

**Parameters**

|                          |                                                                                                     |
|--------------------------|-----------------------------------------------------------------------------------------------------|
| <i>linkStatusControl</i> | Port Status/Control value obtained from a call to <code>CFG_ROUTER_getPort↔StatusControl()</code> . |
|--------------------------|-----------------------------------------------------------------------------------------------------|

|            |               |                                                                                                      |
|------------|---------------|------------------------------------------------------------------------------------------------------|
| <i>out</i> | <i>errors</i> | User provided error structure that will be updated to show any errors present on the SpaceWire link. |
|------------|---------------|------------------------------------------------------------------------------------------------------|

#### Returns

0 if the portStatusControl value passed in is not for a SpaceWire link, otherwise 1.

#### Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

#### Supported devices:

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S, SpaceWire Physical Layer Tester

**7.49.5.9** `int STAR_API_CC CFG_ROUTER_getSpaceWireLinkStatus ( PORT_STATUS_CONTROL linkStatusControl, _Out_ STAR_CFG_SPW_LINK_STATUS * status )`

Gets the status of a SpaceWire Link.

#### Parameters

|            |                          |                                                                                                     |
|------------|--------------------------|-----------------------------------------------------------------------------------------------------|
|            | <i>linkStatusControl</i> | Port Status/Control value obtained from a call to <code>CFG_ROUTER_getPort↵StatusControl()</code> . |
| <i>out</i> | <i>status</i>            | User provided error structure that will be updated to show the status of the the SpaceWire link.    |

#### Returns

0 if the portStatusControl value passed in is not for a SpaceWire link port, otherwise 1.

#### Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

#### Supported devices:

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S, SpaceWire Physical Layer Tester

**7.49.5.10** `int STAR_API_CC CFG_ROUTER_setSpaceWireLinkStatus ( STAR_DEVICE_ID deviceID, U8 linkNum, STAR_CFG_SPW_LINK_STATUS linkStatus )`

Sets the state of a SpaceWire Link.

#### Parameters

|  |                   |                                                                                   |
|--|-------------------|-----------------------------------------------------------------------------------|
|  | <i>deviceID</i>   | Device to have its link state set.                                                |
|  | <i>linkNum</i>    | The link to set state for.                                                        |
|  | <i>linkStatus</i> | The values within this structure are used to set the state of the SpaceWire Link. |

#### Note

The linkState member of this structure is ignored by this function.

**Returns**

1 if the link state was successfully set, else 0.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S, SpaceWire Physical Layer Tester

**7.49.5.11 int STAR\_API\_CC\_CFG\_ROUTER\_startLink ( STAR\_DEVICE\_ID *deviceID*, U8 *linkNum* )**

Starts a SpaceWire link by setting the start bit, and clearing the disable bit.

**Parameters**

|                 |                                  |
|-----------------|----------------------------------|
| <i>deviceID</i> | Device to have its link started. |
| <i>linkNum</i>  | The link to start.               |

**Returns**

1 if the link was successfully started, else 0.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S, SpaceWire Physical Layer Tester

**7.49.5.12 int STAR\_API\_CC\_CFG\_ROUTER\_stopLink ( STAR\_DEVICE\_ID *deviceID*, U8 *linkNum* )**

Stops a SpaceWire link by clearing the start bit, and setting the disable bit.

**Note**

If auto-start is enabled the link will start again as soon as it receives NULLs.

**Parameters**

|                 |                                  |
|-----------------|----------------------------------|
| <i>deviceID</i> | Device to have its link stopped. |
| <i>linkNum</i>  | The link to stop.                |

**Returns**

1 if the link was successfully stopped, else 0.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S, SpaceWire Physical Layer Tester

## 7.50 Routing Table

Functions in this group deal with the management of a device's routing table.

Collaboration diagram for Routing Table:



### Data Structures

- struct `STAR_CFG_GAR_ENTRY`

*A Routing table entry. [More...](#)*

### Functions

- int `STAR_API_CC_CFG_ROUTER_getRoutingTableEntry` (`STAR_DEVICE_ID` deviceID, U8 logicalAddress, `_Out_ STAR_CFG_GAR_ENTRY *`entry)  
*Gets a routing table entry for a given logical address.*
- int `STAR_API_CC_CFG_ROUTER_setRoutingTableEntry` (`STAR_DEVICE_ID` deviceID, U8 logicalAddress, `STAR_CFG_GAR_ENTRY` entry)  
*Sets a routing table entry for a given logical address.*

#### 7.50.1 Detailed Description

Functions in this group deal with the management of a device's routing table.

This is the table that maps Logical Addresses to one or more physical ports. There is one entry in the routing table for each possible Logical Address.

#### Warning

Care must be taken to avoid infinite loops when setting up routing tables. If, for example, there is a SpaceWire link made between two ports of the router, and a logical address routes a packet out of one of these ports, it will arrive back at the other, and then be routed out again continually, forever. If the packet is large enough it may block instead.

#### 7.50.2 Data Structure Documentation

##### 7.50.2.1 struct `STAR_CFG_GAR_ENTRY`

A Routing table entry.

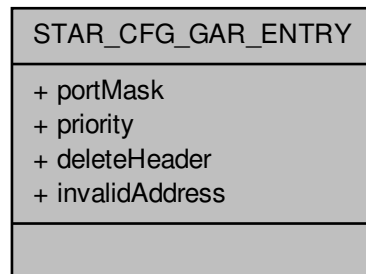
Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

`routing_table_example.c.`

Collaboration diagram for STAR\_CFG\_GAR\_ENTRY:

**Data Fields**

|      |                |                                                                                                                                                                                                                                          |
|------|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| char | deleteHeader   | When set the leading header byte of the input packet will be removed before it is transferred to the output port.                                                                                                                        |
| char | invalidAddress | When set this indicates that the corresponding logical address is invalid. In this case, any packets arriving at the router with an invalid address are spilt and an address error is reported in the port status register.              |
| U32  | portMask       | Bitmask of output ports the logical address will arbitrate for. Valid bits set are 1 through 28.<br><br><b>Note</b><br>Bit 1 corresponds to port 1<br>It is not possible to access the configuration port (0) through logical addresses. |
| char | priority       | Packets with the logical address for this entry with the priority bit set will be granted access to a particular output port in preference to packets whose logical addresses in the routing table have their priority bit set to zero.  |

**7.50.3 Function Documentation**

**7.50.3.1** `int STAR_API_CC CFG_ROUTER_getRoutingTableEntry ( STAR_DEVICE_ID deviceID, U8 logicalAddress, _Out_ STAR_CFG_GAR_ENTRY * entry )`

Gets a routing table entry for a given logical address.

**Parameters**

|                       |                                             |
|-----------------------|---------------------------------------------|
| <i>deviceID</i>       | Device to get GAR table entry for.          |
| <i>logicalAddress</i> | Logical address to get GAR table entry for. |

|            |              |                                                    |
|------------|--------------|----------------------------------------------------|
| <i>out</i> | <i>entry</i> | The GAR table entry for the given logical address. |
|------------|--------------|----------------------------------------------------|

**Returns**

1 if the entry was successfully obtained from the device, else 0.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S, SpaceWire Physical Layer Tester

**7.50.3.2** `int STAR_API_CC_CFG_ROUTER_setRoutingTableEntry ( STAR_DEVICE_ID deviceID, U8 logicalAddress, STAR_CFG_GAR_ENTRY entry )`

Sets a routing table entry for a given logical address.

**Parameters**

|                       |                                                                              |
|-----------------------|------------------------------------------------------------------------------|
| <i>deviceID</i>       | Device to set GAR table entry on.                                            |
| <i>logicalAddress</i> | Logical address to set GAR table entry for. Valid values are 32 through 255. |

**Note**

Logical address 255 is reserved for future use and should not be used. See section 10.3.3n of the SpaceWire standard for more information.

**Parameters**

|              |                                                    |
|--------------|----------------------------------------------------|
| <i>entry</i> | The GAR table entry for the given logical address. |
|--------------|----------------------------------------------------|

**Returns**

1 if the entry was successfully set, else 0.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

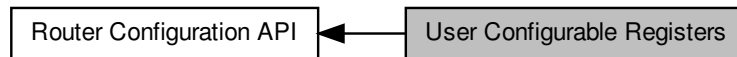
**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S, SpaceWire Physical Layer Tester

## 7.51 User Configurable Registers

Functions in this group deal with reading and writing to the user configurable registers.

Collaboration diagram for User Configurable Registers:



### Functions

- `int STAR_API_CC_CFG_MK2_setTimeCodeDistributionPorts (STAR_DEVICE_ID deviceId, U32 portMask)`  
*This function is used to specify which output ports time-codes are forwarded on.*
- `int STAR_API_CC_CFG_MK2_getTimeCodeDistributionPorts (STAR_DEVICE_ID deviceId, _Out_ U32 *pPortMask)`  
*This function is used to obtain which output ports time-codes are forwarded on.*
- `int STAR_API_CC_CFG_ROUTER_setNetworkIdentity (STAR_DEVICE_ID deviceId, REGISTER value)`  
*Sets the value of the network identity register.*
- `int STAR_API_CC_CFG_ROUTER_getNetworkIdentity (STAR_DEVICE_ID deviceId, _Out_ REGISTER *value)`  
*Gets the value of the network identity register.*
- `int STAR_API_CC_CFG_ROUTER_setGeneralPurpose (STAR_DEVICE_ID deviceId, REGISTER value)`  
*Sets the value of the general purpose register.*
- `int STAR_API_CC_CFG_ROUTER_getGeneralPurpose (STAR_DEVICE_ID deviceId, _Out_ REGISTER *value)`  
*Gets the value of the general purpose register.*
- `int STAR_API_CC_CFG_ROUTER_setTimeCodeDistributionPorts (STAR_DEVICE_ID deviceId, U32 portMask)`  
*This function is used to specify which output ports time-codes are forwarded on.*
- `int STAR_API_CC_CFG_ROUTER_getTimeCodeDistributionPorts (STAR_DEVICE_ID deviceId, _Out_ U32 *portMask)`  
*This function is used to obtain which output ports time-codes are forwarded on.*

### 7.51.1 Detailed Description

Functions in this group deal with reading and writing to the user configurable registers.

These registers have no effect upon the operation of the router.

### 7.51.2 Function Documentation

**7.51.2.1** `int STAR_API_CC_CFG_MK2_getTimeCodeDistributionPorts ( STAR_DEVICE_ID deviceId, _Out_ U32 *pPortMask )`

This function is used to obtain which output ports time-codes are forwarded on.

**Parameters**

|     |                  |                                                                     |
|-----|------------------|---------------------------------------------------------------------|
|     | <i>deviceID</i>  | Device to get the time-code distribution ports for.                 |
| out | <i>pPortMask</i> | Bit mask of ports that time code distribution should be enabled on. |

**Note**

Bit 1 corresponds to port 1.

**Returns**

1 if the register was successfully obtained, else 0.

**Added in version:**

STAR-System v4.00 Beta 1 - 25th September 2018

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S

### 7.51.2.2 int STAR\_API\_CC\_CFG\_MK2\_setTimeCodeDistributionPorts ( STAR\_DEVICE\_ID deviceID, U32 portMask )

This function is used to specify which output ports time-codes are forwarded on.

**Parameters**

|  |                 |                                                                                                                                                       |
|--|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <i>deviceID</i> | Device to set the time-code distribution ports for.                                                                                                   |
|  | <i>portMask</i> | Bit mask of ports that time code distribution should be enabled on. Valid ports to forward time-codes on are 1 through to 13 depending on the device. |

**Note**

Bit 1 corresponds to port 1.

Not all external ports allow time-code forwarding. Check your device's user manual for more details.

**Returns**

1 if the register was successfully obtained, else 0.

**Added in version:**

STAR-System v4.00 Beta 1 - 25th September 2018

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S

### 7.51.2.3 int STAR\_API\_CC\_CFG\_ROUTER\_getGeneralPurpose ( STAR\_DEVICE\_ID deviceID, \_Out\_ REGISTER \* value )

Gets the value of the general purpose register.

This is a user defined 32 bit value that can be set by the user as required for their purposes. This value has no effect on the operation of the router.



**Parameters**

|            |                 |                                          |
|------------|-----------------|------------------------------------------|
|            | <i>deviceID</i> | Device to get the network identity from. |
| <i>out</i> | <i>value</i>    | Value of the register                    |

**Returns**

1 if the register was successfully obtained, else 0.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S, SpaceWire Physical Layer Tester

**7.51.2.4** `int STAR_API_CC_CFG_ROUTER_getNetworkIdentity ( STAR_DEVICE_ID deviceID, _Out_ REGISTER * value )`

Gets the value of the network identity register.

This is a 32 bit value that can be used to identify the device, typically set by a network manager. It may also be used for any other purpose. This value has no effect on the operation of the router.

**Parameters**

|            |                 |                                          |
|------------|-----------------|------------------------------------------|
|            | <i>deviceID</i> | Device to get the network identity from. |
| <i>out</i> | <i>value</i>    | Value of the register.                   |

**Returns**

1 if the register was successfully obtained, else 0.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S, SpaceWire Physical Layer Tester

**7.51.2.5** `int STAR_API_CC_CFG_ROUTER_getTimeCodeDistributionPorts ( STAR_DEVICE_ID deviceID, _Out_ U32 * portMask )`

This function is used to obtain which output ports time-codes are forwarded on.

**Parameters**

|            |                 |                                                                     |
|------------|-----------------|---------------------------------------------------------------------|
|            | <i>deviceID</i> | Device to get the time-code distribution ports for.                 |
| <i>out</i> | <i>portMask</i> | Bit mask of ports that time code distribution should be enabled on. |

**Note**

Bit 1 corresponds to port 1.

**Returns**

1 if the register was successfully obtained, else 0.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester

### 7.51.2.6 `int STAR_API_CC_CFG_ROUTER_setGeneralPurpose ( STAR_DEVICE_ID deviceId, REGISTER value )`

Sets the value of the general purpose register.

This is a user defined 32 bit value that can be set by the user as required for their purposes. This value has no effect on the operation of the router.

**Parameters**

|                 |                                                |
|-----------------|------------------------------------------------|
| <i>deviceId</i> | Device to set the general purpose register on. |
| <i>value</i>    | Value to set the general purpose register to.  |

**Returns**

1 if the register was successfully set, else 0.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S, SpaceWire Physical Layer Tester

### 7.51.2.7 `int STAR_API_CC_CFG_ROUTER_setNetworkIdentity ( STAR_DEVICE_ID deviceId, REGISTER value )`

Sets the value of the network identity register.

This is a 32 bit value that can be used to identify the device, typically set by a network manager. It may also be used for any other purpose. This value has no effect on the operation of the router.

**Parameters**

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceId</i> | Device to set the network identity for. |
| <i>value</i>    | Value to set the network identity to.   |

**Returns**

1 if the register was successfully set, else 0.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S, SpaceWire Physical Layer Tester

7.51.2.8 `int STAR_API_CC CFG_ROUTER_setTimeCodeDistributionPorts ( STAR_DEVICE_ID deviceId, U32 portMask )`

This function is used to specify which output ports time-codes are forwarded on.

**Parameters**

---

|                 |                                                                                                                                                       |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>deviceID</i> | Device to set the time-code distribution ports for.                                                                                                   |
| <i>portMask</i> | Bit mask of ports that time code distribution should be enabled on. Valid ports to forward time-codes on are 1 through to 11 depending on the device. |

---

**Note**

Bit 1 corresponds to port 1.

Not all external ports allow time-code forwarding. Check your device's user manual for more details.

**Returns**

1 if the register was successfully obtained, else 0.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

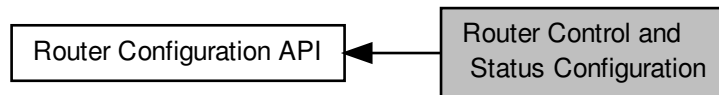
**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester

## 7.52 Router Control and Status Configuration

Functions in this group deal with configuring the router's internal state, such as timeout periods self addressing, time code forwarding etc.

Collaboration diagram for Router Control and Status Configuration:



### Data Structures

- struct `STAR_CFG_ROUTER_GLOBAL_STATE`

Global router settings. [More...](#)

### Enumerations

- enum `STAR_CFG_PORT_TIMEOUT` {  
`STAR_CFG_PORT_TIMEOUT_100US = 0x0`, `STAR_CFG_PORT_TIMEOUT_1MS = 0x1`, `STAR_CFG_PORT_TIMEOUT_10MS = 0x2`, `STAR_CFG_PORT_TIMEOUT_100MS = 0x3`,  
`STAR_CFG_PORT_TIMEOUT_1S = 0x4` }

Length of port timeout.

- enum `STAR_CFG_TIMEOUT_MODE` { `STAR_CFG_TIMEOUT_MODE_BLOCKING`, `STAR_CFG_TIMEOUT_MODE_WATCHDOG` }

Timeout mode used by the router.

### Functions

- int `STAR_API_CC_CFG_MK2_setTimeCodeFlagMode (STAR_DEVICE_ID deviceId, U8 mode)`  
*Sets the time-code flag interpretation mode:*
- int `STAR_API_CC_CFG_MK2_getTimeCodeFlagMode (STAR_DEVICE_ID deviceId, _Out_ U8 *pMode)`  
*Obtains the time-code flag interpretation mode:*
- int `STAR_API_CC_CFG_ROUTER_setTimeCodeFlagMode (STAR_DEVICE_ID deviceId, U8 mode)`  
*Sets the time-code flag interpretation mode:*
- int `STAR_API_CC_CFG_ROUTER_getTimeCodeFlagMode (STAR_DEVICE_ID deviceId, _Out_ U8 *mode)`  
*Obtains the time-code flag interpretation mode:*
- int `STAR_API_CC_CFG_ROUTER_getTimeCodeValue (STAR_DEVICE_ID deviceId, _Out_ U8 *value)`  
*Obtains the current value of the router's internal time-code counter.*
- int `STAR_API_CC_CFG_ROUTER_setRouterGlobalSettings (STAR_DEVICE_ID deviceId, STAR_CFG_ROUTER_GLOBAL_STATE state)`  
*Sets the router's global settings.*
- int `STAR_API_CC_CFG_ROUTER_getRouterGlobalSettings (STAR_DEVICE_ID deviceId, _Out_ STAR_CFG_ROUTER_GLOBAL_STATE *state)`  
*Gets the router's global settings.*

### 7.52.1 Detailed Description

Functions in this group deal with configuring the router's internal state, such as timeout periods self addressing, time code forwarding etc.

### 7.52.2 Data Structure Documentation

#### 7.52.2.1 struct STAR\_CFG\_ROUTER\_GLOBAL\_STATE

Global router settings.

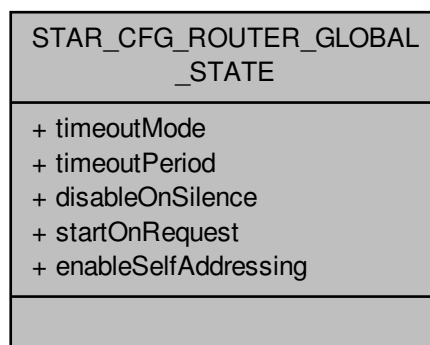
Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

rmap\_target\_example.c, router\_configuration\_example.c, and timestamp\_test.c.

Collaboration diagram for STAR\_CFG\_ROUTER\_GLOBAL\_STATE:



## Data Fields

|                       |                      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-----------------------|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| char                  | disableOnSilence     | If true, links will be disabled after the timeout period when data transfer completes. The link will only be disabled if the link was started automatically.                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| char                  | enableSelfAddressing | If true, a packet can be routed out of the port on which it arrived on. This is for debugging purposes. If this is false and a packet is to be routed through the same port an address error is reported and the packet is discarded. If false, and a group adaptive routing packet is received (a packet which can be routed through two or more ports, dependent on the group adaptive routing table contents) which can be routed through the port it arrived on then the packet is routed through one of the other ports and not the port on which the packet arrived on. An address error is not reported. |
| char                  | startOnRequest       | If true, links will be automatically started when they have data to transfer. If the link cannot be started packets are discarded after the timeout period.                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| STAR_CFG_TIMEOUT_MODE | timeoutMode          | Timeout mode.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| STAR_CFG_PORT_TIMEOUT | timeoutPeriod        | Timeout period.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |

## 7.52.3 Enumeration Type Documentation

## 7.52.3.1 enum STAR\_CFG\_PORT\_TIMEOUT

Length of port timeout.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

## Enumerator

**STAR\_CFG\_PORT\_TIMEOUT\_100US** 60-80us

**STAR\_CFG\_PORT\_TIMEOUT\_1MS** ~1.3ms

**STAR\_CFG\_PORT\_TIMEOUT\_10MS** ~10ms

**STAR\_CFG\_PORT\_TIMEOUT\_100MS** ~82ms

**STAR\_CFG\_PORT\_TIMEOUT\_1S** ~1.3s

## 7.52.3.2 enum STAR\_CFG\_TIMEOUT\_MODE

Timeout mode used by the router.

This affects how blocked packets will be handled.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

## Enumerator

**STAR\_CFG\_TIMEOUT\_MODE\_BLOCKING** Blocking allowed. When blocking mode is enabled packets will wait forever to be routed unless the packet is routed to a port that is not started. In this case the packet will be discarded.

**STAR\_CFG\_TIMEOUT\_MODE\_WATCHDOG** Watchdog timer mode. When watchdog mode is enabled packets which are waiting to be routed at source ports will be discarded after the timeout period. Packet tails will also be discarded if the packet becomes blocked for the timeout period. In this case the packet will be ended with an EEP.

## 7.52.4 Function Documentation

### 7.52.4.1 `int STAR_API_CC_CFG_MK2_getTimeCodeFlagMode ( STAR_DEVICE_ID deviceID, _Out_ U8 * pMode )`

Obtains the time-code flag interpretation mode:

0 - Time-code control bit flags are distributed with valid time-code values regardless of the value of the time-code control flags

1 - When the time-code control bit flags are "00" then valid time-codes are distributed. When the time-code control flags are not "00" then the time-code is discarded and the internal time-code register is not updated.

#### Parameters

|     |                 |                                             |
|-----|-----------------|---------------------------------------------|
|     | <i>deviceID</i> | Device to get the time-code flag mode from. |
| out | <i>pMode</i>    | Time code flag mode,                        |

#### Returns

1 if the register was successfully obtained, else 0.

#### Added in version:

STAR-System v4.00 Beta 1 - 25th September 2018

#### Supported devices:

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S

### 7.52.4.2 `int STAR_API_CC_CFG_MK2_setTimeCodeFlagMode ( STAR_DEVICE_ID deviceID, U8 mode )`

Sets the time-code flag interpretation mode:

0 - Time-code control bit flags are distributed with valid time-code values regardless of the value of the time-code control flags

1 - When the time-code control bit flags are "00" then valid time-codes are distributed. When the time-code control flags are not "00" then the time-code is discarded and the internal time-code register is not updated.

#### Parameters

|  |                 |                                             |
|--|-----------------|---------------------------------------------|
|  | <i>deviceID</i> | Device to set the time-code flag modes for. |
|  | <i>mode</i>     | Mode to set the time code flag mode to,     |

#### Returns

1 if the register was successfully obtained, else 0.

#### Added in version:

STAR-System v4.00 Beta 1 - 25th September 2018

#### Supported devices:

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S



```
7.52.4.3 int STAR_API_CC_CFG_ROUTER_getRouterGlobalSettings (STAR_DEVICE_ID deviceId, _Out_
 STAR_CFG_ROUTER_GLOBAL_STATE * state)
```

Gets the router's global settings.

**Parameters**

|            |                 |                                     |
|------------|-----------------|-------------------------------------|
|            | <i>deviceID</i> | Device to get global settings from. |
| <i>out</i> | <i>state</i>    | Global settings for the router.     |

**Returns**

1 if the entry was successfully obtained, else 0.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S, SpaceWire Physical Layer Tester

#### 7.52.4.4 int STAR\_API\_CC\_CFG\_ROUTER\_getTimeCodeFlagMode ( STAR\_DEVICE\_ID *deviceID*, \_Out\_ U8 \* *mode* )

Obtains the time-code flag interpretation mode:

0 - Time-code control bit flags are distributed with valid time-code values regardless of the value of the time-code control flags

1 - When the time-code control bit flags are "00" then valid time-codes are distributed. When the time-code control flags are not "00" then the time-code is discarded and the internal time-code register is not updated.

**Parameters**

|            |                 |                                             |
|------------|-----------------|---------------------------------------------|
|            | <i>deviceID</i> | Device to get the time-code flag mode from. |
| <i>out</i> | <i>mode</i>     | Time code flag mode,                        |

**Returns**

1 if the register was successfully obtained, else 0.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester

#### 7.52.4.5 int STAR\_API\_CC\_CFG\_ROUTER\_getTimeCodeValue ( STAR\_DEVICE\_ID *deviceID*, \_Out\_ U8 \* *value* )

Obtains the current value of the router's internal time-code counter.

**Parameters**

|            |                 |                                         |
|------------|-----------------|-----------------------------------------|
| •          | <i>deviceID</i> | Device to get the time-code value from. |
| <i>out</i> | <i>value</i>    | Time-code value,                        |

**Returns**

1 if the register was successfully obtained, else 0.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S, SpaceWire Physical Layer Tester

#### 7.52.4.6 `int STAR_API_CC_CFG_ROUTER_setRouterGlobalSettings ( STAR_DEVICE_ID deviceId, STAR_CFG_ROUTER_GLOBAL_STATE state )`

Sets the router's global settings.

**Parameters**

|                 |                                    |
|-----------------|------------------------------------|
| <i>deviceId</i> | Device to set global settings for. |
| <i>state</i>    | Global settings for the router.    |

**Returns**

1 if the entry was successfully set, else 0.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S, SpaceWire Physical Layer Tester

#### 7.52.4.7 `int STAR_API_CC_CFG_ROUTER_setTimeCodeFlagMode ( STAR_DEVICE_ID deviceId, U8 mode )`

Sets the time-code flag interpretation mode:

0 - Time-code control bit flags are distributed with valid time-code values regardless of the value of the time-code control flags

1 - When the time-code control bit flags are "00" then valid time-codes are distributed. When the time-code control flags are not "00" then the time-code is discarded and the internal time-code register is not updated.

**Parameters**

|                 |                                             |
|-----------------|---------------------------------------------|
| <i>deviceId</i> | Device to set the time-code flag modes for. |
| <i>mode</i>     | Mode to set the time code flag mode to,     |

**Returns**

1 if the register was successfully obtained, else 0.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

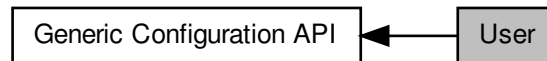
**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester

## 7.53 User

Functions in this group are user functions.

Collaboration diagram for User:



### Functions

- `int STAR_API_CC_CFG_updateDeviceInfo (STAR_DEVICE_ID deviceId)`  
*This function is used to update a device's information held in the shared memory.*
- `int STAR_API_CC_CFG_getNetworkIdentity (STAR_DEVICE_ID deviceId, _Out_ REGISTER *pValue)`  
*Gets the value of the network identity register.*
- `int STAR_API_CC_CFG_setNetworkIdentity (STAR_DEVICE_ID deviceId, REGISTER value)`  
*Sets the value of the network identity register.*
- `int STAR_API_CC_CFG_getTimeCodeDistributionPorts (STAR_DEVICE_ID deviceId, _Out_ U32 *pPortMask)`  
*This function is used to obtain which output ports time-codes are forwarded on.*
- `int STAR_API_CC_CFG_setTimeCodeDistributionPorts (STAR_DEVICE_ID deviceId, U32 portMask)`  
*This function is used to specify which output ports time-codes are forwarded on.*
- `int STAR_API_CC_CFG_getGeneralPurpose (STAR_DEVICE_ID deviceId, _Out_ REGISTER *pValue)`  
*Gets the value of the general purpose register.*
- `int STAR_API_CC_CFG_setGeneralPurpose (STAR_DEVICE_ID deviceId, REGISTER value)`  
*Sets the value of the general purpose register.*

### 7.53.1 Detailed Description

Functions in this group are user functions.

### 7.53.2 Function Documentation

#### 7.53.2.1 `int STAR_API_CC_CFG_getGeneralPurpose ( STAR_DEVICE_ID deviceId, _Out_ REGISTER * pValue )`

Gets the value of the general purpose register.

This is a user defined 32 bit value that can be set by the user as required for their purposes. This value has no effect on the operation of the router.

#### Parameters

|                 |                                          |
|-----------------|------------------------------------------|
| <i>deviceId</i> | Device to get the network identity from. |
|-----------------|------------------------------------------|

|     |               |                       |
|-----|---------------|-----------------------|
| out | <i>pValue</i> | Value of the register |
|-----|---------------|-----------------------|

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

### 7.53.2.2 int STAR\_API\_CC\_CFG\_getNetworkIdentity ( STAR\_DEVICE\_ID *deviceId*, \_Out\_ REGISTER \* *pValue* )

Gets the value of the network identity register.

This is a 32 bit value that can be used to identify the device, typically set by a network manager. It may also be used for any other purpose. This value has no effect on the operation of the router.

**Parameters**

|     |                 |                                          |
|-----|-----------------|------------------------------------------|
|     | <i>deviceId</i> | Device to get the network identity from. |
| out | <i>pValue</i>   | Value of the register.                   |

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

### 7.53.2.3 int STAR\_API\_CC\_CFG\_getTimeCodeDistributionPorts ( STAR\_DEVICE\_ID *deviceId*, \_Out\_ U32 \* *pPortMask* )

This function is used to obtain which output ports time-codes are forwarded on.

**Parameters**

|     |                  |                                                                     |
|-----|------------------|---------------------------------------------------------------------|
|     | <i>deviceId</i>  | Device to get the time-code distribution ports for.                 |
| out | <i>pPortMask</i> | Bit mask of ports that time code distribution should be enabled on. |

**Note**

Bit 1 corresponds to port 1.

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

#### 7.53.2.4 `int STAR_API_CC_CFG_setGeneralPurpose ( STAR_DEVICE_ID deviceId, REGISTER value )`

Sets the value of the general purpose register.

This is a user defined 32 bit value that can be set by the user as required for their purposes. This value has no effect on the operation of the router.

**Parameters**

|                 |                                                |
|-----------------|------------------------------------------------|
| <i>deviceId</i> | Device to set the general purpose register on. |
| <i>value</i>    | Value to set the general purpose register to.  |

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

#### 7.53.2.5 `int STAR_API_CC_CFG_setNetworkIdentity ( STAR_DEVICE_ID deviceId, REGISTER value )`

Sets the value of the network identity register.

This is a 32 bit value that can be used to identify the device, typically set by a network manager. It may also be used for any other purpose. This value has no effect on the operation of the router.

**Parameters**

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceId</i> | Device to set the network identity for. |
| <i>value</i>    | Value to set the network identity to.   |

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

#### 7.53.2.6 int STAR\_API\_CC\_CFG\_setTimeCodeDistributionPorts ( STAR\_DEVICE\_ID *deviceId*, U32 *portMask* )

This function is used to specify which output ports time-codes are forwarded on.

**Parameters**

|                 |                                                                                                                                                       |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>deviceId</i> | Device to set the time-code distribution ports for.                                                                                                   |
| <i>portMask</i> | Bit mask of ports that time code distribution should be enabled on. Valid ports to forward time-codes on are 1 through to 13 depending on the device. |

**Note**

Bit 1 corresponds to port 1.

Not all external ports allow time-code forwarding. Check your device's user manual for more details.

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

#### 7.53.2.7 int STAR\_API\_CC\_CFG\_updateDeviceInfo ( STAR\_DEVICE\_ID *deviceId* )

This function is used to update a device's information held in the shared memory.

This is only required for remote devices if they have been removed and replaced.

**Parameters**

|                 |                                       |
|-----------------|---------------------------------------|
| <i>deviceId</i> | Device to update the information for. |
|-----------------|---------------------------------------|

**Returns**

1 if the device's information was successfully updated, else 0.

**Added in version:**

STAR-System v4.00 - 19th November 2019

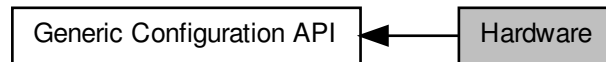
**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

## 7.54 Hardware

Functions in this group are hardware related functions.

Collaboration diagram for Hardware:



### Data Structures

- struct [STAR\\_CFG\\_FPGA\\_INFO](#)

*Defines a structure used to store a device's FPGA version info. [More...](#)*

### Functions

- int [STAR\\_API\\_CC\\_CFG\\_getFPGAInfo](#) (STAR\_DEVICE\_ID deviceId, \_Out\_ STAR\_CFG\_FPGA\_INFO \*pFPGAInfo)

*Reads information about the hardware version of a device and updates the [STAR\\_CFG\\_FPGA\\_INFO](#) structure pointed to by the passed in pointer to contain the received version information.*

- void [STAR\\_API\\_CC\\_CFG\\_FPGAInfoToString](#) (STAR\_DEVICE\_ID deviceId, \_In\_ STAR\_CFG\_FPGA\_INFO \*pFPGAInfo, \_Out\_z\_cap\_c\_(STAR\_CFG\_MK2\_VERSION\_STR\_MAX\_LEN) char \*pVersion, \_Out\_z\_cap\_c\_(STAR\_CFG\_MK2\_BUILD\_DATE\_STR\_MAX\_LEN) char \*pBuildDate)

*Creates a string representation of a [STAR\\_CFG\\_FPGA\\_INFO](#) structure.*

- int [STAR\\_API\\_CC\\_CFG\\_identify](#) (STAR\_DEVICE\_ID deviceId)

*Flashes the front panel LEDs of a device.*

#### 7.54.1 Detailed Description

Functions in this group are hardware related functions.

#### 7.54.2 Data Structure Documentation

##### 7.54.2.1 struct STAR\_CFG\_FPGA\_INFO

Defines a structure used to store a device's FPGA version info.

#### Examples:

[brickMk2\\_configuration\\_example.c](#), [link\\_events.c](#), [mk2\\_configuration\\_example.c](#), [pciMk2\\_configuration\\_example.c](#), [routerMk2S\\_configuration\\_example.c](#), [star-test.c](#), and [timestamp\\_test.c](#).



Collaboration diagram for STAR\_CFG\_FPGA\_INFO:



#### Data Fields

|     |        |                            |
|-----|--------|----------------------------|
| U8  | day    | FPGA build day.            |
| U16 | edit   | FPGA edit number.          |
| U8  | hour   | FPGA build hour.           |
| U8  | major  | FPGA major version number. |
| U8  | minor  | FPGA minor version number. |
| U8  | minute | FPGA build minute.         |
| U8  | month  | FPGA build month.          |
| U8  | patch  | FPGA patch number.         |
| U16 | year   | FPGA build year.           |

### 7.54.3 Function Documentation

**7.54.3.1** void STAR\_API\_CC CFG\_FPGAInfoToString ( STAR\_DEVICE\_ID *deviceId*, \_In\_ STAR\_CFG\_FPGA\_INFO \* *pFPGAInfo*, \_Out\_z\_cap\_c\_(STAR\_CFG\_MK2\_VERSION\_STR\_MAX\_LEN) char \* *pVersion*, \_Out\_z\_cap\_c\_(STAR\_CFG\_MK2\_BUILD\_DATE\_STR\_MAX\_LEN) char \* *pBuildDate* )

Creates a string representation of a [STAR\\_CFG\\_FPGA\\_INFO](#) structure.

#### Parameters

|     |                  |                                                                                                                                                                    |
|-----|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|     | <i>deviceId</i>  | Identifier for the device to get the info string from.                                                                                                             |
| in  | <i>pFPGAInfo</i> | FPGA information obtained from a call to <a href="#">CFG_getFPGAInfo()</a> .                                                                                       |
| out | <i>pVersion</i>  | String of length <a href="#">STAR_CFG_MK2_VERSION_STR_MAX_LEN</a> that will be updated to contain a null-terminated string representation of the hardware version. |

|     |                   |                                                                                                                                                                    |
|-----|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| out | <i>pBuildDate</i> | String of length <code>STAR_CFG_MK2_VERSION_STR_MAX_LEN</code> that will be updated to contain a null-terminated string representation of the hardware build date. |
|-----|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

**7.54.3.2** `int STAR_API_CC CFG_getFPGAInfo ( STAR_DEVICE_ID deviceId, _Out_ STAR_CFG_FPGA_INFO * pFPGAInfo )`

Reads information about the hardware version of a device and updates the `STAR_CFG_FPGA_INFO` structure pointed to by the passed in pointer to contain the received version information.

**Parameters**

|     |                  |                                                                                                                             |
|-----|------------------|-----------------------------------------------------------------------------------------------------------------------------|
|     | <i>deviceId</i>  | Identifier for the device to get hardware info from.                                                                        |
| out | <i>pFPGAInfo</i> | Pointer to a <code>STAR_CFG_FPGA_INFO</code> structure that will be updated to contain hardware information for the device. |

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

**7.54.3.3** `int STAR_API_CC CFG_identify ( STAR_DEVICE_ID deviceId )`

Flashes the front panel LEDs of a device.

This can be used to identify the physical device to which a device identifier refers to.

**Parameters**

|  |                 |                                  |
|--|-----------------|----------------------------------|
|  | <i>deviceId</i> | Device to have its LEDs flashed. |
|--|-----------------|----------------------------------|

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

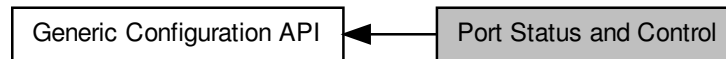
**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

## 7.55 Port Status and Control

Functions in this group deal with port status and control.

Collaboration diagram for Port Status and Control:



### Functions

- `int STAR_API_CC_CFG_clearPortErrors (STAR_DEVICE_ID deviceId, U8 portNum)`  
*Clears all errors on a given port.*
- `int STAR_API_CC_CFG_getConfigPortErrors (PORT_STATUS_CONTROL portStatusControl, _Out_ STAR_CFG_CONFIG_PORT_ERRORS *pErrors)`  
*Gets any errors present on the config port.*
- `int STAR_API_CC_CFG_setSpaceWireLinkStatus (STAR_DEVICE_ID deviceId, U8 linkNum, _In_ STAR_CFG_SPW_LINK_STATUS *pLinkStatus)`  
*Sets the state of a SpaceWire Link.*
- `int STAR_API_CC_CFG_startLink (STAR_DEVICE_ID deviceId, U8 linkNum)`  
*Starts a SpaceWire link by setting the start bit, and clearing the disable bit.*
- `int STAR_API_CC_CFG_stopLink (STAR_DEVICE_ID deviceId, U8 linkNum)`  
*Stops a SpaceWire link by clearing the start bit, and setting the disable bit.*
- `int STAR_API_CC_CFG_getSpaceWireLinkErrors (PORT_STATUS_CONTROL portStatusControl, _Out_ STAR_CFG_SPW_LINK_ERRORS *pErrors)`  
*Gets any errors present on a SpaceWire link.*
- `int STAR_API_CC_CFG_getSpaceWireLinkStatus (PORT_STATUS_CONTROL portStatusControl, _Out_ STAR_CFG_SPW_LINK_STATUS *pStatus)`  
*Gets the status of a SpaceWire Link.*
- `int STAR_API_CC_CFG_getPortStatusControl (STAR_DEVICE_ID deviceId, U8 portNum, _Out_ PORT_STATUS_CONTROL *pPortStatusControl)`  
*Gets the value of a port or link's status/control register, which can then be used by the various functions within the Port Status and Control group.*
- `STAR_CFG_PORT_TYPE STAR_API_CC_CFG_getPortType (PORT_STATUS_CONTROL portStatusControl)`  
*Identifies the type of port attributed to a given port number.*
- `int STAR_API_CC_CFG_getExternalPortErrors (PORT_STATUS_CONTROL portStatusControl, _Out_ STAR_CFG_EXTERNAL_PORT_ERRORS *pErrors)`  
*Gets any errors present on an external port.*
- `int STAR_API_CC_CFG_getExternalPortStatus (PORT_STATUS_CONTROL portStatusControl, _Out_ STAR_CFG_EXTERNAL_PORT_STATUS *pStatus)`  
*Gets the status of an external port.*
- `int STAR_API_CC_CFG_getPortConnection (PORT_STATUS_CONTROL portStatusControl)`  
*Identifies the output port to which the source port is currently connected whilst routing is in operation.*

### 7.55.1 Detailed Description

Functions in this group deal with port status and control.

### 7.55.2 Function Documentation

#### 7.55.2.1 `int STAR_API_CC CFG_clearPortErrors ( STAR_DEVICE_ID deviceID, U8 portNum )`

Clears all errors on a given port.

##### Note

Errors on a port are latched until cleared with this function.

##### Parameters

|                 |                                      |
|-----------------|--------------------------------------|
| <i>deviceID</i> | Device to clear port errors on.      |
| <i>portNum</i>  | The port to have its errors cleared. |

##### Returns

0 for success and a negative value on error.

##### Added in version:

STAR-System v4.00 - 19th November 2019

##### Supported devices:

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

#### 7.55.2.2 `int STAR_API_CC CFG_getConfigPortErrors ( PORT_STATUS_CONTROL portStatusControl, _Out_ STAR_CFG_CONFIG_PORT_ERRORS * pErrors )`

Gets any errors present on the config port.

These are errors that arise when malformed or invalid configuration commands are sent to the configuration port.

##### Note

Errors on the configuration port are latched until cleared with `CFG_clearPortErrors()`.

##### Parameters

|                          |                                                                                                          |
|--------------------------|----------------------------------------------------------------------------------------------------------|
| <i>portStatusControl</i> | Port Status/Control value obtained from a call to <code>CFG_getPortStatusControl()</code> .              |
| <i>out pErrors</i>       | User provided error structure that will be updated to show any errors present on the configuration port. |

##### Returns

0 for success and a negative value on error.

##### Added in version:

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

### 7.55.2.3 int STAR\_API\_CC CFG\_getExternalPortErrors ( PORT\_STATUS\_CONTROL *portStatusControl*, \_Out\_ STAR\_CFG\_EXTERNAL\_PORT\_ERRORS \* *pErrors* )

Gets any errors present on an external port.

**Note**

Errors on a port are latched until cleared with `CFG_ROUTER_clearPortErrors()`.

**Parameters**

|     |                          |                                                                                                     |
|-----|--------------------------|-----------------------------------------------------------------------------------------------------|
|     | <i>portStatusControl</i> | Port Status/Control value obtained from a call to <code>CFG_getPortStatusControl()</code> .         |
| out | <i>pErrors</i>           | User provided error structure that will be updated to show any errors present on the external port. |

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

### 7.55.2.4 int STAR\_API\_CC CFG\_getExternalPortStatus ( PORT\_STATUS\_CONTROL *portStatusControl*, \_Out\_ STAR\_CFG\_EXTERNAL\_PORT\_STATUS \* *pStatus* )

Gets the status of an external port.

**Parameters**

|     |                          |                                                                                             |
|-----|--------------------------|---------------------------------------------------------------------------------------------|
|     | <i>portStatusControl</i> | Port Status/Control value obtained from a call to <code>CFG_getPortStatusControl()</code> . |
| out | <i>pStatus</i>           | User provided error structure that will be updated to show the status of the external port. |

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

7.55.2.5 `int STAR_API_CC CFG_getPortConnection ( PORT_STATUS_CONTROL portStatusControl )`

Identifies the output port to which the source port is currently connected whilst routing is in operation.

**Parameters**


---

|                                |                                                                                             |
|--------------------------------|---------------------------------------------------------------------------------------------|
| <i>portStatus↔<br/>Control</i> | Port Status/Control value obtained from a call to <code>CFG_getPortStatusControl()</code> . |
|--------------------------------|---------------------------------------------------------------------------------------------|

---

**Returns**

Output port connected. This will have a value of 31 if there is no current port connected.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

**7.55.2.6** `int STAR_API_CC CFG_getPortStatusControl ( STAR_DEVICE_ID deviceID, U8 portNum, _Out_ PORT_STATUS_CONTROL * pPortStatusControl )`

Gets the value of a port or link's status/control register, which can then be used by the various functions within the Port Status and Control group.

**Parameters**


---

|            |                                 |                                                                |
|------------|---------------------------------|----------------------------------------------------------------|
|            | <i>deviceID</i>                 | Device to obtain info from.                                    |
|            | <i>portNum</i>                  | The port number to determine the type of.                      |
| <i>out</i> | <i>pPortStatus↔<br/>Control</i> | Value that will be updated with the ports status/control value |

---

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

**7.55.2.7** `STAR_CFG_PORT_TYPE STAR_API_CC CFG_getPortType ( PORT_STATUS_CONTROL portStatusControl )`

Identifies the type of port attributed to a given port number.

**Parameters**


---

|                                |                                                                                             |
|--------------------------------|---------------------------------------------------------------------------------------------|
| <i>portStatus↔<br/>Control</i> | Port Status/Control value obtained from a call to <code>CFG_getPortStatusControl()</code> . |
|--------------------------------|---------------------------------------------------------------------------------------------|

---



**Returns**

Type of the port

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

**7.55.2.8** `int STAR_API_CC CFG_getSpaceWireLinkErrors ( PORT_STATUS_CONTROL portStatusControl, _Out_ STAR_CFG_SPW_LINK_ERRORS * pErrors )`

Gets any errors present on a SpaceWire link.

**Note**

Errors on a port are latched until cleared with `CFG_clearPortErrors()`.

**Parameters**

|     |                          |                                                                                                      |
|-----|--------------------------|------------------------------------------------------------------------------------------------------|
|     | <i>portStatusControl</i> | Port Status/Control value obtained from a call to <code>CFG_getPortStatusControl()</code> .          |
| out | <i>pErrors</i>           | User provided error structure that will be updated to show any errors present on the SpaceWire link. |

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

**7.55.2.9** `int STAR_API_CC CFG_getSpaceWireLinkStatus ( PORT_STATUS_CONTROL portStatusControl, _Out_ STAR_CFG_SPW_LINK_STATUS * pStatus )`

Gets the status of a SpaceWire Link.

**Parameters**

|  |                          |                                                                                             |
|--|--------------------------|---------------------------------------------------------------------------------------------|
|  | <i>portStatusControl</i> | Port Status/Control value obtained from a call to <code>CFG_getPortStatusControl()</code> . |
|--|--------------------------|---------------------------------------------------------------------------------------------|

|            |                |                                                                                                  |
|------------|----------------|--------------------------------------------------------------------------------------------------|
| <i>out</i> | <i>pStatus</i> | User provided error structure that will be updated to show the status of the the SpaceWire link. |
|------------|----------------|--------------------------------------------------------------------------------------------------|

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

7.55.2.10 **int STAR\_API\_CC\_CFG\_setSpaceWireLinkStatus ( STAR\_DEVICE\_ID *deviceID*, U8 *linkNum*, \_In\_ STAR\_CFG\_SPW\_LINK\_STATUS \* *pLinkStatus* )**

Sets the state of a SpaceWire Link.

**Parameters**

|           |                    |                                                                                   |
|-----------|--------------------|-----------------------------------------------------------------------------------|
|           | <i>deviceID</i>    | Device to have its link state set.                                                |
|           | <i>linkNum</i>     | The link to set state for.                                                        |
| <i>in</i> | <i>pLinkStatus</i> | The values within this structure are used to set the state of the SpaceWire Link. |

**Note**

The linkState member of this structure is ignored by this function.

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

7.55.2.11 **int STAR\_API\_CC\_CFG\_startLink ( STAR\_DEVICE\_ID *deviceID*, U8 *linkNum* )**

Starts a SpaceWire link by setting the start bit, and clearing the disable bit.

**Parameters**

---

|                 |                                  |
|-----------------|----------------------------------|
| <i>deviceID</i> | Device to have its link started. |
| <i>linkNum</i>  | The link to start.               |

---

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

**7.55.2.12 int STAR\_API\_CC\_CFG\_stopLink ( STAR\_DEVICE\_ID deviceID, U8 linkNum )**

Stops a SpaceWire link by clearing the start bit, and setting the disable bit.

**Note**

If auto-start is enabled the link will start again as soon as it receives NULLs.

**Parameters**

---

|                 |                                  |
|-----------------|----------------------------------|
| <i>deviceID</i> | Device to have its link stopped. |
| <i>linkNum</i>  | The link to stop.                |

---

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

## 7.56 Device Identifier

Functions in this group deal with device identification.

Collaboration diagram for Device Identifier:



### Functions

- `int STAR_API_CC CFG_getNetworkDiscoveryInfo (STAR_DEVICE_ID deviceId, _Out_ STAR_CFG_NETWORK_DISCOVERY_INFO *pInfo)`  
*Reads the network discovery information of a device.*
- `int STAR_API_CC CFG_getDeviceIdentificationInfo (STAR_DEVICE_ID deviceId, _Out_ STAR_CFG_DEVICE_IDENTIFIER_INFO *pInfo)`  
*Reads the device identifier information of a device, i.e.*
- `void STAR_API_CC CFG_getDeviceManufacturerAsString (_In_ STAR_CFG_DEVICE_IDENTIFIER_INFO *pDeviceIdentifierInfo, _Out_z_cap_c_(STAR_CFG_MANUFACTURER_STR_MAX_LEN) char *pManufacturerStr)`  
*If the manufacturer is known, this function obtains a string representation of the manufacturers name.*
- `void STAR_API_CC CFG_getDeviceTypeAsString (_In_ STAR_CFG_DEVICE_IDENTIFIER_INFO *pDeviceIdentifierInfo, _Out_z_cap_c_(STAR_CFG_DEVICE_STR_MAX_LEN) char *pDeviceTypeStr)`  
*If the manufacturer and device type is known, this function obtains a string representation of the device's type.*

### 7.56.1 Detailed Description

Functions in this group deal with device identification.

### 7.56.2 Function Documentation

#### 7.56.2.1 `int STAR_API_CC CFG_getDeviceIdentificationInfo ( STAR_DEVICE_ID deviceId, _Out_ STAR_CFG_DEVICE_IDENTIFIER_INFO * pInfo )`

Reads the device identifier information of a device, i.e.

the Manufacturer ID, Chip type and version.

#### Parameters

|     | <i>deviceId</i> | Device to obtain info from.                                                              |
|-----|-----------------|------------------------------------------------------------------------------------------|
| out | <i>pInfo</i>    | Structure that will be updated to contain the Identification information for the device. |

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

**7.56.2.2** void **STAR\_API\_CC** CFG\_getDeviceManufacturerAsString ( \_In\_ STAR\_CFG\_DEVICE\_IDENTIFIER\_INFO \* *pDeviceIdentifierInfo*, \_Out\_z\_cap\_c\_(STAR\_CFG\_MANUFACTURER\_STR\_MAX\_LEN) char \* *pManufacturerStr* )

If the manufacturer is known, this function obtains a string representation of the manufacturers name.

**Parameters**

|     |                              |                                                                                                                                                                   |
|-----|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| in  | <i>pDeviceIdentifierInfo</i> | Pointer to device identification information obtained from a call to <a href="#">CFG_getDeviceIdentificationInfo()</a> .                                          |
| out | <i>pManufacturerStr</i>      | User allocated zero terminated string of max length <a href="#">STAR_CFG_MANUFACTURER_STR_MAX_LEN</a> that will be updated with the manufacturers name, if known. |

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

**7.56.2.3** void **STAR\_API\_CC** CFG\_getDeviceTypeAsString ( \_In\_ STAR\_CFG\_DEVICE\_IDENTIFIER\_INFO \* *pDeviceIdentifierInfo*, \_Out\_z\_cap\_c\_(STAR\_CFG\_DEVICE\_STR\_MAX\_LEN) char \* *pDeviceTypeStr* )

If the manufacturer and device type is known, this function obtains a string representation of the device's type.

**Parameters**

|     |                              |                                                                                                                                                      |
|-----|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| in  | <i>pDeviceIdentifierInfo</i> | Pointer to device identification information obtained from a call to <a href="#">CFG_getDeviceIdentificationInfo()</a> .                             |
| out | <i>pDeviceTypeStr</i>        | User allocated zero terminated string of max length <a href="#">STAR_CFG_DEVICE_STR_MAX_LEN</a> that will be updated with the device name, if known. |

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

7.56.2.4 `int STAR_API_CC CFG_getNetworkDiscoveryInfo ( STAR_DEVICE_ID deviceId, _Out_  
STAR_CFG_NETWORK_DISCOVERY_INFO * pInfo )`

Reads the network discovery information of a device.

This information can be used by a network manager to determine the layout of the network.

#### Parameters

|            |                 |                                                                                             |
|------------|-----------------|---------------------------------------------------------------------------------------------|
|            | <i>deviceId</i> | Device to obtain info from.                                                                 |
| <i>out</i> | <i>pInfo</i>    | Structure that will be updated to contain the network discovery information for the device. |

#### Returns

0 for success and a negative value on error.

#### Added in version:

STAR-System v4.00 - 19th November 2019

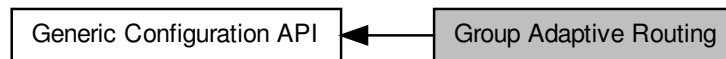
#### Supported devices:

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

## 7.57 Group Adaptive Routing

Functions in this group deal with group adaptive routing.

Collaboration diagram for Group Adaptive Routing:



### Functions

- `int STAR_API_CC_CFG_getRoutingTableEntry (STAR_DEVICE_ID deviceId, U8 logicalAddress, _Out_ STAR_CFG_GAR_ENTRY *pEntry)`  
*Gets a routing table entry for a given logical address.*
- `int STAR_API_CC_CFG_setRoutingTableEntry (STAR_DEVICE_ID deviceId, U8 logicalAddress, _In_ STAR_CFG_GAR_ENTRY *pEntry)`  
*Sets a routing table entry for a given logical address.*

### 7.57.1 Detailed Description

Functions in this group deal with group adaptive routing.

### 7.57.2 Function Documentation

**7.57.2.1** `int STAR_API_CC_CFG_getRoutingTableEntry ( STAR_DEVICE_ID deviceId, U8 logicalAddress, _Out_ STAR_CFG_GAR_ENTRY * pEntry )`

Gets a routing table entry for a given logical address.

#### Parameters

|            |                       |                                                    |
|------------|-----------------------|----------------------------------------------------|
|            | <i>deviceId</i>       | Device to get GAR table entry for.                 |
|            | <i>logicalAddress</i> | Logical address to get GAR table entry for.        |
| <i>out</i> | <i>pEntry</i>         | The GAR table entry for the given logical address. |

#### Returns

0 for success and a negative value on error.

#### Added in version:

STAR-System v4.00 - 19th November 2019

#### Supported devices:

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

7.57.2.2 `int STAR_API_CC CFG_setRoutingTableEntry ( STAR_DEVICE_ID deviceID, U8 logicalAddress, _In_ STAR_CFG_GAR_ENTRY * pEntry )`

Sets a routing table entry for a given logical address.



**Parameters**

|                       |                                                                              |
|-----------------------|------------------------------------------------------------------------------|
| <i>deviceID</i>       | Device to set GAR table entry on.                                            |
| <i>logicalAddress</i> | Logical address to set GAR table entry for. Valid values are 32 through 255. |

**Note**

Logical address 255 is reserved for future use and should not be used. See section 10.3.3n of the SpaceWire standard for more information.

**Parameters**

|           |               |                                                    |
|-----------|---------------|----------------------------------------------------|
| <i>in</i> | <i>pEntry</i> | The GAR table entry for the given logical address. |
|-----------|---------------|----------------------------------------------------|

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

## 7.58 Interface Mode

Functions in this group deal with interface mode.

Collaboration diagram for Interface Mode:



### Functions

- `int STAR_API_CC_CFG_getPortRoutingAddress (STAR_DEVICE_ID deviceId, U8 portNum, _Out_ U8 *pAddress)`  
*Gets the address a packet received on a given port should be routed to when interface mode is enabled.*
- `int STAR_API_CC_CFG_setPortRoutingAddress (STAR_DEVICE_ID deviceId, U8 portNum, U8 address)`  
*Sets the address a packet received on a given port should be routed to when interface mode is enabled.*
- `int STAR_API_CC_CFG_disableIdentifySource (STAR_DEVICE_ID deviceId)`  
*Disables identification of source ports for interface mode globally.*
- `int STAR_API_CC_CFG_enableIdentifySource (STAR_DEVICE_ID deviceId)`  
*When interface mode is enabled on a port, this function globally enables the addition of a leading byte to each received packet indicating which port the packet was received on.*
- `int STAR_API_CC_CFG_getIdentifySourceEnabled (STAR_DEVICE_ID deviceId, _Out_ int *pEnabled)`  
*Gets whether identify source is enabled.*
- `int STAR_API_CC_CFG_disableIdentifySourceOnPort (STAR_DEVICE_ID deviceId, U8 portNum)`  
*Disables source port identification for a given port.*
- `int STAR_API_CC_CFG_enableIdentifySourceOnPort (STAR_DEVICE_ID deviceId, U8 portNum)`  
*Enables identification of source port during interface mode for a given port.*
- `int STAR_API_CC_CFG_getIdentifySourceOnPortEnabled (STAR_DEVICE_ID deviceId, U8 portNum, _Out_ int *pEnabled)`  
*Gets whether identify source is enabled for a specific port.*
- `int STAR_API_CC_CFG_disableInterfaceMode (STAR_DEVICE_ID deviceId)`  
*Disables interface mode on a device.*
- `int STAR_API_CC_CFG_enableInterfaceMode (STAR_DEVICE_ID deviceId)`  
*In interface mode a packet which is received on an external port, from a SpaceWire link or from the configuration port will be routed to the port specified by the port routing register of the port (set by `CFG_setPortRoutingAddress()`).*
- `int STAR_API_CC_CFG_getInterfaceModeEnabled (STAR_DEVICE_ID deviceId, _Out_ int *pEnabled)`  
*Gets whether interface mode has been enabled on a device.*
- `int STAR_API_CC_CFG_disableInterfaceModeOnPort (STAR_DEVICE_ID deviceId, U8 portNum)`  
*Selectively disables interface mode on a given port.*
- `int STAR_API_CC_CFG_enableInterfaceModeOnPort (STAR_DEVICE_ID deviceId, U8 portNum)`  
*Selectively enables interface mode on a given port.*
- `int STAR_API_CC_CFG_getInterfaceModeOnPortEnabled (STAR_DEVICE_ID deviceId, U8 portNum, _Out_ int *pEnabled)`  
*Gets whether interface mode is enabled on a specific port.*

### 7.58.1 Detailed Description

Functions in this group deal with interface mode.

### 7.58.2 Function Documentation

#### 7.58.2.1 `int STAR_API_CC CFG_disableIdentifySource ( STAR_DEVICE_ID deviceID )`

Disables identification of source ports for interface mode globally.

##### Parameters

|                 |                                                  |
|-----------------|--------------------------------------------------|
| <i>deviceID</i> | Device to disable source port identification on. |
|-----------------|--------------------------------------------------|

##### Returns

0 for success and a negative value on error.

##### Added in version:

STAR-System v4.00 - 19th November 2019

##### Supported devices:

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

#### 7.58.2.2 `int STAR_API_CC CFG_disableIdentifySourceOnPort ( STAR_DEVICE_ID deviceID, U8 portNum )`

Disables source port identification for a given port.

##### Parameters

|                 |                                                                   |
|-----------------|-------------------------------------------------------------------|
| <i>deviceID</i> | Device to disable source port identification for a given port on. |
| <i>portNum</i>  | Port to disable source identification on.                         |

##### Returns

0 for success and a negative value on error.

##### Added in version:

STAR-System v4.00 - 19th November 2019

##### Supported devices:

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

#### 7.58.2.3 `int STAR_API_CC CFG_disableInterfaceMode ( STAR_DEVICE_ID deviceID )`

Disables interface mode on a device.

When interface mode is disabled, the device operates in routing mode.

**Parameters**


---

|                 |                                      |
|-----------------|--------------------------------------|
| <i>deviceID</i> | Device to disable interface mode on. |
|-----------------|--------------------------------------|

---

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

#### 7.58.2.4 int STAR\_API\_CC\_CFG\_disableInterfaceModeOnPort ( STAR\_DEVICE\_ID *deviceID*, U8 *portNum* )

Selectively disables interface mode on a given port.

When interface mode is disabled, the device operates in routing mode.

**Parameters**


---

|                 |                                                 |
|-----------------|-------------------------------------------------|
| <i>deviceID</i> | Device to disable interface mode for a port on. |
| <i>portNum</i>  | Port to disable interface mode on.              |

---

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

#### 7.58.2.5 int STAR\_API\_CC\_CFG\_enableIdentifySource ( STAR\_DEVICE\_ID *deviceID* )

When interface mode is enabled on a port, this function globally enables the addition of a leading byte to each received packet indicating which port the packet was received on.

For each source port on which this behaviour is desired, a call to `CFG_enableIdentifySourceOnPort()` must be made.

**Parameters**


---

|                 |                                                 |
|-----------------|-------------------------------------------------|
| <i>deviceID</i> | Device to enable source port identification on. |
|-----------------|-------------------------------------------------|

---

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

### 7.58.2.6 int STAR\_API\_CC\_CFG\_enableIdentifySourceOnPort ( STAR\_DEVICE\_ID *deviceId*, U8 *portNum* )

Enables identification of source port during interface mode for a given port.

Interface mode (CFG\_enableInterfaceMode()) and Identify Source (CFG\_enableIdentifySource()) must be enabled globally for this to have an effect.

**Parameters**

|                 |                                                                  |
|-----------------|------------------------------------------------------------------|
| <i>deviceId</i> | Device to enable source port identification for a given port on. |
| <i>portNum</i>  | Port to enable source identification on.                         |

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

### 7.58.2.7 int STAR\_API\_CC\_CFG\_enableInterfaceMode ( STAR\_DEVICE\_ID *deviceId* )

In interface mode a packet which is received on an external port, from a SpaceWire link or from the configuration port will be routed to the port specified by the port routing register of the port (set by CFG\_setPortRoutingAddress()).

**Parameters**

|                 |                                     |
|-----------------|-------------------------------------|
| <i>deviceId</i> | Device to enable interface mode on. |
|-----------------|-------------------------------------|

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

#### 7.58.2.8 `int STAR_API_CC_CFG_enableInterfaceModeOnPort ( STAR_DEVICE_ID deviceId, U8 portNum )`

Selectively enables interface mode on a given port.

Interface mode must be enabled globally (`CFG_enableInterfaceMode()`) for interface mode on a port to be enabled.

##### Parameters

|                 |                                                |
|-----------------|------------------------------------------------|
| <i>deviceId</i> | Device to enable interface mode for a port on. |
| <i>portNum</i>  | Port to enable interface mode on.              |

##### Returns

0 for success and a negative value on error.

##### Added in version:

STAR-System v4.00 - 19th November 2019

##### Supported devices:

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

#### 7.58.2.9 `int STAR_API_CC_CFG_getIdentifySourceEnabled ( STAR_DEVICE_ID deviceId, _Out_ int * pEnabled )`

Gets whether identify source is enabled.

##### Parameters

|            |                 |                                                                                      |
|------------|-----------------|--------------------------------------------------------------------------------------|
|            | <i>deviceId</i> | Device to check.                                                                     |
| <i>out</i> | <i>pEnabled</i> | User supplied value that will be updated to 1 if identify source is enabled, else 0. |

##### Returns

0 for success and a negative value on error.

##### Added in version:

STAR-System v4.00 - 19th November 2019

##### Supported devices:

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

#### 7.58.2.10 `int STAR_API_CC_CFG_getIdentifySourceOnPortEnabled ( STAR_DEVICE_ID deviceId, U8 portNum, _Out_ int * pEnabled )`

Gets whether identify source is enabled for a specific port.

##### Note

SpaceWire PCI Mk2 and cPCI Mk2 and SpaceWire PCIe devices prior to version 1.07 have a bug which means that the value returned in *pEnabled* is always 0.

**Parameters**

|     |                 |                                                                                      |
|-----|-----------------|--------------------------------------------------------------------------------------|
|     | <i>deviceID</i> | Device to check.                                                                     |
|     | <i>portNum</i>  | Port to check.                                                                       |
| out | <i>pEnabled</i> | User supplied value that will be updated to 1 if identify source is enabled, else 0. |

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

#### 7.58.2.11 int STAR\_API\_CC\_CFG\_getInterfaceModeEnabled ( STAR\_DEVICE\_ID *deviceID*, \_Out\_ int \* *pEnabled* )

Gets whether interface mode has been enabled on a device.

**Parameters**

|     |                 |                                                                                     |
|-----|-----------------|-------------------------------------------------------------------------------------|
|     | <i>deviceID</i> | The device to check.                                                                |
| out | <i>pEnabled</i> | User supplied value that will be updated to 1 if interface mode is enabled, else 0. |

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

#### 7.58.2.12 int STAR\_API\_CC\_CFG\_getInterfaceModeOnPortEnabled ( STAR\_DEVICE\_ID *deviceID*, U8 *portNum*, \_Out\_ int \* *pEnabled* )

Gets whether interface mode is enabled on a specific port.

**Note**

SpaceWire PCI Mk2 and cPCI Mk2 and SpaceWire PCIe devices prior to version 1.07 have a bug which means that the value returned in *pEnabled* is always 0.

**Parameters**

|     |                 |                                                                                     |
|-----|-----------------|-------------------------------------------------------------------------------------|
|     | <i>deviceId</i> | Device to get interface mode enabled for a port on.                                 |
|     | <i>portNum</i>  | Port to get information for.                                                        |
| out | <i>pEnabled</i> | User supplied value that will be updated to 1 if interface mode is enabled, else 0. |

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

**7.58.2.13** int **STAR\_API\_CC\_CFG\_getPortRoutingAddress** ( **STAR\_DEVICE\_ID** *deviceId*, **U8** *portNum*, **\_Out\_ U8 \*** *pAddress* )

Gets the address a packet received on a given port should be routed to when interface mode is enabled.

**Note**

This is an advanced feature and not necessary for normal usage.

SpaceWire PCI Mk2 and cPCI Mk2 and SpaceWire PCIe devices prior to version 1.07 have a bug which means that the value returned in *pAddress* is always 0.

**Parameters**

|     |                 |                                                               |
|-----|-----------------|---------------------------------------------------------------|
|     | <i>deviceId</i> | Device to get port routing address from.                      |
|     | <i>portNum</i>  | Port to get port routing address from.                        |
| out | <i>pAddress</i> | Pointer to value that will be updated to contain the address. |

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

**7.58.2.14** int **STAR\_API\_CC\_CFG\_setPortRoutingAddress** ( **STAR\_DEVICE\_ID** *deviceId*, **U8** *portNum*, **U8** *address* )

Sets the address a packet received on a given port should be routed to when interface mode is enabled.

**Note**

This is an advanced feature and not necessary for normal usage.



**Parameters**

|                 |                                                               |
|-----------------|---------------------------------------------------------------|
| <i>deviceID</i> | Device to have one of its ports port routing address changed. |
| <i>portNum</i>  | Port to have its port routing address modified.               |
| <i>address</i>  | New port routing address.                                     |

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

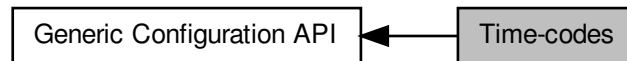
**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

## 7.59 Time-codes

Functions in this group deal with time-codes.

Collaboration diagram for Time-codes:



### Functions

- `int STAR_API_CC_CFG_disableTimeCodeMaster (STAR_DEVICE_ID deviceId)`  
*Disables the device as a time-code master.*
- `int STAR_API_CC_CFG_enableTimeCodeMaster (STAR_DEVICE_ID deviceId)`  
*Enables the device as a time-code master.*
- `int STAR_API_CC_CFG_getTimeCodeMasterEnabled (STAR_DEVICE_ID deviceId, _Out_ int *pEnabled)`  
*Gets whether the device has been enabled as a time-code master.*
- `int STAR_API_CC_CFG_disableExternalTimeCodeSelection (STAR_DEVICE_ID deviceId)`  
*Disables external time-code selection for the device.*
- `int STAR_API_CC_CFG_enableExternalTimeCodeSelection (STAR_DEVICE_ID deviceId)`  
*Enables external time-code selection for the device.*
- `int STAR_API_CC_CFG_getExternalTimeCodeSelectionEnabled (STAR_DEVICE_ID deviceId, _Out_ int *pEnabled)`  
*Gets whether external time-code selection has been enabled for the device.*
- `int STAR_API_CC_CFG_setTimeCodePeriod (STAR_DEVICE_ID deviceId, U32 period)`  
*Sets the period between time-code master ticks.*
- `int STAR_API_CC_CFG_getTimeCodePeriod (STAR_DEVICE_ID deviceId, _Out_ U32 *pPeriod)`  
*Gets the period between time-code master ticks.*
- `int STAR_API_CC_CFG_disableTimeCodeCounterBypassMode (STAR_DEVICE_ID deviceId)`  
*Disables time-code counter bypass mode for the device.*
- `int STAR_API_CC_CFG_enableTimeCodeCounterBypassMode (STAR_DEVICE_ID deviceId)`  
*Enables time-code counter bypass mode for the device.*
- `int STAR_API_CC_CFG_getTimeCodeCounterBypassModeEnabled (STAR_DEVICE_ID deviceId, _Out_ int *pEnabled)`  
*Gets whether time-code counter bypass mode has been enabled for the device.*

### 7.59.1 Detailed Description

Functions in this group deal with time-codes.

## 7.59.2 Function Documentation

### 7.59.2.1 `int STAR_API_CC_CFG_disableExternalTimeCodeSelection ( STAR_DEVICE_ID deviceID )`

Disables external time-code selection for the device.

When external time-code selection is disabled and a time-code is transmitted from an application, the value specified for the time-code is ignored, and the next valid time-code is transmitted by the device.

**Parameters**

---

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <i>deviceID</i> | The device to disable external time-code selection for. |
|-----------------|---------------------------------------------------------|

---

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

**7.59.2.2 int STAR\_API\_CC\_CFG\_disableTimeCodeCounterBypassMode ( STAR\_DEVICE\_ID *deviceID* )**

Disables time-code counter bypass mode for the device.

When time-code counter bypass mode is disabled, any time-code which is received will be checked against the current value of the counter for validity before being forwarded.

**Note**

This function is currently only supported by [SpaceWire PCIe](#) devices.

**Parameters**

---

|                 |                                                          |
|-----------------|----------------------------------------------------------|
| <i>deviceID</i> | The device to disable time-code counter bypass mode for. |
|-----------------|----------------------------------------------------------|

---

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

[SpaceWire PCIe](#)

**7.59.2.3 int STAR\_API\_CC\_CFG\_disableTimeCodeMaster ( STAR\_DEVICE\_ID *deviceID* )**

Disables the device as a time-code master.

**Parameters**

---

|                 |                                              |
|-----------------|----------------------------------------------|
| <i>deviceID</i> | The device to disable as a time-code master. |
|-----------------|----------------------------------------------|

---

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

#### 7.59.2.4 int STAR\_API\_CC\_CFG\_enableExternalTimeCodeSelection ( STAR\_DEVICE\_ID *deviceId* )

Enables external time-code selection for the device.

When external time-code selection is enabled and a time-code is transmitted from an application, the value specified for the time-code is used. Note that the time-code will only be transmitted by the device if it is the next valid time-code.

**Parameters**


---

|                 |                                                        |
|-----------------|--------------------------------------------------------|
| <i>deviceId</i> | The device to enable external time-code selection for. |
|-----------------|--------------------------------------------------------|

---

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

#### 7.59.2.5 int STAR\_API\_CC\_CFG\_enableTimeCodeCounterBypassMode ( STAR\_DEVICE\_ID *deviceId* )

Enables time-code counter bypass mode for the device.

When time-code counter bypass mode is enabled, any time-code which is received will be forwarded out of the other ports on the router, without checking against the current value of the counter. The time-code register will be updated on each received time-code.

**Note**

This function is currently only supported by [SpaceWire PCIe](#) devices.

**Parameters**


---

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <i>deviceId</i> | The device to enable time-code counter bypass mode for. |
|-----------------|---------------------------------------------------------|

---

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

[SpaceWire PCIe](#)

#### 7.59.2.6 int STAR\_API\_CC\_CFG\_enableTimeCodeMaster ( STAR\_DEVICE\_ID *deviceId* )

Enables the device as a time-code master.

**Parameters**

|                 |                                             |
|-----------------|---------------------------------------------|
| <i>deviceID</i> | The device to enable as a time-code master. |
|-----------------|---------------------------------------------|

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

**7.59.2.7** `int STAR_API_CC_CFG_getExternalTimeCodeSelectionEnabled ( STAR_DEVICE_ID deviceID, _Out_ int * pEnabled )`

Gets whether external time-code selection has been enabled for the device.

When external time-code selection is disabled and a time-code is transmitted from an application, the value specified for the time-code is ignored, and the next valid time-code is transmitted by the device. When external time-code selection is enabled and a time-code is transmitted from an application, the value specified for the time-code is used. Note that the time-code will only be transmitted by the device if it is the next valid time-code.

**Parameters**

|            |                 |                                                                                                                  |
|------------|-----------------|------------------------------------------------------------------------------------------------------------------|
|            | <i>deviceID</i> | The device to check.                                                                                             |
| <i>out</i> | <i>pEnabled</i> | User supplied value that will be updated to 1 if external time-code selection is enabled for the device, else 0. |

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

**7.59.2.8** `int STAR_API_CC_CFG_getTimeCodeCounterBypassModeEnabled ( STAR_DEVICE_ID deviceID, _Out_ int * pEnabled )`

Gets whether time-code counter bypass mode has been enabled for the device.

When time-code counter bypass mode is disabled, any time-code which is received will be checked against the current value of the counter for validity before being forwarded. When time-code counter bypass mode is enabled, any time-code which is received will be forwarded out of the other ports on the router, without checking against the current value of the counter. The time-code register will be updated on each received time-code.

**Note**

This function is currently only supported by SpaceWire PCIe devices.

**Parameters**

|     |                 |                                                                                                                   |
|-----|-----------------|-------------------------------------------------------------------------------------------------------------------|
|     | <i>deviceID</i> | The device to check.                                                                                              |
| out | <i>pEnabled</i> | User supplied value that will be updated to 1 if time-code counter bypass mode is enabled for the device, else 0. |

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire PCIe

### 7.59.2.9 int STAR\_API\_CC\_CFG\_getTimeCodeMasterEnabled ( STAR\_DEVICE\_ID *deviceID*, \_Out\_ int \* *pEnabled* )

Gets whether the device has been enabled as a time-code master.

**Parameters**

|     |                 |                                                                                                       |
|-----|-----------------|-------------------------------------------------------------------------------------------------------|
|     | <i>deviceID</i> | The device to check.                                                                                  |
| out | <i>pEnabled</i> | User supplied value that will be updated to 1 if the device is enabled as a time-code master, else 0. |

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

### 7.59.2.10 int STAR\_API\_CC\_CFG\_getTimeCodePeriod ( STAR\_DEVICE\_ID *deviceID*, \_Out\_ U32 \* *pPeriod* )

Gets the period between time-code master ticks.

This is the number of microseconds between time-codes.

**Parameters**

|     |                 |                                                                                    |
|-----|-----------------|------------------------------------------------------------------------------------|
|     | <i>deviceID</i> | Device to get time-code period from                                                |
| out | <i>pPeriod</i>  | A pointer to a variable that will be updated to contain the period in microseconds |

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

**7.59.2.11 int STAR\_API\_CC\_CFG\_setTimeCodePeriod ( STAR\_DEVICE\_ID *deviceId*, U32 *period* )**

Sets the period between time-code master ticks.

This is the number of microseconds between time-codes.

**Parameters**

|                 |                                      |
|-----------------|--------------------------------------|
| <i>deviceId</i> | Device to set time-code period for   |
| <i>period</i>   | The period to be set in microseconds |

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

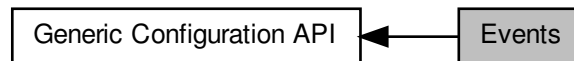
SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3



## 7.60 Events

Functions in this group deal with events.

Collaboration diagram for Events:



### Functions

- `int STAR_API_CC_CFG_disableSpeedChangeEventsOnPort (STAR_DEVICE_ID deviceId, U8 portNum)`  
*Selectively disables speed change events on a given port.*
- `int STAR_API_CC_CFG_enableSpeedChangeEventsOnPort (STAR_DEVICE_ID deviceId, U8 portNum)`  
*Selectively enables speed change events on a given port.*
- `int STAR_API_CC_CFG_getSpeedChangeEventsOnPortEnabled (STAR_DEVICE_ID deviceId, U8 portNum, _Out_ int *pEnabled)`  
*Gets whether speed change events are enabled on a specific port.*
- `int STAR_API_CC_CFG_disableStateChangeEventsOnPort (STAR_DEVICE_ID deviceId, U8 portNum)`  
*Selectively disables state change events on a given port.*
- `int STAR_API_CC_CFG_enableStateChangeEventsOnPort (STAR_DEVICE_ID deviceId, U8 portNum)`  
*Selectively enables state change events on a given port.*
- `int STAR_API_CC_CFG_getStateChangeEventsOnPortEnabled (STAR_DEVICE_ID deviceId, U8 portNum, _Out_ int *pEnabled)`  
*Gets whether state change events are enabled on a specific port.*
- `int STAR_API_CC_CFG_disableTimeCodeEventsOnPort (STAR_DEVICE_ID deviceId, U8 portNum)`  
*Selectively disables time-code notifications on a the channel attached to a given port.*
- `int STAR_API_CC_CFG_enableTimeCodeEventsOnPort (STAR_DEVICE_ID deviceId, U8 portNum)`  
*Selectively enables time-code notifications on a the channel attached to a given port.*
- `int STAR_API_CC_CFG_getTimeCodeEventsOnPortEnabled (STAR_DEVICE_ID deviceId, U8 portNum, _Out_ int *pEnabled)`  
*Gets whether time-code notifications are enabled on a specific port.*

### 7.60.1 Detailed Description

Functions in this group deal with events.

### 7.60.2 Function Documentation

#### 7.60.2.1 `int STAR_API_CC_CFG_disableSpeedChangeEventsOnPort ( STAR_DEVICE_ID deviceId, U8 portNum )`

Selectively disables speed change events on a given port.

Disabling speed change events will result in speed change event traffic no longer being received on channel 0 when the link speed changes.

**Note**

On the [SpaceWire PCIe](#), link speed events are received on the channel associated with the port rather than channel 0.

**Parameters**

|                 |                                                      |
|-----------------|------------------------------------------------------|
| <i>deviceID</i> | Device to disable speed change events for a port on. |
| <i>portNum</i>  | Port to disable speed change events on.              |

**Returns**

0 for success and a negative value on error.

**Added in version:**

[STAR-System v4.00](#) - 19th November 2019

**Supported devices:**

[SpaceWire Router Mk2S](#), [SpaceWire Brick Mk2](#), [SpaceWire Brick Mk3](#), [SpaceWire PCIe](#), [SpaceWire Physical Layer Tester](#) [SpaceWire PXI Interface](#) and [PXI Interface Mk2](#), [SpaceWire PXI 12 Port Router](#) and [PXI 12 Port Router Mk2](#), [STAR Fire Mk3](#)

#### 7.60.2.2 `int STAR_API_CC_CFG_disableStateChangeEventsOnPort ( STAR_DEVICE_ID deviceID, U8 portNum )`

Selectively disables state change events on a given port.

Disabling state change events will result in state change event traffic no longer being received on channel 0 when the link changes to running or disconnected.

**Note**

On the [SpaceWire PCIe](#), state events are received on the channel associated with the port rather than channel 0.

**Parameters**

|                 |                                                      |
|-----------------|------------------------------------------------------|
| <i>deviceID</i> | Device to disable state change events for a port on. |
| <i>portNum</i>  | Port to disable state change events on.              |

**Returns**

0 for success and a negative value on error.

**Added in version:**

[STAR-System v4.00](#) - 19th November 2019

**Supported devices:**

[SpaceWire Router Mk2S](#), [SpaceWire Brick Mk2](#), [SpaceWire Brick Mk3](#), [SpaceWire PCIe](#), [SpaceWire Physical Layer Tester](#) [SpaceWire PXI Interface](#) and [PXI Interface Mk2](#), [SpaceWire PXI 12 Port Router](#) and [PXI 12 Port Router Mk2](#), [STAR Fire Mk3](#)

#### 7.60.2.3 `int STAR_API_CC_CFG_disableTimeCodeEventsOnPort ( STAR_DEVICE_ID deviceID, U8 portNum )`

Selectively disables time-code notifications on a the channel attached to a given port.

**Note**

This function is currently compatible with [SpaceWire PCIe](#) devices and [SpaceWire PXI Interface](#) and [PXI Interface Mk2](#) devices.

**Parameters**

|                 |                                                          |
|-----------------|----------------------------------------------------------|
| <i>deviceID</i> | Device to disable time-code notifications for a port on. |
| <i>portNum</i>  | Port to disable time-code notifications on.              |

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

#### 7.60.2.4 int STAR\_API\_CC\_CFG\_enableSpeedChangeEventsOnPort ( STAR\_DEVICE\_ID *deviceID*, U8 *portNum* )

Selectively enables speed change events on a given port.

Enabling speed change events will result in speed change event traffic being received on channel 0 when the link speed changes.

**Note**

On the [SpaceWire PCIe](#), link speed events are received on the channel associated with the port rather than channel 0.

**Parameters**

|                 |                                                     |
|-----------------|-----------------------------------------------------|
| <i>deviceID</i> | Device to enable speed change events for a port on. |
| <i>portNum</i>  | Port to enable speed change events on.              |

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

#### 7.60.2.5 int STAR\_API\_CC\_CFG\_enableStateChangeEventsOnPort ( STAR\_DEVICE\_ID *deviceID*, U8 *portNum* )

Selectively enables state change events on a given port.

Enabling state change events will result in state change event traffic being received on channel 0 when the link changes to running or disconnected.

**Note**

On the [SpaceWire PCIe](#), state events are received on the channel associated with the port rather than channel 0.

**Parameters**

|                 |                                                     |
|-----------------|-----------------------------------------------------|
| <i>deviceID</i> | Device to enable state change events for a port on. |
| <i>portNum</i>  | Port to enable state change events on.              |

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

#### 7.60.2.6 int STAR\_API\_CC\_CFG\_enableTimeCodeEventsOnPort ( STAR\_DEVICE\_ID *deviceID*, U8 *portNum* )

Selectively enables time-code notifications on a the channel attached to a given port.

**Note**

This function is currently compatible with SpaceWire PCIe devices and SpaceWire PXI Interface and PXI Interface Mk2 devices.

**Parameters**

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <i>deviceID</i> | Device to enable time-code notifications for a port on. |
| <i>portNum</i>  | Port to enable time-code notifications on.              |

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

#### 7.60.2.7 int STAR\_API\_CC\_CFG\_getSpeedChangeEventsOnPortEnabled ( STAR\_DEVICE\_ID *deviceID*, U8 *portNum*, \_Out\_ int \* *pEnabled* )

Gets whether speed change events are enabled on a specific port.

**Parameters**

|                 |                                                                   |
|-----------------|-------------------------------------------------------------------|
| <i>deviceID</i> | Device to get whether speed change events are enabled for a port. |
|-----------------|-------------------------------------------------------------------|

|     |                 |                                                                                           |
|-----|-----------------|-------------------------------------------------------------------------------------------|
|     | <i>portNum</i>  | Port to get information for.                                                              |
| out | <i>pEnabled</i> | User supplied value that will be updated to 1 if speed change events are enabled, else 0. |

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

**7.60.2.8** `int STAR_API_CC_CFG_getStateChangeEventsOnPortEnabled ( STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int * pEnabled )`

Gets whether state change events are enabled on a specific port.

**Parameters**

|     |                 |                                                                                           |
|-----|-----------------|-------------------------------------------------------------------------------------------|
|     | <i>deviceID</i> | Device to get whether state change events are enabled for a port.                         |
|     | <i>portNum</i>  | Port to get information for.                                                              |
| out | <i>pEnabled</i> | User supplied value that will be updated to 1 if state change events are enabled, else 0. |

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

**7.60.2.9** `int STAR_API_CC_CFG_getTimeCodeEventsOnPortEnabled ( STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int * pEnabled )`

Gets whether time-code notifications are enabled on a specific port.

**Note**

This function is currently compatible with [SpaceWire PCIe](#) devices and [SpaceWire PXI Interface and PXI Interface Mk2](#) devices.

**Parameters**

|     |                 |                                                                                                       |
|-----|-----------------|-------------------------------------------------------------------------------------------------------|
|     | <i>deviceID</i> | Device to get whether time-code notifications are enabled for a port.                                 |
|     | <i>portNum</i>  | Port to get information for.                                                                          |
| out | <i>pEnabled</i> | User supplied value that will be updated to 1 if time-code notifications on port are enabled, else 0. |

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

## 7.61 Link Speed

Functions in this group deal with measuring link speed.

Collaboration diagram for Link Speed:



### Functions

- `int STAR_API_CC CFG_getMeasuredLinkSpeed (STAR_DEVICE_ID deviceID, U8 portNum, _Out_ U16 *pLinkSpeed)`  
*Gets the current link speed measured on a specific port.*

#### 7.61.1 Detailed Description

Functions in this group deal with measuring link speed.

#### 7.61.2 Function Documentation

7.61.2.1 `int STAR_API_CC CFG_getMeasuredLinkSpeed ( STAR_DEVICE_ID deviceID, U8 portNum, _Out_ U16 *pLinkSpeed )`

Gets the current link speed measured on a specific port.

##### Parameters

|     |                   |                                                                                                                                                           |
|-----|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
|     | <i>deviceID</i>   | Device to get the measured link speed for a port.                                                                                                         |
|     | <i>portNum</i>    | Port to get information for.                                                                                                                              |
| out | <i>pLinkSpeed</i> | User supplied value that will be updated to contain the measured link speed in 100 Kbit/s units, see <a href="#">STAR_CFG_MK2_LINK_SPEED_UNITS_KBPS</a> . |

##### Returns

0 for success and a negative value on error.

##### Added in version:

STAR-System v4.00 - 19th November 2019

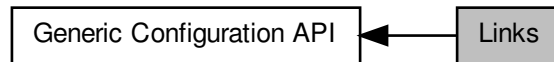
##### Supported devices:

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

## 7.62 Links

Functions in this group deal with controlling link speed.

Collaboration diagram for Links:



### Functions

- `int STAR_API_CC CFG_getTransmitSignallingRate (STAR_DEVICE_ID deviceId, U8 linkNum, _Out_ U32 *pSignallingRate)`  
*Gets the transmit bit rate for a given link.*
- `int STAR_API_CC CFG_setTransmitSignallingRate (STAR_DEVICE_ID deviceId, U8 linkNum, U32 signallingRate)`  
*Sets the transmit bit rate on a given link.*

### 7.62.1 Detailed Description

Functions in this group deal with controlling link speed.

### 7.62.2 Function Documentation

**7.62.2.1** `int STAR_API_CC CFG_getTransmitSignallingRate ( STAR_DEVICE_ID deviceId, U8 linkNum, _Out_ U32 * pSignallingRate )`

Gets the transmit bit rate for a given link.

#### Note

This function is compatible with all devices.

#### Parameters

|     |                        |                                                                                          |
|-----|------------------------|------------------------------------------------------------------------------------------|
|     | <i>deviceId</i>        | Device to get the transmit bit rate from.                                                |
|     | <i>linkNum</i>         | Link to get the transmit bit rate for.                                                   |
| out | <i>pSignallingRate</i> | Pointer to value that will be updated with the given link's transmit bit rate in Mbit/s. |

#### Returns

0 for success and a negative value on error.

Added in version:

STAR-System v4.00 - 19th November 2019



**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

### 7.62.2.2 `int STAR_API_CC_CFG_setTransmitSignallingRate ( STAR_DEVICE_ID deviceID, U8 linkNum, U32 signallingRate )`

Sets the transmit bit rate on a given link.

The transmit bit rate of a link is given in Mbit/s. The bit rate will be between 2 Mbit/s and 400 Mbit/s inclusive, although 400 Mbit/s may not be achievable with every device type.

**Note**

This function is compatible with all devices. However, depending upon the capabilities of the device, an exact bit rate may not be achievable.

For the SpaceWire PXI 12 Port Router device, clocks are shared by pairs of links. For example, setting the transmit bit rate for link 1 will also affect link 2. Similarly, links 3 and 4, 5 and 6, 7 and 8, 9 and 10, and 11 and 12 share clocks. Note that the odd-numbered link must be used to set the transmit clock frequency for each pair of links.

**Parameters**

|                       |                                                 |
|-----------------------|-------------------------------------------------|
| <i>deviceID</i>       | Device to modify a link's transmit bit rate on. |
| <i>linkNum</i>        | Link to have its transmit bit rate modified.    |
| <i>signallingRate</i> | Transmit bit rate in Mbit/s.                    |

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

## 7.63 Precision Links

Functions in this group deal with controlling link speed with greater precision.

Collaboration diagram for Precision Links:



### Functions

- `int STAR_API_CC_CFG_disablePrecisionTransmitRate (STAR_DEVICE_ID deviceId)`  
*Disables precision transmit rate.*
- `int STAR_API_CC_CFG_enablePrecisionTransmitRate (STAR_DEVICE_ID deviceId)`  
*Enables precision transmit rate.*
- `int STAR_API_CC_CFG_getPrecisionTransmitRateEnabled (STAR_DEVICE_ID deviceId, _Out_ int *pEnabled)`  
*Determine if precision transmit rate is enabled.*
- `int STAR_API_CC_CFG_getPrecisionTransmitRateInUse (STAR_DEVICE_ID deviceId, _Out_ int *pInUse)`  
*Determine whether precision transmit rate is in use.*
- `int STAR_API_CC_CFG_getPrecisionTransmitRate (STAR_DEVICE_ID deviceId, _Out_ double *pTransmitRate)`  
*Get the current precision transmit rate on a device in Mbit/s.*
- `int STAR_API_CC_CFG_setPrecisionTransmitRate (STAR_DEVICE_ID deviceId, double transmitRate)`  
*Set the current precision transmit rate on a device in Mbit/s.*

### 7.63.1 Detailed Description

Functions in this group deal with controlling link speed with greater precision.

### 7.63.2 Function Documentation

#### 7.63.2.1 `int STAR_API_CC_CFG_disablePrecisionTransmitRate ( STAR_DEVICE_ID deviceId )`

Disables precision transmit rate.

Precision transmit rate allows the transmit rate of the device to be expressed accurately in Mbit/s as a floating point number using the function `CFG_setPrecisionTransmitRate()`. Note that after being disabled there may be a short delay (less than 250 microseconds) before precision transmit rate is actually disabled. Call `CFG_getPrecisionTransmitRateInUse` to determine if precision transmit rate is no longer in use.

#### Note

This function is currently compatible with [SpaceWire Router Mk2S](#) devices.

**Parameters**

---

|                 |                                               |
|-----------------|-----------------------------------------------|
| <i>deviceID</i> | Device to disable precision transmit rate on. |
|-----------------|-----------------------------------------------|

---

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S

**7.63.2.2 int STAR\_API\_CC CFG\_enablePrecisionTransmitRate ( STAR\_DEVICE\_ID *deviceID* )**

Enables precision transmit rate.

Precision transmit rate allows the transmit rate of the device to be expressed accurately in Mbit/s as a floating point number using the function [CFG\\_setPrecisionTransmitRate\(\)](#). Note that after being enabled it can take up to 250 microseconds before precision transmit rate is actually used. Call [CFG\\_getPrecisionTransmitRateInUse](#) to determine if precision transmit rate is actually in use.

**Note**

This function is currently compatible with [SpaceWire Router Mk2S](#) devices.

**Parameters**

---

|                 |                                              |
|-----------------|----------------------------------------------|
| <i>deviceID</i> | Device to enable precision transmit rate on. |
|-----------------|----------------------------------------------|

---

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S

**7.63.2.3 int STAR\_API\_CC CFG\_getPrecisionTransmitRate ( STAR\_DEVICE\_ID *deviceID*, \_Out\_ double \* *pTransmitRate* )**

Get the current precision transmit rate on a device in Mbit/s.

Precision transmit rate allows the transmit rate of the device to be expressed accurately in Mbit/s as a floating point number using the function [CFG\\_setPrecisionTransmitRate\(\)](#).

**Note**

This function is currently compatible with [SpaceWire Router Mk2S](#) devices.

**Parameters**

|     |                      |                                                                                            |
|-----|----------------------|--------------------------------------------------------------------------------------------|
|     | <i>deviceID</i>      | Device to get the precision transmit rate.                                                 |
| out | <i>pTransmitRate</i> | User supplied value that will be updated to contain the precision transmit rate in Mbit/s. |

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S

#### 7.63.2.4 int STAR\_API\_CC CFG\_getPrecisionTransmitRateEnabled ( STAR\_DEVICE\_ID *deviceID*, \_Out\_ int \* *pEnabled* )

Determine if precision transmit rate is enabled.

Precision transmit rate allows the transmit rate of the device to be expressed accurately in Mbit/s as a floating point number using the function [CFG\\_setPrecisionTransmitRate\(\)](#). Note that although precision transmit rate may be enabled, it can take up to 250 microseconds before precision transmit rate is actually used. Call [CFG\\_getPrecisionTransmitRateInUse](#) to determine if precision transmit rate is actually in use.

**Note**

This function is currently compatible with [SpaceWire Router Mk2S](#) devices.

**Parameters**

|     |                 |                                                                                              |
|-----|-----------------|----------------------------------------------------------------------------------------------|
|     | <i>deviceID</i> | Device to get whether precision transmit rate is enabled.                                    |
| out | <i>pEnabled</i> | User supplied value that will be updated to 1 if precision transmit rate is enabled, else 0. |

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S

#### 7.63.2.5 int STAR\_API\_CC CFG\_getPrecisionTransmitRateInUse ( STAR\_DEVICE\_ID *deviceID*, \_Out\_ int \* *pInUse* )

Determine whether precision transmit rate is in use.

After being enabled or disabled, it can take up to 250 microseconds before precision transmit is actually used or not used. This function determines whether precision transmit rate is currently in use and can be used to determine if an enable or disable operation has completed.

**Note**

This function is currently compatible with [SpaceWire Router Mk2S](#) devices.

**Parameters**

|     |                 |                                                                                             |
|-----|-----------------|---------------------------------------------------------------------------------------------|
|     | <i>deviceID</i> | Device to get whether precision transmit rate is in use.                                    |
| out | <i>pinUse</i>   | User supplied value that will be updated to 1 if precision transmit rate is in use, else 0. |

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S

**7.63.2.6 int STAR\_API\_CC\_CFG\_setPrecisionTransmitRate ( STAR\_DEVICE\_ID *deviceID*, double *transmitRate* )**

Set the current precision transmit rate on a device in Mbit/s.

Precision transmit rate allows the transmit rate of the device to be expressed accurately in Mbit/s as a floating point number.

**Note**

This function is currently compatible with [SpaceWire Router Mk2S](#) devices.

**Parameters**

|  |                     |                                                |
|--|---------------------|------------------------------------------------|
|  | <i>deviceID</i>     | Device to set the precision transmit rate for. |
|  | <i>transmitRate</i> | The precision transmit rate in Mbit/s to set.  |

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

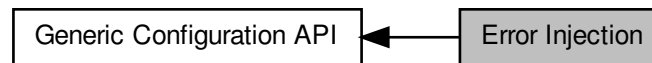
**Supported devices:**

SpaceWire Router Mk2S

## 7.64 Error Injection

Functions in this group deal with error injection.

Collaboration diagram for Error Injection:



### Functions

- `int STAR_API_CC_CFG_injectError (STAR_DEVICE_ID deviceId, U8 portNum, SPW_ERROR error)`  
*Immediately injects the specified error on a given device's port.*
- `int STAR_API_CC_CFG_injectErrors (STAR_DEVICE_ID deviceId, U8 portNum, _In_ STAR_CFG_BRICK_MK2_ERRORS *pErrors)`  
*Inject the specified errors on a given port.*

### 7.64.1 Detailed Description

Functions in this group deal with error injection.

### 7.64.2 Function Documentation

#### 7.64.2.1 `int STAR_API_CC_CFG_injectError ( STAR_DEVICE_ID deviceId, U8 portNum, SPW_ERROR error )`

Immediately injects the specified error on a given device's port.

#### Note

This function is compatible SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire Router Mk2S, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester and SpaceWire PXI Interface and PXI Interface Mk2 devices.

Error injection makes use of the same on-board registers as periodic actions on devices which support it, therefore error injection cannot be used on those devices when periodic actions are in progress.

#### Parameters

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <i>deviceId</i> | Identifier of the device to have the error injected on. |
| <i>portNum</i>  | Port the error is to be injected on.                    |
| <i>error</i>    | SpaceWire error to be injected.                         |

#### Returns

0 for success and a negative value on error.

Added in version:

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

7.64.2.2 `int STAR_API_CC_CFG_injectErrors ( STAR_DEVICE_ID deviceID, U8 portNum, _In_ STAR_CFG_BRICK_MK2_ERRORS * pErrors )`

Inject the specified errors on a given port.

**Note**

This function is compatible with [SpaceWire Brick Mk2](#), [SpaceWire Brick Mk3](#) and [SpaceWire Router Mk2S](#) devices. Please refer to the [Configuration API Functions Supported by Device Types](#) page for details of which errors are supported by each device.

**Parameters**

|           |                 |                                                                  |
|-----------|-----------------|------------------------------------------------------------------|
|           | <i>deviceID</i> | Device to inject the errors on.                                  |
|           | <i>portNum</i>  | Port to inject the errors on.                                    |
| <i>in</i> | <i>pErrors</i>  | Pointer to structure specifying which errors are to be injected. |

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

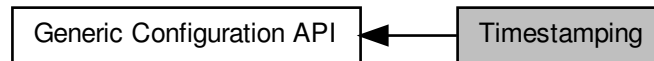
**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

## 7.65 Timestamping

Functions in this group deal with timestampings.

Collaboration diagram for Timestamping:



### Functions

- `int STAR_API_CC_CFG_disableRxTimestampEventsOnPort (STAR_DEVICE_ID deviceId, U8 portNum)`  
*Selectively disables receive timestamp events on a given port.*
- `int STAR_API_CC_CFG_enableRxTimestampEventsOnPort (STAR_DEVICE_ID deviceId, U8 portNum)`  
*Selectively enables receive timestamp events on a given port.*
- `int STAR_API_CC_CFG_getRxTimestampEventsOnPortEnabled (STAR_DEVICE_ID deviceId, U8 portNum, _Out_ int *pEnabled)`  
*Gets whether receive timestamp events are enabled on a specific port.*
- `int STAR_API_CC_CFG_getPulseGeneratorFrequency (STAR_DEVICE_ID deviceId, _Out_ STAR_CFG_BRICK_MK3_PULSE_FREQ *pFrequency)`  
*Gets the frequency of each generated pulse.*
- `int STAR_API_CC_CFG_setPulseGeneratorFrequency (STAR_DEVICE_ID deviceId, STAR_CFG_BRICK_MK3_PULSE_FREQ frequency)`  
*Sets the frequency of each generated pulse.*
- `int STAR_API_CC_CFG_getTimestampMethod (STAR_DEVICE_ID deviceId, _Out_ STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD *pMethod)`  
*Gets the timestamping method that is currently in use for the given device.*
- `int STAR_API_CC_CFG_setTimestampMethod (STAR_DEVICE_ID deviceId, STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD method)`  
*Sets the timestamping method to use for the given device.*
- `int STAR_API_CC_CFG_getTimestampValue (STAR_DEVICE_ID deviceId, _Out_ U32 *pValue)`  
*Gets the current timestamp value.*
- `int STAR_API_CC_CFG_setTimestampValue (STAR_DEVICE_ID deviceId, U32 value)`  
*Sets the current timestamp value for the given device which will be incremented on the next synchronisation pulse.*

### 7.65.1 Detailed Description

Functions in this group deal with timestampings.

### 7.65.2 Function Documentation

#### 7.65.2.1 `int STAR_API_CC_CFG_disableRxTimestampEventsOnPort ( STAR_DEVICE_ID deviceId, U8 portNum )`

Selectively disables receive timestamp events on a given port.

Disabling receive timestamp events will result in no timestamp information being provided after the packet.



**Note**

This function is currently compatible with [SpaceWire Brick Mk3](#) devices and [SpaceWire PXI Interface and PXI Interface Mk2](#) Interface and RMAP devices.

**Parameters**

|                 |                                                           |
|-----------------|-----------------------------------------------------------|
| <i>deviceId</i> | Device to disable receive timestamp events for a port on. |
| <i>portNum</i>  | Port to disable receive timestamp events on.              |

**Returns**

0 for success and a negative value on error.

**Added in version:**

[STAR-System v4.00](#) - 19th November 2019

**Supported devices:**

[SpaceWire Brick Mk3](#), [SpaceWire PXI Interface](#) and [PXI Interface Mk2](#)

### 7.65.2.2 int STAR\_API\_CC\_CFG\_enableRxTimestampEventsOnPort ( STAR\_DEVICE\_ID *deviceId*, U8 *portNum* )

Selectively enables receive timestamp events on a given port.

Enabling receive timestamp events will result in the timestamp that the last packet was received at to be provided as a STAR\_STREAM\_ITEM\_TYPE\_RX\_TIMESTAMP.

**Note**

This function is currently compatible with [SpaceWire Brick Mk3](#) devices and [SpaceWire PXI Interface and PXI Interface Mk2](#) Interface and RMAP devices.

**Parameters**

|                 |                                                          |
|-----------------|----------------------------------------------------------|
| <i>deviceId</i> | Device to enable receive timestamp events for a port on. |
| <i>portNum</i>  | Port to enable receive timestamp events on.              |

**Returns**

0 for success and a negative value on error.

**Added in version:**

[STAR-System v4.00](#) - 19th November 2019

**Supported devices:**

[SpaceWire Brick Mk3](#), [SpaceWire PXI Interface](#) and [PXI Interface Mk2](#)

### 7.65.2.3 int STAR\_API\_CC\_CFG\_getPulseGeneratorFrequency ( STAR\_DEVICE\_ID *deviceId*, \_Out\_ STAR\_CFG\_BRICK\_MK3\_PULSE\_FREQ \* *pFrequency* )

Gets the frequency of each generated pulse.

**Note**

This function is currently compatible with [SpaceWire Brick Mk3](#) devices and [SpaceWire PXI Interface and PXI Interface Mk2](#) Interface and RMAP devices.

**Parameters**

|     |                   |                                                                                    |
|-----|-------------------|------------------------------------------------------------------------------------|
|     | <i>deviceId</i>   | Device to get the pulse generator frequency value for.                             |
| out | <i>pFrequency</i> | Pointer to value that will be updated with the frequency between each pulse cycle. |

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Brick Mk3, SpaceWire PXI Interface and PXI Interface Mk2

7.65.2.4 `int STAR_API_CC_CFG_getRxTimestampEventsOnPortEnabled ( STAR_DEVICE_ID deviceId, U8 portNum, _Out_ int * pEnabled )`

Gets whether receive timestamp events are enabled on a specific port.

**Note**

This function is currently compatible with [SpaceWire Brick Mk3](#) devices and [SpaceWire PXI Interface and PXI Interface Mk2](#) Interface and RMAP devices.

**Parameters**

|     |                 |                                                                                                |
|-----|-----------------|------------------------------------------------------------------------------------------------|
|     | <i>deviceId</i> | Device to get whether receive timestamp events are enabled for a port.                         |
|     | <i>portNum</i>  | Port to get receive timestamp events for.                                                      |
| out | <i>pEnabled</i> | User supplied value that will be updated to 1 if receive timestamp events are enabled, else 0. |

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Brick Mk3, SpaceWire PXI Interface and PXI Interface Mk2

7.65.2.5 `int STAR_API_CC_CFG_getTimestampMethod ( STAR_DEVICE_ID deviceId, _Out_ STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD * pMethod )`

Gets the timestamping method that is currently in use for the given device.

**Note**

This function is currently compatible with [SpaceWire Brick Mk3](#) devices and [SpaceWire PXI Interface and PXI Interface Mk2](#) Interface and RMAP devices.

**Parameters**

|     |                 |                                                                               |
|-----|-----------------|-------------------------------------------------------------------------------|
|     | <i>deviceID</i> | Device to get the timestamping method for.                                    |
| out | <i>pMethod</i>  | Pointer to value that will be updated with the given link's timestamp method. |

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Brick Mk3, SpaceWire PXI Interface and PXI Interface Mk2

### 7.65.2.6 int STAR\_API\_CC\_CFG\_getTimestampValue ( STAR\_DEVICE\_ID *deviceID*, \_Out\_ U32 \* *pValue* )

Gets the current timestamp value.

**Note**

This function is currently compatible with [SpaceWire Brick Mk3](#) devices and [SpaceWire PXI Interface and PXI Interface Mk2](#) Interface and RMAP devices.

**Parameters**

|     |                 |                                                                              |
|-----|-----------------|------------------------------------------------------------------------------|
|     | <i>deviceID</i> | Device to get the timestamp value for.                                       |
| out | <i>pValue</i>   | Pointer to value that will be updated with the given link's timestamp value. |

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Brick Mk3, SpaceWire PXI Interface and PXI Interface Mk2

### 7.65.2.7 int STAR\_API\_CC\_CFG\_setPulseGeneratorFrequency ( STAR\_DEVICE\_ID *deviceID*, STAR\_CFG\_BRICK\_MK3\_PULSE\_FREQ *frequency* )

Sets the frequency of each generated pulse.

**Note**

This function is currently compatible with [SpaceWire Brick Mk3](#) devices and [SpaceWire PXI Interface and PXI Interface Mk2](#) Interface and RMAP devices.

**Parameters**

|                  |                                                  |
|------------------|--------------------------------------------------|
| <i>deviceID</i>  | Device to set the pulse generator frequency for. |
| <i>frequency</i> | The frequency value to set.                      |

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Brick Mk3, SpaceWire PXI Interface and PXI Interface Mk2

**7.65.2.8** `int STAR_API_CC_CFG_setTimestampMethod ( STAR_DEVICE_ID deviceID,  
STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD method )`

Sets the timestamping method to use for the given device.

**Note**

This function is currently compatible with [SpaceWire Brick Mk3](#) devices and [SpaceWire PXI Interface and PXI Interface Mk2](#) Interface and RMAP devices.

The Brick Mk3 will trigger on the rising edge of the external pulse but the Triggering API can be used to change it to trigger on the falling edge using [TRIGGER\\_BRICK\\_MK3\\_enableExtTriggerEdgeDetectMode\(\)](#) and [TRIGGER\\_BRICK\\_MK3\\_enableExtTriggerInvert\(\)](#).

**Parameters**

|                 |                                            |
|-----------------|--------------------------------------------|
| <i>deviceID</i> | Device to set the timestamping method for. |
| <i>method</i>   | Timestamp method to enable for device.     |

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Brick Mk3, SpaceWire PXI Interface and PXI Interface Mk2

**7.65.2.9** `int STAR_API_CC_CFG_setTimestampValue ( STAR_DEVICE_ID deviceID, U32 value )`

Sets the current timestamp value for the given device which will be incremented on the next synchronisation pulse.

**Note**

This function is currently compatible with [SpaceWire Brick Mk3](#) devices and [SpaceWire PXI Interface and PXI Interface Mk2](#) Interface and RMAP devices.

**Parameters**

---

|                 |                                        |
|-----------------|----------------------------------------|
| <i>deviceID</i> | Device to set the timestamp value for. |
| <i>value</i>    | Timestamp value to set for device.     |

---

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

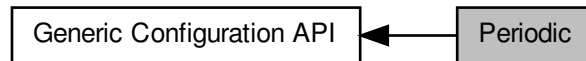
**Supported devices:**

SpaceWire Brick Mk3, SpaceWire PXI Interface and PXI Interface Mk2

## 7.66 Periodic

Functions in this group deal with periodic functions.

Collaboration diagram for Periodic:



### Functions

- `int STAR_API_CC_CFG_getDeviceClockRate (STAR_DEVICE_ID deviceId)`  
*Gets a device's internal clock rate.*
- `int STAR_API_CC_CFG_startPeriodicAction (STAR_DEVICE_ID deviceId, unsigned char port, U32 count, SPW_ACTION action)`  
*Starts a periodic action on a given device's port.*
- `int STAR_API_CC_CFG_stopPeriodicAction (STAR_DEVICE_ID deviceId, unsigned char port)`  
*Stops a periodic action on a given device's port.*

### 7.66.1 Detailed Description

Functions in this group deal with periodic functions.

### 7.66.2 Function Documentation

#### 7.66.2.1 `int STAR_API_CC_CFG_getDeviceClockRate ( STAR_DEVICE_ID deviceId )`

Gets a device's internal clock rate.

#### Note

This function is currently only compatible with [SpaceWire PCI Mk2](#) and [cPCI Mk2](#) devices.

#### Parameters

|                 |                                    |
|-----------------|------------------------------------|
| <i>deviceId</i> | Identifier of the device to query. |
|-----------------|------------------------------------|

#### Returns

The device's clock rate in Hz, or zero if the device does not support periodic actions.

#### Added in version:

[STAR-System v4.00](#) - 19th November 2019

#### Supported devices:

[SpaceWire PCI Mk2](#) and [cPCI Mk2](#)

**7.66.2.2** `int STAR_API_CC CFG_startPeriodicAction ( STAR_DEVICE_ID deviceID, unsigned char port, U32 count, SPW_ACTION action )`

Starts a periodic action on a given device's port.

To stop a periodic action use `CFG_stopPeriodicAction()`.

#### Note

This function is currently only compatible with [SpaceWire PCI Mk2](#) and [cPCI Mk2](#) devices.

Periodic actions make use of the same on-board registers as error injection, therefore error injection cannot be used when periodic actions are in progress.

#### Parameters

|                 |                                                                                                                                               |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| <i>deviceID</i> | Identifier of the device to have periodic action created for.                                                                                 |
| <i>port</i>     | Port the periodic action is to be performed on.                                                                                               |
| <i>count</i>    | Number of device clock cycles between action repetitions. See <a href="#">CFG_getDeviceClockRate()</a> for obtaining the device's clock rate. |
| <i>action</i>   | Action to be performed. This can be <a href="#">SPW_TRANSMIT_PACKET</a> or any of the <a href="#">SPW_ERR↔OR</a> values.                      |

#### Returns

0 for success and a negative value on error.

#### Added in version:

[STAR-System v4.00](#) - 19th November 2019

#### Supported devices:

[SpaceWire PCI Mk2](#) and [cPCI Mk2](#)

**7.66.2.3** `int STAR_API_CC CFG_stopPeriodicAction ( STAR_DEVICE_ID deviceID, unsigned char port )`

Stops a periodic action on a given device's port.

Periodic actions are started by calling `CFG_startPeriodicAction()`.

#### Note

This function is currently only compatible with [SpaceWire PCI Mk2](#) and [cPCI Mk2](#) devices.

#### Parameters

|                 |                                                          |
|-----------------|----------------------------------------------------------|
| <i>deviceID</i> | Identifier of the device performing the periodic action. |
| <i>port</i>     | Port the periodic action is being performed on.          |

#### Returns

0 for success and a negative value on error.

#### Added in version:

[STAR-System v4.00](#) - 19th November 2019

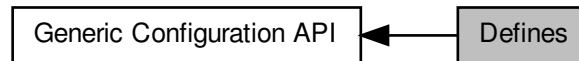
#### Supported devices:

[SpaceWire PCI Mk2](#) and [cPCI Mk2](#)

## 7.67 Defines

Function return error values are defined here.

Collaboration diagram for Defines:



### Macros

- `#define CFG_STATUS_SUCCESS (0)`  
*Positive retvals are success, negative retvals are errors.*
- `#define CFG_SUCCESS(status) ((status) >= CFG_STATUS_SUCCESS ? 1 : 0)`  
*Macro to allow testing for success or failure.*
- `#define CFG_STATUS_FAILURE (-256)`  
*The API call has failed.*
- `#define CFG_STATUS_NOT_COMPATIBLE_WITH_THIS_DEVICE_TYPE (-257)`  
*The API call is not compatible with this device type.*
- `#define CFG_STATUS_NOT_COMPATIBLE_WITH_THIS_DEVICES_FPGA_VERSION (-258)`  
*The API call is not compatible with this device's FPGA version.*
- `#define CFG_STATUS_GET_DEVICE_INFO_FAILURE (-259)`  
*Failed to get "Device Information" for this device.*
- `#define CFG_STATUS_TIME_CODE_DISTRIBUTION_PORT_MASK_IS_INVALID (-260)`  
*The Time-code distribution port mask is invalid.*
- `#define CFG_STATUS_TIME_CODE_FLAG_MODE_IS_INVALID (-261)`  
*The Time-code flag mode is invalid.*
- `#define CFG_STATUS_TIME_CODE_PERIOD_IS_INVALID (-262)`  
*The Time-code period is invalid.*
- `#define CFG_STATUS_TIME_CODE_PORT_IS_INVALID (-263)`  
*The Time-code port is invalid.*
- `#define CFG_STATUS_IDENTIFY_SOURCE_PORT_IS_INVALID (-264)`  
*The Identify Source port is invalid.*
- `#define CFG_STATUS_INTERFACE_MODE_PORT_IS_INVALID (-265)`  
*The Interface Mode port is invalid.*
- `#define CFG_STATUS_PORT_IS_INVALID (-266)`  
*The port is invalid.*
- `#define CFG_STATUS_LOGICAL_ADDRESS_IS_INVALID (-267)`  
*The logical address is invalid.*
- `#define CFG_STATUS_SPACEWIRE_LINK_IS_INVALID (-268)`  
*The SpaceWire link is invalid.*
- `#define CFG_STATUS_MEASURED_SPEED_PORT_IS_INVALID (-269)`  
*The measured port speed is invalid.*
- `#define CFG_STATUS_SPEED_CHANGE_EVENTS_PORT_IS_INVALID (-270)`  
*The Speed Change Events port is invalid.*



- `#define CFG_STATUS_STATE_CHANGE_EVENTS_PORT_IS_INVALID (-271)`  
*The State Change Events port is invalid.*
- `#define CFG_STATUS_PORT_ROUTING_PORT_IS_INVALID (-272)`  
*The Port Routing port is invalid.*
- `#define CFG_STATUS_INJECT_ERROR_PORT_IS_INVALID (-273)`  
*The Inject Error port is invalid.*
- `#define CFG_STATUS_PRECISION_TRANSMIT_RATE_IS_INVALID (-274)`  
*The Precision Transmit Rate port is invalid.*
- `#define CFG_STATUS_PERIODIC_ACTION_PORT_IS_INVALID (-275)`  
*The Periodic Action port is invalid.*
- `#define CFG_STATUS_TIMESTAMP_EVENTS_PORT_IS_INVALID (-276)`  
*The Timestamp Events port is invalid.*
- `#define CFG_STATUS_CLOCK_LINK_IS_INVALID (-277)`  
*The Clock link is invalid.*
- `#define CFG_STATUS_LINK_RATE_DIVIDER_IS_INVALID (-278)`  
*The Link Rate divider is invalid.*
- `#define CFG_STATUS_TRANSMIT_CLOCK_LINK_IS_INVALID (-279)`  
*The Transmit Clock link is invalid.*
- `#define CFG_STATUS_BIT_RATE_IS_INVALID (-280)`  
*The Bit-rate is invalid.*
- `#define CFG_STATUS_ROUTER_TIMEOUT_MODE_IS_INVALID (-281)`  
*The Router Timeout mode is invalid.*
- `#define CFG_STATUS_ROUTER_TIMEOUT_PERIOD_IS_INVALID (-282)`  
*The Router Timeout period is invalid.*
- `#define CFG_STATUS_ROUTER_DISABLE_ON_SILENCE_IS_INVALID (-283)`  
*The Router Disable-on-Silence is invalid.*
- `#define CFG_STATUS_ROUTER_START_ON_REQUEST_IS_INVALID (-284)`  
*The Router Start-on-Request is invalid.*
- `#define CFG_STATUS_ROUTER_ENABLE_SELF_ADDRESSING_IS_INVALID (-285)`  
*The Router Enable-Self-Addressing is invalid.*
- `#define CFG_STATUS_GROUP_ADAPTIVE_ROUTING_PORT_MASK_IS_INVALID (-286)`  
*The Group Adaptive Routing port mask is invalid.*
- `#define CFG_STATUS_GROUP_ADAPTIVE_ROUTING_PRIORITY_IS_INVALID (-287)`  
*The Group Adaptive Routing priority is invalid.*
- `#define CFG_STATUS_GROUP_ADAPTIVE_ROUTING_DELETE_HEADER_IS_INVALID (-288)`  
*The Group Adaptive Routing delete header is invalid.*
- `#define CFG_STATUS_GROUP_ADAPTIVE_ROUTING_ADDRESS_IS_INVALID (-289)`  
*The Group Adaptive Routing address is invalid.*
- `#define CFG_STATUS_SPACEWIRE_LINK_TRISTATE_IS_INVALID (-290)`  
*The SpaceWire link tri-state flag is invalid.*
- `#define CFG_STATUS_SPACEWIRE_LINK_DISABLE_IS_INVALID (-291)`  
*The SpaceWire link disable flag is invalid.*
- `#define CFG_STATUS_SPACEWIRE_LINK_START_IS_INVALID (-292)`  
*The SpaceWire link start flag is invalid.*
- `#define CFG_STATUS_SPACEWIRE_LINK_AUTOSTART_IS_INVALID (-293)`  
*The SpaceWire link autostart flag is invalid.*
- `#define CFG_STATUS_SPACEWIRE_LINK_RUNNING_IS_INVALID (-294)`  
*The SpaceWire link running flag is invalid.*
- `#define CFG_STATUS_SPACEWIRE_LINK_STATE_IS_INVALID (-295)`  
*The SpaceWire link state is invalid.*
- `#define CFG_STATUS_PORT_ROUTING_ADDRESS_IS_INVALID (-296)`

- The Port Routing address is invalid.*

  - `#define CFG_STATUS_INJECT_ERROR_ERROR_IS_INVALID (-297)`
- The Inject Error error is invalid.*

  - `#define CFG_STATUS_INJECT_ERRORS_PARITY_ERROR_IS_INVALID (-298)`
- The Inject Errors parity error is invalid.*

  - `#define CFG_STATUS_INJECT_ERRORS_ESCAPE_ERROR_IS_INVALID (-299)`
- The Inject Errors escape error is invalid.*

  - `#define CFG_STATUS_INJECT_ERRORS_INSERT_FCT_ERROR_IS_INVALID (-300)`
- The Inject Errors insert FCT error is invalid.*

  - `#define CFG_STATUS_INJECT_ERRORS_SUPPRESS_FCT_ERROR_IS_INVALID (-301)`
- The Inject Errors suppress FCT error is invalid.*

  - `#define CFG_STATUS_INJECT_ERRORS_INCREMENT_CREDIT_ERROR_IS_INVALID (-302)`
- The Inject Errors increment credit error is invalid.*

  - `#define CFG_STATUS_INJECT_ERRORS_DECREMENT_CREDIT_ERROR_IS_INVALID (-303)`
- The Inject Errors decrement credit error is invalid.*

  - `#define CFG_STATUS_INJECT_ERRORS_DISCONNECT_ERROR_IS_INVALID (-304)`
- The Inject Errors disconnect error is invalid.*

  - `#define CFG_STATUS_PERIODIC_ACTION_ACTION_IS_INVALID (-305)`
- The Periodic Action is invalid.*

  - `#define CFG_STATUS_TIMESTAMP_METHOD_IS_INVALID (-306)`
- The Timestamp method is invalid.*

  - `#define CFG_STATUS_PULSE_GENERATOR_FREQUENCY_IS_INVALID (-307)`
- The Pulse Generator frequency is invalid.*

  - `#define CFG_STATUS_CLOCK_RATE_MULTIPLIER_IS_INVALID (-308)`
- The Clock Rate multiplier is invalid.*

  - `#define CFG_STATUS_CLOCK_RATE_DIVISOR_IS_INVALID (-309)`
- The Clock Rate divisor is invalid.*

  - `#define CFG_STATUS_CLOCK_RATE_PARAMETERS_ARE_INVALID (-310)`
- The Clock Rate parameters are invalid.*

  - `#define CFG_STATUS_CANT_GET_CLOCK_RATE_PARAMETERS_FROM_BIT_RATE (-311)`
- Can't get Clock Rate parameters from bit-rate.*

  - `#define CFG_STATUS_POINTER_PARAMETER_IS_NULL (-312)`
- The pointer parameter is NULL.*

  - `#define CFG_STATUS_TRANSMIT_BIT_RATE_IS_INVALID (-313)`
- The Transmit bit-rate is invalid.*

## 7.67.1 Detailed Description

Function return error values are defined here.

## 7.67.2 Macro Definition Documentation

### 7.67.2.1 `#define CFG_STATUS_NOT_COMPATIBLE_WITH_THIS_DEVICES_FPGA_VERSION (-258)`

The API call is not compatible with this device's FPGA version.

The FPGA can be updated to allow this additional functionality.

### 7.67.2.2 `#define CFG_STATUS_SUCCESS (0)`

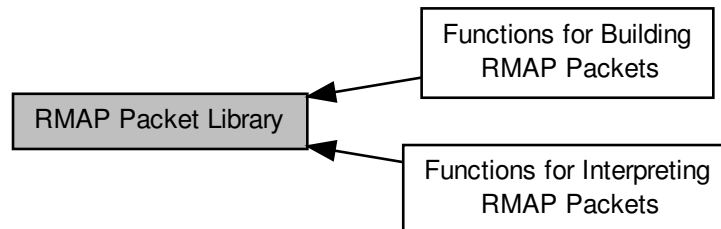
Positive retvals are success, negative retvals are errors.

Values > 0 should be interpreted on a per function basis.

## 7.68 RMAP Packet Library

The SpaceWire Remote Memory Access Protocol (RMAP) is a protocol designed to be used over SpaceWire for a number of purposes, including the configuration of devices and networks.

Collaboration diagram for RMAP Packet Library:



### Modules

- [Functions for Building RMAP Packets](#)  
*These functions can be used to build the various types of RMAP packets.*
- [Functions for Interpreting RMAP Packets](#)  
*These functions can be used to determine if an RMAP packet is valid, and the fields in that RMAP packet.*

### Data Structures

- [struct RMAP\\_PACKET](#)  
*A structure used to describe an RMAP packet. [More...](#)*

### Macros

- [#define RMAPPACKETLIBRARY\\_CC](#)  
*The calling convention used by functions exported by the RMAP Packet Library.*
- [#define RMAP\\_PROTOCOL\\_IDENTIFIER 1](#)  
*The value used in the protocol identifier field of all RMAP packets.*
- [#define PNP\\_PROTOCOL\\_IDENTIFIER 3](#)  
*The value used in the protocol identifier field of all SpaceWire-PnP packets.*
- [#define RMAP\\_RESERVED\\_BIT 0x80](#)  
*A mask for the reserved bit of the Instruction field.*
- [#define RMAP\\_COMMAND\\_BIT 0x40](#)  
*A packet is a command if the bit specified by this mask is set in the Instruction field.*
- [#define RMAP\\_WRITE\\_OPERATION\\_BIT 0x20](#)  
*A packet is a write operation if the bit specified by this mask is set in the Instruction field.*
- [#define RMAP\\_VERIFY\\_BEFORE\\_WRITE\\_BIT 0x10](#)  
*An operation verifies before writing if the bit specified by this mask is set in the Instruction field.*
- [#define RMAP\\_REPLY\\_BIT 0x08](#)  
*An operation is acknowledged if the bit specified by this mask is set in the Instruction field.*

- `#define RMAP_INCREMENT_ADDRESS_BIT 0x04`  
*A read or write address is incremented if the bit specified by this mask is set in the Instruction field.*
- `#define RMAP_REPLY_ADDRESS_LENGTH_BITS 0x03`  
*A mask for the bits in the Instruction field containing the length of the reply address.*
- `#define RMAP_GET_VERSION_MAJOR(versionInfo) (((version) & 0xff000000) >> 24)`  
*Return the major version number from the specified version information.*
- `#define RMAP_GET_VERSION_MINOR(versionInfo) (((version) & 0x00ff0000) >> 16)`  
*Return the minor version number from the specified version information.*
- `#define RMAP_GET_VERSION_EDIT(versionInfo) (((version) & 0x0000ffc0) >> 6)`  
*Return the edit number from the specified version information.*
- `#define RMAP_GET_VERSION_PATCH(versionInfo) ((version) & 0x0000003f)`  
*Return the patch level from the specified version information.*

## Enumerations

- `enum RMAP_STATUS {`  
`RMAP_SUCCESS = 0x00, RMAP_GENERAL_ERROR = 0x01, RMAP_UNUSED_PACKET_TYPE_OR_↵`  
`COMMAND_CODE = 0x02, RMAP_INVALID_KEY = 0x03,`  
`RMAP_INVALID_DATA_CRC = 0x04, RMAP_EARLY_EOP = 0x05, RMAP_TOO_MUCH_DATA = 0x06,`  
`RMAP_EEP = 0x07,`  
`RMAP_VERIFY_BUFFER_OVERRUN = 0x09, RMAP_COMMAND_NOT_IMPLEMENTED_OR_AUTHO↵`  
`RISED = 0x0a, RMAP_RMW_DATA_LENGTH_ERROR = 0x0b, RMAP_INVALID_TARGET_LOGICAL_A↵`  
`DDRESS = 0x0c,`  
`RMAP_INVALID_STATUS = 0xff }`  
*Possible values for the status of RMAP operations.*
- `enum RMAP_PACKET_TYPE {`  
`RMAP_WRITE_COMMAND = (RMAP_COMMAND_BIT | RMAP_WRITE_OPERATION_BIT), RMAP_WR↵`  
`ITE_REPLY = (RMAP_WRITE_OPERATION_BIT), RMAP_READ_COMMAND = (RMAP_COMMAND_BIT),`  
`RMAP_READ_REPLY = (0),`  
`RMAP_READ_MODIFY_WRITE_COMMAND = (RMAP_COMMAND_BIT | RMAP_VERIFY_BEFORE_W↵`  
`RITE_BIT), RMAP_READ_MODIFY_WRITE_REPLY = (RMAP_VERIFY_BEFORE_WRITE_BIT), RMAP↵`  
`_INVALID_PACKET_TYPE = (0xff) }`  
*Possible values for the packet type.*

## Functions

- `U32 RMAPPACKETLIBRARY_CC RMAP_GetVersion (void)`  
*Return the current version information for the RMAP Packet Library.*
- `U8 RMAPPACKETLIBRARY_CC RMAP_CalculateCRC (void *pBuffer, unsigned long len)`  
*Calculate an 8-bit CRC for the given buffer.*
- `U8 RMAPPACKETLIBRARY_CC RMAP_CalculateCRCWithSeed (void *pBuffer, unsigned long len, U8 crc)`  
*Calculate an 8-bit CRC for the given buffer, starting with a seed CRC.*
- `char RMAPPACKETLIBRARY_CC RMAP_IsCRCValid (void *pBuffer, unsigned long len, U8 crc)`  
*Determine if the specified 8-bit CRC is valid for the given buffer.*
- `void RMAPPACKETLIBRARY_CC RMAP_FreeBuffer (void *pBuffer)`  
*Free a previously allocated buffer containing a packet created using one of the RMAP\_Build\* or RMAP\_Fill\* functions.*

### 7.68.1 Detailed Description

The SpaceWire Remote Memory Access Protocol (RMAP) is a protocol designed to be used over SpaceWire for a number of purposes, including the configuration of devices and networks.

More information on SpaceWire and RMAP can be found at the ESA SpaceWire Webpage at <http://spacewire.esa.int>. The STAR-Dundee RMAP Packet Library provides an API which can be used to build RMAP packets, check the validity of RMAP packets, and to read the fields of an RMAP packet.

### 7.68.2 Functionality Provided

The STAR-Dundee RMAP Packet Library provides functions which can be called to create RMAP packets, to check the format of RMAP packets and to determine the values of fields in RMAP packets. It does not send or receive packets using a SpaceWire device; this work must be done by the programmer as this is application and device specific.

Functions are provided to build read, write and read/modify/write commands and replies. Each packet built using the functions which allocate a buffer for the packet must be freed using the [RMAP\\_FreeBuffer](#) function. Functions are also provided to fill previously allocated buffers with specific RMAP packets. Functions are also provided to determine the length of the potential packet so that the amount of memory to allocate is known.

The format of RMAP packets can be checked using the [RMAP\\_CheckPacketValid](#) function, which confirms that the fields have valid values, and that the length of the packet is correct. When building packets or checking a packet's format, the functions can create a structure which can be used in the other functions provided by the library. These functions can be used to determine the values of individual fields within the RMAP packet, such as the packet type and the data.

### 7.68.3 Supported Platforms

The RMAP Packet Library is currently available for Windows (2000, XP, Vista, Windows 7 and Windows 8), Linux (v2.6 and 3 kernels for x86-32 and x86-64), QNX, VxWorks and RTEMS platforms, along with a native ELF version for the LEON processors. As no access to SpaceWire hardware is made, the library can be used with any SpaceWire device.

If you require the library for a different operating system or processor, please contact STAR-Dundee using the contact information in the [Contact Information](#) section.

A Java version of the RMAP Packet Library (described in the [Java Library](#) section) is also available for Windows and Linux.

### 7.68.4 Using The Library

The RMAP Packet Library is provided in the form of two or three files:

- [rmap\\_packet\\_library.h](#)
- [rmap\\_packet\\_library.lib](#) (Windows), [librmap\\_packet\\_library.so](#) (Linux, QNX) or [librmap\\_packet\\_library.a](#) (VxWorks and LEON)
- [rmap\\_packet\\_library.dll](#) (Windows only)

When writing programs which call functions provided by the RMAP Packet Library, the [rmap\\_packet\\_library.h](#) file must be included. This file includes "star\_dundee\_types.h", which must also be provided. To compile programs to use the library "librmap\_packet\_library.so", or "rmap\_packet\_library.lib" on Windows, must be linked in. When running programs which use the library, on Windows "rmap\_packet\_library.dll" must either be in the "Windows\System32" directory of the machine in use, or in the directory from which the program was run. On Linux "librmap\_packet\_library.so" must be present in "/usr/lib".

### 7.68.5 Java Library

The RMAP Packet Library is also provided with the SpaceWire USB and PCI Drivers for Windows and Linux Driver as a jar file, to allow Java code to be written to use the library. Further information on this can be found in the help files provided in the "spacewire\_java" directory under the driver's install directory (see "index.html"). To compile a Java program using the library, a classpath should be specified which includes "RMAPLibrary.jar". To run an application which uses the library, the jar file should also be in the classpath.

### 7.68.6 Byte Alignment

The functions in the library which can be used to build packets take an alignment argument. Normally this should be set to 1, but if the device being used to send the RMAP packet can only send packets which are a multiple of four bytes, for example, then this alignment parameter can be set to 4. This will result in the packet being built containing extra 0s in the packet's path address, prior to the logical address, to make the packet a multiple of four bytes in length. Other alignment values can also be used. Note that this feature is not part of the RMAP standard but is implemented in a number of SpaceWire devices, including the ESA SpaceWire Router. Use an alignment of 1 to maintain full compliance to the standard.

### 7.68.7 Contact Information

If you have any comments or queries regarding the RMAP Packet Library or documentation please contact:

STAR-Dundee

STAR House

166 Nethergate

Dundee, DD1 4EE

Scotland, UK

E-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Further support information and details of all STAR-Dundee products can be found at <http://www.star-dundee.com>.

### 7.68.8 Data Structure Documentation

#### 7.68.8.1 struct RMAP\_PACKET

A structure used to describe an RMAP packet.

This structure should not be accessed directly, but should be populated with values using one of the RMAP\_Build\* or RMAP\_Fill\* functions. The fields should then be accessed using the RMAP\_Get\* functions.

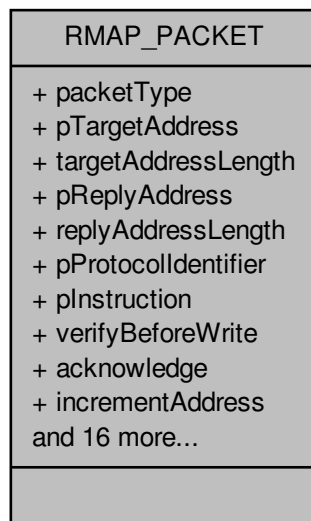
Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

[check\\_packet\\_example.c](#).

Collaboration diagram for RMAP\_PACKET:



#### Data Fields

|                         |                                     |                                                                                                                                                                                                                                                                 |
|-------------------------|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| char                    | acknowledge                         | Whether acknowledgement is enabled in the instruction byte in the packet.                                                                                                                                                                                       |
| U32                     | dataLength                          | The length of the data field in the packet, if present, otherwise 0. If the packet is read/modify/write command packet, then this value will be different from the data length field in the packet, which is the total length of both the data and mask fields. |
| unsigned long           | headerLength                        | The length of the header.                                                                                                                                                                                                                                       |
| char                    | increment↔<br>Address               | Whether incrementing of memory addresses is enabled in the instruction byte in the packet.                                                                                                                                                                      |
| U8                      | maskLength                          | The length of the mask field in the packet, if present, otherwise 0. This is half of the data length field in a read/modify write command packet.                                                                                                               |
| RMAP_PACKET↔<br>ET_TYPE | packetType                          | The type of the RMAP packet.                                                                                                                                                                                                                                    |
| U8 *                    | pData                               | A pointer to the data field in the packet, if present, otherwise NULL.                                                                                                                                                                                          |
| U8 *                    | pDataCRC                            | A pointer to the data CRC byte in the packet, if present, otherwise NULL.                                                                                                                                                                                       |
| void *                  | pDataLength                         | A pointer to the three byte data length field in the packet, if present, otherwise NULL.                                                                                                                                                                        |
| U8 *                    | pExtended↔<br>ReadWrite↔<br>Address | A pointer to the extended read or write memory address byte in the packet, if present, otherwise NULL.                                                                                                                                                          |
| U8 *                    | pHeader                             | A pointer to the header in the packet, starting from the last byte in the address at the start of the packet.                                                                                                                                                   |
| U8 *                    | pHeaderCRC                          | A pointer to the header CRC byte in the packet, or NULL if the field could not be identified.                                                                                                                                                                   |

|               |                             |                                                                                                                     |
|---------------|-----------------------------|---------------------------------------------------------------------------------------------------------------------|
| U8 *          | pInstruction                | A pointer to the instruction byte in the packet, or NULL if the field could not be identified.                      |
| U8 *          | pKey                        | A pointer to the key byte in the packet, if present, otherwise NULL.                                                |
| U8 *          | pMask                       | A pointer to the mask field in the packet, if present, otherwise NULL.                                              |
| U8 *          | pProtocol↔<br>Identifier    | A pointer to the protocol identifier byte in the packet, or NULL if the field could not be identified.              |
| U8 *          | pRawPacket                  | A pointer to the raw packet.                                                                                        |
| U32 *         | pReadWrite↔<br>Address      | A pointer to the four byte read or write memory address field in the packet, if present, otherwise NULL.            |
| U8 *          | pReplyAddress               | A pointer to the reply address in the packet, or NULL if the reply address could not be identified.                 |
| U8 *          | pStatus                     | A pointer to the status byte in the packet, if present, otherwise NULL.                                             |
| U8 *          | pTargetAddress              | A pointer to the target address in the packet, or NULL if the target address could not be identified.               |
| U16 *         | pTransaction↔<br>Identifier | A pointer to the two byte transaction identifier field in the packet, or NULL if the field could not be identified. |
| unsigned long | rawPacket↔<br>Length        | The length of the raw packet.                                                                                       |
| unsigned long | replyAddress↔<br>Length     | The length of the reply address in the packet.                                                                      |
| unsigned long | targetAddress↔<br>Length    | The length of the target address in the packet.                                                                     |
| char          | verifyBefore↔<br>Write      | Whether verify before write is enabled in the instruction byte in the packet.                                       |

## 7.68.9 Macro Definition Documentation

### 7.68.9.1 #define RMAP\_COMMAND\_BIT 0x40

A packet is a command if the bit specified by this mask is set in the Instruction field.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

### 7.68.9.2 #define RMAP\_GET\_VERSION\_EDIT( *versionInfo* ) (((version) & 0x0000ffc0) >> 6)

Return the edit number from the specified version information.

The version information can be obtained from a call to [RMAP\\_GetVersion](#), which returns a value with the edit number in bits 6 to 15.

#### Parameters

---

|                    |                                                                               |
|--------------------|-------------------------------------------------------------------------------|
| <i>versionInfo</i> | the version information returned by a call to <a href="#">RMAP_GetVersion</a> |
|--------------------|-------------------------------------------------------------------------------|

---

#### Returns

the edit number

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

[version\\_example.c](#).



**7.68.9.3** `#define RMAP_GET_VERSION_MAJOR( versionInfo ) (((version) & 0xff000000) >> 24)`

Return the major version number from the specified version information.

The version information can be obtained from a call to [RMAP\\_GetVersion](#), which returns a value with the major version number in the most significant 8 bits, bits 24 to 31.

**Parameters**

---

|                    |                                                                               |
|--------------------|-------------------------------------------------------------------------------|
| <i>versionInfo</i> | the version information returned by a call to <a href="#">RMAP_GetVersion</a> |
|--------------------|-------------------------------------------------------------------------------|

---

**Returns**

the major version number

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

[version\\_example.c](#).

**7.68.9.4** `#define RMAP_GET_VERSION_MINOR( versionInfo ) (((version) & 0x00ff0000) >> 16)`

Return the minor version number from the specified version information.

The version information can be obtained from a call to [RMAP\\_GetVersion](#), which returns a value with the minor version number in bits 16 to 23.

**Parameters**

---

|                    |                                                                               |
|--------------------|-------------------------------------------------------------------------------|
| <i>versionInfo</i> | the version information returned by a call to <a href="#">RMAP_GetVersion</a> |
|--------------------|-------------------------------------------------------------------------------|

---

**Returns**

the minor version number

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

[version\\_example.c](#).

**7.68.9.5** `#define RMAP_GET_VERSION_PATCH( versionInfo ) ((version) & 0x0000003f)`

Return the patch level from the specified version information.

The version information can be obtained from a call to [RMAP\\_GetVersion](#), which returns a value with the patch level in bits 0 to 5. The patch level should be 0 in a release version of the RMAP Packet Library.

**Parameters**

---

|                    |                                                                               |
|--------------------|-------------------------------------------------------------------------------|
| <i>versionInfo</i> | the version information returned by a call to <a href="#">RMAP_GetVersion</a> |
|--------------------|-------------------------------------------------------------------------------|

---

**Returns**

the patch level

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

`version_example.c.`

**7.68.9.6 #define RMAP\_INCREMENT\_ADDRESS\_BIT 0x04**

A read or write address is incremented if the bit specified by this mask is set in the Instruction field.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**7.68.9.7 #define RMAP\_PROTOCOL\_IDENTIFIER 1**

The value used in the protocol identifier field of all RMAP packets.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**7.68.9.8 #define RMAP\_REPLY\_ADDRESS\_LENGTH\_BITS 0x03**

A mask for the bits in the Instruction field containing the length of the reply address.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**7.68.9.9 #define RMAP\_REPLY\_BIT 0x08**

An operation is acknowledged if the bit specified by this mask is set in the Instruction field.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**7.68.9.10 #define RMAP\_RESERVED\_BIT 0x80**

A mask for the reserved bit of the Instruction field.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

#### 7.68.9.11 `#define RMAP_VERIFY_BEFORE_WRITE_BIT 0x10`

An operation verifies before writing if the bit specified by this mask is set in the Instruction field.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

#### 7.68.9.12 `#define RMAP_WRITE_OPERATION_BIT 0x20`

A packet is a write operation if the bit specified by this mask is set in the Instruction field.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

#### 7.68.9.13 `#define RMAPPACKETLIBRARY_CC`

The calling convention used by functions exported by the RMAP Packet Library.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

### 7.68.10 Enumeration Type Documentation

#### 7.68.10.1 `enum RMAP_PACKET_TYPE`

Possible values for the packet type.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Enumerator

***RMAP\_WRITE\_COMMAND*** The write command packet type.

***RMAP\_WRITE\_REPLY*** The write reply packet type.

***RMAP\_READ\_COMMAND*** The read command packet type.

***RMAP\_READ\_REPLY*** The read reply packet type.

***RMAP\_READ\_MODIFY\_WRITE\_COMMAND*** The read/modify/write command packet type.

***RMAP\_READ\_MODIFY\_WRITE\_REPLY*** The read/modify/write reply packet type.

***RMAP\_INVALID\_PACKET\_TYPE*** The packet type used when a valid packet type cannot be determined.

#### 7.68.10.2 `enum RMAP_STATUS`

Possible values for the status of RMAP operations.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Enumerator

***RMAP\_SUCCESS*** The status value indicating success.

***RMAP\_GENERAL\_ERROR*** The status value indicating that the detected error does not fit into the other error cases.

***RMAP\_UNUSED\_PACKET\_TYPE\_OR\_COMMAND\_CODE*** The status value indicating the packet type is reserved or the command is not used by the RMAP protocol.

***RMAP\_INVALID\_KEY*** The status value indicating the key did not match that expected by the target user application.

***RMAP\_INVALID\_DATA\_CRC*** The status value indicating there was an error in the data CRC.

***RMAP\_EARLY\_EOP*** The status value indicating an EOP was detected before the end of the data.

***RMAP\_TOO\_MUCH\_DATA*** The status value indicating there was more data than was expected.

***RMAP\_EEP*** The status value indicating that an EEP was encountered in the packet after the header.

***RMAP\_VERIFY\_BUFFER\_OVERRUN*** The status value indicating that verify before write was enabled in the command but not enough buffer space was available to receive the full command.

***RMAP\_COMMAND\_NOT\_IMPLEMENTED\_OR\_AUTHORISED*** The status value indicating the target user application did not authorise the requested operation.

***RMAP\_RMW\_DATA\_LENGTH\_ERROR*** The status value indicating the amount of data in a read/modify/write command is invalid.

***RMAP\_INVALID\_TARGET\_LOGICAL\_ADDRESS*** The status value indicating the target logical address was not the value expected by the target.

***RMAP\_INVALID\_STATUS*** The status value indicating that an invalid status value was encountered, or the status could not be determined. Note that this is not a standard RMAP error.

## 7.68.11 Function Documentation

### 7.68.11.1 U8 RMAPPACKETLIBRARY\_CC RMAP\_CalculateCRC ( void \* *pBuffer*, unsigned long *len* )

Calculate an 8-bit CRC for the given buffer.

#### Parameters

|                |                                     |
|----------------|-------------------------------------|
| <i>pBuffer</i> | the buffer to calculate the CRC for |
| <i>len</i>     | the length of the buffer            |

#### Returns

the 8-bit CRC for the specified buffer

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

`crc_example.c`.

### 7.68.11.2 U8 RMAPPACKETLIBRARY\_CC RMAP\_CalculateCRCWithSeed ( void \* *pBuffer*, unsigned long *len*, U8 *crc* )

Calculate an 8-bit CRC for the given buffer, starting with a seed CRC.

#### Parameters

|                |                                                  |
|----------------|--------------------------------------------------|
| <i>pBuffer</i> | the buffer to calculate the CRC for              |
| <i>len</i>     | the length of the buffer                         |
| <i>crc</i>     | the seed CRC to be used when calculating the CRC |

**Returns**

the 8-bit CRC for the specified buffer

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

`crc_example.c`.

**7.68.11.3 void RMAPPACKETLIBRARY\_CC RMAP\_FreeBuffer ( void \* *pBuffer* )**

Free a previously allocated buffer containing a packet created using one of the RMAP\_Build\* or RMAP\_Fill\* functions.

**Parameters**

|                |                        |
|----------------|------------------------|
| <i>pBuffer</i> | the buffer to be freed |
|----------------|------------------------|

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

`check_packet_example.c`, `read_command_packet_example.c`, `read_reply_packet_example.c`, `rmap_target_↵_example.c`, `rmw_command_packet_example.c`, `rmw_reply_packet_example.c`, `write_command_packet_↵_example.c`, and `write_reply_packet_example.c`.

**7.68.11.4 U32 RMAPPACKETLIBRARY\_CC RMAP\_GetVersion ( void )**

Return the current version information for the RMAP Packet Library.

This can be used with the RMAP\_GET\_VERSION\_MAJOR, RMAP\_GET\_VERSION\_MINOR, RMAP\_GET\_VE↵RSION\_EDIT and RMAP\_GET\_VERSION\_PATCH macros to obtain the version number in a useable form.

**Returns**

the version information for the RMAP Packet Library stored in a 32-bit number. The most significant 8 bits will contain the major version number, the next significant 8 bits the minor version number, the next significant 10 bits the edit number, and the 6 least significant bits the patch level. In a release version of the library, the patch level should be 0

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

`version_example.c`.

**7.68.11.5 char RMAPPACKETLIBRARY\_CC RMAP\_IsCRCValid ( void \* *pBuffer*, unsigned long *len*, U8 *crc* )**

Determine if the specified 8-bit CRC is valid for the given buffer.

**Parameters**

---

|                |                                 |
|----------------|---------------------------------|
| <i>pBuffer</i> | the buffer to check the CRC for |
| <i>len</i>     | the length of the buffer        |
| <i>crc</i>     | the CRC to check against        |

---

**Returns**

1 if the CRC is correct for the buffer, otherwise 0

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

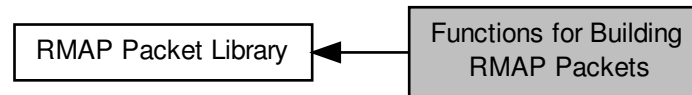
**Examples:**

`crc_example.c`.

## 7.69 Functions for Building RMAP Packets

These functions can be used to build the various types of RMAP packets.

Collaboration diagram for Functions for Building RMAP Packets:



### Functions

- unsigned long [RMAPPACKETLIBRARY\\_CC RMAP\\_CalculateWriteCommandPacketLength](#) (unsigned long targetAddressLength, unsigned long replyAddressLength, U32 dataLength, char alignment)  
*Calculate the number of bytes required for a write command packet, given the properties of the packet.*
- char [RMAPPACKETLIBRARY\\_CC RMAP\\_FillWriteCommandPacket](#) (U8 \*pTargetAddress, unsigned long targetAddressLength, U8 \*pReplyAddress, unsigned long replyAddressLength, char verifyBeforeWrite, char acknowledge, char incrementAddress, U8 key, U16 transactionIdentifier, U32 writeAddress, U8 extendedWriteAddress, U8 \*pData, U32 dataLength, unsigned long \*pRawPacketLength, [RMAP\\_PACKET](#) \*pPacketStruct, char alignment, U8 \*pRawPacket, unsigned long rawPacketLength)  
*Fill a previously allocated buffer with an RMAP write command packet which can be sent to perform an RMAP write command.*
- void [\\*RMAPPACKETLIBRARY\\_CC RMAP\\_BuildWriteCommandPacket](#) (U8 \*pTargetAddress, unsigned long targetAddressLength, U8 \*pReplyAddress, unsigned long replyAddressLength, char verifyBeforeWrite, char acknowledge, char incrementAddress, U8 key, U16 transactionIdentifier, U32 writeAddress, U8 extendedWriteAddress, U8 \*pData, U32 dataLength, unsigned long \*pRawPacketLength, [RMAP\\_PACKET](#) \*pPacketStruct, char alignment)  
*Build an RMAP write command packet which can be sent to perform an RMAP write command.*
- void [\\*RMAPPACKETLIBRARY\\_CC RMAP\\_BuildWriteRegisterPacket](#) (U8 \*pTargetAddress, unsigned long targetAddressLength, U8 \*pReplyAddress, unsigned long replyAddressLength, char verifyBeforeWrite, char acknowledge, char incrementAddress, U8 key, U16 transactionIdentifier, U32 writeAddress, U8 extendedWriteAddress, U32 registerValue, unsigned long \*pRawPacketLength, [RMAP\\_PACKET](#) \*pPacketStruct, char alignment)  
*Build an RMAP write command packet which can be sent to perform an RMAP write command on a 4-byte register.*
- unsigned long [RMAPPACKETLIBRARY\\_CC RMAP\\_CalculateWriteReplyPacketLength](#) (unsigned long initiatorAddressLength, char alignment)  
*Calculate the number of bytes required for a write reply packet, given the properties of the packet.*
- char [RMAPPACKETLIBRARY\\_CC RMAP\\_FillWriteReplyPacket](#) (U8 \*pInitiatorAddress, unsigned long initiatorAddressLength, U8 targetAddress, char verifyBeforeWrite, char incrementAddress, [RMAP\\_STATUS](#) status, U16 transactionIdentifier, unsigned long \*pRawPacketLength, [RMAP\\_PACKET](#) \*pPacketStruct, char alignment, U8 \*pRawPacket, unsigned long rawPacketLength)  
*Fill a previously allocated buffer with an RMAP write reply packet which can be sent to respond to an RMAP write command.*
- void [\\*RMAPPACKETLIBRARY\\_CC RMAP\\_BuildWriteReplyPacket](#) (U8 \*pInitiatorAddress, unsigned long initiatorAddressLength, U8 targetAddress, char verifyBeforeWrite, char incrementAddress, [RMAP\\_STATUS](#) status, U16 transactionIdentifier, unsigned long \*pRawPacketLength, [RMAP\\_PACKET](#) \*pPacketStruct, char alignment)  
*Build an RMAP write reply packet which can be sent to respond to an RMAP write command.*

- unsigned long [RMAPPACKETLIBRARY\\_CC RMAP\\_CalculateReadCommandPacketLength](#) (unsigned long targetAddressLength, unsigned long replyAddressLength, char alignment)

*Calculate the number of bytes required for a read command packet, given the properties of the packet.*

- char [RMAPPACKETLIBRARY\\_CC RMAP\\_FillReadCommandPacket](#) (U8 \*pTargetAddress, unsigned long targetAddressLength, U8 \*pReplyAddress, unsigned long replyAddressLength, char incrementAddress, U8 key, U16 transactionIdentifier, U32 readAddress, U8 extendedReadAddress, U32 dataLength, unsigned long \*pRawPacketLength, [RMAP\\_PACKET](#) \*pPacketStruct, char alignment, U8 \*pRawPacket, unsigned long rawPacketLength)

*Fill a previously allocated buffer with an RMAP read command packet which can be sent to perform an RMAP read command.*

- void [\\*RMAPPACKETLIBRARY\\_CC RMAP\\_BuildReadCommandPacket](#) (U8 \*pTargetAddress, unsigned long targetAddressLength, U8 \*pReplyAddress, unsigned long replyAddressLength, char incrementAddress, U8 key, U16 transactionIdentifier, U32 readAddress, U8 extendedReadAddress, U32 dataLength, unsigned long \*pRawPacketLength, [RMAP\\_PACKET](#) \*pPacketStruct, char alignment)

*Build an RMAP read command packet which can be sent to perform an RMAP read command.*

- void [\\*RMAPPACKETLIBRARY\\_CC RMAP\\_BuildReadRegisterPacket](#) (U8 \*pTargetAddress, unsigned long targetAddressLength, U8 \*pReplyAddress, unsigned long replyAddressLength, char incrementAddress, U8 key, U16 transactionIdentifier, U32 readAddress, U8 extendedReadAddress, unsigned long \*pRawPacketLength, [RMAP\\_PACKET](#) \*pPacketStruct, char alignment)

*Build an RMAP read command packet which can be sent to perform an RMAP read command on a 4-byte register.*

- unsigned long [RMAPPACKETLIBRARY\\_CC RMAP\\_CalculateReadReplyPacketLength](#) (unsigned long initiatorAddressLength, U32 dataLength, char alignment)

*Calculate the number of bytes required for a read reply packet, given the properties of the packet.*

- char [RMAPPACKETLIBRARY\\_CC RMAP\\_FillReadReplyPacket](#) (U8 \*pInitiatorAddress, unsigned long initiatorAddressLength, U8 targetAddress, char incrementAddress, [RMAP\\_STATUS](#) status, U16 transactionIdentifier, U8 \*pData, U32 dataLength, unsigned long \*pRawPacketLength, [RMAP\\_PACKET](#) \*pPacketStruct, char alignment, U8 \*pRawPacket, unsigned long rawPacketLength)

*Fill a previously allocated buffer with an RMAP read reply packet which can be sent to respond to an RMAP read command.*

- void [\\*RMAPPACKETLIBRARY\\_CC RMAP\\_BuildReadReplyPacket](#) (U8 \*pInitiatorAddress, unsigned long initiatorAddressLength, U8 targetAddress, char incrementAddress, [RMAP\\_STATUS](#) status, U16 transactionIdentifier, U8 \*pData, U32 dataLength, unsigned long \*pRawPacketLength, [RMAP\\_PACKET](#) \*pPacketStruct, char alignment)

*Build an RMAP read reply packet which can be sent to respond to an RMAP read command.*

- unsigned long [RMAPPACKETLIBRARY\\_CC RMAP\\_CalculateReadModifyWriteCommandPacketLength](#) (unsigned long targetAddressLength, unsigned long replyAddressLength, U32 dataAndMaskLength, char alignment)

*Calculate the number of bytes required for a read/modify/write command packet, given the properties of the packet.*

- char [RMAPPACKETLIBRARY\\_CC RMAP\\_FillReadModifyWriteCommandPacket](#) (U8 \*pTargetAddress, unsigned long targetAddressLength, U8 \*pReplyAddress, unsigned long replyAddressLength, U8 key, U16 transactionIdentifier, U32 readModifyWriteAddress, U8 extendedReadModifyWriteAddress, U8 dataAndMaskLength, U8 \*pData, U8 \*pMask, unsigned long \*pRawPacketLength, [RMAP\\_PACKET](#) \*pPacketStruct, char alignment, U8 \*pRawPacket, unsigned long rawPacketLength)

*Fill a previously allocated buffer with an RMAP read/modify/write command packet which can be sent to perform an RMAP read/modify/write command.*

- void [\\*RMAPPACKETLIBRARY\\_CC RMAP\\_BuildReadModifyWriteCommandPacket](#) (U8 \*pTargetAddress, unsigned long targetAddressLength, U8 \*pReplyAddress, unsigned long replyAddressLength, U8 key, U16 transactionIdentifier, U32 readModifyWriteAddress, U8 extendedReadModifyWriteAddress, U8 dataAndMaskLength, U8 \*pData, U8 \*pMask, unsigned long \*pRawPacketLength, [RMAP\\_PACKET](#) \*pPacketStruct, char alignment)

*Build an RMAP read/modify/write command packet which can be sent to perform an RMAP read/modify/write command.*

- void [\\*RMAPPACKETLIBRARY\\_CC RMAP\\_BuildReadModifyWriteRegisterPacket](#) (U8 \*pTargetAddress, unsigned long targetAddressLength, U8 \*pReplyAddress, unsigned long replyAddressLength, U8 key, U16 transactionIdentifier, U32 readModifyWriteAddress, U8 extendedReadModifyWriteAddress, U32 registerValue, U32 mask, unsigned long \*pRawPacketLength, [RMAP\\_PACKET](#) \*pPacketStruct, char alignment)



*Build an RMAP read/modify/write command packet which can be sent to perform an RMAP read/modify/write command on a 4 byte register.*

- unsigned long [RMAPPACKETLIBRARY\\_CC](#) [RMAP\\_CalculateReadModifyWriteReplyPacketLength](#) (unsigned long initiatorAddressLength, U32 dataLength, char alignment)

*Calculate the number of bytes required for a read/modify/write reply packet, given the properties of the packet.*

- char [RMAPPACKETLIBRARY\\_CC](#) [RMAP\\_FillReadModifyWriteReplyPacket](#) (U8 \*pInitiatorAddress, unsigned long initiatorAddressLength, U8 targetAddress, [RMAP\\_STATUS](#) status, U16 transactionIdentifier, unsigned long dataLength, U8 \*pData, unsigned long \*pRawPacketLength, [RMAP\\_PACKET](#) \*pPacketStruct, char alignment, U8 \*pRawPacket, unsigned long rawPacketLength)

*Fill a previously allocated buffer with an RMAP read/modify/write reply packet which can be sent to respond to an RMAP read/modify/write command.*

- void \*[RMAPPACKETLIBRARY\\_CC](#) [RMAP\\_BuildReadModifyWriteReplyPacket](#) (U8 \*pInitiatorAddress, unsigned long initiatorAddressLength, U8 targetAddress, [RMAP\\_STATUS](#) status, U16 transactionIdentifier, unsigned long dataLength, U8 \*pData, unsigned long \*pRawPacketLength, [RMAP\\_PACKET](#) \*pPacketStruct, char alignment)

*Build an RMAP read/modify/write reply packet which can be sent to respond to an RMAP read/modify/write command.*

- void [RMAPPACKETLIBRARY\\_CC](#) [RMAP\\_SetProtocolIdentifier](#) ([RMAP\\_PACKET](#) \*pPacketStruct, U8 protocolIdentifier)

*Set the protocol identifier of a previously created packet, updating the packet's header CRC so the packet is still valid.*

### 7.69.1 Detailed Description

These functions can be used to build the various types of RMAP packets.

These functions are useful when transmitting RMAP packets.

### 7.69.2 Function Documentation

- 7.69.2.1 void\* [RMAPPACKETLIBRARY\\_CC](#) [RMAP\\_BuildReadCommandPacket](#) ( U8 \* pTargetAddress, unsigned long targetAddressLength, U8 \* pReplyAddress, unsigned long replyAddressLength, char incrementAddress, U8 key, U16 transactionIdentifier, U32 readAddress, U8 extendedReadAddress, U32 dataLength, unsigned long \* pRawPacketLength, [RMAP\\_PACKET](#) \* pPacketStruct, char alignment )

Build an RMAP read command packet which can be sent to perform an RMAP read command.

Note that this function performs a memory allocation, use [RMAP\\_FillReadCommandPacket](#) to build a packet using previously allocated memory. When the buffer allocated by this function is no longer required, it should be freed by calling [RMAP\\_FreeBuffer](#).

#### Parameters

|                              |                                                                                                                                                        |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pTargetAddress</i>        | a pointer to the SpaceWire path or logical address of the device to be read from                                                                       |
| <i>targetAddressLength</i>   | the length of the target address                                                                                                                       |
| <i>pReplyAddress</i>         | a pointer to the SpaceWire path or logical address that will be used to send any responses back to the device from which the command will be sent from |
| <i>replyAddressLength</i>    | the length of the reply address                                                                                                                        |
| <i>incrementAddress</i>      | whether or not the target read address should be incremented when reading                                                                              |
| <i>key</i>                   | the key expected by the destination device                                                                                                             |
| <i>transactionIdentifier</i> | an identifier for the transaction                                                                                                                      |

|                            |                                                                                                                                                                                                                                                                                                   |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>readAddress</i>         | the memory address at the destination to read from                                                                                                                                                                                                                                                |
| <i>extendedReadAddress</i> | the extended memory address at the destination to read from                                                                                                                                                                                                                                       |
| <i>dataLength</i>          | the length of data to read, which is a 24-bit number and so should be at most 0xfffff                                                                                                                                                                                                             |
| <i>pRawPacketLength</i>    | a pointer to a previously allocated variable which will be updated to contain the length of the packet returned                                                                                                                                                                                   |
| <i>pPacketStruct</i>       | an optional structure which will be updated to contain details of the fields in the packet. This structure should not be accessed directly, but by using other functions provided by the API. Alternatively the parameter can be left as NULL if the properties of the packet are not of interest |
| <i>alignment</i>           | the word size used by the device sending the packet, normally 1. See <a href="#">Byte Alignment</a> for more information                                                                                                                                                                          |

### Returns

the packet generated, or NULL if there was an error, for example if one of the fields is invalid. The buffer returned should be freed by calling [RMAP\\_FreeBuffer](#)

### Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

### Examples:

[read\\_command\\_packet\\_example.c](#).

**7.69.2.2** void\* **RMAPPACKETLIBRARY\_CC** RMAP\_BuildReadModifyWriteCommandPacket ( U8 \* *pTargetAddress*, unsigned long *targetAddressLength*, U8 \* *pReplyAddress*, unsigned long *replyAddressLength*, U8 *key*, U16 *transactionIdentifier*, U32 *readModifyWriteAddress*, U8 *extendedReadModifyWriteAddress*, U8 *dataAndMaskLength*, U8 \* *pData*, U8 \* *pMask*, unsigned long \* *pRawPacketLength*, RMAP\_PACKET \* *pPacketStruct*, char *alignment* )

Build an RMAP read/modify/write command packet which can be sent to perform an RMAP read/modify/write command.

Note that this function performs a memory allocation, use [RMAP\\_FillReadModifyWriteCommandPacket](#) to build a packet using previously allocated memory. When the buffer allocated by this function is no longer required, it should be freed by calling [RMAP\\_FreeBuffer](#).

### Parameters

|                                       |                                                                                                                                                        |
|---------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pTargetAddress</i>                 | a pointer to the SpaceWire path or logical address of the device to be written to                                                                      |
| <i>targetAddressLength</i>            | the length of the target address                                                                                                                       |
| <i>pReplyAddress</i>                  | a pointer to the SpaceWire path or logical address that will be used to send any responses back to the device from which the command will be sent from |
| <i>replyAddressLength</i>             | the length of the reply address                                                                                                                        |
| <i>key</i>                            | the key expected by the destination device                                                                                                             |
| <i>transactionIdentifier</i>          | an identifier for the transaction                                                                                                                      |
| <i>readModifyWriteAddress</i>         | the memory address at the destination to read from and write to                                                                                        |
| <i>extendedReadModifyWriteAddress</i> | the extended memory address at the destination to read from and write to                                                                               |

|                                |                                                                                                                                                                                                                                                                                                    |
|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>dataAndMask↔<br/>Length</i> | the combined length in bytes of the data and mask fields, this should be an even number between 0 and 8                                                                                                                                                                                            |
| <i>pData</i>                   | the data to be written                                                                                                                                                                                                                                                                             |
| <i>pMask</i>                   | the mask to be applied to the data                                                                                                                                                                                                                                                                 |
| <i>pRawPacket↔<br/>Length</i>  | a pointer to a previously allocated variable which will be updated to contain the length of the packet returned                                                                                                                                                                                    |
| <i>pPacketStruct</i>           | an optional structure which will be updated to contain details of the fields in the packet. This structure should not be accessed directly, but by using other functions provided by the A↔PI. Alternatively the parameter can be left as NULL if the properties of the packet are not of interest |
| <i>alignment</i>               | the word size used by the device sending the packet, normally 1. See <a href="#">Byte Alignment</a> for more information                                                                                                                                                                           |

#### Returns

the packet generated, or NULL if there was an error, for example if one of the fields is invalid. The buffer returned should be freed by calling [RMAP\\_FreeBuffer](#)

#### Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

#### Examples:

[rmw\\_command\\_packet\\_example.c](#).

**7.69.2.3** void\* **RMAPPACKETLIBRARY\_CC** RMAP\_BuildReadModifyWriteRegisterPacket ( U8 \* *pTargetAddress*, unsigned long *targetAddressLength*, U8 \* *pReplyAddress*, unsigned long *replyAddressLength*, U8 *key*, U16 *transactionIdentifier*, U32 *readModifyWriteAddress*, U8 *extendedReadModifyWriteAddress*, U32 *registerValue*, U32 *mask*, unsigned long \* *pRawPacketLength*, RMAP\_PACKET \* *pPacketStruct*, char *alignment* )

Build an RMAP read/modify/write command packet which can be sent to perform an RMAP read/modify/write command on a 4 byte register.

Note that this function performs a memory allocation, use [RMAP\\_FillReadModifyWriteCommandPacket](#) to build a packet using previously allocated memory. When the buffer allocated by this function is no longer required, it should be freed by calling [RMAP\\_FreeBuffer](#).

#### Parameters

|                                                   |                                                                                                                                                        |
|---------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pTargetAddress</i>                             | a pointer to the SpaceWire path or logical address of the device to be written to                                                                      |
| <i>targetAddress↔<br/>Length</i>                  | the length of the target address                                                                                                                       |
| <i>pReplyAddress</i>                              | a pointer to the SpaceWire path or logical address that will be used to send any responses back to the device from which the command will be sent from |
| <i>replyAddress↔<br/>Length</i>                   | the length of the reply address                                                                                                                        |
| <i>key</i>                                        | the key expected by the destination device                                                                                                             |
| <i>transaction↔<br/>Identifier</i>                | an identifier for the transaction                                                                                                                      |
| <i>readModify↔<br/>WriteAddress</i>               | the memory address at the destination to read from and write to                                                                                        |
| <i>extendedRead↔<br/>ModifyWrite↔<br/>Address</i> | the extended memory address at the destination to read from and write to                                                                               |

|                         |                                                                                                                                                                                                                                                                                                    |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>registerValue</i>    | the value to be written to the register                                                                                                                                                                                                                                                            |
| <i>mask</i>             | the mask to be applied to the data                                                                                                                                                                                                                                                                 |
| <i>pRawPacketLength</i> | a pointer to a previously allocated variable which will be updated to contain the length of the packet returned                                                                                                                                                                                    |
| <i>pPacketStruct</i>    | an optional structure which will be updated to contain details of the fields in the packet. This structure should not be accessed directly, but by using other functions provided by the A↔PI. Alternatively the parameter can be left as NULL if the properties of the packet are not of interest |
| <i>alignment</i>        | the word size used by the device sending the packet, normally 1. See <a href="#">Byte Alignment</a> for more information                                                                                                                                                                           |

#### Returns

the packet generated, or NULL if there was an error, for example if one of the fields is invalid. The buffer returned should be freed by calling [RMAP\\_FreeBuffer](#)

#### Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

#### Examples:

[rmw\\_command\\_packet\\_example.c](#).

**7.69.2.4** void\* **RMAPPACKETLIBRARY\_CC** RMAP\_BuildReadModifyWriteReplyPacket ( U8 \* *pInitiatorAddress*, unsigned long *initiatorAddressLength*, U8 *targetAddress*, RMAP\_STATUS *status*, U16 *transactionIdentifier*, unsigned long *dataLength*, U8 \* *pData*, unsigned long \* *pRawPacketLength*, RMAP\_PACKET \* *pPacketStruct*, char *alignment* )

Build an RMAP read/modify/write reply packet which can be sent to respond to an RMAP read/modify/write command.

Note that this function performs a memory allocation, use [RMAP\\_FillReadModifyWriteReplyPacket](#) to build a packet using previously allocated memory. When the buffer allocated by this function is no longer required, it should be freed by calling [RMAP\\_FreeBuffer](#).

#### Parameters

|                               |                                                                                                                                                                                                                                                                                                    |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pInitiatorAddress</i>      | a pointer to the SpaceWire path or logical address of the device to which the reply should be sent                                                                                                                                                                                                 |
| <i>initiatorAddressLength</i> | the length of the initiator address                                                                                                                                                                                                                                                                |
| <i>targetAddress</i>          | the SpaceWire logical address of the device that sent the command being responded to                                                                                                                                                                                                               |
| <i>status</i>                 | the status of the read operation                                                                                                                                                                                                                                                                   |
| <i>transactionIdentifier</i>  | an identifier for the transaction                                                                                                                                                                                                                                                                  |
| <i>dataLength</i>             | the length in bytes of the data field, this can be at most 4 bytes                                                                                                                                                                                                                                 |
| <i>pData</i>                  | the data read                                                                                                                                                                                                                                                                                      |
| <i>pRawPacketLength</i>       | a pointer to a previously allocated variable which will be updated to contain the length of the packet returned                                                                                                                                                                                    |
| <i>pPacketStruct</i>          | an optional structure which will be updated to contain details of the fields in the packet. This structure should not be accessed directly, but by using other functions provided by the A↔PI. Alternatively the parameter can be left as NULL if the properties of the packet are not of interest |

|                  |                                                                                                                          |
|------------------|--------------------------------------------------------------------------------------------------------------------------|
| <i>alignment</i> | the word size used by the device sending the packet, normally 1. See <a href="#">Byte Alignment</a> for more information |
|------------------|--------------------------------------------------------------------------------------------------------------------------|

**Returns**

the packet generated, or NULL if there was an error, for example if one of the fields is invalid. The buffer returned should be freed by calling [RMAP\\_FreeBuffer](#)

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

[rmw\\_reply\\_packet\\_example.c](#).

**7.69.2.5** void\* **RMAPPACKETLIBRARY\_CC** [RMAP\\_BuildReadRegisterPacket](#) ( U8 \* *pTargetAddress*, unsigned long *targetAddressLength*, U8 \* *pReplyAddress*, unsigned long *replyAddressLength*, char *incrementAddress*, U8 *key*, U16 *transactionIdentifier*, U32 *readAddress*, U8 *extendedReadAddress*, unsigned long \* *pRawPacketLength*, **RMAP\_PACKET** \* *pPacketStruct*, char *alignment* )

Build an RMAP read command packet which can be sent to perform an RMAP read command on a 4-byte register.

Note that this function performs a memory allocation, use [RMAP\\_FillReadCommandPacket](#) to build a packet using previously allocated memory. When the buffer allocated by this function is no longer required, it should be freed by calling [RMAP\\_FreeBuffer](#).

**Parameters**

|                                    |                                                                                                                                                                                                                                                                                                    |
|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pTargetAddress</i>              | a pointer to the SpaceWire path or logical address of the device to be read from                                                                                                                                                                                                                   |
| <i>targetAddress↔<br/>Length</i>   | the length of the target address                                                                                                                                                                                                                                                                   |
| <i>pReplyAddress</i>               | a pointer to the SpaceWire path or logical address that will be used to send any responses back to the device from which the command will be sent from                                                                                                                                             |
| <i>replyAddress↔<br/>Length</i>    | the length of the reply address                                                                                                                                                                                                                                                                    |
| <i>increment↔<br/>Address</i>      | whether or not the target read address should be incremented when reading                                                                                                                                                                                                                          |
| <i>key</i>                         | the key expected by the destination device                                                                                                                                                                                                                                                         |
| <i>transaction↔<br/>Identifier</i> | an identifier for the transaction                                                                                                                                                                                                                                                                  |
| <i>readAddress</i>                 | the memory address at the destination to read from                                                                                                                                                                                                                                                 |
| <i>extendedRead↔<br/>Address</i>   | the extended memory address at the destination to read from                                                                                                                                                                                                                                        |
| <i>pRawPacket↔<br/>Length</i>      | a pointer to a previously allocated variable which will be updated to contain the length of the packet returned                                                                                                                                                                                    |
| <i>pPacketStruct</i>               | an optional structure which will be updated to contain details of the fields in the packet. This structure should not be accessed directly, but by using other functions provided by the A↔PI. Alternatively the parameter can be left as NULL if the properties of the packet are not of interest |

---

|                  |                                                                                                                          |
|------------------|--------------------------------------------------------------------------------------------------------------------------|
| <i>alignment</i> | the word size used by the device sending the packet, normally 1. See <a href="#">Byte Alignment</a> for more information |
|------------------|--------------------------------------------------------------------------------------------------------------------------|

---

#### Returns

the packet generated, or NULL if there was an error, for example if one of the fields is invalid. The buffer returned should be freed by calling [RMAP\\_FreeBuffer](#)

#### Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

#### Examples:

[read\\_command\\_packet\\_example.c](#).

**7.69.2.6** `void* RMAPPACKETLIBRARY_CC RMAP_BuildReadReplyPacket ( U8 * pInitiatorAddress, unsigned long initiatorAddressLength, U8 targetAddress, char incrementAddress, RMAP_STATUS status, U16 transactionIdentifier, U8 * pData, U32 dataLength, unsigned long * pRawPacketLength, RMAP_PACKET * pPacketStruct, char alignment )`

Build an RMAP read reply packet which can be sent to respond to an RMAP read command.

Note that this function performs a memory allocation, use [RMAP\\_FillReadReplyPacket](#) to build a packet using previously allocated memory. When the buffer allocated by this function is no longer required, it should be freed by calling [RMAP\\_FreeBuffer](#).

#### Parameters

|                                |                                                                                                                                                                                                                                                                                                    |
|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pInitiatorAddress</i>       | a pointer to the SpaceWire path or logical address of the device to which the reply should be sent                                                                                                                                                                                                 |
| <i>initiator↔AddressLength</i> | the length of the initiator address                                                                                                                                                                                                                                                                |
| <i>targetAddress</i>           | the SpaceWire logical address of the device that sent the command being responded to                                                                                                                                                                                                               |
| <i>increment↔Address</i>       | whether or not the command indicated that the target read address should be incremented when reading                                                                                                                                                                                               |
| <i>status</i>                  | the status of the read operation                                                                                                                                                                                                                                                                   |
| <i>transaction↔Identifier</i>  | an identifier for the transaction                                                                                                                                                                                                                                                                  |
| <i>pData</i>                   | the data read                                                                                                                                                                                                                                                                                      |
| <i>dataLength</i>              | the length of data, which is a 24-bit number and so should be at most 0xfffff                                                                                                                                                                                                                      |
| <i>pRawPacket↔Length</i>       | a pointer to a previously allocated variable which will be updated to contain the length of the packet returned                                                                                                                                                                                    |
| <i>pPacketStruct</i>           | an optional structure which will be updated to contain details of the fields in the packet. This structure should not be accessed directly, but by using other functions provided by the A↔PI. Alternatively the parameter can be left as NULL if the properties of the packet are not of interest |
| <i>alignment</i>               | the word size used by the device sending the packet, normally 1. See <a href="#">Byte Alignment</a> for more information                                                                                                                                                                           |

---

#### Returns

the packet generated, or NULL if there was an error, for example if one of the fields is invalid. The buffer returned should be freed by calling [RMAP\\_FreeBuffer](#)

#### Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

## Examples:

[read\\_reply\\_packet\\_example.c](#).

**7.69.2.7** `void* RMAPPACKETLIBRARY_CC RMAP_BuildWriteCommandPacket ( U8 * pTargetAddress, unsigned long targetAddressLength, U8 * pReplyAddress, unsigned long replyAddressLength, char verifyBeforeWrite, char acknowledge, char incrementAddress, U8 key, U16 transactionIdentifier, U32 writeAddress, U8 extendedWriteAddress, U8 * pData, U32 dataLength, unsigned long * pRawPacketLength, RMAP_PACKET * pPacketStruct, char alignment )`

Build an RMAP write command packet which can be sent to perform an RMAP write command.

Note that this function performs a memory allocation, use `RMAP_FillWriteCommandPacket` to build a packet using previously allocated memory. When the buffer allocated by this function is no longer required, it should be freed by calling `RMAP_FreeBuffer`.

## Parameters

|                                    |                                                                                                                                                                                                                                                                                                    |
|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pTargetAddress</i>              | a pointer to the SpaceWire path or logical address of the device to be written to                                                                                                                                                                                                                  |
| <i>targetAddress↔<br/>Length</i>   | the length of the target address                                                                                                                                                                                                                                                                   |
| <i>pReplyAddress</i>               | a pointer to the SpaceWire path or logical address that will be used to send any responses back to the device from which the command will be sent from                                                                                                                                             |
| <i>replyAddress↔<br/>Length</i>    | the length of the reply address                                                                                                                                                                                                                                                                    |
| <i>verifyBefore↔<br/>Write</i>     | whether or not the data should be verified before writing                                                                                                                                                                                                                                          |
| <i>acknowledge</i>                 | whether or not the command should be acknowledged                                                                                                                                                                                                                                                  |
| <i>increment↔<br/>Address</i>      | whether or not the target write address should be incremented when writing                                                                                                                                                                                                                         |
| <i>key</i>                         | the key expected by the destination device                                                                                                                                                                                                                                                         |
| <i>transaction↔<br/>Identifier</i> | an identifier for the transaction                                                                                                                                                                                                                                                                  |
| <i>writeAddress</i>                | the memory address at the destination to write to                                                                                                                                                                                                                                                  |
| <i>extendedWrite↔<br/>Address</i>  | the extended memory address at the destination to write to                                                                                                                                                                                                                                         |
| <i>pData</i>                       | the data to be written                                                                                                                                                                                                                                                                             |
| <i>dataLength</i>                  | the length of data, which is a 24-bit number and so should be at most 0xffff                                                                                                                                                                                                                       |
| <i>pRawPacket↔<br/>Length</i>      | a pointer to a previously allocated variable which will be updated to contain the length of the packet returned                                                                                                                                                                                    |
| <i>pPacketStruct</i>               | an optional structure which will be updated to contain details of the fields in the packet. This structure should not be accessed directly, but by using other functions provided by the A↔PI. Alternatively the parameter can be left as NULL if the properties of the packet are not of interest |
| <i>alignment</i>                   | the word size used by the device sending the packet, normally 1. See <a href="#">Byte Alignment</a> for more information                                                                                                                                                                           |

## Returns

the packet generated, or NULL if there was an error, for example if one of the fields is invalid. The buffer returned should be freed by calling `RMAP_FreeBuffer`

## Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

## Examples:

[rmap\\_target\\_example.c](#), and [write\\_command\\_packet\\_example.c](#).



**7.69.2.8** void\* **RMAPPACKETLIBRARY\_CC** RMAP\_BuildWriteRegisterPacket ( U8 \* *pTargetAddress*, unsigned long *targetAddressLength*, U8 \* *pReplyAddress*, unsigned long *replyAddressLength*, char *verifyBeforeWrite*, char *acknowledge*, char *incrementAddress*, U8 *key*, U16 *transactionIdentifier*, U32 *writeAddress*, U8 *extendedWriteAddress*, U32 *registerValue*, unsigned long \* *pRawPacketLength*, RMAP\_PACKET \* *pPacketStruct*, char *alignment* )

Build an RMAP write command packet which can be sent to perform an RMAP write command on a 4-byte register.

Note that this function performs a memory allocation, use `RMAP_FillWriteCommandPacket` to build a packet using previously allocated memory. When the buffer allocated by this function is no longer required, it should be freed by calling `RMAP_FreeBuffer`.

#### Parameters

|                                           |                                                                                                                                                                                                                                                                                                    |
|-------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pTargetAddress</i>                     | a pointer to the SpaceWire path or logical address of the device to be written to                                                                                                                                                                                                                  |
| <i>targetAddress</i> ↔<br><i>Length</i>   | the length of the target address                                                                                                                                                                                                                                                                   |
| <i>pReplyAddress</i>                      | a pointer to the SpaceWire path or logical address that from will be used the device to send any responses back to the device which the command will be sent from                                                                                                                                  |
| <i>replyAddress</i> ↔<br><i>Length</i>    | the length of the reply address                                                                                                                                                                                                                                                                    |
| <i>verifyBefore</i> ↔<br><i>Write</i>     | whether or not the data should be verified before writing                                                                                                                                                                                                                                          |
| <i>acknowledge</i>                        | whether or not the command should be acknowledged                                                                                                                                                                                                                                                  |
| <i>increment</i> ↔<br><i>Address</i>      | whether or not the target write address should be incremented when writing                                                                                                                                                                                                                         |
| <i>key</i>                                | the key expected by the destination device                                                                                                                                                                                                                                                         |
| <i>transaction</i> ↔<br><i>Identifier</i> | an identifier for the transaction                                                                                                                                                                                                                                                                  |
| <i>writeAddress</i>                       | the memory address at the destination to write to                                                                                                                                                                                                                                                  |
| <i>extendedWrite</i> ↔<br><i>Address</i>  | the extended memory address at the destination to write to                                                                                                                                                                                                                                         |
| <i>registerValue</i>                      | the value to be written to the register                                                                                                                                                                                                                                                            |
| <i>pRawPacket</i> ↔<br><i>Length</i>      | a pointer to a previously allocated variable which will be updated to contain the length of the packet returned                                                                                                                                                                                    |
| <i>pPacketStruct</i>                      | an optional structure which will be updated to contain details of the fields in the packet. This structure should not be accessed directly, but by using other functions provided by the A↔PI. Alternatively the parameter can be left as NULL if the properties of the packet are not of interest |
| <i>alignment</i>                          | the word size used by the device sending the packet, normally 1. See <a href="#">Byte Alignment</a> for more information                                                                                                                                                                           |

#### Returns

the packet generated, or NULL if there was an error, for example if one of the fields is invalid. The buffer returned should be freed by calling `RMAP_FreeBuffer`

#### Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

#### Examples:

`check_packet_example.c`, and `write_command_packet_example.c`.



**7.69.2.9** void\* **RMAPPACKETLIBRARY\_CC** RMAP\_BuildWriteReplyPacket ( U8 \* *pInitiatorAddress*, unsigned long *initiatorAddressLength*, U8 *targetAddress*, char *verifyBeforeWrite*, char *incrementAddress*, RMAP\_STATUS *status*, U16 *transactionIdentifier*, unsigned long \* *pRawPacketLength*, RMAP\_PACKET \* *pPacketStruct*, char *alignment* )

Build an RMAP write reply packet which can be sent to respond to an RMAP write command.

Note that this function performs a memory allocation, use RMAP\_FillWriteReplyPacket to build a packet using previously allocated memory. When the buffer allocated by this function is no longer required, it should be freed by calling [RMAP\\_FreeBuffer](#).

#### Parameters

|                               |                                                                                                                                                                                                                                                                                                    |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pInitiatorAddress</i>      | a pointer to the SpaceWire path or logical address of the device to which the reply should be sent                                                                                                                                                                                                 |
| <i>initiatorAddressLength</i> | the length of the initiator address                                                                                                                                                                                                                                                                |
| <i>targetAddress</i>          | the SpaceWire logical address of the device that sent the command being responded to                                                                                                                                                                                                               |
| <i>verifyBeforeWrite</i>      | whether or not the command indicated that data should be verified before writing                                                                                                                                                                                                                   |
| <i>incrementAddress</i>       | whether or not the command indicated that the target write address should be incremented when writing                                                                                                                                                                                              |
| <i>status</i>                 | the status of the write operation                                                                                                                                                                                                                                                                  |
| <i>transactionIdentifier</i>  | an identifier for the transaction                                                                                                                                                                                                                                                                  |
| <i>pRawPacketLength</i>       | a pointer to a previously allocated variable which will be updated to contain the length of the packet returned                                                                                                                                                                                    |
| <i>pPacketStruct</i>          | an optional structure which will be updated to contain details of the fields in the packet. This structure should not be accessed directly, but by using other functions provided by the A↔PI. Alternatively the parameter can be left as NULL if the properties of the packet are not of interest |
| <i>alignment</i>              | the word size used by the device sending the packet, normally 1. See <a href="#">Byte Alignment</a> for more information                                                                                                                                                                           |

#### Returns

the packet generated, or NULL if there was an error, for example if one of the fields is invalid. The buffer returned should be freed by calling [RMAP\\_FreeBuffer](#)

#### Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

#### Examples:

[write\\_reply\\_packet\\_example.c](#).

**7.69.2.10** unsigned long **RMAPPACKETLIBRARY\_CC** RMAP\_CalculateReadCommandPacketLength ( unsigned long *targetAddressLength*, unsigned long *replyAddressLength*, char *alignment* )

Calculate the number of bytes required for a read command packet, given the properties of the packet.

#### Parameters

|                            |                                                         |
|----------------------------|---------------------------------------------------------|
| <i>targetAddressLength</i> | the length in bytes of the target address in the packet |
|----------------------------|---------------------------------------------------------|

|                                 |                                                                                                                          |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <i>replyAddress↔<br/>Length</i> | the length in bytes of the reply address in the packet                                                                   |
| <i>alignment</i>                | the word size used by the device sending the packet, normally 1. See <a href="#">Byte Alignment</a> for more information |

#### Returns

the calculated length of the read command packet in bytes, or 0 if there was an error

#### Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

#### Examples:

[read\\_command\\_packet\\_example.c](#).

**7.69.2.11** unsigned long **RMAPPACKETLIBRARY\_CC** RMAP\_CalculateReadModifyWriteCommandPacketLength ( unsigned long *targetAddressLength*, unsigned long *replyAddressLength*, U32 *dataAndMaskLength*, char *alignment* )

Calculate the number of bytes required for a read/modify/write command packet, given the properties of the packet.

#### Parameters

|                                  |                                                                                                                          |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <i>targetAddress↔<br/>Length</i> | the length in bytes of the target address in the packet                                                                  |
| <i>replyAddress↔<br/>Length</i>  | the length in bytes of the reply address in the packet                                                                   |
| <i>dataAndMask↔<br/>Length</i>   | the combined length in bytes of the data and mask fields, this should be an even number between 0 and 8                  |
| <i>alignment</i>                 | the word size used by the device sending the packet, normally 1. See <a href="#">Byte Alignment</a> for more information |

#### Returns

the calculated length of the write command packet in bytes, or 0 if there was an error

#### Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

#### Examples:

[rmw\\_command\\_packet\\_example.c](#).

**7.69.2.12** unsigned long **RMAPPACKETLIBRARY\_CC** RMAP\_CalculateReadModifyWriteReplyPacketLength ( unsigned long *initiatorAddressLength*, U32 *dataLength*, char *alignment* )

Calculate the number of bytes required for a read/modify/write reply packet, given the properties of the packet.

#### Parameters

|                                     |                                                            |
|-------------------------------------|------------------------------------------------------------|
| <i>initiator↔<br/>AddressLength</i> | the length in bytes of the initiator address in the packet |
|-------------------------------------|------------------------------------------------------------|

|                   |                                                                                                                          |
|-------------------|--------------------------------------------------------------------------------------------------------------------------|
| <i>dataLength</i> | the length in bytes of the data field, this can be at most 4 bytes                                                       |
| <i>alignment</i>  | the word size used by the device sending the packet, normally 1. See <a href="#">Byte Alignment</a> for more information |

**Returns**

the calculated length of the read/modify/write reply packet in bytes, or 0 if there was an error

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

[rmw\\_reply\\_packet\\_example.c](#).

#### 7.69.2.13 unsigned long RMAPPACKETLIBRARY\_CC RMAP\_CalculateReadReplyPacketLength ( unsigned long initiatorAddressLength, U32 dataLength, char alignment )

Calculate the number of bytes required for a read reply packet, given the properties of the packet.

**Parameters**

|                                     |                                                                                                                          |
|-------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <i>initiator↔<br/>AddressLength</i> | the length in bytes of the initiator address in the packet                                                               |
| <i>dataLength</i>                   | the length in bytes of the data field, this can be at most 0xfffff bytes                                                 |
| <i>alignment</i>                    | the word size used by the device sending the packet, normally 1. See <a href="#">Byte Alignment</a> for more information |

**Returns**

the calculated length of the read reply packet in bytes, or 0 if there was an error

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

[read\\_reply\\_packet\\_example.c](#).

#### 7.69.2.14 unsigned long RMAPPACKETLIBRARY\_CC RMAP\_CalculateWriteCommandPacketLength ( unsigned long targetAddressLength, unsigned long replyAddressLength, U32 dataLength, char alignment )

Calculate the number of bytes required for a write command packet, given the properties of the packet.

**Parameters**

|                                  |                                                                          |
|----------------------------------|--------------------------------------------------------------------------|
| <i>targetAddress↔<br/>Length</i> | the length in bytes of the target address in the packet                  |
| <i>replyAddress↔<br/>Length</i>  | the length in bytes of the reply address in the packet                   |
| <i>dataLength</i>                | the length in bytes of the data field, this can be at most 0xfffff bytes |

---

|                  |                                                                                                                          |
|------------------|--------------------------------------------------------------------------------------------------------------------------|
| <i>alignment</i> | the word size used by the device sending the packet, normally 1. See <a href="#">Byte Alignment</a> for more information |
|------------------|--------------------------------------------------------------------------------------------------------------------------|

---

**Returns**

the calculated length of the write command packet in bytes, or 0 if there was an error

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

[write\\_command\\_packet\\_example.c](#).

**7.69.2.15** `unsigned long RMAPPACKETLIBRARY_CC RMAP_CalculateWriteReplyPacketLength ( unsigned long initiatorAddressLength, char alignment )`

Calculate the number of bytes required for a write reply packet, given the properties of the packet.

**Parameters**


---

|                                     |                                                                                                                          |
|-------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <i>initiator↔<br/>AddressLength</i> | the length in bytes of the initiator address in the packet                                                               |
| <i>alignment</i>                    | the word size used by the device sending the packet, normally 1. See <a href="#">Byte Alignment</a> for more information |

---

**Returns**

the calculated length of the write reply packet in bytes, or 0 if there was an error

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

[write\\_reply\\_packet\\_example.c](#).

**7.69.2.16** `char RMAPPACKETLIBRARY_CC RMAP_FillReadCommandPacket ( U8 * pTargetAddress, unsigned long targetAddressLength, U8 * pReplyAddress, unsigned long replyAddressLength, char incrementAddress, U8 key, U16 transactionIdentifier, U32 readAddress, U8 extendedReadAddress, U32 dataLength, unsigned long * pRawPacketLength, RMAP_PACKET * pPacketStruct, char alignment, U8 * pRawPacket, unsigned long rawPacketLength )`

Fill a previously allocated buffer with an RMAP read command packet which can be sent to perform an RMAP read command.

The length of the packet to be built, and therefore the amount of memory required, can be determined by calling [RMAP\\_CalculateReadCommandPacketLength](#). This function performs no memory allocation. See [RMAP\\_Build↔ReadCommandPacket](#) for a function which builds a read command packet, performing the memory allocation.

**Parameters**

|                                           |                                                                                                                                                                                                                                                                                                    |
|-------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pTargetAddress</i>                     | a pointer to the SpaceWire path or logical address of the device to be read from                                                                                                                                                                                                                   |
| <i>targetAddress</i> ↔<br><i>Length</i>   | the length of the target address                                                                                                                                                                                                                                                                   |
| <i>pReplyAddress</i>                      | a pointer to the SpaceWire path or logical address that will be used to send any responses back to the device from which the command will be sent from                                                                                                                                             |
| <i>replyAddress</i> ↔<br><i>Length</i>    | the length of the reply address                                                                                                                                                                                                                                                                    |
| <i>increment</i> ↔<br><i>Address</i>      | whether or not the target read address should be incremented when reading                                                                                                                                                                                                                          |
| <i>key</i>                                | the key expected by the destination device                                                                                                                                                                                                                                                         |
| <i>transaction</i> ↔<br><i>Identifier</i> | an identifier for the transaction                                                                                                                                                                                                                                                                  |
| <i>readAddress</i>                        | the memory address at the destination to read from                                                                                                                                                                                                                                                 |
| <i>extendedRead</i> ↔<br><i>Address</i>   | the extended memory address at the destination to read from                                                                                                                                                                                                                                        |
| <i>dataLength</i>                         | the length of data to read, which is a 24-bit number and so should be at most 0xfffff                                                                                                                                                                                                              |
| <i>pRawPacket</i> ↔<br><i>Length</i>      | a pointer to a previously allocated variable which will be updated to contain the length of the packet returned                                                                                                                                                                                    |
| <i>pPacketStruct</i>                      | an optional structure which will be updated to contain details of the fields in the packet. This structure should not be accessed directly, but by using other functions provided by the A↔PI. Alternatively the parameter can be left as NULL if the properties of the packet are not of interest |
| <i>alignment</i>                          | the word size used by the device sending the packet, normally 1. See <a href="#">Byte Alignment</a> for more information                                                                                                                                                                           |
| <i>pRawPacket</i>                         | a pointer to a previously allocated buffer which will be updated to contain the read command packet                                                                                                                                                                                                |
| <i>rawPacket</i> ↔<br><i>Length</i>       | the length of the buffer provided for the packet                                                                                                                                                                                                                                                   |

**Returns**

1 if the buffer was successfully filled with the RMAP packet, or 0 if there was an error, for example if one of the fields is invalid

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

[read\\_command\\_packet\\_example.c](#).

```
7.69.2.17 char RMAPPACKETLIBRARY_CC RMAP_FillReadModifyWriteCommandPacket (U8 * pTargetAddress,
unsigned long targetAddressLength, U8 * pReplyAddress, unsigned long replyAddressLength, U8 key,
U16 transactionIdentifier, U32 readModifyWriteAddress, U8 extendedReadModifyWriteAddress, U8
dataAndMaskLength, U8 * pData, U8 * pMask, unsigned long * pRawPacketLength, RMAP_PACKET *
pPacketStruct, char alignment, U8 * pRawPacket, unsigned long rawPacketLength)
```

Fill a previously allocated buffer with an RMAP read/modify/write command packet which can be sent to perform an RMAP read/modify/write command.

The length of the packet to be built, and therefore the amount of memory required, can be determined by calling [RMAP\\_CalculateReadModifyWriteCommandPacketLength](#). This function performs no memory allocation. See [RMAP\\_BuildReadModifyWriteCommandPacket](#) for a function which builds a read/modify/write command packet, performing the memory allocation.

**Parameters**

|                                                                 |                                                                                                                                                                                                                                                                                                    |
|-----------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pTargetAddress</i>                                           | a pointer to the SpaceWire path or logical address of the device to be written to                                                                                                                                                                                                                  |
| <i>targetAddress</i> ↔<br><i>Length</i>                         | the length of the target address                                                                                                                                                                                                                                                                   |
| <i>pReplyAddress</i>                                            | a pointer to the SpaceWire path or logical address that will be used to send any responses back to the device from which the command will be sent from                                                                                                                                             |
| <i>replyAddress</i> ↔<br><i>Length</i>                          | the length of the reply address                                                                                                                                                                                                                                                                    |
| <i>key</i>                                                      | the key expected by the destination device                                                                                                                                                                                                                                                         |
| <i>transaction</i> ↔<br><i>Identifier</i>                       | an identifier for the transaction                                                                                                                                                                                                                                                                  |
| <i>readModify</i> ↔<br><i>WriteAddress</i>                      | the memory address at the destination to read from and write to                                                                                                                                                                                                                                    |
| <i>extendedRead</i> ↔<br><i>ModifyWrite</i> ↔<br><i>Address</i> | the extended memory address at the destination to read from and write to                                                                                                                                                                                                                           |
| <i>dataAndMask</i> ↔<br><i>Length</i>                           | the combined length in bytes of the data and mask fields, this should be an even number between 0 and 8                                                                                                                                                                                            |
| <i>pData</i>                                                    | the data to be written                                                                                                                                                                                                                                                                             |
| <i>pMask</i>                                                    | the mask to be applied to the data                                                                                                                                                                                                                                                                 |
| <i>pRawPacket</i> ↔<br><i>Length</i>                            | a pointer to a previously allocated variable which will be updated to contain the length of the packet returned                                                                                                                                                                                    |
| <i>pPacketStruct</i>                                            | an optional structure which will be updated to contain details of the fields in the packet. This structure should not be accessed directly, but by using other functions provided by the A↔PI. Alternatively the parameter can be left as NULL if the properties of the packet are not of interest |
| <i>alignment</i>                                                | the word size used by the device sending the packet, normally 1. See <a href="#">Byte Alignment</a> for more information                                                                                                                                                                           |
| <i>pRawPacket</i>                                               | a pointer to a previously allocated buffer which will be updated to contain the read/modify/write command packet                                                                                                                                                                                   |
| <i>rawPacket</i> ↔<br><i>Length</i>                             | the length of the buffer provided for the packet                                                                                                                                                                                                                                                   |

**Returns**

1 if the buffer was successfully filled with the RMAP packet, or 0 if there was an error, for example if one of the fields is invalid

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

[rmw\\_command\\_packet\\_example.c](#).

```
7.69.2.18 char RMAPPACKETLIBRARY_CC RMAP_FillReadModifyWriteReplyPacket (U8 * pInitiatorAddress, unsigned long initiatorAddressLength, U8 targetAddress, RMAP_STATUS status, U16 transactionIdentifier, unsigned long dataLength, U8 * pData, unsigned long * pRawPacketLength, RMAP_PACKET * pPacketStruct, char alignment, U8 * pRawPacket, unsigned long rawPacketLength)
```

Fill a previously allocated buffer with an RMAP read/modify/write reply packet which can be sent to respond to an RMAP read/modify/write command.

The length of the packet to be built, and therefore the amount of memory required, can be determined by calling [RMAP\\_CalculateReadModifyWriteReplyPacketLength](#). This function performs no memory allocation. See [RMAP\\_BuildReadModifyWriteReplyPacket](#) for a function which builds a read/modify/write reply packet, performing the memory allocation.

**Parameters**

|                                     |                                                                                                                                                                                                                                                                                                    |
|-------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pInitiatorAddress</i>            | a pointer to the SpaceWire path or logical address of the device to which the reply should be sent                                                                                                                                                                                                 |
| <i>initiator↔<br/>AddressLength</i> | the length of the initiator address                                                                                                                                                                                                                                                                |
| <i>targetAddress</i>                | the SpaceWire logical address of the device that sent the command being responded to                                                                                                                                                                                                               |
| <i>status</i>                       | the status of the read operation                                                                                                                                                                                                                                                                   |
| <i>transaction↔<br/>Identifier</i>  | an identifier for the transaction                                                                                                                                                                                                                                                                  |
| <i>dataLength</i>                   | the length in bytes of the data field, this can be at most 4 bytes                                                                                                                                                                                                                                 |
| <i>pData</i>                        | the data read                                                                                                                                                                                                                                                                                      |
| <i>pRawPacket↔<br/>Length</i>       | a pointer to a previously allocated variable which will be updated to contain the length of the packet returned                                                                                                                                                                                    |
| <i>pPacketStruct</i>                | an optional structure which will be updated to contain details of the fields in the packet. This structure should not be accessed directly, but by using other functions provided by the A↔PI. Alternatively the parameter can be left as NULL if the properties of the packet are not of interest |
| <i>alignment</i>                    | the word size used by the device sending the packet, normally 1. See <a href="#">Byte Alignment</a> for more information                                                                                                                                                                           |
| <i>pRawPacket</i>                   | a pointer to a previously allocated buffer which will be updated to contain the read reply packet                                                                                                                                                                                                  |
| <i>rawPacket↔<br/>Length</i>        | the length of the buffer provided for the packet                                                                                                                                                                                                                                                   |

**Returns**

1 if the buffer was successfully filled with the RMAP packet, or 0 if there was an error, for example if one of the fields is invalid

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

[rmw\\_reply\\_packet\\_example.c](#).

**7.69.2.19** `char RMAPPACKETLIBRARY_CC RMAP_FillReadReplyPacket ( U8 * pInitiatorAddress, unsigned long initiatorAddressLength, U8 targetAddress, char incrementAddress, RMAP_STATUS status, U16 transactionIdentifier, U8 * pData, U32 dataLength, unsigned long * pRawPacketLength, RMAP_PACKET * pPacketStruct, char alignment, U8 * pRawPacket, unsigned long rawPacketLength )`

Fill a previously allocated buffer with an RMAP read reply packet which can be sent to respond to an RMAP read command.

The length of the packet to be built, and therefore the amount of memory required, can be determined by calling [RMAP\\_CalculateReadReplyPacketLength](#). This function performs no memory allocation. See [RMAP\\_BuildRead↔ReplyPacket](#) for a function which builds a read reply packet, performing the memory allocation.

**Parameters**

|                                     |                                                                                                    |
|-------------------------------------|----------------------------------------------------------------------------------------------------|
| <i>pInitiatorAddress</i>            | a pointer to the SpaceWire path or logical address of the device to which the reply should be sent |
| <i>initiator↔<br/>AddressLength</i> | the length of the initiator address                                                                |

|                                    |                                                                                                                                                                                                                                                                                                    |
|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>targetAddress</i>               | the SpaceWire logical address of the device that sent the command being responded to                                                                                                                                                                                                               |
| <i>increment↔<br/>Address</i>      | whether or not the command indicated that the target read address should be incremented when reading                                                                                                                                                                                               |
| <i>status</i>                      | the status of the read operation                                                                                                                                                                                                                                                                   |
| <i>transaction↔<br/>Identifier</i> | an identifier for the transaction                                                                                                                                                                                                                                                                  |
| <i>pData</i>                       | the data read                                                                                                                                                                                                                                                                                      |
| <i>dataLength</i>                  | the length of data, which is a 24-bit number and so should be at most 0xfffff                                                                                                                                                                                                                      |
| <i>pRawPacket↔<br/>Length</i>      | a pointer to a previously allocated variable which will be updated to contain the length of the packet returned                                                                                                                                                                                    |
| <i>pPacketStruct</i>               | an optional structure which will be updated to contain details of the fields in the packet. This structure should not be accessed directly, but by using other functions provided by the A↔PI. Alternatively the parameter can be left as NULL if the properties of the packet are not of interest |
| <i>alignment</i>                   | the word size used by the device sending the packet, normally 1. See <a href="#">Byte Alignment</a> for more information                                                                                                                                                                           |
| <i>pRawPacket</i>                  | a pointer to a previously allocated buffer which will be updated to contain the read reply packet                                                                                                                                                                                                  |
| <i>rawPacket↔<br/>Length</i>       | the length of the buffer provided for the packet                                                                                                                                                                                                                                                   |

### Returns

1 if the buffer was successfully filled with the RMAP packet, or 0 if there was an error, for example if one of the fields is invalid

### Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

### Examples:

[read\\_reply\\_packet\\_example.c](#).

**7.69.2.20** `char RMAPPACKETLIBRARY_CC RMAP_FillWriteCommandPacket ( U8 * pTargetAddress, unsigned long targetAddressLength, U8 * pReplyAddress, unsigned long replyAddressLength, char verifyBeforeWrite, char acknowledge, char incrementAddress, U8 key, U16 transactionIdentifier, U32 writeAddress, U8 extendedWriteAddress, U8 * pData, U32 dataLength, unsigned long * pRawPacketLength, RMAP_PACKET * pPacketStruct, char alignment, U8 * pRawPacket, unsigned long rawPacketLength )`

Fill a previously allocated buffer with an RMAP write command packet which can be sent to perform an RMAP write command.

The length of the packet to be built, and therefore the amount of memory required, can be determined by calling [RMAP\\_CalculateWriteCommandPacketLength](#). This function performs no memory allocation. See [RMAP\\_Build↔WriteCommandPacket](#) for a function which builds a write command packet, performing the memory allocation.

### Parameters

|                                  |                                                                                                                                                        |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pTargetAddress</i>            | a pointer to the SpaceWire path or logical address of the device to be written to                                                                      |
| <i>targetAddress↔<br/>Length</i> | the length of the target address                                                                                                                       |
| <i>pReplyAddress</i>             | a pointer to the SpaceWire path or logical address that will be used to send any responses back to the device from which the command will be sent from |



|                                           |                                                                                                                                                                                                                                                                                                    |
|-------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>replyAddress</i> ↔<br><i>Length</i>    | the length of the reply address                                                                                                                                                                                                                                                                    |
| <i>verifyBefore</i> ↔<br><i>Write</i>     | whether or not the data should be verified before writing                                                                                                                                                                                                                                          |
| <i>acknowledge</i>                        | whether or not the command should be acknowledged                                                                                                                                                                                                                                                  |
| <i>increment</i> ↔<br><i>Address</i>      | whether or not the target write address should be incremented when writing                                                                                                                                                                                                                         |
| <i>key</i>                                | the key expected by the destination device                                                                                                                                                                                                                                                         |
| <i>transaction</i> ↔<br><i>Identifier</i> | an identifier for the transaction                                                                                                                                                                                                                                                                  |
| <i>writeAddress</i>                       | the memory address at the destination to write to                                                                                                                                                                                                                                                  |
| <i>extendedWrite</i> ↔<br><i>Address</i>  | the extended memory address at the destination to write to                                                                                                                                                                                                                                         |
| <i>pData</i>                              | the data to be written                                                                                                                                                                                                                                                                             |
| <i>dataLength</i>                         | the length of data, which is a 24-bit number and so should be at most 0xfffff                                                                                                                                                                                                                      |
| <i>pRawPacket</i> ↔<br><i>Length</i>      | a pointer to a previously allocated variable which will be updated to contain the length of the packet returned                                                                                                                                                                                    |
| <i>pPacketStruct</i>                      | an optional structure which will be updated to contain details of the fields in the packet. This structure should not be accessed directly, but by using other functions provided by the A↔PI. Alternatively the parameter can be left as NULL if the properties of the packet are not of interest |
| <i>alignment</i>                          | the word size used by the device sending the packet, normally 1. See <a href="#">Byte Alignment</a> for more information                                                                                                                                                                           |
| <i>pRawPacket</i>                         | a pointer to a previously allocated buffer which will be updated to contain the write command packet                                                                                                                                                                                               |
| <i>rawPacket</i> ↔<br><i>Length</i>       | the length of the buffer provided for the packet                                                                                                                                                                                                                                                   |

**Returns**

1 if the buffer was successfully filled with the RMAP packet, or 0 if there was an error, for example if one of the fields is invalid

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

[write\\_command\\_packet\\_example.c](#).

**7.69.2.21** `char RMAPPACKETLIBRARY_CC RMAP_FillWriteReplyPacket ( U8 * pInitiatorAddress, unsigned long initiatorAddressLength, U8 targetAddress, char verifyBeforeWrite, char incrementAddress, RMAP_STATUS status, U16 transactionIdentifier, unsigned long * pRawPacketLength, RMAP_PACKET * pPacketStruct, char alignment, U8 * pRawPacket, unsigned long rawPacketLength )`

Fill a previously allocated buffer with an RMAP write reply packet which can be sent to respond to an RMAP write command.

The length of the packet to be built, and therefore the amount of memory required, can be determined by calling [RMAP\\_CalculateWriteReplyPacketLength](#). This function performs no memory allocation. See [RMAP\\_BuildWrite↔ReplyPacket](#) for a function which builds a write reply packet, performing the memory allocation.

**Parameters**

|                                     |                                                                                                                                                                                                                                                                                                    |
|-------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pInitiatorAddress</i>            | a pointer to the SpaceWire path or logical address of the device to which the reply should be sent                                                                                                                                                                                                 |
| <i>initiator↔<br/>AddressLength</i> | the length of the initiator address                                                                                                                                                                                                                                                                |
| <i>targetAddress</i>                | the SpaceWire logical address of the device that sent the command being responded to                                                                                                                                                                                                               |
| <i>verifyBefore↔<br/>Write</i>      | whether or not the command indicated that data should be verified before writing                                                                                                                                                                                                                   |
| <i>increment↔<br/>Address</i>       | whether or not the command indicated that the target write address should be incremented when writing                                                                                                                                                                                              |
| <i>status</i>                       | the status of the write operation                                                                                                                                                                                                                                                                  |
| <i>transaction↔<br/>Identifier</i>  | an identifier for the transaction                                                                                                                                                                                                                                                                  |
| <i>pRawPacket↔<br/>Length</i>       | a pointer to a previously allocated variable which will be updated to contain the length of the packet returned                                                                                                                                                                                    |
| <i>pPacketStruct</i>                | an optional structure which will be updated to contain details of the fields in the packet. This structure should not be accessed directly, but by using other functions provided by the A↔PI. Alternatively the parameter can be left as NULL if the properties of the packet are not of interest |
| <i>alignment</i>                    | the word size used by the device sending the packet, normally 1. See <a href="#">Byte Alignment</a> for more information                                                                                                                                                                           |
| <i>pRawPacket</i>                   | a pointer to a previously allocated buffer which will be updated to contain the write reply packet                                                                                                                                                                                                 |
| <i>rawPacket↔<br/>Length</i>        | the length of the buffer provided for the packet                                                                                                                                                                                                                                                   |

#### Returns

1 if the buffer was successfully filled with the RMAP packet, or 0 if there was an error, for example if one of the fields is invalid

#### Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

#### Examples:

[write\\_reply\\_packet\\_example.c](#).

#### 7.69.2.22 void RMAPPACKETLIBRARY\_CC RMAP\_SetProtocolIdentifier ( RMAP\_PACKET \* pPacketStruct, U8 protocolIdentifier )

Set the protocol identifier of a previously created packet, updating the packet's header CRC so the packet is still valid.

This can be used to create packets which use the RMAP packet structure, but use a different protocol identifier, such as SpaceWire-PnP packets. The packet and packet structure can be created by calling one of the RMAP\_Build\* or RMAP\_Fill\* functions.

#### Parameters

|                           |                                                                                                |
|---------------------------|------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i>      | a structure representing the packet, created by one of the RMAP_Build* or RMAP_Fill* functions |
| <i>protocolIdentifier</i> | the value to set the packet's protocol identifier to                                           |

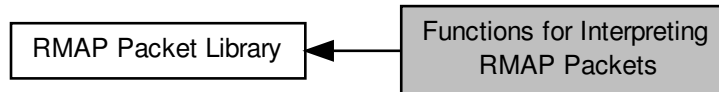
#### Added in version:

STAR-System v2.0 - 3rd July 2013

## 7.70 Functions for Interpreting RMAP Packets

These functions can be used to determine if an RMAP packet is valid, and the fields in that RMAP packet.

Collaboration diagram for Functions for Interpreting RMAP Packets:



### Functions

- **RMAP\_STATUS RMAPPACKETLIBRARY\_CC RMAP\_CheckPacketValid** (void \*pRawPacket, unsigned long packetLength, **RMAP\_PACKET** \*pPacketStruct, char checkPacketTooLong)
 

*Check that the packet specified is in the correct format for an RMAP packet.*
- **RMAP\_STATUS RMAPPACKETLIBRARY\_CC RMAP\_CheckPacketValidIgnoreProtocol** (void \*pRawPacket, unsigned long packetLength, **RMAP\_PACKET** \*pPacketStruct, char checkPacketTooLong)
 

*Check that the packet specified is in the correct format for an RMAP packet, but do not check that the protocol identifier used by the packet is the RMAP protocol ID.*
- **RMAP\_PACKET\_TYPE RMAPPACKETLIBRARY\_CC RMAP\_GetPacketType** (**RMAP\_PACKET** \*pPacketStruct)
 

*Get the type of the packet (whether it is a read, write or read/modify/write command or reply) from a packet structure which has already been created.*
- **U8 \*RMAPPACKETLIBRARY\_CC RMAP\_GetTargetAddress** (**RMAP\_PACKET** \*pPacketStruct, unsigned long \*pTargetAddressLength)
 

*Get the target address of the packet from a packet structure which has already been created.*
- **char RMAPPACKETLIBRARY\_CC RMAP\_GetVerifyBeforeWrite** (**RMAP\_PACKET** \*pPacketStruct)
 

*Get whether the packet represented by a previously created packet structure has verify before write enabled, to check any data before writing it to memory.*
- **char RMAPPACKETLIBRARY\_CC RMAP\_GetPerformAcknowledgement** (**RMAP\_PACKET** \*pPacketStruct)
 

*Get whether the packet represented by a previously created packet structure has acknowledgement enabled, to acknowledge the command with a reply packet.*
- **char RMAPPACKETLIBRARY\_CC RMAP\_GetIncrementAddress** (**RMAP\_PACKET** \*pPacketStruct)
 

*Get whether the packet represented by a previously created packet structure has incrementing of memory addresses enabled, to read from or write to sequential memory.*
- **U8 RMAPPACKETLIBRARY\_CC RMAP\_GetKey** (**RMAP\_PACKET** \*pPacketStruct)
 

*Get the key field in the packet represented by a previously created packet structure.*
- **U8 \*RMAPPACKETLIBRARY\_CC RMAP\_GetReplyAddress** (**RMAP\_PACKET** \*pPacketStruct, unsigned long \*pReplyAddressLength)
 

*Get the reply address of the packet from a packet structure which has already been created.*
- **U16 RMAPPACKETLIBRARY\_CC RMAP\_GetTransactionID** (**RMAP\_PACKET** \*pPacketStruct)
 

*Get the transaction identifier in the packet represented by a previously created packet structure.*
- **U32 RMAPPACKETLIBRARY\_CC RMAP\_GetAddress** (**RMAP\_PACKET** \*pPacketStruct, **U8** \*pExtendedAddress)
 

*Get the memory address and extended memory address of the packet from a packet structure which has already been created.*
- **U8 \*RMAPPACKETLIBRARY\_CC RMAP\_GetData** (**RMAP\_PACKET** \*pPacketStruct, **U32** \*pDataLength)

*Get the data in the packet from a packet structure which has already been created.*

- [U32 RMAPPACKETLIBRARY\\_CC RMAP\\_GetReadLength \(RMAP\\_PACKET \\*pPacketStruct\)](#)  
*Gets the data length property in the read command packet from a packet structure which has already been created.*
- [RMAP\\_STATUS RMAPPACKETLIBRARY\\_CC RMAP\\_GetStatus \(RMAP\\_PACKET \\*pPacketStruct\)](#)  
*Get the value of the status field in the packet represented by a previously created packet structure.*
- [U8 RMAPPACKETLIBRARY\\_CC RMAP\\_GetHeaderCRC \(RMAP\\_PACKET \\*pPacketStruct\)](#)  
*Get the value of the header CRC field in the packet represented by a previously created packet structure.*
- [U8 RMAPPACKETLIBRARY\\_CC RMAP\\_GetDataCRC \(RMAP\\_PACKET \\*pPacketStruct\)](#)  
*Get the value of the data CRC field in the packet represented by a previously created packet structure.*
- [U8 \\*RMAPPACKETLIBRARY\\_CC RMAP\\_GetMask \(RMAP\\_PACKET \\*pPacketStruct, U8 \\*pMaskLength\)](#)  
*Get the mask in the packet from a packet structure which has already been created.*
- [U8 RMAPPACKETLIBRARY\\_CC RMAP\\_GetProtocolIdentifier \(RMAP\\_PACKET \\*pPacketStruct\)](#)  
*Get the protocol identifier of the packet from a packet structure which has already been created.*

### 7.70.1 Detailed Description

These functions can be used to determine if an RMAP packet is valid, and the fields in that RMAP packet.

These functions are useful when receiving RMAP packets.

### 7.70.2 Function Documentation

#### 7.70.2.1 [RMAP\\_STATUS RMAPPACKETLIBRARY\\_CC RMAP\\_CheckPacketValid \( void \\* pRawPacket, unsigned long packetLength, RMAP\\_PACKET \\* pPacketStruct, char checkPacketTooLong \)](#)

Check that the packet specified is in the correct format for an RMAP packet.

Note that this function expects that the address at the start of the packet (the target address in a command and the reply address in a response) is only one byte in length (e.g. it is the packet when it reaches its destination). This is because without this information it would be otherwise impossible to determine with certainty if packet was a valid RMAP packet. Offsets of packets with longer addresses at the start can be passed in using pointer arithmetic.

#### Parameters

|                           |                                                                                                                                                                                                                                                                                                                                                                                                      |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pRawPacket</i>         | a buffer containing an RMAP packet                                                                                                                                                                                                                                                                                                                                                                   |
| <i>packetLength</i>       | the length of the RMAP packet                                                                                                                                                                                                                                                                                                                                                                        |
| <i>pPacketStruct</i>      | an optional (previously allocated) structure which can be provided, or left as NULL. If a structure is supplied, this will be updated to contain the packet's properties. Although it is not recommended to access this structure directly, functions such as <a href="#">RMAP_GetReplyAddress</a> and <a href="#">RMAP_GetKey</a> can be called to retrieve the values of fields from the structure |
| <i>checkPacketTooLong</i> | whether to check if the packet specified is longer than it should be, based on the values in the RMAP fields. This should be set to 0 if the packet was received using a device which can only receive multiples of 4 bytes, for example                                                                                                                                                             |

#### Returns

an RMAP error code indicating if the packet contained any errors, for example if any fields have invalid values, or the packet is too short or long

#### Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

#### Examples:

[check\\_packet\\_example.c](#).

### 7.70.2.2 RMAP\_STATUS RMAPPACKETLIBRARY\_CC RMAP\_CheckPacketValidIgnoreProtocol ( void \* *pRawPacket*, unsigned long *packetLength*, RMAP\_PACKET \* *pPacketStruct*, char *checkPacketTooLong* )

Check that the packet specified is in the correct format for an RMAP packet, but do not check that the protocol identifier used by the packet is the RMAP protocol ID.

This can be used to check the format of packets for other protocols such as SpaceWire-PnP, which use the RMAP packet format. The protocol identifier can then be checked by calling [RMAP\\_GetProtocolIdentifier\(\)](#). Note that this function expects that the address at the start of the packet (the target address in a command and the reply address in a response) is only one byte in length (e.g. it is the packet when it reaches its destination). This is because without this information it would be otherwise impossible to determine with certainty if the packet was a valid RMAP packet. Offsets of packets with longer addresses at the start can be passed in using pointer arithmetic.

#### Parameters

|                           |                                                                                                                                                                                                                                                                                                                                                                                                      |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pRawPacket</i>         | a buffer containing an RMAP packet                                                                                                                                                                                                                                                                                                                                                                   |
| <i>packetLength</i>       | the length of the RMAP packet                                                                                                                                                                                                                                                                                                                                                                        |
| <i>pPacketStruct</i>      | an optional (previously allocated) structure which can be provided, or left as NULL. If a structure is supplied, this will be updated to contain the packet's properties. Although it is not recommended to access this structure directly, functions such as <a href="#">RMAP_GetReplyAddress</a> and <a href="#">RMAP_GetKey</a> can be called to retrieve the values of fields from the structure |
| <i>checkPacketTooLong</i> | whether to check if the packet specified is longer than it should be, based on the values in the RMAP fields. This should be set to 0 if the packet was received using a device which can only receive multiples of 4 bytes, for example                                                                                                                                                             |

#### Returns

an RMAP error code indicating if the packet contained any errors, for example if any fields have invalid values, or the packet is too short or long

#### Added in version:

STAR-System v2.0 - 3rd July 2013

### 7.70.2.3 U32 RMAPPACKETLIBRARY\_CC RMAP\_GetAddress ( RMAP\_PACKET \* *pPacketStruct*, U8 \* *pExtendedAddress* )

Get the memory address and extended memory address of the packet from a packet structure which has already been created.

The memory address is the address to be read from or written to at the target. The extended address can be used to extend the 32-bit memory address to 40 bits. The packet structure can be created when calling one of the [RMAP\\_Build\\*](#) or [RMAP\\_Fill\\*](#) functions to create a packet to perform a specific operation, or when calling [RMAP\\_CheckPacketValid](#) to check that a packet is in the correct format.

#### Parameters

|                         |                                                                                                                                                                                      |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i>    | a structure representing the packet, created by the <a href="#">RMAP_CheckPacketValid</a> function or one of the <a href="#">RMAP_Build*</a> or <a href="#">RMAP_Fill*</a> functions |
| <i>pExtendedAddress</i> | a pointer to a previously allocated variable which will be updated to contain the extended address. This can be NULL if the extended address is not of interest                      |

#### Returns

the memory address in the packet, or 0 if the packet is invalid or is of a type which does not contain a memory address

#### Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

[check\\_packet\\_example.c](#).

**7.70.2.4 U8\* RMAPPACKETLIBRARY\_CC RMAP\_GetData ( RMAP\_PACKET \* *pPacketStruct*, U32 \* *pDataLength* )**

Get the data in the packet from a packet structure which has already been created.

The data bytes are the bytes to be written or the bytes that have been read. The packet structure can be created when calling one of the [RMAP\\_Build\\*](#) or [RMAP\\_Fill\\*](#) functions to create a packet to perform a specific operation, or when calling [RMAP\\_CheckPacketValid](#) to check that a packet is in the correct format.

**Parameters**

|                      |                                                                                                                                                                                      |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i> | a structure representing the packet, created by the <a href="#">RMAP_CheckPacketValid</a> function or one of the <a href="#">RMAP_Build*</a> or <a href="#">RMAP_Fill*</a> functions |
| <i>pDataLength</i>   | a pointer to a previously allocated variable which will be updated to contain the length of the data field. This can be NULL if the length is not of interest                        |

**Returns**

a pointer to the data in the packet, or NULL if the packet is invalid or is of a type which does not contain data

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

[check\\_packet\\_example.c](#).

**7.70.2.5 U8 RMAPPACKETLIBRARY\_CC RMAP\_GetDataCRC ( RMAP\_PACKET \* *pPacketStruct* )**

Get the value of the data CRC field in the packet represented by a previously created packet structure.

The data CRC field is present in packet types with data fields and contains an 8-bit CRC covering each byte in the data and mask (in read/modify/write commands) fields. The packet structure can be created when calling one of the [RMAP\\_Build\\*](#) or [RMAP\\_Fill\\*](#) functions to create a packet to perform a specific operation, or when calling [RMAP\\_CheckPacketValid](#) to check that a packet is in the correct format.

**Parameters**

|                      |                                                                                                                                                                                      |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i> | a structure representing the packet, created by the <a href="#">RMAP_CheckPacketValid</a> function or one of the <a href="#">RMAP_Build*</a> or <a href="#">RMAP_Fill*</a> functions |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Returns**

the value of the data CRC field in the packet, if present

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

[check\\_packet\\_example.c](#).

**7.70.2.6 U8 RMAPPACKETLIBRARY\_CC RMAP\_GetHeaderCRC ( RMAP\_PACKET \* pPacketStruct )**

Get the value of the header CRC field in the packet represented by a previously created packet structure.

The header CRC field contains an 8-bit CRC covering each byte in the header. The packet structure can be created when calling one of the RMAP\_Build\* or RMAP\_Fill\* functions to create a packet to perform a specific operation, or when calling [RMAP\\_CheckPacketValid](#) to check that a packet is in the correct format.

**Parameters**


---

|                      |                                                                                                                                                      |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i> | a structure representing the packet, created by the <a href="#">RMAP_CheckPacketValid</a> function or one of the RMAP_Build* or RMAP_Fill* functions |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|

---

**Returns**

the value of the header CRC field in the packet

**Added in version:**

[STAR-System v0.8 \(Beta 1\) - 22nd August 2011](#)

**Examples:**

[check\\_packet\\_example.c](#).

**7.70.2.7 char RMAPPACKETLIBRARY\_CC RMAP\_GetIncrementAddress ( RMAP\_PACKET \* pPacketStruct )**

Get whether the packet represented by a previously created packet structure has incrementing of memory addresses enabled, to read from or write to sequential memory.

The packet structure can be created when calling one of the RMAP\_Build\* or RMAP\_Fill\* functions to create a packet to perform a specific operation, or when calling [RMAP\\_CheckPacketValid](#) to check that a packet is in the correct format.

**Parameters**


---

|                      |                                                                                                                                                      |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i> | a structure representing the packet, created by the <a href="#">RMAP_CheckPacketValid</a> function or one of the RMAP_Build* or RMAP_Fill* functions |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|

---

**Returns**

whether incrementing addresses is enabled in the packet

**Added in version:**

[STAR-System v0.8 \(Beta 1\) - 22nd August 2011](#)

**Examples:**

[check\\_packet\\_example.c](#).

**7.70.2.8 U8 RMAPPACKETLIBRARY\_CC RMAP\_GetKey ( RMAP\_PACKET \* pPacketStruct )**

Get the key field in the packet represented by a previously created packet structure.

The key is matched by the target user application in order for the command to be authorised. The packet structure can be created when calling one of the RMAP\_Build\* or RMAP\_Fill\* functions to create a packet to perform a specific operation, or when calling [RMAP\\_CheckPacketValid](#) to check that a packet is in the correct format.

**Parameters**


---

|                      |                                                                                                                                                                                      |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i> | a structure representing the packet, created by the <a href="#">RMAP_CheckPacketValid</a> function or one of the <a href="#">RMAP_Build*</a> or <a href="#">RMAP_Fill*</a> functions |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

---

**Returns**

the value of the key field in the packet

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

[check\\_packet\\_example.c](#).

### 7.70.2.9 U8\* RMAPPACKETLIBRARY\_CC RMAP\_GetMask ( RMAP\_PACKET \* pPacketStruct, U8 \* pMaskLength )

Get the mask in the packet from a packet structure which has already been created.

The mask bytes are present in read/modify/write commands and defines how the data written to memory is formed in an application dependent manner. The packet structure can be created when calling one of the [RMAP\\_Build\\*](#) or [RMAP\\_Fill\\*](#) functions to create a packet to perform a specific operation, or when calling [RMAP\\_CheckPacketValid](#) to check that a packet is in the correct format.

**Parameters**


---

|                      |                                                                                                                                                                                      |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i> | a structure representing the packet, created by the <a href="#">RMAP_CheckPacketValid</a> function or one of the <a href="#">RMAP_Build*</a> or <a href="#">RMAP_Fill*</a> functions |
| <i>pMaskLength</i>   | a pointer to a previously allocated variable which will be updated to contain the length of the mask field. This can be NULL if the length is not of interest                        |

---

**Returns**

a pointer to the mask in the packet, or NULL if the packet is invalid or is not a read/modify/write command

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

[check\\_packet\\_example.c](#).

### 7.70.2.10 RMAP\_PACKET\_TYPE RMAPPACKETLIBRARY\_CC RMAP\_GetPacketType ( RMAP\_PACKET \* pPacketStruct )

Get the type of the packet (whether it is a read, write or read/modify/write command or reply) from a packet structure which has already been created.

The packet structure can be created when calling one of the [RMAP\\_Build\\*](#) or [RMAP\\_Fill\\*](#) functions to create a packet to perform a specific operation, or when calling [RMAP\\_CheckPacketValid](#) to check that a packet is in the correct format.



**Parameters**


---

|                      |                                                                                                                                                                                      |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i> | a structure representing the packet, created by the <a href="#">RMAP_CheckPacketValid</a> function or one of the <a href="#">RMAP_Build*</a> or <a href="#">RMAP_Fill*</a> functions |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

---

**Returns**

the type of the packet, or `RMAP_INVALID_PACKET_TYPE` if the packet is invalid

**Added in version:**

[STAR-System v0.8 \(Beta 1\) - 22nd August 2011](#)

**Examples:**

[check\\_packet\\_example.c](#).

#### 7.70.2.11 `char RMAPPACKETLIBRARY_CC RMAP_GetPerformAcknowledgement ( RMAP_PACKET * pPacketStruct )`

Get whether the packet represented by a previously created packet structure has acknowledgement enabled, to acknowledge the command with a reply packet.

The packet structure can be created when calling one of the [RMAP\\_Build\\*](#) or [RMAP\\_Fill\\*](#) functions to create a packet to perform a specific operation, or when calling [RMAP\\_CheckPacketValid](#) to check that a packet is in the correct format.

**Parameters**


---

|                      |                                                                                                                                                                                      |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i> | a structure representing the packet, created by the <a href="#">RMAP_CheckPacketValid</a> function or one of the <a href="#">RMAP_Build*</a> or <a href="#">RMAP_Fill*</a> functions |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

---

**Returns**

whether acknowledgement is enabled in the packet

**Added in version:**

[STAR-System v0.8 \(Beta 1\) - 22nd August 2011](#)

**Examples:**

[check\\_packet\\_example.c](#).

#### 7.70.2.12 `U8 RMAPPACKETLIBRARY_CC RMAP_GetProtocolIdentifier ( RMAP_PACKET * pPacketStruct )`

Get the protocol identifier of the packet from a packet structure which has already been created.

Normally this will be the RMAP protocol ID of 1, but may be the protocol ID of another protocol which uses the RMAP packet format, such as SpaceWire-PnP. The packet structure can be created when calling one of the [RMAP\\_Build\\*](#) or [RMAP\\_Fill\\*](#) functions to create a packet to perform a specific operation, or when calling [RMAP\\_CheckPacketValid](#) to check that a packet is in the correct format.

**Parameters**


---

|                      |                                                                                                                                                                                      |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i> | a structure representing the packet, created by the <a href="#">RMAP_CheckPacketValid</a> function or one of the <a href="#">RMAP_Build*</a> or <a href="#">RMAP_Fill*</a> functions |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

---

**Returns**

the packet's protocol identifier, or 0 if the packet is invalid

**Added in version:**

[STAR-System v2.0 - 3rd July 2013](#)

#### 7.70.2.13 U32 RMAPPACKETLIBRARY\_CC RMAP\_GetReadLength ( RMAP\_PACKET \* *pPacketStruct* )

Gets the data length property in the read command packet from a packet structure which has already been created.

The packet structure can be created when calling one of the `RMAP_BuildReadCommandPacket` or `RMAP_FillReadCommandPacket` functions to create a packet to perform a read command operation, or when calling `RMAP_CheckPacketValid` to check that a packet is in the correct format.

##### Parameters

---

|                      |                                                                                                                                                                                                             |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i> | a structure representing the packet, created by the <code>RMAP_CheckPacketValid</code> function or one of the <code>RMAP_BuildReadCommandPacket</code> or <code>RMAP_FillReadCommandPacket</code> functions |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

---

##### Returns

the data length of the read command packet

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

#### 7.70.2.14 U8\* RMAPPACKETLIBRARY\_CC RMAP\_GetReplyAddress ( RMAP\_PACKET \* *pPacketStruct*, unsigned long \* *pReplyAddressLength* )

Get the reply address of the packet from a packet structure which has already been created.

Note that the reply address may contain 0 byte padding if the packet is a command. The packet structure can be created when calling one of the `RMAP_Build*` or `RMAP_Fill*` functions to create a packet to perform a specific operation, or when calling `RMAP_CheckPacketValid` to check that a packet is in the correct format.

##### Parameters

---

|                            |                                                                                                                                                                             |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i>       | a structure representing the packet, created by the <code>RMAP_CheckPacketValid</code> function or one of the <code>RMAP_Build*</code> or <code>RMAP_Fill*</code> functions |
| <i>pReplyAddressLength</i> | a pointer to a previously allocated variable which will be updated to contain the length of the reply address. This can be NULL if the length is not of interest            |

---

##### Returns

a pointer to the reply address in the packet, or NULL if the packet is invalid

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

`check_packet_example.c`.

#### 7.70.2.15 RMAP\_STATUS RMAPPACKETLIBRARY\_CC RMAP\_GetStatus ( RMAP\_PACKET \* *pPacketStruct* )

Get the value of the status field in the packet represented by a previously created packet structure.

The status field is present in reply packets to indicate the status of the command. The packet structure can be created when calling one of the `RMAP_Build*` or `RMAP_Fill*` functions to create a packet to perform a specific operation, or when calling `RMAP_CheckPacketValid` to check that a packet is in the correct format.

**Parameters**


---

|                      |                                                                                                                                                                                      |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i> | a structure representing the packet, created by the <a href="#">RMAP_CheckPacketValid</a> function or one of the <a href="#">RMAP_Build*</a> or <a href="#">RMAP_Fill*</a> functions |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

---

**Returns**

the value of the status field in the packet, if present

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

[check\\_packet\\_example.c](#).

#### 7.70.2.16 U8\* RMAPPACKETLIBRARY\_CC RMAP\_GetTargetAddress ( RMAP\_PACKET \* pPacketStruct, unsigned long \* pTargetAddressLength )

Get the target address of the packet from a packet structure which has already been created.

The packet structure can be created when calling one of the [RMAP\\_Build\\*](#) or [RMAP\\_Fill\\*](#) functions to create a packet to perform a specific operation, or when calling [RMAP\\_CheckPacketValid](#) to check that a packet is in the correct format.

**Parameters**


---

|                             |                                                                                                                                                                                      |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i>        | a structure representing the packet, created by the <a href="#">RMAP_CheckPacketValid</a> function or one of the <a href="#">RMAP_Build*</a> or <a href="#">RMAP_Fill*</a> functions |
| <i>pTargetAddressLength</i> | a pointer to a previously allocated variable which will be updated to contain the length of the target address. This can be NULL if the length is not of interest                    |

---

**Returns**

a pointer to the target address in the packet, or NULL if the packet is invalid

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

[check\\_packet\\_example.c](#).

#### 7.70.2.17 U16 RMAPPACKETLIBRARY\_CC RMAP\_GetTransactionID ( RMAP\_PACKET \* pPacketStruct )

Get the transaction identifier in the packet represented by a previously created packet structure.

The transaction identifier is used to associate a reply with the command that caused the reply. The packet structure can be created when calling one of the [RMAP\\_Build\\*](#) or [RMAP\\_Fill\\*](#) functions to create a packet to perform a specific operation, or when calling [RMAP\\_CheckPacketValid](#) to check that a packet is in the correct format.

**Parameters**


---

|                      |                                                                                                                                                                                      |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i> | a structure representing the packet, created by the <a href="#">RMAP_CheckPacketValid</a> function or one of the <a href="#">RMAP_Build*</a> or <a href="#">RMAP_Fill*</a> functions |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

---

**Returns**

the value of the transaction identifier field in the packet

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

[check\\_packet\\_example.c](#).

**7.70.2.18 char RMAPPACKETLIBRARY\_CC RMAP\_GetVerifyBeforeWrite ( RMAP\_PACKET \* *pPacketStruct* )**

Get whether the packet represented by a previously created packet structure has verify before write enabled, to check any data before writing it to memory.

The packet structure can be created when calling one of the [RMAP\\_Build\\*](#) or [RMAP\\_Fill\\*](#) functions to create a packet to perform a specific operation, or when calling [RMAP\\_CheckPacketValid](#) to check that a packet is in the correct format.

**Parameters**

---

|                      |                                                                                                                                                                                      |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i> | a structure representing the packet, created by the <a href="#">RMAP_CheckPacketValid</a> function or one of the <a href="#">RMAP_Build*</a> or <a href="#">RMAP_Fill*</a> functions |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

---

**Returns**

whether verify before write is enabled in the packet

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

[check\\_packet\\_example.c](#).

## 7.71 STAR-Dundee Types

This section contains the definitions of types used in STAR-Dundee software drivers and APIs.

### Macros

- `#define TRUE 1`  
*A value that can be used to represent the boolean value of true.*
- `#define FALSE 0`  
*A value that can be used to represent the boolean value of false.*
- `#define U16_MAX (0xffff)`  
*The maximum value that can be represented using an unsigned 16-bit number.*

### Typedefs

- `typedef unsigned char U8`  
*A type that can be used to represent an unsigned 8-bit number.*
- `typedef unsigned short U16`  
*A type that can be used to represent an unsigned 16-bit number.*
- `typedef unsigned int U32`  
*A type that can be used to represent an unsigned 32-bit number.*
- `typedef U32 REGISTER`  
*A type that can be used to represent a 4-byte register.*

#### 7.71.1 Detailed Description

This section contains the definitions of types used in STAR-Dundee software drivers and APIs.

#### 7.71.2 Macro Definition Documentation

##### 7.71.2.1 `#define FALSE 0`

A value that can be used to represent the boolean value of false.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

`link_events.c`, `router_configuration_example.c`, and `star-test.c`.

##### 7.71.2.2 `#define TRUE 1`

A value that can be used to represent the boolean value of true.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

`link_events.c`, `router_configuration_example.c`, `star-test.c`, `star_performance_tester.c`, and `ui.c`.

#### 7.71.2.3 U16\_MAX (0xffff)

The maximum value that can be represented using an unsigned 16-bit number.

Added in version:

[STAR-System v1.8 - 24th August 2012](#)

### 7.71.3 Typedef Documentation

#### 7.71.3.1 REGISTER

A type that can be used to represent a 4-byte register.

Added in version:

[STAR-System v0.8 \(Beta 1\) - 22nd August 2011](#)

#### 7.71.3.2 U16

A type that can be used to represent an unsigned 16-bit number.

Added in version:

[STAR-System v0.8 \(Beta 1\) - 22nd August 2011](#)

#### 7.71.3.3 U32

A type that can be used to represent an unsigned 32-bit number.

Added in version:

[STAR-System v0.8 \(Beta 1\) - 22nd August 2011](#)

Examples:

[packet\\_subsystem\\_example.c](#).

#### 7.71.3.4 U8

A type that can be used to represent an unsigned 8-bit number.

Added in version:

[STAR-System v0.8 \(Beta 1\) - 22nd August 2011](#)

Examples:

[utility.c](#).

## Chapter 8

# File Documentation

### 8.1 `cfg_api_brick_mk2.h` File Reference

Functions used to configure STAR-Dundee Brick Mk2 and compatible devices.

```
#include "star-api.h"
#include "cfg_api_brick_mk2_types.h"
```

#### Functions

- int `STAR_API_CC_CFG_BRICK_MK2_enableInterfaceModeOnPort` (STAR\_DEVICE\_ID deviceId, U8 port↔Num)  
*Selectively enables interface mode on a given port of a supported USB device.*
- int `STAR_API_CC_CFG_BRICK_MK2_getInterfaceModeOnPortEnabled` (STAR\_DEVICE\_ID deviceId, U8 portNum, \_Out\_ int \*pEnabled)  
*Gets whether interface mode is enabled on a specific port of a supported USB device.*
- int `STAR_API_CC_CFG_BRICK_MK2_disableInterfaceModeOnPort` (STAR\_DEVICE\_ID deviceId, U8 port↔Num)  
*Selectively disables interface mode on a given port of a supported USB device.*
- int `STAR_API_CC_CFG_BRICK_MK2_enableIdentifySourceOnPort` (STAR\_DEVICE\_ID deviceId, U8 port↔Num)  
*Enables identification of the source port of a received packet on a given port, when in interface mode.*
- int `STAR_API_CC_CFG_BRICK_MK2_getIdentifySourceOnPortEnabled` (STAR\_DEVICE\_ID deviceId, U8 portNum, \_Out\_ int \*pEnabled)  
*Gets whether identify source is enabled for a specific port.*
- int `STAR_API_CC_CFG_BRICK_MK2_disableIdentifySourceOnPort` (STAR\_DEVICE\_ID deviceId, U8 port↔Num)  
*Disables source port identification for a given port.*
- int `STAR_API_CC_CFG_BRICK_MK2_setPortRoutingAddress` (STAR\_DEVICE\_ID deviceId, U8 portNum, U8 address)  
*Sets the address a packet received on a given port should be routed to when interface mode is enabled.*
- int `STAR_API_CC_CFG_BRICK_MK2_getPortRoutingAddress` (STAR\_DEVICE\_ID deviceId, U8 portNum, \_Out\_ U8 \*pAddress)  
*Gets the address a packet received on a given port should be routed to when interface mode is enabled.*
- int `STAR_API_CC_CFG_BRICK_MK2_setLinkClockFrequency` (STAR\_DEVICE\_ID deviceId, U8 linkNum, STAR\_CFG\_BRICK\_MK2\_LINK\_FREQ linkFreq)  
*Sets the Link Clock Frequency on a given link.*
- int `STAR_API_CC_CFG_BRICK_MK2_getLinkClockFrequency` (STAR\_DEVICE\_ID deviceId, U8 linkNum, \_Out\_ STAR\_CFG\_BRICK\_MK2\_LINK\_FREQ \*pLinkFreq)

*Gets the Link Clock Frequency for a given link.*

- int `STAR_API_CC_CFG_BRICK_MK2_injectErrors` (STAR\_DEVICE\_ID deviceId, U8 portNum, \_In\_ STAR\_CFG\_BRICK\_MK2\_ERRORS \*pErrors)

*Inject the specified errors on a given port.*

- int `STAR_API_CC_CFG_BRICK_MK2_enableStateChangeEventsOnPort` (STAR\_DEVICE\_ID deviceId, U8 portNum)

*Selectively enables state change events on a given port.*

- int `STAR_API_CC_CFG_BRICK_MK2_getStateChangeEventsOnPortEnabled` (STAR\_DEVICE\_ID deviceId, U8 portNum, \_Out\_ int \*pEnabled)

*Gets whether state change events are enabled on a specific port.*

- int `STAR_API_CC_CFG_BRICK_MK2_disableStateChangeEventsOnPort` (STAR\_DEVICE\_ID deviceId, U8 portNum)

*Selectively disables state change events on a given port.*

- int `STAR_API_CC_CFG_BRICK_MK2_enableSpeedChangeEventsOnPort` (STAR\_DEVICE\_ID deviceId, U8 portNum)

*Selectively enables speed change events on a given port.*

- int `STAR_API_CC_CFG_BRICK_MK2_getSpeedChangeEventsOnPortEnabled` (STAR\_DEVICE\_ID deviceId, U8 portNum, \_Out\_ int \*pEnabled)

*Gets whether speed change events are enabled on a specific port.*

- int `STAR_API_CC_CFG_BRICK_MK2_disableSpeedChangeEventsOnPort` (STAR\_DEVICE\_ID deviceId, U8 portNum)

*Selectively disables speed change events on a given port.*

### 8.1.1 Detailed Description

Functions used to configure STAR-Dundee Brick Mk2 and compatible devices.

#### Author

STAR-Dundee Ltd.  
 STAR House  
 166 Nethergate  
 Dundee, DD1 4EE  
 Scotland, UK  
 e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2012 STAR-Dundee Ltd.

## 8.2 `cfg_api_brick_mk2_types.h` File Reference

Types used with the STAR-Dundee Brick Mk2 Configuration API.

### Data Structures

- struct `STAR_CFG_BRICK_MK2_ERRORS`

*Structure used to indicate which errors should be generated when calling `CFG_BRICK_MK2_injectErrors()`. [More...](#)*

### Macros

- `#define STAR_CFG_BRICK_MK2_LINK_SPEED_UNITS_KBPS 200`

*The units in Kbit/s used to represent the link speed returned when calling `CFG_BRICK_MK2_getMeasuredLinkSpeed()`.*



## Enumerations

- enum STAR\_CFG\_BRICK\_MK2\_LINK\_FREQ {  
    STAR\_CFG\_BRICK\_MK2\_LINK\_FREQ\_120 = 0, STAR\_CFG\_BRICK\_MK2\_LINK\_FREQ\_130 = 1, STAR\_CFG\_BRICK\_MK2\_LINK\_FREQ\_140 = 2, STAR\_CFG\_BRICK\_MK2\_LINK\_FREQ\_150 = 3,  
    STAR\_CFG\_BRICK\_MK2\_LINK\_FREQ\_160 = 4, STAR\_CFG\_BRICK\_MK2\_LINK\_FREQ\_180 = 5, STAR\_CFG\_BRICK\_MK2\_LINK\_FREQ\_200 = 6 }

*Frequencies a link on a Brick Mk2 compatible device may run at.*

### 8.2.1 Detailed Description

Types used with the STAR-Dundee Brick Mk2 Configuration API.

#### Author

STAR-Dundee Ltd.  
STAR House  
166 Nethergate  
Dundee, DD1 4EE  
Scotland, UK  
e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2012 STAR-Dundee Ltd.

### 8.2.2 Data Structure Documentation

#### 8.2.2.1 struct STAR\_CFG\_BRICK\_MK2\_ERRORS

Structure used to indicate which errors should be generated when calling `CFG_BRICK_MK2_injectErrors()`.

If the element is set to a non-zero value, an error of that type will be injected.

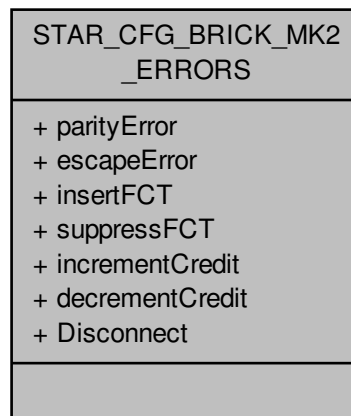
Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

Examples:

[brickMk2\\_configuration\\_example.c](#).

Collaboration diagram for STAR\_CFG\_BRICK\_MK2\_ERRORS:



#### Data Fields

|      |                 |                                                                                                      |
|------|-----------------|------------------------------------------------------------------------------------------------------|
| char | decrementCredit | Whether credit should be decremented to cause an error.                                              |
| char | Disconnect      | Whether a disconnect should occur.<br><br>Added in version:<br><br>STAR-System v3.6 - 3rd April 2017 |
| char | escapeError     | Whether an escape error should be injected.                                                          |
| char | incrementCredit | Whether credit should be incremented to cause an error.                                              |
| char | insertFCT       | Whether an extra FCT should be inserted to cause an error.                                           |
| char | parityError     | Whether a parity error should be injected.                                                           |
| char | suppressFCT     | Whether an FCT should be suppressed to cause an error.                                               |

## 8.3 cfg\_api\_brick\_mk3.h File Reference

Functions used to configure STAR-Dundee Brick Mk3 and compatible devices.

```
#include "star-api.h"
#include "cfg_api_mk2_types.h"
#include "cfg_api_brick_mk3_types.h"
#include "cfg_api_router_types.h"
#include "cfg_api_router.h"
```

#### Functions

- int STAR\_API\_CC CFG\_BRICK\_MK3\_setBaseTransmitClock (STAR\_DEVICE\_ID deviceID, U8 linkNum, STAR\_CFG\_MK2\_BASE\_TRANSMIT\_CLOCK clockRateParams)  
*Sets the base transmit clock rate on a given link of a Brick Mk3 device:*
- int STAR\_API\_CC CFG\_BRICK\_MK3\_getBaseTransmitClock (STAR\_DEVICE\_ID deviceID, U8 linkNum, ↵ \_Out\_ STAR\_CFG\_MK2\_BASE\_TRANSMIT\_CLOCK \*pClockRateParams)  
*Gets the base transmit clock rate parameters for a given link of a Brick Mk3 device.*

- int STAR\_API\_CC\_CFG\_BRICK\_MK3\_setTransmitClock (STAR\_DEVICE\_ID deviceID, U8 linkNum, STAR\_CFG\_MK2\_BASE\_TRANSMIT\_CLOCK clockRateParams)  
*Sets the transmit clock rate on a given link.*
- int STAR\_API\_CC\_CFG\_BRICK\_MK3\_getTransmitClock (STAR\_DEVICE\_ID deviceID, U8 linkNum, \_Out\_ STAR\_CFG\_MK2\_BASE\_TRANSMIT\_CLOCK \*pClockRateParams)  
*Gets the transmit clock rate parameters for a given link.*
- int STAR\_API\_CC\_CFG\_BRICK\_MK3\_setTimestampMethod (STAR\_DEVICE\_ID deviceID, STAR\_CFG\_BRICK\_MK3\_TIMESTAMP\_METHOD timestampMethod)  
*Sets the timestamping method to use for the given device.*
- int STAR\_API\_CC\_CFG\_BRICK\_MK3\_getTimestampMethod (STAR\_DEVICE\_ID deviceID, \_Out\_ STAR\_CFG\_BRICK\_MK3\_TIMESTAMP\_METHOD \*pTimestampMethod)  
*Gets the timestamping method that is currently in use for the given device.*
- int STAR\_API\_CC\_CFG\_BRICK\_MK3\_enableRxTimestampEventsOnPort (STAR\_DEVICE\_ID deviceID, U8 portNum)  
*Selectively enables receive timestamp events on a given port.*
- int STAR\_API\_CC\_CFG\_BRICK\_MK3\_getRxTimestampEventsOnPortEnabled (STAR\_DEVICE\_ID deviceID, U8 portNum, \_Out\_ int \*pEnabled)  
*Gets whether receive timestamp events are enabled on a specific port.*
- int STAR\_API\_CC\_CFG\_BRICK\_MK3\_disableRxTimestampEventsOnPort (STAR\_DEVICE\_ID deviceID, U8 portNum)  
*Selectively disables receive timestamp events on a given port.*
- int STAR\_API\_CC\_CFG\_BRICK\_MK3\_setTimestampValue (STAR\_DEVICE\_ID deviceID, U32 value)  
*Sets the current timestamp value for the given device which will be incremented on the next synchronisation pulse.*
- int STAR\_API\_CC\_CFG\_BRICK\_MK3\_getTimestampValue (STAR\_DEVICE\_ID deviceID, \_Out\_ U32 \*pValue)  
*Gets the current timestamp value.*
- int STAR\_API\_CC\_CFG\_BRICK\_MK3\_setPulseGeneratorFrequency (STAR\_DEVICE\_ID deviceID, STAR\_CFG\_BRICK\_MK3\_PULSE\_FREQ frequency)  
*Sets the frequency of each generated pulse.*
- int STAR\_API\_CC\_CFG\_BRICK\_MK3\_getPulseGeneratorFrequency (STAR\_DEVICE\_ID deviceID, \_Out\_ STAR\_CFG\_BRICK\_MK3\_PULSE\_FREQ \*pFrequency)  
*Gets the frequency of each generated pulse.*

### 8.3.1 Detailed Description

Functions used to configure STAR-Dundee Brick Mk3 and compatible devices.

#### Author

STAR-Dundee Ltd.  
 STAR House  
 166 Nethergate  
 Dundee, DD1 4EE  
 Scotland, UK  
 e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2014 STAR-Dundee Ltd.

## 8.4 cfg\_api\_brick\_mk3\_types.h File Reference

Types used with the STAR-Dundee Brick Mk3 Configuration API.

## Enumerations

- enum STAR\_CFG\_BRICK\_MK3\_TIMESTAMP\_METHOD { STAR\_CFG\_BRICK\_MK3\_TIMESTAMP\_METHOD\_NONE = 0, STAR\_CFG\_BRICK\_MK3\_TIMESTAMP\_METHOD\_EXTERNAL\_TRIGGER = 1, STAR\_CFG\_BRICK\_MK3\_TIMESTAMP\_METHOD\_PULSE\_GENERATOR = 2 }

*Timestamps can be generated by either an external trigger signal or by an internal pulse generator.*

- enum STAR\_CFG\_BRICK\_MK3\_PULSE\_FREQ { STAR\_CFG\_BRICK\_MK3\_PULSE\_FREQ\_1 = 0, STAR\_CFG\_BRICK\_MK3\_PULSE\_FREQ\_10 = 1, STAR\_CFG\_BRICK\_MK3\_PULSE\_FREQ\_100 = 2, STAR\_CFG\_BRICK\_MK3\_PULSE\_FREQ\_1000 = 3 }

*Defines frequencies that can be used by the global pulse generator.*

### 8.4.1 Detailed Description

Types used with the STAR-Dundee Brick Mk3 Configuration API.

#### Author

STAR-Dundee Ltd.  
 STAR House  
 166 Nethergate  
 Dundee, DD1 4EE  
 Scotland, UK  
 e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2013 STAR-Dundee Ltd.

## 8.5 cfg\_api\_generic.c File Reference

Generic Configuration API used to configure all STAR-Dundee devices.

```
#include <transfer_types.h>
#include <stream_item_types.h>
#include <star-core.h>
#include <star-core_device.h>
#include <star-core_physical.h>
#include <star-core_shared_memory.h>
#include <general.h>
#include "spacewire_router_hardware_v53.h"
#include "cfg_api_utils.h"
#include "cfg_api_generic.h"
#include "cfg_api_mk2.h"
#include "cfg_api_mk2_types.h"
#include "cfg_api_brick_mk2.h"
#include "cfg_api_brick_mk3.h"
#include "cfg_api_router.h"
#include "cfg_api_router_mk2s.h"
#include "cfg_api_pci_mk2.h"
#include "cfg_api_pxi.h"
#include "cfg_api_gbe_brick.h"
```

## Functions

- int STAR\_API\_CFG\_setTimeCodeDistributionPorts (STAR\_DEVICE\_ID deviceId, U32 portMask)

*This function is used to specify which output ports time-codes are forwarded on.*

- int STAR\_API\_CC\_CFG\_getTimeCodeDistributionPorts (STAR\_DEVICE\_ID deviceID, \_Out\_ U32 \*pPortMask)  
*This function is used to obtain which output ports time-codes are forwarded on.*
- int STAR\_API\_CC\_CFG\_setTimeCodeFlagMode (STAR\_DEVICE\_ID deviceID, U8 mode)  
*Sets the time-code flag interpretation mode:*
- int STAR\_API\_CC\_CFG\_getTimeCodeFlagMode (STAR\_DEVICE\_ID deviceID, \_Out\_ U8 \*pMode)  
*Obtains the time-code flag interpretation mode:*
- int STAR\_API\_CC\_CFG\_getTimeCodeValue (STAR\_DEVICE\_ID deviceID, \_Out\_ U8 \*pValue)  
*Obtains the current value of the router's internal time-code counter.*
- int STAR\_API\_CC\_CFG\_disableTimeCodeMaster (STAR\_DEVICE\_ID deviceID)  
*Disables the device as a time-code master.*
- int STAR\_API\_CC\_CFG\_enableTimeCodeMaster (STAR\_DEVICE\_ID deviceID)  
*Enables the device as a time-code master.*
- int STAR\_API\_CC\_CFG\_getTimeCodeMasterEnabled (STAR\_DEVICE\_ID deviceID, \_Out\_ int \*pEnabled)  
*Gets whether the device has been enabled as a time-code master.*
- int STAR\_API\_CC\_CFG\_setTimeCodePeriod (STAR\_DEVICE\_ID deviceID, U32 period)  
*Sets the period between time-code master ticks.*
- int STAR\_API\_CC\_CFG\_getTimeCodePeriod (STAR\_DEVICE\_ID deviceID, \_Out\_ U32 \*pPeriod)  
*Gets the period between time-code master ticks.*
- int STAR\_API\_CC\_CFG\_disableExternalTimeCodeSelection (STAR\_DEVICE\_ID deviceID)  
*Disables external time-code selection for the device.*
- int STAR\_API\_CC\_CFG\_enableExternalTimeCodeSelection (STAR\_DEVICE\_ID deviceID)  
*Enables external time-code selection for the device.*
- int STAR\_API\_CC\_CFG\_getExternalTimeCodeSelectionEnabled (STAR\_DEVICE\_ID deviceID, \_Out\_ int \*pEnabled)  
*Gets whether external time-code selection has been enabled for the device.*
- int STAR\_API\_CC\_CFG\_disableTimeCodeEventsOnPort (STAR\_DEVICE\_ID deviceID, U8 portNum)  
*Selectively disables time-code notifications on a the channel attached to a given port.*
- int STAR\_API\_CC\_CFG\_enableTimeCodeEventsOnPort (STAR\_DEVICE\_ID deviceID, U8 portNum)  
*Selectively enables time-code notifications on a the channel attached to a given port.*
- int STAR\_API\_CC\_CFG\_getTimeCodeEventsOnPortEnabled (STAR\_DEVICE\_ID deviceID, U8 portNum, \_Out\_ int \*pEnabled)  
*Gets whether time-code notifications are enabled on a specific port.*
- int STAR\_API\_CC\_CFG\_disableTimeCodeCounterBypassMode (STAR\_DEVICE\_ID deviceID)  
*Disables time-code counter bypass mode for the device.*
- int STAR\_API\_CC\_CFG\_enableTimeCodeCounterBypassMode (STAR\_DEVICE\_ID deviceID)  
*Enables time-code counter bypass mode for the device.*
- int STAR\_API\_CC\_CFG\_getTimeCodeCounterBypassModeEnabled (STAR\_DEVICE\_ID deviceID, \_Out\_ int \*pEnabled)  
*Gets whether time-code counter bypass mode has been enabled for the device.*
- int STAR\_API\_CC\_CFG\_disableIdentifySourceOnPort (STAR\_DEVICE\_ID deviceID, U8 portNum)  
*Disables source port identification for a given port.*
- int STAR\_API\_CC\_CFG\_enableIdentifySourceOnPort (STAR\_DEVICE\_ID deviceID, U8 portNum)  
*Enables identification of source port during interface mode for a given port.*
- int STAR\_API\_CC\_CFG\_getIdentifySourceOnPortEnabled (STAR\_DEVICE\_ID deviceID, U8 portNum, \_Out\_ int \*pEnabled)  
*Gets whether identify source is enabled for a specific port.*
- int STAR\_API\_CC\_CFG\_disableInterfaceModeOnPort (STAR\_DEVICE\_ID deviceID, U8 portNum)  
*Selectively disables interface mode on a given port.*
- int STAR\_API\_CC\_CFG\_enableInterfaceModeOnPort (STAR\_DEVICE\_ID deviceID, U8 portNum)  
*Selectively enables interface mode on a given port.*

- `int STAR_API_CC_CFG_getInterfaceModeOnPortEnabled (STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int *pEnabled)`  
*Gets whether interface mode is enabled on a specific port.*
- `int STAR_API_CC_CFG_clearPortErrors (STAR_DEVICE_ID deviceID, U8 portNum)`  
*Clears all errors on a given port.*
- `int STAR_API_CC_CFG_getConfigPortErrors (PORT_STATUS_CONTROL portStatusControl, _Out_ STAR_CFG_CONFIG_PORT_ERRORS *pErrors)`  
*Gets any errors present on the config port.*
- `int STAR_API_CC_CFG_getDeviceIdentificationInfo (STAR_DEVICE_ID deviceID, _Out_ STAR_CFG_DEVICE_IDENTIFIER_INFO *pInfo)`  
*Reads the device identifier information of a device, i.e.*
- `void STAR_API_CC_CFG_getDeviceManufacturerAsString (_In_ STAR_CFG_DEVICE_IDENTIFIER_INFO *pDeviceIdentifierInfo, _Out_z_cap_c_ (STAR_CFG_MANUFACTURER_STR_MAX_LEN) char *pManufacturerStr)`  
*If the manufacturer is known, this function obtains a string representation of the manufacturers name.*
- `void STAR_API_CC_CFG_getDeviceTypeAsString (_In_ STAR_CFG_DEVICE_IDENTIFIER_INFO *pDeviceIdentifierInfo, _Out_z_cap_c_ (STAR_CFG_DEVICE_STR_MAX_LEN) char *pDeviceTypeStr)`  
*If the manufacturer and device type is known, this function obtains a string representation of the device's type.*
- `int STAR_API_CC_CFG_getExternalPortErrors (PORT_STATUS_CONTROL portStatusControl, _Out_ STAR_CFG_EXTERNAL_PORT_ERRORS *pErrors)`  
*Gets any errors present on an external port.*
- `int STAR_API_CC_CFG_getExternalPortStatus (PORT_STATUS_CONTROL portStatusControl, _Out_ STAR_CFG_EXTERNAL_PORT_STATUS *pStatus)`  
*Gets the status of an external port.*
- `int STAR_API_CC_CFG_getGeneralPurpose (STAR_DEVICE_ID deviceID, _Out_ REGISTER *pValue)`  
*Gets the value of the general purpose register.*
- `int STAR_API_CC_CFG_getNetworkDiscoveryInfo (STAR_DEVICE_ID deviceID, _Out_ STAR_CFG_NETWORK_DISCOVERY_INFO *pInfo)`  
*Reads the network discovery information of a device.*
- `int STAR_API_CC_CFG_getNetworkIdentity (STAR_DEVICE_ID deviceID, _Out_ REGISTER *pValue)`  
*Gets the value of the network identity register.*
- `int STAR_API_CC_CFG_getPortConnection (PORT_STATUS_CONTROL portStatusControl)`  
*Identifies the output port to which the source port is currently connected whilst routing is in operation.*
- `int STAR_API_CC_CFG_getPortStatusControl (STAR_DEVICE_ID deviceID, U8 portNum, _Out_ PORT_STATUS_CONTROL *pPortStatusControl)`  
*Gets the value of a port or link's status/control register, which can then be used by the various functions within the Port Status and Control group.*
- `STAR_CFG_PORT_TYPE STAR_API_CC_CFG_getPortType (PORT_STATUS_CONTROL portStatusControl)`  
*Identifies the type of port attributed to a given port number.*
- `int STAR_API_CC_CFG_getRouterGlobalSettings (STAR_DEVICE_ID deviceID, _Out_ STAR_CFG_ROUTER_GLOBAL_STATE *pState)`  
*Gets the router's global settings.*
- `int STAR_API_CC_CFG_getRoutingTableEntry (STAR_DEVICE_ID deviceID, U8 logicalAddress, _Out_ STAR_CFG_GAR_ENTRY *pEntry)`  
*Gets a routing table entry for a given logical address.*
- `int STAR_API_CC_CFG_getSpaceWireLinkErrors (PORT_STATUS_CONTROL portStatusControl, _Out_ STAR_CFG_SPW_LINK_ERRORS *pErrors)`  
*Gets any errors present on a SpaceWire link.*
- `int STAR_API_CC_CFG_getSpaceWireLinkStatus (PORT_STATUS_CONTROL portStatusControl, _Out_ STAR_CFG_SPW_LINK_STATUS *pLinkStatus)`  
*Gets the status of a SpaceWire Link.*
- `int STAR_API_CC_CFG_setGeneralPurpose (STAR_DEVICE_ID deviceID, REGISTER value)`

- Sets the value of the general purpose register.*

  - int `STAR_API_CC_CFG_setNetworkIdentity` (STAR\_DEVICE\_ID deviceId, REGISTER value)

*Sets the value of the network identity register.*
- int `STAR_API_CC_CFG_setRouterGlobalSettings` (STAR\_DEVICE\_ID deviceId, \_In\_ STAR\_CFG\_ROUTER\_GLOBAL\_STATE \*pState)

*Sets the router's global settings.*
- int `STAR_API_CC_CFG_setRoutingTableEntry` (STAR\_DEVICE\_ID deviceId, U8 logicalAddress, \_In\_ STAR\_CFG\_ROUTING\_TABLE\_ENTRY \*pEntry)

*Sets a routing table entry for a given logical address.*
- int `STAR_API_CC_CFG_setSpaceWireLinkStatus` (STAR\_DEVICE\_ID deviceId, U8 portNum, \_In\_ STAR\_CFG\_SPW\_LINK\_STATUS \*pLinkStatus)

*Sets the state of a SpaceWire Link.*
- int `STAR_API_CC_CFG_startLink` (STAR\_DEVICE\_ID deviceId, U8 portNum)

*Starts a SpaceWire link by setting the start bit, and clearing the disable bit.*
- int `STAR_API_CC_CFG_stopLink` (STAR\_DEVICE\_ID deviceId, U8 portNum)

*Stops a SpaceWire link by clearing the start bit, and setting the disable bit.*
- int `STAR_API_CC_CFG_disableIdentifySource` (STAR\_DEVICE\_ID deviceId)

*Disables identification of source ports for interface mode globally.*
- int `STAR_API_CC_CFG_disableInterfaceMode` (STAR\_DEVICE\_ID deviceId)

*Disables interface mode on a device.*
- int `STAR_API_CC_CFG_enableIdentifySource` (STAR\_DEVICE\_ID deviceId)

*When interface mode is enabled on a port, this function globally enables the addition of a leading byte to each received packet indicating which port the packet was received on.*
- int `STAR_API_CC_CFG_enableInterfaceMode` (STAR\_DEVICE\_ID deviceId)

*In interface mode a packet which is received on an external port, from a SpaceWire link or from the configuration port will be routed to the port specified by the port routing register of the port (set by `CFG_setPortRoutingAddress()`).*
- int `STAR_API_CC_CFG_getFPGAInfo` (STAR\_DEVICE\_ID deviceId, \_Out\_ STAR\_CFG\_FPGA\_INFO \*pFPGAInfo)

*Reads information about the hardware version of a device and updates the `STAR_CFG_FPGA_INFO` structure pointed to by the passed in pointer to contain the received version information.*
- void `STAR_API_CC_CFG_FPGAInfoToString` (STAR\_DEVICE\_ID deviceId, \_In\_ STAR\_CFG\_FPGA\_INFO \*pFPGAInfo, \_Out\_z\_cap\_c\_ (STAR\_CFG\_MK2\_VERSION\_STR\_MAX\_LEN) char \*pVersion, \_Out\_z\_cap\_c\_ (STAR\_CFG\_MK2\_BUILD\_DATE\_STR\_MAX\_LEN) char \*pBuildDate)

*Creates a string representation of a `STAR_CFG_FPGA_INFO` structure.*
- int `STAR_API_CC_CFG_identify` (STAR\_DEVICE\_ID deviceId)

*Flashes the front panel LEDs of a device.*
- int `STAR_API_CC_CFG_getIdentifySourceEnabled` (STAR\_DEVICE\_ID deviceId, \_Out\_ int \*pEnabled)

*Gets whether identify source is enabled.*
- int `STAR_API_CC_CFG_getInterfaceModeEnabled` (STAR\_DEVICE\_ID deviceId, \_Out\_ int \*pEnabled)

*Gets whether interface mode has been enabled on a device.*
- int `STAR_API_CC_CFG_getMeasuredLinkSpeed` (STAR\_DEVICE\_ID deviceId, U8 portNum, \_Out\_ U16 \*pLinkSpeed)

*Gets the current link speed measured on a specific port.*
- int `STAR_API_CC_CFG_disableSpeedChangeEventsOnPort` (STAR\_DEVICE\_ID deviceId, U8 portNum)

*Selectively disables speed change events on a given port.*
- int `STAR_API_CC_CFG_enableSpeedChangeEventsOnPort` (STAR\_DEVICE\_ID deviceId, U8 portNum)

*Selectively enables speed change events on a given port.*
- int `STAR_API_CC_CFG_getSpeedChangeEventsOnPortEnabled` (STAR\_DEVICE\_ID deviceId, U8 portNum, \_Out\_ int \*pEnabled)

*Gets whether speed change events are enabled on a specific port.*
- int `STAR_API_CC_CFG_disableStateChangeEventsOnPort` (STAR\_DEVICE\_ID deviceId, U8 portNum)

*Selectively disables state change events on a given port.*



- `int STAR_API_CC_CFG_enableStateChangeEventsOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`  
*Selectively enables state change events on a given port.*
- `int STAR_API_CC_CFG_getStateChangeEventsOnPortEnabled (STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int *pEnabled)`  
*Gets whether state change events are enabled on a specific port.*
- `int STAR_API_CC_CFG_getPortRoutingAddress (STAR_DEVICE_ID deviceID, U8 portNum, _Out_ U8 *pAddress)`  
*Gets the address a packet received on a given port should be routed to when interface mode is enabled.*
- `int STAR_API_CC_CFG_setPortRoutingAddress (STAR_DEVICE_ID deviceID, U8 portNum, U8 address)`  
*Sets the address a packet received on a given port should be routed to when interface mode is enabled.*
- `int STAR_API_CC_CFG_injectError (STAR_DEVICE_ID deviceID, U8 portNum, SPW_ERROR error)`  
*Immediately injects the specified error on a given device's port.*
- `int STAR_API_CC_CFG_injectErrors (STAR_DEVICE_ID deviceID, U8 portNum, _In_ STAR_CFG_BRICK_MK2_ERRORS *pErrors)`  
*Inject the specified errors on a given port.*
- `int STAR_API_CC_CFG_disablePrecisionTransmitRate (STAR_DEVICE_ID deviceID)`  
*Disables precision transmit rate.*
- `int STAR_API_CC_CFG_enablePrecisionTransmitRate (STAR_DEVICE_ID deviceID)`  
*Enables precision transmit rate.*
- `int STAR_API_CC_CFG_getPrecisionTransmitRate (STAR_DEVICE_ID deviceID, _Out_ double *pTransmitRate)`  
*Get the current precision transmit rate on a device in Mbit/s.*
- `int STAR_API_CC_CFG_getPrecisionTransmitRateEnabled (STAR_DEVICE_ID deviceID, _Out_ int *pEnabled)`  
*Determine if precision transmit rate is enabled.*
- `int STAR_API_CC_CFG_getPrecisionTransmitRateInUse (STAR_DEVICE_ID deviceID, _Out_ int *pInUse)`  
*Determine whether precision transmit rate is in use.*
- `int STAR_API_CC_CFG_setPrecisionTransmitRate (STAR_DEVICE_ID deviceID, double TransmitRate)`  
*Set the current precision transmit rate on a device in Mbit/s.*
- `int STAR_API_CC_CFG_getDeviceClockRate (STAR_DEVICE_ID deviceID)`  
*Gets a device's internal clock rate.*
- `int STAR_API_CC_CFG_startPeriodicAction (STAR_DEVICE_ID deviceID, unsigned char port, U32 count, SPW_ACTION action)`  
*Starts a periodic action on a given device's port.*
- `int STAR_API_CC_CFG_stopPeriodicAction (STAR_DEVICE_ID deviceID, unsigned char port)`  
*Stops a periodic action on a given device's port.*
- `int STAR_API_CC_CFG_getTimestampMethod (STAR_DEVICE_ID deviceID, _Out_ STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD *pMethod)`  
*Gets the timestamping method that is currently in use for the given device.*
- `int STAR_API_CC_CFG_setTimestampMethod (STAR_DEVICE_ID deviceID, STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD method)`  
*Sets the timestamping method to use for the given device.*
- `int STAR_API_CC_CFG_disableRxTimestampEventsOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`  
*Selectively disables receive timestamp events on a given port.*
- `int STAR_API_CC_CFG_enableRxTimestampEventsOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`  
*Selectively enables receive timestamp events on a given port.*
- `int STAR_API_CC_CFG_getRxTimestampEventsOnPortEnabled (STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int *pEnabled)`  
*Gets whether receive timestamp events are enabled on a specific port.*
- `int STAR_API_CC_CFG_setTimestampValue (STAR_DEVICE_ID deviceID, U32 value)`  
*Sets the current timestamp value for the given device which will be incremented on the next synchronisation pulse.*
- `int STAR_API_CC_CFG_getTimestampValue (STAR_DEVICE_ID deviceID, _Out_ U32 *pValue)`



*Gets the current timestamp value.*

- int STAR\_API\_CC\_CFG\_setPulseGeneratorFrequency (STAR\_DEVICE\_ID deviceId, STAR\_CFG\_BRICK↔\_MK3\_PULSE\_FREQ freq)

*Sets the frequency of each generated pulse.*

- int STAR\_API\_CC\_CFG\_getPulseGeneratorFrequency (STAR\_DEVICE\_ID deviceId, \_Out\_ STAR\_CFG↔\_BRICK\_MK3\_PULSE\_FREQ \*pFreq)

*Gets the frequency of each generated pulse.*

- int STAR\_API\_CC\_CFG\_getLinkBitRate (STAR\_DEVICE\_ID deviceId, U8 linkNum, \_Out\_ U32 \*pBitRate)
- int STAR\_API\_CC\_CFG\_setLinkBitRate (STAR\_DEVICE\_ID deviceId, U8 linkNum, U32 bitRate)
- int STAR\_API\_CC\_CFG\_getLinkRateDivider (STAR\_DEVICE\_ID deviceId, U8 linkNum, \_Out\_ U8 \*p↔Divider)
- int STAR\_API\_CC\_CFG\_setLinkRateDivider (STAR\_DEVICE\_ID deviceId, U8 linkNum, U8 divider)
- int STAR\_API\_CC\_CFG\_getTransmitClock (STAR\_DEVICE\_ID deviceId, U8 linkNum, \_Out\_ STAR\_CF↔G\_MK2\_BASE\_TRANSMIT\_CLOCK \*pClockRateParams)
- int STAR\_API\_CC\_CFG\_setTransmitClock (STAR\_DEVICE\_ID deviceId, U8 linkNum, \_In\_ STAR\_CFG↔\_MK2\_BASE\_TRANSMIT\_CLOCK \*pClockRateParams)
- int STAR\_API\_CC\_CFG\_getTransmitSignallingRate (STAR\_DEVICE\_ID deviceId, U8 linkNum, \_Out\_ U32 \*pSignallingRate)

*Gets the transmit bit rate for a given link.*

- int STAR\_API\_CC\_CFG\_setTransmitSignallingRate (STAR\_DEVICE\_ID deviceId, U8 linkNum, U32 signallingRate)

*Sets the transmit bit rate on a given link.*

### 8.5.1 Detailed Description

Generic Configuration API used to configure all STAR-Dundee devices.

#### Author

STAR-Dundee Ltd.  
STAR House  
166 Nethergate  
Dundee, DD1 4EE  
Scotland, UK  
e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2012 STAR-Dundee Ltd.

The content of this source file is CONFIDENTIAL and PROPRIETARY property of STAR-Dundee Ltd.

### 8.5.2 Function Documentation

#### 8.5.2.1 int STAR\_API\_CC\_CFG\_getRouterGlobalSettings ( STAR\_DEVICE\_ID deviceId, \_Out\_ STAR\_CFG\_ROUTER\_GLOBAL\_STATE \* pState )

Gets the router's global settings.

#### Parameters

|            |                 |                                     |
|------------|-----------------|-------------------------------------|
|            | <i>deviceId</i> | Device to get global settings from. |
| <i>out</i> | <i>pState</i>   | Global settings for the router.     |

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

**Examples:**

rmap\_target\_example.c, router\_configuration\_example.c, and timestamp\_test.c.

### 8.5.2.2 int STAR\_API\_CC\_CFG\_getTimeCodeFlagMode ( STAR\_DEVICE\_ID *deviceId*, \_Out\_ U8 \* *pMode* )

Obtains the time-code flag interpretation mode:

0 - Time-code control bit flags are distributed with valid time-code values regardless of the value of the time-code control flags

1 - When the time-code control bit flags are "00" then valid time-codes are distributed. When the time-code control flags are not "00" then the time-code is discarded and the internal time-code register is not updated.

**Parameters**

|     |                 |                                             |
|-----|-----------------|---------------------------------------------|
|     | <i>deviceId</i> | Device to get the time-code flag mode from. |
| out | <i>pMode</i>    | Time code flag mode,                        |

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

**Examples:**

router\_configuration\_example.c.

### 8.5.2.3 int STAR\_API\_CC\_CFG\_getTimeCodeValue ( STAR\_DEVICE\_ID *deviceId*, \_Out\_ U8 \* *pValue* )

Obtains the current value of the router's internal time-code counter.

• **Parameters**

---

|     |                 |                                             |
|-----|-----------------|---------------------------------------------|
|     | <i>deviceId</i> | Device to get the time-code flag mode from. |
| out | <i>pValue</i>   | Time-code value,                            |

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

**Examples:**

router\_configuration\_example.c.

**8.5.2.4** `int STAR_API_CC_CFG_setRouterGlobalSettings ( STAR_DEVICE_ID deviceId, _In_ STAR_CFG_ROUTER_GLOBAL_STATE * pState )`

Sets the router's global settings.

**Parameters**

|    |                 |                                    |
|----|-----------------|------------------------------------|
|    | <i>deviceId</i> | Device to set global settings for. |
| in | <i>pState</i>   | Global settings for the router.    |

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

**Examples:**

rmap\_target\_example.c, router\_configuration\_example.c, and timestamp\_test.c.

**8.5.2.5** `int STAR_API_CC_CFG_setTimeCodeFlagMode ( STAR_DEVICE_ID deviceId, U8 mode )`

Sets the time-code flag interpretation mode:

0 - Time-code control bit flags are distributed with valid time-code values regardless of the value of the time-code control flags

1 - When the time-code control bit flags are "00" then valid time-codes are distributed. When the time-code control flags are not "00" then the time-code is discarded and the internal time-code register is not updated.

**Parameters**

|                 |                                             |
|-----------------|---------------------------------------------|
| <i>deviceID</i> | Device to set the time-code flag modes for. |
| <i>mode</i>     | Mode to set the time code flag mode to,     |

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

## 8.6 `cfg_api_generic.h` File Reference

Types used with the STAR-Dundee Generic Configuration API.

```
#include "star-api.h"
#include "cfg_api_remote.h"
#include "cfg_api_mk2.h"
#include "cfg_api_mk2_types.h"
#include "cfg_api_brick_mk2_types.h"
#include "cfg_api_brick_mk3_types.h"
#include "cfg_api_router_types.h"
#include "cfg_api_generic_common.h"
```

**Macros**

- `#define CFG_STATUS_SUCCESS (0)`  
*Positive retvals are success, negative retvals are errors.*
- `#define CFG_SUCCESS(status) ((status) >= CFG_STATUS_SUCCESS ? 1 : 0)`  
*Macro to allow testing for success or failure.*
- `#define CFG_STATUS_FAILURE (-256)`  
*The API call has failed.*
- `#define CFG_STATUS_NOT_COMPATIBLE_WITH_THIS_DEVICE_TYPE (-257)`  
*The API call is not compatible with this device type.*
- `#define CFG_STATUS_NOT_COMPATIBLE_WITH_THIS_DEVICES_FPGA_VERSION (-258)`  
*The API call is not compatible with this device's FPGA version.*
- `#define CFG_STATUS_GET_DEVICE_INFO_FAILURE (-259)`  
*Failed to get "Device Information" for this device.*
- `#define CFG_STATUS_TIME_CODE_DISTRIBUTION_PORT_MASK_IS_INVALID (-260)`  
*The Time-code distribution port mask is invalid.*
- `#define CFG_STATUS_TIME_CODE_FLAG_MODE_IS_INVALID (-261)`  
*The Time-code flag mode is invalid.*
- `#define CFG_STATUS_TIME_CODE_PERIOD_IS_INVALID (-262)`  
*The Time-code period is invalid.*
- `#define CFG_STATUS_TIME_CODE_PORT_IS_INVALID (-263)`  
*The Time-code port is invalid.*

- `#define CFG_STATUS_IDENTIFY_SOURCE_PORT_IS_INVALID (-264)`  
*The Identify Source port is invalid.*
- `#define CFG_STATUS_INTERFACE_MODE_PORT_IS_INVALID (-265)`  
*The Interface Mode port is invalid.*
- `#define CFG_STATUS_PORT_IS_INVALID (-266)`  
*The port is invalid.*
- `#define CFG_STATUS_LOGICAL_ADDRESS_IS_INVALID (-267)`  
*The logical address is invalid.*
- `#define CFG_STATUS_SPACEWIRE_LINK_IS_INVALID (-268)`  
*The SpaceWire link is invalid.*
- `#define CFG_STATUS_MEASURED_SPEED_PORT_IS_INVALID (-269)`  
*The measured port speed is invalid.*
- `#define CFG_STATUS_SPEED_CHANGE_EVENTS_PORT_IS_INVALID (-270)`  
*The Speed Change Events port is invalid.*
- `#define CFG_STATUS_STATE_CHANGE_EVENTS_PORT_IS_INVALID (-271)`  
*The State Change Events port is invalid.*
- `#define CFG_STATUS_PORT_ROUTING_PORT_IS_INVALID (-272)`  
*The Port Routing port is invalid.*
- `#define CFG_STATUS_INJECT_ERROR_PORT_IS_INVALID (-273)`  
*The Inject Error port is invalid.*
- `#define CFG_STATUS_PRECISION_TRANSMIT_RATE_IS_INVALID (-274)`  
*The Precision Transmit Rate port is invalid.*
- `#define CFG_STATUS_PERIODIC_ACTION_PORT_IS_INVALID (-275)`  
*The Periodic Action port is invalid.*
- `#define CFG_STATUS_TIMESTAMP_EVENTS_PORT_IS_INVALID (-276)`  
*The Timestamp Events port is invalid.*
- `#define CFG_STATUS_CLOCK_LINK_IS_INVALID (-277)`  
*The Clock link is invalid.*
- `#define CFG_STATUS_LINK_RATE_DIVIDER_IS_INVALID (-278)`  
*The Link Rate divider is invalid.*
- `#define CFG_STATUS_TRANSMIT_CLOCK_LINK_IS_INVALID (-279)`  
*The Transmit Clock link is invalid.*
- `#define CFG_STATUS_BIT_RATE_IS_INVALID (-280)`  
*The Bit-rate is invalid.*
- `#define CFG_STATUS_ROUTER_TIMEOUT_MODE_IS_INVALID (-281)`  
*The Router Timeout mode is invalid.*
- `#define CFG_STATUS_ROUTER_TIMEOUT_PERIOD_IS_INVALID (-282)`  
*The Router Timeout period is invalid.*
- `#define CFG_STATUS_ROUTER_DISABLE_ON_SILENCE_IS_INVALID (-283)`  
*The Router Disable-on-Silence is invalid.*
- `#define CFG_STATUS_ROUTER_START_ON_REQUEST_IS_INVALID (-284)`  
*The Router Start-on-Request is invalid.*
- `#define CFG_STATUS_ROUTER_ENABLE_SELF_ADDRESSING_IS_INVALID (-285)`  
*The Router Enable-Self-Addressing is invalid.*
- `#define CFG_STATUS_GROUP_ADAPTIVE_ROUTING_PORT_MASK_IS_INVALID (-286)`  
*The Group Adaptive Routing port mask is invalid.*
- `#define CFG_STATUS_GROUP_ADAPTIVE_ROUTING_PRIORITY_IS_INVALID (-287)`  
*The Group Adaptive Routing priority is invalid.*
- `#define CFG_STATUS_GROUP_ADAPTIVE_ROUTING_DELETE_HEADER_IS_INVALID (-288)`  
*The Group Adaptive Routing delete header is invalid.*
- `#define CFG_STATUS_GROUP_ADAPTIVE_ROUTING_ADDRESS_IS_INVALID (-289)`

- The Group Adaptive Routing address is invalid.*

  - #define `CFG_STATUS_SPACEWIRE_LINK_TRISTATE_IS_INVALID` (-290)
- The SpaceWire link tri-state flag is invalid.*

  - #define `CFG_STATUS_SPACEWIRE_LINK_DISABLE_IS_INVALID` (-291)
- The SpaceWire link disable flag is invalid.*

  - #define `CFG_STATUS_SPACEWIRE_LINK_START_IS_INVALID` (-292)
- The SpaceWire link start flag is invalid.*

  - #define `CFG_STATUS_SPACEWIRE_LINK_AUTOSTART_IS_INVALID` (-293)
- The SpaceWire link autostart flag is invalid.*

  - #define `CFG_STATUS_SPACEWIRE_LINK_RUNNING_IS_INVALID` (-294)
- The SpaceWire link running flag is invalid.*

  - #define `CFG_STATUS_SPACEWIRE_LINK_STATE_IS_INVALID` (-295)
- The SpaceWire link state is invalid.*

  - #define `CFG_STATUS_PORT_ROUTING_ADDRESS_IS_INVALID` (-296)
- The Port Routing address is invalid.*

  - #define `CFG_STATUS_INJECT_ERROR_ERROR_IS_INVALID` (-297)
- The Inject Error error is invalid.*

  - #define `CFG_STATUS_INJECT_ERRORS_PARITY_ERROR_IS_INVALID` (-298)
- The Inject Errors parity error is invalid.*

  - #define `CFG_STATUS_INJECT_ERRORS_ESCAPE_ERROR_IS_INVALID` (-299)
- The Inject Errors escape error is invalid.*

  - #define `CFG_STATUS_INJECT_ERRORS_INSERT_FCT_ERROR_IS_INVALID` (-300)
- The Inject Errors insert FCT error is invalid.*

  - #define `CFG_STATUS_INJECT_ERRORS_SUPPRESS_FCT_ERROR_IS_INVALID` (-301)
- The Inject Errors suppress FCT error is invalid.*

  - #define `CFG_STATUS_INJECT_ERRORS_INCREMENT_CREDIT_ERROR_IS_INVALID` (-302)
- The Inject Errors increment credit error is invalid.*

  - #define `CFG_STATUS_INJECT_ERRORS_DECREMENT_CREDIT_ERROR_IS_INVALID` (-303)
- The Inject Errors decrement credit error is invalid.*

  - #define `CFG_STATUS_INJECT_ERRORS_DISCONNECT_ERROR_IS_INVALID` (-304)
- The Inject Errors disconnect error is invalid.*

  - #define `CFG_STATUS_PERIODIC_ACTION_ACTION_IS_INVALID` (-305)
- The Periodic Action is invalid.*

  - #define `CFG_STATUS_TIMESTAMP_METHOD_IS_INVALID` (-306)
- The Timestamp method is invalid.*

  - #define `CFG_STATUS_PULSE_GENERATOR_FREQUENCY_IS_INVALID` (-307)
- The Pulse Generator frequency is invalid.*

  - #define `CFG_STATUS_CLOCK_RATE_MULTIPLIER_IS_INVALID` (-308)
- The Clock Rate multiplier is invalid.*

  - #define `CFG_STATUS_CLOCK_RATE_DIVISOR_IS_INVALID` (-309)
- The Clock Rate divisor is invalid.*

  - #define `CFG_STATUS_CLOCK_RATE_PARAMETERS_ARE_INVALID` (-310)
- The Clock Rate parameters are invalid.*

  - #define `CFG_STATUS_CANT_GET_CLOCK_RATE_PARAMETERS_FROM_BIT_RATE` (-311)
- Can't get Clock Rate parameters from bit-rate.*

  - #define `CFG_STATUS_POINTER_PARAMETER_IS_NULL` (-312)
- The pointer parameter is NULL.*

  - #define `CFG_STATUS_TRANSMIT_BIT_RATE_IS_INVALID` (-313)
- The Transmit bit-rate is invalid.*

## Functions

- int `STAR_API_CC_CFG_updateDeviceInfo` (STAR\_DEVICE\_ID deviceId)  
*This function is used to update a device's information held in the shared memory.*
- int `STAR_API_CC_CFG_getNetworkIdentity` (STAR\_DEVICE\_ID deviceId, \_Out\_ REGISTER \*pValue)  
*Gets the value of the network identity register.*
- int `STAR_API_CC_CFG_setNetworkIdentity` (STAR\_DEVICE\_ID deviceId, REGISTER value)  
*Sets the value of the network identity register.*
- int `STAR_API_CC_CFG_getTimeCodeDistributionPorts` (STAR\_DEVICE\_ID deviceId, \_Out\_ U32 \*pPortMask)  
*This function is used to obtain which output ports time-codes are forwarded on.*
- int `STAR_API_CC_CFG_setTimeCodeDistributionPorts` (STAR\_DEVICE\_ID deviceId, U32 portMask)  
*This function is used to specify which output ports time-codes are forwarded on.*
- int `STAR_API_CC_CFG_getGeneralPurpose` (STAR\_DEVICE\_ID deviceId, \_Out\_ REGISTER \*pValue)  
*Gets the value of the general purpose register.*
- int `STAR_API_CC_CFG_setGeneralPurpose` (STAR\_DEVICE\_ID deviceId, REGISTER value)  
*Sets the value of the general purpose register.*
- int `STAR_API_CC_CFG_getFPGAInfo` (STAR\_DEVICE\_ID deviceId, \_Out\_ STAR\_CFG\_FPGA\_INFO \*pFPGAInfo)  
*Reads information about the hardware version of a device and updates the STAR\_CFG\_FPGA\_INFO structure pointed to by the passed in pointer to contain the received version information.*
- void `STAR_API_CC_CFG_FPGAInfoToString` (STAR\_DEVICE\_ID deviceId, \_In\_ STAR\_CFG\_FPGA\_INFO \*pFPGAInfo, \_Out\_z\_cap\_c\_ (STAR\_CFG\_MK2\_VERSION\_STR\_MAX\_LEN) char \*pVersion, \_Out\_z\_cap\_c\_ (STAR\_CFG\_MK2\_BUILD\_DATE\_STR\_MAX\_LEN) char \*pBuildDate)  
*Creates a string representation of a STAR\_CFG\_FPGA\_INFO structure.*
- int `STAR_API_CC_CFG_identify` (STAR\_DEVICE\_ID deviceId)  
*Flashes the front panel LEDs of a device.*
- int `STAR_API_CC_CFG_clearPortErrors` (STAR\_DEVICE\_ID deviceId, U8 portNum)  
*Clears all errors on a given port.*
- int `STAR_API_CC_CFG_getConfigPortErrors` (PORT\_STATUS\_CONTROL portStatusControl, \_Out\_ STAR\_CFG\_CONFIG\_PORT\_ERRORS \*pErrors)  
*Gets any errors present on the config port.*
- int `STAR_API_CC_CFG_setSpaceWireLinkStatus` (STAR\_DEVICE\_ID deviceId, U8 linkNum, \_In\_ STAR\_CFG\_SPW\_LINK\_STATUS \*pLinkStatus)  
*Sets the state of a SpaceWire Link.*
- int `STAR_API_CC_CFG_startLink` (STAR\_DEVICE\_ID deviceId, U8 linkNum)  
*Starts a SpaceWire link by setting the start bit, and clearing the disable bit.*
- int `STAR_API_CC_CFG_stopLink` (STAR\_DEVICE\_ID deviceId, U8 linkNum)  
*Stops a SpaceWire link by clearing the start bit, and setting the disable bit.*
- int `STAR_API_CC_CFG_getSpaceWireLinkErrors` (PORT\_STATUS\_CONTROL portStatusControl, \_Out\_ STAR\_CFG\_SPW\_LINK\_ERRORS \*pErrors)  
*Gets any errors present on a SpaceWire link.*
- int `STAR_API_CC_CFG_getSpaceWireLinkStatus` (PORT\_STATUS\_CONTROL portStatusControl, \_Out\_ STAR\_CFG\_SPW\_LINK\_STATUS \*pStatus)  
*Gets the status of a SpaceWire Link.*
- int `STAR_API_CC_CFG_getPortStatusControl` (STAR\_DEVICE\_ID deviceId, U8 portNum, \_Out\_ PORT\_STATUS\_CONTROL \*pPortStatusControl)  
*Gets the value of a port or link's status/control register, which can then be used by the various functions within the Port Status and Control group.*
- `STAR_CFG_PORT_TYPE` `STAR_API_CC_CFG_getPortType` (PORT\_STATUS\_CONTROL portStatusControl)  
*Identifies the type of port attributed to a given port number.*

- `int STAR_API_CC_CFG_getExternalPortErrors (PORT_STATUS_CONTROL portStatusControl, _Out_ STAR_CFG_EXTERNAL_PORT_ERRORS *pErrors)`  
*Gets any errors present on an external port.*
- `int STAR_API_CC_CFG_getExternalPortStatus (PORT_STATUS_CONTROL portStatusControl, _Out_ STAR_CFG_EXTERNAL_PORT_STATUS *pStatus)`  
*Gets the status of an external port.*
- `int STAR_API_CC_CFG_getPortConnection (PORT_STATUS_CONTROL portStatusControl)`  
*Identifies the output port to which the source port is currently connected whilst routing is in operation.*
- `int STAR_API_CC_CFG_getNetworkDiscoveryInfo (STAR_DEVICE_ID deviceId, _Out_ STAR_CFG_NETWORK_DISCOVERY_INFO *pInfo)`  
*Reads the network discovery information of a device.*
- `int STAR_API_CC_CFG_getDeviceIdentificationInfo (STAR_DEVICE_ID deviceId, _Out_ STAR_CFG_DEVICE_IDENTIFIER_INFO *pInfo)`  
*Reads the device identifier information of a device, i.e.*
- `void STAR_API_CC_CFG_getDeviceManufacturerAsString (_In_ STAR_CFG_DEVICE_IDENTIFIER_INFO *pDeviceIdentifierInfo, _Out_z_cap_c_ (STAR_CFG_MANUFACTURER_STR_MAX_LEN) char *pManufacturerStr)`  
*If the manufacturer is known, this function obtains a string representation of the manufacturers name.*
- `void STAR_API_CC_CFG_getDeviceTypeAsString (_In_ STAR_CFG_DEVICE_IDENTIFIER_INFO *pDeviceIdentifierInfo, _Out_z_cap_c_ (STAR_CFG_DEVICE_STR_MAX_LEN) char *pDeviceTypeStr)`  
*If the manufacturer and device type is known, this function obtains a string representation of the device's type.*
- `int STAR_API_CC_CFG_getRouterGlobalSettings (STAR_DEVICE_ID deviceId, _Out_ STAR_CFG_ROUTER_GLOBAL_STATE *pState)`  
*Gets the router's global settings.*
- `int STAR_API_CC_CFG_setRouterGlobalSettings (STAR_DEVICE_ID deviceId, _In_ STAR_CFG_ROUTER_GLOBAL_STATE *pState)`  
*Sets the router's global settings.*
- `int STAR_API_CC_CFG_getTimeCodeFlagMode (STAR_DEVICE_ID deviceId, _Out_ U8 *pMode)`  
*Obtains the time-code flag interpretation mode:*
- `int STAR_API_CC_CFG_setTimeCodeFlagMode (STAR_DEVICE_ID deviceId, U8 mode)`  
*Sets the time-code flag interpretation mode:*
- `int STAR_API_CC_CFG_getTimeCodeValue (STAR_DEVICE_ID deviceId, _Out_ U8 *pValue)`  
*Obtains the current value of the router's internal time-code counter.*
- `int STAR_API_CC_CFG_getRoutingTableEntry (STAR_DEVICE_ID deviceId, U8 logicalAddress, _Out_ STAR_CFG_ROUTING_TABLE_ENTRY *pEntry)`  
*Gets a routing table entry for a given logical address.*
- `int STAR_API_CC_CFG_setRoutingTableEntry (STAR_DEVICE_ID deviceId, U8 logicalAddress, _In_ STAR_CFG_ROUTING_TABLE_ENTRY *pEntry)`  
*Sets a routing table entry for a given logical address.*
- `int STAR_API_CC_CFG_getPortRoutingAddress (STAR_DEVICE_ID deviceId, U8 portNum, _Out_ U8 *pAddress)`  
*Gets the address a packet received on a given port should be routed to when interface mode is enabled.*
- `int STAR_API_CC_CFG_setPortRoutingAddress (STAR_DEVICE_ID deviceId, U8 portNum, U8 address)`  
*Sets the address a packet received on a given port should be routed to when interface mode is enabled.*
- `int STAR_API_CC_CFG_disableIdentifySource (STAR_DEVICE_ID deviceId)`  
*Disables identification of source ports for interface mode globally.*
- `int STAR_API_CC_CFG_enableIdentifySource (STAR_DEVICE_ID deviceId)`  
*When interface mode is enabled on a port, this function globally enables the addition of a leading byte to each received packet indicating which port the packet was received on.*
- `int STAR_API_CC_CFG_getIdentifySourceEnabled (STAR_DEVICE_ID deviceId, _Out_ int *pEnabled)`  
*Gets whether identify source is enabled.*
- `int STAR_API_CC_CFG_disableIdentifySourceOnPort (STAR_DEVICE_ID deviceId, U8 portNum)`



- Disables source port identification for a given port.*

  - int [STAR\\_API\\_CC\\_CFG\\_enableIdentifySourceOnPort](#) (STAR\_DEVICE\_ID deviceId, U8 portNum)

*Enables identification of source port during interface mode for a given port.*

  - int [STAR\\_API\\_CC\\_CFG\\_getIdentifySourceOnPortEnabled](#) (STAR\_DEVICE\_ID deviceId, U8 portNum, [\\_Out\\_int](#) \*pEnabled)

*Gets whether identify source is enabled for a specific port.*

  - int [STAR\\_API\\_CC\\_CFG\\_disableInterfaceMode](#) (STAR\_DEVICE\_ID deviceId)

*Disables interface mode on a device.*

  - int [STAR\\_API\\_CC\\_CFG\\_enableInterfaceMode](#) (STAR\_DEVICE\_ID deviceId)

*In interface mode a packet which is received on an external port, from a SpaceWire link or from the configuration port will be routed to the port specified by the port routing register of the port (set by [CFG\\_setPortRoutingAddress\(\)](#)).*

  - int [STAR\\_API\\_CC\\_CFG\\_getInterfaceModeEnabled](#) (STAR\_DEVICE\_ID deviceId, [\\_Out\\_int](#) \*pEnabled)

*Gets whether interface mode has been enabled on a device.*

  - int [STAR\\_API\\_CC\\_CFG\\_disableInterfaceModeOnPort](#) (STAR\_DEVICE\_ID deviceId, U8 portNum)

*Selectively disables interface mode on a given port.*

  - int [STAR\\_API\\_CC\\_CFG\\_enableInterfaceModeOnPort](#) (STAR\_DEVICE\_ID deviceId, U8 portNum)

*Selectively enables interface mode on a given port.*

  - int [STAR\\_API\\_CC\\_CFG\\_getInterfaceModeOnPortEnabled](#) (STAR\_DEVICE\_ID deviceId, U8 portNum, [\\_Out\\_int](#) \*pEnabled)

*Gets whether interface mode is enabled on a specific port.*

  - int [STAR\\_API\\_CC\\_CFG\\_disableTimeCodeMaster](#) (STAR\_DEVICE\_ID deviceId)

*Disables the device as a time-code master.*

  - int [STAR\\_API\\_CC\\_CFG\\_enableTimeCodeMaster](#) (STAR\_DEVICE\_ID deviceId)

*Enables the device as a time-code master.*

  - int [STAR\\_API\\_CC\\_CFG\\_getTimeCodeMasterEnabled](#) (STAR\_DEVICE\_ID deviceId, [\\_Out\\_int](#) \*pEnabled)

*Gets whether the device has been enabled as a time-code master.*

  - int [STAR\\_API\\_CC\\_CFG\\_disableExternalTimeCodeSelection](#) (STAR\_DEVICE\_ID deviceId)

*Disables external time-code selection for the device.*

  - int [STAR\\_API\\_CC\\_CFG\\_enableExternalTimeCodeSelection](#) (STAR\_DEVICE\_ID deviceId)

*Enables external time-code selection for the device.*

  - int [STAR\\_API\\_CC\\_CFG\\_getExternalTimeCodeSelectionEnabled](#) (STAR\_DEVICE\_ID deviceId, [\\_Out\\_int](#) \*pEnabled)

*Gets whether external time-code selection has been enabled for the device.*

  - int [STAR\\_API\\_CC\\_CFG\\_setTimeCodePeriod](#) (STAR\_DEVICE\_ID deviceId, U32 period)

*Sets the period between time-code master ticks.*

  - int [STAR\\_API\\_CC\\_CFG\\_getTimeCodePeriod](#) (STAR\_DEVICE\_ID deviceId, [\\_Out\\_U32](#) \*pPeriod)

*Gets the period between time-code master ticks.*

  - int [STAR\\_API\\_CC\\_CFG\\_disableTimeCodeCounterBypassMode](#) (STAR\_DEVICE\_ID deviceId)

*Disables time-code counter bypass mode for the device.*

  - int [STAR\\_API\\_CC\\_CFG\\_enableTimeCodeCounterBypassMode](#) (STAR\_DEVICE\_ID deviceId)

*Enables time-code counter bypass mode for the device.*

  - int [STAR\\_API\\_CC\\_CFG\\_getTimeCodeCounterBypassModeEnabled](#) (STAR\_DEVICE\_ID deviceId, [\\_Out\\_int](#) \*pEnabled)

*Gets whether time-code counter bypass mode has been enabled for the device.*

  - int [STAR\\_API\\_CC\\_CFG\\_disableSpeedChangeEventsOnPort](#) (STAR\_DEVICE\_ID deviceId, U8 portNum)

*Selectively disables speed change events on a given port.*

  - int [STAR\\_API\\_CC\\_CFG\\_enableSpeedChangeEventsOnPort](#) (STAR\_DEVICE\_ID deviceId, U8 portNum)

*Selectively enables speed change events on a given port.*

  - int [STAR\\_API\\_CC\\_CFG\\_getSpeedChangeEventsOnPortEnabled](#) (STAR\_DEVICE\_ID deviceId, U8 portNum, [\\_Out\\_int](#) \*pEnabled)

*Gets whether speed change events are enabled on a specific port.*

  - int [STAR\\_API\\_CC\\_CFG\\_disableStateChangeEventsOnPort](#) (STAR\_DEVICE\_ID deviceId, U8 portNum)

- Selectively disables state change events on a given port.*

  - int `STAR_API_CC_CFG_enableStateChangeEventsOnPort` (STAR\_DEVICE\_ID deviceId, U8 portNum)

*Selectively enables state change events on a given port.*

  - int `STAR_API_CC_CFG_getStateChangeEventsOnPortEnabled` (STAR\_DEVICE\_ID deviceId, U8 portNum, \_Out\_ int \*pEnabled)

*Gets whether state change events are enabled on a specific port.*

  - int `STAR_API_CC_CFG_disableTimeCodeEventsOnPort` (STAR\_DEVICE\_ID deviceId, U8 portNum)

*Selectively disables time-code notifications on a the channel attached to a given port.*

  - int `STAR_API_CC_CFG_enableTimeCodeEventsOnPort` (STAR\_DEVICE\_ID deviceId, U8 portNum)

*Selectively enables time-code notifications on a the channel attached to a given port.*

  - int `STAR_API_CC_CFG_getTimeCodeEventsOnPortEnabled` (STAR\_DEVICE\_ID deviceId, U8 portNum, \_↔\_Out\_ int \*pEnabled)

*Gets whether time-code notifications are enabled on a specific port.*

  - int `STAR_API_CC_CFG_getMeasuredLinkSpeed` (STAR\_DEVICE\_ID deviceId, U8 portNum, \_Out\_ U16 \*pLinkSpeed)

*Gets the current link speed measured on a specific port.*

  - int `STAR_API_CC_CFG_getTransmitSignallingRate` (STAR\_DEVICE\_ID deviceId, U8 linkNum, \_Out\_ U32 \*pSignallingRate)

*Gets the transmit bit rate for a given link.*

  - int `STAR_API_CC_CFG_setTransmitSignallingRate` (STAR\_DEVICE\_ID deviceId, U8 linkNum, U32 signallingRate)

*Sets the transmit bit rate on a given link.*

  - int `STAR_API_CC_CFG_disablePrecisionTransmitRate` (STAR\_DEVICE\_ID deviceId)

*Disables precision transmit rate.*

  - int `STAR_API_CC_CFG_enablePrecisionTransmitRate` (STAR\_DEVICE\_ID deviceId)

*Enables precision transmit rate.*

  - int `STAR_API_CC_CFG_getPrecisionTransmitRateEnabled` (STAR\_DEVICE\_ID deviceId, \_Out\_ int \*p↔Enabled)

*Determine if precision transmit rate is enabled.*

  - int `STAR_API_CC_CFG_getPrecisionTransmitRateInUse` (STAR\_DEVICE\_ID deviceId, \_Out\_ int \*pInUse)

*Determine whether precision transmit rate is in use.*

  - int `STAR_API_CC_CFG_getPrecisionTransmitRate` (STAR\_DEVICE\_ID deviceId, \_Out\_ double \*p↔TransmitRate)

*Get the current precision transmit rate on a device in Mbit/s.*

  - int `STAR_API_CC_CFG_setPrecisionTransmitRate` (STAR\_DEVICE\_ID deviceId, double transmitRate)

*Set the current precision transmit rate on a device in Mbit/s.*

  - int `STAR_API_CC_CFG_injectError` (STAR\_DEVICE\_ID deviceId, U8 portNum, SPW\_ERROR error)

*Immediately injects the specified error on a given device's port.*

  - int `STAR_API_CC_CFG_injectErrors` (STAR\_DEVICE\_ID deviceId, U8 portNum, \_In\_ STAR\_CFG\_BRIC↔K\_MK2\_ERRORS \*pErrors)

*Inject the specified errors on a given port.*

  - int `STAR_API_CC_CFG_disableRxTimestampEventsOnPort` (STAR\_DEVICE\_ID deviceId, U8 portNum)

*Selectively disables receive timestamp events on a given port.*

  - int `STAR_API_CC_CFG_enableRxTimestampEventsOnPort` (STAR\_DEVICE\_ID deviceId, U8 portNum)

*Selectively enables receive timestamp events on a given port.*

  - int `STAR_API_CC_CFG_getRxTimestampEventsOnPortEnabled` (STAR\_DEVICE\_ID deviceId, U8 portNum, \_Out\_ int \*pEnabled)

*Gets whether receive timestamp events are enabled on a specific port.*

  - int `STAR_API_CC_CFG_getPulseGeneratorFrequency` (STAR\_DEVICE\_ID deviceId, \_Out\_ STAR\_CFG\_↔BRICK\_MK3\_PULSE\_FREQ \*pFrequency)

*Gets the frequency of each generated pulse.*

- `int STAR_API_CC_CFG_setPulseGeneratorFrequency (STAR_DEVICE_ID deviceId, STAR_CFG_BRICK↔_MK3_PULSE_FREQ frequency)`  
*Sets the frequency of each generated pulse.*
- `int STAR_API_CC_CFG_getTimestampMethod (STAR_DEVICE_ID deviceId, _Out_ STAR_CFG_BRICK↔_MK3_TIMESTAMP_METHOD *pMethod)`  
*Gets the timestamping method that is currently in use for the given device.*
- `int STAR_API_CC_CFG_setTimestampMethod (STAR_DEVICE_ID deviceId, STAR_CFG_BRICK_MK3_↔TIMESTAMP_METHOD method)`  
*Sets the timestamping method to use for the given device.*
- `int STAR_API_CC_CFG_getTimestampValue (STAR_DEVICE_ID deviceId, _Out_ U32 *pValue)`  
*Gets the current timestamp value.*
- `int STAR_API_CC_CFG_setTimestampValue (STAR_DEVICE_ID deviceId, U32 value)`  
*Sets the current timestamp value for the given device which will be incremented on the next synchronisation pulse.*
- `int STAR_API_CC_CFG_getDeviceClockRate (STAR_DEVICE_ID deviceId)`  
*Gets a device's internal clock rate.*
- `int STAR_API_CC_CFG_startPeriodicAction (STAR_DEVICE_ID deviceId, unsigned char port, U32 count, SPW_ACTION action)`  
*Starts a periodic action on a given device's port.*
- `int STAR_API_CC_CFG_stopPeriodicAction (STAR_DEVICE_ID deviceId, unsigned char port)`  
*Stops a periodic action on a given device's port.*

### 8.6.1 Detailed Description

Types used with the STAR-Dundee Generic Configuration API.

#### Author

STAR-Dundee Ltd.  
STAR House  
166 Nethergate  
Dundee, DD1 4EE  
Scotland, UK  
e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2012 STAR-Dundee Ltd.

### 8.6.2 Function Documentation

#### 8.6.2.1 `int STAR_API_CC_CFG_getRouterGlobalSettings ( STAR_DEVICE_ID deviceId, _Out_ STAR_CFG_ROUTER_GLOBAL_STATE * pState )`

Gets the router's global settings.

#### Parameters

|            |                 |                                     |
|------------|-----------------|-------------------------------------|
|            | <i>deviceId</i> | Device to get global settings from. |
| <i>out</i> | <i>pState</i>   | Global settings for the router.     |

#### Returns

0 for success and a negative value on error.

Added in version:

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

### 8.6.2.2 int STAR\_API\_CC\_CFG\_getTimeCodeFlagMode ( STAR\_DEVICE\_ID *deviceId*, \_Out\_ U8 \* *pMode* )

Obtains the time-code flag interpretation mode:

0 - Time-code control bit flags are distributed with valid time-code values regardless of the value of the time-code control flags

1 - When the time-code control bit flags are "00" then valid time-codes are distributed. When the time-code control flags are not "00" then the time-code is discarded and the internal time-code register is not updated.

**Parameters**

|     |                 |                                             |
|-----|-----------------|---------------------------------------------|
|     | <i>deviceId</i> | Device to get the time-code flag mode from. |
| out | <i>pMode</i>    | Time code flag mode,                        |

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

### 8.6.2.3 int STAR\_API\_CC\_CFG\_getTimeCodeValue ( STAR\_DEVICE\_ID *deviceId*, \_Out\_ U8 \* *pValue* )

Obtains the current value of the router's internal time-code counter.

**Parameters**

|     |                 |                                             |
|-----|-----------------|---------------------------------------------|
| •   | <i>deviceId</i> | Device to get the time-code flag mode from. |
| out | <i>pValue</i>   | Time-code value,                            |

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

### 8.6.2.4 int STAR\_API\_CC\_CFG\_setRouterGlobalSettings ( STAR\_DEVICE\_ID *deviceId*, \_In\_ STAR\_CFG\_ROUTER\_GLOBAL\_STATE \* *pState* )

Sets the router's global settings.

**Parameters**

|           |                 |                                    |
|-----------|-----------------|------------------------------------|
|           | <i>deviceID</i> | Device to set global settings for. |
| <i>in</i> | <i>pState</i>   | Global settings for the router.    |

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

**8.6.2.5 `int STAR_API_CC_CFG_setTimeCodeFlagMode ( STAR_DEVICE_ID deviceID, U8 mode )`**

Sets the time-code flag interpretation mode:

0 - Time-code control bit flags are distributed with valid time-code values regardless of the value of the time-code control flags

1 - When the time-code control bit flags are "00" then valid time-codes are distributed. When the time-code control flags are not "00" then the time-code is discarded and the internal time-code register is not updated.

**Parameters**

|  |                 |                                             |
|--|-----------------|---------------------------------------------|
|  | <i>deviceID</i> | Device to set the time-code flag modes for. |
|  | <i>mode</i>     | Mode to set the time code flag mode to,     |

**Returns**

0 for success and a negative value on error.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, STAR Fire Mk3

**8.7 `cfg_api_generic_common.h` File Reference**

Additional types used with the STAR-Dundee Generic Configuration API.

```
#include "star-api.h"
#include "cfg_api_remote.h"
#include "cfg_api_mk2_types.h"
#include "cfg_api_pci_mk2_types.h"
#include "cfg_api_brick_mk2_types.h"
#include "cfg_api_brick_mk3_types.h"
#include "cfg_api_router_types.h"
```

## Data Structures

- struct `STAR_CFG_FPGA_INFO`

*Defines a structure used to store a device's FPGA version info. [More...](#)*

### 8.7.1 Detailed Description

Additional types used with the STAR-Dundee Generic Configuration API.

#### Author

STAR-Dundee Ltd.  
 STAR House  
 166 Nethergate  
 Dundee, DD1 4EE  
 Scotland, UK  
 e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2012 STAR-Dundee Ltd.

## 8.8 `cfg_api_mk2.h` File Reference

Functions used to configure STAR-Dundee Mk2 compatible devices.

```
#include "star-api.h"
#include "cfg_api_mk2_types.h"
#include "cfg_api_remote.h"
```

#### Macros

- `#define STAR_CFG_MK2_BUILD_DATE_STR_MAX_LEN 256`  
*Maximum length for the string representation of a hardware build date.*
- `#define STAR_CFG_MK2_VERSION_STR_MAX_LEN 256`  
*Maximum length for the string representation of a hardware build date.*
- `#define CFG_API_MK2_MODULE_NAME "CFG-API_MK2"`  
*The name of the module, used for logging purposes.*

#### Functions

- `int STAR_API_CC CFG_MK2_getHardwareInfo (STAR_DEVICE_ID deviceId, _Out_ STAR_CFG_MK2_↔  
HARDWARE_INFO *pInfo)`  
*Reads information about the hardware version of a device and updates the `STAR_CFG_MK2_HARDWARE_INFO` structure pointed to by the passed in pointer to contain the received version information.*
- `void STAR_API_CC CFG_MK2_hardwareInfoToString (STAR_CFG_MK2_HARDWARE_INFO info, _Out_ ↔  
z_cap_c_(STAR_CFG_MK2_VERSION_STR_MAX_LEN) char *pVersion, _Out_z_cap_c_(STAR_CFG_↔  
MK2_BUILD_DATE_STR_MAX_LEN) char *pBuildDate)`  
*Creates a string representation of a `STAR_CFG_MK2_HARDWARE_INFO` structure.*
- `int STAR_API_CC CFG_MK2_enableTimeCodeMaster (STAR_DEVICE_ID deviceId)`  
*Enables the device as a time-code master.*
- `int STAR_API_CC CFG_MK2_getTimeCodeMasterEnabled (STAR_DEVICE_ID deviceId, int *pEnabled)`  
*Gets whether the device has been enabled as a time-code master.*
- `int STAR_API_CC CFG_MK2_disableTimeCodeMaster (STAR_DEVICE_ID deviceId)`

- Disables the device as a time-code master.*

  - int [STAR\\_API\\_CC\\_CFG\\_MK2\\_enableExternalTimeCodeSelection](#) (STAR\_DEVICE\_ID deviceId)

*Enables external time-code selection for the device.*

  - int [STAR\\_API\\_CC\\_CFG\\_MK2\\_getExternalTimeCodeSelectionEnabled](#) (STAR\_DEVICE\_ID deviceId, int \*pEnabled)

*Gets whether external time-code selection has been enabled for the device.*

  - int [STAR\\_API\\_CC\\_CFG\\_MK2\\_disableExternalTimeCodeSelection](#) (STAR\_DEVICE\_ID deviceId)

*Disables external time-code selection for the device.*

  - int [STAR\\_API\\_CC\\_CFG\\_MK2\\_setTimeCodePeriod](#) (STAR\_DEVICE\_ID deviceId, U32 period)

*Sets the period between time-code master ticks.*

  - int [STAR\\_API\\_CC\\_CFG\\_MK2\\_getTimeCodePeriod](#) (STAR\_DEVICE\_ID deviceId, \_Out\_ U32 \*pPeriod)

*Gets the period between time-code master ticks.*

  - int [STAR\\_API\\_CC\\_CFG\\_MK2\\_enableTimeCodeCounterBypassMode](#) (STAR\_DEVICE\_ID deviceId)

*Enables time-code counter bypass mode for the device.*

  - int [STAR\\_API\\_CC\\_CFG\\_MK2\\_getTimeCodeCounterBypassModeEnabled](#) (STAR\_DEVICE\_ID deviceId, int \*pEnabled)

*Gets whether time-code counter bypass mode has been enabled for the device.*

  - int [STAR\\_API\\_CC\\_CFG\\_MK2\\_disableTimeCodeCounterBypassMode](#) (STAR\_DEVICE\_ID deviceId)

*Disables time-code counter bypass mode for the device.*

  - int [STAR\\_API\\_CC\\_CFG\\_MK2\\_identify](#) (STAR\_DEVICE\_ID deviceId)

*Flashes the front panel LEDs of a device.*

  - int [STAR\\_API\\_CC\\_CFG\\_MK2\\_enableInterfaceMode](#) (STAR\_DEVICE\_ID deviceId)

*In interface mode a packet which is received on an external port, from a SpaceWire link or from the configuration port will be routed to the port specified by the port routing register of the port (set by [CFG\\_MK2\\_setPortRoutingAddress\(\)](#)).*

  - int [STAR\\_API\\_CC\\_CFG\\_MK2\\_getInterfaceModeEnabled](#) (STAR\_DEVICE\_ID deviceId, int \*pEnabled)

*Gets whether interface mode has been enabled on a device.*

  - int [STAR\\_API\\_CC\\_CFG\\_MK2\\_enableInterfaceModeOnPort](#) (STAR\_DEVICE\_ID deviceId, U8 portNum)

*Selectively enables interface mode on a given port.*

  - int [STAR\\_API\\_CC\\_CFG\\_MK2\\_getInterfaceModeOnPortEnabled](#) (STAR\_DEVICE\_ID deviceId, U8 portNum, int \*pEnabled)

*Gets whether interface mode is enabled on a specific port.*

  - int [STAR\\_API\\_CC\\_CFG\\_MK2\\_disableInterfaceMode](#) (STAR\_DEVICE\_ID deviceId)

*Disables interface mode on a device.*

  - int [STAR\\_API\\_CC\\_CFG\\_MK2\\_disableInterfaceModeOnPort](#) (STAR\_DEVICE\_ID deviceId, U8 portNum)

*Selectively disables interface mode on a given port.*

  - int [STAR\\_API\\_CC\\_CFG\\_MK2\\_enableIdentifySource](#) (STAR\_DEVICE\_ID deviceId)

*When interface mode is enabled on a port, this function globally enables the addition of a leading byte to each received packet indicating which port the packet was received on.*

  - int [STAR\\_API\\_CC\\_CFG\\_MK2\\_getIdentifySourceEnabled](#) (STAR\_DEVICE\_ID deviceId, int \*pEnabled)

*Gets whether identify source is enabled.*

  - int [STAR\\_API\\_CC\\_CFG\\_MK2\\_disableIdentifySource](#) (STAR\_DEVICE\_ID deviceId)

*Disables identification of source ports for interface mode globally.*

  - int [STAR\\_API\\_CC\\_CFG\\_MK2\\_enableIdentifySourceOnPort](#) (STAR\_DEVICE\_ID deviceId, U8 portNum)

*Enables identification of source port during interface mode for a given port.*

  - int [STAR\\_API\\_CC\\_CFG\\_MK2\\_getIdentifySourceOnPortEnabled](#) (STAR\_DEVICE\_ID deviceId, U8 portNum, int \*pEnabled)

*Gets whether identify source is enabled for a specific port.*

  - int [STAR\\_API\\_CC\\_CFG\\_MK2\\_disableIdentifySourceOnPort](#) (STAR\_DEVICE\_ID deviceId, U8 portNum)

*Disables source port identification for a given port.*

  - int [STAR\\_API\\_CC\\_CFG\\_MK2\\_setPortRoutingAddress](#) (STAR\_DEVICE\_ID deviceId, U8 portNum, U8 address)

*Sets the address a packet received on a given port should be routed to when interface mode is enabled.*



- `int STAR_API_CC_CFG_MK2_getPortRoutingAddress (STAR_DEVICE_ID deviceID, U8 portNum, _Out_ U8 *pAddress)`  
*Gets the address a packet received on a given port should be routed to when interface mode is enabled.*
- `int STAR_API_CC_CFG_MK2_setLinkRateDivider (STAR_DEVICE_ID deviceID, U8 linkNum, U8 divider)`  
*Sets the Link rate divider for a given link.*
- `int STAR_API_CC_CFG_MK2_getLinkRateDivider (STAR_DEVICE_ID deviceID, U8 linkNum, _Out_ U8 *pDivider)`  
*Gets the Link rate divider for a given link.*
- `int STAR_API_CC_CFG_MK2_setBaseTransmitClock (STAR_DEVICE_ID deviceID, U8 linkNum, STAR_CFG_MK2_BASE_TRANSMIT_CLOCK clockRateParams)`  
*Sets the base transmit clock rate on a given link:*
- `int STAR_API_CC_CFG_MK2_getBaseTransmitClock (STAR_DEVICE_ID deviceID, U8 linkNum, _Out_ STAR_CFG_MK2_BASE_TRANSMIT_CLOCK *pClockRateParams)`  
*Gets the base transmit clock rate parameters for a given link.*
- `int STAR_API_CC_CFG_MK2_setTransmitClock (STAR_DEVICE_ID deviceID, U8 linkNum, STAR_CFG_MK2_BASE_TRANSMIT_CLOCK clockRateParams)`  
*Sets the transmit clock rate on a given link.*
- `int STAR_API_CC_CFG_MK2_getTransmitClock (STAR_DEVICE_ID deviceID, U8 linkNum, _Out_ STAR_CFG_MK2_BASE_TRANSMIT_CLOCK *pClockRateParams)`  
*Gets the transmit clock rate parameters for a given link.*
- `int STAR_API_CC_CFG_MK2_injectError (STAR_DEVICE_ID deviceID, unsigned char port, SPW_ERROR error)`  
*Immediately injects the specified error on a given device's port.*
- `int STAR_API_CC_CFG_MK2_startPeriodicAction (STAR_DEVICE_ID deviceID, unsigned char port, U32 count, SPW_ACTION action)`  
*Starts a periodic action on a given device's port.*
- `int STAR_API_CC_CFG_MK2_stopPeriodicAction (STAR_DEVICE_ID deviceID, unsigned char port)`  
*Stops a periodic action on a given device's port.*
- `U32 STAR_API_CC_CFG_MK2_getDeviceClockRate (STAR_DEVICE_ID deviceID)`  
*Gets a device's internal clock rate.*
- `int STAR_API_CC_CFG_MK2_enableStateChangeEventsOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`  
*Selectively enables state change events on a given port.*
- `int STAR_API_CC_CFG_MK2_getStateChangeEventsOnPortEnabled (STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int *pEnabled)`  
*Gets whether state change events are enabled on a specific port.*
- `int STAR_API_CC_CFG_MK2_disableStateChangeEventsOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`  
*Selectively disables state change events on a given port.*
- `int STAR_API_CC_CFG_MK2_enableSpeedChangeEventsOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`  
*Selectively enables speed change events on a given port.*
- `int STAR_API_CC_CFG_MK2_getSpeedChangeEventsOnPortEnabled (STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int *pEnabled)`  
*Gets whether speed change events are enabled on a specific port.*
- `int STAR_API_CC_CFG_MK2_disableSpeedChangeEventsOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`  
*Selectively disables speed change events on a given port.*
- `int STAR_API_CC_CFG_MK2_enableTimeCodeEventsOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`  
*Selectively enables time-code notifications on a the channel attached to a given port.*
- `int STAR_API_CC_CFG_MK2_getTimeCodeEventsOnPortEnabled (STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int *pEnabled)`  
*Gets whether time-code notifications are enabled on a specific port.*



- int [STAR\\_API\\_CC\\_CFG\\_MK2\\_disableTimeCodeEventsOnPort](#) (STAR\_DEVICE\_ID deviceID, U8 portNum)  
*Selectively disables time-code notifications on a the channel attached to a given port.*
- int [STAR\\_API\\_CC\\_CFG\\_MK2\\_getMeasuredLinkSpeed](#) (STAR\_DEVICE\_ID deviceID, U8 portNum, \_Out\_ U16 \*pLinkSpeed)  
*Gets the current link speed measured on a specific port.*
- int [STAR\\_API\\_CC\\_CFG\\_MK2\\_setTimeCodeDistributionPorts](#) (STAR\_DEVICE\_ID deviceID, U32 portMask)  
*This function is used to specify which output ports time-codes are forwarded on.*
- int [STAR\\_API\\_CC\\_CFG\\_MK2\\_getTimeCodeDistributionPorts](#) (STAR\_DEVICE\_ID deviceID, \_Out\_ U32 \*pPortMask)  
*This function is used to obtain which output ports time-codes are forwarded on.*
- int [STAR\\_API\\_CC\\_CFG\\_MK2\\_setTimeCodeFlagMode](#) (STAR\_DEVICE\_ID deviceID, U8 mode)  
*Sets the time-code flag interpretation mode:*
- int [STAR\\_API\\_CC\\_CFG\\_MK2\\_getTimeCodeFlagMode](#) (STAR\_DEVICE\_ID deviceID, \_Out\_ U8 \*pMode)  
*Obtains the time-code flag interpretation mode:*

### 8.8.1 Detailed Description

Functions used to configure STAR-Dundee Mk2 compatible devices.

#### Author

STAR-Dundee Ltd.  
STAR House  
166 Nethergate  
Dundee, DD1 4EE  
Scotland, UK  
e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2012 STAR-Dundee Ltd.

### 8.8.2 Macro Definition Documentation

#### 8.8.2.1 #define CFG\_API\_MK2\_MODULE\_NAME "CFG-API\_MK2"

The name of the module, used for logging purposes.

## 8.9 cfg\_api\_mk2\_types.h File Reference

Types used with the STAR-Dundee Mk2 compatible device configuration API.

### Data Structures

- struct [STAR\\_CFG\\_MK2\\_HARDWARE\\_INFO](#)  
*Hardware version, and synthesis date of the FPGA code. [More...](#)*
- struct [STAR\\_CFG\\_MK2\\_BASE\\_TRANSMIT\\_CLOCK](#)  
*Base transmit clock rate control properties. [More...](#)*

### Macros

- #define [STAR\\_CFG\\_MK2\\_LINK\\_SPEED\\_UNITS\\_KBPS](#) 100  
*The units in Kbit/s used to represent the link speed returned when calling [CFG\\_MK2\\_getMeasuredLinkSpeed\(\)](#).*

## Enumerations

- enum `SPW_ERROR` {  
`SPW_ERROR_DISCONNECT` = 1, `SPW_ERROR_PARITY` = 2, `SPW_ERROR_ESCAPE` = 3, `SPW_ERROR_INSERT_FCT` = 4,  
`SPW_ERROR_SUPPRESS_FCT` = 5, `SPW_ERROR_INCREMENT_CREDIT` = 6, `SPW_ERROR_DECREMENT_CREDIT` = 7 }

*Errors that can be injected onto the SpaceWire link.*

- enum `SPW_ACTION` { `SPW_TRANSMIT_PACKET` = 0 }

*Actions which can be preformed at specified periodic intervals.*

### 8.9.1 Detailed Description

Types used with the STAR-Dundee Mk2 compatible device configuration API.

#### Author

STAR-Dundee Ltd.  
 STAR House  
 166 Nethergate  
 Dundee, DD1 4EE  
 Scotland, UK  
 e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2012 STAR-Dundee Ltd.

### 8.9.2 Enumeration Type Documentation

#### 8.9.2.1 enum `SPW_ACTION`

Actions which can be preformed at specified periodic intervals.

#### Added in version:

STAR-System v3.0 Beta 3 - 4th September 2015

#### Enumerator

**`SPW_TRANSMIT_PACKET`** Transmits a SpaceWire packet.

## 8.10 `cfg_api_pci_mk2.h` File Reference

Functions used to configure STAR-Dundee PCI Mk2 devices.

```
#include "star-api.h"
#include "cfg_api_pci_mk2_types.h"
#include "cfg_api_remote.h"
```

## Macros

- #define `CFG_API_PCI_MK2_MODULE_NAME` "CFG-API-PCI-MK2"

*The name of the module, used for logging purposes.*

## Functions

- int STAR\_API\_CC\_CFG\_PCIMK2\_setLinkClockFrequency (STAR\_DEVICE\_ID deviceID, U8 linkNum, STAR\_CFG\_PCIMK2\_LINK\_FREQ linkFreq)  
*Sets the Link Clock Frequency on a given link.*
- int STAR\_API\_CC\_CFG\_PCIMK2\_getLinkClockFrequency (STAR\_DEVICE\_ID deviceID, U8 linkNum, STAR\_CFG\_PCIMK2\_LINK\_FREQ \*pLinkFreq)  
*Gets the Link Clock Frequency for a given link.*

### 8.10.1 Detailed Description

Functions used to configure STAR-Dundee PCI Mk2 devices.

#### Author

STAR-Dundee Ltd.  
STAR House  
166 Nethergate  
Dundee, DD1 4EE  
Scotland, UK  
e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2012 STAR-Dundee Ltd.

### 8.10.2 Macro Definition Documentation

#### 8.10.2.1 #define CFG\_API\_PCI\_MK2\_MODULE\_NAME "CFG-API-PCI-MK2"

The name of the module, used for logging purposes.

## 8.11 cfg\_api\_pci\_mk2\_types.h File Reference

Types used with the STAR-Dundee PCI Mk2 Configuration API.

## Enumerations

- enum STAR\_CFG\_PCIMK2\_LINK\_FREQ {  
STAR\_CFG\_PCIMK2\_LINK\_FREQ\_120 = 0, STAR\_CFG\_PCIMK2\_LINK\_FREQ\_128 = 5, STAR\_CFG\_PCIMK2\_LINK\_FREQ\_140 = 1, STAR\_CFG\_PCIMK2\_LINK\_FREQ\_150 = 6,  
STAR\_CFG\_PCIMK2\_LINK\_FREQ\_160 = 2, STAR\_CFG\_PCIMK2\_LINK\_FREQ\_180 = 3, STAR\_CFG\_PCIMK2\_LINK\_FREQ\_200 = 4 }  
*Frequencies a link may run at.*

### 8.11.1 Detailed Description

Types used with the STAR-Dundee PCI Mk2 Configuration API.

**Author**

STAR-Dundee Ltd.  
 STAR House  
 166 Nethergate  
 Dundee, DD1 4EE  
 Scotland, UK  
 e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2012 STAR-Dundee Ltd.

**8.12 cfg\_api\_pxi.h File Reference**

Functions used to configure STAR-Dundee PXI devices.

```
#include "star-api.h"
#include "cfg_api_mk2_types.h"
```

**Functions**

- int [STAR\\_API\\_CC\\_CFG\\_PXI\\_setLinkRateDivider](#) (STAR\_DEVICE\_ID deviceId, U8 divider)  
*Sets the Link rate divider for a PXI device.*
- int [STAR\\_API\\_CC\\_CFG\\_PXI\\_getLinkRateDivider](#) (STAR\_DEVICE\_ID deviceId, \_Out\_ U8 \*pDivider)  
*Gets the Link rate divider for a PXI device.*
- int [STAR\\_API\\_CC\\_CFG\\_PXI\\_setBaseTransmitClock](#) (STAR\_DEVICE\_ID deviceId, U8 linkNum, STAR\_CFG\_MK2\_BASE\_TRANSMIT\_CLOCK clockRateParams)  
*Sets the base transmit clock rate on a given link:*
- int [STAR\\_API\\_CC\\_CFG\\_PXI\\_getBaseTransmitClock](#) (STAR\_DEVICE\_ID deviceId, U8 linkNum, \_Out\_ STAR\_CFG\_MK2\_BASE\_TRANSMIT\_CLOCK \*clockRateParams)  
*Gets the base transmit clock rate parameters for a given link.*
- int [STAR\\_API\\_CC\\_CFG\\_PXI\\_setTransmitClock](#) (STAR\_DEVICE\_ID deviceId, U8 linkNum, STAR\_CFG\_MK2\_BASE\_TRANSMIT\_CLOCK clockRateParams)  
*Sets the transmit clock rate on a given link.*
- int [STAR\\_API\\_CC\\_CFG\\_PXI\\_getTransmitClock](#) (STAR\_DEVICE\_ID deviceId, U8 linkNum, \_Out\_ STAR\_CFG\_MK2\_BASE\_TRANSMIT\_CLOCK \*clockRateParams)  
*Gets the transmit clock rate parameters for a given link.*
- int [STAR\\_API\\_CC\\_CFG\\_PXI\\_injectError](#) (STAR\_DEVICE\_ID deviceId, unsigned char port, SPW\_ERROR error)  
*Immediately injects the specified error on a given device's port.*
- int [STAR\\_API\\_CC\\_CFG\\_PXI\\_enableInterfaceModeOnPort](#) (STAR\_DEVICE\_ID deviceId, U8 portNum)  
*Selectively enables interface mode on a given port of a supported PXI device.*
- int [STAR\\_API\\_CC\\_CFG\\_PXI\\_getInterfaceModeOnPortEnabled](#) (STAR\_DEVICE\_ID deviceId, U8 portNum, \_Out\_ int \*pEnabled)  
*Gets whether interface mode is enabled on a specific port of a supported PXI device.*
- int [STAR\\_API\\_CC\\_CFG\\_PXI\\_disableInterfaceModeOnPort](#) (STAR\_DEVICE\_ID deviceId, U8 portNum)  
*Selectively disables interface mode on a given port of a supported PXI device.*
- int [STAR\\_API\\_CC\\_CFG\\_PXI\\_enableIdentifySourceOnPort](#) (STAR\_DEVICE\_ID deviceId, U8 portNum)  
*Enables identification of the source port of a received packet on a given port, when in interface mode.*
- int [STAR\\_API\\_CC\\_CFG\\_PXI\\_getIdentifySourceOnPortEnabled](#) (STAR\_DEVICE\_ID deviceId, U8 portNum, \_Out\_ int \*pEnabled)  
*Gets whether identify source is enabled for a specific port.*
- int [STAR\\_API\\_CC\\_CFG\\_PXI\\_disableIdentifySourceOnPort](#) (STAR\_DEVICE\_ID deviceId, U8 portNum)  
*Disables source port identification for a given port.*

- int STAR\_API\_CC CFG\_PXI\_setPortRoutingAddress (STAR\_DEVICE\_ID deviceID, U8 portNum, U8 address)  
*Sets the address a packet received on a given port should be routed to when interface mode is enabled.*
- int STAR\_API\_CC CFG\_PXI\_getPortRoutingAddress (STAR\_DEVICE\_ID deviceID, U8 portNum, \_Out\_ U8 \*pAddress)  
*Gets the address a packet received on a given port should be routed to when interface mode is enabled.*
- int STAR\_API\_CC CFG\_PXI\_enableStateChangeEventsOnPort (STAR\_DEVICE\_ID deviceID, U8 portNum)  
*Selectively enables state change events on a given port.*
- int STAR\_API\_CC CFG\_PXI\_getStateChangeEventsOnPortEnabled (STAR\_DEVICE\_ID deviceID, U8 portNum, \_Out\_ int \*pEnabled)  
*Gets whether state change events are enabled on a specific port.*
- int STAR\_API\_CC CFG\_PXI\_disableStateChangeEventsOnPort (STAR\_DEVICE\_ID deviceID, U8 portNum)  
*Selectively disables state change events on a given port.*
- int STAR\_API\_CC CFG\_PXI\_enableSpeedChangeEventsOnPort (STAR\_DEVICE\_ID deviceID, U8 portNum)  
*Selectively enables speed change events on a given port.*
- int STAR\_API\_CC CFG\_PXI\_getSpeedChangeEventsOnPortEnabled (STAR\_DEVICE\_ID deviceID, U8 portNum, \_Out\_ int \*pEnabled)  
*Gets whether speed change events are enabled on a specific port.*
- int STAR\_API\_CC CFG\_PXI\_disableSpeedChangeEventsOnPort (STAR\_DEVICE\_ID deviceID, U8 portNum)  
*Selectively disables speed change events on a given port.*
- int STAR\_API\_CC CFG\_PXI\_enableTimeCodeEventsOnPort (STAR\_DEVICE\_ID deviceID, U8 portNum)  
*Selectively enables time-code notifications on a channel attached to a given port.*
- int STAR\_API\_CC CFG\_PXI\_getTimeCodeEventsOnPortEnabled (STAR\_DEVICE\_ID deviceID, U8 portNum, \_Out\_ int \*pEnabled)  
*Gets whether time-code notifications are enabled on a specific port.*
- int STAR\_API\_CC CFG\_PXI\_disableTimeCodeEventsOnPort (STAR\_DEVICE\_ID deviceID, U8 portNum)  
*Selectively disables time-code notifications on a channel attached to a given port.*

### 8.12.1 Detailed Description

Functions used to configure STAR-Dundee PXI devices.

#### Author

STAR-Dundee Ltd.  
 STAR House  
 166 Nethergate  
 Dundee, DD1 4EE  
 Scotland, UK  
 e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2015 STAR-Dundee Ltd.

## 8.13 cfg\_api\_remote.h File Reference

Functions used to create, configure and destroy paths to remote devices.

```
#include "star-api.h"
```

## Functions

- `STAR_DEVICE_ID STAR_API_CC STAR_CFG_createRemoteDeviceIdentifier (STAR_DEVICE_ID deviceId, unsigned char localChannel, char *name, STAR_SPACEWIRE_ADDRESS *pathTo, STAR_SPACEWIRE_ADDRESS *returnPath)`  
*Creates a Remote Device Identifier.*
- `int STAR_API_CC STAR_CFG_destroyRemoteDeviceIdentifier (STAR_DEVICE_ID remoteDeviceId)`  
*Destroys a Remote Device Identifier.*
- `_Ret_opt_cap_ count STAR_DEVICE_ID *STAR_API_CC STAR_CFG_getRemoteDeviceDescriptionList (_Out_ U32 *count)`  
*Gets an array of all remote devices that have been created by all processes.*
- `void STAR_API_CC STAR_CFG_destroyRemoteDeviceDescriptionList (_Post_ptr_invalid_ STAR_DEVICE_ID *pRemoteDeviceDescriptionList)`  
*Frees a remote device description list previously created by a call to `STAR_CFG_getRemoteDeviceDescriptionList()`.*
- `int STAR_API_CC STAR_CFG_getRemoteDeviceLocalChannel (STAR_DEVICE_ID remoteDeviceId, unsigned char *localChannel)`  
*Gets the number of the local device channel used to communicate with the remote device on.*
- `int STAR_API_CC STAR_CFG_getRemoteDeviceLocalDevice (STAR_DEVICE_ID remoteDeviceId, STAR_DEVICE_ID *localDeviceId)`  
*Gets the local device used to communicate with the remote device on.*
- `int STAR_API_CC STAR_CFG_getRemoteDevicePath (STAR_DEVICE_ID remoteDeviceId, _Out_opt_ STAR_SPACEWIRE_ADDRESS **pPath)`  
*Gets the path to the specified remote device.*
- `int STAR_API_CC STAR_CFG_getRemoteDeviceRetPath (STAR_DEVICE_ID remoteDeviceId, _Out_opt_ STAR_SPACEWIRE_ADDRESS **pRetPath)`  
*Gets the return path from to the specified remote device.*
- `_Check_return_ _Ret_opt_z_ char *STAR_API_CC STAR_CFG_getRemoteDeviceDescriptionNameString (STAR_DEVICE_ID deviceId)`  
*Gets the name of the remote device description identified by a given identifier.*

### 8.13.1 Detailed Description

Functions used to create, configure and destroy paths to remote devices.

#### Author

STAR-Dundee Ltd.  
 STAR House  
 166 Nethergate  
 Dundee, DD1 4EE  
 Scotland, UK  
 e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2012 STAR-Dundee Ltd.

## 8.14 cfg\_api\_router.h File Reference

Functions used to configure STAR-Dundee routing devices.

```
#include "star-api.h"
#include "cfg_api_remote.h"
#include "cfg_api_router_types.h"
```

## Functions

- `int STAR_API_CC CFG_ROUTER_getDeviceIdentificationInfo (STAR_DEVICE_ID deviceID, _Out_ STAR_CFG_DEVICE_IDENTIFIER_INFO *pInfo)`  
*Reads the device identifier information of a device, i.e.*
- `void STAR_API_CC CFG_ROUTER_getDeviceManufacturerAsString (STAR_CFG_DEVICE_IDENTIFIER_INFO *pDeviceIdentifierInfo, _Out_z_cap_c_ (STAR_CFG_MANUFACTURER_STR_MAX_LEN) char *manufacturerStr)`  
*If the manufacturer is known, this function obtains a string representation of the manufacturers name.*
- `void STAR_API_CC CFG_ROUTER_getDeviceTypeAsString (STAR_CFG_DEVICE_IDENTIFIER_INFO *pDeviceIdentifierInfo, _Out_z_cap_c_ (STAR_CFG_DEVICE_STR_MAX_LEN) char *deviceTypeStr)`  
*If the manufacturer and device type is known, this function obtains a string representation of the device's type.*
- `int STAR_API_CC CFG_ROUTER_getNetworkDiscoveryInfo (STAR_DEVICE_ID deviceID, _Out_ STAR_CFG_NETWORK_DISCOVERY_INFO *pInfo)`  
*Reads the network discovery information of a device.*
- `int STAR_API_CC CFG_ROUTER_getPortStatusControl (STAR_DEVICE_ID deviceID, U8 portNum, _Out_ PORT_STATUS_CONTROL *portStatusControl)`  
*Gets the value of a port or link's status/control register, which can then be used by the various functions within the Port Status and Control group.*
- `STAR_CFG_PORT_TYPE STAR_API_CC CFG_ROUTER_getPortType (PORT_STATUS_CONTROL portStatusControl)`  
*Identifies the type of port attributed to a given port number.*
- `U8 STAR_API_CC CFG_ROUTER_getPortConnection (PORT_STATUS_CONTROL portStatusControl)`  
*Identifies the output port to which the source port is currently connected whilst routing is in operation.*
- `int STAR_API_CC CFG_ROUTER_clearPortErrors (STAR_DEVICE_ID deviceID, U8 portNum)`  
*Clears all errors on a given port.*
- `int STAR_API_CC CFG_ROUTER_getConfigPortErrors (PORT_STATUS_CONTROL portStatusControl, _Out_ STAR_CFG_CONFIG_PORT_ERRORS *errors)`  
*Gets any errors present on the config port.*
- `int STAR_API_CC CFG_ROUTER_getSpaceWireLinkErrors (PORT_STATUS_CONTROL linkStatusControl, _Out_ STAR_CFG_SPW_LINK_ERRORS *errors)`  
*Gets any errors present on a SpaceWire link.*
- `int STAR_API_CC CFG_ROUTER_getSpaceWireLinkStatus (PORT_STATUS_CONTROL linkStatusControl, _Out_ STAR_CFG_SPW_LINK_STATUS *status)`  
*Gets the status of a SpaceWire Link.*
- `int STAR_API_CC CFG_ROUTER_setSpaceWireLinkStatus (STAR_DEVICE_ID deviceID, U8 linkNum, STAR_CFG_SPW_LINK_STATUS linkStatus)`  
*Sets the state of a SpaceWire Link.*
- `int STAR_API_CC CFG_ROUTER_startLink (STAR_DEVICE_ID deviceID, U8 linkNum)`  
*Starts a SpaceWire link by setting the start bit, and clearing the disable bit.*
- `int STAR_API_CC CFG_ROUTER_stopLink (STAR_DEVICE_ID deviceID, U8 linkNum)`  
*Stops a SpaceWire link by clearing the start bit, and setting the disable bit.*
- `int STAR_API_CC CFG_ROUTER_getExternalPortErrors (PORT_STATUS_CONTROL portStatusControl, _Out_ STAR_CFG_EXTERNAL_PORT_ERRORS *errors)`  
*Gets any errors present on an external port.*
- `int STAR_API_CC CFG_ROUTER_getExternalPortStatus (PORT_STATUS_CONTROL portStatusControl, _Out_ STAR_CFG_EXTERNAL_PORT_STATUS *status)`  
*Gets the status of an external port.*
- `int STAR_API_CC CFG_ROUTER_getRoutingTableEntry (STAR_DEVICE_ID deviceID, U8 logicalAddress, _Out_ STAR_CFG_GAR_ENTRY *entry)`  
*Gets a routing table entry for a given logical address.*
- `int STAR_API_CC CFG_ROUTER_setRoutingTableEntry (STAR_DEVICE_ID deviceID, U8 logicalAddress, STAR_CFG_GAR_ENTRY entry)`

*Sets a routing table entry for a given logical address.*

- int STAR\_API\_CC\_CFG\_ROUTER\_setNetworkIdentity (STAR\_DEVICE\_ID deviceId, REGISTER value)

*Sets the value of the network identity register.*

- int STAR\_API\_CC\_CFG\_ROUTER\_getNetworkIdentity (STAR\_DEVICE\_ID deviceId, \_Out\_ REGISTER \*value)

*Gets the value of the network identity register.*

- int STAR\_API\_CC\_CFG\_ROUTER\_setGeneralPurpose (STAR\_DEVICE\_ID deviceId, REGISTER value)

*Sets the value of the general purpose register.*

- int STAR\_API\_CC\_CFG\_ROUTER\_getGeneralPurpose (STAR\_DEVICE\_ID deviceId, \_Out\_ REGISTER \*value)

*Gets the value of the general purpose register.*

- int STAR\_API\_CC\_CFG\_ROUTER\_setTimeCodeDistributionPorts (STAR\_DEVICE\_ID deviceId, U32 portMask)

*This function is used to specify which output ports time-codes are forwarded on.*

- int STAR\_API\_CC\_CFG\_ROUTER\_getTimeCodeDistributionPorts (STAR\_DEVICE\_ID deviceId, \_Out\_ U32 \*portMask)

*This function is used to obtain which output ports time-codes are forwarded on.*

- int STAR\_API\_CC\_CFG\_ROUTER\_setTimeCodeFlagMode (STAR\_DEVICE\_ID deviceId, U8 mode)

*Sets the time-code flag interpretation mode:*

- int STAR\_API\_CC\_CFG\_ROUTER\_getTimeCodeFlagMode (STAR\_DEVICE\_ID deviceId, \_Out\_ U8 \*mode)

*Obtains the time-code flag interpretation mode:*

- int STAR\_API\_CC\_CFG\_ROUTER\_getTimeCodeValue (STAR\_DEVICE\_ID deviceId, \_Out\_ U8 \*value)

*Obtains the current value of the router's internal time-code counter.*

- int STAR\_API\_CC\_CFG\_ROUTER\_setRouterGlobalSettings (STAR\_DEVICE\_ID deviceId, STAR\_CFG\_ROUTER\_GLOBAL\_STATE state)

*Sets the router's global settings.*

- int STAR\_API\_CC\_CFG\_ROUTER\_getRouterGlobalSettings (STAR\_DEVICE\_ID deviceId, \_Out\_ STAR\_CFG\_ROUTER\_GLOBAL\_STATE \*state)

*Gets the router's global settings.*

### 8.14.1 Detailed Description

Functions used to configure STAR-Dundee routing devices.

#### Author

STAR-Dundee Ltd.  
 STAR House  
 166 Nethergate  
 Dundee, DD1 4EE  
 Scotland, UK  
 e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2012 STAR-Dundee Ltd.

## 8.15 cfg\_api\_router\_mk2s.h File Reference

Functions used to configure STAR-Dundee Router Mk2S devices.

```
#include "star-api.h"
```



## Functions

- `int STAR_API_CC CFG_ROUTER_MK2S_enablePrecisionTransmitRate (STAR_DEVICE_ID deviceID)`  
*Enables precision transmit rate on a SpaceWire Router Mk2S device.*
- `int STAR_API_CC CFG_ROUTER_MK2S_getPrecisionTransmitRateEnabled (STAR_DEVICE_ID deviceID, _Out_ int *pEnabled)`  
*Determine if precision transmit rate is enabled on a Router Mk2S device.*
- `int STAR_API_CC CFG_ROUTER_MK2S_disablePrecisionTransmitRate (STAR_DEVICE_ID deviceID)`  
*Disables precision transmit rate on a SpaceWire Router Mk2S device.*
- `int STAR_API_CC CFG_ROUTER_MK2S_getPrecisionTransmitRateInUse (STAR_DEVICE_ID deviceID, _Out_ int *pInUse)`  
*Determine whether precision transmit rate is in use on a SpaceWire Router Mk2S device.*
- `int STAR_API_CC CFG_ROUTER_MK2S_getPrecisionTransmitRate (STAR_DEVICE_ID deviceID, _Out_ double *pTransmitRate)`  
*Get the current precision transmit rate on a SpaceWire Router Mk2S device in Mbit/s.*
- `int STAR_API_CC CFG_ROUTER_MK2S_setPrecisionTransmitRate (STAR_DEVICE_ID deviceID, double transmitRate)`  
*Set the current precision transmit rate on a SpaceWire Router Mk2S device in Mbit/s.*

### 8.15.1 Detailed Description

Functions used to configure STAR-Dundee Router Mk2S devices.

#### Author

STAR-Dundee Ltd.  
STAR House  
166 Nethergate  
Dundee, DD1 4EE  
Scotland, UK  
e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2012 STAR-Dundee Ltd.

## 8.16 `cfg_api_router_types.h` File Reference

Types used with the STAR-Dundee Router API.

```
#include "star-api.h"
```

## Data Structures

- struct `STAR_CFG_DEVICE_IDENTIFIER_INFO`  
*Information obtained from a Device's Identifier Register, identifying the router's version and type. [More...](#)*
- struct `STAR_CFG_NETWORK_DISCOVERY_INFO`  
*Information obtained from a device's network discovery register. [More...](#)*
- struct `STAR_CFG_CONFIG_PORT_ERRORS`  
*Errors that may be present on a device's configuration port. [More...](#)*
- struct `STAR_CFG_SPW_LINK_ERRORS`  
*Errors that may be present on a SpaceWire Link. [More...](#)*
- struct `STAR_CFG_SPW_LINK_STATUS`  
*Status of a SpaceWire link. [More...](#)*

- struct `STAR_CFG_EXTERNAL_PORT_ERRORS`  
*Errors that may be present on an external port. [More...](#)*
- struct `STAR_CFG_EXTERNAL_PORT_STATUS`  
*Status of an external port. [More...](#)*
- struct `STAR_CFG_GAR_ENTRY`  
*A Routing table entry. [More...](#)*
- struct `STAR_CFG_ROUTER_GLOBAL_STATE`  
*Global router settings. [More...](#)*

## Macros

- `#define STAR_CFG_MANUFACTURER_STR_MAX_LEN 256`  
*Maximum length for the string representation of a manufacturer name.*
- `#define STAR_CFG_DEVICE_STR_MAX_LEN 256`  
*Maximum length for the string representation of a device type.*

## Typedefs

- typedef `REGISTER_PORT_STATUS_CONTROL`  
*The type used to represent a port status and control register value.*

## Enumerations

- enum `STAR_CFG_DEVICE_TYPE` { `STAR_CFG_DEVICE_TYPE_ROUTER`, `STAR_CFG_DEVICE_TYPE_UNKNOWN`, `STAR_CFG_DEVICE_TYPE_INVALID` }  
*Device types permitted within the network discovery register.*
- enum `STAR_CFG_PORT_TYPE` { `STAR_CFG_PORT_TYPE_CONFIGURATION`, `STAR_CFG_PORT_TYPE_LINK`, `STAR_CFG_PORT_TYPE_EXTERNAL`, `STAR_CFG_PORT_TYPE_INVALID` }  
*Port types for Routing devices.*
- enum `STAR_CFG_SPW_LINK_STATE` { `STAR_CFG_SPW_LINK_STATE_ERROR_RESET`, `STAR_CFG_SPW_LINK_STATE_ERROR_WAIT`, `STAR_CFG_SPW_LINK_STATE_READY`, `STAR_CFG_SPW_LINK_STATE_STARTED`, `STAR_CFG_SPW_LINK_STATE_CONNECTING`, `STAR_CFG_SPW_LINK_STATE_RUN`, `STAR_CFG_SPW_LINK_STATE_INVALID` }  
*The state of the interface state machine in the SpaceWire link.*
- enum `STAR_CFG_PORT_TIMEOUT` { `STAR_CFG_PORT_TIMEOUT_100US` = 0x0, `STAR_CFG_PORT_TIMEOUT_1MS` = 0x1, `STAR_CFG_PORT_TIMEOUT_10MS` = 0x2, `STAR_CFG_PORT_TIMEOUT_100MS` = 0x3, `STAR_CFG_PORT_TIMEOUT_1S` = 0x4 }  
*Length of port timeout.*
- enum `STAR_CFG_TIMEOUT_MODE` { `STAR_CFG_TIMEOUT_MODE_BLOCKING`, `STAR_CFG_TIMEOUT_MODE_WATCHDOG` }  
*Timeout mode used by the router.*

### 8.16.1 Detailed Description

Types used with the STAR-Dundee Router API.

**Author**

STAR-Dundee Ltd.  
STAR House  
166 Nethergate  
Dundee, DD1 4EE  
Scotland, UK  
e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2012 STAR-Dundee Ltd.

## 8.17 common.h File Reference

STAR-Dundee API macros and types required by STAR API modules.

```
#include <string.h>
#include "star-dundee_annotations.h"
```

**Macros**

- `#define STAR_API_MODULE_NAME "STAR-API"`  
*The name of the module, used for logging purposes.*
- `#define STAR_API_CC`  
*The calling convention used by functions exported by the API.*

### 8.17.1 Detailed Description

STAR-Dundee API macros and types required by STAR API modules.

**Author**

STAR-Dundee Ltd.  
STAR House  
166 Nethergate  
Dundee, DD1 4EE  
Scotland, UK  
e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

This file contains the definitions of macros, structures and types required by modules in the STAR-Dundee software stack.

Copyright © 2012 STAR-Dundee Ltd.

### 8.17.2 Macro Definition Documentation

#### 8.17.2.1 `#define STAR_API_CC`

The calling convention used by functions exported by the API.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

[channel\\_callback\\_example.c](#), [packet\\_subsystem\\_example.c](#), [rmap\\_target\\_example.c](#), and [transfer\\_callback\\_example.c](#).

### 8.17.2.2 #define STAR\_API\_MODULE\_NAME "STAR-API"

The name of the module, used for logging purposes.

## 8.18 general.h File Reference

General functions provided by STAR-API.

```
#include "common.h"
#include "types.h"
#include "stream_item_types.h"
#include "version.h"
```

### Functions

- [STAR\\_VERSION\\_INFO \\*STAR\\_API\\_CC STAR\\_getApiVersion \(void\)](#)  
*Gets the Version of STAR-API.*
- [void STAR\\_API\\_CC STAR\\_destroyVersionInfo \(\\_Post\\_ptr\\_invalid\\_ STAR\\_VERSION\\_INFO \\*pVersionInfo\)](#)  
*Frees a version information structure previously created by a call to [STAR\\_getApiVersion\(\)](#), [STAR\\_getDriverVersion\(\)](#) or [STAR\\_getDeviceFirmwareVersion\(\)](#).*
- [\\_Ret\\_opt\\_cap\\_ count STAR\\_VERSION\\_INFO \\*STAR\\_API\\_CC STAR\\_getAllVersions \(\\_Out\\_ U32 \\*count\)](#)  
*Gets an array of all the version info structures provided by all modules of STAR-System.*
- [void STAR\\_API\\_CC STAR\\_destroyVersionInfoList \(\\_Post\\_ptr\\_invalid\\_ STAR\\_VERSION\\_INFO \\*pVersionInfoList\)](#)  
*Frees a version information list previously created by a call to [STAR\\_getAllVersions\(\)](#).*
- [int STAR\\_API\\_CC STAR\\_setApplicationName \(\\_In\\_z\\_ char \\*name\)](#)  
*Sets the name of the application using STAR-API.*
- [\\_Check\\_return\\_ \\_Ret\\_opt\\_z\\_ char \\*STAR\\_API\\_CC STAR\\_getApplicationName \(void\)](#)  
*Gets the name of the calling process (this process), set by [STAR\\_setApplicationName\(\)](#).*
- [STAR\\_APP\\_ID STAR\\_API\\_CC STAR\\_getApplicationID \(void\)](#)  
*Gets the application ID of the calling process.*
- [\\_Check\\_return\\_ \\_Ret\\_opt\\_z\\_ char \\*STAR\\_API\\_CC STAR\\_getApplicationNameForID \(STAR\\_APP\\_ID appld\)](#)  
*Gets the name of the application with the given ID.*
- [void STAR\\_API\\_CC STAR\\_destroyString \(\\_Post\\_ptr\\_invalid\\_ \\_In\\_z\\_ char \\*string\)](#)  
*Destroy a string returned by STAR-API, such as from [STAR\\_getApplicationName\(\)](#) or [STAR\\_getDeviceTypeAsString\(\)](#).*
- [\\_Ret\\_opt\\_cap\\_ count STAR\\_DRIVER\\_ID \\*STAR\\_API\\_CC STAR\\_getDriverList \(int physicalDeviceDrivers, int virtualDeviceDrivers, \\_Out\\_ U32 \\*count\)](#)  
*Gets an array of driver identifiers for all drivers of the requested type(s).*
- [void STAR\\_API\\_CC STAR\\_destroyDriverList \(\\_Post\\_ptr\\_invalid\\_ STAR\\_DRIVER\\_ID \\*pDriverList\)](#)  
*Frees a driver list previously created by a call to [STAR\\_getDriverList\(\)](#).*
- [\\_Check\\_return\\_ STAR\\_DRIVER\\_LISTENER\\_ID STAR\\_API\\_CC STAR\\_registerDriverListener \(STAR\\_DriverEventFunc listenerFunc, \\_In\\_opt\\_ void \\*pContextInfo, int notifyForCurrent\)](#)  
*Registers a Call-back function that will be called whenever a Driver is added or removed.*
- [int STAR\\_API\\_CC STAR\\_unregisterDriverListener \(STAR\\_DRIVER\\_LISTENER\\_ID listenerId\)](#)  
*Unregister a Call-back function that was previously registered to be called whenever a driver is added or removed.*
- [STAR\\_VERSION\\_INFO \\*STAR\\_API\\_CC STAR\\_getDriverVersion \(STAR\\_DRIVER\\_ID driverId\)](#)  
*Gets the Version of a given driver.*
- [STAR\\_DRIVER\\_TYPE STAR\\_API\\_CC STAR\\_getDriverType \(STAR\\_DRIVER\\_ID driverId\)](#)  
*Gets the type (USB/PCI/specific virtual type identifier) of a given driver.*
- [\\_Ret\\_opt\\_cap\\_ count STAR\\_DEVICE\\_ID \\*STAR\\_API\\_CC STAR\\_getDeviceList \(\\_Out\\_ U32 \\*count\)](#)

- Gets an array of identifiers for all devices present for all drivers.*
- `_Ret_opt_cap_ pCount STAR_DEVICE_ID *STAR_API_CC STAR_getDeviceListForType (STAR_DEVICE_ID deviceType, _Out_ U32 *pCount)`
- Gets an array of identifiers for all devices present for the given device type.*
- `_Ret_opt_cap_ pCount STAR_DEVICE_ID *STAR_API_CC STAR_getDeviceListForTypes (STAR_DEVICE_ID deviceTypes[], U32 numRequestedTypes, _Out_ U32 *pCount)`
- Gets an array of identifiers for all devices present for the given device types in an array.*
- `_Ret_opt_cap_ count STAR_DEVICE_ID *STAR_API_CC STAR_getDeviceListForDriver (STAR_DRIVER_ID driverId, _Out_ U32 *count)`
- Gets an array of device identifiers for all devices present for a specific driver.*
- `void STAR_API_CC STAR_destroyDeviceList (_Post_ptr_invalid_ STAR_DEVICE_ID *pDeviceList)`
- Frees a device list previously created by a call to `STAR_getDeviceList()` or `STAR_getDeviceListForDriver()`.*
- `_Check_return_ STAR_DEVICE_LISTENER_ID STAR_API_CC STAR_registerDeviceListener (_In_ STAR_DEVICE_ID deviceId, STAR_DeviceEventFunc listenerFunc, _In_opt_ void *pContextInfo, int notifyForCurrent)`
- Registers a Call-back function that will be called whenever a device is added or removed.*
- `_Check_return_ STAR_DEVICE_LISTENER_ID STAR_API_CC STAR_registerDeviceListenerForDriver (STAR_DRIVER_ID driverId, _In_ STAR_DeviceEventFunc listenerFunc, _In_opt_ void *pContextInfo, int notifyForCurrent)`
- Registers a Call-back function that will be called whenever a device for a specific driver is added or removed.*
- `_Check_return_ STAR_DEVICE_LISTENER_ID STAR_API_CC STAR_registerDeviceListenerForDevice (STAR_DEVICE_ID deviceId, _In_ STAR_DeviceEventFunc listenerFunc, _In_opt_ void *pContextInfo)`
- Registers a Call-back function that will be called whenever a specific device is removed.*
- `int STAR_API_CC STAR_unregisterDeviceListener (STAR_DEVICE_LISTENER_ID listenerId)`
- Unregister a Call-back function that was previously registered to be called whenever a device is added or removed.*
- `int STAR_API_CC STAR_resetDevice (STAR_DEVICE_ID deviceId)`
- Resets a device.*
- `STAR_VERSION_INFO *STAR_API_CC STAR_getDeviceFirmwareVersion (STAR_DEVICE_ID deviceId)`
- Gets the version of a device's firmware.*
- `STAR_BUS_TYPE STAR_API_CC STAR_getDeviceBusType (STAR_DEVICE_ID deviceId)`
- Gets the bus type (PCI, USB, Virtual, etc) for the device.*
- `_Check_return_ _Ret_opt_z_ char *STAR_API_CC STAR_getDeviceBusTypeAsString (STAR_DEVICE_ID deviceId)`
- Gets a string representation of a device's bus type.*
- `int STAR_API_CC STAR_getDeviceIndex (STAR_DEVICE_ID deviceId)`
- Gets the device index by Driver.*
- `STAR_DEVICE_TYPE STAR_API_CC STAR_getDeviceType (STAR_DEVICE_ID deviceId)`
- Gets the device's type identifier.*
- `_Check_return_ _Ret_opt_z_ char *STAR_API_CC STAR_getDeviceTypeAsString (STAR_DEVICE_ID deviceId)`
- Gets a string representation of a device's type.*
- `STAR_DRIVER_ID STAR_API_CC STAR_getDeviceDriver (STAR_DEVICE_ID deviceId)`
- Gets the identifier of the device's driver.*
- `_Check_return_ _Ret_opt_z_ char *STAR_API_CC STAR_getDeviceName (STAR_DEVICE_ID deviceId)`
- Gets the name of the device identified by a given identifier.*
- `int STAR_API_CC STAR_setDeviceName (STAR_DEVICE_ID deviceId, _In_z_ char *newName)`
- Sets the name of the device (friendly name).*
- `int STAR_API_CC STAR_resetDeviceName (STAR_DEVICE_ID deviceId)`
- Resets a device's name to its default value.*
- `_Check_return_ _Ret_opt_z_ char *STAR_API_CC STAR_getDeviceSerialNumber (STAR_DEVICE_ID deviceId)`
- Gets the serial number (unique alphanumeric identifier) of the device identified by a given identifier.*
- `STAR_DEVICE_TYPE STAR_API_CC STAR_getDeviceTxRxCapabilities (STAR_DEVICE_ID deviceId)`

*Determine whether the device is capable of transmitting and receiving packets on channels.*

- `STAR_DEVICE_TYPE STAR_API_CC STAR_getDeviceConfigCapabilities (STAR_DEVICE_ID deviceId)`

*Determine whether the device is capable of being configured.*

- `STAR_CHANNEL_MASK STAR_API_CC STAR_getDeviceChannels (STAR_DEVICE_ID deviceId)`

*Gets the channels on a device.*

- `_Check_return_ STAR_CHANNEL_ID STAR_API_CC STAR_openChannelBetweenLocalDevices (STAR_DEVICE_ID deviceA, unsigned char channelA, STAR_DEVICE_ID deviceB, unsigned char channelB)`

*Open a channel between two devices on the specified channel numbers.*

- `_Check_return_ STAR_CHANNEL_ID STAR_API_CC STAR_openChannelToLocalDevice (STAR_DEVICE_ID device, STAR_CHANNEL_DIRECTION direction, unsigned char channelNumber, int isQueued)`

*Open a channel to a device on the specified channel number.*

- `_Check_return_ STAR_CHANNEL_ID STAR_API_CC STAR_openChannelToLocalDeviceEx (_In_ STAR_DEVICE_ID device, _In_ STAR_CHANNEL_DIRECTION direction, _In_ unsigned char channelNumber, _In_ int isQueued)`

*This function is an alternative to `STAR_openChannelToLocalDevice` that uses an optimised receive strategy.*

- `int STAR_API_CC STAR_closeChannel (STAR_CHANNEL_ID channelId)`

*Close a previously opened channel.*

- `STAR_CHANNEL_TYPE STAR_API_CC STAR_isChannelOpen (STAR_DEVICE_ID deviceId, unsigned int channelNumber)`

*Returns whether a given channel on a device is open and, if it is open, whether it is connected to a device or an application.*

- `_Check_return_ STAR_CHANNEL_DIRECTION STAR_API_CC STAR_getChannelDirection (STAR_CHANNEL_ID channelId)`

*Get the direction information of an open channel.*

- `STAR_DEVICE_ID STAR_API_CC STAR_getLocalDeviceAttachedToChannel (STAR_DEVICE_ID deviceId, unsigned int channelNumber)`

*Returns the id of a device attached to a given channel on a given device.*

- `STAR_APP_ID STAR_API_CC STAR_getApplicationAttachedToChannel (STAR_DEVICE_ID deviceId, unsigned int channelNumber)`

*Returns the ID of an application attached to a given channel on a given device.*

- `_Check_return_ STAR_CHANNEL_LISTENER_ID STAR_API_CC STAR_registerChannelListener (_In_ STAR_ChannelEventFunc listenerFunc, _In_opt_ void *pContextInfo, int notifyForCurrent)`

*Registers a Call-back function that will be called whenever any channel is opened or closed.*

- `_Check_return_ STAR_CHANNEL_LISTENER_ID STAR_API_CC STAR_registerChannelListenerForDriver (STAR_DRIVER_ID driverId, _In_ STAR_ChannelEventFunc listenerFunc, _In_opt_ void *pContextInfo, int notifyForCurrent)`

*Registers a Call-back function that will be called whenever a channel is opened or closed on a device which has the specific driver.*

- `_Check_return_ STAR_CHANNEL_LISTENER_ID STAR_API_CC STAR_registerChannelListenerForDevice (STAR_DEVICE_ID deviceId, _In_ STAR_ChannelEventFunc listenerFunc, _In_opt_ void *pContextInfo, int notifyForCurrent)`

*Registers a Call-back function that will be called whenever a channel on the specific device is opened or closed.*

- `int STAR_API_CC STAR_unregisterChannelListener (STAR_CHANNEL_LISTENER_ID listenerId)`

*Unregister a Call-back function that was previously registered to be called whenever a channel is opened or closed.*

- `int STAR_API_CC STAR_isDriverVirtual (STAR_DRIVER_ID driverId)`

*Gets whether or not a given driver is virtual.*

- `int STAR_API_CC STAR_isDeviceVirtual (STAR_DEVICE_ID deviceId)`

*Gets whether or not a given device is virtual.*

### 8.18.1 Detailed Description

General functions provided by STAR-API.

#### Author

STAR-Dundee Ltd.  
 STAR House  
 166 Nethergate  
 Dundee, DD1 4EE  
 Scotland, UK  
 e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

This file contains the declarations of the general functions provided by STAR-API, the STAR-Dundee software API, relating to the API itself, along with the functions used for managing channels.

Copyright © 2012 STAR-Dundee Ltd.

## 8.19 packet\_subsystem.h File Reference

Definitions of the functions provided by the PCI Mk2 packet subsystem API.

```
#include "star-api.h"
#include <limits.h>
```

### Data Structures

- struct [PKT\\_SUBSYS\\_STATISTICS](#)

*Statistics that can be obtained from the PCI Mk2 device packet sink subsystem. [More...](#)*

### Macros

- `#define` [PKT\\_SUBSYS\\_MODULE\\_NAME](#) "PACKET\_SUBSYSTEM"

*The name of the module, used for logging purposes.*

### Typedefs

- typedef void([STAR\\_API\\_CC](#) \* [PKT\\_SUBSYS\\_StatisticsFunc](#)) ([STAR\\_DEVICE\\_ID](#) deviceId, unsigned int subsystem, [PKT\\_SUBSYS\\_STATISTICS](#) statistics)

*The function type used by the statistics loop function, which is called when a monitored subsystem's statistics are updated.*

- typedef [U32](#) [STATISTICS\\_LOOP\\_ID](#)

*Type of identifier used to identify a statistics polling thread.*

### Enumerations

- enum [PKT\\_SUBSYS\\_PATTERN](#) {  
[PKT\\_SUBSYS\\_PATTERN\\_ALL\\_ZEROES](#), [PKT\\_SUBSYS\\_PATTERN\\_ALL\\_ONES](#), [PKT\\_SUBSYS\\_PATTERN\\_ALTERNATING\\_BITS](#), [PKT\\_SUBSYS\\_PATTERN\\_INCREMENTING\\_BYTES](#),  
[PKT\\_SUBSYS\\_PATTERN\\_DECREMENTING\\_BYTES](#), [PKT\\_SUBSYS\\_PATTERN\\_WALKING\\_ONES](#),  
[PKT\\_SUBSYS\\_PATTERN\\_WALKING\\_ZEROES](#) }

*Available patterns to use with [PKT\\_SUBSYS\\_fillBufferWithPattern\(\)](#)*

## Functions

- void `STAR_API_CC PKT_SUBSYS_fillBufferWithPattern` (`PKT_SUBSYS_PATTERN` pattern, U16 bufferSize, `_Out_bytecap_(bufferSize)` void \*pBuffer)  
*Convenience function that fills a user provided buffer with a known pattern.*
- int `STAR_API_CC PKT_SUBSYS_writeMemory` (`STAR_DEVICE_ID` deviceId, U16 address, U16 writeSize, `_In_bytecount_(writeSize)` void \*pBuffer)  
*Writes a buffer to the PCI Mk2's dual port memory.*
- int `STAR_API_CC PKT_SUBSYS_readMemory` (`STAR_DEVICE_ID` deviceId, U16 address, U16 readSize, `_Inout_bytecap_(readSize)` void \*pBuffer)  
*Reads from the PCI Mk2's dual port memory to a user allocated buffer.*
- int `STAR_API_CC PKT_SUBSYS_setPacketGeneratorFormat` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem, U16 headerPointer, U16 headerLength, U16 bodyPointer, U16 bodyLength, U32 packetLength, `STAR_EOP_TYPE` eop, U16 gap)  
*Sets up the format of the packet to be inserted by the packet generator into the router.*
- int `STAR_API_CC PKT_SUBSYS_setGeneratorBlockingParameters` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem, U16 blockFrequency, U16 blockDuration)  
*Sets up the packet generation subsystem blocking parameters.*
- int `STAR_API_CC PKT_SUBSYS_startPacketGeneration` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem, U16 sequenceLength)  
*Starts the packet generator inserting packets into the router.*
- int `STAR_API_CC PKT_SUBSYS_waitForPacketGenerationSequenceComplete` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem, unsigned int timeout)  
*Blocks the current thread until packet generation has stopped.*
- int `STAR_API_CC PKT_SUBSYS_stopPacketGeneration` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem)  
*Stops the packet generator inserting packets into the router.*
- int `STAR_API_CC PKT_SUBSYS_setPacketSinkParameters` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem, U16 memoryPointer, U16 memoryLength)  
*Sets up the circular buffer for the packet sink subsystem.*
- int `STAR_API_CC PKT_SUBSYS_setSinkBlockingParameters` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem, U16 blockFrequency, U16 blockDuration)  
*Sets up the packet sink subsystem blocking parameters.*
- int `STAR_API_CC PKT_SUBSYS_startSink` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem)  
*Starts the packet sink reading packets into memory.*
- int `STAR_API_CC PKT_SUBSYS_stopSink` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem)  
*Stops the packet sink reading packets into memory.*
- int `STAR_API_CC PKT_SUBSYS_getSinkDataPointers` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem, U16 \*startPointer, U16 \*endPointer)  
*Gets the address for the start and end of valid data in the circular buffer in device memory.*
- int `STAR_API_CC PKT_SUBSYS_startPacketChecking` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem, int disableSinkOnError, U16 disableSinkdelay)  
*Starts a packet checker running on a packet sink.*
- int `STAR_API_CC PKT_SUBSYS_stopPacketChecking` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem)  
*Stops the packet checker.*
- int `STAR_API_CC PKT_SUBSYS_waitForPacketCheckingError` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem)  
*Blocks the current thread until either a character mismatch is detected and the packet checker is automatically stopped, or the user manually stops the packet checker.*
- int `STAR_API_CC PKT_SUBSYS_cancelWaitsForPacketCheckingError` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem)  
*Cancels all waits started by `PKT_SUBSYS_waitForPacketCheckingError()` for packet checker errors on a given device and subsystem.*



- int `STAR_API_CC PKT_SUBSYS_setPacketCheckingFormat` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem, U16 headerPointer, U16 headerLength, U16 bodyPointer, U16 bodyLength, U32 packetLength, `STAR_EOP_TYPE` eop)

*Sets up the packet checker to check packets match the format of packets provided by user entered parameters.*

- int `STAR_API_CC PKT_SUBSYS_getPacketCheckingMismatchCount` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem, U64 \*errorCount)

*Gets the number of mismatched data characters observed on the SpaceWire interface.*

- int `STAR_API_CC PKT_SUBSYS_getStatistics` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem, `PKT_SUBSYS_STATISTICS` \*pStatistics)

*Gets statistics for a given subsystem.*

- `STATISTICS_LOOP_ID` `STAR_API_CC PKT_SUBSYS_startGetStatisticsLoop` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem, `PKT_SUBSYS_StatisticsFunc` statisticsHandler)

*Starts a thread that polls statistics on a given subsystem, and calls a given function with the updated statistics when they are retrieved.*

- void `STAR_API_CC PKT_SUBSYS_stopGetStatisticsLoop` (`STATISTICS_LOOP_ID` loopId)

*Stops a statistics gathering loop set up by `PKT_SUBSYS_startGetStatisticsLoop()`.*

### 8.19.1 Detailed Description

Definitions of the functions provided by the PCI Mk2 packet subsystem API.

#### Author

STAR-Dundee Ltd.  
 STAR House  
 166 Nethergate  
 Dundee, DD1 4EE  
 Scotland, UK  
 e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2012 STAR-Dundee Ltd.

### 8.19.2 Macro Definition Documentation

#### 8.19.2.1 `#define PKT_SUBSYS_MODULE_NAME "PACKET_SUBSYSTEM"`

The name of the module, used for logging purposes.

## 8.20 rmap\_packet\_library.h File Reference

Declarations of the functions provided by the STAR-Dundee RMAP Packet Library.

```
#include "star-dundee_types.h"
```

### Data Structures

- struct `RMAP_PACKET`

*A structure used to describe an RMAP packet. [More...](#)*

## Macros

- `#define RMAPPACKETLIBRARY_CC`  
*The calling convention used by functions exported by the RMAP Packet Library.*
- `#define RMAP_PROTOCOL_IDENTIFIER 1`  
*The value used in the protocol identifier field of all RMAP packets.*
- `#define PNP_PROTOCOL_IDENTIFIER 3`  
*The value used in the protocol identifier field of all SpaceWire-PnP packets.*
- `#define RMAP_RESERVED_BIT 0x80`  
*A mask for the reserved bit of the Instruction field.*
- `#define RMAP_COMMAND_BIT 0x40`  
*A packet is a command if the bit specified by this mask is set in the Instruction field.*
- `#define RMAP_WRITE_OPERATION_BIT 0x20`  
*A packet is a write operation if the bit specified by this mask is set in the Instruction field.*
- `#define RMAP_VERIFY_BEFORE_WRITE_BIT 0x10`  
*An operation verifies before writing if the bit specified by this mask is set in the Instruction field.*
- `#define RMAP_REPLY_BIT 0x08`  
*An operation is acknowledged if the bit specified by this mask is set in the Instruction field.*
- `#define RMAP_INCREMENT_ADDRESS_BIT 0x04`  
*A read or write address is incremented if the bit specified by this mask is set in the Instruction field.*
- `#define RMAP_REPLY_ADDRESS_LENGTH_BITS 0x03`  
*A mask for the bits in the Instruction field containing the length of the reply address.*
- `#define RMAP_GET_VERSION_MAJOR(versionInfo) (((version) & 0xff000000) >> 24)`  
*Return the major version number from the specified version information.*
- `#define RMAP_GET_VERSION_MINOR(versionInfo) (((version) & 0x00ff0000) >> 16)`  
*Return the minor version number from the specified version information.*
- `#define RMAP_GET_VERSION_EDIT(versionInfo) (((version) & 0x0000ffc0) >> 6)`  
*Return the edit number from the specified version information.*
- `#define RMAP_GET_VERSION_PATCH(versionInfo) ((version) & 0x0000003f)`  
*Return the patch level from the specified version information.*

## Enumerations

- `enum RMAP_STATUS {`  
`RMAP_SUCCESS = 0x00, RMAP_GENERAL_ERROR = 0x01, RMAP_UNUSED_PACKET_TYPE_OR_↵`  
`COMMAND_CODE = 0x02, RMAP_INVALID_KEY = 0x03,`  
`RMAP_INVALID_DATA_CRC = 0x04, RMAP_EARLY_EOP = 0x05, RMAP_TOO_MUCH_DATA = 0x06,`  
`RMAP_EEP = 0x07,`  
`RMAP_VERIFY_BUFFER_OVERRUN = 0x09, RMAP_COMMAND_NOT_IMPLEMENTED_OR_AUTHO↵`  
`RISED = 0x0a, RMAP_RMW_DATA_LENGTH_ERROR = 0x0b, RMAP_INVALID_TARGET_LOGICAL_A↵`  
`DDRESS = 0x0c,`  
`RMAP_INVALID_STATUS = 0xff }`  
*Possible values for the status of RMAP operations.*
- `enum RMAP_PACKET_TYPE {`  
`RMAP_WRITE_COMMAND = (RMAP_COMMAND_BIT | RMAP_WRITE_OPERATION_BIT), RMAP_WR↵`  
`ITE_REPLY = (RMAP_WRITE_OPERATION_BIT), RMAP_READ_COMMAND = (RMAP_COMMAND_BIT),`  
`RMAP_READ_REPLY = (0),`  
`RMAP_READ_MODIFY_WRITE_COMMAND = (RMAP_COMMAND_BIT | RMAP_VERIFY_BEFORE_W↵`  
`RITE_BIT), RMAP_READ_MODIFY_WRITE_REPLY = (RMAP_VERIFY_BEFORE_WRITE_BIT), RMAP↵`  
`_INVALID_PACKET_TYPE = (0xff) }`  
*Possible values for the packet type.*

## Functions

- **U32 RMAPPACKETLIBRARY\_CC RMAP\_GetVersion** (void)  
*Return the current version information for the RMAP Packet Library.*
- **U8 RMAPPACKETLIBRARY\_CC RMAP\_CalculateCRC** (void \*pBuffer, unsigned long len)  
*Calculate an 8-bit CRC for the given buffer.*
- **U8 RMAPPACKETLIBRARY\_CC RMAP\_CalculateCRCWithSeed** (void \*pBuffer, unsigned long len, U8 crc)  
*Calculate an 8-bit CRC for the given buffer, starting with a seed CRC.*
- **char RMAPPACKETLIBRARY\_CC RMAP\_IsCRCValid** (void \*pBuffer, unsigned long len, U8 crc)  
*Determine if the specified 8-bit CRC is valid for the given buffer.*
- **RMAP\_STATUS RMAPPACKETLIBRARY\_CC RMAP\_CheckPacketValid** (void \*pRawPacket, unsigned long packetLength, RMAP\_PACKET \*pPacketStruct, char checkPacketTooLong)  
*Check that the packet specified is in the correct format for an RMAP packet.*
- **RMAP\_STATUS RMAPPACKETLIBRARY\_CC RMAP\_CheckPacketValidIgnoreProtocol** (void \*pRawPacket, unsigned long packetLength, RMAP\_PACKET \*pPacketStruct, char checkPacketTooLong)  
*Check that the packet specified is in the correct format for an RMAP packet, but do not check that the protocol identifier used by the packet is the RMAP protocol ID.*
- **RMAP\_PACKET\_TYPE RMAPPACKETLIBRARY\_CC RMAP\_GetPacketType** (RMAP\_PACKET \*pPacketStruct)  
*Get the type of the packet (whether it is a read, write or read/modify/write command or reply) from a packet structure which has already been created.*
- **U8 \*RMAPPACKETLIBRARY\_CC RMAP\_GetTargetAddress** (RMAP\_PACKET \*pPacketStruct, unsigned long \*pTargetAddressLength)  
*Get the target address of the packet from a packet structure which has already been created.*
- **char RMAPPACKETLIBRARY\_CC RMAP\_GetVerifyBeforeWrite** (RMAP\_PACKET \*pPacketStruct)  
*Get whether the packet represented by a previously created packet structure has verify before write enabled, to check any data before writing it to memory.*
- **char RMAPPACKETLIBRARY\_CC RMAP\_GetPerformAcknowledgement** (RMAP\_PACKET \*pPacketStruct)  
*Get whether the packet represented by a previously created packet structure has acknowledgement enabled, to acknowledge the command with a reply packet.*
- **char RMAPPACKETLIBRARY\_CC RMAP\_GetIncrementAddress** (RMAP\_PACKET \*pPacketStruct)  
*Get whether the packet represented by a previously created packet structure has incrementing of memory addresses enabled, to read from or write to sequential memory.*
- **U8 RMAPPACKETLIBRARY\_CC RMAP\_GetKey** (RMAP\_PACKET \*pPacketStruct)  
*Get the key field in the packet represented by a previously created packet structure.*
- **U8 \*RMAPPACKETLIBRARY\_CC RMAP\_GetReplyAddress** (RMAP\_PACKET \*pPacketStruct, unsigned long \*pReplyAddressLength)  
*Get the reply address of the packet from a packet structure which has already been created.*
- **U16 RMAPPACKETLIBRARY\_CC RMAP\_GetTransactionID** (RMAP\_PACKET \*pPacketStruct)  
*Get the transaction identifier in the packet represented by a previously created packet structure.*
- **U32 RMAPPACKETLIBRARY\_CC RMAP\_GetAddress** (RMAP\_PACKET \*pPacketStruct, U8 \*pExtendedAddress)  
*Get the memory address and extended memory address of the packet from a packet structure which has already been created.*
- **U8 \*RMAPPACKETLIBRARY\_CC RMAP\_GetData** (RMAP\_PACKET \*pPacketStruct, U32 \*pDataLength)  
*Get the data in the packet from a packet structure which has already been created.*
- **U32 RMAPPACKETLIBRARY\_CC RMAP\_GetReadLength** (RMAP\_PACKET \*pPacketStruct)  
*Gets the data length property in the read command packet from a packet structure which has already been created.*
- **RMAP\_STATUS RMAPPACKETLIBRARY\_CC RMAP\_GetStatus** (RMAP\_PACKET \*pPacketStruct)  
*Get the value of the status field in the packet represented by a previously created packet structure.*
- **U8 RMAPPACKETLIBRARY\_CC RMAP\_GetHeaderCRC** (RMAP\_PACKET \*pPacketStruct)  
*Get the value of the header CRC field in the packet represented by a previously created packet structure.*
- **U8 RMAPPACKETLIBRARY\_CC RMAP\_GetDataCRC** (RMAP\_PACKET \*pPacketStruct)

*Get the value of the data CRC field in the packet represented by a previously created packet structure.*

- `U8 *RMAPPACKETLIBRARY_CC RMAP_GetMask (RMAP_PACKET *pPacketStruct, U8 *pMaskLength)`

*Get the mask in the packet from a packet structure which has already been created.*

- unsigned long `RMAPPACKETLIBRARY_CC RMAP_CalculateWriteCommandPacketLength` (unsigned long targetAddressLength, unsigned long replyAddressLength, U32 dataLength, char alignment)

*Calculate the number of bytes required for a write command packet, given the properties of the packet.*

- char `RMAPPACKETLIBRARY_CC RMAP_FillWriteCommandPacket` (U8 \*pTargetAddress, unsigned long targetAddressLength, U8 \*pReplyAddress, unsigned long replyAddressLength, char verifyBeforeWrite, char acknowledge, char incrementAddress, U8 key, U16 transactionIdentifier, U32 writeAddress, U8 extendedWriteAddress, U8 \*pData, U32 dataLength, unsigned long \*pRawPacketLength, RMAP\_PACKET \*pPacketStruct, char alignment, U8 \*pRawPacket, unsigned long rawPacketLength)

*Fill a previously allocated buffer with an RMAP write command packet which can be sent to perform an RMAP write command.*

- void `*RMAPPACKETLIBRARY_CC RMAP_BuildWriteCommandPacket` (U8 \*pTargetAddress, unsigned long targetAddressLength, U8 \*pReplyAddress, unsigned long replyAddressLength, char verifyBeforeWrite, char acknowledge, char incrementAddress, U8 key, U16 transactionIdentifier, U32 writeAddress, U8 extendedWriteAddress, U8 \*pData, U32 dataLength, unsigned long \*pRawPacketLength, RMAP\_PACKET \*pPacketStruct, char alignment)

*Build an RMAP write command packet which can be sent to perform an RMAP write command.*

- void `*RMAPPACKETLIBRARY_CC RMAP_BuildWriteRegisterPacket` (U8 \*pTargetAddress, unsigned long targetAddressLength, U8 \*pReplyAddress, unsigned long replyAddressLength, char verifyBeforeWrite, char acknowledge, char incrementAddress, U8 key, U16 transactionIdentifier, U32 writeAddress, U8 extendedWriteAddress, U32 registerValue, unsigned long \*pRawPacketLength, RMAP\_PACKET \*pPacketStruct, char alignment)

*Build an RMAP write command packet which can be sent to perform an RMAP write command on a 4-byte register.*

- unsigned long `RMAPPACKETLIBRARY_CC RMAP_CalculateWriteReplyPacketLength` (unsigned long initiatorAddressLength, char alignment)

*Calculate the number of bytes required for a write reply packet, given the properties of the packet.*

- char `RMAPPACKETLIBRARY_CC RMAP_FillWriteReplyPacket` (U8 \*pInitiatorAddress, unsigned long initiatorAddressLength, U8 targetAddress, char verifyBeforeWrite, char incrementAddress, RMAP\_STATUS status, U16 transactionIdentifier, unsigned long \*pRawPacketLength, RMAP\_PACKET \*pPacketStruct, char alignment, U8 \*pRawPacket, unsigned long rawPacketLength)

*Fill a previously allocated buffer with an RMAP write reply packet which can be sent to respond to an RMAP write command.*

- void `*RMAPPACKETLIBRARY_CC RMAP_BuildWriteReplyPacket` (U8 \*pInitiatorAddress, unsigned long initiatorAddressLength, U8 targetAddress, char verifyBeforeWrite, char incrementAddress, RMAP\_STATUS status, U16 transactionIdentifier, unsigned long \*pRawPacketLength, RMAP\_PACKET \*pPacketStruct, char alignment)

*Build an RMAP write reply packet which can be sent to respond to an RMAP write command.*

- unsigned long `RMAPPACKETLIBRARY_CC RMAP_CalculateReadCommandPacketLength` (unsigned long targetAddressLength, unsigned long replyAddressLength, char alignment)

*Calculate the number of bytes required for a read command packet, given the properties of the packet.*

- char `RMAPPACKETLIBRARY_CC RMAP_FillReadCommandPacket` (U8 \*pTargetAddress, unsigned long targetAddressLength, U8 \*pReplyAddress, unsigned long replyAddressLength, char incrementAddress, U8 key, U16 transactionIdentifier, U32 readAddress, U8 extendedReadAddress, U32 dataLength, unsigned long \*pRawPacketLength, RMAP\_PACKET \*pPacketStruct, char alignment, U8 \*pRawPacket, unsigned long rawPacketLength)

*Fill a previously allocated buffer with an RMAP read command packet which can be sent to perform an RMAP read command.*

- void `*RMAPPACKETLIBRARY_CC RMAP_BuildReadCommandPacket` (U8 \*pTargetAddress, unsigned long targetAddressLength, U8 \*pReplyAddress, unsigned long replyAddressLength, char incrementAddress, U8 key, U16 transactionIdentifier, U32 readAddress, U8 extendedReadAddress, U32 dataLength, unsigned long \*pRawPacketLength, RMAP\_PACKET \*pPacketStruct, char alignment)

*Build an RMAP read command packet which can be sent to perform an RMAP read command.*

- void `*RMAPPACKETLIBRARY_CC RMAP_BuildReadRegisterPacket` (U8 \*pTargetAddress, unsigned long targetAddressLength, U8 \*pReplyAddress, unsigned long replyAddressLength, char incrementAddress, U8 key, U16 transactionIdentifier, U32 readAddress, U8 extendedReadAddress, unsigned long \*pRawPacketLength, `RMAP_PACKET` \*pPacketStruct, char alignment)

*Build an RMAP read command packet which can be sent to perform an RMAP read command on a 4-byte register.*

- unsigned long `RMAPPACKETLIBRARY_CC RMAP_CalculateReadReplyPacketLength` (unsigned long initiatorAddressLength, U32 dataLength, char alignment)

*Calculate the number of bytes required for a read reply packet, given the properties of the packet.*

- char `RMAPPACKETLIBRARY_CC RMAP_FillReadReplyPacket` (U8 \*pInitiatorAddress, unsigned long initiatorAddressLength, U8 targetAddress, char incrementAddress, `RMAP_STATUS` status, U16 transactionIdentifier, U8 \*pData, U32 dataLength, unsigned long \*pRawPacketLength, `RMAP_PACKET` \*pPacketStruct, char alignment, U8 \*pRawPacket, unsigned long rawPacketLength)

*Fill a previously allocated buffer with an RMAP read reply packet which can be sent to respond to an RMAP read command.*

- void `*RMAPPACKETLIBRARY_CC RMAP_BuildReadReplyPacket` (U8 \*pInitiatorAddress, unsigned long initiatorAddressLength, U8 targetAddress, char incrementAddress, `RMAP_STATUS` status, U16 transactionIdentifier, U8 \*pData, U32 dataLength, unsigned long \*pRawPacketLength, `RMAP_PACKET` \*pPacketStruct, char alignment)

*Build an RMAP read reply packet which can be sent to respond to an RMAP read command.*

- unsigned long `RMAPPACKETLIBRARY_CC RMAP_CalculateReadModifyWriteCommandPacketLength` (unsigned long targetAddressLength, unsigned long replyAddressLength, U32 dataAndMaskLength, char alignment)

*Calculate the number of bytes required for a read/modify/write command packet, given the properties of the packet.*

- char `RMAPPACKETLIBRARY_CC RMAP_FillReadModifyWriteCommandPacket` (U8 \*pTargetAddress, unsigned long targetAddressLength, U8 \*pReplyAddress, unsigned long replyAddressLength, U8 key, U16 transactionIdentifier, U32 readModifyWriteAddress, U8 extendedReadModifyWriteAddress, U8 dataAndMaskLength, U8 \*pData, U8 \*pMask, unsigned long \*pRawPacketLength, `RMAP_PACKET` \*pPacketStruct, char alignment, U8 \*pRawPacket, unsigned long rawPacketLength)

*Fill a previously allocated buffer with an RMAP read/modify/write command packet which can be sent to perform an RMAP read/modify/write command.*

- void `*RMAPPACKETLIBRARY_CC RMAP_BuildReadModifyWriteCommandPacket` (U8 \*pTargetAddress, unsigned long targetAddressLength, U8 \*pReplyAddress, unsigned long replyAddressLength, U8 key, U16 transactionIdentifier, U32 readModifyWriteAddress, U8 extendedReadModifyWriteAddress, U8 dataAndMaskLength, U8 \*pData, U8 \*pMask, unsigned long \*pRawPacketLength, `RMAP_PACKET` \*pPacketStruct, char alignment)

*Build an RMAP read/modify/write command packet which can be sent to perform an RMAP read/modify/write command.*

- void `*RMAPPACKETLIBRARY_CC RMAP_BuildReadModifyWriteRegisterPacket` (U8 \*pTargetAddress, unsigned long targetAddressLength, U8 \*pReplyAddress, unsigned long replyAddressLength, U8 key, U16 transactionIdentifier, U32 readModifyWriteAddress, U8 extendedReadModifyWriteAddress, U32 registerValue, U32 mask, unsigned long \*pRawPacketLength, `RMAP_PACKET` \*pPacketStruct, char alignment)

*Build an RMAP read/modify/write command packet which can be sent to perform an RMAP read/modify/write command on a 4 byte register.*

- unsigned long `RMAPPACKETLIBRARY_CC RMAP_CalculateReadModifyWriteReplyPacketLength` (unsigned long initiatorAddressLength, U32 dataLength, char alignment)

*Calculate the number of bytes required for a read/modify/write reply packet, given the properties of the packet.*

- char `RMAPPACKETLIBRARY_CC RMAP_FillReadModifyWriteReplyPacket` (U8 \*pInitiatorAddress, unsigned long initiatorAddressLength, U8 targetAddress, `RMAP_STATUS` status, U16 transactionIdentifier, unsigned long dataLength, U8 \*pData, unsigned long \*pRawPacketLength, `RMAP_PACKET` \*pPacketStruct, char alignment, U8 \*pRawPacket, unsigned long rawPacketLength)

*Fill a previously allocated buffer with an RMAP read/modify/write reply packet which can be sent to respond to an RMAP read/modify/write command.*

- void `*RMAPPACKETLIBRARY_CC RMAP_BuildReadModifyWriteReplyPacket` (U8 \*pInitiatorAddress, unsigned long initiatorAddressLength, U8 targetAddress, `RMAP_STATUS` status, U16 transactionIdentifier, unsigned long dataLength, U8 \*pData, unsigned long \*pRawPacketLength, `RMAP_PACKET` \*pPacketStruct, char alignment)

- Build an RMAP read/modify/write reply packet which can be sent to respond to an RMAP read/modify/write command.*
- void `RMAPPACKETLIBRARY_CC RMAP_FreeBuffer` (void \*pBuffer)  
*Free a previously allocated buffer containing a packet created using one of the `RMAP_Build*` or `RMAP_Fill*` functions.*
  - U8 `RMAPPACKETLIBRARY_CC RMAP_GetProtocolIdentifier` (RMAP\_PACKET \*pPacketStruct)  
*Get the protocol identifier of the packet from a packet structure which has already been created.*
  - void `RMAPPACKETLIBRARY_CC RMAP_SetProtocolIdentifier` (RMAP\_PACKET \*pPacketStruct, U8 protocolIdentifier)  
*Set the protocol identifier of a previously created packet, updating the packet's header CRC so the packet is still valid.*

### 8.20.1 Detailed Description

Declarations of the functions provided by the STAR-Dundee RMAP Packet Library.

#### Author

STAR-Dundee Ltd.  
 STAR House  
 166 Nethergate  
 Dundee, DD1 4EE  
 Scotland, UK  
 e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

This file contains the declarations of the functions provided by the STAR-Dundee RMAP Packet Library, along with constants and types used by the library. The RMAP Packet Library provides functions for building and interpreting RMAP packets.

#### IMPORTANT NOTE:

#### Note

If you are experiencing compilation errors indicating that U8 is already defined, for example, please add the following line to your code prior to including this file:

```
#define NO_STAR_TYPES
```

Alternatively you can compile your code with a flag of `-DNO_STAR_TYPES`.

(Copied from `star-dundee_types.h`)

Copyright © 2013 STAR-Dundee Ltd

## 8.21 rmap\_target\_pxi\_if.h File Reference

Functions used to configure and control the RMAP target capabilities of the STAR-Dundee PXI Interface devices.

```
#include "rmap_target_types.h"
```

#### Functions

- int `STAR_API_CC RMAP_TARGET_PXI_IF_getStatus` (STAR\_DEVICE\_ID deviceId, U32 target, TARGET\_STATUS \*pStatus)  
*Gets the RMAP target status for a specific target.*
- int `STAR_API_CC RMAP_TARGET_PXI_IF_setAddressOffset` (STAR\_DEVICE\_ID deviceId, U32 target, U32 offset)  
*Sets the address offset for a specific target.*
- int `STAR_API_CC RMAP_TARGET_PXI_IF_getAddressOffset` (STAR\_DEVICE\_ID deviceId, U32 target, U32 \*pOffset)  
*Gets the address offset from a specific target.*



- int [STAR\\_API\\_CC RMAP\\_TARGET\\_PXI\\_IF\\_setAuthControlMode](#) (STAR\_DEVICE\_ID deviceld, U32 target, TARGET\_AUTH\_MODE mode)  
*Sets the authorisation control mode for a specific target.*
- int [STAR\\_API\\_CC RMAP\\_TARGET\\_PXI\\_IF\\_getAuthControlMode](#) (STAR\_DEVICE\_ID deviceld, U32 target, TARGET\_AUTH\_MODE \*pMode)  
*Gets the authorisation control mode from a specific target.*
- int [STAR\\_API\\_CC RMAP\\_TARGET\\_PXI\\_IF\\_setAuthLogicalAddressRange](#) (STAR\_DEVICE\_ID deviceld, U32 target, U8 lowest, U8 highest)  
*Sets the authorised target logical address range for a specific target.*
- int [STAR\\_API\\_CC RMAP\\_TARGET\\_PXI\\_IF\\_getAuthLogicalAddressRange](#) (STAR\_DEVICE\_ID deviceld, U32 target, U8 \*pLowest, U8 \*pHighest)  
*Gets the authorised target logical address range from a specific target.*
- int [STAR\\_API\\_CC RMAP\\_TARGET\\_PXI\\_IF\\_setAuthProtocolId](#) (STAR\_DEVICE\_ID deviceld, U32 target, U8 protocolId)  
*Sets the authorised protocol ID for a specific target.*
- int [STAR\\_API\\_CC RMAP\\_TARGET\\_PXI\\_IF\\_getAuthProtocolId](#) (STAR\_DEVICE\_ID deviceld, U32 target, U8 \*pProtocolId)  
*Gets the authorised protocol ID from a specific target.*
- int [STAR\\_API\\_CC RMAP\\_TARGET\\_PXI\\_IF\\_setAuthCommands](#) (STAR\_DEVICE\_ID deviceld, U32 target, TARGET\_AUTH\_CMD\_MASK commands)  
*Sets the authorised RMAP commands for a specific target.*
- int [STAR\\_API\\_CC RMAP\\_TARGET\\_PXI\\_IF\\_getAuthCommands](#) (STAR\_DEVICE\_ID deviceld, U32 target, TARGET\_AUTH\_CMD\_MASK \*pCommands)  
*Gets the authorised RMAP commands from a specific target.*
- int [STAR\\_API\\_CC RMAP\\_TARGET\\_PXI\\_IF\\_setAuthKeyRange](#) (STAR\_DEVICE\_ID deviceld, U32 target, U8 lowest, U8 highest)  
*Sets the authorised key range for a specific target.*
- int [STAR\\_API\\_CC RMAP\\_TARGET\\_PXI\\_IF\\_getAuthKeyRange](#) (STAR\_DEVICE\_ID deviceld, U32 target, U8 \*pLowest, U8 \*pHighest)  
*Gets the authorised key range from a specific target.*
- int [STAR\\_API\\_CC RMAP\\_TARGET\\_PXI\\_IF\\_setAuthMemoryAddressRange](#) (STAR\_DEVICE\_ID deviceld, U32 target, U32 lowerBoundary, U32 upperBoundary)  
*Sets the authorised memory address range for a specific target.*
- int [STAR\\_API\\_CC RMAP\\_TARGET\\_PXI\\_IF\\_getAuthMemoryAddressRange](#) (STAR\_DEVICE\_ID deviceld, U32 target, U32 \*pLowerBoundary, U32 \*pUpperBoundary)  
*Gets the authorised memory address range from a specific target.*
- int [STAR\\_API\\_CC RMAP\\_TARGET\\_PXI\\_IF\\_setInterfaceMode](#) (STAR\_DEVICE\_ID deviceld, U32 port, IF↔\_MODE mode)  
*Sets the RMAP interface mode for a specific port.*
- int [STAR\\_API\\_CC RMAP\\_TARGET\\_PXI\\_IF\\_getInterfaceMode](#) (STAR\_DEVICE\_ID deviceld, U32 port, IF↔\_MODE \*pMode)  
*Gets the RMAP interface mode for a specific port.*
- int [STAR\\_API\\_CC RMAP\\_TARGET\\_PXI\\_IF\\_isAuthRequired](#) (STAR\_DEVICE\_ID deviceld, U32 target)  
*Gets whether or not there is an RMAP command waiting to be authorised.*
- int [STAR\\_API\\_CC RMAP\\_TARGET\\_PXI\\_IF\\_getWaitingCommand](#) (STAR\_DEVICE\_ID deviceld, U32 target, RmapCommandParameters \*pCommand)  
*Gets the parameters of the RMAP command waiting to be authorised.*
- int [STAR\\_API\\_CC RMAP\\_TARGET\\_PXI\\_IF\\_authoriseWaitingCommand](#) (STAR\_DEVICE\_ID deviceld, U32 target)  
*Authorises a waiting RMAP command.*
- int [STAR\\_API\\_CC RMAP\\_TARGET\\_PXI\\_IF\\_rejectWaitingCommand](#) (STAR\_DEVICE\_ID deviceld, U32 target, REJECTION\_REASON reason)  
*Rejects a waiting RMAP command.*

- int `STAR_API_CC RMAP_TARGET_PXI_IF_setEnabledNotifications` (STAR\_DEVICE\_ID deviceId, U32 target, NOTIF\_TYPE\_MASK notifications)  
*Set the notifications that are enabled for a target.*
- int `STAR_API_CC RMAP_TARGET_PXI_IF_getEnabledNotifications` (STAR\_DEVICE\_ID deviceId, U32 target, NOTIF\_TYPE\_MASK \*pNotifications)  
*Get the notifications that are enabled for a target.*
- int `STAR_API_CC RMAP_TARGET_PXI_IF_registerNotificationListener` (STAR\_DEVICE\_ID deviceId, U32 target, NOTIF\_TYPE type, NotifListenerFunc listenerFunc)  
*Register a notification listener function.*
- int `STAR_API_CC RMAP_TARGET_PXI_IF_registerNotificationListenerWithContext` (STAR\_DEVICE\_ID deviceId, U32 target, NOTIF\_TYPE type, NotifListenerWithContextFunc listenerFunc, void \*pContext)  
*Register a notification listener with context function.*
- int `STAR_API_CC RMAP_TARGET_PXI_IF_unregisterNotificationListener` (STAR\_DEVICE\_ID deviceId, U32 target, NOTIF\_TYPE type)  
*Unregister a notification listener function.*
- int `STAR_API_CC RMAP_TARGET_PXI_IF_unregisterNotificationListenerWithContext` (STAR\_DEVICE\_ID deviceId, U32 target, NOTIF\_TYPE type)  
*Unregister a notification listener with context function.*
- int `STAR_API_CC RMAP_TARGET_PXI_IF_startReceivingNotifications` (STAR\_DEVICE\_ID deviceId)  
*Start receiving notifications for a device.*
- int `STAR_API_CC RMAP_TARGET_PXI_IF_stopReceivingNotifications` (STAR\_DEVICE\_ID deviceId)  
*Stop receiving notifications for a device.*
- int `STAR_API_CC RMAP_TARGET_PXI_IF_readMemory` (STAR\_DEVICE\_ID deviceId, U32 target, U32 address, U32 length, unsigned char \*pBuffer)  
*Reads an area of RMAP target memory into a user-supplied buffer.*
- int `STAR_API_CC RMAP_TARGET_PXI_IF_writeMemory` (STAR\_DEVICE\_ID deviceId, U32 target, U32 address, U32 length, const unsigned char \*pBuffer)  
*Writes an area of RMAP target memory from a user-supplied buffer.*

### 8.21.1 Detailed Description

Functions used to configure and control the RMAP target capabilities of the STAR-Dundee PXI Interface devices.

#### Author

STAR-Dundee Ltd.  
 STAR House  
 166 Nethergate  
 Dundee, DD1 4EE  
 Scotland, UK  
 e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2016 STAR-Dundee Ltd.

## 8.22 rmap\_target\_types.h File Reference

Types used with the STAR-Dundee RMAP Target API.

```
#include <star-api.h>
```



## Data Structures

- struct `TARGET_ENGINE`  
*Hardware capabilities. [More...](#)*
- struct `RmapCommandParameters`  
*Parameters of an RMAP command. [More...](#)*

## Typedefs

- typedef `U32 TARGET_AUTH_CMD_MASK`  
*Mask describing which commands are authorised.*
- typedef `U32 NOTIF_TYPE_MASK`  
*Mask describing which notifications are enabled.*
- typedef `void(STAR_API_CC * NotifListenerFunc) (U32 target, void *pNotif)`  
*Notification listener function pointer: target is the index of the target which generated the notification.*
- typedef `void(STAR_API_CC * NotifListenerWithContextFunc) (STAR_DEVICE_ID deviceId, U32 target, void *pNotif, void *pContext)`  
*Notification listener with context function pointer: deviceId is the ID of the device that issued the notification target is the index of the target which generated the notification.*

## Enumerations

- enum `TARGET_AUTH_MODE` { `TARGET_AUTH_MODE_MANUAL` = 0, `TARGET_AUTH_MODE_AUTO←`  
`MATIC` = 1 }
  - enum `TARGET_STATUS` {  
`TARGET_STATUS_SUCCESS` = 0, `TARGET_STATUS_GENERAL_ERROR` = 1, `TARGET_STATUS_H←`  
`EADER_EOP_ERROR` = 2, `TARGET_STATUS_HEADER_EEP_ERROR` = 3,  
`TARGET_STATUS_PID_ERROR` = 4, `TARGET_STATUS_REPLY_ERROR` = 5, `TARGET_STATUS_HE←`  
`ADER_CRC_ERROR` = 6, `TARGET_STATUS_HEADER_EEP_AFTER_CRC` = 7,  
`TARGET_STATUS_PACKET_TYPE_ERROR` = 8, `TARGET_STATUS_COMMAND_TYPE_ERROR` = 9,  
`TARGET_STATUS_RMW_DATALEN_ERROR` = 10, `TARGET_STATUS_CARGO_TOO_LARGE` = 11,  
`TARGET_STATUS_KEY_ERROR` = 12, `TARGET_STATUS_LOGICAL_ADDR_ERROR` = 13, `TARGET←`  
`_STATUS_AUTHORISED_ERROR` = 14, `TARGET_STATUS_VERIFY_BUFFER_OVERRUN` = 15,  
`TARGET_STATUS_DATA_CRC_ERROR` = 16, `TARGET_STATUS_BUS_ERROR` = 17, `TARGET_STA←`  
`TUS_DATA_EOP_ERROR` = 18, `TARGET_STATUS_DATA_EEP_ERROR` = 19,  
`TARGET_STATUS_DATA_EEP_AFTER_CRC` = 20 }
  - enum `TARGET_AUTH_CMD` {  
`TARGET_AUTH_CMD_READ` = (1 << 0), `TARGET_AUTH_CMD_READ_INCR` = (1 << 1), `TARGET←`  
`_AUTH_CMD_READ_MODIFY_WRITE` = (1 << 2), `TARGET_AUTH_CMD_WRITE` = (1 << 3),  
`TARGET_AUTH_CMD_WRITE_INCR` = (1 << 4), `TARGET_AUTH_CMD_WRITE_REPLY` = (1 << 5),  
`TARGET_AUTH_CMD_WRITE_INCR_REPLY` = (1 << 6), `TARGET_AUTH_CMD_WRITE_VERIFY` = (1  
<< 7),  
`TARGET_AUTH_CMD_WRITE_VERIFY_INCR` = (1 << 8), `TARGET_AUTH_CMD_WRITE_VERIFY_R←`  
`EPLY` = (1 << 9), `TARGET_AUTH_CMD_WRITE_VERIFY_INCR_REPLY` = (1 << 10), `TARGET_AUT←`  
`H_CMD_ALL` = (0x7FF),  
`TARGET_AUTH_CMD_NONE` = (0) }
  - enum `REJECTION_REASON` { `REJECTION_REASON_LOGICAL_ADDRESS`, `REJECTION_REASON←`  
`KEY`, `REJECTION_REASON_PROTOCOL_ID`, `REJECTION_REASON_OTHER` }
  - enum `IF_MODE` { `IF_MODE_RMAP_DISABLED` = 0, `IF_MODE_RMAP_ENABLED` = 1 }
- Method of authorising incoming RMAP commands.*
- Status of the RMAP target.*
- Command types which are authorised by the target.*
- Reason for rejecting a waiting RMAP command.*
- Control whether or not the RMAP target is enabled for a port.*

- enum NOTIF\_TYPE { NOTIF\_TYPE\_AUTH\_REQUEST = (1 << 0), NOTIF\_TYPE\_CMD\_COMPLETE = (1 << 2), NOTIF\_TYPE\_ALL = (0x5), NOTIF\_TYPE\_NONE = (0) }

*Types of notifications.*

### 8.22.1 Detailed Description

Types used with the STAR-Dundee RMAP Target API.

#### Author

STAR-Dundee Ltd.  
 STAR House  
 166 Nethergate  
 Dundee, DD1 4EE  
 Scotland, UK  
 e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2016 STAR-Dundee Ltd.

## 8.23 star-api.h File Reference

Declarations of all functions provided by STAR-API.

```
#include "star-dundee_annotations.h"
#include "general.h"
#include "stream_item.h"
#include "transfer.h"
```

### 8.23.1 Detailed Description

Declarations of all functions provided by STAR-API.

#### Author

STAR-Dundee Ltd.  
 STAR House  
 166 Nethergate  
 Dundee, DD1 4EE  
 Scotland, UK  
 e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

This file contains includes to the declarations of all functions provided by STAR-API, the STAR-Dundee software API.

Copyright © 2013 STAR-Dundee Ltd.

## 8.24 star-dundee\_types.h File Reference

Definitions of STAR-Dundee commonly used types.

```
#include <limits.h>
```

## Macros

- `#define U16_MAX (0xffff)`  
*The maximum value that can be represented using an unsigned 16-bit number.*
- `#define TRUE 1`  
*A value that can be used to represent the boolean value of true.*
- `#define FALSE 0`  
*A value that can be used to represent the boolean value of false.*

## Typedefs

- `typedef unsigned char U8`  
*A type that can be used to represent an unsigned 8-bit number.*
- `typedef unsigned short U16`  
*A type that can be used to represent an unsigned 16-bit number.*
- `typedef unsigned int U32`  
*A type that can be used to represent an unsigned 32-bit number.*
- `typedef U32 REGISTER`  
*A type that can be used to represent a 4-byte register.*

### 8.24.1 Detailed Description

Definitions of STAR-Dundee commonly used types.

#### Author

STAR-Dundee Ltd.  
STAR House  
166 Nethergate  
Dundee, DD1 4EE  
Scotland, UK  
e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

This file contains the definitions of common types used by STAR-Dundee software drivers and APIs.

#### IMPORTANT NOTE:

##### Note

If you are experiencing compilation errors indicating that U8 is already defined, for example, please add the following line to your code prior to including this file:

```
#define NO_STAR_TYPES
```

Alternatively you can compile your code with a flag of `-DNO_STAR_TYPES`.

Copyright © 2013 STAR-Dundee Ltd.

The content of this source file is CONFIDENTIAL and PROPRIETARY property of STAR-Dundee Ltd.

## 8.25 stream\_item.h File Reference

Functions create and modify items that can be sent over a SpaceWire network.

```
#include "common.h"
#include "stream_item_types.h"
```

## Functions

- void `STAR_API_CC STAR_destroyStreamItem` (`_In_ _Post_ptr_invalid_ STAR_STREAM_ITEM *item`)  
*Disposes of a given stream item.*
- `_Check_return_ STAR_STREAM_ITEM *STAR_API_CC STAR_createPacket` (`_In_opt_ STAR_SPACEWIRE_ADDRESS *address, _In_opt_count_(dataLen) unsigned char *data, unsigned int dataLen, STAR_EOP_TYPE eopType`)  
*Creates a new SpaceWire packet stream item from user provided data.*
- `_Check_return_ STAR_SPACEWIRE_ADDRESS *STAR_API_CC STAR_createAddress` (`_In_count_(pathLen) unsigned char *path, U16 pathLen`)  
*Creates a SpaceWire address.*
- void `STAR_API_CC STAR_destroyAddress` (`_In_ _Post_ptr_invalid_ STAR_SPACEWIRE_ADDRESS *address`)  
*Disposes of a SpaceWire address created with `STAR_createAddress()`.*
- int `STAR_API_CC STAR_setPacketAddress` (`_Inout_ STAR_SPACEWIRE_PACKET *packet, _In_ STAR_SPACEWIRE_ADDRESS *address`)  
*Updates the address of a given packet.*
- `_Ret_opt_bytecap_x_ dataLength unsigned char *STAR_API_CC STAR_getPacketData` (`_In_ STAR_SPACEWIRE_PACKET *packet, _Out_ unsigned int *dataLength`)  
*Gets the packet's data as an array of bytes.*
- void `STAR_API_CC STAR_destroyPacketData` (`_In_ _Post_ptr_invalid_ unsigned char *pData`)  
*Frees packet data previously created by a call to `STAR_getPacketData()`.*
- `STAR_SPACEWIRE_ADDRESS *STAR_API_CC STAR_getPacketAddress` (`_In_ STAR_SPACEWIRE_PACKET *packet`)  
*Gets a copy of a packet's address structure.*
- unsigned int `STAR_API_CC STAR_getPacketLength` (`_In_ STAR_SPACEWIRE_PACKET *packet`)  
*Gets the length of a given packet.*
- `STAR_EOP_TYPE STAR_API_CC STAR_getPacketEOP` (`_In_ STAR_SPACEWIRE_PACKET *packet`)  
*Gets the end of packet marker type of a packet.*
- int `STAR_API_CC STAR_setPacketEOP` (`_In_ STAR_SPACEWIRE_PACKET *packet, STAR_EOP_TYPE eopType`)  
*Sets the end of packet marker type of a packet.*
- `_Check_return_ STAR_STREAM_ITEM *STAR_API_CC STAR_createDataChunk` (`_In_opt_count_(dataLen) unsigned char *data, unsigned int dataLen, int isStart, STAR_EOP_TYPE eopType`)  
*Creates a data chunk stream item, used to refer to part of a packet.*
- unsigned char `*STAR_API_CC STAR_getChunkData` (`_In_ STAR_DATA_CHUNK *chunk, _Out_ unsigned int *len`)  
*Gets a pointer to a data chunk stream item's data.*
- unsigned int `STAR_API_CC STAR_getChunkDataLength` (`_In_ STAR_DATA_CHUNK *chunk`)  
*Gets the length of a data chunk stream item's data.*
- `STAR_EOP_TYPE STAR_API_CC STAR_getChunkEop` (`_In_ STAR_DATA_CHUNK *chunk`)  
*Gets the End Of Packet marker type of a data chunk stream item.*
- int `STAR_API_CC STAR_getChunkSop` (`_In_ STAR_DATA_CHUNK *chunk`)  
*Gets whether a data chunk stream item is at the start of a packet.*
- `STAR_STREAM_ITEM *STAR_API_CC STAR_createTimeCode` (U8 value)  
*Creates a Time-code stream item.*
- U8 `STAR_API_CC STAR_getTimeCodeValue` (`_In_ STAR_TIMECODE *pTimeCode`)  
*Gets the value of a given time-code stream item.*
- void `STAR_API_CC STAR_setTimeCodeValue` (`_In_ STAR_TIMECODE *pTimeCode, U8 value`)  
*Sets the value of a given time-code stream item.*
- `STAR_STREAM_ITEM *STAR_API_CC STAR_createErrorInData` (`STAR_ERROR_IN_DATA_TYPE errorType`)

- Creates an error in data stream item.*
- `U8 STAR_API_CC STAR_getLinkStateEventCount (_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent)`  
*Gets the count of a given link state event stream item, indicating the number of times the specified event has occurred.*
  - `U8 STAR_API_CC STAR_getLinkStateEventPort (_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent)`  
*Gets the port related to a given link state change event stream item.*
  - `int STAR_API_CC STAR_isLinkStateEventDisconnectError (_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent)`  
*Gets whether a link state event stream item includes a disconnect error.*
  - `int STAR_API_CC STAR_isLinkStateEventEscapeError (_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent)`  
*Gets whether a link state event stream item includes an escape error.*
  - `int STAR_API_CC STAR_isLinkStateEventLinkRunning (_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent)`  
*Gets whether a link state event stream item includes a link running state change.*
  - `int STAR_API_CC STAR_isLinkStateEventParityError (_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent)`  
*Gets whether a link state event stream item includes a parity error.*
  - `int STAR_API_CC STAR_isLinkStateEventReceiveCreditError (_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent)`  
*Gets whether a link state event stream item includes a receive credit error.*
  - `int STAR_API_CC STAR_isLinkStateEventTransmitCreditError (_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent)`  
*Gets whether a link state event stream item includes a transmit credit error.*
  - `U32 STAR_API_CC STAR_getLinkSpeedEventSpeed (_In_ STAR_LINK_SPEED_EVENT *pLinkSpeedEvent)`  
*Gets the link speed of a given link speed change event stream item.*
  - `U8 STAR_API_CC STAR_getLinkSpeedEventPort (_In_ STAR_LINK_SPEED_EVENT *pLinkSpeedEvent)`  
*Gets the port related to a given link speed change event stream item.*
  - `void STAR_API_CC STAR_getTimestampEventStartValue (_In_ STAR_TIMESTAMP_EVENT *pTimestampEvent, U32 syncPulseFrequency, _Out_ U32 *pSeconds, _Out_ U32 *pRemainder)`  
*Gets the start time value from a timestamp event stream item.*
  - `void STAR_API_CC STAR_getTimestampEventEndValue (_In_ STAR_TIMESTAMP_EVENT *pTimestampEvent, U32 syncPulseFrequency, _Out_ U32 *pSeconds, _Out_ U32 *pRemainder)`  
*Gets the end time value from a timestamp event stream item.*
  - `STAR_TIMESTAMP_TYPE STAR_API_CC STAR_getTimestampEventType (_In_ STAR_TIMESTAMP_EVENT *pTimestampEvent)`  
*Gets the timestamp type information from a timestamp event stream item.*
  - `STAR_TIMESTAMP_DIRECTION STAR_API_CC STAR_getTimestampEventDirection (_In_ STAR_TIMESTAMP_EVENT *pTimestampEvent)`  
*Gets the timestamp direction information from a timestamp event stream item.*
  - `void STAR_API_CC STAR_getTimestampEventRawCounters (_In_ STAR_TIMESTAMP_EVENT *pTimestampEvent, _Out_opt_ U32 *pStartSyncPulseCount, _Out_opt_ U32 *pStartClockCycleCount, _Out_opt_ U32 *pStartTotalCycleCount, _Out_opt_ U32 *pEndSyncPulseCount, _Out_opt_ U32 *pEndClockCycleCount, _Out_opt_ U32 *pEndTotalCycleCount, _Out_opt_ U32 *pClockFrequency)`  
*Gets the timestamp counter values as returned by the hardware, from a timestamp event stream item.*
  - `STAR_STREAM_ITEM *STAR_API_CC STAR_createBroadcastMessage (_In_ U8 channel, _In_ U8 type, _In_ U8 status, _In_ U32 dataWord1, _In_ U32 dataWord2)`  
*Creates a broadcast message stream item.*
  - `U8 STAR_API_CC STAR_getBroadcastMessageChannel (_In_ STAR_BROADCAST_MESSAGE *pBroadcastMessage)`  
*Gets the broadcast message channel from a broadcast message stream item.*
  - `U8 STAR_API_CC STAR_getBroadcastMessageType (_In_ STAR_BROADCAST_MESSAGE *pBroadcastMessage)`

*Gets the broadcast message type from a broadcast message stream item.*

- [U8 STAR\\_API\\_CC STAR\\_getBroadcastMessageStatus](#) ([\\_In\\_ STAR\\_BROADCAST\\_MESSAGE](#) \*p↔ BroadcastMessage)

*Gets the broadcast message status from a broadcast message stream item.*

- [U8 STAR\\_API\\_CC STAR\\_getBroadcastMessageLateFlag](#) ([\\_In\\_ STAR\\_BROADCAST\\_MESSAGE](#) \*p↔ BroadcastMessage)

*Gets the broadcast message late flag from a broadcast message stream item.*

- [U8 STAR\\_API\\_CC STAR\\_getBroadcastMessageDelayedFlag](#) ([\\_In\\_ STAR\\_BROADCAST\\_MESSAGE](#) \*p↔ BroadcastMessage)

*Gets the broadcast message delayed flag from a broadcast message stream item.*

- [U32 STAR\\_API\\_CC STAR\\_getBroadcastMessageDataWord1](#) ([\\_In\\_ STAR\\_BROADCAST\\_MESSAGE](#) \*p↔ BroadcastMessage)

*Gets the broadcast message first data word from a broadcast message stream item.*

- [U32 STAR\\_API\\_CC STAR\\_getBroadcastMessageDataWord2](#) ([\\_In\\_ STAR\\_BROADCAST\\_MESSAGE](#) \*p↔ BroadcastMessage)

*Gets the broadcast message second data word from a broadcast message stream item.*

### 8.25.1 Detailed Description

Functions create and modify items that can be sent over a SpaceWire network.

#### Author

STAR-Dundee Ltd.  
 STAR House  
 166 Nethergate  
 Dundee, DD1 4EE  
 Scotland, UK  
 e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2012 STAR-Dundee Ltd.

## 8.26 stream\_item\_types.h File Reference

Structures used to represent stream items such as packets, time-codes and link control tokens.

```
#include "star-dundee_types.h"
```

### Data Structures

- [struct STAR\\_LINK\\_STATE\\_EVENT](#)  
*Events that can occur which affect the state of the SpaceWire link. [More...](#)*
- [struct STAR\\_SPACEWIRE\\_ADDRESS](#)  
*A structure used to describe a SpaceWire address. [More...](#)*
- [struct STAR\\_DATA\\_CHUNK](#)  
*A structure used to describe a chunk of contiguous SpaceWire data, within a single packet. [More...](#)*
- [struct STAR\\_SPACEWIRE\\_PACKET](#)  
*A structure used to describe a SpaceWire packet. [More...](#)*
- [struct STAR\\_TIMECODE](#)  
*A structure used to describe a SpaceWire time-code. [More...](#)*
- [struct STAR\\_LINK\\_SPEED\\_EVENT](#)  
*A structure used to describe a link speed change event. [More...](#)*

- struct `STAR_ERROR_IN_DATA_INJECT`  
A structure used to describe an error in data event. [More...](#)
- struct `STAR_TIMESTAMP_EVENT`  
A structure used to describe a timestamp on receipt of data on the link. [More...](#)
- struct `STAR_BROADCAST_MESSAGE`  
A structure used to describe a SpaceFibre broadcast message. [More...](#)
- struct `STAR_STREAM_ITEM`  
A wrapper for Stream Item objects. [More...](#)

## Enumerations

- enum `STAR_STREAM_ITEM_TYPE` {  
`STAR_STREAM_ITEM_TYPE_SPACEWIRE_PACKET`, `STAR_STREAM_ITEM_TYPE_TIMECODE`, `STAR_STREAM_ITEM_TYPE_LINK_STATE_EVENT`, `STAR_STREAM_ITEM_TYPE_DATA_CHUNK`,  
`STAR_STREAM_ITEM_TYPE_LINK_SPEED_EVENT`, `STAR_STREAM_ITEM_TYPE_ERROR_INJECT`,  
`STAR_STREAM_ITEM_TYPE_TIMESTAMP_EVENT`, `STAR_STREAM_ITEM_TYPE_BROADCAST_MESSAGE` }  
The kinds of object that are represented as stream items.
- enum `STAR_EOP_TYPE` { `STAR_EOP_TYPE_INVALID`, `STAR_EOP_TYPE_EOP`, `STAR_EOP_TYPE_EEP`, `STAR_EOP_TYPE_NONE` }  
The kinds of End of packet marker that may be present.
- enum `STAR_ERROR_IN_DATA_TYPE` { `STAR_ERROR_IN_DATA_PARITY`, `STAR_ERROR_IN_DATA_DISCONNECT` }  
The types of error that may be injected in to data in a packet.
- enum `STAR_TIMESTAMP_TYPE` {  
`STAR_TIMESTAMP_TYPE_DATA`, `STAR_TIMESTAMP_TYPE_LINK_STATE`, `STAR_TIMESTAMP_TYPE_LINK_SPEED`, `STAR_TIMESTAMP_TYPE_TIMECODE`,  
`STAR_TIMESTAMP_TYPE_ERROR` }  
An enum used to define different types of timestamps that can be received.
- enum `STAR_TIMESTAMP_DIRECTION` { `STAR_TIMESTAMP_DIRECTION_TRANSMIT`, `STAR_TIMESTAMP_DIRECTION_RECEIVE` }  
An enum used to define the direction of a timestamp.
- enum `STAR_BROADCAST_MESSAGE_STATUS_FLAG` { `STAR_BROADCAST_MESSAGE_STATUS_FLAG_LATE` = 0x1, `STAR_BROADCAST_MESSAGE_STATUS_FLAG_DELAYED` = 0x2 }  
An enum used to define the status flags of a broadcast message.

### 8.26.1 Detailed Description

Structures used to represent stream items such as packets, time-codes and link control tokens.

#### Author

STAR-Dundee Ltd.  
 STAR House  
 166 Nethergate  
 Dundee, DD1 4EE  
 Scotland, UK  
 e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

The types in this file are used to represent the traffic and link control tokens that may be sent on a SpaceWire link.  
 Copyright © 2012 STAR-Dundee Ltd.

## 8.27 transfer.h File Reference

Declarations of functions provided by STAR-API relating to transfer operations.

```
#include "types.h"
#include "transfer_types.h"
#include "stream_item_types.h"
#include "common.h"
```

### Functions

- [STAR\\_TRANSFER\\_STATUS STAR\\_API\\_CC STAR\\_receivePacket](#) ([\\_In\\_ STAR\\_CHANNEL\\_ID](#) channelId, [\\_Out\\_bytecap\\_](#)(\*pPacketLength) void \*pPacketData, [\\_Inout\\_ unsigned int](#) \*pPacketLength, [\\_Out\\_ STAR\\_EOP\\_TYPE](#) \*pEopType, [\\_In\\_ int](#) timeout)  
*Receive a single packet on a previously opened channel in to the buffer specified.*
- [STAR\\_TRANSFER\\_STATUS STAR\\_API\\_CC STAR\\_transmitPacket](#) ([\\_In\\_ STAR\\_CHANNEL\\_ID](#) channelId, [\\_In\\_opt\\_bytecount\\_](#)(packetLength) void \*pPacketData, [\\_In\\_ unsigned int](#) packetLength, [\\_In\\_ STAR\\_EOP\\_TYPE](#) eopType, [\\_In\\_ int](#) timeout)  
*Transmit a single packet on a previously opened channel from the buffer specified.*
- [\\_Check\\_return\\_ STAR\\_TRANSFER\\_OPERATION \\*STAR\\_API\\_CC STAR\\_createTxOperation](#) ([\\_In\\_count\\_](#)(streamItemCount) [STAR\\_STREAM\\_ITEM](#) \*\*ppltems, unsigned int streamItemCount)  
*Creates a transmit operation that can be submitted later.*
- [\\_Check\\_return\\_ STAR\\_TRANSFER\\_OPERATION \\*STAR\\_API\\_CC STAR\\_createRxOperation](#) (int itemCount, [STAR\\_RECEIVE\\_MASK](#) mask)  
*Creates a receive operation that can then be submitted.*
- [\\_Check\\_return\\_ STAR\\_TRANSFER\\_OPERATION \\*STAR\\_API\\_CC STAR\\_createRxOperationEx](#) ([\\_In\\_count\\_](#)(numBuffers) [U8](#) \*\*ppBuffers, [\\_In\\_ unsigned int](#) numBuffers, [\\_In\\_ unsigned int](#) bufferSize, [\\_In\\_ STAR\\_RECEIVE\\_MASK](#) mask)  
*This function is an alternative to STAR\_createRxOperation that creates receive operations to be submitted to a channel opened with the STAR\_openChannelToLocalDeviceEx function.*
- [int STAR\\_API\\_CC STAR\\_submitTransferOperation](#) ([\\_In\\_ STAR\\_CHANNEL\\_ID](#) channelId, [\\_In\\_ STAR\\_TRANSFER\\_OPERATION](#) \*transferOp)  
*Submits a single transfer operation.*
- [int STAR\\_API\\_CC STAR\\_submitTransferOperationList](#) ([STAR\\_CHANNEL\\_ID](#) channelId, [\\_In\\_count\\_](#)(count) [STAR\\_TRANSFER\\_OPERATION](#) \*\*transferOps, unsigned int count)  
*Submits a list of transfer operations.*
- [STAR\\_TRANSFER\\_STATUS STAR\\_API\\_CC STAR\\_waitOnTransferOperationCompletion](#) ([\\_In\\_ STAR\\_TRANSFER\\_OPERATION](#) \*pTransferOperation, int timeout)  
*Blocks until a given transfer operation has completed, or the timeout period expires.*
- [STAR\\_TRANSFER\\_STATUS STAR\\_API\\_CC STAR\\_getTransferOperationStatus](#) ([\\_In\\_ STAR\\_TRANSFER\\_OPERATION](#) \*pTransferOperation)  
*Gets the status of a transfer operation.*
- [void STAR\\_API\\_CC STAR\\_cancelTransferOperationWaits](#) ([\\_In\\_ STAR\\_TRANSFER\\_OPERATION](#) \*pTransferOperation)  
*Cancels any waits in progress on a transfer operation.*
- [unsigned int STAR\\_API\\_CC STAR\\_getTransferItemCount](#) ([\\_In\\_ STAR\\_TRANSFER\\_OPERATION](#) \*pReceiveOperation)  
*Gets the current count of stream items received by the operation and available to be obtained using STAR\_getTransferItem().*
- [STAR\\_STREAM\\_ITEM \\*STAR\\_API\\_CC STAR\\_getTransferItem](#) ([\\_In\\_ STAR\\_TRANSFER\\_OPERATION](#) \*pReceiveOperation, unsigned int index)  
*Gets the stream item at a given index of a receive operation.*
- [STAR\\_STREAM\\_ITEM \\*STAR\\_API\\_CC STAR\\_getNextTransferItem](#) ([STAR\\_STREAM\\_ITEM](#) \*pStreamItem)



- Gets the next stream item in the list of received stream items.*
- `_Ret_opt_cap_count STAR_STREAM_ITEM **STAR_API_CC STAR_getTransferItemList (_In_ STAR_TRANSFER_OPERATION *pReceiveOperation, _Out_ unsigned int *count)`
- Gets the entire list of transfer items associated with a receive.*
- `void STAR_API_CC STAR_destroyTransferItemList (_In_ _Post_ptr_invalid_ STAR_STREAM_ITEM **transferItemList, unsigned int transferItemListCount, int destroyItems)`
- Destroys a given stream item array, returned by a call to `STAR_getTransferItemList()`, optionally destroying each of the elements in the array.*
- `int STAR_API_CC STAR_cancelTransferOperation (_In_ STAR_TRANSFER_OPERATION *pTransferOperation)`
- Cancels a given transfer operation.*
- `int STAR_API_CC STAR_disposeTransferOperation (_In_ _Post_ptr_invalid_ STAR_TRANSFER_OPERATION *pTransferOperation)`
- Cancels a transfer operation (if it is not already complete) and indicates that the memory associated with the operation should be freed when the operation is no longer in use.*
- `int STAR_API_CC STAR_registerTransferCompletionListener (_In_ STAR_TRANSFER_OPERATION *pTransferOperation, STAR_TransferOperationListenerFunc listenerFunc, _In_opt_ void *pContextInfo)`
- Registers a call-back function that will be called when an operation completes.*
- `int STAR_API_CC STAR_unregisterTransferCompletionListener (_In_ STAR_TRANSFER_OPERATION *pTransferOperation, STAR_TransferOperationListenerFunc listenerFunc)`
- Unregisters a call-back function that will be called when an operation completes.*
- `U8 *STAR_API_CC STAR_getSpaceWireAddressPath (STAR_SPACEWIRE_ADDRESS *pSpaceWireAddress)`
- Access function to get a SpaceWire address path.*
- `U16 STAR_API_CC STAR_getSpaceWireAddressPathLength (STAR_SPACEWIRE_ADDRESS *pSpaceWireAddress)`
- Access function to get a SpaceWire address path length.*
- `int STAR_API_CC STAR_rxOperationExGetNumItems (_In_ STAR_TRANSFER_OPERATION *pTransferOp)`
- Gets the number of items received by an ex receive operation.*
- `STAR_STREAM_ITEM_TYPE STAR_API_CC STAR_rxOperationExGetItemType (_In_ STAR_TRANSFER_OPERATION *pTransferOp, _In_ unsigned int item)`
- Gets the stream item type for an item received by an ex receive operation.*
- `int STAR_API_CC STAR_rxOperationExGetDataChunk (_In_ STAR_TRANSFER_OPERATION *pTransferOp, _In_ unsigned int item, _Out_ STAR_DATA_CHUNK *pDataChunk)`
- Gets a data chunk from an item received by an ex receive operation.*
- `int STAR_API_CC STAR_rxOperationExGetBroadcastMessage (_In_ STAR_TRANSFER_OPERATION *pTransferOp, _In_ unsigned int item, _Out_ STAR_BROADCAST_MESSAGE *pBroadcastMessage)`
- Gets a broadcast message from an item received by an ex receive operation.*
- `STAR_TRANSFER_STATUS STAR_API_CC STAR_rxOperationExGetStatus (_In_ STAR_TRANSFER_OPERATION *pTransferOp)`
- Gets the status of an ex receive operation.*

### 8.27.1 Detailed Description

Declarations of functions provided by STAR-API relating to transfer operations.

#### Author

STAR-Dundee Ltd.  
 STAR House  
 166 Nethergate  
 Dundee, DD1 4EE  
 Scotland, UK  
 e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2012 STAR-Dundee Ltd.

## 8.28 transfer\_types.h File Reference

Types used to describe transmit and receive operations.

```
#include "types.h"
```

### Typedefs

- typedef void [STAR\\_TRANSFER\\_OPERATION](#)  
*A structure used to identify a transfer operation.*
- typedef void([STAR\\_API\\_CC](#) \* [STAR\\_TransferOperationListenerFunc](#)) ([STAR\\_TRANSFER\\_OPERATION](#) N \*pOperation, [STAR\\_TRANSFER\\_STATUS](#) status, void \*pContextInfo)  
*The function type for transfer operation listener functions which are called when a transfer operation completes, is cancelled, or encounters an error.*

### Enumerations

- enum [STAR\\_TRANSFER\\_STATUS](#) {  
[STAR\\_TRANSFER\\_STATUS\\_NOT\\_STARTED](#), [STAR\\_TRANSFER\\_STATUS\\_STARTED](#), [STAR\\_TRANSFER\\_STATUS\\_COMPLETE](#), [STAR\\_TRANSFER\\_STATUS\\_CANCELLED](#),  
[STAR\\_TRANSFER\\_STATUS\\_ERROR](#) }  
*Possible states a transfer operation can be in.*
- enum [STAR\\_RECEIVE\\_MASK](#) {  
[STAR\\_RECEIVE\\_PACKETS](#) = 1U << 0, [STAR\\_RECEIVE\\_CHUNKS](#) = 1U << 1, [STAR\\_RECEIVE\\_TIMECODES](#) = 1U << 2, [STAR\\_RECEIVE\\_FCTS](#) = 1U << 3,  
[STAR\\_RECEIVE\\_NULL](#) = 1U << 4, [STAR\\_RECEIVE\\_LINK\\_STATE\\_EVENTS](#) = 1U << 5, [STAR\\_RECEIVE\\_LINK\\_SPEED\\_EVENTS](#) = 1U << 6, [STAR\\_RECEIVE\\_TIMESTAMP\\_EVENTS](#) = 1U << 7,  
[STAR\\_RECEIVE\\_BROADCAST\\_MESSAGES](#) = 1U << 8 }  
*Flags used to determine what kind of stream items to receive on a receive operation.*

### 8.28.1 Detailed Description

Types used to describe transmit and receive operations.

#### Author

STAR-Dundee Ltd.  
 STAR House  
 166 Nethergate  
 Dundee, DD1 4EE  
 Scotland, UK  
 e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2012 STAR-Dundee Ltd.

## 8.29 triggering\_brick\_mk3.h File Reference

Functions used to configure and control the triggering capabilities of the STAR-Dundee Brick Mk3 devices.

```
#include <star-api.h>
#include "triggering_types.h"
```

## Functions

- [int STAR\\_API\\_CC TRIGGER\\_BRICK\\_MK3\\_setExtTriggerInputEvents](#) (STAR\_DEVICE\_ID deviceId, U32 extTrigger, U32 trigger, EXT\_TRIGGER\_EVENT\_MASK events)  
*Sets the input events for an external trigger which will cause an internal trigger to be set.*
- [int STAR\\_API\\_CC TRIGGER\\_BRICK\\_MK3\\_setCounterInputEvents](#) (STAR\_DEVICE\_ID deviceId, U32 counter, U32 trigger, COUNTER\_EVENT\_MASK events)  
*Sets the input events for a counter which will cause an internal trigger to be set.*
- [int STAR\\_API\\_CC TRIGGER\\_BRICK\\_MK3\\_setPortInputEvents](#) (STAR\_DEVICE\_ID deviceId, U32 port, U32 trigger, PORT\_EVENT\_MASK events)  
*Sets the input events for a port which will cause an internal trigger to be set.*
- [int STAR\\_API\\_CC TRIGGER\\_BRICK\\_MK3\\_setTimeCodeInputEvents](#) (STAR\_DEVICE\_ID deviceId, U32 timeCode, U32 trigger, TIME\_CODE\_EVENT\_MASK events)  
*Sets the input events for a time-code engine which will cause an internal trigger to be set.*
- [int STAR\\_API\\_CC TRIGGER\\_BRICK\\_MK3\\_setTriggerInputEvents](#) (STAR\_DEVICE\_ID deviceId, U32 causeTrigger, U32 trigger, TRIGGER\_EVENT\_MASK events)  
*Sets the input events for an internal trigger which will cause another internal trigger to be set.*
- [int STAR\\_API\\_CC TRIGGER\\_BRICK\\_MK3\\_getExtTriggerInputEvents](#) (STAR\_DEVICE\_ID deviceId, U32 extTrigger, U32 trigger, EXT\_TRIGGER\_EVENT\_MASK \*pEvents)  
*Gets the input events from an external trigger which will cause an internal trigger to be set.*
- [int STAR\\_API\\_CC TRIGGER\\_BRICK\\_MK3\\_getCounterInputEvents](#) (STAR\_DEVICE\_ID deviceId, U32 counter, U32 trigger, COUNTER\_EVENT\_MASK \*pEvents)  
*Gets the input events from a counter which will cause an internal trigger to be set.*
- [int STAR\\_API\\_CC TRIGGER\\_BRICK\\_MK3\\_getPortInputEvents](#) (STAR\_DEVICE\_ID deviceId, U32 port, U32 trigger, PORT\_EVENT\_MASK \*pEvents)  
*Gets the input events from a port which will cause an internal trigger to be set.*
- [int STAR\\_API\\_CC TRIGGER\\_BRICK\\_MK3\\_getTimeCodeInputEvents](#) (STAR\_DEVICE\_ID deviceId, U32 timeCode, U32 trigger, TIME\_CODE\_EVENT\_MASK \*pEvents)  
*Gets the input events from a time-code engine which will cause an internal trigger to be set.*
- [int STAR\\_API\\_CC TRIGGER\\_BRICK\\_MK3\\_getTriggerInputEvents](#) (STAR\_DEVICE\_ID deviceId, U32 causeTrigger, U32 trigger, TRIGGER\_EVENT\_MASK \*pEvents)  
*Gets the input events for an internal trigger which will cause another internal trigger to be set.*
- [int STAR\\_API\\_CC TRIGGER\\_BRICK\\_MK3\\_setExtTriggerOutputActions](#) (STAR\_DEVICE\_ID deviceId, U32 extTrigger, U32 trigger, EXT\_TRIGGER\_ACTION\_MASK actions)  
*Sets the output actions for an external trigger that are caused by an internal trigger being set.*
- [int STAR\\_API\\_CC TRIGGER\\_BRICK\\_MK3\\_setCounterOutputActions](#) (STAR\_DEVICE\_ID deviceId, U32 counter, U32 trigger, COUNTER\_ACTION\_MASK actions)  
*Sets the output actions for a counter that are caused by an internal trigger being set.*
- [int STAR\\_API\\_CC TRIGGER\\_BRICK\\_MK3\\_setPortOutputActions](#) (STAR\_DEVICE\_ID deviceId, U32 port, U32 trigger, PORT\_ACTION\_MASK actions)  
*Sets the output actions for a port that are caused by an internal trigger being set.*
- [int STAR\\_API\\_CC TRIGGER\\_BRICK\\_MK3\\_setTimeCodeOutputActions](#) (STAR\_DEVICE\_ID deviceId, U32 timeCode, U32 trigger, TIME\_CODE\_ACTION\_MASK actions)  
*Sets the output actions for a time-code engine that are caused by an internal trigger being set.*
- [int STAR\\_API\\_CC TRIGGER\\_BRICK\\_MK3\\_getExtTriggerOutputActions](#) (STAR\_DEVICE\_ID deviceId, U32 extTrigger, U32 trigger, EXT\_TRIGGER\_ACTION\_MASK \*pActions)  
*Gets the output actions from an external trigger that are caused by an internal trigger being set.*
- [int STAR\\_API\\_CC TRIGGER\\_BRICK\\_MK3\\_getCounterOutputActions](#) (STAR\_DEVICE\_ID deviceId, U32 counter, U32 trigger, COUNTER\_ACTION\_MASK \*pActions)  
*Gets the output actions from a counter that are caused by an internal trigger being set.*
- [int STAR\\_API\\_CC TRIGGER\\_BRICK\\_MK3\\_getPortOutputActions](#) (STAR\_DEVICE\_ID deviceId, U32 port, U32 trigger, PORT\_ACTION\_MASK \*pActions)  
*Gets the output actions from a port that are caused by an internal trigger being set.*

- `int STAR_API_CC TRIGGER_BRICK_MK3_getTimeCodeOutputActions (STAR_DEVICE_ID deviceId, U32 timeCode, U32 trigger, TIME_CODE_ACTION_MASK *pActions)`  
*Gets the output actions from a time-code engine that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_enableExtTriggerEdgeDetectMode (STAR_DEVICE_ID deviceId, U32 extTrigger)`  
*Enables edge detect mode for a specific external trigger.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_disableExtTriggerEdgeDetectMode (STAR_DEVICE_ID deviceId, U32 extTrigger)`  
*Disables edge detect mode for a specific external trigger.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_getExtTriggerEdgeDetectModeEnabled (STAR_DEVICE_ID deviceId, U32 extTrigger, U32 *pEnabled)`  
*Gets whether edge detect mode is enabled for a specific external trigger.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_enableExtTriggerInvert (STAR_DEVICE_ID deviceId, U32 extTrigger)`  
*Enables invert for a specific external trigger.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_disableExtTriggerInvert (STAR_DEVICE_ID deviceId, U32 extTrigger)`  
*Disables invert for a specific external trigger.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_getExtTriggerInvertEnabled (STAR_DEVICE_ID deviceId, U32 extTrigger, U32 *pEnabled)`  
*Gets whether invert is enabled for a specific external trigger.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_enableExtTriggerOutput (STAR_DEVICE_ID deviceId, U32 extTrigger)`  
*Enables output for a specific external trigger.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_disableExtTriggerOutput (STAR_DEVICE_ID deviceId, U32 extTrigger)`  
*Disables output for a specific external trigger.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_getExtTriggerOutputEnabled (STAR_DEVICE_ID deviceId, U32 extTrigger, U32 *pEnabled)`  
*Gets whether output is enabled for a specific external trigger.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_setExtTriggerExtend (STAR_DEVICE_ID deviceId, U32 extTrigger, U32 extend)`  
*Sets the extend duration for a specific external trigger.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_getExtTriggerExtend (STAR_DEVICE_ID deviceId, U32 extTrigger, U32 *pExtend)`  
*Gets the extend duration for a specific external trigger.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_enableCounterAutoReload (STAR_DEVICE_ID deviceId, U32 counter)`  
*Enables auto reload for a specific counter.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_disableCounterAutoReload (STAR_DEVICE_ID deviceId, U32 counter)`  
*Disables auto reload for a specific counter.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_getCounterAutoReloadEnabled (STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)`  
*Gets whether auto reload is enabled for a specific counter.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_enableCounterTriggerCount (STAR_DEVICE_ID deviceId, U32 counter)`  
*Enables trigger count for a specific counter.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_disableCounterTriggerCount (STAR_DEVICE_ID deviceId, U32 counter)`  
*Disables trigger count for a specific counter.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_getCounterTriggerCountEnabled (STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)`

- Gets whether trigger count is enabled for a specific counter.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_enableCounterStartMode` (STAR\_DEVICE\_ID deviceId, U32 counter)
- Enables start mode for a specific counter.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_disableCounterStartMode` (STAR\_DEVICE\_ID deviceId, U32 counter)
- Disables start mode for a specific counter.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_getCounterStartModeEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)
- Gets whether start mode is enabled for a specific counter.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_enableCounterStartStopMode` (STAR\_DEVICE\_ID deviceId, U32 counter)
- Enables start stop mode for a specific counter.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_disableCounterStartStopMode` (STAR\_DEVICE\_ID deviceId, U32 counter)
- Disables start stop mode for a specific counter.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_getCounterStartStopModeEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)
- Gets whether start stop mode is enabled for a specific counter.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_enableCounterLoadZero` (STAR\_DEVICE\_ID deviceId, U32 counter)
- Enables load zero for a specific counter.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_disableCounterLoadZero` (STAR\_DEVICE\_ID deviceId, U32 counter)
- Disables load zero for a specific counter.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_getCounterLoadZeroEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)
- Gets whether load zero is enabled for a specific counter.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_setCounterReloadValue` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 reloadValue)
- Sets the reload value for a specific counter.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_getCounterReloadValue` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pReloadValue)
- Gets the reload value for a specific counter.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_forceCounterReload` (STAR\_DEVICE\_ID deviceId, U32 counter)
- Force the reload of a specific counter.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_getCounterValue` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pValue)
- Gets the counter value for a specific counter.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_enablePortTransmitPacketMode` (STAR\_DEVICE\_ID deviceId, U32 port)
- Enables packet transmit mode for a specific port.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_disablePortTransmitPacketMode` (STAR\_DEVICE\_ID deviceId, U32 port)
- Disables packet transmit mode for a specific port.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_getPortTransmitPacketModeEnabled` (STAR\_DEVICE\_ID deviceId, U32 port, U32 \*pEnabled)
- Gets whether packet transmit mode is enabled for a specific port.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_setTriggerInputMode` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_INPUT\_MODE mode)
- Sets the input mode for a specific internal trigger.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_getTriggerInputMode` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_INPUT\_MODE \*pMode)

- Gets the input mode for a specific internal trigger.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_enableTrigger` (STAR\_DEVICE\_ID deviceId, U32 trigger)  
*Enables a specific internal trigger.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_disableTrigger` (STAR\_DEVICE\_ID deviceId, U32 trigger)  
*Disables a specific internal trigger.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_getTriggerEnabled` (STAR\_DEVICE\_ID deviceId, U32 trigger, U32 \*pEnabled)  
*Gets whether a trigger is enabled.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_forceTrigger` (STAR\_DEVICE\_ID deviceId, U32 trigger)  
*Force the triggering of a specific internal trigger.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_setTriggerInvertMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 mask)  
*Sets the invert mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_getTriggerInvertMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 \*pMask)  
*Gets the invert mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_setTriggerAndMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 mask)  
*Sets the AND/OR mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_getTriggerAndMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 \*pMask)  
*Gets the AND/OR mask for a trigger type for a specific internal trigger.*

### 8.29.1 Detailed Description

Functions used to configure and control the triggering capabilities of the STAR-Dundee Brick Mk3 devices.

The Brick Mk3 devices have the following triggering resources available: 2 external triggers (indexed from 0-1), 4 internal counters (indexed from 0-3), 2 SpaceWire ports (indexed from 1-2), 1 time-code interface (indexed as 0) and 8 internal triggers (indexed from 0-7).

#### Author

STAR-Dundee Ltd.  
STAR House  
166 Nethergate  
Dundee, DD1 4EE  
Scotland, UK  
e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2016 STAR-Dundee Ltd.

## 8.30 triggering\_pcie.h File Reference

Functions used to configure and control the triggering capabilities of the STAR-Dundee PCIe Interface devices.

```
#include <star-api.h>
#include "triggering_types.h"
```

#### Functions

- int `STAR_API_CC TRIGGER_PCIE_IF_setCounterInputEvents` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 trigger, COUNTER\_EVENT\_MASK events)

*Sets the input events for a counter which will cause an internal trigger to be set.*

- int `STAR_API_CC TRIGGER_PCIE_IF_setPortInputEvents` (STAR\_DEVICE\_ID deviceId, U32 port, U32 trigger, PORT\_EVENT\_MASK events)

*Sets the input events for a port which will cause an internal trigger to be set.*

- int `STAR_API_CC TRIGGER_PCIE_IF_setTriggerInputEvents` (STAR\_DEVICE\_ID deviceId, U32 cause↔Trigger, U32 trigger, TRIGGER\_EVENT\_MASK events)

*Sets the input events for an internal trigger which will cause another internal trigger to be set.*

- int `STAR_API_CC TRIGGER_PCIE_IF_getCounterInputEvents` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 trigger, COUNTER\_EVENT\_MASK \*pEvents)

*Gets the input events from a counter which will cause an internal trigger to be set.*

- int `STAR_API_CC TRIGGER_PCIE_IF_getPortInputEvents` (STAR\_DEVICE\_ID deviceId, U32 port, U32 trigger, PORT\_EVENT\_MASK \*pEvents)

*Gets the input events from a port which will cause an internal trigger to be set.*

- int `STAR_API_CC TRIGGER_PCIE_IF_getTriggerInputEvents` (STAR\_DEVICE\_ID deviceId, U32 cause↔Trigger, U32 trigger, TRIGGER\_EVENT\_MASK \*pEvents)

*Gets the input events for an internal trigger which will cause another internal trigger to be set.*

- int `STAR_API_CC TRIGGER_PCIE_IF_setCounterOutputActions` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 trigger, COUNTER\_ACTION\_MASK actions)

*Sets the output actions for a counter that are caused by an internal trigger being set.*

- int `STAR_API_CC TRIGGER_PCIE_IF_setPortOutputActions` (STAR\_DEVICE\_ID deviceId, U32 port, U32 trigger, PORT\_ACTION\_MASK actions)

*Sets the output actions for a port that are caused by an internal trigger being set.*

- int `STAR_API_CC TRIGGER_PCIE_IF_getCounterOutputActions` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 trigger, COUNTER\_ACTION\_MASK \*pActions)

*Gets the output actions from a counter that are caused by an internal trigger being set.*

- int `STAR_API_CC TRIGGER_PCIE_IF_getPortOutputActions` (STAR\_DEVICE\_ID deviceId, U32 port, U32 trigger, PORT\_ACTION\_MASK \*pActions)

*Gets the output actions from a port that are caused by an internal trigger being set.*

- int `STAR_API_CC TRIGGER_PCIE_IF_enableCounterAutoReload` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Enables auto reload for a specific counter.*

- int `STAR_API_CC TRIGGER_PCIE_IF_disableCounterAutoReload` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Disables auto reload for a specific counter.*

- int `STAR_API_CC TRIGGER_PCIE_IF_getCounterAutoReloadEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)

*Gets whether auto reload is enabled for a specific counter.*

- int `STAR_API_CC TRIGGER_PCIE_IF_enableCounterTriggerCount` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Enables trigger count for a specific counter.*

- int `STAR_API_CC TRIGGER_PCIE_IF_disableCounterTriggerCount` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Disables trigger count for a specific counter.*

- int `STAR_API_CC TRIGGER_PCIE_IF_getCounterTriggerCountEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)

*Gets whether trigger count is enabled for a specific counter.*

- int `STAR_API_CC TRIGGER_PCIE_IF_enableCounterStartMode` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Enables start mode for a specific counter.*

- int `STAR_API_CC TRIGGER_PCIE_IF_disableCounterStartMode` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Disables start mode for a specific counter.*



- int `STAR_API_CC TRIGGER_PCIE_IF_getCounterStartModeEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)  
*Gets whether start mode is enabled for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_IF_enableCounterStartStopMode` (STAR\_DEVICE\_ID deviceId, U32 counter)  
*Enables start stop mode for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_IF_disableCounterStartStopMode` (STAR\_DEVICE\_ID deviceId, U32 counter)  
*Disables start stop mode for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_IF_getCounterStartStopModeEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)  
*Gets whether start stop mode is enabled for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_IF_enableCounterLoadZero` (STAR\_DEVICE\_ID deviceId, U32 counter)  
*Enables load zero for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_IF_disableCounterLoadZero` (STAR\_DEVICE\_ID deviceId, U32 counter)  
*Disables load zero for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_IF_getCounterLoadZeroEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)  
*Gets whether load zero is enabled for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_IF_setCounterReloadValue` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 reloadValue)  
*Sets the reload value for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_IF_getCounterReloadValue` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pReloadValue)  
*Gets the reload value for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_IF_forceCounterReload` (STAR\_DEVICE\_ID deviceId, U32 counter)  
*Force the reload of a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_IF_getCounterValue` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pValue)  
*Gets the counter value for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_IF_setTriggerInputMode` (STAR\_DEVICE\_ID deviceId, U32 trigger, T↔RIGGER\_INPUT\_MODE mode)  
*Sets the input mode for a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PCIE_IF_getTriggerInputMode` (STAR\_DEVICE\_ID deviceId, U32 trigger, T↔RIGGER\_INPUT\_MODE \*pMode)  
*Gets the input mode for a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PCIE_IF_enableTrigger` (STAR\_DEVICE\_ID deviceId, U32 trigger)  
*Enables a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PCIE_IF_disableTrigger` (STAR\_DEVICE\_ID deviceId, U32 trigger)  
*Disables a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PCIE_IF_getTriggerEnabled` (STAR\_DEVICE\_ID deviceId, U32 trigger, U32 \*pEnabled)  
*Gets whether a trigger is enabled.*
- int `STAR_API_CC TRIGGER_PCIE_IF_forceTrigger` (STAR\_DEVICE\_ID deviceId, U32 trigger)  
*Force the triggering of a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PCIE_IF_setTriggerInvertMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, T↔RIGGER\_TYPE type, U32 mask)  
*Sets the invert mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PCIE_IF_getTriggerInvertMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, T↔RIGGER\_TYPE type, U32 \*pMask)  
*Gets the invert mask for a trigger type for a specific internal trigger.*



- int `STAR_API_CC TRIGGER_PCIE_IF_setTriggerAndMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 mask)  
*Sets the AND/OR mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PCIE_IF_getTriggerAndMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 \*pMask)  
*Gets the AND/OR mask for a trigger type for a specific internal trigger.*

### 8.30.1 Detailed Description

Functions used to configure and control the triggering capabilities of the STAR-Dundee PCIe Interface devices.

The PCIe Interface devices have the following triggering resources available: 6 internal counters (indexed from 0-5), 3 SpaceWire ports (indexed from 1-3) and 6 internal triggers (indexed from 0-5).

#### Author

STAR-Dundee Ltd.  
STAR House  
166 Nethergate  
Dundee, DD1 4EE  
Scotland, UK  
e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2016 STAR-Dundee Ltd.

## 8.31 triggering\_pxi\_if.h File Reference

Functions used to configure and control the triggering capabilities of the STAR-Dundee PXI Interface devices.

```
#include <star-api.h>
#include "triggering_types.h"
```

### Functions

- int `STAR_API_CC TRIGGER_PXI_IF_setExtTriggerInputEvents` (STAR\_DEVICE\_ID deviceId, U32 extTrigger, U32 trigger, EXT\_TRIGGER\_EVENT\_MASK events)  
*Sets the input events for an external trigger which will cause an internal trigger to be set.*
- int `STAR_API_CC TRIGGER_PXI_IF_setCounterInputEvents` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 trigger, COUNTER\_EVENT\_MASK events)  
*Sets the input events for a counter which will cause an internal trigger to be set.*
- int `STAR_API_CC TRIGGER_PXI_IF_setPortInputEvents` (STAR\_DEVICE\_ID deviceId, U32 port, U32 trigger, PORT\_EVENT\_MASK events)  
*Sets the input events for a port which will cause an internal trigger to be set.*
- int `STAR_API_CC TRIGGER_PXI_IF_setTimeCodeInputEvents` (STAR\_DEVICE\_ID deviceId, U32 timeCode, U32 trigger, TIME\_CODE\_EVENT\_MASK events)  
*Sets the input events for a time-code engine which will cause an internal trigger to be set.*
- int `STAR_API_CC TRIGGER_PXI_IF_setTriggerInputEvents` (STAR\_DEVICE\_ID deviceId, U32 causeTrigger, U32 trigger, TRIGGER\_EVENT\_MASK events)  
*Sets the input events for an internal trigger which will cause another internal trigger to be set.*
- int `STAR_API_CC TRIGGER_PXI_IF_getExtTriggerInputEvents` (STAR\_DEVICE\_ID deviceId, U32 extTrigger, U32 trigger, EXT\_TRIGGER\_EVENT\_MASK \*pEvents)  
*Gets the input events from an external trigger which will cause an internal trigger to be set.*
- int `STAR_API_CC TRIGGER_PXI_IF_getCounterInputEvents` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 trigger, COUNTER\_EVENT\_MASK \*pEvents)

*Gets the input events from a counter which will cause an internal trigger to be set.*

- int [STAR\\_API\\_CC\\_TRIGGER\\_PXI\\_IF\\_getPortInputEvents](#) (STAR\_DEVICE\_ID deviceId, U32 port, U32 trigger, PORT\_EVENT\_MASK \*pEvents)

*Gets the input events from a port which will cause an internal trigger to be set.*

- int [STAR\\_API\\_CC\\_TRIGGER\\_PXI\\_IF\\_getTimeCodeInputEvents](#) (STAR\_DEVICE\_ID deviceId, U32 timeCode, U32 trigger, TIME\_CODE\_EVENT\_MASK \*pEvents)

*Gets the input events from a time-code engine which will cause an internal trigger to be set.*

- int [STAR\\_API\\_CC\\_TRIGGER\\_PXI\\_IF\\_getTriggerInputEvents](#) (STAR\_DEVICE\_ID deviceId, U32 causeTrigger, U32 trigger, TRIGGER\_EVENT\_MASK \*pEvents)

*Gets the input events for an internal trigger which will cause another internal trigger to be set.*

- int [STAR\\_API\\_CC\\_TRIGGER\\_PXI\\_IF\\_setExtTriggerOutputActions](#) (STAR\_DEVICE\_ID deviceId, U32 extTrigger, U32 trigger, EXT\_TRIGGER\_ACTION\_MASK actions)

*Sets the output actions for an external trigger that are caused by an internal trigger being set.*

- int [STAR\\_API\\_CC\\_TRIGGER\\_PXI\\_IF\\_setCounterOutputActions](#) (STAR\_DEVICE\_ID deviceId, U32 counter, U32 trigger, COUNTER\_ACTION\_MASK actions)

*Sets the output actions for a counter that are caused by an internal trigger being set.*

- int [STAR\\_API\\_CC\\_TRIGGER\\_PXI\\_IF\\_setPortOutputActions](#) (STAR\_DEVICE\_ID deviceId, U32 port, U32 trigger, PORT\_ACTION\_MASK actions)

*Sets the output actions for a port that are caused by an internal trigger being set.*

- int [STAR\\_API\\_CC\\_TRIGGER\\_PXI\\_IF\\_setTimeCodeOutputActions](#) (STAR\_DEVICE\_ID deviceId, U32 timeCode, U32 trigger, TIME\_CODE\_ACTION\_MASK actions)

*Sets the output actions for a time-code engine that are caused by an internal trigger being set.*

- int [STAR\\_API\\_CC\\_TRIGGER\\_PXI\\_IF\\_getExtTriggerOutputActions](#) (STAR\_DEVICE\_ID deviceId, U32 extTrigger, U32 trigger, EXT\_TRIGGER\_ACTION\_MASK \*pActions)

*Gets the output actions from an external trigger that are caused by an internal trigger being set.*

- int [STAR\\_API\\_CC\\_TRIGGER\\_PXI\\_IF\\_getCounterOutputActions](#) (STAR\_DEVICE\_ID deviceId, U32 counter, U32 trigger, COUNTER\_ACTION\_MASK \*pActions)

*Gets the output actions from a counter that are caused by an internal trigger being set.*

- int [STAR\\_API\\_CC\\_TRIGGER\\_PXI\\_IF\\_getPortOutputActions](#) (STAR\_DEVICE\_ID deviceId, U32 port, U32 trigger, PORT\_ACTION\_MASK \*pActions)

*Gets the output actions from a port that are caused by an internal trigger being set.*

- int [STAR\\_API\\_CC\\_TRIGGER\\_PXI\\_IF\\_getTimeCodeOutputActions](#) (STAR\_DEVICE\_ID deviceId, U32 timeCode, U32 trigger, TIME\_CODE\_ACTION\_MASK \*pActions)

*Gets the output actions from a time-code engine that are caused by an internal trigger being set.*

- int [STAR\\_API\\_CC\\_TRIGGER\\_PXI\\_IF\\_enableExtTriggerEdgeDetectMode](#) (STAR\_DEVICE\_ID deviceId, U32 extTrigger)

*Enables edge detect mode for a specific external trigger.*

- int [STAR\\_API\\_CC\\_TRIGGER\\_PXI\\_IF\\_disableExtTriggerEdgeDetectMode](#) (STAR\_DEVICE\_ID deviceId, U32 extTrigger)

*Disables edge detect mode for a specific external trigger.*

- int [STAR\\_API\\_CC\\_TRIGGER\\_PXI\\_IF\\_getExtTriggerEdgeDetectModeEnabled](#) (STAR\_DEVICE\_ID deviceId, U32 extTrigger, U32 \*pEnabled)

*Gets whether edge detect mode is enabled for a specific external trigger.*

- int [STAR\\_API\\_CC\\_TRIGGER\\_PXI\\_IF\\_enableExtTriggerInvert](#) (STAR\_DEVICE\_ID deviceId, U32 extTrigger)

*Enables invert for a specific external trigger.*

- int [STAR\\_API\\_CC\\_TRIGGER\\_PXI\\_IF\\_disableExtTriggerInvert](#) (STAR\_DEVICE\_ID deviceId, U32 extTrigger)

*Disables invert for a specific external trigger.*

- int [STAR\\_API\\_CC\\_TRIGGER\\_PXI\\_IF\\_getExtTriggerInvertEnabled](#) (STAR\_DEVICE\_ID deviceId, U32 extTrigger, U32 \*pEnabled)

*Gets whether invert is enabled for a specific external trigger.*

- int [STAR\\_API\\_CC\\_TRIGGER\\_PXI\\_IF\\_enableExtTriggerOutput](#) (STAR\_DEVICE\_ID deviceId, U32 extTrigger)

*Enables output for a specific external trigger.*

- int [STAR\\_API\\_CC TRIGGER\\_PXI\\_IF\\_disableExtTriggerOutput](#) (STAR\_DEVICE\_ID deviceId, U32 extTrigger)
 

*Disables output for a specific external trigger.*
- int [STAR\\_API\\_CC TRIGGER\\_PXI\\_IF\\_getExtTriggerOutputEnabled](#) (STAR\_DEVICE\_ID deviceId, U32 extTrigger, U32 \*pEnabled)
 

*Gets whether output is enabled for a specific external trigger.*
- int [STAR\\_API\\_CC TRIGGER\\_PXI\\_IF\\_setExtTriggerExtend](#) (STAR\_DEVICE\_ID deviceId, U32 extTrigger, U32 extend)
 

*Sets the extend duration for a specific external trigger.*
- int [STAR\\_API\\_CC TRIGGER\\_PXI\\_IF\\_getExtTriggerExtend](#) (STAR\_DEVICE\_ID deviceId, U32 extTrigger, U32 \*pExtend)
 

*Gets the extend duration for a specific external trigger.*
- int [STAR\\_API\\_CC TRIGGER\\_PXI\\_IF\\_enableCounterAutoReload](#) (STAR\_DEVICE\_ID deviceId, U32 counter)
 

*Enables auto reload for a specific counter.*
- int [STAR\\_API\\_CC TRIGGER\\_PXI\\_IF\\_disableCounterAutoReload](#) (STAR\_DEVICE\_ID deviceId, U32 counter)
 

*Disables auto reload for a specific counter.*
- int [STAR\\_API\\_CC TRIGGER\\_PXI\\_IF\\_getCounterAutoReloadEnabled](#) (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)
 

*Gets whether auto reload is enabled for a specific counter.*
- int [STAR\\_API\\_CC TRIGGER\\_PXI\\_IF\\_enableCounterTriggerCount](#) (STAR\_DEVICE\_ID deviceId, U32 counter)
 

*Enables trigger count for a specific counter.*
- int [STAR\\_API\\_CC TRIGGER\\_PXI\\_IF\\_disableCounterTriggerCount](#) (STAR\_DEVICE\_ID deviceId, U32 counter)
 

*Disables trigger count for a specific counter.*
- int [STAR\\_API\\_CC TRIGGER\\_PXI\\_IF\\_getCounterTriggerCountEnabled](#) (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)
 

*Gets whether trigger count is enabled for a specific counter.*
- int [STAR\\_API\\_CC TRIGGER\\_PXI\\_IF\\_enableCounterStartMode](#) (STAR\_DEVICE\_ID deviceId, U32 counter)
 

*Enables start mode for a specific counter.*
- int [STAR\\_API\\_CC TRIGGER\\_PXI\\_IF\\_disableCounterStartMode](#) (STAR\_DEVICE\_ID deviceId, U32 counter)
 

*Disables start mode for a specific counter.*
- int [STAR\\_API\\_CC TRIGGER\\_PXI\\_IF\\_getCounterStartModeEnabled](#) (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)
 

*Gets whether start mode is enabled for a specific counter.*
- int [STAR\\_API\\_CC TRIGGER\\_PXI\\_IF\\_enableCounterStartStopMode](#) (STAR\_DEVICE\_ID deviceId, U32 counter)
 

*Enables start stop mode for a specific counter.*
- int [STAR\\_API\\_CC TRIGGER\\_PXI\\_IF\\_disableCounterStartStopMode](#) (STAR\_DEVICE\_ID deviceId, U32 counter)
 

*Disables start stop mode for a specific counter.*
- int [STAR\\_API\\_CC TRIGGER\\_PXI\\_IF\\_getCounterStartStopModeEnabled](#) (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)
 

*Gets whether start stop mode is enabled for a specific counter.*
- int [STAR\\_API\\_CC TRIGGER\\_PXI\\_IF\\_enableCounterLoadZero](#) (STAR\_DEVICE\_ID deviceId, U32 counter)
 

*Enables load zero for a specific counter.*
- int [STAR\\_API\\_CC TRIGGER\\_PXI\\_IF\\_disableCounterLoadZero](#) (STAR\_DEVICE\_ID deviceId, U32 counter)
 

*Disables load zero for a specific counter.*
- int [STAR\\_API\\_CC TRIGGER\\_PXI\\_IF\\_getCounterLoadZeroEnabled](#) (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)
 

*Gets whether load zero is enabled for a specific counter.*

- `int STAR_API_CC TRIGGER_PXI_IF_setCounterReloadValue (STAR_DEVICE_ID deviceId, U32 counter, U32 reloadValue)`  
*Sets the reload value for a specific counter.*
- `int STAR_API_CC TRIGGER_PXI_IF_getCounterReloadValue (STAR_DEVICE_ID deviceId, U32 counter, U32 *pReloadValue)`  
*Gets the reload value for a specific counter.*
- `int STAR_API_CC TRIGGER_PXI_IF_forceCounterReload (STAR_DEVICE_ID deviceId, U32 counter)`  
*Force the reload of a specific counter.*
- `int STAR_API_CC TRIGGER_PXI_IF_getCounterValue (STAR_DEVICE_ID deviceId, U32 counter, U32 *pValue)`  
*Gets the counter value for a specific counter.*
- `int STAR_API_CC TRIGGER_PXI_IF_enablePortTransmitPacketMode (STAR_DEVICE_ID deviceId, U32 port)`  
*Enables packet transmit mode for a specific port.*
- `int STAR_API_CC TRIGGER_PXI_IF_disablePortTransmitPacketMode (STAR_DEVICE_ID deviceId, U32 port)`  
*Disables packet transmit mode for a specific port.*
- `int STAR_API_CC TRIGGER_PXI_IF_getPortTransmitPacketModeEnabled (STAR_DEVICE_ID deviceId, U32 port, U32 *pEnabled)`  
*Gets whether packet transmit mode is enabled for a specific port.*
- `int STAR_API_CC TRIGGER_PXI_IF_setTriggerInputMode (STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_INPUT_MODE mode)`  
*Sets the input mode for a specific internal trigger.*
- `int STAR_API_CC TRIGGER_PXI_IF_getTriggerInputMode (STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_INPUT_MODE *pMode)`  
*Gets the input mode for a specific internal trigger.*
- `int STAR_API_CC TRIGGER_PXI_IF_enableTrigger (STAR_DEVICE_ID deviceId, U32 trigger)`  
*Enables a specific internal trigger.*
- `int STAR_API_CC TRIGGER_PXI_IF_disableTrigger (STAR_DEVICE_ID deviceId, U32 trigger)`  
*Disables a specific internal trigger.*
- `int STAR_API_CC TRIGGER_PXI_IF_getTriggerEnabled (STAR_DEVICE_ID deviceId, U32 trigger, U32 *pEnabled)`  
*Gets whether a trigger is enabled.*
- `int STAR_API_CC TRIGGER_PXI_IF_forceTrigger (STAR_DEVICE_ID deviceId, U32 trigger)`  
*Force the triggering of a specific internal trigger.*
- `int STAR_API_CC TRIGGER_PXI_IF_setTriggerInvertMask (STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_TYPE type, U32 mask)`  
*Sets the invert mask for a trigger type for a specific internal trigger.*
- `int STAR_API_CC TRIGGER_PXI_IF_getTriggerInvertMask (STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_TYPE type, U32 *pMask)`  
*Gets the invert mask for a trigger type for a specific internal trigger.*
- `int STAR_API_CC TRIGGER_PXI_IF_setTriggerAndMask (STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_TYPE type, U32 mask)`  
*Sets the AND/OR mask for a trigger type for a specific internal trigger.*
- `int STAR_API_CC TRIGGER_PXI_IF_getTriggerAndMask (STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_TYPE type, U32 *pMask)`  
*Gets the AND/OR mask for a trigger type for a specific internal trigger.*

### 8.31.1 Detailed Description

Functions used to configure and control the triggering capabilities of the STAR-Dundee PXI Interface devices.

The PXI Interface devices have the following triggering resources available: 4 external triggers (indexed from 0-3), 4 internal counters (indexed from 0-3), 4 SpaceWire ports (indexed from 1-4), 1 time-code interface (indexed as 0) and 8 internal triggers (indexed from 0-7).

#### Author

STAR-Dundee Ltd.  
 STAR House  
 166 Nethergate  
 Dundee, DD1 4EE  
 Scotland, UK  
 e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2016 STAR-Dundee Ltd.

## 8.32 triggering\_pxi\_ro.h File Reference

Functions used to configure and control the triggering capabilities of the STAR-Dundee PXI Router devices.

```
#include <star-api.h>
#include "triggering_types.h"
```

### Functions

- int [STAR\\_API\\_CC TRIGGER\\_PXI\\_ROUTER\\_setCounterInputEvents](#) (STAR\_DEVICE\_ID deviceId, U32 counter, U32 trigger, COUNTER\_EVENT\_MASK events)  
*Sets the input events for a counter which will cause an internal trigger to be set.*
- int [STAR\\_API\\_CC TRIGGER\\_PXI\\_ROUTER\\_setPortInputEvents](#) (STAR\_DEVICE\_ID deviceId, U32 port, U32 trigger, PORT\_EVENT\_MASK events)  
*Sets the input events for a port which will cause an internal trigger to be set.*
- int [STAR\\_API\\_CC TRIGGER\\_PXI\\_ROUTER\\_setTimeCodeInputEvents](#) (STAR\_DEVICE\_ID deviceId, U32 timeCode, U32 trigger, TIME\_CODE\_EVENT\_MASK events)  
*Sets the input events for a time-code engine which will cause an internal trigger to be set.*
- int [STAR\\_API\\_CC TRIGGER\\_PXI\\_ROUTER\\_setTriggerInputEvents](#) (STAR\_DEVICE\_ID deviceId, U32 causeTrigger, U32 trigger, TRIGGER\_EVENT\_MASK events)  
*Sets the input events for an internal trigger which will cause another internal trigger to be set.*
- int [STAR\\_API\\_CC TRIGGER\\_PXI\\_ROUTER\\_getCounterInputEvents](#) (STAR\_DEVICE\_ID deviceId, U32 counter, U32 trigger, COUNTER\_EVENT\_MASK \*pEvents)  
*Gets the input events from a counter which will cause an internal trigger to be set.*
- int [STAR\\_API\\_CC TRIGGER\\_PXI\\_ROUTER\\_getPortInputEvents](#) (STAR\_DEVICE\_ID deviceId, U32 port, U32 trigger, PORT\_EVENT\_MASK \*pEvents)  
*Gets the input events from a port which will cause an internal trigger to be set.*
- int [STAR\\_API\\_CC TRIGGER\\_PXI\\_ROUTER\\_getTimeCodeInputEvents](#) (STAR\_DEVICE\_ID deviceId, U32 timeCode, U32 trigger, TIME\_CODE\_EVENT\_MASK \*pEvents)  
*Gets the input events from a time-code engine which will cause an internal trigger to be set.*
- int [STAR\\_API\\_CC TRIGGER\\_PXI\\_ROUTER\\_getTriggerInputEvents](#) (STAR\_DEVICE\_ID deviceId, U32 causeTrigger, U32 trigger, TRIGGER\_EVENT\_MASK \*pEvents)  
*Gets the input events for an internal trigger which will cause another internal trigger to be set.*
- int [STAR\\_API\\_CC TRIGGER\\_PXI\\_ROUTER\\_setCounterOutputActions](#) (STAR\_DEVICE\_ID deviceId, U32 counter, U32 trigger, COUNTER\_ACTION\_MASK actions)

*Sets the output actions for a counter that are caused by an internal trigger being set.*

- int `STAR_API_CC TRIGGER_PXI_ROUTER_setPortOutputActions` (STAR\_DEVICE\_ID deviceId, U32 port, U32 trigger, PORT\_ACTION\_MASK actions)

*Sets the output actions for a port that are caused by an internal trigger being set.*

- int `STAR_API_CC TRIGGER_PXI_ROUTER_setTimeCodeOutputActions` (STAR\_DEVICE\_ID deviceId, U32 timeCode, U32 trigger, TIME\_CODE\_ACTION\_MASK actions)

*Sets the output actions for a time-code engine that are caused by an internal trigger being set.*

- int `STAR_API_CC TRIGGER_PXI_ROUTER_getCounterOutputActions` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 trigger, COUNTER\_ACTION\_MASK \*pActions)

*Gets the output actions from a counter that are caused by an internal trigger being set.*

- int `STAR_API_CC TRIGGER_PXI_ROUTER_getPortOutputActions` (STAR\_DEVICE\_ID deviceId, U32 port, U32 trigger, PORT\_ACTION\_MASK \*pActions)

*Gets the output actions from a port that are caused by an internal trigger being set.*

- int `STAR_API_CC TRIGGER_PXI_ROUTER_getTimeCodeOutputActions` (STAR\_DEVICE\_ID deviceId, U32 timeCode, U32 trigger, TIME\_CODE\_ACTION\_MASK \*pActions)

*Gets the output actions from a time-code engine that are caused by an internal trigger being set.*

- int `STAR_API_CC TRIGGER_PXI_ROUTER_enableCounterAutoReload` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Enables auto reload for a specific counter.*

- int `STAR_API_CC TRIGGER_PXI_ROUTER_disableCounterAutoReload` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Disables auto reload for a specific counter.*

- int `STAR_API_CC TRIGGER_PXI_ROUTER_getCounterAutoReloadEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)

*Gets whether auto reload is enabled for a specific counter.*

- int `STAR_API_CC TRIGGER_PXI_ROUTER_enableCounterTriggerCount` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Enables trigger count for a specific counter.*

- int `STAR_API_CC TRIGGER_PXI_ROUTER_disableCounterTriggerCount` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Disables trigger count for a specific counter.*

- int `STAR_API_CC TRIGGER_PXI_ROUTER_getCounterTriggerCountEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)

*Gets whether trigger count is enabled for a specific counter.*

- int `STAR_API_CC TRIGGER_PXI_ROUTER_enableCounterStartMode` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Enables start mode for a specific counter.*

- int `STAR_API_CC TRIGGER_PXI_ROUTER_disableCounterStartMode` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Disables start mode for a specific counter.*

- int `STAR_API_CC TRIGGER_PXI_ROUTER_getCounterStartModeEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)

*Gets whether start mode is enabled for a specific counter.*

- int `STAR_API_CC TRIGGER_PXI_ROUTER_enableCounterStartStopMode` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Enables start stop mode for a specific counter.*

- int `STAR_API_CC TRIGGER_PXI_ROUTER_disableCounterStartStopMode` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Disables start stop mode for a specific counter.*

- int `STAR_API_CC TRIGGER_PXI_ROUTER_getCounterStartStopModeEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)

*Gets whether start stop mode is enabled for a specific counter.*



- int `STAR_API_CC TRIGGER_PXI_ROUTER_enableCounterLoadZero` (STAR\_DEVICE\_ID deviceId, U32 counter)  
*Enables load zero for a specific counter.*
- int `STAR_API_CC TRIGGER_PXI_ROUTER_disableCounterLoadZero` (STAR\_DEVICE\_ID deviceId, U32 counter)  
*Disables load zero for a specific counter.*
- int `STAR_API_CC TRIGGER_PXI_ROUTER_getCounterLoadZeroEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)  
*Gets whether load zero is enabled for a specific counter.*
- int `STAR_API_CC TRIGGER_PXI_ROUTER_setCounterReloadValue` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 reloadValue)  
*Sets the reload value for a specific counter.*
- int `STAR_API_CC TRIGGER_PXI_ROUTER_getCounterReloadValue` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pReloadValue)  
*Gets the reload value for a specific counter.*
- int `STAR_API_CC TRIGGER_PXI_ROUTER_forceCounterReload` (STAR\_DEVICE\_ID deviceId, U32 counter)  
*Force the reload of a specific counter.*
- int `STAR_API_CC TRIGGER_PXI_ROUTER_getCounterValue` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pValue)  
*Gets the counter value for a specific counter.*
- int `STAR_API_CC TRIGGER_PXI_ROUTER_enablePortTransmitPacketMode` (STAR\_DEVICE\_ID deviceId, U32 port)  
*Enables packet transmit mode for a specific port.*
- int `STAR_API_CC TRIGGER_PXI_ROUTER_disablePortTransmitPacketMode` (STAR\_DEVICE\_ID deviceId, U32 port)  
*Disables packet transmit mode for a specific port.*
- int `STAR_API_CC TRIGGER_PXI_ROUTER_getPortTransmitPacketModeEnabled` (STAR\_DEVICE\_ID deviceId, U32 port, U32 \*pEnabled)  
*Gets whether packet transmit mode is enabled for a specific port.*
- int `STAR_API_CC TRIGGER_PXI_ROUTER_setTriggerInputMode` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_INPUT\_MODE mode)  
*Sets the input mode for a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PXI_ROUTER_getTriggerInputMode` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_INPUT\_MODE \*pMode)  
*Gets the input mode for a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PXI_ROUTER_enableTrigger` (STAR\_DEVICE\_ID deviceId, U32 trigger)  
*Enables a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PXI_ROUTER_disableTrigger` (STAR\_DEVICE\_ID deviceId, U32 trigger)  
*Disables a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PXI_ROUTER_getTriggerEnabled` (STAR\_DEVICE\_ID deviceId, U32 trigger, U32 \*pEnabled)  
*Gets whether a trigger is enabled.*
- int `STAR_API_CC TRIGGER_PXI_ROUTER_forceTrigger` (STAR\_DEVICE\_ID deviceId, U32 trigger)  
*Force the triggering of a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PXI_ROUTER_setTriggerInvertMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 mask)  
*Sets the invert mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PXI_ROUTER_getTriggerInvertMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 \*pMask)  
*Gets the invert mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PXI_ROUTER_setTriggerAndMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 mask)

*Sets the AND/OR mask for a trigger type for a specific internal trigger.*

- int [STAR\\_API\\_CC\\_TRIGGER\\_PXI\\_ROUTER\\_getTriggerAndMask](#) (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 \*pMask)

*Gets the AND/OR mask for a trigger type for a specific internal trigger.*

### 8.32.1 Detailed Description

Functions used to configure and control the triggering capabilities of the STAR-Dundee PXI Router devices.

#### Author

STAR-Dundee Ltd.  
 STAR House  
 166 Nethergate  
 Dundee, DD1 4EE  
 Scotland, UK  
 e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2016 STAR-Dundee Ltd.

## 8.33 triggering\_types.h File Reference

Types used with the STAR-Dundee Triggering API.

```
#include <star-api.h>
```

### Data Structures

- struct [TRIGGER\\_MATRIX](#)  
*Hardware capabilities. [More...](#)*

### Typedefs

- typedef U32 [EXT\\_TRIGGER\\_EVENT\\_MASK](#)  
*External trigger event mask.*
- typedef U32 [EXT\\_TRIGGER\\_ACTION\\_MASK](#)  
*External trigger action mask.*
- typedef U32 [COUNTER\\_EVENT\\_MASK](#)  
*Counter event mask.*
- typedef U32 [COUNTER\\_ACTION\\_MASK](#)  
*Counter action mask.*
- typedef U32 [PORT\\_EVENT\\_MASK](#)  
*Port event mask.*
- typedef U32 [PORT\\_ACTION\\_MASK](#)  
*Port action mask.*
- typedef U32 [TIME\\_CODE\\_EVENT\\_MASK](#)  
*Time-code event mask.*
- typedef U32 [TIME\\_CODE\\_ACTION\\_MASK](#)  
*Time-code action mask.*
- typedef U32 [TRIGGER\\_EVENT\\_MASK](#)  
*Trigger event mask.*



## Enumerations

- enum `EXT_TRIGGER_EVENT` { `EXT_TRIGGER_EVENT_IN` = 1 << 0, `EXT_TRIGGER_EVENT_NONE` = 0, `EXT_TRIGGER_EVENT_ALL` = 0x1 }

*External trigger events.*

- enum `EXT_TRIGGER_ACTION` { `EXT_TRIGGER_ACTION_OUT` = 1 << 0, `EXT_TRIGGER_ACTION_NONE` = 0, `EXT_TRIGGER_ACTION_ALL` = 0x1 }

*External trigger actions.*

- enum `COUNTER_EVENT` { `COUNTER_EVENT_COUNT` = 1 << 0, `COUNTER_EVENT_ZERO_SINGLE` = 1 << 1, `COUNTER_EVENT_ZERO` = 1 << 2, `COUNTER_EVENT_RELOAD` = 1 << 3, `COUNTER_EVENT_NONE` = 0, `COUNTER_EVENT_ALL` = 0xf }

*Counter events.*

- enum `COUNTER_ACTION` { `COUNTER_ACTION_COUNT` = 1 << 0, `COUNTER_ACTION_RELOAD` = 1 << 1, `COUNTER_ACTION_START` = 1 << 2, `COUNTER_ACTION_STOP` = 1 << 3, `COUNTER_ACTION_NONE` = 0, `COUNTER_ACTION_ALL` = 0xf }

*Counter actions.*

- enum `PORT_EVENT` { `PORT_EVENT_RX_SOP` = 1 << 0, `PORT_EVENT_RX_EOP` = 1 << 1, `PORT_EVENT_RX_EEP` = 1 << 2, `PORT_EVENT_TX_SOP` = 1 << 3, `PORT_EVENT_TX_EOP` = 1 << 4, `PORT_EVENT_TX_PKT_PENDING` = 1 << 5, `PORT_EVENT_RUNNING` = 1 << 6, `PORT_EVENT_PARITY_ERROR` = 1 << 7, `PORT_EVENT_ESCAPE_ERROR` = 1 << 8, `PORT_EVENT_CREDIT_ERROR` = 1 << 9, `PORT_EVENT_DISCONNECT` = 1 << 10, `PORT_EVENT_RX_TIME_CODE` = 1 << 11, `PORT_EVENT_TX_TIME_CODE` = 1 << 12, `PORT_EVENT_NONE` = 0, `PORT_EVENT_ALL` = 0x1fff, `PORT_EVENT_ALL_PCIE` = 0xffff }

*Port events.*

- enum `PORT_ACTION` { `PORT_ACTION_TRANSMIT_PKT` = 1 << 0, `PORT_ACTION_DISCONNECT` = 1 << 1, `PORT_ACTION_PARITY_ERROR` = 1 << 2, `PORT_ACTION_ESCAPE_ERROR` = 1 << 3, `PORT_ACTION_INSERT_FCT` = 1 << 4, `PORT_ACTION_SUPPRESS_FCT` = 1 << 5, `PORT_ACTION_INCR_CREDIT` = 1 << 6, `PORT_ACTION_DECR_CREDIT` = 1 << 7, `PORT_ACTION_STOP_RECEPTION` = 1 << 8, `PORT_ACTION_DISABLE_LVDS` = 1 << 9, `PORT_ACTION_NONE` = 0, `PORT_ACTION_ALL` = 0x3ff, `PORT_ACTION_ALL_PCIE` = 0x1ff }

*Port actions.*

- enum `TIME_CODE_EVENT` { `TIME_CODE_EVENT_TICK` = 1 << 0, `TIME_CODE_EVENT_NONE` = 0, `TIME_CODE_EVENT_ALL` = 0x1 }

*Time-code events.*

- enum `TIME_CODE_ACTION` { `TIME_CODE_ACTION_TX` = 1 << 0, `TIME_CODE_ACTION_NONE` = 0, `TIME_CODE_ACTION_ALL` = 0x1 }

*Time-code actions.*

- enum `TRIGGER_EVENT` { `TRIGGER_EVENT_IN` = 1 << 0, `TRIGGER_EVENT_NONE` = 0, `TRIGGER_EVENT_ALL` = 0x1 }

*Internal trigger events.*

- enum `TRIGGER_INPUT_MODE` { `TRIGGER_INPUT_MODE_OR` = 0, `TRIGGER_INPUT_MODE_AND` = 1 }

*Internal trigger input modes.*

- enum `TRIGGER_TYPE` { `TRIGGER_TYPE_EXT_TRIGGER` = 0, `TRIGGER_TYPE_COUNTER` = 1, `TRIGGER_TYPE_PORT` = 2, `TRIGGER_TYPE_TIME_CODE` = 3, `TRIGGER_TYPE_TRIGGER` = 4 }

*Trigger types.*

- enum `TRIGGER_DEVICE` {  
`TRIGGER_DEVICE_INVALID` = 0, `TRIGGER_DEVICE_BRICK_MK3` = 1, `TRIGGER_DEVICE_PXI_IF` = 2,  
`TRIGGER_DEVICE_PXI_ROUTER` = 3,  
`TRIGGER_DEVICE_PCIE_IF` = 4 }

*Trigger devices.*

### 8.33.1 Detailed Description

Types used with the STAR-Dundee Triggering API.

#### Author

STAR-Dundee Ltd.  
 STAR House  
 166 Nethergate  
 Dundee, DD1 4EE  
 Scotland, UK  
 e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2016 STAR-Dundee Ltd.

### 8.33.2 Data Structure Documentation

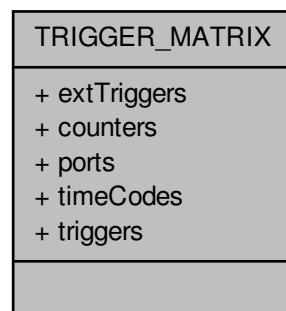
#### 8.33.2.1 struct `TRIGGER_MATRIX`

Hardware capabilities.

#### Added in version:

STAR-System v3.0 - 26th August 2016

Collaboration diagram for `TRIGGER_MATRIX`:



## Data Fields

|     |             |  |
|-----|-------------|--|
| U32 | counters    |  |
| U32 | extTriggers |  |
| U32 | ports       |  |
| U32 | timeCodes   |  |
| U32 | triggers    |  |

## 8.33.3 Enumeration Type Documentation

## 8.33.3.1 enum TRIGGER\_DEVICE

Trigger devices.

Added in version:

STAR-System v3.0 - 26th August 2016

## 8.33.3.2 enum TRIGGER\_TYPE

Trigger types.

Added in version:

STAR-System v3.0 - 26th August 2016

## 8.34 types.h File Reference

Types, structures and function types used by STAR-API.

```
#include "common.h"
#include "star-dundee_types.h"
```

## Macros

- `#define CFG_INFO_ID_TYPE U32`
- `#define STAR_DEVICE_TYPE U32`  
*Type of device that a device is.*
- `#define STAR_DEVICE_SERIAL_SIZE 16`  
*Size in bytes (including null terminator) of a device serial number.*
- `#define STAR_CHANNEL_MASK U32`  
*Type used to represent channels that exist on a device.*
- `#define STAR_DEVICE_UNKNOWN 0U`  
*The "unknown" device type.*
- `#define STAR_DEVICE_PCI 1U`  
*The STAR-Dundee SpaceWire PCI-2 and cPCI device type.*
- `#define STAR_DEVICE_ROUTER_USB 2U`  
*The STAR-Dundee SpaceWire Router-USB device type.*
- `#define STAR_DEVICE_USB_BRICK 3U`  
*The STAR-Dundee SpaceWire-USB Brick device type.*
- `#define STAR_DEVICE_LINK_ANALYSER 4U`  
*The STAR-Dundee SpaceWire Link Analyser device type.*

- `#define STAR_DEVICE_CONFORMANCE_TESTER 5U`  
*The STAR-Dundee SpaceWire Conformance Tester device type.*
- `#define STAR_DEVICE_IP_TUNNEL 6U`  
*The STAR-Dundee SpaceWire IP-Tunnel device type.*
- `#define STAR_DEVICE_ROUTER_MK2S 7U`  
*The STAR-Dundee SpaceWire Router Mk2S device type.*
- `#define STAR_DEVICE_PCI_MK2 8U`  
*The STAR-Dundee SpaceWire PCI Mk2 device type.*
- `#define STAR_DEVICE_BRICK_MK2 9U`  
*The STAR-Dundee SpaceWire Brick Mk2 device type.*
- `#define STAR_DEVICE_LINK_ANALYSER_MK2 10U`  
*The STAR-Dundee SpaceWire Link Analyser Mk2 device type.*
- `#define STAR_DEVICE_PCIE 11U`  
*The STAR-Dundee SpaceWire PCI Express device type.*
- `#define STAR_DEVICE_RTC 12U`  
*The STAR-Dundee SpaceWire RTC device type.*
- `#define STAR_DEVICE_EGSE 13U`  
*The STAR-Dundee SpaceWire EGSE device type.*
- `#define STAR_DEVICE_CPCI_MK2 14U`  
*The STAR-Dundee SpaceWire cPCI Mk2 device type.*
- `#define STAR_DEVICE_SPLT 15U`  
*The STAR-Dundee SpaceWire Physical Layer Tester device type.*
- `#define STAR_DEVICE_STAR_FIRE 16U`  
*The STAR-Dundee STAR Fire device type.*
- `#define STAR_DEVICE_WBS_II 17U`  
*The STAR-Dundee Wide Band Spectrometer II device type.*
- `#define STAR_DEVICE_RECORDER 18U`  
*The STAR-Dundee SpaceWire Recorder device type.*
- `#define STAR_DEVICE_BRICK_MK3 19U`  
*The STAR-Dundee SpaceWire Brick Mk3 device type.*
- `#define STAR_DEVICE_TRAFFIC_GENERATOR 20U`  
*The Shimafuji Traffic Generator device type.*
- `#define STAR_DEVICE_PXI_INTERFACE 21U`  
*The STAR-Dundee SpaceWire PXI Interface device type.*
- `#define STAR_DEVICE_PXI_RMAP 22U`  
*The STAR-Dundee SpaceWire PXI RMAP device type.*
- `#define STAR_DEVICE_PXI_ROUTER_8 23U`  
*The STAR-Dundee SpaceWire PXI 8 port Router device type.*
- `#define STAR_DEVICE_PXI_ROUTER_12 24U`  
*The STAR-Dundee SpaceWire PXI 12 port Router device type.*
- `#define STAR_DEVICE_SPFIPCI 25U`  
*The STAR-Dundee SpaceFibre PCI Express device type.*
- `#define STAR_DEVICE_STAR_FIRE_MK3 26U`  
*The STAR-Dundee STAR Fire Mk3 device type.*
- `#define STAR_DEVICE_PCI_MK3 27U`  
*The STAR-Dundee SpaceWire PCI Mk3 device type.*
- `#define STAR_DEVICE_LINK_ANALYSER_MK3 28U`  
*The STAR-Dundee SpaceWire Link Analyser Mk3 device type.*
- `#define STAR_DEVICE_CONFORMANCE_TESTER_MK2 29U`  
*The STAR-Dundee SpaceWire Conformance Tester Mk2 device type.*
- `#define STAR_DEVICE_EGSE_MK2 30U`

- The STAR-Dundee SpaceWire EGSE Mk2 device type.*

  - `#define STAR_DEVICE_GBE_BRICK 31U`

*The STAR-Dundee SpaceWire GbE Brick device type.*
- `#define STAR_DEVICE_STAR_ULTRA_PCIE 32U`

*The STAR-Dundee STAR-Ultra-PCIe device type.*
- `#define STAR_DEVICE_PXI_INTERFACE_MK2 33U`

*The STAR-Dundee SpaceWire PXI Interface Mk2 device type.*
- `#define STAR_DEVICE_PXI_RMAP_MK2 34U`

*The STAR-Dundee SpaceWire PXI RMAP Mk2 device type.*
- `#define STAR_DEVICE_PXI_ROUTER_MK2 35U`

*The STAR-Dundee SpaceWire PXI Router Mk2 device type.*
- `#define STAR_DEVICE_CONFIG_SUPPORTED 0x00ffffbU`

*A device which is capable of being configured.*
- `#define STAR_DEVICE_CONFIG_NOT_SUPPORTED 0x00ffffcU`

*A device which is not capable of being configured.*
- `#define STAR_DEVICE_TXRX_SUPPORTED 0x00ffffdU`

*A device which is capable of transmitting and receiving packets.*
- `#define STAR_DEVICE_TXRX_NOT_SUPPORTED 0x00ffffeU`

*A device which is not capable of transmitting and receiving packets.*
- `#define STAR_DEVICE_ALL 0x00fffffU`

*All devices.*

## Typedefs

- `typedef CFG_INFO_ID_TYPE STAR_DRIVER_ID`

*Unique value used to identify an individual driver.*
- `typedef CFG_INFO_ID_TYPE STAR_DEVICE_ID`

*Unique value used to identify an individual device.*
- `typedef CFG_INFO_ID_TYPE STAR_CHANNEL_ID`

*Unique value used to identify an open channel.*
- `typedef CFG_INFO_ID_TYPE STAR_APP_ID`

*Unique value used to identify an application using STAR-System.*
- `typedef CFG_INFO_ID_TYPE STAR_DEVICE_LISTENER_ID`

*Unique value used to identify an individual device listener.*
- `typedef CFG_INFO_ID_TYPE STAR_DRIVER_LISTENER_ID`

*Unique value used to identify an individual driver listener.*
- `typedef CFG_INFO_ID_TYPE STAR_CHANNEL_LISTENER_ID`

*Unique value used to identify an individual channel listener.*
- `typedef void(STAR_API_CC * STAR_DriverEventFunc) (STAR_DRIVER_LISTENER_ID driverListener↵ Identifier, STAR_DRIVER_ID driverIdentifier, int driverAdded, void *pContextInfo)`

*The function type for driver listener functions which are called when a new driver is added, or an existing one removed.*
- `typedef void(STAR_API_CC * STAR_DeviceEventFunc) (STAR_DEVICE_LISTENER_ID deviceListener↵ Identifier, STAR_DRIVER_ID driverIdentifier, STAR_DEVICE_ID deviceIdIdentifier, int deviceAdded, void *p↵ ContextInfo)`

*The function type for device listener functions which are called when a device is added or removed.*
- `typedef void(STAR_API_CC * STAR_ChannelEventFunc) (STAR_CHANNEL_LISTENER_ID channel↵ ListenerIdentifier, STAR_DRIVER_ID driverIdentifier, STAR_DEVICE_ID deviceIdIdentifier, STAR_CHANN↵ EL_ID channelIdIdentifier, int channelOpened, U8 channelNumber, void *pContextInfo)`

*The function type for channel listener functions which are called when a device channel is opened or closed.*

## Enumerations

- enum `STAR_BUS_TYPE` {  
`STAR_BUS_UNKNOWN` = 0, `STAR_BUS_PCI` = 1, `STAR_BUS_USB` = 2, `STAR_BUS_PCIE` = 3,  
`STAR_BUS_TCP` = 4, `STAR_BUS_CPCI` = 5, `STAR_BUS_VIRTUAL` = 255 }

*The different types of bus that can be used to connect a SpaceWire device to a PC.*

- enum `STAR_DRIVER_TYPE` {  
`STAR_DRIVER_INVALID` = 0, `STAR_DRIVER_TYPE_USB` = 1, `STAR_DRIVER_TYPE_PCI` = 2, `STAR_DRIVER_TYPE_TCPIP` = 4,  
`STAR_DRIVER_TYPE_VIRTUAL` = 5 }

*Available driver types.*

- enum `STAR_CHANNEL_TYPE` { `STAR_CHANNEL_TYPE_NOT_OPEN`, `STAR_CHANNEL_TYPE_DEVICE`, `STAR_CHANNEL_TYPE_APPLICATION` }

*Channel types.*

- enum `STAR_CHANNEL_DIRECTION` { `STAR_CHANNEL_DIRECTION_IN` = 1, `STAR_CHANNEL_DIRECTION_OUT` = 2, `STAR_CHANNEL_DIRECTION_INOUT` }

*Channel directions.*

### 8.34.1 Detailed Description

Types, structures and function types used by STAR-API.

Types used to describe drivers, devices and channels.

#### Author

STAR-Dundee Ltd.  
 STAR House  
 166 Nethergate  
 Dundee, DD1 4EE  
 Scotland, UK  
 e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2012 STAR-Dundee Ltd.

### 8.34.2 Macro Definition Documentation

#### 8.34.2.1 `#define CFG_INFO_ID_TYPE U32`

## 8.35 version.h File Reference

Version information for a module.

```
#include <stdio.h>
#include "star-dundee_types.h"
```

## Data Structures

- struct `STAR_VERSION_INFO`

*A structure used to store version information for a module. [More...](#)*

## Macros

- `#define VERSION_MAJOR (4)`  
*The module's major version number.*
- `#define VERSION_MINOR (1)`  
*The module's minor version number.*
- `#define VERSION_EDIT (9809)`  
*The module's edit number.*
- `#define VERSION_PATCH (0)`  
*The module's patch number.*
- `#define STAR_STR_MAX_LEN 256`  
*Length of strings stored in a version information structure, inclusive of null terminator.*
- `#define PRINT_FULL_VERSION_INFO(o)`  
*Macro to print the full version information, including name and author.*
- `#define PRINT_VERSION(o)`  
*Macro to print the version information.*
- `#define PRINT_VERSION_EDIT(o) (void)fprintf(o, "(%d)", VERSION_EDIT)`  
*Macro to print the version's edit number if present.*
- `#define PRINT_VERSION_PATCH(o)`  
*Macro to print the version's patch number if present.*
- `#define PRINT_VERSION_NUM(o)`  
*Macro to print the version number.*
- `#define POPULATE_VERSION(v)`  
*Macro to populate a version information structure.*

### 8.35.1 Detailed Description

Version information for a module.

#### Author

STAR-Dundee Ltd.  
 STAR House  
 166 Nethergate  
 Dundee, DD1 4EE  
 Scotland, UK  
 e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

This file contains the current version information used by each of the modules in STAR-System, the STAR-Dundee software stack. Before including this file, `VERSION_NAME` and `VERSION_AUTHOR` macros should be defined to provide the name and authors of the module, e.g.

```
#define VERSION_NAME ("SpaceWire PCI Driver for Windows")
#define VERSION_AUTHOR ("Stuart Mills, STAR-Dundee Ltd.")
```

Copyright © 2015 STAR-Dundee Ltd.

### 8.35.2 Data Structure Documentation

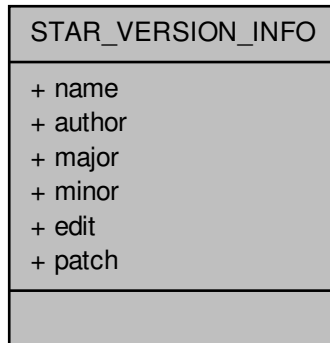
#### 8.35.2.1 struct STAR\_VERSION\_INFO

A structure used to store version information for a module.

**Examples:**

[all\\_version\\_example.c](#), [api\\_version\\_example.c](#), [firmware\\_version\\_example.c](#), [star-test.c](#), and [utilities.c](#).

Collaboration diagram for STAR\_VERSION\_INFO:

**Data Fields**

|      |                          |                                                                      |
|------|--------------------------|----------------------------------------------------------------------|
| char | author[STAR_STR_MAX_LEN] | The author of the module.                                            |
| U16  | edit                     | The edit number of this module.                                      |
| U16  | major                    | The major version number of this module.                             |
| U16  | minor                    | The minor version number of this module.                             |
| char | name[STAR_STR_MAX_LEN]   | The name of the module.                                              |
| U16  | patch                    | The patch number of this module. Edit will be 0 if patch is non zero |

**8.35.3 Macro Definition Documentation****8.35.3.1 #define POPULATE\_VERSION( v )****Value:**

```

{
 (v).major = VERSION_MAJOR;
 (v).minor = VERSION_MINOR;
 (v).edit = VERSION_EDIT;
 (v).patch = VERSION_PATCH;

 strcpy_s((v).name, STAR_STR_MAX_LEN, VERSION_NAME);
 strcpy_s((v).author, STAR_STR_MAX_LEN, VERSION_AUTHOR);
}

```

Macro to populate a version information structure.

**Parameters**



---

*v* the version information structure to be updated

---

### 8.35.3.2 #define PRINT\_FULL\_VERSION\_INFO( *o* )

**Value:**

```
{
 fprintf(o, "Module Name: %s\n", VERSION_NAME); \
 fprintf(o, "Module Author: %s\n", VERSION_AUTHOR); \
 fprintf(o, "Version: "); \
 PRINT_VERSION_NUM(o); \
 fprintf(o, "\n"); \
}
```

Macro to print the full version information, including name and author.

**Parameters**

---

*o* the output stream

---

### 8.35.3.3 #define PRINT\_VERSION( *o* )

**Value:**

```
{
 (void)fprintf(o, "%s", VERSION_NAME); \
 PRINT_VERSION_NUM(o); \
}
```

Macro to print the version information.

**Parameters**

---

*o* the output stream

---

### 8.35.3.4 #define PRINT\_VERSION\_EDIT( *o* ) (void)fprintf(o, "(%d)", VERSION\_EDIT)

Macro to print the version's edit number if present.

Used by [PRINT\\_VERSION\\_NUM\(FILE \\*\)](#) but contained in a separate macro to avoid "conditional expression is constant" warnings.

**Parameters**

---

*o* the output stream

---

### 8.35.3.5 #define PRINT\_VERSION\_NUM( *o* )

**Value:**

```
{
 (void)fprintf(o, "v%d.%02d", VERSION_MAJOR, VERSION_MINOR); \
 PRINT_VERSION_EDIT(o); \
 PRINT_VERSION_PATCH(o); \
}
```

Macro to print the version number.

---

**Parameters**

---

|          |                   |
|----------|-------------------|
| <i>o</i> | the output stream |
|----------|-------------------|

---

**8.35.3.6 #define PRINT\_VERSION\_PATCH( *o* )**

Macro to print the version's patch number if present.

Used by [PRINT\\_VERSION\\_NUM\(FILE \\*\)](#) but contained in a separate macro to avoid "conditional expression is constant" warnings.

**Parameters**

---

|          |                   |
|----------|-------------------|
| <i>o</i> | the output stream |
|----------|-------------------|

---

**8.35.3.7 #define VERSION\_EDIT (9809)**

The module's edit number.

**8.35.3.8 #define VERSION\_MAJOR (4)**

The module's major version number.

**8.35.3.9 #define VERSION\_MINOR (1)**

The module's minor version number.

**8.35.3.10 #define VERSION\_PATCH (0)**

The module's patch number.

## Chapter 9

# Example Documentation

### 9.1 advanced\_address\_example.c

This example assumes that channel 1 (defined in general header for api\_test\_app project) is connected to link 2 on a SpaceWire USB Brick or other routing device. Any packets sent out of channel 1 will end up coming back to where they started.

#### PCI Mk2 Brick

Link 1  Link 2

Link 2 

Link 3 

```
#include "examples.h"

#include "general.h"
#include "utilities.h"

#include "star-dundee_types.h"
#include "star-api.h"

void advancedAddressExample()
{
 /* Get first device */
 STAR_DEVICE_ID firstDevice = getFirstDevice();

 /* If there is a first device */
 if(firstDevice)
 {
 /* Open channel to send out of and receive into */
 STAR_CHANNEL_ID channel = STAR_openChannelToLocalDevice
 (firstDevice,
 STAR_CHANNEL_DIRECTION_OUT, CHANNEL1, 0);

 /* If transmit channel was opened */
 if(channel)
 {
 /* Create address path */
 unsigned char addressPath[] = {2};

 /* Get address path length */
 U16 addressPathLength = sizeof(addressPath) / sizeof(unsigned char);

 /* Create SpaceWire address */
 STAR_SPACEWIRE_ADDRESS * address =
 STAR_createAddress(addressPath,
 addressPathLength);

 /* If address was created */
 if(address)
 {
 /* Define packet data to be sent out */
 unsigned char packetData[] = {1, 2, 3, 4, 5, 6, 7, 8, 9};

 /* Get packet length */
 unsigned int packetDataLength = sizeof(packetData) /
 sizeof(unsigned char);
```

```

/* Create packet */
STAR_STREAM_ITEM * packet = STAR_createPacket(address,
 (unsigned char *)packetData, packetDataLength,
 STAR_EOP_TYPE_EOP);

/* If packet was created */
if(packet)
{
 /* Create send transfer operation */
 STAR_TRANSFER_OPERATION * sendTransferOperation =
 STAR_createTxOperation(&packet, 1);

 /* If send transfer operation was created */
 if (sendTransferOperation)
 {
 /* Define transfer status */
 STAR_TRANSFER_STATUS status;

 /* Start transmitting the packet */
 STAR_submitTransferOperation(channel,
 sendTransferOperation);

 /* Wait indefinitely for transfer to complete */
 status = STAR_waitOnTransferOperationCompletion
(
 sendTransferOperation, -1);

 /* Close transmit channel */
 STAR_closeChannel(channel);

 /* Open receive channel */
 channel = STAR_openChannelToLocalDevice(firstDevice,
 STAR_CHANNEL_DIRECTION_IN, CHANNEL2, 0);

 /* If receive channel was opened */
 if(channel)
 {
 /* If packet was sent */
 if(status == STAR_TRANSFER_STATUS_COMPLETE)
 {
 /* Create receive transfer operation */
 STAR_TRANSFER_OPERATION
 *receiveTransferOperation =
 STAR_createRxOperation(1,
 STAR_RECEIVE_PACKETS);

 /* If receive transfer operation was created */
 if(receiveTransferOperation)
 {
 /* Start receiving packet */
 STAR_submitTransferOperation(channel,
 receiveTransferOperation);

 /* Wait for packet to be received */
 status =
STAR_waitOnTransferOperationCompletion
(receiveTransferOperation, -1);

 /* If valid packet was received */
 if(status == STAR_TRANSFER_STATUS_COMPLETE)
)
 {
 /* Get received packet */
 STAR_STREAM_ITEM * streamItem =
 STAR_getTransferItem(
 receiveTransferOperation, 0);

 /* Initialise data length to 0 */
 unsigned int dataLength = 0;

 /* Get packet data and its length */
 unsigned char * data =
 STAR_getPacketData(
 (STAR_SPACEWIRE_PACKET *)streamItem->
 item, &dataLength);
 if(data != NULL)
 {
 /* Print received packet */
 printPacketContents(data,
 dataLength);

 /* Destroy packet data now that it
 has been output */
 STAR_destroyPacketData(data);
 }
 }
 }
 }
 }
 }
}

```

```

 /* Dispose receive transfer operation */
 STAR_disposeTransferOperation(
 receiveTransferOperation);
 }
}

/* Dispose send transfer operation */
STAR_disposeTransferOperation(sendTransferOperation);
}

/* Destroy packet */
STAR_destroyStreamItem(packet);
}

/* Dispose address */
STAR_destroyAddress(address);
}

/* Close channel */
STAR_closeChannel(channel);
}
}
}

```

## 9.2 advanced\_continuous\_receive\_example.c

Loops over the receive functions until 10 packets have been received. After a packet is received, the contents are displayed.

```

#include "examples.h"

#include "general.h"
#include "utilities.h"

#include "star-dundee_types.h"
#include "star-api.h"

void advancedContinuousReceiveExample()
{
 /* Get first device */
 STAR_DEVICE_ID firstDevice = getFirstDevice();

 /* If there is a first device */
 if(firstDevice)
 {
 /* Open receive channel */
 STAR_CHANNEL_ID receiveChannel =
 STAR_openChannelToLocalDevice(
 firstDevice, STAR_CHANNEL_DIRECTION_IN, CHANNEL1, 1);

 /* Initialise received packet count to 0 */
 unsigned int receivedPacketCount = 0;

 /* Create receive transfer operation */
 STAR_TRANSFER_OPERATION * receiveTransferOperation =
 STAR_createRxOperation(1, STAR_RECEIVE_PACKETS);

 /* While less than 10 packets have been received */
 while(receivedPacketCount < 10)
 {
 /* Define transfer status for receive transfer status updates */
 STAR_TRANSFER_STATUS status;

 /* Start receiving packet */
 STAR_submitTransferOperation(receiveChannel,
 receiveTransferOperation);

 /* Wait for packet to be received */
 status = STAR_waitOnTransferOperationCompletion(
 receiveTransferOperation, -1);

 /* If packet was received */
 if(status == STAR_TRANSFER_STATUS_COMPLETE)
 {
 /* Get received stream item */
 STAR_STREAM_ITEM * streamItem =
 STAR_getTransferItem(
 receiveTransferOperation, 0);
 }
 }
 }
}

```

```

 /* Initialise data length to 0 */
 unsigned int dataLength = 0;

 /* Get packet data and its length */
 unsigned char * data = STAR_getPacketData(
 (STAR_SPACEWIRE_PACKET *)streamItem->
item,
 &dataLength);

 if (data != NULL)
 {
 /* Prints the packet contents from the receive buffer */
 printPacketContents(data, dataLength);

 /* Destroy packet data now that it has been output */
 STAR_destroyPacketData(data);
 }

 /* Increment received packet count */
 receivedPacketCount++;
 }
}

/* Dispose receive transfer operation */
STAR_disposeTransferOperation(receiveTransferOperation);

/* Close receive channel */
STAR_closeChannel(receiveChannel);
}
}

```

### 9.3 advanced\_multiple\_packet\_example.c

Creates 3 packets, puts them in an array and then transmits the array in a single transfer operation. The packets are then received and printed.

```

#include "examples.h"

#include "general.h"
#include "utilities.h"

#include "star-dundee_types.h"
#include "star-api.h"

void advancedMultiplePacketExample()
{
 /* Get first device */
 STAR_DEVICE_ID firstDevice = getFirstDevice();

 /* If there is a first device */
 if(firstDevice)
 {
 /* Define EOP type */
 STAR_EOP_TYPE eopType = STAR_EOP_TYPE_EOP;

 /* Create first packet data */
 unsigned char packetData1[] = {1, 2, 3, 4, 5};

 /* Get length of first packet */
 unsigned int packetData1Length = sizeof(packetData1) /
 sizeof(unsigned char);

 /* Create first packet */
 STAR_STREAM_ITEM * packet1 = STAR_createPacket(NULL, packetData1,
 packetData1Length, eopType);

 /* Create second packet data */
 unsigned char packetData2[] = {6, 7, 8, 9, 10};

 /* Get length of second packet */
 unsigned int packetData2Length = sizeof(packetData2) /
 sizeof(unsigned char);

 /* Create second packet */
 STAR_STREAM_ITEM * packet2 = STAR_createPacket(NULL, packetData2,
 packetData2Length, eopType);

 /* Create third packet data */
 unsigned char packetData3[] = {11, 12, 13, 14, 15};
 }
}

```

```

/* Get length of third packet */
unsigned int packetData3Length = sizeof(packetData3) /
 sizeof(unsigned char);

/* Create third packet */
STAR_STREAM_ITEM * packet3 = STAR_createPacket(NULL, packetData3,
 packetData3Length, eopType);

/* If valid packets */
if(packet1 && packet2 && packet3)
{
 /* Open channel to send packets out of */
 STAR_CHANNEL_ID sendChannel =
 STAR_openChannelToLocalDevice(
 firstDevice, STAR_CHANNEL_DIRECTION_OUT, CHANNEL1, 0);

 /* If send channel was opened */
 if(sendChannel)
 {
 /* Define send transfer operation for sending packets */
 STAR_TRANSFER_OPERATION * sendTransferOperation;

 /* Wrap individual packets in an array */
 STAR_STREAM_ITEM * streamItems[3];
 streamItems[0] = packet1;
 streamItems[1] = packet2;
 streamItems[2] = packet3;

 /* Create send operation to send packets */
 sendTransferOperation =
 STAR_createTxOperation(streamItems, 3);

 /* If transfer operation was created */
 if(sendTransferOperation)
 {
 /* Define transfer status */
 STAR_TRANSFER_STATUS transmitStatus;

 /* Start transmitting the packets */
 STAR_submitTransferOperation(sendChannel,
 sendTransferOperation);

 /* Wait for packets to be transmitted */
 transmitStatus = STAR_waitOnTransferOperationCompletion
(
 sendTransferOperation, -1);

 /* If packets were transmitted */
 if(transmitStatus == STAR_TRANSFER_STATUS_COMPLETE)
 {
 /* Open receive channel */
 STAR_CHANNEL_ID receiveChannel =
 STAR_openChannelToLocalDevice(firstDevice,
 STAR_CHANNEL_DIRECTION_IN, CHANNEL2, 1);

 /* If receive channel was opened */
 if(receiveChannel)
 {
 /* Counter for loop */
 unsigned int index;

 /* Create transfer operation to receive 1 packet */
 STAR_TRANSFER_OPERATION * receiveTransferOperation =
 STAR_createRxOperation(3,
STAR_RECEIVE_PACKETS);

 /* If transfer operation was created */
 if(receiveTransferOperation)
 {
 /* Define transfer status */
 STAR_TRANSFER_STATUS receiveStatus;

 /* Start receiving packet */
 STAR_submitTransferOperation(receiveChannel,
 receiveTransferOperation);

 /* Wait for packet to be received */
 receiveStatus =
 STAR_waitOnTransferOperationCompletion
(
 receiveTransferOperation, -1);

 /* If valid stream item was received */
 if (receiveStatus ==
 STAR_TRANSFER_STATUS_COMPLETE)
 {
 /* Get transfer item count */

```





```

 &selectedChannelNumber, TRANSMIT_TYPE_RECEIVE);

/* If channel was opened */
if(selectedChannel)
{
 /* Create transfer operation to receive 1 packet */
 STAR_TRANSFER_OPERATION * transferOperation =
 STAR_createRxOperation
 (1, STAR_RECEIVE_PACKETS);

 /* If transfer operation was created */
 if(transferOperation)
 {
 /* Define transfer status */
 STAR_TRANSFER_STATUS status;

 /* Start receiving the packet */
 STAR_submitTransferOperation(selectedChannel, transferOperation);

 /* Print waiting on incoming packet message */
 printf("Waiting on incoming packet...\n");

 /* Wait for packet to be received */
 status = STAR_waitOnTransferOperationCompletion(
 transferOperation, -1);

 /* If packet was received */
 if(status == STAR_TRANSFER_STATUS_COMPLETE)
 {
 /* Get received stream item */
 STAR_STREAM_ITEM * streamItem =
 STAR_getTransferItem(
 transferOperation, 0);

 /* If valid stream item was received */
 if(streamItem)
 {
 /* Initialise data length to 0 */
 unsigned int dataLength = 0;

 /* Get packet data and its length */
 unsigned char * data = STAR_getPacketData(
 (STAR_SPACEWIRE_PACKET *)streamItem->
item,
 &dataLength);

 if (data != NULL)
 {
 /* Prints the packet contents from the receive buffer */
 printPacketContents(data, dataLength);

 /* Destroy packet data now that it has been output */
 STAR_destroyPacketData(data);
 }
 }

 /* Dispose transfer operation */
 STAR_disposeTransferOperation(transferOperation);
 }

 /* Close channel after transmitting packet */
 STAR_closeChannel(selectedChannel);
 }
}

```

## 9.5 advanced\_send\_and\_receive\_example.c

A simple packet is created, sent using a transfer operation out of one channel which is then received on another. The received packet is printed.

```

#include "examples.h"

#include "general.h"
#include "utilities.h"

#include "star-dundee_types.h"
#include "star-api.h"

void advancedSendAndReceiveExample()

```

```

{
 /* Define transfer status for send and receive status information */
 STAR_TRANSFER_STATUS transmitStatus;

 /* Get first device */
 STAR_DEVICE_ID firstDevice = getFirstDevice();

 /* Define packet data to be sent */
 unsigned char packetData[] = {1, 2, 3, 4, 5, 6, 7, 8, 9};

 /* If there is a first device */
 if(firstDevice)
 {
 /* Open send channel */
 STAR_CHANNEL_ID sendChannel =
 STAR_openChannelToLocalDevice(
 firstDevice, STAR_CHANNEL_DIRECTION_OUT, CHANNEL1, 1);

 /* If send channel was opened */
 if(sendChannel)
 {
 /* Get packet length */
 unsigned int packetLength = sizeof(packetData) /
 sizeof(unsigned char);

 /* Create packet */
 STAR_STREAM_ITEM * packet = STAR_createPacket(NULL,
 (unsigned char *)packetData, packetLength, STAR_EOP_TYPE_EOP);

 /* If packet was created */
 if(packet)
 {
 /* Create transfer operation to send packet */
 STAR_TRANSFER_OPERATION * sendTransferOperation =
 STAR_createTxOperation(&packet, 1);

 /* If transfer operation was created */
 if(sendTransferOperation)
 {
 /* Start transmitting the packet */
 STAR_submitTransferOperation(sendChannel,
 sendTransferOperation);

 /* Wait for packet to be transmitted */
 transmitStatus = STAR_waitOnTransferOperationCompletion

(
 sendTransferOperation, -1);

 /* Dispose transfer operation */
 STAR_disposeTransferOperation(sendTransferOperation);

 /* Destroy packet after it was sent */
 STAR_destroyStreamItem(packet);

 /* If packet was sent */
 if(transmitStatus == STAR_TRANSFER_STATUS_COMPLETE)
 {
 /* Open receive channel */
 STAR_CHANNEL_ID receiveChannel =
 STAR_openChannelToLocalDevice(firstDevice,
 STAR_CHANNEL_DIRECTION_IN, CHANNEL2, 1);

 /* If receive channel was opened */
 if(receiveChannel)
 {
 /* Create transfer operation to receive 1 packet */
 STAR_TRANSFER_OPERATION * receiveTransferOperation =
 STAR_createRxOperation(1,
STAR_RECEIVE_PACKETS);

 /* If transfer operation was created */
 if(receiveTransferOperation)
 {
 /* Define transfer status */
 STAR_TRANSFER_STATUS receiveStatus;

 /* Start receiving packet */
 STAR_submitTransferOperation(receiveChannel,
 receiveTransferOperation);

 /* Wait for packet to be received */
 receiveStatus =
 STAR_waitOnTransferOperationCompletion

(
 receiveTransferOperation, -1);

 /* If valid stream item was received */

```

```

 if(receiveStatus ==
 STAR_TRANSFER_STATUS_COMPLETE)
 {
 /* Get received stream item */
 STAR_STREAM_ITEM * streamItem =
 STAR_getTransferItem(
 receiveTransferOperation, 0);

 /* Initialise data length to 0 */
 unsigned int dataLength = 0;

 /* Get packet data and its length */
 unsigned char *data = STAR_getPacketData(
 (STAR_SPACEWIRE_PACKET *)streamItem->
 item, &dataLength);

 if (data != NULL)
 {
 /* Prints the packet contents from the
 receive buffer */
 printPacketContents(data, dataLength);

 /* Destroy packet data now that it has
 been output */
 STAR_destroyPacketData(data);
 }
 }

 /* Dispose receive transfer operation */
 STAR_disposeTransferOperation(
 receiveTransferOperation);
 }

 /* Close receive channel */
 STAR_closeChannel(receiveChannel);
}

}

}

/* Close send channel */
STAR_closeChannel(sendChannel);
}

}
}

```

## 9.6 advanced\_send\_example.c

The user is prompted for the device to use, the channel to send out of and a series of bytes to be sent in the packet. The packet is sent using the advanced transfer functions.

```

#include "examples.h"

#include "general.h"
#include "utilities.h"

#include "star-dundee_types.h"
#include "star-api.h"

#include <stdlib.h>

void advancedSendExample()
{
 /* Pointer to a list of available SpaceWire devices */
 STAR_DEVICE_ID * devices = NULL;

 /* Define selected channel */
 int selectedChannelNumber;

 /* Get selected device */
 STAR_DEVICE_ID selectedDevice = promptForDevice(TRANSMIT_TYPE_SEND);

 /* Get selected channel */
 STAR_CHANNEL_ID selectedChannel = promptForChannel(selectedDevice,
 &selectedChannelNumber, TRANSMIT_TYPE_SEND);

 /* If channel was opened */
 if(selectedChannel)
 {
 /* Allocate array of bytes to be populated */

```

```

unsigned char bytes[256];
U16 readBytesLength = 0;

/* Define packet that will hold the packet buffer data */
STAR_STREAM_ITEM * packet;

/* Ask user to enter a series of bytes to send */
printf("\nPlease enter a series of bytes to be transmitted over "
 "SpaceWire:\n");

/* If valid bytes read input bytes from console */
if(readBytes(bytes, 256, &readBytesLength))
{
 /* Create packet */
 packet = STAR_createPacket(NULL, bytes,
 (unsigned int)readBytesLength, STAR_EOP_TYPE_EOP);

 /* If packet was created */
 if(packet)
 {
 /* Create transfer operation */
 STAR_TRANSFER_OPERATION * transferOperation =
 STAR_createTxOperation(&packet, 1);

 /* If transfer operation was created */
 if(transferOperation)
 {
 /* Define transfer status for result of transfer
 operation */
 STAR_TRANSFER_STATUS status;

 /* Start transmitting the packet */
 STAR_submitTransferOperation(selectedChannel,
 transferOperation);

 /* Wait for packet to be transmitted */
 status = STAR_waitOnTransferOperationCompletion(
 transferOperation, -1);

 /* Check transfer status */
 if (status != STAR_TRANSFER_STATUS_COMPLETE)
 {
 printf("Unexpected transfer status: %d\n", status);
 }

 /* Dispose transfer operation */
 STAR_disposeTransferOperation(transferOperation);

 /* Destroy packet after it was sent */
 STAR_destroyStreamItem(packet);
 }
 }

 /* Close channel after transmitting packet */
 STAR_closeChannel(selectedChannel);
}
else
{
 /* Print error */
 printf("Channel %d could not be opened.\n", selectedChannelNumber);
}

/* Destroy device list */
STAR_destroyDeviceList(devices);
}

```

## 9.7 advanced\_two\_way\_example.c

Two packets are constructed and then sent in either direction using two way channels and the advanced transfer functions.

```

#include "examples.h"

#include "general.h"
#include "utilities.h"

#include "star-dundee_types.h"
#include "star-api.h"

```

```

void receiveAdvancedPacket(STAR_CHANNEL_ID channel)
{
 /* Create transfer operation to receive 1 packet */
 STAR_TRANSFER_OPERATION * receiveTransferOperation =
 STAR_createRxOperation(1, STAR_RECEIVE_PACKETS);

 /* If transfer operation was created */
 if(receiveTransferOperation)
 {
 /* Define transfer status for receive status information */
 STAR_TRANSFER_STATUS status;

 /* Start receiving packet */
 STAR_submitTransferOperation(channel,
 receiveTransferOperation);

 /* Wait for packet to be received */
 status = STAR_waitOnTransferOperationCompletion
 (receiveTransferOperation, -1);

 /* If valid stream item was received */
 if(status == STAR_TRANSFER_STATUS_COMPLETE)
 {
 /* Get received stream item */
 STAR_STREAM_ITEM * streamItem =
 STAR_getTransferItem(
 receiveTransferOperation, 0);

 /* Initialise data length to 0 */
 unsigned int dataLength = 0;

 /* Get packet data and its length */
 unsigned char * data = STAR_getPacketData(
 (STAR_SPACEWIRE_PACKET *)streamItem->
 item, &dataLength);

 if (data != NULL)
 {
 /* Prints the packet contents from the receive buffer */
 printPacketContents(data, dataLength);

 /* Destroy packet data now that it has been output */
 STAR_destroyPacketData(data);
 }

 /* Dispose transfer operation */
 STAR_disposeTransferOperation(receiveTransferOperation);
 }
 }
}

void advancedTwoWayExample()
{
 /* Define transfer status for send and receive status information */
 STAR_TRANSFER_STATUS status;

 /* Get first device */
 STAR_DEVICE_ID firstDevice = getFirstDevice();

 /* Define packet data to be sent out of first channel */
 unsigned char packetData1[] = {1, 2, 3, 4, 5, 6, 7, 8, 9};

 /* Define packet data to be sent out of second channel */
 unsigned char packetData2[] = {9, 8, 7, 6, 5, 4, 3, 2, 1};

 /* If there is a device to use */
 if(firstDevice)
 {
 /* Open channel to send and receive on */
 STAR_CHANNEL_ID channel1 = STAR_openChannelToLocalDevice
 (
 firstDevice, STAR_CHANNEL_DIRECTION_INOUT, CHANNEL1, 1);

 /* Open channel to send and receive on */
 STAR_CHANNEL_ID channel2 = STAR_openChannelToLocalDevice
 (
 firstDevice, STAR_CHANNEL_DIRECTION_INOUT, CHANNEL2, 1);

 /* If both channels were opened */
 if(channel1 && channel2)
 {
 /* Get packet length */
 unsigned int packetLength = sizeof(packetData1) /
 sizeof(unsigned char);

 /* Create packet to send out of first channel */
 STAR_STREAM_ITEM * packet1 = STAR_createPacket(NULL,

```

```

 (unsigned char *)packetData1, packetLength, STAR_EOP_TYPE_EOP);

/* If packet was created */
if(packet1)
{
 /* Create transfer operation to send packet out of first
 channel */
 STAR_TRANSFER_OPERATION * sendTransferOperation1 =
 STAR_createTxOperation(&packet1, 1);

 /* If transfer operation was created */
 if(sendTransferOperation1)
 {
 /* Start transmitting the packet */
 STAR_submitTransferOperation(channel1,
 sendTransferOperation1);

 /* Wait for packet to be trasmitted */
 status = STAR_waitOnTransferOperationCompletion(
 sendTransferOperation1, -1);

 /* Dispose transfer operation */
 STAR_disposeTransferOperation(sendTransferOperation1);

 /* Destroy packet after it was sent */
 STAR_destroyStreamItem(packet1);

 /* If packet was sent */
 if(status == STAR_TRANSFER_STATUS_COMPLETE)
 {
 /* Define packet to be sent out of second channel */
 STAR_STREAM_ITEM * packet2;

 /* Receive packet on channel 2 */
 receiveAdvancedPacket(channel2);

 /* Get packet length */
 packetLength = sizeof(packetData2) /
 sizeof(unsigned char);

 /* Create packet to send out of second channel */
 packet2 = STAR_createPacket(NULL,
 (unsigned char *)packetData2, packetLength,
 STAR_EOP_TYPE_EOP);

 /* If packet was created */
 if(packet2)
 {
 /* Create transfer operation to send packet out of
 second channel */
 STAR_TRANSFER_OPERATION * sendTransferOperation2 =
 STAR_createTxOperation(&packet2, 1);

 /* If transfer operation was created */
 if(sendTransferOperation2)
 {
 /* Start transmitting the packet */
 STAR_submitTransferOperation(channel2,
 sendTransferOperation2);

 /* Wait for packet to be transmitted */
 status = STAR_waitOnTransferOperationCompletion
 (
 sendTransferOperation2, -1);

 /* Dispose transfer operation */
 STAR_disposeTransferOperation(
 sendTransferOperation2);

 /* Destroy packet after it was sent */
 STAR_destroyStreamItem(packet2);

 /* If packet was sent */
 if(status == STAR_TRANSFER_STATUS_COMPLETE)
 {
 /* Receive packet on first channel */
 receiveAdvancedPacket(channel1);
 }
 }
 }
 }
 }
}

/* Close first channel */
STAR_closeChannel(channel1);

```

```

 /* Close second channel */
 STAR_closeChannel(channel2);
 }
}

```

## 9.8 all\_version\_example.c

Uses the `STAR_getAllVersions()` function to retrieve a list of versions of the modules that are in use and prints the version information.

```

#include "examples.h"

#include "utilities.h"

#include "star-api.h"

void allVersionExample()
{
 /* Define number of versions */
 U32 numberOfVersions = 0;

 /* Define counter for loop */
 unsigned int index;

 /* Get all versions */
 STAR_VERSION_INFO *versionInfos = STAR_getAllVersions(&
 numberOfVersions);

 /* For number of versions */
 for(index = 0; (versionInfos != NULL) && (index < numberOfVersions);
 index++)
 {
 /* Print version information string */
 printVersionInfo(&versionInfos[index]);

 /* If not the last version */
 if (index < (numberOfVersions - 1))
 {
 /* Print blank line to separate */
 printf("\n");
 }
 }

 /* Destroy version info string */
 STAR_destroyVersionInfoList(versionInfos);
}

```

## 9.9 api\_test\_app.c

This file is the entry point to various example functions that demonstrate the capabilities of STAR-API including transmitting and receiving packets.

```

#include "examples.h"

#if !defined(UNREFERENCED_PARAMETER)
 #define UNREFERENCED_PARAMETER(a) ((void) (a))
#endif

#include <stdlib.h>

#include "star-api.h"

int __cdecl main(int argc, char *argv[])
{
 UNREFERENCED_PARAMETER(argc);
 UNREFERENCED_PARAMETER(argv);

 STAR_setApplicationName("STAR-System API Test Application");

 /* Configure the transmit and receive channels in general.h
 Uncomment the example(s) below to run */

```

```

//advancedAddressExample();
//advancedContinuousReceiveExample();
//advancedMultiplePacketExample();
//advancedReceiveExample();
//advancedSendAndReceiveExample();
//advancedSendExample();
//advancedTwoWayExample();
//allVersionExample();
//apiVersionExample();
//applicationNameExample();
//channelCallbackExample();
//driverListExample();
//firmwareVersionExample();
//simpleReceiveExample();
//simpleSendAndReceiveExample();
//simpleSendExample();
//simpleTwoWayExample();
//timecodeExample();
//transferCallbackExample();
//transferOperationListSendExample();
//transferOperationListSendReceiveExample();
//transmitErrorsExample();

return 0;
}

```

## 9.10 api\_version\_example.c

Uses the `STAR_getApiVersion()` function to retrieve the current API version that is in use and prints it.

```

#include "examples.h"
#include "utilities.h"
#include "star-api.h"

void apiVersionExample()
{
 /* Get API version info */
 STAR_VERSION_INFO * versionInfo = STAR_getApiVersion();

 /* Print version info */
 printVersionInfo(versionInfo);

 /* Destroy version info */
 STAR_destroyVersionInfo(versionInfo);
}

```

## 9.11 application\_name\_example.c

Prints the initial application name, sets it to a new value and then prints the new value.

```

#include "examples.h"
#include "star-api.h"

void applicationNameExample()
{
 /* Get initial application name */
 char * applicationName = STAR_getApplicationName();

 if (applicationName != NULL)
 {
 /* Print initial application name */
 printf("Initial application name: %s\n", applicationName);

 /* Destroy application name */
 STAR_destroyString(applicationName);
 }

 /* Set application name */
 STAR_setApplicationName("Test STAR-System Application");

 /* Get application name */
}

```



```

applicationName = STAR_getApplicationName();

if (applicationName != NULL)
{
 /* Print application name */
 printf("Application name: %s\n", applicationName);

 /* Destroy application name */
 STAR_destroyString(applicationName);
}
}

```

## 9.12 brickMk2\_configuration\_example.c

Example usage of this API.

```

#include <stdio.h>
#include "star-api.h"
#include "cfg_api_generic.h"
#include "ui.h"

int __cdecl main()
{
 STAR_DEVICE_ID deviceID;
 STAR_CFG_FPGA_INFO fpgaInfo;
 char versionStr[STAR_CFG_MK2_VERSION_STR_MAX_LEN];
 char buildDateStr[STAR_CFG_MK2_BUILD_DATE_STR_MAX_LEN];
 int enabled;
 U16 linkSpeed;

 STAR_CHANNEL_ID channelID;
 STAR_TRANSFER_OPERATION *linkEventRxOp;
 STAR_STREAM_ITEM *linkEvent;
 STAR_CFG_BRICK_MK2_ERRORS linkErrors;
 U32 newLinkSpeed;

 STAR_DEVICE_TYPE aDeviceTypes[2];
 aDeviceTypes[0] = STAR_DEVICE_BRICK_MK2;
 aDeviceTypes[1] = STAR_DEVICE_ROUTER_MK2S;

 /* Select device to configure */
 deviceID = STAR_UI_chooseDevice(aDeviceTypes, 2);
 if (!deviceID)
 {
 return 0;
 }

 STAR_getDeviceType(deviceID);

 /* Get hardware info*/
 CFG_getFPGAInfo(deviceID, &fpgaInfo);

 CFG_FPGAInfoToString(deviceID, &fpgaInfo, versionStr, buildDateStr);

 /* Display the hardware info*/
 printf("\nVersion: %s", versionStr);
 printf("\nBuildDate: %s", buildDateStr);

 /* Set general purpose register to 0xABCD */
 CFG_setGeneralPurpose(deviceID, 0xABCD);

 /* Flash the device's LEDs*/
 CFG_identify(deviceID);

 /* Set the device to be a time-code master*/
 CFG_enableTimeCodeMaster(deviceID);

 /* Set 2 second delay between time-codes */
 CFG_setTimeCodePeriod(deviceID, 2000000);

 /* Enable interface mode */
 CFG_enableInterfaceMode(deviceID);

 /* Enable interface mode on port 1 only */
 CFG_enableInterfaceModeOnPort(deviceID, 1);

 /* Disable interface mode on port 2 only */
 CFG_disableInterfaceModeOnPort(deviceID, 2);

 CFG_getInterfaceModeOnPortEnabled(deviceID, 2, &enabled);
}

```

```

printf("\nInterface mode %s on port 2", enabled ? "enabled" : "disabled");

/* Enable adding the source port number as a leading byte
 to received packets on ports 1 and 2 */

CFG_enableIdentifySource(deviceID);
CFG_enableIdentifySourceOnPort(deviceID, 1);
CFG_enableIdentifySourceOnPort(deviceID, 2);

/* Check if identify source is enabled for port 1 */
CFG_getIdentifySourceOnPortEnabled(deviceID, 1, &enabled);
printf("\nIdentify source %s on port 1", enabled ? "enabled" : "disabled");

CFG_disableIdentifySourceOnPort(deviceID, 1);

/* Set link speed for link 1 to 100 Mbit/s */
CFG_setTransmitSignallingRate(deviceID, 1, 100);

/* Get measured link speed */
CFG_getMeasuredLinkSpeed(deviceID, 1, &linkSpeed);

printf("\nMeasured Link Speed: %d Kbit/s",
 linkSpeed * STAR_CFG_MK2_LINK_SPEED_UNITS_KBPS);

/* Inject an error */
linkErrors.suppressFCT = 1;
CFG_injectErrors(deviceID, 2, &linkErrors);

/* Enable Link Speed Events */
CFG_enableSpeedChangeEventsOnPort(deviceID, 1);

/* To receive link speed events, we must open a connection to device
 channel 0. This will prevent configuration operations from being
 performed until we close this channel again. */

channelID = STAR_openChannelToLocalDevice(deviceID,
 STAR_CHANNEL_DIRECTION_IN,
 0,
 0);

linkEventRxOp = STAR_createRxOperation(1,
 STAR_RECEIVE_LINK_SPEED_EVENTS);

STAR_submitTransferOperation(channelID, linkEventRxOp);

/* Wait for up to 10 seconds for link speed event to occur */
printf("\nWaiting for Link speed event...");
STAR_waitOnTransferOperationCompletion(linkEventRxOp, 10000);

/* Display new link speed */
switch(STAR_getTransferOperationStatus(linkEventRxOp)){
case STAR_TRANSFER_STATUS_COMPLETE:
 linkEvent = STAR_getTransferItem(linkEventRxOp, 0);
 newLinkSpeed = STAR_getLinkSpeedEventSpeed(
 (STAR_LINK_SPEED_EVENT*)(linkEvent->item));
 printf("\nNew Link Speed: %u Kbit/s", newLinkSpeed/1000);
 break;
case STAR_TRANSFER_STATUS_STARTED:
case STAR_TRANSFER_STATUS_NOT_STARTED:
 printf("\nLink speed event did not occur");
 STAR_cancelTransferOperation(linkEventRxOp);
 break;
default:
 printf("\nError occurred while waiting for link speed event");
}

STAR_disposeTransferOperation(linkEventRxOp);
STAR_closeChannel(channelID);

CFG_disableSpeedChangeEventsOnPort(deviceID, 1);

return 0;
}

```

## 9.13 channel\_callback\_example.c

Opens and closes various channels, printing messages when the channel callback notifications are received.

```
#include "examples.h"
```

```

#include "general.h"
#include "utilities.h"

#include "star-dundee_types.h"
#include "star-api.h"

void STAR_API_CC ChannelCallback(STAR_CHANNEL_LISTENER_ID
 channelListenerIdentifier, STAR_DRIVER_ID driverIdentifier,
 STAR_DEVICE_ID deviceIdentifier, STAR_CHANNEL_ID channelIdentifier,
 int channelOpened, U8 channelNumber, void *pContextInfo)
{
 /* Unreferenced parameters */
 UNREFERENCED_PARAMETER(pContextInfo);
 UNREFERENCED_PARAMETER(deviceIdentifier);
 UNREFERENCED_PARAMETER(driverIdentifier);
 UNREFERENCED_PARAMETER(channelListenerIdentifier);
 UNREFERENCED_PARAMETER(channelIdentifier);

 /* If channel was opened */
 if(channelOpened)
 {
 /* Print channel opened message */
 printf("Opened channel %u.\n", channelNumber);
 }
 else
 {
 /* Print channel closed message */
 printf("Closed channel %u.\n", channelNumber);
 }
}

void channelCallbackExample()
{
 /* Define first out channel */
 STAR_CHANNEL_ID outChannel1;

 /* Define second out channel */
 STAR_CHANNEL_ID outChannel2;

 /* Define first in channel */
 STAR_CHANNEL_ID inChannel1;

 /* Define second in channel */
 STAR_CHANNEL_ID inChannel2;

 /* Get first device */
 STAR_DEVICE_ID firstDevice = getFirstDevice();

 /* Register channel listener */
 STAR_CHANNEL_LISTENER_ID channelListener =
 STAR_registerChannelListener(
 ChannelCallback, NULL, 0);

 /* If channel listener was registered */
 if(channelListener)
 {
 /* Open first channel with out direction */
 outChannel1 = STAR_openChannelToLocalDevice(firstDevice,
 STAR_CHANNEL_DIRECTION_OUT, CHANNEL1, 0);

 /* Open second channel with out direction */
 outChannel2 = STAR_openChannelToLocalDevice(firstDevice,
 STAR_CHANNEL_DIRECTION_OUT, CHANNEL2, 0);

 /* Open first channel with in direction */
 inChannel1 = STAR_openChannelToLocalDevice(firstDevice,
 STAR_CHANNEL_DIRECTION_IN, CHANNEL1, 0);

 /* Open second channel with in direction */
 inChannel2 = STAR_openChannelToLocalDevice(firstDevice,
 STAR_CHANNEL_DIRECTION_IN, CHANNEL2, 0);

 /* Close first out channel */
 STAR_closeChannel(outChannel1);

 /* Close second out channel */
 STAR_closeChannel(outChannel2);

 /* Close first in channel */
 STAR_closeChannel(inChannel1);

 /* Close second in channel */
 STAR_closeChannel(inChannel2);

 /* Unregister channel listener */
 STAR_unregisterChannelListener(channelListener);
 }
}

```

```
}
```

## 9.14 check\_packet\_example.c

This is an example of how to check that the format of a received RMAP packet is correct, then access the fields in the packet.

```
#include <stdio.h>

#include "rmap_packet_library.h"

/* Display the type of a packet. */
void display_packet_type(RMAP_PACKET_TYPE type)
{
 switch (type)
 {
 case RMAP_WRITE_COMMAND:
 puts("The packet is an RMAP write command.");
 break;

 case RMAP_WRITE_REPLY:
 puts("The packet is an RMAP write reply.");
 break;

 case RMAP_READ_COMMAND:
 puts("The packet is an RMAP read command.");
 break;

 case RMAP_READ_REPLY:
 puts("The packet is an RMAP read reply.");
 break;

 case RMAP_READ_MODIFY_WRITE_COMMAND:
 puts("The packet is an RMAP read/modify/write command.");
 break;

 case RMAP_READ_MODIFY_WRITE_REPLY:
 puts("The packet is an RMAP read/modify/write reply.");
 break;

 case RMAP_INVALID_PACKET_TYPE:
 puts("The packet is of an invalid RMAP packet type.");
 break;

 default:
 puts("The packet is of an unexpected type.");
 }
}

/* Display the status of a packet. */
void display_packet_status(RMAP_STATUS status)
{
 switch (status)
 {
 case RMAP_SUCCESS:
 puts("The status indicates the command completed successfully.");
 break;

 case RMAP_GENERAL_ERROR:
 puts("The status indicates a general error that does not fit into the other error cases.");
 break;

 case RMAP_UNUSED_PACKET_TYPE_OR_COMMAND_CODE:
 puts("The status indicates the packet type or command is an unexpected value.");
 break;

 case RMAP_INVALID_KEY:
 puts("The status indicates the key did not match that expected by the target user application.");
 break;

 case RMAP_INVALID_DATA_CRC:
 puts("The status indicates an invalid data CRC.");
 break;

 case RMAP_EARLY_EOP:
 puts("The status indicates an EOP was detected before the end of the data.");
 break;

 case RMAP_TOO_MUCH_DATA:
 }
```

```

 puts("The status indicates there was more data than was expected.");
 break;

 case RMAP_EEP:
 puts("The status indicates an EEP was encountered in the packet after the header.");
 break;

 case RMAP_VERIFY_BUFFER_OVERRUN:
 puts("The status indicates verify before write was enabled in the command but not enough buffer
space was available to receive the full command.");
 break;

 case RMAP_COMMAND_NOT_IMPLEMENTED_OR_AUTHORISED:
 puts("The status indicates the target user application did not authorise the requested
operation.");
 break;

 case RMAP_RMW_DATA_LENGTH_ERROR:
 puts("The status indicates the amount of data in a read/modify/write command is invalid.");
 break;

 case RMAP_INVALID_TARGET_LOGICAL_ADDRESS:
 puts("The status indicates the target logical address was not the value expected by the target.
");
 break;

 default:
 puts("The status is an unexpected value.");
 }
}

void check_packet_example()
{
 void *pPacket;
 unsigned long packetLen, addressLen, i;
 U8 pTarget[] = {0, 254};
 U8 pReply[] = {254};
 RMAP_PACKET packetStruct;
 RMAP_STATUS status;
 RMAP_PACKET_TYPE type;
 U8 *pAddress, *pData, *pMask;
 char enabled;
 U8 key, extendedMemoryAddress, crc, maskLen;
 U16 transactionID;
 U32 memoryAddress, dataLen;

 /* Build an RMAP write command packet to write to a 4-byte register */
 pPacket = RMAP_BuildWriteRegisterPacket(pTarget, 2, pReply, 1, 1, 1, 0,
0x20, 0, 0x106, 0, 0x12345678, &packetLen, &packetStruct, 1);

 /* Check the format of the packet, making sure it's not longer than */
 /* expected. Note that the first byte is skipped as RMAP_CheckPacketValid */
 /* expects the address at the start to be 1 byte long. */
 status = RMAP_CheckPacketValid((U8 *)pPacket + 1, packetLen - 1,
&packetStruct, 1);
 printf("The packet is %sa valid RMAP packet.\n",
(status == RMAP_SUCCESS) ? "" : "not ");

 /* Get the type of the packet */
 type = RMAP_GetPacketType(&packetStruct);

 /* Display the type of the packet */
 display_packet_type(type);

 /* Get the target address of the packet */
 pAddress = RMAP_GetTargetAddress(&packetStruct, &addressLen);
 if ((!pAddress) || (addressLen == 0))
 {
 puts("No valid target address is present in the packet.");
 }
 else
 {
 /* Display the target address of the packet */
 printf("The packet has a target address of:");
 for (i = 0; i < addressLen; i++)
 {
 printf(" %x", pAddress[i]);
 }
 puts("");
 }

 /* Determine if verify before write is enabled */
 enabled = RMAP_GetVerifyBeforeWrite(&packetStruct);

 /* Display whether verify before write is enabled */
 printf("Verify before write is %senabled for the packet.\n",
enabled ? "" : "not ");
}

```

```

/* Determine if acknowledgement is enabled */
enabled = RMAP_GetPerformAcknowledgement(&packetStruct);

/* Display whether acknowledgement is enabled */
printf("Acknowledgement is %senabled for the packet.\n",
 enabled ? "" : "not ");

/* Determine if incrementing addresses is enabled */
enabled = RMAP_GetIncrementAddress(&packetStruct);

/* Display whether incrementing addresses is enabled */
printf("Incrementing addresses is %senabled for the packet.\n",
 enabled ? "" : "not ");

/* Get the key of the packet */
key = RMAP_GetKey(&packetStruct);

/* Display the key of the packet */
printf("The value of the key is 0x%x.\n", key);

/* Get the transaction identifier of the packet */
transactionID = RMAP_GetTransactionID(&packetStruct);

/* Display the transaction identifier of the packet */
printf("The value of the transaction identifier is %d.\n", transactionID);

/* Get the reply address of the packet */
pAddress = RMAP_GetReplyAddress(&packetStruct, &addressLen);
if ((!pAddress) || (addressLen == 0))
{
 puts("No valid reply address is present in the packet.");
}
else
{
 /* Display the reply address of the packet */
 printf("The packet has a reply address of:");
 for (i = 0; i < addressLen; i++)
 {
 printf(" %x", pAddress[i]);
 }
 puts("");
}

/* Get the memory address and extended memory address of the packet */
memoryAddress = RMAP_GetAddress(&packetStruct, &extendedMemoryAddress);

/* Display the transaction identifier of the packet */
printf("The value of the memory address is 0x%x ", memoryAddress);
printf("and the extended address is 0x%x.\n", extendedMemoryAddress);

/* Get the data in the packet */
pData = RMAP_GetData(&packetStruct, &dataLen);
if ((!pData) || (dataLen == 0))
{
 puts("No valid data is present in the packet.");
}
else
{
 /* Display the data in the packet */
 printf("The packet has the following data:");
 for (i = 0; i < dataLen; i++)
 {
 printf(" %x", pData[i]);
 }
 puts("");
}

/* Get the status of the packet */
status = RMAP_GetStatus(&packetStruct);

/* Display the status of the packet */
display_packet_status(status);

/* Get the header CRC of the packet */
crc = RMAP_GetHeaderCRC(&packetStruct);

/* Display the header CRC of the packet */
printf("The value of the header CRC is 0x%x.\n", crc);

/* Get the data CRC of the packet */
crc = RMAP_GetDataCRC(&packetStruct);

/* Display the data CRC of the packet */
printf("The value of the data CRC is 0x%x.\n", crc);

/* Get the mask in the packet */

```

```

pMask = RMAP_GetMask(&packetStruct, &maskLen);
if ((!pMask) || (maskLen == 0))
{
 puts("No valid mask is present in the packet.");
}
else
{
 /* Display the data in the packet */
 printf("The packet has the following mask:");
 for (i = 0; i < maskLen; i++)
 {
 printf(" %x", pMask[i]);
 }
 puts("");
}

/* Free the RMAP write command packet created above */
RMAP_FreeBuffer(pPacket);
}

```

## 9.15 crc\_example.c

This is an example of how to use the CRC functions.

```

#include <stdio.h>

#include "rmap_packet_library.h"

#define BUFFER_LEN 32
#define HEADER_LEN 10

void crc_example()
{
 U8 pBuffer[BUFFER_LEN], crc;
 int i;

 /* Fill the data buffer */
 for (i = 0; i < BUFFER_LEN; i++)
 {
 pBuffer[i] = i;
 }

 /* Calculate a CRC for the header part of the buffer */
 crc = RMAP_CalculateCRC(pBuffer, HEADER_LEN);

 printf("The CRC of the first %d bytes of the buffer is 0x%x\n", HEADER_LEN,
 crc);

 /* Calculate a CRC for the entire buffer, */
 /* using the CRC already calculated as a seed */
 crc = RMAP_CalculateCRCWithSeed(pBuffer + HEADER_LEN,
 BUFFER_LEN - HEADER_LEN, crc);

 printf("The CRC of the full buffer is 0x%x\n", crc);

 /* Check the CRC is correct for the buffer */
 printf("The CRC of the buffer is %svalid\n",
 RMAP_IsCRCValid(pBuffer, BUFFER_LEN, crc) ? "" : "not ");
}

```

## 9.16 device\_identifier\_example.c

This example demonstrates the use of the device identifier functions.

```

#include <stdio.h>
#include "star-api.h"
#include "cfg_api_generic.h"

STAR_DEVICE_ID chooseStarDevice()
{
 STAR_DEVICE_ID *devices;

```

```

unsigned int devCount = 0, chosen, i, status;
STAR_DEVICE_ID deviceID;
char *deviceName, s[256];

/* Get the list of devices present which can be configured */
devices = STAR_getDeviceListForType(
 STAR_DEVICE_CONFIG_SUPPORTED,
 &devCount);

/* If there was an error getting the list of devices */
if (!devices)
{
 /* Return an error */
 puts("No SpaceWire devices detected!");
 return 0;
}

/* Display the devices detected */
if (devCount == 1)
{
 puts("One SpaceWire device detected:");
}
else
{
 printf("%u SpaceWire devices detected:", devCount);
}

/* For each device */
for (i = 0; i < devCount; i++)
{
 /* Display its name */
 deviceName = STAR_getDeviceName(devices[i]);
 if (deviceName)
 {
 printf("\t%u - %s\n", i, deviceName);
 STAR_destroyString(deviceName);
 }
 else
 {
 printf("\t%u - Unknown SpaceWire Device\n", i);
 }
}

/* If there's only 1 device on the list */
if (devCount == 1)
{
 /* Return that device */
 deviceID = devices[0];
 STAR_destroyDeviceList(devices);
 return deviceID;
}

/* Ask the user which device to use */
printf("\nPlease select which device to open: ");
fgets(s, 256, stdin);
status = sscanf(s, "%d", &chosen);
if ((!status) || (chosen > devCount - 1))
{
 puts("Incorrect device number selected.");
 STAR_destroyDeviceList(devices);
 return 0;
}

/* Return the chosen device */
deviceID = devices[chosen];
STAR_destroyDeviceList(devices);
return deviceID;
}

int __cdecl main()
{
 STAR_DEVICE_ID deviceID;
 STAR_CFG_DEVICE_IDENTIFIER_INFO deviceIdentifierInfo;
 STAR_CFG_NETWORK_DISCOVERY_INFO networkInfo;
 char manufacturerStr[STAR_CFG_MANUFACTURER_STR_MAX_LEN];
 char deviceStr[STAR_CFG_DEVICE_STR_MAX_LEN];
 int i;

 /* Select device to configure */
 deviceID = chooseStarDevice();

 /* Get the router Identification Information */
 CFG_getDeviceIdentificationInfo(deviceID, &deviceIdentifierInfo);

 /* Display the Identification Information*/
 printf("\nManufacturer ID:\t%u", deviceIdentifierInfo.manufacturerID);

```



```

printf("\nChip ID:\t%u", deviceIdentifierInfo.chipType);
printf("\nVersion:\t%u", deviceIdentifierInfo.versionNum);

/* Display parsed information */
CFG_getDeviceManufacturerAsString(&deviceIdentifierInfo,
 manufacturerStr);
printf("\nManufacturer Name:\t%s", manufacturerStr);

CFG_getDeviceTypeAsString(&deviceIdentifierInfo, deviceStr);
printf("\nDevice Type:\t%s", deviceStr);

/* Get Device network information */
CFG_getNetworkDiscoveryInfo(deviceID, &networkInfo);

/* Display device network information */
switch(networkInfo.deviceType)
{
case STAR_CFG_DEVICE_TYPE_ROUTER:
 printf("\nDevice Type:\tRouter");
 break;
case STAR_CFG_DEVICE_TYPE_UNKNOWN:
 printf("\nDevice Type:\tUnknown");
 break;
case STAR_CFG_DEVICE_TYPE_INVALID:
 printf("\nDevice Type:\tInvalid");
 break;
}

printf("\nNumber of Ports (Total/Running):\t%u/%u", networkInfo.portCount, networkInfo.
 runningPortsCount);
printf("\nRunning port numbers: ");
/* Test each bit in mask to see if it is set */
for(i = 1; i < 32; i++)
{
 if((1 << i) & networkInfo.runningPortsMask)
 printf(" %d", i);
}

printf("\nReturn Port:\t%u", networkInfo.returnPort);

return 0;
}

```

## 9.17 driver\_list\_example.c

Gets a list of available drivers using `STAR_getDriverList()` then iterates over them and prints the type of driver.

```

#include "examples.h"

#include "utilities.h"

#include "star-api.h"

void printDriverType(STAR_DRIVER_TYPE driverType)
{
 /* Switch on driver type */
 switch(driverType)
 {
 /* Case Invalid */
 case STAR_DRIVER_INVALID:
 /* Print invalid driver */
 printf("Invalid driver detected.\n");

 break;

 /* Case USB */
 case STAR_DRIVER_TYPE_USB:
 /* Print USB driver type */
 printf("USB driver detected.\n");

 break;

 /* Case PCI */
 case STAR_DRIVER_TYPE_PCI:
 /* Print PCI driver type */
 printf("PCI driver detected.\n");

 break;

 /* Case TCP/IP */
 case STAR_DRIVER_TYPE_TCPIP:
 /* Print TCP/IP driver type */

```

```

 printf("TCP/IP driver detected.\n");

 break;
 /* Case Virtual */
 case STAR_DRIVER_TYPE_VIRTUAL:
 /* Print virtual driver type */
 printf("Virtual driver detected.\n");

 break;
 }
}

void driverListExample()
{
 /* The number of drivers that were found */
 U32 driverCount = 0;

 /* The current driver index */
 unsigned int index;

 /* Get list of drivers */
 STAR_DRIVER_ID * drivers = STAR_getDriverList(1, 1, &driverCount);

 /* For all drivers */
 for(index = 0; (drivers != NULL) && (index < driverCount); index++)
 {
 /* Get driver type */
 STAR_DRIVER_TYPE driverType = STAR_getDriverType(drivers[index]);

 /* Print driver type */
 printDriverType(driverType);
 }

 /* Destroy device list */
 STAR_destroyDeviceList(drivers);
}

```

## 9.18 firmware\_version\_example.c

Gets a list of available devices using `STAR_getDeviceList()` and then iterates over them. The firmware version for each device is retrieved and printed.

```

#include "examples.h"

#include "utilities.h"

#include "star-dundee_types.h"
#include "star-api.h"

void firmwareVersionExample()
{
 /* Initialise device count to 0 */
 U32 deviceCount = 0;

 /* Counter for loop */
 unsigned int index;

 /* Get device list */
 STAR_DEVICE_ID *devices = STAR_getDeviceList(&deviceCount);

 /* For all devices */
 for(index = 0; (devices != NULL) && (index < deviceCount); index++)
 {
 /* Get firmware version */
 STAR_VERSION_INFO * firmwareVersion =
 STAR_getDeviceFirmwareVersion(
 devices[index]);

 /* Print firmware version label */
 printf("Firmware version:");

 /* Print firmware version */
 printVersionInfo(firmwareVersion);
 }

 /* Destroy device list */
 STAR_destroyDeviceList(devices);
}

```

## 9.19 link\_events.c

This file contains example code demonstrating how to receive link speed and state change events.

```
#include <stdio.h>

#include "star-api.h"
#include "cfg_api_mk2.h"
#include "cfg_api_brick_mk2.h"
#include "cfg_api_router.h"
#include "cfg_api_brick_mk3.h"
#include "cfg_api_pxi.h"
#include "cfg_api_pci_mk2.h"
#include "cfg_api_generic.h"
#include "ui.h"

//-----

#define TEST_ERROR_STATUS(Status,sText,sError) \
if (Status == 0) \
{ \
 HandleError(sError); \
 return (1); \
} \
else \
{ \
 printf("%s", sText); \
}

#define TEST_FUNC_STATUS(Status,sText,sError) \
if (Status == 0) \
{ \
 printf("%s", sError); \
 return (1); \
} \
else \
{ \
 printf("%s", sText); \
}

#define TEST_SUB_FUNC_STATUS(Status,sText,sError) \
if (Status == 0) \
{ \
 printf("%s", sError); \
 return (0); \
} \
else \
{ \
 printf("%s", sText); \
}

#define TEST_GENERIC_ERROR_STATUS(Status,sText,sError) \
if (!CFG_SUCCESS(Status)) \
{ \
 HandleError(sError); \
 return (1); \
} \
else \
{ \
 printf("%s", sText); \
}

#define TEST_GENERIC_FUNC_STATUS(Status,sText,sError) \
if (!CFG_SUCCESS(Status)) \
{ \
 printf("%s", sError); \
 return (1); \
} \
else \
{ \
 printf("%s", sText); \
}

#define TEST_GENERIC_SUB_FUNC_STATUS(Status,sText,sError) \
if (!CFG_SUCCESS(Status)) \
{ \
 printf("%s", sError); \
 return (0); \
} \
else \
{ \
 printf("%s", sText); \
}
```

```

//-----

#ifdef _WIN32
#include <windows.h>
#elif (defined(__linux__) || defined(__CYGWIN__) || defined(__QNX__) || defined(__rtems__) ||
 defined(__vxworks))
#include <pthread.h>
#include <unistd.h>
#endif

//-----

void DelayMS(U32 _DelayMS)
{
#ifdef _WIN32
 Sleep(_DelayMS);
#elif (defined(__linux__) || defined(__CYGWIN__) || defined(__QNX__) || defined(__rtems__))
 usleep(_DelayMS * 1000);
#elif defined(__vxworks)
 taskDelay(_DelayMS);
#endif
}

//-----

void HandleError(const char* _psText)
{
 printf("%s", _psText);
 printf("\nPress Enter to exit...\n");
 getchar();
}

//-----

int SetLinkSpeed(STAR_DEVICE_ID _DeviceID, U8 _Port, U8 _Divisor)
{
 int Status = CFG_setTransmitSignallingRate(_DeviceID, _Port, 200 /
 _Divisor);
 TEST_GENERIC_SUB_FUNC_STATUS(Status, "", "");
 return (Status);
}

//-----

int PrintLinkSpeed(STAR_DEVICE_ID _DeviceID, U8 _TxPort, U8 _RxPort, const char* _psText)
{
 U16 LinkSpeed;
 int Status = CFG_getMeasuredLinkSpeed(_DeviceID, _RxPort, &LinkSpeed);
 if (Status >= 0)
 {
 U32 TempLinkSpeed = (LinkSpeed * STAR_CFG_MK2_LINK_SPEED_UNITS_KBPS
);
 printf(_psText, _TxPort, _RxPort, _RxPort, TempLinkSpeed / 1000, (TempLinkSpeed % 1000) / 100);
 }
 return (Status);
}

//-----

STAR_DEVICE_ID GetRemoteDeviceID(STAR_DEVICE_ID _DeviceID,
 U8 _RemoteChannel)
{
 unsigned char aPath[] = { 0, 254 };
 unsigned char aRetPath[] = { 254 };

 STAR_SPACEWIRE_ADDRESS* pPath = STAR_createAddress(aPath,
 sizeof(aPath));
 STAR_SPACEWIRE_ADDRESS* pRetPath = STAR_createAddress(aRetPath,
 sizeof(aRetPath));

 STAR_DEVICE_ID RemoteDeviceID =
 STAR_CFG_createRemoteDeviceIdentifier(
 _DeviceID, _RemoteChannel, NULL, pPath, pRetPath);

 STAR_destroyAddress(pRetPath);
 STAR_destroyAddress(pPath);

 return (RemoteDeviceID);
}

//-----

int EnableLinkEvents(STAR_DEVICE_ID _DeviceID, U8 _Port)
{
 int Status = CFG_enableSpeedChangeEventsOnPort(_DeviceID, _Port);
 TEST_GENERIC_SUB_FUNC_STATUS(Status, "", "");
 Status = CFG_enableStateChangeEventsOnPort(_DeviceID, _Port);
}

```

```

 TEST_GENERIC_SUB_FUNC_STATUS(Status, "", "");
 return (Status);
}

//-----

int DisableLinkEvents(STAR_DEVICE_ID _DeviceID, U8 _Port)
{
 int Status = CFG_disableSpeedChangeEventsOnPort(_DeviceID, _Port);
 TEST_GENERIC_SUB_FUNC_STATUS(Status, "", "");
 Status = CFG_disableStateChangeEventsOnPort(_DeviceID, _Port);
 TEST_GENERIC_SUB_FUNC_STATUS(Status, "", "");
 return (Status);
}

//-----

int WaitEvents(STAR_CHANNEL_ID _ChannelID, STAR_TRANSFER_OPERATION*
 _pLinkEventRxOperation)
{
 int Status = 0;
 U32 Index;

 STAR_TRANSFER_STATUS TransferOperationStatus;
 do
 {
 TransferOperationStatus = STAR_waitOnTransferOperationCompletion
 (_pLinkEventRxOperation, 2000);
 switch (TransferOperationStatus)
 {
 case STAR_TRANSFER_STATUS_COMPLETE:
 {
 U32 ItemCount = STAR_getTransferItemCount(
 _pLinkEventRxOperation);
 for (Index=0; Index<ItemCount; Index++)
 {
 STAR_STREAM_ITEM* pLinkEvent =
 STAR_getTransferItem(_pLinkEventRxOperation, Index);
 STAR_STREAM_ITEM_TYPE EventType = (
 STAR_STREAM_ITEM_TYPE)pLinkEvent->itemType;
 if (EventType == STAR_STREAM_ITEM_TYPE_LINK_STATE_EVENT
)
 {
 STAR_LINK_STATE_EVENT* pLinkStateEvent = (
 STAR_LINK_STATE_EVENT*)pLinkEvent->item;
 U8 Port = STAR_getLinkStateEventPort(pLinkStateEvent);

 int bIsLinkRunning = STAR_isLinkStateEventLinkRunning
 (pLinkStateEvent);
 if (bIsLinkRunning == FALSE)
 {
 printf("\nPort %i state event: Link not running.", Port);
 }
 else
 {
 printf("\nPort %i state event: Link running.", Port);
 }
 }
 else if (EventType == STAR_STREAM_ITEM_TYPE_LINK_SPEED_EVENT
)
 {
 STAR_LINK_SPEED_EVENT* pLinkSpeedEvent = (
 STAR_LINK_SPEED_EVENT*)pLinkEvent->item;
 U8 Port = STAR_getLinkSpeedEventPort(pLinkSpeedEvent);
 U32 NewLinkSpeed = STAR_getLinkSpeedEventSpeed(
 pLinkSpeedEvent);
 printf("\nPort %i speed event: Link speed %u.%u Mbit/s", Port, NewLinkSpeed / 1000
 000, (NewLinkSpeed % 1000000) / 100000);
 }
 break;
 }
 }
 case STAR_TRANSFER_STATUS_STARTED:
 {
 printf("\nNo more events\n");
 Status = STAR_cancelTransferOperation(_pLinkEventRxOperation);
 TEST_SUB_FUNC_STATUS(Status, "", "");
 break;
 }
 case STAR_TRANSFER_STATUS_NOT_STARTED:
 {
 printf("\nXfer not started and link speed event did not occur");
 Status = STAR_cancelTransferOperation(_pLinkEventRxOperation);
 TEST_SUB_FUNC_STATUS(Status, "", "");
 break;
 }
 case STAR_TRANSFER_STATUS_CANCELLED:

```

```

 {
 //printf("\nCancelled");
 break;
 }
 case STAR_TRANSFER_STATUS_ERROR:
 default:
 {
 printf("\nError occurred while waiting for link speed event");
 break;
 }
 }

 if (TransferOperationStatus != STAR_TRANSFER_STATUS_STARTED)
 {
 Status = STAR_submitTransferOperation(_ChannelID,
 _pLinkEventRxOperation);
 TEST_SUB_FUNC_STATUS(Status, "", "\nSTAR_submitTransferOperation Status = 0.");
 }
 while (TransferOperationStatus != STAR_TRANSFER_STATUS_STARTED);
 return (Status);
}

//-----
void PrintDeviceType(STAR_DEVICE_ID _DeviceID)
{
 char* psDeviceType = STAR_getDeviceTypeAsString(_DeviceID);
 if (psDeviceType == NULL)
 {
 puts("Testing an unknown device type");
 }
 else
 {
 printf("\nTesting the %s\n", psDeviceType);
 STAR_destroyString(psDeviceType);
 }
}

//-----
int DisableTriState(STAR_DEVICE_ID _DeviceID, U8 _Port)
{
 PORT_STATUS_CONTROL PortStatusControl;
 STAR_CFG_SPW_LINK_STATUS LinkStatus;

 int Status = CFG_getPortStatusControl(_DeviceID, _Port, &PortStatusControl);
 TEST_GENERIC_SUB_FUNC_STATUS(Status, "", "");

 Status = CFG_getSpaceWireLinkStatus(PortStatusControl, &LinkStatus);
 TEST_GENERIC_SUB_FUNC_STATUS(Status, "", "");

 LinkStatus.triState = FALSE;
 Status = CFG_setSpaceWireLinkStatus(_DeviceID, _Port, &LinkStatus);
 TEST_GENERIC_SUB_FUNC_STATUS(Status, "", "");

 return (Status);
}

//-----
int bAreLinkEventsOnChannel0Only(U32 _DeviceType)
{
 if ((_DeviceType == STAR_DEVICE_BRICK_MK2) ||
 (_DeviceType == STAR_DEVICE_BRICK_MK3) ||
 (_DeviceType == STAR_DEVICE_ROUTER_MK2S) ||
 (_DeviceType == STAR_DEVICE_SPLT) ||
 (_DeviceType == STAR_DEVICE_STAR_FIRE_MK3))
 {
 return (TRUE);
 }
 return (FALSE);
}

//-----
#define NUM_DEVICE_TYPES (14)

int __cdecl main()
{
 STAR_DEVICE_ID DeviceID;
 STAR_DEVICE_ID RemoteDeviceID;
 STAR_CHANNEL_ID ChannelID;
 STAR_CFG_FPGA_INFO FPGAInfo;
 STAR_TRANSFER_OPERATION* pLinkEventRxOperation;
 STAR_CHANNEL_MASK ChannelMask;

```

```

STAR_CHANNEL_MASK UsableChannelsMask;

U32 DeviceType;
int Status;
U8 TxPort;
U8 RxPort;
U8 Channel;
U8 RemoteChannel;
U32 Index;
char sBuffer[96];

char sVersion[STAR_CFG_MK2_VERSION_STR_MAX_LEN];
char sBuildDate[STAR_CFG_MK2_BUILD_DATE_STR_MAX_LEN];

STAR_DEVICE_TYPE aDeviceTypes[NUM_DEVICE_TYPES];
aDeviceTypes[0] = STAR_DEVICE_BRICK_MK2;
aDeviceTypes[1] = STAR_DEVICE_ROUTER_MK2S;
aDeviceTypes[2] = STAR_DEVICE_BRICK_MK3;
aDeviceTypes[3] = STAR_DEVICE_PCIE;
aDeviceTypes[4] = STAR_DEVICE_PXI_INTERFACE;
aDeviceTypes[5] = STAR_DEVICE_PXI_RMAP;
aDeviceTypes[6] = STAR_DEVICE_PXI_ROUTER_12;
aDeviceTypes[7] = STAR_DEVICE_SPLT;
aDeviceTypes[8] = STAR_DEVICE_PCI_MK2;
aDeviceTypes[9] = STAR_DEVICE_CPCI_MK2;
aDeviceTypes[10] = STAR_DEVICE_STAR_FIRE_MK3;
aDeviceTypes[11] = STAR_DEVICE_PXI_INTERFACE_MK2;
aDeviceTypes[12] = STAR_DEVICE_PXI_RMAP_MK2;
aDeviceTypes[13] = STAR_DEVICE_PXI_ROUTER_MK2;

STAR_setApplicationName("STAR-System Link Events Example Application");

// Select device to configure
DeviceID = STAR_UI_chooseDevice(aDeviceTypes, NUM_DEVICE_TYPES);
TEST_ERROR_STATUS(DeviceID, "", "\nChosen device has invalid ID");

DeviceType = STAR_getDeviceType(DeviceID);
TEST_ERROR_STATUS(DeviceType, "", "\nCan't get device type");

PrintDeviceType(DeviceID);

// Get hardware info
Status = CFG_getFPGAInfo(DeviceID, &FPGAInfo);
TEST_GENERIC_ERROR_STATUS(Status, "", "\nCan't get device FPGA Info");
CFG_FPGAInfoToString(DeviceID, &FPGAInfo, sVersion, sBuildDate);

// Display the hardware info
printf("\nVersion: %s", sVersion);
printf("\nBuildDate: %s\n", sBuildDate);

if (DeviceType == STAR_DEVICE_PCIE)
{
 if (!((FPGAInfo.major > 1) || ((FPGAInfo.major == 1) && (FPGAInfo.minor >= 11))))
 {
 TEST_ERROR_STATUS(0, "", "\nPlease upgrade this device's firmware\n");
 }
}
else if ((DeviceType == STAR_DEVICE_PCI_MK2) ||
 (DeviceType == STAR_DEVICE_CPCI_MK2))
{
 if (!((FPGAInfo.major > 1) || ((FPGAInfo.major == 1) && (FPGAInfo.minor >= 10))))
 {
 TEST_ERROR_STATUS(0, "", "\nPlease upgrade this device's firmware\n");
 }
}
else if ((DeviceType == STAR_DEVICE_PXI_INTERFACE) ||
 (DeviceType == STAR_DEVICE_PXI_RMAP) ||
 (DeviceType == STAR_DEVICE_PXI_ROUTER_12))
{
 if (!((FPGAInfo.major > 1) || ((FPGAInfo.major == 1) && (FPGAInfo.minor >= 1))))
 {
 TEST_ERROR_STATUS(0, "", "\nPlease upgrade this device's firmware\n");
 }
}

//----
Status = CFG_disableInterfaceMode(DeviceID);
TEST_GENERIC_ERROR_STATUS(Status, "", "\nCan't disable interface mode");
DelayMS(3000); // Waiting for Config Service to close
channel 0

ChannelMask = STAR_getDeviceChannels(DeviceID);
UsableChannelsMask = (ChannelMask >> 1); // Step over channel 0

```

```

RemoteChannel = 0;

/* To receive link speed events on some devices, we must open a connection
to device channel 0. This will prevent local configuration operations
from being performed until we close this channel again, so we create a
remote device for configuration operations, using a different channel, if
necessary. */
if (bAreLinkEventsOnChannel0Only(DeviceType) == TRUE)
{
 Channel = 0;
 RemoteChannel = 1;

 RemoteDeviceID = GetRemoteDeviceID(DeviceID, RemoteChannel);
 TEST_ERROR_STATUS(RemoteDeviceID, "", "\nCan't get remote device ID");
}
else
{
 RemoteDeviceID = DeviceID;
}

Index = 1;
while (UsableChannelsMask != 0)
{
 UsableChannelsMask = (UsableChannelsMask >> 1);

 if ((Index % 2) == 1)
 {
 TxPort = Index;
 RxPort = (Index + 1);
 }
 else
 {
 TxPort = Index;
 RxPort = (Index - 1);
 }
 Index++;

 if (bAreLinkEventsOnChannel0Only(DeviceType) == FALSE)
 {
 Channel = RxPort;
 }

 printf("\n\nTx port %d, Rx port %d, Channel %d\n", TxPort, RxPort, Channel);

 if ((DeviceType == STAR_DEVICE_PXI_INTERFACE) ||
 (DeviceType == STAR_DEVICE_PXI_RMAP) ||
 (DeviceType == STAR_DEVICE_PXI_ROUTER_12) ||
 (DeviceType == STAR_DEVICE_PXI_INTERFACE_MK2) ||
 (DeviceType == STAR_DEVICE_PXI_RMAP_MK2) ||
 (DeviceType == STAR_DEVICE_PXI_ROUTER_MK2))
 {
 DisableTriState(RemoteDeviceID, TxPort);
 DisableTriState(RemoteDeviceID, RxPort);
 }

 Status = EnableLinkEvents(RemoteDeviceID, RxPort); // Enable Link Speed and Link State Events
 TEST_GENERIC_ERROR_STATUS(Status, "", "\nCan't enable link events");

 ChannelID = STAR_openChannelToLocalDevice(DeviceID,
STAR_CHANNEL_DIRECTION_IN, Channel, TRUE);
 TEST_ERROR_STATUS(ChannelID, "", "\nCan't open channel to local device");

 pLinkEventRxOperation = STAR_createRxOperation(1, (
STAR_RECEIVE_MASK)(STAR_RECEIVE_LINK_STATE_EVENTS |
STAR_RECEIVE_LINK_SPEED_EVENTS));
 TEST_ERROR_STATUS(pLinkEventRxOperation, "", "\nSTAR_createRxOperation returned NULL pointer");

 Status = STAR_submitTransferOperation(ChannelID, pLinkEventRxOperation);
 TEST_ERROR_STATUS(Status, "", "\nCan't submit transfer operation");

 Status = CFG_startLink(RemoteDeviceID, TxPort); // Remote in to start link
 sprintf(sBuffer, "\nStart link from port %i to port %i", TxPort, RxPort);
 TEST_GENERIC_ERROR_STATUS(Status, sBuffer, "\nCan't start link");

 Status = WaitEvents(ChannelID, pLinkEventRxOperation);
 TEST_ERROR_STATUS(Status, "", "\nWaitEvents failed");

 Status = SetLinkSpeed(RemoteDeviceID, TxPort, 2);
 sprintf(sBuffer, "\nSet port %i tx speed : 100 Mbit/s", TxPort);
 TEST_GENERIC_ERROR_STATUS(Status, sBuffer, "\nCan't set link speed");

 Status = WaitEvents(ChannelID, pLinkEventRxOperation);
 TEST_ERROR_STATUS(Status, "", "\nWaitEvents failed");

 Status = SetLinkSpeed(RemoteDeviceID, TxPort, 1);
 sprintf(sBuffer, "\nSet port %i tx speed : 200 Mbit/s", TxPort);
 TEST_GENERIC_ERROR_STATUS(Status, sBuffer, "\nCan't set link speed");
}

```



```

 Status = WaitEvents(ChannelID, pLinkEventRxOperation);
 TEST_ERROR_STATUS(Status, "", "\nWaitEvents failed");

 Status = STAR_disposeTransferOperation(pLinkEventRxOperation);
 TEST_ERROR_STATUS(Status, "", "\nCan't dispose transfer operation");

 Status = STAR_closeChannel(ChannelID);
 TEST_ERROR_STATUS(Status, "", "\nCan't close channel");

 Status = DisableLinkEvents(RemoteDeviceID, RxPort); // Disable Link Speed and Link State Events
 TEST_GENERIC_ERROR_STATUS(Status, "", "\nCan't disable link events");

 Status = PrintLinkSpeed(RemoteDeviceID, TxPort, RxPort, "\nLink from port %i to port %i running.
 Link speed measured at port %i : %d.%d Mbit/s");
 TEST_GENERIC_ERROR_STATUS(Status, "", "\nCan't measure link speed");

 Status = CFG_stopLink(RemoteDeviceID, TxPort);
 TEST_GENERIC_ERROR_STATUS(Status, "", "\nCan't stop link");
 DelayMS(1000); // Wait for link to stop

 Status = PrintLinkSpeed(RemoteDeviceID, TxPort, RxPort, "\nLink from port %i to port %i stopped.
 Link speed measured at port %i : %d.%d Mbit/s");
 TEST_GENERIC_ERROR_STATUS(Status, "", "\nCan't measure link speed");
}

if (RemoteChannel != 0)
{
 Status = STAR_CFG_destroyRemoteDeviceIdentifier(
 RemoteDeviceID);
 TEST_ERROR_STATUS(Status, "", "\nCan't destroy remote device ID");
}

Status = CFG_enableInterfaceMode(DeviceID);
TEST_GENERIC_ERROR_STATUS(Status, "", "\nCan't enable interface mode");

printf("\n\nPress Enter to exit...\n");
getchar();
return (0);
}

//-----

```

## 9.20 mk2\_configuration\_example.c

Example usage of this API.

```

#include <stdio.h>
#include "star-api.h"
#include "cfg_api_generic.h"

#if defined(__rtems__)
#include <bsp.h>
#include <pthread.h>
#include "cfg_service_api.h"

int __cdecl main();

void *POSIX_Init(void *pArgs)
{
 UNREFERENCED_PARAMETER(pArgs);

 /* Start the Config Service */
 STAR_runConfigService();

 main();

 exit(0);
 return NULL;
}

#define CONFIGURE_APPLICATION_NEEDS_CONSOLE_DRIVER
#define CONFIGURE_APPLICATION_NEEDS_CLOCK_DRIVER

#define CONFIGURE_OBJECTS_UNLIMITED
#define CONFIGURE_UNIFIED_WORK_AREAS

#define CONFIGURE_MAXIMUM_POSIX_THREADS rtems_resource_unlimited(4)
#define CONFIGURE_MAXIMUM_POSIX_MUTEXES rtems_resource_unlimited(4)
#define CONFIGURE_MAXIMUM_POSIX_SEMAPHORES rtems_resource_unlimited(4)

```

```

#define CONFIGURE_POSIX_INIT_THREAD_TABLE

#define CONFIGURE_INIT
#include <rtems/confdefs.h>

#include <drvmgr/drvmgr_confdefs.h>
#endif

STAR_DEVICE_ID chooseStarDevice()
{
 STAR_DEVICE_ID *devices;
 unsigned int devCount = 0, chosen, i, status;
 STAR_DEVICE_ID deviceID;
 char *deviceName, s[256];

 /* Get the list of devices present which can be configured */
 devices = STAR_getDeviceListForType(
 STAR_DEVICE_CONFIG_SUPPORTED,
 &devCount);

 /* If there was an error getting the list of devices */
 if (!devices)
 {
 /* Return an error */
 puts("No SpaceWire devices detected!");
 return 0;
 }

 /* Display the devices detected */
 if (devCount == 1)
 {
 puts("One SpaceWire device detected:");
 }
 else
 {
 printf("%u SpaceWire devices detected:", devCount);
 }

 /* For each device */
 for (i = 0; i < devCount; i++)
 {
 /* Display its name */
 deviceName = STAR_getDeviceName(devices[i]);
 if (deviceName)
 {
 printf("\t%u - %s\n", i, deviceName);
 STAR_destroyString(deviceName);
 }
 else
 {
 printf("\t%u - Unknown SpaceWire Device\n", i);
 }
 }

 /* If there's only 1 device on the list */
 if (devCount == 1)
 {
 /* Return that device */
 deviceID = devices[0];
 STAR_destroyDeviceList(devices);
 return deviceID;
 }

 /* Ask the user which device to use */
 printf("\nPlease select which device to open: ");
 fgets(s, 256, stdin);
 status = sscanf(s, "%d", &chosen);
 if ((!status) || (chosen > devCount - 1))
 {
 puts("Incorrect device number selected.");
 STAR_destroyDeviceList(devices);
 return 0;
 }

 /* Return the chosen device */
 deviceID = devices[chosen];
 STAR_destroyDeviceList(devices);
 return deviceID;
}

int __cdec1 main()
{
 STAR_DEVICE_ID deviceID;
 STAR_CFG_FPGA_INFO fpgaInfo;
 char versionStr[STAR_CFG_MK2_VERSION_STR_MAX_LEN];
 char buildDateStr[STAR_CFG_MK2_BUILD_DATE_STR_MAX_LEN];

```

```

/* Select device to configure */
deviceID = chooseStarDevice();
if (!deviceID)
{
 return 0;
}

/* Get hardware info*/
CFG_getFPGAInfo(deviceID, &fpgaInfo);

CFG_FPGAInfoToString(deviceID, &fpgaInfo, versionStr, buildDateStr);

/* Display the hardware info*/
printf("\nVersion: %s", versionStr);
printf("\nBuildDate: %s", buildDateStr);

/* Set general purpose register to 0xABCD */
CFG_setGeneralPurpose(deviceID, 0xABCD);

/* Flash the devices LEDs*/
CFG_identify(deviceID);

/* Set the device to be a time-code master*/
CFG_enableTimeCodeMaster(deviceID);

/* Set 2 second delay between time-codes */
CFG_setTimeCodePeriod(deviceID, 2000000);

/* Enable interface mode */
CFG_enableInterfaceMode(deviceID);

/* Disable interface mode on port 2 only */
CFG_disableInterfaceModeOnPort(deviceID, 2);

/* Enable adding the source port number as a leading byte
 to received packets on ports 1 and 3 */

CFG_enableIdentifySource(deviceID);
CFG_enableIdentifySourceOnPort(deviceID, 1);
CFG_enableIdentifySourceOnPort(deviceID, 3);

/* Set port 2 to have a base transmit rate of 150 Mbit/s */
CFG_setTransmitSignallingRate(deviceID, 2, 150);

/* Inject a disconnect error on link 1 */
CFG_injectError(deviceID, 1, SPW_ERROR_DISCONNECT);

return 0;
}

```

## 9.21 packet\_subsystem\_example.c

Example usage of this API.

```

#include "packet_subsystem.h"
#include "cfg_api_generic.h"
#include "ui.h"
#include <stdlib.h>
#include <stdio.h>

#define VERSION_INFO "PCI Mk2 Packet Subsystem Test Application"

/* Currently the PCI Mk2 only contains a single packet subsystem on port 8*/
#define PACKET_SUBSYSTEM_NUMBER 1
#define SUBSYSTEM_PORT_START 7

#define MIN(x, y) ((x) < (y)) ? (x) : (y)

#if !defined(UNREFERENCED_PARAMETER)
#define UNREFERENCED_PARAMETER(a) ((void) (a))
#endif

void printVersionInformation()
{
 puts(VERSION_INFO);
 puts("Copyright (C) 2012 STAR-Dundee Ltd.");
 puts("http://www.star-dundee.com/ \n");
}

```

```

int runTest(STAR_DEVICE_ID generatorDevice,
 STAR_DEVICE_ID checkerDevice)
{
 /* Buffer that will be filled with bytes to be used for generated packet headers */
 U8 *pHeaderBuffer;

 /* Number of header bytes to use */
 U16 headerBytes = 2;
 /* Determine how many words header bytes occupy */
 U16 headerWords = (headerBytes + 3) / 4;

 /* Sizes for expected headerdata used for checker*/
 U16 expectedHeaderWords = 0;

 /* Buffer that will be filled with bytes to be used for generated packet bodies */
 U8 *pBodyBuffer;

 /* Number of 4 byte words to use when generating packet bodies */
 U16 bodyWords = 12;

 /* Size of packets to be generated. Generated packets will be made up of the specified
 header bytes, followed by the body bytes specified repeating as necessary to make
 the packet up to the length specified.*/
 U16 packetLen = 100 + headerBytes;

 /* Size of expected packets received by the packet checker. Header deletion by the router
 will strip the path address. */
 U16 expectedPacketLen = packetLen - headerBytes;

 /* Here we decide on the the physical location in the device dual port memory to use
 for the packet generator and checker information, and for the sink's receive buffer.
 These locations and the order we specify them are arbitrary. */

 /* We decide to put the header data at address 0 */
 U16 memHeaderGenStart = 0;

 /* We put the body data after the end of the header data */
 U16 memBodyGenStart = memHeaderGenStart + headerWords;

 /* Put the checker memory after the end of the generator data */
 U16 memHeaderCheckerStart = memBodyGenStart + bodyWords;
 U16 memBodyCheckerStart = memHeaderCheckerStart + expectedHeaderWords;

 /* Put the sink buffer after the checker memory */
 U16 memSinkStart = memBodyCheckerStart + bodyWords;

 U16 sinkLenWords;

 /* Number of bytes to wait before turning off sink after detecting a character mismatch */
 U16 sinkDelayOnMismatch = 16;

 /* Count of byte mismatches caught by the packet checker */
 U64 mismatchCount = 0;

 /* Circular buffer pointers used when reading data back from the packet sink */
 U16 startPointer = 0 , endPointer = 0;

 /* Buffer that will be filled with data read back from the packet sink*/
 U8* pReadBackBuffer;

 /* Values used when displaying results*/
 unsigned int offset= 0, address = 0;

 /* Create local buffers which we will copy to the devices on board memory */
 pHeaderBuffer = (U8*)malloc(headerWords * 4);
 if(!pHeaderBuffer)
 {
 return 0;
 }
 pBodyBuffer = (U8*)malloc(bodyWords * 4);
 if(!pBodyBuffer)
 {
 free(pHeaderBuffer);
 return 0;
 }

 printVersionInformation();

 /* In this example (with a cable looped back from port 1 to port 3) we want the packet
 to have a path address of [1, 8]. 1 to send packet out of port one, 8 to receive the
 packet back at the packet subsystem.*/
 *pHeaderBuffer = 1;
 *(pHeaderBuffer + 1) = SUBSYSTEM_PORT_START + PACKET_SUBSYSTEM_NUMBER;

 /* Fill the packet body with a repeating pattern*/
 PKT_SUBSYS_fillBufferWithPattern(PKT_SUBSYS_PATTERN_ALTERNATING_BITS,
 bodyWords * 4, pBodyBuffer);

```

```

/* Write packet header and body to device memory */
PKT_SUBSYS_writeMemory(generatorDevice, memHeaderGenStart, headerWords * 4,
 pHeaderBuffer);
PKT_SUBSYS_writeMemory(generatorDevice, memBodyGenStart, bodyWords * 4,
 pBodyBuffer);
free(pHeaderBuffer);
pHeaderBuffer = NULL;

/* Write expected packet body to checker device memory. As we don't expect the packet body to change
 we can just use the same buffer as before.
 Note that header deletion will strip the path address, so we will not expect to see a header*/
PKT_SUBSYS_writeMemory(checkerDevice, memBodyCheckerStart, bodyWords * 4,
 pBodyBuffer);
free(pBodyBuffer);
pBodyBuffer = NULL;

/* Instruct subsystem A packet generator to use supplied buffers */
PKT_SUBSYS_setPacketGeneratorFormat(generatorDevice,
 PACKET_SUBSYSTEM_NUMBER,
 memHeaderGenStart,
 headerBytes,
 memBodyGenStart,
 bodyWords,
 packetLen,
 STAR_EOP_TYPE_EOP,
 0);

/* Set up a packet sink on subsystem B to receive generated packets:
 We set up circular buffer with room for 20 packets,
 ie sink size in words = ((20 packets * (packetLen in bytes + 2 bytes for end of packet marker) +
 3) / 4
 Note that 3 is added to ensure there are enough words if (packet length + EOP) is not a multiple of
 4
*/
sinkLenWords = ((20 * (expectedPacketLen+2)) + 3) / 4;
PKT_SUBSYS_setPacketSinkParameters(checkerDevice,
 PACKET_SUBSYSTEM_NUMBER, memSinkStart, sinkLenWords);

/* Set up the packet checker on subsystem B to check the received generated packets
 Here we just re-use the buffers already written to device to generate packets from.
 As we do not expect a packet header, we set its length to 0.*/
PKT_SUBSYS_setPacketCheckingFormat(checkerDevice,
 PACKET_SUBSYSTEM_NUMBER,
 memHeaderCheckerStart,
 0,
 memBodyCheckerStart,
 bodyWords,
 expectedPacketLen,
 STAR_EOP_TYPE_EOP);

/* Initiate packet generation and sinking run */
PKT_SUBSYS_startSink(checkerDevice, PACKET_SUBSYSTEM_NUMBER);
/* Start the packet checker, disable the the packet sink after a short delay on receiving a mismatched
 character */
PKT_SUBSYS_startPacketChecking(checkerDevice, PACKET_SUBSYSTEM_NUMBER, 1,
 sinkDelayOnMismatch);
/* Kick off packet generation of 10 packets */
PKT_SUBSYS_startPacketGeneration(generatorDevice,
 PACKET_SUBSYSTEM_NUMBER, 10);
/* Wait for packet generation to complete */
PKT_SUBSYS_waitForPacketGenerationSequenceComplete(
 generatorDevice, PACKET_SUBSYSTEM_NUMBER, 0);
/* Get a count of the number of received bytes that did not match the expected pattern */
PKT_SUBSYS_getPacketCheckingMismatchCount(checkerDevice,
 PACKET_SUBSYSTEM_NUMBER, &mismatchCount);
/* Stop packet checking*/
PKT_SUBSYS_stopPacketChecking(checkerDevice, PACKET_SUBSYSTEM_NUMBER);
PKT_SUBSYS_stopSink(checkerDevice, PACKET_SUBSYSTEM_NUMBER);

printf("Mismatch count: %llu\n", mismatchCount);

/* Display sink contents if we received a mismatched character */
if(mismatchCount)
{
 /* Read back the packet sink data from the device */
 PKT_SUBSYS_getSinkDataPointers(checkerDevice, PACKET_SUBSYSTEM_NUMBER
 , &startPointer, &endPointer);
 if(endPointer < startPointer)
 {
 /* wraparound */
 pReadBackBuffer = (U8*)calloc(sinkLenWords, sizeof(U32));
 if(!pReadBackBuffer)
 {

```

```

 return 0;
 }

 PKT_SUBSYS_readMemory(checkerDevice, startPoint, ((memSinkStart+
sinkLenWords) - startPoint) * sizeof(U32), pReadBackBuffer);
 PKT_SUBSYS_readMemory(checkerDevice, memSinkStart, (endPointer -
memSinkStart) * sizeof(U32), pReadBackBuffer + ((memSinkStart+sinkLenWords) - endPointer) * sizeof(
U32));
 }
 else
 {
 /* no wraparound */
 sinkLenWords = endPointer - startPoint;
 pReadBackBuffer = (U8*)calloc(sinkLenWords, sizeof(U32));
 if(!pReadBackBuffer)
 {
 return 0;
 }
 PKT_SUBSYS_readMemory(checkerDevice, startPoint, (endPointer -
startPointer) * sizeof(U32), pReadBackBuffer);
 }

 /* Display all data received after the packet sink was stopped automatically on
character mismatch. Note that if the packet sink was stopped manually before this
delay was reached, some displayed data will be from before the first mismatch occurred. */

 if((sinkDelayOnMismatch+3)/4 >= sinkLenWords)
 {
 /* sink disable delay is larger than sink; display whole sink */
 offset = address = 0;
 }
 else
 {
 /* display only last (sink delay words + mismatch word) of sink */
 offset = (sinkLenWords * 4 - sinkDelayOnMismatch) - 4;
 address = (sinkLenWords - (sinkDelayOnMismatch+3)/4) - 1;
 }

 printf("Packet Sink contents (Last %d words):\n", MIN(sinkLenWords, ((sinkDelayOnMismatch+3)/4) + 1
));

 for(; offset <= sinkLenWords * sizeof(U32) - 4; offset += 4)
 {
 printf(" %-5d | %02x %02x %02x %02x\n", address++,
 ((U8 *)pReadBackBuffer)[offset], ((U8 *)pReadBackBuffer)[offset + 1],
 ((U8 *)pReadBackBuffer)[offset + 2], ((U8 *)pReadBackBuffer)[offset + 3]);
 }

 free(pReadBackBuffer);
 pReadBackBuffer = NULL;
}

return 0;
}

void STAR_API_CC displayStats(
 STAR_DEVICE_ID deviceIdentifier,
 unsigned int subsystem,
 PKT_SUBSYS_STATISTICS statistics)
{
 UNREFERENCED_PARAMETER(deviceIdentifier);
 UNREFERENCED_PARAMETER(subsystem);

 printf("Data Character Rate: %u/s \n", statistics.dataCharacterRate);
 printf("Data Characters Received: %llu \n", statistics.
 dataCharactersReceived);
 printf("EEP Character Rate: %u/s \n", statistics.eepCharacterRate);
 printf("EEP Characters Received: %llu \n", statistics.eepCharactersReceived);
 printf("EOP Character Rate: %u/s \n", statistics.eopCharacterRate);
 printf("EOP Characters Received: %llu \n", statistics.eopCharactersReceived);
 puts("Press enter to stop");
}

void showStatistics(STAR_DEVICE_ID deviceId)
{
 char string [256];
 STATISTICS_LOOP_ID id;

 /* Enable packet sink */
 PKT_SUBSYS_startSink(deviceId, PACKET_SUBSYSTEM_NUMBER);

 puts("Ensure the device is not running in interface mode.");
 puts("Press enter to start.");
 fgets(string, 256, stdin);

```

```

/* Begin polling statistics */
id = PKT_SUBSYS_startGetStatisticsLoop(deviceId,
 PACKET_SUBSYSTEM_NUMBER, displayStats);
if(!id)
{
 puts("Could not start statistics loop");
 return;
}
puts("Press enter to stop:");
fgets(string, 256, stdin);

/* End loop, disable packet sink again */
PKT_SUBSYS_stopGetStatisticsLoop(id);
PKT_SUBSYS_stopSink(deviceId, PACKET_SUBSYSTEM_NUMBER);
}

void usage(char *argv[], int error)
{
 puts("");
 if(error)
 fprintf(stderr, "usage: %s [-v][-s][-help]\n", argv[0]);
 else
 fprintf(stdout, "usage: %s [-v][-s][-help]\n", argv[0]);
}

void showHelp(void)
{
 puts("");
 puts("Description:");
 puts(" A program to demonstrate usage of the PCI Mk2");
 puts(" packet subsystem API. Sets up a packet generator");
 puts(" and checker, starts them, then displays results.");
 puts("");
 puts(" Note that interface mode will be disabled on the");
 puts(" the device(s) chosen.");
 puts("");
 puts("Optional Switches:");
 puts("");
 puts(" -v Displays version information and exits.");
 puts("");
 puts(" -help Displays this help message and exits.");
 puts("");
 puts(" -s Polls and displays device statistics until");
 puts(" user hits enter.");
 puts("");
 puts(" When called without arguments, the program sets up a packet");
 puts(" generator a chosen device to send 10 102 byte packets,");
 puts(" addressed to the packet checking subsystem on a second chosen");
 puts(" device. The packets are built from 2 address bytes");
 puts(" (1, then subsystem port) followed by 100 bytes of a repeating");
 puts(" alternating bits pattern.");
}

/*Set up a packet generator on subsystem one to send packets to a sink on subsystem 2*/
int __cdecl main(int argc, char* argv[])
{
 int i;
 int version = 0; /* Value for the "-v" optional argument. */
 int help = 0; /* Value for the "-help" optional argument. */
 int statistics = 0; /* Value for the "-s" optional argument. */
 STAR_DEVICE_ID deviceA, deviceB;

 STAR_DEVICE_TYPE aDeviceTypes[1];
 aDeviceTypes[0] = STAR_DEVICE_PCI_MK2;

 /* If no args provided, run with default settings */
 if (argc <= 1)
 {
 puts("Select device to run packet generator on:");
 deviceA = STAR_UI_chooseDevice(aDeviceTypes,1);

 puts("Select device to run packet checker on:");
 deviceB = STAR_UI_chooseDevice(aDeviceTypes,1);

 if (!deviceA || !deviceB)
 {
 return 1;
 }

 CFG_disableInterfaceMode(deviceB);

 runTest(deviceA, deviceB);
 return 0;
 }
}

```

```

/* Currently we only expect 1 optional argument */
if (argc > 2)
{
 usage(argv, 1);
 return 1;
}

/* Process command line args*/
for (i = 1; i < argc; i++)
{
 if (strcmp(argv[i], "-v") == 0)
 {
 version = 1;
 }
 else if (strcmp(argv[i], "-help") == 0)
 {
 help = 1;
 }
 else if (strcmp(argv[i], "-s") == 0)
 {
 statistics = 1;
 }
}

if (help)
{
 usage(argv, 0);
 showHelp();
}
else if (version)
{
 printVersionInformation();
}
else if (statistics)
{
 puts("Select device:");
 //deviceA = chooseDevice();
 deviceA = STAR_UI_chooseDevice(aDeviceTypes, 1);

 CFG_disableInterfaceMode(deviceA);

 if (!deviceA)
 {
 return 1;
 }

 showStatistics(deviceA);
}
else
{
 usage(argv, 1);
 return 1;
}

return 0;
}

```

## 9.22 pciMk2\_configuration\_example.c

Example usage of this API.

```

#include <stdio.h>
#include "star-api.h"
#include "cfg_api_generic.h"
#include "ui.h"

int __cdecl main()
{
 STAR_DEVICE_ID deviceID;
 STAR_CFG_FPGA_INFO fpgaInfo;
 char versionStr[STAR_CFG_MK2_VERSION_STR_MAX_LEN];
 char buildDateStr[STAR_CFG_MK2_BUILD_DATE_STR_MAX_LEN];
 U32 signallingRate;

 STAR_DEVICE_TYPE aDeviceTypes[1];
 aDeviceTypes[0] = STAR_DEVICE_PCI_MK2;

 /* Select device to configure */
 deviceID = STAR_UI_chooseDevice(aDeviceTypes, 1);
 if (!deviceID)

```



```

{
 return 0;
}

STAR_getDeviceType(deviceID);

/* Get hardware info*/
CFG_getFPGAInfo(deviceID, &fpgaInfo);

CFG_FPGAInfoToString(deviceID, &fpgaInfo, versionStr, buildDateStr);

/* Display the hardware info*/
printf("\nVersion: %s", versionStr);
printf("\nBuildDate: %s", buildDateStr);

/* Set general purpose register to 0xABCD */
CFG_setGeneralPurpose(deviceID, 0xABCD);

/* Flash the device's LEDs*/
CFG_identify(deviceID);

/* Set the device to be a time-code master*/
CFG_enableTimeCodeMaster(deviceID);

/* Set 2 second delay between time-codes */
CFG_setTimeCodePeriod(deviceID, 2000000);

/* Enable interface mode */
CFG_enableInterfaceMode(deviceID);

/* Disable interface mode on port 2 only */
CFG_disableInterfaceModeOnPort(deviceID, 2);

/* Enable adding the source port number as a leading byte
 to received packets on ports 1 and 3 */

CFG_enableIdentifySource(deviceID);
CFG_enableIdentifySourceOnPort(deviceID, 1);
CFG_enableIdentifySourceOnPort(deviceID, 3);

/* Set port 2 to have a link speed of 100 Mbit/s */
CFG_setTransmitSignallingRate(deviceID, 2, 100);

/* Read Link speed of port 1 */
CFG_getTransmitSignallingRate(deviceID, 1, &signallingRate);
printf("\nLink 1 transmit rate: %d Mbit/s", signallingRate);

/* Inject a disconnect error on link 1 */
CFG_injectError(deviceID, 1, SPW_ERROR_DISCONNECT);

return 0;
}

```

## 9.23 port\_control\_status\_example.c

This example demonstrates the use of the Port Status and Control functions.

```

#include <stdio.h>
#include "star-api.h"
#include "cfg_api_generic.h"

STAR_DEVICE_ID chooseStarDevice()
{
 STAR_DEVICE_ID * devices;
 unsigned int devCount = 0, chosen, i, status;
 STAR_DEVICE_ID deviceID;
 char *deviceName, s[256];

 /* Get the list of devices present which can be configured */
 devices = STAR_getDeviceListForType(
 STAR_DEVICE_CONFIG_SUPPORTED,
 &devCount);

 /* If there was an error getting the list of devices */
 if (!devices)
 {
 /* Return an error */
 puts("No SpaceWire devices detected!");
 return 0;
 }
}

```

```

/* Display the devices detected */
if (devCount == 1)
{
 puts("One SpaceWire device detected:");
}
else
{
 printf("%u SpaceWire devices detected:", devCount);
}

/* For each device */
for (i = 0; i < devCount; i++)
{
 /* Display its name */
 deviceName = STAR_getDeviceName(devices[i]);
 if (deviceName)
 {
 printf("\t%u - %s\n", i, deviceName);
 STAR_destroyString(deviceName);
 }
 else
 {
 printf("\t%u - Unknown SpaceWire Device\n", i);
 }
}

/* If there's only 1 device on the list */
if (devCount == 1)
{
 /* Return that device */
 deviceID = devices[0];
 STAR_destroyDeviceList(devices);
 return deviceID;
}

/* Ask the user which device to use */
printf("\nPlease select which device to open: ");
fgets(s, 256, stdin);
status = sscanf(s, "%d", &chosen);
if ((!status) || (chosen > devCount - 1))
{
 puts("Incorrect device number selected.");
 STAR_destroyDeviceList(devices);
 return 0;
}

/* Return the chosen device */
deviceID = devices[chosen];
STAR_destroyDeviceList(devices);
return deviceID;
}

int __cdecl main()
{
 STAR_DEVICE_ID deviceID;
 PORT_STATUS_CONTROL registerValue;
 U8 portNum = 0; /* Configuration port */

 STAR_CFG_CONFIG_PORT_ERRORS configPortErrors;
 STAR_CFG_SPW_LINK_STATUS spwPortStatus;

 /* Select device to configure */
 deviceID = chooseStarDevice();

 /* Get port information */
 CFG_getPortStatusControl(deviceID, portNum, ®isterValue);

 /* Display port type */
 switch(CFG_getPortType(registerValue))
 {
 case STAR_CFG_PORT_TYPE_CONFIGURATION:
 printf("\nPort type of port %u: Configuration Port", portNum);
 break;
 case STAR_CFG_PORT_TYPE_LINK:
 printf("\nPort type of port %u: SpaceWire Link Port", portNum);
 break;
 case STAR_CFG_PORT_TYPE_EXTERNAL:
 printf("\nPort type of port %u: External Port", portNum);
 break;
 case STAR_CFG_PORT_TYPE_INVALID:
 printf("\nPort type of port %u: Invalid port value", portNum);
 break;
 }

 /* Display current internal connection */

```

```

printf("\nPort %d is currently connected to port %d ",
 portNum, CFG_getPortConnection(registerValue));

/* Get configuration port errors (if portNum == 0) */
CFG_getConfigPortErrors(registerValue, &configPortErrors);
if (configPortErrors.errorCount)
{
 printf("\n%d error(s) present on configuration port ", configPortErrors.
 errorCount);
}

/* Clear port errors */
CFG_clearPortErrors(deviceID, portNum);

/* Get SpaceWire port Status */
portNum = 1; /* Assume for the purpose of this example that port One
 for this device is a SpaceWire port */

CFG_getPortStatusControl(deviceID, portNum, ®isterValue);

/* Obtain the links status values */
CFG_getSpaceWireLinkStatus(registerValue, &spwPortStatus);

/* Display some details about the links status
 Note that the links state machine state can be obtained from the linkState member*/
printf("\n\nLink %u %s running.", portNum, spwPortStatus.running ? "is":"is not");
printf("\nLink %u %s set to autostart.", portNum, spwPortStatus.autoStart ? "is":"is not");
printf("\nLink %u %s set to start.", portNum, spwPortStatus.start ? "is":"is not");
printf("\nLink %u %s disabled.", portNum, spwPortStatus.disable ? "is":"is not");
printf("\nLink %u %s in tri-state mode.", portNum, spwPortStatus.triState ? "is":"is not");

/* Disable the link (Set the disable bit, clear the start bit)*/
spwPortStatus.disable = 1;
spwPortStatus.start = 0;
CFG_setSpaceWireLinkStatus(deviceID, portNum, &spwPortStatus);

/* Start and stop the link using the shortcut functions.*/
CFG_startLink(deviceID, portNum);
CFG_stopLink(deviceID, portNum);

return 0;
}

```

## 9.24 read\_command\_packet\_example.c

This is an example of how to build RMAP read commands.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#include "rmap_packet_library.h"

void read_command_packet_example()
{
 void *pFillPacket, *pBuildPacket, *pBuildPacketRegister;
 unsigned long fillPacketLenCalculated, fillPacketLen;
 unsigned long buildPacketLen, buildPacketRegisterLen;
 U8 pTarget[] = {0, 254};
 U8 pReply[] = {254};
 char status;

 /* Calculate the length of a read command packet, with 4 bytes of data */
 fillPacketLenCalculated = RMAP_CalculateReadCommandPacketLength(2,
 1, 1);
 printf("The read command packet will have a length of %ld bytes\n",
 fillPacketLenCalculated);

 /* Allocate memory for the packet */
 pFillPacket = malloc(fillPacketLenCalculated);
 if (!pFillPacket)
 {
 puts("Couldn't allocate the memory for the command packet");
 return;
 }

 /* Fill the memory with the packet */
 status = RMAP_FillReadCommandPacket(pTarget, 2, pReply, 1, 0, 0x20, 0,
 0x106, 0, 4, &fillPacketLen, NULL, 1, (U8 *)pFillPacket,
 fillPacketLenCalculated);
}

```

```

if (!status)
{
 puts("Couldn't fill the read command packet");
 free(pFillPacket);
 return;
}

/* Build an RMAP read command packet to read 4-bytes */
pBuildPacket = RMAP_BuildReadCommandPacket(pTarget, 2, pReply, 1, 0, 0x20,
0, 0x106, 0, 4, &buildPacketLen, NULL, 1);
if (!pBuildPacket)
{
 puts("Couldn't build the read command packet");
 free(pFillPacket);
 return;
}

/* Build an RMAP read command packet to read from a 4-byte register */
pBuildPacketRegister = RMAP_BuildReadRegisterPacket(pTarget, 2, pReply, 1,
0, 0x20, 0, 0x106, 0, &buildPacketRegisterLen, NULL, 1);
if (!pBuildPacketRegister)
{
 puts("Couldn't build the read register packet");
 RMAP_FreeBuffer(pBuildPacket);
 free(pFillPacket);
 return;
}

/* Check the lengths are all the same */
if ((fillPacketLenCalculated != fillPacketLen) ||
(fillPacketLen != buildPacketLen) ||
(buildPacketLen != buildPacketRegisterLen))
{
 puts("The lengths are different:");
 printf("\tLength calculated for packet: %ld\n",
 fillPacketLenCalculated);
 printf("\tLength of filled packet: %ld\n", fillPacketLen);
 printf("\tLength of built packet: %ld\n", buildPacketLen);
 printf("\tLength of built register packet: %ld\n", buildPacketRegisterLen);
}
else
{
 puts("The packet lengths are all the same.");

 /* Check the packets are identical, despite being built differently */
 if (memcmp(pFillPacket, pBuildPacket, fillPacketLen) != 0)
 {
 puts("The filled packet is different to the built packet.");
 }
 else if (memcmp(pFillPacket, pBuildPacketRegister, fillPacketLen) != 0)
 {
 puts("The built register packet is different to the built packet and the filled packet.");
 }
 else
 {
 puts("The packets are identical.");
 }
}

/* Free the memory allocated for each of the packets. Note that the */
/* memory allocated by the library should be freed by calling */
/* RMAP_FreeBuffer. */
RMAP_FreeBuffer(pBuildPacketRegister);
RMAP_FreeBuffer(pBuildPacket);
free(pFillPacket);
}

```

## 9.25 read\_reply\_packet\_example.c

This is an example of how to build RMAP read replies.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#include "rmap_packet_library.h"

void read_reply_packet_example()
{

```

```

void *pFillPacket, *pBuildPacket;
unsigned long fillPacketLenCalculated, fillPacketLen, buildPacketLen;
U8 pReply[] = {254};
U8 target = 254;
U8 pData[] = {0x12, 0x34, 0x56, 0x78};
char status;

/* Calculate the length of a read reply packet, with 4 bytes of data */
fillPacketLenCalculated = RMAP_CalculateReadReplyPacketLength(1, 4,
1);
printf("The read reply packet will have a length of %ld bytes\n",
fillPacketLenCalculated);

/* Allocate memory for the packet */
pFillPacket = malloc(fillPacketLenCalculated);
if (!pFillPacket)
{
puts("Couldn't allocate the memory for the reply packet");
return;
}

/* Fill the memory with the packet */
status = RMAP_FillReadReplyPacket(pReply, 1, target, 0,
RMAP_SUCCESS, 0,
pData, 4, &fillPacketLen, NULL, 1, (U8 *)pFillPacket,
fillPacketLenCalculated);
if (!status)
{
puts("Couldn't fill the read reply packet");
free(pFillPacket);
return;
}

/* Build an RMAP read reply packet */
pBuildPacket = RMAP_BuildReadReplyPacket(pReply, 1, target, 0,
RMAP_SUCCESS, 0, pData, 4, &buildPacketLen, NULL, 1);
if (!pBuildPacket)
{
puts("Couldn't build the read reply packet");
free(pFillPacket);
return;
}

/* Check the lengths are all the same */
if ((fillPacketLenCalculated != fillPacketLen) ||
(fillPacketLen != buildPacketLen))
{
puts("The lengths are different:");
printf("\tLength calculated for packet: %ld\n",
fillPacketLenCalculated);
printf("\tLength of filled packet: %ld\n", fillPacketLen);
printf("\tLength of built packet: %ld\n", buildPacketLen);
}
else
{
puts("The packet lengths are all the same.");

/* Check the packets are identical, despite being built differently */
if (memcmp(pFillPacket, pBuildPacket, fillPacketLen) != 0)
{
puts("The filled packet is different to the built packet.");
}
else
{
puts("The packets are identical.");
}
}

/* Free the memory allocated for each of the packets. Note that the */
/* memory allocated by the library should be freed by calling */
/* RMAP_FreeBuffer. */
RMAP_FreeBuffer(pBuildPacket);
free(pFillPacket);
}

```

## 9.26 rmap\_target\_example.c

This file contains example code demonstrating how to use the RMAP Target API.

```

#include "rmap_target_pxi_if.h"

#include <stdlib.h>

#include "rmap_packet_library.h"
#include "cfg_api_generic.h"

#ifdef _WIN32
 #include "windows.h"

 #define SLEEP(milliseconds) Sleep(milliseconds);
#else
 #include <unistd.h>

 #define SLEEP(milliseconds) usleep(milliseconds * 1000);
#endif

#if !defined(UNREFERENCED_PARAMETER)
 #define UNREFERENCED_PARAMETER(a) ((void) (a))
#endif

/* The four RMAP targets are accessed through external ports 6-9 */
#define TARGET_PORT_START (6)
#define TARGET_PORT_END (9)

typedef enum
{
 MENU_CHOICE_MANUAL_AUTH = 1,
 MENU_CHOICE_NOTIFICATIONS,
 MENU_CHOICE_EXIT,
 MENU_CHOICE_INVALID
} MENU_CHOICE;

void STAR_API_CC notifListener(STAR_DEVICE_ID deviceId,
 U32 target,
 void *pNotif, void *pContext)
{
 int i;

 /* Cast the notification parameter to an array of bytes */
 U8 *notif = (U8*)pNotif;

 /* Cast the context variable to a U32 */
 U32 context = *((U32*)pContext);

 /* Print the device ID and the target index */
 printf("Device %u (Target = %u, Context = %u): ", deviceId, target,
 context);

 /* Print each of the notification bytes */
 /* See the STAR-System documentation for the notification structure */
 for (i = 0; i < 22; ++i)
 {
 printf("0x%02x ", notif[i]);
 }
 printf("\n");
}

void sendRandomCommand(STAR_DEVICE_ID deviceId, U32 target)
{
 U8 buffer[1024];
 U8 cmdPath[2], repPath[1];
 U32 length, address;
 U8 iniLa, tarLa, key;
 STAR_CHANNEL_ID channelId;
 STAR_STREAM_ITEM *streamItem;
 STAR_TRANSFER_OPERATION *transferOp;
 STAR_TRANSFER_STATUS status;
 void *command;
 unsigned long rawPacketLength;

 /* Set the RMAP command parameters to random values */
 length = rand() % 1024;
 address = rand();
 iniLa = 32 + (rand() % 222);
 tarLa = 32 + (rand() % 222);
 key = rand();

 /* Set the buffer to 0xAB, repeating */
 memset(buffer, 0xAB, 1024);

 /* Set the command and reply paths */
 cmdPath[0] = 6 + target, cmdPath[1] = tarLa;
 repPath[0] = iniLa;

 /* Open a channel */
 channelId = STAR_openChannelToLocalDevice(deviceId,

```

```

 STAR_CHANNEL_DIRECTION_OUT, 4, 1);
if (!channelId)
{
 printf("Failed to open channel 4\n");
 return;
}

/* Build the command */
command = RMAP_BuildWriteCommandPacket(cmdPath, 2, repPath, 1, 0, 0, 1, key
, 0,
 address, 0, buffer, length, &rawPacketLength, NULL, 1);

/* Create the stream item */
streamItem = STAR_createPacket(0, command, rawPacketLength,
 STAR_EOP_TYPE_EOP);
if (!streamItem)
{
 RMAP_FreeBuffer(command);
 printf("Failed to create stream item\n");
 return;
}

/* Create the transfer operation */
transferOp = STAR_createTxOperation(&streamItem, 1);
if (!transferOp)
{
 STAR_destroyStreamItem(streamItem);
 RMAP_FreeBuffer(command);
 printf("Failed to create transfer operation\n");
 return;
}

/* Submit the transfer operation */
if (!STAR_submitTransferOperation(channelId, transferOp))
{
 STAR_destroyStreamItem(streamItem);
 RMAP_FreeBuffer(command);
 printf("Failed to submit transfer operation\n");
 return;
}

/* Wait on the transfer operation completing */
status = STAR_waitOnTransferOperationCompletion(transferOp, -1);
if (status != STAR_TRANSFER_STATUS_COMPLETE)
{
 STAR_destroyStreamItem(streamItem);
 RMAP_FreeBuffer(command);
 printf("Failed to transmit the command\n");
 return;
}

STAR_destroyStreamItem(streamItem);
RMAP_FreeBuffer(command);
STAR_closeChannel(channelId);
}

void reset(STAR_DEVICE_ID deviceId)
{
 int i;

 /* Reset the RMAP parameters for each target */
 for (i = 0; i < 4; ++i)
 {
 RMAP_TARGET_PXI_IF_setInterfaceMode(deviceId, TARGET_PORT_START
+ i,
 IF_MODE_RMAP_ENABLED);
 RMAP_TARGET_PXI_IF_setAuthControlMode(deviceId, i,
 TARGET_AUTH_MODE_AUTOMATIC);
 RMAP_TARGET_PXI_IF_setAuthCommands(deviceId, i,
 TARGET_AUTH_CMD_ALL);
 RMAP_TARGET_PXI_IF_setAuthKeyRange(deviceId, i, 0, 255);
 RMAP_TARGET_PXI_IF_setAuthLogicalAddressRange(deviceId
, i, 32, 255);
 RMAP_TARGET_PXI_IF_setAuthProtocolId(deviceId, i, 1);
 RMAP_TARGET_PXI_IF_setAuthMemoryAddressRange(deviceId,
i, 0,
 0xffffffff);
 RMAP_TARGET_PXI_IF_setAddressOffset(deviceId, i, 0);
 RMAP_TARGET_PXI_IF_setEnabledNotifications(deviceId, i,
 NOTIF_TYPE_NONE);
 RMAP_TARGET_PXI_IF_unregisterNotificationListener(
 deviceId, i,
 NOTIF_TYPE_AUTH_REQUEST);
 RMAP_TARGET_PXI_IF_unregisterNotificationListener(
 deviceId, i,
 NOTIF_TYPE_CMD_COMPLETE);
 RMAP_TARGET_PXI_IF_stopReceivingNotifications(deviceId

```

```

 });
}

MENU_CHOICE getMenuChoice(void)
{
 char buffer[32];
 unsigned int choice;

 /* Show menu to user */
 printf("Please select:\n");
 printf("%d. Manual RMAP command authorisation example.\n",
 MENU_CHOICE_MANUAL_AUTH);
 printf("%d. Command complete notifications example.\n",
 MENU_CHOICE_NOTIFICATIONS);
 printf("%d. Exit\n", MENU_CHOICE_EXIT);
 printf("> ");

 /* Validate chosen number */
 if (!fgets(buffer, 32, stdin)) return MENU_CHOICE_INVALID;

 /* Get menu choice value */
 if (!sscanf(buffer, "%u", &choice) ||
 choice >= MENU_CHOICE_INVALID) return MENU_CHOICE_INVALID;

 /* Return the chosen value */
 return choice;
}

void manualAuthExample(STAR_DEVICE_ID deviceId)
{
 char buffer[32];
 unsigned int choice;
 RmapCommandParameters command;

 /* Disable router timeouts so RMAP commands aren't truncated whilst
 awaiting authorisation */
 STAR_CFG_ROUTER_GLOBAL_STATE globalState;
 CFG_getRouterGlobalSettings(deviceId, &globalState);
 globalState.timeoutMode = STAR_CFG_TIMEOUT_MODE_BLOCKING;
 CFG_setRouterGlobalSettings(deviceId, &globalState);

 /* Reset the RMAP targets */
 reset(deviceId);

 /* Disable the RMAP target for target 3 (port 9) so the channel can be used
 to transmit commands */
 RMAP_TARGET_PXI_IF_setInterfaceMode(deviceId, 9,
 IF_MODE_RMAP_DISABLED);

 /* Enable manual authorisation mode for target 0 */
 RMAP_TARGET_PXI_IF_setAuthControlMode(deviceId, 0,
 TARGET_AUTH_MODE_MANUAL);

 printf("\nGenerating randomised RMAP command for target 0...\n");

 /* Send a random command to target 0 */
 sendRandomCommand(deviceId, 0);

 /* Get the waiting command */
 RMAP_TARGET_PXI_IF_getWaitingCommand(deviceId, 0, &command);

 /* Print the command parameters */
 printf("\nParameters: tar LA = 0x%x, command = 0x%x, key = 0x%x, address = 0x%x, length = 0x%x, ini LA
 = 0x%x\n",
 command.targetLogicalAddress, command.command, command.key,
 command.address, command.dataLength, command.initiatorLogicalAddress);

 /* Print the authorisation menu */
 printf("1. Authorise.\n");
 printf("2. Reject.\n");
 printf("> ");

 if (!fgets(buffer, 32, stdin)) choice = 0;

 if (!sscanf(buffer, "%u", &choice) ||
 choice >= MENU_CHOICE_INVALID) choice = 0;

 /* Authorise or reject the command */
 if (choice == 1)
 {
 printf("Authorising command...\n");
 RMAP_TARGET_PXI_IF_authoriseWaitingCommand(deviceId, 0);
 }
 else if (choice == 2)
 {
 printf("Rejecting command...\n");
 }
}

```



```

 RMAP_TARGET_PXI_IF_rejectWaitingCommand(deviceId, 0,
 REJECTION_REASON_OTHER);
 }
}

void notificationsExample(STAR_DEVICE_ID deviceId)
{
 U32 target;
 U32 context0, context1, context2;

 /* Reset the RMAP targets */
 reset(deviceId);

 /* Disable the RMAP target for target 3 (port 9) so the channel can be used
 to transmit commands */
 RMAP_TARGET_PXI_IF_setInterfaceMode(deviceId, 9,
 IF_MODE_RMAP_DISABLED);

 /* Enable notifications for targets 0-2 */
 RMAP_TARGET_PXI_IF_setEnabledNotifications(deviceId, 0,
 NOTIF_TYPE_CMD_COMPLETE);
 RMAP_TARGET_PXI_IF_setEnabledNotifications(deviceId, 1,
 NOTIF_TYPE_CMD_COMPLETE);
 RMAP_TARGET_PXI_IF_setEnabledNotifications(deviceId, 2,
 NOTIF_TYPE_CMD_COMPLETE);

 /* Set the context variables */
 context0 = 123;
 context1 = 456;
 context2 = 789;

 /* Set the command complete notification listener for targets 0-2 */
 RMAP_TARGET_PXI_IF_registerNotificationListenerWithContext
 (deviceId, 0,
 NOTIF_TYPE_CMD_COMPLETE, notifListener, &context0);
 RMAP_TARGET_PXI_IF_registerNotificationListenerWithContext
 (deviceId, 1,
 NOTIF_TYPE_CMD_COMPLETE, notifListener, &context1);
 RMAP_TARGET_PXI_IF_registerNotificationListenerWithContext
 (deviceId, 2,
 NOTIF_TYPE_CMD_COMPLETE, notifListener, &context2);

 /* Start receiving notifications for the device */
 RMAP_TARGET_PXI_IF_startReceivingNotifications(deviceId);

 printf("\nGenerating randomised RMAP commands for target 0-2...\n");

 /* Send random commands to targets 0-2 every 200 milliseconds */
 while (1)
 {
 target = rand() % 3;
 sendRandomCommand(deviceId, target);
 SLEEP(200);
 }
}

STAR_DEVICE_ID getDeviceId(void)
{
 U32 count;
 STAR_DEVICE_ID *devices;
 STAR_DEVICE_ID deviceId;

 /* Get the first PXI RMAP Target device */
 devices = STAR_getDeviceListForType(
 STAR_DEVICE_PXI_RMAP, &count);
 deviceId = count ? devices[0] : 0;
 STAR_destroyDeviceList(devices);
 return deviceId;
}

int main(int argc, char **argv)
{
 STAR_DEVICE_ID deviceId;
 MENU_CHOICE choice;

 UNREFERENCED_PARAMETER(argc);
 UNREFERENCED_PARAMETER(argv);

 /* Print the program header */
 printf("----- RMAP Target API Examples ----- \n");

 deviceId = getDeviceId();
 if (!deviceId)
 {
 printf("No devices were found.\n");
 return 0;
 }
}

```

```

}

/* Reset RMAP targets */
reset(deviceId);

/* This example requires interface mode to be disabled */
CFG_disableInterfaceMode(deviceId);

/* Loop menu */
choice = MENU_CHOICE_INVALID;
do
{
 choice = getMenuChoice();
 switch (choice)
 {
 /* Manual authorisation example */
 case MENU_CHOICE_MANUAL_AUTH: manualAuthExample(deviceId); break;
 /* Notifications example */
 case MENU_CHOICE_NOTIFICATIONS: notificationsExample(deviceId); break;

 /* Handle unused cases */
 case MENU_CHOICE_INVALID: break;
 case MENU_CHOICE_EXIT: break;
 }
} while (choice != MENU_CHOICE_EXIT);

return 0;
}

```

## 9.27 rmw\_command\_packet\_example.c

This is an example of how to build RMAP read/modify/write commands.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#include "rmw_packet_library.h"

void rmw_command_packet_example()
{
 void *pFillPacket, *pBuildPacket, *pBuildPacketRegister;
 unsigned long fillPacketLenCalculated, fillPacketLen;
 unsigned long buildPacketLen, buildPacketRegisterLen;
 U8 pTarget[] = {0, 254};
 U8 pReply[] = {254};
 U8 pData[] = {0x12, 0x34, 0x56, 0x78};
 U8 pMask[] = {0x0f, 0x0f, 0x0f, 0x0f};
 char status;

 /* Calculate the length of a read/modify/write command packet, */
 /* with 4 bytes of data and mask */
 fillPacketLenCalculated = RMAP_CalculateReadModifyWriteCommandPacketLength
 (
 2, 1, 8, 1);
 printf("The read/modify/write command packet will have a length of %ld bytes\n",
 fillPacketLenCalculated);

 /* Allocate memory for the packet */
 pFillPacket = malloc(fillPacketLenCalculated);
 if (!pFillPacket)
 {
 puts("Couldn't allocate the memory for the command packet");
 return;
 }

 /* Fill the memory with the packet */
 status = RMAP_FillReadModifyWriteCommandPacket(pTarget, 2, pReply,
 1, 0x20,
 0, 0x106, 0, 8, pData, pMask, &fillPacketLen, NULL, 1,
 (U8 *)pFillPacket, fillPacketLenCalculated);
 if (!status)
 {
 puts("Couldn't fill the read/modify/write command packet");
 free(pFillPacket);
 return;
 }

 /* Build an RMAP read/modify/write command packet to write 4-bytes */
 pBuildPacket = RMAP_BuildReadModifyWriteCommandPacket(pTarget, 2,
 pReply, 1,

```

```

 0x20, 0, 0x106, 0, 8, pData, pMask, &buildPacketLen, NULL, 1);
if (!pBuildPacket)
{
 puts("Couldn't build the read/modify/write command packet");
 free(pFillPacket);
 return;
}

/* Build an RMAP read/modify/write command packet */
/* to write to a 4-byte register */
pBuildPacketRegister = RMAP_BuildReadModifyWriteRegisterPacket(
 pTarget, 2,
 pReply, 1, 0x20, 0, 0x106, 0, 0x12345678, 0x0f0f0f0f,
 &buildPacketRegisterLen, NULL, 1);
if (!pBuildPacketRegister)
{
 puts("Couldn't build the read/modify/write register packet");
 RMAP_FreeBuffer(pBuildPacket);
 free(pFillPacket);
 return;
}

/* Check the lengths are all the same */
if ((fillPacketLenCalculated != fillPacketLen) ||
 (fillPacketLen != buildPacketLen) ||
 (buildPacketLen != buildPacketRegisterLen))
{
 puts("The lengths are different:");
 printf("\tLength calculated for packet: %ld\n",
 fillPacketLenCalculated);
 printf("\tLength of filled packet: %ld\n", fillPacketLen);
 printf("\tLength of built packet: %ld\n", buildPacketLen);
 printf("\tLength of built register packet: %ld\n", buildPacketRegisterLen);
}
else
{
 puts("The packet lengths are all the same.");

 /* Check the packets are identical, despite being built differently */
 if (memcmp(pFillPacket, pBuildPacket, fillPacketLen) != 0)
 {
 puts("The filled packet is different to the built packet.");
 }
 else if (memcmp(pFillPacket, pBuildPacketRegister, fillPacketLen) != 0)
 {
 puts("The built register packet is different to the built packet and the filled packet.");
 }
 else
 {
 puts("The packets are identical.");
 }
}

/* Free the memory allocated for each of the packets. Note that the */
/* memory allocated by the library should be freed by calling */
/* RMAP_FreeBuffer. */
RMAP_FreeBuffer(pBuildPacketRegister);
RMAP_FreeBuffer(pBuildPacket);
free(pFillPacket);
}

```

## 9.28 rmw\_reply\_packet\_example.c

This is an example of how to build RMAP read/modify/write replies.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#include "rmap_packet_library.h"

void rmw_reply_packet_example()
{
 void *pFillPacket, *pBuildPacket;
 unsigned long fillPacketLenCalculated, fillPacketLen, buildPacketLen;
 U8 pReply[] = {254};
 U8 target = 254;
 U8 pData[] = {0x12, 0x34, 0x56, 0x78};
 char status;

```

```

/* Calculate the length of a read/modify/write reply packet, */
/* with 4 bytes of data */
fillPacketLenCalculated = RMAP_CalculateReadModifyWriteReplyPacketLength
(1,
 4, 1);
printf("The read/modify/write reply packet will have a length of %ld bytes\n",
 fillPacketLenCalculated);

/* Allocate memory for the packet */
pFillPacket = malloc(fillPacketLenCalculated);
if (!pFillPacket)
{
 puts("Couldn't allocate the memory for the reply packet");
 return;
}

/* Fill the memory with the packet */
status = RMAP_FillReadModifyWriteReplyPacket(pReply, 1, target,
 RMAP_SUCCESS, 0, 4, pData, &fillPacketLen, NULL, 1, (U8 *)pFillPacket,
 fillPacketLenCalculated);
if (!status)
{
 puts("Couldn't fill the read/modify/write reply packet");
 free(pFillPacket);
 return;
}

/* Build an RMAP read/modify/write reply packet */
pBuildPacket = RMAP_BuildReadModifyWriteReplyPacket(pReply, 1,
 target,
 RMAP_SUCCESS, 0, 4, pData, &buildPacketLen, NULL, 1);
if (!pBuildPacket)
{
 puts("Couldn't build the read/modify/write reply packet");
 free(pFillPacket);
 return;
}

/* Check the lengths are all the same */
if ((fillPacketLenCalculated != fillPacketLen) ||
 (fillPacketLen != buildPacketLen))
{
 puts("The lengths are different:");
 printf("\tLength calculated for packet: %ld\n",
 fillPacketLenCalculated);
 printf("\tLength of filled packet: %ld\n", fillPacketLen);
 printf("\tLength of built packet: %ld\n", buildPacketLen);
}
else
{
 puts("The packet lengths are all the same.");

 /* Check the packets are identical, despite being built differently */
 if (memcmp(pFillPacket, pBuildPacket, fillPacketLen) != 0)
 {
 puts("The filled packet is different to the built packet.");
 }
 else
 {
 puts("The packets are identical.");
 }
}

/* Free the memory allocated for each of the packets. Note that the */
/* memory allocated by the library should be freed by calling */
/* RMAP_FreeBuffer. */
RMAP_FreeBuffer(pBuildPacket);
free(pFillPacket);
}

```

## 9.29 router\_configuration\_example.c

This example demonstrates how to use the user router state configuration functions.

```

#include <stdio.h>
#include "star-api.h"
#include "cfg_api_generic.h"

```

```

#ifndef TRUE
#define TRUE 1
#endif
#ifndef FALSE
#define FALSE 0
#endif

STAR_DEVICE_ID chooseStarDevice()
{
 STAR_DEVICE_ID * devices;
 unsigned int devCount = 0, chosen, i, status;
 STAR_DEVICE_ID deviceID;
 char *deviceName, s[256];

 /* Get the list of devices present which can be configured */
 devices = STAR_getDeviceListForType(
 STAR_DEVICE_CONFIG_SUPPORTED,
 &devCount);

 /* If there was an error getting the list of devices */
 if (!devices)
 {
 /* Return an error */
 puts("No SpaceWire devices detected!");
 return 0;
 }

 /* Display the devices detected */
 if (devCount == 1)
 {
 puts("One SpaceWire device detected:");
 }
 else
 {
 printf("%u SpaceWire devices detected:", devCount);
 }

 /* For each device */
 for (i = 0; i < devCount; i++)
 {
 /* Display its name */
 deviceName = STAR_getDeviceName(devices[i]);
 if (deviceName)
 {
 printf("\t%u - %s\n", i, deviceName);
 STAR_destroyString(deviceName);
 }
 else
 {
 printf("\t%u - Unknown SpaceWire Device\n", i);
 }
 }

 /* If there's only 1 device on the list */
 if (devCount == 1)
 {
 /* Return that device */
 deviceID = devices[0];
 STAR_destroyDeviceList(devices);
 return deviceID;
 }

 /* Ask the user which device to use */
 printf("\nPlease select which device to open: ");
 fgets(s, 256, stdin);
 status = sscanf(s, "%d", &chosen);
 if ((!status) || (chosen > devCount - 1))
 {
 puts("Incorrect device number selected.");
 STAR_destroyDeviceList(devices);
 return 0;
 }

 /* Return the chosen device */
 deviceID = devices[chosen];
 STAR_destroyDeviceList(devices);
 return deviceID;
}

int __cdecl main()
{
 STAR_DEVICE_ID deviceID;
 U32 portMask;
 U8 tcFlagMode;
 U8 tcValue;
 STAR_CFG_ROUTER_GLOBAL_STATE globalState;
 unsigned int i;

```

```

/* Select device to configure */
deviceID = chooseStarDevice();

/* Get Time-code output ports */
CFG_getTimeCodeDistributionPorts(deviceID, &portMask);

printf("\nTime-codes are bieng forwarded on ports: ");

/* Test each bit in mask to see if it is set */
for(i = 1; i < 32; i++)
{
 if((1 << i) & portMask)
 printf(" %u", i);
}

/* Forward time codes on ports 1, 2 and 3 only */
CFG_setTimeCodeDistributionPorts(deviceID, 0xE);

/* Get time-code flags mode */
CFG_getTimeCodeFlagMode(deviceID, &tcFlagMode);

printf("\nTime-code flag interpretation mode: %s", tcFlagMode ?
 "Flags ignored" :
 "Time-code discarded on flag not \"00\"");

/* Get current time-code value */
CFG_getTimeCodeValue(deviceID, &tcValue);
printf("\n Current time-code value: %u", tcValue);

/* Get Router global settings */
CFG_getRouterGlobalSettings(deviceID, &globalState);

printf("\n\nDisable on silence: %s", globalState.disableOnSilence ? "Enabled" : "
Disabled");
printf("\nSelf addressing: %s", globalState.enableSelfAddressing ? "Enabled" : "
Disabled");
printf("\nStart on request: %s", globalState.startOnRequest ? "Enabled" : "Disabled");

switch(globalState.timeoutMode)
{
case STAR_CFG_TIMEOUT_MODE_WATCHDOG:
 printf("\nWatchdog mode enabled.");
 break;
case STAR_CFG_TIMEOUT_MODE_BLOCKING:
 printf("\nWatchdog mode disabled.");
 break;
}

printf("\nTimeout period: ");
switch(globalState.timeoutPeriod)
{
case STAR_CFG_PORT_TIMEOUT_100US:
 printf("60-100us");
 break;
case STAR_CFG_PORT_TIMEOUT_1MS:
 printf("1.3ms");
 break;
case STAR_CFG_PORT_TIMEOUT_10MS:
 printf("10ms");
 break;
case STAR_CFG_PORT_TIMEOUT_100MS:
 printf("82ms");
 break;
case STAR_CFG_PORT_TIMEOUT_1S:
 printf("1.3s");
 break;
}

/* Set Global settings */
globalState.disableOnSilence = FALSE;
globalState.enableSelfAddressing = TRUE;
globalState.startOnRequest = TRUE;
globalState.timeoutMode = STAR_CFG_TIMEOUT_MODE_WATCHDOG;
globalState.timeoutPeriod = STAR_CFG_PORT_TIMEOUT_10MS;

CFG_setRouterGlobalSettings(deviceID, &globalState);

return 0;
}

```

## 9.30 routerMk2S\_configuration\_example.c

Example usage of this API.

```
#include <stdio.h>
#include "star-api.h"
#include "cfg_api_generic.h"
#include "ui.h"

int main()
{
 STAR_DEVICE_ID deviceID;
 STAR_CFG_FPGA_INFO fpgaInfo;
 char versionStr[STAR_CFG_MK2_VERSION_STR_MAX_LEN];
 char buildDateStr[STAR_CFG_MK2_BUILD_DATE_STR_MAX_LEN];
 int enabled;
 double precisionTxRate;

 STAR_DEVICE_TYPE aDeviceTypes[1];
 aDeviceTypes[0] = STAR_DEVICE_ROUTER_MK2S;

 /* Select device to configure */
 deviceID = STAR_UI_chooseDevice(aDeviceTypes, 1);
 if (!deviceID)
 {
 return 0;
 }

 STAR_getDeviceType(deviceID);

 /* Get hardware info*/
 CFG_getFPGAInfo(deviceID, &fpgaInfo);

 CFG_FPGAInfoToString(deviceID, &fpgaInfo, versionStr, buildDateStr);

 /* Display the hardware info*/
 printf("\nVersion: %s", versionStr);
 printf("\nBuildDate: %s", buildDateStr);

 /* Set general purpose register to 0xABCD */
 CFG_setGeneralPurpose(deviceID, 0xABCD);

 /* Flash the device's LEDs*/
 CFG_identify(deviceID);

 /* Set the device to be a time-code master*/
 CFG_enableTimeCodeMaster(deviceID);

 /* Set 2 second delay between time-codes */
 CFG_setTimeCodePeriod(deviceID, 2000000);

 /* Enable interface mode */
 CFG_enableInterfaceMode(deviceID);

 /* Enable interface mode on port 1 only */
 CFG_enableInterfaceModeOnPort(deviceID, 1);

 /* Disable interface mode on port 2 only */
 CFG_disableInterfaceModeOnPort(deviceID, 2);

 CFG_getInterfaceModeOnPortEnabled(deviceID, 2, &enabled);

 printf("\nInterface mode %s on port 2", enabled ? "enabled" : "disabled");

 /* Enable adding the source port number as a leading byte
 to received packets on ports 1 and 2 */

 CFG_enableIdentifySource(deviceID);
 CFG_enableIdentifySourceOnPort(deviceID, 1);
 CFG_enableIdentifySourceOnPort(deviceID, 2);

 /* Check if identify source is enabled for port 1 */
 CFG_getIdentifySourceOnPortEnabled(deviceID, 1, &enabled);
 printf("\nIdentify source %s on port 1", enabled ? "enabled" : "disabled");

 CFG_disableIdentifySourceOnPort(deviceID, 1);

 /* Test for precision transmit rate */
 CFG_getPrecisionTransmitRateEnabled(deviceID, &enabled);
 printf("\nPrecision transmit rate is %s", enabled ? "enabled" : "disabled");

 /* Enable precision transmit rate */
 CFG_enablePrecisionTransmitRate(deviceID);
```

```

/* Wait until precision transmit rate is in use. This can sometimes take
 250us */
do
{
 CFG_getPrecisionTransmitRateInUse(deviceID, &enabled);
}
while(!enabled);

/* Get current precision transmit rate */
CFG_getPrecisionTransmitRate(deviceID, &precisionTxRate);

printf("\nPrecision transmit rate is %f Mbit/s", precisionTxRate);

/* Set precision transmit rate */
CFG_setPrecisionTransmitRate(deviceID, 123.4);
CFG_getPrecisionTransmitRate(deviceID, &precisionTxRate);
printf("\nPrecision transmit rate is %f Mbit/s", precisionTxRate);

return 0;
}

```

### 9.31 routing\_table\_example.c

This example demonstrates the use of the routing table functions.

```

#include <stdio.h>
#include "star-api.h"
#include "cfg_api_generic.h"

#define BIT1 0x02
#define BIT3 0x08

STAR_DEVICE_ID chooseStarDevice()
{
 STAR_DEVICE_ID * devices;
 unsigned int devCount = 0, chosen, i, status;
 STAR_DEVICE_ID deviceID;
 char *deviceName, s[256];

 /* Get the list of devices present which can be configured */
 devices = STAR_getDeviceListForType(
 STAR_DEVICE_CONFIG_SUPPORTED,
 &devCount);

 /* If there was an error getting the list of devices */
 if (!devices)
 {
 /* Return an error */
 puts("No SpaceWire devices detected!");
 return 0;
 }

 /* Display the devices detected */
 if (devCount == 1)
 {
 puts("One SpaceWire device detected:");
 }
 else
 {
 printf("%u SpaceWire devices detected:", devCount);
 }

 /* For each device */
 for (i = 0; i < devCount; i++)
 {
 /* Display its name */
 deviceName = STAR_getDeviceName(devices[i]);
 if (deviceName)
 {
 printf("\t%u - %s\n", i, deviceName);
 STAR_destroyString(deviceName);
 }
 else
 {
 printf("\t%u - Unknown SpaceWire Device\n", i);
 }
 }

 /* If there's only 1 device on the list */

```



```

 if (devCount == 1)
 {
 /* Return that device */
 deviceID = devices[0];
 STAR_destroyDeviceList(devices);
 return deviceID;
 }

 /* Ask the user which device to use */
 printf("\nPlease select which device to open: ");
 fgets(s, 256, stdin);
 status = sscanf(s, "%d", &chosen);
 if ((!status) || (chosen > devCount - 1))
 {
 puts("Incorrect device number selected.");
 STAR_destroyDeviceList(devices);
 return 0;
 }

 /* Return the chosen device */
 deviceID = devices[chosen];
 STAR_destroyDeviceList(devices);
 return deviceID;
}

int __cdecl main()
{
 STAR_DEVICE_ID deviceID;
 STAR_CFG_GAR_ENTRY entry;
 U8 logicalAddress = 106;
 unsigned int i;

 /* Select device to configure */
 deviceID = chooseStarDevice();

 /* Get a routing table entry */
 CFG_getRoutingTableEntry(deviceID, logicalAddress, &entry);

 /* Display routing table entry details */

 printf("\nRouting table entry for address %u.\n", logicalAddress);

 printf("\n\tAddress is %s.", entry.invalidAddress ? "invalid":"valid");
 printf("\n\tHeader deletion %s.", entry.deleteHeader ? "enabled":"disabled");
 printf("\n\tPriority: %s.", entry.priority ? "high":"low");

 printf("\n\tOutput ports: ");

 /* Test each bit in mask to see if it is set */
 for(i = 1; i < 32; i++)
 {
 if((1 << i) & entry.portMask)
 printf(" %u", i);
 }

 /* Set a routing table entry:
 Logical address 44, Header deletion enabled, High priority
 Output ports: 1,3 */
 entry.deleteHeader = 1;
 entry.invalidAddress = 0;
 entry.priority = 1;
 entry.portMask = BIT1 | BIT3;

 CFG_setRoutingTableEntry(deviceID, 44, &entry);

 return 0;
}

```

## 9.32 simple\_receive\_example.c

Prompts the user for the device to use and the channel to receive into. The simple receive function is then used to wait for an incoming packet. The received packet contents are then printed.

```

#include "examples.h"

#include "general.h"
#include "utilities.h"

#include "star-dundee_types.h"

```

```

#include "star-api.h"

void simpleReceiveExample()
{
 /* Get selected device */
 STAR_DEVICE_ID selectedDevice = promptForDevice(TRANSMIT_TYPE_RECEIVE);

 /* Define selected channel */
 int selectedChannelNumber;

 /* Get selected channel */
 STAR_CHANNEL_ID selectedChannel = promptForChannel(selectedDevice,
 &selectedChannelNumber, TRANSMIT_TYPE_RECEIVE);

 /* If channel was opened */
 if(selectedChannel)
 {
 /* Define receive buffer of maximum packet length */
 unsigned char receiveBuffer[MAX_PACKET_LENGTH];

 /* Initialise receive buffer length to size of receive buffer array */
 unsigned int receiveBufferLength = sizeof(receiveBuffer);

 /* Define eop type */
 STAR_EOP_TYPE eopType;

 /* Define transfer status */
 STAR_TRANSFER_STATUS status;

 /* Print waiting on incoming packet message */
 printf("Waiting on incoming packet...\n");

 /* Receive packet on selected channel */
 status = STAR_receivePacket(selectedChannel, receiveBuffer,
 &receiveBufferLength, &eopType, -1);

 /* If packet was received successfully */
 if(status == STAR_TRANSFER_STATUS_COMPLETE)
 {
 /* Prints the packet contents from the receive buffer */
 printPacketContents(receiveBuffer, receiveBufferLength);
 }
 else
 {
 /* Switch on transfer status */
 switch(status)
 {
 /* Case cancelled */
 case STAR_TRANSFER_STATUS_CANCELLED:
 /* Print error when packet transfer was cancelled */
 printf("The packet was not received because the transfer "
 "was cancelled.\n");
 break;
 default:
 /* Print unexpected error when receiving packet */
 printf("An unexpected error occurred when receiving the "
 "packet.\n");
 break;
 }
 }

 /* Close channel after transmitting packet */
 STAR_closeChannel(selectedChannel);
 }
}

```

### 9.33 simple\_send\_and\_receive\_example.c

Creates a simple packet and sends it out of a channel and receives it into another channel. Uses the simple send and receive functions.

```

#include "examples.h"

#include "general.h"
#include "utilities.h"

#include "star-dundee_types.h"
#include "star-api.h"

void simpleSendAndReceiveExample()

```

```

{
 /* Get first device */
 STAR_DEVICE_ID firstDevice = getFirstDevice();

 /* Define packet data to be sent */
 unsigned char packetData[] = {1, 2, 3, 4, 5, 6, 7, 8, 9};

 /* If there is a first device */
 if(firstDevice)
 {
 /* Open send channel */
 STAR_CHANNEL_ID sendChannel =
 STAR_openChannelToLocalDevice(
 firstDevice, STAR_CHANNEL_DIRECTION_OUT, CHANNEL1, 1);

 /* If send channel was opened */
 if(sendChannel)
 {
 /* Get packet length */
 unsigned int packetLength = sizeof(packetData) /
 sizeof(unsigned char);

 /* Transmit packet and wait indefinitely */
 STAR_TRANSFER_STATUS status =
 STAR_transmitPacket(sendChannel,
 packetData, packetLength, STAR_EOP_TYPE_EOP, -1);

 /* Close send channel */
 STAR_closeChannel(sendChannel);

 /* If packet was sent */
 if(status == STAR_TRANSFER_STATUS_COMPLETE)
 {
 /* Open receive channel */
 STAR_CHANNEL_ID receiveChannel =
 STAR_openChannelToLocalDevice
 (firstDevice, STAR_CHANNEL_DIRECTION_IN, CHANNEL2, 1);

 /* If receive channel was opened */
 if(receiveChannel)
 {
 /* Define receive buffer of maximum packet length */
 unsigned char receiveBuffer[MAX_PACKET_LENGTH];

 /* Initialise receive buffer length to size of receive
 buffer array */
 unsigned int receiveBufferLength = sizeof(receiveBuffer);

 /* Define EOP type */
 STAR_EOP_TYPE eopType = STAR_EOP_TYPE_EOP;

 /* Receive packet and wait indefinitely */
 status = STAR_receivePacket(receiveChannel, receiveBuffer,
 &receiveBufferLength, &eopType, -1);

 /* If packet was received */
 if(status == STAR_TRANSFER_STATUS_COMPLETE)
 {
 /* Print packet contents */
 printPacketContents(receiveBuffer, receiveBufferLength);
 }

 /* Close receive channel */
 STAR_closeChannel(receiveChannel);
 }
 }
 }
 }
}

```

## 9.34 simple\_send\_example.c

Creates a simple packet and sends it out of a channel. Uses the simple send and receive functions.

```

#include "examples.h"

#include "general.h"
#include "utilities.h"

#include "star-dundee_types.h"
#include "star-api.h"

```

```

#include <stdlib.h>

void simpleSendExample()
{
 /* Pointer to a list of available SpaceWire devices */
 STAR_DEVICE_ID * devices = NULL;

 /* Define transfer status */
 STAR_TRANSFER_STATUS status;

 /* Define selected channel */
 int selectedChannelNumber;

 /* Get selected device */
 STAR_DEVICE_ID selectedDevice = promptForDevice(TRANSMIT_TYPE_SEND);

 /* Get selected channel */
 STAR_CHANNEL_ID selectedChannel = promptForChannel(selectedDevice,
 &selectedChannelNumber, TRANSMIT_TYPE_SEND);

 /* If channel was opened */
 if(selectedChannel)
 {
 /* Allocate array of bytes to be populated */
 unsigned char bytes[256];
 U16 readBytesLength = 0;

 /* Ask user to enter a series of bytes to send */
 printf("\nPlease enter a series of bytes to be transmitted over "
 "SpaceWire:\n");

 /* If valid bytes read input bytes from console */
 if(readBytes(bytes, 256, &readBytesLength))
 {
 /* Send packet out of channel */
 status = STAR_transmitPacket(selectedChannel, bytes,
 (unsigned int)readBytesLength, STAR_EOP_TYPE_EOP, 2);

 /* If packet was transmitted */
 if(status == STAR_TRANSFER_STATUS_COMPLETE)
 {
 /* Print success */
 printf("\nPacket was sent successfully over channel %d.\n",
 selectedChannelNumber);
 }
 else
 {
 /* Print error */
 printf("\nPacket could not be sent.\n");
 }
 }

 /* Close channel after transmitting packet */
 STAR_closeChannel(selectedChannel);
 }
 else
 {
 /* Print error */
 printf("Channel %d could not be opened.\n", selectedChannelNumber);
 }

 /* Destroy device list */
 STAR_destroyDeviceList(devices);
}

```

## 9.35 simple\_two\_way\_example.c

Two packets are constructed and then sent in either direction using two way channels and the simple transmit functions.

```

#include "examples.h"

#include "general.h"
#include "utilities.h"

#include "star-dundee_types.h"
#include "star-api.h"

void receiveSimplePacket(STAR_CHANNEL_ID channel)

```

```

{
 /* Define transfer status for receive status information */
 STAR_TRANSFER_STATUS status;

 /* Define receive buffer of maximum packet length */
 unsigned char receiveBuffer[MAX_PACKET_LENGTH];

 /* Initialise receive buffer length to size of receive
 buffer array */
 unsigned int receiveBufferLength = sizeof(receiveBuffer);

 /* Define EOP type */
 STAR_EOP_TYPE eopType = STAR_EOP_TYPE_EOP;

 /* Receive packet and wait indefinitely */
 status = STAR_receivePacket(channel, receiveBuffer,
 &receiveBufferLength, &eopType, -1);

 /* If packet was received */
 if(status == STAR_TRANSFER_STATUS_COMPLETE)
 {
 /* Print packet contents */
 printPacketContents(receiveBuffer, receiveBufferLength);
 }
}

void simpleTwoWayExample()
{
 /* Get first device */
 STAR_DEVICE_ID firstDevice = getFirstDevice();

 /* Define packet data to be sent out of first channel */
 unsigned char packetData1[] = {1, 2, 3, 4, 5, 6, 7, 8, 9};

 /* Define packet data to be sent out of second channel */
 unsigned char packetData2[] = {9, 8, 7, 6, 5, 4, 3, 2, 1};

 /* If there is a device to use */
 if(firstDevice)
 {
 /* Open first channel to send and receive on */
 STAR_CHANNEL_ID channel1 = STAR_openChannelToLocalDevice
 (
 firstDevice, STAR_CHANNEL_DIRECTION_INOUT, CHANNEL1, 1);

 /* Open second channel to send and receive on */
 STAR_CHANNEL_ID channel2 = STAR_openChannelToLocalDevice
 (
 firstDevice, STAR_CHANNEL_DIRECTION_INOUT, CHANNEL2, 1);

 /* If both channels were opened */
 if(channel1 && channel2)
 {
 /* Get packet length */
 unsigned int packetLength = sizeof(packetData1) /
 sizeof(unsigned char);

 /* Define EOP type */
 STAR_EOP_TYPE eopType = STAR_EOP_TYPE_EOP;

 /* Transmit packet out of first channel */
 STAR_TRANSFER_STATUS status =
 STAR_transmitPacket(channel1,
 packetData1, packetLength, eopType, -1);

 /* If packet was transmitted */
 if(status == STAR_TRANSFER_STATUS_COMPLETE)
 {
 /* Receive packet on second channel */
 receiveSimplePacket(channel2);

 /* Get packet length */
 packetLength = sizeof(packetData2) / sizeof(unsigned char);

 /* Transmit packet out of second channel */
 status = STAR_transmitPacket(channel2, packetData2,
 packetLength, eopType, -1);

 /* If packet was transmitted */
 if(status == STAR_TRANSFER_STATUS_COMPLETE)
 {
 /* Receive packet on first channel */
 receiveSimplePacket(channel1);
 }
 }

 /* Close first channel */

```

```

 STAR_closeChannel(channel1);

 /* Close second channel */
 STAR_closeChannel(channel2);
 }
}

```

## 9.36 star-test.c

This file contains the bulk of the functions used by the STAR-System Test program. This is provided as an example for developers wishing to write programs using STAR-API.

```

#include "star-test.h"
#include "star-api.h"
#include "cfg_api_generic.h"

#define VERSION_INFO "star-system_test v2.0"

#include <errno.h>

/* Menu option codes */
#define MENU_DISPLAY_INFORMATION 1
#define MENU_LOOPBACK_SINGLE 2
#define MENU_LOOPBACK_MULTI 3
#define MENU_LOOPBACK_DOUBLE_SINGLE 4
#define MENU_LOOPBACK_DOUBLE_MULTI 5

#define MENU_TRANSMIT_SINGLE 6
#define MENU_TRANSMIT_MULTI 7
#define MENU_RECEIVE_SINGLE 8
#define MENU_RECEIVE_MULTI 9
#define MENU_TRANSMIT_FILE_WHOLE 10
#define MENU_RECEIVE_FILE_WHOLE 11
#define MENU_TRANSMIT_FILE_SPLIT 12
#define MENU_RECEIVE_FILE_SPLIT 13
#define MENU_RESET_DEVICE 14
#define MENU_IDENTIFY_DEVICE 15

#define MENU_EXIT 0

#define STAR_INFINITE -1

void DisplayMenu(void)
{
 puts("\nSelect Option:");
 printf(" (%d) Display Device and Version Information\n",
 MENU_DISPLAY_INFORMATION);
 printf(" (%d) Loopback Test, Single Packet (Data Compare, Link Speed)\n",
 MENU_LOOPBACK_SINGLE);
 printf(" (%d) Loopback Test, Multiple Packets (Data Compare, Link Speed)\n",
 MENU_LOOPBACK_MULTI);
 printf(" (%d) Double Loopback Test, Single Packet (Data Compare, Link Speed)\n",
 MENU_LOOPBACK_DOUBLE_SINGLE);
 printf(" (%d) Double Loopback Test, Multiple Packets (Data Compare, Link Speed)\n",
 MENU_LOOPBACK_DOUBLE_MULTI);
 printf(" (%d) Transmit Single Packet (Link Speed)\n", MENU_TRANSMIT_SINGLE);
 printf(" (%d) Transmit Multiple Packets (Link Speed)\n",
 MENU_TRANSMIT_MULTI);
 printf(" (%d) Receive Packet (Link Speed)\n", MENU_RECEIVE_SINGLE);
 printf(" (%d) Receive Multiple Packets (Link Speed)\n", MENU_RECEIVE_MULTI);
 printf(" (%d) Transmit From File, Single Packet\n", MENU_TRANSMIT_FILE_WHOLE);
 printf(" (%d) Receive To File, Single Packet\n", MENU_RECEIVE_FILE_WHOLE);
 printf(" (%d) Transmit From File, Multiple Packets\n", MENU_TRANSMIT_FILE_SPLIT);
 printf(" (%d) Receive To File, Multiple Packets (Link Speed)\n", MENU_RECEIVE_FILE_SPLIT);
 printf(" (%d) Reset Device\n", MENU_RESET_DEVICE);
 printf(" (%d) Identify Device\n", MENU_IDENTIFY_DEVICE);
 printf(" (%d) Exit\n", MENU_EXIT);
 printf("Please Select Menu Option: ");
 fflush(stdout);
}

void DisplayInformation(void)
{
 STAR_VERSION_INFO *versions;
 STAR_CFG_FPGA_INFO fpgaVersion;
 int getFpgaResult = 0;

 U32 versionCount = 0U;

```

```

STAR_DEVICE_ID* devices;
U32 deviceCount = 0U;

char *deviceStr;
STAR_CHANNEL_MASK channelMask;
unsigned int deviceNum, i;

STAR_CFG_DEVICE_IDENTIFIER_INFO deviceInfo;
char manufacturerStr[STAR_CFG_MANUFACTURER_STR_MAX_LEN];
char deviceTypeStr[STAR_CFG_DEVICE_STR_MAX_LEN];
STAR_DEVICE_TYPE configCapabilities;

puts("\n\nMODULE VERSION INFORMATION");
puts("=====");

/* Get the list of module versions */
versions = STAR_getAllVersions(&versionCount);

/* If there are modules present */
if (versions != NULL)
{
 /* For each module version item in the list */
 for (i = 0U; i < versionCount; i++)
 {
 /* Display the module's version information */
 printf(" Module name: %s\n", versions[i].name);
 printf(" Module author: %s\n", versions[i].author);
 printf(" Module version: v%u.%02u", versions[i].major,
 versions[i].minor);
 if (versions[i].edit != 0U)
 {
 printf(" (%u)", versions[i].edit);
 }
 if (versions[i].patch != 0U)
 {
 printf("p%u", versions[i].patch);
 }
 puts("\n");
 }

 /* Dispose of the version list */
 STAR_destroyVersionInfoList(versions);
}
else
{
 puts("No modules present.\n");
}

puts("\n\nDEVICE INFORMATION");
puts("=====");

/* Get the list of devices */
devices = STAR_getDeviceList(&deviceCount);

/* If there are devices present */
if (devices != NULL)
{
 /* For each device in the list */
 for (deviceNum = 0U; deviceNum < deviceCount; deviceNum++)
 {
 /* Get and display the device's name */
 deviceStr = STAR_getDeviceName(devices[deviceNum]);
 if (deviceStr == NULL)
 {
 puts(" Name: Unable to get device name!");
 }
 else
 {
 printf(" Name: %s\n", deviceStr);
 STAR_destroyString(deviceStr);
 }

 /* Get and display the device's type */
 deviceStr = STAR_getDeviceTypeAsString(devices[deviceNum]);
 if (deviceStr == NULL)
 {
 puts(" Device type: Unable to get device type!");
 }
 else
 {
 printf(" Device type: %s\n", deviceStr);
 STAR_destroyString(deviceStr);
 }

 deviceStr = STAR_getDeviceBusTypeAsString(devices[deviceNum]);
 if (deviceStr == NULL)

```

```

{
 puts(" Bus type: Unable to get bus type!");
}
else
{
 printf(" Bus type: %s\n", deviceStr);
 STAR_destroyString(deviceStr);
}

/* Get and display the device's firmware version */
versions = STAR_getDeviceFirmwareVersion(devices[deviceNum]);
if (versions == NULL)
{
 puts("Firmware version: Unable to get the firmware version!");
}
else
{
 printf("Firmware version: v%u.%u", versions->major,
 versions->minor);
 if (versions->edit != 0U)
 {
 printf("(%u)", versions->edit);
 }
 if (versions->patch != 0U)
 {
 printf("p%u", versions->patch);
 }
 puts("");
 STAR_destroyVersionInfo(versions);
}

/* If the device can be configured */
configCapabilities = STAR_getDeviceConfigCapabilities(
 devices[deviceNum]);
if (configCapabilities == STAR_DEVICE_CONFIG_SUPPORTED)
{
 /* Get and display the device's FPGA version */
 getFpgaResult = CFG_getFPGAInfo(devices[deviceNum],
 &fpgaVersion);
 if (!CFG_SUCCESS(getFpgaResult))
 {
 puts(" FPGA version: Unable to get the FPGA version!");
 }
 else
 {
 printf(" FPGA version: v%u.%02u", fpgaVersion.major,
 fpgaVersion.minor);
 if (fpgaVersion.edit != 0U)
 {
 printf("(%u)", fpgaVersion.edit);
 }
 if (fpgaVersion.patch != 0U)
 {
 printf("p%u", fpgaVersion.patch);
 }
 puts("");
 }
}

/* Get and display the device's serial number */
deviceStr = STAR_getDeviceSerialNumber(devices[deviceNum]);
if (deviceStr == NULL)
{
 puts(" Serial number: Unable to get serial number!");
}
else
{
 printf(" Serial number: %s\n", deviceStr);
 STAR_destroyString(deviceStr);
}

/* Get and display the device's index */
printf(" Index: %d\n",
 STAR_getDeviceIndex(devices[deviceNum]));

/* Get and display the device's channels */
printf(" Channels:");
channelMask = STAR_getDeviceChannels(devices[deviceNum]);
if (channelMask == 0U)
{
 puts(" None");
}
else
{
 for (i = 0U; i < 32U; i++)
 {
 if (((channelMask >> i) & 1U) != 0U)

```



```

 {
 printf(" %u", i);
 }
 }
 printf("\n");
}

if (configCapabilities == STAR_DEVICE_CONFIG_SUPPORTED)
{
 memset(&deviceInfo, 0, sizeof(deviceInfo));
 if (!CFG_SUCCESS(CFG_getDeviceIdentificationInfo(
 devices[deviceNum],
 &deviceInfo)))
 {
 strcpy(manufacturerStr, "Unable to get the manufacturer!");
 strcpy(deviceTypeStr, "Unable to get the chip type!");
 }
 else
 {
 CFG_getDeviceManufacturerAsString(&deviceInfo,
 manufacturerStr);
 CFG_getDeviceTypeAsString(&deviceInfo,
 deviceTypeStr);
 }
 printf(" Manufacturer ID: %u\t(%s)\n",
 deviceInfo.manufacturerID, manufacturerStr);
 printf(" Chip Type: %u\t(%s)\n", deviceInfo.chipType,
 deviceTypeStr);
}
puts("\n");
}

/* Dispose of the device list */
STAR_destroyDeviceList(devices);
}
else
{
 puts("No devices present.\n");
}

puts("\n");
}

STAR_DEVICE_ID chooseDevice(const char * const descriptionStr,
 const STAR_DEVICE_TYPE deviceType)
{
 STAR_DEVICE_ID* devices;
 U32 deviceCount = 0U, i;
 unsigned int chosen;
 int status;

 STAR_DEVICE_ID deviceId;
 char *deviceName, s[256];

 /* Get the list of devices of the specified type */
 devices = STAR_getDeviceListForType(deviceType, &deviceCount);

 /* If there are no devices present */
 if (devices == NULL)
 {
 /* Return an error */
 puts("No SpaceWire devices detected!");
 return STAR_DEVICE_UNKNOWN;
 }

 /* Display the devices detected */
 if (deviceCount == 1U)
 {
 printf("One %s device detected:", descriptionStr);
 }
 else
 {
 printf("%u %s devices detected:\n", deviceCount, descriptionStr);
 }

 /* For each device */
 for (i = 0U; i < deviceCount; i++)
 {
 /* Display its name */
 if (devices[i] != STAR_DEVICE_UNKNOWN)
 {
 deviceName = STAR_getDeviceName(devices[i]);
 if (deviceName != NULL)
 {
 printf("\t%u - %s\n", i, deviceName);
 STAR_destroyString(deviceName);
 }
 }
 }
}

```

```

 else
 {
 printf("\t%u - Unknown SpaceWire Device\n", i);
 }
 }
 else
 {
 printf("\t%u - Unable to access device\n", i);
 }
}

/* If there's only 1 device on the list */
if (deviceCount == 1U)
{
 /* Return that device */
 deviceId = devices[0U];
 STAR_destroyDeviceList(devices);
 return deviceId;
}

/* Ask the user which device to use */
printf("Please select which %s device to use: ", descriptionStr);
fflush(stdout);
if (fgets(s, 256, stdin) == NULL)
{
 puts("No device number selected.");
 STAR_destroyDeviceList(devices);
 return STAR_DEVICE_UNKNOWN;
}
status = sscanf(s, "%u", &chosen);
if ((status == 0) || (chosen > (deviceCount - 1U)))
{
 puts("Incorrect device number selected.");
 STAR_destroyDeviceList(devices);
 return STAR_DEVICE_UNKNOWN;
}

/* Return the chosen device */
deviceId = devices[chosen];
STAR_destroyDeviceList(devices);
return deviceId;
}

unsigned char chooseChannel(const char * const descriptionStr,
 const STAR_DEVICE_ID deviceId)
{
 STAR_CHANNEL_MASK channelMask;
 unsigned int channelNumber;
 unsigned char channelCount;
 int status;
 unsigned char i;
 char s[256];

 /* Get the channels present on the device */
 channelMask = STAR_getDeviceChannels(deviceId);
 if ((channelMask == 0U) || (channelMask == 1U))
 {
 printf("ERROR: The %s device doesn't appear to have any valid channels.\n",
 descriptionStr);
 return 0U;
 }

 /* Determine the number of channels on the device, */
 /* ignoring the configuration channel */
 channelCount = 0U;
 channelNumber = 0U;
 for (i = 1U; i < 32U; i++)
 {
 if (((channelMask >> i) & 1U) != 0U)
 {
 channelCount++;
 channelNumber = i;
 }
 }

 /* If there's only one channel on the device, use this channel */
 if (channelCount == 1U)
 {
 printf("Using channel %u as the %s channel, as this is the only channel on the device.\n",
 channelNumber, descriptionStr);
 return (unsigned char)channelNumber;
 }

 /* Display the available channels */
 printf("Enter %s channel (", descriptionStr);
 for (i = 1U; i < 32U; i++)
 {

```

```

 /* If the channel exists */
 if (((channelMask >> i) & 1U) != 0U)
 {
 printf("%u", i);
 channelCount--;
 if (channelCount == 1U)
 {
 printf(" or ");
 }
 else if (channelCount > 1U)
 {
 printf(", ");
 }
 else
 {
 break;
 }
 }
 }
 printf("): ");
 fflush(stdout);

 /* Read in the channel to use */
 if (fgets(s, 256, stdin) == NULL)
 {
 puts("\nERROR: No channel specified");
 return 0U;
 }
 status = sscanf(s, "%u", &channelNumber);
 if ((status == 0) || ((1U << channelNumber) & channelMask) == 0U)
 {
 puts("\nERROR: The channel specified is not present");
 return 0U;
 }

 return (unsigned char)channelNumber;
}

int chooseDeviceAndChannel(const char * const descriptionStr,
 STAR_CHANNEL_ID * const pChannelId,
 const STAR_CHANNEL_DIRECTION direction)
{
 STAR_DEVICE_ID deviceId;
 unsigned char channelNumber;

 deviceId = chooseDevice(descriptionStr, STAR_DEVICE_TXRX_SUPPORTED);
 if (deviceId == STAR_DEVICE_UNKNOWN)
 {
 return 0;
 }

 channelNumber = chooseChannel(descriptionStr, deviceId);
 if (channelNumber == 0U)
 {
 return 0;
 }

 /* Open the channel */
 *pChannelId = STAR_openChannelToLocalDevice(deviceId, direction,
 channelNumber, TRUE);
 if ((*pChannelId) == 0U)
 {
 printf("\nERROR: Unable to open %s channel\n", descriptionStr);
 return 0;
 }

 puts("");

 return 1;
}

STAR_SPACEWIRE_ADDRESS *getTransmitPathAddress()
{
 char s[256];
 STAR_SPACEWIRE_ADDRESS *pAddress;
 unsigned char newPath[256];
 U16 pathLen = 0U;
 char *pos = (char *)s;
 unsigned long value;

 /* Ask the user to enter the path to add to the front of packets */
 puts("Enter an optional path to add to the front of the packets to be sent:");
 puts("(Values should be in hex, separated by a space, i.e.: 01 0f 02)");

 /* Read in the path */
 if (fgets(s, 256, stdin) == NULL)
 {

```

```

 puts("No address entered");
 return NULL;
 }

 /* Read until end of buffer or line feed */
 while ((*pos != '\0') && (pathLen < 256U) && (*pos != '\n'))
 {
 errno = 0;
 value = strtoul(pos, &pos, 16);
 if (errno != 0)
 {
 puts("Invalid value entered in address");
 return NULL;
 }

 newPath[pathLen] = (unsigned char)value;
 pathLen++;
 }

 /* Create a SpaceWire address from the path */
 pAddress = STAR_createAddress(newPath, pathLen);

 /* Return the completed address */
 return pAddress;
}

int getPacketSize(unsigned long * const pPacketSize)
{
 char s[256];
 int status;

 printf("Enter buffer size for each packet in bytes: ");
 fflush(stdout);
 if (fgets(s, 256, stdin) == NULL)
 {
 puts("\nERROR: No buffer size specified");
 return 0;
 }
 status = sscanf(s, "%lu", pPacketSize);
 if ((status == 0) || (*pPacketSize == 0U))
 {
 puts("\nERROR: Invalid buffer size specified");
 return 0;
 }

 return 1;
}

int chooseCheckData(int *pCompare)
{
 char s[256];

 printf("Would you like to check received data for errors? (y/n): ");
 fflush(stdout);
 if (fgets(s, 256, stdin) == NULL)
 {
 puts("ERROR: No value selected");
 return 0;
 }
 if ((s[0U] == 'y') || (s[0U] == 'Y') || (s[0U] == 't') || (s[0U] == 'T') ||
 (s[0U] == '1'))
 {
 *pCompare = 1;
 }
 else if ((s[0U] == 'n') || (s[0U] == 'N') || (s[0U] == 'f') ||
 (s[0U] == 'F') || (s[0U] == '0'))
 {
 *pCompare = 0;
 }
 else
 {
 puts("ERROR: Invalid value selected");
 return 0;
 }

 return 1;
}

int getLoopCount(unsigned long * const pLoopCount)
{
 char s[256];
 int status;

 printf("Enter the number of times to run the test: ");
 fflush(stdout);
 if (fgets(s, 256, stdin) == NULL)
 {

```

```

 puts("\nERROR: No loop count specified");
 return 0;
 }
 status = sscanf(s, "%lu", pLoopCount);
 if ((status == 0) || (*pLoopCount == 0U))
 {
 puts("\nERROR: Invalid loop count specified");
 return 0;
 }

 return 1;
}

int getPacketCount(unsigned long * const pPacketCount)
{
 char s[256];
 int status;

 printf("Enter the number of packets to transmit/receive in the test: ");
 fflush(stdout);
 if (fgets(s, 256, stdin) == NULL)
 {
 puts("\nERROR: No packet count specified");
 return 0;
 }
 status = sscanf(s, "%lu", pPacketCount);
 if ((status == 0) || (*pPacketCount == 0U))
 {
 puts("\nERROR: Invalid packet count specified");
 return 0;
 }

 return 1;
}

unsigned long comparePackets(STAR_TRANSFER_OPERATION * const pTransferOp,
 const unsigned long packetCount, const unsigned long packetSize,
 char * const pBuffer)
{
 unsigned long errorCount = 0U, i;
 unsigned int rxPacketCount;
 STAR_STREAM_ITEM* pRxStreamItem = NULL;
 char *pRxBuffer;
 U32 rxPacketLength;
 U32 bRestart;

 /* Get the number of traffic items received */
 rxPacketCount = STAR_getTransferItemCount(pTransferOp);
 if (rxPacketCount != packetCount)
 {
 printf("\nERROR expected to receive %lu packets but received %u.\n",
 packetCount, rxPacketCount);
 errorCount++;
 }
 else
 {
 {
 bRestart = TRUE;
 /* For each traffic item received */
 for (i = 0U; i < rxPacketCount; i++)
 {
 if (bRestart == TRUE)
 {
 pRxStreamItem = STAR_getTransferItem(pTransferOp, i);
 bRestart = FALSE;
 }
 else
 {
 pRxStreamItem = STAR_getNextTransferItem(pRxStreamItem);
 }

 /* Get the packet */
 if ((pRxStreamItem == NULL) || (pRxStreamItem->itemType !=
 STAR_STREAM_ITEM_TYPE_SPACEWIRE_PACKET) ||
 (pRxStreamItem->item == NULL))
 {
 printf("\nERROR received an unexpected traffic type, or empty traffic item in item %lu\n",
 i);
 errorCount++;
 bRestart = TRUE;
 }
 else
 {
 pRxBuffer = (char *)STAR_getPacketData(
 (STAR_SPACEWIRE_PACKET *)pRxStreamItem->
item,
 &rxPacketLength);
 if ((pRxBuffer == NULL) || (rxPacketLength != packetSize))

```

```

 {
 printf("\nERROR received a packet of length %u, expected length %lu in item %lu\n",
 rxPacketLength, packetSize, i);
 errorCount++;
 }
 else
 {
 /* Compare the buffers and increment the error count if */
 /* the buffers do not match */
 errorCount += BufferCompareChar(pBuffer + (packetSize * i),
 pRxBuffer, packetSize);
 }
 if (pRxBuffer != NULL)
 {
 STAR_destroyPacketData((unsigned char *)pRxBuffer);
 }
 }
}

/* Return the error count */
return errorCount;
}

unsigned long getPacketsLengths(STAR_TRANSFER_OPERATION * const pTransferOp,
 const unsigned long packetCount)
{
 unsigned long rxPacketLength = 0U, i;
 unsigned int rxPacketCount;
 STAR_STREAM_ITEM *pRxStreamItem;

 /* Get the number of traffic items received */
 rxPacketCount = STAR_getTransferItemCount(pTransferOp);
 if (rxPacketCount != packetCount)
 {
 printf("\nERROR expected to receive %lu packets but received %u.\n",
 packetCount, rxPacketCount);
 }

 /* For each traffic item received */
 for (i = 0U; i < rxPacketCount; i++)
 {
 /* Get the packet */
 pRxStreamItem = STAR_getTransferItem(pTransferOp, i);
 if ((pRxStreamItem == NULL) || (pRxStreamItem->itemType !=
 STAR_STREAM_ITEM_TYPE_SPACEWIRE_PACKET) ||
 (pRxStreamItem->item == NULL))
 {
 printf("\nERROR received an unexpected traffic type, or empty traffic item in item %lu\n",
 i);
 }
 else
 {
 rxPacketLength += STAR_getPacketLength(
 (STAR_SPACEWIRE_PACKET *)pRxStreamItem->
 item);
 }
 }

 /* Return the total packet length */
 return rxPacketLength;
}

void DisplayResults(const clock_t start, const clock_t finish,
 const int compared, const unsigned long byteSize,
 const unsigned long packetCount, const unsigned long loopCount,
 const unsigned long errorCount, const char * const descriptionStr)
{
 double bitsSent, duration;

 puts("Test complete.");

 /* Calculate the time taken and the number of bits sent */
 duration = (double)(finish - start) / TIME_DIVIDER;
 bitsSent = (double)byteSize * (double)8 * (double)packetCount *
 (double)loopCount;

 /* Display the data rates */
 printf("\n**** %s, Results **** \n", descriptionStr);
 printf("\tTime Taken = %9.5g Seconds\n", duration);
 printf("\tAverage Speed = %9.2E bit/s (%3.2f Mbit/s)\n\n",
 bitsSent / duration, (bitsSent / duration) / 1000000.0);

 /* Display the number of errors encountered */
 if (compared != 0)
 {
 printf("Total Errors = 0x%-7lx \n", errorCount);
 }
}

```

```

 }

 /* Display whether the test was successful */
 if (errorCount == 0U)
 {
 puts("Test successful.\n");
 }
 else
 {
 puts("Test failed.\n");
 }
}

void LoopBack_SinglePacket(void)
{
 STAR_CHANNEL_ID rxChannelId = 0U, txChannelId = 0U;
 char *pTxBuffer = NULL;
 STAR_TRANSFER_OPERATION *pTxTransferOp = NULL, *pRxTransferOp = NULL;
 STAR_STREAM_ITEM *pTxStreamItem = NULL;
 unsigned long loopCount = 0U, errorCount = 0U, byteSize = 0U, testNum;
 clock_t start, finish;
 int compare;
 STAR_TRANSFER_STATUS rxStatus, txStatus;
 STAR_SPACEWIRE_ADDRESS *pAddressPath;

 /* Select the transmit device and channel to be used */
 if (chooseDeviceAndChannel("transmit", &txChannelId,
 STAR_CHANNEL_DIRECTION_OUT) == 0)
 {
 return;
 }

 /* Select the path to be added to the front of packets transmitted */
 pAddressPath = getTransmitPathAddress();

 /* Select the receive device and channel to be used */
 if (chooseDeviceAndChannel("receive", &rxChannelId,
 STAR_CHANNEL_DIRECTION_IN) == 0)
 {
 goto LoopBack_SinglePacket_Finish;
 }

 /* Get packet size */
 if (getPacketSize(&byteSize) == 0)
 {
 goto LoopBack_SinglePacket_Finish;
 }

 /* Get loop count */
 if (getLoopCount(&loopCount) == 0)
 {
 goto LoopBack_SinglePacket_Finish;
 }

 /* Compare? */
 if (chooseCheckData(&compare) == 0)
 {
 goto LoopBack_SinglePacket_Finish;
 }

 /* Allocate memory for the transmit buffer */
 pTxBuffer = (char *)calloc(1U, byteSize);
 if (pTxBuffer == NULL)
 {
 puts("\nERROR: Unable to allocate memory for transmit buffer");
 goto LoopBack_SinglePacket_Finish;
 }

 /* Fill transmit buffer with random data */
 FillBufferRandomChar(pTxBuffer, byteSize, DATA_TYPE_RANDOM);

 /* Create receive operation to receive 1 packet */
 pRxTransferOp = STAR_createRxOperation(1,
 STAR_RECEIVE_PACKETS);
 if (pRxTransferOp == NULL)
 {
 puts("\nERROR: Unable to create receive operation");
 goto LoopBack_SinglePacket_Finish;
 }

 /* Create the packet to be transmitted */
 pTxStreamItem = STAR_createPacket(pAddressPath, (U8 *)pTxBuffer, byteSize,
 STAR_EOP_TYPE_EOP);
 if (pTxStreamItem == NULL)
 {
 puts("\nERROR: Unable to create the packet to be transmitted");
 goto LoopBack_SinglePacket_Finish;
 }

```

```

}

/* Create the transmit transfer operation for the packet */
pTxTransferOp = STAR_createTxOperation(&pTxStreamItem, 1U);
if (pTxTransferOp == NULL)
{
 puts("\nERROR: Unable to create the transfer operation to be transmitted");
 goto LoopBack_SinglePacket_Finish;
}

puts("Running Single-Packet Loopback Test...");

/* Start time */
start = GET_TIME();

for (testNum = 0U; testNum < loopCount; testNum++)
{
 /* Submit the receive operation */
 if (STAR_submitTransferOperation(rxChannelId, pRxTransferOp) == 0)
 {
 printf("\nERROR occurred during receive. Test %lu failed.\n",
 testNum + 1U);
 goto LoopBack_SinglePacket_Finish;
 }

 /* Submit the transmit operation */
 if (STAR_submitTransferOperation(txChannelId, pTxTransferOp) == 0)
 {
 printf("\nERROR occurred during transmit. Test %lu failed.\n",
 testNum + 1U);
 goto LoopBack_SinglePacket_Finish;
 }

 /* Wait on the transmit operation completing */
 txStatus = STAR_waitOnTransferOperationCompletion(
 pTxTransferOp,
 STAR_INFINITE);
 if (txStatus != STAR_TRANSFER_STATUS_COMPLETE)
 {
 printf("\nERROR occurred during transmit. Test %lu failed.\n",
 testNum + 1U);
 goto LoopBack_SinglePacket_Finish;
 }

 /* Wait on the receive operation completing */
 rxStatus = STAR_waitOnTransferOperationCompletion(
 pRxTransferOp,
 STAR_INFINITE);
 if (rxStatus != STAR_TRANSFER_STATUS_COMPLETE)
 {
 printf("\nERROR occurred during receive. Test %lu failed.\n",
 testNum + 1U);
 goto LoopBack_SinglePacket_Finish;
 }

 /* If the received packet is to be compared */
 if (compare != 0)
 {
 /* Compare the received packet to the buffer transmitted */
 errorCount += comparePackets(pRxTransferOp, 1U, byteSize,
 pTxBuffer);
 }
 printf("\rTest - %lu", testNum + 1);
}
printf("\n");

/* End time */
finish = GET_TIME();

/* Display the results */
DisplayResults(start, finish, compare, byteSize, 1U, loopCount, errorCount,
 "Single-Packet Loopback Test");

LoopBack_SinglePacket_Finish:

/* Dispose of the transfer operations */
if (pTxTransferOp != NULL)
{
 STAR_disposeTransferOperation(pTxTransferOp);
}
if (pRxTransferOp != NULL)
{
 STAR_disposeTransferOperation(pRxTransferOp);
}

/* Destroy the packet transmitted */
if (pTxStreamItem != NULL)

```



```

 {
 STAR_destroyStreamItem(pTxStreamItem);
 }

 /* Free the transmit buffer */
 if (pTxBuffer != NULL)
 {
 free(pTxBuffer);
 }

 /* Free the address path */
 if (pAddressPath != NULL)
 {
 STAR_destroyAddress(pAddressPath);
 }

 /* Close the channels */
 if (rxChannelId != 0U)
 {
 STAR_closeChannel(rxChannelId);
 }
 if (txChannelId != 0U)
 {
 STAR_closeChannel(txChannelId);
 }
}

void LoopBack_MultiPacket(void)
{
 STAR_CHANNEL_ID rxChannelId = 0U, txChannelId = 0U;
 unsigned long loopCount, errorCount = 0U, byteSize, testNum;
 unsigned long i, packetCount = 0U;
 int compare;
 char *pTxBuffer = NULL;
 STAR_STREAM_ITEM **ppTxStreamItems = NULL;
 STAR_TRANSFER_OPERATION *pTxTransferOp = NULL, *pRxTransferOp = NULL;
 STAR_TRANSFER_STATUS rxStatus, txStatus;
 clock_t start, finish;
 STAR_SPACEWIRE_ADDRESS *pAddressPath;

 /* Select the transmit device and channel to be used */
 if (chooseDeviceAndChannel("transmit", &txChannelId,
 STAR_CHANNEL_DIRECTION_OUT) == 0)
 {
 return;
 }

 /* Select the path to be added to the front of packets transmitted */
 pAddressPath = getTransmitPathAddress();

 /* Select the receive device and channel to be used */
 if (chooseDeviceAndChannel("receive", &rxChannelId,
 STAR_CHANNEL_DIRECTION_IN) == 0)
 {
 goto LoopBack_MultiPacket_Finish;
 }

 /* Get packet size */
 if (getPacketSize(&byteSize) == 0)
 {
 goto LoopBack_MultiPacket_Finish;
 }

 /* Get packet count */
 if (getPacketCount(&packetCount) == 0)
 {
 goto LoopBack_MultiPacket_Finish;
 }

 /* Get loop count */
 if (getLoopCount(&loopCount) == 0)
 {
 goto LoopBack_MultiPacket_Finish;
 }

 /* Compare? */
 if (chooseCheckData(&compare) == 0)
 {
 goto LoopBack_MultiPacket_Finish;
 }

 /* Allocate memory for the transmit buffer */
 pTxBuffer = (char *)calloc(1U, byteSize * packetCount);
 if (pTxBuffer == NULL)
 {
 puts("\nERROR: Unable to allocate memory for transmit buffer");
 goto LoopBack_MultiPacket_Finish;
 }

```

```

}

/* Fill transmit buffer with random data */
FillBufferRandomChar(pTxBuffer, byteSize * packetCount, DATA_TYPE_RANDOM);

/* Create receive operation to receive the packets */
pRxTransferOp = STAR_createRxOperation(packetCount,
 STAR_RECEIVE_PACKETS);
if (pRxTransferOp == NULL)
{
 puts("\nERROR: Unable to create receive operation");
 goto LoopBack_MultiPacket_Finish;
}

/* Allocate memory for the transmit stream item pointers */
ppTxStreamItems = (STAR_STREAM_ITEM **)calloc(packetCount,
 sizeof(STAR_STREAM_ITEM *));
if (ppTxStreamItems == NULL)
{
 puts("\nERROR: Unable to allocate memory for transmit stream items");
 goto LoopBack_MultiPacket_Finish;
}

/* For each packet */
for (i = 0U; i < packetCount; i++)
{
 /* Create the stream item */
 ppTxStreamItems[i] = STAR_createPacket(pAddressPath,
 (U8 *)pTxBuffer + (byteSize * i), byteSize, STAR_EOP_TYPE_EOP);
 if (ppTxStreamItems[i] == NULL)
 {
 printf("\nERROR: Unable to create packet %lu to be transmitted\n", i);
 goto LoopBack_MultiPacket_Finish;
 }
}

/* Create the transmit transfer operation for the packets */
pTxTransferOp = STAR_createTxOperation(ppTxStreamItems, packetCount);
if (pTxTransferOp == NULL)
{
 puts("\nERROR: Unable to create the transfer operation to be transmitted");
 goto LoopBack_MultiPacket_Finish;
}

puts("Running Multi-Packet Loopback Test...");

/* Start time */
start = GET_TIME();

for (testNum = 0U; testNum < loopCount; testNum++)
{
 /* Submit the receive operation */
 if (STAR_submitTransferOperation(rxChannelId,
 pRxTransferOp) == 0)
 {
 printf("\nERROR occurred during receive. Test %lu failed.\n",
 testNum + 1U);
 goto LoopBack_MultiPacket_Finish;
 }

 /* Submit the transmit operation */
 if (STAR_submitTransferOperation(txChannelId, pTxTransferOp) == 0)
 {
 printf("\nERROR occurred during transmit. Test %lu failed.\n",
 testNum + 1U);
 goto LoopBack_MultiPacket_Finish;
 }

 /* Wait on the transmit operation completing */
 txStatus = STAR_waitOnTransferOperationCompletion(
 pTxTransferOp,
 STAR_INFINITE);
 if (txStatus != STAR_TRANSFER_STATUS_COMPLETE)
 {
 printf("\nERROR occurred during transmit. Test %lu failed.\n",
 testNum + 1U);
 goto LoopBack_MultiPacket_Finish;
 }

 /* Wait on the receive operation completing */
 rxStatus = STAR_waitOnTransferOperationCompletion(
 pRxTransferOp,
 STAR_INFINITE);
 if (rxStatus != STAR_TRANSFER_STATUS_COMPLETE)
 {
 printf("\nERROR occurred during receive. Test %lu failed.\n",
 testNum + 1U);

```

```

 goto LoopBack_MultiPacket_Finish;
 }

 /* If the received packets are to be compared */
 if (compare != 0)
 {
 /* Compare the received packets to the buffer transmitted */
 errorCount += comparePackets(pRxTransferOp, packetCount, byteSize,
 pTxBuffer);
 }
 printf("\rTest - %lu", testNum + 1);
}
printf("\n");

/* End time */
finish = GET_TIME();

/* Display the results */
DisplayResults(start, finish, compare, byteSize, packetCount, loopCount,
 errorCount, "Multiple-Packet Loopback Test");

LoopBack_MultiPacket_Finish:

/* Dispose of the transfer operations */
if (pTxTransferOp != NULL)
{
 STAR_disposeTransferOperation(pTxTransferOp);
}
if (pRxTransferOp != NULL)
{
 STAR_disposeTransferOperation(pRxTransferOp);
}

/* Destroy the packets transmitted */
if (ppTxStreamItems != NULL)
{
 for (i = 0U; i < packetCount; i++)
 {
 if (ppTxStreamItems[i] != NULL)
 {
 STAR_destroyStreamItem(ppTxStreamItems[i]);
 }
 }
 free(ppTxStreamItems);
}

/* Free the transmit buffer */
if (pTxBuffer != NULL)
{
 free(pTxBuffer);
}

/* Free the address path */
if (pAddressPath != NULL)
{
 STAR_destroyAddress(pAddressPath);
}

/* Close the opened channels */
if (rxChannelId != 0U)
{
 STAR_closeChannel(rxChannelId);
}
if (txChannelId != 0U)
{
 STAR_closeChannel(txChannelId);
}
}

void LoopBackDouble_SinglePacket(void)
{
 STAR_CHANNEL_ID channelId[2] = { 0U, 0U };
 unsigned long loopCount, errorCount = 0U, byteSize, testNum, i;
 int compare;
 char *pTxBuffer[2] = { NULL, NULL };
 STAR_STREAM_ITEM *pTxStreamItem[2] = { NULL, NULL };
 STAR_TRANSFER_OPERATION *pTxTransferOp[2] = { NULL, NULL };
 STAR_TRANSFER_OPERATION *pRxTransferOp[2] = { NULL, NULL };
 STAR_TRANSFER_OPERATION *pTransferOpList[2][2] = { { NULL } };
 STAR_TRANSFER_STATUS rxStatus, txStatus;
 clock_t start, finish;
 STAR_SPACEWIRE_ADDRESS *pAddressPath[2] = { NULL, NULL };

 /* Select the first device and channel to be used */
 if (chooseDeviceAndChannel("first", &channelId[0U],
 STAR_CHANNEL_DIRECTION_INOUT) == 0)
 {

```

```

 return;
}

/* Select the first path to be added to the front of packets transmitted */
pAddressPath[0U] = getTransmitPathAddress();

/* Select the second device and channel to be used */
if (chooseDeviceAndChannel("second", &channelId[1U],
 STAR_CHANNEL_DIRECTION_INOUT) == 0)
{
 goto LoopBackDouble_SinglePacket_Finish;
}

/* Select the second path to be added to the front of packets transmitted */
pAddressPath[1U] = getTransmitPathAddress();

/* Get packet size */
if (getPacketSize(&byteSize) == 0)
{
 goto LoopBackDouble_SinglePacket_Finish;
}

/* Get loop count */
if (getLoopCount(&loopCount) == 0)
{
 goto LoopBackDouble_SinglePacket_Finish;
}

/* Compare? */
if (chooseCheckData(&compare) == 0)
{
 goto LoopBackDouble_SinglePacket_Finish;
}

/* For each channel */
for (i = 0U; i < 2U; i++)
{
 /* Allocate memory for the transmit buffer */
 pTxBuffer[i] = (char *)calloc(1U, byteSize);
 if (pTxBuffer[i] == NULL)
 {
 puts("\nERROR: Unable to allocate memory for transmit buffer");
 goto LoopBackDouble_SinglePacket_Finish;
 }

 /* Fill transmit buffer with random data */
 FillBufferRandomChar(pTxBuffer[i], byteSize, DATA_TYPE_RANDOM);

 /* Create receive operation to receive 1 packet */
 pRxTransferOp[i] = STAR_createRxOperation(1,
 STAR_RECEIVE_PACKETS);
 if (pRxTransferOp[i] == NULL)
 {
 puts("\nERROR: Unable to create receive operation");
 goto LoopBackDouble_SinglePacket_Finish;
 }

 /* Create the packet to be transmitted */
 pTxStreamItem[i] = STAR_createPacket(pAddressPath[i],
 (U8 *)pTxBuffer[i], byteSize, STAR_EOP_TYPE_EOP);
 if (pTxStreamItem[i] == NULL)
 {
 puts("\nERROR: Unable to create the packet to be transmitted");
 goto LoopBackDouble_SinglePacket_Finish;
 }

 /* Create the transmit transfer operation for the packet */
 pTxTransferOp[i] = STAR_createTxOperation(&pTxStreamItem[i], 1U);
 if (pTxTransferOp[i] == NULL)
 {
 puts("\nERROR: Unable to create the transfer operation to be transmitted");
 goto LoopBackDouble_SinglePacket_Finish;
 }

 /* Add transfer operation to list for channel */
 pTransferOpList[i][0U] = pRxTransferOp[i];
 pTransferOpList[i][1U] = pTxTransferOp[i];
}

puts("Starting Single-Packet Double Loopback Test...");

/* Start time */
start = GET_TIME();

for (testNum = 0U; testNum < loopCount; testNum++)
{
 /* Submit the receive operations */

```

```

/* First Channel Rx */
if (STAR_submitTransferOperation(channelId[0U],
 pTransferOpList[0U][0U]) == 0)
{
 printf("\nERROR occurred submitting operation list. Test %lu failed.\n",
 testNum + 1U);
 goto LoopBackDouble_SinglePacket_Finish;
}
/* Second Channel Rx */
if (STAR_submitTransferOperation(channelId[1U],
 pTransferOpList[1U][0U]) == 0)
{
 printf("\nERROR occurred submitting operation list. Test %lu failed.\n",
 testNum + 1U);
 goto LoopBackDouble_SinglePacket_Finish;
}

/* Submit the transmit operations */

/* First Channel Tx */
if (STAR_submitTransferOperation(channelId[0U],
 pTransferOpList[0U][1U]) == 0)
{
 printf("\nERROR occurred submitting operation list. Test %lu failed.\n",
 testNum + 1U);
 goto LoopBackDouble_SinglePacket_Finish;
}
/* Second Channel Tx */
if (STAR_submitTransferOperation(channelId[1U],
 pTransferOpList[1U][1U]) == 0)
{
 printf("\nERROR occurred submitting operation list. Test %lu failed.\n",
 testNum + 1U);
 goto LoopBackDouble_SinglePacket_Finish;
}

/* For each channel */
for (i = 0U; i < 2U; i++)
{
 /* Wait on the transmit operation completing */
 txStatus = STAR_waitOnTransferOperationCompletion(
pTxTransferOp[i],
 STAR_INFINITE);
 if (txStatus != STAR_TRANSFER_STATUS_COMPLETE)
 {
 printf("\nERROR occurred during transmit. Test %lu failed.\n",
 testNum + 1U);
 goto LoopBackDouble_SinglePacket_Finish;
 }
}

/* For each channel */
for (i = 0U; i < 2U; i++)
{
 /* Wait on the receive operation completing */
 rxStatus = STAR_waitOnTransferOperationCompletion(
pRxTransferOp[i],
 STAR_INFINITE);
 if (rxStatus != STAR_TRANSFER_STATUS_COMPLETE)
 {
 printf("\nERROR occurred during receive. Test %lu failed.\n",
 testNum + 1U);
 goto LoopBackDouble_SinglePacket_Finish;
 }
}

if (compare != 0)
{
 /* Compare the received packets to the buffer transmitted */
 errorCount += comparePackets(pRxTransferOp[1U], 1U, byteSize,
 pTxBuffer[0U]);
 errorCount += comparePackets(pRxTransferOp[0U], 1U, byteSize,
 pTxBuffer[1U]);
}
printf("\rTest - %lu", testNum + 1);
}
printf("\n");

/* End time */
finish = GET_TIME();

/* Display the results */
DisplayResults(start, finish, compare, byteSize, 1U, loopCount, errorCount,
 "Single-Packet Double Loopback Test");

LoopBackDouble_SinglePacket_Finish:

```

```

/* For each channel */
for (i = 0U; i < 2U; i++)
{
 /* Dispose of the transfer operations */
 STAR_disposeTransferOperation(pTransferOpList[i][0U]);
 STAR_disposeTransferOperation(pTransferOpList[i][1U]);

 /* Destroy the packet transmitted */
 if (pTxStreamItem[i] != NULL)
 {
 STAR_destroyStreamItem(pTxStreamItem[i]);
 }

 /* Free the transmit buffer */
 if (pTxBuffer[i] != NULL)
 {
 free(pTxBuffer[i]);
 }

 /* Free the address path */
 if (pAddressPath[i] != NULL)
 {
 STAR_destroyAddress(pAddressPath[i]);
 }

 /* Close the channel */
 if (channelId[i] != 0U)
 {
 STAR_closeChannel(channelId[i]);
 }
}

void LoopBackDouble_MultiPacket(void)
{
 STAR_CHANNEL_ID channelId[2] = { 0U, 0U };
 unsigned long loopCount, errorCount = 0U, byteSize, testNum, i;
 unsigned long packetNum, packetCount = 0U;
 int compare;
 char *pTxBuffer[2] = { NULL, NULL };
 STAR_STREAM_ITEM **ppTxStreamItems[2] = { NULL, NULL };
 STAR_TRANSFER_OPERATION *pTxTransferOp[2] = { NULL, NULL };
 STAR_TRANSFER_OPERATION *pRxTransferOp[2] = { NULL, NULL };
 STAR_TRANSFER_OPERATION *pTransferOpList[2][2] = { { NULL } };
 STAR_TRANSFER_STATUS rxStatus, txStatus;
 clock_t start, finish;
 STAR_SPACEWIRE_ADDRESS *pAddressPath[2] = { NULL, NULL };

 /* Select the first device and channel to be used */
 if (chooseDeviceAndChannel("first", &channelId[0U],
 STAR_CHANNEL_DIRECTION_INOUT) == 0)
 {
 return;
 }

 /* Select the first path to be added to the front of packets transmitted */
 pAddressPath[0U] = getTransmitPathAddress();

 /* Select the second device and channel to be used */
 if (chooseDeviceAndChannel("second", &channelId[1U],
 STAR_CHANNEL_DIRECTION_INOUT) == 0)
 {
 goto LoopBackDouble_MultiPacket_Finish;
 }

 /* Select the second path to be added to the front of packets transmitted */
 pAddressPath[1U] = getTransmitPathAddress();

 /* Get packet size */
 if (getPacketSize(&byteSize) == 0)
 {
 goto LoopBackDouble_MultiPacket_Finish;
 }

 /* Get packet count */
 if (getPacketCount(&packetCount) == 0)
 {
 goto LoopBackDouble_MultiPacket_Finish;
 }

 /* Get loop count */
 if (getLoopCount(&loopCount) == 0)
 {
 goto LoopBackDouble_MultiPacket_Finish;
 }
}

```

```

/* Compare? */
if (chooseCheckData(&compare) == 0)
{
 goto LoopBackDouble_MultiPacket_Finish;
}

/* For each channel */
for (i = 0U; i < 2U; i++)
{
 /* Allocate memory for the transmit buffer */
 pTxBuffer[i] = (char *)calloc(1U, byteSize * packetCount);
 if (pTxBuffer[i] == NULL)
 {
 puts("\nERROR: Unable to allocate memory for transmit buffer");
 goto LoopBackDouble_MultiPacket_Finish;
 }

 /* Fill transmit buffer with random data */
 FillBufferRandomChar(pTxBuffer[i], byteSize * packetCount,
 DATA_TYPE_RANDOM);

 /* Create receive operation to receive the packets */
 pRxTransferOp[i] = STAR_createRxOperation(packetCount,
 STAR_RECEIVE_PACKETS);
 if (pRxTransferOp[i] == NULL)
 {
 puts("\nERROR: Unable to create receive operation");
 goto LoopBackDouble_MultiPacket_Finish;
 }

 /* Allocate memory for the transmit stream item pointers */
 ppTxStreamItems[i] = (STAR_STREAM_ITEM **)calloc(packetCount,
 sizeof(STAR_STREAM_ITEM *));
 if (ppTxStreamItems[i] == NULL)
 {
 puts("\nERROR: Unable to allocate memory for transmit stream items");
 goto LoopBackDouble_MultiPacket_Finish;
 }

 /* For each packet */
 for (packetNum = 0U; packetNum < packetCount; packetNum++)
 {
 /* Create the packet to be transmitted */
 ppTxStreamItems[i][packetNum] = STAR_createPacket(pAddressPath[i],
 (U8 *)pTxBuffer[i] + (byteSize * packetNum), byteSize,
 STAR_EOP_TYPE_EOP);
 if (ppTxStreamItems[i][packetNum] == NULL)
 {
 printf("\nERROR: Unable to create the packet %lu to be transmitted\n",
 packetNum);
 goto LoopBackDouble_MultiPacket_Finish;
 }
 }

 /* Create the transmit transfer operation for the packets */
 pTxTransferOp[i] = STAR_createTxOperation(ppTxStreamItems[i],
 packetCount);
 if (pTxTransferOp[i] == NULL)
 {
 puts("\nERROR: Unable to create the transfer operation to be transmitted");
 goto LoopBackDouble_MultiPacket_Finish;
 }

 /* Add transfer operation to list for channel */
 pTransferOpList[i][0U] = pRxTransferOp[i];
 pTransferOpList[i][1U] = pTxTransferOp[i];
}

puts("Running Multi-Packet Double Loopback Test...");

/* Start time */
start = GET_TIME();

for (testNum = 0U; testNum < loopCount; testNum++)
{
 /* Submit the receive operations */

 /* First Channel Rx */
 if (STAR_submitTransferOperation(channelId[0U],
 pTransferOpList[0U][0U]) == 0)
 {
 printf("\nERROR occurred submitting operation list. Test %lu failed.\n",
 testNum + 1U);
 goto LoopBackDouble_MultiPacket_Finish;
 }

 /* Second Channel Rx */
 if (STAR_submitTransferOperation(channelId[1U],

```

```

 pTransferOpList[1U][0U]) == 0)
 {
 printf("\nERROR occurred submitting operation list. Test %lu failed.\n",
 testNum + 1U);
 goto LoopBackDouble_MultiPacket_Finish;
 }

 /* Submit the transmit operations */

 /* First Channel Tx */
 if (STAR_submitTransferOperation(channelId[0U],
 pTransferOpList[0U][1U]) == 0)
 {
 printf("\nERROR occurred submitting operation list. Test %lu failed.\n",
 testNum + 1U);
 goto LoopBackDouble_MultiPacket_Finish;
 }
 /* Second Channel Tx */
 if (STAR_submitTransferOperation(channelId[1U],
 pTransferOpList[1U][1U]) == 0)
 {
 printf("\nERROR occurred submitting operation list. Test %lu failed.\n",
 testNum + 1U);
 goto LoopBackDouble_MultiPacket_Finish;
 }

 /* For each channel */
 for (i = 0U; i < 2U; i++)
 {
 /* Wait on the transmit operation completing */
 txStatus = STAR_waitOnTransferOperationCompletion(
 pTxTransferOp[i],
 STAR_INFINITE);
 if (txStatus != STAR_TRANSFER_STATUS_COMPLETE)
 {
 printf("\nERROR occurred during transmit. Test %lu failed.\n",
 testNum + 1U);
 goto LoopBackDouble_MultiPacket_Finish;
 }
 }

 /* For each channel */
 for (i = 0U; i < 2U; i++)
 {
 /* Wait on the receive operation completing */
 rxStatus = STAR_waitOnTransferOperationCompletion(
 pRxTransferOp[i],
 STAR_INFINITE);
 if (rxStatus != STAR_TRANSFER_STATUS_COMPLETE)
 {
 printf("\nERROR occurred during receive. Test %lu failed - status %d.\n",
 testNum + 1U, rxStatus);
 goto LoopBackDouble_MultiPacket_Finish;
 }
 }

 if (compare != 0)
 {
 /* Compare the received packets to the buffer transmitted */
 errorCount += comparePackets(pRxTransferOp[1U], packetCount,
 byteSize, pTxBuffer[0U]);
 errorCount += comparePackets(pRxTransferOp[0U], packetCount,
 byteSize, pTxBuffer[1U]);
 }
 printf("\rTest - %lu", testNum + 1);
}
printf("\n");

/* End time */
finish = GET_TIME();

/* Display the results */
DisplayResults(start, finish, compare, byteSize, packetCount, loopCount,
 errorCount, "Multi-Packet Double Loopback Test");

LoopBackDouble_MultiPacket_Finish:

/* For each channel */
for (i = 0U; i < 2U; i++)
{
 /* Dispose of transfer operations, */
 STAR_disposeTransferOperation(pTransferOpList[i][0U]);
 STAR_disposeTransferOperation(pTransferOpList[i][1U]);

 /* Destroy the packets transmitted */
 if (ppTxStreamItems[i] != NULL)
 {

```



```

 for (packetNum = 0U; packetNum < packetCount; packetNum++)
 {
 if (ppTxStreamItems[i][packetNum] != NULL)
 {
 STAR_destroyStreamItem(ppTxStreamItems[i][packetNum]);
 }
 }
 free(ppTxStreamItems[i]);
 }

 /* Free the transmit buffer */
 if (pTxBuffer[i] != NULL)
 {
 free(pTxBuffer[i]);
 }

 /* Free the address path */
 if (pAddressPath[i] != NULL)
 {
 STAR_destroyAddress(pAddressPath[i]);
 }

 /* Close the channel */
 if (channelId[i] != 0U)
 {
 STAR_closeChannel(channelId[i]);
 }
}

void Transmit_SinglePacket(void)
{
 STAR_CHANNEL_ID txChannelId = 0U;
 char *pTxBuffer = NULL;
 STAR_TRANSFER_OPERATION *pTxTransferOp = NULL;
 STAR_STREAM_ITEM *pTxStreamItem = NULL;
 unsigned long loopCount, byteSize, testNum;
 clock_t start, finish;
 STAR_TRANSFER_STATUS txStatus;
 STAR_SPACEWIRE_ADDRESS *pAddressPath;

 /* Select the transmit device and channel to be used */
 if (chooseDeviceAndChannel("transmit", &txChannelId,
 STAR_CHANNEL_DIRECTION_OUT) == 0)
 {
 return;
 }

 /* Select the path to be added to the front of packets transmitted */
 pAddressPath = getTransmitPathAddress();

 /* Get packet size */
 if (getPacketSize(&byteSize) == 0)
 {
 goto Transmit_SinglePacket_Finish;
 }

 /* Get loop count */
 if (getLoopCount(&loopCount) == 0)
 {
 goto Transmit_SinglePacket_Finish;
 }

 /* Allocate memory for the transmit buffer */
 pTxBuffer = (char *)calloc(1U, byteSize);
 if (pTxBuffer == NULL)
 {
 puts("\nERROR: Unable to allocate memory for transmit buffer");
 goto Transmit_SinglePacket_Finish;
 }

 /* Fill transmit buffer with random data, receive buffer with zeros */
 FillBufferRandomChar(pTxBuffer, byteSize, DATA_TYPE_RANDOM);

 /* Create the packet to be transmitted */
 pTxStreamItem = STAR_createPacket(pAddressPath, (U8 *)pTxBuffer, byteSize,
 STAR_EOP_TYPE_EOP);
 if (pTxStreamItem == NULL)
 {
 puts("\nERROR: Unable to create the packet to be transmitted");
 goto Transmit_SinglePacket_Finish;
 }

 /* Create the transmit transfer operation for the packet */
 pTxTransferOp = STAR_createTxOperation(&pTxStreamItem, 1U);
 if (pTxTransferOp == NULL)
 {

```

```

 puts("\nERROR: Unable to create the transfer operation to be transmitted");
 goto Transmit_SinglePacket_Finish;
 }

 puts("Starting Single-Packet Transmit Test...");

 /* Start time */
 start = GET_TIME();

 for (testNum = 0U; testNum < loopCount; testNum++)
 {
 /* Submit the transmit operation */
 if (STAR_submitTransferOperation(txChannelId, pTxTransferOp) == 0)
 {
 printf("\nERROR occurred during transmit. Test %lu failed.\n",
 testNum + 1U);
 goto Transmit_SinglePacket_Finish;
 }

 /* Wait on the transmit operation completing */
 txStatus = STAR_waitOnTransferOperationCompletion(
 pTxTransferOp,
 STAR_INFINITE);
 if (txStatus != STAR_TRANSFER_STATUS_COMPLETE)
 {
 printf("\nERROR occurred during transmit. Test %lu failed.\n",
 testNum + 1U);
 goto Transmit_SinglePacket_Finish;
 }
 printf("\rTest - %lu", testNum + 1);
 }
 printf("\n");

 /* End time */
 finish = GET_TIME();

 /* Display the results */
 DisplayResults(start, finish, 0, byteSize, 1U, loopCount, 0U,
 "Single-Packet Transmit Test");

Transmit_SinglePacket_Finish:

 /* Dispose of the transfer operation */
 if (pTxTransferOp != NULL)
 {
 STAR_disposeTransferOperation(pTxTransferOp);
 }

 /* Destroy the packet transmitted */
 if (pTxStreamItem != NULL)
 {
 STAR_destroyStreamItem(pTxStreamItem);
 }

 /* Free the transmit buffer */
 if (pTxBuffer != NULL)
 {
 free(pTxBuffer);
 }

 /* Free the address */
 if (pAddressPath != NULL)
 {
 STAR_destroyAddress(pAddressPath);
 }

 /* Close the channel */
 if (txChannelId != 0u)
 {
 STAR_closeChannel(txChannelId);
 }
}

void Transmit_MultiPacket(void)
{
 STAR_CHANNEL_ID txChannelId = 0U;
 unsigned long loopCount, byteSize, testNum, packetCount = 0U, i;
 char *pTxBuffer = NULL;
 STAR_STREAM_ITEM **ppTxStreamItems = NULL;
 STAR_TRANSFER_OPERATION *pTxTransferOp = NULL;
 STAR_TRANSFER_STATUS txStatus;
 clock_t start, finish;
 STAR_SPACEWIRE_ADDRESS *pAddressPath;

 /* Select the transmit device and channel to be used */
 if (chooseDeviceAndChannel("transmit", &txChannelId,
 STAR_CHANNEL_DIRECTION_OUT) == 0)

```

```

{
 return;
}

/* Select the path to be added to the front of packets transmitted */
pAddressPath = getTransmitPathAddress();

/* Get packet size */
if (getPacketSize(&byteSize) == 0)
{
 goto Transmit_MultiPacket_Finish;
}

/* Get packet count */
if (getPacketCount(&packetCount) == 0)
{
 goto Transmit_MultiPacket_Finish;
}

/* Get loop count */
if (getLoopCount(&loopCount) == 0)
{
 goto Transmit_MultiPacket_Finish;
}

/* Allocate memory for the transmit buffer */
pTxBuffer = (char *)calloc(1U, byteSize * packetCount);
if (pTxBuffer == NULL)
{
 puts("\nERROR: Unable to allocate memory for transmit buffer");
 goto Transmit_MultiPacket_Finish;
}

/* Fill transmit buffer with random data */
FillBufferRandomChar(pTxBuffer, byteSize * packetCount, DATA_TYPE_RANDOM);

/* Allocate memory for the transmit stream item pointers */
ppTxStreamItems = (STAR_STREAM_ITEM **)calloc(packetCount,
 sizeof(STAR_STREAM_ITEM *));
if (ppTxStreamItems == NULL)
{
 puts("\nERROR: Unable to allocate memory for transmit stream items");
 goto Transmit_MultiPacket_Finish;
}

/* For each packet */
for (i = 0U; i < packetCount; i++)
{
 /* Create the stream item */
 ppTxStreamItems[i] = STAR_createPacket(pAddressPath,
 (U8 *)pTxBuffer + (byteSize * i), byteSize, STAR_EOP_TYPE_EOP);
 if (ppTxStreamItems[i] == NULL)
 {
 printf("\nERROR: Unable to create packet %lu to be transmitted\n",
 i);
 goto Transmit_MultiPacket_Finish;
 }
}

/* Create the transmit transfer operation for the packets */
pTxTransferOp = STAR_createTxOperation(ppTxStreamItems, packetCount);
if (pTxTransferOp == NULL)
{
 puts("\nERROR: Unable to create the transfer operation to be transmitted");
 goto Transmit_MultiPacket_Finish;
}

puts("Starting Multi-Packet Transmit Test...");

/* Start time */
start = GET_TIME();

for (testNum = 0U; testNum < loopCount; testNum++)
{
 /* Submit the transmit operation */
 if (STAR_submitTransferOperation(txChannelId, pTxTransferOp) == 0)
 {
 printf("\nERROR occurred during transmit. Test %lu failed.\n",
 testNum + 1U);
 goto Transmit_MultiPacket_Finish;
 }

 /* Wait on the transmit operation completing */
 txStatus = STAR_waitOnTransferOperationCompletion(
 pTxTransferOp,
 STAR_INFINITE);
 if (txStatus != STAR_TRANSFER_STATUS_COMPLETE)

```

```

 {
 printf("\nERROR occurred during transmit. Test %lu failed.\n",
 testNum + 1U);
 goto Transmit_MultiPacket_Finish;
 }
 printf("\rTest - %lu", testNum + 1);
 }
 printf("\n");

 /* End time */
 finish = GET_TIME();

 /* Display the results */
 DisplayResults(start, finish, 0, byteSize, packetCount, loopCount, 0U,
 "Multiple-Packet Transmit Test");

Transmit_MultiPacket_Finish:

 /* Dispose of the transfer operation */
 if (pTxTransferOp != NULL)
 {
 STAR_disposeTransferOperation(pTxTransferOp);
 }

 /* Destroy the packets transmitted */
 if (ppTxStreamItems != NULL)
 {
 for (i = 0U; i < packetCount; i++)
 {
 if (ppTxStreamItems[i] != NULL)
 {
 STAR_destroyStreamItem(ppTxStreamItems[i]);
 }
 }
 free(ppTxStreamItems);
 }

 /* Free the transmit buffer */
 if (pTxBuffer != NULL)
 {
 free(pTxBuffer);
 }

 /* Free the address */
 if (pAddressPath != NULL)
 {
 STAR_destroyAddress(pAddressPath);
 }

 /* Close the channel */
 if (txChannelId != 0U)
 {
 STAR_closeChannel(txChannelId);
 }
}

void Receive_SinglePacket(void)
{
 STAR_CHANNEL_ID rxChannelId = 0U;
 STAR_TRANSFER_OPERATION *pRxTransferOp = NULL;
 unsigned long loopCount, rxByteSize = 0U, testNum;
 clock_t start, finish;
 STAR_TRANSFER_STATUS rxStatus;

 /* Select the receive device and channel to be used */
 if (chooseDeviceAndChannel("receive", &rxChannelId,
 STAR_CHANNEL_DIRECTION_IN) == 0)
 {
 goto Receive_SinglePacket_Finish;
 }

 /* Get loop count */
 if (getLoopCount(&loopCount) == 0)
 {
 goto Receive_SinglePacket_Finish;
 }

 /* Create receive operation to receive 1 packet */
 pRxTransferOp = STAR_createRxOperation(1,
 STAR_RECEIVE_PACKETS);
 if (pRxTransferOp == NULL)
 {
 puts("\nERROR: Unable to create receive operation");
 goto Receive_SinglePacket_Finish;
 }

 puts("Running Single-Packet Receive Test...");
}

```

```

/* Start time (this should be updated when the first packet is received) */
start = GET_TIME();

for (testNum = 0U; testNum < loopCount; testNum++)
{
 /* Submit the receive operation */
 if (STAR_submitTransferOperation(rxChannelId, pRxTransferOp) == 0)
 {
 printf("\nERROR occurred during receive. Test %lu failed.\n",
 testNum + 1U);
 goto Receive_SinglePacket_Finish;
 }

 /* Wait on the receive operation completing */
 rxStatus = STAR_waitOnTransferOperationCompletion(
 pRxTransferOp,
 STAR_INFINITE);
 if (rxStatus != STAR_TRANSFER_STATUS_COMPLETE)
 {
 printf("\nERROR occurred during receive. Test %lu failed.\n",
 testNum + 1U);
 goto Receive_SinglePacket_Finish;
 }

 /* Start time after first packet received */
 if (testNum == 0U)
 {
 start = GET_TIME();
 }
 else
 {
 rxByteSize += getPacketLengths(pRxTransferOp, 1U);
 }
 printf("\rTest - %lu", testNum + 1);
}
printf("\n");

/* End time */
finish = GET_TIME();

/* Display the results */
DisplayResults(start, finish, 0, rxByteSize, 1U, loopCount, 0U,
 "Single-Packet Receive Test");

Receive_SinglePacket_Finish:

/* Dispose of the transfer operation */
if (pRxTransferOp != NULL)
{
 STAR_disposeTransferOperation(pRxTransferOp);
}

/* Close the channel */
if (rxChannelId != 0U)
{
 STAR_closeChannel(rxChannelId);
}
}

void Receive_MultiPacket(void)
{
 STAR_CHANNEL_ID rxChannelId = 0U;
 unsigned long loopCount, rxByteSize = 0U, testNum, packetCount;
 STAR_TRANSFER_OPERATION *pRxTransferOp = NULL;
 STAR_TRANSFER_STATUS rxStatus;
 clock_t start, finish;

 /* Select the receive device and channel to be used */
 if (chooseDeviceAndChannel("receive", &rxChannelId,
 STAR_CHANNEL_DIRECTION_IN) == 0)
 {
 goto Receive_MultiPacket_Finish;
 }

 /* Get packet count */
 if (getPacketCount(&packetCount) == 0)
 {
 goto Receive_MultiPacket_Finish;
 }

 /* Get loop count */
 if (getLoopCount(&loopCount) == 0)
 {
 goto Receive_MultiPacket_Finish;
 }
}

```

```

/* Create receive operation to receive the packets */
pRxTransferOp = STAR_createRxOperation(packetCount,
 STAR_RECEIVE_PACKETS);
if (pRxTransferOp == NULL)
{
 puts("\nERROR: Unable to create receive operation");
 goto Receive_MultiPacket_Finish;
}

puts("Running Multiple-Packet Receive Test...");

/* Start time */
/* (this should be updated when the first packets are received) */
start = GET_TIME();

for (testNum = 0U; testNum < loopCount; testNum++)
{
 /* Submit the receive operation */
 if (STAR_submitTransferOperation(rxChannelId, pRxTransferOp) == 0)
 {
 printf("\nERROR occurred during receive. Test %lu failed.\n",
 testNum + 1U);
 goto Receive_MultiPacket_Finish;
 }

 /* Wait on the receive operation completing */
 rxStatus = STAR_waitOnTransferOperationCompletion(
 pRxTransferOp,
 STAR_INFINITE);
 if (rxStatus != STAR_TRANSFER_STATUS_COMPLETE)
 {
 printf("\nERROR occurred during receive. Test %lu failed.\n",
 testNum + 1U);
 goto Receive_MultiPacket_Finish;
 }

 /* Start time after first packet received */
 if (testNum == 0U)
 {
 start = GET_TIME();
 }
 else
 {
 rxByteSize += getPacketLengths(pRxTransferOp, packetCount);
 }
 printf("\rTest - %lu", testNum + 1);
}
printf("\n");

/* End time */
finish = GET_TIME();

/* Display the results */
DisplayResults(start, finish, 0, rxByteSize, packetCount, loopCount, 0U,
 "Multi-Packet Receive Test");

Receive_MultiPacket_Finish:

/* Dispose of the transfer operation */
if (pRxTransferOp != NULL)
{
 STAR_disposeTransferOperation(pRxTransferOp);
}

/* Close the channel */
if (rxChannelId != 0U)
{
 STAR_closeChannel(rxChannelId);
}
}

int ConfirmYes(void)
{
 char response[256] = { '\0' };
 size_t responseLen = 0U;

 for (;;)
 {
 if (fgets(response, 256, stdin) == NULL)
 {
 puts("ERROR: Invalid Input");
 return 0;
 }

 /* Strip newline */
 responseLen = strlen(response);
 if (response[responseLen - 1U] == '\n')

```

```

 {
 response[responseLen - 1U] = '\\0';
 }

 if ((strcmp(response, "y") == 0) || (strcmp(response, "Y") == 0))
 {
 return 1;
 }
 else if ((strcmp(response, "n") == 0) || (strcmp(response, "N") == 0))
 {
 return 0;
 }
 else
 {
 puts("Please enter Y/N");
 }
 }
}

void TransmitFile_Whole(void)
{
 STAR_CHANNEL_ID txChannelId = 0U;
 char *pTxBuffer = NULL;
 STAR_TRANSFER_OPERATION *pTxTransferOp = NULL;
 STAR_STREAM_ITEM *pTxStreamItem = NULL;
 STAR_TRANSFER_STATUS txStatus;
 STAR_SPACEWIRE_ADDRESS *pAddressPath;
 char sFile[256];
 int fileSize = 0;

 size_t sFileLen;

 /* Select the transmit device and channel to be used */
 if (chooseDeviceAndChannel("transmit", &txChannelId,
 STAR_CHANNEL_DIRECTION_OUT) == 0)
 {
 return;
 }

 /* Select the path to be added to the front of packets transmitted */
 pAddressPath = getTransmitPathAddress();

 /* Get the File name */
 printf("Enter name of file to transmit: ");
 fflush(stdout);
 if (fgets(sFile, 256, stdin) == NULL)
 {
 puts("ERROR: Invalid Input");
 return;
 }

 /* Strip newline */
 sFileLen = strlen(sFile);
 if (sFile[sFileLen - 1U] == '\\n')
 {
 sFile[sFileLen - 1U] = '\\0';
 }

 printf("Filename = \"%s\\n\", sFile);
 printf("Continue with these values ? (Y/N): ");
 fflush(stdout);

 if (ConfirmYes() == 0)
 {
 puts("File Transfer aborted");
 goto Transmit_File_Whole_Finish;
 }

 fileSize = SizeOfFile(sFile);
 if (fileSize <= 0)
 {
 puts("File size invalid, aborting transfer.");
 goto Transmit_File_Whole_Finish;
 }

 /* Allocate memory to read file into */
 pTxBuffer = (char *)malloc(fileSize);
 if (pTxBuffer == NULL)
 {
 puts("ERROR: Could not allocate buffer for file");
 goto Transmit_File_Whole_Finish;
 }

 /* Read the file */
 if (file_read_chunk(sFile, pTxBuffer, 0U, fileSize) == 0)
 {
 puts("ERROR: Could not read file");
 }
}

```

```

 goto Transmit_File_Whole_Finish;
 }

 pTxStreamItem = STAR_createPacket(pAddressPath, (U8 *)pTxBuffer, fileSize,
 STAR_EOP_TYPE_EOP);
 if (pTxStreamItem == NULL)
 {
 puts("ERROR: Unable to create the packet to be transmitted");
 goto Transmit_File_Whole_Finish;
 }

 /* Create the transmit transfer operation for the packet */
 pTxTransferOp = STAR_createTxOperation(&pTxStreamItem, 1U);
 if (pTxTransferOp == NULL)
 {
 puts("ERROR: Unable to create the transfer operation to be transmitted");
 goto Transmit_File_Whole_Finish;
 }

 puts("Starting File Transmit...");

 if (STAR_submitTransferOperation(txChannelId, pTxTransferOp) == 0)
 {
 puts("ERROR occurred during transmit");
 goto Transmit_File_Whole_Finish;
 }

 /* Wait on the transmit operation completing */
 txStatus = STAR_waitOnTransferOperationCompletion(pTxTransferOp,
 STAR_INFINITE);
 if (txStatus != STAR_TRANSFER_STATUS_COMPLETE)
 {
 puts("ERROR occurred during transmit");
 goto Transmit_File_Whole_Finish;
 }

 puts("Complete.");

Transmit_File_Whole_Finish:
 /* Dispose of the transfer operation */
 if (pTxTransferOp != NULL)
 {
 STAR_disposeTransferOperation(pTxTransferOp);
 }

 /* Destroy the packet transmitted */
 if (pTxStreamItem != NULL)
 {
 STAR_destroyStreamItem(pTxStreamItem);
 }

 /* Free the transmit buffer */
 if (pTxBuffer != NULL)
 {
 free(pTxBuffer);
 }

 /* Free the address */
 if (pAddressPath != NULL)
 {
 STAR_destroyAddress(pAddressPath);
 }

 /* Close the channel */
 if (txChannelId != 0U)
 {
 STAR_closeChannel(txChannelId);
 }
}

void ReceiveFile_Whole(void)
{
 STAR_CHANNEL_ID rxChannelId = 0U;
 STAR_TRANSFER_OPERATION *pRxTransferOp = NULL;
 STAR_TRANSFER_STATUS rxStatus;
 char sFile[256];
 int writeStatus;
 STAR_SPACEWIRE_PACKET *pReceivedPacket = NULL;
 size_t sFileLen;
 U8 *receivedData = NULL;
 unsigned int receivedDataLen = 0U;

 /* Select the receive device and channel to be used */
 if (chooseDeviceAndChannel("receive", &rxChannelId,
 STAR_CHANNEL_DIRECTION_IN) == 0)
 {
 goto Receive_File_Whole_Finish;
 }

```



```

}

/* Get the File name */
printf("Enter name of file to write data to: ");
fflush(stdout);
if (fgets(sFile, 256, stdin) == NULL)
{
 puts("ERROR: Invalid Input");
 return;
}

/* Strip newline */
sFileLen = strlen(sFile);
if (sFile[sFileLen - 1U] == '\n')
{
 sFile[sFileLen - 1U] = '\0';
}

printf("Filename = \"%s\"\n", sFile);
printf("Continue with these values ? (Y/N): ");
fflush(stdout);

if (ConfirmYes() == 0)
{
 printf("File Transfer aborted\n");
 goto Receive_File_Whole_Finish;
}

puts("Running File Receive...");

/* Create receive operation to receive 1 packet */
pRxTransferOp = STAR_createRxOperation(1,
 STAR_RECEIVE_PACKETS);
if (pRxTransferOp == NULL)
{
 puts("ERROR: Unable to create receive operation");
 goto Receive_File_Whole_Finish;
}

/* Submit the receive operation */
if (STAR_submitTransferOperation(rxChannelId, pRxTransferOp) == 0)
{
 puts("ERROR: Could not submit receive operation");
 goto Receive_File_Whole_Finish;
}

/* Wait on the receive operation completing */
rxStatus = STAR_waitOnTransferOperationCompletion(pRxTransferOp,
 STAR_INFINITE);
if (rxStatus != STAR_TRANSFER_STATUS_COMPLETE)
{
 printf("ERROR: Error of %d occurred during receive: \n", rxStatus);
 goto Receive_File_Whole_Finish;
}

/* Get received data*/
pReceivedPacket = (STAR_SPACEWIRE_PACKET*)
 STAR_getTransferItem(
 pRxTransferOp, 0U)->item;

receivedData = STAR_getPacketData(pReceivedPacket, &receivedDataLen);
if (receivedData == NULL)
{
 puts("ERROR: No data received");
 goto Receive_File_Whole_Finish;
}

writeStatus = file_append_chunk(receivedData, receivedDataLen, sFile);
STAR_destroyPacketData(receivedData);

if (writeStatus == 0)
{
 puts("ERROR: Could not write file");
 goto Receive_File_Whole_Finish;
}

puts("Complete.");

Receive_File_Whole_Finish:

/* Dispose of the transfer operation */
if (pRxTransferOp != NULL)
{
 STAR_disposeTransferOperation(pRxTransferOp);
}

```

```

 /* Close the channel */
 if (rxChannelId != 0U)
 {
 STAR_closeChannel(rxChannelId);
 }
}

void TransmitFile_Split(void)
{
 STAR_CHANNEL_ID txChannelId = 0U;
 unsigned long chunkSize, dataPacketCount = 0U, i;
 U8 *pTxBuffer = NULL;
 STAR_TRANSFER_OPERATION *pTxTransferOp = NULL;
 STAR_STREAM_ITEM** packets = NULL;
 STAR_TRANSFER_STATUS txStatus;
 STAR_SPACEWIRE_ADDRESS *pAddressPath;
 char sFile[256], sChunkSize[256];
 int status;
 int fileSize = 0;
 U32 fileSizeToSend = 0U;
 unsigned int offset = 0U;
 size_t sFileLen;

 /* Select the transmit device and channel to be used */
 if (chooseDeviceAndChannel("transmit", &txChannelId,
 STAR_CHANNEL_DIRECTION_OUT) == 0)
 {
 return;
 }

 /* Select the path to be added to the front of packets transmitted */
 pAddressPath = getTransmitPathAddress();

 /* Get the File name */
 printf("Enter name of file to transmit: ");
 fflush(stdout);
 if (fgets(sFile, 256, stdin) == NULL)
 {
 puts("ERROR: Invalid Input");
 return;
 }

 /* Strip newline */
 sFileLen = strlen(sFile);
 if (sFile[sFileLen - 1U] == '\n')
 {
 sFile[sFileLen - 1U] = '\0';
 }

 printf("Filename = \"%s\"\n", sFile);
 printf("Continue with these values ? (Y/N): ");
 fflush(stdout);

 if (ConfirmYes() == 0)
 {
 printf("File Transfer aborted\n");
 goto Transmit_File_Packets_Finish;
 }

 /* Allocate memory to read file into */
 fileSize = SizeOfFile(sFile);
 if (fileSize <= 0)
 {
 printf("File size invalid, aborting transfer.\n");
 goto Transmit_File_Packets_Finish;
 }

 /* Read the file into a buffer */
 pTxBuffer = (U8 *)malloc(fileSize);
 if (pTxBuffer == NULL)
 {
 puts("ERROR: Could not allocate buffer for file");
 goto Transmit_File_Packets_Finish;
 }

 if (file_read_chunk(sFile, (char*)pTxBuffer, 0U, fileSize) == 0)
 {
 puts("ERROR: Could not read file");
 goto Transmit_File_Packets_Finish;
 }

 /* Get packet size */
 printf("Enter maximum size of packets to transmit, in bytes: ");
 fflush(stdout);
 if (fgets(sChunkSize, 256, stdin) == NULL)
 {
 puts("ERROR: Invalid input");
 }
}

```

```

 goto Transmit_File_Packets_Finish;
 }
 status = sscanf(sChunkSize, "%lu", &chunkSize);
 if (status == 0)
 {
 puts("ERROR: Invalid input");
 goto Transmit_File_Packets_Finish;
 }

 /* Calculate number of packets to send */
 dataPacketCount = ((fileSize / chunkSize) +
 ((fileSize % chunkSize) > 0U) ? 1U : 0U));

 /* Prepare packets for transmission */
 packets = (STAR_STREAM_ITEM **)calloc(dataPacketCount + 1U,
 sizeof(STAR_STREAM_ITEM *));
 if (packets == NULL)
 {
 puts("ERROR: Could not allocate buffer for packets");
 goto Transmit_File_Packets_Finish;
 }

 /* Initial four byte packet size */
 CopyNumberToMemory(&fileSizeToSend, fileSize, sizeof(U32));
 packets[0U] = STAR_createPacket(pAddressPath, (U8*)&fileSizeToSend,
 sizeof(U32), STAR_EOP_TYPE_EOP);

 /* Data packets */
 for (i = 1U; i < dataPacketCount; i++)
 {
 packets[i] = STAR_createPacket(pAddressPath, pTxBuffer + offset,
 chunkSize, STAR_EOP_TYPE_EOP);
 offset += chunkSize;
 }

 /* Final Data packet */
 if ((fileSize % chunkSize) > 0)
 {
 packets[dataPacketCount] = STAR_createPacket(pAddressPath,
 pTxBuffer + offset, fileSize % chunkSize, STAR_EOP_TYPE_EOP);
 }
 else
 {
 packets[dataPacketCount] = STAR_createPacket(pAddressPath,
 pTxBuffer + offset, chunkSize, STAR_EOP_TYPE_EOP);
 }

 pTxTransferOp = STAR_createTxOperation(packets, dataPacketCount + 1U);

 puts("Starting File Transmit...");

 if (STAR_submitTransferOperation(txChannelId, pTxTransferOp) == 0)
 {
 puts("ERROR: Could not submit transmit operation");
 goto Packet_Cleanup;
 }

 /* Wait on the transmit operation completing */
 txStatus = STAR_waitOnTransferOperationCompletion(pTxTransferOp,
 STAR_INFINITE);
 if (txStatus != STAR_TRANSFER_STATUS_COMPLETE)
 {
 printf("ERROR: Error %d occurred during transmit\n", txStatus);
 goto Packet_Cleanup;
 }

 puts("Complete.");

Packet_Cleanup:
 /* Cleanup packets */
 for (i = 0U; i <= dataPacketCount; i++)
 {
 STAR_destroyStreamItem(packets[i]);
 }

Transmit_File_Packets_Finish:

 free(packets);

 /* Dispose of the transfer operation */
 if (pTxTransferOp != NULL)
 {
 STAR_disposeTransferOperation(pTxTransferOp);
 }

 /* Free the transmit buffer */
 if (pTxBuffer != NULL)

```

```

 {
 free(pTxBuffer);
 }

 /* Free the address path */
 if (pAddressPath != NULL)
 {
 STAR_destroyAddress(pAddressPath);
 }

 /* Close the channel */
 if (txChannelId != 0U)
 {
 STAR_closeChannel(txChannelId);
 }
}

void ReceiveFile_Split(void)
{
 STAR_CHANNEL_ID rxChannelId = 0U;
 STAR_TRANSFER_OPERATION *pRxTransferOp = NULL;
 STAR_TRANSFER_STATUS rxStatus;
 char sFile[256];
 int writeStatus;
 STAR_SPACEWIRE_PACKET *pReceivedPacket = NULL;
 size_t sFileLen = 0U;
 clock_t start, finish;
 U8 *receivedData = NULL;
 unsigned int receivedDataLen = 0U, expectedFileSize = 0U;
 unsigned int actualFileSize = 0U;

 /* Select the receive device and channel to be used */
 if (chooseDeviceAndChannel("receive", &rxChannelId,
 STAR_CHANNEL_DIRECTION_IN) == 0)
 {
 goto Receive_File_Split_Finish;
 }

 /* Get the File name */
 printf("Enter name of file to write data to: ");
 fflush(stdout);
 if (fgets(sFile, 256, stdin) == NULL)
 {
 puts("ERROR: Invalid input");
 return;
 }

 /* Strip newline */
 sFileLen = strlen(sFile);
 if (sFile[sFileLen - 1U] == '\n')
 {
 sFile[sFileLen - 1U] = '\0';
 }

 printf("Filename = \"%s\"\n", sFile);
 printf("Continue with these values ? (Y/N): ");
 fflush(stdout);

 if (ConfirmYes() == 0)
 {
 puts("File Transfer aborted");
 goto Receive_File_Split_Finish;
 }

 puts("Running File Receive...");

 /* Create receive operation to receive Header packet */
 pRxTransferOp = STAR_createRxOperation(1,
 STAR_RECEIVE_PACKETS);
 if (pRxTransferOp == NULL)
 {
 puts("ERROR: Unable to create receive operation");
 goto Receive_File_Split_Finish;
 }

 /* Submit the receive operation */
 if (STAR_submitTransferOperation(rxChannelId, pRxTransferOp) == 0)
 {
 puts("ERROR: Could not submit receive operation");
 goto Receive_File_Split_Finish;
 }

 /* Wait on the receive operation completing */
 rxStatus = STAR_waitOnTransferOperationCompletion(pRxTransferOp,
 STAR_INFINITE);
 if (rxStatus != STAR_TRANSFER_STATUS_COMPLETE)
 {

```

```

 printf("ERROR: Error %d occurred during receive\n", rxStatus);
 goto Receive_File_Split_Finish;
}

pReceivedPacket = (STAR_SPACEWIRE_PACKET *)
 STAR_getTransferItem(
 pRxTransferOp, 0U)->item;
receivedData = STAR_getPacketData(pReceivedPacket, &receivedDataLen);
if (receivedData == NULL)
{
 puts("ERROR: No data received");
 goto Receive_File_Split_Finish;
}

CopyNumberFromMemory(&expectedFileSize, receivedData,
 sizeof(expectedFileSize));

STAR_destroyPacketData(receivedData);

/* Start the timer */
start = GET_TIME();

/* Read a packet at a time until we get all expected data */
while(expectedFileSize > actualFileSize)
{
 if (STAR_submitTransferOperation(rxChannelId, pRxTransferOp) == 0)
 {
 puts("ERROR: Could not submit receive operation");
 goto Receive_File_Split_Finish;
 }

 rxStatus = STAR_waitOnTransferOperationCompletion(
 pRxTransferOp,
 STAR_INFINITE);
 if (rxStatus != STAR_TRANSFER_STATUS_COMPLETE)
 {
 printf("ERROR: Error %d occurred during receive\n", rxStatus);
 goto Receive_File_Split_Finish;
 }

 pReceivedPacket = (STAR_SPACEWIRE_PACKET *)
 STAR_getTransferItem(
 pRxTransferOp, 0U)->item;

 receivedData = STAR_getPacketData(pReceivedPacket, &receivedDataLen);
 if (receivedData == NULL)
 {
 puts("ERROR: No data received");
 goto Receive_File_Split_Finish;
 }

 writeStatus = file_append_chunk(receivedData, receivedDataLen, sFile);

 STAR_destroyPacketData(receivedData);

 if (writeStatus == 0)
 {
 puts("ERROR: Could not write to file");
 goto Receive_File_Split_Finish;
 }

 actualFileSize+=receivedDataLen;
}

if (actualFileSize != expectedFileSize)
{
 puts("ERROR: Expected and actual file size do not match");
}

/* End time */
finish = GET_TIME();

/* Display the results */
DisplayResults(start, finish, 0, actualFileSize, 1U, 1U, 0U,
 "Receive File from multiple packets");

Receive_File_Split_Finish:

/* Dispose of the transfer operation */
if (pRxTransferOp != NULL)
{
 STAR_disposeTransferOperation(pRxTransferOp);
}

/* Close the channel */
if (rxChannelId != 0U)
{

```

```

 STAR_closeChannel(rxChannelId);
 }
}

void ResetDevice(void)
{
 STAR_DEVICE_ID deviceId;

 deviceId = chooseDevice("", STAR_DEVICE_ALL);
 if (deviceId == STAR_DEVICE_UNKNOWN)
 {
 return;
 }

 if (STAR_resetDevice(deviceId) == 0)
 {
 puts("ERROR: Unable to reset device");
 }
}

void IdentifyDevice(void)
{
 STAR_DEVICE_ID deviceId;

 deviceId = chooseDevice("", STAR_DEVICE_CONFIG_SUPPORTED);
 if (deviceId == STAR_DEVICE_UNKNOWN)
 {
 return;
 }

 if (!CFG_SUCCESS(CFG_identify(deviceId)))
 {
 puts("ERROR: Unable to identify device");
 }
}

void runInteractive(void)
{
 char s[256];
 int menuSelect;
 int bExit = 0;

 puts("STAR-System Test Program");
 puts("Copyright STAR-Dundee Ltd. (c) 2011-2018");
 puts("www.star-dundee.com");

 /* Loop until exit option is chosen */
 while (bExit == 0)
 {
 DisplayMenu();

 if (fgets(s, 256, stdin) == NULL)
 {
 puts("\nERROR: No value specified");
 }
 else if (sscanf(s, "%d", &menuSelect) == 0)
 {
 puts("\nERROR: Invalid value specified");
 }
 else
 {
 switch(menuSelect)
 {
 case MENU_DISPLAY_INFORMATION:
 DisplayInformation();
 break;

 case MENU_LOOPBACK_SINGLE:
 LoopBack_SinglePacket();
 break;

 case MENU_LOOPBACK_MULTI:
 LoopBack_MultiPacket();
 break;

 case MENU_LOOPBACK_DOUBLE_SINGLE:
 LoopBackDouble_SinglePacket();
 break;

 case MENU_LOOPBACK_DOUBLE_MULTI:
 LoopBackDouble_MultiPacket();
 break;

 case MENU_TRANSMIT_SINGLE:
 Transmit_SinglePacket();
 break;
 }
 }
 }
}

```

```

 case MENU_TRANSMIT_MULTI:
 Transmit_MultiPacket();
 break;

 case MENU_RECEIVE_SINGLE:
 Receive_SinglePacket();
 break;

 case MENU_RECEIVE_MULTI:
 Receive_MultiPacket();
 break;

 case MENU_TRANSMIT_FILE_WHOLE:
 TransmitFile_Whole();
 break;

 case MENU_RECEIVE_FILE_WHOLE:
 ReceiveFile_Whole();
 break;

 case MENU_TRANSMIT_FILE_SPLIT:
 TransmitFile_Split();
 break;

 case MENU_RECEIVE_FILE_SPLIT:
 ReceiveFile_Split();
 break;

 case MENU_RESET_DEVICE:
 ResetDevice();
 break;

 case MENU_IDENTIFY_DEVICE:
 IdentifyDevice();
 break;

 case MENU_EXIT:
 puts("\nExiting SpaceWire test program");
 bExit = 1;
 break;

 default:
 puts("\nERROR: Incorrect menu option");
 break;
 }
}

puts("Exiting...\n");
}

void usage(char *argv[], const int error)
{
 puts("");

 if (error != 0)
 {
 fprintf(stderr, "usage: %s [-v] [-help]\n", argv[0]);
 }
 else
 {
 fprintf(stdout, "usage: %s [-v] [-help]\n", argv[0]);
 }
}

void showHelp(void)
{
 puts("");
 puts("Description:");
 puts(" A program to display information about STAR-System,");
 puts(" the STAR-Dundee API stack, and perform basic transmit");
 puts(" and receive tests using attached hardware.");
 puts("");
 puts("Optional Arguments:");
 puts("");
 puts(" -v Displays version information and exits.");
 puts("");
 puts(" -help Displays this help message and exits.");
 puts("");
 puts(" When called without arguments, the program is run in");
 puts(" interactive mode.");
}

int __cdecl main(int argc, char *argv[])
{
 int i;
 int version = 0; /* Value for the "-v" optional argument. */

```

```

int help = 0; /* Value for the "-help" optional argument. */

STAR_setApplicationName("STAR-System Test Application");

/* If no args provided, interactive mode */
if (argc <= 1)
{
 runInteractive();
 return 0;
}

/* Currently we only expect 1 optional argument */
if (argc > 2)
{
 usage(argv, 1);
 return 1;
}

/* Process command line args*/
for (i = 1; i < argc; i++)
{
 if (strcmp(argv[i], "-v") == 0)
 {
 version = 1;
 }
 else if (strcmp(argv[i], "-help") == 0)
 {
 help = 1;
 }
}

if (help != 0)
{
 usage(argv, 0);
 showHelp();
}
else if (version != 0)
{
 puts(VERSION_INFO);
 puts("Copyright STAR-Dundee Ltd. (c) 2011-2018");
 puts("www.star-dundee.com");

 DisplayInformation();
}
else
{
 usage(argv, 1);
 return 1;
}

return 0;
}

```

### 9.37 star\_performance\_tester.c

This file contains the functions for the SpaceWire Performance Tester, a program to test the performance of SpaceWire devices using the STAR-System software stack.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <errno.h>

#include "star-api.h"

#if !defined(__vxworks)
#define TRUE 1
#define FALSE 0
#else
/* Required for strcasecmp() */
#include <strings.h>
#endif

#define STAR_INFINITE -1

#ifdef _WIN32

#include <windows.h>

```



```

#define strcasecmp(s1, s2) _stricmp((s1), (s2))

#define GET_TIME() clock()

#define TIME_DIVIDER CLOCKS_PER_SEC

#define SLEEP(time) Sleep(time)

#define DISABLE_SYSTEM_SLEEP() \
 SetThreadExecutionState(ES_CONTINUOUS | ES_SYSTEM_REQUIRED)

#define ENABLE_SYSTEM_SLEEP() \
 SetThreadExecutionState(ES_CONTINUOUS)

typedef struct
{
 FILETIME idleTime;

 FILETIME kernelTime;

 FILETIME userTime;

 FILETIME processKernelTime;

 FILETIME processUserTime;
} CPU_USAGE_PROPERTIES;

static void STORE_CPU_USAGE(CPU_USAGE_PROPERTIES * const pCpuProperties)
{
 FILETIME creationTime, exitTime;

 GetSystemTimes(&pCpuProperties->idleTime, &pCpuProperties->kernelTime,
 &pCpuProperties->userTime);
 GetProcessTimes(GetCurrentProcess(), &creationTime, &exitTime,
 &pCpuProperties->processKernelTime,
 &pCpuProperties->processUserTime);
}

static void PRINT_CPU_USAGE_HEADERS(FILE * const stream)
{
 fprintf(stream, "\tProcess CPU usage (%) \tTotal CPU usage (%) \n");
}

static void PRINT_CPU_USAGE(FILE * const stream,
 const CPU_USAGE_PROPERTIES * const pStartCpuProperties,
 const CPU_USAGE_PROPERTIES * const pEndCpuProperties)
{
 ULARGE_INTEGER originalIdleTime, originalKernelTime, originalUserTime;
 ULARGE_INTEGER idleTime, kernelTime, userTime, totalTime;
 double totalUsage, processUsage;

 /* Subtract the original total CPU figures from the final figures */
 originalIdleTime.HighPart =
 pStartCpuProperties->idleTime.dwHighDateTime;
 originalIdleTime.LowPart =
 pStartCpuProperties->idleTime.dwLowDateTime;
 idleTime.HighPart = pEndCpuProperties->idleTime.dwHighDateTime;
 idleTime.LowPart = pEndCpuProperties->idleTime.dwLowDateTime;
 idleTime.QuadPart -= originalIdleTime.QuadPart;
 originalKernelTime.HighPart =
 pStartCpuProperties->kernelTime.dwHighDateTime;
 originalKernelTime.LowPart =
 pStartCpuProperties->kernelTime.dwLowDateTime;
 kernelTime.HighPart = pEndCpuProperties->kernelTime.dwHighDateTime;
 kernelTime.LowPart = pEndCpuProperties->kernelTime.dwLowDateTime;
 kernelTime.QuadPart -= originalKernelTime.QuadPart;
 originalUserTime.HighPart =
 pStartCpuProperties->userTime.dwHighDateTime;
 originalUserTime.LowPart =
 pStartCpuProperties->userTime.dwLowDateTime;
 userTime.HighPart = pEndCpuProperties->userTime.dwHighDateTime;
 userTime.LowPart = pEndCpuProperties->userTime.dwLowDateTime;
 userTime.QuadPart -= originalUserTime.QuadPart;

 /* Calculate the total time */
 totalTime.QuadPart = userTime.QuadPart + kernelTime.QuadPart;

 /* Calculate the total usage */
 if (totalTime.QuadPart != 0U)
 {
 totalUsage =
 (double)((totalTime.QuadPart - idleTime.QuadPart) * 100U) /
 (double)totalTime.QuadPart;
 }
}

```

```

 }
 else
 {
 totalUsage = 0.0;
 }

 /* Subtract the original process CPU figures from the final figures */
 originalKernelTime.HighPart =
 pStartCpuProperties->processKernelTime.dwHighDateTime;
 originalKernelTime.LowPart =
 pStartCpuProperties->processKernelTime.dwLowDateTime;
 kernelTime.HighPart =
 pEndCpuProperties->processKernelTime.dwHighDateTime;
 kernelTime.LowPart = pEndCpuProperties->processKernelTime.dwLowDateTime;
 kernelTime.QuadPart -= originalKernelTime.QuadPart;
 originalUserTime.HighPart =
 pStartCpuProperties->processUserTime.dwHighDateTime;
 originalUserTime.LowPart =
 pStartCpuProperties->processUserTime.dwLowDateTime;
 userTime.HighPart =
 pEndCpuProperties->processUserTime.dwHighDateTime;
 userTime.LowPart =
 pEndCpuProperties->processUserTime.dwLowDateTime;
 userTime.QuadPart -= originalUserTime.QuadPart;

 /* Calculate the process's usage */
 if (totalTime.QuadPart != 0U)
 {
 processUsage =
 (double)((userTime.QuadPart + kernelTime.QuadPart) * 100U) /
 (double)totalTime.QuadPart;
 }
 else
 {
 processUsage = 0.0;
 }

 if (stream != NULL)
 {
 fprintf(stream, "\t%-3.2f\t%-3.2f\n", processUsage, totalUsage);
 }
 else
 {
 printf("\t\t%-3.2f%% Process CPU usage\t%-3.2f%% Total CPU usage\n",
 processUsage, totalUsage);
 }
}

#else

#if defined(__vxworks)
#include <time.h>
#include <utime.h>
#include <sys/times.h>
#include <taskLib.h>
#include <sysLib.h>
#else
#include <sys/time.h>
#include <sys/times.h>
#endif

#include <unistd.h>

clock_t GET_TIME()
{
 struct timeval tv;
 struct timezone tz;

 gettimeofday(&tv, &tz);
 return tv.tv_sec * 1000000 + tv.tv_usec;
}

#define TIME_DIVIDER 1000000

#if defined(__vxworks)
void SLEEP(unsigned int delay)
{
 double rate = (double)sysClkRateGet();
 unsigned int ticks = (int)((rate / 1000.0) * delay + 0.5);
 taskDelay(ticks);
}
#else
#define SLEEP(time) usleep(time * 1000)
#endif

#define MAX_PATH 1024

```

```

#define DISABLE_SYSTEM_SLEEP()

#define ENABLE_SYSTEM_SLEEP()

typedef struct
{
#if !defined(__vxworks)

 struct tms processTimes;

#endif

 clock_t currentTime;

 unsigned long userTime;

 unsigned long systemTime;

 unsigned long niceTime;

 unsigned long idleTime;

 unsigned long iowaitTime;

} CPU_USAGE_PROPERTIES;

void STORE_CPU_USAGE(CPU_USAGE_PROPERTIES *pCpuProperties)
{
 FILE *procFile;
#if !defined(__vxworks)
 pCpuProperties->currentTime = times(&pCpuProperties->processTimes);
#endif
 pCpuProperties->userTime = 0;
 pCpuProperties->systemTime = 0;
 pCpuProperties->niceTime = 0;
 pCpuProperties->idleTime = 0;
 pCpuProperties->iowaitTime = 0;
 procFile = fopen("/proc/stat", "r");
 if (procFile)
 {
 char line[256];
 if (fgets(line, 256, procFile))
 {
 sscanf(line, "%*s %lu %lu %lu %lu %lu",
 &pCpuProperties->userTime, &pCpuProperties->systemTime,
 &pCpuProperties->niceTime, &pCpuProperties->idleTime,
 &pCpuProperties->iowaitTime);
 }

 fclose(procFile);
 }
}

void PRINT_CPU_USAGE_HEADERS(FILE *stream)
{
 fprintf(stream, "\tProcess CPU usage (%%)\tTotal CPU usage (%%)\n");
}

#if defined(__QNX__)
#include <sys/syspage.h>
int GetCpuCount()
{
 return _syspage_ptr->num_cpu;
}
#elif defined(__vxworks)
/* First version of vxworks driver is for single core targets only */
int GetCpuCount()
{
 return 1;
}
#else
int GetCpuCount()
{
 return (int)sysconf(_SC_NPROCESSORS_ONLN);
}
#endif

void PRINT_CPU_USAGE(FILE *stream,
 CPU_USAGE_PROPERTIES const *pStartCpuProperties,
 CPU_USAGE_PROPERTIES const *pEndCpuProperties)
{
 unsigned long userTime, systemTime, niceTime, idleTime, iowaitTime;
 unsigned long totalTime;

```

```

clock_t processKernelTime, processUserTime, processTotalTime;
double totalUsage, processUsage;
int cpuCount;

/* Determine the number of CPUs */
cpuCount = GetCpuCount();

/* Subtract the original total CPU figures from the final figures */
userTime = pEndCpuProperties->userTime -
 pStartCpuProperties->userTime;
systemTime = pEndCpuProperties->systemTime -
 pStartCpuProperties->systemTime;
niceTime = pEndCpuProperties->niceTime -
 pStartCpuProperties->niceTime;
idleTime = pEndCpuProperties->idleTime -
 pStartCpuProperties->idleTime;
iowaitTime = pEndCpuProperties->iowaitTime -
 pStartCpuProperties->iowaitTime;

/* Calculate the total time */
totalTime = userTime + systemTime + niceTime + idleTime + iowaitTime;

/* Calculate the total usage */
if (totalTime)
{
 totalUsage =
 (double)((totalTime - idleTime) * 100) / (double)totalTime;
}
else
{
 totalUsage = 0;
}

#if !defined(__vxworks)
/* Subtract the original process CPU figures from the final figures */
processKernelTime = pEndCpuProperties->processTimes.tms_stime -
 pStartCpuProperties->processTimes.tms_stime;
processUserTime = pEndCpuProperties->processTimes.tms_utime -
 pStartCpuProperties->processTimes.tms_utime;
#else
processKernelTime = 0;
processUserTime = 0;
#endif

/* Calculate the total process time */
processTotalTime = pEndCpuProperties->currentTime -
 pStartCpuProperties->currentTime;

/* Calculate the process's usage */
if (processTotalTime && cpuCount)
{
 processUsage =
 (double)((processUserTime + processKernelTime) * 100) /
 (double)(processTotalTime * cpuCount);
}
else
{
 processUsage = 0;
}

/* Note: the kernel time may not be needed, if this is included in */
/* the user time (at least according to some manuals!) */
/* processUsage = (double)(processUserTime * 100) / */
/* (double)(processTotalTime * cpuCount); */

if (stream)
{
 fprintf(stream, "\t%-3.2f\t%-3.2f\n", processUsage, totalUsage);
}
else
{
 printf("\t\t%-3.2f%% Process CPU usage\t%-3.2f%% Total CPU usage\n",
 processUsage, totalUsage);
}
}

#endif

#define PERFORM_RECEIVE 1U
#define PERFORM_TRANSMIT 2U

#define PERFORMING_RECEIVES(_mode_) (((_mode_) & PERFORM_RECEIVE) != 0U)
#define PERFORMING_TRANSMITS(_mode_) (((_mode_) & PERFORM_TRANSMIT) != 0U)

```

```

#define MIN(a, b) (((a) > (b)) ? (b) : (a))

#define MAX(a, b) (((a) < (b)) ? (b) : (a))

#define BUFFER_NUM 4

/* The following values ensure that each test runs for at least 10 seconds */
/* (at 200 Mbit/s) */

#define BUFFER_SIZE 200000

#define LOOP_NUM 1000

typedef enum
{
 RATE_TEST,

 RANDOM_TEST,

 LATENCY_TEST
} TEST_TYPE;

typedef struct
{
 U8 mode;

 TEST_TYPE testType;

 int testCount;

 int packetStart;

 int packetEnd;

 int packetStep;

 STAR_CHANNEL_ID *pTxChannelIds;

 STAR_SPACEWIRE_ADDRESS **pAddressPaths;

 STAR_CHANNEL_ID *pRxChannelIds;

 char pFilePath[MAX_PATH + 1];

 char pFileDescription[1000];

 int useChannel0;
} TEST_PROPERTIES;

static STAR_DEVICE_ID chooseDevice(const char * const deviceTypeStr,
 const int argCount, const char * const arguments[],
 int * const pCurrentArgument)
{
 STAR_DEVICE_ID *devices;
 U32 devCount = 0U, i, chosen;
 int status;
 STAR_DEVICE_ID deviceID;
 char *deviceName, s[256];

 /* Get the list of devices which can transmit and receive packets */
 devices = STAR_getDeviceListForType(
 STAR_DEVICE_TXRX_SUPPORTED, &devCount);

 /* If there was an error getting the list of devices */
 if (devices == NULL)
 {
 /* Return an error */
 puts("Error occurred detecting devices!");
 return 0U;
 }

 /* Display the devices detected */
 if (devCount == 1U)
 {
 puts("One SpaceWire device detected:");
 }
 else
 {
 printf("%u SpaceWire devices detected:\n", devCount);
 }

 /* For each device */
 for (i = 0U; i < devCount; i++)

```

```

{
 if (devices[i] != 0U)
 {
 deviceName = STAR_getDeviceName(devices[i]);
 if (deviceName != NULL)
 {
 printf("\t%u - %s\n", i, deviceName);
 STAR_destroyString(deviceName);
 }
 else
 {
 printf("\t%u - Unknown SpaceWire Device\n", i);
 }
 }
 else
 {
 printf("\t%u - Unable to access device\n", i);
 }
}

/* If there's only 1 device on the list */
if (devCount == 1U)
{
 /* Return that device */
 deviceID = devices[0U];
 STAR_destroyDeviceList(devices);
 return deviceID;
}

/* Ask the user which device to use */
printf("\nPlease select which %s device to use: ", deviceTypeStr);
fflush(stdout);
if ((*pCurrentArgument) < argCount)
{
 puts(arguments[*pCurrentArgument]);
 strncpy(s, arguments[*pCurrentArgument], 255U);
 s[255U] = '\0';
 (*pCurrentArgument)++;
}
else
{
 if (fgets(s, 256, stdin) == NULL)
 {
 puts("No device number selected.");
 STAR_destroyDeviceList(devices);
 return 0U;
 }
}
status = sscanf(s, "%u", &chosen);
if ((status == 0) || (chosen >= devCount))
{
 puts("Incorrect device number selected.");
 STAR_destroyDeviceList(devices);
 return 0U;
}

/* Return the chosen device */
deviceID = devices[chosen];
STAR_destroyDeviceList(devices);
return deviceID;
}

static STAR_CHANNEL_ID openChannel(const char * const channelTypeStr,
 const int testNumber, const STAR_CHANNEL_DIRECTION direction,
 const int argCount, const char * const arguments[], int * const pCurrentArgument,
 const int lowestChannel)
{
 STAR_CHANNEL_MASK channelMask;
 int maximumChannelNumber = 0, i, channelNumber;
 STAR_DEVICE_ID deviceId;
 STAR_CHANNEL_ID channelId;

 /* Choose the device to be used */
 deviceId = chooseDevice(channelTypeStr, argCount, arguments,
 pCurrentArgument);

 /* Check for an invalid device id */
 if (deviceId == 0U)
 {
 printf("\nERROR: Failed to open %s device for test %d.\n",
 channelTypeStr, testNumber);
 return 0U;
 }

 /* Check for a device with zero channels or an invalid list of channels */

```

```

channelMask = STAR_getDeviceChannels(deviceId);
if (channelMask == 0U)
{
 printf("\nERROR: The chosen %s device doesn't appear to have any valid channels.\n",
 channelTypeStr);
 return 0U;
}

/* For each possible channel */
for (i = 31; i >= lowestChannel; i--)
{
 /* If the channel exists */
 if (((channelMask >> (U32)i) & 1U) != 0U)
 {
 /* This is the maximum channel number */
 maximumChannelNumber = i;
 break;
 }
}

if (maximumChannelNumber < lowestChannel)
{
 printf("\nERROR: The chosen %s device doesn't appear to have any valid channels.\n",
 channelTypeStr);
 return 0U;
}
else if (maximumChannelNumber == lowestChannel)
{
 printf("\nUsing %s channel %d for test %d\n", channelTypeStr,
 lowestChannel, testNumber);
 channelNumber = lowestChannel;
}
else
{
 char s[256];
 int status;

 /* Get channel and check for validity */
 printf("\nEnter %s channel (%d..%d) for test %d: ", channelTypeStr,
 lowestChannel, maximumChannelNumber, testNumber);
 fflush(stdout);
 if ((*pCurrentArgument) < argCount)
 {
 puts(arguments[*pCurrentArgument]);
 strncpy(s, arguments[*pCurrentArgument], 255U);
 s[255U] = '\0';
 (*pCurrentArgument)++;
 }
 else
 {
 if (fgets(s, 256, stdin) == NULL)
 {
 puts("\nERROR: No channel number specified");
 return 0U;
 }
 }
 status = sscanf(s, "%d", &channelNumber);
 if ((status == 0) || (channelNumber < lowestChannel) ||
 (channelNumber > maximumChannelNumber))
 {
 puts("\nERROR: Incorrect channel number specified");
 return 0U;
 }
 if ((1U << (U32)channelNumber) & channelMask) == 0U)
 {
 puts("\nERROR: The channel specified is not present");
 return 0U;
 }
}

/* Open the channel */
channelId = STAR_openChannelToLocalDevice(deviceId, direction,
 (unsigned char)channelNumber, TRUE);
if (channelId == 0U)
{
 printf("\nFailed to open %s channel %d for test %d\n", channelTypeStr,
 channelNumber, testNumber);
 return 0U;
}

return channelId;
}

static void cleanupTest(const TEST_PROPERTIES * const pTestProperties)
{
 int i;

```

```

/* Close any channels which were opened */
if (pTestProperties->pRxChannelIds != NULL)
{
 for (i = 0; i < pTestProperties->testCount; i++)
 {
 STAR_closeChannel(pTestProperties->pRxChannelIds[i]);
 }
 free(pTestProperties->pRxChannelIds);
}
if (pTestProperties->pTxChannelIds != NULL)
{
 for (i = 0; i < pTestProperties->testCount; i++)
 {
 STAR_closeChannel(pTestProperties->pTxChannelIds[i]);
 }
 free(pTestProperties->pTxChannelIds);
}

/* Clean up the address paths */
if (pTestProperties->pAddressPaths != NULL)
{
 for (i = 0; i < pTestProperties->testCount; i++)
 {
 if (pTestProperties->pAddressPaths[i] != NULL)
 {
 STAR_destroyAddress(pTestProperties->pAddressPaths[i]);
 }
 }
 free(pTestProperties->pAddressPaths);
}
}

static STAR_SPACEWIRE_ADDRESS *readSpaceWireAddress(const int argCount,
const char * const arguments[], int * const pCurrentArgument)
{
 char s[256];
 STAR_SPACEWIRE_ADDRESS *pAddress;
 unsigned char newPath[256];
 U16 pathLen = 0U;
 char *pos = (char *)s;

 /* Read in the path */
 if ((*pCurrentArgument) < argCount)
 {
 puts(arguments[*pCurrentArgument]);
 strncpy(s, arguments[*pCurrentArgument], 255U);
 s[255U] = '\0';
 (*pCurrentArgument)++;
 }
 else
 {
 if (fgets(s, 256, stdin) == NULL)
 {
 puts("No address entered");
 return NULL;
 }
 }

 /* Read until end of buffer or line feed */
 while ((*pos != '\0') && (pathLen < 256U) && (*pos != '\n'))
 {
 unsigned long value;

 errno = 0;
 value = strtoul(pos, &pos, 16);
 if (errno != 0)
 {
 puts("Invalid value entered in address");
 return NULL;
 }

 newPath[pathLen] = (unsigned char)value;
 pathLen++;
 }

 /* Create a SpaceWire address from the path */
 pAddress = STAR_createAddress(newPath, pathLen);

 /* Return the completed address */
 return pAddress;
}

static int readTestParameters(TEST_PROPERTIES * const pTestProperties,
const int argCount, const char * const arguments[])

```



```

{
 char s[256], txRxStr[100];
 size_t len;
 int status, testNumber, currentArgument = 0;

 puts("STAR-System Performance Tester");
 puts("Copyright STAR-Dundee Ltd. (c) 2011-2018");
 puts("www.star-dundee.com\n");

 /* Ask the user which test they wish to perform */
 puts("Which test type do you wish to perform?");
 puts("\t(m) Maximum data rate test");
 puts("\t(r) Random packet size data rate test");
 puts("\t(l) Latency test");

 /* Read in the test type */
 if (currentArgument < argCount)
 {
 puts(arguments[currentArgument]);
 strncpy(s, arguments[currentArgument], 255U);
 s[255U] = '\0';
 currentArgument++;
 }
 else
 {
 if (fgets(s, 256, stdin) == NULL)
 {
 puts("ERROR: No test type selected");
 return 0;
 }
 }
 if ((s[0U] == 'm') || (s[0U] == 'M'))
 {
 pTestProperties->testType = RATE_TEST;
 }
 else if ((s[0U] == 'r') || (s[0U] == 'R'))
 {
 pTestProperties->testType = RANDOM_TEST;
 }
 else if ((s[0U] == 'l') || (s[0U] == 'L'))
 {
 pTestProperties->testType = LATENCY_TEST;
 }
 else
 {
 puts("ERROR: Invalid test type selected");
 return 0;
 }

 /* Ask the user to how many tests they wish to perform */
 printf("\nHow many tests do you wish to perform: ");
 fflush(stdout);
 if (currentArgument < argCount)
 {
 puts(arguments[currentArgument]);
 strncpy(s, arguments[currentArgument], 255U);
 s[255U] = '\0';
 currentArgument++;
 }
 else
 {
 if (fgets(s, 256, stdin) == NULL)
 {
 puts("\nERROR: No test count specified");
 return 0;
 }
 }
 status = sscanf(s, "%d", &pTestProperties->testCount);
 if ((status == 0) || (pTestProperties->testCount < 1) ||
 (pTestProperties->testCount > 100))
 {
 puts("\nERROR: No valid test count specified, enter a value between 1 and 100");
 return 0;
 }

 /* Ask the user if the test should transmit, receive or */
 /* both transmit and receive packets */
 printf("Should these tests transmit packets only (\t\t), receive packets only (\tr\t), or both (\tb\t\n)? ");
 fflush(stdout);
 if (currentArgument < argCount)
 {
 puts(arguments[currentArgument]);
 strncpy(s, arguments[currentArgument], 255U);
 s[255U] = '\0';
 currentArgument++;
 }
}

```

```

else
{
 if (fgets(s, 256, stdin) == NULL)
 {
 puts("\nERROR: No transmit/receive option specified");
 return 0;
 }
}
pTestProperties->mode = 0U;
if ((s[0U] == 't') || (s[0U] == 'T') || (s[0U] == 's') || (s[0U] == 'S') ||
 (s[0U] == 'b') || (s[0U] == 'B'))
{
 strcpy(txRxStr, "transmit");
 pTestProperties->mode |= PERFORM_TRANSMIT;
}
if ((s[0U] == 'r') || (s[0U] == 'R') || (s[0U] == 'b') || (s[0U] == 'B'))
{
 if (pTestProperties->mode != 0U)
 {
 strcat(txRxStr, "/receive");
 }
 else
 {
 strcpy(txRxStr, "receive");
 }
 pTestProperties->mode |= PERFORM_RECEIVE;
}
if (pTestProperties->mode == 0U)
{
 puts("\nERROR: No valid transmit/receive option specified");
 return 0;
}

/* Ask the user to enter a starting packet size */
printf("\nEnter the minimum size of packet to %s: ", txRxStr);
fflush(stdout);
if (currentArgument < argCount)
{
 puts(arguments[currentArgument]);
 strncpy(s, arguments[currentArgument], 255U);
 s[255U] = '\0';
 currentArgument++;
}
else
{
 if (fgets(s, 256, stdin) == NULL)
 {
 puts("\nERROR: No minimum packet size specified");
 return 0;
 }
}
status = sscanf(s, "%d", &pTestProperties->packetStart);
if ((status == 0) || (pTestProperties->packetStart < 1) ||
 (pTestProperties->packetStart > (1024 * 1024)))
{
 printf("\nERROR: No valid minimum packet size specified, enter a packet size between 1 and %d\n",
 1024 * 1024);
 return 0;
}

/* Ask the user to enter an ending packet size */
printf("\nEnter the maximum size of packet to %s: ", txRxStr);
fflush(stdout);
if (currentArgument < argCount)
{
 puts(arguments[currentArgument]);
 strncpy(s, arguments[currentArgument], 255U);
 s[255U] = '\0';
 currentArgument++;
}
else
{
 if (fgets(s, 256, stdin) == NULL)
 {
 puts("\nERROR: No maximum packet size specified");
 return 0;
 }
}
status = sscanf(s, "%d", &pTestProperties->packetEnd);
if ((status == 0) || (pTestProperties->packetEnd < 1) ||
 (pTestProperties->packetEnd > (1024 * 1024)))
{
 printf("\nERROR: No valid maximum packet size specified, enter a packet size between 1 and %d\n",
 1024 * 1024);
 return 0;
}
if (pTestProperties->packetStart > pTestProperties->packetEnd)

```

```

{
 puts("\nERROR: Enter a maximum packet size greater than or equal to the minimum packet size");
 return 0;
}

/* Ask the user to enter the number of bytes to increment the packet size */
/* by in each loop */
if (pTestProperties->packetStart == pTestProperties->packetEnd)
{
 pTestProperties->packetStep = 1;
}
else
{
 printf(
 "\nEnter the number of bytes to increment the packet size by in each loop (0 for log-like
 pattern): ");
 fflush(stdout);
 if (currentArgument < argCount)
 {
 puts(arguments[currentArgument]);
 strncpy(s, arguments[currentArgument], 255U);
 s[255U] = '\0';
 currentArgument++;
 }
 else
 {
 if (fgets(s, 256, stdin) == NULL)
 {
 puts("\nERROR: No packet size increment specified");
 return 0;
 }
 }
 status = sscanf(s, "%d", &pTestProperties->packetStep);
 if ((status == 0) || (pTestProperties->packetStep < 0))
 {
 puts("\nERROR: No valid packet size increment specified");
 return 0;
 }
}

/* Allocate memory for the channels */
if (PERFORMING_TRANSMITS(pTestProperties->mode))
{
 pTestProperties->pTxChannelIds = (STAR_CHANNEL_ID *)calloc(
 pTestProperties->testCount, sizeof(STAR_CHANNEL_ID));
 if (pTestProperties->pTxChannelIds == NULL)
 {
 puts(
 "\nERROR: Unable to allocate memory for the transmit channels");
 return 0;
 }
 pTestProperties->pAddressPaths = (STAR_SPACEWIRE_ADDRESS **)calloc(
 pTestProperties->testCount, sizeof(STAR_SPACEWIRE_ADDRESS *));
 if (pTestProperties->pAddressPaths == NULL)
 {
 puts(
 "\nERROR: Unable to allocate memory for the address paths");
 return 0;
 }
}
if (PERFORMING_RECEIVES(pTestProperties->mode))
{
 pTestProperties->pRxChannelIds = (STAR_CHANNEL_ID *)calloc(
 pTestProperties->testCount, sizeof(STAR_CHANNEL_ID));
 if (pTestProperties->pRxChannelIds == NULL)
 {
 puts("\nERROR: Unable to allocate memory for the receive channels");
 return 0;
 }
}

/* For each test to perform */
for (testNumber = 0; testNumber < pTestProperties->testCount; testNumber++)
{
 /* If transmitting packets */
 if (PERFORMING_TRANSMITS(pTestProperties->mode))
 {
 /* Choose the transmit channel */
 pTestProperties->pTxChannelIds[testNumber] = openChannel("transmit",
 testNumber + 1, STAR_CHANNEL_DIRECTION_OUT, argCount, arguments,
 ¤tArgument, (pTestProperties->useChannel0 != 0) ? 0 : 1);
 if (pTestProperties->pTxChannelIds[testNumber] == 0U)
 {
 return 0;
 }
 }

 /* Ask the user to enter the path to add to the front of packets */

```

```

printf("\nEnter the path to add to the front of packets sent for test %d:\n",
 testNumber);
puts("(Values should be in hex, separated by a space, i.e.: 01 0f 02)");

/* Read in the path */
pTestProperties->pAddressPaths[testNumber] = readSpaceWireAddress(
 argCount, arguments, ¤tArgument);
}

/* If receiving packets */
if (PERFORMING_RECEIVES (pTestProperties->mode))
{
 /* Choose the receive channel */
 pTestProperties->pRxChannelIds[testNumber] = openChannel("receive",
 testNumber + 1, STAR_CHANNEL_DIRECTION_IN, argCount, arguments,
 ¤tArgument, (pTestProperties->useChannel0 != 0) ? 0 : 1);
 if (pTestProperties->pRxChannelIds[testNumber] == 0U)
 {
 return 0;
 }
}

/* Ask the user to enter the path to the file to use, */
/* or to press enter to not store to a file */
printf("\nEnter the path and the file to be used to store the results (hit enter if the results should
 not be stored): ");
fflush(stdout);

/* Read in the file path */
if (currentArgument < argCount)
{
 puts(arguments[currentArgument]);
 strncpy(pTestProperties->pFilePath, arguments[currentArgument],
 MAX_PATH);
 pTestProperties->pFilePath[MAX_PATH] = '\0';
 currentArgument++;
}
else
{
 if (fgets(pTestProperties->pFilePath, MAX_PATH, stdin) == NULL)
 {
 strcpy(pTestProperties->pFilePath, "");
 }
}

len = strlen(pTestProperties->pFilePath);
if (len != 0U)
{
 while ((len > 0U) &&
 ((pTestProperties->pFilePath[len - 1] == '\r') ||
 (pTestProperties->pFilePath[len - 1] == '\n')))
 {
 pTestProperties->pFilePath[len - 1] = '\0';
 len--;
 }
}

/* if writing the results to file */
if (len > 0U)
{
 printf("\nEnter a description of the test: ");
 fflush(stdout);
 if (currentArgument < argCount)
 {
 puts(arguments[currentArgument]);
 strncpy(pTestProperties->pFileDescription,
 arguments[currentArgument], 999U);
 pTestProperties->pFileDescription[999U] = '\0';
 }
 else
 {
 if (fgets(pTestProperties->pFileDescription, 999U, stdin) == NULL)
 {
 strcpy(pTestProperties->pFileDescription, "");
 }
 }
}

puts("\n");

return 1;
}

static void performRateTestForPacketSize(const int packetSize, const U8 mode,
 const int testCount, const STAR_CHANNEL_ID * const pTxChannels,

```

```

STAR_SPACEWIRE_ADDRESS ** const pAddressPaths,
const STAR_CHANNEL_ID * const pRxChannels, double * const pDataRate,
double * const pPacketRate, double * const pPacketTime,
CPU_USAGE_PROPERTIES * const pStartCpuProperties,
CPU_USAGE_PROPERTIES * const pEndCpuProperties)
{
 int i, j, bufferNum, buffersUsed, packetNum, loopNum;
 STAR_TRANSFER_OPERATION **ppRxTransOp = NULL, **ppTxTransOp = NULL;
 clock_t start = 0, finish;
 double duration, bitsSent;
 STAR_TRANSFER_STATUS status = STAR_TRANSFER_STATUS_ERROR;
 unsigned char *pTransmitBuffer = NULL;
 STAR_STREAM_ITEM **ppTxPackets = NULL;
 unsigned int packetItem;
 unsigned int rxPackSize;
 unsigned int receivedItemCount;
 STAR_STREAM_ITEM** receivedItems;
 STAR_SPACEWIRE_PACKET* pReceivedPacket;
 STAR_EOP_TYPE rxEopType;

 /* Calculate the number of packets to transmit in each loop and */
 /* the number of loops to perform */
 packetNum = MAX(BUFFER_NUM, BUFFER_SIZE / packetSize);
 loopNum = LOOP_NUM;

#ifdef DEBUG
 printf("\nPackets per loop:\t%d", packetNum);
 printf("\nNumber of loops:\t%d", loopNum);
 printf("\nBytes per packet:\t%d", packetSize);
 printf("\nExpected packets:\t%u", loopNum * packetNum);
 printf("\nExpected bytes:\t%f\n",
 (double)packetSize * (double)packetNum * (double)loopNum);
 printf("\n");
#endif

 /* if performing receives */
 if (PERFORMING_RECEIVES(mode))
 {
 /* Allocate memory for the receive transfer operations */
 ppRxTransOp = (STAR_TRANSFER_OPERATION **)calloc(BUFFER_NUM * testCount,
 sizeof(STAR_TRANSFER_OPERATION *));
 if (ppRxTransOp == NULL)
 {
 puts("Couldn't allocate memory for receive operations, exiting.");
 goto Rate_End_Cleanup;
 }

 /* Create transfer operations to receive all packets in each buffer */
 for (i = 0; i < (BUFFER_NUM * testCount); i++)
 {
 ppRxTransOp[i] =
 STAR_createRxOperation(packetNum,
 STAR_RECEIVE_PACKETS);
 if (ppRxTransOp[i] == NULL)
 {
 puts("Couldn't create receive operation, exiting.");
 goto Rate_End_Cleanup;
 }
 }
 }

 /* if performing transmits */
 if (PERFORMING_TRANSMITS(mode))
 {
 /* Allocate transmit buffer */
 /* This buffer will be reused for all packets sent */
 pTransmitBuffer = (unsigned char *)calloc(1U, packetSize);
 if (pTransmitBuffer == NULL)
 {
 puts("Couldn't allocate memory for transmit buffer, exiting.");
 goto Rate_End_Cleanup;
 }

 /* Set the first byte of the transmit buffer to 0xfe. This is the */
 /* byte that will be at the front of the packet at the destination */
 /* (if path addressing is used) */
 if (packetSize > 0)
 {
 pTransmitBuffer[0U] = 0xfeU;
 }

 /* Allocate memory for packet pointers */
 ppTxPackets = (STAR_STREAM_ITEM **)calloc(
 packetNum * BUFFER_NUM * testCount, sizeof(STAR_STREAM_ITEM *));
 if (ppTxPackets == NULL)
 {
 puts("Couldn't allocate memory for transmit packets, exiting.");
 }
 }
}

```

```

 goto Rate_End_Cleanup;
 }

 /* Allocate memory for the transmit identifiers */
 ppTxTransOp = (STAR_TRANSFER_OPERATION **)calloc(BUFFER_NUM * testCount,
 sizeof(STAR_TRANSFER_OPERATION *));
 if (ppTxTransOp == NULL)
 {
 puts("Couldn't allocate memory for transmit operations, exiting.");
 goto Rate_End_Cleanup;
 }

 /* Construct packets */
 for (i = 0; i < (packetNum * BUFFER_NUM); i++)
 {
 for (j = 0; j < testCount; j++)
 {
 /* Create stream item using the transmit buffer */
 ppTxPackets[(i * testCount) + j] = STAR_createPacket(
 pAddressPaths[j], pTransmitBuffer, packetSize,
 STAR_EOP_TYPE_EOP);
 if (ppTxPackets[(i * testCount) + j] == NULL)
 {
 puts("Couldn't create packet for transmit operation, exiting.");
 goto Rate_End_Cleanup;
 }
 }
 }

 /* Create transfer operations to transmit all packets in each buffer */
 for (i = 0; i < (BUFFER_NUM * testCount); i++)
 {
 ppTxTransOp[i] = STAR_createTxOperation(
 ppTxPackets + (packetNum * i), packetNum);
 if (ppTxTransOp[i] == NULL)
 {
 puts("Couldn't create transmit operation, exiting.");
 goto Rate_End_Cleanup;
 }
 }

 /* Start the clock */
 start = GET_TIME();
 STORE_CPU_USAGE(pStartCpuProperties);
}

bufferNum = 0;

/* for each loop to perform */
for (i = 0; i < loopNum; i++)
{
 /* If performing receives */
 if (PERFORMING_RECEIVES(mode))
 {
 /* If a receive has already been submitted for this buffer */
 if (i >= BUFFER_NUM)
 {
 for (j = 0; j < testCount; j++)
 {
 /* Wait on packets being received */
 status = STAR_waitOnTransferOperationCompletion(
 ppRxTransOp[(bufferNum * testCount) + j],
 STAR_INFINITE);

 receivedItems = STAR_getTransferItemList(
 ppRxTransOp[(bufferNum * testCount) + j],
 &receivedItemCount);
 if (status != STAR_TRANSFER_STATUS_COMPLETE)
 {
 printf("ERROR: Could not complete receive in loop %d, error of %d\n",
 i, status);
 goto Rate_End_Cleanup;
 }

 if (receivedItems == NULL)
 {
 printf("ERROR: Receive completed but no data was received. Loop: %d\n",
 i);
 goto Rate_End_Cleanup;
 }

 for (packetItem = 0; packetItem < receivedItemCount;
 packetItem++)
 {
 pReceivedPacket = (STAR_SPACEWIRE_PACKET*)receivedItems[
 packetItem]->item;

```

```

 rxPacSize = STAR_getPacketLength(pReceivedPacket);
 if (rxPacSize != (unsigned int)packetSize)
 {
 printf("Rx operation completed, received incorrect packet size: %u, Loop %d \n",
 rxPacSize, i);
 }

 rxEopType = STAR_getPacketEOP(pReceivedPacket);
 if (rxEopType != STAR_EOP_TYPE_EOP)
 {
 printf("Rx operation completed, received unexpected EOP type: %d, Loop %d \n",
 rxEopType, i);
 }
 }
 STAR_destroyTransferItemList(receivedItems,
 receivedItemCount, 0);
}

/* If this is the first receive and not performing transmits */
if ((i == BUFFER_NUM) && (!PERFORMING_TRANSMITS(mode)))
{
 /* Start the clock */
 start = GET_TIME();
 STORE_CPU_USAGE(pStartCpuProperties);
}

/* Start receiving the next group of packets */
for (j = 0; j < testCount; j++)
{
 STAR_submitTransferOperation(pRxChannels[j],
 ppRxTransOp[(bufferNum * testCount) + j]);

 status = STAR_getTransferOperationStatus(ppRxTransOp[
 (bufferNum * testCount) + j]);
 if ((status != STAR_TRANSFER_STATUS_STARTED) &&
 (status != STAR_TRANSFER_STATUS_COMPLETE))
 {
 printf("Could not perform receive in loop %d, error of %d\n",
 i, status);
 goto Rate_End_Cleanup;
 }
}

/* if performing transmits */
if (PERFORMING_TRANSMITS(mode))
{
 /* If a transmit has already been submitted for this buffer */
 if (i >= BUFFER_NUM)
 {
 /* Wait on the last transmit completing */
 for (j = 0; j < testCount; j++)
 {
 status = STAR_waitOnTransferOperationCompletion(
 ppTxTransOp[(bufferNum * testCount) + j],
 STAR_INFINITE);

 if (status != STAR_TRANSFER_STATUS_COMPLETE)
 {
 printf("ERROR: Could not complete transmit in loop %d, error of %d\n",
 i, status);
 goto Rate_End_Cleanup;
 }
 }
 }

 /* Start transmitting the next packets */
 for (j = 0; j < testCount; j++)
 {
 STAR_submitTransferOperation(pTxChannels[j],
 ppTxTransOp[(bufferNum * testCount) + j]);

 status = STAR_getTransferOperationStatus(
 ppTxTransOp[(bufferNum * testCount) + j]);
 if ((status != STAR_TRANSFER_STATUS_STARTED) &&
 (status != STAR_TRANSFER_STATUS_COMPLETE))
 {
 printf("Could not perform transmit in loop %d, error of %d\n",
 i, status);
 goto Rate_End_Cleanup;
 }
 }
}

/* Move to the next buffer */

```

```

 bufferNum++;
 if (bufferNum == BUFFER_NUM)
 {
 bufferNum = 0;
 }
 }

 /* Account for LOOP_NUM less than BUFFER_NUM */
 buffersUsed = BUFFER_NUM;
 if (LOOP_NUM < BUFFER_NUM)
 {
 buffersUsed = LOOP_NUM;
 }

 /* For each buffer used */
 for (bufferNum = 0; bufferNum < buffersUsed; bufferNum++)
 {
 /* If performing transmits */
 if (PERFORMING_TRANSMITS(mode))
 {
 /* Wait on the next transmit completing */
 for (j = 0; j < testCount; j++)
 {
 status = STAR_waitOnTransferOperationCompletion(
 ppTxTransOp[(bufferNum * testCount) + j], STAR_INFINITE);
 if (status != STAR_TRANSFER_STATUS_COMPLETE)
 {
 printf("ERROR: Could not complete transmit in loop %d, error of %d\n",
 i, status);
 goto Rate_End_Cleanup;
 }
 }
 }

 /* If performing receives */
 if (PERFORMING_RECEIVES(mode))
 {
 /* Wait on packets being received */
 for (j = 0; j < testCount; j++)
 {
 status = STAR_waitOnTransferOperationCompletion(
 ppRxTransOp[(bufferNum * testCount) + j], STAR_INFINITE);

 receivedItems = STAR_getTransferItemList(
 ppRxTransOp[(bufferNum * testCount) + j],
 &receivedItemCount);

 if (receivedItems == NULL)
 {
 printf("ERROR: Receive completed but no data was received. Loop: %d\n",
 loopNum);
 goto Rate_End_Cleanup;
 }

 for (packetItem = 0U; packetItem < receivedItemCount;
 packetItem++)
 {
 pReceivedPacket =
 (STAR_SPACEWIRE_PACKET*)receivedItems[packetItem]->item;

 rxPackSize = STAR_getPacketLength(pReceivedPacket);
 if (rxPackSize != (unsigned int)packetSize)
 {
 printf("Rx operation completed, received incorrect packet size: %u, Loop %d \n",
 rxPackSize, loopNum);
 }

 rxEopType = STAR_getPacketEOP(pReceivedPacket);
 if (rxEopType != STAR_EOP_TYPE_EOP)
 {
 printf("Rx operation completed, received unexpected EOP type: %d, Loop %d \n",
 rxEopType, loopNum);
 }
 }
 STAR_destroyTransferItemList(receivedItems, receivedItemCount,
 0);

 if (status != STAR_TRANSFER_STATUS_COMPLETE)
 {
 printf("ERROR: Could not complete receive in loop %d, error of %d\n",
 i, status);
 goto Rate_End_Cleanup;
 }
 }
 }
 }
}

```



```

/* Stop the clock */
finish = GET_TIME();
STORE_CPU_USAGE(pEndCpuProperties);

duration = (double)(finish - start) / TIME_DIVIDER;

bitsSent = (double)testCount * (double)packetSize;
if (pAddressPaths != NULL)
{
 for (i = 0; i < testCount; i++)
 {
 if (pAddressPaths[i] != NULL)
 {
 bitsSent += STAR_getSpaceWireAddressPathLength(
 pAddressPaths[i]);
 }
 }
}
bitsSent = bitsSent * (double)packetNum * (double)loopNum * (double)8;

/* Calculate the data and packet rate */
if (duration > 0.0)
{
 *pDataRate = (bitsSent / duration) / 1000000.0;
 *pPacketRate = (double)(packetNum * testCount * loopNum) / duration;
}
else
{
 *pDataRate = 0.0;
 *pPacketRate = 0.0;
}
*pPacketTime = (duration * 1000000.0) /
 (double)(packetNum * testCount * loopNum);

/* Clean-up */
Rate_End_Cleanup:

/* If performing receives */
if (PERFORMING_RECEIVES(mode))
{
 /* Dispose of all receive transfer operations */
 if (ppRxTransOp != NULL)
 {
 for (i = 0; i < (BUFFER_NUM * testCount); i++)
 {
 if (ppRxTransOp[i] != NULL)
 {
 STAR_disposeTransferOperation(ppRxTransOp[i]);
 }
 }
 free(ppRxTransOp);
 }
}

/* If performing transmits */
if (PERFORMING_TRANSMITS(mode))
{
 if (ppTxTransOp != NULL)
 {
 /* Dispose of all transmit transfer operations */
 for (i = 0; i < (BUFFER_NUM * testCount); i++)
 {
 if (ppTxTransOp[i] != NULL)
 {
 STAR_disposeTransferOperation(ppTxTransOp[i]);
 }
 }
 }

 if (ppTxPackets != NULL)
 {
 /* Dispose of stream item buffer */
 for (i = 0; i < (packetNum * BUFFER_NUM * testCount); i++)
 {
 if (ppTxPackets[i] != NULL)
 {
 STAR_destroyStreamItem(ppTxPackets[i]);
 }
 }

 /* Free the packet pointer array*/
 free(ppTxPackets);
 }

 /* Free the transfer operation pointers */
 if (ppTxTransOp != NULL)

```

```

 {
 free(ppTxTransOp);
 }

 /* Free the transmit buffer */
 if (pTransmitBuffer != NULL)
 {
 free(pTransmitBuffer);
 }
 }

 if (status != STAR_TRANSFER_STATUS_COMPLETE)
 {
 *pDataRate = 0.0;
 *pPacketRate = 0.0;
 *pPacketTime = 0.0;
 }
}

static void performRateTest(const TEST_PROPERTIES * const pTestProperties,
 FILE * const pOutputFile)
{
 int packetSize;
 double dataRate = 0, packetRate = 0, packetTime = 0;
 CPU_USAGE_PROPERTIES startCpuProperties, endCpuProperties;

 /* If a file is to be written to */
 if (pOutputFile != NULL)
 {
 /* Write the statistics column headers to the file */
 fprintf(pOutputFile, "Packet Size\tData Rate (Mbit/s)\t");
 fprintf(pOutputFile,
 "Packet Rate (packets/s)\tAverage Packet Time (microseconds)");
 PRINT_CPU_USAGE_HEADERS(pOutputFile);
 }

 /* for each packet size to test */
 packetSize = pTestProperties->packetStart;
 while (packetSize <= pTestProperties->packetEnd)
 {
 /* Display the packet size on screen */
 printf(
 "Performing maximum data rate test using packets of %d bytes...\n",
 packetSize);

 /* Perform the test for the current packet size */
 performRateTestForPacketSize(packetSize, pTestProperties->mode,
 pTestProperties->testCount, pTestProperties->pTxChannelIds,
 pTestProperties->pAddressPaths, pTestProperties->pRxChannelIds,
 &dataRate, &packetRate, &packetTime, &startCpuProperties,
 &endCpuProperties);

 /* Display the results of the test */
 printf(
 "\t%-3.2f Mbit/s %-3.2f packets/s %-3.2f microseconds/packet\n",
 dataRate, packetRate, packetTime);
 PRINT_CPU_USAGE(NULL, &startCpuProperties, &endCpuProperties);

 /* If writing the results to file */
 if (pOutputFile != NULL)
 {
 /* Write the results to the file */
 fprintf(pOutputFile, "%8d\t%-3.2f\t%-3.2f\t%-3.2f", packetSize,
 dataRate, packetRate, packetTime);
 PRINT_CPU_USAGE(pOutputFile, &startCpuProperties,
 &endCpuProperties);
 fflush(pOutputFile);
 }

 /* if not performing receives and this isn't the last loop */
 if ((!PERFORMING_RECEIVES(pTestProperties->mode)) &&
 (packetSize < pTestProperties->packetEnd))
 {
 /* Sleep for 5 seconds to allow all packets to be sent */
 SLEEP(5000U);
 }

 /* Move to the next packet size */
 if (packetSize == pTestProperties->packetEnd)
 {
 packetSize++;
 }
 else if (pTestProperties->packetStep == 0)
 {
 packetSize = MIN(2 * packetSize, pTestProperties->packetEnd);
 }
 }
}

```

```

 else
 {
 packetSize = MIN(packetSize + pTestProperties->packetStep,
 pTestProperties->packetEnd);
 }
 }
}

static void performRandomTestForPacketSize(const int minPacketSize,
 const int maxPacketSize, const U8 mode, const int testCount,
 const STAR_CHANNEL_ID * const pTxChannels,
 STAR_SPACEWIRE_ADDRESS ** const pAddressPaths,
 const STAR_CHANNEL_ID * const pRxChannels, double * const pDataRate,
 double * const pPacketRate, double * const pPacketTime,
 double * const pPacketSize,
 CPU_USAGE_PROPERTIES * const pStartCpuProperties,
 CPU_USAGE_PROPERTIES * const pEndCpuProperties)
{
 int i, j, bufferNum, buffersUsed, packetNum, loopNum;
 STAR_TRANSFER_OPERATION **ppRxTransOp = NULL, **ppTxTransOp = NULL;
 clock_t start = 0, finish;
 double duration, bytesSent = 0.0;
 STAR_TRANSFER_STATUS status = STAR_TRANSFER_STATUS_ERROR;
 unsigned char *pTransmitBuffer = NULL;
 STAR_STREAM_ITEM **ppTxPackets = NULL;
 int *pPacketLengths = NULL;

 /* Seed random number function */
 srand((unsigned int)time(NULL));

 /* Calculate the number of packets to transmit in each loop and */
 /* the number of loops to perform */
 packetNum = MAX(BUFFER_NUM, BUFFER_SIZE / maxPacketSize);
 loopNum = LOOP_NUM;

 /* Allocate memory to hold length info for packets */
 pPacketLengths = (int *)calloc(packetNum * BUFFER_NUM, sizeof(int));
 if (pPacketLengths == NULL)
 {
 puts("Couldn't allocate memory for the packet lengths, exiting.");
 goto Random_End_Cleanup;
 }

 for (i = 0; i < (packetNum * BUFFER_NUM); i++)
 {
 /* Set the initial random lengths for the packets */
 pPacketLengths[i] = ((int)(((double)rand() / ((double)RAND_MAX + 1.0)) *
 (maxPacketSize - minPacketSize))) + minPacketSize;
 }

 /* if performing receives */
 if (PERFORMING_RECEIVES(mode))
 {
 /* Allocate memory for the receive transfer operations */
 ppRxTransOp = (STAR_TRANSFER_OPERATION **)calloc(BUFFER_NUM * testCount,
 sizeof(STAR_TRANSFER_OPERATION *));
 if (ppRxTransOp == NULL)
 {
 puts("Couldn't allocate memory for receive operations, exiting.");
 goto Random_End_Cleanup;
 }

 /* Create transfer operations to receive all packets in each buffer */
 for (i = 0; i < (BUFFER_NUM * testCount); i++)
 {
 ppRxTransOp[i] =
 STAR_createRxOperation(packetNum,
 STAR_RECEIVE_PACKETS);
 if (ppRxTransOp[i] == NULL)
 {
 puts("Couldn't create receive operation, exiting.");
 goto Random_End_Cleanup;
 }
 }
 }

 /* if performing transmits */
 if (PERFORMING_TRANSMITS(mode))
 {
 /* Allocate transmit buffer */
 pTransmitBuffer = (unsigned char *)calloc(1U, maxPacketSize);
 if (pTransmitBuffer == NULL)
 {
 puts("Couldn't allocate memory for transmit buffer, exiting.");
 goto Random_End_Cleanup;
 }
 }
}

```

```

/* Allocate memory for packet pointers */
ppTxPackets = (STAR_STREAM_ITEM **)calloc(
 packetNum * BUFFER_NUM * testCount, sizeof(STAR_STREAM_ITEM *));
if (ppTxPackets == NULL)
{
 puts("Couldn't allocate memory for transmit packets, exiting.");
 goto Random_End_Cleanup;
}

/* Set the first byte of the transmit buffer to 0xfe. This is the */
/* byte that will be at the front of the packet at the destination */
/* (if path addressing is used) */
if (maxPacketSize > 0)
{
 pTransmitBuffer[0U] = 0xfeU;
}

/* Allocate memory for the transmit identifiers */
ppTxTransOp = (STAR_TRANSFER_OPERATION **)calloc(BUFFER_NUM * testCount,
 sizeof(STAR_TRANSFER_OPERATION *));
if (ppTxTransOp == NULL)
{
 puts("Couldn't allocate memory for transmit operations, exiting.");
 goto Random_End_Cleanup;
}

/* Construct packets */
for (i = 0; i < (packetNum * BUFFER_NUM); i++)
{
 for (j = 0; j < testCount; j++)
 {
 /* Create stream item using the transmit buffer */
 ppTxPackets[(i * testCount) + j] = STAR_createPacket(
 pAddressPaths[j], pTransmitBuffer, pPacketLengths[i],
 STAR_EOP_TYPE_EOP);
 if (ppTxPackets[(i * testCount) + j] == NULL)
 {
 puts("Couldn't create packets for transmit operations, exiting.");
 goto Random_End_Cleanup;
 }
 }
}

/* Create transfer operations to transmit all packets in each buffer */
for (i = 0; i < (BUFFER_NUM * testCount); i++)
{
 ppTxTransOp[i] = STAR_createTxOperation(
 ppTxPackets + (packetNum * i), packetNum);
 if (ppTxTransOp[i] == NULL)
 {
 puts("Couldn't create transmit operations, exiting.");
 goto Random_End_Cleanup;
 }
}

/* Start the clock */
start = GET_TIME();
STORE_CPU_USAGE(pStartCpuProperties);
}

bufferNum = 0;

/* for each loop to perform */
for (i = 0; i < loopNum; i++)
{
 /* If performing receives */
 if (PERFORMING_RECEIVES(mode))
 {
 /* If a receive has already been submitted for this buffer */
 if (i >= BUFFER_NUM)
 {
 for (j = 0; j < testCount; j++)
 {
 /* Wait on packets being received */
 status = STAR_waitOnTransferOperationCompletion(
 ppRxTransOp[(bufferNum * testCount) + j],
 STAR_INFINITE);
 if (status != STAR_TRANSFER_STATUS_COMPLETE)
 {
 printf("ERROR: Could not complete receive in loop %d, error of %d\n",
 i, status);
 goto Random_End_Cleanup;
 }
 }

 /* If this is the first receive and not performing transmits */

```

```

 if ((i == BUFFER_NUM) && (!PERFORMING_TRANSMITS(mode)))
 {
 /* Start the clock */
 start = GET_TIME();
 STORE_CPU_USAGE(pStartCpuProperties);
 }
 }

 /* Start receiving the next group of packets */
 for (j = 0; j < testCount; j++)
 {
 STAR_submitTransferOperation(pRxChannels[j],
 ppRxTransOp[(bufferNum * testCount) + j]);

 status = STAR_getTransferOperationStatus(
 ppRxTransOp[(bufferNum * testCount) + j]);
 if ((status != STAR_TRANSFER_STATUS_STARTED) &&
 (status != STAR_TRANSFER_STATUS_COMPLETE))
 {
 printf("Could not perform receive in loop %d, error of %d\n",
 i, status);
 goto Random_End_Cleanup;
 }
 }
}

/* if performing transmits */
if (PERFORMING_TRANSMITS(mode))
{
 /* If a transmit op has already been submitted for this buffer */
 if (i >= BUFFER_NUM)
 {
 /* Wait on the last transmit completing */
 for (j = 0; j < testCount; j++)
 {
 status = STAR_waitOnTransferOperationCompletion(
 ppTxTransOp[(bufferNum * testCount) + j], STAR_INFINITE);

 if (status != STAR_TRANSFER_STATUS_COMPLETE)
 {
 printf("ERROR: Could not complete transmit in loop %d, error of %d\n",
 i, status);
 goto Random_End_Cleanup;
 }
 }
 }
}

/* Update the number of bytes transmitted/received */
for (j = 0; j < packetNum; j++)
{
 bytesSent +=
 (pPacketLengths[(packetNum * bufferNum) + j] * testCount);
}
if (pAddressPaths != NULL)
{
 for (j = 0; j < testCount; j++)
 {
 if (pAddressPaths[j] != NULL)
 {
 bytesSent += (packetNum * STAR_getSpaceWireAddressPathLength
 (pAddressPaths[j]));
 }
 }
}

/* if performing transmits */
if (PERFORMING_TRANSMITS(mode))
{
 /* Note that this current version of the Performance Tester does */
 /* not change the size of the data in each loop. This is due to */
 /* an as yet unfixed bug in STAR_setPacketData */

 /* Start transmitting the next packets */
 for (j = 0; j < testCount; j++)
 {
 STAR_submitTransferOperation(pTxChannels[j],
 ppTxTransOp[(bufferNum * testCount) + j]);

 status = STAR_getTransferOperationStatus(
 ppTxTransOp[(bufferNum * testCount) + j]);
 if ((status != STAR_TRANSFER_STATUS_STARTED) &&
 (status != STAR_TRANSFER_STATUS_COMPLETE))
 {
 printf("Could not perform transmit in loop %d, error of %d\n",
 i, status);
 goto Random_End_Cleanup;
 }
 }
}

```

```

 }
}

/* Move to the next buffer */
bufferNum++;
if (bufferNum == BUFFER_NUM)
{
 bufferNum = 0;
}

/* Account for LOOP_NUM less than BUFFER_NUM */
buffersUsed = BUFFER_NUM;
if (LOOP_NUM < BUFFER_NUM)
{
 buffersUsed = LOOP_NUM;
}

/* For each buffer used */
for (bufferNum = 0; bufferNum < buffersUsed; bufferNum++)
{
 /* If performing transmits */
 if (PERFORMING_TRANSMITS(mode))
 {
 /* Wait on the next transmit completing */
 for (j = 0; j < testCount; j++)
 {
 status = STAR_waitOnTransferOperationCompletion(
 ppTxTransOp[(bufferNum * testCount) + j], STAR_INFINITE);
 if (status != STAR_TRANSFER_STATUS_COMPLETE)
 {
 printf("ERROR: Could not complete transmit in loop %d, error of %d\n",
 i, status);
 goto Random_End_Cleanup;
 }
 }

 /* If performing receives */
 if (PERFORMING_RECEIVES(mode))
 {
 /* Wait on packets being received */
 for (j = 0; j < testCount; j++)
 {
 status = STAR_waitOnTransferOperationCompletion(
 ppRxTransOp[(bufferNum * testCount) + j], STAR_INFINITE);
 if (status != STAR_TRANSFER_STATUS_COMPLETE)
 {
 printf("ERROR: Could not complete receive in loop %d, error of %d\n",
 i, status);
 goto Random_End_Cleanup;
 }
 }
 }
 }
}

/* Stop the clock */
finish = GET_TIME();
STORE_CPU_USAGE(pEndCpuProperties);

duration = (double)(finish - start) / TIME_DIVIDER;

/* Calculate the average packet size */
*pPacketSize = (double)bytesSent / (packetNum * testCount * loopNum);

/* Calculate the data and packet rate */
if (duration > 0.0)
{
 *pDataRate = ((bytesSent * 8.0) / duration) / 1000000.0;
 *pPacketRate = (double)(packetNum * testCount * loopNum) / duration;
}
else
{
 *pDataRate = 0.0;
 *pPacketRate = 0.0;
}

*pPacketTime = (duration * 1000000.0) /
 (double)(packetNum * testCount * loopNum);

/* Clean-up */
Random_End_Cleanup:

/* If performing receives */
if (PERFORMING_RECEIVES(mode))
{

```

```

 if (ppRxTransOp != NULL)
 {
 for (i = 0; i < (BUFFER_NUM * testCount); i++)
 {
 /* Dispose of all receive transfer operations */
 if (ppRxTransOp[i] != NULL)
 {
 STAR_disposeTransferOperation(ppRxTransOp[i]);
 }
 }

 free(ppRxTransOp);
 }
 }

 /* If performing transmits */
 if (PERFORMING_TRANSMITS(mode))
 {
 if (ppTxTransOp != NULL)
 {
 /* Dispose of all transmit transfer operations */
 for (i = 0; i < (BUFFER_NUM * testCount); i++)
 {
 if (ppTxTransOp[i] != NULL)
 {
 STAR_disposeTransferOperation(ppTxTransOp[i]);
 }
 }
 }

 /* Free transmit stream items (packets) */
 if (ppTxPackets != NULL)
 {
 for (i = 0; i < (packetNum * BUFFER_NUM * testCount); i++)
 {
 if (ppTxPackets[i] != NULL)
 {
 STAR_destroyStreamItem(ppTxPackets[i]);
 }
 }

 /* Free the packet pointer array*/
 free(ppTxPackets);
 }

 /* Free the transfer operation pointers */
 if (ppTxTransOp != NULL)
 {
 free(ppTxTransOp);
 }

 /* Free the transmit buffer */
 if (pTransmitBuffer != NULL)
 {
 free(pTransmitBuffer);
 }
 }

 /* Free the array of packet lengths */
 if (pPacketLengths != NULL)
 {
 free(pPacketLengths);
 }

 if (status != STAR_TRANSFER_STATUS_COMPLETE)
 {
 *pDataRate = 0.0;
 *pPacketRate = 0.0;
 *pPacketTime = 0.0;
 }
}

static void performRandomTest(const TEST_PROPERTIES * const pTestProperties,
 FILE * const pOutputFile)
{
 int minPacketSize;
 double dataRate = 0, packetRate = 0, packetTime = 0, packetSize = 0;
 CPU_USAGE_PROPERTIES startCpuProperties, endCpuProperties;

 /* If a file is to be written to */
 if (pOutputFile != NULL)
 {
 /* Write the statistics column headers to the file */
 fprintf(pOutputFile,
 "Minimum Packet Size\tMaximum Packet Size\tData Rate (Mbit/s)\t");
 fprintf(pOutputFile,

```

```

 "Packet Rate (packets/s)\tAverage Packet Time (microseconds)\t");
 fprintf(pOutputFile,
 "Average Packet Size (bytes)");
 PRINT_CPU_USAGE_HEADERS(pOutputFile);
}

/* for each packet size to test */
minPacketSize = pTestProperties->packetStart;
while (minPacketSize <= pTestProperties->packetEnd)
{
 /* Display the packet size on screen */
 printf("Performing random packet size data rate test using packets of size %d to %d bytes...\n",
 minPacketSize, pTestProperties->packetEnd);

 /* Perform the test for the current packet size */
 performRandomTestForPacketSize(minPacketSize,
 pTestProperties->packetEnd, pTestProperties->mode,
 pTestProperties->testCount, pTestProperties->pTxChannelIds,
 pTestProperties->pAddressPaths, pTestProperties->pRxChannelIds,
 &dataRate, &packetRate, &packetTime, &packetSize,
 &startCpuProperties, &endCpuProperties);

 /* Display the results of the test */
 printf("\t%-3.2f Mbit/s %-3.2f packets/s %-3.2f microseconds/packet %-3.2f average packet size\n",
 dataRate, packetRate, packetTime, packetSize);
 PRINT_CPU_USAGE(NULL, &startCpuProperties, &endCpuProperties);

 /* If writing the results to file */
 if (pOutputFile != NULL)
 {
 /* Write the results to the file */
 fprintf(pOutputFile, "%8d\t%8d\t%-3.2f\t%-3.2f\t%-3.2f\t%-3.2f",
 minPacketSize, pTestProperties->packetEnd, dataRate, packetRate, packetTime,
 packetSize);
 PRINT_CPU_USAGE(pOutputFile, &startCpuProperties,
 &endCpuProperties);
 fflush(pOutputFile);
 }

 /* if not performing receives and this isn't the last loop */
 if ((!PERFORMING_RECEIVES(pTestProperties->mode)) &&
 (minPacketSize < pTestProperties->packetEnd))
 {
 /* Sleep for 5 seconds to allow all packets to be sent */
 SLEEP(5000U);
 }

 /* Move to the next packet size */
 if (minPacketSize == pTestProperties->packetEnd)
 {
 minPacketSize++;
 }
 else if ((minPacketSize == pTestProperties->packetStart) &&
 (pTestProperties->packetStep != 0) && ((minPacketSize +
 pTestProperties->packetStep) > pTestProperties->packetEnd))
 {
 minPacketSize = pTestProperties->packetEnd + 1;
 }
 else if (pTestProperties->packetStep == 0)
 {
 minPacketSize = MIN(2 * minPacketSize, pTestProperties->packetEnd);
 }
 else
 {
 minPacketSize = MIN(minPacketSize + pTestProperties->packetStep,
 pTestProperties->packetEnd);
 }
}
}

static void performLatencyTestForPacketSize(const int packetSize, const U8 mode,
 const int testCount, const STAR_CHANNEL_ID * const pTxChannels,
 STAR_SPACEWIRE_ADDRESS ** const pAddressPaths,
 const STAR_CHANNEL_ID * const pRxChannels, double * const pDataRate,
 double * const pPacketRate, double * const pPacketTime,
 CPU_USAGE_PROPERTIES * const pStartCpuProperties,
 CPU_USAGE_PROPERTIES * const pEndCpuProperties)
{
 int i, j, loopNum;
 STAR_TRANSFER_OPERATION **ppRxTransOp = NULL, **ppTxTransOp = NULL;
 clock_t start = 0, finish;
 double duration, bitsSent;
 STAR_TRANSFER_STATUS status = STAR_TRANSFER_STATUS_ERROR;
 unsigned char *pTransmitBuffer = NULL;
 STAR_STREAM_ITEM **ppTxPackets = NULL;

```



```

/* Calculate the number of loops to perform */
loopNum = LOOP_NUM;

/* if performing receives */
if (PERFORMING_RECEIVES(mode))
{
 /* Allocate memory for the receive transfer operations */
 ppRxTransOp = (STAR_TRANSFER_OPERATION **)calloc(testCount,
 sizeof(STAR_TRANSFER_OPERATION *));
 if (ppRxTransOp == NULL)
 {
 puts("Couldn't allocate memory for receive operations, exiting.");
 goto Latency_End_Cleanup;
 }

 for (i = 0; i < testCount; i++)
 {
 /* Create a transfer operation for receiving */
 ppRxTransOp[i] = STAR_createRxOperation(1,
 STAR_RECEIVE_PACKETS);
 if (ppRxTransOp[i] == NULL)
 {
 puts("Couldn't create receive operation, exiting.");
 goto Latency_End_Cleanup;
 }
 }
}

/* if performing transmits */
if (PERFORMING_TRANSMITS(mode))
{
 /* Allocate transmit buffer */
 pTransmitBuffer = (unsigned char *)calloc(1U, packetSize);
 if (pTransmitBuffer == NULL)
 {
 puts("Couldn't allocate memory for transmit buffer, exiting.");
 goto Latency_End_Cleanup;
 }

 /* Set the first byte of the transmit buffer to 0xfe. This is the */
 /* byte that will be at the front of the packet at the destination */
 /* (if path addressing is used) */
 if (packetSize > 0)
 {
 pTransmitBuffer[0U] = 0xfeU;
 }

 /* Allocate memory for packet pointers */
 ppTxPackets = (STAR_STREAM_ITEM **)calloc(testCount,
 sizeof(STAR_STREAM_ITEM *));
 if (ppTxPackets == NULL)
 {
 puts("Couldn't allocate memory for transmit packets, exiting.");
 goto Latency_End_Cleanup;
 }

 /* Allocate memory for the transmit identifiers */
 ppTxTransOp = (STAR_TRANSFER_OPERATION **)calloc(testCount,
 sizeof(STAR_TRANSFER_OPERATION *));
 if (ppTxTransOp == NULL)
 {
 puts("Couldn't allocate memory for transmit operations, exiting.");
 goto Latency_End_Cleanup;
 }

 for (i = 0; i < testCount; i++)
 {
 /* Create a packet to be transferred */
 ppTxPackets[i] = STAR_createPacket(pAddressPaths[i],
 pTransmitBuffer, packetSize, STAR_EOP_TYPE_EOP);
 if (ppTxPackets[i] == NULL)
 {
 puts("Couldn't create the packet to be transferred, exiting.");
 goto Latency_End_Cleanup;
 }

 /* Create a transfer operation to transmit the packet */
 ppTxTransOp[i] = STAR_createTxOperation(&ppTxPackets[i], 1);
 if (ppTxTransOp[i] == NULL)
 {
 puts("Couldn't create transmit operation, exiting.");
 goto Latency_End_Cleanup;
 }
 }
}

/* Start the clock */

```

```

 start = GET_TIME();
 STORE_CPU_USAGE(pStartCpuProperties);
}

/* for each loop to perform */
for (i = 0; i < loopNum; i++)
{
 /* if performing transmits */
 if (PERFORMING_TRANSMITS(mode))
 {
 for (j = 0; j < testCount; j++)
 {
 /* Start transmitting the packet */
 STAR_submitTransferOperation(pTxChannels[j], ppTxTransOp[j]);

 status = STAR_getTransferOperationStatus(ppTxTransOp[j]);
 if ((status != STAR_TRANSFER_STATUS_STARTED) &&
 (status != STAR_TRANSFER_STATUS_COMPLETE))
 {
 printf("Could not perform transmit in loop %d, error of %d\n",
 i, status);
 goto Latency_End_Cleanup;
 }
 }

 /* If performing receives */
 if (PERFORMING_RECEIVES(mode))
 {
 /* Start receiving the next packet */
 for (j = 0; j < testCount; j++)
 {
 STAR_submitTransferOperation(pRxChannels[j], ppRxTransOp[j]);
 }

 /* Wait on packets being received */
 for (j = 0; j < testCount; j++)
 {
 status = STAR_waitOnTransferOperationCompletion(
 ppRxTransOp[j],
 STAR_INFINITE);
 if (status != STAR_TRANSFER_STATUS_COMPLETE)
 {
 printf("Could not receive packet in loop %d, error of %d\n",
 i, status);
 goto Latency_End_Cleanup;
 }
 }

 /* If this is the first receive and not performing transmits */
 if ((i == 0) && (!PERFORMING_TRANSMITS(mode)))
 {
 /* Start the clock */
 start = GET_TIME();
 STORE_CPU_USAGE(pStartCpuProperties);
 }
 }

 /* If performing transmits but not performing receives */
 if (PERFORMING_TRANSMITS(mode) && (!PERFORMING_RECEIVES(mode)))
 {
 /* Wait on the transmit completing */
 for (j = 0; j < testCount; j++)
 {
 status = STAR_waitOnTransferOperationCompletion(
 ppTxTransOp[j],
 STAR_INFINITE);
 if (status != STAR_TRANSFER_STATUS_COMPLETE)
 {
 printf("Could not complete transmit in loop %d, error of %d\n",
 i, status);
 goto Latency_End_Cleanup;
 }
 }
 }
 }
}

/* Stop the clock */
finish = GET_TIME();
STORE_CPU_USAGE(pEndCpuProperties);

duration = (double)(finish - start) / TIME_DIVIDER;
bitsSent = (double)packetSize * (double)testCount;
if (pAddressPaths != NULL)
{
 for (i = 0; i < testCount; i++)

```

```

 {
 if (pAddressPaths[i] != NULL)
 {
 bitsSent += STAR_getSpaceWireAddressPathLength(
 pAddressPaths[i]);
 }
 }
 }
 bitsSent = bitsSent * (double)loopNum * (double)8;

 /* Calculate the data and packet rate */
 if (duration > 0.0)
 {
 *pDataRate = (bitsSent / duration) / 1000000.0;
 *pPacketRate = (double)(loopNum * testCount) / duration;
 }
 else
 {
 *pDataRate = 0.0;
 *pPacketRate = 0.0;
 }
 *pPacketTime = (duration * 1000000.0) / (double)(loopNum * testCount);

/* Clean-up */
Latency_End_Cleanup:

/* If performing receives */
if (PERFORMING_RECEIVES(mode))
{
 /* Dispose of the receive operations */
 if (ppRxTransOp != NULL)
 {
 for (j = 0; j < testCount; j++)
 {
 STAR_disposeTransferOperation(ppRxTransOp[j]);
 }

 free(ppRxTransOp);
 }
}

/* If performing transmits */
if (PERFORMING_TRANSMITS(mode))
{
 /* Dispose of the transmit operations */
 if (ppTxTransOp != NULL)
 {
 for (j = 0; j < testCount; j++)
 {
 STAR_disposeTransferOperation(ppTxTransOp[j]);
 }
 }

 /* Destroy the transmit packet */
 if (ppTxPackets != NULL)
 {
 for (j = 0; j < testCount; j++)
 {
 STAR_destroyStreamItem(ppTxPackets[j]);
 }

 /* Free the packet pointer array*/
 free(ppTxPackets);
 }

 /* Free the transfer operation pointers */
 if (ppTxTransOp != NULL)
 {
 free(ppTxTransOp);
 }

 /* Free the transmit buffer */
 if (pTransmitBuffer != NULL)
 {
 free(pTransmitBuffer);
 }
}

/* In the event of an error */
if (status != STAR_TRANSFER_STATUS_COMPLETE)
{
 *pDataRate = 0.0;
 *pPacketRate = 0.0;
 *pPacketTime = 0.0;
}
}

```

```

static void performLatencyTest(const TEST_PROPERTIES * const pTestProperties,
 FILE * const pOutputFile)
{
 int packetSize;
 double dataRate = 0, packetRate = 0, packetTime = 0;
 CPU_USAGE_PROPERTIES startCpuProperties, endCpuProperties;

 /* If a file is to be written to */
 if (pOutputFile != NULL)
 {
 /* Write the statistics column headers to the file */
 fprintf(pOutputFile, "Packet Size\tData Rate (Mbit/s)\t");
 fprintf(pOutputFile,
 "Packet Rate (packets/s)\tAverage Packet Time (microseconds)");
 PRINT_CPU_USAGE_HEADERS(pOutputFile);
 }

 /* for each packet size to test */
 packetSize = pTestProperties->packetStart;
 while (packetSize <= pTestProperties->packetEnd)
 {
 /* Display the packet size on screen */
 printf("Performing latency test using packets of %d bytes...\n",
 packetSize);

 /* Perform the test for the current packet size */
 performLatencyTestForPacketSize(packetSize, pTestProperties->mode,
 pTestProperties->testCount, pTestProperties->pTxChannelIds,
 pTestProperties->pAddressPaths, pTestProperties->pRxChannelIds,
 &dataRate, &packetRate, &packetTime, &startCpuProperties,
 &endCpuProperties);

 /* Display the results of the test */
 printf(
 "\t%-3.2f Mbit/s %-3.2f packets/s %-3.2f microseconds/packet\n",
 dataRate, packetRate, packetTime);
 PRINT_CPU_USAGE(NULL, &startCpuProperties, &endCpuProperties);

 /* If writing the results to file */
 if (pOutputFile != NULL)
 {
 /* Write the results to the file */
 fprintf(pOutputFile, "%8d\t%-3.2f\t%-3.2f\t%-3.2f", packetSize,
 dataRate, packetRate, packetTime);
 PRINT_CPU_USAGE(pOutputFile, &startCpuProperties,
 &endCpuProperties);
 fflush(pOutputFile);
 }

 /* if not performing receives and this isn't the last loop */
 if ((!PERFORMING_RECEIVES(pTestProperties->mode)) &&
 (packetSize < pTestProperties->packetEnd))
 {
 /* Sleep for 5 seconds to allow all packets to be sent */
 SLEEP(5000U);
 }

 /* Move to the next packet size */
 if (packetSize == pTestProperties->packetEnd)
 {
 packetSize++;
 }
 else if (pTestProperties->packetStep == 0)
 {
 packetSize = MIN(2 * packetSize, pTestProperties->packetEnd);
 }
 else
 {
 packetSize = MIN(packetSize + pTestProperties->packetStep,
 pTestProperties->packetEnd);
 }
 }
}

static int readArguments(TEST_PROPERTIES * const pTestProperties,
 const int argCount, const char * const arguments[])
{
 int processedArguments = 1;

 /* While not reached the end of the arguments */
 while (processedArguments < argCount)
 {
 /* If the argument begins with a hyphen */
 if ((strlen(arguments[processedArguments]) > 1U) &&

```

```

 (arguments[processedArguments][0U] == '-')
 {
 /* If the argument is to use channel 0, enable its use */
 if (strcasecmp(arguments[processedArguments], "-usechannel0") == 0)
 {
 pTestProperties->useChannel0 = 1;
 }
 /* Otherwise this argument isn't supported, so display a warning */
 else
 {
 printf("Unsupported argument \"%s\" ignored\n",
 arguments[processedArguments]);
 }
 processedArguments++;
 }
 else
 {
 break;
 }
}

return processedArguments;
}

int main(int argc, char* argv[])
{
 TEST_PROPERTIES testProperties = { 0 };
 FILE *pOutputFile = NULL;
 int processedArguments;

 STAR_setApplicationName("STAR-System Performance Tester Application");

 /* Read in any arguments */
 processedArguments = readArguments(&testProperties, argc,
 (const char * const *)argv);

 /* Read the test parameters */
 if (readTestParameters(&testProperties, argc - processedArguments,
 (const char * const *) (argv + processedArguments)) == 0)
 {
 cleanupTest(&testProperties);
 return 0;
 }

 /* if writing the results to file */
 if (strlen(testProperties.pFilePath) > 0U)
 {
 /* Open the file */
 pOutputFile = fopen(testProperties.pFilePath, "wt");
 if (pOutputFile == NULL)
 {
 puts("Couldn't open the file to store the statistics, exiting");
 cleanupTest(&testProperties);
 return 0;
 }

 /* Write the description to the file */
 fprintf(pOutputFile, "%s\n\n", testProperties.pFileDescription);
 }

 /* Test is beginning, so prevent system sleeping */
 DISABLE_SYSTEM_SLEEP();

 switch (testProperties.testType)
 {
 case RATE_TEST:
 performRateTest(&testProperties, pOutputFile);
 break;

 case RANDOM_TEST:
 performRandomTest(&testProperties, pOutputFile);
 break;

 case LATENCY_TEST:
 performLatencyTest(&testProperties, pOutputFile);
 break;

 default:
 puts("Unexpected test type encountered, exiting");
 break;
 }

 ENABLE_SYSTEM_SLEEP();

 /* if writing the results to file */
 if (pOutputFile != NULL)

```

```

 {
 /* Close the file */
 fclose(pOutputFile);
 }

 /* Clean-up the test by freeing any memory and closing any open channels */
 cleanupTest(&testProperties);

 return 0;
}

```

## 9.38 time-code\_example.c

Demonstrates sending a time-code using the advanced transmit operations.

```

#include "examples.h"

#include "general.h"
#include "utilities.h"

#include "star-dundee_types.h"
#include "star-api.h"

void timecodeExample()
{
 /* Get first device */
 STAR_DEVICE_ID firstDevice = getFirstDevice();

 /* If there is a first device */
 if(firstDevice)
 {
 /* Open send channel */
 STAR_CHANNEL_ID sendChannel =
 STAR_openChannelToLocalDevice(
 firstDevice, STAR_CHANNEL_DIRECTION_OUT, CHANNEL0, 1);

 /* If send channel was opened */
 if(sendChannel)
 {
 /* Define transfer status for transfer operation status updates */
 STAR_TRANSFER_STATUS status;

 /* Define transfer operation for sending packets */
 STAR_TRANSFER_OPERATION * sendTransferOperation;

 /* Initialise time-code value to 1 */
 U8 timecodeValue = 1;

 /* Create time-code */
 STAR_STREAM_ITEM * timecode = STAR_createTimeCode(
 timecodeValue);

 /* If time-code was created */
 if(timecode)
 {
 /* Create transmit operation to send time-code */
 sendTransferOperation = STAR_createTxOperation(&timecode, 1);

 /* Start transmitting the time-code */
 STAR_submitTransferOperation(sendChannel,
 sendTransferOperation);

 /* Wait indefinitely for transfer to complete */
 status = STAR_waitOnTransferOperationCompletion(
 sendTransferOperation, -1);

 /* If time-code was sent successfully */
 if(status == STAR_TRANSFER_STATUS_COMPLETE)
 {
 /* Print success message */
 printf("Time-code sent successfully.\n");
 }

 /* Dispose of transfer operation */
 STAR_disposeTransferOperation(sendTransferOperation);

 /* Destroy time-code */
 STAR_destroyStreamItem(timecode);
 }
 }
 }
}

```

```

 /* Close send channel */
 STAR_closeChannel(sendChannel);
 }
}

```

## 9.39 time-code\_test.c

The STAR-System Time-code Test application is a command line program that can be used to configure the time-code properties of a device, and to transmit and receive time-codes. The source code also provides useful examples for performing these tasks.

```

#include <stdio.h>
#include <stdlib.h>

#include "star-api.h"
#include "cfg_api_generic.h"

#if defined(_WIN32)

 #include <process.h>
 #include <windows.h>

 #define STAR_THREAD HANDLE
 #define STAR_THREAD_RETURN_TYPE unsigned int WINAPI
 #define STAR_THREAD_RETURN_VAL 0

 #define STAR_CreateThread(thread, func, arg) \
 ((thread = (HANDLE)_beginthreadex(NULL, 0, func, arg, 0, NULL)) != 0)

 #define STAR_CloseThread(thread) CloseHandle(thread)

 #define STAR_WaitForThreadCompletion(thread) \
 (WaitForSingleObject(thread, INFINITE) == WAIT_OBJECT_0)

#else

 #include <pthread.h>
 #include <unistd.h>

 #define STAR_THREAD pthread_t
 #define STAR_THREAD_RETURN_TYPE void *
 #define STAR_THREAD_RETURN_VAL NULL

 #define STAR_CreateThread(thread, func, arg) \
 (!pthread_create(&(thread), NULL, func, arg))

 #define STAR_CloseThread(thread) \
 ((thread) ? (!pthread_detach(thread)) : 1)

 #define STAR_WaitForThreadCompletion(thread) \
 (!!pthread_join(thread, NULL) && (!(thread) = 0)))

#endif

#if !defined(UNREFERENCED_PARAMETER)
 #define UNREFERENCED_PARAMETER(a) ((void)(a))
#endif

U8 g_currentTimecode = 0;

STAR_THREAD g_receiveThread = 0;

int g_receiveThreadRunning = 0;

STAR_CHANNEL_ID g_receiveChannel = 0;

static void transmitTimecode(STAR_DEVICE_ID deviceId)
{
 /* Open channel 0 to transmit */
 /* Note that STAR-System devices currently only support transmitting and */
 /* receiving time-codes on channel 0 */
 STAR_CHANNEL_ID txChannel = STAR_openChannelToLocalDevice(

```

```

 deviceId,
 STAR_CHANNEL_DIRECTION_OUT, 0, 0);

/* If transmit channel was opened */
if(txChannel)
{
 STAR_TRANSFER_STATUS status;
 STAR_TRANSFER_OPERATION * transferOperation;

 /* Create a time-code */
 STAR_STREAM_ITEM *timecode = STAR_createTimeCode(
g_currentTimecode);

 /* If time-code was created */
 if (timecode)
 {
 /* Create transmit operation to transmit time-code */
 transferOperation = STAR_createTxOperation(&timecode, 1);

 /* Start transmitting the time-code */
 STAR_submitTransferOperation(txChannel, transferOperation);

 /* Wait indefinitely for transfer to complete */
 status = STAR_waitOnTransferOperationCompletion(
transferOperation,
-1);

 /* If time-code was transmitted successfully */
 if (status == STAR_TRANSFER_STATUS_COMPLETE)
 {
 /* Print success message */
 printf("Time-code %u transmitted successfully.\n",
g_currentTimecode);
 }
 else
 {
 /* Print error message */
 printf("Error transmitting time-code %u, status = %d.\n",
g_currentTimecode, status);
 }

 /* Dispose of transfer operation */
 STAR_disposeTransferOperation(transferOperation);

 /* Destroy time-code */
 STAR_destroyStreamItem(timecode);

 /* Move to the next time-code value */
 g_currentTimecode++;
 if (g_currentTimecode == 64)
 {
 g_currentTimecode = 0;
 }
 }

 /* Close transmit channel */
 STAR_closeChannel(txChannel);
}

static void setTimecodePeriod(STAR_DEVICE_ID deviceId)
{
 char s[256];

 /* Display the current time-code period */
 U32 period;
 if (CFG_SUCCESS(CFG_getTimeCodePeriod(deviceId, &period)))
 {
 printf("Current time-code period: %u microseconds\n", period);
 }
 else
 {
 puts("Unable to read the current time-code period");
 }

 /* Read in the time-code period to be set */
 printf("Please enter the time-code period in microseconds: ");
 fflush(stdout);
 if (!fgets(s, 256, stdin))
 {
 puts("No period specified.");
 }
 else
 {
 int status = sscanf(s, "%u", &period);
 if (!status)

```



```

 {
 puts("Invalid time-code period specified.");
 }
 else
 {
 /* Set the time-code period */
 if (CFG_SUCCESS(CFG_setTimeCodePeriod(deviceId, period)))
 {
 printf("Time-code period set to %u\n", period);
 }
 else
 {
 puts("Error setting time-code period");
 }
 }
 }
}

static void enableTimecodeMaster(STAR_DEVICE_ID deviceId)
{
 /* Enable the device as a time-code master */
 if (CFG_SUCCESS(CFG_enableTimeCodeMaster(deviceId)))
 {
 puts("Device enabled as a time-code master");
 }
 else
 {
 puts("Error enabling device as a time-code master");
 }
}

static void disableTimecodeMaster(STAR_DEVICE_ID deviceId)
{
 /* Disable the device as a time-code master */
 if (CFG_SUCCESS(CFG_disableTimeCodeMaster(deviceId)))
 {
 puts("Device disabled as a time-code master");
 }
 else
 {
 puts("Error disabling device as a time-code master");
 }
}

STAR_THREAD_RETURN_TYPE timecodeReceiveThread(void *pArg)
{
 STAR_TRANSFER_OPERATION *pRxTransferOp;

 /* Get the device ID from the argument passed to the thread */
 STAR_DEVICE_ID deviceId = *(STAR_DEVICE_ID *)pArg;
 free(pArg);

 /* Open channel 0 to receive */
 /* Note that STAR-System devices currently only support transmitting and */
 /* receiving time-codes on channel 0 */
 g_receiveChannel = STAR_openChannelToLocalDevice(deviceId,
 STAR_CHANNEL_DIRECTION_IN, 0, 1);
 if (!g_receiveChannel)
 {
 puts("Unable to open channel to receive time-codes");
 goto timecodeReceiveThread_openChannelFail;
 }

 /* Create a receive operation to receive 1 time-code */
 pRxTransferOp = STAR_createRxOperation(1,
 STAR_RECEIVE_TIMECODES);
 if (!pRxTransferOp)
 {
 puts("Unable to create receive operation to receive a time-code");
 goto timecodeReceiveThread_createRxOperationFail;
 }

 /* While the thread has not been stopped */
 while (g_receiveThreadRunning)
 {
 STAR_TRANSFER_STATUS rxStatus;
 STAR_STREAM_ITEM *pRxStreamItem;

 /* Submit the receive operation */
 if (!STAR_submitTransferOperation(g_receiveChannel, pRxTransferOp))
 {
 if (g_receiveThreadRunning)
 {
 puts("Unable to submit receive operation to receive a time-code");
 }
 }
 }
}

```

```

 }
 break;
 }

 /* Wait indefinitely on the receive operation completing */
 rxStatus = STAR_waitOnTransferOperationCompletion(
pRxTransferOp, -1);
 if (rxStatus != STAR_TRANSFER_STATUS_COMPLETE)
 {
 if (g_receiveThreadRunning)
 {
 printf("Error waiting for receive operation to receive a time-code, status = %d\n",
rxStatus);
 }
 break;
 }

 /* Get the received time-code */
 pRxStreamItem = STAR_getTransferItem(pRxTransferOp, 0);
 if ((!pRxStreamItem) || (pRxStreamItem->itemType !=
 STAR_STREAM_ITEM_TYPE_TIMECODE) || (!pRxStreamItem->
item))
 {
 if (g_receiveThreadRunning)
 {
 puts("Error getting receive stream item");
 }
 break;
 }

 /* Display the received time-code */
 printf("\t\tReceived time-code with value: %2u\n",
 STAR_getTimeCodeValue((STAR_TIMECODE *)pRxStreamItem->
item));
}

/* Dispose the receive operation */
STAR_disposeTransferOperation(pRxTransferOp);

timecodeReceiveThread_createRxOperationFail:
/* Close the receive channel */
STAR_closeChannel(g_receiveChannel);

timecodeReceiveThread_openChannelFail:
g_receiveThreadRunning = 0;

 return STAR_THREAD_RETURN_VAL;
}

static void startReceivingTimecodes(STAR_DEVICE_ID deviceId)
{
 /* Check the thread isn't already running */
 if (g_receiveThread)
 {
 puts("The receive thread is already running");
 }
 else
 {
 /* Create the argument to be passed to the thread */
 STAR_DEVICE_ID *pDeviceId;
 pDeviceId = (STAR_DEVICE_ID *)calloc(1, sizeof(
STAR_DEVICE_ID));
 if (!pDeviceId)
 {
 puts("Unable to create argument for thread to receive time-codes");
 }
 else
 {
 *pDeviceId = deviceId;

 /* Start a thread to receive time-codes */
 g_receiveThreadRunning = 1;
 if (!STAR_CreateThread(g_receiveThread, timecodeReceiveThread,
 pDeviceId))
 {
 puts("Unable to create thread to receive time-codes");
 g_receiveThreadRunning = 0;
 g_receiveThread = 0;
 }
 }
 }
}

static void stopReceivingTimecodes()
{

```

```

 if (!g_receiveThread)
 {
 puts("The receive thread is not currently running");
 }
 else
 {
 /* Close the receive channel to stop the thread receiving time-codes */
 g_receiveThreadRunning = 0;
 STAR_closeChannel(g_receiveChannel);

 /* Wait on the thread completing */
 if (!STAR_WaitForThreadCompletion(g_receiveThread))
 {
 puts("Error waiting for the receive thread to complete");
 }

 /* Close the thread */
 (void)STAR_CloseThread(g_receiveThread);
 g_receiveThread = 0;
 }
}

static void enableExternalTimecodeSelection(STAR_DEVICE_ID deviceId)
{
 /* Enable external time-codes selection for the device */
 if (CFG_SUCCESS(CFG_enableExternalTimeCodeSelection(
 deviceId)))
 {
 puts("External time-code selection enabled for the device");
 }
 else
 {
 puts("Error enabling external time-code selection for the device");
 }
}

static void disableExternalTimecodeSelection(STAR_DEVICE_ID deviceId)
{
 /* Disable external time-codes selection for the device */
 if (CFG_SUCCESS(CFG_disableExternalTimeCodeSelection(
 deviceId)))
 {
 puts("External time-code selection disabled for the device");
 }
 else
 {
 puts("Error disabling external time-code selection for the device");
 }
}

static STAR_DEVICE_ID displayDeviceList(void)
{
 U32 deviceCount, deviceNum;
 char *deviceName, s[256];
 unsigned int chosenDevice;
 int status;

 /* Get the list of configurable devices */
 STAR_DEVICE_ID *devices = STAR_getDeviceListForType(
 STAR_DEVICE_CONFIG_SUPPORTED, &deviceCount);
 STAR_DEVICE_ID deviceId = 0;

 /* If there are devices present */
 if (devices)
 {
 if (!deviceCount)
 {
 puts("No devices present.\n");
 }
 else if (deviceCount == 1)
 {
 deviceName = STAR_getDeviceName(devices[0]);
 if (deviceName)
 {
 printf("One device present: %s\n", deviceName);
 STAR_destroyString(deviceName);
 }
 else
 {
 puts("One device present: Unable to determine device name");
 }
 deviceId = devices[0];
 }
 else
 }
}

```

```

 {
 puts("Select device:");

 /* For each device in the list */
 for (deviceNum = 0; deviceNum < deviceCount; deviceNum++)
 {
 /* Display its name */
 deviceName = STAR_getDeviceName(devices[deviceNum]);
 if (deviceName)
 {
 printf("%u: %s\n", deviceNum, deviceName);
 STAR_destroyString(deviceName);
 }
 else
 {
 printf("%u: Unable to determine device name\n", deviceNum);
 }
 }

 /* Get the device to use */
 if (!fgets(s, 256, stdin))
 {
 puts("No device number selected.");
 }
 else
 {
 status = sscanf(s, "%u", &chosenDevice);
 if ((!status) || (chosenDevice > deviceCount - 1))
 {
 puts("Incorrect device number selected.");
 }
 else
 {
 deviceId = devices[chosenDevice];
 }
 }
 }

 /* Destroy the device list */
 STAR_destroyDeviceList(devices);
}
else
{
 puts("No devices present.\n");
}

return deviceId;
}

int __cdecl main(int argc, char *argv[])
{
 char exitSelected = 0;
 char s[256];
 STAR_DEVICE_ID deviceId;

 UNREFERENCED_PARAMETER(argc);
 UNREFERENCED_PARAMETER(argv);

 STAR_setApplicationName("STAR-System Time-code Test Application");

 /* Get the device to be used */
 deviceId = displayDeviceList();
 if (!deviceId)
 {
 return 0;
 }

 /* Enable time-code forwarding on all ports of the device */
 if (!CFG_SUCCESS(CFG_setTimeCodeDistributionPorts(deviceId,
 0x00000fff)))
 {
 puts("Unable to enable time-code distribution on all ports");
 }

 do
 {
 /* Display Menu */
 puts("\n");
 puts("Select option:");
 puts(" 0: Exit");
 puts(" 1: Transmit a time-code");
 puts(" 2: Set period of time-code master (cannot be done while receiving)");
 puts(" 3: Enable time-code master (cannot be done while receiving)");
 puts(" 4: Disable time-code master (cannot be done while receiving)");
 puts(" 5: Start receiving time-codes");
 puts(" 6: Stop receiving time-codes");
 }
}

```

```

puts(" 7: Enable external time-code selection (cannot be done while receiving)");
puts(" 8: Disable external time-code selection (cannot be done while receiving)");
puts("");

/* Read in the operation type */
if (!fgets(s, 256, stdin))
{
 puts("Error reading input, exiting");
 exitSelected = 1;
}
else
{
 puts("");
 switch (s[0])
 {
 case '0':
 exitSelected = 1;
 break;

 case '1':
 transmitTimecode(deviceId);
 break;

 case '2':
 setTimecodePeriod(deviceId);
 break;

 case '3':
 enableTimecodeMaster(deviceId);
 break;

 case '4':
 disableTimecodeMaster(deviceId);
 break;

 case '5':
 startReceivingTimecodes(deviceId);
 break;

 case '6':
 stopReceivingTimecodes();
 break;

 case '7':
 enableExternalTimecodeSelection(deviceId);
 break;

 case '8':
 disableExternalTimecodeSelection(deviceId);
 break;

 default:
 printf("Invalid menu option selected: %c\n", s[0]);
 }
}
while (!exitSelected);

puts("Exiting");

return 0;
}

```

## 9.40 timestamp\_test.c

This file contains example code demonstrating how to use the timestamping API for the Brick Mk3.

```

#include "star-api.h"
#include "cfg_api_generic.h"
#include "triggering_brick_mk3.h"

#include <stdio.h>

#ifdef _WIN32
 #include "windows.h"

 #define SLEEP(milliseconds) Sleep(milliseconds);
#else
 #include <unistd.h>

 #define SLEEP(milliseconds) usleep(milliseconds * 1000);

```

```

#endif

#if !defined(UNREFERENCED_PARAMETER)
#define UNREFERENCED_PARAMETER(a) ((void) (a))
#endif

/* Comment this out to use external trigger in example. */
#define GLOBAL_PULSE_GENERATOR

/* Uncomment this to trigger on falling edge of external trigger */
// #define EXTERNAL_TRIGGER_FALLING_EDGE

#ifdef GLOBAL_PULSE_GENERATOR

/* Set sync pulse frequency to 1Hz (global pulse generator supports 1Hz,
 10Hz, 100Hz or 1000Hz). */
#define SYNC_PULSE_FREQUENCY 1 /* 1Hz */

#else

/* Set sync pulse frequency to match external trigger pulse generation
 frequency. */
#define SYNC_PULSE_FREQUENCY 1 /* 1Hz */

#endif

/* Define transmit and receive links */
#define TRANSMIT_LINK 1
#define RECEIVE_LINK 2

STAR_DEVICE_ID getDeviceId()
{
 /* The selected device to use for the example */
 STAR_DEVICE_ID selectedDevice = 0;

 /* Get all Brick Mk3 devices */
 U32 deviceCount = 0;
 STAR_DEVICE_ID *pDeviceList = STAR_getDeviceListForType(
 STAR_DEVICE_BRICK_MK3, &deviceCount);

 /* If at least one device */
 if(deviceCount > 0)
 {
 U32 x;

 /* For all devices */
 for(x = 0; x < deviceCount; x++)
 {
 /* Get the current device */
 STAR_DEVICE_ID currentDevice = pDeviceList[x];

 /* Get hardware info for current device */
 STAR_CFG_FPGA_INFO hardwareInfo;
 CFG_getFPGAInfo(currentDevice, &hardwareInfo);

 /* If Brick Mk3 is v1.02 or later */
 if(hardwareInfo.major > 1 ||
 (hardwareInfo.major == 1 && hardwareInfo.minor >= 2))
 {
 /* Use current device for test */
 selectedDevice = currentDevice;
 break;
 }
 }

 /* Destroy the device list */
 STAR_destroyDeviceList(pDeviceList);

 /* Return the selected device */
 return selectedDevice;
 }
}

void printPacket(unsigned char *pPacketData, unsigned int packetLength)
{
 /* Print packet contents */
 unsigned int x;
 for(x = 0; x < packetLength; x++)
 {
 printf("%02X ", pPacketData[x]);
 }
}

void transmitPackets(STAR_CHANNEL_ID channel, unsigned int transmitPacketCount)
{
 U8 buffer[32];
 unsigned int x;

```

```

STAR_STREAM_ITEM *pPacket;
STAR_TRANSFER_OPERATION *pTransmitOperation;

/* For all packets to be transmitted */
for(x = 1; x <= transmitPacketCount; x++)
{
 /* Populate packet buffer */
 memset(buffer, 0xAB, 32);
 memset(buffer, x, 1);

 /* Create packet from buffer */
 pPacket = STAR_createPacket(NULL, buffer, 32,
STAR_EOP_TYPE_EOP);

 /* Create transmit operation from packet */
 pTransmitOperation = STAR_createTxOperation(&pPacket, 1);

 /* Transmit the packet */
 STAR_submitTransferOperation(channel, pTransmitOperation);

 /* Wait for transmit operation to complete */
 STAR_waitOnTransferOperationCompletion(pTransmitOperation, -1
);

 /* Cleanup resources required for transmit operation */
 STAR_disposeTransferOperation(pTransmitOperation);
 STAR_destroyStreamItem(pPacket);

 /* Print packet transmit status */
 printf("Packet %d transmitted : ", x);
 printPacket(buffer, 32);
 printf("\n");

 /* Sleep for 100 milliseconds */
 SLEEP(100);
}
}

double convertToDouble(U32 seconds, U32 nanoseconds)
{
 /* Convert value to double */
 double doubleValue = (double)seconds + ((double)nanoseconds / 1E9);

 /* Return value as double */
 return doubleValue;
}

void receivePacketsAndTimestamps(STAR_CHANNEL_ID channel,
 unsigned int receiveCount, U32 syncPulseFrequency)
{
 STAR_TRANSFER_OPERATION *pReceiveOperation;
 unsigned int x;
 unsigned int actualReceiveCount;

 /* Create receive operation for packets and timestamp events */
 pReceiveOperation = STAR_createRxOperation(receiveCount,
STAR_RECEIVE_PACKETS | STAR_RECEIVE_TIMESTAMP_EVENTS
);

 /* Submit the receive operation */
 STAR_submitTransferOperation(channel, pReceiveOperation);

 /* Wait for receive operation to complete */
 STAR_waitOnTransferOperationCompletion(pReceiveOperation, -1);

 /* Get the number of stream items that were received */
 actualReceiveCount = STAR_getTransferItemCount(pReceiveOperation);

 /* For all stream items that were received */
 for(x = 0; x < actualReceiveCount; x++)
 {
 /* Get current stream item */
 STAR_STREAM_ITEM *pReceivedItem =
 STAR_getTransferItem(pReceiveOperation, x);

 /* If current stream item is a packet */
 if(pReceivedItem->itemType ==
STAR_STREAM_ITEM_TYPE_SPACEWIRE_PACKET)
 {
 unsigned char *pPacketData;
 unsigned int packetLength;

 /* Get the packet data */
 STAR_SPACEWIRE_PACKET *pPacket =
 (STAR_SPACEWIRE_PACKET *)pReceivedItem->
item;
 pPacketData = STAR_getPacketData(pPacket, &packetLength);

```

```

 /* Print the packet data */
 printf("Packet %d received : ", pPacketData[0]);
 printPacket(pPacketData, packetLength);
 printf("\n");

 /* Destroy the packet data now that it has been used */
 STAR_destroyPacketData(pPacketData);
 }
 /* Else if current stream item is a timestamp event */
 else if(pReceivedItem->itemType ==
 STAR_STREAM_ITEM_TYPE_TIMESTAMP_EVENT)
 {
 U32 startSeconds;
 U32 startRemainder;
 U32 endSeconds;
 U32 endRemainder;

 /* Get the timestamp data */
 STAR_TIMESTAMP_EVENT *pTimestampEvent =
 (STAR_TIMESTAMP_EVENT *)pReceivedItem->item;

 /* Get start and end timestamp values in seconds and a remainder of
 nanoseconds */
 STAR_getTimestampEventStartValue(pTimestampEvent,
 syncPulseFrequency, &startSeconds, &startRemainder);
 STAR_getTimestampEventEndValue(pTimestampEvent,
 syncPulseFrequency, &endSeconds, &endRemainder);

 /* Print the start and end of packet timestamps */
 printf("Start of packet timestamp : %f seconds.\n",
 convertToDouble(startSeconds, startRemainder));
 printf("End of packet timestamp : %f seconds.\n\n",
 convertToDouble(endSeconds, endRemainder));
 }
}

/* Cleanup receive operation */
STAR_disposeTransferOperation(pReceiveOperation);
}

void enableExternalSyncPulse(STAR_DEVICE_ID deviceId)
{
 /* The following code uses the STAR-System triggering API to generate a
 pulse every second. An SMB lead should be connected between triggers
 A and B on the Brick Mk3. */

 /* Set timer 0 reload value to 1 second */
 TRIGGER_BRICK_MK3_setCounterReloadValue(deviceId, 0, 60000000);

 /* Enable timer auto reload */
 TRIGGER_BRICK_MK3_enableCounterAutoReload(deviceId, 0);

 /* COUNTER_EVENT_RELOAD event on timer 0 causes internal trigger 0 to
 be set */
 TRIGGER_BRICK_MK3_setCounterInputEvents(deviceId, 0, 0,
 COUNTER_EVENT_RELOAD);

 /* Internal trigger 0 causes TIME_CODE_ACTION_TX action on time-code
 engine 0 */
 TRIGGER_BRICK_MK3_enableExtTriggerOutput(deviceId, 1);
 TRIGGER_BRICK_MK3_setExtTriggerExtend(deviceId, 1, 50);
 TRIGGER_BRICK_MK3_setExtTriggerOutputActions(deviceId, 1, 0
 , EXT_TRIGGER_ACTION_OUT);

 /* Enable internal trigger 0 */
 TRIGGER_BRICK_MK3_enableTrigger(deviceId, 0);
}

void disableExternalSyncPulse(STAR_DEVICE_ID deviceId)
{
 /* Disable the constant pulse on the external trigger */
 TRIGGER_BRICK_MK3_disableExtTriggerOutput(deviceId, 1);
 TRIGGER_BRICK_MK3_disableTrigger(deviceId, 0);
}

void timestampTest(STAR_DEVICE_ID deviceId)
{
 STAR_CHANNEL_ID transmitChannel;
 STAR_CHANNEL_ID receiveChannel;

 /* Transmit 10 packets */
 unsigned int transmitPacketCount = 10;

 printf("Setting up timestamp system...\n");

```



```

/* Ensure that timestamp counters are disabled before configuring */
CFG_setTimestampMethod(deviceId,
 STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD_NONE);

/* Set initial timestamp value of 10 seconds */
CFG_setTimestampValue(deviceId, 10);

/* Enable timestamp events on port 2 */
CFG_enableRxTimestampEventsOnPort(deviceId, RECEIVE_LINK);

#ifdef GLOBAL_PULSE_GENERATOR

#if SYNC_PULSE_FREQUENCY == 1

 /* Set pulse generator frequency to 1Hz */
 CFG_setPulseGeneratorFrequency(deviceId,
 STAR_CFG_BRICK_MK3_PULSE_FREQ_1);

#elif SYNC_PULSE_FREQUENCY == 10

 /* Set pulse generator frequency to 10Hz */
 CFG_setPulseGeneratorFrequency(deviceId,
 STAR_CFG_BRICK_MK3_PULSE_FREQ_10);

#elif SYNC_PULSE_FREQUENCY == 100

 /* Set pulse generator frequency to 100Hz */
 CFG_setPulseGeneratorFrequency(deviceId,
 STAR_CFG_BRICK_MK3_PULSE_FREQ_100);

#elif SYNC_PULSE_FREQUENCY == 1000

 /* Set pulse generator frequency to 1kHz */
 CFG_setPulseGeneratorFrequency(deviceId,
 STAR_CFG_BRICK_MK3_PULSE_FREQ_1000);

#else

 /* Print error when unexpected global pulse frequency is detected */
 printf("ERROR: Unexpected global pulse frequency of %dHz (valid values are: 1Hz, 10Hz, 100Hz, 1000Hz).\n");
 return;

#endif

 /* Enable global pulse generator */
 CFG_setTimestampMethod(deviceId,
 STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD_PULSE_GENERATOR
);

#else

 /* Enable external trigger pulse detection */
 CFG_setTimestampMethod(deviceId,
 STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD_EXTERNAL_TRIGGER
);

 /* Enable the external sync pulse before transmitting packets */
 enableExternalSyncPulse(deviceId);

#endif

 /* Create transmit and receive channels */
 transmitChannel = STAR_openChannelToLocalDevice(deviceId,
 STAR_CHANNEL_DIRECTION_OUT, TRANSMIT_LINK, 1);
 receiveChannel = STAR_openChannelToLocalDevice(deviceId,
 STAR_CHANNEL_DIRECTION_IN, RECEIVE_LINK, 1);

 /* Sleep to allow timer to run before transmitting packets */
 SLEEP(2000);

 /* Transmit a group of packets */
 transmitPackets(transmitChannel, transmitPacketCount);

 /* Receive the packets and timestamps */
 receivePacketsAndTimestamps(receiveChannel, transmitPacketCount * 2,
 SYNC_PULSE_FREQUENCY);

#ifdef GLOBAL_PULSE_GENERATOR

 /* Disable the external sync pulse after receiving packets and timestamps */
 disableExternalSyncPulse(deviceId);

#endif

 /* Disable timestamps after receiving packets and timestamps */
 CFG_setTimestampMethod(deviceId,

```

```

 STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD_NONE);

/* Close the channels */
STAR_closeChannel(transmitChannel);
STAR_closeChannel(receiveChannel);
}

int main(int argc, char **argv)
{
 /* Select device to use in examples */
 STAR_DEVICE_ID deviceId = getDeviceId();

 UNREFERENCED_PARAMETER(argc);
 UNREFERENCED_PARAMETER(argv);

 STAR_setApplicationName("STAR-System Timestamp Test Application");

 /* Print the program header */
 printf("----- Timestamping API Example -----\\n\\n");

 /* If compatible device was found */
 if(deviceId)
 {
 STAR_CFG_ROUTER_GLOBAL_STATE globalState;
 PORT_STATUS_CONTROL portStatusControl;
 STAR_CFG_SPW_LINK_STATUS linkStatus;

 /* Get device name and serial */
 char * pDeviceName = STAR_getDeviceName(deviceId);
 char * pDeviceSerial = STAR_getDeviceSerialNumber(deviceId);

 /* Print selected serial */
 printf("%s with serial %s selected.\\n\\n", pDeviceName, pDeviceSerial);

 /* Disable router timeouts so that timestamps are not blocked */
 CFG_getRouterGlobalSettings(deviceId, &globalState);
 globalState.timeoutMode = STAR_CFG_TIMEOUT_MODE_BLOCKING;
 CFG_setRouterGlobalSettings(deviceId, &globalState);

 /* Timestamping requires interface mode to be enabled */
 CFG_enableInterfaceMode(deviceId);

 /* Start the link since watchdog timer is disabled */
 CFG_getPortStatusControl(deviceId, 1, &portStatusControl);
 CFG_getSpaceWireLinkStatus(portStatusControl, &linkStatus);
 linkStatus.start = 1;
 CFG_setSpaceWireLinkStatus(deviceId, 1, &linkStatus);

#ifdef EXTERNAL_TRIGGER_FALLING_EDGE
 /* Trigger on falling edge */
 TRIGGER_BRICK_MK3_enableExtTriggerEdgeDetectMode(
 deviceId, 1);
 TRIGGER_BRICK_MK3_enableExtTriggerInvert(deviceId, 1);
#else
 /* Trigger on rising edge */
 TRIGGER_BRICK_MK3_enableExtTriggerEdgeDetectMode(
 deviceId, 1);
 TRIGGER_BRICK_MK3_disableExtTriggerInvert(deviceId, 1);
#endif

 /* Perform timestamp test */
 timestampTest(deviceId);
 }
 else
 {
 printf("No compatible devices were found.\\n");
 }

 return 0;
}

```

## 9.41 transfer\_callback\_example.c

Creates a simple packet and sends it using the advanced transmit operations. The packet is received using the transfer completion listener function.

```

#include "examples.h"

#include "general.h"
#include "utilities.h"

```

```

#include "star-dundee_types.h"
#include "star-api.h"

/* Define receive channel */
STAR_CHANNEL_ID receiveChannel;

/* Initialise transfer complete to false */
unsigned int transferComplete = 0;

void STAR_API_CC TransferCallback(STAR_TRANSFER_OPERATION *pOperation,
 STAR_TRANSFER_STATUS status, void *pContextInfo)
{
 /* Unreferenced parameters */
 UNREFERENCED_PARAMETER(pContextInfo);

 /* Unregister transfer completion listener */
 STAR_unregisterTransferCompletionListener(pOperation,
 TransferCallback);

 /* If valid stream item was received */
 if(status == STAR_TRANSFER_STATUS_COMPLETE)
 {
 /* Get received stream item */
 STAR_STREAM_ITEM * streamItem = STAR_getTransferItem(pOperation
 , 0);

 /* Initialise data length to 0 */
 unsigned int dataLength = 0;

 /* Get packet data and its length */
 unsigned char * data = STAR_getPacketData((
 STAR_SPACEWIRE_PACKET *)
 streamItem->item, &dataLength);

 if (data != NULL)
 {
 /* Prints the packet contents from the receive buffer */
 printPacketContents(data, dataLength);

 /* Destroy packet data now that it has been output */
 STAR_destroyPacketData(data);
 }
 }

 /* Dispose transfer operation */
 STAR_disposeTransferOperation(pOperation);

 /* Close receive channel */
 STAR_closeChannel(receiveChannel);

 /* Set transfer complete to true */
 transferComplete = 1;
}

void transferCallbackExample()
{
 /* Get first device */
 STAR_DEVICE_ID firstDevice = getFirstDevice();

 /* If there is a first device */
 if(firstDevice)
 {
 /* Open receive channel */
 receiveChannel = STAR_openChannelToLocalDevice(firstDevice,
 STAR_CHANNEL_DIRECTION_IN, CHANNEL1, 1);

 /* If receive channel was opened */
 if(receiveChannel)
 {
 /* Create transfer operation to receive 1 packet */
 STAR_TRANSFER_OPERATION * receiveTransferOperation =
 STAR_createRxOperation(1,
 STAR_RECEIVE_PACKETS);

 /* If transfer operation was created */
 if(receiveTransferOperation)
 {
 /* Register transfer completion listener */
 STAR_registerTransferCompletionListener(
 receiveTransferOperation, TransferCallback, NULL);

 /* Start receiving packet */
 STAR_submitTransferOperation(receiveChannel,
 receiveTransferOperation);

 /* Transfer completion listener is unregistered in callback

```

```

 function */
 }

 /* Receive transfer operation is disposed in callback function */

 /* Receive channel is closed in callback function */
}

/* While transfer isn't complete, keep application alive */
while(!transferComplete)
{
 /* Sleep to relieve the processor */
 sleep(0);
}
}

```

## 9.42 transfer\_operation\_list\_send\_example.c

This example dynamically creates multiple packets and sends them using the `STAR_submitTransferOperationList()` function.

```

#include "examples.h"

#include "general.h"
#include "utilities.h"

#include "star-dundee_types.h"
#include "star-api.h"

#include <stdlib.h>

void transferOperationListSendExample()
{
 /* Get first device */
 STAR_DEVICE_ID firstDevice = getFirstDevice();

 /* If there is a first device */
 if(firstDevice)
 {
 /* Open channel to send out of */
 STAR_CHANNEL_ID channel = STAR_openChannelToLocalDevice(
 firstDevice,
 STAR_CHANNEL_DIRECTION_OUT, CHANNEL1, 1);

 /* If channel was opened */
 if(channel)
 {
 /* Initialise number of packets to 10 */
 unsigned int numberOfPackets = 10;

 /* Create array to hold transfer operations */
 STAR_TRANSFER_OPERATION **transferOperations =
 (STAR_TRANSFER_OPERATION **)malloc(numberOfPackets *
 sizeof(STAR_TRANSFER_OPERATION *));

 /* Create array to hold packets */
 STAR_STREAM_ITEM **packets = (STAR_STREAM_ITEM **)malloc(
 numberOfPackets * sizeof(STAR_STREAM_ITEM *));

 if ((transferOperations != NULL) && (packets != NULL))
 {
 /* Define counter for loop */
 unsigned char index;

 /* For number of packets */
 for (index = 0; index < numberOfPackets; index++)
 {
 /* Create packet data with current index */
 unsigned char packetData[1];
 packetData[0] = index;

 {
 /* Create packet from packet data */
 STAR_STREAM_ITEM * packet =
 STAR_createPacket(NULL,
 packetData, 1, STAR_EOP_TYPE_EOP);

 /* Create send transfer operation */
 STAR_TRANSFER_OPERATION * sendTransferOperation =

```

```

 STAR_createTxOperation(&packet, 1);

 /* Store send transfer operation in array */
 transferOperations[index] = sendTransferOperation;

 /* Store packet so that it can be disposed later */
 packets[index] = packet;
 }

 /* Start transmitting packets */
 STAR_submitTransferOperationList(channel,
 transferOperations,
 numberOfPackets);

 /* For number of packets */
 for (index = 0; index < numberOfPackets; index++)
 {
 /* Get current transfer operation */
 STAR_TRANSFER_OPERATION * transferOperation =
 transferOperations[index];

 /* Get current packet */
 STAR_STREAM_ITEM * packet = packets[index];

 /* Wait for transfer operation to complete */
 STAR_waitOnTransferOperationCompletion(
 transferOperation, -1);

 /* Dispose transfer operation */
 STAR_disposeTransferOperation(transferOperation);

 /* Destroy packet */
 STAR_destroyStreamItem(packet);
 }

 if (packets != NULL)
 {
 /* Free array of packets */
 free(packets);
 }

 if (transferOperations != NULL)
 {
 /* Free array of transfer operations */
 free(transferOperations);
 }

 /* Close channel */
 STAR_closeChannel(channel);
}
}
}

```

## 9.43 transfer\_operation\_list\_send\_receive\_example.c

This example assumes that channel 1 (defined in general header for api\_test\_app project) is connected to link 2 on a SpaceWire USB Brick or other routing device. Any packets sent out of channel 1 will end up coming back to where they started.

### PCI Mk2 Brick

Link 1  Link 2

Link 2 

Link 3 

```

#include "examples.h"

#include "general.h"
#include "utilities.h"

#include "star-dundee_types.h"
#include "star-api.h"

void transferOperationListSendReceiveExample()
{
 /* Get first device */
 STAR_DEVICE_ID firstDevice = getFirstDevice();

```

```

/* If there is a first device */
if(firstDevice)
{
 /* Open channel to send out of */
 STAR_CHANNEL_ID channel = STAR_openChannelToLocalDevice
(firstDevice,
 STAR_CHANNEL_DIRECTION_INOUT, CHANNEL1, 1);

 /* If channel was opened */
 if(channel)
 {
 /* Create address path */
 unsigned char addressPath[] = {2};

 /* Get address path length */
 U16 addressPathLength = sizeof(addressPath) / sizeof(unsigned char);

 /* Create SpaceWire address */
 STAR_SPACEWIRE_ADDRESS * address =
STAR_createAddress(addressPath,
 addressPathLength);

 /* Define packet data to be sent out */
 unsigned char packetData[] = {1, 2, 3, 4, 5, 6, 7, 8, 9};

 /* Get packet length */
 unsigned int packetDataLength = sizeof(packetData) /
sizeof(unsigned char);

 /* Create packet */
 STAR_STREAM_ITEM * packet = STAR_createPacket(address,
packetData,
 packetDataLength, STAR_EOP_TYPE_EOP);

 /* If packet was created */
 if(packet)
 {
 /* Create send transfer operation */
 STAR_TRANSFER_OPERATION * sendTransferOperation =
STAR_createTxOperation(&packet, 1);

 /* Create receive transfer operation */
 STAR_TRANSFER_OPERATION * receiveTransferOperation =
STAR_createRxOperation(1,
STAR_RECEIVE_PACKETS);

 /* If send and receive transfer operations were created */
 if ((sendTransferOperation != NULL) &&
(receiveTransferOperation != NULL))
 {
 /* Define transfer status */
 STAR_TRANSFER_STATUS status;

 /* Put send and receive transfer operations in an array */
 STAR_TRANSFER_OPERATION * transferOperations[2];
transferOperations[0] = sendTransferOperation;
transferOperations[1] = receiveTransferOperation;

 /* Submit transfer operation list */
 STAR_submitTransferOperationList(channel,
transferOperations, 2);

 /* Wait for send transfer operation to complete */
 status = STAR_waitOnTransferOperationCompletion(
sendTransferOperation, -1);

 /* If packet was sent successfully */
 if(status == STAR_TRANSFER_STATUS_COMPLETE)
 {
 /* Wait for receive transfer operation to complete */
 status = STAR_waitOnTransferOperationCompletion
(
 receiveTransferOperation, -1);

 /* If packet was received successfully */
 if (status == STAR_TRANSFER_STATUS_COMPLETE)
 {
 /* Get received packet */
 STAR_STREAM_ITEM * streamItem =
STAR_getTransferItem(
 receiveTransferOperation, 0);

 /* Initialise data length to 0 */
 unsigned int dataLength = 0;

 /* Get packet data and its length */

```

```

 unsigned char * data = STAR_getPacketData(
 (STAR_SPACEWIRE_PACKET *)streamItem->
 item, &dataLength);

 /* Check that packet data is valid */
 if (data)
 {
 /* Print received packet */
 printPacketContents(data, dataLength);

 /* Destroy packet data now that it has been
 output */
 STAR_destroyPacketData(data);
 }
 }

 /* Dispose send transfer operation */
 STAR_disposeTransferOperation(sendTransferOperation);
}

/* Dispose receive transfer operation */
STAR_disposeTransferOperation(receiveTransferOperation);
}

/* Destroy packet */
STAR_destroyStreamItem(packet);
}

/* Close channel */
STAR_closeChannel(channel);
}
}
}

```

## 9.44 transmit\_errors\_example.c

The user is prompted for the device to use, the channel to send out of and a series of bytes to be sent in the packets. The packet is sent twice, the first time containing a disconnect error in the middle and the second time containing a parity error in the middle.

```

#include "examples.h"

#include "general.h"
#include "utilities.h"

#include "star-dundee_types.h"
#include "star-api.h"

#include <stdlib.h>

void transmitErrorsExample()
{
 /* Pointer to a list of available SpaceWire devices */
 STAR_DEVICE_ID *devices = NULL;

 /* Define selected channel */
 int selectedChannelNumber;

 /* Get selected device */
 STAR_DEVICE_ID selectedDevice = promptForDevice(TRANSMIT_TYPE_SEND);

 /* Get selected channel */
 STAR_CHANNEL_ID selectedChannel = promptForChannel(selectedDevice,
 &selectedChannelNumber, TRANSMIT_TYPE_SEND);

 /* If channel was opened */
 if (selectedChannel)
 {
 /* Allocate array of bytes to be populated */
 unsigned char bytes[256];
 U16 readBytesLength = 0;

 /* Length of start of packet to be transmitted */
 U16 packetStartLength;

 /* Define the stream items that will hold the packet and errors */
 STAR_STREAM_ITEM *packetStart, *packetEnd;
 STAR_STREAM_ITEM *disconnectErrorItem, *parityErrorItem;
 STAR_STREAM_ITEM *packet[3];
 }
}

```

```

/* Ask user to enter a series of bytes to send */
printf("\nPlease enter a series of bytes to be transmitted over "
 "SpaceWire:\n");

/* If valid bytes read input bytes from console */
if(readBytes(bytes, 256, &readBytesLength))
{
 /* Get length of packet string */
 packetStartLength = readBytesLength / 2;

 /* Create the stream items */
 packetStart = STAR_createDataChunk(bytes,
 packetStartLength, 1, STAR_EOP_TYPE_NONE);
 packetEnd = STAR_createDataChunk(
 bytes + packetStartLength,
 readBytesLength - packetStartLength, 0, STAR_EOP_TYPE_EOP);
 disconnectErrorItem = STAR_createErrorInData(
 STAR_ERROR_IN_DATA_DISCONNECT);
 parityErrorItem = STAR_createErrorInData(
 STAR_ERROR_IN_DATA_PARITY);

 /* Make packet with from stream items with disconnect error */
 packet[0] = packetStart;
 packet[1] = disconnectErrorItem;
 packet[2] = packetEnd;

 /* If the stream items were created */
 if((packetStart != NULL) && (packetEnd != NULL) &&
 (disconnectErrorItem != NULL) && (parityErrorItem != NULL))
 {
 /* Create transfer operation */
 STAR_TRANSFER_OPERATION *transferOperation =
 STAR_createTxOperation(packet, 3);

 /* If transfer operation was created */
 if(transferOperation != NULL)
 {
 /* Define transfer status for result of transfer */
 /* operation */
 STAR_TRANSFER_STATUS status;

 /* Start transmitting the packet */
 STAR_submitTransferOperation(selectedChannel,
 transferOperation);

 /* Wait for packet to be transmitted */
 status = STAR_waitOnTransferOperationCompletion(
 transferOperation, -1);

 /* Dispose transfer operation */
 STAR_disposeTransferOperation(transferOperation);

 /* If the transfer operation completed successfully */
 if (status == STAR_TRANSFER_STATUS_COMPLETE)
 {
 /* Change the packet to contain a parity error */
 packet[1] = parityErrorItem;

 /* Create transfer operation */
 transferOperation = STAR_createTxOperation(packet, 3);

 /* If transfer operation was created */
 if (transferOperation != NULL)
 {
 /* Start transmitting the packet */
 STAR_submitTransferOperation(selectedChannel,
 transferOperation);

 /* Wait for packet to be transmitted */
 status = STAR_waitOnTransferOperationCompletion
 (
 transferOperation, -1);

 /* Check transfer status */
 if (status != STAR_TRANSFER_STATUS_COMPLETE)
 {
 printf("Unexpected transfer status: %d\n",
 status);
 }

 /* Dispose transfer operation */
 STAR_disposeTransferOperation(transferOperation);
 }
 }
 else
 {
 /* Unexpected transfer status */

```



```

 printf("Unexpected transfer status: %d\n", status);
 }
}

/* Destroy the stream items */
if(packetStart != NULL)
{
 STAR_destroyStreamItem(packetStart);
}
if(packetEnd != NULL)
{
 STAR_destroyStreamItem(packetEnd);
}
if(disconnectErrorItem != NULL)
{
 STAR_destroyStreamItem(disconnectErrorItem);
}
if(parityErrorItem != NULL)
{
 STAR_destroyStreamItem(parityErrorItem);
}
}

/* Close channel after transmitting packet */
STAR_closeChannel(selectedChannel);
}
else
{
 /* Print error */
 printf("Channel %d could not be opened.\n", selectedChannelNumber);
}

/* Destroy device list */
STAR_destroyDeviceList(devices);
}

```

## 9.45 triggering\_example.c

This file contains example code demonstrating how to use the Triggering API.

```

#include "utility.h"

#include "triggering_brick_mk3.h"
#include "triggering_pxi_if.h"
#include "triggering_pxi_ro.h"
#include "triggering_pcie.h"

#define CYCLES_PER_SECOND 60000000

typedef enum
{
 MENU_CHOICE_TX_TC_ON_EXT_TRIGGER = 1,
 MENU_CHOICE_TX_TC_ON_COUNTER,
 MENU_CHOICE_PKT_TX_ON_EXT_TRIGGER,
 MENU_CHOICE_PKT_TX_ON_COUNTER,
 MENU_CHOICE_PKT_TX_ON_TC,
 MENU_CHOICE_MULTIPLE_PKT_TX_ON_EXT_TRIGGER,
 MENU_CHOICE_MULTIPLE_PKT_TX_ON_TC,
 MENU_CHOICE_TIMED_PKT_TX_ON_TC,
 MENU_CHOICE_STOP_RECEIVING,
 MENU_CHOICE_PRINT_TRIGGER_CONF,
 MENU_CHOICE_RESET,
 MENU_CHOICE_EXIT,
 MENU_CHOICE_INVALID
} MENU_CHOICE;

void txTcOnExtTrigger(STAR_DEVICE_ID deviceId)
{
 if (gTriggerDevice == TRIGGER_DEVICE_PCIE_IF ||
 gTriggerDevice == TRIGGER_DEVICE_PXI_ROUTER)
 {
 printf("\n Test not supported by this device.\n");
 return;
 }

 reset(deviceId);
 printf("\n----- Trigger Example (Transmit time-code on external trigger) "
 "-----\n");

 if (gTriggerDevice == TRIGGER_DEVICE_BRICK_MK3)

```

```

{
 /* Enable external trigger 0 edge detect mode */
 TRIGGER_BRICK_MK3_enableExtTriggerEdgeDetectMode(
 deviceId, 0);

 /* EXT_TRIGGER_EVENT_IN event on external trigger 0 causes internal
 trigger 0 to be set */
 TRIGGER_BRICK_MK3_setExtTriggerInputEvents(deviceId, 0, 0
 ,
 EXT_TRIGGER_EVENT_IN);

 /* Internal trigger 0 causes TIME_CODE_ACTION_RX action on external
 trigger 0 */
 TRIGGER_BRICK_MK3_setTimeCodeOutputActions(deviceId, 0, 0
 ,
 TIME_CODE_ACTION_TX);

 /* Enable internal trigger 0 */
 TRIGGER_BRICK_MK3_enableTrigger(deviceId, 0);
}
else if (gTriggerDevice == TRIGGER_DEVICE_PXI_IF)
{
 /* Enable external trigger 0 edge detect mode */
 TRIGGER_PXI_IF_enableExtTriggerEdgeDetectMode(deviceId
 , 0);

 /* EXT_TRIGGER_EVENT_IN event on external trigger 0 causes internal
 trigger 0 to be set */
 TRIGGER_PXI_IF_setExtTriggerInputEvents(deviceId, 0, 0,
 EXT_TRIGGER_EVENT_IN);

 /* Internal trigger 0 causes TIME_CODE_ACTION_RX action on external
 trigger 0 */
 TRIGGER_PXI_IF_setTimeCodeOutputActions(deviceId, 0, 0,
 TIME_CODE_ACTION_TX);

 /* Enable internal trigger 0 */
 TRIGGER_PXI_IF_enableTrigger(deviceId, 0);
}

printf("Time-codes will now be transmitted when external trigger 0 is "
 "pulsed.\n\n");
}

void txTcOnCounter(STAR_DEVICE_ID deviceId)
{
 if (gTriggerDevice == TRIGGER_DEVICE_PCIE_IF)
 {
 printf("\n Test not supported by this device.\n");
 return;
 }

 reset(deviceId);
 printf("\n----- Trigger Example (Transmit time-code on timer) ----- \n");

 if (gTriggerDevice == TRIGGER_DEVICE_BRICK_MK3)
 {
 /* Set timer 0 reload value to 1 second */
 TRIGGER_BRICK_MK3_setCounterReloadValue(deviceId, 0,
 CYCLES_PER_SECOND);

 /* Enable timer auto reload */
 TRIGGER_BRICK_MK3_enableCounterAutoReload(deviceId, 0);

 /* COUNTER_EVENT_RELOAD event on timer 0 causes internal trigger 0 to
 be set */
 TRIGGER_BRICK_MK3_setCounterInputEvents(deviceId, 0, 0,
 COUNTER_EVENT_RELOAD);

 /* Internal trigger 0 causes TIME_CODE_ACTION_TX action on time-code
 engine 0 */
 TRIGGER_BRICK_MK3_setTimeCodeOutputActions(deviceId, 0, 0
 ,
 TIME_CODE_ACTION_TX);

 /* Enable internal trigger 0 */
 TRIGGER_BRICK_MK3_enableTrigger(deviceId, 0);
 }
 else if (gTriggerDevice == TRIGGER_DEVICE_PXI_IF)
 {
 /* Set timer 0 reload value to 1 second */
 TRIGGER_PXI_IF_setCounterReloadValue(deviceId, 0,
 CYCLES_PER_SECOND);

 /* Enable timer auto reload */
 TRIGGER_PXI_IF_enableCounterAutoReload(deviceId, 0);
 }
}

```

```

 /* COUNTER_EVENT_RELOAD event on timer 0 causes internal trigger 0 to
 be set */
 TRIGGER_PXI_IF_setCounterInputEvents(deviceId, 0, 0,
 COUNTER_EVENT_RELOAD);

 /* Internal trigger 0 causes TIME_CODE_ACTION_TX action on time-code
 engine 0 */
 TRIGGER_PXI_IF_setTimeCodeOutputActions(deviceId, 0, 0,
 TIME_CODE_ACTION_TX);

 /* Enable internal trigger 0 */
 TRIGGER_PXI_IF_enableTrigger(deviceId, 0);
}
else if (gTriggerDevice == TRIGGER_DEVICE_PXI_ROUTER)
{
 /* Set timer 0 reload value to 1 second */
 TRIGGER_PXI_ROUTER_setCounterReloadValue(deviceId, 0,
 CYCLES_PER_SECOND);

 /* Enable timer auto reload */
 TRIGGER_PXI_ROUTER_enableCounterAutoReload(deviceId, 0);

 /* COUNTER_EVENT_RELOAD event on timer 0 causes internal trigger 0 to
 be set */
 TRIGGER_PXI_ROUTER_setCounterInputEvents(deviceId, 0, 0,
 COUNTER_EVENT_RELOAD);

 /* Internal trigger 0 causes TIME_CODE_ACTION_TX action on time-code
 engine 0 */
 TRIGGER_PXI_ROUTER_setTimeCodeOutputActions(deviceId, 0,
0,
 TIME_CODE_ACTION_TX);

 /* Enable internal trigger 0 */
 TRIGGER_PXI_ROUTER_enableTrigger(deviceId, 0);
}

printf("Time-codes will now be transmitted every second.\n\n");
}

void pktTxOnExtTrigger(STAR_DEVICE_ID deviceId)
{
 if (gTriggerDevice == TRIGGER_DEVICE_PCIE_IF ||
 gTriggerDevice == TRIGGER_DEVICE_PXI_ROUTER)
 {
 printf("\n Test not supported by this device.\n");
 return;
 }

 reset(deviceId);
 printf("\n----- Trigger Example (Packet transmit on external trigger) "
 "-----\n");

 if (gTriggerDevice == TRIGGER_DEVICE_BRICK_MK3)
 {
 /* Enable external trigger 0 edge detect mode */
 TRIGGER_BRICK_MK3_enableExtTriggerEdgeDetectMode(
 deviceId, 0);

 /* EXT_TRIGGER_EVENT_IN event on external trigger 0 causes internal
 trigger 0 to be set */
 TRIGGER_BRICK_MK3_setExtTriggerInputEvents(deviceId, 0, 0
 ,
 EXT_TRIGGER_EVENT_IN);

 /* Enable port transmit packet mode on port 1 */
 TRIGGER_BRICK_MK3_enablePortTransmitPacketMode(
 deviceId, 1);

 /* Internal trigger 0 causes PORT_ACTION_TRANSMIT_PKT action on
 port 1 */
 TRIGGER_BRICK_MK3_setPortOutputActions(deviceId, 1, 0,
 PORT_ACTION_TRANSMIT_PKT);

 /* Queue up some packets */
 queuePackets(deviceId, 8);

 /* Enable internal trigger 0 */
 TRIGGER_BRICK_MK3_enableTrigger(deviceId, 0);
 }
 else if (gTriggerDevice == TRIGGER_DEVICE_PXI_IF)
 {
 /* Enable external trigger 0 edge detect mode */
 TRIGGER_PXI_IF_enableExtTriggerEdgeDetectMode(deviceId
 , 0);

 /* EXT_TRIGGER_EVENT_IN event on external trigger 0 causes internal

```

```

 trigger 0 to be set */
 TRIGGER_PXI_IF_setExtTriggerInputEvents(deviceId, 0, 0,
 EXT_TRIGGER_EVENT_IN);

 /* Enable port transmit packet mode on port 1 */
 TRIGGER_PXI_IF_enablePortTransmitPacketMode(deviceId, 1)
;

 /* Internal trigger 0 causes PORT_ACTION_TRANSMIT_PKT action on
 port 1 */
 TRIGGER_PXI_IF_setPortOutputActions(deviceId, 1, 0,
 PORT_ACTION_TRANSMIT_PKT);

 /* Queue up some packets */
 queuePackets(deviceId, 8);

 /* Enable internal trigger 0 */
 TRIGGER_PXI_IF_enableTrigger(deviceId, 0);
 }

 printf("8 packets have been queued and will now be transmitted on port 1 "
 "when external trigger 0 is pulsed.\n\n");
}

void pktTxOnCounter(STAR_DEVICE_ID deviceId)
{
 reset(deviceId);
 printf("\n----- Trigger Example (Packet transmit on timer) ----- \n");

 if (gTriggerDevice == TRIGGER_DEVICE_BRICK_MK3)
 {
 /* Set timer 0 reload value to 1 second */
 TRIGGER_BRICK_MK3_setCounterReloadValue(deviceId, 0,
 CYCLES_PER_SECOND);

 /* Enable timer auto reload */
 TRIGGER_BRICK_MK3_enableCounterAutoReload(deviceId, 0);

 /* COUNTER_EVENT_RELOAD event on timer 0 causes internal trigger 0 to
 be set */
 TRIGGER_BRICK_MK3_setCounterInputEvents(deviceId, 0, 0,
 COUNTER_EVENT_RELOAD);

 /* Enable port transmit packet mode on port 1 */
 TRIGGER_BRICK_MK3_enablePortTransmitPacketMode(
 deviceId, 1);

 /* Internal trigger 0 causes PORT_ACTION_TRANSMIT_PKT action on
 port 1 */
 TRIGGER_BRICK_MK3_setPortOutputActions(deviceId, 1, 0,
 PORT_ACTION_TRANSMIT_PKT);

 /* Queue up some packets */
 queuePackets(deviceId, 8);

 /* Enable internal trigger 0 */
 TRIGGER_BRICK_MK3_enableTrigger(deviceId, 0);
 }
 else if (gTriggerDevice == TRIGGER_DEVICE_PXI_IF)
 {
 /* Set timer 0 reload value to 1 second */
 TRIGGER_PXI_IF_setCounterReloadValue(deviceId, 0,
 CYCLES_PER_SECOND);

 /* Enable timer auto reload */
 TRIGGER_PXI_IF_enableCounterAutoReload(deviceId, 0);

 /* COUNTER_EVENT_RELOAD event on timer 0 causes internal trigger 0 to
 be set */
 TRIGGER_PXI_IF_setCounterInputEvents(deviceId, 0, 0,
 COUNTER_EVENT_RELOAD);

 /* Enable port transmit packet mode on port 1 */
 TRIGGER_PXI_IF_enablePortTransmitPacketMode(deviceId, 1)
;

 /* Internal trigger 0 causes PORT_ACTION_TRANSMIT_PKT action on
 port 1 */
 TRIGGER_PXI_IF_setPortOutputActions(deviceId, 1, 0,
 PORT_ACTION_TRANSMIT_PKT);

 /* Queue up some packets */
 queuePackets(deviceId, 8);

 /* Enable internal trigger 0 */
 TRIGGER_PXI_IF_enableTrigger(deviceId, 0);
 }
}

```

```

else if (gTriggerDevice == TRIGGER_DEVICE_PXI_ROUTER)
{
 /* Set timer 0 reload value to 1 second */
 TRIGGER_PXI_ROUTER_setCounterReloadValue(deviceId, 0,
 CYCLES_PER_SECOND);

 /* Enable timer auto reload */
 TRIGGER_PXI_ROUTER_enableCounterAutoReload(deviceId, 0);

 /* COUNTER_EVENT_RELOAD event on timer 0 causes internal trigger 0 to
 be set */
 TRIGGER_PXI_ROUTER_setCounterInputEvents(deviceId, 0, 0,
 COUNTER_EVENT_RELOAD);

 /* Enable port transmit packet mode on port 1 */
 TRIGGER_PXI_ROUTER_enablePortTransmitPacketMode(
 deviceId, 1);

 /* Internal trigger 0 causes PORT_ACTION_TRANSMIT_PKT action on
 port 1 */
 TRIGGER_PXI_ROUTER_setPortOutputActions(deviceId, 1, 0,
 PORT_ACTION_TRANSMIT_PKT);

 /* Queue up some packets */
 queuePackets(deviceId, 8);

 /* Enable internal trigger 0 */
 TRIGGER_PXI_ROUTER_enableTrigger(deviceId, 0);
}
else if (gTriggerDevice == TRIGGER_DEVICE_PCIE_IF)
{
 /* Set timer 0 reload value to 1 second */
 TRIGGER_PCIE_IF_setCounterReloadValue(deviceId, 0,
 CYCLES_PER_SECOND);

 /* Enable timer auto reload */
 TRIGGER_PCIE_IF_enableCounterAutoReload(deviceId, 0);

 /* COUNTER_EVENT_RELOAD event on timer 0 causes internal trigger 0 to
 be set */
 TRIGGER_PCIE_IF_setCounterInputEvents(deviceId, 0, 0,
 COUNTER_EVENT_RELOAD);

 /* Internal trigger 0 causes PORT_ACTION_TRANSMIT_PKT action on
 port 1 */
 TRIGGER_PCIE_IF_setPortOutputActions(deviceId, 1, 0,
 PORT_ACTION_TRANSMIT_PKT);

 /* Queue up some packets */
 queuePackets(deviceId, 8);

 /* Enable internal trigger 0 */
 TRIGGER_PCIE_IF_enableTrigger(deviceId, 0);
}

printf("8 packets have been queued and will now be transmitted every "
 "second.\n\n");
}

void pktTxOnTc(STAR_DEVICE_ID deviceId)
{
 if (gTriggerDevice == TRIGGER_DEVICE_PCIE_IF)
 {
 printf("\n Test not supported by this device.\n");
 return;
 }

 reset(deviceId);
 printf("\n----- Trigger Example (Packet transmit on time-code) ----- \n");

 if (gTriggerDevice == TRIGGER_DEVICE_BRICK_MK3)
 {
 /* TIME_CODE_EVENT_TICK event on time-code engine 0 causes internal
 trigger 0 to be set */
 TRIGGER_BRICK_MK3_setTimeCodeInputEvents(deviceId, 0, 0,
 TIME_CODE_EVENT_TICK);

 /* Enable port transmit packet mode on port 1 */
 TRIGGER_BRICK_MK3_enablePortTransmitPacketMode(
 deviceId, 1);

 /* Internal trigger 0 causes PORT_ACTION_TRANSMIT_PKT action on
 port 1 */
 TRIGGER_BRICK_MK3_setPortOutputActions(deviceId, 1, 0,
 PORT_ACTION_TRANSMIT_PKT);

 /* Queue up some packets */

```

```

 queuePackets(deviceId, 8);

 /* Enable internal trigger 0 */
 TRIGGER_BRICK_MK3_enableTrigger(deviceId, 0);
}
else if (gTriggerDevice == TRIGGER_DEVICE_PXI_IF)
{
 /* TIME_CODE_EVENT_TICK event on time-code engine 0 causes internal
 trigger 0 to be set */
 TRIGGER_PXI_IF_setTimeCodeInputEvents(deviceId, 0, 0,
 TIME_CODE_EVENT_TICK);

 /* Enable port transmit packet mode on port 1 */
 TRIGGER_PXI_IF_enablePortTransmitPacketMode(deviceId, 1)
;

 /* Internal trigger 0 causes PORT_ACTION_TRANSMIT_PKT action on
 port 1 */
 TRIGGER_PXI_IF_setPortOutputActions(deviceId, 1, 0,
 PORT_ACTION_TRANSMIT_PKT);

 /* Queue up some packets */
 queuePackets(deviceId, 8);

 /* Enable internal trigger 0 */
 TRIGGER_PXI_IF_enableTrigger(deviceId, 0);
}
else if (gTriggerDevice == TRIGGER_DEVICE_PXI_ROUTER)
{
 /* TIME_CODE_EVENT_TICK event on time-code engine 0 causes internal
 trigger 0 to be set */
 TRIGGER_PXI_ROUTER_setTimeCodeInputEvents(deviceId, 0, 0,
 TIME_CODE_EVENT_TICK);

 /* Enable port transmit packet mode on port 1 */
 TRIGGER_PXI_ROUTER_enablePortTransmitPacketMode(
deviceId, 1);

 /* Internal trigger 0 causes PORT_ACTION_TRANSMIT_PKT action on
 port 1 */
 TRIGGER_PXI_ROUTER_setPortOutputActions(deviceId, 1, 0,
 PORT_ACTION_TRANSMIT_PKT);

 /* Queue up some packets */
 queuePackets(deviceId, 8);

 /* Enable internal trigger 0 */
 TRIGGER_PXI_ROUTER_enableTrigger(deviceId, 0);
}

printf("8 packets have been queued and will now be transmitted when "
 "time-codes are received.\n\n");
}

void multiplePktTxOnExtTrigger(STAR_DEVICE_ID deviceId)
{
 if (gTriggerDevice == TRIGGER_DEVICE_PCIE_IF ||
 gTriggerDevice == TRIGGER_DEVICE_PXI_ROUTER)
 {
 printf("\n Test not supported by this device.\n");
 return;
 }

 reset(deviceId);
 printf("\n----- Trigger Example (Multiple packet transmit on external "
 "trigger) ----- \n");

 if (gTriggerDevice == TRIGGER_DEVICE_BRICK_MK3)
 {
 /* Enable timer trigger count on timer 0 so the timer only counts on
 triggers */
 TRIGGER_BRICK_MK3_enableCounterTriggerCount(deviceId, 0)
;

 /* Set timer 0 reload value to 3 */
 TRIGGER_BRICK_MK3_setCounterReloadValue(deviceId, 0, 3);

 /* Enable external trigger 0 edge detect mode */
 TRIGGER_BRICK_MK3_enableExtTriggerEdgeDetectMode(
deviceId, 0);

 /* Enable port transmit packet mode on port 1 */
 TRIGGER_BRICK_MK3_enablePortTransmitPacketMode(
deviceId, 1);

 /* EXT_TRIGGER_EVENT_IN event on external trigger 0 causes internal
 trigger 0 to be set */

```

```

 TRIGGER_BRICK_MK3_setExtTriggerInputEvents(deviceId, 0, 0
,
 EXT_TRIGGER_EVENT_IN);

 /* External trigger 0 causes COUNTER_ACTION_RELOAD action on timer 0 */
 TRIGGER_BRICK_MK3_setCounterOutputActions(deviceId, 0, 0,
 COUNTER_ACTION_RELOAD);

 /* Internal trigger 0 causes PORT_ACTION_TRANSMIT_PKT action on
 port 1 */
 TRIGGER_BRICK_MK3_setPortOutputActions(deviceId, 1, 0,
 PORT_ACTION_TRANSMIT_PKT);

 /* PORT_EVENT_TX_EOP event causes internal trigger 1 to be set */
 TRIGGER_BRICK_MK3_setPortInputEvents(deviceId, 1, 1,
 PORT_EVENT_TX_EOP);

 /* Internal trigger 1 causes COUNTER_ACTION_COUNT action on timer 0 */
 TRIGGER_BRICK_MK3_setCounterOutputActions(deviceId, 0, 1,
 COUNTER_ACTION_COUNT);

 /* COUNTER_EVENT_COUNT event on timer 0 causes internal trigger 2 to be
 set */
 TRIGGER_BRICK_MK3_setCounterInputEvents(deviceId, 0, 2,
 COUNTER_EVENT_COUNT);

 /* Internal trigger 2 causes PORT_ACTION_TRANSMIT_PKT action on
 port 1 */
 TRIGGER_BRICK_MK3_setPortOutputActions(deviceId, 1, 2,
 PORT_ACTION_TRANSMIT_PKT);

 /* Queue up some packets */
 queuePackets(deviceId, 32);

 /* Enable internal triggers 0, 1 and 2 */
 TRIGGER_BRICK_MK3_enableTrigger(deviceId, 0);
 TRIGGER_BRICK_MK3_enableTrigger(deviceId, 1);
 TRIGGER_BRICK_MK3_enableTrigger(deviceId, 2);
}
else if (gTriggerDevice == TRIGGER_DEVICE_PXI_IF)
{
 /* Enable timer trigger count on timer 0 so the timer only counts on
 triggers */
 TRIGGER_PXI_IF_enableCounterTriggerCount(deviceId, 0);

 /* Set timer 0 reload value to 3 */
 TRIGGER_PXI_IF_setCounterReloadValue(deviceId, 0, 3);

 /* Enable external trigger 0 edge detect mode */
 TRIGGER_PXI_IF_enableExtTriggerEdgeDetectMode(deviceId
, 0);

 /* Enable port transmit packet mode on port 1 */
 TRIGGER_PXI_IF_enablePortTransmitPacketMode(deviceId, 1)
;

 /* EXT_TRIGGER_EVENT_IN event on external trigger 0 causes internal
 trigger 0 to be set */
 TRIGGER_PXI_IF_setExtTriggerInputEvents(deviceId, 0, 0,
 EXT_TRIGGER_EVENT_IN);

 /* External trigger 0 causes COUNTER_ACTION_RELOAD action on timer 0 */
 TRIGGER_PXI_IF_setCounterOutputActions(deviceId, 0, 0,
 COUNTER_ACTION_RELOAD);

 /* Internal trigger 0 causes PORT_ACTION_TRANSMIT_PKT action on
 port 1 */
 TRIGGER_PXI_IF_setPortOutputActions(deviceId, 1, 0,
 PORT_ACTION_TRANSMIT_PKT);

 /* PORT_EVENT_TX_EOP event causes internal trigger 1 to be set */
 TRIGGER_PXI_IF_setPortInputEvents(deviceId, 1, 1,
 PORT_EVENT_TX_EOP);

 /* Internal trigger 1 causes COUNTER_ACTION_COUNT action on timer 0 */
 TRIGGER_PXI_IF_setCounterOutputActions(deviceId, 0, 1,
 COUNTER_ACTION_COUNT);

 /* COUNTER_EVENT_COUNT event on timer 0 causes internal trigger 2 to be
 set */
 TRIGGER_PXI_IF_setCounterInputEvents(deviceId, 0, 2,
 COUNTER_EVENT_COUNT);

 /* Internal trigger 2 causes PORT_ACTION_TRANSMIT_PKT action on
 port 1 */
 TRIGGER_PXI_IF_setPortOutputActions(deviceId, 1, 2,
 PORT_ACTION_TRANSMIT_PKT);

```

```

 /* Queue up some packets */
 queuePackets(deviceId, 32);

 /* Enable internal triggers 0, 1 and 2 */
 TRIGGER_PXI_IF_enableTrigger(deviceId, 0);
 TRIGGER_PXI_IF_enableTrigger(deviceId, 1);
 TRIGGER_PXI_IF_enableTrigger(deviceId, 2);
 }

 printf("32 packets have been queued and will now be transmitted in groups "
 "of four on port 1 when external trigger 0 is pulsed.\n\n");
}

void multiplePktTxOnTc(STAR_DEVICE_ID deviceId)
{
 if (gTriggerDevice == TRIGGER_DEVICE_PCIE_IF)
 {
 printf("\n Test not supported by this device.\n");
 return;
 }

 reset(deviceId);
 printf("\n----- Trigger Example (Multiple packet transmit on time-code) "
 "-----\n");

 if (gTriggerDevice == TRIGGER_DEVICE_BRICK_MK3)
 {
 /* Enable timer trigger count on timer 0 so the timer only counts on
 triggers */
 TRIGGER_BRICK_MK3_enableCounterTriggerCount(deviceId, 0);

 /* Set timer 0 reload value to 3 */
 TRIGGER_BRICK_MK3_setCounterReloadValue(deviceId, 0, 3);

 /* Enable port transmit packet mode on port 1 */
 TRIGGER_BRICK_MK3_enablePortTransmitPacketMode(
 deviceId, 1);

 /* TIME_CODE_EVENT_TICK event on time-code engine 0 causes internal
 trigger 0 to be set */
 TRIGGER_BRICK_MK3_setTimeCodeInputEvents(deviceId, 0, 0,
 TIME_CODE_EVENT_TICK);

 /* Internal trigger 0 causes COUNTER_ACTION_RELOAD action on timer 0 */
 TRIGGER_BRICK_MK3_setCounterOutputActions(deviceId, 0, 0,
 COUNTER_ACTION_RELOAD);

 /* Internal trigger 0 causes PORT_ACTION_TRANSMIT_PKT action on
 port 1 */
 TRIGGER_BRICK_MK3_setPortOutputActions(deviceId, 1, 0,
 PORT_ACTION_TRANSMIT_PKT);

 /* PORT_EVENT_TX_EOP event causes internal trigger 1 to be set */
 TRIGGER_BRICK_MK3_setPortInputEvents(deviceId, 1, 1,
 PORT_EVENT_TX_EOP);

 /* Internal trigger 1 causes COUNTER_ACTION_COUNT action on timer 0 */
 TRIGGER_BRICK_MK3_setCounterOutputActions(deviceId, 0, 1,
 COUNTER_ACTION_COUNT);

 /* COUNTER_EVENT_COUNT event on timer 0 causes internal trigger 2 to be
 set */
 TRIGGER_BRICK_MK3_setCounterInputEvents(deviceId, 0, 2,
 COUNTER_EVENT_COUNT);

 /* Internal trigger 2 causes PORT_ACTION_TRANSMIT_PKT action on
 port 1 */
 TRIGGER_BRICK_MK3_setPortOutputActions(deviceId, 1, 2,
 PORT_ACTION_TRANSMIT_PKT);

 /* Queue up some packets */
 queuePackets(deviceId, 32);

 /* Enable internal triggers 0, 1 and 2 */
 TRIGGER_BRICK_MK3_enableTrigger(deviceId, 0);
 TRIGGER_BRICK_MK3_enableTrigger(deviceId, 1);
 TRIGGER_BRICK_MK3_enableTrigger(deviceId, 2);
 }
 else if (gTriggerDevice == TRIGGER_DEVICE_PXI_IF)
 {
 /* Enable timer trigger count on timer 0 so the timer only counts on
 triggers */
 TRIGGER_PXI_IF_enableCounterTriggerCount(deviceId, 0);

 /* Set timer 0 reload value to 3 */

```



```

 TRIGGER_PXI_IF_setCounterReloadValue(deviceId, 0, 3);

 /* Enable port transmit packet mode on port 1 */
 TRIGGER_PXI_IF_enablePortTransmitPacketMode(deviceId, 1)
;

 /* TIME_CODE_EVENT_TICK event on time-code engine 0 causes internal
 trigger 0 to be set */
 TRIGGER_PXI_IF_setTimeCodeInputEvents(deviceId, 0, 0,
 TIME_CODE_EVENT_TICK);

 /* Internal trigger 0 causes COUNTER_ACTION_RELOAD action on timer 0 */
 TRIGGER_PXI_IF_setCounterOutputActions(deviceId, 0, 0,
 COUNTER_ACTION_RELOAD);

 /* Internal trigger 0 causes PORT_ACTION_TRANSMIT_PKT action on
 port 1 */
 TRIGGER_PXI_IF_setPortOutputActions(deviceId, 1, 0,
 PORT_ACTION_TRANSMIT_PKT);

 /* PORT_EVENT_TX_EOP event causes internal trigger 1 to be set */
 TRIGGER_PXI_IF_setPortInputEvents(deviceId, 1, 1,
 PORT_EVENT_TX_EOP);

 /* Internal trigger 1 causes COUNTER_ACTION_COUNT action on timer 0 */
 TRIGGER_PXI_IF_setCounterOutputActions(deviceId, 0, 1,
 COUNTER_ACTION_COUNT);

 /* COUNTER_EVENT_COUNT event on timer 0 causes internal trigger 2 to be
 set */
 TRIGGER_PXI_IF_setCounterInputEvents(deviceId, 0, 2,
 COUNTER_EVENT_COUNT);

 /* Internal trigger 2 causes PORT_ACTION_TRANSMIT_PKT action on
 port 1 */
 TRIGGER_PXI_IF_setPortOutputActions(deviceId, 1, 2,
 PORT_ACTION_TRANSMIT_PKT);

 /* Queue up some packets */
 queuePackets(deviceId, 32);

 /* Enable internal triggers 0, 1 and 2 */
 TRIGGER_PXI_IF_enableTrigger(deviceId, 0);
 TRIGGER_PXI_IF_enableTrigger(deviceId, 1);
 TRIGGER_PXI_IF_enableTrigger(deviceId, 2);
}
else if (gTriggerDevice == TRIGGER_DEVICE_PXI_ROUTER)
{
 /* Enable timer trigger count on timer 0 so the timer only counts on
 triggers */
 TRIGGER_PXI_ROUTER_enableCounterTriggerCount(deviceId,
0);

 /* Set timer 0 reload value to 3 */
 TRIGGER_PXI_ROUTER_setCounterReloadValue(deviceId, 0, 3);

 /* Enable port transmit packet mode on port 1 */
 TRIGGER_PXI_ROUTER_enablePortTransmitPacketMode(
deviceId, 1);

 /* TIME_CODE_EVENT_TICK event on time-code engine 0 causes internal
 trigger 0 to be set */
 TRIGGER_PXI_ROUTER_setTimeCodeInputEvents(deviceId, 0, 0,
 TIME_CODE_EVENT_TICK);

 /* Internal trigger 0 causes COUNTER_ACTION_RELOAD action on timer 0 */
 TRIGGER_PXI_ROUTER_setCounterOutputActions(deviceId, 0, 0
,
 COUNTER_ACTION_RELOAD);

 /* Internal trigger 0 causes PORT_ACTION_TRANSMIT_PKT action on
 port 1 */
 TRIGGER_PXI_ROUTER_setPortOutputActions(deviceId, 1, 0,
 PORT_ACTION_TRANSMIT_PKT);

 /* PORT_EVENT_TX_EOP event causes internal trigger 1 to be set */
 TRIGGER_PXI_ROUTER_setPortInputEvents(deviceId, 1, 1,
 PORT_EVENT_TX_EOP);

 /* Internal trigger 1 causes COUNTER_ACTION_COUNT action on timer 0 */
 TRIGGER_PXI_ROUTER_setCounterOutputActions(deviceId, 0, 1
,
 COUNTER_ACTION_COUNT);

 /* COUNTER_EVENT_COUNT event on timer 0 causes internal trigger 2 to be
 set */
 TRIGGER_PXI_ROUTER_setCounterInputEvents(deviceId, 0, 2,

```

```

 COUNTER_EVENT_COUNT);

 /* Internal trigger 2 causes PORT_ACTION_TRANSMIT_PKT action on
 port 1 */
 TRIGGER_PXI_ROUTER_setPortOutputActions(deviceId, 1, 2,
 PORT_ACTION_TRANSMIT_PKT);

 /* Queue up some packets */
 queuePackets(deviceId, 32);

 /* Enable internal triggers 0, 1 and 2 */
 TRIGGER_PXI_ROUTER_enableTrigger(deviceId, 0);
 TRIGGER_PXI_ROUTER_enableTrigger(deviceId, 1);
 TRIGGER_PXI_ROUTER_enableTrigger(deviceId, 2);
}

printf("32 packets have been queued and will now be transmitted in groups "
 "of four on port 1 when time-codes are received.\n\n");
}

void timedPktTxOnTc(STAR_DEVICE_ID deviceId)
{
 if (gTriggerDevice == TRIGGER_DEVICE_PCIE_IF)
 {
 printf("\n Test not supported by this device.\n");
 return;
 }

 reset(deviceId);
 printf("\n----- Trigger Example (Timed Packet Transmit on Time-Code) "
 "-----\n");

 if (gTriggerDevice == TRIGGER_DEVICE_BRICK_MK3)
 {
 /* Setup timer 0 to transmit time-codes every second and reload/start
 timer 1 */
 TRIGGER_BRICK_MK3_setCounterReloadValue(deviceId, 0,
 CYCLES_PER_SECOND);
 TRIGGER_BRICK_MK3_enableCounterAutoReload(deviceId, 0);
 TRIGGER_BRICK_MK3_setCounterInputEvents(deviceId, 0, 0,
 COUNTER_EVENT_RELOAD);
 TRIGGER_BRICK_MK3_setTimeCodeOutputActions(deviceId, 0, 0,
 TIME_CODE_ACTION_TX);
 TRIGGER_BRICK_MK3_setCounterOutputActions(deviceId, 1, 0,
 COUNTER_ACTION_RELOAD);
 TRIGGER_BRICK_MK3_setCounterOutputActions(deviceId, 1, 0,
 COUNTER_ACTION_START);

 /* Setup timer 1 to expire every 200 ms and enable start/stop mode */
 TRIGGER_BRICK_MK3_setCounterReloadValue(deviceId, 1,
 CYCLES_PER_SECOND / 5);
 TRIGGER_BRICK_MK3_enableCounterAutoReload(deviceId, 1);
 TRIGGER_BRICK_MK3_enableCounterStartStopMode(deviceId,
 1);

 /* Setup timer 1 to decrement timer 2 (packet counter timer) */
 TRIGGER_BRICK_MK3_setCounterInputEvents(deviceId, 1, 1,
 COUNTER_EVENT_ZERO_SINGLE);
 TRIGGER_BRICK_MK3_setCounterOutputActions(deviceId, 2, 1,
 COUNTER_ACTION_COUNT);

 /* Setup timer 2 to count from 4 to 0 and enable trigger count mode */
 TRIGGER_BRICK_MK3_setCounterReloadValue(deviceId, 2, 4);
 TRIGGER_BRICK_MK3_enableCounterAutoReload(deviceId, 2);
 TRIGGER_BRICK_MK3_enableCounterTriggerCount(deviceId, 2);

 /* Setup timer 2 to transmit packets */
 TRIGGER_BRICK_MK3_setCounterInputEvents(deviceId, 2, 2,
 COUNTER_EVENT_COUNT);
 TRIGGER_BRICK_MK3_setPortOutputActions(deviceId, 1, 2,
 PORT_ACTION_TRANSMIT_PKT);

 /* Setup timer 2 to stop timer 1 when it hits 0 */
 TRIGGER_BRICK_MK3_setCounterInputEvents(deviceId, 2, 3,
 COUNTER_EVENT_ZERO_SINGLE);
 TRIGGER_BRICK_MK3_setCounterOutputActions(deviceId, 1, 3,
 COUNTER_ACTION_STOP);

 /* Enable port transmit packet mode on port 1 */
 TRIGGER_BRICK_MK3_enablePortTransmitPacketMode(
 deviceId, 1);

 /* Reload timers */
 TRIGGER_BRICK_MK3_forceCounterReload(deviceId, 0);
 TRIGGER_BRICK_MK3_forceCounterReload(deviceId, 1);
 }
}

```

```

 TRIGGER_BRICK_MK3_forceCounterReload(deviceId, 2);

 /* Enable internal triggers 0-3 */
 TRIGGER_BRICK_MK3_enableTrigger(deviceId, 0);
 TRIGGER_BRICK_MK3_enableTrigger(deviceId, 1);
 TRIGGER_BRICK_MK3_enableTrigger(deviceId, 2);
 TRIGGER_BRICK_MK3_enableTrigger(deviceId, 3);

 queuePackets(deviceId, 32);
}
else if (gTriggerDevice == TRIGGER_DEVICE_PXI_IF)
{
 /* Setup timer 0 to transmit time-codes every second and reload/start
 timer 1 */
 TRIGGER_PXI_IF_setCounterReloadValue(deviceId, 0,
 CYCLES_PER_SECOND);
 TRIGGER_PXI_IF_enableCounterAutoReload(deviceId, 0);
 TRIGGER_PXI_IF_setCounterInputEvents(deviceId, 0, 0,
 COUNTER_EVENT_RELOAD);
 TRIGGER_PXI_IF_setTimeCodeOutputActions(deviceId, 0, 0,
 TIME_CODE_ACTION_TX);
 TRIGGER_PXI_IF_setCounterOutputActions(deviceId, 1, 0,
 COUNTER_ACTION_RELOAD);
 TRIGGER_PXI_IF_setCounterOutputActions(deviceId, 1, 0,
 COUNTER_ACTION_START);

 /* Setup timer 1 to expire every 200 ms and enable start/stop mode */
 TRIGGER_PXI_IF_setCounterReloadValue(deviceId, 1,
 CYCLES_PER_SECOND / 5);
 TRIGGER_PXI_IF_enableCounterAutoReload(deviceId, 1);
 TRIGGER_PXI_IF_enableCounterStartStopMode(deviceId, 1);

 /* Setup timer 1 to decrement timer 2 (packet counter timer) */
 TRIGGER_PXI_IF_setCounterInputEvents(deviceId, 1, 1,
 COUNTER_EVENT_ZERO_SINGLE);
 TRIGGER_PXI_IF_setCounterOutputActions(deviceId, 2, 1,
 COUNTER_ACTION_COUNT);

 /* Setup timer 2 to count from 4 to 0 and enable trigger count mode */
 TRIGGER_PXI_IF_setCounterReloadValue(deviceId, 2, 4);
 TRIGGER_PXI_IF_enableCounterAutoReload(deviceId, 2);
 TRIGGER_PXI_IF_enableCounterTriggerCount(deviceId, 2);

 /* Setup timer 2 to transmit packets */
 TRIGGER_PXI_IF_setCounterInputEvents(deviceId, 2, 2,
 COUNTER_EVENT_COUNT);
 TRIGGER_PXI_IF_setPortOutputActions(deviceId, 1, 2,
 PORT_ACTION_TRANSMIT_PKT);

 /* Setup timer 2 to stop timer 1 when it hits 0 */
 TRIGGER_PXI_IF_setCounterInputEvents(deviceId, 2, 3,
 COUNTER_EVENT_ZERO_SINGLE);
 TRIGGER_PXI_IF_setCounterOutputActions(deviceId, 1, 3,
 COUNTER_ACTION_STOP);

 /* Enable port transmit packet mode on port 1 */
 TRIGGER_PXI_IF_enablePortTransmitPacketMode(deviceId, 1)
;

 /* Reload timers */
 TRIGGER_PXI_IF_forceCounterReload(deviceId, 0);
 TRIGGER_PXI_IF_forceCounterReload(deviceId, 1);
 TRIGGER_PXI_IF_forceCounterReload(deviceId, 2);

 /* Enable internal triggers 0-3 */
 TRIGGER_PXI_IF_enableTrigger(deviceId, 0);
 TRIGGER_PXI_IF_enableTrigger(deviceId, 1);
 TRIGGER_PXI_IF_enableTrigger(deviceId, 2);
 TRIGGER_PXI_IF_enableTrigger(deviceId, 3);

 queuePackets(deviceId, 32);
}
else if (gTriggerDevice == TRIGGER_DEVICE_PXI_ROUTER)
{
 /* Setup timer 0 to transmit time-codes every second and reload/start
 timer 1 */
 TRIGGER_PXI_ROUTER_setCounterReloadValue(deviceId, 0,
 CYCLES_PER_SECOND);
 TRIGGER_PXI_ROUTER_enableCounterAutoReload(deviceId, 0);
 TRIGGER_PXI_ROUTER_setCounterInputEvents(deviceId, 0, 0,
 COUNTER_EVENT_RELOAD);
 TRIGGER_PXI_ROUTER_setTimeCodeOutputActions(deviceId, 0,
 0,
 TIME_CODE_ACTION_TX);
 TRIGGER_PXI_ROUTER_setCounterOutputActions(deviceId, 1, 0
 ,
 COUNTER_ACTION_RELOAD);

```

```

 TRIGGER_PXI_ROUTER_setCounterOutputActions(deviceId, 1, 0
,
 COUNTER_ACTION_START);

 /* Setup timer 1 to expire every 200 ms and enable start/stop mode */
 TRIGGER_PXI_ROUTER_setCounterReloadValue(deviceId, 1,
 CYCLES_PER_SECOND / 5);
 TRIGGER_PXI_ROUTER_enableCounterAutoReload(deviceId, 1);
 TRIGGER_PXI_ROUTER_enableCounterStartStopMode(deviceId
, 1);

 /* Setup timer 1 to decrement timer 2 (packet counter timer) */
 TRIGGER_PXI_ROUTER_setCounterInputEvents(deviceId, 1, 1,
 COUNTER_EVENT_ZERO_SINGLE);
 TRIGGER_PXI_ROUTER_setCounterOutputActions(deviceId, 2, 1
,
 COUNTER_ACTION_COUNT);

 /* Setup timer 2 to count from 4 to 0 and enable trigger count mode */
 TRIGGER_PXI_ROUTER_setCounterReloadValue(deviceId, 2, 4);
 TRIGGER_PXI_ROUTER_enableCounterAutoReload(deviceId, 2);
 TRIGGER_PXI_ROUTER_enableCounterTriggerCount(deviceId,
2);

 /* Setup timer 2 to transmit packets */
 TRIGGER_PXI_ROUTER_setCounterInputEvents(deviceId, 2, 2,
 COUNTER_EVENT_COUNT);
 TRIGGER_PXI_ROUTER_setPortOutputActions(deviceId, 1, 2,
 PORT_ACTION_TRANSMIT_PKT);

 /* Setup timer 2 to stop timer 1 when it hits 0 */
 TRIGGER_PXI_ROUTER_setCounterInputEvents(deviceId, 2, 3,
 COUNTER_EVENT_ZERO_SINGLE);
 TRIGGER_PXI_ROUTER_setCounterOutputActions(deviceId, 1, 3
,
 COUNTER_ACTION_STOP);

 /* Enable port transmit packet mode on port 1 */
 TRIGGER_PXI_ROUTER_enablePortTransmitPacketMode(
deviceId, 1);

 /* Reload timers */
 TRIGGER_PXI_ROUTER_forceCounterReload(deviceId, 0);
 TRIGGER_PXI_ROUTER_forceCounterReload(deviceId, 1);
 TRIGGER_PXI_ROUTER_forceCounterReload(deviceId, 2);

 /* Enable internal triggers 0-3 */
 TRIGGER_PXI_ROUTER_enableTrigger(deviceId, 0);
 TRIGGER_PXI_ROUTER_enableTrigger(deviceId, 1);
 TRIGGER_PXI_ROUTER_enableTrigger(deviceId, 2);
 TRIGGER_PXI_ROUTER_enableTrigger(deviceId, 3);

 queuePackets(deviceId, 32);
}

printf("32 packets have been queued and will now be transmitted in groups "
 "of 4 at 200 ms intervals when time-codes are transmitted.\n\n");
}

void stopReceiving(STAR_DEVICE_ID deviceId)
{
 if (gTriggerDevice != TRIGGER_DEVICE_BRICK_MK3)
 {
 printf("\n Test not supported by this device.\n");
 return;
 }

 reset(deviceId);
 printf("\n----- Trigger Example (Stop Receiving on Port 2 For 5 Seconds "
 "Every 10 Seconds) -----");

 /* Setup counter 0 to fire every 10 seconds and reload counter 1 */
 TRIGGER_BRICK_MK3_setCounterReloadValue(deviceId, 0,
 10 * CYCLES_PER_SECOND);
 TRIGGER_BRICK_MK3_enableCounterAutoReload(deviceId, 0);
 TRIGGER_BRICK_MK3_setCounterInputEvents(deviceId, 0, 0,
 COUNTER_EVENT_RELOAD);
 TRIGGER_BRICK_MK3_setCounterOutputActions(deviceId, 1, 0,
 COUNTER_ACTION_RELOAD);

 /* Setup counter 1 to fire a single-shot 5 second timer which, while */
 /* not zero, causes the STOP_RECEPTION action on port 2 */
 TRIGGER_BRICK_MK3_setCounterReloadValue(deviceId, 1,
 5 * CYCLES_PER_SECOND);
 TRIGGER_BRICK_MK3_disableCounterAutoReload(deviceId, 1);
 TRIGGER_BRICK_MK3_setCounterInputEvents(deviceId, 1, 1,
 COUNTER_EVENT_ZERO);

```

```

 TRIGGER_BRICK_MK3_setPortOutputActions(deviceId, 2, 1,
 PORT_ACTION_STOP_RECEPTION);

 /* Setup internal trigger 1 to receive inverted input from counter 1 */
 TRIGGER_BRICK_MK3_setTriggerInvertMask(deviceId, 1,
 TRIGGER_TYPE_COUNTER, 0x2);

 /* Enable internal triggers 0-1 */
 TRIGGER_BRICK_MK3_enableTrigger(deviceId, 0);
 TRIGGER_BRICK_MK3_enableTrigger(deviceId, 1);

 printf("Reception will now be stopped for 5 seconds every 10 seconds.\n\n");
}

void resetTriggerConf(STAR_DEVICE_ID deviceId)
{
 reset(deviceId);
 printf("\n----- Trigger Example (Reset triggers) ----- \n");
 printf("Triggers have been reset.\n\n");
}

MENU_CHOICE getMenuChoice(void)
{
 char buffer[32];
 unsigned int choice;

 /* Show menu to user */
 printf("Please select example to run:\n");
 printf("%d. Transmit time-code on external trigger.\n",
 MENU_CHOICE_TX_TC_ON_EXT_TRIGGER);
 printf("%d. Transmit time-code on timer.\n",
 MENU_CHOICE_TX_TC_ON_COUNTER);
 printf("%d. Transmit packet on external trigger.\n",
 MENU_CHOICE_PKT_TX_ON_EXT_TRIGGER);
 printf("%d. Transmit packet on timer.\n",
 MENU_CHOICE_PKT_TX_ON_COUNTER);
 printf("%d. Transmit packet on time-code.\n",
 MENU_CHOICE_PKT_TX_ON_TC);
 printf("%d. Transmit multiple packets on external trigger.\n",
 MENU_CHOICE_MULTIPLE_PKT_TX_ON_EXT_TRIGGER);
 printf("%d. Transmit multiple packets on time-code.\n",
 MENU_CHOICE_MULTIPLE_PKT_TX_ON_TC);
 printf("%d. Transmit packets at intervals with time-code.\n",
 MENU_CHOICE_TIMED_PKT_TX_ON_TC);
 printf("%d. Stop receiving for 5 seconds every 10 seconds.\n",
 MENU_CHOICE_STOP_RECEIVING);
 printf("%d. Print trigger configuration.\n",
 MENU_CHOICE_PRINT_TRIGGER_CONF);
 printf("%d. Reset triggers.\n", MENU_CHOICE_RESET);
 printf("%d. Exit\n", MENU_CHOICE_EXIT);
 printf("> ");

 /* Validate chosen number */
 if (!fgets(buffer, 32, stdin)) return MENU_CHOICE_INVALID;

 /* Get menu choice value */
 if (!sscanf(buffer, "%u", &choice) ||
 choice >= MENU_CHOICE_INVALID) return MENU_CHOICE_INVALID;

 /* Return the chosen value */
 return choice;
}

STAR_DEVICE_ID getTriggerDeviceID()
{
 char *pDeviceName, *pDeviceSerial, buffer[256];
 unsigned int chosen;
 int status;

 /* List devices that support triggering API */
 U32 deviceTypes[5] = { STAR_DEVICE_BRICK_MK3,
 STAR_DEVICE_PXI_INTERFACE,
 STAR_DEVICE_PXI_RMAP,
 STAR_DEVICE_PXI_ROUTER_12,
 STAR_DEVICE_PCIE };

 /* Get available devices that support triggering */
 U32 deviceCount;
 STAR_DEVICE_ID* pDevices = STAR_getDeviceListForTypes(
 deviceTypes, 5,
 &deviceCount);
 STAR_DEVICE_ID selectedDeviceID = STAR_DEVICE_UNKNOWN;

 /* If more than one device is available */
 if (deviceCount > 1)
 {
 U32 x;

```

```

printf("The following devices are available that support triggering:"
 "\n\n");

/* For each device */
for (x = 0; x < deviceCount; x++)
{
 /* Display its name */
 if (pDevices[x] != STAR_DEVICE_UNKNOWN)
 {
 pDeviceName = STAR_getDeviceName(pDevices[x]);
 pDeviceSerial = STAR_getDeviceSerialNumber(pDevices[x]);
 if (pDeviceName != NULL && pDeviceSerial != NULL)
 {
 printf("\t%u - %s with serial number %s\n", x, pDeviceName,
 pDeviceSerial);
 }
 else
 {
 printf("\t%u - Unknown SpaceWire Device\n", x);
 }

 STAR_destroyString(pDeviceName);
 STAR_destroyString(pDeviceSerial);
 }
 else
 {
 printf("\t%u - Unable to access device\n", x);
 }
}

printf("\n");

/* Ask the user which device to use */
do
{
 printf("Please select which device to use: ");
 fflush(stdout);

 /* Read selected device number */
 if (fgets(buffer, 256, stdin) == NULL)
 {
 puts("No device number selected.");

 continue;
 }
 status = sscanf(buffer, "%u", &chosen);
 if ((status == 0) || (chosen > (deviceCount - 1U)))
 {
 puts("Invalid device number selected.");

 continue;
 }

 /* Get the selected device id */
 selectedDeviceID = pDevices[chosen];
} while (selectedDeviceID == STAR_DEVICE_UNKNOWN);

/* Else if just one device is available */
else if (deviceCount == 1)
{
 /* Select the first device id */
 selectedDeviceID = pDevices[0];
}

/* If valid device was selected */
if (selectedDeviceID != STAR_DEVICE_UNKNOWN)
{
 /* Get device name and serial */
 pDeviceName = STAR_getDeviceName(selectedDeviceID);
 pDeviceSerial = STAR_getDeviceSerialNumber(selectedDeviceID);

 /* Print selected device */
 printf("%s selected with serial number %s\n\n", pDeviceName,
 pDeviceSerial);

 /* Destroy device and serial */
 STAR_destroyString(pDeviceName);
 STAR_destroyString(pDeviceSerial);
}

STAR_destroyDeviceList(pDevices);

/* Return the selected device id */
return selectedDeviceID;
}

```

```

TRIGGER_DEVICE getTriggerDeviceType(U32 deviceType)
{
 if (deviceType == STAR_DEVICE_BRICK_MK3)
 {
 return TRIGGER_DEVICE_BRICK_MK3;
 }
 else if (deviceType == STAR_DEVICE_PXI_INTERFACE ||
 deviceType == STAR_DEVICE_PXI_RMAP)
 {
 return TRIGGER_DEVICE_PXI_IF;
 }
 else if (deviceType == STAR_DEVICE_PXI_ROUTER_12)
 {
 return TRIGGER_DEVICE_PXI_ROUTER;
 }
 else if (deviceType == STAR_DEVICE_PCIE)
 {
 return TRIGGER_DEVICE_PCIE_IF;
 }
 return TRIGGER_DEVICE_INVALID;
}

int main(int argc, char **argv)
{
 STAR_DEVICE_ID deviceId;
 MENU_CHOICE choice;
 U32 deviceType = STAR_DEVICE_UNKNOWN;

 UNREFERENCED_PARAMETER(argc);
 UNREFERENCED_PARAMETER(argv);

 /* Print the program header */
 printf("----- Trigger API Examples -----\\n");

 /* Select the device to use */
 deviceId = getTriggerDeviceID();

 /* Check that a device was selected */
 if (deviceId == 0)
 {
 printf("No devices were found.\\n");
 return 0;
 }

 /* Get type of device */
 deviceType = STAR_getDeviceType(deviceId);
 gTriggerDevice = getTriggerDeviceType(deviceType);

 /* Reset triggers */
 reset(deviceId);

 /* Loop menu */
 choice = MENU_CHOICE_INVALID;
 do
 {
 choice = getMenuChoice();
 switch (choice)
 {
 /* Time-code on external trigger (not supported on PCIe or
 * PXI Router) */
 case MENU_CHOICE_TX_TC_ON_EXT_TRIGGER:
 txTcOnExtTrigger(deviceId);
 break;
 /* Time-code on counter (not supported on PCIe) */
 case MENU_CHOICE_TX_TC_ON_COUNTER:
 txTcOnCounter(deviceId);
 break;
 /* Transmit packet on external trigger (not supported on PCIe or
 * PXI Router) */
 case MENU_CHOICE_PKT_TX_ON_EXT_TRIGGER:
 pktTxOnExtTrigger(deviceId);
 break;
 /* Transmit packet on counter */
 case MENU_CHOICE_PKT_TX_ON_COUNTER:
 pktTxOnCounter(deviceId);
 break;
 /* Transmit packet on time-code (not supported on PCIe) */
 case MENU_CHOICE_PKT_TX_ON_TC:
 pktTxOnTc(deviceId);
 break;
 /* Transmit multiple packets on external trigger
 * (not supported on PCIe or PXI Router) */
 case MENU_CHOICE_MULTIPLE_PKT_TX_ON_EXT_TRIGGER:
 multiplePktTxOnExtTrigger(deviceId);
 break;
 }
 } while (choice != MENU_CHOICE_INVALID);
}

```

```

 /* Transmit multiple packets on time-code (not supported on PCIe) */
 case MENU_CHOICE_MULTIPLE_PKT_TX_ON_TC:
 multiplePktTxOnTc(deviceId);
 break;
 /* Timed packet transmit on time-code (not supported on PCIe) */
 case MENU_CHOICE_TIMED_PKT_TX_ON_TC:
 timedPktTxOnTc(deviceId);
 break;
 /* Stop receiving for 5 seconds every 10 seconds (only
 supported on Brick Mk3) */
 case MENU_CHOICE_STOP_RECEIVING:
 stopReceiving(deviceId);
 break;
 /* Print trigger configuration */
 case MENU_CHOICE_PRINT_TRIGGER_CONF:
 printTriggerConf(deviceId);
 break;
 /* Reset trigger configuration */
 case MENU_CHOICE_RESET: resetTriggerConf(deviceId); break;

 /* Handle unused cases */
 case MENU_CHOICE_INVALID: break;
 case MENU_CHOICE_EXIT: break;
}
} while (choice != MENU_CHOICE_EXIT);

/* Reset triggers */
reset(deviceId);

return 0;
}

```

## 9.46 ui.c

This file contains UI functions that are common across the example applications.

```

#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include "star-api.h"

STAR_DEVICE_ID STAR_UI_chooseDevice(STAR_DEVICE_TYPE aDeviceTypes[],
 U32 NumRequestedTypes)
{
 STAR_DEVICE_ID* devices;
 U32 deviceCount = 0, i;
 unsigned int chosen;
 int status;

 STAR_DEVICE_ID deviceId;
 char *deviceName, s[256];

 devices = STAR_getDeviceListForTypes(aDeviceTypes, NumRequestedTypes, &
 deviceCount);

 /* If there are no devices present */
 if (!devices)
 {
 /* Return an error */
 puts("No SpaceWire devices detected!");
 return 0;
 }

 /* Display the devices detected */
 if (deviceCount == 1)
 {
 printf("One device detected:");
 }
 else
 {
 printf("%u devices detected:\n", deviceCount);
 }

 /* For each device */
 for (i = 0; i < deviceCount; i++)
 {
 /* Display its name */
 if (devices[i])
 {
 deviceName = STAR_getDeviceName(devices[i]);

```



```

 if (deviceName)
 {
 printf("\t%u - %s\n", i, deviceName);
 STAR_destroyString(deviceName);
 }
 else
 {
 printf("\t%u - Unknown SpaceWire Device\n", i);
 }
 }
 else
 {
 printf("\t%u - Unable to access device\n", i);
 }
}

/* If there's only 1 device on the list */
if (deviceCount == 1)
{
 /* Return that device */
 deviceId = devices[0];
 STAR_destroyDeviceList(devices);
 return deviceId;
}

/* Ask the user which device to use */
printf("Please select which device to use: ");
if (!fgets(s, 256, stdin))
{
 puts("No device number selected.");
 STAR_destroyDeviceList(devices);
 return 0;
}
status = sscanf(s, "%u", &chosen);
if ((!status) || (chosen > deviceCount - 1))
{
 puts("Incorrect device number selected.");
 STAR_destroyDeviceList(devices);
 return 0;
}

/* Return the chosen device */
deviceId = devices[chosen];
STAR_destroyDeviceList(devices);
return deviceId;
}

unsigned char STAR_UI_chooseChannel(STAR_DEVICE_ID deviceId)
{
 STAR_CHANNEL_MASK channelMask;
 unsigned int channelNumber;
 unsigned char channelCount;
 int status;
 unsigned char i;
 char s[256];

 /* Get the channels present on the device */
 channelMask = STAR_getDeviceChannels(deviceId);
 if (channelMask == 0 || channelMask == 1)
 {
 printf("ERROR: The device doesn't appear to have any valid channels.");
 return 0;
 }

 /* Determine the number of channels on the device, */
 /* ignoring the configuration channel */
 channelCount = 0;
 channelNumber = 0;
 for (i = 1; i < 32; i++)
 {
 if ((channelMask >> i) & 1)
 {
 channelCount++;
 channelNumber = i;
 }
 }

 /* If there's only one channel on the device, use this channel */
 if (channelCount == 1)
 {
 printf("Using channel %u as the channel, as this is the only channel on the device.",
 channelNumber);
 return (unsigned char)channelNumber;
 }

 /* Display the available channels */
 printf("Enter channel (");

```

```

 for (i = 1; i < 32; i++)
 {
 /* If the channel exists */
 if ((channelMask >> i) & 1)
 {
 printf("%d", i);
 channelCount--;
 if (channelCount == 1)
 {
 printf(" or ");
 }
 else if (channelCount > 1)
 {
 printf(", ");
 }
 else
 {
 break;
 }
 }
 }
 printf("): ");

 /* Read in the channel to use */
 if (!fgets(s, 256, stdin))
 {
 puts("\nERROR: No channel specified");
 return 0;
 }
 status = sscanf(s, "%u", &channelNumber);
 if ((!status) || (!(1 << channelNumber) & channelMask))
 {
 puts("\nERROR: The channel specified is not present");
 return 0;
 }

 return (unsigned char)channelNumber;
}

int STAR_UI_chooseDeviceAndChannel(STAR_DEVICE_TYPE aDeviceTypes[],
 U32 NumRequestedTypes,
 STAR_DEVICE_ID *pDeviceId,
 STAR_CHANNEL_ID *pChannelId,
 STAR_CHANNEL_DIRECTION direction)
{
 unsigned char channelNumber;

 *pDeviceId = STAR_UI_chooseDevice(aDeviceTypes, NumRequestedTypes);
 if (!*pDeviceId)
 {
 return 0;
 }

 channelNumber = STAR_UI_chooseChannel(*pDeviceId);
 if (!channelNumber)
 {
 return 0;
 }

 /* Open the channel */
 *pChannelId = STAR_openChannelToLocalDevice(*pDeviceId, direction,
 channelNumber, TRUE);
 if (!(*pChannelId))
 {
 printf("\nERROR: Unable to open channel\n");
 return 0;
 }

 puts("");

 return 1;
}

STAR_SPACEWIRE_ADDRESS *STAR_UI_getAddress()
{
 char s[256];
 STAR_SPACEWIRE_ADDRESS *pAddress;
 unsigned char newPath[256];
 U16 pathLen = 0;
 char *pos = (char *)s;

 /* Ask the user to enter the path to add to the front of packets */
 puts("Enter SpaceWire Address:");
 puts("(Values should be in hex, separated by a space, i.e.: 01 0f 02)");

 /* Read in the path */
 if (!fgets(s, 256, stdin))

```

```

{
 puts("No address entered");
 return NULL;
}

/* Read until end of buffer or line feed */
while ((*pos) && (pathLen < 256) && (*pos != '\n'))
{
 unsigned long value;

 errno = 0;
 value = strtoul(pos, &pos, 16);
 if (errno)
 {
 puts("Invalid value entered in address");
 return NULL;
 }

 newPath[pathLen] = (unsigned char)value;
 pathLen++;
}

/* Create a SpaceWire address from the path */
pAddress = STAR_createAddress(newPath, pathLen);

/* Return the completed address */
return pAddress;
}

```

## 9.47 user\_registers\_example.c

This example demonstrates how to use the user configurable device registers.

```

#include <stdio.h>
#include "star-api.h"
#include "cfg_api_generic.h"

STAR_DEVICE_ID chooseStarDevice()
{
 STAR_DEVICE_ID * devices;
 unsigned int devCount = 0, chosen, i, status;
 STAR_DEVICE_ID deviceId;
 char *deviceName, s[256];

 /* Get the list of devices present which can be configured */
 devices = STAR_getDeviceListForType(
 STAR_DEVICE_CONFIG_SUPPORTED,
 &devCount);

 /* If there was an error getting the list of devices */
 if (!devices)
 {
 /* Return an error */
 puts("No SpaceWire devices detected!");
 return 0;
 }

 /* Display the devices detected */
 if (devCount == 1)
 {
 puts("One SpaceWire device detected:");
 }
 else
 {
 printf("%u SpaceWire devices detected:", devCount);
 }

 /* For each device */
 for (i = 0; i < devCount; i++)
 {
 /* Display its name */
 deviceName = STAR_getDeviceName(devices[i]);
 if (deviceName)
 {
 printf("\t%u - %s\n", i, deviceName);
 STAR_destroyString(deviceName);
 }
 else
 {
 printf("\t%u - Unknown SpaceWire Device\n", i);
 }
 }
}

```

```

 }

 /* If there's only 1 device on the list */
 if (devCount == 1)
 {
 /* Return that device */
 deviceID = devices[0];
 STAR_destroyDeviceList(devices);
 return deviceID;
 }

 /* Ask the user which device to use */
 printf("\nPlease select which device to open: ");
 fgets(s, 256, stdin);
 status = sscanf(s, "%d", &chosen);
 if ((!status) || (chosen > devCount - 1))
 {
 puts("Incorrect device number selected.");
 STAR_destroyDeviceList(devices);
 return 0;
 }

 /* Return the chosen device */
 deviceID = devices[chosen];
 STAR_destroyDeviceList(devices);
 return deviceID;
}

int __cdecl main()
{
 STAR_DEVICE_ID deviceID;
 REGISTER value;

 /* Select device to configure */
 deviceID = chooseStarDevice();

 /* Set general purpose register to 0xABCD */
 CFG_setGeneralPurpose(deviceID, 0xABCD);

 /* Set device identity register to 0xCAFE */
 CFG_setNetworkIdentity(deviceID, 0xCAFE);

 /* Display user register values*/
 CFG_getGeneralPurpose(deviceID, &value);
 printf("General Purpose register value: %x", value);
 CFG_getNetworkIdentity(deviceID, &value);
 printf("\nNetwork Identity register value: %x", value);

 return 0;
}

```

## 9.48 utilities.c

This file provides common utilities used throughout the `api_test_app` example project.

```

#include "utilities.h"

#include "star-api.h"
#include "star-dundee_types.h"

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>

int readBytes(unsigned char * bytes, unsigned int maximumBytesLength,
 U16 * actualBytesLength)
{
 U16 bytesRead = 0U;
 unsigned long value;
 char currentChar;
 char * pos;

 /* Allocate string for input */
 char * input = (char *)calloc(maximumBytesLength, sizeof(char));

 /* Check that input was allocated successfully */
 if (input == NULL)
 {
 return 0;
 }
}

```

```

/* Flush the input stream */
while ((currentChar = getchar()) != '\n' && currentChar != EOF);

/* Read in the path */
if (fgets(input, maximumBytesLength, stdin) == NULL)
{
 puts("No bytes entered.");
 free(input);
 return 0;
}

/* Get start of input */
pos = (char *)input;

/* Detect no bytes */
if (*pos == '\n')
{
 puts("No bytes entered.");
 free(input);
 return 0;
}

/* Read until end of buffer or line feed */
while ((*pos != '\0') && (bytesRead < maximumBytesLength) && (*pos != '\n'))
{
 errno = 0;
 value = strtoul(pos, &pos, 16);
 if (errno != 0)
 {
 puts("Invalid value entered in bytes.");
 free(input);
 return 0;
 }

 bytes[bytesRead] = (unsigned char)value;
 bytesRead++;
}

/* Free the input string */
free(input);

/* Update actual number of bytes that were read */
*actualBytesLength = bytesRead;

return 1;
}

STAR_DEVICE_ID promptForDevice(TRANSMIT_TYPE transmitType)
{
 /* The number of devices that were found */
 U32 deviceCount = 0;

 /* Get the list of available SpaceWire devices */
 STAR_DEVICE_ID * devices = STAR_getDeviceList(&deviceCount);

 /* If at least one device is present */
 if(devices)
 {
 /* Initialise selected device index to -1 */
 int selectedDeviceIndex = -1;

 /* Define selected device */
 STAR_DEVICE_ID selectedDevice;

 /* Counter to iterate over device array */
 int index;

 /* Initialise transmit type string to null by default */
 char * transmitTypeString = NULL;

 /* Print header statement */
 printf("The following SpaceWire devices were detected:\n");

 /* For all devices found on system */
 for(index = 0; index < (int)deviceCount; index++)
 {
 /* Get current device */
 STAR_DEVICE_ID device = devices[index];

 /* Get current device name */
 char * deviceName = STAR_getDeviceName(device);

 /* Get current device serial number */
 char * deviceSerial = STAR_getDeviceSerialNumber(device);

 if ((deviceSerial != NULL) && (deviceName != NULL))

```

```

 {
 /* Print current device */
 printf("%d: %s (Serial Number: %s)\n", index + 1, deviceName,
 deviceSerial);
 }

 if (deviceName != NULL)
 {
 /* Destroy device name string */
 STAR_destroyString(deviceName);
 }

 if (deviceSerial)
 {
 /* Destroy device serial string */
 STAR_destroyString(deviceSerial);
 }
 }

 /* Switch on transmit type */
 switch(transmitType)
 {
 /* Case send */
 case TRANSMIT_TYPE_SEND:
 /* Set transmit type string to "send out of" */
 transmitTypeString = "send out of";

 break;

 /* Case receive */
 case TRANSMIT_TYPE_RECEIVE:
 /* Set transmit type string to "receive into" */
 transmitTypeString = "receive into";

 break;
 }

 /* If there is a transmit type string */
 if(transmitTypeString)
 {
 /* Ask which device to use with transmit type string */
 printf("\nPlease enter the number of the device that you would like"
 " to %s:\n", transmitTypeString);
 }
 else
 {
 /* Ask which device to use */
 printf("\nPlease enter the number of the device that you would like"
 " use:\n");
 }

 /* Get user input */
 (void)scanf("%d", &selectedDeviceIndex);

 /* While invalid user input */
 while(selectedDeviceIndex < 1 || selectedDeviceIndex > (int)deviceCount)
 {
 /* Flush input buffer */
 fflush(stdin);

 /* Ask again */
 printf("Invalid device number. Please enter a number between 1 and"
 " %u.\n", deviceCount);

 /* Get user input */
 (void)scanf("%d", &selectedDeviceIndex);
 }

 /* Decrement selected device index */
 selectedDeviceIndex--;

 /* Get selected device */
 selectedDevice = devices[selectedDeviceIndex];

 /* Destroy device list */
 STAR_destroyDeviceList(devices);

 /* Return selected device */
 return selectedDevice;
}

/* Return 0 if no selected device */
return 0;
}

STAR_DEVICE_ID getFirstDevice()
{
 /* The number of devices that were found */

```

```

unsigned int deviceCount = 0;

/* Get the list of available SpaceWire devices */
STAR_DEVICE_ID * devices = STAR_getDeviceList(&deviceCount);

/* If at least one device is present */
if(devices)
{
 /* Get first device */
 STAR_DEVICE_ID firstDevice = devices[0];

 /* Destroy device list */
 STAR_destroyDriverList(devices);

 /* Return first device */
 return firstDevice;
}

/* Return 0 if no devices exist */
return 0;
}

int isChannelValid(STAR_DEVICE_ID device, unsigned int channel)
{
 /* Get available channels */
 U32 channelMask = STAR_getDeviceChannels(device);

 /* Define counter for loop */
 unsigned int index;

 /* For all possible channels (apart from port 0) */
 for (index = 1; index < 32; index++)
 {
 /* If the channel exists */
 if ((channelMask >> index) & 1)
 {
 /* If this is the channel that we are looking for */
 if(index == channel)
 {
 /* Return true */
 return 1;
 }
 }
 }

 /* Return false if channel is invalid */
 return 0;
}

STAR_CHANNEL_ID promptForChannel(STAR_DEVICE_ID device, int *
selectedChannelNumber, TRANSMIT_TYPE transmitType)
{
 /* Define selected channel id */
 STAR_CHANNEL_ID selectedChannel;

 /* Initialise transmit type string to null by default */
 char * transmitTypeString = NULL;

 /* Initialise channel direction to inout by default */
 STAR_CHANNEL_DIRECTION channelDirection =
 STAR_CHANNEL_DIRECTION_INOUT;

 /* Initialise selected channel number to -1 */
 * selectedChannelNumber = -1;

 /* Switch on transmit type */
 switch(transmitType)
 {
 /* Case send */
 case TRANSMIT_TYPE_SEND:
 /* Set transmit type string to "send out of" */
 transmitTypeString = "send out of";

 /* Set channel direction to out */
 channelDirection = STAR_CHANNEL_DIRECTION_OUT;

 break;
 /* Case receive */
 case TRANSMIT_TYPE_RECEIVE:
 /* Set transmit type string to "receive into" */
 transmitTypeString = "receive into";

 /* Set channel direction to in */
 channelDirection = STAR_CHANNEL_DIRECTION_IN;

 break;
 }
}

```

```

/* If there is a transmit type string */
if(transmitTypeString)
{
 /* Ask user for channel number to use with transmit type string */
 printf("\nPlease enter a channel number between 1 and 31 to %s:\n",
 transmitTypeString);
}
else
{
 /* Ask user for channel number to use */
 printf("\nPlease enter a channel number between 1 and 31 to use:\n");
}

/* Get user input */
(void)scanf("%d", &selectedChannelNumber);

/* While invalid user input */
while(!isChannelValid(device, &selectedChannelNumber))
{
 /* Flush input buffer */
 fflush(stdin);

 /* Ask again */
 printf("Invalid channel number. Please enter a valid channel number "
 "between 1 and 31:\n");

 /* Get user input */
 (void)scanf("%d", &selectedChannelNumber);
}

/* Open channel to device */
selectedChannel = STAR_openChannelToLocalDevice(device, channelDirection,
 (unsigned char)* selectedChannelNumber, 1);

/* Return selected channel */
return selectedChannel;
}

void printPacketContents(unsigned char * packetContents,
 unsigned int receiveBufferLength)
{
 /* Counter for loop */
 unsigned int index;

 /* Print packet contents label */
 printf("Received packet of length %u bytes, contents:", receiveBufferLength);

 /* For all characters in receive buffer */
 for(index = 0; index < receiveBufferLength; index++)
 {
 /* Print current character with leading zero */
 printf(" %02x", packetContents[index]);
 }

 /* Print full stop */
 printf(".\n");
}

void printVersionInfo(STAR_VERSION_INFO * versionInfo)
{
 /* Get module name */
 char * moduleName = versionInfo->name;

 /* Get module author */
 char * moduleAuthor = versionInfo->author;

 /* Get major version number */
 U16 major = versionInfo->major;

 /* Get minor version number */
 U16 minor = versionInfo->minor;

 /* Get edit version number */
 U16 edit = versionInfo->edit;

 /* Get patch version number */
 U16 patch = versionInfo->patch;

 /* Print version information string */
 printf("%s %u.%u.%u.%u\n", moduleName, major, minor, edit, patch);

 /* If module has a name */
 if(strlen(moduleName) > 0)
 {
 /* Print module name */
 printf(" Module name: %s\n", moduleName);
 }
}

```



```

 }

 /* If module has an author */
 if(strlen(moduleAuthor) > 0)
 {
 /* Print module author */
 printf(" Module author: %s\n", moduleAuthor);
 }

 /* Print module author */
 printf("Module version: v%u.%u", major, minor);

 /* If there is an edit version number */
 if (edit)
 {
 /* Print edit version number */
 printf("(%u)", edit);
 }

 /* If there is a patch version number */
 if (patch)
 {
 /* Print patch version number */
 printf("p%u", patch);
 }

 /* Take new line */
 printf("\n");
}

```

## 9.49 utility.c

This file contains utility functions used by the STAR-System Test program.

```

#include <stdio.h>
#include <stdlib.h>
#ifdef __vxworks
 #include <memLib.h>
#elif defined(__APPLE__)
#else
 #include <malloc.h>
#endif
#include <string.h>
#include <errno.h>

#include "utility.h"

/*****
/*
/* Generates a 16-bit random number
/*
/*
*****/
unsigned int random16(void)
{
 unsigned int value = rand();

 if ((unsigned int)rand() > (RAND_MAX / 2))
 {
 value += RAND_MAX;
 }

 return value;
}

/*****
/*
/* Generates a 32-bit random number
/*
/*
*****/
unsigned int random32(void)
{
 unsigned int value;

 value = random16();
 value *= 0x10000U;
 value += random16();

 return value;
}

```

```

}

/*****
/*
/* Plain format-less File read/write
/* Reads a file into a buffer
/*
/*
*****/
int file_read(const char fname[], char * const pBuffer,
 unsigned int * const byteSize)
{
 FILE *infile;
 unsigned int count;

 *byteSize = 0U;
 infile = fopen(fname, "rb");
 if (infile == NULL)
 {
 printf("file_read:Trouble opening file\n");
 return 0;
 }

 /* Cycle until end of file reached: */
 while (feof(infile) == 0)
 {
 /* Attempt to read in a chunk of bytes */
 count = fread(pBuffer + *byteSize, sizeof(char), 100U, infile);
 if (ferror(infile) != 0)
 {
 printf("Read error in file_read: After reading %u\n", *byteSize);
 fclose(infile);
 return 0;
 }
 *byteSize += count;
 }

 fclose(infile);
 return 1;
}

/*****
/*
/* Plain format-less File read/write
/* Reads PART of a file into a buffer
/*
/*
*****/
int file_read_chunk(const char fname[], char * const pBuffer, const long offset,
 const unsigned int size)
{
 FILE *infile;
 unsigned int count;

 infile = fopen(fname, "rb");
 if (infile == NULL)
 {
 printf("file_read:Trouble opening file\n");
 return 0;
 }

 /* Move file steam pointer to offset */
 fseek(infile, offset, SEEK_SET);

 /* Attempt to read in a chunk of bytes */
 count = (unsigned int)fread(pBuffer, sizeof(char), size, infile);

 if ((ferror(infile) != 0) || (count < size))
 {
 printf("ERROR trying to read file %s at byte point %ld.\n", fname, offset);
 fclose(infile);
 return 0;
 }

 fclose(infile);
 return 1;
}

/*****
/*
/* Receives a fixed size message.
/* Returns FALSE if data could not be read.
/*
*****/

```

```

/*****
int file_append_chunk(const unsigned char * const pData, const long dataSize,
const char fname[])
{
 FILE *outfile;
 int returnValue = 1;

 /* Open the file to write to */
 outfile = fopen(fname, "ab");
 if (outfile == NULL)
 {
 printf("\nfile_append_chunk: Trouble opening file: %s\n", fname);
 return 0;
 }

 fwrite(pData, 1U, dataSize, outfile);

 if (ferror(outfile) != 0)
 {
 printf("\nWrite error in file_append_chunk.\n");
 returnValue = 0;
 }

 fclose(outfile);

 return returnValue;
}

/*****
/*
/* FillBuffer
/* Fills a buffer with different data types. (zeros, ones, random, count)
/*
/*
/*****
void FillBufferRandomChar(char * const pBuffer, const unsigned int size,
const int dataType)
{
 unsigned int n;
 switch (dataType)
 {
 case DATA_TYPE_0: /* Fill with zeros */
 memset(pBuffer, 0, size);
 break;

 case DATA_TYPE_1: /* Fill with ones */
 memset(pBuffer, 1, size);
 break;

 case DATA_TYPE_RANDOM: /* Fill with random values */
 for (n = 0U; n < size; n++)
 {
 *(pBuffer + n) = (char) random32();
 }
 break;

 case DATA_TYPE_COUNT: /* Fill with simple count */
 for (n = 0U; n < size; n++)
 {
 *(pBuffer + n) = (char)n;
 }
 break;

 case DATA_TYPE_NOT_COUNT: /* Fill with not of simple count */
 for (n = 0U; n < size; n++)
 {
 *(pBuffer + n) = 0xFF - (char)n;
 }
 break;
 }
}

/*****
/*
/* Buffer compare function: Used instead of memcmp() for debugging
/* Returns the number of errors found
/*
/*
/*****
unsigned long BufferCompareChar(const char * const pBuffer1,
const char * const pBuffer2, const unsigned long size)
{
 unsigned int errorCount = 0U;
 unsigned long i;

```

```

 for (i = 0U; i < size; i++)
 {
 if (*(pBuffer1 + i) != *(pBuffer2 + i))
 {
 errorCount++;
 printf("Error in byte %8lu, should be 0x%02x actually 0x%02x\n", i,
 (unsigned char)*(pBuffer1 + i), (unsigned char)*(pBuffer2 + i));
 }
 }

 return errorCount;
}

void CopyNumberToMemory(void * const pBuffer, const U32 number,
 const unsigned long len)
{
 unsigned long i;

 for (i = 0U; i < len; i++)
 {
 ((U8 *)pBuffer)[i] = (U8)((number >> ((len - i) - 1U) * 8U) & 0xffU);
 }
}

void CopyNumberFromMemory(U32 * const pNumber, void * const pBuffer,
 const unsigned long len)
{
 unsigned long i;

 *pNumber = 0U;
 for (i = 0U; i < len; i++)
 {
 *pNumber = (*pNumber << 8U) + ((U8 *)pBuffer)[i];
 }
}

int SizeOfFile(const char filePath[])
{
 FILE *infile;
 int size;

 infile = fopen(filePath, "rb");
 if (infile == NULL)
 {
 printf("Error: Trouble obtaining file size\n");
 return 0;
 }

 fseek(infile, 0, SEEK_END); /* seek to end of file */
 size = ftell(infile); /* get current file pointer */
 fseek(infile, 0, SEEK_SET); /* seek back to beginning of file */

 fclose(infile);

 return size;
}

```

## 9.50 version\_example.c

This is an example of how to obtain the version information of the RMAP Packet Library.

```

#include <stdio.h>

#include "rmap_packet_library.h"

void version_example()
{
 U32 version;
 U16 edit;
 U8 patch;

```

```

version = RMAP_GetVersion();

printf("RMAP Packet Library v%d.%02d", RMAP_GET_VERSION_MAJOR(version),
 RMAP_GET_VERSION_MINOR(version));
edit = RMAP_GET_VERSION_EDIT(version);
if (edit)
{
 printf(" edit %d", edit);
}
patch = RMAP_GET_VERSION_PATCH(version);
if (patch)
{
 printf(" patch level %d", patch);
}
puts("");
}

```

## 9.51 write\_command\_packet\_example.c

This is an example of how to build RMAP write commands.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#include "rmap_packet_library.h"

void write_command_packet_example()
{
 void *pFillPacket, *pBuildPacket, *pBuildPacketRegister;
 unsigned long fillPacketLenCalculated, fillPacketLen;
 unsigned long buildPacketLen, buildPacketRegisterLen;
 U8 pTarget[] = {0, 254};
 U8 pReply[] = {254};
 U8 pData[] = {0x12, 0x34, 0x56, 0x78};
 char status;

 /* Calculate the length of a write command packet, with 4 bytes of data */
 fillPacketLenCalculated = RMAP_CalculateWriteCommandPacketLength(
 2, 1, 4,
 1);
 printf("The write command packet will have a length of %ld bytes\n",
 fillPacketLenCalculated);

 /* Allocate memory for the packet */
 pFillPacket = malloc(fillPacketLenCalculated);
 if (!pFillPacket)
 {
 puts("Couldn't allocate the memory for the command packet");
 return;
 }

 /* Fill the memory with the packet */
 status = RMAP_FillWriteCommandPacket(pTarget, 2, pReply, 1, 1, 1, 0, 0x20,
 0, 0x106, 0, pData, 4, &fillPacketLen, NULL, 1, (U8 *)pFillPacket,
 fillPacketLenCalculated);
 if (!status)
 {
 puts("Couldn't fill the write command packet");
 free(pFillPacket);
 return;
 }

 /* Build an RMAP write command packet to write 4-bytes */
 pBuildPacket = RMAP_BuildWriteCommandPacket(pTarget, 2, pReply, 1, 1, 1, 0,
 0x20, 0, 0x106, 0, pData, 4, &buildPacketLen, NULL, 1);
 if (!pBuildPacket)
 {
 puts("Couldn't build the write command packet");
 free(pFillPacket);
 return;
 }

 /* Build an RMAP write command packet to write to a 4-byte register */
 pBuildPacketRegister = RMAP_BuildWriteRegisterPacket(pTarget, 2, pReply, 1
 ,
 1, 1, 0, 0x20, 0, 0x106, 0, 0x12345678, &buildPacketRegisterLen, NULL,
 1);
 if (!pBuildPacketRegister)

```

```

 {
 puts("Couldn't build the write register packet");
 RMAP_FreeBuffer(pBuildPacket);
 free(pFillPacket);
 return;
 }

 /* Check the lengths are all the same */
 if ((fillPacketLenCalculated != fillPacketLen) ||
 (fillPacketLen != buildPacketLen) ||
 (buildPacketLen != buildPacketRegisterLen))
 {
 puts("The lengths are different:");
 printf("\tLength calculated for packet: %ld\n",
 fillPacketLenCalculated);
 printf("\tLength of filled packet: %ld\n", fillPacketLen);
 printf("\tLength of built packet: %ld\n", buildPacketLen);
 printf("\tLength of built register packet: %ld\n", buildPacketLen);
 }
 else
 {
 puts("The packet lengths are all the same.");

 /* Check the packets are identical, despite being built differently */
 if (memcmp(pFillPacket, pBuildPacket, fillPacketLen) != 0)
 {
 puts("The filled packet is different to the built packet.");
 }
 else if (memcmp(pFillPacket, pBuildPacketRegister, fillPacketLen) != 0)
 {
 puts("The built register packet is different to the built packet and the filled packet.");
 }
 else
 {
 puts("The packets are identical.");
 }
 }

 /* Free the memory allocated for each of the packets. Note that the */
 /* memory allocated by the library should be freed by calling */
 /* RMAP_FreeBuffer. */
 RMAP_FreeBuffer(pBuildPacketRegister);
 RMAP_FreeBuffer(pBuildPacket);
 free(pFillPacket);
}

```

## 9.52 write\_reply\_packet\_example.c

This is an example of how to build RMAP write replies.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#include "rmap_packet_library.h"

void write_reply_packet_example()
{
 void *pFillPacket, *pBuildPacket;
 unsigned long fillPacketLenCalculated, fillPacketLen, buildPacketLen;
 U8 pReply[] = {254};
 U8 target = 254;
 char status;

 /* Calculate the length of a write reply packet */
 fillPacketLenCalculated = RMAP_CalculateWriteReplyPacketLength(1, 1
);
 printf("The write reply packet will have a length of %ld bytes\n",
 fillPacketLenCalculated);

 /* Allocate memory for the packet */
 pFillPacket = malloc(fillPacketLenCalculated);
 if (!pFillPacket)
 {
 puts("Couldn't allocate the memory for the reply packet");
 return;
 }

 /* Fill the memory with the packet */

```

```

status = RMAP_FillWriteReplyPacket(pReply, 1, target, 1, 0,
 RMAP_SUCCESS, 0,
 &fillPacketLen, NULL, 1, (U8 *)pFillPacket, fillPacketLenCalculated);
if (!status)
{
 puts("Couldn't fill the write reply packet");
 free(pFillPacket);
 return;
}

/* Build an RMAP write reply packet */
pBuildPacket = RMAP_BuildWriteReplyPacket(pReply, 1, target, 1, 0,
 RMAP_SUCCESS, 0, &buildPacketLen, NULL, 1);
if (!pBuildPacket)
{
 puts("Couldn't build the write reply packet");
 free(pFillPacket);
 return;
}

/* Check the lengths are all the same */
if ((fillPacketLenCalculated != fillPacketLen) ||
 (fillPacketLen != buildPacketLen))
{
 puts("The lengths are different:");
 printf("\tLength calculated for packet: %ld\n",
 fillPacketLenCalculated);
 printf("\tLength of filled packet: %ld\n", fillPacketLen);
 printf("\tLength of built packet: %ld\n", buildPacketLen);
}
else
{
 puts("The packet lengths are all the same.");

 /* Check the packets are identical, despite being built differently */
 if (memcmp(pFillPacket, pBuildPacket, fillPacketLen) != 0)
 {
 puts("The filled packet is different to the built packet.");
 }
 else
 {
 puts("The packets are identical.");
 }
}

/* Free the memory allocated for each of the packets. Note that the */
/* memory allocated by the library should be freed by calling */
/* RMAP_FreeBuffer. */
RMAP_FreeBuffer(pBuildPacket);
free(pFillPacket);
}

```





# Index

## Actions, 403

- COUNTER\_ACTION, 405
- COUNTER\_ACTION\_ALL, 405
- COUNTER\_ACTION\_COUNT, 405
- COUNTER\_ACTION\_NONE, 405
- COUNTER\_ACTION\_RELOAD, 405
- COUNTER\_ACTION\_START, 405
- COUNTER\_ACTION\_STOP, 405
- EXT\_TRIGGER\_ACTION, 405
- EXT\_TRIGGER\_ACTION\_ALL, 405
- EXT\_TRIGGER\_ACTION\_NONE, 405
- EXT\_TRIGGER\_ACTION\_OUT, 405
- PORT\_ACTION, 405
- PORT\_ACTION\_ALL, 406
- PORT\_ACTION\_DECR\_CREDIT, 406
- PORT\_ACTION\_DISABLE\_LVDS, 406
- PORT\_ACTION\_DISCONNECT, 406
- PORT\_ACTION\_ESCAPE\_ERROR, 406
- PORT\_ACTION\_INCR\_CREDIT, 406
- PORT\_ACTION\_INSERT\_FCT, 406
- PORT\_ACTION\_NONE, 406
- PORT\_ACTION\_PARITY\_ERROR, 406
- PORT\_ACTION\_STOP\_RECEPTION, 406
- PORT\_ACTION\_SUPPRESS\_FCT, 406
- PORT\_ACTION\_TRANSMIT\_PKT, 406
- TIME\_CODE\_ACTION, 406
- TIME\_CODE\_ACTION\_ALL, 406
- TIME\_CODE\_ACTION\_NONE, 406
- TIME\_CODE\_ACTION\_TX, 406
- TRIGGER\_BRICK\_MK3\_getCounterOutput↔  
Actions, 406
- TRIGGER\_BRICK\_MK3\_getExtTriggerOutput↔  
Actions, 407
- TRIGGER\_BRICK\_MK3\_getPortOutputActions,  
407
- TRIGGER\_BRICK\_MK3\_getTimeCodeOutput↔  
Actions, 408
- TRIGGER\_BRICK\_MK3\_setCounterOutput↔  
Actions, 408
- TRIGGER\_BRICK\_MK3\_setExtTriggerOutput↔  
Actions, 408
- TRIGGER\_BRICK\_MK3\_setPortOutputActions,  
409
- TRIGGER\_BRICK\_MK3\_setTimeCodeOutput↔  
Actions, 409
- TRIGGER\_PCIE\_IF\_getCounterOutputActions,  
410
- TRIGGER\_PCIE\_IF\_getPortOutputActions, 410
- TRIGGER\_PCIE\_IF\_setCounterOutputActions,

411

- TRIGGER\_PCIE\_IF\_setPortOutputActions, 411
- TRIGGER\_PXI\_IF\_getCounterOutputActions, 412
- TRIGGER\_PXI\_IF\_getExtTriggerOutputActions,  
412
- TRIGGER\_PXI\_IF\_getPortOutputActions, 412
- TRIGGER\_PXI\_IF\_getTimeCodeOutputActions,  
413
- TRIGGER\_PXI\_IF\_setCounterOutputActions, 413
- TRIGGER\_PXI\_IF\_setExtTriggerOutputActions,  
414
- TRIGGER\_PXI\_IF\_setPortOutputActions, 414
- TRIGGER\_PXI\_IF\_setTimeCodeOutputActions,  
415
- TRIGGER\_PXI\_ROUTER\_getCounterOutput↔  
Actions, 415
- TRIGGER\_PXI\_ROUTER\_getPortOutputActions,  
415
- TRIGGER\_PXI\_ROUTER\_getTimeCodeOutput↔  
Actions, 417
- TRIGGER\_PXI\_ROUTER\_setCounterOutput↔  
Actions, 417
- TRIGGER\_PXI\_ROUTER\_setPortOutputActions,  
418
- TRIGGER\_PXI\_ROUTER\_setTimeCodeOutput↔  
Actions, 418

Brick Mk2 Configuration API, 566

Brick Mk3 Configuration API, 567

CFG\_API\_MK2\_MODULE\_NAME

cfg\_api\_mk2.h, 739

CFG\_API\_PCI\_MK2\_MODULE\_NAME

cfg\_api\_pci\_mk2.h, 741

CFG\_BRICK\_MK2\_disableIdentifySourceOnPort  
Interface Mode, 300

CFG\_BRICK\_MK2\_disableInterfaceModeOnPort  
Interface Mode, 301

CFG\_BRICK\_MK2\_disableSpeedChangeEventsOnPort  
Events, 307

CFG\_BRICK\_MK2\_disableStateChangeEventsOnPort  
Events, 309

CFG\_BRICK\_MK2\_enableIdentifySourceOnPort  
Interface Mode, 301

CFG\_BRICK\_MK2\_enableInterfaceModeOnPort  
Interface Mode, 303

CFG\_BRICK\_MK2\_enableSpeedChangeEventsOnPort  
Events, 309

CFG\_BRICK\_MK2\_enableStateChangeEventsOnPort  
Events, 310

- CFG\_BRICK\_MK2\_getIdentifySourceOnPortEnabled
  - Interface Mode, [303](#)
- CFG\_BRICK\_MK2\_getInterfaceModeOnPortEnabled
  - Interface Mode, [304](#)
- CFG\_BRICK\_MK2\_getLinkClockFrequency
  - Link Speed, [298](#)
- CFG\_BRICK\_MK2\_getPortRoutingAddress
  - Interface Mode, [304](#)
- CFG\_BRICK\_MK2\_getSpeedChangeEventsOnPort↔
  - Enabled
  - Events, [310](#)
- CFG\_BRICK\_MK2\_getStateChangeEventsOnPort↔
  - Enabled
  - Events, [311](#)
- CFG\_BRICK\_MK2\_injectErrors
  - Error Injection, [306](#)
- CFG\_BRICK\_MK2\_setLinkClockFrequency
  - Link Speed, [298](#)
- CFG\_BRICK\_MK2\_setPortRoutingAddress
  - Interface Mode, [305](#)
- CFG\_BRICK\_MK3\_disableRxTimestampEventsOnPort
  - Timestamping, [318](#)
- CFG\_BRICK\_MK3\_enableRxTimestampEventsOnPort
  - Timestamping, [319](#)
- CFG\_BRICK\_MK3\_getBaseTransmitClock
  - Link Speed, [313](#)
- CFG\_BRICK\_MK3\_getPulseGeneratorFrequency
  - Timestamping, [319](#)
- CFG\_BRICK\_MK3\_getRxTimestampEventsOnPort↔
  - Enabled
  - Timestamping, [320](#)
- CFG\_BRICK\_MK3\_getTimestampMethod
  - Timestamping, [320](#)
- CFG\_BRICK\_MK3\_getTimestampValue
  - Timestamping, [321](#)
- CFG\_BRICK\_MK3\_getTransmitClock
  - Link Speed, [314](#)
- CFG\_BRICK\_MK3\_setBaseTransmitClock
  - Link Speed, [314](#)
- CFG\_BRICK\_MK3\_setPulseGeneratorFrequency
  - Timestamping, [321](#)
- CFG\_BRICK\_MK3\_setTimestampMethod
  - Timestamping, [322](#)
- CFG\_BRICK\_MK3\_setTimestampValue
  - Timestamping, [322](#)
- CFG\_BRICK\_MK3\_setTransmitClock
  - Link Speed, [315](#)
- CFG\_FPGAInfoToString
  - Hardware, [611](#)
- CFG\_INFO\_ID\_TYPE
  - types.h, [792](#)
- CFG\_MK2\_disableExternalTimeCodeSelection
  - Time-code Master, [515](#)
- CFG\_MK2\_disableIdentifySource
  - Interface Mode, [525](#)
- CFG\_MK2\_disableIdentifySourceOnPort
  - Interface Mode, [525](#)
- CFG\_MK2\_disableInterfaceMode
  - Interface Mode, [525](#)
- CFG\_MK2\_disableInterfaceModeOnPort
  - Interface Mode, [526](#)
- CFG\_MK2\_disableSpeedChangeEventsOnPort
  - Events, [501](#)
- CFG\_MK2\_disableStateChangeEventsOnPort
  - Events, [501](#)
- CFG\_MK2\_disableTimeCodeCounterBypassMode
  - Time-code Master, [516](#)
- CFG\_MK2\_disableTimeCodeEventsOnPort
  - Events, [502](#)
- CFG\_MK2\_disableTimeCodeMaster
  - Time-code Master, [516](#)
- CFG\_MK2\_enableExternalTimeCodeSelection
  - Time-code Master, [517](#)
- CFG\_MK2\_enableIdentifySource
  - Interface Mode, [526](#)
- CFG\_MK2\_enableIdentifySourceOnPort
  - Interface Mode, [527](#)
- CFG\_MK2\_enableInterfaceMode
  - Interface Mode, [527](#)
- CFG\_MK2\_enableInterfaceModeOnPort
  - Interface Mode, [527](#)
- CFG\_MK2\_enableSpeedChangeEventsOnPort
  - Events, [502](#)
- CFG\_MK2\_enableStateChangeEventsOnPort
  - Events, [502](#)
- CFG\_MK2\_enableTimeCodeCounterBypassMode
  - Time-code Master, [517](#)
- CFG\_MK2\_enableTimeCodeEventsOnPort
  - Events, [503](#)
- CFG\_MK2\_enableTimeCodeMaster
  - Time-code Master, [517](#)
- CFG\_MK2\_getBaseTransmitClock
  - Link Speed, [507](#)
- CFG\_MK2\_getDeviceClockRate
  - Periodic Actions, [534](#)
- CFG\_MK2\_getExternalTimeCodeSelectionEnabled
  - Time-code Master, [518](#)
- CFG\_MK2\_getHardwareInfo
  - Device Information, [522](#)
- CFG\_MK2\_getIdentifySourceEnabled
  - Interface Mode, [528](#)
- CFG\_MK2\_getIdentifySourceOnPortEnabled
  - Interface Mode, [528](#)
- CFG\_MK2\_getInterfaceModeEnabled
  - Interface Mode, [529](#)
- CFG\_MK2\_getInterfaceModeOnPortEnabled
  - Interface Mode, [529](#)
- CFG\_MK2\_getLinkRateDivider
  - Link Speed, [508](#)
- CFG\_MK2\_getMeasuredLinkSpeed
  - Measured Link Speed, [512](#)
- CFG\_MK2\_getPortRoutingAddress
  - Interface Mode, [530](#)
- CFG\_MK2\_getSpeedChangeEventsOnPortEnabled
  - Events, [503](#)
- CFG\_MK2\_getStateChangeEventsOnPortEnabled

- Events, [504](#)
- CFG\_MK2\_getTimeCodeCounterBypassModeEnabled
  - Time-code Master, [518](#)
- CFG\_MK2\_getTimeCodeDistributionPorts
  - User Configurable Registers, [593](#)
- CFG\_MK2\_getTimeCodeEventsOnPortEnabled
  - Events, [504](#)
- CFG\_MK2\_getTimeCodeFlagMode
  - Router Control and Status Configuration, [602](#)
- CFG\_MK2\_getTimeCodeMasterEnabled
  - Time-code Master, [519](#)
- CFG\_MK2\_getTimeCodePeriod
  - Time-code Master, [519](#)
- CFG\_MK2\_getTransmitClock
  - Link Speed, [508](#)
- CFG\_MK2\_hardwareInfoToString
  - Device Information, [523](#)
- CFG\_MK2\_identify
  - Device Information, [523](#)
- CFG\_MK2\_injectError
  - Error Injection, [533](#)
- CFG\_MK2\_setBaseTransmitClock
  - Link Speed, [509](#)
- CFG\_MK2\_setLinkRateDivider
  - Link Speed, [510](#)
- CFG\_MK2\_setPortRoutingAddress
  - Interface Mode, [530](#)
- CFG\_MK2\_setTimeCodeDistributionPorts
  - User Configurable Registers, [594](#)
- CFG\_MK2\_setTimeCodeFlagMode
  - Router Control and Status Configuration, [602](#)
- CFG\_MK2\_setTimeCodePeriod
  - Time-code Master, [520](#)
- CFG\_MK2\_setTransmitClock
  - Link Speed, [510](#)
- CFG\_MK2\_startPeriodicAction
  - Periodic Actions, [534](#)
- CFG\_MK2\_stopPeriodicAction
  - Periodic Actions, [535](#)
- CFG\_PCIMK2\_getLinkClockFrequency
  - Link Speed, [295](#)
- CFG\_PCIMK2\_setLinkClockFrequency
  - Link Speed, [295](#)
- CFG\_PXI\_disableIdentifySourceOnPort
  - Interface Mode, [330](#)
- CFG\_PXI\_disableInterfaceModeOnPort
  - Interface Mode, [331](#)
- CFG\_PXI\_disableSpeedChangeEventsOnPort
  - Events, [335](#)
- CFG\_PXI\_disableStateChangeEventsOnPort
  - Events, [336](#)
- CFG\_PXI\_disableTimeCodeEventsOnPort
  - Events, [336](#)
- CFG\_PXI\_enableIdentifySourceOnPort
  - Interface Mode, [331](#)
- CFG\_PXI\_enableInterfaceModeOnPort
  - Interface Mode, [331](#)
- CFG\_PXI\_enableSpeedChangeEventsOnPort
  - Events, [337](#)
- CFG\_PXI\_enableStateChangeEventsOnPort
  - Events, [337](#)
- CFG\_PXI\_enableTimeCodeEventsOnPort
  - Events, [338](#)
- CFG\_PXI\_getBaseTransmitClock
  - Link Speed, [324](#)
- CFG\_PXI\_getIdentifySourceOnPortEnabled
  - Interface Mode, [332](#)
- CFG\_PXI\_getInterfaceModeOnPortEnabled
  - Interface Mode, [332](#)
- CFG\_PXI\_getLinkRateDivider
  - Link Speed, [325](#)
- CFG\_PXI\_getPortRoutingAddress
  - Interface Mode, [333](#)
- CFG\_PXI\_getSpeedChangeEventsOnPortEnabled
  - Events, [338](#)
- CFG\_PXI\_getStateChangeEventsOnPortEnabled
  - Events, [338](#)
- CFG\_PXI\_getTimeCodeEventsOnPortEnabled
  - Events, [339](#)
- CFG\_PXI\_getTransmitClock
  - Link Speed, [325](#)
- CFG\_PXI\_injectError
  - Error Injection, [329](#)
- CFG\_PXI\_setBaseTransmitClock
  - Link Speed, [326](#)
- CFG\_PXI\_setLinkRateDivider
  - Link Speed, [326](#)
- CFG\_PXI\_setPortRoutingAddress
  - Interface Mode, [333](#)
- CFG\_PXI\_setTransmitClock
  - Link Speed, [327](#)
- CFG\_ROUTER\_MK2S\_disablePrecisionTransmitRate
  - Link Speed, [496](#)
- CFG\_ROUTER\_MK2S\_enablePrecisionTransmitRate
  - Link Speed, [497](#)
- CFG\_ROUTER\_MK2S\_getPrecisionTransmitRate
  - Link Speed, [497](#)
- CFG\_ROUTER\_MK2S\_getPrecisionTransmitRate↔
  - Enabled
  - Link Speed, [498](#)
- CFG\_ROUTER\_MK2S\_getPrecisionTransmitRateInUse
  - Link Speed, [498](#)
- CFG\_ROUTER\_MK2S\_setPrecisionTransmitRate
  - Link Speed, [498](#)
- CFG\_ROUTER\_clearPortErrors
  - Port Status and Control, [583](#)
- CFG\_ROUTER\_getConfigPortErrors
  - Port Status and Control, [583](#)
- CFG\_ROUTER\_getDeviceIdentificationInfo
  - Device Information, [573](#)
- CFG\_ROUTER\_getDeviceManufacturerAsString
  - Device Information, [573](#)
- CFG\_ROUTER\_getDeviceTypeAsString
  - Device Information, [574](#)
- CFG\_ROUTER\_getExternalPortErrors
  - Port Status and Control, [584](#)

- CFG\_ROUTER\_getExternalPortStatus
  - Port Status and Control, [584](#)
- CFG\_ROUTER\_getGeneralPurpose
  - User Configurable Registers, [594](#)
- CFG\_ROUTER\_getNetworkDiscoveryInfo
  - Device Information, [574](#)
- CFG\_ROUTER\_getNetworkIdentity
  - User Configurable Registers, [595](#)
- CFG\_ROUTER\_getPortConnection
  - Port Status and Control, [586](#)
- CFG\_ROUTER\_getPortStatusControl
  - Port Status and Control, [586](#)
- CFG\_ROUTER\_getPortType
  - Port Status and Control, [587](#)
- CFG\_ROUTER\_getRouterGlobalSettings
  - Router Control and Status Configuration, [602](#)
- CFG\_ROUTER\_getRoutingTableEntry
  - Routing Table, [591](#)
- CFG\_ROUTER\_getSpaceWireLinkErrors
  - Port Status and Control, [587](#)
- CFG\_ROUTER\_getSpaceWireLinkStatus
  - Port Status and Control, [588](#)
- CFG\_ROUTER\_getTimeCodeDistributionPorts
  - User Configurable Registers, [595](#)
- CFG\_ROUTER\_getTimeCodeFlagMode
  - Router Control and Status Configuration, [604](#)
- CFG\_ROUTER\_getTimeCodeValue
  - Router Control and Status Configuration, [604](#)
- CFG\_ROUTER\_setGeneralPurpose
  - User Configurable Registers, [596](#)
- CFG\_ROUTER\_setNetworkIdentity
  - User Configurable Registers, [596](#)
- CFG\_ROUTER\_setRouterGlobalSettings
  - Router Control and Status Configuration, [605](#)
- CFG\_ROUTER\_setRoutingTableEntry
  - Routing Table, [592](#)
- CFG\_ROUTER\_setSpaceWireLinkStatus
  - Port Status and Control, [588](#)
- CFG\_ROUTER\_setTimeCodeDistributionPorts
  - User Configurable Registers, [596](#)
- CFG\_ROUTER\_setTimeCodeFlagMode
  - Router Control and Status Configuration, [605](#)
- CFG\_ROUTER\_startLink
  - Port Status and Control, [589](#)
- CFG\_ROUTER\_stopLink
  - Port Status and Control, [589](#)
- CFG\_STATUS\_NOT\_COMPATIBLE\_WITH\_THIS\_D↔EVICES\_FPGA\_VERSION
  - Defines, [668](#)
- CFG\_STATUS\_SUCCESS
  - Defines, [668](#)
- CFG\_clearPortErrors
  - Port Status and Control, [615](#)
- CFG\_disableExternalTimeCodeSelection
  - Time-codes, [637](#)
- CFG\_disableIdentifySource
  - Interface Mode, [629](#)
- CFG\_disableIdentifySourceOnPort
  - Interface Mode, [629](#)
- CFG\_disableInterfaceMode
  - Interface Mode, [629](#)
- CFG\_disableInterfaceModeOnPort
  - Interface Mode, [630](#)
- CFG\_disablePrecisionTransmitRate
  - Precision Links, [652](#)
- CFG\_disableRxTimestampEventsOnPort
  - Timestamping, [658](#)
- CFG\_disableSpeedChangeEventsOnPort
  - Events, [643](#)
- CFG\_disableStateChangeEventsOnPort
  - Events, [644](#)
- CFG\_disableTimeCodeCounterBypassMode
  - Time-codes, [638](#)
- CFG\_disableTimeCodeEventsOnPort
  - Events, [644](#)
- CFG\_disableTimeCodeMaster
  - Time-codes, [638](#)
- CFG\_enableExternalTimeCodeSelection
  - Time-codes, [639](#)
- CFG\_enableIdentifySource
  - Interface Mode, [630](#)
- CFG\_enableIdentifySourceOnPort
  - Interface Mode, [631](#)
- CFG\_enableInterfaceMode
  - Interface Mode, [631](#)
- CFG\_enableInterfaceModeOnPort
  - Interface Mode, [631](#)
- CFG\_enablePrecisionTransmitRate
  - Precision Links, [653](#)
- CFG\_enableRxTimestampEventsOnPort
  - Timestamping, [659](#)
- CFG\_enableSpeedChangeEventsOnPort
  - Events, [645](#)
- CFG\_enableStateChangeEventsOnPort
  - Events, [645](#)
- CFG\_enableTimeCodeCounterBypassMode
  - Time-codes, [639](#)
- CFG\_enableTimeCodeEventsOnPort
  - Events, [646](#)
- CFG\_enableTimeCodeMaster
  - Time-codes, [639](#)
- CFG\_getConfigPortErrors
  - Port Status and Control, [615](#)
- CFG\_getDeviceClockRate
  - Periodic, [664](#)
- CFG\_getDeviceIdentificationInfo
  - Device Identifier, [622](#)
- CFG\_getDeviceManufacturerAsString
  - Device Identifier, [623](#)
- CFG\_getDeviceTypeAsString
  - Device Identifier, [623](#)
- CFG\_getExternalPortErrors
  - Port Status and Control, [616](#)
- CFG\_getExternalPortStatus
  - Port Status and Control, [616](#)
- CFG\_getExternalTimeCodeSelectionEnabled

- Time-codes, 640
- CFG\_getFPGAInfo
  - Hardware, 612
- CFG\_getGeneralPurpose
  - User, 606
- CFG\_getIdentifySourceEnabled
  - Interface Mode, 632
- CFG\_getIdentifySourceOnPortEnabled
  - Interface Mode, 632
- CFG\_getInterfaceModeEnabled
  - Interface Mode, 633
- CFG\_getInterfaceModeOnPortEnabled
  - Interface Mode, 633
- CFG\_getMeasuredLinkSpeed
  - Link Speed, 649
- CFG\_getNetworkDiscoveryInfo
  - Device Identifier, 623
- CFG\_getNetworkIdentity
  - User, 607
- CFG\_getPortConnection
  - Port Status and Control, 616
- CFG\_getPortRoutingAddress
  - Interface Mode, 634
- CFG\_getPortStatusControl
  - Port Status and Control, 618
- CFG\_getPortType
  - Port Status and Control, 618
- CFG\_getPrecisionTransmitRate
  - Precision Links, 653
- CFG\_getPrecisionTransmitRateEnabled
  - Precision Links, 654
- CFG\_getPrecisionTransmitRateInUse
  - Precision Links, 654
- CFG\_getPulseGeneratorFrequency
  - Timestamping, 659
- CFG\_getRouterGlobalSettings
  - cfg\_api\_generic.c, 723
  - cfg\_api\_generic.h, 733
- CFG\_getRoutingTableEntry
  - Group Adaptive Routing, 625
- CFG\_getRxTimestampEventsOnPortEnabled
  - Timestamping, 660
- CFG\_getSpaceWireLinkErrors
  - Port Status and Control, 619
- CFG\_getSpaceWireLinkStatus
  - Port Status and Control, 619
- CFG\_getSpeedChangeEventsOnPortEnabled
  - Events, 646
- CFG\_getStateChangeEventsOnPortEnabled
  - Events, 647
- CFG\_getTimeCodeCounterBypassModeEnabled
  - Time-codes, 640
- CFG\_getTimeCodeDistributionPorts
  - User, 607
- CFG\_getTimeCodeEventsOnPortEnabled
  - Events, 647
- CFG\_getTimeCodeFlagMode
  - cfg\_api\_generic.c, 724
  - cfg\_api\_generic.h, 734
- CFG\_getTimeCodeMasterEnabled
  - Time-codes, 641
- CFG\_getTimeCodePeriod
  - Time-codes, 641
- CFG\_getTimeCodeValue
  - cfg\_api\_generic.c, 724
  - cfg\_api\_generic.h, 734
- CFG\_getTimestampMethod
  - Timestamping, 660
- CFG\_getTimestampValue
  - Timestamping, 661
- CFG\_getTransmitSignallingRate
  - Links, 650
- CFG\_identify
  - Hardware, 612
- CFG\_injectError
  - Error Injection, 656
- CFG\_injectErrors
  - Error Injection, 657
- CFG\_setGeneralPurpose
  - User, 608
- CFG\_setNetworkIdentity
  - User, 608
- CFG\_setPortRoutingAddress
  - Interface Mode, 634
- CFG\_setPrecisionTransmitRate
  - Precision Links, 655
- CFG\_setPulseGeneratorFrequency
  - Timestamping, 661
- CFG\_setRouterGlobalSettings
  - cfg\_api\_generic.c, 725
  - cfg\_api\_generic.h, 734
- CFG\_setRoutingTableEntry
  - Group Adaptive Routing, 625
- CFG\_setSpaceWireLinkStatus
  - Port Status and Control, 620
- CFG\_setTimeCodeDistributionPorts
  - User, 609
- CFG\_setTimeCodeFlagMode
  - cfg\_api\_generic.c, 725
  - cfg\_api\_generic.h, 735
- CFG\_setTimeCodePeriod
  - Time-codes, 642
- CFG\_setTimestampMethod
  - Timestamping, 662
- CFG\_setTimestampValue
  - Timestamping, 662
- CFG\_setTransmitSignallingRate
  - Links, 651
- CFG\_startLink
  - Port Status and Control, 620
- CFG\_startPeriodicAction
  - Periodic, 664
- CFG\_stopLink
  - Port Status and Control, 621
- CFG\_stopPeriodicAction
  - Periodic, 665



- CFG\_updateDeviceInfo
  - User, 609
- COUNTER\_ACTION
  - Actions, 405
- COUNTER\_ACTION\_ALL
  - Actions, 405
- COUNTER\_ACTION\_COUNT
  - Actions, 405
- COUNTER\_ACTION\_MASK
  - Events, 383
- COUNTER\_ACTION\_NONE
  - Actions, 405
- COUNTER\_ACTION\_RELOAD
  - Actions, 405
- COUNTER\_ACTION\_START
  - Actions, 405
- COUNTER\_ACTION\_STOP
  - Actions, 405
- COUNTER\_EVENT
  - Events, 384
- COUNTER\_EVENT\_ALL
  - Events, 384
- COUNTER\_EVENT\_COUNT
  - Events, 384
- COUNTER\_EVENT\_MASK
  - Events, 383
- COUNTER\_EVENT\_NONE
  - Events, 384
- COUNTER\_EVENT\_RELOAD
  - Events, 384
- COUNTER\_EVENT\_ZERO
  - Events, 384
- COUNTER\_EVENT\_ZERO\_SINGLE
  - Events, 384
- cfg\_api\_brick\_mk2.h, 713
- cfg\_api\_brick\_mk2\_types.h, 714
- cfg\_api\_brick\_mk3.h, 716
- cfg\_api\_brick\_mk3\_types.h, 717
- cfg\_api\_generic.c, 718
  - CFG\_getRouterGlobalSettings, 723
  - CFG\_getTimeCodeFlagMode, 724
  - CFG\_getTimeCodeValue, 724
  - CFG\_setRouterGlobalSettings, 725
  - CFG\_setTimeCodeFlagMode, 725
- cfg\_api\_generic.h, 726
  - CFG\_getRouterGlobalSettings, 733
  - CFG\_getTimeCodeFlagMode, 734
  - CFG\_getTimeCodeValue, 734
  - CFG\_setRouterGlobalSettings, 734
  - CFG\_setTimeCodeFlagMode, 735
- cfg\_api\_generic\_common.h, 735
- cfg\_api\_mk2.h, 736
  - CFG\_API\_MK2\_MODULE\_NAME, 739
- cfg\_api\_mk2\_types.h, 739
  - SPW\_ACTION, 740
  - SPW\_TRANSMIT\_PACKET, 740
- cfg\_api\_pci\_mk2.h, 740
  - CFG\_API\_PCI\_MK2\_MODULE\_NAME, 741
- cfg\_api\_pci\_mk2\_types.h, 741
- cfg\_api\_pxi.h, 742
- cfg\_api\_remote.h, 743
- cfg\_api\_router.h, 744
- cfg\_api\_router\_mk2s.h, 746
- cfg\_api\_router\_types.h, 747
- Channel Management, 235
  - STAR\_CHANNEL\_DIRECTION, 237
  - STAR\_CHANNEL\_DIRECTION\_IN, 237
  - STAR\_CHANNEL\_DIRECTION\_INOUT, 237
  - STAR\_CHANNEL\_DIRECTION\_OUT, 237
  - STAR\_CHANNEL\_ID, 236
  - STAR\_CHANNEL\_LISTENER\_ID, 236
  - STAR\_CHANNEL\_TYPE, 237
  - STAR\_CHANNEL\_TYPE\_APPLICATION, 237
  - STAR\_CHANNEL\_TYPE\_DEVICE, 237
  - STAR\_CHANNEL\_TYPE\_NOT\_OPEN, 237
  - STAR\_ChannelEventFunc, 236
  - STAR\_closeChannel, 237
  - STAR\_getChannelDirection, 238
  - STAR\_openChannelBetweenLocalDevices, 238
  - STAR\_openChannelToLocalDevice, 238
  - STAR\_openChannelToLocalDeviceEx, 239
  - STAR\_registerChannelListener, 240
  - STAR\_registerChannelListenerForDevice, 240
  - STAR\_registerChannelListenerForDriver, 242
  - STAR\_unregisterChannelListener, 242
- common.h, 749
  - STAR\_API\_CC, 749
  - STAR\_API\_MODULE\_NAME, 749
- Configuration, 346, 420
  - IF\_MODE, 349
  - IF\_MODE\_RMAP\_DISABLED, 349
  - IF\_MODE\_RMAP\_ENABLED, 349
  - RMAP\_TARGET\_PXI\_IF\_getAddressOffset, 350
  - RMAP\_TARGET\_PXI\_IF\_getAuthCommands, 350
  - RMAP\_TARGET\_PXI\_IF\_getAuthControlMode, 350
  - RMAP\_TARGET\_PXI\_IF\_getAuthKeyRange, 351
  - RMAP\_TARGET\_PXI\_IF\_getAuthLogical↔
    - AddressRange, 351
  - RMAP\_TARGET\_PXI\_IF\_getAuthMemory↔
    - AddressRange, 352
  - RMAP\_TARGET\_PXI\_IF\_getAuthProtocolId, 352
  - RMAP\_TARGET\_PXI\_IF\_getInterfaceMode, 352
  - RMAP\_TARGET\_PXI\_IF\_getStatus, 354
  - RMAP\_TARGET\_PXI\_IF\_readMemory, 354
  - RMAP\_TARGET\_PXI\_IF\_setAddressOffset, 355
  - RMAP\_TARGET\_PXI\_IF\_setAuthCommands, 355
  - RMAP\_TARGET\_PXI\_IF\_setAuthControlMode, 356
  - RMAP\_TARGET\_PXI\_IF\_setAuthKeyRange, 356
  - RMAP\_TARGET\_PXI\_IF\_setAuthLogicalAddress↔
    - Range, 357
  - RMAP\_TARGET\_PXI\_IF\_setAuthMemory↔
    - AddressRange, 357
  - RMAP\_TARGET\_PXI\_IF\_setAuthProtocolId, 357
  - RMAP\_TARGET\_PXI\_IF\_setInterfaceMode, 358

- RMAP\_TARGET\_PXI\_IF\_writeMemory, 358
- TARGET\_AUTH\_CMD, 349
- TARGET\_AUTH\_CMD\_MASK, 349
- TARGET\_AUTH\_MODE, 349
- TARGET\_AUTH\_MODE\_AUTOMATIC, 349
- TARGET\_AUTH\_MODE\_MANUAL, 349
- TARGET\_STATUS, 349
- TRIGGER\_BRICK\_MK3\_disableCounterAuto↔
  - Reload, 428
- TRIGGER\_BRICK\_MK3\_disableCounterLoad↔
  - Zero, 429
- TRIGGER\_BRICK\_MK3\_disableCounterStart↔
  - Mode, 429
- TRIGGER\_BRICK\_MK3\_disableCounterStart↔
  - StopMode, 430
- TRIGGER\_BRICK\_MK3\_disableCounterTrigger↔
  - Count, 430
- TRIGGER\_BRICK\_MK3\_disableExtTriggerEdge↔
  - DetectMode, 430
- TRIGGER\_BRICK\_MK3\_disableExtTriggerInvert, 431
- TRIGGER\_BRICK\_MK3\_disableExtTriggerOutput, 431
- TRIGGER\_BRICK\_MK3\_disablePortTransmit↔
  - PacketMode, 432
- TRIGGER\_BRICK\_MK3\_disableTrigger, 432
- TRIGGER\_BRICK\_MK3\_enableCounterAuto↔
  - Reload, 433
- TRIGGER\_BRICK\_MK3\_enableCounterLoadZero, 433
- TRIGGER\_BRICK\_MK3\_enableCounterStart↔
  - Mode, 433
- TRIGGER\_BRICK\_MK3\_enableCounterStart↔
  - StopMode, 435
- TRIGGER\_BRICK\_MK3\_enableCounterTrigger↔
  - Count, 435
- TRIGGER\_BRICK\_MK3\_enableExtTriggerEdge↔
  - DetectMode, 436
- TRIGGER\_BRICK\_MK3\_enableExtTriggerInvert, 436
- TRIGGER\_BRICK\_MK3\_enableExtTriggerOutput, 437
- TRIGGER\_BRICK\_MK3\_enablePortTransmit↔
  - PacketMode, 437
- TRIGGER\_BRICK\_MK3\_enableTrigger, 438
- TRIGGER\_BRICK\_MK3\_forceCounterReload, 438
- TRIGGER\_BRICK\_MK3\_forceTrigger, 439
- TRIGGER\_BRICK\_MK3\_getCounterAutoReload↔
  - Enabled, 439
- TRIGGER\_BRICK\_MK3\_getCounterLoadZero↔
  - Enabled, 439
- TRIGGER\_BRICK\_MK3\_getCounterReloadValue, 440
- TRIGGER\_BRICK\_MK3\_getCounterStartMode↔
  - Enabled, 440
- TRIGGER\_BRICK\_MK3\_getCounterStartStop↔
  - ModeEnabled, 441
- TRIGGER\_BRICK\_MK3\_getCounterTrigger↔
  - CountEnabled, 441
- TRIGGER\_BRICK\_MK3\_getCounterValue, 441
- TRIGGER\_BRICK\_MK3\_getExtTriggerEdge↔
  - DetectModeEnabled, 442
- TRIGGER\_BRICK\_MK3\_getExtTriggerExtend, 442
- TRIGGER\_BRICK\_MK3\_getExtTriggerInvert↔
  - Enabled, 443
- TRIGGER\_BRICK\_MK3\_getExtTriggerOutput↔
  - Enabled, 443
- TRIGGER\_BRICK\_MK3\_getPortTransmitPacket↔
  - ModeEnabled, 443
- TRIGGER\_BRICK\_MK3\_getTriggerAndMask, 444
- TRIGGER\_BRICK\_MK3\_getTriggerEnabled, 444
- TRIGGER\_BRICK\_MK3\_getTriggerInputMode, 445
- TRIGGER\_BRICK\_MK3\_getTriggerInvertMask, 445
- TRIGGER\_BRICK\_MK3\_setCounterReloadValue, 445
- TRIGGER\_BRICK\_MK3\_setExtTriggerExtend, 447
- TRIGGER\_BRICK\_MK3\_setTriggerAndMask, 447
- TRIGGER\_BRICK\_MK3\_setTriggerInputMode, 448
- TRIGGER\_BRICK\_MK3\_setTriggerInvertMask, 448
- TRIGGER\_INPUT\_MODE, 428
- TRIGGER\_INPUT\_MODE\_AND, 428
- TRIGGER\_INPUT\_MODE\_OR, 428
- TRIGGER\_PCIE\_IF\_disableCounterAutoReload, 449
- TRIGGER\_PCIE\_IF\_disableCounterLoadZero, 449
- TRIGGER\_PCIE\_IF\_disableCounterStartMode, 450
- TRIGGER\_PCIE\_IF\_disableCounterStartStop↔
  - Mode, 450
- TRIGGER\_PCIE\_IF\_disableCounterTriggerCount, 450
- TRIGGER\_PCIE\_IF\_disableTrigger, 452
- TRIGGER\_PCIE\_IF\_enableCounterAutoReload, 452
- TRIGGER\_PCIE\_IF\_enableCounterLoadZero, 453
- TRIGGER\_PCIE\_IF\_enableCounterStartMode, 453
- TRIGGER\_PCIE\_IF\_enableCounterStartStop↔
  - Mode, 453
- TRIGGER\_PCIE\_IF\_enableCounterTriggerCount, 454
- TRIGGER\_PCIE\_IF\_enableTrigger, 454
- TRIGGER\_PCIE\_IF\_forceCounterReload, 455
- TRIGGER\_PCIE\_IF\_forceTrigger, 455
- TRIGGER\_PCIE\_IF\_getCounterAutoReload↔
  - Enabled, 455
- TRIGGER\_PCIE\_IF\_getCounterLoadZero↔
  - Enabled, 456
- TRIGGER\_PCIE\_IF\_getCounterReloadValue, 456
- TRIGGER\_PCIE\_IF\_getCounterStartMode↔

- Enabled, [457](#)
- TRIGGER\_PCIE\_IF\_getCounterStartStopMode↔  
Enabled, [457](#)
- TRIGGER\_PCIE\_IF\_getCounterTriggerCount↔  
Enabled, [457](#)
- TRIGGER\_PCIE\_IF\_getCounterValue, [458](#)
- TRIGGER\_PCIE\_IF\_getTriggerAndMask, [458](#)
- TRIGGER\_PCIE\_IF\_getTriggerEnabled, [459](#)
- TRIGGER\_PCIE\_IF\_getTriggerInputMode, [459](#)
- TRIGGER\_PCIE\_IF\_getTriggerInvertMask, [459](#)
- TRIGGER\_PCIE\_IF\_setCounterReloadValue, [460](#)
- TRIGGER\_PCIE\_IF\_setTriggerAndMask, [460](#)
- TRIGGER\_PCIE\_IF\_setTriggerInputMode, [461](#)
- TRIGGER\_PCIE\_IF\_setTriggerInvertMask, [461](#)
- TRIGGER\_PXI\_IF\_disableCounterAutoReload,  
[462](#)
- TRIGGER\_PXI\_IF\_disableCounterLoadZero, [462](#)
- TRIGGER\_PXI\_IF\_disableCounterStartMode, [462](#)
- TRIGGER\_PXI\_IF\_disableCounterStartStopMode,  
[463](#)
- TRIGGER\_PXI\_IF\_disableCounterTriggerCount,  
[463](#)
- TRIGGER\_PXI\_IF\_disableExtTriggerEdge↔  
DetectMode, [464](#)
- TRIGGER\_PXI\_IF\_disableExtTriggerInvert, [464](#)
- TRIGGER\_PXI\_IF\_disableExtTriggerOutput, [464](#)
- TRIGGER\_PXI\_IF\_disablePortTransmitPacket↔  
Mode, [465](#)
- TRIGGER\_PXI\_IF\_disableTrigger, [465](#)
- TRIGGER\_PXI\_IF\_enableCounterAutoReload,  
[466](#)
- TRIGGER\_PXI\_IF\_enableCounterLoadZero, [466](#)
- TRIGGER\_PXI\_IF\_enableCounterStartMode, [466](#)
- TRIGGER\_PXI\_IF\_enableCounterStartStopMode,  
[467](#)
- TRIGGER\_PXI\_IF\_enableCounterTriggerCount,  
[467](#)
- TRIGGER\_PXI\_IF\_enableExtTriggerEdgeDetect↔  
Mode, [468](#)
- TRIGGER\_PXI\_IF\_enableExtTriggerInvert, [468](#)
- TRIGGER\_PXI\_IF\_enableExtTriggerOutput, [469](#)
- TRIGGER\_PXI\_IF\_enablePortTransmitPacket↔  
Mode, [469](#)
- TRIGGER\_PXI\_IF\_enableTrigger, [470](#)
- TRIGGER\_PXI\_IF\_forceCounterReload, [470](#)
- TRIGGER\_PXI\_IF\_forceTrigger, [470](#)
- TRIGGER\_PXI\_IF\_getCounterAutoReload↔  
Enabled, [471](#)
- TRIGGER\_PXI\_IF\_getCounterLoadZeroEnabled,  
[471](#)
- TRIGGER\_PXI\_IF\_getCounterReloadValue, [471](#)
- TRIGGER\_PXI\_IF\_getCounterStartModeEnabled,  
[472](#)
- TRIGGER\_PXI\_IF\_getCounterStartStopMode↔  
Enabled, [472](#)
- TRIGGER\_PXI\_IF\_getCounterTriggerCount↔  
Enabled, [473](#)
- TRIGGER\_PXI\_IF\_getCounterValue, [473](#)
- TRIGGER\_PXI\_IF\_getExtTriggerEdgeDetect↔  
ModeEnabled, [473](#)
- TRIGGER\_PXI\_IF\_getExtTriggerExtend, [475](#)
- TRIGGER\_PXI\_IF\_getExtTriggerInvertEnabled,  
[475](#)
- TRIGGER\_PXI\_IF\_getExtTriggerOutputEnabled,  
[476](#)
- TRIGGER\_PXI\_IF\_getPortTransmitPacketMode↔  
Enabled, [476](#)
- TRIGGER\_PXI\_IF\_getTriggerAndMask, [476](#)
- TRIGGER\_PXI\_IF\_getTriggerEnabled, [477](#)
- TRIGGER\_PXI\_IF\_getTriggerInputMode, [477](#)
- TRIGGER\_PXI\_IF\_getTriggerInvertMask, [478](#)
- TRIGGER\_PXI\_IF\_setCounterReloadValue, [478](#)
- TRIGGER\_PXI\_IF\_setExtTriggerExtend, [479](#)
- TRIGGER\_PXI\_IF\_setTriggerAndMask, [479](#)
- TRIGGER\_PXI\_IF\_setTriggerInputMode, [479](#)
- TRIGGER\_PXI\_IF\_setTriggerInvertMask, [480](#)
- TRIGGER\_PXI\_ROUTER\_disableCounterAuto↔  
Reload, [480](#)
- TRIGGER\_PXI\_ROUTER\_disableCounterLoad↔  
Zero, [481](#)
- TRIGGER\_PXI\_ROUTER\_disableCounterStart↔  
Mode, [481](#)
- TRIGGER\_PXI\_ROUTER\_disableCounterStart↔  
StopMode, [481](#)
- TRIGGER\_PXI\_ROUTER\_disableCounter↔  
TriggerCount, [482](#)
- TRIGGER\_PXI\_ROUTER\_disablePortTransmit↔  
PacketMode, [482](#)
- TRIGGER\_PXI\_ROUTER\_disableTrigger, [483](#)
- TRIGGER\_PXI\_ROUTER\_enableCounterAuto↔  
Reload, [483](#)
- TRIGGER\_PXI\_ROUTER\_enableCounterLoad↔  
Zero, [484](#)
- TRIGGER\_PXI\_ROUTER\_enableCounterStart↔  
Mode, [484](#)
- TRIGGER\_PXI\_ROUTER\_enableCounterStart↔  
StopMode, [484](#)
- TRIGGER\_PXI\_ROUTER\_enableCounter↔  
TriggerCount, [485](#)
- TRIGGER\_PXI\_ROUTER\_enablePortTransmit↔  
PacketMode, [485](#)
- TRIGGER\_PXI\_ROUTER\_enableTrigger, [486](#)
- TRIGGER\_PXI\_ROUTER\_forceCounterReload,  
[486](#)
- TRIGGER\_PXI\_ROUTER\_forceTrigger, [487](#)
- TRIGGER\_PXI\_ROUTER\_getCounterAuto↔  
ReloadEnabled, [487](#)
- TRIGGER\_PXI\_ROUTER\_getCounterLoadZero↔  
Enabled, [487](#)
- TRIGGER\_PXI\_ROUTER\_getCounterReload↔  
Value, [489](#)
- TRIGGER\_PXI\_ROUTER\_getCounterStart↔  
ModeEnabled, [489](#)
- TRIGGER\_PXI\_ROUTER\_getCounterStartStop↔  
ModeEnabled, [490](#)
- TRIGGER\_PXI\_ROUTER\_getCounterTrigger↔



- CountEnabled, 490
- TRIGGER\_PXI\_ROUTER\_getCounterValue, 490
- TRIGGER\_PXI\_ROUTER\_getPortTransmit↔
  - PacketModeEnabled, 491
- TRIGGER\_PXI\_ROUTER\_getTriggerAndMask, 491
- TRIGGER\_PXI\_ROUTER\_getTriggerEnabled, 492
- TRIGGER\_PXI\_ROUTER\_getTriggerInputMode, 492
- TRIGGER\_PXI\_ROUTER\_getTriggerInvertMask, 492
- TRIGGER\_PXI\_ROUTER\_setCounterReload↔
  - Value, 493
- TRIGGER\_PXI\_ROUTER\_setTriggerAndMask, 493
- TRIGGER\_PXI\_ROUTER\_setTriggerInputMode, 494
- TRIGGER\_PXI\_ROUTER\_setTriggerInvertMask, 494
- Defines, 666
  - CFG\_STATUS\_NOT\_COMPATIBLE\_WITH\_TH↔
    - IS\_DEVICES\_FPGA\_VERSION, 668
  - CFG\_STATUS\_SUCCESS, 668
- Device and Driver Management, 206
  - STAR\_BUS\_CPCI, 219
  - STAR\_BUS\_PCI, 219
  - STAR\_BUS\_PCIE, 219
  - STAR\_BUS\_TCP, 219
  - STAR\_BUS\_TYPE, 219
  - STAR\_BUS\_UNKNOWN, 219
  - STAR\_BUS\_USB, 219
  - STAR\_BUS\_VIRTUAL, 219
  - STAR\_CHANNEL\_MASK, 210
  - STAR\_DEVICE\_ALL, 210
  - STAR\_DEVICE\_BRICK\_MK2, 210
  - STAR\_DEVICE\_BRICK\_MK3, 211
  - STAR\_DEVICE\_CONFIG\_NOT\_SUPPORTED, 211
  - STAR\_DEVICE\_CONFIG\_SUPPORTED, 211
  - STAR\_DEVICE\_CONFORMANCE\_TESTER, 211
  - STAR\_DEVICE\_CONFORMANCE\_TESTER\_M↔
    - K2, 211
  - STAR\_DEVICE\_CPCI\_MK2, 212
  - STAR\_DEVICE\_EGSE, 212
  - STAR\_DEVICE\_EGSE\_MK2, 212
  - STAR\_DEVICE\_ID, 218
  - STAR\_DEVICE\_IP\_TUNNEL, 212
  - STAR\_DEVICE\_LINK\_ANALYSER, 212
  - STAR\_DEVICE\_LINK\_ANALYSER\_MK2, 212
  - STAR\_DEVICE\_LINK\_ANALYSER\_MK3, 213
  - STAR\_DEVICE\_LISTENER\_ID, 218
  - STAR\_DEVICE\_PCI, 213
  - STAR\_DEVICE\_PCI\_MK2, 213
  - STAR\_DEVICE\_PCIE, 213
  - STAR\_DEVICE\_PXI\_INTERFACE, 213
  - STAR\_DEVICE\_PXI\_INTERFACE\_MK2, 213
  - STAR\_DEVICE\_PXI\_RMAP, 214
  - STAR\_DEVICE\_PXI\_RMAP\_MK2, 214
  - STAR\_DEVICE\_PXI\_ROUTER\_12, 214
  - STAR\_DEVICE\_PXI\_ROUTER\_8, 214
  - STAR\_DEVICE\_PXI\_ROUTER\_MK2, 214
  - STAR\_DEVICE\_RECORDER, 215
  - STAR\_DEVICE\_ROUTER\_MK2S, 215
  - STAR\_DEVICE\_ROUTER\_USB, 215
  - STAR\_DEVICE\_RTC, 215
  - STAR\_DEVICE\_SERIAL\_SIZE, 215
  - STAR\_DEVICE\_SPLT, 215
  - STAR\_DEVICE\_STAR\_FIRE, 216
  - STAR\_DEVICE\_STAR\_FIRE\_MK3, 216
  - STAR\_DEVICE\_STAR\_ULTRA\_PCIE, 216
  - STAR\_DEVICE\_TRAFFIC\_GENERATOR, 216
  - STAR\_DEVICE\_TXRX\_NOT\_SUPPORTED, 216
  - STAR\_DEVICE\_TXRX\_SUPPORTED, 216
  - STAR\_DEVICE\_TYPE, 217
  - STAR\_DEVICE\_UNKNOWN, 217
  - STAR\_DEVICE\_USB\_BRICK, 217
  - STAR\_DEVICE\_WBS\_II, 217
  - STAR\_DRIVER\_ID, 218
  - STAR\_DRIVER\_INVALID, 219
  - STAR\_DRIVER\_LISTENER\_ID, 218
  - STAR\_DRIVER\_TYPE, 219
  - STAR\_DRIVER\_TYPE\_PCI, 219
  - STAR\_DRIVER\_TYPE\_TCPIP, 219
  - STAR\_DRIVER\_TYPE\_USB, 219
  - STAR\_DRIVER\_TYPE\_VIRTUAL, 219
  - STAR\_DeviceEventFunc, 218
  - STAR\_DriverEventFunc, 218
  - STAR\_destroyDeviceList, 220
  - STAR\_destroyDriverList, 221
  - STAR\_getDeviceBusType, 221
  - STAR\_getDeviceBusTypeAsString, 221
  - STAR\_getDeviceChannels, 222
  - STAR\_getDeviceConfigCapabilities, 222
  - STAR\_getDeviceDriver, 222
  - STAR\_getDeviceFirmwareVersion, 223
  - STAR\_getDeviceIndex, 223
  - STAR\_getDeviceList, 223
  - STAR\_getDeviceListForDriver, 225
  - STAR\_getDeviceListForType, 225
  - STAR\_getDeviceListForTypes, 226
  - STAR\_getDeviceName, 226
  - STAR\_getDeviceSerialNumber, 227
  - STAR\_getDeviceTxRxCapabilities, 227
  - STAR\_getDeviceType, 228
  - STAR\_getDeviceTypeAsString, 228
  - STAR\_getDriverList, 228
  - STAR\_getDriverType, 230
  - STAR\_getDriverVersion, 230
  - STAR\_getLocalDeviceAttachedToChannel, 231
  - STAR\_isChannelOpen, 231
  - STAR\_registerDeviceListener, 231
  - STAR\_registerDeviceListenerForDevice, 232
  - STAR\_registerDeviceListenerForDriver, 232
  - STAR\_registerDriverListener, 232
  - STAR\_resetDevice, 233
  - STAR\_resetDeviceName, 233

- STAR\_setDeviceName, 233
- STAR\_unregisterDeviceListener, 234
- STAR\_unregisterDriverListener, 234
- Device Identifier, 622
  - CFG\_getDeviceIdentificationInfo, 622
  - CFG\_getDeviceManufacturerAsString, 623
  - CFG\_getDeviceTypeAsString, 623
  - CFG\_getNetworkDiscoveryInfo, 623
- Device Information, 521, 570
  - CFG\_MK2\_getHardwareInfo, 522
  - CFG\_MK2\_hardwareInfoToString, 523
  - CFG\_MK2\_identify, 523
  - CFG\_ROUTER\_getDeviceIdentificationInfo, 573
  - CFG\_ROUTER\_getDeviceManufacturerAsString, 573
  - CFG\_ROUTER\_getDeviceTypeAsString, 574
  - CFG\_ROUTER\_getNetworkDiscoveryInfo, 574
  - STAR\_CFG\_DEVICE\_STR\_MAX\_LEN, 572
  - STAR\_CFG\_DEVICE\_TYPE, 573
  - STAR\_CFG\_DEVICE\_TYPE\_INVALID, 573
  - STAR\_CFG\_DEVICE\_TYPE\_ROUTER, 573
  - STAR\_CFG\_DEVICE\_TYPE\_UNKNOWN, 573
  - STAR\_CFG\_MANUFACTURER\_STR\_MAX\_LEN, 572
- EXT\_TRIGGER\_ACTION
  - Actions, 405
- EXT\_TRIGGER\_ACTION\_ALL
  - Actions, 405
- EXT\_TRIGGER\_ACTION\_MASK
  - Events, 383
- EXT\_TRIGGER\_ACTION\_NONE
  - Actions, 405
- EXT\_TRIGGER\_ACTION\_OUT
  - Actions, 405
- EXT\_TRIGGER\_EVENT
  - Events, 384
- EXT\_TRIGGER\_EVENT\_ALL
  - Events, 385
- EXT\_TRIGGER\_EVENT\_IN
  - Events, 385
- EXT\_TRIGGER\_EVENT\_MASK
  - Events, 383
- EXT\_TRIGGER\_EVENT\_NONE
  - Events, 385
- Error Injection, 306, 329, 532, 656
  - CFG\_BRICK\_MK2\_injectErrors, 306
  - CFG\_MK2\_injectError, 533
  - CFG\_PXI\_injectError, 329
  - CFG\_injectError, 656
  - CFG\_injectErrors, 657
  - SPW\_ERROR, 532
  - SPW\_ERROR\_DECREMENT\_CREDIT, 533
  - SPW\_ERROR\_DISCONNECT, 532
  - SPW\_ERROR\_ESCAPE, 532
  - SPW\_ERROR\_INCREMENT\_CREDIT, 533
  - SPW\_ERROR\_INSERT\_FCT, 532
  - SPW\_ERROR\_PARITY, 532
  - SPW\_ERROR\_SUPPRESS\_FCT, 532
- Events, 307, 335, 380, 500, 643
  - CFG\_BRICK\_MK2\_disableSpeedChangeEvents↔
    - OnPort, 307
  - CFG\_BRICK\_MK2\_disableStateChangeEvents↔
    - OnPort, 309
  - CFG\_BRICK\_MK2\_enableSpeedChangeEvents↔
    - OnPort, 309
  - CFG\_BRICK\_MK2\_enableStateChangeEvents↔
    - OnPort, 310
  - CFG\_BRICK\_MK2\_getSpeedChangeEventsOn↔
    - PortEnabled, 310
  - CFG\_BRICK\_MK2\_getStateChangeEventsOn↔
    - PortEnabled, 311
  - CFG\_MK2\_disableSpeedChangeEventsOnPort, 501
  - CFG\_MK2\_disableStateChangeEventsOnPort, 501
  - CFG\_MK2\_disableTimeCodeEventsOnPort, 502
  - CFG\_MK2\_enableSpeedChangeEventsOnPort, 502
  - CFG\_MK2\_enableStateChangeEventsOnPort, 502
  - CFG\_MK2\_enableTimeCodeEventsOnPort, 503
  - CFG\_MK2\_getSpeedChangeEventsOnPort↔
    - Enabled, 503
  - CFG\_MK2\_getStateChangeEventsOnPort↔
    - Enabled, 504
  - CFG\_MK2\_getTimeCodeEventsOnPortEnabled, 504
  - CFG\_PXI\_disableSpeedChangeEventsOnPort, 335
  - CFG\_PXI\_disableStateChangeEventsOnPort, 336
  - CFG\_PXI\_disableTimeCodeEventsOnPort, 336
  - CFG\_PXI\_enableSpeedChangeEventsOnPort, 337
  - CFG\_PXI\_enableStateChangeEventsOnPort, 337
  - CFG\_PXI\_enableTimeCodeEventsOnPort, 338
  - CFG\_PXI\_getSpeedChangeEventsOnPort↔
    - Enabled, 338
  - CFG\_PXI\_getStateChangeEventsOnPortEnabled, 338
  - CFG\_PXI\_getTimeCodeEventsOnPortEnabled, 339
  - CFG\_disableSpeedChangeEventsOnPort, 643
  - CFG\_disableStateChangeEventsOnPort, 644
  - CFG\_disableTimeCodeEventsOnPort, 644
  - CFG\_enableSpeedChangeEventsOnPort, 645
  - CFG\_enableStateChangeEventsOnPort, 645
  - CFG\_enableTimeCodeEventsOnPort, 646
  - CFG\_getSpeedChangeEventsOnPortEnabled, 646
  - CFG\_getStateChangeEventsOnPortEnabled, 647
  - CFG\_getTimeCodeEventsOnPortEnabled, 647
  - COUNTER\_ACTION\_MASK, 383
  - COUNTER\_EVENT, 384
  - COUNTER\_EVENT\_ALL, 384
  - COUNTER\_EVENT\_COUNT, 384
  - COUNTER\_EVENT\_MASK, 383
  - COUNTER\_EVENT\_NONE, 384
  - COUNTER\_EVENT\_RELOAD, 384

- COUNTER\_EVENT\_ZERO, 384
- COUNTER\_EVENT\_ZERO\_SINGLE, 384
- EXT\_TRIGGER\_ACTION\_MASK, 383
- EXT\_TRIGGER\_EVENT, 384
- EXT\_TRIGGER\_EVENT\_ALL, 385
- EXT\_TRIGGER\_EVENT\_IN, 385
- EXT\_TRIGGER\_EVENT\_MASK, 383
- EXT\_TRIGGER\_EVENT\_NONE, 385
- PORT\_ACTION\_MASK, 383
- PORT\_EVENT, 385
- PORT\_EVENT\_ALL, 385
- PORT\_EVENT\_CREDIT\_ERROR, 385
- PORT\_EVENT\_DISCONNECT, 385
- PORT\_EVENT\_ESCAPE\_ERROR, 385
- PORT\_EVENT\_MASK, 384
- PORT\_EVENT\_NONE, 385
- PORT\_EVENT\_PARITY\_ERROR, 385
- PORT\_EVENT\_RUNNING, 385
- PORT\_EVENT\_RX\_EEP, 385
- PORT\_EVENT\_RX\_EOP, 385
- PORT\_EVENT\_RX\_SOP, 385
- PORT\_EVENT\_RX\_TIME\_CODE, 385
- PORT\_EVENT\_TX\_EOP, 385
- PORT\_EVENT\_TX\_PKT\_PENDING, 385
- PORT\_EVENT\_TX\_SOP, 385
- PORT\_EVENT\_TX\_TIME\_CODE, 385
- TIME\_CODE\_ACTION\_MASK, 384
- TIME\_CODE\_EVENT, 385
- TIME\_CODE\_EVENT\_ALL, 385
- TIME\_CODE\_EVENT\_MASK, 384
- TIME\_CODE\_EVENT\_NONE, 385
- TIME\_CODE\_EVENT\_TICK, 385
- TRIGGER\_BRICK\_MK3\_getCounterInputEvents, 386
- TRIGGER\_BRICK\_MK3\_getExtTriggerInput↔Events, 386
- TRIGGER\_BRICK\_MK3\_getPortInputEvents, 387
- TRIGGER\_BRICK\_MK3\_getTimeCodeInput↔Events, 387
- TRIGGER\_BRICK\_MK3\_getTriggerInputEvents, 387
- TRIGGER\_BRICK\_MK3\_setCounterInputEvents, 388
- TRIGGER\_BRICK\_MK3\_setExtTriggerInput↔Events, 388
- TRIGGER\_BRICK\_MK3\_setPortInputEvents, 389
- TRIGGER\_BRICK\_MK3\_setTimeCodeInput↔Events, 389
- TRIGGER\_BRICK\_MK3\_setTriggerInputEvents, 390
- TRIGGER\_EVENT, 385
- TRIGGER\_EVENT\_ALL, 386
- TRIGGER\_EVENT\_IN, 386
- TRIGGER\_EVENT\_MASK, 384
- TRIGGER\_EVENT\_NONE, 386
- TRIGGER\_PCIE\_IF\_getCounterInputEvents, 390
- TRIGGER\_PCIE\_IF\_getPortInputEvents, 391
- TRIGGER\_PCIE\_IF\_getTriggerInputEvents, 391
- TRIGGER\_PCIE\_IF\_setCounterInputEvents, 391
- TRIGGER\_PCIE\_IF\_setPortInputEvents, 392
- TRIGGER\_PCIE\_IF\_setTriggerInputEvents, 392
- TRIGGER\_PXI\_IF\_getCounterInputEvents, 393
- TRIGGER\_PXI\_IF\_getExtTriggerInputEvents, 393
- TRIGGER\_PXI\_IF\_getPortInputEvents, 394
- TRIGGER\_PXI\_IF\_getTimeCodeInputEvents, 394
- TRIGGER\_PXI\_IF\_getTriggerInputEvents, 394
- TRIGGER\_PXI\_IF\_setCounterInputEvents, 396
- TRIGGER\_PXI\_IF\_setExtTriggerInputEvents, 396
- TRIGGER\_PXI\_IF\_setPortInputEvents, 397
- TRIGGER\_PXI\_IF\_setTimeCodeInputEvents, 397
- TRIGGER\_PXI\_IF\_setTriggerInputEvents, 398
- TRIGGER\_PXI\_ROUTER\_getCounterInput↔Events, 398
- TRIGGER\_PXI\_ROUTER\_getPortInputEvents, 399
- TRIGGER\_PXI\_ROUTER\_getTimeCodeInput↔Events, 399
- TRIGGER\_PXI\_ROUTER\_getTriggerInputEvents, 399
- TRIGGER\_PXI\_ROUTER\_setCounterInputEvents, 400
- TRIGGER\_PXI\_ROUTER\_setPortInputEvents, 400
- TRIGGER\_PXI\_ROUTER\_setTimeCodeInput↔Events, 401
- TRIGGER\_PXI\_ROUTER\_setTriggerInputEvents, 401
- FALSE
  - STAR-Dundee Types, 711
- Functions for Building RMAP Packets, 681
  - RMAP\_BuildReadCommandPacket, 683
  - RMAP\_BuildReadModifyWriteCommandPacket, 684
  - RMAP\_BuildReadModifyWriteRegisterPacket, 685
  - RMAP\_BuildReadModifyWriteReplyPacket, 686
  - RMAP\_BuildReadRegisterPacket, 687
  - RMAP\_BuildReadReplyPacket, 688
  - RMAP\_BuildWriteCommandPacket, 689
  - RMAP\_BuildWriteRegisterPacket, 689
  - RMAP\_BuildWriteReplyPacket, 690
  - RMAP\_CalculateReadCommandPacketLength, 691
  - RMAP\_CalculateReadModifyWriteCommand↔PacketLength, 692
  - RMAP\_CalculateReadModifyWriteReplyPacket↔Length, 692
  - RMAP\_CalculateReadReplyPacketLength, 693
  - RMAP\_CalculateWriteCommandPacketLength, 693
  - RMAP\_CalculateWriteReplyPacketLength, 694
  - RMAP\_FillReadCommandPacket, 694
  - RMAP\_FillReadModifyWriteCommandPacket, 695
  - RMAP\_FillReadModifyWriteReplyPacket, 696
  - RMAP\_FillReadReplyPacket, 697
  - RMAP\_FillWriteCommandPacket, 698
  - RMAP\_FillWriteReplyPacket, 699



- CFG\_PXI\_getLinkRateDivider, 325
- CFG\_PXI\_getTransmitClock, 325
- CFG\_PXI\_setBaseTransmitClock, 326
- CFG\_PXI\_setLinkRateDivider, 326
- CFG\_PXI\_setTransmitClock, 327
- CFG\_ROUTER\_MK2S\_disablePrecisionTransmit↔  
Rate, 496
- CFG\_ROUTER\_MK2S\_enablePrecisionTransmit↔  
Rate, 497
- CFG\_ROUTER\_MK2S\_getPrecisionTransmitRate,  
497
- CFG\_ROUTER\_MK2S\_getPrecisionTransmit↔  
RateEnabled, 498
- CFG\_ROUTER\_MK2S\_getPrecisionTransmit↔  
RateInUse, 498
- CFG\_ROUTER\_MK2S\_setPrecisionTransmitRate,  
498
- CFG\_getMeasuredLinkSpeed, 649
- STAR\_CFG\_BRICK\_MK2\_LINK\_FREQ, 297
- STAR\_CFG\_BRICK\_MK2\_LINK\_FREQ\_120, 297
- STAR\_CFG\_BRICK\_MK2\_LINK\_FREQ\_130, 297
- STAR\_CFG\_BRICK\_MK2\_LINK\_FREQ\_140, 297
- STAR\_CFG\_BRICK\_MK2\_LINK\_FREQ\_150, 298
- STAR\_CFG\_BRICK\_MK2\_LINK\_FREQ\_160, 298
- STAR\_CFG\_BRICK\_MK2\_LINK\_FREQ\_180, 298
- STAR\_CFG\_BRICK\_MK2\_LINK\_FREQ\_200, 298
- STAR\_CFG\_PCIMK2\_LINK\_FREQ, 294
- STAR\_CFG\_PCIMK2\_LINK\_FREQ\_120, 294
- STAR\_CFG\_PCIMK2\_LINK\_FREQ\_128, 294
- STAR\_CFG\_PCIMK2\_LINK\_FREQ\_140, 295
- STAR\_CFG\_PCIMK2\_LINK\_FREQ\_150, 295
- STAR\_CFG\_PCIMK2\_LINK\_FREQ\_160, 295
- STAR\_CFG\_PCIMK2\_LINK\_FREQ\_180, 295
- STAR\_CFG\_PCIMK2\_LINK\_FREQ\_200, 295
- Links, 650
  - CFG\_getTransmitSignallingRate, 650
  - CFG\_setTransmitSignallingRate, 651
- Manual Authorisation, 342
  - REJECTION\_REASON, 343
  - RMAP\_TARGET\_PXI\_IF\_authoriseWaiting↔  
Command, 344
  - RMAP\_TARGET\_PXI\_IF\_getWaitingCommand,  
344
  - RMAP\_TARGET\_PXI\_IF\_isAuthRequired, 344
  - RMAP\_TARGET\_PXI\_IF\_rejectWaitingCommand,  
345
- Measured Link Speed, 312, 512
  - CFG\_MK2\_getMeasuredLinkSpeed, 512
  - STAR\_CFG\_BRICK\_MK2\_LINK\_SPEED\_UNIT↔  
S\_KBPS, 312
  - STAR\_CFG\_MK2\_LINK\_SPEED\_UNITS\_KBPS,  
512
- Mk2 Compatible Interface and Router Device Configura-  
tion API, 564
- NOTIF\_TYPE
  - Notifications, 362
- NOTIF\_TYPE\_ALL
  - Notifications, 362
- NOTIF\_TYPE\_AUTH\_REQUEST
  - Notifications, 362
- NOTIF\_TYPE\_CMD\_COMPLETE
  - Notifications, 362
- NOTIF\_TYPE\_MASK
  - Notifications, 361
- NOTIF\_TYPE\_NONE
  - Notifications, 362
- NotifListenerFunc
  - Notifications, 361
- NotifListenerWithContextFunc
  - Notifications, 361
- Notifications, 360
  - NOTIF\_TYPE, 362
  - NOTIF\_TYPE\_ALL, 362
  - NOTIF\_TYPE\_AUTH\_REQUEST, 362
  - NOTIF\_TYPE\_CMD\_COMPLETE, 362
  - NOTIF\_TYPE\_MASK, 361
  - NOTIF\_TYPE\_NONE, 362
  - NotifListenerFunc, 361
  - NotifListenerWithContextFunc, 361
  - RMAP\_TARGET\_PXI\_IF\_getEnabledNotifications,  
362
  - RMAP\_TARGET\_PXI\_IF\_registerNotification↔  
Listener, 364
  - RMAP\_TARGET\_PXI\_IF\_registerNotification↔  
ListenerWithContext, 364
  - RMAP\_TARGET\_PXI\_IF\_setEnabledNotifications,  
365
  - RMAP\_TARGET\_PXI\_IF\_startReceivingNotifications,  
365
  - RMAP\_TARGET\_PXI\_IF\_stopReceivingNotifications,  
366
  - RMAP\_TARGET\_PXI\_IF\_unregisterNotification↔  
Listener, 366
  - RMAP\_TARGET\_PXI\_IF\_unregisterNotification↔  
ListenerWithContext, 366
- Other Device Configuration APIs, 555
- PCI Mk2 Configuration API, 565
- PCI Mk2 Packet Subsystem, 536
  - PKT\_SUBSYS\_PATTERN, 539
  - PKT\_SUBSYS\_StatisticsFunc, 539
  - PKT\_SUBSYS\_cancelWaitsForPacketChecking↔  
Error, 539
  - PKT\_SUBSYS\_fillBufferWithPattern, 541
  - PKT\_SUBSYS\_getPacketCheckingMismatch↔  
Count, 541
  - PKT\_SUBSYS\_getSinkDataPointers, 542
  - PKT\_SUBSYS\_getStatistics, 542
  - PKT\_SUBSYS\_readMemory, 542
  - PKT\_SUBSYS\_setGeneratorBlockingParameters,  
543
  - PKT\_SUBSYS\_setPacketCheckingFormat, 543
  - PKT\_SUBSYS\_setPacketGeneratorFormat, 545
  - PKT\_SUBSYS\_setPacketSinkParameters, 545
  - PKT\_SUBSYS\_setSinkBlockingParameters, 547



- PKT\_SUBSYS\_startGetStatisticsLoop, 547
- PKT\_SUBSYS\_startPacketChecking, 548
- PKT\_SUBSYS\_startPacketGeneration, 548
- PKT\_SUBSYS\_startSink, 549
- PKT\_SUBSYS\_stopGetStatisticsLoop, 549
- PKT\_SUBSYS\_stopPacketChecking, 550
- PKT\_SUBSYS\_stopPacketGeneration, 550
- PKT\_SUBSYS\_stopSink, 550
- PKT\_SUBSYS\_waitForPacketCheckingError, 551
- PKT\_SUBSYS\_waitForPacketGeneration↔
  - SequenceComplete, 551
- PKT\_SUBSYS\_writeMemory, 552
- STATISTICS\_LOOP\_ID, 539
- PKT\_SUBSYS\_MODULE\_NAME
  - packet\_subsystem.h, 755
- PKT\_SUBSYS\_PATTERN
  - PCI Mk2 Packet Subsystem, 539
- PKT\_SUBSYS\_STATISTICS, 538
- PKT\_SUBSYS\_StatisticsFunc
  - PCI Mk2 Packet Subsystem, 539
- PKT\_SUBSYS\_cancelWaitsForPacketCheckingError
  - PCI Mk2 Packet Subsystem, 539
- PKT\_SUBSYS\_fillBufferWithPattern
  - PCI Mk2 Packet Subsystem, 541
- PKT\_SUBSYS\_getPacketCheckingMismatchCount
  - PCI Mk2 Packet Subsystem, 541
- PKT\_SUBSYS\_getSinkDataPointers
  - PCI Mk2 Packet Subsystem, 542
- PKT\_SUBSYS\_getStatistics
  - PCI Mk2 Packet Subsystem, 542
- PKT\_SUBSYS\_readMemory
  - PCI Mk2 Packet Subsystem, 542
- PKT\_SUBSYS\_setGeneratorBlockingParameters
  - PCI Mk2 Packet Subsystem, 543
- PKT\_SUBSYS\_setPacketCheckingFormat
  - PCI Mk2 Packet Subsystem, 543
- PKT\_SUBSYS\_setPacketGeneratorFormat
  - PCI Mk2 Packet Subsystem, 545
- PKT\_SUBSYS\_setPacketSinkParameters
  - PCI Mk2 Packet Subsystem, 545
- PKT\_SUBSYS\_setSinkBlockingParameters
  - PCI Mk2 Packet Subsystem, 547
- PKT\_SUBSYS\_startGetStatisticsLoop
  - PCI Mk2 Packet Subsystem, 547
- PKT\_SUBSYS\_startPacketChecking
  - PCI Mk2 Packet Subsystem, 548
- PKT\_SUBSYS\_startPacketGeneration
  - PCI Mk2 Packet Subsystem, 548
- PKT\_SUBSYS\_startSink
  - PCI Mk2 Packet Subsystem, 549
- PKT\_SUBSYS\_stopGetStatisticsLoop
  - PCI Mk2 Packet Subsystem, 549
- PKT\_SUBSYS\_stopPacketChecking
  - PCI Mk2 Packet Subsystem, 550
- PKT\_SUBSYS\_stopPacketGeneration
  - PCI Mk2 Packet Subsystem, 550
- PKT\_SUBSYS\_stopSink
  - PCI Mk2 Packet Subsystem, 550
- PKT\_SUBSYS\_waitForPacketCheckingError
  - PCI Mk2 Packet Subsystem, 551
- PKT\_SUBSYS\_waitForPacketGenerationSequence↔
  - Complete
    - PCI Mk2 Packet Subsystem, 551
- PKT\_SUBSYS\_writeMemory
  - PCI Mk2 Packet Subsystem, 552
- POPULATE\_VERSION
  - version.h, 794
- PORT\_ACTION
  - Actions, 405
- PORT\_ACTION\_ALL
  - Actions, 406
- PORT\_ACTION\_DECR\_CREDIT
  - Actions, 406
- PORT\_ACTION\_DISABLE\_LVDS
  - Actions, 406
- PORT\_ACTION\_DISCONNECT
  - Actions, 406
- PORT\_ACTION\_ESCAPE\_ERROR
  - Actions, 406
- PORT\_ACTION\_INCR\_CREDIT
  - Actions, 406
- PORT\_ACTION\_INSERT\_FCT
  - Actions, 406
- PORT\_ACTION\_MASK
  - Events, 383
- PORT\_ACTION\_NONE
  - Actions, 406
- PORT\_ACTION\_PARITY\_ERROR
  - Actions, 406
- PORT\_ACTION\_STOP\_RECEPTION
  - Actions, 406
- PORT\_ACTION\_SUPPRESS\_FCT
  - Actions, 406
- PORT\_ACTION\_TRANSMIT\_PKT
  - Actions, 406
- PORT\_EVENT
  - Events, 385
- PORT\_EVENT\_ALL
  - Events, 385
- PORT\_EVENT\_CREDIT\_ERROR
  - Events, 385
- PORT\_EVENT\_DISCONNECT
  - Events, 385
- PORT\_EVENT\_ESCAPE\_ERROR
  - Events, 385
- PORT\_EVENT\_MASK
  - Events, 384
- PORT\_EVENT\_NONE
  - Events, 385
- PORT\_EVENT\_PARITY\_ERROR
  - Events, 385
- PORT\_EVENT\_RUNNING
  - Events, 385
- PORT\_EVENT\_RX\_EEP
  - Events, 385
- PORT\_EVENT\_RX\_EOP

- Events, [385](#)
- PORT\_EVENT\_RX\_SOP
  - Events, [385](#)
- PORT\_EVENT\_RX\_TIME\_CODE
  - Events, [385](#)
- PORT\_EVENT\_TX\_EOP
  - Events, [385](#)
- PORT\_EVENT\_TX\_PKT\_PENDING
  - Events, [385](#)
- PORT\_EVENT\_TX\_SOP
  - Events, [385](#)
- PORT\_EVENT\_TX\_TIME\_CODE
  - Events, [385](#)
- PORT\_STATUS\_CONTROL
  - Port Status and Control, [582](#)
- PRINT\_FULL\_VERSION\_INFO
  - version.h, [795](#)
- PRINT\_VERSION
  - version.h, [795](#)
- PRINT\_VERSION\_EDIT
  - version.h, [795](#)
- PRINT\_VERSION\_NUM
  - version.h, [795](#)
- PRINT\_VERSION\_PATCH
  - version.h, [796](#)
- PXI Configuration API, [569](#)
- packet\_subsystem.h, [753](#)
  - PKT\_SUBSYS\_MODULE\_NAME, [755](#)
- Periodic, [664](#)
  - CFG\_getDeviceClockRate, [664](#)
  - CFG\_startPeriodicAction, [664](#)
  - CFG\_stopPeriodicAction, [665](#)
- Periodic Actions, [534](#)
  - CFG\_MK2\_getDeviceClockRate, [534](#)
  - CFG\_MK2\_startPeriodicAction, [534](#)
  - CFG\_MK2\_stopPeriodicAction, [535](#)
- Port Status and Control, [576, 614](#)
  - CFG\_ROUTER\_clearPortErrors, [583](#)
  - CFG\_ROUTER\_getConfigPortErrors, [583](#)
  - CFG\_ROUTER\_getExternalPortErrors, [584](#)
  - CFG\_ROUTER\_getExternalPortStatus, [584](#)
  - CFG\_ROUTER\_getPortConnection, [586](#)
  - CFG\_ROUTER\_getPortStatusControl, [586](#)
  - CFG\_ROUTER\_getPortType, [587](#)
  - CFG\_ROUTER\_getSpaceWireLinkErrors, [587](#)
  - CFG\_ROUTER\_getSpaceWireLinkStatus, [588](#)
  - CFG\_ROUTER\_setSpaceWireLinkStatus, [588](#)
  - CFG\_ROUTER\_startLink, [589](#)
  - CFG\_ROUTER\_stopLink, [589](#)
  - CFG\_clearPortErrors, [615](#)
  - CFG\_getConfigPortErrors, [615](#)
  - CFG\_getExternalPortErrors, [616](#)
  - CFG\_getExternalPortStatus, [616](#)
  - CFG\_getPortConnection, [616](#)
  - CFG\_getPortStatusControl, [618](#)
  - CFG\_getPortType, [618](#)
  - CFG\_getSpaceWireLinkErrors, [619](#)
  - CFG\_getSpaceWireLinkStatus, [619](#)
  - CFG\_setSpaceWireLinkStatus, [620](#)
  - CFG\_startLink, [620](#)
  - CFG\_stopLink, [621](#)
  - PORT\_STATUS\_CONTROL, [582](#)
  - STAR\_CFG\_PORT\_TYPE, [582](#)
  - STAR\_CFG\_PORT\_TYPE\_CONFIGURATION, [582](#)
  - STAR\_CFG\_PORT\_TYPE\_EXTERNAL, [583](#)
  - STAR\_CFG\_PORT\_TYPE\_INVALID, [583](#)
  - STAR\_CFG\_PORT\_TYPE\_LINK, [583](#)
  - STAR\_CFG\_SPW\_LINK\_STATE, [583](#)
  - STAR\_CFG\_SPW\_LINK\_STATE\_CONNECTING, [583](#)
  - STAR\_CFG\_SPW\_LINK\_STATE\_ERROR\_RES←ET, [583](#)
  - STAR\_CFG\_SPW\_LINK\_STATE\_ERROR\_WAIT, [583](#)
  - STAR\_CFG\_SPW\_LINK\_STATE\_INVALID, [583](#)
  - STAR\_CFG\_SPW\_LINK\_STATE\_READY, [583](#)
  - STAR\_CFG\_SPW\_LINK\_STATE\_RUN, [583](#)
  - STAR\_CFG\_SPW\_LINK\_STATE\_STARTED, [583](#)
- Precision Links, [652](#)
  - CFG\_disablePrecisionTransmitRate, [652](#)
  - CFG\_enablePrecisionTransmitRate, [653](#)
  - CFG\_getPrecisionTransmitRate, [653](#)
  - CFG\_getPrecisionTransmitRateEnabled, [654](#)
  - CFG\_getPrecisionTransmitRateInUse, [654](#)
  - CFG\_setPrecisionTransmitRate, [655](#)
- REGISTER
  - STAR-Dundee Types, [712](#)
- REJECTION\_REASON
  - Manual Authorisation, [343](#)
- RMAP Packet Library, [669](#)
  - RMAP\_COMMAND\_BIT, [674](#)
  - RMAP\_COMMAND\_NOT\_IMPLEMENTED\_OR←\_AUTHORISED, [678](#)
  - RMAP\_CalculateCRC, [678](#)
  - RMAP\_CalculateCRCWithSeed, [678](#)
  - RMAP\_EARLY\_EOP, [678](#)
  - RMAP\_EEP, [678](#)
  - RMAP\_FreeBuffer, [679](#)
  - RMAP\_GENERAL\_ERROR, [677](#)
  - RMAP\_GET\_VERSION\_EDIT, [674](#)
  - RMAP\_GET\_VERSION\_MAJOR, [674](#)
  - RMAP\_GET\_VERSION\_MINOR, [675](#)
  - RMAP\_GET\_VERSION\_PATCH, [675](#)
  - RMAP\_GetVersion, [679](#)
  - RMAP\_INCREMENT\_ADDRESS\_BIT, [676](#)
  - RMAP\_INVALID\_DATA\_CRC, [678](#)
  - RMAP\_INVALID\_KEY, [678](#)
  - RMAP\_INVALID\_PACKET\_TYPE, [677](#)
  - RMAP\_INVALID\_STATUS, [678](#)
  - RMAP\_INVALID\_TARGET\_LOGICAL\_ADDRE←SS, [678](#)
  - RMAP\_IsCRCValid, [679](#)
  - RMAP\_PACKET\_TYPE, [677](#)
  - RMAP\_PROTOCOL\_IDENTIFIER, [676](#)
  - RMAP\_READ\_COMMAND, [677](#)





- Functions for Interpreting RMAP Packets, 708
- RMAP\_GetTargetAddress
  - Functions for Interpreting RMAP Packets, 709
- RMAP\_GetTransactionID
  - Functions for Interpreting RMAP Packets, 709
- RMAP\_GetVerifyBeforeWrite
  - Functions for Interpreting RMAP Packets, 710
- RMAP\_GetVersion
  - RMAP Packet Library, 679
- RMAP\_INCREMENT\_ADDRESS\_BIT
  - RMAP Packet Library, 676
- RMAP\_INVALID\_DATA\_CRC
  - RMAP Packet Library, 678
- RMAP\_INVALID\_KEY
  - RMAP Packet Library, 678
- RMAP\_INVALID\_PACKET\_TYPE
  - RMAP Packet Library, 677
- RMAP\_INVALID\_STATUS
  - RMAP Packet Library, 678
- RMAP\_INVALID\_TARGET\_LOGICAL\_ADDRESS
  - RMAP Packet Library, 678
- RMAP\_IsCRCValid
  - RMAP Packet Library, 679
- RMAP\_PACKET, 672
- RMAP\_PACKET\_TYPE
  - RMAP Packet Library, 677
- RMAP\_PROTOCOL\_IDENTIFIER
  - RMAP Packet Library, 676
- RMAP\_READ\_COMMAND
  - RMAP Packet Library, 677
- RMAP\_READ\_MODIFY\_WRITE\_COMMAND
  - RMAP Packet Library, 677
- RMAP\_READ\_MODIFY\_WRITE\_REPLY
  - RMAP Packet Library, 677
- RMAP\_READ\_REPLY
  - RMAP Packet Library, 677
- RMAP\_REPLY\_ADDRESS\_LENGTH\_BITS
  - RMAP Packet Library, 676
- RMAP\_REPLY\_BIT
  - RMAP Packet Library, 676
- RMAP\_RESERVED\_BIT
  - RMAP Packet Library, 676
- RMAP\_RMW\_DATA\_LENGTH\_ERROR
  - RMAP Packet Library, 678
- RMAP\_STATUS
  - RMAP Packet Library, 677
- RMAP\_SUCCESS
  - RMAP Packet Library, 677
- RMAP\_SetProtocolIdentifier
  - Functions for Building RMAP Packets, 700
- RMAP\_TARGET\_PXI\_IF\_authoriseWaitingCommand
  - Manual Authorisation, 344
- RMAP\_TARGET\_PXI\_IF\_getAddressOffset
  - Configuration, 350
- RMAP\_TARGET\_PXI\_IF\_getAuthCommands
  - Configuration, 350
- RMAP\_TARGET\_PXI\_IF\_getAuthControlMode
  - Configuration, 350
- RMAP\_TARGET\_PXI\_IF\_getAuthKeyRange
  - Configuration, 351
- RMAP\_TARGET\_PXI\_IF\_getAuthLogicalAddress↔
  - Range
  - Configuration, 351
- RMAP\_TARGET\_PXI\_IF\_getAuthMemoryAddress↔
  - Range
  - Configuration, 352
- RMAP\_TARGET\_PXI\_IF\_getAuthProtocolId
  - Configuration, 352
- RMAP\_TARGET\_PXI\_IF\_getEnabledNotifications
  - Notifications, 362
- RMAP\_TARGET\_PXI\_IF\_getInterfaceMode
  - Configuration, 352
- RMAP\_TARGET\_PXI\_IF\_getStatus
  - Configuration, 354
- RMAP\_TARGET\_PXI\_IF\_getWaitingCommand
  - Manual Authorisation, 344
- RMAP\_TARGET\_PXI\_IF\_isAuthRequired
  - Manual Authorisation, 344
- RMAP\_TARGET\_PXI\_IF\_readMemory
  - Configuration, 354
- RMAP\_TARGET\_PXI\_IF\_registerNotificationListener
  - Notifications, 364
- RMAP\_TARGET\_PXI\_IF\_registerNotificationListener↔
  - WithContext
  - Notifications, 364
- RMAP\_TARGET\_PXI\_IF\_rejectWaitingCommand
  - Manual Authorisation, 345
- RMAP\_TARGET\_PXI\_IF\_setAddressOffset
  - Configuration, 355
- RMAP\_TARGET\_PXI\_IF\_setAuthCommands
  - Configuration, 355
- RMAP\_TARGET\_PXI\_IF\_setAuthControlMode
  - Configuration, 356
- RMAP\_TARGET\_PXI\_IF\_setAuthKeyRange
  - Configuration, 356
- RMAP\_TARGET\_PXI\_IF\_setAuthLogicalAddress↔
  - Range
  - Configuration, 357
- RMAP\_TARGET\_PXI\_IF\_setAuthMemoryAddress↔
  - Range
  - Configuration, 357
- RMAP\_TARGET\_PXI\_IF\_setAuthProtocolId
  - Configuration, 357
- RMAP\_TARGET\_PXI\_IF\_setEnabledNotifications
  - Notifications, 365
- RMAP\_TARGET\_PXI\_IF\_setInterfaceMode
  - Configuration, 358
- RMAP\_TARGET\_PXI\_IF\_startReceivingNotifications
  - Notifications, 365
- RMAP\_TARGET\_PXI\_IF\_stopReceivingNotifications
  - Notifications, 366
- RMAP\_TARGET\_PXI\_IF\_unregisterNotificationListener
  - Notifications, 366
- RMAP\_TARGET\_PXI\_IF\_unregisterNotification↔
  - ListenerWithContext
  - Notifications, 366

- RMAP\_TARGET\_PXI\_IF\_writeMemory Configuration, 358
- RMAP\_TOO\_MUCH\_DATA
  - RMAP Packet Library, 678
- RMAP\_UNUSED\_PACKET\_TYPE\_OR\_COMMAND↵\_CODE
  - RMAP Packet Library, 678
- RMAP\_VERIFY\_BEFORE\_WRITE\_BIT
  - RMAP Packet Library, 676
- RMAP\_VERIFY\_BUFFER\_OVERRUN
  - RMAP Packet Library, 678
- RMAP\_WRITE\_COMMAND
  - RMAP Packet Library, 677
- RMAP\_WRITE\_OPERATION\_BIT
  - RMAP Packet Library, 677
- RMAP\_WRITE\_REPLY
  - RMAP Packet Library, 677
- RMAPPACKETLIBRARY\_CC
  - RMAP Packet Library, 677
- Remote Devices, 557
  - STAR\_CFG\_createRemoteDeviceIdentifier, 558
  - STAR\_CFG\_destroyRemoteDeviceDescription↵List, 558
  - STAR\_CFG\_destroyRemoteDeviceIdentifier, 558
  - STAR\_CFG\_getRemoteDeviceDescriptionList, 559
  - STAR\_CFG\_getRemoteDeviceDescriptionName↵String, 559
  - STAR\_CFG\_getRemoteDeviceLocalChannel, 559
  - STAR\_CFG\_getRemoteDeviceLocalDevice, 561
  - STAR\_CFG\_getRemoteDevicePath, 561
  - STAR\_CFG\_getRemoteDeviceRetPath, 561
- rmap\_packet\_library.h, 755
- rmap\_target\_pxi\_if.h, 760
- rmap\_target\_types.h, 762
- RmapCommandParameters, 342
- Router Configuration API, 563
- Router Control and Status Configuration, 599
  - CFG\_MK2\_getTimeCodeFlagMode, 602
  - CFG\_MK2\_setTimeCodeFlagMode, 602
  - CFG\_ROUTER\_getRouterGlobalSettings, 602
  - CFG\_ROUTER\_getTimeCodeFlagMode, 604
  - CFG\_ROUTER\_getTimeCodeValue, 604
  - CFG\_ROUTER\_setRouterGlobalSettings, 605
  - CFG\_ROUTER\_setTimeCodeFlagMode, 605
  - STAR\_CFG\_PORT\_TIMEOUT, 601
  - STAR\_CFG\_PORT\_TIMEOUT\_100MS, 601
  - STAR\_CFG\_PORT\_TIMEOUT\_100US, 601
  - STAR\_CFG\_PORT\_TIMEOUT\_10MS, 601
  - STAR\_CFG\_PORT\_TIMEOUT\_1MS, 601
  - STAR\_CFG\_PORT\_TIMEOUT\_1S, 601
  - STAR\_CFG\_TIMEOUT\_MODE, 601
  - STAR\_CFG\_TIMEOUT\_MODE\_BLOCKING, 601
  - STAR\_CFG\_TIMEOUT\_MODE\_WATCHDOG, 601
- Router Mk2S Configuration API, 568
- Routing Table, 590
  - CFG\_ROUTER\_getRoutingTableEntry, 591
  - CFG\_ROUTER\_setRoutingTableEntry, 592
- SPW\_ACTION
  - cfg\_api\_mk2\_types.h, 740
- SPW\_ERROR
  - Error Injection, 532
- SPW\_ERROR\_DECREMENT\_CREDIT
  - Error Injection, 533
- SPW\_ERROR\_DISCONNECT
  - Error Injection, 532
- SPW\_ERROR\_ESCAPE
  - Error Injection, 532
- SPW\_ERROR\_INCREMENT\_CREDIT
  - Error Injection, 533
- SPW\_ERROR\_INSERT\_FCT
  - Error Injection, 532
- SPW\_ERROR\_PARITY
  - Error Injection, 532
- SPW\_ERROR\_SUPPRESS\_FCT
  - Error Injection, 532
- SPW\_TRANSMIT\_PACKET
  - cfg\_api\_mk2\_types.h, 740
- STAR-API, 199
- STAR-Dundee Types, 711
  - FALSE, 711
  - REGISTER, 712
  - TRUE, 711
  - U16, 712
  - U16\_MAX, 711
  - U32, 712
  - U8, 712
- STAR\_API\_CC
  - common.h, 749
- STAR\_API\_MODULE\_NAME
  - common.h, 749
- STAR\_APP\_ID
  - General API Functions, 202
- STAR\_BROADCAST\_MESSAGE, 253
- STAR\_BROADCAST\_MESSAGE\_STATUS\_FLAG
  - Stream Items, 255
- STAR\_BROADCAST\_MESSAGE\_STATUS\_FLAG↵\_DELAYED
  - Stream Items, 255
- STAR\_BROADCAST\_MESSAGE\_STATUS\_FLAG↵\_ATE
  - Stream Items, 255
- STAR\_BUS\_CPCI
  - Device and Driver Management, 219
- STAR\_BUS\_PCI
  - Device and Driver Management, 219
- STAR\_BUS\_PCIE
  - Device and Driver Management, 219
- STAR\_BUS\_TCP
  - Device and Driver Management, 219
- STAR\_BUS\_TYPE
  - Device and Driver Management, 219
- STAR\_BUS\_UNKNOWN
  - Device and Driver Management, 219
- STAR\_BUS\_USB

- Device and Driver Management, 219
- STAR\_BUS\_VIRTUAL
  - Device and Driver Management, 219
- STAR\_CFG\_BRICK\_MK2\_ERRORS, 715
- STAR\_CFG\_BRICK\_MK2\_LINK\_FREQ
  - Link Speed, 297
- STAR\_CFG\_BRICK\_MK2\_LINK\_FREQ\_120
  - Link Speed, 297
- STAR\_CFG\_BRICK\_MK2\_LINK\_FREQ\_130
  - Link Speed, 297
- STAR\_CFG\_BRICK\_MK2\_LINK\_FREQ\_140
  - Link Speed, 297
- STAR\_CFG\_BRICK\_MK2\_LINK\_FREQ\_150
  - Link Speed, 298
- STAR\_CFG\_BRICK\_MK2\_LINK\_FREQ\_160
  - Link Speed, 298
- STAR\_CFG\_BRICK\_MK2\_LINK\_FREQ\_180
  - Link Speed, 298
- STAR\_CFG\_BRICK\_MK2\_LINK\_FREQ\_200
  - Link Speed, 298
- STAR\_CFG\_BRICK\_MK2\_LINK\_SPEED\_UNITS\_KB↔PS
  - Measured Link Speed, 312
- STAR\_CFG\_BRICK\_MK3\_PULSE\_FREQ
  - Timestamping, 318
- STAR\_CFG\_BRICK\_MK3\_PULSE\_FREQ\_1
  - Timestamping, 318
- STAR\_CFG\_BRICK\_MK3\_PULSE\_FREQ\_10
  - Timestamping, 318
- STAR\_CFG\_BRICK\_MK3\_PULSE\_FREQ\_100
  - Timestamping, 318
- STAR\_CFG\_BRICK\_MK3\_PULSE\_FREQ\_1000
  - Timestamping, 318
- STAR\_CFG\_BRICK\_MK3\_TIMESTAMP\_METHOD
  - Timestamping, 318
- STAR\_CFG\_BRICK\_MK3\_TIMESTAMP\_METHOD↔EXTERNAL\_TRIGGER
  - Timestamping, 318
- STAR\_CFG\_BRICK\_MK3\_TIMESTAMP\_METHOD↔NONE
  - Timestamping, 318
- STAR\_CFG\_BRICK\_MK3\_TIMESTAMP\_METHOD↔PULSE\_GENERATOR
  - Timestamping, 318
- STAR\_CFG\_CONFIG\_PORT\_ERRORS, 577
- STAR\_CFG\_DEVICE\_IDENTIFIER\_INFO, 571
- STAR\_CFG\_DEVICE\_STR\_MAX\_LEN
  - Device Information, 572
- STAR\_CFG\_DEVICE\_TYPE
  - Device Information, 573
- STAR\_CFG\_DEVICE\_TYPE\_INVALID
  - Device Information, 573
- STAR\_CFG\_DEVICE\_TYPE\_ROUTER
  - Device Information, 573
- STAR\_CFG\_DEVICE\_TYPE\_UNKNOWN
  - Device Information, 573
- STAR\_CFG\_EXTERNAL\_PORT\_ERRORS, 581
- STAR\_CFG\_EXTERNAL\_PORT\_STATUS, 581
- STAR\_CFG\_FPGA\_INFO, 610
- STAR\_CFG\_GAR\_ENTRY, 590
- STAR\_CFG\_MANUFACTURER\_STR\_MAX\_LEN
  - Device Information, 572
- STAR\_CFG\_MK2\_BASE\_TRANSMIT\_CLOCK, 506
- STAR\_CFG\_MK2\_HARDWARE\_INFO, 521
- STAR\_CFG\_MK2\_LINK\_SPEED\_UNITS\_KBPS
  - Measured Link Speed, 512
- STAR\_CFG\_NETWORK\_DISCOVERY\_INFO, 571
- STAR\_CFG\_PCIMK2\_LINK\_FREQ
  - Link Speed, 294
- STAR\_CFG\_PCIMK2\_LINK\_FREQ\_120
  - Link Speed, 294
- STAR\_CFG\_PCIMK2\_LINK\_FREQ\_128
  - Link Speed, 294
- STAR\_CFG\_PCIMK2\_LINK\_FREQ\_140
  - Link Speed, 295
- STAR\_CFG\_PCIMK2\_LINK\_FREQ\_150
  - Link Speed, 295
- STAR\_CFG\_PCIMK2\_LINK\_FREQ\_160
  - Link Speed, 295
- STAR\_CFG\_PCIMK2\_LINK\_FREQ\_180
  - Link Speed, 295
- STAR\_CFG\_PCIMK2\_LINK\_FREQ\_200
  - Link Speed, 295
- STAR\_CFG\_PORT\_TIMEOUT
  - Router Control and Status Configuration, 601
- STAR\_CFG\_PORT\_TIMEOUT\_100MS
  - Router Control and Status Configuration, 601
- STAR\_CFG\_PORT\_TIMEOUT\_100US
  - Router Control and Status Configuration, 601
- STAR\_CFG\_PORT\_TIMEOUT\_10MS
  - Router Control and Status Configuration, 601
- STAR\_CFG\_PORT\_TIMEOUT\_1MS
  - Router Control and Status Configuration, 601
- STAR\_CFG\_PORT\_TIMEOUT\_1S
  - Router Control and Status Configuration, 601
- STAR\_CFG\_PORT\_TYPE
  - Port Status and Control, 582
- STAR\_CFG\_PORT\_TYPE\_CONFIGURATION
  - Port Status and Control, 582
- STAR\_CFG\_PORT\_TYPE\_EXTERNAL
  - Port Status and Control, 583
- STAR\_CFG\_PORT\_TYPE\_INVALID
  - Port Status and Control, 583
- STAR\_CFG\_PORT\_TYPE\_LINK
  - Port Status and Control, 583
- STAR\_CFG\_ROUTER\_GLOBAL\_STATE, 600
- STAR\_CFG\_SPW\_LINK\_ERRORS, 579
- STAR\_CFG\_SPW\_LINK\_STATE
  - Port Status and Control, 583
- STAR\_CFG\_SPW\_LINK\_STATE\_CONNECTING
  - Port Status and Control, 583
- STAR\_CFG\_SPW\_LINK\_STATE\_ERROR\_RESET
  - Port Status and Control, 583
- STAR\_CFG\_SPW\_LINK\_STATE\_ERROR\_WAIT
  - Port Status and Control, 583
- STAR\_CFG\_SPW\_LINK\_STATE\_INVALID

- Port Status and Control, [583](#)
- STAR\_CFG\_SPW\_LINK\_STATE\_READY
  - Port Status and Control, [583](#)
- STAR\_CFG\_SPW\_LINK\_STATE\_RUN
  - Port Status and Control, [583](#)
- STAR\_CFG\_SPW\_LINK\_STATE\_STARTED
  - Port Status and Control, [583](#)
- STAR\_CFG\_SPW\_LINK\_STATUS, [580](#)
- STAR\_CFG\_TIMEOUT\_MODE
  - Router Control and Status Configuration, [601](#)
- STAR\_CFG\_TIMEOUT\_MODE\_BLOCKING
  - Router Control and Status Configuration, [601](#)
- STAR\_CFG\_TIMEOUT\_MODE\_WATCHDOG
  - Router Control and Status Configuration, [601](#)
- STAR\_CFG\_createRemoteDeviceIdentifier
  - Remote Devices, [558](#)
- STAR\_CFG\_destroyRemoteDeviceDescriptionList
  - Remote Devices, [558](#)
- STAR\_CFG\_destroyRemoteDeviceIdentifier
  - Remote Devices, [558](#)
- STAR\_CFG\_getRemoteDeviceDescriptionList
  - Remote Devices, [559](#)
- STAR\_CFG\_getRemoteDeviceDescriptionNameString
  - Remote Devices, [559](#)
- STAR\_CFG\_getRemoteDeviceLocalChannel
  - Remote Devices, [559](#)
- STAR\_CFG\_getRemoteDeviceLocalDevice
  - Remote Devices, [561](#)
- STAR\_CFG\_getRemoteDevicePath
  - Remote Devices, [561](#)
- STAR\_CFG\_getRemoteDeviceRetPath
  - Remote Devices, [561](#)
- STAR\_CHANNEL\_DIRECTION
  - Channel Management, [237](#)
- STAR\_CHANNEL\_DIRECTION\_IN
  - Channel Management, [237](#)
- STAR\_CHANNEL\_DIRECTION\_INOUT
  - Channel Management, [237](#)
- STAR\_CHANNEL\_DIRECTION\_OUT
  - Channel Management, [237](#)
- STAR\_CHANNEL\_ID
  - Channel Management, [236](#)
- STAR\_CHANNEL\_LISTENER\_ID
  - Channel Management, [236](#)
- STAR\_CHANNEL\_MASK
  - Device and Driver Management, [210](#)
- STAR\_CHANNEL\_TYPE
  - Channel Management, [237](#)
- STAR\_CHANNEL\_TYPE\_APPLICATION
  - Channel Management, [237](#)
- STAR\_CHANNEL\_TYPE\_DEVICE
  - Channel Management, [237](#)
- STAR\_CHANNEL\_TYPE\_NOT\_OPEN
  - Channel Management, [237](#)
- STAR\_ChannelEventFunc
  - Channel Management, [236](#)
- STAR\_DATA\_CHUNK, [248](#)
- STAR\_DEVICE\_ALL
  - Device and Driver Management, [210](#)
- STAR\_DEVICE\_BRICK\_MK2
  - Device and Driver Management, [210](#)
- STAR\_DEVICE\_BRICK\_MK3
  - Device and Driver Management, [211](#)
- STAR\_DEVICE\_CONFIG\_NOT\_SUPPORTED
  - Device and Driver Management, [211](#)
- STAR\_DEVICE\_CONFIG\_SUPPORTED
  - Device and Driver Management, [211](#)
- STAR\_DEVICE\_CONFORMANCE\_TESTER
  - Device and Driver Management, [211](#)
- STAR\_DEVICE\_CONFORMANCE\_TESTER\_MK2
  - Device and Driver Management, [211](#)
- STAR\_DEVICE\_CPCI\_MK2
  - Device and Driver Management, [212](#)
- STAR\_DEVICE\_EGSE
  - Device and Driver Management, [212](#)
- STAR\_DEVICE\_EGSE\_MK2
  - Device and Driver Management, [212](#)
- STAR\_DEVICE\_ID
  - Device and Driver Management, [218](#)
- STAR\_DEVICE\_IP\_TUNNEL
  - Device and Driver Management, [212](#)
- STAR\_DEVICE\_LINK\_ANALYSER
  - Device and Driver Management, [212](#)
- STAR\_DEVICE\_LINK\_ANALYSER\_MK2
  - Device and Driver Management, [212](#)
- STAR\_DEVICE\_LINK\_ANALYSER\_MK3
  - Device and Driver Management, [213](#)
- STAR\_DEVICE\_LISTENER\_ID
  - Device and Driver Management, [218](#)
- STAR\_DEVICE\_PCI
  - Device and Driver Management, [213](#)
- STAR\_DEVICE\_PCI\_MK2
  - Device and Driver Management, [213](#)
- STAR\_DEVICE\_PCIE
  - Device and Driver Management, [213](#)
- STAR\_DEVICE\_PXI\_INTERFACE
  - Device and Driver Management, [213](#)
- STAR\_DEVICE\_PXI\_INTERFACE\_MK2
  - Device and Driver Management, [213](#)
- STAR\_DEVICE\_PXI\_RMAP
  - Device and Driver Management, [214](#)
- STAR\_DEVICE\_PXI\_RMAP\_MK2
  - Device and Driver Management, [214](#)
- STAR\_DEVICE\_PXI\_ROUTER\_12
  - Device and Driver Management, [214](#)
- STAR\_DEVICE\_PXI\_ROUTER\_8
  - Device and Driver Management, [214](#)
- STAR\_DEVICE\_PXI\_ROUTER\_MK2
  - Device and Driver Management, [214](#)
- STAR\_DEVICE\_RECORDER
  - Device and Driver Management, [215](#)
- STAR\_DEVICE\_ROUTER\_MK2S
  - Device and Driver Management, [215](#)
- STAR\_DEVICE\_ROUTER\_USB
  - Device and Driver Management, [215](#)
- STAR\_DEVICE\_RTC

- Device and Driver Management, 215
- STAR\_DEVICE\_SERIAL\_SIZE
  - Device and Driver Management, 215
- STAR\_DEVICE\_SPLT
  - Device and Driver Management, 215
- STAR\_DEVICE\_STAR\_FIRE
  - Device and Driver Management, 216
- STAR\_DEVICE\_STAR\_FIRE\_MK3
  - Device and Driver Management, 216
- STAR\_DEVICE\_STAR\_ULTRA\_PCIE
  - Device and Driver Management, 216
- STAR\_DEVICE\_TRAFFIC\_GENERATOR
  - Device and Driver Management, 216
- STAR\_DEVICE\_TXRX\_NOT\_SUPPORTED
  - Device and Driver Management, 216
- STAR\_DEVICE\_TXRX\_SUPPORTED
  - Device and Driver Management, 216
- STAR\_DEVICE\_TYPE
  - Device and Driver Management, 217
- STAR\_DEVICE\_UNKNOWN
  - Device and Driver Management, 217
- STAR\_DEVICE\_USB\_BRICK
  - Device and Driver Management, 217
- STAR\_DEVICE\_WBS\_II
  - Device and Driver Management, 217
- STAR\_DRIVER\_ID
  - Device and Driver Management, 218
- STAR\_DRIVER\_INVALID
  - Device and Driver Management, 219
- STAR\_DRIVER\_LISTENER\_ID
  - Device and Driver Management, 218
- STAR\_DRIVER\_TYPE
  - Device and Driver Management, 219
- STAR\_DRIVER\_TYPE\_PCI
  - Device and Driver Management, 219
- STAR\_DRIVER\_TYPE\_TCPIP
  - Device and Driver Management, 219
- STAR\_DRIVER\_TYPE\_USB
  - Device and Driver Management, 219
- STAR\_DRIVER\_TYPE\_VIRTUAL
  - Device and Driver Management, 219
- STAR\_DeviceEventFunc
  - Device and Driver Management, 218
- STAR\_DriverEventFunc
  - Device and Driver Management, 218
- STAR\_EOP\_TYPE
  - Stream Items, 255
- STAR\_EOP\_TYPE\_EEP
  - Stream Items, 256
- STAR\_EOP\_TYPE\_EOP
  - Stream Items, 255
- STAR\_EOP\_TYPE\_INVALID
  - Stream Items, 255
- STAR\_EOP\_TYPE\_NONE
  - Stream Items, 256
- STAR\_ERROR\_IN\_DATA\_DISCONNECT
  - Stream Items, 256
- STAR\_ERROR\_IN\_DATA\_INJECT, 251
- STAR\_ERROR\_IN\_DATA\_PARITY
  - Stream Items, 256
- STAR\_ERROR\_IN\_DATA\_TYPE
  - Stream Items, 256
- STAR\_LINK\_SPEED\_EVENT, 250
- STAR\_LINK\_STATE\_EVENT, 246
- STAR\_RECEIVE\_BROADCAST\_MESSAGES
  - Transfer Operations, 278
- STAR\_RECEIVE\_CHUNKS
  - Transfer Operations, 278
- STAR\_RECEIVE\_FCTS
  - Transfer Operations, 278
- STAR\_RECEIVE\_LINK\_SPEED\_EVENTS
  - Transfer Operations, 278
- STAR\_RECEIVE\_LINK\_STATE\_EVENTS
  - Transfer Operations, 278
- STAR\_RECEIVE\_MASK
  - Transfer Operations, 278
- STAR\_RECEIVE\_NULL
  - Transfer Operations, 278
- STAR\_RECEIVE\_PACKETS
  - Transfer Operations, 278
- STAR\_RECEIVE\_TIMECODES
  - Transfer Operations, 278
- STAR\_RECEIVE\_TIMESTAMP\_EVENTS
  - Transfer Operations, 278
- STAR\_SPACEWIRE\_ADDRESS, 247
- STAR\_SPACEWIRE\_PACKET, 249
- STAR\_STREAM\_ITEM, 254
- STAR\_STREAM\_ITEM\_TYPE
  - Stream Items, 256
- STAR\_STREAM\_ITEM\_TYPE\_BROADCAST\_MESSAGE
  - Stream Items, 256
- STAR\_STREAM\_ITEM\_TYPE\_DATA\_CHUNK
  - Stream Items, 256
- STAR\_STREAM\_ITEM\_TYPE\_ERROR\_INJECT
  - Stream Items, 256
- STAR\_STREAM\_ITEM\_TYPE\_LINK\_SPEED\_EVENT
  - Stream Items, 256
- STAR\_STREAM\_ITEM\_TYPE\_LINK\_STATE\_EVENT
  - Stream Items, 256
- STAR\_STREAM\_ITEM\_TYPE\_SPACEWIRE\_PACKET
  - Stream Items, 256
- STAR\_STREAM\_ITEM\_TYPE\_TIMECODE
  - Stream Items, 256
- STAR\_STREAM\_ITEM\_TYPE\_TIMESTAMP\_EVENT
  - Stream Items, 256
- STAR\_TIMECODE, 250
- STAR\_TIMESTAMP\_DIRECTION
  - Stream Items, 256
- STAR\_TIMESTAMP\_DIRECTION\_RECEIVE
  - Stream Items, 257
- STAR\_TIMESTAMP\_DIRECTION\_TRANSMIT
  - Stream Items, 257
- STAR\_TIMESTAMP\_EVENT, 252
- STAR\_TIMESTAMP\_TYPE



- Stream Items, 257
- STAR\_TIMESTAMP\_TYPE\_DATA
  - Stream Items, 257
- STAR\_TIMESTAMP\_TYPE\_ERROR
  - Stream Items, 257
- STAR\_TIMESTAMP\_TYPE\_LINK\_SPEED
  - Stream Items, 257
- STAR\_TIMESTAMP\_TYPE\_LINK\_STATE
  - Stream Items, 257
- STAR\_TIMESTAMP\_TYPE\_TIMECODE
  - Stream Items, 257
- STAR\_TRANSFER\_OPERATION
  - Transfer Operations, 277
- STAR\_TRANSFER\_STATUS
  - Transfer Operations, 278
- STAR\_TRANSFER\_STATUS\_CANCELLED
  - Transfer Operations, 279
- STAR\_TRANSFER\_STATUS\_COMPLETE
  - Transfer Operations, 279
- STAR\_TRANSFER\_STATUS\_ERROR
  - Transfer Operations, 279
- STAR\_TRANSFER\_STATUS\_NOT\_STARTED
  - Transfer Operations, 279
- STAR\_TRANSFER\_STATUS\_STARTED
  - Transfer Operations, 279
- STAR\_TransferOperationListenerFunc
  - Transfer Operations, 277
- STAR\_VERSION\_INFO, 793
- STAR\_cancelTransferOperation
  - Transfer Operations, 279
- STAR\_cancelTransferOperationWaits
  - Transfer Operations, 279
- STAR\_closeChannel
  - Channel Management, 237
- STAR\_createAddress
  - Stream Items, 257
- STAR\_createBroadcastMessage
  - Stream Items, 258
- STAR\_createDataChunk
  - Stream Items, 258
- STAR\_createErrorInData
  - Stream Items, 258
- STAR\_createPacket
  - Stream Items, 259
- STAR\_createRxOperation
  - Transfer Operations, 279
- STAR\_createRxOperationEx
  - Transfer Operations, 280
- STAR\_createTimeCode
  - Stream Items, 260
- STAR\_createTxOperation
  - Transfer Operations, 281
- STAR\_destroyAddress
  - Stream Items, 260
- STAR\_destroyDeviceList
  - Device and Driver Management, 220
- STAR\_destroyDriverList
  - Device and Driver Management, 221
- STAR\_destroyPacketData
  - Stream Items, 260
- STAR\_destroyStreamItem
  - Stream Items, 261
- STAR\_destroyString
  - General API Functions, 202
- STAR\_destroyTransferItemList
  - Transfer Operations, 281
- STAR\_destroyVersionInfo
  - General API Functions, 202
- STAR\_destroyVersionInfoList
  - General API Functions, 202
- STAR\_disposeTransferOperation
  - Transfer Operations, 282
- STAR\_getAllVersions
  - General API Functions, 203
- STAR\_getApiVersion
  - General API Functions, 203
- STAR\_getApplicationAttachedToChannel
  - General API Functions, 203
- STAR\_getApplicationID
  - General API Functions, 204
- STAR\_getApplicationName
  - General API Functions, 204
- STAR\_getApplicationNameForID
  - General API Functions, 204
- STAR\_getBroadcastMessageChannel
  - Stream Items, 261
- STAR\_getBroadcastMessageDataWord1
  - Stream Items, 261
- STAR\_getBroadcastMessageDataWord2
  - Stream Items, 262
- STAR\_getBroadcastMessageDelayedFlag
  - Stream Items, 262
- STAR\_getBroadcastMessageLateFlag
  - Stream Items, 262
- STAR\_getBroadcastMessageStatus
  - Stream Items, 263
- STAR\_getBroadcastMessageType
  - Stream Items, 263
- STAR\_getChannelDirection
  - Channel Management, 238
- STAR\_getChunkData
  - Stream Items, 263
- STAR\_getChunkDataLength
  - Stream Items, 264
- STAR\_getChunkEop
  - Stream Items, 264
- STAR\_getChunkSop
  - Stream Items, 264
- STAR\_getDeviceBusType
  - Device and Driver Management, 221
- STAR\_getDeviceBusTypeAsString
  - Device and Driver Management, 221
- STAR\_getDeviceChannels
  - Device and Driver Management, 222
- STAR\_getDeviceConfigCapabilities
  - Device and Driver Management, 222

- STAR\_getDeviceDriver
  - Device and Driver Management, 222
- STAR\_getDeviceFirmwareVersion
  - Device and Driver Management, 223
- STAR\_getDeviceIndex
  - Device and Driver Management, 223
- STAR\_getDeviceList
  - Device and Driver Management, 223
- STAR\_getDeviceListForDriver
  - Device and Driver Management, 225
- STAR\_getDeviceListForType
  - Device and Driver Management, 225
- STAR\_getDeviceListForTypes
  - Device and Driver Management, 226
- STAR\_getDeviceName
  - Device and Driver Management, 226
- STAR\_getDeviceSerialNumber
  - Device and Driver Management, 227
- STAR\_getDeviceTxRxCapabilities
  - Device and Driver Management, 227
- STAR\_getDeviceType
  - Device and Driver Management, 228
- STAR\_getDeviceTypeAsString
  - Device and Driver Management, 228
- STAR\_getDriverList
  - Device and Driver Management, 228
- STAR\_getDriverType
  - Device and Driver Management, 230
- STAR\_getDriverVersion
  - Device and Driver Management, 230
- STAR\_getLinkSpeedEventPort
  - Stream Items, 265
- STAR\_getLinkSpeedEventSpeed
  - Stream Items, 265
- STAR\_getLinkStateEventCount
  - Stream Items, 265
- STAR\_getLinkStateEventPort
  - Stream Items, 266
- STAR\_getLocalDeviceAttachedToChannel
  - Device and Driver Management, 231
- STAR\_getNextTransferItem
  - Transfer Operations, 282
- STAR\_getPacketAddress
  - Stream Items, 266
- STAR\_getPacketData
  - Stream Items, 266
- STAR\_getPacketEOP
  - Stream Items, 267
- STAR\_getPacketLength
  - Stream Items, 267
- STAR\_getSpaceWireAddressPath
  - Transfer Operations, 284
- STAR\_getSpaceWireAddressPathLength
  - Transfer Operations, 284
- STAR\_getTimeCodeValue
  - Stream Items, 267
- STAR\_getTimestampEventDirection
  - Stream Items, 268
- STAR\_getTimestampEventEndValue
  - Stream Items, 268
- STAR\_getTimestampEventRawCounters
  - Stream Items, 268
- STAR\_getTimestampEventStartValue
  - Stream Items, 270
- STAR\_getTimestampEventType
  - Stream Items, 270
- STAR\_getTransferItem
  - Transfer Operations, 284
- STAR\_getTransferItemCount
  - Transfer Operations, 285
- STAR\_getTransferItemList
  - Transfer Operations, 285
- STAR\_getTransferOperationStatus
  - Transfer Operations, 286
- STAR\_isChannelOpen
  - Device and Driver Management, 231
- STAR\_isDeviceVirtual
  - Virtual Devices and Drivers, 292
- STAR\_isDriverVirtual
  - Virtual Devices and Drivers, 292
- STAR\_isLinkStateEventDisconnectError
  - Stream Items, 271
- STAR\_isLinkStateEventEscapeError
  - Stream Items, 271
- STAR\_isLinkStateEventLinkRunning
  - Stream Items, 271
- STAR\_isLinkStateEventParityError
  - Stream Items, 272
- STAR\_isLinkStateEventReceiveCreditError
  - Stream Items, 272
- STAR\_isLinkStateEventTransmitCreditError
  - Stream Items, 272
- STAR\_openChannelBetweenLocalDevices
  - Channel Management, 238
- STAR\_openChannelToLocalDevice
  - Channel Management, 238
- STAR\_openChannelToLocalDeviceEx
  - Channel Management, 239
- STAR\_receivePacket
  - Transfer Operations, 286
- STAR\_registerChannelListener
  - Channel Management, 240
- STAR\_registerChannelListenerForDevice
  - Channel Management, 240
- STAR\_registerChannelListenerForDriver
  - Channel Management, 242
- STAR\_registerDeviceListener
  - Device and Driver Management, 231
- STAR\_registerDeviceListenerForDevice
  - Device and Driver Management, 232
- STAR\_registerDeviceListenerForDriver
  - Device and Driver Management, 232
- STAR\_registerDriverListener
  - Device and Driver Management, 232
- STAR\_registerTransferCompletionListener
  - Transfer Operations, 287

- STAR\_resetDevice
  - Device and Driver Management, 233
- STAR\_resetDeviceName
  - Device and Driver Management, 233
- STAR\_rxOperationExGetBroadcastMessage
  - Transfer Operations, 287
- STAR\_rxOperationExGetDataChunk
  - Transfer Operations, 288
- STAR\_rxOperationExGetItemType
  - Transfer Operations, 288
- STAR\_rxOperationExGetNumItems
  - Transfer Operations, 288
- STAR\_rxOperationExGetStatus
  - Transfer Operations, 289
- STAR\_setApplicationName
  - General API Functions, 205
- STAR\_setDeviceName
  - Device and Driver Management, 233
- STAR\_setPacketAddress
  - Stream Items, 272
- STAR\_setPacketEOP
  - Stream Items, 274
- STAR\_setTimeCodeValue
  - Stream Items, 274
- STAR\_submitTransferOperation
  - Transfer Operations, 289
- STAR\_submitTransferOperationList
  - Transfer Operations, 289
- STAR\_transmitPacket
  - Transfer Operations, 290
- STAR\_unregisterChannelListener
  - Channel Management, 242
- STAR\_unregisterDeviceListener
  - Device and Driver Management, 234
- STAR\_unregisterDriverListener
  - Device and Driver Management, 234
- STAR\_unregisterTransferCompletionListener
  - Transfer Operations, 290
- STAR\_waitOnTransferOperationCompletion
  - Transfer Operations, 291
- STATISTICS\_LOOP\_ID
  - PCI Mk2 Packet Subsystem, 539
- star-api.h, 764
- star-dundee\_types.h, 764
- Stream Items, 243
  - STAR\_BROADCAST\_MESSAGE\_STATUS\_FL↔AG, 255
  - STAR\_BROADCAST\_MESSAGE\_STATUS\_FL↔AG\_DELAYED, 255
  - STAR\_BROADCAST\_MESSAGE\_STATUS\_FL↔AG\_LATE, 255
  - STAR\_EOP\_TYPE, 255
  - STAR\_EOP\_TYPE\_EEP, 256
  - STAR\_EOP\_TYPE\_EOP, 255
  - STAR\_EOP\_TYPE\_INVALID, 255
  - STAR\_EOP\_TYPE\_NONE, 256
  - STAR\_ERROR\_IN\_DATA\_DISCONNECT, 256
  - STAR\_ERROR\_IN\_DATA\_PARITY, 256
  - STAR\_ERROR\_IN\_DATA\_TYPE, 256
  - STAR\_STREAM\_ITEM\_TYPE, 256
  - STAR\_STREAM\_ITEM\_TYPE\_BROADCAST\_↔MESSAGE, 256
  - STAR\_STREAM\_ITEM\_TYPE\_DATA\_CHUNK, 256
  - STAR\_STREAM\_ITEM\_TYPE\_ERROR\_INJECT, 256
  - STAR\_STREAM\_ITEM\_TYPE\_LINK\_SPEED\_E↔VENT, 256
  - STAR\_STREAM\_ITEM\_TYPE\_LINK\_STATE\_E↔VENT, 256
  - STAR\_STREAM\_ITEM\_TYPE\_SPACEWIRE\_P↔ACKET, 256
  - STAR\_STREAM\_ITEM\_TYPE\_TIMECODE, 256
  - STAR\_STREAM\_ITEM\_TYPE\_TIMESTAMP\_E↔VENT, 256
  - STAR\_TIMESTAMP\_DIRECTION, 256
  - STAR\_TIMESTAMP\_DIRECTION\_RECEIVE, 257
  - STAR\_TIMESTAMP\_DIRECTION\_TRANSMIT, 257
  - STAR\_TIMESTAMP\_TYPE, 257
  - STAR\_TIMESTAMP\_TYPE\_DATA, 257
  - STAR\_TIMESTAMP\_TYPE\_ERROR, 257
  - STAR\_TIMESTAMP\_TYPE\_LINK\_SPEED, 257
  - STAR\_TIMESTAMP\_TYPE\_LINK\_STATE, 257
  - STAR\_TIMESTAMP\_TYPE\_TIMECODE, 257
  - STAR\_createAddress, 257
  - STAR\_createBroadcastMessage, 258
  - STAR\_createDataChunk, 258
  - STAR\_createErrorInData, 258
  - STAR\_createPacket, 259
  - STAR\_createTimeCode, 260
  - STAR\_destroyAddress, 260
  - STAR\_destroyPacketData, 260
  - STAR\_destroyStreamItem, 261
  - STAR\_getBroadcastMessageChannel, 261
  - STAR\_getBroadcastMessageDataWord1, 261
  - STAR\_getBroadcastMessageDataWord2, 262
  - STAR\_getBroadcastMessageDelayedFlag, 262
  - STAR\_getBroadcastMessageLateFlag, 262
  - STAR\_getBroadcastMessageStatus, 263
  - STAR\_getBroadcastMessageType, 263
  - STAR\_getChunkData, 263
  - STAR\_getChunkDataLength, 264
  - STAR\_getChunkEop, 264
  - STAR\_getChunkSop, 264
  - STAR\_getLinkSpeedEventPort, 265
  - STAR\_getLinkSpeedEventSpeed, 265
  - STAR\_getLinkStateEventCount, 265
  - STAR\_getLinkStateEventPort, 266
  - STAR\_getPacketAddress, 266
  - STAR\_getPacketData, 266
  - STAR\_getPacketEOP, 267
  - STAR\_getPacketLength, 267
  - STAR\_getTimeCodeValue, 267
  - STAR\_getTimestampEventDirection, 268
  - STAR\_getTimestampEventEndValue, 268



- STAR\_getTimestampEventRawCounters, 268
- STAR\_getTimestampEventStartValue, 270
- STAR\_getTimestampEventType, 270
- STAR\_isLinkStateEventDisconnectError, 271
- STAR\_isLinkStateEventEscapeError, 271
- STAR\_isLinkStateEventLinkRunning, 271
- STAR\_isLinkStateEventParityError, 272
- STAR\_isLinkStateEventReceiveCreditError, 272
- STAR\_isLinkStateEventTransmitCreditError, 272
- STAR\_setPacketAddress, 272
- STAR\_setPacketEOP, 274
- STAR\_setTimeCodeValue, 274
- stream\_item.h, 765
- stream\_item\_types.h, 768
- TARGET\_AUTH\_CMD
  - Configuration, 349
- TARGET\_AUTH\_CMD\_MASK
  - Configuration, 349
- TARGET\_AUTH\_MODE
  - Configuration, 349
- TARGET\_AUTH\_MODE\_AUTOMATIC
  - Configuration, 349
- TARGET\_AUTH\_MODE\_MANUAL
  - Configuration, 349
- TARGET\_ENGINE, 348
- TARGET\_STATUS
  - Configuration, 349
- TIME\_CODE\_ACTION
  - Actions, 406
- TIME\_CODE\_ACTION\_ALL
  - Actions, 406
- TIME\_CODE\_ACTION\_MASK
  - Events, 384
- TIME\_CODE\_ACTION\_NONE
  - Actions, 406
- TIME\_CODE\_ACTION\_TX
  - Actions, 406
- TIME\_CODE\_EVENT
  - Events, 385
- TIME\_CODE\_EVENT\_ALL
  - Events, 385
- TIME\_CODE\_EVENT\_MASK
  - Events, 384
- TIME\_CODE\_EVENT\_NONE
  - Events, 385
- TIME\_CODE\_EVENT\_TICK
  - Events, 385
- TRIGGER\_BRICK\_MK3\_disableCounterAutoReload
  - Configuration, 428
- TRIGGER\_BRICK\_MK3\_disableCounterLoadZero
  - Configuration, 429
- TRIGGER\_BRICK\_MK3\_disableCounterStartMode
  - Configuration, 429
- TRIGGER\_BRICK\_MK3\_disableCounterStartStop↔
  - Mode
  - Configuration, 430
- TRIGGER\_BRICK\_MK3\_disableCounterTriggerCount
  - Configuration, 430
- TRIGGER\_BRICK\_MK3\_disableExtTriggerEdge↔
  - DetectMode
  - Configuration, 430
- TRIGGER\_BRICK\_MK3\_disableExtTriggerInvert
  - Configuration, 431
- TRIGGER\_BRICK\_MK3\_disableExtTriggerOutput
  - Configuration, 431
- TRIGGER\_BRICK\_MK3\_disablePortTransmitPacket↔
  - Mode
  - Configuration, 432
- TRIGGER\_BRICK\_MK3\_disableTrigger
  - Configuration, 432
- TRIGGER\_BRICK\_MK3\_enableCounterAutoReload
  - Configuration, 433
- TRIGGER\_BRICK\_MK3\_enableCounterLoadZero
  - Configuration, 433
- TRIGGER\_BRICK\_MK3\_enableCounterStartMode
  - Configuration, 433
- TRIGGER\_BRICK\_MK3\_enableCounterStartStopMode
  - Configuration, 435
- TRIGGER\_BRICK\_MK3\_enableCounterTriggerCount
  - Configuration, 435
- TRIGGER\_BRICK\_MK3\_enableExtTriggerEdge↔
  - DetectMode
  - Configuration, 436
- TRIGGER\_BRICK\_MK3\_enableExtTriggerInvert
  - Configuration, 436
- TRIGGER\_BRICK\_MK3\_enableExtTriggerOutput
  - Configuration, 437
- TRIGGER\_BRICK\_MK3\_enablePortTransmitPacket↔
  - Mode
  - Configuration, 437
- TRIGGER\_BRICK\_MK3\_enableTrigger
  - Configuration, 438
- TRIGGER\_BRICK\_MK3\_forceCounterReload
  - Configuration, 438
- TRIGGER\_BRICK\_MK3\_forceTrigger
  - Configuration, 439
- TRIGGER\_BRICK\_MK3\_getCounterAutoReload↔
  - Enabled
  - Configuration, 439
- TRIGGER\_BRICK\_MK3\_getCounterInputEvents
  - Events, 386
- TRIGGER\_BRICK\_MK3\_getCounterLoadZeroEnabled
  - Configuration, 439
- TRIGGER\_BRICK\_MK3\_getCounterOutputActions
  - Actions, 406
- TRIGGER\_BRICK\_MK3\_getCounterReloadValue
  - Configuration, 440
- TRIGGER\_BRICK\_MK3\_getCounterStartModeEnabled
  - Configuration, 440
- TRIGGER\_BRICK\_MK3\_getCounterStartStopMode↔
  - Enabled
  - Configuration, 441
- TRIGGER\_BRICK\_MK3\_getCounterTriggerCount↔
  - Enabled
  - Configuration, 441
- TRIGGER\_BRICK\_MK3\_getCounterValue

- Configuration, [441](#)
- TRIGGER\_BRICK\_MK3\_getExtTriggerEdgeDetect↔
  - ModeEnabled
- Configuration, [442](#)
- TRIGGER\_BRICK\_MK3\_getExtTriggerExtend
  - Configuration, [442](#)
- TRIGGER\_BRICK\_MK3\_getExtTriggerInputEvents
  - Events, [386](#)
- TRIGGER\_BRICK\_MK3\_getExtTriggerInvertEnabled
  - Configuration, [443](#)
- TRIGGER\_BRICK\_MK3\_getExtTriggerOutputActions
  - Actions, [407](#)
- TRIGGER\_BRICK\_MK3\_getExtTriggerOutputEnabled
  - Configuration, [443](#)
- TRIGGER\_BRICK\_MK3\_getPortInputEvents
  - Events, [387](#)
- TRIGGER\_BRICK\_MK3\_getPortOutputActions
  - Actions, [407](#)
- TRIGGER\_BRICK\_MK3\_getPortTransmitPacket↔
  - ModeEnabled
- Configuration, [443](#)
- TRIGGER\_BRICK\_MK3\_getTimeCodeInputEvents
  - Events, [387](#)
- TRIGGER\_BRICK\_MK3\_getTimeCodeOutputActions
  - Actions, [408](#)
- TRIGGER\_BRICK\_MK3\_getTriggerAndMask
  - Configuration, [444](#)
- TRIGGER\_BRICK\_MK3\_getTriggerEnabled
  - Configuration, [444](#)
- TRIGGER\_BRICK\_MK3\_getTriggerInputEvents
  - Events, [387](#)
- TRIGGER\_BRICK\_MK3\_getTriggerInputMode
  - Configuration, [445](#)
- TRIGGER\_BRICK\_MK3\_getTriggerInvertMask
  - Configuration, [445](#)
- TRIGGER\_BRICK\_MK3\_setCounterInputEvents
  - Events, [388](#)
- TRIGGER\_BRICK\_MK3\_setCounterOutputActions
  - Actions, [408](#)
- TRIGGER\_BRICK\_MK3\_setCounterReloadValue
  - Configuration, [445](#)
- TRIGGER\_BRICK\_MK3\_setExtTriggerExtend
  - Configuration, [447](#)
- TRIGGER\_BRICK\_MK3\_setExtTriggerInputEvents
  - Events, [388](#)
- TRIGGER\_BRICK\_MK3\_setExtTriggerOutputActions
  - Actions, [408](#)
- TRIGGER\_BRICK\_MK3\_setPortInputEvents
  - Events, [389](#)
- TRIGGER\_BRICK\_MK3\_setPortOutputActions
  - Actions, [409](#)
- TRIGGER\_BRICK\_MK3\_setTimeCodeInputEvents
  - Events, [389](#)
- TRIGGER\_BRICK\_MK3\_setTimeCodeOutputActions
  - Actions, [409](#)
- TRIGGER\_BRICK\_MK3\_setTriggerAndMask
  - Configuration, [447](#)
- TRIGGER\_BRICK\_MK3\_setTriggerInputEvents
  - Events, [390](#)
- TRIGGER\_BRICK\_MK3\_setTriggerInputMode
  - Configuration, [448](#)
- TRIGGER\_BRICK\_MK3\_setTriggerInvertMask
  - Configuration, [448](#)
- TRIGGER\_DEVICE
  - triggering\_types.h, [789](#)
- TRIGGER\_EVENT
  - Events, [385](#)
- TRIGGER\_EVENT\_ALL
  - Events, [386](#)
- TRIGGER\_EVENT\_IN
  - Events, [386](#)
- TRIGGER\_EVENT\_MASK
  - Events, [384](#)
- TRIGGER\_EVENT\_NONE
  - Events, [386](#)
- TRIGGER\_INPUT\_MODE
  - Configuration, [428](#)
- TRIGGER\_INPUT\_MODE\_AND
  - Configuration, [428](#)
- TRIGGER\_INPUT\_MODE\_OR
  - Configuration, [428](#)
- TRIGGER\_MATRIX, [788](#)
- TRIGGER\_PCIE\_IF\_disableCounterAutoReload
  - Configuration, [449](#)
- TRIGGER\_PCIE\_IF\_disableCounterLoadZero
  - Configuration, [449](#)
- TRIGGER\_PCIE\_IF\_disableCounterStartMode
  - Configuration, [450](#)
- TRIGGER\_PCIE\_IF\_disableCounterStartStopMode
  - Configuration, [450](#)
- TRIGGER\_PCIE\_IF\_disableCounterTriggerCount
  - Configuration, [450](#)
- TRIGGER\_PCIE\_IF\_disableTrigger
  - Configuration, [452](#)
- TRIGGER\_PCIE\_IF\_enableCounterAutoReload
  - Configuration, [452](#)
- TRIGGER\_PCIE\_IF\_enableCounterLoadZero
  - Configuration, [453](#)
- TRIGGER\_PCIE\_IF\_enableCounterStartMode
  - Configuration, [453](#)
- TRIGGER\_PCIE\_IF\_enableCounterStartStopMode
  - Configuration, [453](#)
- TRIGGER\_PCIE\_IF\_enableCounterTriggerCount
  - Configuration, [454](#)
- TRIGGER\_PCIE\_IF\_enableTrigger
  - Configuration, [454](#)
- TRIGGER\_PCIE\_IF\_forceCounterReload
  - Configuration, [455](#)
- TRIGGER\_PCIE\_IF\_forceTrigger
  - Configuration, [455](#)
- TRIGGER\_PCIE\_IF\_getCounterAutoReloadEnabled
  - Configuration, [455](#)
- TRIGGER\_PCIE\_IF\_getCounterInputEvents
  - Events, [390](#)
- TRIGGER\_PCIE\_IF\_getCounterLoadZeroEnabled
  - Configuration, [456](#)

- TRIGGER\_PCIE\_IF\_getCounterOutputActions
  - Actions, [410](#)
- TRIGGER\_PCIE\_IF\_getCounterReloadValue
  - Configuration, [456](#)
- TRIGGER\_PCIE\_IF\_getCounterStartModeEnabled
  - Configuration, [457](#)
- TRIGGER\_PCIE\_IF\_getCounterStartStopMode↔  
Enabled
  - Configuration, [457](#)
- TRIGGER\_PCIE\_IF\_getCounterTriggerCountEnabled
  - Configuration, [457](#)
- TRIGGER\_PCIE\_IF\_getCounterValue
  - Configuration, [458](#)
- TRIGGER\_PCIE\_IF\_getPortInputEvents
  - Events, [391](#)
- TRIGGER\_PCIE\_IF\_getPortOutputActions
  - Actions, [410](#)
- TRIGGER\_PCIE\_IF\_getTriggerAndMask
  - Configuration, [458](#)
- TRIGGER\_PCIE\_IF\_getTriggerEnabled
  - Configuration, [459](#)
- TRIGGER\_PCIE\_IF\_getTriggerInputEvents
  - Events, [391](#)
- TRIGGER\_PCIE\_IF\_getTriggerInputMode
  - Configuration, [459](#)
- TRIGGER\_PCIE\_IF\_getTriggerInvertMask
  - Configuration, [459](#)
- TRIGGER\_PCIE\_IF\_setCounterInputEvents
  - Events, [391](#)
- TRIGGER\_PCIE\_IF\_setCounterOutputActions
  - Actions, [411](#)
- TRIGGER\_PCIE\_IF\_setCounterReloadValue
  - Configuration, [460](#)
- TRIGGER\_PCIE\_IF\_setPortInputEvents
  - Events, [392](#)
- TRIGGER\_PCIE\_IF\_setPortOutputActions
  - Actions, [411](#)
- TRIGGER\_PCIE\_IF\_setTriggerAndMask
  - Configuration, [460](#)
- TRIGGER\_PCIE\_IF\_setTriggerInputEvents
  - Events, [392](#)
- TRIGGER\_PCIE\_IF\_setTriggerInputMode
  - Configuration, [461](#)
- TRIGGER\_PCIE\_IF\_setTriggerInvertMask
  - Configuration, [461](#)
- TRIGGER\_PXI\_IF\_disableCounterAutoReload
  - Configuration, [462](#)
- TRIGGER\_PXI\_IF\_disableCounterLoadZero
  - Configuration, [462](#)
- TRIGGER\_PXI\_IF\_disableCounterStartMode
  - Configuration, [462](#)
- TRIGGER\_PXI\_IF\_disableCounterStartStopMode
  - Configuration, [463](#)
- TRIGGER\_PXI\_IF\_disableCounterTriggerCount
  - Configuration, [463](#)
- TRIGGER\_PXI\_IF\_disableExtTriggerEdgeDetectMode
  - Configuration, [464](#)
- TRIGGER\_PXI\_IF\_disableExtTriggerInvert
  - Configuration, [464](#)
- TRIGGER\_PXI\_IF\_disableExtTriggerOutput
  - Configuration, [464](#)
- TRIGGER\_PXI\_IF\_disablePortTransmitPacketMode
  - Configuration, [465](#)
- TRIGGER\_PXI\_IF\_disableTrigger
  - Configuration, [465](#)
- TRIGGER\_PXI\_IF\_enableCounterAutoReload
  - Configuration, [466](#)
- TRIGGER\_PXI\_IF\_enableCounterLoadZero
  - Configuration, [466](#)
- TRIGGER\_PXI\_IF\_enableCounterStartMode
  - Configuration, [466](#)
- TRIGGER\_PXI\_IF\_enableCounterStartStopMode
  - Configuration, [467](#)
- TRIGGER\_PXI\_IF\_enableCounterTriggerCount
  - Configuration, [467](#)
- TRIGGER\_PXI\_IF\_enableExtTriggerEdgeDetectMode
  - Configuration, [468](#)
- TRIGGER\_PXI\_IF\_enableExtTriggerInvert
  - Configuration, [468](#)
- TRIGGER\_PXI\_IF\_enableExtTriggerOutput
  - Configuration, [469](#)
- TRIGGER\_PXI\_IF\_enablePortTransmitPacketMode
  - Configuration, [469](#)
- TRIGGER\_PXI\_IF\_enableTrigger
  - Configuration, [470](#)
- TRIGGER\_PXI\_IF\_forceCounterReload
  - Configuration, [470](#)
- TRIGGER\_PXI\_IF\_forceTrigger
  - Configuration, [470](#)
- TRIGGER\_PXI\_IF\_getCounterAutoReloadEnabled
  - Configuration, [471](#)
- TRIGGER\_PXI\_IF\_getCounterInputEvents
  - Events, [393](#)
- TRIGGER\_PXI\_IF\_getCounterLoadZeroEnabled
  - Configuration, [471](#)
- TRIGGER\_PXI\_IF\_getCounterOutputActions
  - Actions, [412](#)
- TRIGGER\_PXI\_IF\_getCounterReloadValue
  - Configuration, [471](#)
- TRIGGER\_PXI\_IF\_getCounterStartModeEnabled
  - Configuration, [472](#)
- TRIGGER\_PXI\_IF\_getCounterStartStopModeEnabled
  - Configuration, [472](#)
- TRIGGER\_PXI\_IF\_getCounterTriggerCountEnabled
  - Configuration, [473](#)
- TRIGGER\_PXI\_IF\_getCounterValue
  - Configuration, [473](#)
- TRIGGER\_PXI\_IF\_getExtTriggerEdgeDetectMode↔  
Enabled
  - Configuration, [473](#)
- TRIGGER\_PXI\_IF\_getExtTriggerExtend
  - Configuration, [475](#)
- TRIGGER\_PXI\_IF\_getExtTriggerInputEvents
  - Events, [393](#)
- TRIGGER\_PXI\_IF\_getExtTriggerInvertEnabled
  - Configuration, [475](#)

- TRIGGER\_PXI\_IF\_getExtTriggerOutputActions
  - Actions, [412](#)
- TRIGGER\_PXI\_IF\_getExtTriggerOutputEnabled
  - Configuration, [476](#)
- TRIGGER\_PXI\_IF\_getPortInputEvents
  - Events, [394](#)
- TRIGGER\_PXI\_IF\_getPortOutputActions
  - Actions, [412](#)
- TRIGGER\_PXI\_IF\_getPortTransmitPacketMode↔
  - Enabled
  - Configuration, [476](#)
- TRIGGER\_PXI\_IF\_getTimeCodeInputEvents
  - Events, [394](#)
- TRIGGER\_PXI\_IF\_getTimeCodeOutputActions
  - Actions, [413](#)
- TRIGGER\_PXI\_IF\_getTriggerAndMask
  - Configuration, [476](#)
- TRIGGER\_PXI\_IF\_getTriggerEnabled
  - Configuration, [477](#)
- TRIGGER\_PXI\_IF\_getTriggerInputEvents
  - Events, [394](#)
- TRIGGER\_PXI\_IF\_getTriggerInputMode
  - Configuration, [477](#)
- TRIGGER\_PXI\_IF\_getTriggerInvertMask
  - Configuration, [478](#)
- TRIGGER\_PXI\_IF\_setCounterInputEvents
  - Events, [396](#)
- TRIGGER\_PXI\_IF\_setCounterOutputActions
  - Actions, [413](#)
- TRIGGER\_PXI\_IF\_setCounterReloadValue
  - Configuration, [478](#)
- TRIGGER\_PXI\_IF\_setExtTriggerExtend
  - Configuration, [479](#)
- TRIGGER\_PXI\_IF\_setExtTriggerInputEvents
  - Events, [396](#)
- TRIGGER\_PXI\_IF\_setExtTriggerOutputActions
  - Actions, [414](#)
- TRIGGER\_PXI\_IF\_setPortInputEvents
  - Events, [397](#)
- TRIGGER\_PXI\_IF\_setPortOutputActions
  - Actions, [414](#)
- TRIGGER\_PXI\_IF\_setTimeCodeInputEvents
  - Events, [397](#)
- TRIGGER\_PXI\_IF\_setTimeCodeOutputActions
  - Actions, [415](#)
- TRIGGER\_PXI\_IF\_setTriggerAndMask
  - Configuration, [479](#)
- TRIGGER\_PXI\_IF\_setTriggerInputEvents
  - Events, [398](#)
- TRIGGER\_PXI\_IF\_setTriggerInputMode
  - Configuration, [479](#)
- TRIGGER\_PXI\_IF\_setTriggerInvertMask
  - Configuration, [480](#)
- TRIGGER\_PXI\_ROUTER\_disableCounterAutoReload
  - Configuration, [480](#)
- TRIGGER\_PXI\_ROUTER\_disableCounterLoadZero
  - Configuration, [481](#)
- TRIGGER\_PXI\_ROUTER\_disableCounterStartMode
  - Configuration, [481](#)
- TRIGGER\_PXI\_ROUTER\_disableCounterStartStop↔
  - Mode
  - Configuration, [481](#)
- TRIGGER\_PXI\_ROUTER\_disableCounterTriggerCount
  - Configuration, [482](#)
- TRIGGER\_PXI\_ROUTER\_disablePortTransmit↔
  - PacketMode
  - Configuration, [482](#)
- TRIGGER\_PXI\_ROUTER\_disableTrigger
  - Configuration, [483](#)
- TRIGGER\_PXI\_ROUTER\_enableCounterAutoReload
  - Configuration, [483](#)
- TRIGGER\_PXI\_ROUTER\_enableCounterLoadZero
  - Configuration, [484](#)
- TRIGGER\_PXI\_ROUTER\_enableCounterStartMode
  - Configuration, [484](#)
- TRIGGER\_PXI\_ROUTER\_enableCounterStartStop↔
  - Mode
  - Configuration, [484](#)
- TRIGGER\_PXI\_ROUTER\_enableCounterTriggerCount
  - Configuration, [485](#)
- TRIGGER\_PXI\_ROUTER\_enablePortTransmitPacket↔
  - Mode
  - Configuration, [485](#)
- TRIGGER\_PXI\_ROUTER\_enableTrigger
  - Configuration, [486](#)
- TRIGGER\_PXI\_ROUTER\_forceCounterReload
  - Configuration, [486](#)
- TRIGGER\_PXI\_ROUTER\_forceTrigger
  - Configuration, [487](#)
- TRIGGER\_PXI\_ROUTER\_getCounterAutoReload↔
  - Enabled
  - Configuration, [487](#)
- TRIGGER\_PXI\_ROUTER\_getCounterInputEvents
  - Events, [398](#)
- TRIGGER\_PXI\_ROUTER\_getCounterLoadZero↔
  - Enabled
  - Configuration, [487](#)
- TRIGGER\_PXI\_ROUTER\_getCounterOutputActions
  - Actions, [415](#)
- TRIGGER\_PXI\_ROUTER\_getCounterReloadValue
  - Configuration, [489](#)
- TRIGGER\_PXI\_ROUTER\_getCounterStartMode↔
  - Enabled
  - Configuration, [489](#)
- TRIGGER\_PXI\_ROUTER\_getCounterStartStopMode↔
  - Enabled
  - Configuration, [490](#)
- TRIGGER\_PXI\_ROUTER\_getCounterTriggerCount↔
  - Enabled
  - Configuration, [490](#)
- TRIGGER\_PXI\_ROUTER\_getCounterValue
  - Configuration, [490](#)
- TRIGGER\_PXI\_ROUTER\_getPortInputEvents
  - Events, [399](#)
- TRIGGER\_PXI\_ROUTER\_getPortOutputActions
  - Actions, [415](#)

- TRIGGER\_PXI\_ROUTER\_getPortTransmitPacket↔
  - ModeEnabled
  - Configuration, 491
- TRIGGER\_PXI\_ROUTER\_getTimeCodeInputEvents
  - Events, 399
- TRIGGER\_PXI\_ROUTER\_getTimeCodeOutputActions
  - Actions, 417
- TRIGGER\_PXI\_ROUTER\_getTriggerAndMask
  - Configuration, 491
- TRIGGER\_PXI\_ROUTER\_getTriggerEnabled
  - Configuration, 492
- TRIGGER\_PXI\_ROUTER\_getTriggerInputEvents
  - Events, 399
- TRIGGER\_PXI\_ROUTER\_getTriggerInputMode
  - Configuration, 492
- TRIGGER\_PXI\_ROUTER\_getTriggerInvertMask
  - Configuration, 492
- TRIGGER\_PXI\_ROUTER\_setCounterInputEvents
  - Events, 400
- TRIGGER\_PXI\_ROUTER\_setCounterOutputActions
  - Actions, 417
- TRIGGER\_PXI\_ROUTER\_setCounterReloadValue
  - Configuration, 493
- TRIGGER\_PXI\_ROUTER\_setPortInputEvents
  - Events, 400
- TRIGGER\_PXI\_ROUTER\_setPortOutputActions
  - Actions, 418
- TRIGGER\_PXI\_ROUTER\_setTimeCodeInputEvents
  - Events, 401
- TRIGGER\_PXI\_ROUTER\_setTimeCodeOutputActions
  - Actions, 418
- TRIGGER\_PXI\_ROUTER\_setTriggerAndMask
  - Configuration, 493
- TRIGGER\_PXI\_ROUTER\_setTriggerInputEvents
  - Events, 401
- TRIGGER\_PXI\_ROUTER\_setTriggerInputMode
  - Configuration, 494
- TRIGGER\_PXI\_ROUTER\_setTriggerInvertMask
  - Configuration, 494
- TRIGGER\_TYPE
  - triggering\_types.h, 789
- TRUE
  - STAR-Dundee Types, 711
- Time-code Master, 514
  - CFG\_MK2\_disableExternalTimeCodeSelection, 515
  - CFG\_MK2\_disableTimeCodeCounterBypass↔ Mode, 516
  - CFG\_MK2\_disableTimeCodeMaster, 516
  - CFG\_MK2\_enableExternalTimeCodeSelection, 517
  - CFG\_MK2\_enableTimeCodeCounterBypassMode, 517
  - CFG\_MK2\_enableTimeCodeMaster, 517
  - CFG\_MK2\_getExternalTimeCodeSelection↔ Enabled, 518
  - CFG\_MK2\_getTimeCodeCounterBypassMode↔ Enabled, 518
  - CFG\_MK2\_getTimeCodeMasterEnabled, 519
  - CFG\_MK2\_getTimeCodePeriod, 519
  - CFG\_MK2\_setTimeCodePeriod, 520
- Time-codes, 636
  - CFG\_disableExternalTimeCodeSelection, 637
  - CFG\_disableTimeCodeCounterBypassMode, 638
  - CFG\_disableTimeCodeMaster, 638
  - CFG\_enableExternalTimeCodeSelection, 639
  - CFG\_enableTimeCodeCounterBypassMode, 639
  - CFG\_enableTimeCodeMaster, 639
  - CFG\_getExternalTimeCodeSelectionEnabled, 640
  - CFG\_getTimeCodeCounterBypassModeEnabled, 640
  - CFG\_getTimeCodeMasterEnabled, 641
  - CFG\_getTimeCodePeriod, 641
  - CFG\_setTimeCodePeriod, 642
- Timestamping, 317, 658
  - CFG\_BRICK\_MK3\_disableRxTimestampEvents↔ OnPort, 318
  - CFG\_BRICK\_MK3\_enableRxTimestampEvents↔ OnPort, 319
  - CFG\_BRICK\_MK3\_getPulseGeneratorFrequency, 319
  - CFG\_BRICK\_MK3\_getRxTimestampEventsOn↔ PortEnabled, 320
  - CFG\_BRICK\_MK3\_getTimestampMethod, 320
  - CFG\_BRICK\_MK3\_getTimestampValue, 321
  - CFG\_BRICK\_MK3\_setPulseGeneratorFrequency, 321
  - CFG\_BRICK\_MK3\_setTimestampMethod, 322
  - CFG\_BRICK\_MK3\_setTimestampValue, 322
  - CFG\_disableRxTimestampEventsOnPort, 658
  - CFG\_enableRxTimestampEventsOnPort, 659
  - CFG\_getPulseGeneratorFrequency, 659
  - CFG\_getRxTimestampEventsOnPortEnabled, 660
  - CFG\_getTimestampMethod, 660
  - CFG\_getTimestampValue, 661
  - CFG\_setPulseGeneratorFrequency, 661
  - CFG\_setTimestampMethod, 662
  - CFG\_setTimestampValue, 662
  - STAR\_CFG\_BRICK\_MK3\_PULSE\_FREQ, 318
  - STAR\_CFG\_BRICK\_MK3\_PULSE\_FREQ\_1, 318
  - STAR\_CFG\_BRICK\_MK3\_PULSE\_FREQ\_10, 318
  - STAR\_CFG\_BRICK\_MK3\_PULSE\_FREQ\_100, 318
  - STAR\_CFG\_BRICK\_MK3\_PULSE\_FREQ\_1000, 318
  - STAR\_CFG\_BRICK\_MK3\_TIMESTAMP\_METH↔ OD, 318
  - STAR\_CFG\_BRICK\_MK3\_TIMESTAMP\_METH↔ OD\_EXTERNAL\_TRIGGER, 318
  - STAR\_CFG\_BRICK\_MK3\_TIMESTAMP\_METH↔ OD\_NONE, 318
  - STAR\_CFG\_BRICK\_MK3\_TIMESTAMP\_METH↔ OD\_PULSE\_GENERATOR, 318
- Transfer Operations, 275



- STAR\_RECEIVE\_BROADCAST\_MESSAGE↵  
S, 278
- STAR\_RECEIVE\_CHUNKS, 278
- STAR\_RECEIVE\_FCTS, 278
- STAR\_RECEIVE\_LINK\_SPEED\_EVENTS, 278
- STAR\_RECEIVE\_LINK\_STATE\_EVENTS, 278
- STAR\_RECEIVE\_MASK, 278
- STAR\_RECEIVE\_NULL, 278
- STAR\_RECEIVE\_PACKETS, 278
- STAR\_RECEIVE\_TIMECODES, 278
- STAR\_RECEIVE\_TIMESTAMP\_EVENTS, 278
- STAR\_TRANSFER\_OPERATION, 277
- STAR\_TRANSFER\_STATUS, 278
- STAR\_TRANSFER\_STATUS\_CANCELLED, 279
- STAR\_TRANSFER\_STATUS\_COMPLETE, 279
- STAR\_TRANSFER\_STATUS\_ERROR, 279
- STAR\_TRANSFER\_STATUS\_NOT\_STARTED,  
279
- STAR\_TRANSFER\_STATUS\_STARTED, 279
- STAR\_TransferOperationListenerFunc, 277
- STAR\_cancelTransferOperation, 279
- STAR\_cancelTransferOperationWaits, 279
- STAR\_createRxOperation, 279
- STAR\_createRxOperationEx, 280
- STAR\_createTxOperation, 281
- STAR\_destroyTransferItemList, 281
- STAR\_disposeTransferOperation, 282
- STAR\_getNextTransferItem, 282
- STAR\_getSpaceWireAddressPath, 284
- STAR\_getSpaceWireAddressPathLength, 284
- STAR\_getTransferItem, 284
- STAR\_getTransferItemCount, 285
- STAR\_getTransferItemList, 285
- STAR\_getTransferOperationStatus, 286
- STAR\_receivePacket, 286
- STAR\_registerTransferCompletionListener, 287
- STAR\_rxOperationExGetBroadcastMessage, 287
- STAR\_rxOperationExGetDataChunk, 288
- STAR\_rxOperationExGetItemType, 288
- STAR\_rxOperationExGetNumItems, 288
- STAR\_rxOperationExGetStatus, 289
- STAR\_submitTransferOperation, 289
- STAR\_submitTransferOperationList, 289
- STAR\_transmitPacket, 290
- STAR\_unregisterTransferCompletionListener, 290
- STAR\_waitOnTransferOperationCompletion, 291
- transfer.h, 770
- transfer\_types.h, 772
- Triggering API, 368
- triggering\_brick\_mk3.h, 772
- triggering\_pcie.h, 776
- triggering\_pxi\_if.h, 779
- triggering\_pxi\_ro.h, 783
- triggering\_types.h, 786
  - TRIGGER\_DEVICE, 789
  - TRIGGER\_TYPE, 789
- types.h, 789
  - CFG\_INFO\_ID\_TYPE, 792
- U16
  - STAR-Dundee Types, 712
- U16\_MAX
  - STAR-Dundee Types, 711
- U32
  - STAR-Dundee Types, 712
- U8
  - STAR-Dundee Types, 712
- User, 606
  - CFG\_getGeneralPurpose, 606
  - CFG\_getNetworkIdentity, 607
  - CFG\_getTimeCodeDistributionPorts, 607
  - CFG\_setGeneralPurpose, 608
  - CFG\_setNetworkIdentity, 608
  - CFG\_setTimeCodeDistributionPorts, 609
  - CFG\_updateDeviceInfo, 609
- User Configurable Registers, 593
  - CFG\_MK2\_getTimeCodeDistributionPorts, 593
  - CFG\_MK2\_setTimeCodeDistributionPorts, 594
  - CFG\_ROUTER\_getGeneralPurpose, 594
  - CFG\_ROUTER\_getNetworkIdentity, 595
  - CFG\_ROUTER\_getTimeCodeDistributionPorts,  
595
  - CFG\_ROUTER\_setGeneralPurpose, 596
  - CFG\_ROUTER\_setNetworkIdentity, 596
  - CFG\_ROUTER\_setTimeCodeDistributionPorts,  
596
- VERSION\_EDIT
  - version.h, 796
- VERSION\_MAJOR
  - version.h, 796
- VERSION\_MINOR
  - version.h, 796
- VERSION\_PATCH
  - version.h, 796
- version.h, 792
  - POPULATE\_VERSION, 794
  - PRINT\_FULL\_VERSION\_INFO, 795
  - PRINT\_VERSION, 795
  - PRINT\_VERSION\_EDIT, 795
  - PRINT\_VERSION\_NUM, 795
  - PRINT\_VERSION\_PATCH, 796
  - VERSION\_EDIT, 796
  - VERSION\_MAJOR, 796
  - VERSION\_MINOR, 796
  - VERSION\_PATCH, 796
- Virtual Devices and Drivers, 292
  - STAR\_isDeviceVirtual, 292
  - STAR\_isDriverVirtual, 292